PostGIS

공식 가이드북 2.3ver

지은이 PostGIS 프로젝트 운영위원회

펴낸이 한국오픈소스GIS포럼

PostGIS 공식 가이드북 2.3 ver

이 책은 한국오픈소스 GIS 포럼의 지원을 받아 제작되었음을 알립니다.

초판 1 쇄 인쇄: 2016 년 11 월 28 일
초판 1 쇄 발행: 2016 년 11 월 28 일

지은이: PostGIS 프로젝트 운영위원회
옮긴이: 권용찬
감수: 김지윤, 장병진
표지 디자인: 신명순
편집: 김서인, 임호준

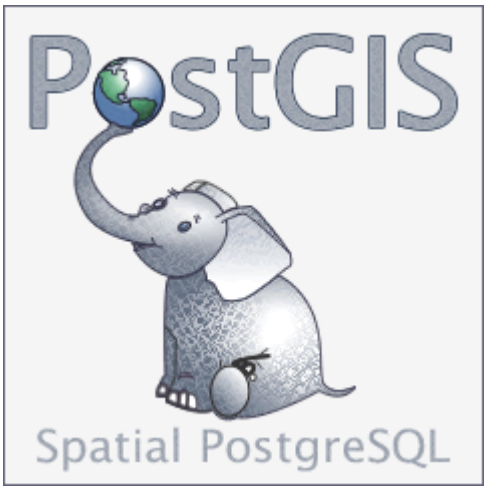
펴낸이: 한국오픈소스 GIS 포럼
펴낸곳: 가이아쓰리디㈜
주소: 대전 유성구 관평동 1359 한신에스메카 230 호
전화: 042-330-0400
팩스: 042-330-0410
출판등록: 제 2012-000016 호
ISBN: 978-89-969532-8-9 (93000)

이 책은 비매품입니다.

이 도서의 국립중앙도서관 출판시도서목록(CIP)은 서지정보유통지원시스템 홈페이지(http://seoji.nl.go.kr)와 국가자료공동목록시스템(http://www.nl.go.kr/kolisnet)에서 이용하실 수 있습니다.(CIP 제어번호: CIP2016028654)

Abstract

PostGIS 는 객체 관계형 데이터베이스 시스템인 PostgreSQL 의 확장 프로그램으로, 데이터베이스에 GIS(지리정보 시스템) 객체를 저장할 수 있게 해줍니다. PostGIS 는 GiST 기반 R-Tree 공간 인덱스를 지원하며, GIS 객체의 분석 및 공간 처리를 위한 기능을 포함하고 있습니다.





이 문서는 2.3.0dev 버전입니다.



이 저작물은 크리에이티브 커먼즈 저작자표시-동일조건변경허락 3.0 라이선스를 따릅니다. 사용자가 원하는 대로 이 문서를 사용할 수 있지만, 우리는 사용자가 PostGIS 프로젝트의 저작권을 명시하고 가능한 위치에 http://postgis.net 으로 연결되는 링크를 삽입하도록 요구합니다.

목 차

Abstract
목 차 i

Chapter 1. 소개 1
1.1. 프로젝트 운영 위원회 1
1.2. 핵심 공헌자 - 현재 2
1.3. 핵심 공헌자 – 과거 2
1.4. 기타 공헌자 3
1.5. 추가 정보 4
Chapter 2. 설치 5
2.1. 짧은 설명 5
2.2. 설치 요구 사양 7
2.3. 소스 획득 8
2.4. 소스를 컴파일하고 설치하기: 상세 설명 8
2.5. 확장 프로그램을 활용한 공간 데이터베이스 생성 23
2.6. 확장 프로그램을 활용하지 않은 공간 데이터베이스 생성 24
2.7. 주소 표준화 도구 설치 및 활용 25
2.8. TIGER 지오코딩 도구의 설치, 업그레이드 및 데이터 로딩 26
2.9. 템플릿을 이용해서 공간정보 처리 가능한 데이터베이스 생성 31
2.10. 업그레이드 31
2.11. 설치 과정에서 흔히 발생하는 문제들 34
2.12. 로더/덤퍼(Loader/Dumper) 35
Chapter 3. PostGIS FAQ 36
Chapter 4. PostGIS 활용: 데이터 관리 및 쿼리 43
4.1. GIS 객체 43
4.2. PostGIS Geography Type 46
4.3. OpenGIS 표준 이용 50
4.4. GIS (벡터) 데이터 로드 62
4.5. GIS 데이터 가져오기 64
4.6. 인덱스 빌드 작업 67
4.7. 복잡 쿼리 69
Chapter 5. 래스터 데이터의 관리, 쿼리, 및 응용 74
5.1. 래스터 로드 및 생성 74
5.2. 래스터 카탈로그 79
5.3. PostGIS 래스터를 이용하는 사용자 지정 응용 프로그램 빌드하기 82
Chapter 6. PostGIS 도형 활용: 응용 프로그램 빌드 88
6.1. MapServer 활용 88
6.2. Java 클라이언트 (JDBC) 93
6.3. C 클라이언트 (libpq) 94
Chapter 7. 성능 향상 비법 95
7.1. 대용량 도형을 담은 저용량 테이블 95
7.2. 도형 인덱스에 대한 클러스터 작업 96

7.3. 차원변환 피하기 97
7.4. 사용자 설정 세부 조정 97
Chapter 8. PostGIS 참조 문서 99
8.1. PostgreSQL PostGIS 도형/지리형/상자 유형 99
8.2. PostGIS GUC(Grand Unified Custom Variable) 101
8.3. 관리 함수 105
8.4. 도형 작성자(constructor) 116
8.5. 지오메트리 접근자 151
8.6. 도형 편집자(editor) 194
8.7. 도형 출력물(output) 225
8.8. 연산자(operator) 245
8.9. 공간 관계성 및 측정 272
8.10. SFCGAL 함수 334
8.11. 도형 공간 처리 348
8.12. 선형 참조(Linear Referencing) 402
8.13. 시계열 지원 410
8.14. 장기 트랜잭션(Long Transaction) 지원 413
8.15. 기타 함수들 417
8.16. 예외적인 함수들 427
Chapter 9. 래스터 참조 문서 430
9.1. 래스터 지원 데이터형 431
9.2. 래스터 데이터 관리 434
9.3. 래스터 생성자 444
9.4. 래스터 접근자 455
9.5. 래스터 밴드 접근자 471
9.6. 래스터 픽셀 접근자 및 설정 476
9.7. 래스터 편집자 496
9.8. 래스터 밴드 편집기 507
9.9. 래스터 밴드 통계 및 분석 510
9.10. 래스터 출력 522
9.11. 래스터 처리 526
9.12. 래스터 연산자들 584
9.13. 래스터 및 래스터 밴드 공간적 관계성 589
Chapter 10. PostGIS 래스터 FAQ 601
Chapter 11. 지형(topology) 608
11.1. 지형 유형 608
11.2. 지형 도메인 610
11.3. 지형 및 TopoGeometry 관리 612
11.4. 지형 작성자 615
11.5. 지형 편집자 620
11.6. 지형 접근자 629
11.7. 지형 공간 처리 635
11.8. TopoGeometry 작성자 640
11.9. TopoGeometry 편집자 644
11.10. TopoGeometry 접근자 646
11.11. TopoGeometry 출력물 647
11.12. 지형 공간 관계성 651
Chapter 12. 주소 표준화 도구 653
12.1. 파싱 도구의 작동 방식 653

12.2. 주소 표준화 도구 유형 654
12.3. 주소 표준화 도구 테이블 655
12.4. 주소 표준화 도구 함수 659
Chapter 13. PostGIS 추가 기능 663
13.1. TIGER 지오코딩 도구 663
Chapter 14. PostGIS 공간 함수 색인 689
14.1. PostGIS 종합 함수 689
14.2. PostGIS Window Functions 689
14.3. PostGIS SQL-MM 호환 함수 690
14.4. PostGIS 지리형 지원 함수 696
14.5. PostGIS 래스터 지원 함수 697
14.6. PostGIS 도형/지리형/래스터 덤프 함수 704
14.7. PostGIS 상자 함수 704
14.8. 3 차원을 지원하는 PostGIS 함수 706
14.9. PostGIS 폐합 도형 지원 함수 712
14.10. PostGIS 다면체 표면 지원 함수 716
14.11. PostGIS 함수 지원 매트릭스 720
14.12. 신규, 개선, 변경된 PostGIS 함수 727
Chapter 15. 문제 보고 732
15.1. 소프트웨어 버그 보고하기 732
15.2. 문서 문제점 보고하기 732

Chapter 1.소개

PostGIS 는 리프랙션스 리서치(Refractions Research Inc.)가 개발한 공간 데이터베이스 기술연구 프로젝트입니다. 리프랙션스 리서치는 캐나다 브리티시컬럼비아 주 빅토리아 시에 위치한 GIS 및 데이터베이스 자문회사로 데이터 통합 및 고객 지향 소프트웨어 개발 전문 기업입니다. 우리는 완전한 OpenGIS 지원, 고급 위상 구조(커버리지, 표면, 네트워크), GIS 데이터의 열람 및 편집 작업을 위한 데스크톱 사용자 인터페이스 도구, 웹 기반 접근 도구를 포함하는 일련의 중요 GIS 기능을 지원하도록 PostGIS 를 지원하고 개발할 계획입니다.

PostGIS 는 OSGeo 재단의 육성 프로젝트 중 하나입니다. 여러 FOSS4G 개발자들은 물론 PostGIS 의 기능성 및 다목적성으로부터 막대한 이익을 얻고 있는 전세계의 많은 기업들이 PostGIS 를 끊임없이 개선하고 후원하고 있습니다.

1.1. 프로젝트 운영 위원회

PostGIS 프로젝트 운영 위원회(Project Steering Committee; PSC)는 PostGIS 프로젝트의 대략적인 방향, 발표 주기, 문서화 및 원조 활동을 조정합니다. PSC 는 또한 일반 사용자 지원을 제공하고, PostGIS 커뮤니티 전반에서 개발하는 소프트웨어 패치를 받아들여 승인하며, 개발자 커밋 접근, PSC 신입회원 결정 또는 중요한 API 변경 등 PostGIS 에 관한 여러 가지 사안에 대해 투표를 실시합니다.

마크 케이브-에일런드(Mark Cave-Ayland)

버그 수정 및 유지보수 활동, PostGIS 와 PostgreSQL 의 출시 시기, 공간 인덱스 선택도 및 바인딩, 로더/덤퍼, shapefile GUI 로더, 새 기능과 새로운 기능 개선의 통합을 조정

레지나 오베(Regina Obe)

빌드봇 유지보수, 윈도우 용 제품 생산 및 실험적인 버전 빌드, 문서화, PostGIS 뉴스그룹에서 개괄적인 사용자 지원, X3D 지원, TIGER 지오코딩 도구(TIGER Geocoder) 지원, 관리 기능, 새로운 기능 또는 대대적인 코드 변경에 대한 스모크 테스트를 담당

박 뽀리(Bborie Park)

래스터 개발, GDAL 과의 통합, 래스터 로더, 사용자 지원, 개괄적인 버그 수정, 다양한 OS(슬랙웨어, 맥, 윈도우 등)에서의 테스트를 담당

폴 램지(Paul Ramsey) (의장)

PostGIS 프로젝트의 공동 창설자. 개괄적인 버그 수정, 지리학(geography) 지원, 지형(geography) 및 도형(geometry) 인덱스 지원(2D, 3D, nD 인덱스와 모든 공간 인덱스), 도형 내부 구조의 근원, PointCloud(개발중), GEOS 출시 시기와 GEOS 기능성 통합의 조정, 로더/덤퍼, shapefile GUI 로더를 담당

산드로 산틸리(Sandro Santilli)

버그 수정, 유지보수, GEOS 출시 시기와 GEOS 기능성 통합의 조정, 지형(topology) 지원, 래스터 프레임워크 및 저레벨 API 함수를 담당

1.2. 핵심 공헌자 - 현재

호르헤 아레발로(Jorge Arévalo)

래스터 개발, GDAL 드라이버 지원, 로더를 담당

니클라스 아벤(Nicklas Avén)

거리 함수 개선(3D 거리와 관계 함수 포함) 및 추가 작업, TWKB(Tiny WKB) 출력 포맷(개발중), 개괄적인 사용자 지원을 담당

댄 배스턴(Dan Baston)

도형 클러스터 작업 함수 및 추가 작업, 기타 도형 알고리즘 개선, 개괄적인 사용자 지원을 담당

올리비에 쿠르텡(Olivier Courtin)

XML(KML, GML)/GeoJSON 입출력 기능, 3D 지원 및 버그 수정을 담당

마테우스 로스코트(Mateusz Loskot)

PostGIS 용 CMake 지원, 파이썬 래스터 로더 원본 개발, 저레벨 래스터 API 함수 개발을 담당

피에르 라신(Pierre Racine)

래스터의 전반적 아키텍처, 프로토타이핑, 프로그래밍 지원을 담당

다피트 츠바르크(David Zwarg)

래스터 개발(대부분 맵 대수학 분석 기능들)을 담당

1.3. 핵심 공헌자 – 과거

크리스 호지슨(Chris Hodgson)

전 PSC 회원. 개괄적인 개발, 사이트 및 빌드봇 유지보수, OSGeo 육성 프로젝트 관리를 담당

케빈 뉴펠드(Kevin Neufeld)

전 PSC 회원. 문서화 및 문서화 지원 도구, 빌드봇 유지보수, PostGIS 뉴스그룹에서 고급 사용자 지원, PostGIS 유지보수 기능 개선을 담당

데이브 블래스비(Dave Blasby)

PostGIS 의 원 개발자이자 공동 창립자. 서버측 객체, 인덱스 바인딩 및 서버측의 여러 분석 기능들을 작업

제프 라운스버리(Jeff Lounsbury)

shapefile 로더/덤퍼 원본을 개발. 현재 PostGIS 프로젝트 소유자 대표

마크 레슬리(Mark Leslie)

진행중인 유지보수 및 핵심 기능 개발. 곡선 지원 개선, shapefile GUI 로더를 담당

1.4. 기타 공헌자

개인 공헌자

알파벳 순서: Alex Bodnaru, Alex Mayrhofer, Andrea Peri, Andreas Forø Tollefsen, Andreas Neumann, Anne Ghisla, Barbara Phillipot, Ben Jubb, Bernhard Reiter, Brian Hamlin, Bruce Rindahl, Bruno Wolff III, Bryce L. Nordgren, Carl Anderson, Charlie Savage, Dane Springmeyer, David Skea, David Techer, Eduin Carrillo, Even Rouault, Frank Warmerdam, George Silva, Gerald Fenoy, Gino Lucrezi, Guillaume Lelarge, IIDA Tetsushi, Ingvild Nystuen, Jason Smith, Jeff Adams, Jose Carlos Martinez Llari, Julien Rouhaud, Kashif Rasul, Klaus Foerster, Kris Jurka, Leo Hsu, Loic Dachary, Luca S. Percich, Maria Arias de Reyna, Mark Sondheim, Markus Schaber, Maxime Guillaud, Maxime van Noppen, Michael Fuhr, Mike Toews, Nathan Wagner, Nathaniel Clay, Nikita Shulga, Norman Vine, Rafal Magda, Ralph Mason, Rémi Cura, Richard Greenwood, Silvio Grosso, Steffen Macke, Stephen Frost, Tom van Tilburg, Vincent Mora, Vincent Picavet

후원 기업

PostGIS 프로젝트에 직접적으로 금전을 후원하거나, 개발자 시간 및 호스팅에 기여를 한 기업들입니다.

알파벳 순서: Arrival 3D, Associazione Italiana per l'Informazione Geografica Libera (GFOSS.it), AusVet, Avencia, Azavea, Cadcorp, CampToCamp, CartoDB, City of Boston (DND), Clever Elephant Solutions, Cooperativa Alveo, Deimos Space, Faunalia, Geographic Data BC, Hunter Systems Group, Lidwala Consulting Engineers, LisaSoft, Logical Tracking & Tracing International AG, Maponics, Michigan Tech Research Institute, Natural Resources Canada, Norwegian Forest and Landscape Institute, Boundless (former OpenGeo), OSGeo, Oslandia, Palantir Technologies, Paragon Corporation, R3 GIS, Refractions Research, Regione Toscana - SITA, Safe Software, Sirius Corporation plc, Stadt Uster, UC Davis Center for Vectorborne Diseases, University of Laval, U.S Department of State (HIU), Zonar Systems

크라우드 펀딩 캠페인

크라우드 펀딩 캠페인이란 수많은 사람들에게 서비스할 수 있는, 우리가 간절히 원하는 기능들을 후원받기 위해 진행하는 캠페인입니다. 각 캠페인은 특정 기능 또는 일련의 기능에 특화되어 있습니다. 각 후원자는 필요한 펀딩의 작은 일부분을 담당하며, 충분한 공헌자 또는 조직이 모일 경우 많은 사용자를 도울 수 있는 작업을 위한 비용을 충당할 수 있습니다. 여러분이 생각하기에 다른 사람들이 기꺼이 공동 펀딩에 참여할 만한 아이디어가 있다면, PostGIS 뉴스그룹 에 여러분의 생각을 포스팅해서 우리 모두가 함께 실현할 수 있습니다.

PostGIS 2.0.0 은 이런 전략 아래 탄생한 첫 번째 출시작입니다. 우리는 PledgBank 를 이용했으며 이를 통해 캠페인을 두 번 성공적으로 진행했습니다.

postgistopology - 10 명 이상의 후원자들이 2.0.0 버전에서 toTopGeometry 함수 빌드와 지형 지원 강화를 위해 250 달러씩 후원했습니다.

postgis64windows - 후원자 20 여 명이 윈도우 용 64 비트 PostGIS 의 문제점을 해결하기 위한 작업에 100 달러씩 지원했고, 성공했습니다. 이제 PostgreSQL 스택 빌더에서 PostGIS 2.0.1 64 비트 버전을 다운로드할 수 있습니다.

주요 지원 라이브러리

GEOS 도형 작업 라이브러리 - 마틴 데이비스가 알고리즘 작업을 통해 모든 것을 작동하도록 만들었고, 마테우스 로스코트, 산드로 산틸리(strk), 폴 램지 외 다른 이들이 유지보수 및 지원 작업을 진행 중입니다.

GDAL 공간지리 데이터 추출 라이브러리 - 프랭크 바르메르담(Frank Warmerdam) 등이 만든 이 라이브러리는 PostGIS 2.0.0 에 포함된 래스터 기능성 대부분을 강화하는 데 쓰입니다. 기브앤테이크처럼, PostGIS 를 지원하기 위해 GDAL 이 필요로 하는 개선은 역으로 GDAL 프로젝트에 공헌합니다.

Proj4 지도제작 투영 라이브러리 - 제럴드 에벤던(Gerald Evenden)과 프랭크 바르메르담이 이 라이브러리를 개발하고 유지보수하고 있습니다.

마지막으로 그러나 역시 중요한, PostGIS 의 기반이 된 PostgreSQL DBMS - PostGIS 의 유동성과 속도의 대부분은 PostgreSQL 가 제공하는 확장성, 강력한 쿼리 플래너, GiST 인덱스, 그리고 다양한 SQL 기능들 없이는 불가능했습니다.

1.5. 추가 정보

- 최신 PostGIS 소프트웨어, 문서 및 뉴스들을 다음 웹사이트 http://postgis.net 에서 찾아볼 수 있습니다.
- GEOS 도형 작업 라이브러리에 대한 상세 정보는 http://trac.osgeo.org/geos/에서 확인할 수 있습니다.
- Proj4 재투영 라이브러리에 대한 상세 정보는 http://trac.osgeo.org/proj/에서 확인할 수 있습니다.
- PostgreSQL 데이터베이스 서버에 대한 상세 정보는 PostgreSQL 사이트 http://www.postgresql.org 에서 확인할 수 있습니다.
- GiST 인덱싱에 대한 상세 정보는 PostgreSQL GiST 개발 사이트 http://www.sai.msu.su/~megera/postgres/gist/에서 확인할 수 있습니다.
- 인터넷 맵 서버인 MapServer 에 대한 상세 정보는 http://mapserver.org 에서 확인할 수 있습니다.
- "Simple Features for Specification for SQL"은 OpenGIS Consortium 웹 사이트 http://www.opengeospatial.org/ 에서 확인할 수 있습니다.

Chapter 2.설치

이 장에서는 PostGIS 설치에 요구되는 모든 과정을 설명합니다.

2.1. 짧은 설명

사용자의 검색 경로에 모든 의존성이 설정되어 있을 때 컴파일하려면:

```
tar xvfz postgis-2.3.0dev.tar.gz
cd postgis-2.3.0dev
./configure
make
make install
```

PostGIS 가 설치되면, PostGIS 를 사용하고자 하는 모든 데이터베이스 각각에서 활성화해야 합니다.



현재 래스터 지원은 옵션으로 선택할 수 있지만, 설치에 기본적으로 포함됩니다. 래스터 지원을 활성화시키려면 PostgreSQL 9.1+ 확장 프로그램 모델 래스터를 사용해야 합니다. 확장 프로그램 활성화 처리가 좀 더 사용자 친화적이고 선호되는 방법입니다. 사용자 데이터베이스를 공간적으로 활성화시키려면:

```
psql -d yourdatabase -c "CREATE EXTENSION postgis;"
psql -d yourdatabase -c "CREATE EXTENSION postgis_topology;"
-- SFCGAL 지원과 함께 빌드했을 경우 --
psql -d yourdatabase -c "CREATE EXTENSION postgis_sfcgal;"

-- TIGER 지오코더를 설치하려 할 경우 --
psql -d yourdatabase -c "CREATE EXTENSION fuzzystrmatch"
psql -d yourdatabase -c "CREATE EXTENSION postgis_tiger_geocoder;"

-- PCRE 와 함께 설치한 경우
-- 주소 표준화 도구 확장 프로그램도 설치해야 합니다.
psql -d yourdatabase -c "CREATE EXTENSION address_standardizer;"
```

설치된/사용가능한 확장 프로그램을 쿼리하거나, 확장 프로그램을 업그레이드하거나, 확장 프로그램 제외 설치를 확장 프로그램 포함 설치로 변환하는 데 대한 상세한 정보는 Section 2.4.3, “PostGIS 확장 프로그램 빌드 및 배포” 를 참조하십시오.

어떤 이유에서든 래스터 지원을 제외하고 컴파일했거나, 그저 구식을 선호하는 사용자를 위한 더 길고 더 강력한 지침이 존재합니다.

설치시 모든 .sql 파일은 사용자의 PostgreSQL 설치 디렉터리의 share/contrib/postgis-2.2 폴더에 설치됩니다.

```
createdb yourdatabase
createlang plpgsql yourdatabase
psql -d yourdatabase -f postgis.sql
psql -d yourdatabase -f postgis_comments.sql
psql -d yourdatabase -f spatial_ref_sys.sql
psql -d yourdatabase -f topology.sql
psql -d yourdatabase -f topology_comments.sql

-- only if you compiled with raster (GDAL)
psql -d yourdatabase -f rtpostgis.sql
psql -d yourdatabase -f raster_comments.sql

--if you built with sfcgal support --
psql -d yourdatabase -f sfcgal.sql
psql -d yourdatabase -f sfcgal_comments.sql
```

이 장 나머지 부분에서는 앞에 설명한 각 설치 순서에 관해 더 자세히 다룹니다.

PostGIS 2.1.3 버전부터 DB 외부 래스터(out-of-db raster; 래스터를 DB 에 넣지 않고 Path 만 넣는 방식)와 모든 래스터 드라이버는 기본적으로 비활성화되어 있습니다. 이를 다시 활성화하려면 서버 환경에서 다음 POSTGIS_GDAL_ENABLED_DRIVERS 와 POSTGIS_ENABLE_OUTDB_RASTERS 환경변수들을 설정해야 합니다. PostGIS 2.2 버전의 경우, Section 8.2, "PostGIS GUC(Grand Unified Custom Variable)"에 해당하는 보다 크로스 플랫폼적인 설정 방식을 쓸 수 있습니다.

오프라인 래스터를 활성화하고자 할 경우:

```
POSTGIS_ENABLE_OUTDB_RASTERS=1
```

설정을 이와 다르게 하거나, 아예 하지 않을 경우 DB 외부 래스터가 비활성화될 것입니다.

사용자가 설치한 GDAL 에서 사용 가능한 모든 GDAL 드라이버를 활성화하려면, 환경변수를 다음과 같이 설정하십시오.

```
POSTGIS_GDAL_ENABLED_DRIVERS="GTiff PNG JPEG GIF XYZ"
```



윈도우 환경일 경우 드라이버 목록에 따옴표를 쓰지 않습니다.

환경변수 설정 작업은 OS 에 따라 달라집니다. apt-postgresql 을 통해 우분투 또는 데비안에 설치된 PostgreSQL 의 경우 /etc/postgresql/9.3/main/environment 을 편집하는 방식이 선호됩니다. 이때 9.3 은 PostgreSQL 의 버전이고 main 은 클러스터를 가리킵니다.

윈도우의 경우, 서비스 형태로 실행하고 있다면 Computer->Properties Advanced System Settings 를 오른쪽 클릭하거나(윈도우 7) 파일 탐색기에서 Control Panel\All Control Panel Items\System 을 찾아 시스템 변수를 설정할 수 있습니다. 그 다음 Advanced System Settings ->Advanced->Environment Variables 를 클릭해서 새 시스템 변수를 추가하십시오.

환경변수를 설정한 다음, 변경 사항을 적용하려면 PostgreSQL 서비스를 재시작해야 합니다.

2.2. 설치 요구 사양

PostGIS 를 빌드하고 사용하기 위해서는 다음과 같은 요구사항들을 만족해야 합니다.

필수 사항

- PostgreSQL 9.2 또는 이후 버전. PostgreSQL 을 (서버 헤더 포함) 완전 설치해야 합니다. PostgreSQL 은 http://www.postgresql.org 에서 다운로드할 수 있습니다.

  전체 PostgreSQL/PostGIS 지원 매트릭스 및 PostGIS/GEOS 지원 매트릭스는 http://trac.osgeo.org/postgis/wiki/UsersWikiPostgreSQLPostGIS 를 참조하십시오.

- GNU C 컴파일러(gcc). PostGIS 를 컴파일하기 위해 그 밖에 다른 ANSI C 컴파일러들을 사용할 수 있으나 gcc 로 컴파일했을 경우 오류가 훨씬 적게 발생합니다.

- GNU Make(gmake 또는 make). 많은 시스템들에서 GNU make 는 make 의 기본 버전입니다. make -v 를 통해 버전을 확인하십시오. 다른 버전의 make 는 PostGIS Makefile 을 제대로 처리하지 못 할 수도 있습니다.

- Proj4 재투영 라이브러리 4.6.0 또는 이후 버전. Proj4 라이브러리는 PostGIS 내에서 좌표 재투영을 제공하는 데 사용됩니다. Proj4 는 http://trac.osgeo.org/proj/ 에서 다운로드할 수 있습니다.

- GEOS 도형 라이브러리 3.3 또는 이후 버전이 필요하지만, 모든 새로운 함수 및 기능을 완전히 사용하려면 GEOS 3.5 이상 버전을 권장합니다. GEOS 3.5 버전이 없다면 ST_ClipByBox2D 및 ST_Subdivide 등과 같은 주요 개선 사항을 놓치게 될 겁니다. GEOS 는 http://trac.osgeo.org/geos/ 에서 다운로드할 수 있으며, 3.4 이상 버전은 구 버전들과 호환되므로 안전하게 업그레이드할 수 있습니다.

- LibXML2, version 2.5.x 또는 이후 버전. LibXML2 는 현재 몇몇 임포트 함수(ST_GeomFromGML 및 ST_GeomFromKML)에 사용되고 있습니다. LibXML2 는 http://xmlsoft.org/downloads.html 에서 다운로드할 수 있습니다.

- JSON-C 0.9 또는 이후 버전. JSON-C 는 현재 ST_GeomFromGeoJson 함수를 통해 GeoJSON 을 임포트하는 데 사용되고 있습니다. JSON-C 는 https://github.com/json-c/json-c/releases/ 에서 다운로드할 수 있습니다.

- GDAL 1.8 또는 이후 버전(구 버전에서 제대로 동작하지 않거나 다르게 동작하는 기능들이 있으므로 1.9 이상 버전을 강력히 권장합니다). GDAL 은 래스터 지원에 필요하며, CREATE EXTENSION postgis 명령어를 통해 설치할 수 있으므로 PostgreSQL 9.1 이상 버전을 실행하는 사용자에게 특히 권장합니다. http://trac.osgeo.org/gdal/wiki/DownloadSource.

선택 사항

- 사용자가 래스터 기능을 원하지 않고 CREATE EXTENSION postgis 와 함께 설치하지 않아도 될 경우에만 (가(假)선택적) GDAL 을 빼놓을 수 있습니다. PostGIS 를 확장 프로그램으로서 설치하지 않는다면, 다른 확장 프로그램을 설치하지 못 하게 막는 필수 PostGIS 확장 프로그램이 있을 수도 있다는 점을 기억하십시오. 따라서 GDAL 지원과 함께 컴파일할 것을 강력히 권장합니다.

  또 Section 2.1, “짧은 설명” 에 설명된 대로 사용자가 사용하길 원하는 드라이버를 활성화하는 작업도 잊지 마십시오.

- shapefile 로더 shp2pgsql-gui 를 컴파일하기 위한 GTK(GTK+2.0, 2.8+ 필요). http://www.gtk.org/.

- PostGIS 에 추가적인 2D 및 3D 고급 분석 기능을 제공하는 데 SFCGAL 1.1 (또는 그 이후) 버전을 사용할 수 있습니다. Section 8.10, "SFCGAL 함수" 를 참조하십시오. 또 양쪽 백엔드가 제공하는 GEOS 의 몇몇 2D 기능 대신 (예를 들어 ST_Intersection 또는 ST_Area) SFCGAL 을 사용하게 할 수도 있습니다. SFCGAL 을 설치했을 경우 (기본적으로는 GEOS 설치) PostgreSQL 설정 변수 postgis.backend 를 통해 말단 사용자가 원하는 백엔드를 조정할 수 있습니다. 주의: SFCGAL 1.2 버전은 최소 CGAL 4.3 과 Boost 1.54 를 필요로 합니다(http://oslandia.github.io/SFCGAL/installation.html 참조). https://github.com/Oslandia/SFCGAL.

- Chapter 12, 주소 표준화 도구 를 빌드하려면 PCRE 도 필요합니다(일반적으로 유닉스 파생 시스템에 이미 설치되어 있습니다). parseaddress-stcities.h 파일에 인코딩되어 있는 데이터를 다시 빌드하려 하는 경우에만 Regex::Assemble 펄 CPAN 패키지가 필요합니다. Chapter 12, 주소 표준화 도구 는 PCRE 라이브러리를 감지할 경우 자동적으로 빌드될 것입니다. 또는 설정 과정에서 유효한 --with-pcre-dir=/path/to/pcre 경로 변수를 입력할 수도 있습니다.

- CUnit(CUnit). 회기 검증을 하는 데 필요합니다. http://cunit.sourceforge.net/

- DocBook(xsltproc)은 문서를 발행할 때 필요합니다. DocBook 은 http://www.docbook.org/ 에서 다운로드할 수 있습니다.

- DBLatex(dblatex)는 PDF 형식의 문서를 발행할 때 필요합니다. DBLatex 는 http://dblatex.sourceforge.net/ 에서 다운로드할 수 있습니다.

- ImageMagick(convert)은 문서에 사용되는 이미지를 생성하는 데 필요합니다. ImageMagick 은 http://www.imagemagick.org/ 에서 다운로드할 수 있습니다.

2.3. 소스 획득

PostGIS 소스 압축파일을 다운로드 웹사이트 http://postgis.net/stuff/postgis-2.3.0dev.tar.gz 에서 다운로드하십시오.

```
wget http://postgis.net/stuff/postgis-2.3.0dev.tar.gz
tar -xvzf postgis-2.3.0dev.tar.gz
```

이 명령어를 실행하면 현재 작업 디렉토리에 postgis-2.3.0dev (이)라는 명칭의 디렉터리가 생길 것입니다.

다른 방법으로는, svn 저장소 http://svn.osgeo.org/postgis/trunk/ 에서 소스를 체크아웃(checkout)할 수 있습니다.

```
svn checkout http://svn.osgeo.org/postgis/trunk/ postgis-2.3.0dev
```

설치를 계속하려면 새로 만든 `postgis-2.3.0dev` 디렉터리로 이동합니다.

2.4. 소스를 컴파일하고 설치하기: 상세 설명



현재 많은 OS 시스템들이 PostgreSQL/PostGIS 용 사전 구축된 패키지를 포함하고 있습니다. 많은 경우, 최첨단 버전이 필요하거나 패키지 관리자인 경우가 아니라면 컴파일할 필요가 없습니다.

이 단원에서는 일반적인 컴파일에 대해 설명하고 있으므로, 더 상세한 설명을 원할 경우 PostGIS User contributed compile guides 와 PostGIS Dev Wiki 를 참조하십시오.

PostGIS Pre-built Packages 에서 다양한 OS 를 지원하는 사전 빌드된 패키지들을 확인할 수 있습니다.
만약 윈도우 사용자라면 Stackbuilder 또는 PostGIS Windows download site 를 통해 안정적인 버전을 받을 수 있습니다. 또 1~2 주에 한 번 혹은 뭔가 특이한 일이 일어날 경우 빌드를 하는 very bleeding-edge windows experimental builds 가 있습니다. 사용자는 이를 통해 진행중인 PostGIS 배포본을 실행해볼 수 있습니다.

PostGIS 모듈은 PostgreSQL 백엔드 서버로의 확장 프로그램입니다. PostGIS 2.3.0dev 를 컴파일하기 위해서는 완전한 PostgreSQL 서버 헤더 접근이 필요합니다. 이때 PostgreSQL 9.2 또는 이후 버전을 대상으로 컴파일할 수 있습니다. PostgreSQL 이전 버전은 지원하지 않습니다

아직 PostgreSQL 을 설치하지 않았다면 PostgreSQL 설치 지침서를 참조하십시오. http://www.postgresql.org .



GEOS 기능성을 위해 PostgreSQL 설치 시 표준 C++ 라이브러리를 명확하게 링크해야 할 수도 있습니다.

```
LDFLAGS=-lstdc++ ./configure [YOUR OPTIONS HERE]
```

이것은 구 버전 개발 도구 이용시 거짓 C++ 예외 상호작용을 피하기 위한 방법입니다. 만약 사용자가 이상한 문제(백엔드가 갑자기 끊어진다거나 또는 비슷한 증상)를 겪는다면 PostgreSQL 을 다시 컴파일해야 할 수도 있습니다.

다음은 PostGIS 소스를 설정하고 취합하는 단계를 설명합니다. 리눅스 사용자를 위한 설명이므로 윈도우 또는 맥 사용자에겐 해당되지 않습니다.

2.4.1. 설정

대부분의 리눅스 설치에서 첫 번째 단계는 소스 코드를 빌드하는 데 사용할 Makefile 을 생성하는 것입니다. 셸 스크립트를 실행해서 Makefile 을 작성합니다.

./configure

추가 파라미터 없이 사용하면, 이 명령은 자동적으로 PostGIS 소스코드를 사용자 시스템에 빌드하는 데 필요한 필수 구성 요소들과 라이브러리의 위치를 확인하려 합니다. ./configure 명령어의 가장 흔한 사용법이긴 하지만, 이 스크립트는 비표준적인 위치에 있는 필수 라이브러리와 프로그램들에 대한 몇몇 파라미터를 받습니다.

다음은 가장 많이 사용되는 파라미터들만 나열한 목록입니다. 전체 목록은 --help 또는 --help=short 파라미터를 사용하십시오.

--prefix=PREFIX PostGIS 라이브러리와 SQL 스크립트가 설치될 경로입니다. 기본적으로 PostgreSQL 가 설치된 경로와 같은 곳에 설치됩니다.



이 파라미터는 현재 작동되지 않습니다. 패키지가 PostgreSQL 설치 경로에만 설치될 것이기 때문입니다. 해당 버그를 추적하려면 http://trac.osgeo.org/postgis/ticket/635 를 참조하십시오.

--with-pgconfig=FILE PostgreSQL 은 PostGIS 같은 확장 프로그램이 PostgreSQL 설치 디렉토리의 위치를 확인하게 해주는 **pg_config** 라는 유틸리티를 제공합니다. PostGIS 빌드 시 대상이 될 특정 PostgreSQL 설치 디렉터리를 사용자가 직접 설정하려면 이 파라미터**(--with-pgconfig=/path/to/pg_config**)를 사용하십시오.

--with-gdalconfig=FILE GDAL 은 래스터 지원을 위한 기능을 제공하는 필수 라이브러리로, GDAL 설치 디렉터리의 위치를 확인하는 소프트웨어 설치를 활성화하기 위한 **gdal-config** 를 지원합니다. PostGIS 빌드 시 대상이 될 특정 GDAL 설치 디렉터리를 사용자가 직접 설정하려면 이 파라미터**(--with-gdalconfig=/path/to/gdal-config**)를 사용하십시오.

--with-geosconfig=FILE GEOS 는 필수 도형 라이브러리로, GEOS 설치 디렉터리의 위치를 확인하는 소프트웨어 설치를 활성화하기 위한 **geos-config** 라는 유틸리티를 제공합니다. PostGIS 빌드 시 대상이 될 특정 GEOS 설치 디렉터리를 사용자가 직접 설정하려면 이 파라미터**(--with-geosconfig=/path/to/geos-config**)를 사용하십시오.

--with-xml2config=FILE LibXML 은 GeomFromKML/GML 프로세스를 진행하기 위해 필요한 라이브러리입니다. 일반적으로 libxml 을 설치하면 찾을 수 있지만, 설치하지 않았거나 특정 버전을 사용하기 바랄 경우 LibXML 설치 디렉터리의 위치를 확인하는 소프트웨어 설치를 활성화하기 위해 xml2-config 라는 설정 파일에 PostGIS 의 위치를 지정해야 합니다. PostGIS 빌드 시 대상이 될 특정 LibXML 설치 디렉터리를 사용자가 직접 설정하려면 이 파라미터(**--with-xml2config=/path/to/xml2-config**)를 사용하십시오.

--with-projdir=DIR Proj4 는 PostGIS 필수 재투영 라이브러리입니다. PostGIS 빌드 시 대상이 될 특정 Proj4 설치 디렉터리를 사용자가 직접 설정하려면 이 파라미터(**--with-projdir=/path/to/projdir**)를 사용하십시오.

--with-libiconv=DIR Iconv 설치 경로

--with-jsondir=DIR JSON-C 는 MIT-라이선스의 JSON 라이브러리로, PostGIS 의 ST_GeomFromJSON 지원에 필요합니다. PostGIS 빌드 시 대상이 될 특정 JSON-C 설치 디렉터리를 사용자가 직접 설정하려면 이 파라미터(**--with-jsondir=/path/to/jsondir**)를 사용하십시오.

--with-pcredir=DIR PCRE 는 BSD-라이선스의 펄 호환 가능 정규 표현식 라이브러리로, address_standardizer 확장 프로그램이 필요합니다. PostGIS 빌드 시 대상이 될 특정 PCRE 설치 디렉터리를 사용자가 직접 설정하려면 이 파라미터(**--with-pcredir=/path/to/pcredir**)를 사용하십시오.

--with-gui 데이터 임포트 GUI 컴파일(GTK+2.0 필요). shp2pgsql-gui 의 shp2pgsql 에 대한 그래픽 인테페이스를 생성합니다.

--with-raster 래스터 지원이 되도록 컴파일. rtpostgis-2.3.0dev 라이브러리와 rtpostgis.sql 파일을 빌드합니다. 최종 배포본에서는 기본적으로 래스터 지원을 빌드할 것이기 때문에 필요하지 않을 수도 있습니다.

--with-topology 지형 지원이 되도록 빌드합니다. topology.sql 파일을 빌드할 것입니다. 지형에 필요한 모든 논리가 postgis-2.3.0dev 라이브러리 내에 존재하기 때문에, 이에 상응하는 다른 라이브러리는 없습니다.

--with-gettext=no 기본적으로 PostGIS 는 gettext 지원을 감지해서 함께 컴파일하지만, 로더의 파손을 야기하는 비호환성 문제가 발생할 경우 이 명령어로 gettext 지원을 완전히 비활성화시킬 수 있습니다. 이런 방법으로 설정을 변경해서 문제를 해결하는 예는 버그 티켓 http://trac.osgeo.org/postgis/ticket/748 을 참조하십시오. 주의: gettext 지원을 끈다고 해서 별다른

문제는 없습니다. gettext 지원은 아직 문서화되지도 않았고 검증중에 있는 GUI 로더 용 국제 도움말/라벨 지원에 사용됩니다.

--with-sfcgal=PATH 기본적으로 PostGIS 는 이 스위치 없이는 sfcgal 지원과 함께 설치되지 않습니다. PATH 는 sfcgal-config 를 가리키는 대체 경로를 지정하도록 해주는 선택적인 인자입니다.



PostGIS 를 SVN 저장소 에서 얻었다면, 먼저 다음 스크립트를 실행하십시오.
./autogen.sh
이 스크립트는 **configure** 스크립트를 생성하는데, 이 스크립트는 PostGIS 의 사용자 지정 설치를 위해 이용됩니다.
만약 tar 파일 형태로 PostGIS 를 얻었다면 이미 **configure** 가 생성되었기 때문에 **./autogen.sh** 를 실행할 필요는 없습니다.

2.4.2. 빌드

일단 Makefile 이 생성되면 PostGIS 빌드 작업은 실행만큼이나 쉽습니다.

make 산출물의 마지막 줄에 "PostGIS was built successfully. Ready to install."이란 문장이 보여야 합니다.

PostGIS 1.4.0 버전부터, 모든 함수는 문서에서 생성된 주석을 달고 있습니다. 이후 공간 데이터베이스에 이 주석들을 넣고 싶다면 docbook 을 필요로 하는 명령어를 실행하십시오. postgis_comments.sql 과 다른 패키지 주석 파일인 raster_comments.sql, topology_comments.sql 은 doc 폴더 안의 tar.gz 배포 파일에 패키징되어 있으므로, tar 파일로부터 설치한 경우에는 따로 주석을 만들 필요가 없습니다.

make comments PostGIS 2.0 버전부터 소개되었습니다. 빠른 참조 또는 학습용 유인물에 적합한 참조 자료(cheat sheet) html 파일을 생성합니다. 파일 생성에 xsltproc 가 필요하며, doc 폴더 안에 다음 topology_cheatsheet.html, tiger_geocoder_cheatsheet.html, raster_cheatsheet.html, postgis_cheatsheet.html 4 개의 파일을 생성할 것입니다.

html 및 pdf 형식으로 미리 만들어진 파일들을 PostGIS / PostgreSQL Study Guides 에서 다운로드받을 수 있습니다.

make cheatsheets

2.4.3. PostGIS 확장 프로그램 빌드 및 배포

PostgreSQL 9.1 이상 버전을 사용중이라면 PostGIS 확장 프로그램은 자동적으로 빌드 및 설치됩니다.

소스 저장소로부터 빌드할 경우, 먼저 함수 설명(function description)부터 빌드해야 합니다. docbook 을 설치한 경우 빌드됩니다. 다음 명령어를 통해 직접 생성할 수도 있습니다.

```
make comments
```

사용자가 tar 파일 배포본으로부터 빌드한다면 미리 빌드된 주석이 tar 파일과 함께 패키지되기 때문에 주석을 따로 빌드할 필요가 없습니다.

PostgreSQL 9.1 버전을 대상으로 빌드하는 경우, make install 과정의 한 단계에서 확장 프로그램을 자동적으로 빌드할 것입니다. 필요할 경우 확장 프로그램 폴더로부터 빌드하거나, 다른 서버에서 필요할 경우 파일을 복사할 수 있습니다.

```
cd extensions
cd postgis
make clean
make
make install
cd ..
cd postgis_topology
make clean
make
make install
cd ..
cd postgis_sfcgal
make clean
make
make install

cd ..
cd address_standardizer
make clean
make
make install
make installcheck

cd ..
cd postgis_tiger_geocoder
make clean
make
make install
make installcheck
```

PostGIS 버전만 같다면 OS 에 상관없이 확장 프로그램 파일은 언제나 동일할 것입니다. 따라서 사용자 서버에 이미 PostGIS 바이너리가 설치되어 있다면 한 OS 에서 다른 OS 로 확장 프로그램 파일만 복사해도 문제가 없습니다.

만약 확장 프로그램을 사용자 개발 서버가 아닌 다른 서버에 직접 설치하고 싶다면, 다음 파일들을 확장 프로그램 폴더에서 다른 서버에 있는 PostgreSQL 설치 경로의 PostgreSQL / share / extension 폴더에 복사해야 합니다. 이때 해당 서버에 PostGIS 가 설치되어 있지 않다면 정식 PostGIS 의 바이너리 또한 복사해줘야 합니다.

- postgis.control, postgis_topology.control 은, 지정되어 있지 않을 경우, 설치할 확장 프로그램의 버전 등과 같은 정보를 표시하는 제어 파일입니다.
- extensions/postgis/sql/*.sql, extensions/postgis_topology/sql/*.sql 은 각 확장 프로그램의 /sql 폴더 안의 모든 파일들을 나타냅니다. PostgreSQL 의 share/extension 폴더 루트에 복사해야 한다는 점을 주의하십시오.

복사가 끝나면, PgAdmin -> extensions 메뉴에서 postgis, postgis_topology 가 사용할 수 있는 확장 프로그램으로 나타난 것을 볼 수 있을 겁니다.

만약 PSQL 을 이용하고 있다면 다음 쿼리를 실행해서 확장 프로그램이 설치된 것을 확인할 수 있습니다.

```
SELECT name, default_version,installed_version
```

```
FROM pg_available_extensions WHERE name LIKE 'postgis%' or name LIKE
'address%';

          name           | default_version | installed_version
-----------------------------+-----------------+-------------------
 address_standardizer       | 2.3.0dev        | 2.3.0dev
 address_standardizer_data_us | 2.3.0dev        | 2.3.0dev
 postgis                    | 2.3.0dev        | 2.3.0dev
 postgis_sfcgal             | 2.3.0dev        |
 postgis_tiger_geocoder     | 2.3.0dev        | 2.3.0dev
 postgis_topology           | 2.3.0dev        |
(6 rows)
```

사용자가 쿼리한 데이터베이스에 확장 프로그램이 설치되었다면 installed_version 열에 언급된 것을 볼 수 있을 겁니다. 반환되는 레코드가 없다면, 애초에 해당 서버에 PostGIS 확장 프로그램이 설치되지 않았다는 뜻입니다. PgAdmin III 1.14 이상 버전 또한 데이터베이스 브라우저 트리의 extensions 부분에서 이 정보를 제공하며, 오른쪽 클릭을 통해 확장 프로그램을 업그레이드 또는 설치 해제할 수도 있습니다.

이용 가능한 확장 프로그램이 있다면, pgAdmin 확장 프로그램 인터페이스 또는 다음 sql 명령어를 통해 사용자가 선택한 데이터베이스에 PostGIS 확장 프로그램을 설치할 수 있습니다.

```
CREATE EXTENSION postgis;
CREATE EXTENSION postgis_sfcgal;
CREATE EXTENSION fuzzystrmatch; --needed for postgis_tiger_geocoder
--optional used by postgis_tiger_geocoder, or can be used standalone
CREATE EXTENSION address_standardizer;
CREATE EXTENSION address_standardizer_data_us;
CREATE EXTENSION postgis_tiger_geocoder;
CREATE EXTENSION postgis_topology;
```

PSQL 에서 다음 명령어를 사용하면 어떤 버전을 설치했는지, 어떤 스키마로 설치했는지 알 수 있습니다.

```
\connect mygisdb
\x
\dx postgis*
List of installed extensions
-[ RECORD 1 ]-------------------------------------------------
-
Name       | postgis
Version    | 2.3.0dev
Schema     | public
Description | PostGIS geometry, geography, and raster spat..
-[ RECORD 2 ]-------------------------------------------------
-
Name       | postgis_tiger_geocoder
Version    | 2.3.0dev
Schema     | tiger
Description | PostGIS tiger geocoder and reverse geocoder
-[ RECORD 3 ]-------------------------------------------------
-
Name       | postgis_topology
Version    | 2.3.0dev
Schema     | topology
Description | PostGIS topology spatial types and functions
```



spatial_ref_sys, layer, topology 확장 프로그램 테이블을 확실히 백업할 수는 없습니다. postgis 또는 postgis_topology 확장 프로그램을 각각 백업하는 경우에만 -- 데이터베이스 전체를 백업해야 확장 프로그램이 백업되는 것 같습니다 -- 확장 프로그램 테이블을 백업할 수 있습니다. PostGIS 2.0.1 버전부터 데이터베이스를 백업할 때 PostGIS 로 패키지하지 않은 SRID 레코드만 백업되기 때문에, 패키징한 SRID 를 변경한 다음 그 변경사항이 남아 있으리라고 기대하지는 마십시오. 문제점을 발견했다면 버그 티켓을 제출하십시오. 확장 프로그램 테이블의 구조는 CREATE EXTENSION 으로 생성되었으며 확장 프로그램의 주어진 버전에서 동일할 것으로 가정하므로 백업되지 않습니다. 이런 동작은 현재 PostgreSQL 확장 프로그램 모델에 내장되어 있기 때문에 어떻게 할 수 있는 방법은 없습니다.

우리의 멋진 확장 프로그램 시스템 없이 2.3.0dev 을 설치했다면, 먼저 다음 업그레이드 스크립트를 실행해서 확장 프로그램 기반 최신 버전으로 변경할 수 있습니다:

```
postgis_upgrade_22_minor.sql,raster_upgrade_22_minor.sql,topology_upgrade_22_minor.sql.
```

만일 래스터 지원 없이 PostGIS 를 설치했다면, 래스터 지원을 먼저 설치해야 합니다(rtpostgis.sql 전체 사용).

그런 다음 확장 프로그램 각각의 함수들을 패키징하기 위한 다음 명령어들을 실행할 수 있습니다.

```
CREATE EXTENSION postgis FROM unpackaged;
CREATE EXTENSION postgis_topology FROM unpackaged;
CREATE EXTENSION postgis_tiger_geocoder FROM unpackaged;
```

2.4.4. 테스트

만약 PostGIS 빌드를 테스트하고 싶다면, 다음을 실행하십시오.

Make check 이 명령어는 실제 PostgreSQL 데이터베이스를 대상으로 생성된 라이브러리를 이용하여 여러 가지 검사 및 회귀 테스트를 실행할 것입니다.



표준이 아닌 PostgreSQL, GEOS, 또는 Proj4 위치를 이용해서 PostGIS 를 설치한 경우, LD_LIBRARY_PATH 환경변수에 해당 라이브러리 경로를 설정해주어야 할 수도 있습니다.



현재 **make check** 명령어로 검사 작업을 할 때, 설정 파라미터 **--with-pgconfig** 를 통해 지정되었을 수도 있는 PostgreSQL 버전을 사용하지 않고 PATH 및 PGPORT 환경변수에 의존합니다. 따라서 설정 과정에서 PostgreSQL 설치 경로와 일치하도록 사용자의 PATH 를 수정하거나, 또는 곧 닥칠 두통에 대비하십시오.

성공하였다면 테스트의 결과는 아래와 비슷하게 나올 것입니다.

```
CUnit - A unit testing framework for C - Version 2.1-2
    http://cunit.sourceforge.net/


Suite: computational_geometry
  Test: test_lw_segment_side ...passed
  Test: test_lw_segment_intersects ...passed
```

```
  Test: test_lwline_crossing_short_lines ...passed
  Test: test_lwline_crossing_long_lines ...passed
  Test: test_lwline_crossing_bugs ...passed
  Test: test_lwpoint_set_ordinate ...passed
  Test: test_lwpoint_get_ordinate ...passed
  Test: test_point_interpolate ...passed
  Test: test_lwline_clip ...passed
  Test: test_lwline_clip_big ...passed
  Test: test_lwmline_clip ...passed
  Test: test_geohash_point ...passed
  Test: test_geohash_precision ...passed
  Test: test_geohash ...passed
  Test: test_geohash_point_as_int ...passed
  Test: test_isclosed ...passed
  Test: test_lwgeom_simplify ...passed
Suite: buildarea
  Test: buildarea1 ...passed
  Test: buildarea2 ...passed
  Test: buildarea3 ...passed
  Test: buildarea4 ...passed
  Test: buildarea4b ...passed
  Test: buildarea5 ...passed
  Test: buildarea6 ...passed
  Test: buildarea7 ...passed
Suite: geometry_clean
  Test: test_lwgeom_make_valid ...passed
Suite: clip_by_rectangle
  Test: test_lwgeom_clip_by_rect ...passed
Suite: force_sfs
  Test: test_sfs_11 ...passed
  Test: test_sfs_12 ...passed
  Test: test_sqlmm ...passed
Suite: geodetic
  Test: test_sphere_direction ...passed
  Test: test_sphere_project ...passed
  Test: test_lwgeom_area_sphere ...passed
  Test: test_signum ...passed
  Test: test_gbox_from_spherical_coordinates ...passed
  Test: test_gserialized_get_gbox_geocentric ...passed
  Test: test_clairaut ...passed
  Test: test_edge_intersection ...passed
  Test: test_edge_intersects ...passed
  Test: test_edge_distance_to_point ...passed
  Test: test_edge_distance_to_edge ...passed
  Test: test_lwgeom_distance_sphere ...passed
  Test: test_lwgeom_check_geodetic ...passed
  Test: test_gserialized_from_lwgeom ...passed
  Test: test_spheroid_distance ...passed
  Test: test_spheroid_area ...passed
  Test: test_lwpoly_covers_point2d ...passed
  Test: test_gbox_utils ...passed
  Test: test_vector_angle ...passed
  Test: test_vector_rotate ...passed
  Test: test_lwgeom_segmentize_sphere ...passed
  Test: test_ptarray_contains_point_sphere ...passed
  Test: test_ptarray_contains_point_sphere_iowa ...passed
Suite: GEOS
  Test: test_geos_noop ...passed
  Test: test_geos_subdivide ...passed
  Test: test_geos_linemerge ...passed
Suite: Clustering
  Test: basic_test ...passed
```

```
  Test: nonsequential_test ...passed
  Test: basic_distance_test ...passed
  Test: single_input_test ...passed
  Test: empty_inputs_test ...passed
Suite: Clustering Union-Find
  Test: test_unionfind_create ...passed
  Test: test_unionfind_union ...passed
  Test: test_unionfind_ordered_by_cluster ...passed
Suite: homogenize
  Test: test_coll_point ...passed
  Test: test_coll_line ...passed
  Test: test_coll_poly ...passed
  Test: test_coll_coll ...passed
  Test: test_geom ...passed
  Test: test_coll_curve ...passed
Suite: encoded_polyline_input
  Test: in_encoded_polyline_test_geoms ...passed
  Test: in_encoded_polyline_test_precision ...passed
Suite: geojson_input
  Test: in_geojson_test_srid ...passed
  Test: in_geojson_test_bbox ...passed
  Test: in_geojson_test_geoms ...passed
Suite: twkb_input
  Test: test_twkb_in_point ...passed
  Test: test_twkb_in_linestring ...passed
  Test: test_twkb_in_polygon ...passed
  Test: test_twkb_in_multipoint ...passed
  Test: test_twkb_in_multilinestring ...passed
  Test: test_twkb_in_multipolygon ...passed
  Test: test_twkb_in_collection ...passed
  Test: test_twkb_in_precision ...passed
Suite: serialization/deserialization
  Test: test_typmod_macros ...passed
  Test: test_flags_macros ...passed
  Test: test_serialized_srid ...passed
  Test: test_gserialized_from_lwgeom_size ...passed
  Test: test_gbox_serialized_size ...passed
  Test: test_lwgeom_from_gserialized ...passed
  Test: test_lwgeom_count_vertices ...passed
  Test: test_on_gser_lwgeom_count_vertices ...passed
  Test: test_geometry_type_from_string ...passed
  Test: test_lwcollection_extract ...passed
  Test: test_lwgeom_free ...passed
  Test: test_lwgeom_flip_coordinates ...passed
  Test: test_f2d ...passed
  Test: test_lwgeom_clone ...passed
  Test: test_lwgeom_force_clockwise ...passed
  Test: test_lwgeom_calculate_gbox ...passed
  Test: test_lwgeom_is_empty ...passed
  Test: test_lwgeom_same ...passed
  Test: test_lwline_from_lwmpoint ...passed
  Test: test_lwgeom_as_curve ...passed
  Test: test_lwgeom_scale ...passed
  Test: test_gserialized_is_empty ...passed
  Test: test_gbox_same_2d ...passed
Suite: measures
  Test: test_mindistance2d_tolerance ...passed
  Test: test_rect_tree_contains_point ...passed
  Test: test_rect_tree_intersects_tree ...passed
  Test: test_lwgeom_segmentize2d ...passed
  Test: test_lwgeom_locate_along ...passed
  Test: test_lw_dist2d_pt_arc ...passed
  Test: test_lw_dist2d_seg_arc ...passed
```

```
  Test: test_lw_dist2d_arc_arc ...passed
  Test: test_lw_arc_length ...passed
  Test: test_lw_dist2d_pt_ptarrayarc ...passed
  Test: test_lw_dist2d_ptarray_ptarrayarc ...passed
  Test: test_lwgeom_tcpa ...passed
  Test: test_lwgeom_is_trajectory ...passed
Suite: effectivearea
  Test: do_test_lwgeom_effectivearea_lines ...passed
  Test: do_test_lwgeom_effectivearea_polys ...passed
Suite: miscellaneous
  Test: test_misc_force_2d ...passed
  Test: test_misc_simplify ...passed
  Test: test_misc_count_vertices ...passed
  Test: test_misc_area ...passed
  Test: test_misc_wkb ...passed
  Test: test_grid ...passed
Suite: noding
  Test: test_lwgeom_node ...passed
Suite: encoded_polyline_output
  Test: out_encoded_polyline_test_geoms ...passed
  Test: out_encoded_polyline_test_srid ...passed
  Test: out_encoded_polyline_test_precision ...passed
Suite: geojson_output
  Test: out_geojson_test_precision ...passed
  Test: out_geojson_test_dims ...passed
  Test: out_geojson_test_srid ...passed
  Test: out_geojson_test_bbox ...passed
  Test: out_geojson_test_geoms ...passed
Suite: gml_output
  Test: out_gml_test_precision ...passed
  Test: out_gml_test_srid ...passed
  Test: out_gml_test_dims ...passed
  Test: out_gml_test_geodetic ...passed
  Test: out_gml_test_geoms ...passed
  Test: out_gml_test_geoms_prefix ...passed
  Test: out_gml_test_geoms_nodims ...passed
  Test: out_gml2_extent ...passed
  Test: out_gml3_extent ...passed
Suite: kml_output
  Test: out_kml_test_precision ...passed
  Test: out_kml_test_dims ...passed
  Test: out_kml_test_geoms ...passed
  Test: out_kml_test_prefix ...passed
Suite: svg_output
  Test: out_svg_test_precision ...passed
  Test: out_svg_test_dims ...passed
  Test: out_svg_test_relative ...passed
  Test: out_svg_test_geoms ...passed
  Test: out_svg_test_srid ...passed
Suite: x3d_output
  Test: out_x3d3_test_precision ...passed
  Test: out_x3d3_test_geoms ...passed
  Test: out_x3d3_test_option ...passed
Suite: ptarray
  Test: test_ptarray_append_point ...passed
  Test: test_ptarray_append_ptarray ...passed
  Test: test_ptarray_locate_point ...passed
  Test: test_ptarray_isccw ...passed
  Test: test_ptarray_signed_area ...passed
  Test: test_ptarray_unstroke ...passed
  Test: test_ptarray_insert_point ...passed
  Test: test_ptarray_contains_point ...passed
```

```
  Test: test_ptarrayarc_contains_point ...passed
  Test: test_ptarray_scale ...passed
Suite: printing
  Test: test_lwprint_default_format ...passed
  Test: test_lwprint_format_orders ...passed
  Test: test_lwprint_optional_format ...passed
  Test: test_lwprint_oddball_formats ...passed
  Test: test_lwprint_bad_formats ...passed
Suite: SFCGAL
  Test: test_sfcgal_noop ...passed
Suite: split
  Test: test_lwline_split_by_point_to ...passed
  Test: test_lwgeom_split ...passed
Suite: stringbuffer
  Test: test_stringbuffer_append ...passed
  Test: test_stringbuffer_aprintf ...passed
Suite: surface
  Test: triangle_parse ...passed
  Test: tin_parse ...passed
  Test: polyhedralsurface_parse ...passed
  Test: surface_dimension ...passed
Suite: Internal Spatial Trees
  Test: test_tree_circ_create ...passed
  Test: test_tree_circ_pip ...passed
  Test: test_tree_circ_pip2 ...passed
  Test: test_tree_circ_distance ...passed
  Test: test_tree_circ_distance_threshold ...passed
Suite: triangulate
  Test: test_lwgeom_delaunay_triangulation ...passed
Suite: twkb_output
  Test: test_twkb_out_point ...passed
  Test: test_twkb_out_linestring ...passed
  Test: test_twkb_out_polygon ...passed
  Test: test_twkb_out_multipoint ...passed
  Test: test_twkb_out_multilinestring ...passed
  Test: test_twkb_out_multipolygon ...passed
  Test: test_twkb_out_collection ...passed
  Test: test_twkb_out_idlist ...passed
Suite: varint
  Test: test_zigzag ...passed
  Test: test_varint ...passed
  Test: test_varint_roundtrip ...passed
Suite: wkb_input
  Test: test_wkb_in_point ...passed
  Test: test_wkb_in_linestring ...passed
  Test: test_wkb_in_polygon ...passed
  Test: test_wkb_in_multipoint ...passed
  Test: test_wkb_in_multilinestring ...passed
  Test: test_wkb_in_multipolygon ...passed
  Test: test_wkb_in_collection ...passed
  Test: test_wkb_in_circularstring ...passed
  Test: test_wkb_in_compoundcurve ...passed
  Test: test_wkb_in_curvpolygon ...passed
  Test: test_wkb_in_multicurve ...passed
  Test: test_wkb_in_multisurface ...passed
  Test: test_wkb_in_malformed ...passed
Suite: wkb_output
  Test: test_wkb_out_point ...passed
  Test: test_wkb_out_linestring ...passed
  Test: test_wkb_out_polygon ...passed
  Test: test_wkb_out_multipoint ...passed
  Test: test_wkb_out_multilinestring ...passed
  Test: test_wkb_out_multipolygon ...passed
```

```
  Test: test_wkb_out_collection ...passed
  Test: test_wkb_out_circularstring ...passed
  Test: test_wkb_out_compoundcurve ...passed
  Test: test_wkb_out_curvpolygon ...passed
  Test: test_wkb_out_multicurve ...passed
  Test: test_wkb_out_multisurface ...passed
  Test: test_wkb_out_polyhedralsurface ...passed
Suite: wkt_input
  Test: test_wkt_in_point ...passed
  Test: test_wkt_in_linestring ...passed
  Test: test_wkt_in_polygon ...passed
  Test: test_wkt_in_multipoint ...passed
  Test: test_wkt_in_multilinestring ...passed
  Test: test_wkt_in_multipolygon ...passed
  Test: test_wkt_in_collection ...passed
  Test: test_wkt_in_circularstring ...passed
  Test: test_wkt_in_compoundcurve ...passed
  Test: test_wkt_in_curvpolygon ...passed
  Test: test_wkt_in_multicurve ...passed
  Test: test_wkt_in_multisurface ...passed
  Test: test_wkt_in_tin ...passed
  Test: test_wkt_in_polyhedralsurface ...passed
  Test: test_wkt_in_errlocation ...passed
Suite: wkt_output
  Test: test_wkt_out_point ...passed
  Test: test_wkt_out_linestring ...passed
  Test: test_wkt_out_polygon ...passed
  Test: test_wkt_out_multipoint ...passed
  Test: test_wkt_out_multilinestring ...passed
  Test: test_wkt_out_multipolygon ...passed
  Test: test_wkt_out_collection ...passed
  Test: test_wkt_out_circularstring ...passed
  Test: test_wkt_out_compoundcurve ...passed
  Test: test_wkt_out_curvpolygon ...passed
  Test: test_wkt_out_multicurve ...passed
  Test: test_wkt_out_multisurface ...passed

Run Summary:    Type  Total    Ran Passed Failed Inactive
              suites     38     38    n/a      0        0
               tests    251    251    251      0        0
             asserts   2468   2468   2468      0      n/a

Elapsed time =    0.298 seconds

Creating database 'postgis_reg'
Loading PostGIS into 'postgis_reg'
  /projects/postgis/branches/2.2/regress/00-regress-
install/share/contrib/postgis/postgis.sql
  /projects/postgis/branches/2.2/regress/00-regress-
install/share/contrib/postgis/postgis_comments.sql
Loading SFCGAL into 'postgis_reg'
  /projects/postgis/branches/2.2/regress/00-regress-
install/share/contrib/postgis/sfcgal.sql
  /projects/postgis/branches/2.2/regress/00-regress-
install/share/contrib/postgis/sfcgal_comments.sql
PostgreSQL 9.4.4, compiled by Visual C++ build 1800, 32-bit
  Postgis 2.2.0dev - r13980 - 2015-08-23 06:13:07
  scripts 2.2.0dev r13980
  GEOS: 3.5.0-CAPI-1.9.0 r4088
  PROJ: Rel. 4.9.1, 04 March 2015
  SFCGAL: 1.1.0
```

```
Running tests

 loader/Point ............. ok
 loader/PointM ............. ok
 loader/PointZ ............. ok
 loader/MultiPoint ............. ok
 loader/MultiPointM ............. ok
 loader/MultiPointZ ............. ok
 loader/Arc ............. ok
 loader/ArcM ............. ok
 loader/ArcZ ............. ok
 loader/Polygon ............. ok
 loader/PolygonM ............. ok
 loader/PolygonZ ............. ok
 loader/TSTPolygon ......... ok
 loader/TSIPolygon ......... ok
 loader/TSTIPolygon ......... ok
 loader/PointWithSchema ..... ok
 loader/NoTransPoint ......... ok
 loader/NotReallyMultiPoint ......... ok
 loader/MultiToSinglePoint ......... ok
 loader/ReprojectPts ........ ok
 loader/ReprojectPtsGeog ........ ok
 loader/Latin1 .... ok
 loader/Latin1-implicit .... ok
 loader/mfile .... ok
 dumper/literalsrid ....... ok
 dumper/realtable ....... ok
 affine .. ok
 bestsrid .. ok
 binary .. ok
 boundary .. ok
 cluster .. ok
 concave_hull .. ok
 ctors .. ok
 dump .. ok
 dumppoints .. ok
 empty .. ok
 forcecurve .. ok
 geography .. ok
 in_geohash .. ok
 in_gml .. ok
 in_kml .. ok
 in_encodedpolyline .. ok
 iscollection .. ok
 legacy .. ok
 long_xact .. ok
 lwgeom_regress .. ok
 measures .. ok
 operators .. ok
 out_geometry .. ok
 out_geography .. ok
 polygonize .. ok
 polyhedralsurface .. ok
 postgis_type_name .. ok
 regress .. ok
 regress_bdpoly .. ok
 regress_index .. ok
 regress_index_nulls .. ok
 regress_management .. ok
 regress_selectivity .. ok
 regress_lrs .. ok
 regress_ogc .. ok
```

```
 regress_ogc_cover .. ok
 regress_ogc_prep .. ok
 regress_proj .. ok
 relate .. ok
 remove_repeated_points .. ok
 removepoint .. ok
 setpoint .. ok
 simplify .. ok
 simplifyvw .. ok
 size .. ok
 snaptogrid .. ok
 split .. ok
 sql-mm-serialize .. ok
 sql-mm-circularstring .. ok
 sql-mm-compoundcurve .. ok
 sql-mm-curvepoly .. ok
 sql-mm-general .. ok
 sql-mm-multicurve .. ok
 sql-mm-multisurface .. ok
 swapordinates .. ok
 summary .. ok
 temporal .. ok
 tickets .. ok
 twkb .. ok
 typmod .. ok
 wkb .. ok
 wkt .. ok
 wmsservers .. ok
 knn .. ok
 hausdorff .. ok
 regress_buffer_params .. ok
 offsetcurve .. ok
 relatematch .. ok
 isvaliddetail .. ok
 sharedpaths .. ok
 snap .. ok
 node .. ok
 unaryunion .. ok
 clean .. ok
 relate_bnr .. ok
 delaunaytriangles .. ok
 clipbybox2d .. ok
 subdivide .. ok
 in_geojson .. ok
 regress_sfcgal .. ok
 sfcgal/empty .. ok
 sfcgal/geography .. ok
 sfcgal/legacy .. ok
 sfcgal/measures .. ok
 sfcgal/regress_ogc_prep .. ok
 sfcgal/regress_ogc .. ok
 sfcgal/regress .. ok
 sfcgal/tickets .. ok
 sfcgal/concave_hull .. ok
 sfcgal/wmsservers .. ok
 sfcgal/approximatemedialaxis .. ok
 uninstall    .         /projects/postgis/branches/2.2/regress/00-regress-
install/share/contrib/postgis/uninstall_sfcgal.sql
  /projects/postgis/branches/2.2/regress/00-regress-
install/share/contrib/postgis/uninstall_postgis.sql
. ok (4336)
```

```
Run tests: 118
Failed: 0

-- --with-gui 옵션으로 빌드했을 경우, 다음 내용도 볼 수 있어야 합니다.

    CUnit - A unit testing framework for C - Version 2.1-2
    http://cunit.sourceforge.net/


Suite: Shapefile Loader File shp2pgsql Test
  Test: test_ShpLoaderCreate() ...passed
  Test: test_ShpLoaderDestroy() ...passed
Suite: Shapefile Loader File pgsql2shp Test
  Test: test_ShpDumperCreate() ...passed
  Test: test_ShpDumperDestroy() ...passed

Run Summary:    Type  Total    Ran Passed Failed Inactive
              suites      2      2    n/a      0        0
               tests      4      4      4      0        0
             asserts      4      4      4      0      n/a
```

postgis_tiger_geocoder 와 address_standardizer 확장 프로그램은 현재 표준 PostgreSQL 설치검사(installcheck)만을 지원합니다. 이 확장 프로그램들을 테스트하려면 다음을 실행하십시오. 주의: 이미 PostGIS 코드 폴더의 루트에서 make install 을 실행했다면 다시 실행할 필요는 없습니다.

address_standardizer 의 경우:

```
cd extensions/address_standardizer
make install
make installcheck
```

다음과 같은 결과가 나와야 합니다:

```
============== dropping database "contrib_regression" ==============
DROP DATABASE
============== creating database "contrib_regression" ==============
CREATE DATABASE
ALTER DATABASE
============== running regression test queries       ==============
test test-init-extensions    ... ok
test test-parseaddress       ... ok
test test-standardize_address_1 ... ok
test test-standardize_address_2 ... ok

======================
 All 4 tests passed.
======================
```

TIGER 지오코딩 도구의 경우, 사용자의 PostgreSQL 인스턴스 안에서 PostGIS 및 fuzzystrmatch 확장 프로그램을 이용할 수 있는지 확인하십시오. address_standardizer 지원이 되도록 PostGIS 를 빌드했다면 address_standardizer 테스트도 함께 실행될 것입니다.

```
cd extensions/postgis_tiger_geocoder
make install
make installcheck
```

다음과 같은 결과가 나와야 합니다:

```
============== dropping database "contrib_regression" ==============
DROP DATABASE
============== creating database "contrib_regression" ==============
CREATE DATABASE
ALTER DATABASE
============== installing fuzzystrmatch            ==============
CREATE EXTENSION
============== installing postgis                  ==============
CREATE EXTENSION
============== installing postgis_tiger_geocoder     ==============
CREATE EXTENSION
============== installing address_standardizer      ==============
CREATE EXTENSION
============== running regression test queries      ==============
test test-normalize_address   ... ok
test test-pagc_normalize_address ... ok

======================
All 2 tests passed.
======================
```

2.4.5. 설치

PostGIS 를 설치하려면 다음을 입력하십시오.

make install 이렇게 하면 --prefix 설정 파라미터에 지정된 적절한 하위 디렉터리에 PostGIS 설치 파일을 복사할 것입니다.

- 로더(loader) 및 덤퍼(dumper) 바이너리는 [prefix]/bin 에 설치됩니다.
- postgis.sql 와 같은 SQL 파일들은 [prefix]/share/contrib 에 설치됩니다.
- PostGIS 라이브러리들은 [prefix]/lib 에 설치됩니다.

만약 이미 postgis_comments.sql, raster_comments.sql 파일을 생성하기 위해 make comments 명령어를 실행했다면, 다음을 실행해 sql 파일을 설치하십시오.

make comments-install



postgis_comments.sql, raster_comments.sql, topology_comments.sql 은 **xsltproc** 에 추가 의존성을 가지기 때문에 일반적인 빌드 및 설치 대상으로부터 분리되었습니다.

2.5. 확장 프로그램을 활용한 공간 데이터베이스 생성

PostgreSQL 9.1 이상 버전을 사용하며 확장 프로그램/PostGIS 모듈을 컴파일하고 설치했다면 다음과 같은 새로운 방법으로 공간 데이터베이스를 생성할 수 있습니다.

createdb [yourdatabase]

핵심 PostGIS 확장 프로그램은

```
CREATE EXTENSION postgis;
```

라는 단순한 명령어를 통해 PostGIS 의 도형(geometry), 지리(geography), 래스터, spatial_ref_sys 및 모든 함수와 주석을 설치합니다.

psql -d [yourdatabase] -c "CREATE EXTENSION postgis;"

지형(topology)은 별도의 확장 프로그램으로 패키징되어 있고, 다음 명령어로 설치할 수 있습니다:

psql -d [yourdatabase] -c "CREATE EXTENSION postgis_topology;"

만약 이 새 DB 에 이전 버전의 백업을 복원하고 싶다면 다음을 실행하십시오:

psql -d [yourdatabase] -f legacy.sql

복구 및 청소(cleanup)를 마친 뒤 앞으로 사용하지 않을 함수들을 제거하려면 이후 uninstall_legacy.sql 을 실행할 수 있습니다.

2.6. 확장 프로그램을 활용하지 않은 공간 데이터베이스 생성



일반적으로 사용자가 래스터 지원을 빼고 PostGIS 를 빌드할 경우에만 필요한 방법입니다. 래스터 함수들이 PostGIS 확장 프로그램의 일부이기 때문에, 래스터 지원을 빼고 PostGIS 를 빌드할 경우 확장 프로그램 지원은 활성화되지 않습니다.

PostGIS 데이터베이스를 생성하기 위한 첫 번째 단계는 단순한 PostgreSQL 데이터베이스를 생성하는 것입니다.

createdb [yourdatabase] 많은 PostGIS 함수들이 PL/pgSQL 절차형 언어로 작성됩니다. 그러므로 PostGIS 데이터베이스를 생성하기 위한 두 번째 단계는 새로운 데이터베이스에 PL/pgSQL 언어를 활성화하는 것입니다. 다음 명령어를 실행해서 활성화할 수 있습니다. PostgreSQL 8.4 이상 버전의 경우, 일반적으로 이미 설치되어 있습니다.

createlang plpgsql [yourdatabase] 이제 postgis.sql 정의 파일(설정 단계에서 지정한 대로 [prefix]/share/contrib 안에 있습니다)을 로드해서 PostGIS 오브젝트 및 함수 정의를 사용자 데이터베이스에 로드하십시오.

psql -d [yourdatabase] -f postgis.sql EPSG 좌표계 정의 식별자를 완전히 설치하려면, spatial_ref_sys.sql 정의 파일을 로드해서 spatial_ref_sys 테이블에 채울 수도 있습니다. 이렇게 하면 ST_Transform() 작업을 수행할 수 있습니다.

psql -d [yourdatabase] -f spatial_ref_sys.sql 주석을 PostGIS 함수에 추가하고 싶다면 마지막으로 공간 데이터베이스에 postgis_comments.sql 을 로드합니다. PSQL 터미널 창에서 \dd [function_name] 을 입력하는 것만으로도 주석을 조회할 수 있습니다.

psql -d [yourdatabase] -f postgis_comments.sql 래스터 지원 설치

psql -d [yourdatabase] -f rtpostgis.sql 래스터 지원 주석 설치. 이렇게 하면 PSQL, PgAdmin 또는 함수 주석을 표출할 수 있는 어떤 PostgreSQL 도구를 사용해도 각 래스터 함수에 대한 빠른 도움말 정보를 제공할 것입니다.

psql -d [yourdatabase] -f raster_comments.sql 지형 지원 설치

psql -d [yourdatabase] -f topology/topology.sql 지형 지원 주석 설치. 이렇게 하면 PSQL, PgAdmin 또는 함수 주석을 표출할 수 있는 어떤 PostgreSQL 도구를 사용해도 각 지형 함수/유형에 대한 빠른 도움말 정보를 제공할 것입니다.

psql -d [yourdatabase] -f topology/topology_comments.sql 만약 이 새 DB 에 이전 버전의 백업을 복원하고 싶다면 다음을 실행하십시오:

psql -d [yourdatabase] -f legacy.sql



MapServer 와 GeoServer 같은 응용 프로그램과 연동하는 테이블을 복구하는 데 필요한 베어본(barebone)을 설치하는 대신 실행할 수 있는 legacy_minimal.sql 이라는 대안이 있습니다. 거리, 길이 등과 같은 정보를 표출하는 뷰(view)가 있을 경우, 완전한 legacy.sql 이 필요할 것입니다.

복구 및 청소(cleanup)를 마친 뒤 앞으로 사용하지 않을 함수들을 제거하려면 이후 uninstall_legacy.sql 을 실행할 수 있습니다.

2.7. 주소 표준화 도구 설치 및 활용

address_standardizer 확장 프로그램은 별도로 다운로드해야 하는 별도의 패키지였습니다. PostGIS 2.2 버전부터는 내장되어 있습니다. 이 확장 프로그램이 무슨 일을 하고 사용자의 필요에 따라 어떻게 설정하는지에 대한 자세한 정보는 Chapter 12, 주소 표준화 도구 를 참조하십시오.

이 표준화 도구는 Normalize_Address 대신 PostGIS 용으로 패키징된 TIGER 지오코딩 도구(geocoder)와 함께 쓰일 수 있습니다. 이렇게 대신 사용하는 방법은 Section 2.8.3, “주소 표준화 도구를 TIGER 지오코딩 도구와 함께 사용” 을 참조하십시오. 주소 표준화 도구를 사용자의 다른 지오코딩 도구를 위한 구성 요소(building block)로 사용하거나, 주소를 더 쉽게 비교하기 위해 사용자 주소를 표준화하는 데 사용할 수도 있습니다.

주소 표준화 도구는 PCRE 에 의존성을 갖습니다. PCRE 는 많은 유닉스 파생 시스템에 일반적으로 이미 설치되어 있지만, http://www.pcre.org 에서 최신 버전을 다운로드할 수 있습니다. Section 2.4.1, “설정” 과정에서 PCRE 를 찾았다면, 주소 표준화 도구 확장 프로그램을 자동적으로 빌드할 것입니다. 사용자가 사용하고자 하는 PCRE 를 따로 설치한 경우, 설정 파라미터 --with-pcredir=/path/to/pcre 의 /path/to/pcre 부분에 사용자의 PCRE include 및 lib 디렉터리의 루트 폴더를 입력하십시오.

윈도우 사용자의 경우 PostGIS 2.1 이상 버전 번들은 이미 address_standardizer 와 함께 패키징되어 있으므로 컴파일할 필요없이 바로 CREATE EXTENSION 단계로 건너뛸 수 있습니다.

설치를 완료했다면 사용자 데이터베이스에 연결해서 SQL 을 실행할 수 있습니다:

```
CREATE EXTENSION address_standardizer;
```

다음 테스트에는 어떤 rules, gaz, 또는 lex 테이블도 필요없습니다.

```
SELECT num, street, city, state, zip
 FROM parse_address('1 Devonshire Place, Boston, MA 02109');
```

다음과 같은 결과가 나와야 합니다:

```
num |       street        |  city  | state |  zip
-----+----------------------+--------+-------+-------
 1   | Devonshire Place PH301 | Boston | MA    | 02109
```

2.7.1. Regex::Assemble 설치

address_standardizer 확장 프로그램을 컴파일하는 데 펄 Regex:Assemble 은 더 이상 필요없습니다. 펄 Regex:Assemble 이 생성하는 파일들이 소스 트리의 일부로 통합되었기 때문입니다. 하지만 usps-st-city-orig.txt 또는 usps-st-city-orig.txt usps-st-city-adds.tx 파일을 편집해야 할 경우, parseaddress-stcities.h 를 다시 빌드해야 하는데 이때 Regex:Assemble 이 필요합니다.

```
cpan Regexp::Assemble
```

또는 우분투/데비안 시스템의 경우 다음 작업을 해야 할 수도 있습니다.

```
sudo perl -MCPAN -e "install Regexp::Assemble"
```

2.8. TIGER 지오코딩 도구의 설치, 업그레이드 및 데이터 로딩

TIGER 지오코딩 도구 같은 추가 프로그램이 사용자의 PostGIS 배포판에 패키징되어 있지 않을 수도 있지만, postgis-2.3.0dev.tar.gz 파일에는 언제나 포함되어 있을 것입니다. 이 문서에서 제공하는 지침은 extras/tiger_geocoder/README 에서도 볼 수 있습니다.

윈도우 사용자이며 tar 를 설치하지 않았다면, PostGIS tar 파일의 압축을 풀기 위해 http://www.7-zip.org/를 이용할 수 있습니다.

2.8.1. TIGER 지오코딩 도구로 사용자의 PostGIS 데이터베이스 활성화: 확장 프로그램 이용

PostgreSQL 9.1 이상 버전과 PostGIS 2.1 이상 버전을 사용중이라면, TIGER 지오코딩 도구를 설치하는 데 새로운 확장 프로그램 모델을 활용할 수 있습니다. 그 방법은 다음과 같습니다.

1. 먼저 PostGIS 2.1 이상 버전의 바이너리를 다운로드하거나 컴파일해서 일반적인 방법으로 설치하십시오. TIGER 지오코딩 도구는 물론 필수 확장 프로그램도 함께 설치될 것입니다.

2. PSQL, pgAdmin 또는 다른 도구를 통해 사용자 데이터베이스에 연결해서 다음 SQL 명령어를 실행하십시오. 이미 PostGIS 가 설치된 데이터베이스에 설치하는 경우, 첫 번째 단계를 수행할 필요는 없다는 사실을 주의하십시오. 이미 fuzzystrmatch 확장 프로그램이 설치되어 있다면 이 두 번째 단계도 수행할 필요가 없습니다.

```
CREATE EXTENSION postgis;
CREATE EXTENSION fuzzystrmatch;
-- 규칙 기반 표준화 도구(pagc_normalize_address)를 이용하고자 할 경우 이 명령어는 선택적입니다.
CREATE EXTENSION address_standardizer;
CREATE EXTENSION postgis_tiger_geocoder;
```

이미 postgis_tiger_geocoder 확장 프로그램을 설치했고 최신 버전으로 업데이트만 하고자 할 경우:

```
ALTER EXTENSION postgis UPDATE;
```

```
ALTER EXTENSION postgis_tiger_geocoder UPDATE;
```

tiger.loader_platform 과 tiger.loader_variables 에 사용자 지정 항목을 만들었거나 변경한 경우 이 테이블들도 업데이트해야 할 수도 있습니다.

3. 제대로 설치되었는지 확인하려면 사용자 데이터베이스에 다음 SQL 을 실행하십시오:

```
SELECT na.address, na.streetname,na.streettypeabbrev, na.zip
 FROM normalize_address('1 Devonshire Place, Boston, MA 02109') AS na;
```

다음과 같은 결과가 나와야 합니다:

```
address | streetname | streettypeabbrev |  zip
---------+------------+------------------+-------
        1 | Devonshire | Pl               | 02109
```

4. tiger.loader_platform 테이블에 사용자의 실행 파일과 서버의 경로를 새 레코드로 생성하십시오.

   예를 들어 sh 규약(convention)을 따르는 debbie 라는 프로파일을 생성하려면 다음과 같이 해야 합니다.

```
INSERT INTO tiger.loader_platform(os, declare_sect, pgbin, wget,
unzip_command, psql, path_sep,
              loader, environ_set_command, county_process_command)
SELECT 'debbie', declare_sect, pgbin, wget, unzip_command, psql,
path_sep,
        loader, environ_set_command, county_process_command
  FROM tiger.loader_platform
  WHERE os = 'sh';
```

   그 다음 debbie 의 pg, unzip,shp2pgsql, PSQL 등의 경로 위치에 맞도록 declare_sect 열의 경로를 편집하십시오.

   이 loader_platform 테이블을 편집하지 않을 경우, 각 항목의 흔히 있는(common case) 위치만을 담게 되며 스크립트가 생생된 후 생성된 스크립트를 직접 편집해야 할 것입니다.

5. 서버의 루트, 또는 서버에 충분히 빠른 네트워크로 연결된 경우 사용자 PC 의 루트에 gisdata 라는 폴더를 생성하십시오. 이 폴더로 TIGER 파일을 다운로드해서 처리할 것입니다. 서버의 루트에 폴더를 만드는 게 마음에 안 들거나, 또는 단순히 다른 폴더로 변경하고 싶다면, tiger.loader_variables 테이블의 staging_fold 항목을 편집하십시오.

6. gisdata 폴더 또는 사용자가 staging_fold 에 지정한 폴더 안에 temp 라는 폴더를 생성하십시오. 다운로드한 TIGER 데이터를 로더가 이 temp 폴더에 압축해제할 것입니다.

7. 그 다음 Loader_Generate_Nation_Script 및 Loader_Generate_Script SQL 함수를 실행하십시오. 사용자가 지정한 프로파일 명칭을 사용하는지, 스크립트를 .sh 또는 .bat 파일로 복사했는지 확인하십시오. 예를 들어 nation 로드 작업 시 앞에서 만든 새 프로파일을 사용하려면 다음과 같이 해야 합니다:

```
SELECT Loader_Generate_Nation_Script('debbie');
```

```
SELECT Loader_Generate_Script(ARRAY['MA'], 'debbie');
```

8. 생성된 명령어 스크립트를 실행하십시오.

9. 모든 데이터를 로딩한 다음 또는 어떤 정지 지점에서 모든 TIGER 테이블을 분석해서 (상속된 통계를 포함한) 통계(stat)를 업데이트하는 것이 좋습니다.

```
SELECT install_missing_indexes();
vacuum analyze verbose tiger.addr;
vacuum analyze verbose tiger.edges;
```

```
vacuum analyze verbose tiger.faces;
vacuum analyze verbose tiger.featnames;
vacuum analyze verbose tiger.place;
vacuum analyze verbose tiger.cousub;
vacuum analyze verbose tiger.county;
vacuum analyze verbose tiger.state;
```

2.8.1.1. TIGER 지오코딩 도구 정규 설치를 확장 프로그램 모델로 변환

확장 프로그램 모델을 사용하지 않고 TIGER 지오코딩 도구를 설치했다면, 다음과 같이 확장 프로그램 모델로 변환시킬 수 있습니다:

1. 확장 프로그램 제외(non-extension) 모델을 업그레이드하려면 Section 2.8.5, "TIGER 지오코딩 도구 설치 업그레이드"의 지침을 따라해보십시오.
2. PSQL 또는 pgAdmin 을 통해 사용자 데이터베이스에 연결하고 다음 명령어를 실행하십시오:

```
CREATE EXTENSION postgis_tiger_geocoder FROM unpackaged;
```

2.8.2. TIGER 지오코딩 도구로 사용자의 PostGIS 데이터베이스 활성화: 확장 프로그램 없이

먼저 앞에서 설명한 바에 따라 PostGIS 를 설치하십시오.

extras 폴더가 없을 경우, http://postgis.net/stuff/postgis-2.3.0dev.tar.gz 에서 다운로드하십시오.

tar xvfz postgis-2.3.0dev.tar.gz

cd postgis-2.3.0dev/extras/tiger_geocoder

tiger_loader_2015.sql (또는 다른 년도를 로드하고자 할 경우, 사용자가 찾을 수 있는 최신 로더)를 사용자의 실행 가능한 서버 등의 경로로 편집하거나, 그 대신 설치 완료 후 loader_platform 테이블을 업데이트할 수도 있습니다. 이 파일 또는 loader_platform 테이블을 편집지 않는다면, 각 항목의 흔히 쓰이는(common case) 위치만 담게 되어 Loader_Generate_Nation_Script 및 Loader_Generate_Script SQL 함수를 실행해서 생성된 스크립트를 직접 편집해야 할 것입니다.

TIGER 지오코딩 도구를 처음 설치하는 경우, 윈도우 시스템이라면 create_geocode.bat 스크립트를, Linux/Unix/Mac OSX 시스템이라면 create_geocode.sh 를 사용자의 PostgreSQL 에 특화된 설정으로 편집한 다음 명령 프롬프트에서 각각 상응하는 스크립트를 실행하십시오.

tiger 스키마가 데이터베이스에 있는지와 사용자 데이터베이스의 search_path 에 포함되는지 검증하십시오. 아닐 경우, 다음과 같은 명령어와 함께 추가하십시오.

```
ALTER DATABASE geocoder SET search_path=public, tiger;
```

정규화된 주소 기능은 까다로운 주소를 제외하고는 동작합니다. 이 테스트를 실행한 결과가 다음처럼 보이는지 검증하십시오.

```
SELECT pprint_addy(normalize_address('202 East Fremont Street, Las Vegas,
Nevada 89101')) As pretty_address;
pretty_address
---------------------------------------
```

```
202 E Fremont St, Las Vegas, NV 89101
```

2.8.3. 주소 표준화 도구를 TIGER 지오코딩 도구와 함께 사용

사용자들의 많은 불평 가운데 하나는 주소 정규화 도구 Normalize_Address 함수가 지오코딩 작업 전 준비 과정에서 주소를 정규화한다는 것입니다. 정규화 도구는 완벽하지 않아 그 불완전함을 수정하려면 막대한 노력이 필요합니다. 그래서 우리는 훨씬 나은 주소 표준화 도구 엔진을 가진 또다른 프로젝트와 통합시켰습니다. 이 새로운 address_standardizer 를 이용하려면, Section 2.7, "주소 표준화 도구 설치 및 활용"에 설명된 대로 확장 프로그램을 컴파일해서 사용자 데이터베이스에 확장 프로그램으로 설치하십시오.

postgis_tiger_geocoder 를 설치했던 데이터베이스에 이 확장 프로그램을 설치했다면, Normalize_Address 대신 Pagc_Normalize_Address 를 이용할 수 있습니다. 이 확장 프로그램은 TIGER 유무와 상관없이 동작하므로, 국제 주소와 같은 다른 데이터 소스와 함께 사용할 수 있습니다. 실제로 TIGER 지오코딩 도구 확장 프로그램은 규칙 테이블 (tiger.pagc_rules) , 지명 색인 테이블 (tiger.pagc_gaz), 그리고 어휘 목록 테이블 (tiger.pagc_lex) 자체 수정 버전과 함께 패키징되어 있습니다. 사용자 자신의 필요에 따라 이들을 추가하고 업데이트해서 표준화 작업 과정을 향상시킬 수 있습니다.

2.8.4. TIGER 데이터 로드

TIGER 데이터를 로드하는 데 대한 자세한 설명을 extras/tiger_geocoder/tiger_2011/README 에서 볼 수 있습니다. 이 문서에서는 개괄적인 단계만 설명합니다.

데이터 로드 과정은 다음과 같습니다. 해당 국가의 인구조사 웹사이트에서 필요한 지역의 데이터를 다운로드합니다. 파일의 압축을 해제한 다음 각 행정구역의 데이터를 대응하는 각각의 행정구역 테이블로 로드합니다. 각 행정구역 테이블은 tiger 스키마에 정의된 테이블을 상속하기 때문에, 어떤 행정구역을 다시 로드해야 하거나 더 이상 어떤 행정구역이 필요없을 경우 언제라도 Drop_State_Tables_Generate_Script 를 통해 테이블을 쿼리해서 모든 데이터에 접근하거나 행정구역 테이블을 드롭시킬 수 있습니다.

데이터를 로드하려면 다음과 같은 도구들이 필요합니다:

- 인구조사 웹사이트에서 받은 .zip 파일을 풀기 위한 도구.

Unix 계열 시스템의 경우: 대부분의 Unix 계열 시스템에는 일반적으로 이미 unzip 실행 파일이 설치되어 있습니다.

윈도우의 경우: 무료 압축/압축해제 툴인 7-zip 을 http://www.7-zip.org/ 에서 다운로드할 수 있습니다.

- shp2pgsql 명령어는 PostGIS 설치시 기본적으로 설치됩니다.
- 웹 다운로드 도구인 wget 은 대부분의 Unix/Linux 시스템에 일반적으로 설치되어 있습니다.

윈도우를 사용중이라면 http://gnuwin32.sourceforge.net/packages/wget.htm 에서 미리 컴파일된 바이너리를 다운로드할 수 있습니다.

tiger_2010 을 업그레이드하는 경우, 먼저 Drop_Nation_Tables_Generate_Script 를 생성한 다음 실행해야 합니다. 행정구역 데이터를 로드하기 전에 로더 스크립트를 생성하는 Loader_Generate_Nation_Script 를 통해 전국 데이터를 로드해야 합니다. Loader_Generate_Nation_Script 를 사용하면 (2010 년에서) 업그레이드는 물론 새로운 설치까지 한번에 끝날 것입니다.

사용자의 플랫폼에서 사용자가 지정한 행정구역 데이터를 로드하는 스크립트를 생성하려면 Loader_Generate_Script 를 참조하십시오. 이 데이터들을 개별적으로 로드할 수 있다는 점을 기억하십시오. 사용자가 지정한 모든 행정구역을 한번에 로드하지 않아도 됩니다. 필요할 때마다 로드할 수 있습니다.

사용자가 지정한 행정구역 데이터를 로드한 다음, Install_Missing_Indexes 에 설명된 대로

```
SELECT install_missing_indexes();
```

를 꼭 실행하십시오.

작업이 제대로 됐는지 확인하려면 Geocode 를 이용해서 사용자의 행정구역에 있는 주소에 지오코딩을 실행해보십시오.

2.8.5. TIGER 지오코딩 도구 설치 업그레이드

PostGIS 2.0 이상 버전과 함께 패키징된 TIGER 지오코딩 도구를 이미 설치했다면, 그리고 사용자가 간절히 바라는 수정 사항이 있다면, 언제라도, 심지어 tar 파일 내부에서도 함수를 업그레이드할 수 있습니다. 이 작업은 확장 프로그램과 함께 설치되지 않은 TIGER 지오코딩 도구라야만 가능합니다.

extras 폴더가 없을 경우, http://postgis.net/stuff/postgis-2.3.0dev.tar.gz 에서 다운로드하십시오.

tar xvfz postgis-2.3.0dev.tar.gz

cd postgis-2.3.0dev/extras/tiger_geocoder/tiger_2011

윈도우 시스템이라면 upgrade_geocoder.bat 스크립트를, Linux/Unix/Mac OSX 시스템이라면 upgrade_geocoder.sh 스크립트를 찾으십시오. 사용자의 PostGIS 데이터베이스 사양에 맞춰 파일을 편집하십시오.

2010 이나 2011 을 업그레이드하는 경우, 로더 스크립트의 해당 라인을 주석 처리(unremark out)해야 2012 데이터를 로드하기 위한 최신 스크립트를 얻을 수 있다는 점을 기억하십시오.

명령 프롬프트에서 각 플랫폼에 상응하는 스크립트를 실행하십시오.

다음으로 모든 nation 테이블을 드롭하고 새 테이블을 불러옵니다. Drop_Nation_Tables_Generate_Script 에 자세히 설명되어 있는 다음 SQL 선언문으로 드롭 스크립트를 생성하십시오.

```
SELECT drop_nation_tables_generate_script();
```

생성된 드롭 SQL 선언문을 실행하십시오.

Loader_Generate_Nation_Script 에 자세히 설명되어 있는 이 SELECT 선언문으로 nation 로드 스크립트를 생성하십시오.

윈도우용

```
SELECT loader_generate_nation_script('windows');
```

Unix/Linux 용

```
SELECT loader_generate_nation_script('sh');
```

생성된 스크립트를 실행하는 방법에 대해서는 Section 2.8.4, “TIGER 데이터 로드” 를 참조하십시오. 이 작업은 딱 한번만 하면 됩니다.



2010/2011 행정구역 테이블을 합칠 수도 있고 각 행정구역을 개별적으로 각각 업그레이드할 수도 있습니다. 어떤 행정구역을 2011 년으로 업그레이드하기 전에, 먼저 Drop_State_Tables_Generate_Script 를 이용해서 해당 행정구역의 2010 년 테이블을 드롭해야 합니다.

2.9. 템플릿을 이용해서 공간정보 처리 가능한 데이터베이스 생성

일부 PostGIS 배포 패키지(특히 PostGIS 1.1.5 이상 버전의 Win32 인스톨러)는 template_postgis 라는 템플릿 데이터베이스에 PostGIS 함수들을 로드합니다. 사용자의 PostgreSQL 설치에 template_postgis 데이터베이스가 존재한다면 사용자 그리고/또는 응용 프로그램이 단일 명령어를 통해 공간정보를 처리 가능한(spatially-enabled) 데이터베이스를 생성할 수 있습니다. 두 경우 모두 데이터베이스 사용자가 새로운 데이터베이스를 생성할 수 있는 권한을 가지고 있어야 한다는 점을 주의하십시오.

셸(shell)에서:

```
# createdb -T template_postgis my_spatial_db
```

SQL 에서:

```
postgres=# CREATE DATABASE my_spatial_db TEMPLATE=template_postgis
```

2.10. 업그레이드

기존 공간 데이터베이스를 업그레이드하는 작업은 새 PostGIS 객체 정의 또는 이를 대신할 것이 필요하기 때문에 까다로울 수 있습니다.

안타깝게도 실제 실행중인 데이터베이스 내에서는 모든 정의들을 쉽게 대체할 수 있는 것이 아닙니다. 따라서 덤프/리로드(dump/reload) 방식이 최고의 선택일 수 있습니다.

PostGIS 는 버그 수정 또는 사소한 배포판을 위한 소프트 업그레이드와 주요 배포판을 위한 하드 업그레이드를 제공합니다.

PostGIS 업그레이드에 앞서 언제나 데이터를 미리 백업해두는 것이 중요합니다. pg_dump 실행시 –Fc 플래그(flag)를 이용하면 하드 업그레이드 작업시 언제나 덤프를 복원할 수 있을 것입니다.

2.10.1. 소프트 업그레이드

확장 프로그램을 이용해서 사용자 데이터베이스를 설치한 경우, 업그레이드 또한 확장 프로그램을 이용해야 합니다. 구식 SQL 스크립트 방식으로 설치한 경우, 마찬가지로 SQL 스크립트 방식으로 업그레이드해야 합니다. 사용자 상황에 맞는 방법을 이용하십시오.

2.10.1.1. 9.1 미만 버전 또는 확장 프로그램 제외 소프트 업그레이드

이 단원은 확장 프로그램을 이용하지 않고 PostGIS 를 설치한 경우에만 해당합니다. 만약 확장 프로그램을 설치한 상태에서 이 단원에서 설명하는대로 업그레이드한다면 다음과 같은 메시지를 받게 될 것입니다:

```
can't drop ... because postgis extension depends on it
```

컴파일 및 설치(make install)가 끝나면 설치 폴더에 postgis_upgrade.sql 과 rtpostgis_upgrade.sql 이 있을 것입니다. 예를 들면 /usr/share/postgresql/9.3/contrib/postgis_upgrade.sql 과 같은 경로에 말입니다. postgis_upgrade.sql 을 설치하십시오. 래스터 기능을 설치했다면, /usr/share/postgresql/9.3/contrib/postgis_upgrade.sql 도 설치해야 합니다. PostGIS 1.* 버전에서 2.* 버전으로 또는 PostGIS 2.* r7409 이전 버전에서 업그레이드하는 경우 하드 업그레이드 작업을 해야 합니다.

```
psql -f postgis_upgrade.sql -d your_spatial_database
```

래스터 및 지형 확장 프로그램에도 동일한 절차가 적용됩니다. 이때 업그레이드 파일은 각각 rtpostgis_upgrade*.sql 과 topology_upgrade*.sql 입니다. 이 작업이 필요할 경우:

```
psql -f rtpostgis_upgrade.sql -d your_spatial_database
```

```
psql -f topology_upgrade.sql -d your_spatial_database
```



사용자의 버전을 업그레이드하기 위한 특정 postgis_upgrade*.sql 스크립트를 찾을 수 없다면 소프트 업그레이드를 하기에는 사용자의 버전이 너무 이전 것이라는 의미입니다. 이럴 경우 하드 업그레이드 작업을 해야 합니다.

PostGIS_Full_Version 함수는 "procs need upgrade" 메시지를 통해 이런 종류의 업그레이드가 필요한지에 대해 알려줍니다.

2.10.1.2. 확장 프로그램을 통한 9.1 버전 이상의 소프트 업그레이드

확장 프로그램과 함께 PostGIS 를 설치한 경우, 마찬가지로 확장 프로그램을 이용해서 업그레이드해야 합니다. 확장 프로그램을 통한 사소한 업그레이드는 쉬운 편입니다.

```
ALTER EXTENSION postgis UPDATE TO "2.3.0dev";
ALTER EXTENSION postgis_topology UPDATE TO "2.3.0dev ";
```

다음과 같은 에러 메시지가 표출되는 경우:

```
No migration path defined for ... to 2.3.0dev
```

사용자 데이터베이스를 백업하고, Section 2.5, “확장 프로그램을 활용한 공간 데이터베이스 생성” 에서 설명하는 대로 새로운 데이터베이스를 생성한 다음 이 새 데이터베이스에 사용자의 백업을 복원해야 합니다.

다음과 같은 에러 메시지가 표출되는 경우:

```
Version "2.3.0dev" of extension "postgis" is already installed
```

이미 모든 것이 최신 상태라는 뜻이므로 무시해도 괜찮습니다. 한 SVN 버전에서 다음 버전으로 (실제 버전은 올라가지 않습니다) 업그레이드하려는 게 아니라면 말입니다. 이런 경우엔 버전 문자열에 "next"를 덧붙일 수 있습니다. 다음 업그레이드 작업시 이 "next" 접미사(suffix)를 다시 삭제해야 할 것입니다.

```
ALTER EXTENSION postgis UPDATE TO "2.3.0devnext";
ALTER EXTENSION postgis_topology UPDATE TO "2.3.0devnext";
```



원래 버전을 지정하지 않고 PostGIS 를 설치한 경우, 복원 작업 전에 PostGIS 확장 프로그램을 재설치하는 과정을 건너뛸 수도 있습니다. 백업 파일에 CREATE EXTENSION postgis 가 포함되어 있으므로 복원 작업 도중 최신 버전을 받아오기 때문입니다.

2.10.2. 하드 업그레이드

하드 업그레이드란 PostGIS 를 사용 가능한(postgis-enabled) 데이터베이스의 완전한 덤프/리로드(dump/reload)를 뜻합니다. PostGIS 객체의 내부 저장소가 변경되거나 또는 소프트 업그레이드를 할 수 없을 경우 하드 업그레이드가 필요합니다. Release Notes 부록에서 업그레이드 시 덤프/리로드(하드 업그레이드)가 필요한 버전에 대해 각각 설명하고 있습니다.

덤프/리로드 과정은 PostGIS(구 버전들 포함)에 속해 있는 모든 정의를 덤프하는 작업을 건너뛸 수 있도록 하는 postgis_restore.pl 스크립트의 지원을 받습니다. 이 스크립트를 사용하면 심볼이 중복되거나 더 이상 필요없는 객체를 그대로 가져오는 오류 없이 PostGIS 가 설치된 데이터베이스에 사용자의 스키마 및 데이터를 복원할 수 있습니다.

윈도우 사용자를 위한 추가적인 지침을 Windows Hard upgrade 에서 확인할 수 있습니다.

하드 업그레이드 절차는 다음과 같습니다:

1. 업그레이드하고자 하는 데이터베이스의 "사용자 지정 유형(custom-format)" 덤프(olddb 라고 합시다)를 BLOB(-b)과 상세 진행 정보(-v) 출력을 포함해서 생성합니다. 사용자가 꼭 PostgreSQL 슈퍼 계정일 필요는 없고, DB 의 소유자일 수는 있습니다.

   ```
   pg_dump -h localhost -p 5432 -U postgres -Fc -b -v -f
      "/somepath/olddb.backup" olddb
   ```

2. 새 데이터베이스에 PostGIS 를 새로 설치하십시오. 이 데이터베이스를 newdb 라고 하겠습니다. 이 작업을 어떻게 하는지에 대해서는 Section 2.6, “Create a spatially-enabled database without using extensions” 및 Section 2.5, “확장 프로그램을 활용한 공간 데이터베이스 생성” 을 참조하십시오.

   덤프에 담겨 있는 spatial_ref_sys 항목들은 복구되지만, spatial_ref_sys 에 있는 기존 항목을 덮어쓰지는 않을 것입니다. 공식 패키지에 있는 수정 사항이 복원된 데이터베이스에서도 제대로 유지되도록 하기 위해서 입니다. 어떤 이유에서건 사용자의 덤프 항목을 표준 항목에 덮어씌우고자 할 경우, 새 DB 생성시 spatial_ref_sys.sql 파일을 로드하지 않기만 하면 됩니다.

사용자 데이터베이스가 아주 구버전이거나 사용자 뷰 또는 기능에서 오래 전에 필요없어진 함수를 이용한다는 사실을 알고 있다면, 사용자의 모든 기능 및 뷰 등을 제대로 돌려놓기 위해 legacy.sql 을 로드해야 할 수도 있습니다. 정말로 필요할 경우에만 이 작업을 수행하십시오. 가능하다면 이 작업 대신 덤핑하기 전에 사용자 뷰 및 기능을 업그레이드하는 것도 고려해보십시오. 더 이상 필요없어진 기능들은 나중에 uninstall_legacy.sql 을 로드해서 제거할 수 있습니다.

3. 사용자의 새로운 newdb 데이터베이스에 사용자 백업을 postgis_restore.pl 스크립트를 이용해서 복원하십시오. 예상 밖의 오류가 하나라도 발생한다면 PSQL 이 표준 오류 스트림으로 표출할 것입니다. 이 오류 로그를 삭제하지 마십시오.

```
perl utils/postgis_restore.pl "/somepath/olddb.backup" | psql -h
   localhost -p 5432 -U postgres newdb 2
> errors.txt
```

다음과 같은 경우에 오류가 발생할 수 있습니다:

1. 사용자 뷰 또는 기능의 일부가 더 이상 필요없는 PosgtGIS 객체를 사용하는 경우입니다. 이 문제를 해결하려면 복원에 앞서 legacy.sql 를 로드하거나, 아직 해당 객체를 포함하고 있는 PostGIS 버전을 복구하고 사용자 코드를 포팅한 다음 이송(migration)을 시도해야 할 것입니다. legacy.sql 방식이 잘 작동했다면, 더 이상 필요없는 않은 함수들의 사용을 중단하도록 사용자 코드를 수정하고 uninstall_legacy.sql 을 로드해서 이들을 삭제하는 것을 잊지 마십시오.

2. 덤프 파일 내부에 있는 spatial_ref_sys 의 사용자 지정 레코드 가운데 일부는 유효하지 않은 SRID 값을 가지고 있습니다. 유효한 SRID 값은 0 보다 크고 999,000 보다 작습니다. 999000 에서 999999 까지의 값은 내부용으로 할당되어 있는 반면 999,999 보다 큰 값은 전혀 쓸 수도 없습니다. 유효하지 않은 SRID 값을 가진 모든 사용자 지정 레코드는 유지될 것입니다. 999,999 보다 큰 값은 할당된 범위로 이동되지만, spatial_ref_sys 테이블이 담을 수 있는 불변값을 보호하는 CHECK 제약조건을 잃게 될 수도 있고, 또 (복수의 유효하지 않은 SRID 가 할당된 SRID 값 가운데 동일한 값으로 변환될 때) 기본 키(primary key)도 잃게 될 수 있습니다.

   이 문제를 해결하기 위해선 사용자 지정 SRS 를 유효한 값의 SRID 로 (아마도 910000~910999 범위로) 복사하고, 모든 테이블을 새 SRID 로 전환한 다음(UpdateGeometrySRID 참조), 다음과 같이 spatial_ref_sys 에서 유효하지 않은 항목을 삭제하고 CHECK 제약조건(들)을 재구성해야 합니다:

```
ALTER TABLE spatial_ref_sys ADD CONSTRAINT spatial_ref_sys_srid_check
check (srid > 0 AND srid < 999000 );
ALTER TABLE spatial_ref_sys ADD PRIMARY KEY(srid));
```

2.11. 설치 과정에서 흔히 발생하는 문제들

설치 또는 업그레이드가 잘 되지 않을 경우 확인해야 할 몇 가지 사항이 있습니다.

1. PostgreSQL 9.2 버전 또는 그 이상 버전을 설치했는지, 현재 실행되고 있는 PostgreSQL 버전과 동일한 버전의 PostgreSQL 소스를 바탕으로 컴파일했는지 확인하십시오. 사용자의 (Linux) 배포판이 이미 PostgreSQL 을 설치했거나, 또는 이전에 설치한 사실을 잊어버린 경우 중복 설치가 될 수 있습니다. PostGIS 는 PostgreSQL 9.2 이상 버전에서만 작동하므로 구 버전 이용시 이상하고 예상치 못 한 오류들이 발생할 수 있습니다. 현재 실행중인 PostgreSQL 버전을 확인하려면 PSQL 을 통해 데이터베이스에 연결해서 다음 쿼리를 실행하십시오:

```
SELECT version();
```

RPM 기반 배포판을 실행중이라면 다음과 같은 rpm 명령어로 이전에 설치된 패키지의 존재를 확인할 수 있습니다: **rpm -qa | grep postgresql**

2. 업그레이드가 실패한 경우, 기존 PostGIS 가 설치된 데이터베이스로 복원해야 합니다.

```
SELECT postgis_full_version();
```

또 설정 과정에서 PostreSQL, Proj4 라이브러리 및 GEOS 라이브러리의 버전과 경로를 제대로 감지했는지 확인하십시오.

1. 설정 과정에서 나온 결과물은 postgis_config.h 파일을 생성하는 데 사용됩니다. POSTGIS_PGSQL_VERSION, POSTGIS_PROJ_VERSION 및 POSTGIS_GEOS_VERSION 변수가 정확히 설정됐는지 확인하십시오.

2.12. 로더/덤퍼(Loader/Dumper)

데이터 로더와 덤퍼는 PostGIS 빌드 작업의 한 부분으로, 자동적으로 빌드 및 설치됩니다. 로더와 덤퍼를 수동으로 설치하려면:

```
# cd postgis-2.3.0dev/loader
# make
# make install
```

로더는 shp2pgsql 이라고 불리며 PostGIS/PostgreSQL 에 로드하기 위해 ESRI shapefile 을 SQL 에 적합하도록 변환합니다. 덤퍼는 pgsql2shp 이라 불리며 PostGIS 테이블(또는 쿼리)을 ESRI shapefile 로 전환합니다. 더 상세한 설명을 원한다면 온라인 도움말 및 사용자 지침서를 참조하십시오.

Chapter 3.PostGIS FAQ

1. **PostGIS 활용에 대한 예제, 지침서, 워크숍 등을 어디에서 찾아볼 수 있을까요?**

OpenGeo 가 단계별 예제 지침 워크숍 PostGIS 개론 을 제공하고 있습니다. 이 워크숍은 패키징된 데이터는 물론 OpenGeo 스위트(suite) 활용법에 대한 소개를 포함하고 있습니다. 아마도 PostGIS 에 대한 최고의 예제일 겁니다.

BostonGIS 에서도 PostGIS: 멍청이도 할 수 있는 지침서를 제공합니다. 이 지침서는 윈도우 사용자를 더 중시하고 있습니다.

2. **PostGIS 1.5 에서 돌아가던 응용 프로그램과 데스크톱 도구들이 PostGIS 2.0 에서는 동작하지 않습니다. 어떻게 고칠 수 있을까요?**

PostGIS 2.0 버전에서 더 이상 지원되지 않는 상당수의 함수들이 PostGIS 코드 기반에서 제거되었습니다. GeoServer, MapServer, QuantumGIS, 그리고 OpenJump 등과 같은 몇몇 제 3 자(third-party) 도구들뿐만 아니라, 응용 프로그램들도 이 영향을 받고 있습니다. 두어 가지 방법으로 이 문제를 해결할 수 있습니다. 제 3 자 응용 프로그램의 경우, 이 문제들의 상당수가 수정된 최신 버전으로 업그레이드해볼 수 있습니다. 사용자가 직접 개발한 코드인 경우, 제거된 함수들을 더 이상 사용하지 않도록 코드를 변경할 수 있습니다. 이 함수들의 대부분은 ST_Union, ST_Length 등 ST_* 명칭의 함수가 아닙니다. 마지막 수단으로, legacy.sql 을 전체 설치하거나, 사용자가 필요한 만큼의 legacy.sql 을 일부 설치하십시오.

legacy.sql 파일은 postgis.sql 과 같은 폴더에 있습니다. 제거했던 200 여 개의 구 버전 함수를 다시 얻으려면 postgis.sql 과 spatial_ref_sys.sql 을 설치한 뒤 legacy.sql 을 설치하면 됩니다.

3. **osm2pgsql 을 통해 OpenStreetMap 데이터를 로드할 때 다음과 같은 오류가 발생합니다. ERROR: operator class "gist_geometry_ops" does not exist for access method "gist". PostGIS 1.5 에서는 잘 됐는데 말이죠.**

PostGIS 2 버전부터 기본 도형 연산자 클래스가 gist_geometry_ops 에서 gist_geometry_ops_2d 로 변경되었고, gist_geometry_ops 는 완전히 삭제되었습니다. PostGIS 2 버전부터 3D 를 지원하기 위해 N-D 공간 인덱스를 도입했는데, 구 명칭 gist_geometry_ops 가 부정확하고 혼동된다고 여겨졌기 때문입니다.

테이블 및 인덱스를 생성하는 과정의 일부인 몇몇 구 버전 응용 프로그램은 연산자 클래스 명을 정확히 참조합니다. 기본 2D 인덱스를 사용하고자 하는 경우 이렇게 정확히 참조할 필요는 없습니다. 따라서 이런 경우라면 인덱스 생성 명령어를 다음과 같이 변경하십시오:

나쁜 예에서:

```
CREATE INDEX idx_my_table_geom ON my_table USING gist(geom
gist_geometry_ops);
```

좋은 예로:

```
CREATE INDEX idx_my_table_geom ON my_table USING gist(geom);
```

사용자가 연산자 클래스를 지정해야만 하는 유일한 경우는 다음과 같이 3D 공간 인덱스를 생성하고자 할 때뿐입니다:

```
CREATE INDEX idx_my_super3d_geom ON my_super3d USING gist(geom
gist_geometry_ops_nd);
```

안타깝게도 사용자가 변경할 수 없는, 구 버전 gist_geometry_ops 가 하드 코딩되어 있는 컴파일된 코드를 써야만 할 경우, PostGIS 2.0.2 이상 버전에 패키징된 legacy_gist.sql 을 이용해서 구 버전 클래스를 생성할 수 있습니다. 하지만 이 해결 방법을 쓸 경우, 이후 어떤 시점에서 해당 인덱스를 삭제하고 연산자 클래스 없이 재생성하는 것이 좋습니다. 이렇게 하면 향후 다시 업그레이드를 해야 할 때 수고를 덜게 될 것입니다.

4. PostgreSQL 9.0 을 실행중인데 OpenJump, Safe FME 및 몇몇 다른 도구들에서 도형을 더 이상 읽거나 볼 수 없습니다.

PostgreSQL 9.0 이상 버전부터 bytea 데이터 용 기본 인코딩이 헥스(hex)로 바뀌었는데, 예전 JDBC 드라이버는 여전히 이스케이프 형식(escape format)을 취합니다. 예전 JDBC 드라이버를 사용하는 Java 응용 프로그램이나, 구 버전 ST_AsBinary 의 방식을 요구하는 예전 npgsql 드라이버를 사용하는 .NET 응용 프로그램과 같은 몇몇 응용 프로그램이 이 영향을 받습니다. 두 가지 접근 방법으로 이 오류를 바로잡을 수 있습니다.

JDBC 드라이버를 최신 PostgreSQL 9.0 버전으로 업그레이드할 수 있습니다. http://jdbc.postgresql.org/download.html 에서 최신 PosgtreSQL 버전을 다운로드할 수 있습니다.

.NET 응용 프로그램을 실행중이라면 http://pgfoundry.org/frs/?group_id=1000140 에서 다운로드할 수 있는 Npgsql 2.0.11 또는 이후 버전을 사용할 수 있습니다. Npgsql 2.0.11 에 대한 정보는 Francisco Figueiredo's NpgSQL 2.0.11 released blog entry 를 참조하십시오.

PostgreSQL 드라이버를 업그레이드할 수 없는 경우, 다음과 같이 변경해서 기본값을 예전 방식으로 설정할 수 있습니다:

```
ALTER DATABASE mypostgisdb SET bytea_output='escape';
```

5. PgAdmin 을 사용해서 도형 열을 보려고 했는데 비어있습니다. 이유가 뭐죠?

대용량 도형의 경우 PgAdmin 은 아무것도 보여주지 않습니다. 사용자의 도형 열에 데이터가 들어 있다는 사실을 검증하는 데 가장 좋은 방법은 다음과 같습니다.

```
-- 이 스크립트는 사용자의 모든 geom 항목이 차 있다면 어떤 레코드도 반환하지 않습니다.
SELECT somefield FROM mytable WHERE geom IS NULL;
```

```
-- 사용자 도형이 얼마나 큰지 알고 싶다면 다음과 같이 쿼리하십시오
```

```
-- 사용자의 geom 열에서 가장 많은 포인트 개수를 반환할 것입니다
SELECT MAX(ST_NPoints(geom)) FROM sometable;
```

6. 어떤 종류의 도형 객체를 저장할 수 있습니까?

포인트, 라인스트링, 폴리곤, 멀티포인트, 멀티라인스트링, 멀티폴리곤, 그리고 지오메트리컬렉션(GeometryCollection) 도형을 저장할 수 있습니다. PostGIS 2.0 이상 버전이라면 TINS 및 다면체 표면(Polyhedral Surface)도 기본 도형 유형으로 저장할 수 있습니다. Z, M, ZM 확장자를 가지는 오픈 GIS WKT 형식(Open GIS Well Known Text Format)이 이런 도형들을 지정하고 있습니다. 현재 다음 세 가지 데이터 유형을 지원합니다. 그 세 가지 유형은 측정시 평면좌표계를 이용하는 표준 OGC 도형 데이터 유형, 구체 또는 편구체 상에서 계산하는 측지좌표계를 이용하는 지리 데이터 유형, 그리고 PostGIS 공간 유형 계보에 새롭게 추가된 래스터 데이터 분석 및 저장을 위한 래스터입니다. 래스터 전용 FAQ 도 있습니다. 더 자세한 설명은 Chapter 10, PostGIS 래스터 FAQ 과 Chapter 9, 래스터 참조 문서를 참조하십시오.

7. 혼란스럽군요. 도형 저장소와 지리형 저장소 가운데 어떤 데이터 저장소를 이용해야 합니까?

짧은 답변: 지리형(geography)은 장거리(long range distance) 범위 측정을 지원하는 새로운 데이터 유형이지만, 이 유형을 대상으로 하는 계산 대부분은 도형의 경우보다 느립니다. 지리형을 이용할 경우, 평면좌표계를 자세히 알 필요는 없습니다. 사용자가 전세계에 걸친 데이터를 가지고 있고 거리 및 길이를 측정하는 데에만 관심이 있을 경우 일반적으로 지리형이 최선입니다. 도형 데이터 유형은 훨씬 많은 함수가 지원하고, 제 3 자 도구의 광범위한 지원을 받으며, 도형을 대상으로 하는 연산이 더 빠릅니다 -- 대용량 도형의 경우 때로는 10 배 빠르기도 합니다. 사용자가 공간 참조 시스템(Spatial Reference System)에 꽤 익숙하거나, 사용자 데이터 전부가 단일 공간 참조 시스템(SRID) 의 적용을 받는 국지적인 데이터를 처리하는 경우, 또는 상당한 양의 공간 처리 작업을 해야할 경우 도형이 최선입니다. 주의: 각 유형의 장점을 취하기 위해 1 단계 작업(one-off)만으로 꽤 쉽게 두 유형을 변환시킬 수 있습니다. 현재 어떤 지원을 받는지 그리고 받지 못 하는지 알고 싶다면 Section 14.11, “PostGIS Function Support Matrix” 를 참조하십시오.

긴 답변: Section 4.2.2, “도형 데이터 유형과 지리형 데이터 유형을 중첩해서 이용하는 경우” 과 함수 유형 매트릭스에 있는 더 긴 논의를 참조하십시오.

8. 지리형에 대해 더 자세히 질문하고 싶습니다. 지리형 열에 얼마나 큰 지리적 지역을 저장하고도 괜찮은 답을 얻을 수 있습니까? 모든 항목이 (SQLServer 2008 처럼) 북반구, 남반구 둘 중 하나의 반구에만 있어야 하는 극(pole) 제약이나 속도 제약 등 제약이 있나요?

이 단원에서 제대로 답변하기에는 질문이 너무 깊고 복잡합니다. Section 4.2.3, “지리형 고급 FAQ” 를 참조하십시오.

9. GIS 객체를 어떻게 데이터베이스에 삽입하나요?

먼저 사용자 GIS 데이터를 보관하기 위한 “도형” 또는 “지리형”의 열을 가진 테이블을 생성해야 합니다. 지리형 유형 데이터를 저장하는 일은 도형을 저장하는 일과는 조금 다릅니다. 지리형 저장에 대한 자세한 설명은 Section 4.2.1, “지리형의 기초” 을 참조하십시오.

도형을 저장하는 경우: PSQL 을 통해 데이터베이스에 연결한 다음 SQL 을 실행해보십시오:

```
CREATE TABLE gtest ( gid serial primary key, name varchar(20)
       , geom geometry(LINESTRING) );
```

도형 열 정의가 실패하는 경우, 아마도 PostGIS 함수와 객체를 해당 데이터베이스에 로드하지 않았거나 PostGIS 2.0 미만 버전을 사용하고 있을 겁니다. Section 2.4, "소스를 컴파일하고 설치하기: 상세 설명"를 참조하십시오.

그 다음, SQL 삽입(insert) 구문을 통해 도형을 테이블에 삽입할 수 있습니다. GIS 객체 자체는 OpenGIS 컨소시엄의 WKT(well-known text) 형식을 사용합니다:

```
INSERT INTO gtest (ID, NAME, GEOM)
VALUES (
  1,
  'First Geometry',
  ST_GeomFromText('LINESTRING(2 3,4 5,6 5,7 8)')
);
```

다른 GIS 객체에 대해 더 알아보려면 객체 참조를 살펴보십시오.

테이블의 GIS 데이터를 보려면:

```
SELECT id, name, ST_AsText(geom) AS geom FROM gtest;
```

반환값이 아래와 비슷하게 나타날 겁니다:

```
id | name            | geom
---+-----------------+-----------------------------
 1 | First Geometry | LINESTRING(2 3,4 5,6 5,7 8)
(1 row)
```

10. 공간 쿼리를 어떻게 구성하나요?

모든 다른 데이터베이스 쿼리를 구성하는 방식과 동일합니다. SQL은 반환값, 함수, 불 검사의 조합입니다.

공간 쿼리의 경우, 사용자가 쿼리를 구성할 때 염두에 두어야 할 두 가지 중요한 문제가 있습니다. 활용할 수 있는 공간 인덱스가 있는가? 그리고 대량의 도형에 대해 비용이 많이 드는 계산을 하고 있지는 않은가?

일반적으로, 피처들의 경계 상자(bounding box)가 교차하는지 검사하는 "교차 연산자(&&)"를 사용하고 싶을 것입니다. && 연산자가 쓸모 있는 이유는 검사 속도를 빠르게 하는 데 공간 인덱스를 쓸 수 있다면 && 연산자는 이를 활용할 것이기 때문입니다. 이럴 경우 쿼리 속도가 무척이나 빨라질 수 있습니다.

또한 검색 결과를 좁히기 위해 Distance(), ST_Intersects(), ST_Contains(), ST_Within() 등과 같은 공간 함수를 활용할 것입니다. 공간 쿼리 대부분은 인덱스가 붙은 검사와 공간 함수 검사 둘 모두를 포함합니다. 인덱스 검사는 관심 조건에 부합할 만한 투플만 반환되도록 반환 투플의 개수를 제한합니다. 그 다음 공간 함수를 이용해서 조건을 정확히 만족하는 검사를 실행합니다.

```
SELECT id, the_geom
FROM thetable
WHERE
  ST_Contains(the_geom,'POLYGON((0 0, 0 10, 10 10, 10 0, 0 0))');
```

11. 대용량 테이블에서 공간 쿼리 속도를 어떻게 높일 수 있나요?

대용량 테이블에 대한 빠른 쿼리란 (트랜잭션 지원과 함께) 공간 데이터베이스가 존재하는 이유 입니다. 따라서 적절한 인덱스가 중요합니다.

geometry 열을 가진 테이블에 공간 인덱스를 단들려면 다음과 같이 "CREATE INDEX" 함수를 사용하십시오:

```
CREATE INDEX [indexname] ON [tablename] USING GIST ( [geometrycolumn] );
```

"USING GIST" 옵션은 서버에게 GiST(Generalized Search Tree) 인덱스를 사용하라고 알려줍니다.



GiST 인덱스는 손실을 본다고 가정해야 합니다. 손실 인덱스는 인덱스를 만들기 위해 프록시 객체(공간 데이터의 경우 경계 상자)를 이용합니다.

또 사용자는 PostgreSQL 쿼리 계획자가 사용자 인덱스를 언제 이용할지에 대해 합리적인 판단을 내릴 수 있도록 인덱스에 대한 충분한 정보를 얻을 수 있도록 해야 합니다. 이를 위해 사용자 도형 테이블에 대한 "통계를 수집"해야 합니다.

PostgreSQL 8.0.x 이상 버전의 경우, **VACUUM ANALYZE** 명령어만 실행하면 됩니다.

PostgreSQL 7.4.x 이하 버전의 경우, **SELECT UPDATE_GEOMETRY_STATS()** 명령어를 실행하십시오.

12. 어째서 PostgreSQL R-Tree 인덱스를 지원하지 않나요?

PostGIS 초기 버전들은 PostgreSQL R-Tree 인덱스를 사용했습니다. 하지만 PostgreSQL R-Tree 는 0.6 버전 이후 완전히 폐기되었고, 공간 인덱스 작업은 R-Tree-over-GiST 스키마를 통해 제공됩니다.

테스트 결과 자생(native) R-Tree 와 GiST 의 검색 속도가 비슷하다는 것이 밝혀졌습니다. 자생 PostgreSQL R-Tree 는 GIS 피처와 함께 사용하기에 적합하지 않은 두 가지 한계점이 있습니다(이런 한계점은 일반적인 R-Tree 개념이 아니라 현재의 자생 PostgreSQL R-Tree 실행 때문이라는 점을 주의하십시오):

- PostgreSQL 내부의 R-Tree 인덱스는 8K 이상의 크기를 가지는 피처를 다루지 못 합니다. GiST 인덱스는 피처 자체 대신 경계 상자를 쓰는 "손실" 트릭을 통해 할 수 있습니다.
- PostgreSQL 내부의 R-Tree 인덱스는 "null 값을 처리할 수(null safe)" 없습니다. 따라서 null 도형을 담고 있는 도형 열에 대한 인덱스 생성 작업은 실패할 것입니다.

13. AddGeometryColumn() 함수 및 다른 모든 OpenGIS 기능들을 사용해야 하는 이유가 뭡니까?

만약 OpenGIS 지원 함수들을 사용하고 싶지 않다면 꼭 사용할 필요는 없습니다. 그냥 구 버전에서처럼 테이블을 생성하고 CREATE 구문으로 도형 열을 정의하십시오. 모든 도형의 SRID 값이 -1 이 될 것이며, OpenGIS 메타데이터 테이블은 제대로 채워지지 않을 것입니다. 하지만 이렇게 하면 PostGIS 기반 응용 프로그램 대부분이 오류를 낼 것입니다. 일반적으로 도형 테이블을 생성하려면 AddGeometryColumn() 을 이용하는 편이 좋습니다.

MapServer 는 geometry_columns 메타데이터를 활용하는 유일한 응용 프로그램입니다. 정확히 말하자면 MapServer 는 올바른 맵 좌표계로 피처를 실시간 재투영(on-the-fly reprojection)하기 위해 도형 열의 SRID 를 이용할 수 있습니다.

14. 한 객체의 반경 안에 있는 모든 객체를 찾는 데 가장 좋은 방법은 무엇인가요?

데이터베이스를 가장 효율적으로 사용하려면, 경계 상자 검사와 반경 검사를 결합한 반경 쿼리를 실행하는 것이 최선입니다. 경계 상자 검사는 공간 인덱스를 사용하며, 이후 반경 검사를 적용할 데이터의 서브셋에 빠르게 접근할 수 있도록 해줍니다.

ST_DWithin(geometry, geometry, distance) 함수는 인덱스가 붙은 거리 검색을 수행하는 유용한 방법입니다. 이 함수는 거리 반경을 둘러싸기에 충분한 크기의 검색 직사각형을 생성함으로써 작동합니다. 그 다음 결과값의 인덱스가 붙은 서브셋에 대해 정확한 거리 검색을 수행합니다.

예를 들어, POINT(1000 1000)의 100 미터 안에 있는 모든 객체를 찾으려면 다음과 같은 쿼리를 실행하면 됩니다:

```
SELECT * FROM geotable
WHERE ST_DWithin(geocolumn, 'POINT(1000 1000)', 100.0);
```

15. 어떻게 쿼리의 일부로서 좌표 재투영을 실행하나요?

재투영을 실행하려면, SPATIAL_REF_SYS 테이블에 원본 및 대상 좌표계가 정의되어 있어야 하며, 재투영되는 도형에 이미 SRID 가 설정되어 있어야 합니다. 이를 만족하는 경우, 재투영 작업은 원하는 대상 SRID 를 참조하면 끝나는 간단한 일입니다. 다음은 도형을 NAD 83 경위도로 투영하는 예시입니다. the_geom 의 SRID 가 -1 이 아닐 경우에만 (정의되지 않은 공간 참조일 경우에만) 이 스크립트가 작동할 것입니다.

```
SELECT ST_Transform(the_geom,4269) FROM geotable;
```

16. 제법 큰 용량의 도형에 ST_AsEWKT 와 ST_AsText 를 실행했더니 비어 있는 필드를 반환했습니다. 이유가 뭘까요?

아마도 대용량 텍스트를 출력하지 못 하는 pgAdmin 이나 다른 도구를 쓰고 있기 때문일 것입니다. 만약 사용자 도형이 충분히 크다면, 이런 도구에서는 공백으로 표출될 것입니다. 텍스트를 반드시 확인해야 한다면 PSQL 을 사용하거나 WKT 로 출력하십시오.

```
-- 도형의 개수가 정말로 공백인지 확인하려면
SELECT count(gid) FROM geotable WHERE the_geom IS NULL;
```

17. ST_Intersects 를 실행했는데 두 도형이 교차하지 않는다는 메시지를 받았습니다. 교차한다는 걸 내가 아는데 말이죠. 이유가 뭘까요?

일반적으로 두 가지 흔한 경우에 이런 일이 발생합니다. 사용자 도형이 유효하지 않거나 -- ST_IsValid 확인 -- 또는 ST_AsText 가 숫자를 잘라내서 사용자가 보지 못 하는 자리수가 많기 때문에 두 도형이 교차한다고 착각하는 경우입니다.

18. PostIGIS 를 이용하는 소프트웨어를 출시했는데, 그러면 이 소프트웨어도 PostGIS 처럼 GPL 을 따르는 라이선스를 가져야 합니까? 만약 PostGIS 를 이용한다면 모든 개인 코드를 공개해야 하나요?

그렇지 않습니다. 예를 들어, 리눅스 상에서 실행되는 오라클 데이터베이스를 생각해보십시오. 리눅스는 GPL 을 따르지만 오라클은 그렇지 않습니다. 리눅스에서 실행되는 오라클이 반드시 GPL 을 따라 배포되어야 할까요? 아닙니다. 따라서 사용자의 소프트웨어는 어떤 라이선스를 따르더라도 PostgreSQL/PostGIS 데이터베이스를 사용할 수 있습니다.

유일한 예외는 사용자가 PostGIS 소스코드를 변경하고, 변경된 PostGIS 버전을 배포할 때일 겁니다. 이런 경우라면 사용자가 변경한 PostGIS 의 코드를 공유해야 할 것입니다(그러나 PostGIS 상에서 실행되는 응용 프로그램의 코드는 아닙니다). 이렇게 제한된 경우라고 할지라도, 사용자가 바이너리를 배포하는 사람들에게만 소스코드를 배포하면 될 겁니다. GPL 은 사용자에게 소스코드를 발표(publish) 하라고 요구하지 않습니다. 사용자가 바이너리를 제공한 사람들과만 코드를 공유하도록 요구할 뿐입니다.

Chapter 4. PostGIS 활용: 데이터 관리 및 쿼리

4.1. GIS 객체

PostGIS가 지원하는 GIS 객체는 OpenGIS 컨소시엄(OGC)이 정의한 "단순 피처(simple feature)"의 상위 집합입니다. 0.9 버전부터, PostGIS는 OGC의 "SQL 용 단순 피처(Simple Features for SQL)" 사양서에 정의된 모든 객체 및 함수를 지원합니다.

PostGIS는 3DM, 3DZ 및 4D 좌표 지원을 통해 표준을 확장합니다.

4.1.1. OpenGIS WKB 및 WKT

OpenGIS 사양서에는 공간 객체를 표현하는 두 가지 표준 방식이 정의되어 있습니다. WKT(Well-Known Text) 형식과 WKB(Well-Known Binary) 형식입니다. WKT와 WKB 모두 객체 유형과 객체를 형성하는 좌표에 대한 정보를 포함하고 있습니다.

다음은 공간 객체 피처의 텍스트 문자열 표현식(WKT)의 예시입니다:

- POINT(0 0)
- LINESTRING(0 0,1 1,1 2)
- POLYGON((0 0,4 0,4 4,0 4,0 0),(1 1, 2 1, 2 2, 1 2,1 1))
- MULTIPOINT(0 0,1 2)
- MULTILINESTRING((0 0,1 1,1 2),(2 3,3 2,5 4))
- MULTIPOLYGON(((0 0,4 0,4 4,0 4,0 0),(1 1,2 1,2 2,1 2,1 1)), ((-1 -1,-1 -2,-2 -2,-2 -1,-1 -1)))
- GEOMETRYCOLLECTION(POINT(2 3),LINESTRING(2 3,3 4))

OpenGIS 사양서는 공간 객체의 내부 저장 형식이 공간 참조 시스템 식별자(SRID)를 포함하도록 요구합니다. 데이터베이스에 삽입될 공간 객체 생성시 SRID 가 필요합니다.

다음과 같은 인터페이스를 이용해서 이 형식을 입력/출력할 수 있습니다.

```
bytea WKB = ST_AsBinary(geometry);
text WKT = ST_AsText(geometry);
geometry = ST_GeomFromWKB(bytea WKB, SRID);
geometry = ST_GeometryFromText(text WKT, SRID);
```

예를 들어 OGC 공간 객체를 생성하고 삽입하기 위한 유효한 삽입 구문은 다음과 같을 것입니다:

```
INSERT INTO geotable ( the_geom, the_name )
  VALUES ( ST_GeomFromText('POINT(-126.4 45.32)', 312), 'A Place');
```

4.1.2. PostGIS EWKB, EWKT 및 기본형

OGC 형식은 2D 도형만을 지원하며, 관련 SRID는 입력/출력 표현식에 절대로 내장되지 않습니다.

PostGIS는 현재 OGC 형식의 상위 집합인 형식을 확장합니다(유효한 모든 WKB/WKT는 유효한 EWKB/EWKT가 됩니다). 하지만 향후 달라질 수도 있습니다. 특히 OGC가 PostGIS 확장 프로그램과 상충하는 새 형식을 들고 나온다면 말입니다. 그러므로 이 피처에 의존해서는 안 됩니다!

PostGIS EWKB/EWKT에는 3DM, 3DZ 및 4D 좌표 지원과 내장 SRID 정보가 추가됩니다.

다음은 확장 공간 객체 피처의 텍스트 문자열 표현식(EWKT)의 예시입니다. * 표시는 이번 PostGIS 버전에 새로 추가되었다는 의미입니다.

- POINT(0 0 0) -- XYZ
- SRID=32632;POINT(0 0) – SRID 추가 XY
- POINTM(0 0 0) -- XYM
- POINT(0 0 0 0) -- XYZM
- SRID=4326;MULTIPOINTM(0 0 0,1 2 1) -- SRID 추가 XYM
- MULTILINESTRING((0 0 0,1 1 0,1 2 1),(2 3 1,3 2 1,5 4 1))
- POLYGON((0 0 0,4 0 0,4 4 0,0 4 0,0 0 0),(1 1 0,2 1 0,2 2 0,1 2 0,1 1 0))
- MULTIPOLYGON(((0 0 0,4 0 0,4 4 0,0 4 0,0 0 0),(1 1 0,2 1 0,2 2 0,1 2 0,1 1 0)),((-1 -1 0,-1 -2 0,-2 -2 0,-2 -1 0,-1 -1 0)))
- GEOMETRYCOLLECTIONM(POINTM(2 3 9), LINESTRINGM(2 3 4, 3 4 5))
- MULTICURVE((0 0, 5 5), CIRCULARSTRING(4 0, 4 4, 8 4))
- POLYHEDRALSURFACE(((0 0 0, 0 0 1, 0 1 1, 0 1 0, 0 0 0)), ((0 0 0, 0 1 0, 1 1 0, 1 0 0, 0 0 0)), ((0 0 0, 1 0 0, 1 0 1, 0 0 1, 0 0 0)), ((1 1 0, 1 1 1, 1 0 1, 1 0 0, 1 1 0)), ((0 1 0, 0 1 1, 1 1 1, 1 1 0, 0 1 0)), ((0 0 1, 1 0 1, 1 1 1, 0 1 1, 0 0 1)))
- TRIANGLE ((0 0, 0 9, 9 0, 0 0))
- TIN(((0 0 0, 0 0 1, 0 1 0, 0 0 0)), ((0 0 0, 0 1 0, 1 1 0, 0 0 0)))

다음과 같은 인터페이스를 이용해서 이 형식을 입력/출력할 수 있습니다.

```
bytea EWKB = ST_AsEWKB(geometry);
text EWKT = ST_AsEWKT(geometry);
geometry = ST_GeomFromEWKB(bytea EWKB);
geometry = ST_GeomFromEWKT(text EWKT);
```

예를 들어 PostGIS 공간 객체를 생성하고 삽입하기 위한 유효한 삽입 구문은 다음과 같을 것입니다:

```
INSERT INTO geotable ( the_geom, the_name )
  VALUES ( ST_GeomFromEWKT('SRID=312;POINTM(-126.4 45.32 15)'), 'A Place' )
```

PostgreSQL의 "기본형(canonical form)"은 (어떤 함수도 호출하지 않는) 단순 쿼리로 얻게 되는 표현식을 뜻하며, 간단한 삽입, 업데이트, 또는 복사 쿼리와 함께 사용할 수 있습니다. PostGIS '도형' 유형의 경우 다음과 같은 기본형을 쓸 수 있습니다:

```
- Output
  - binary: EWKB
       ascii: HEXEWKB (EWKB in hex form)
- Input
  - binary: EWKB
       ascii: HEXEWKB|EWKT
```

예를 들어 다음 구문은 EWKT를 읽어들여 기본 ASCII 입력/출력 과정에서 HEXEWKB를 반환합니다:

```
=# SELECT 'SRID=4;POINT(0 0)'::geometry;

geometry
----------------------------------------------------
0101000020040000000000000000000000000000000000000000
(1 row)
```

4.1.3. SQL-MM Part 3

SQL 멀티미디어 응용 프로그램(SQL Multimedia Applications)의 공간 사양서는 일련의 원호 보간 곡선(circularly interpolated curve) 을 정의해서 SQL 사양에 대응하는 단순 피처를 확장합니다.

SQL-MM 정의는 3DM, 3DZ 및 4D 좌표를 포함하지만 SRID 정보를 내장할 수는 없습니다.

아직 WKT 확장 프로그램을 완전히 지원하지는 않습니다. 다음은 단순 곡선 도형의 일부 예시입니다.

- CIRCULARSTRING(0 0, 1 1, 1 0)
  CIRCULARSTRING(0 0, 4 0, 4 4, 0 4, 0 0)
  CIRCULARSTRING 은 기본 곡선 유형으로, 선형계의 LINESTRING과 비슷합니다. 단일 분절에는 시작점과 종단점(첫 번째 및 세 번째) 그리고 곡선 위의 다른 한 점 이렇게 포인트 세 개가 필요합니다. 예외는 닫힌 원으로, 이 경우 시작점과 종단점이 동일합니다. 이 경우 두 번째 포인트는 원호의 중심, 즉 원의 반대편이 되어야만 합니다. 원호를 함께 묶으려면, LINESTRING과 마찬가지로 한 원호의 마지막 포인트가 다음 원호의 첫 번째 포인트가 되어야 합니다. 즉 유효한 원형 스트링은 1을 초과하는 홀수 개수의 포인트들을 가져야만 한다는 뜻입니다.

- COMPOUNDCURVE(CIRCULARSTRING(0 0, 1 1, 1 0),(1 0, 0 1))
  복심곡선(compound curve)이란 곡선(원호) 분절과 선형 분절이 함께 있는 연속적인 단일 곡선을 말합니다. 즉 구성 요소들이 잘 형성되어야 함은 물론, (마지막을 제외한) 모든 구성 요소의 종단점이 다음 구성 요소의 시작점과 일치해야한 한다는 뜻입니다.

- CURVEPOLYGON(CIRCULARSTRING(0 0, 4 0, 4 4, 0 4, 0 0),(1 1, 3 3, 3 1, 1 1))
  만곡 폴리곤 내부의 복심곡선의 예: CURVEPOLYGON(COMPOUNDCURVE(CIRCULARSTRING(0 0,2 0, 2 1, 2 3, 4 3),(4 3, 4 5, 1 4, 0 0)), CIRCULARSTRING(1.7 1, 1.4 0.4, 1.6 0.4, 1.6 0.5, 1.7 1))
  CURVEPOLYGON은 일반 폴리곤입니다. 외곽선과 함께 0개 이상의 내곽선을 가지고 있을 뿐입니다. 차이라면 내외곽선이 원형 스트링, 선형 스트링, 또는 복합 스트링 형태를 할 수 있다는 점입니다. PostGIS는 1.4버전부터 만곡 폴리곤에 대해 복심곡선을 지원합니다.

- MULTICURVE((0 0, 5 5),CIRCULARSTRING(4 0, 4 4, 8 4))
  MULTICURVE는 원형 스트링, 선형 스트링, 복합 스트링을 포함할 수 있는 곡선 집합입니다.

- MULTISURFACE(CURVEPOLYGON(CIRCULARSTRING(0 0, 4 0, 4 4, 0 4, 0 0),(1 1, 3 3, 3 1, 1 1)),((10 10, 14 12, 11 10, 10 10),(11 11, 11.5 11, 11 11.5, 11 11)))
  MULTISURFACE는 면 집합으로, (선형) 폴리곤일 수도 만곡 폴리곤일 수도 있습니다.



PostGIS 1.4 이전 버전은 만곡 폴리곤 안에서 복심곡선을 지원하지 않았지만, 1.4 버전 이후로 만곡 폴리곤 안에서 복심곡선을 지원하고 있습니다.



SQL-MM 실행 과정에서 지정된 허용 오차를 기준으로 부동소수점을 비교합니다. 현재 허용 오차는 1E-8 입니다.

4.2. PostGIS Geography Type

지리형 유형은 (종종 "측지" 좌표, 또는 "위도/경도"나 "경도/위도"라고 불리는) "지리" 좌표로 표현되는 공간 피처를 자체적으로 지원합니다. 지리 좌표는 각도 단위(도)를 사용하는 구면(球面) 좌표입니다.

PostGIS 도형 유형은 평면을 기반으로 합니다. 평면상에서 두 포인트 사이의 가장 짧은 경로는 직선입니다. 즉 데카르트 수학과 직선 벡터를 이용해서 도형에 대해 계산(면적, 거리, 길이, 교차점 등)한다는 뜻입니다.

PostGIS 지리형 유형은 구면을 기반으로 합니다. 구면상에서 두 포인트 사이의 가장 짧은 경로는 대권(大圈; great circle arc)입니다. 즉 지리형에 대한 계산(면적, 거리, 길이, 교차점 등)은 더 복잡한 수학을 이용해서 구면상에서 이루어져야 한다는 뜻입니다. 더 정확하게 측정하려면 지구의 실제 회전타원체(spheroidal shape)를 고려해서 계산해야 하는데, 수학이 아주 복잡해질 수밖에 없습니다.

기저 수학이 훨씬 더 복잡하기 때문에, 지리형 유형을 위해 정의된 함수는 도형 유형의 함수보다 더 적습니다. 시간이 지날수록 새 알고리즘이 추가되어 지리형 유형의 역량은 확장될 것입니다.

한 가지 제약 사항이 있다면 지리형이 WGS84 경위도 좌표계(SRID:4326)만 지원한다는 점입니다. 이 새 유형을 지원하는 GEOS 함수는 하나도 없습니다. 이 문제를 해결하려면 도형과 지리형 유형을 번갈아 가며 변환할 수 있습니다.

이 새로운 지리형 유형은 PostgreSQL 8.3 이상 버전의 typmod 정의 형식을 이용하므로 지리형 필드를 담은 테이블을 한번에 추가시킬 수 있습니다. 곡선을 제외한 모든 표준 OGC 형식을 지원합니다.

4.2.1. Geography Basics

지리형 유형은 단순 피처 중에서도 가장 단순한 것만 지원합니다. 표준 도형 유형 데이터는, SRID 4326인 경우, 지리형으로 자동 변환됩니다. 또한 데이터를 삽입하는 데 EWKT 및 EWKB 규약을 이용할 수도 있습니다.

- POINT: 2D 포인트 도형을 담은 테이블 생성:

```
CREATE TABLE testgeog(gid serial PRIMARY KEY, the_geog geography(POINT,4326) );
```

- Z 좌표 포인트를 담은 테이블 생성:

```
CREATE TABLE testgeog(gid serial PRIMARY KEY, the_geog geography(POINTZ,4326) );
```

- LINESTRING
- POLYGON
- MULTIPOINT
- MULTILINESTRING
- MULTIPOLYGON
- GEOMETRYCOLLECTION

새 지리형 필드는 geometry_columns에 등록되지 않습니다. geography_columns라는 새로운 뷰에 등록되는데, 이 뷰는 시스템 카탈로그를 기반으로 하기 때문에 AddGeom을 사용할 필요없이 자동적으로 항상 최신 상태를 유지합니다. 마치 함수처럼 말이죠.

그러면 "geography_columns" 뷰를 확인해서 사용자 테이블이 목록에 추가됐는지 살펴보십시오.

CREATE TABLE 문법을 이용해서 지리형(GEOGRAPHY) 열을 가진 새 테이블을 생성할 수 있습니다. 도형(GEOMETRY)과는 달리, 메타데이터에 해당 열을 등록하기 위해 별도로 AddGeometryColumns() 함수를 실행할 필요가 없습니다.

```
CREATE TABLE global_points (
   id SERIAL PRIMARY KEY,
   name VARCHAR(64),
   location GEOGRAPHY(POINT,4326)
  );
```

location 열이 지리형 유형인데, 지리형 유형은 두 가지 선택적인 변경자(modifier)를 지원한다는 사실을 주의하십시오. 열에 들어갈 수 있는 형태 및 차원 종류를 제한하는 유형 변경자, 그리고 좌표 참조 식별자를 특정 숫자로 제한하는 SRID 변경자입니다.

유형 변경자가 허용하는 값은 다음과 같습니다. POINT, LINESTRING, POLYGON, MULTIPOINT, MULTILINESTRING, MULTIPOLYGON. 또 이 변경자는 Z, M 및 ZM이라는 접미사를 통해 차원수 제약도 지원합니다. 따라서, 예를 들자면 'LINESTRINGM'의 변경자는 3차원 이하의 라인 스트링만을 허용할 것이며, 세 번째 차원을 기준으로 취급할 것입니다. 마찬가지로 'POINTZM'은 네 가지 차원의 데이터를 입력해야 할 것입니다.

SRID 변경자는 현재 제한적으로 사용됩니다. 허용되는 값이 4326(WGS84)뿐입니다. SRID를 지정하지 않을 경우 0값(정의되지 않은 회전타원체)을 사용하는데, 이와 상관없이 WGS84를 이용해서 계산이 이루어질 것입니다.

향후 WGS84만이 아니라 다른 회전타원체 상에서도 계산할 수 있게 될 겁니다.

사용자 테이블 생성을 완료했다면, GEOGRAPHY_COLUMNS 테이블에서 사용자 테이블을 살펴볼 수 있습니다:

```
-- 메타데이터 뷰의 내용을 살펴봅시다
SELECT * FROM geography_columns;
```

도형 열을 사용하는 경우와 동일한 방법으로 테이블에 데이터를 삽입할 수 있습니다:

```
-- 테스트용 테이블에 데이터를 추가해봅시다
```

```
INSERT INTO global_points (name, location) VALUES ('Town',
ST_GeographyFromText('SRID=4326;POINT(-110 30)') );
INSERT INTO global_points (name, location) VALUES ('Forest',
ST_GeographyFromText('SRID=4326;POINT(-109 29)') );
INSERT INTO global_points (name, location) VALUES ('London',
ST_GeographyFromText('SRID=4326;POINT(0 49)') );
```

도형과 동일한 방법으로 인덱스를 생성합니다. PostGIS가 열 유형이 지리형인 것을 감지하고 일반적인 도형 용 평면 인덱스 대신 적절한 구면 기반 인덱스를 생성할 것입니다.

```
-- 테스트 테이블에 구면 인덱스 생성
  CREATE INDEX global_points_gix ON global_points USING GIST ( location );
```

쿼리 및 측정 함수는 미터 단위를 사용합니다. 따라서 거리 파라미터는 미터로 표현되어야 하고, 반환값도 미터(또는 면적의 경우 평방미터) 단위가 될 것입니다.

```
-- 다음은 거리 쿼리입니다. 주의: 런던이 허용 범위 1000km 밖에 있습니다
  SELECT name FROM global_points WHERE ST_DWithin(location,
ST_GeographyFromText('SRID=4326;POINT(-110 29)'), 1000000);
```

시애틀에서 런던으로 가는 비행기가(LINESTRING(-122.33 47.606, 0.0 51.5)) 레이캬비크에(POINT(-21.96 64.15)) 얼마나 접근하는지 계산해보면, 실제 계산시 지리형이 얼마나 강력한지 알 수 있습니다.

```
-- 지리형을 이용한 거리 계산(122.2km)
  SELECT ST_Distance('LINESTRING(-122.33 47.606, 0.0 51.5)'::geography,
'POINT(-21.96 64.15)':: geography);
```

```
-- 도형을 이용한 거리 계산(13.3"도")
  SELECT ST_Distance('LINESTRING(-122.33 47.606, 0.0 51.5)'::geometry,
'POINT(-21.96 64.15)':: geometry);
```

지리형 유형이 시애틀과 런던을 잇는 대권항로와 레이캬비크 사이의 구면 상 가장 짧은 거리를 실제로 계산할 수 있습니다.

대권 매퍼(Great Circle mapper) 도형 유형은 평면 세계지도 상에서 시애틀과 런던을 직선으로 잇는 경로와 레이캬비크 사이의 아무 의미도 없는 데카르트 거리를 계산합니다. 결과값의 명목상 단위를 "도(degree)"라고 할 수도 있겠지만, 결과값은 세 포인트 사이의 어떤 실제 각도 차이도 반영하지 않기 때문에 "도"라고 하는 것조차 부정확한 일이 됩니다.

4.2.2. 도형 데이터 유형과 지리형 데이터 유형을 중첩해서 이용하는 경우

새로운 지리형 유형은 데이터를 경도/위도 좌표로 저장할 수 있도록 해주지만, 단점도 있습니다. 도형을 대상으로 정의된 함수보다 지리형 대상 함수가 더 적고, 그 정의된 함수도 실행하는 데 CPU 시간을 더 많이 잡아먹습니다.

사용자가 선택한 유형은 사용자가 빌드하는 응용 프로그램 영역에서 기대한대로 동작하도록 적합한 조건을 갖춰야 합니다. 사용자 데이터가 전세계 또는 광대한 대륙 지역을 포괄할 예정입니까 아니면 시, 도, 군 또는 그 이하의 지자체에 국한될 예정입니까?

- 사용자 데이터가 좁은 지역에 국한된다면, 사용 가능한 실행성 및 기능성 관점에서, 적합한 투영을 선택하고 도형을 이용하는 것이 최선의 해결책이 될 수도 있습니다.
- 사용자 데이터가 전세계 또는 대륙에 걸쳐 있을 경우, 지리형을 이용하면 일일이 어떤 투영법을 이용할지 고민하지 않고 시스템을 빌드할 수도 있습니다. 사용자 데이터를 경도/위도로 저장하고, 지리형을 대상으로 정의된 함수를 이용하십시오.
- 투영에 대한 이해가 부족하고, 따로 공부하고 싶지도 않으며, 지리형 사용시 기능성이 제한된다는 사실을 받아들일 준비가 되어 있다면, 도형보다 지리형을 사용하는 편이 더 쉬울 수 있습니다. 그냥 사용자 데이터를 경도/위도로 로드한 다음 작업을 시작하십시오.

지리형과 도형을 각각 지원하는 함수를 비교해보려면 Section 14.11, "PostGIS Function Support Matrix"를 참조하십시오. 지리형 함수의 목록 및 설명을 간단하게 살펴보려면 Section 14.4, "PostGIS Geography Support Functions"를 참조하십시오.

4.2.3. 지리형 고급 FAQ

1. 계산 작업시 구체 상에서 계산하게 됩니까 회전타원체 상에서 하게 됩니까?

기본적으로, 모든 거리 및 면적 계산은 회전타원체 상에서 이루어집니다. 좁은 지역을 대상으로 한 계산의 결과와 해당 지역에 적절한 투영법을 적용한 평면 상 계산 결과는 일치할 것입니다. 더 넓은 지역이라면 투영법을 적용한 평면 상 계산보다 회전타원체 상 계산이 언제나 더 정확할 것입니다.

최종 불 파라미터 'FALSE'를 설정하면 모든 지리형 함수가 구체 상 계산을 할 수 있습니다. 이렇게 하면 계산 속도가 조금 빨라질 것입니다. 특히 도형들이 매우 단순한 경우에 말입니다.

2. 날짜변경선과 남극/북극은 어떻습니까?

모든 계산은 날짜변경선이나 양극을 고려하지 않고 이루어집니다. 좌표가 회전타원체(경도/위도)이기 때문에 날짜변경선을 지나는 형상이라도, 계산이라는 관점에서 보면, 다른 어떤 형상과도 다를 바가 없습니다.

3. 공간 처리할 수 있는 가장 긴 원호가 무엇인가요?

두 포인트 사이의 "보간 라인"으로 대권호(great circle arc)를 이용합니다. 즉 대권을 따라 어느 방향으로 이동하느냐에 따라 두 포인트가 실제로는 두 가지 방식으로 만난다는 뜻입니다. 모든 코드는 포인트들이 대권을 따라 가는 두 경로 가운데 '짧은' 경로로 만난다고 가정합니다. 결과적으로, 180 도 이상의 원호를 가진다면 정확히 모델링된 형상이 아니게 됩니다.

4. 유럽이나 러시아의 면적을 계산하거나 또는 광대한 지역을 삽입하는 작업이 이렇게 느린 이유가 뭐지요?

폴리곤이 너무나 크기 때문이지요! 광대한 지역은 두 가지 이유로 좋지 않습니다. 먼저 경계가 워낙 길기 때문에 어떤 쿼리를 실행하든 인덱스가 피처 전체를 읽어오는 경향이 있습니다. 그리고 꼭짓점

개수도 너무 많아서 거리, 밀폐 여부 등의 테스트를 할 때 적어도 한 번, 때로는 n 번 이상(이때 n 은 다른 후보 피처의 꼭짓점 개수) 꼭짓점 목록 전체를 훑어야 하기 때문입니다.

도형의 경우, 대용량 폴리곤을 대상으로 좁은 지역에 대한 쿼리를 할 때 사용자 도형 데이터를 더 작은 덩어리들로 "비정규화"해서 인덱스가 효율적으로 객체의 일부분을 하위 쿼리(subquery)할 수 있도록 만들어 쿼리 시 매번 전체 객체를 읽어올 필요가 없도록 하는 편이 좋습니다. 유럽 전체를 폴리곤 한 개로 저장할 수 있다고 해서 꼭 그렇게 해야 한다는 뜻은 아닙니다.

4.3. OpenGIS 표준 이용

OpenGIS 의 "SQL 용 단순 피처 사양서(Simple Features Specification for SQL)"는 표준 GIS 객체 유형, 그 유형들을 다루기 위해 필요한 함수, 그리고 메타데이터 테이블의 집합을 정의합니다. 메타데이터의 일관성을 유지하기 위해 공간 열 생성 및 삭제와 같은 작업은 OpenGIS 가 정의한 특별한 과정을 거쳐 이루어집니다.

OpenGIS 메타데이터 테이블이 두 개 있습니다. SPATIAL_REF_SYS 와 GEOMETRY_COLUMNS 입니다. SPATIAL_REF_SYS 테이블은 공간 데이터베이스가 사용하는 좌표계의 숫자 ID 및 텍스트 설명을 담습니다.

4.3.1. SPATIAL_REF_SYS 테이블과 공간 참조 시스템

SPATIAL_REF_SYS 테이블은 OGC를 따르는 PostGIS에 내장된 테이블로, 공간 참조 시스템을 서로 변환/재투영하는 데 필요한 공간 참조 시스템(spatial reference systems) 3000여 개의 목록 및 상세 정보를 가지고 있습니다.

PostGIS의 SPATIAL_REF_SYS 테이블이 proj 라이브러리가 처리할 수 있는, 좀 더 널리 사용되는 공간 참조 시스템 3000여 개를 담고 있긴 하지만 현재까지 알려진 모든 공간 참조 시스템을 다 담고 있지는 않으며, 사용자가 proj4의 구조를 잘 알고 있다면 자기만의 사용자 지정 투영을 정의할 수도 있습니다. 공간 참조 시스템 대부분은 특정 지역에 특화되어 있으며, 특화된 지역 범위 바깥에서 사용할 경우 어떤 의미도 없다는 점을 명심하십시오.

핵심 SPATIAL_REF_SYS 테이블에 정의되지 않은 공간 참조 시스템은 http://spatialreference.org/ 에 훌륭하게 정리되어 있습니다.

좀 더 널리 사용되는 공간 참조 시스템에는 4326 - WGS 84 Long Lat, 4269 - NAD 83 Long Lat, 3395 - WGS 84 World Mercator, 2163 - US National Atlas Equal Area, 그리고 NAD 83 및 WGS 84 UTM 대(帶; zone)의 공간 참조 시스템이 있습니다. 각 UTM 대는 측정에 가장 이상적인 공간 참조 시스템이지만, 6도 범위의 지역에만 특화되어 있습니다.

미국 여러 주의 평면 공간 참조 시스템(미터 또는 피트 기반)은 각 주마다 보통 한 개 또는 두 개가 존재합니다. 미터 기반 공간 참조 시스템 대부분은 SPATIAL_REF_SYS 테이블에 들어 있지만, 피트 기반 또는 ESRI가 생성한 공간 참조 시스템 중 상당수는 사용자가 spatialreference.org 에서 찾아와야 합니다.

사용자 관심 지역에 어떤 UTM 대를 사용할지 결정하는 데에 대한 자세한 정보는 utmzone PostGIS plpgsql helper function 를 확인해보십시오.

SPATIAL_REF_SYS 테이블 정의는 다음과 같습니다:

```
CREATE TABLE spatial_ref_sys (
  srid      INTEGER NOT NULL PRIMARY KEY,
  auth_name  VARCHAR(256),
```

```
  auth_srid  INTEGER,
  srtext     VARCHAR(2048),
  proj4text  VARCHAR(2048)
)
```

SPATIAL_REF_SYS 테이블의 열들은 다음과 같습니다:

SRID 데이터베이스 내부에서 공간 참조 시스템(SRS)을 고유하게 식별하는 정수값입니다.

AUTH_NAME 해당 참조 시스템을 위해 인용되는 표준 또는 표준들 본체의 명칭입니다. 예를 들어 "EPSG"는 유효한 AUTH_NAME 이라고 할 수 있습니다.

AUTH_SRID AUTH_NAME 에 인용된 권위체(Authority)가 정의하는 공간 참조 시스템의 ID입니다. EPSG의 경우, 이 열에 EPSG 투영 코드가 들어갑니다.

SRTEXT 공간 참조 시스템의 WKT(Well-Known Text) 표현식입니다. 다음은 WKT SRS 표현식의 예입니다:

```
PROJCS["NAD83 / UTM Zone 10N",
  GEOGCS["NAD83",
       DATUM["North_American_Datum_1983",
         SPHEROID["GRS 1980",6378137,298.257222101]
       ],
       PRIMEM["Greenwich",0],
       UNIT["degree",0.0174532925199433]
  ],
  PROJECTION["Transverse_Mercator"],
  PARAMETER["latitude_of_origin",0],
  PARAMETER["central_meridian",-123],
  PARAMETER["scale_factor",0.9996],
  PARAMETER["false_easting",500000],
  PARAMETER["false_northing",0],
  UNIT["metre",1]
]
```

EPSG 투영 코드 및 이에 대응하는 WKT 표현식 목록은 http://www.opengeospatial.org/ 를 살펴보십시오. WKT 에 대한 개괄적인 논의는 http://www.opengeospatial.org/standards 에 있는 OpenGIS 의 "좌표 변환 서비스 시행 사양서(Coordinate Transformation Services Implementation Specification)"를 살펴보십시오. EPSG(European Petroleum Survey Group) 및 EPSG 의 공간 참조 시스템 데이터베이스에 대한 정보는 http://www.epsg.org 를 살펴보십시오.

PROJ4TEXT PostGIS 는 좌표 변환 기능을 제공하기 위해 proj4 라이브러리를 이용합니다. PROJ4TEXT 열이 특정 SRID 에 대응하는 proj4 좌표 정의 스트링을 담고 있습니다. 다음은 그 예입니다:

```
+proj=utm +zone=10 +ellps=clrk66 +datum=NAD27 +units=m
```

이에 대한 자세한 정보는 http://trac.osgeo.org/proj/ 주소의 proj4 웹사이트를 참조하십시오. spatial_ref_sys.sql 파일이 모든 EPSG 투영에 대한 SRTEXT 및 PROJ4TEXT 정의를 담고 있습니다.

4.3.2. THE GEOMETRY_COLUMNS VIEW

PostGIS 2.0.0 미만 버전에서는 geometry_columns를 직접 편집할 수 있어서 종종 도형 열의 실제 정의와 달라질 때도 있었습니다. PostGIS 2.0.0 버전부터 GEOMETRY_COLUMNS 가 이전 버전과 동일한 전향(前向; front-facing) 구조이지만 데이터베이스 시스템 카탈로그를 읽어오는 뷰가 되었습니다. 그 구조는 다음과 같습니다:

```
\d geometry_columns
```

```
View "public.geometry_columns"
     Column      |         Type          | Modifiers
-----------------+-----------------------+-----------
 f_table_catalog   | character varying(256) |
 f_table_schema    | character varying(256) |
 f_table_name      | character varying(256) |
 f_geometry_column | character varying(256) |
 coord_dimension   | integer                |
 srid              | integer                |
 type              | character varying(30)  |
```

열의 의미는 이전 버전과 달라지지 않았습니다:

F_TABLE_CATALOG, F_TABLE_SCHEMA, F_TABLE_NAME 도형 열을 담고 있는 피처 테이블의 조건을 완전히 만족하는 명칭입니다. "카탈로그"와 "스키마"가 오라클 용어라는 점을 주목하십시오. "카탈로그"를 대체하는 PostgreSQL 용어가 없기 때문에 해당 열은 공백으로 남게 됩니다. "스키마"의 경우 PostgreSQL 스키마 명칭이 사용됩니다(기본값은 public 입니다).

F_GEOMETRY_COLUMN 피처 테이블이 담고 있는 도형 열의 명칭입니다.

COORD_DIMENSION 열의 공간 차원(2, 3, 또는 4차원)입니다.

SRID 해당 테이블이 담고 있는 도형의 좌표가 사용하는 공간 좌표 시스템의 ID로, SPATIAL_REF_SYS 를 참조하는 외래 키(foreign key)입니다.

TYPE 공간 객체의 유형입니다. 공간 열을 단일 유형으로 제약하려면 다음 유형 가운데 하나를 이용하십시오. POINT, LINESTRING, POLYGON, MULTIPOINT, MULTILINESTRING, MULTIPOLYGON, GEOMETRYCOLLECTION 또는 이에 상응하는 XYM 버전의 POINTM, LINESTRINGM, POLYGONM, MULTIPOINTM, MULTILINESTRINGM, MULTIPOLYGONM, GEOMETRYCOLLECTIONM. 혼합 유형 집합을 이용하려면 유형으로 "GEOMETRY"를 이용할 수 있습니다.



이런 속성은 (아마도) OpenGIS 사양에는 포함되지 않겠지만, 유형의 동질성(homogeneity)을 보장하는 데 필요합니다.

4.3.3. 공간 테이블 생성

공간 데이터를 담은 테이블을 한 번에 생성할 수 있습니다. WGS84 경위도 좌표의 2D 라인스트링 도형 열을 가진 도로 테이블을 생성하는 다음 예시를 살펴보십시오.

```
CREATE TABLE ROADS ( ID int4
             , ROAD_NAME varchar(25), geom geometry(LINESTRING,4326) );
```

3D 라인스트링을 추가하는 다음 예시처럼, 표준 ALTER TABLE 명령어를 사용해서 추가적인 열을 추가할 수 있습니다.

```
ALTER TABLE roads ADD COLUMN geom2 geometry(LINESTRINGZ,4326);
```

이전 버전과의 호환성을 위해, 여전히 공간 테이블을 관리 도구를 사용해서 두 단계에 걸쳐 생성할 수도 있습니다.

- 일반적인 비공간 테이블을 생성하십시오.
  예: **CREATE TABLE ROADS (ID int4, ROAD_NAME varchar(25))**
- OpenGIS “AddGeometryColumn" 함수를 이용해서 테이블에 공간 열을 추가하십시오. 자세한 내용은 AddGeometryColumn 을 참조하십시오.

문법은 다음과 같습니다:

```
AddGeometryColumn(
  <schema_name>,
  <table_name>,
  <column_name>,
  <srid>,
  <type>,
  <dimension>
)
```

또는 현재 스키마를 이용해서:

```
AddGeometryColumn(
  <table_name>,
  <column_name>,
  <srid>,
  <type>,
  <dimension>
)
```

예1: **SELECT AddGeometryColumn('public', 'roads', 'geom', 423, 'LINESTRING', 2)**

예2: **SELECT AddGeometryColumn('roads', 'geom', 423, 'LINESTRING', 2)**

다음은 SQL을 이용해서 테이블을 생성하고 공간 열을 추가하는 예시입니다("128"의 SRID가 이미 존재한다고 가정합니다)

```
CREATE TABLE parks (
  park_id   INTEGER,
  park_name  VARCHAR,
  park_date  DATE,
  park_type  VARCHAR
);
SELECT AddGeometryColumn('parks', 'park_geom', 128, 'MULTIPOLYGON', 2 );
```

다음은 포괄적인 "GEOMETRY" 유형 및 0값을 가진, 정의되지 않은 SRID를 이용하는 또다른 예시입니다:

```
CREATE TABLE roads (
  road_id INTEGER,
  road_name VARCHAR
```

```
);
SELECT AddGeometryColumn( 'roads', 'roads_geom', 0, 'GEOMETRY', 3 );
```

4.3.4. geometry_columns 에 도형 열을 직접 등록하기

AddGeometryColumn() 함수를 이용하면 도형 열을 생성하고, 이 새 열을 geometry_columns 테이블에 등록도 합니다. 사용자의 소프트웨어가 geometry_columns 테이블을 활용하고 있다면, 사용자가 쿼리해야 하는 모든 도형 열을 반드시 이 뷰에 등록해야 합니다. PostGIS 2.0 버전부터 geometry_columns 테이블을 직접 편집할 수 없고 모든 도형 열은 자동 등록됩니다.

하지만 생성시 특정 유형으로 정의하지 않은 열이라면 일반 도형 열로 등록될 수도 있습니다.

AddGeometryColumn() 함수를 쓸 수 없을 때 이런 일이 발생할 수 있는 두 가지 경우가 있는데, SQL 뷰 그리고 대규모 삽입(bulk insert)의 경우입니다. 이런 경우, 해당 열에 제약 조건을 걸어서 geometry_columns 테이블 등록을 바로잡을 수 있습니다. PostGIS 2.0 이상 버전에서는, 사용자 열이 typmod 기반이라면 생성 과정 중에 정확하게 등록할 것이기 때문에 아무것도 할 필요가 없다는 점을 기억하십시오.

```
-- 이렇게 생성된 뷰가 있다고 합시다.
CREATE VIEW  public.vwmytablemercator AS
       SELECT gid, ST_Transform(geom,3395) As geom, f_name
       FROM public.mytable;

-- PostGIS 2.0 이상 버전에 정확하게 등록하려면
-- 도형을 형변환해야 합니다.
--
DROP VIEW public.vwmytablemercator;
CREATE VIEW  public.vwmytablemercator AS
       SELECT  gid,  ST_Transform(geom,3395)::geometry(Geometry,  3395)  As
geom, f_name
       FROM public.mytable;

-- 도형 유형이 2D 폴리곤이란 사실을 확실히 알고 있을 경우 다음과 같이 할 수 있습니다.
DROP VIEW public.vwmytablemercator;
CREATE VIEW  public.vwmytablemercator AS
       SELECT  gid,  ST_Transform(geom,3395)::geometry(Polygon,  3395)  As
geom, f_name
       FROM public.mytable;
```

```
-- 대규모 삽입 작업을 통해 파생 테이블을 생성했다고 합시다.
SELECT poi.gid, poi.geom, citybounds.city_name
INTO myschema.my_special_pois
FROM poi INNER JOIN citybounds ON ST_Intersects(citybounds.geom, poi.geom);

-- 새 테이블에 2D 인덱스를 생성합니다.
CREATE INDEX idx_myschema_myspecialpois_geom_gist
  ON myschema.my_special_pois USING gist(geom);

-- 사용자 포인트가 3D 또는 3M 포인트일 경우,
-- 2D 인덱스가 아니라 nD 인덱스를 생성하는 편이 좋을 수도 있습니다.
-- 다음처럼 말이죠.
CREATE INDEX my_special_pois_geom_gist_nd
       ON my_special_pois USING gist(geom gist_geometry_ops_nd);

-- 이 새 테이블의 도형 열을 geometry_columns 테이블에 직접 등록하는
```

```
-- 다음 방법은 PostGIS 2.0 이후 버전 및 1.4 이후 버전 모두에서 동작합니다.
-- PostGIS 2.0 버전의 경우 열을 typmod 기반으로 만들기 위해
-- 테이블의 기저 구조를 변경할 것입니다.
-- PostGIS 2.0 이전 버전의 경우, 동일한 방법으로 뷰를 등록할 수도 있습니다.
SELECT populate_geometry_columns('myschema.my_special_pois'::regclass);

-- PostGIS 2.0 버전을 사용중이고 어떤 이유에서든
-- 구식 제약조건 기반 정의 동작이 필요한 경우
-- (모든 자식 객체가 동일한 유형과 SRID가 아닌 상속 테이블의 경우 등)
-- 선택적인 새 use_typmod argument 파라미터를 거짓으로 설정하십시오.
SELECT     populate_geometry_columns('myschema.my_special_pois'::regclass,
false);
```

구식 제약조건 기반 방법을 여전히 지원하긴 하지만, 뷰에서 직접적으로 사용되는 제약조건 기반 도형 열은 typmod 기반 열과는 달리 geometry_columns 테이블에 정확하게 등록되지 않을 겁니다. 다음은 typmod를 이용하는 열과 제약조건을 이용하는 또다른 열을 정의하는 예시입니다.

```
CREATE TABLE pois_ny(gid SERIAL PRIMARY KEY
   , poi_name text, cat varchar(20)
   , geom geometry(POINT,4326) );
SELECT AddGeometryColumn('pois_ny', 'geom_2160', 2160, 'POINT', 2, false);
```

PSQL에서 실행할 경우

```
\d pois_ny;
```

두 열이 서로 다르게 정의되었다는 사실을 알 수 있습니다. 하나는 typmod, 다른 하나는 제약조건으로 정의되었습니다.

```
Table "public.pois_ny"
 Column    |     Type            |            Modifiers
-----------+---------------------+-----------------------------------------------------
 gid       | integer             | not null default nextval('pois_ny_gid_seq'::regclass)
 poi_name  | text                |
 cat       | character varying(20)|
 geom      | geometry(Point,4326) |
 geom_2160| geometry             |
Indexes:
   "pois_ny_pkey" PRIMARY KEY, btree (gid)
Check constraints:
   "enforce_dims_geom_2160" CHECK (st_ndims(geom_2160) = 2)
   "enforce_geotype_geom_2160" CHECK (geometrytype(geom_2160) = 'POINT'::text
       OR geom_2160 IS NULL)
   "enforce_srid_geom_2160" CHECK (st_srid(geom_2160) = 2160)
```

둘 다 geometry_columns 테이블에 정확하게 등록됩니다.

```
SELECT f_table_name, f_geometry_column, srid, type
       FROM geometry_columns
       WHERE f_table_name = 'pois_ny';
```

```
f_table_name | f_geometry_column | srid | type
-------------+-------------------+------+-------
```

```
pois_ny      | geom               | 4326 | POINT
pois_ny      | geom_2160          | 2160 | POINT
```

하지만 -- 다음과 같은 뷰를 생성하려 한다면

```
CREATE VIEW vw_pois_ny_parks AS
SELECT *
  FROM pois_ny
  WHERE cat='park';

SELECT f_table_name, f_geometry_column, srid, type
       FROM geometry_columns
       WHERE f_table_name = 'vw_pois_ny_parks';
```

typmod 기반 도형 열은 정확하게 등록되지만, 제약조건 기반 도형 열은 정확하게 등록되지 않습니다.

```
f_table_name      | f_geometry_column | srid |   type
------------------+-------------------+------+----------
 vw_pois_ny_parks | geom              | 4326 | POINT
 vw_pois_ny_parks | geom_2160         |    0 | GEOMETRY
```

PostGIS 향후 버전에서는 변경될 수도 있지만, 현재 버전에서 제약조건 기반 뷰 열을 정확하게 등록하려면 다음과 같이 해야 합니다:

```
DROP VIEW vw_pois_ny_parks;
CREATE VIEW vw_pois_ny_parks AS
SELECT gid, poi_name, cat
  , geom
  , geom_2160::geometry(POINT,2160) As geom_2160
  FROM pois_ny
  WHERE cat='park';
```

```
SELECT f_table_name, f_geometry_column, srid, type
       FROM geometry_columns
       WHERE f_table_name = 'vw_pois_ny_parks';
```

```
    f_table_name   | f_geometry_column | srid | type
-------------------+-------------------+------+-------
 vw_pois_ny_parks | geom              | 4326 | POINT
 vw_pois_ny_parks | geom_2160         | 2160 | POINT
```

4.3.5. 도형의 OpenGIS 준수 보장

PostGIS 는 OGC(Open Geospatial Consortium)의 OpenGIS 사양서를 준수합니다. 따라서 많은 PostGIS 방식이 작업 대상인 도형이 단순하고 또 유효할 것을 요구, 아니, 좀 더 정확히 말하자면 가정합니다. 예를 들어 그 외부에 구멍이 있다고 정의된 폴리곤의 면적을 계산하거나 단순하지 않은 경계선으로부터 폴리곤을 그리는 일 등은 말이 되지 않습니다.

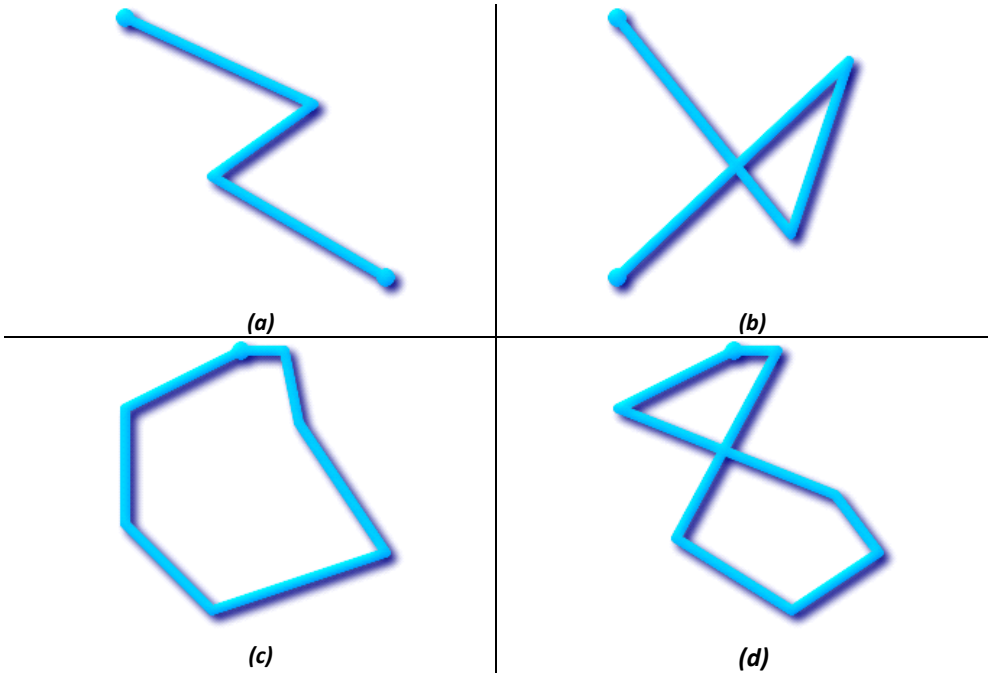
OGC 사양서에 따르면, 단순(simple) 도형은 변칙적인 포인트, 즉 자체적으로 교차하거나 접촉하는 기하학적 포인트가 하나도 없으며, 기본적으로 0 또는 1 차원 도형([MULTI]POINT, [MULTI]LINESTRING)을 참조하는 도형을 말합니다. 반면, 도형의 유효성은 기본적으로 2 차원 도형([MULTI]POLYGON))을 참조하며

유효한 폴리곤을 특징짓는 일련의 진술(assertion)을 정의합니다. 각 도형 클래스의 설명에는 도형의 단순성과 유효성을 더 상세히 알려주는 특정한 조건들이 포함됩니다.

POINT 란 0 차원 도형 객체로서 상속적으로 *단순형* 입니다.

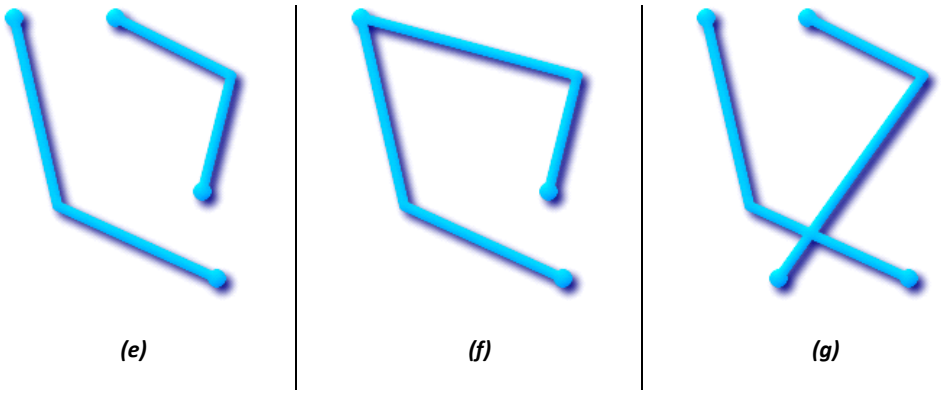
MULTIPOINT 는 어떤 두 좌표(POINT)도 동일하지 않은 (동일한 좌표를 공유하지 않는) *단순형* 입니다.

LINESTRING 은 동일한 POINT 를 두 번 통과하지 않는 경우 (종단점은 예외입니다. 이럴 경우 선형 고리라 불리며, 폐쇄 도형으로 간주됩니다) *단순형* 입니다.



(a) 와 **(c)** 는 단순 LINESTRING이지만, **(b)** 와 **(d)** 는 아닙니다.

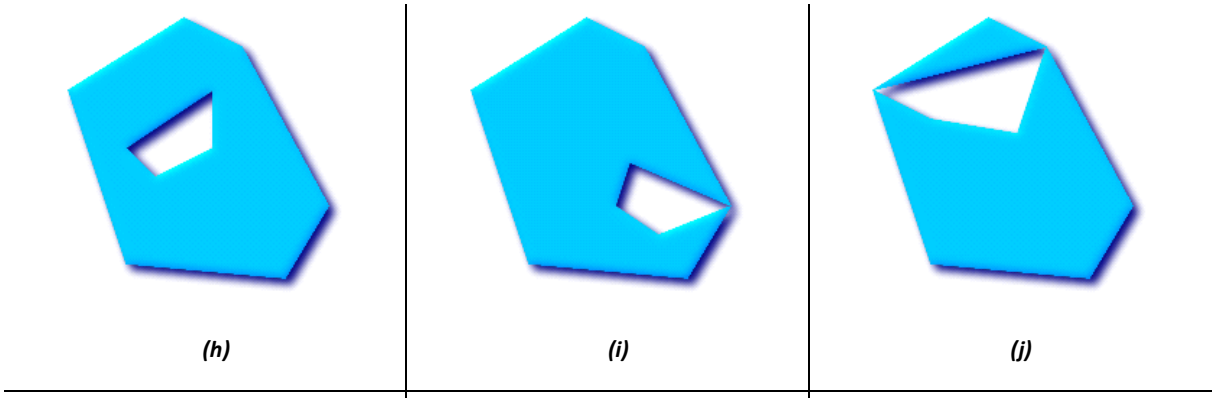
MULTILINESTRING 은 모든 구성 요소가 단순형이며 어떤 두 요소가 해당 두 요소의 경계상에 있는 POINT에서 교차하는 경우에만 단순형 입니다.



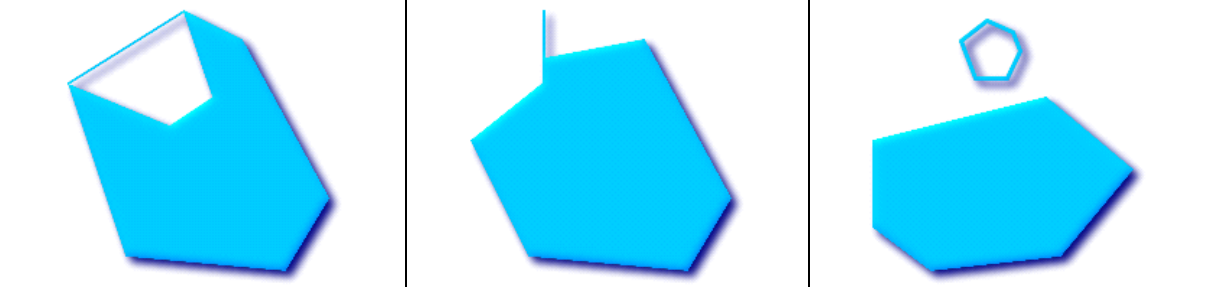
(e) 와 **(f)** 는 단순 MULTILINESTRING이지만, **(g)** 는 아닙니다.

정의에 따르면, POLYGON 은 어떤 경우라도 단순형 입니다. 경계선(boundary) 내부의 (외곽 고리와 내곽 고리로 이루어진) 어떤 두 고리도 교차하지 않을 경우 유효 합니다. POLYGON 의 경계선은 어떤 POINT 에서 교차할 수도 있지만, 접선(즉 라인으로는 겹치지 않는)일 경우에만 가능합니다. POLYGON 대부분은

커트라인(cut line) 또는 스파이크(spike)를 가지고 있지 않으며, 외곽선이 내곽선을 완전히 감싸 담고 있어야 합니다.



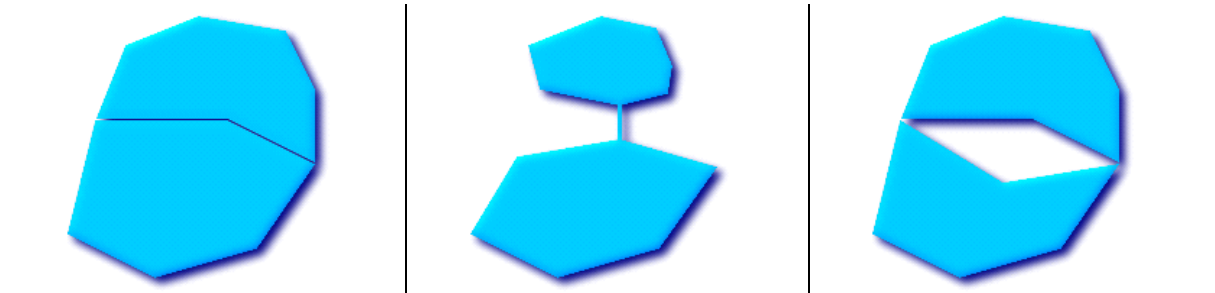
(h) *(i)* *(j)*



(k) *(l)* *(m)*

(h) 와 **(i)** 는 유효한 POLYGON이며 **(j-m)** 은 단일 POLYGON이라고 할 수 없지만 **(j)**와**(m)**은 유효한 MULTIPOLYGON이라고 할 수 있습니다.

MULTIPOLYGON 은 모든 구성 요소가 유효하며 어떤 두 요소의 내부도 교차하지 않을 경우에만 유효합니다. 어느 두 요소의 경계선은 접촉할 수도 있지만, 유한한 개수의 POINT 들에서만 접할 수 있습니다.



(n) *(o)* *(p)*

(n)과 **(o)**은 유효한 MULTIPOLYGON이 아닙니다. 하지만, **(p)**는 유효합니다.

GEOS 라이브러리가 실행하는 함수 대부분은 사용자 도형이 OpenGIS 단순 도형 사양서의 정의대로 유효하다는 가정에 의존합니다. 도형의 단순성 및 유효성을 확인하려면 ST_IsSimple() 및 ST_IsValid() 함수를 이용할 수 있습니다.

```
-- 일반적으로, 선형 피처의 유효성을 확인하는 작업은
-- 언제나 참을 반환하기 때문에 할 필요가 없습니다.
-- 그러니 이 예시에서는, 라인스트링의 두렷이 구분되는 꼭짓점이 2 개 미만일 경우
-- 거짓을 반환하여 PostGIS 가 OGC IsValid 의 정의를 확장합니다.
gisdb=# SELECT
   ST_IsValid('LINESTRING(0 0, 1 1)'),
   ST_IsValid('LINESTRING(0 0, 0 0, 0 0)');
```

```
 st_isvalid | st_isvalid
------------+-----------
     t     |     f
```

PostGIS는 기본적으로 도형 입력시 유효성 검사를 하지 않습니다. 복잡 도형, 특히 폴리곤의 경우 유효성 테스트에 많은 CPU 시간이 들기 때문입니다. 사용자가 자신의 데이터소스를 신용하지 못 할 경우, 사용자 테이블에 다음과 같은 검사 제약조건을 걸어 직접 검사해볼 수 있습니다.

```
ALTER TABLE mytable
  ADD CONSTRAINT geometry_valid_check
       CHECK (ST_IsValid(the_geom));
```

유효한 입력 도형과 함께 PostGIS 함수를 호출했는데 "GEOS Intersection() threw an error!" 또는 "JTS Intersection() threw an error!" 같은 이상한 오류 메시지를 받았다면, 사용자가 PostGIS 또는 PostGIS가 사용하는 라이브러리 중 하나의 오류를 찾았을 가능성이 높습니다. PostGIS 개발자에게 연락해주십시오. PostGIS 함수가 유효한 입력에 대해 유효하지 않은 도형을 반환하는 경우도 마찬가지입니다.



OGC를 엄격히 준수하는 도형은 Z 또는 M 값을 가질 수 없습니다. ST_IsValid() 함수는 고차원 도형을 유효하지 않다고 판단하지 않을 것입니다! AddGeometryColumn() 함수를 호출하면 도형 차원을 검사하는 제약조건을 추가할 것이므로, "2"라고 설정해주면 충분합니다.

4.3.6. 차원적으로 확장된 9 교차 모델(DE-9IM)

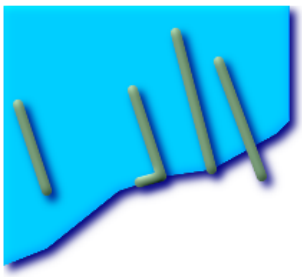
때때로 전형적인 공간 술어(ST_Contains, ST_Crosses, ST_Intersects, ST_Touches, ...) 그 자체로는 원하는 공간 필터를 제대로 구현하는 데 부족한 경우가 있습니다.



예를 들어, 도로망을 표현하는 선형 데이터셋을 생각해보십시오. GIS 분석가가 포인트가 아니라 라인 상에서 겹쳐지는, 아마도 사업 규칙을 무시하는, 모든 도로 구간을 식별해야 할 수도 있습니다. 이런 경우 ST_Crosses 함수는 선형 피처에 대해 오직 포인트 상에서 교차할 경우에만 참 을 반환하기 때문에, 필요한 공간 필터를 제대로 제공하지 못 합니다.

먼저 도로 구간 한 쌍이 공간적으로 교차하는(ST_Intersects) 실제 교차점(ST_Intersection)을 구하는 작업을 수행한 다음, ([MULTI]POINT, [MULTI]LINESTRING 등의 GEOMETRYCOLLECTION 을 반환하는 경우를 적절히 대처하며) 해당 교차점의 ST_GeometryType 을 'LINESTRING'과 비교하는 두 단계의 해법을 이용할 수도 있습니다.

물론 더 우아하고 빠른 해법을 원할 수도 있지요.



두 번째 (이론적인) 예시로, GIS 분석가가 호수의 경계선상을 침범하며 한쪽 끝만 호수 기슭에 올라가 있는 선창 또는 부두의 위치를 찾으려 할 수도 있습니다. 다시 말해, 선창이 호수의 경계선상을 침범하며 호수 내부에 들어가 있지만 완전히 호수 안에 들어가 있지는 않고, 선창의 종단점 두 개 모두 완전히 호수 경계선 내부 및 경계선상에 있는 경우입니다. 이 분석가는 찾고 있는 피처를 분리해내기 위해 공간 술어들의 조합을 이용해야 할 수도 있습니다.

- ST_Contains(lake, wharf) = TRUE
- ST_ContainsProperly(lake, wharf) = FALSE
- ST_GeometryType(ST_Intersection(wharf, lake)) = 'LINESTRING'
- ST_NumGeometries(ST_Multi(ST_Intersection(ST_Boundary(wharf), ST_Boundary(lake)))) = 1
  ... (말할 필요도 없지만, 굉장히 복잡해질 수도 있습니다.)

따라서 차원 확장 9 교차 모델(Dimensionally Extended 9 Intersection Model), 줄여서 DE-9IM이 등장합니다.

4.3.6.1. 이론

SQL 용 OpenGIS 단순 피처 실행 사양서 에 따르면, "두 도형을 비교하는 기본 접근법은 두 도형의 내부, 경계, 외부 사이의 교차점을 쌍으로 테스트한 다음, 해당 결과의 '교차점' 매트릭스를 바탕으로 두 도형 사이의 관계를 분류하는 방법"입니다.

경계(Boundary)

도형의 경계는 한 단계 아래 차원의 도형 집합입니다. POINT 의 경우, 0 차원이므로 경계는 공집합입니다. LINESTRING 의 경계는 두 종단점입니다. POLYGON 의 경우, 경계는 외곽 및 내곽 고리를 이루는 선형 피처입니다.

내부(Interior)

도형의 내부란 경계를 제거했을 때 남는 해당 도형의 포인트들입니다. POINT 의 경우, 내부는 POINT 자체입니다. LINESTRING 의 내부는 두 종단점 사이에 있는 실제 포인트들의 집합입니다. POLYGON 의 경우, 내부는 폴리곤 안의 면적을 가진 면입니다.

외부(Exterior)

도형의 외부란 해당 도형의 내부 또는 경계가 아닌 세계, 면적을 가진 면입니다.

*a*라는 도형이 있을 때 *I(a), B(a), E(a)*이 각각 도형 *a*의 *내부*, *경계*, *외부*를 뜻하며, 해당 매트릭스의 수학적 표현식은 다음과 같습니다:

	내부	경계	외부
내부	*dim(I(a) ∩ I(b))*	*dim(I(a) ∩ B(b))*	*dim(I(a) ∩ E(b))*
경계	*dim(B(a) ∩ I(b))*	*dim(B(a) ∩ B(b))*	*dim(B(a) ∩ E(b))*
외부	*dim(E(a) ∩ I(b))*	*dim(E(a) ∩ B(b))*	*dim(E(a) ∩ E(b))*

이때 dim(a) 는 ST_Dimension 이 지정한 도형 a 의 차원이지만 그 정의역(domain)은 {0,1,2,T,F,*} 입니다.

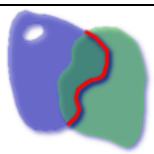
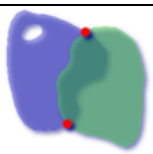
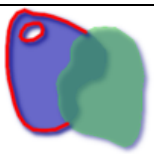
- 0 => 포인트

- 1 => 라인
- 2 => 면
- T => {0,1,2}
- F => 공집합
- * => 상관없음

서로 겹치는 두 폴리곤 도형의 경우를 시각적으로 나타내면 다음과 같습니다:





	내부	***경계***	***외부***
내부	 *dim(...) = **2***	 *dim(...) = **1***	 *dim(...) = **2***
경계	 *dim(...) = **1***	 *dim(...) = **0***	 *dim(...) = **1***
외부	 *dim(...) = **2***	 *dim(...) = **1***	 *dim(...) = **2***

왼쪽에서 오른쪽으로 그리고 위에서 아래로 읽을 경우, 차원 매트릭스는 '212101212' 와 같이 표현됩니다.

따라서 첫 번째 예시에서 나온, 라인 상에서 겹치는 두 라인을 표현하는 관계 매트릭스는 '1*1***1**' 가 될 것입니다.

```
-- 라인 상에 겹치는 도로 구간을 식별하기
SELECT a.id
FROM roads a, roads b
WHERE a.id != b.id
AND a.geom && b.geom
AND ST_Relate(a.geom, b.geom, '1*1***1**');
```

두 번째 예시에서 나온, 호수의 경계선에 일부분을 걸치는 선창을 표현하는 관계 매트릭스는 '102101FF2'가 될 것입니다.

```
-- 호수의 경계선에 일부분을 걸치는 선창을 식별하기
SELECT a.lake_id, b.wharf_id
FROM lakes a, wharfs b
WHERE a.geom && b.geom
AND ST_Relate(a.geom, b.geom, '102101FF2');
```

더 상세한 정보 또는 읽을 거리가 필요하다면 다음을 참조하십시오:

- SQL 용 OpenGIS 단순 피처 실행 사양서 (1.1 버전, 2.1.13.2 단원)
- 차원 확장 9 교차 모델(DE-9IM)
- GeoTools: 포인트 집합 이론 및 DE-9IM 매트릭스
- GIS 백과사전(Encyclopedia of GIS) - 후이 슝(Hui Xiong) 저(著)

4.4. GIS (벡터) 데이터 로드

공간 테이블 생성을 끝냈다면, 사용자가 데이터베이스에 GIS 데이터를 업로드할 준비가 된 것입니다. 현재, 형식화된 SQL 구문을 사용하거나 shapefile 로더/덤퍼를 사용하는 두 가지 방법으로 PostGIS/PostgreSQL 데이터베이스에 데이터를 입력할 수 있습니다.

4.4.1. SQL 을 이용해서 데이터를 로드하기

사용자 데이터를 텍스트 표현식으로 변환할 수 있다면, PostGIS에 사용자 데이터를 입력하는 가장 쉬운 방법은 형식화된(formatted) SQL을 이용하는 것입니다. Oracle이나 다른 SQL 데이터베이스와 마찬가지로, SQL 터미널 모니터에 SQL "INSERT" 선언문으로 가득 찬 대용량 텍스트 파일을 송신하는(piping) 방법으로 데이터를 일괄 로드시킬 수 있습니다.

데이터 업로드 파일(예를 들어 roads.sql)은 다음처럼 보일 것입니다:

```
BEGIN;
INSERT INTO roads (road_id, roads_geom, road_name)
  VALUES  (1,ST_GeomFromText('LINESTRING(191232  243118,191108  243242)',-
1),'Jeff Rd');
INSERT INTO roads (road_id, roads_geom, road_name)
  VALUES  (2,ST_GeomFromText('LINESTRING(189141  244158,189265  244817)',-
1),'Geordie Rd');
INSERT INTO roads (road_id, roads_geom, road_name)
  VALUES  (3,ST_GeomFromText('LINESTRING(192783  228138,192612  229814)',-
1),'Paul St');
INSERT INTO roads (road_id, roads_geom, road_name)
  VALUES  (4,ST_GeomFromText('LINESTRING(189412  252431,189631  259122)',-
1),'Graeme Ave');
INSERT INTO roads (road_id, roads_geom, road_name)
  VALUES  (5,ST_GeomFromText('LINESTRING(190131  224148,190871  228134)',-
1),'Phil Tce');
INSERT INTO roads (road_id, roads_geom, road_name)
  VALUES  (6,ST_GeomFromText('LINESTRING(198231  263418,198213  268322)',-
1),'Dave Cres');
COMMIT;
```

"psql" SQL 터미널 모니터를 이용해서 PostgreSQL로 데이터 파일을 매우 쉽게 송신할 수 있습니다.

```
psql -d [database] -f roads.sql
```

4.4.2. shp2pgsql: ESRI shapefile 로더 이용하기

shp2pgsql 데이터 로더는 ESRI shapefile을, 도형 형식이든 지리형 형식이든, PostGIS/PostgreSQL 데이터베이스로 삽입하기에 적합한 SQL로 변환합니다. 이 로더에는 명령행(command line) 플래그로 구별되는 몇 가지 실행 모드가 존재합니다.

shp2pgsql 명령행 로더 외에, 사용자가 PostGIS를 처음 접하는 경우 스크립트를 사용하지 않고 단 한 번 로드하는 데 더 쉽게 사용할 수 있을 뿐만 아니라 명령행 로더가 가진 대부분의 옵션도 가지고 있는 shp2pgsql-gui 그래픽 인터페이스도 있습니다. shp2pgsql-gui를 pgAdmin III의 플러그인으로 설정할 수도 있습니다.

cladlp -- 이들은 상호배타적인 옵션들입니다:

-c 새 테이블을 생성한 다음 shapefile의 데이터로 해당 테이블을 채웁니다. 이것이 기본 모드입니다.

-a 기존 데이터베이스 테이블에 shapefile의 데이터를 추가합니다. 이 옵션을 이용해서 복수의 파일을 로드하려면, 파일들이 동일한 속성 및 동일한 데이터 유형을 담고 있어야 한다는 점을 주의하십시오.

-d 기존 데이터베이스 테이블을 삭제(drop)한 다음 shapefile의 데이터를 가진 새 테이블을 생성합니다.

-p 테이블을 생성하는 SQL 코드만 생성하고, 어떤 실제 데이터도 추가하지 않습니다. 테이블 생성과 데이터 로드 단계를 완전히 분리해야 할 경우 사용할 수 있습니다.

-? 도움말 화면을 표출합니다.

-D 산출물 데이터의 형식으로 PostgreSQL "덤프(dump)" 형식을 사용합니다. 이 옵션은 -a, -c 및 -d와 함께 사용할 수 있습니다. 이 덤프 형식은 기본 "삽입" SQL 형식보다 훨씬 빨리 로드할 수 있습니다. 대용량 데이터셋의 경우 이 옵션을 사용하십시오.

-s [<FROM_SRID%gt;:]<SRID> 도형 테이블을 생성하고 지정된 SRID로 채웁니다. 입력 shapefile이 주어진 FROM_SRID를 쓰도록 설정하는 옵션도 있습니다. 이런 경우 도형이 목표 SRID로 재투영될 것입니다. FROM_SRID는 -D 옵션과 함께 사용될 수 없습니다.

-k 식별자의 대소문자(열, 스키마 및 속성)를 유지합니다. shapefile 안의 속성은 모두 대문자라는 점을 주의하십시오.

-i DBF 헤더 서명이 64비트 bigint 형식을 보장하더라도, 모든 정수를 표준 32비트 정수로 강제 변환하고 64비트 bigint 형식을 생성하지 않습니다.

-I 도형 열에 GiST 인덱스를 생성합니다.

-m "-m 파일명" 형식으로 (긴) 열 명칭과 10문자 DBF 열 명칭을 매핑하는 목록을 담은 파일을 지정합니다. 이 파일의 내용은 공백으로 구분된 두 명칭으로 이루어진 하나 이상의 행으로, 행 맨 앞과 맨 뒤에는 공백이 없어야 합니다. 다음은 그 예시입니다:

```
COLUMNNAME DBFFIELD1
AVERYLONGCOLUMNNAME DBFFIELD2
```

-S 다중(multi) 도형 대신 단순 도형을 생성합니다. 이 옵션은 모든 도형이 실제로 단일형(예: 단일 외곽선을 가진 다중 폴리곤 또는 단일 꼭짓점을 가진 다중 포인트)일 경우에만 작동합니다.

-t <dimensionality> 산출 도형이 지정된 차원수를 가지도록 강제합니다. 차원수를 지시하는 데 다음 스트링을 사용하십시오: 2D, 3DZ, 3DM, 4D

입력물이 지정된 차원수보다 낮은 차원일 경우, 출력물의 해당 차원은 0으로 채워질 것입니다. 입력물이 지정된 차원수보다 높은 차원일 경우, 필요 없는 차원은 제거될 것입니다.

-w WKB 대신 WKT 형식으로 출력합니다. 정확도가 부족하기 때문에 좌표가 이동될 가능성이 있다는 점을 주의하십시오.

-e 각 선언문을 상호처리를 이용하지 않고 자체적으로 실행합니다. 오류를 생성하는 몇몇 망가진 도형이 있을 경우 이 옵션을 사용하면 괜찮은 데이터 대다수를 로드할 수 있습니다. "덤프" 형식은 항상 상호처리를 이용하기 때문에 -D 플래그와 함께 사용할 수 없다는 점을 주의하십시오.

-W <encoding> 입력 데이터(DBF 파일)의 인코딩을 지정합니다. 이 옵션을 사용하면, DBF의 모든 속성을 지정된 인코딩에서 UTF8로 변환합니다. 그 결과로 생성되는 SQL 출력물은 SET CLIENT_ENCODING to UTF8 명령어를 담게 되어, 백엔드에서 UTF8을 데이터베이스 내부에서 이용하도록 설정된 어떤 인코딩으로든 재변환할 수 있습니다.

-N <policy> NULL 도형 처리 방침 -- insert*(상관없이 삽입), skip(건너뛰기), abort(중단)

-n DBF 파일만 임포트합니다. 사용자 데이터에 대응하는 shapefile이 없다면, 자동적으로 이 모드로 전환하여 DBF만 로드할 것입니다. 따라서 전체 shapefile 집합을 가지고 있지만 도형을 빼고 속성 데이터만 필요한 경우에만 이 플래그를 설정해야 합니다.

-G (경도/위도가 필요한) 도형 대신 WGS84 경위도(SRID=4326)를 쓰는 지리형을 이용합니다.

-T <tablespace> 새 테이블을 위한 테이블스페이스를 지정합니다. -X 파라미터가 함께 쓰인 경우가 아니라면 여전히 인덱스가 기본 테이블스페이스를 이용할 것입니다. PostgreSQL 문서는 사용자 지정 테이블스페이스가 필요한 경우를 잘 설명하고 있습니다.

-X <tablespace> 새 테이블의 인덱스를 위한 테이블스페이스를 지정합니다. 이 옵션은 기본 키(primary key) 인덱스에 적용되며, -I 플래그를 함께 사용하는 경우 GiST 공간 인덱스에도 적용됩니다.

다음은 로더를 이용해서 입력 파일을 생성하고 업로드하는 세션의 예시입니다:

```
# shp2pgsql -c -D -s 4269 -i -I shaperoads.shp myschema.roadstable > roads.sql
# psql -d roadsdb -f roads.sql
```

UNIX 파이프(pipe)를 이용하면 모든 변환 및 업로드 작업을 한 번에 끝낼 수 있습니다:

```
# shp2pgsql shaperoads.shp myschema.roadstable | psql -d roadsdb
```

4.5. GIS 데이터 가져오기

SQL이나 shapefile 로더/덤퍼를 이용해서 데이터베이스로부터 데이터를 추출할 수 있습니다. SQL 단원에서 공간 테이블에 대한 비교 및 쿼리를 할 수 있는 몇몇 연산자에 대해 논의할 것입니다.

4.5.1. SQL 을 이용해 데이터 가져오기

데이터베이스로부터 데이터를 추출하는 가장 간단한 방법은 SQL 선별(select) 쿼리로 반환될 레코드 및 열의 개수를 줄인 다음 해당 결과 열을 파싱 가능한 텍스트 파일로 덤프받는 것입니다:

```
db=# SELECT road_id, ST_AsText(road_geom) AS geom, road_name FROM roads;

road_id | geom                                  | road_name
--------+---------------------------------------+-----------
        1 | LINESTRING(191232 243118,191108 243242) | Jeff Rd
        2 | LINESTRING(189141 244158,189265 244817) | Geordie Rd
        3 | LINESTRING(192783 228138,192612 229814) | Paul St
        4 | LINESTRING(189412 252431,189631 259122) | Graeme Ave
        5 | LINESTRING(190131 224148,190871 228134) | Phil Tce
        6 | LINESTRING(198231 263418,198213 268322) | Dave Cres
        7 | LINESTRING(218421 284121,224123 241231) | Chris Way
(6 rows)
```

하지만, 반환되는 필드의 개수를 줄이기 위해 어떤 종류의 제약이 필요할 때가 있을 것입니다. 속성 기반 제약의 경우, 일반적인 비공간 테이블의 경우와 동일한 SQL 문법을 쓰면 됩니다. 공간 제약의 경우, 다음 유용한 연산자들을 쓸 수 있습니다.

&& 이 연산자는 한 도형의 경계 상자와 다른 도형의 경계 상자가 교차하는지를 알려줍니다.

ST_OrderingEquals 이 연산자는 두 도형이 기하학적으로 동일한지를 테스트합니다.예를 들어, 'POLYGON((0 0,1 1,1 0,0 0))'과 'POLYGON((0 0,1 1,1 0,0 0))'이 동일한지를 말입니다(동일합니다).

= 이 연산자는 두 도형이 기하학적으로 동일한지를 테스트합니다.예를 들어, 'POLYGON((0 0,1 1,1 0,0 0))'과 'POLYGON((0 0,1 1,1 0,0 0))'이 동일한지를 말입니다(동일합니다).

다음으로, 이 연산자들을 쿼리에 쓸 수 있습니다. SQL 명령행에 도형과 경계 상자를 지정할 때, "ST_GeomFromText()" 함수를 이용해서 스트링 표현식을 도형으로 정확하게 변환시켜야 합니다. 해당 데이터와 일치하는 가공의 공간 참조 시스템은 312입니다. 다음은 그 예시입니다:

```
SELECT road_id, road_name
  FROM roads
  WHERE  ST_OrderingEquals(roads_geom ,  ST_GeomFromText('LINESTRING(191232
243118,191108 243242)',312) ) ;
```

이 쿼리는 해당 값과 동일한 도형을 담고 있는 "ROADS_GEOM" 테이블로부터 단일 레코드를 반환할 것입니다.

"&&" 연산자 사용시, 비교 피처로 BOX3D 또는 도형을 지정할 수 있습니다. 하지만 도형을 지정했을 경우, 비교 작업에 해당 경계 상자가 사용될 것입니다.

```
SELECT road_id, road_name
FROM roads
WHERE roads_geom && ST_GeomFromText('POLYGON((...))',312);
```

이 쿼리는 비교 목적으로 폴리곤의 경계 상자를 이용할 것입니다.

가장 흔한 공간 쿼리는 아마도 데이터 브라우저 또는 웹 매퍼 같은 클라이언트 소프트웨어가 화면 표출을 위해 "맵 프레임(map frame)" 용량에 해당하는 데이터를 가져오기 위해 사용하는 "프레임 기반(frame-based)" 쿼리일 것입니다.

```
SELECT ST_AsText(roads_geom) AS geom
FROM roads
WHERE
  roads_geom && ST_MakeEnvelope(191232, 243117,191232, 243119,312);
```

화면에 해당하는 데이터의 투영체를 지정하는 데 SRID 312를 썼다는 사실에 주의하십시오.

4.5.2. 덤퍼 이용하기

pgsql2shp 테이블 덤퍼는 데이터베이스에 직접 연결되어 (아마도 쿼리가 정의했을) 테이블을 shapefile로 변환합니다. 기본 문법은 다음과 같습니다:

```
pgsql2shp [<options>] <database> [<schema>.]<table>
```

```
pgsql2shp [<options>] <database> <query>
```

다음과 같은 명령행 옵션이 있습니다:

-f <filename> 특정 파일명으로 출력물을 작성합니다.

-h <host> 연결할 데이터베이스 호스트를 설정합니다.

-p <port> 데이터베이스 호스트 연결시 사용할 포트를 설정합니다.

-P <password> 데이터베이스 연결에 사용할 비밀번호를 설정합니다.

-u <user> 데이터베이스 연결에 사용할 사용자명을 설정합니다.

-g <geometry column> 복수의 도형 열을 가진 테이블일 경우, shapefile 작성에 이용될 도형 열을 설정합니다.

-b 바이너리 커서를 사용하도록 설정합니다. 이 옵션을 쓰면 실행 속도가 빨라지지만, 테이블 안에 있는 비(非) 도형 속성 가운데 하나라도 텍스트로 작성할 캐스트(cast)가 부족할 경우 실행되지 않을 것입니다.

-r 로(raw) 모드입니다. gid 필드를 삭제하거나, 열 명칭을 제외하지 않습니다.

-d 하위 호환성에 대해: 구 버전(1.0.0 이전) PostGIS 데이터베이스로부터 덤핑받을 때 3차원 shapefile을 작성하십시오(이런 경우 2차원 shapefile 작성이 기본값입니다). PostGIS 1.0.0 버전부터 차원수를 완전히 인코딩합니다.

-m filename 식별자를 10문자 명칭으로 다시 매핑(remap)합니다. 해당 파일의 내용은 공백으로 구분된 두 심볼로 이루어진 복수의 행으로, 행 맨 앞과 맨 뒤에는 공백이 없어야 합니다. VERYLONGSYMBOL SHORTONE ANOTHERVERYLONGSYMBOL SHORTER 등과 같은 예가 있습니다.

4.6. 인덱스 빌드 작업

인덱스 덕분에 공간 데이터베이스가 대용량 데이터셋을 사용할 수 있습니다. 인덱스 작업을 하지 않으면, 어떤 피처를 검색하든 데이터베이스 안의 모든 레코드를 "순차 스캔"해야 할 것입니다. 인덱스 작업은 데이터를 특정 레코드를 찾기 위해 빠르게 훑어갈 수 있는 검색 트리로 조직해서 검색 속도를 향상시킵니다. PostgreSQL 는 기본적으로 B-Tree, R-Tree, GiST 세 종류의 인덱스를 지원합니다.

- B-Tree 인덱스는 하나의 축을 따라 정렬할 수 있는 데이터에 이용됩니다. 숫자, 글자, 날짜 등이 그 예입니다. GIS 데이터는 하나의 축을 따라 논리적으로 정렬될 수 없기 때문에 ((0,0), (0,1), (1,0) 가운데 어느 것이 더 큰 값인가요?) 이 문서에서 B-Tree 인덱스는 쓸모가 없습니다.
- R-Tree 인덱스는 데이터를 장방형(rectangle), 하위장방형(sub-rectangle), 하하위장방형(sub-sub rectangle) 등으로 분해합니다. 몇몇 공간 데이터베이스는 GIS 데이터 인덱스 작업을 위해 R-Tree 인덱스를 이용하지만, PostgreSQL에서의 R-Tree 작업은 GiST 작업만큼 강력하지는 않습니다.
- GiST(Generalized Search Tree) 인덱스는 데이터를 "한 쪽에 있는 것", "겹치는 것", "내부에 있는 것"으로 분해하며 GIS 데이터를 포함한 광범위한 데이터 유형에 쓰일 수 있습니다. PostGIS는 GiST를 써서 GIS 데이터에 인덱스 작업을 한 다음, 해당 데이터에 다시 작업된 R-Tree 인덱스를 이용합니다.

4.6.1. GiST 인덱스

GiST는 "일반화된 검색 트리"의 줄임말로, 인덱스 작업의 포괄적인 형태입니다. GIS 인덱스 작업 외에도, 일반 B-Tree 인덱스 작업으로는 쓸 수 없는 온갖 종류의 비정규 데이터 구조(정수 배열, 분광 데이터 등등)에 대한 검색 속도를 향상시키는 데 GiST를 이용합니다.

GIS 데이터 테이블이 수천 행을 넘게 되면, 데이터 공간 검색의 속도를 향상시키기 위해 인덱스를 빌드하고 싶게 될 것입니다(사용자의 모든 검색이 속성에 기반하는 경우가 아니라면 말입니다. 그런 경우, 속성 필드에 대해 일반 인덱스를 빌드하면 됩니다).

"도형" 열에 대해 GiST 인덱스를 빌드하는 데 필요한 문법은 다음과 같습니다:

```
CREATE INDEX [indexname] ON [tablename] USING GIST ( [geometryfield] );
```

이 문법은 항상 2D 인덱스를 빌드할 것입니다. 해당 도형 유형에 PostGIS 2.0 이상 버전이 지원하는 n차원 인덱스를 얻으려면, 다음 문법으로 생성할 수 있습니다:

```
CREATE INDEX [indexname] ON [tablename] USING GIST ([geometryfield]
gist_geometry_ops_nd);
```

공간 인덱스 빌드 작업은 계산적으로 집중적인 작업입니다. 300MHz 솔라리스 워크스테이션 상에서, 약 1백만 행을 가진 테이블에 대한 GiST 인덱스 빌드 작업에 약 1시간이 걸렸습니다. 인덱스 빌드 후 PostgreSQL가 테이블 통계를 수집하도록 하는 명령을 해야 합니다. 쿼리 설계를 최적화하는 데 이 통계를 이용하기 때문에, 다음 명령이 중요합니다:

```
VACUUM ANALYZE [table_name] [(column_name)];
-- This is only needed for PostgreSQL 7.4 installations and below
SELECT UPDATE_GEOMETRY_STATS([table_name], [column_name]);
```

PostgreSQL에서, GiST 인덱스에는 R-Tree 인덱스에 비해 두 가지 장점이 있습니다. 첫째, GiST 인덱스는 "null값을 처리할 수(null safe)" 있습니다. 즉 null값을 포함하는 열에 대해 인덱스 작업을 할 수 있다는 뜻입니다. 둘째, GiST 인덱스는 PostgreSQL 8K 페이지 크기보다 더 큰 GIS 객체를 다룰 때 중요한 "손실(lossiness)"이라는 개념을 지원합니다. 손실 개념을 통해 PostgreSQL은 인덱스에 객체의 "중요한" 부분만을 저장할 수 있습니다. GIS 객체의 경우, 경계 상자만 저장합니다. 8K보다 더 큰 GIS 객체가 있을 경우, R-Tree 인덱스는 인덱스 빌드 과정 중 오류를 일으킬 것입니다.

4.6.2. Brin 인덱스

BRIN 이란 "Block Range Index"의 약자로 PostgreSQL 9.5 버전에 도입된 인덱스 작업의 일반 형태입니다. BRIN 은 손실이 있는 종류의 인덱스로, 읽기 및 쓰기 성능을 위한 절충안을 제공하는 데 주로 쓰입니다. BRIN 인덱스의 기본 목표는 열의 일부가 테이블 내부에서 자신의 물리적 위치와 어떤 태생적인 상관관계를 가지는 대용량 테이블을 처리하는 것입니다. BRIN 은 GIS 인덱스일 뿐만이 아니라, 여러 종류의 정규 또는 비정규 데이터 구조(정수, 배열 등)에 대한 검색 속도를 향상시키는 데 쓰입니다.

GIS 데이터 테이블이 몇 천 행을 초과한다면, 사용자는 데이터의 검색 속도를 향상시키기 위해 인덱스를 작성하고자 할 것입니다(모든 검색이 속성 기반일 경우를 제외하면 말이죠. 이 경우 사용자는 속성 항목에 대해 정규 인덱스를 작성하고자 할 것입니다). 인덱스 용량이 데이터베이스가 사용할 수 있는 RAM 용량을 초과하지 않으며, 사용자가 쓰기 작업량으로 인한 불이익 및 저장 용량을 감당할 수 있을 경우, GiST 인덱스도 정말 쓸 만합니다. 그렇지 못 할 경우, BRIN 인덱스를 대신 고려해봐도 좋습니다.

BRIN 인덱스의 개념은 테이블 블록 집합 내 모든 행이 담고 있는 모든 도형들을 감싸는, 범위(range)라고 하는 경계 상자만 저장한다는 것입니다. 당연하지만, 각 블록 범위에 대해 산출된 경계 상자들이 서로 겹치지 않도록 데이터가 물리적으로 정렬되어 있는 경우에만 이 인덱스 작업 방식을 쓸 수 있습니다. 이렇게 생성된 인덱스의 용량은 매우 작지만, 많은 경우 GiST 인덱스보다 덜 효율적일 것입니다.

BRIN 인덱스 작성은 GiST 인덱스 작성보다 품이 훨씬 덜 듭니다. BRIN 인덱스 작성에 걸리는 시간이 GiST 인덱스 작성 때보다 10 분의 1 미만인 경우가 대부분입니다. BRIN 인덱스는 테이블 블록 한 개에서 여러 개에 대해 경계 상자 한 개만 저장하므로, 이런 종류의 인덱스가 소모하는 디스크 용량은 1,000 분의 1 미만인 경우가 많습니다.

범위(range)로 요약할 블록의 개수를 선택할 수 있습니다. 이 개수를 줄인다면, 인덱스의 용량은 커지겠지만 검색 결과의 질을 향상시켜줄 것입니다.

"도형" 열에 대해 BRIN 인덱스를 작성하기 위한 구문은 다음과 같습니다:

```
CREATE INDEX [indexname] ON [tablename] USING BRIN ( [geometryfield] );
```

이 구문은 항상 2 차원 인덱스를 작성할 것입니다. PostGIS 2.0 이상 버전에서 도형 유형에 대해 3 차원 인덱스를 작성하려면, 다음 구문을 통해 생성할 수 있습니다:

```
CREATE INDEX [indexname] ON [tablename] USING BRIN ( [geometryfield] ) WITH
(pages_per_range = [number]);
```

또한 BRIN 인덱스는 "지리형" 데이터유형을 지원합니다. "지리형" 열에 대해 BRIN 인덱스를 작성하는 구문은 다음과 같습니다:

```
CREATE INDEX [indexname] ON [tablename] USING BRIN ( [geographyfield] );
```

이 구문은 항상 회전 타원체 상의 지리형 객체에 대한 2 차원 인덱스를 작성할 것입니다.

현재로서는 "포함(inclusion) 지원"만 고려하고 있습니다. 즉 2 차원의 경우 ("도형" 및 "지리형" 모두에 대해) &&, ~ 및 @ 연산자만 사용할 수 있고, 3 차원 도형에 대해서는 &&& 연산자만 사용할 수 있다는 뜻입니다. 현 시점에서는 kNN 검색을 지원하지 않습니다.

```
VACUUM ANALYZE [table_name] [(column_name)];
-- This is only needed for PostgreSQL 7.4 installations and below
SELECT UPDATE_GEOMETRY_STATS([table_name], [column_name]);
```

4.6.3. 인덱스 활용

일반적인 경우라면 인덱스는 눈에 보이지 않게 데이터 접속 속도를 향상시킵니다. 인덱스가 빌드된 후, 쿼리 설계자는 쿼리 설계의 속도를 향상기키는 데 언제 인덱스 정보를 사용할 것인지 투명하게 결정합니다. 안타깝게도 PostgreSQL 쿼리 설계자가 GiST 인덱스의 사용을 제대로 최적화시키지 못 하기 때문에, 종종 공간 인덱스를 활용해야 할 검색이 대신 기본값인 전체 데이터의 순차 스캔을 이용할 때가 있습니다.

사용자의 공간 인덱스가 (또는 사용자의 속성 인덱스가) 활용되지 않고 있다는 사실을 알게 되었다면, 몇 가지 해결 방법이 있습니다:

- 첫 번째, 테이블 안에 있는 값들의 개수 및 분포에 대한 통계를 제대로 수집했는지 확인하십시오. 인덱스 활용에 대한 결정을 내릴 수 있게 하려면 쿼리 설계자에 더 나은 정보를 제공해야 하기 때문입니다. PostgreSQL 7.4 이하 버전에서는 update_geometry_stats([table_name, column_name]) (분포 계산) 및 VACUUM ANALYZE [table_name] [column_name] (값의 개수 계산)을 실행하면 됩니다. PostgreSQL 8.0 이상 버전에서는 VACUUM ANALYZE 를 실행하면 두 계산을 동시에 처리할 것입니다. 어쨌든지 간에 사용자 데이터베이스를 주기적으로 빈공간 분석(vacuum analyze)하는 편이 좋습니다 -- 많은 PostgreSQL DBA가 정기적으로 사용량이 적을 때 자동화된 작업으로 VACUUM 을 실행하고 있습니다.

- 두 번째, 빈공간 분석으로 해결이 안 될 경우 SET ENABLE_SEQSCAN=OFF 명령어를 통해 강제로 쿼리 설계자가 인덱스 정보를 이용하도록 할 수 있습니다. 이 명령어는 공간 인덱스 쿼리일 경우에 한해 드물게 이용해야 합니다. 일반적으로, 쿼리 설계자는 언제 일반 B-Tree 인덱스를 활용해야 하는지 사용자보다 더 잘 알고 있습니다. 사용자 쿼리 실행 후, 다른 쿼리가 평소처럼 쿼리 설계자를 활용하도록 ENABLE_SEQSCAN 을 다시 켤지 고려해봐야 합니다.



0.6 버전부터, ENABLE_SEQSCAN 변수를 통해 강제로 쿼리 설계자가 인덱스를 이용하도록 할 필요는 없을 것입니다.

- 쿼리 설계자가 순차 및 인덱스 스캔의 경중(cost)을 잘못 판단하고 있다면, postgresql.conf 파일의 random_page_cost의 값을 줄여보거나 "SET random_page_cost=#"로 써보십시오. 해당 파라미터의 기본값은 4이지만, 1또는 2로 설정해보십시오. 값을 감소시킬수록 점점 더 쿼리 설계자가 인덱스 스캔을 활용하게 될 것입니다.

4.7. 복잡 쿼리

공간 데이터베이스 기능성의 존재 이유는 원래 데스크탑 GIS 기능성을 필요로 하는 데이터베이스의 내부에서 쿼리를 실행하는 것입니다. GIS를 효율적으로 활용하려면 어떤 공간 함수를 사용할 수 있는지

알아야 하고, 훌륭한 수행을 제공하기 위한 적절한 인덱스가 준비되어 있는지 보장해야 합니다. 이 단원의 예시에서 사용된 SRID 312는 오직 시연을 위한 것입니다. 사용자는 spatial_ref_sys 테이블에 정리되어 있으며 사용자 데이터의 투영체와 일치하는 실제 SRID를 사용해야 합니다. 사용자의 데이터가 어떤 공간 참조 시스템도 지정하고 있지 않을 경우, 어째서 지정하고 있지 않은지 그리고 지정해야 할지를 매우 신중하게 생각해봐야 할 것입니다. 분자 내부 구조 또는 핵전쟁 발발로 인해 인류를 화성으로 이주시키는 데 적합한 화성 상의 위치 등과 같이 정의된 지리 공간 참조 시스템이 없는 대상을 모델링하고 있기 때문이라면, 그냥 SRID를 비워두거나 또는 하나 만들어내서 spatial_ref_sys 테이블에 삽입하십시오.

4.7.1. 인덱스의 장점을 활용하기

쿼리 작성시 && 와 같은 경계 상자 기반 연산자만이 GiST 공간 인덱스의 장점을 취할 수 있다는 사실을 기억해야 합니다. ST_Distance() 같은 함수는 자체 연산을 최적화하는 데 인덱스를 활용할 수 없습니다. 예를 들어, 다음 쿼리는 대용량 테이블 대상일 경우 꽤 느릴 겁니다:

```
SELECT the_geom
FROM geom_table
WHERE ST_Distance(the_geom, ST_GeomFromText('POINT(100000 200000)', 312))
< 100
```

이 쿼리는 geom_table 안에서 포인트 (100000, 200000)로부터 100 단위 안에 있는 모든 도형을 선택하고 있습니다. 지정된 포인트와 테이블이 담고 있는 모든 포인트 사이의 거리를 각각 계산하기 때문에, 예를 들어 테이블의 각 행마다 ST_Distance() 계산을 실행하기 때문에 느릴 수밖에 없습니다. && 연산자를 통해 필요한 거리 계산의 개수를 줄여 속도를 향상시킬 수 있습니다:

```
SELECT the_geom
FROM geom_table
WHERE   ST_DWithin(the_geom,   ST_MakeEnvelope(90900,   190900,   100100,
200100,312), 100)
```

이 쿼리는 동일한 도형을 선택하지만, 더 효율적인 방식을 쓰고 있습니다. the_geom에 대한 GiST 인덱스가 있다고 가정하면, ST_distance() 함수의 결과를 계산하기 전에 쿼리 설계자가 인덱스를 활용해서 행의 개수를 줄일 수 있다는 사실을 알아차릴 것입니다. && 연산에 사용된 ST_MakeEnvelope 도형이 원래 포인트가 중심에 있는 200단위 정사각형이라는 점을 주의하십시오. 이것이 "쿼리 상자"입니다. && 연산자는 "쿼리 상자"와 겹치는 경계 상자를 가진 도형들만으로 결과 집합을 신속하게 줄이기 위해 인덱스를 활용합니다. 이 쿼리 상자가 전체 도형 테이블의 범위보다 훨씬 작다고 가정할 때, 필요한 거리 계산의 개수가 극단적으로 줄어들 것입니다.



작동 방식의 변화

PostGIS 1.3.0 버전부터, 주목할 만한 ST_Disjoint 및 ST_Relate 를 제외한 도형 관계 함수 대부분은 내포된 경계 상자 중첩(overlap) 연산자를 포함하고 있습니다.

4.7.2. 공간 SQL 예시

이 단원의 예시들은 선형 도로 테이블 및 폴리곤형 지자체 경계 테이블을 활용할 것입니다. bc_roads 테이블은 다음과 같이 정의됩니다:

```
Column      | Type            | Description
```

```
------------+-------------------+-------------------
gid        | integer          | Unique ID
name       | character varying | Road Name
the_geom    | geometry         | Location Geometry (Linestring)
```

bc_municipality 테이블은 다음과 같이 정의됩니다:

```
Column     | Type              | Description
-----------+-------------------+-------------------
gid       | integer          | Unique ID
code      | integer          | Unique ID
name      | character varying | City / Town Name
the_geom   | geometry         | Location Geometry (Polygon)
```

1. 모든 도로들의 총연장이 몇 킬로미터입니까?

매우 단순한 SQL 로 이 질문에 답할 수 있습니다:

```
SELECT sum(ST_Length(the_geom))/1000 AS km_roads FROM bc_roads;

km_roads
------------------
70842.1243039643
(1 row)
```

2. 프린스 조지(Prince George) 시의 면적이 몇 헥타르입니까?

이 쿼리는 속성 조건(지자체 명칭)과 공간 계산(면적)을 결합시킵니다:

```
SELECT
  ST_Area(the_geom)/10000 AS hectares
FROM bc_municipality
WHERE name = 'PRINCE GEORGE';

hectares
------------------
32657.9103824927
 (1 row)
```

3. 해당 지역에서 가장 면적이 넓은 지자체는 어디입니까?

이 쿼리는 쿼리 조건에 공간 측정을 넣어야 합니다. 이 문제에 접근하는 데에는 몇 가지 방법이 있지만, 가장 효율적인 방법은 다음과 같습니다:

```
SELECT
  name,
  ST_Area(the_geom)/10000 AS hectares
FROM
  bc_municipality
ORDER BY hectares DESC
LIMIT 1;

name         | hectares
--------------+-----------------
TUMBLER RIDGE  | 155020.02556131
(1 row)
```

이 문제에 답하기 위해 모든 폴리곤의 면적을 계산해야 한다는 점에 주의하십시오. 이런 작업을 많이 해야 한다면, 테이블에 비교를 위해 별도로 인덱스 작업을 한 면적 열을 추가하는 편이 좋을 것입니다. 면적이 넓은 순으로 결과값을 정렬한 다음 PostgreSQL 의 "LIMIT" 명령어를 사용해서 max() 같은 종합 함수를 이용하지 않고도 가장 넓은 값을 쉽게 골라낼 수 있습니다.

4. 각 지자체 안에 완전히 들어가 있는 도로의 총연장은 얼마나 됩니까?

이 문제는 "공간 연결(spatial join)"의 예시입니다. 두 테이블로부터 데이터를 함께 모으지만(연결), 연결 조건으로 흔히 쓰이는 공통 키(common key)를 기반으로 연결하는 관계 접근법 대신 공간 교차 조건("들어가 있는")을 이용하기 때문입니다:

```
SELECT
  m.name,
  sum(ST_Length(r.the_geom))/1000 as roads_km
FROM
  bc_roads AS r,
  bc_municipality AS m
WHERE
  ST_Contains(m.the_geom,r.the_geom)
GROUP BY m.name
ORDER BY roads_km;

name                      | roads_km
--------------------------+------------------
SURREY                    | 1539.47553551242
VANCOUVER                 | 1450.33093486576
LANGLEY DISTRICT          | 833.793392535662
BURNABY                   | 773.769091404338
PRINCE GEORGE             | 694.37554369147
...
```

테이블에 있는 모든 도로를 최종 결과(현재 예시 테이블의 경우 약 250,000 개의 도로)로 요약하기 때문에 이 쿼리는 시간이 걸립니다. 더 작은 오버레이(수백 개의 도로에 대한 수천 레코드 정도)라면, 응답도 매우 빠를 것입니다.

5. 프린스 조지 시 내부에 있는 모든 도로를 가진 새로운 테이블을 생성하십시오.

이 지시는 "오버레이"에 대한 예시입니다. 오버레이 쿼리는 두 테이블을 읽어들여 공간적으로 오려내거나 잘라낸 결과값으로 이루어진 새로운 테이블을 산출합니다. 앞에서 나온 "공간 연결"과는 달리, 이 쿼리는 실제로 새 도형을 생성합니다. 오버레이는 터보차저가 달린 공간 연결이라 할 수 있고, 더 정확한 분석 작업에 유용합니다:

```
CREATE TABLE pg_roads as
SELECT
  ST_Intersection(r.the_geom, m.the_geom) AS intersection_geom,
  ST_Length(r.the_geom) AS rd_orig_length,
  r.*
FROM
  bc_roads AS r,
  bc_municipality AS m
WHERE  m.name = 'PRINCE GEORGE' AND ST_Intersects(r.the_geom, m.the_geom);
```

6. 빅토리아 시의 "더글러스 거리**(Douglas St)**"의 길이는 몇 킬로미터입니까**?**

```
SELECT
  sum(ST_Length(r.the_geom))/1000 AS kilometers
FROM
  bc_roads r,
  bc_municipality m
WHERE  r.name = 'Douglas St' AND m.name = 'VICTORIA'
       AND ST_Contains(m.the_geom, r.the_geom) ;

kilometers
------------------
4.89151904172838
(1 row)
```

7. 구멍이 있는 지자체 폴리곤 가운데 가장 넓은 곳은 어디입니까**?**

```
SELECT gid, name, ST_Area(the_geom) AS area
FROM bc_municipality
WHERE ST_NRings(the_geom) > 1
ORDER BY area DESC LIMIT 1;

gid  | name        | area
-----+-------------+------------------
12   | SPALLUMCHEEN | 257374619.430216
(1 row)
```

Chapter 5.래스터 데이터의 관리, 쿼리, 및 응용

5.1. 래스터 로드 및 생성

대부분의 경우, 사용자는 패키징되어 있는 raster2pgsql 래스터 로더를 통해 기존 래스터 파일을 로드해서 PostGIS 래스터를 생성할 것입니다.

5.1.1. raster2pgsql 을 이용해 래스터를 로드하기

raster2pgsql 은 GDAL 이 지원하는 래스터 형식을 로드해서 PostGIS 래스터 테이블로 로드하는 데 적합한 SQL로 변환하는래스터 로더 실행 파일입니다. 래스터 오버뷰 생성은 물론 래스터 파일 폴더를 로드할 수도 있습니다.

raster2pgsql 이 (사용자 스스로 GDAL 라이브러리를 컴파일하지 않는 한) 가장 흔히 PostGIS 의 일부로서 컴파일되는 파일이기 때문에, 이 실행 파일이 지원하는 래스터 형식은 GDAL 의존성 라이브러리 안에 컴파일되는 래스터 형식과 동일할 것입니다. 사용자 자신의 raster2pgsql 이 지원하는 래스터 형식의 목록을 보려면 -G 스위치를 사용하십시오. 사용자가 양쪽 모두에서 동일한 GDAL 라이브러리를 사용하고 있다면, ST_GDALDrivers 에 문서화된, 사용자 PostGIS 설치가 제공하는 지원 목록과 동일해야 합니다.



이 도구의 구 버전은 파이썬 스크립트였습니다. 현재는 실행 파일이 파이썬 스크립트를 대체했습니다. 사용자가 해당 파이썬 스크립트를 필요로 할 경우, GDAL PostGIS 래스터 드라이버 활용 에서 파이썬 스크립트의 예들을 찾을 수 있습니다. raster2pgsql 파이썬 스크립트가 PostGIS 향후 버전과 호환되지 않을 수 있으며, 더 이상 지원되지도 않는다는 점을 주의해주시기 바랍니다.



정렬된 래스터 집합에서 특정 인자(factor)의 오버뷰를 성생하는 경우, 오버뷰들이 정렬되지 않을 수도 있습니다. http://trac.osgeo.org/postgis/ticket/1764 페이지에서 오버뷰가 정렬되지 않는 예시를 찾아볼 수 있습니다.

활용 예:

```
raster2pgsql raster_options_go_here raster_file someschema.sometable > out.sql
```

-? 도움말 화면을 표출합니다. 어떤 인수도 쓰지 않을 경우에도 도움말이 표출될 것입니다..

-G 지원하는 래스터 형식을 나열합니다.

(c|a|d|p) **공통으로 적용되는 전용 옵션 :**

-c 새 테이블을 생성한 다음 래스터(들)로 해당 테이블을 채웁니다. *이것이 기본 모드입니다.*

-a 기존 테이블에 래스터(들)을 추가합니다.

-d 기존 테이블을 삭제하고, 새 테이블을 생성한 다음 래스터(들)로 해당 테이블을 채웁니다.

-p 준비 모드로, 테이블만 생성합니다.

래스터 공간 처리: 래스터 카탈로그에 제대로 등록하기 위한 제약조건 적용

-C raster_columns 뷰에 래스터를 제대로 등록하기 위한 SRID, 픽셀 크기 등의 래스터 제약조건을 적용합니다.

-x 최대 범위(extent) 제약조건을 해제합니다. -C 플래그와 함께 쓰일 경우에만 적용됩니다.

-r 정규 블록화(regular blocking)를 위한 제약조건(공간적 유일성 및 커버리지 타일)을 설정합니다. -C 플래그와 함께 쓰일 경우에만 적용됩니다.

래스터 공간 처리: 입력 래스터 데이터셋을 조작하는 데 쓰이는 선택적 파라미터

-s <SRID> 출력 래스터에 지정된 SRID 를 부여합니다. 기존에 없거나 0 값인 경우, 적절한 SRID 를 결정하기 위해 래스터의 메타데이터를 확인할 것입니다.

-b BAND 래스터에서 밴드의 (1-기반) 인덱스를 추출합니다. 하나 이상의 밴드 인덱스가 있을 경우, 쉼표(,)로 구분하십시오. 지정하지 않는 경우 래스터의 모든 밴드를 추출합니다.

-t TILE_SIZE 테이블 행 한 개당 하나씩 삽입되도록 래스터를 타일로 자릅니다. TILE_SIZE 는 너비 x 높이로 표현되거나, 또는 "auto" 값으로 설정하면 로더가 첫 번째 래스터를 이용해 적당한 크기를 계산해서 모든 래스터에 적용합니다.

-P 맨 오른쪽 및 맨 아래 타일들의 여백을 메꿔넣어(padding) 모든 타일이 동일한 너비와 높이를 갖도록 보장합니다.

-R, --register 래스터를 파일 시스템 (DB 외부) 래스터로 등록합니다.

데이터베이스에 래스터의 (픽셀이 아니라) 메타데이터 및 경로 위치만 저장될 것입니다.

-l OVERVIEW_FACTOR 래스터의 오버뷰를 생성합니다. 인자가 한 개 이상인 경우, 쉼표(,)로 구분하십시오. 오버뷰 테이블명은 o_overview factor_table 이라는 패턴을 따르는데, 이때 overview factor 는 숫자로 나타낸 오버뷰 인자의 위치지시자(placeholder)이며 table 은 기저 테이블명으로 대체됩니다. 생성된 오버뷰는 데이터베이스에 저장되어, -R 플래그의 영향을 받지 않습니다. 사용자가 생성한 SQL 파일이 주 테이블과 오버뷰 테이블을 모두 담게 될 것이라는 점에 주의하십시오.

-N NODATA "NODATA" 값이 없는 밴드에 사용할 NODATA 값입니다.

데이터 오브젝트를 처리하기 위해 사용되는 선택적 파라미터

-q PostgreSQL 식별자를 따옴표로 감쌉니다.

-f COLUMN 저장될 래스터 열의 명칭을 설정합니다. 기본값은 'rast'입니다.

-F 파일명을 담은 열을 추가합니다.

-n COLUMN 파일명 열의 명칭을 설정합니다. -F 플래그와 함께 써야 합니다.

-q PostgreSQL 식별자를 따옴표로 감쌉니다.

-I 래스터 열에 GiST 인덱스를 생성합니다.

-M 래스터 테이블을 빈공간 분석(vacuum analyze)합니다.

-K 각 래스터 밴드 별로 NODATA 값 확인을 건너뜁니다.

-T tablespace 새 테이블 용 테이블스페이스를 설정합니다. -X 플래그를 함께 사용하지 않으면 (기본 키를 포함한) 인덱스들이 여전히 기본 테이블스페이스를 사용할 것이라는 점에 주의하십시오.

-X tablespace 테이블의 새 인덱스 용 테이블스페이스를 설정합니다. -I 플래그를 함께 사용하면 기본 키와 공간 인덱스에 함께 적용됩니다.

-Y 삽입 선언문 대신 복사 선언문을 사용합니다.

-e 각 선언문을 개별적으로 실행하며, 상호처리(transaction)를 이용하지 않습니다.

-E ENDIAN 생성된 래스터 바이너리 산출물의 메모리 내부 데이터 순서(endianness)를 조정합니다. XDR 은 0, 기본값인 NDR 은 1 로 설정하십시오. 현재, NDR 산출물만 지원합니다.

-V version 산출물 형식의 버전을 설정합니다. 기본값은 0 입니다. 현재, 0 만 지원합니다.

로더를 통해 입력 파일을 생성하고 `100x100` 타일 뭉치로 나누어 업로드하는 세션의 예시는 다음과 같습니다:



예를 들어 public.demelevation 대신 demelevation 을 쓰는 것처럼 스키마 명을 생략할 수 있습니다. 그래도 래스터 테이블은 데이터베이스 또는 사용자의 기본 스키마를 기반으로 생성될 것입니다.

```
raster2pgsql -s 4326 -I -C -M *.tif -F -t 100x100 public.demelevation
> elev.sql
psql -d gisdb -f elev.sql
```

UNIX 파이프를 이용하면 변환 및 업로드 작업을 한 번에 끝낼 수 있습니다:

```
raster2pgsql -s 4326 -I -C -M *.tif -F -t 100x100 public.demelevation |
psql -d gisdb
```

매사추세츠 주의 미터 단위 항공사진 타일 래스터들을 aerial 이라는 스키마로 로드하고, 전체 뷰와 2 레벨 및 4 레벨 오버뷰 테이블을 생성한 다음, 복사 모드를 통해 (중간 단계 파일 없이 DB 로 직접) 삽입하며, 강제로 모든 작업을 상호처리하지 않도록 -e 플래그를 사용하십시오(작업이 완료되길 기다리지 않고 테이블에 들어오는 데이터를 바로 살펴보고자 할 때 유용합니다). 래스터를 128x128 픽셀 타일로 분절한 다음 래스터 제약조건을 적용하십시오. 테이블 삽입 대신 복사 모드를 이용하십시오. 타일들이 잘라져 나온 타일 파일명을 담을 수 있도록 -F 플래그로 "filename"이라는 필드를 포함시키십시오.

```
raster2pgsql -I -C -e -Y -F -s 26986 -t 128x128 -l 2,4
bostonaerials2008/*.jpg aerials.boston | psql -U postgres -d gisdb -h
localhost -p 5432
```

```
-- 지원되는 래스터 유형의 목록을 얻으려면:
raster2pgsql -G
```

-G 옵션이 다음과 같은 목록을 출력할 것입니다:

```
Available GDAL raster formats:
  Virtual Raster
  GeoTIFF
  National Imagery Transmission Format
  Raster Product Format TOC format
  ECRG TOC format
  Erdas Imagine Images (.img)
  CEOS SAR Image
  CEOS Image
  JAXA PALSAR Product Reader (Level 1.1/1.5)
  Ground-based SAR Applications Testbed File Format (.gff)
  ELAS
  Arc/Info Binary Grid
  Arc/Info ASCII Grid
  GRASS ASCII Grid
  SDTS Raster
  DTED Elevation Raster
  Portable Network Graphics
  JPEG JFIF
  In Memory Raster
  Japanese DEM (.mem)
  Graphics Interchange Format (.gif)
  Graphics Interchange Format (.gif)
  Envisat Image Format
  Maptech BSB Nautical Charts
  X11 PixMap Format
  MS Windows Device Independent Bitmap
  SPOT DIMAP
  AirSAR Polarimetric Image
  RadarSat 2 XML Product
  PCIDSK Database File
  PCRaster Raster File
  ILWIS Raster Map
  SGI Image File Format 1.0
  SRTMHGT File Format
  Leveller heightfield
  Terragen heightfield
  USGS Astrogeology ISIS cube (Version 3)
  USGS Astrogeology ISIS cube (Version 2)
  NASA Planetary Data System
  EarthWatch .TIL
  ERMapper .ers Labelled
  NOAA Polar Orbiter Level 1b Data Set
  FIT Image
  GRIdded Binary (.grb)
  Raster Matrix Format
  EUMETSAT Archive native (.nat)
  Idrisi Raster A.1
  Intergraph Raster
  Golden Software ASCII Grid (.grd)
  Golden Software Binary Grid (.grd)
  Golden Software 7 Binary Grid (.grd)
  COSAR Annotated Binary Matrix (TerraSAR-X)
  TerraSAR-X Product
  DRDC COASP SAR Processor Raster
  R Object Data Store
```

```
Portable Pixmap Format (netpbm)
USGS DOQ (Old Style)
USGS DOQ (New Style)
ENVI .hdr Labelled
ESRI .hdr Labelled
Generic Binary (.hdr Labelled)
PCI .aux Labelled
Vexcel MFF Raster
Vexcel MFF2 (HKV) Raster
Fuji BAS Scanner Image
GSC Geogrid
EOSAT FAST Format
VTP .bt (Binary Terrain) 1.3 Format
Erdas .LAN/.GIS
Convair PolGASP
Image Data and Analysis
NLAPS Data Format
Erdas Imagine Raw
DIPEx
FARSITE v.4 Landscape File (.lcp)
NOAA Vertical Datum .GTX
NADCON .los/.las Datum Grid Shift
NTv2 Datum Grid Shift
ACE2
Snow Data Assimilation System
Swedish Grid RIK (.rik)
USGS Optional ASCII DEM (and CDED)
GeoSoft Grid Exchange Format
Northwood Numeric Grid Format .grd/.tab
Northwood Classified Grid Format .grc/.tab
ARC Digitized Raster Graphics
Standard Raster Product (ASRP/USRP)
Magellan topo (.blx)
SAGA GIS Binary Grid (.sdat)
Kml Super Overlay
ASCII Gridded XYZ
HF2/HFZ heightfield raster
OziExplorer Image File
USGS LULC Composite Theme Grid
Arc/Info Export E00 GRID
ZMap Plus Grid
NOAA NGS Geoid Height Grids
```

5.1.2. PostGIS 래스터 함수를 이용해 래스터 생성하기

사용자가 래스터와 래스터 테이블을 데이터베이스 내부에 생성하려 하는 경우가 많을 것입니다. 그런 작업을 위한, 넘치고도 남을 많은 함수가 있습니다. 일반적인 단계는 다음과 같습니다.

1. 새 래스터 레코드를 담을 래스터 열을 가진 테이블을 다음과 같이 생성하십시오:

```
CREATE TABLE myrasters(rid serial primary key, rast raster);
```

2. 해당 목표를 도와줄 함수가 많이 있습니다. 다른 래스터에서 파생되지 않은 래스터를 생성하는 경우, ST_MakeEmptyRaster 함수와 ST_AddBand 함수를 순서대로 사용하는 편이 좋습니다.
   도형으로부터도 래스터를 생성할 수 있습니다. 그러려면 ST_AsRaster 함수를, 아마도 ST_Union , ST_MapAlgebraFct 또는 맵 대수(algebra) 함수 계열의 어떤 함수와도 같은 다른 함수와 함께 사용하는 편이 좋습니다.

기존 테이블로부터 새 래스터 테이블을 생성하는 데에는 더욱 많은 선택지가 있습니다. 예를 들어 ST_Transform 함수를 사용하면 기존 테이블과는 다른 투영이 적용된 래스터 테이블을 생성할 수 있습니다.

3. 일단 사용자 테이블을 채우는 작업을 마쳤다면, 다음과 같이 래스터 열에 대해 공간 인덱스를 생성하는 편이 좋습니다:

```
CREATE INDEX myrasters_rast_st_convexhull_idx ON myrasters USING
gist( ST_ConvexHull(rast) );
```

래스터 연산자 대부분이 래스터의 볼록 껍질(convex hull)을 기반으로 하기 때문에 ST_ConvexHull 함수를 사용했다는 점에 주의하십시오.



PostGIS 2.0 미만 버전에서 래스터는 볼록 껍질보다는 엔벨로프(envelop)를 기반으로 하고 있었습니다. 공간 인덱스가 제대로 작동하려면 구식 인덱스를 삭제하고 볼록 껍질 기반 인덱스로 대체해야 합니다.

4. AddRasterConstraints 를 이용해 래스터 제약조건 적용

5.2. 래스터 카탈로그

PostGIS 와 함께 패키징되는 래스터 카탈로그 뷰는 두 개입니다. 두 뷰 모두 래스터 테이블의 제약조건에 내장된 정보를 활용합니다. 제약조건이 강제적이기 때문에, 결과적으로 카탈로그 뷰는 언제나 테이블 내부의 래스터 데이터와 일관성을 유지합니다.

1. raster_columns 이 뷰는 사용자 데이터베이스에 있는 모든 래스터 테이블 열을 카탈로그화합니다.

2. raster_overviews 이 뷰는 더 세밀한 테이블을 위한 오버뷰 역할을 하는 사용자 데이터베이스에 있는 모든 래스터 테이블 열을 카탈로그화합니다. 로드 과정에서 -I 스위치를 사용할 경우 이 테이블 유형을 생성합니다.

5.2.1. 래스터 열 카탈로그

raster_columns 는 사용자 데이터베이스에 있는 모든 래스터 테이블 열의 래스터 유형 카탈로그입니다. 래스터 열 카탈로그는 테이블에 대한 제약조건을 활용하기 때문에, 설령 사용자가 다른 데이터베이스의 백업으로부터 래스터 테이블을 하나 복원했다 하더라도, 카탈로그 정보는 언제나 일관성을 유지합니다. 다음은 raster_columns 카탈로그 안에 존재하는 열들입니다.

로더를 통해 사용자 테이블을 생성하지 않았거나 로드 과정에서 -C 플래그 설정을 잊었을 경우, 테이블 생성 후에 raster_columns 카탈로그가 사용자의 래스터 타일에 관한 일반 정보를 등록하도록 AddRasterConstraints 함수를 통해 제약조건을 강제할 수 있습니다.

- `r_table_catalog` 테이블을 담고 있는 데이터베이스입니다. 이 열은 언제나 현재 데이터베이스를 읽어들일 것입니다.

- r_table_schema 래스터 테이블이 속해 있는 데이터베이스 스키마입니다.
- r_table_name 래스터 테이블 명입니다.
- r_raster_column 래스터 유형의 r_table_name 테이블 안에 있는 열입니다. PostGIS는 사용자가 테이블 하나 당 몇 개의 래스터 열을 가질 수 있는지 제한하지 않기 때문에, 래스터 테이블 하나를 각각 다른 래스터 열로 몇 번이고 나열할 수 있습니다.
- srid 래스터의 공간 참조 식별자입니다. Section 4.3.1, "SPATIAL_REF_SYS 테이블과 공간 참조 시스템" 목록에 있는 항목이어야 합니다.
- scale_x 기하하적 공간 좌표와 픽셀의 비례(축척)입니다. 래스터 열에 있는 모든 타일이 동일한 scale_x 를 가지며, scale_x 제약조건이 적용될 경우에만 사용할 수 있습니다. 자세한 내용은 ST_ScaleX 를 참조하십시오.
- scale_y 기하하적 공간 좌표와 픽셀의 비례(축척)입니다. 래스터 열에 있는 모든 타일이 동일한 scale_y 를 가지며, scale_y 제약조건이 적용될 경우에만 사용할 수 있습니다. 자세한 내용은 ST_ScaleY 를 참조하십시오.
- blocksize_x 각 래스터 타일의 너비(가로 방향의 픽셀 개수)입니다. 자세한 사항은 ST_Width 를 참조하십시오.
- blocksize_y 각 래스터 타일의 높이(세로 방향의 픽셀 개수)입니다. 자세한 사항은 ST_Height 를 참조하십시오.
- same_alignment 모든 래스터 타일이 동일한 방향으로 정렬된 경우 참인 불 값입니다. 자세한 내용은 ST_SameAlignment 를 참조하십시오.
- regular_blocking 래스터 열이 공간적 유일성 제약조건 및 커버리지 타일 제약조건을 가질 경우, 해당 값은 참입니다. 그 외의 경우는 거짓이 됩니다.
- num_bands 사용자 래스터 집합의 각 타일이 내장한 밴드의 개수입니다. 다음 열과 동일한 정보를 제공합니다. ST_NumBands
- pixel_types 각 밴드에 대한 픽셀 유형을 정의하는 배열입니다. 이 배열에는 사용자 밴드 개수와 동일한 개수의 요소들이 들어갈 것입니다. pixel_types는 다음 ST_BandPixelType 에서 정의된 픽셀 유형 가운데 하나입니다.
- nodata_values 각 밴드에 대한 nodata_value 를 의미하는 배정밀도(double precision) 수(數)입니다. 이 배열에는 사용자 밴드 개수와 동일한 개수의 요소들이 들어갈 것입니다. 이 숫자는 대부분의 연산에서 무시되어야 할 각 밴드에 대한 픽셀 값을 정의합니다. 이 정보는 ST_BandNoDataValue 가 제공하는 정보와 유사합니다.
- out_db 데이터베이스 외부에서 래스터 밴드 데이터를 유지하는지 여부를 보여주는 불 값 플래그의 배열입니다. 이 배열에는 사용자 밴드 개수와 동일한 개수의 요소들이 들어갈 것입니다.
- extent 사용자 래스터 집합이 포함하는 모든 래스터 열의 범위(extent)입니다. 해당 집합의 범위를 변경시킬 추가 데이터를 로드할 계획이라면, 로드 작업 전에 DropRasterConstraints 함수를 실행하고 작업 후에 AddRasterConstraints 함수로 제약조건을 다시 적용시키는 편이 좋습니다.
- spatial_index 래스터 열이 공간 인덱스를 가진 경우 참인 불 값입니다.

5.2.2. 래스터 오버뷰

raster_overviews 는 오버뷰에 이용되는 래스터 테이블 열에 관한 정보 및 오버뷰 활용시 알면 유용한 추가 정보를 카탈로그화합니다. 오버뷰 테이블은 raster_columns 및 raster_overviews 양쪽에 모두 카탈로그화됩니다. 오버뷰가 나름대로 래스터이긴 하지만, 동시에 고해상도 테이블의 저해상도 썸네일이라는 추가적인 특별 목적을 담당하고 있기 때문입니다. 래스터 로드 작업중 -l 스위치를 사용하면 주 래스터 테이블과 함께 오버뷰 테이블을 생성합니다. 또는 AddOverviewConstraints 함수를 통해 직접 생성할 수도 있습니다.

오버뷰 테이블은 다른 래스터 테이블과 동일한 제약조건은 물론 오직 오버뷰에 특화된 제약조건에 관한 추가 정보도 포함합니다.



raster_overviews 에 담긴 정보는 raster_columns 의 정보를 복제한 것이 아닙니다. raster_columns 에서 현재 오버뷰 테이블에 관한 정보를 얻어야 할 경우 raster_overviews 와 raster_columns 을 함께 연결(join)시켜 필요한 완전한 정보를 얻으십시오.

오버뷰의 주된 존재 이유는 다음 두 가지입니다:

1. 빠른 매핑 확대를 위해 흔히 핵심 테이블의 저해상도 모사를 이용합니다.
2. 고해상도 페어런트보다 오버뷰를 기반으로 하는 계산이 일반적으로 더 빠릅니다. 레코드 개수도 더 적고 각 픽셀이 커버하는 면적도 더 크기 때문입니다. 오버뷰가 지원하는 고해상도 테이블 대상 계산보다야 정확하지 않지만, 어림잡이(rule-of-thumb) 계산의 경우 오버뷰도 충분할 수 있습니다.

raster_overviews 카탈로그는 다음과 같은 정보 열을 포함합니다.

- o_table_catalog 오버뷰 테이블을 담고 있는 데이터베이스입니다. 이 열은 언제나 현재 데이터베이스를 읽어들일 것입니다.
- o_table_schema 오버뷰 테이블이 속해 있는 데이터베이스 스키마입니다.
- o_table_name 오버뷰 테이블 명입니다.
- o_raster_column 오버뷰 테이블이 담고 있는 래스터 열입니다.
- r_table_catalog 해당 오버뷰 서비스를 담고 있는 래스터 테이블 카탈로그입니다. 이 열은 언제나 현재 데이터베이스를 읽어들일 것입니다.
- r_table_schema 해당 오버뷰 서비스가 속해 있는 래스터 테이블 스키마입니다.
- r_table_name 해당 오버뷰가 서비스하는 래스터 테이블 명입니다.
- r_raster_column 해당 오버뷰가 서비스하는 래스터 열입니다.
- overview_factor 오버뷰 테이블의 피라미드 레벨입니다. 숫자가 커질수록 테이블의 해상도는 낮아집니다. raster2pgsql 은 이미지 폴더를 받아 각 이미지 파일의 오버뷰를 계산하고 개별적으로 로드합니다. 항상 레벨 1을 원 파일로 가정합니다. 레벨 2는 각 타일이 원래 이미지 4장을 모사하게

됩니다. 따라서 예를 들어 사용자가 125x125 로 뭉치려는 5000x5000 픽셀 이미지 파일들이 있는 폴더의 경우, 각 이미지 파일에 대해 사용자의 기저 테이블이 (5000*5000)/(125*125) = 1600 레코드를 담게 되고, 사용자의 o_2 (l=2) 테이블이 최대 (1600/2^2) = 400 행, o_3 (l=3) 테이블은 최대 (1600/2^3) = 200 행의 레코드를 담게 될 것입니다. 사용자 픽셀 수가 사용자 타일로 나눌 수 있는 숫자가 아닐 경우, 몇몇 파편(scrap) 타일(완전히 채워지지 않은 타일)을 얻게 될 것입니다. raster2pgsql 이 생성한 각 오버뷰 타일의 픽셀 개수가 해당 페어런트와 동일한 개수이지만, 각 픽셀이 표현하는 영역이 더 넓은 (원 이미지의 2^overview_factor 픽셀에 해당하는) 저해상도라는 점에 주의하십시오.

5.3. PostGIS 래스터를 이용하는 사용자 지정 응용 프로그램 빌드하기

PostGIS 래스터가 알려진 이미지 형식으로 래스터를 렌더링하는 SQL 함수를 제공하기 때문에, 래스터 렌더링 옵션이 많습니다. 예를 들어 리브레오피스 기본 리포트로 PostGIS 래스터 그래픽을 렌더링하기 에서 설명하는 대로 오픈오피스/리브레오피스를 이용해서 렌더링할 수도 있습니다. 또한 이 단원에서 설명하는 대로 광범위한 언어를 이용할 수도 있습니다.

5.3.1. 다른 래스터 함수와 함께 ST_AsPNG 를 이용해서 PHP 예제를 출력하기

이 단원에서, PHP PostgreSQL 드라이버와 ST_AsGDALRaster 계열 함수를 사용해서 래스터의 1, 2, 3 밴드를 PHP 요청 스트림(request stream)으로 출력하는 방법을 설명할 것입니다. 이후 PHP 요청 스트림을 "img src" HTML 태그로 임베딩할 수 있습니다.

예시 쿼리는 여러 래스터 함수를 함께 결합(combine)해서 특정 WGS84 경계 상자와 교차하는 모든 타일을 얻은 다음, ST_Union 함수를 통해 교차하는 타일들과 반환되는 모든 밴드를 통합(union)해서, ST_Transform 함수를 이용해 사용자가 지정한 투영으로 변환한 후, ST_AsPNG 함수로 결과값을 PNG 파일로 출력하는 방법을 보여줍니다.

```
http://mywebserver/test_raster.php?srid=2249
```

를 통해 메사추세츠 주의 피트 단위 항공사진의 래스터 이미지를 얻은 후, 다음 쿼리를 호출하십시오.

```
<?php
/** test_raster.php 의 내용 **/
$conn_str    ='dbname=mydb    host=localhost    port=5432    user=myuser
password=mypwd';
$dbconn = pg_connect($conn_str);
header('Content-Type: image/png');
/** 특정 투영체를 요구할 경우 이걸 사용하고, 그렇지 않을 경우 매사추세츠 주 미터 단위 평면을
사용할 것 **/
if (!empty( $_REQUEST['srid'] ) && is_numeric( $_REQUEST['srid']) ){
            $input_srid = intval($_REQUEST['srid']);
}
else { $input_srid = 26986; }
/** PostgreSQL 9.0 이상 버전의 경우 bytea_output 집합이 필요할 수도 있지만, 8.4
버전은 필요없음 **/
$sql = "set bytea_output='escape';
SELECT ST_AsPNG(ST_Transform(
                  ST_AddBand(ST_Union(rast,1),
ARRAY[ST_Union(rast,2),ST_Union(rast,3)])
```

```
                          ,$input_srid) ) As new_rast
 FROM aerials.boston
       WHERE
        ST_Intersects(rast, ST_Transform(ST_MakeEnvelope(-71.1217, 42.227,
-71.1210, 42.218,4326),26986) )";
$result = pg_query($sql);
$row = pg_fetch_row($result);
pg_free_result($result);
if ($row === false) return;
echo pg_unescape_bytea($row[0]);
?>
```

5.3.2. 다른 래스터 함수와 함께 ST_AsPNG 를 이용해서 ASP.NET C# 예제를 출력하기

이 단원에서, npgsql PostgreSQL .NET 드라이버와 ST_AsGDALRaster 계열 함수를 사용해서 래스터의 1, 2, 3 밴드를 PHP 요청 스트림(request stream)으로 출력하는 방법을 설명할 것입니다. 이후 PHP 요청 스트림을 "img src" HTML 태그로 임베딩할 수 있습니다.

이 예제를 실행하려면 npgsql PostgreSQL .NET 드라이버가 필요합니다. http://npgsql.projects.postgresql.org/ 에서 최신 버전을 다운로드할 수 있습니다. 최신 버전을 다운로드해서 사용자의 ASP.NET bin 폴더에 넣기만 하면 준비가 끝납니다.

예시 쿼리는 여러 래스터 함수를 함께 결합(combine)해서 특정 WGS84 경계 상자와 교차하는 모든 타일을 얻은 다음, ST_Union 함수를 통해 교차하는 타일들과 반환되는 모든 밴드를 통합(union)해서, ST_Transform 함수를 이용해 사용자가 지정한 투영으로 변환한 후, ST_AsPNG 함수로 결과값을 PNG 파일로 출력하는 방법을 보여줍니다.

이 예제는 C#으로 실행된다는 점만 제외하면 Section 5.3.1, “다른 래스터 함수와 함께 ST_AsPNG 를 이용해서 PHP 예제를 출력하기” 예제와 동일합니다.

```
http://mywebserver/test_raster.php?srid=2249
```

를 통해 메사추세츠 주의 피트 단위 항공사진의 래스터 이미지를 얻은 후, 다음 쿼리를 호출하십시오.

```
-- web.config 접속 문자열 부분 --
<connectionStrings>
   <add name="DSN"
      connectionString="server=localhost;database=mydb;Port=5432;User
Id=myuser;password=mypwd"/>
</connectionStrings
>
// TestRaster.ashx 용 코드
<%@ WebHandler Language="C#" Class="TestRaster" %>
using System;
using System.Data;
using System.Web;
using Npgsql;

public class TestRaster : IHttpHandler
{
      public void ProcessRequest(HttpContext context)
      {

            context.Response.ContentType = "image/png";
            context.Response.BinaryWrite(GetResults(context));
```

```
	}

	public bool IsReusable {
		get { return false; }
	}

	public byte[] GetResults(HttpContext context)
	{
		byte[] result = null;
		NpgsqlCommand command;
		string sql = null;
		int input_srid = 26986;
	try {
			using (NpgsqlConnection conn = new
NpgsqlConnection(System.Configuration.ConfigurationManager.ConnectionString
s["DSN"].ConnectionString)) {
				conn.Open();

		if (context.Request["srid"] != null)
		{
		   input_srid = Convert.ToInt32(context.Request["srid"]);
		}
		sql = @"SELECT ST_AsPNG(
				ST_Transform(
					ST_AddBand(
				   ST_Union(rast,1),
ARRAY[ST_Union(rast,2),ST_Union(rast,3)])
					,:input_srid) ) As new_rast
			FROM aerials.boston
				WHERE
				   ST_Intersects(rast,
				   ST_Transform(ST_MakeEnvelope(-71.1217,  42.227,
-71.1210, 42.218,4326),26986) )";
				command = new NpgsqlCommand(sql, conn);
		command.Parameters.Add(new NpgsqlParameter("input_srid",
input_srid));


				result = (byte[]) command.ExecuteScalar();
		conn.Close();
			}

		}
	catch (Exception ex)
	{
	   result = null;
	   context.Response.Write(ex.Message.Trim());
	}
		return result;
	}
}
```

5.3.3. 래스터 쿼리를 이미지 파일로 출력하는 Java 콘솔 응용 프로그램

이미지 한 장을 반환해서 지정된 파일로 출력하는 쿼리를 이용하는 단순한 Java 콘솔 응용 프로그램입니다.

http://jdbc.postgresql.org/download.html 에서 최신 PostgreSQL JDBC 드라이버를 다운로드할 수 있습니다.

다음과 같은 명령어를 통해 예시 코드를 컴파일할 수 있습니다:

```
set env CLASSPATH .:..\postgresql-9.0-801.jdbc4.jar
javac SaveQueryImage.java
jar cfm SaveQueryImage.jar Manifest.txt *.class
```

다음과 같은 명령행을 이용해서 컴파일한 코드를 호출하십시오:

```
java -jar SaveQueryImage.jar "SELECT
ST_AsPNG(ST_AsRaster(ST_Buffer(ST_Point(1,5),10, 'quad_segs=2'),150, 150,
'8BUI',100));" "test.png"
```

```
-- Manifest.txt --
Class-Path: postgresql-9.0-801.jdbc4.jar
Main-Class: SaveQueryImage
// SaveQueryImage.java 용 코드
import java.sql.Connection;
import java.sql.SQLException;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.io.*;

public class SaveQueryImage {
  public static void main(String[] argv) {
     System.out.println("Checking    if    Driver    is    registered    with
DriverManager.");

     try {
       //java.sql.DriverManager.registerDriver                              (new
org.postgresql.Driver());
       Class.forName("org.postgresql.Driver");
     }
     catch (ClassNotFoundException cnfe) {
       System.out.println("Couldn't find the driver!");
       cnfe.printStackTrace();
       System.exit(1);
     }

     Connection conn = null;

     try {
       conn                                                                  =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/mydb","myuser
", "mypwd");
       conn.setAutoCommit(false);

       PreparedStatement sGetImg = conn.prepareStatement(argv[0]);

       ResultSet rs = sGetImg.executeQuery();

             FileOutputStream fout;
             try
             {
                   rs.next();
                   /** 사용자가 요청한 파일명으로 출력 **/
                   fout = new FileOutputStream(new File(argv[1]) );
                   fout.write(rs.getBytes(1));
                   fout.close();
             }
             catch(Exception e)
```

```
            {
                  System.out.println("Can't create file");
                  e.printStackTrace();
            }

      rs.close();
            sGetImg.close();
      conn.close();
     }
     catch (SQLException se) {
      System.out.println("Couldn't connect: print out a stack trace and
exit.");
      se.printStackTrace();
      System.exit(1);
     }
  }
}
```

5.3.4. PLPython 을 이용해서 SQL 을 통해 이미지를 덤프하기

서버 디렉터리에 각 레코드를 파일로 생성하는 PLPython 저장 함수입니다. PLPython 을 설치해야 합니다. PLPythonu 와 PLPythonu3u 모두 잘 작동할 것입니다.

```
CREATE OR REPLACE FUNCTION write_file (param_bytes bytea, param_filepath
text)
RETURNS text
AS $$
f = open(param_filepath, 'wb+')
f.write(param_bytes)
return param_filepath
$$ LANGUAGE plpythonu;
```

```
-- PostgreSQL 서버에 여러 크기로 이미지 5 장을 작성합니다.
-- PostgreSQL 데몬(daemon) 계정이 폴더의 쓰기 권한을 가져야 한다는 점에 주의하십시오.
-- 이 쿼리는 생성된 파일명을 화면에 출력합니다;
 SELECT write_file(ST_AsPNG(
       ST_AsRaster(ST_Buffer(ST_Point(1,5),j*5,       'quad_segs=2'),150*j,
150*j, '8BUI',100)),
        'C:/temp/slices'|| j || '.png')
        FROM generate_series(1,5) As j;

    write_file
----------------------
 C:/temp/slices1.png
 C:/temp/slices2.png
 C:/temp/slices3.png
 C:/temp/slices4.png
 C:/temp/slices5.png
```

5.3.5. PSQL 을 이용해서 래스터 출력하기

안타깝게도 PSQL 은 바이너리 출력 작업에 쉽게 사용할 수 있는 내장 기능성을 가지고 있지 않습니다. PostgreSQL 의 레거시 대용량 객체 지원에 어느 정도 숟가락만 얹는 꼼수에 가깝습니다. PSQL 을 사용하려면, 먼저 사용자 데이터베이스에 연결된 사용자의 PSQL 명령행을 실행하십시오.

파이썬 접근법과는 달리, 이 접근법은 사용자의 로컬 시스템에 파일을 생성합니다.

```
SELECT oid, lowrite(lo_open(oid, 131072), png) As num_bytes
 FROM
 ( VALUES (lo_create(0),
   ST_AsPNG( (SELECT rast FROM aerials.boston WHERE rid=1) )
  ) ) As v(oid,png);
-- 다음과 비슷하게 출력될 것입니다. --
   oid   | num_bytes
---------+-----------
 2630819 |    74860

-- 그 다음 oid를 기억하고 c:/test.png를 사용자의
-- 로컬 시스템 파일 경로 위치로 대체하십시오.
 \lo_export 2630819 'C:/temp/aerial_samp.png'

-- 이 구문은 DB에 있는 대용량 객체 저장소에서 파일을 삭제합니다.
SELECT lo_unlink(2630819);
```

Chapter 6.PostGIS 도형 활용: 응용 프로그램 빌드

6.1. MapServer 활용

미네소타 맵서버(Minnesota MapServer)는 OpenGIS 웹 매핑 서버 사양서를 준수하는 인터넷 웹 매핑 서버입니다.

- MapServer 홈페이지는 http://mapserver.org 입니다.
- http://www.opengeospatial.org/standards/wms 에서 OpenGIS 웹 맵 사양서를 찾아볼 수 있습니다.

6.1.1. 기본 활용

MapServer 와 함께 PostGIS 를 사용하려면 MapServer 설정법을 알아야 하는데, MapServer 설정은 이 문서의 범위를 벗어납니다. 이 단원에서는 특정 PostGIS 문제점 및 설정 상세 정보를 다룰 것입니다.

MapServer 와 함께 PostGIS 를 사용하려면 다음 프로그램이 필요합니다:

- PostGIS 0.6 이상 버전
- MapServer 3.5 이상 버전

MapServer 는 다른 어떤 PostgreSQL 클라이언트와도 동일한 방식으로 -- libpq 인터페이스를 이용해서 PostGIS/PostgreSQL 데이터에 접근합니다. 즉 PostGIS 서버에 네트워크 연결된 어떤 컴퓨터에라도 MapServer 를 설치할 수 있고, PostGIS 를 데이터 소스로 이용할 수 있다는 뜻입니다. 두 시스템 간의 연결이 빠를수록 좋습니다.

1. "--with-postgis" 설정 옵션을 포함하는, 사용자가 원하는 옵션으로 MapServer 를 컴파일하고 설치하십시오.
2. 사용자 MapServer 의 맵 파일 안에 PostGIS 레이어를 추가하십시오. 다음은 그 예시입니다.

```
LAYER
  CONNECTIONTYPE postgis
  NAME "widehighways"
  # 원격 공간 데이터베이스에 연결
  CONNECTION "user=dbuser dbname=gisdatabase host=bigserver"
  PROCESSING "CLOSE_CONNECTION=DEFER"
  # 'roads' 테이블의 'geom' 열에서 라인을 획득
  DATA "geom from roads using srid=4326 using unique gid"
  STATUS ON
  TYPE LINE
  # 범위 내부의 라인 가운데 넓은 고속도로만 렌더링
```

```
  FILTER "type = 'highway' and numlanes >= 4"
  CLASS
    # 초고속도로를 2 픽셀 너비의 밝은 색으로
    EXPRESSION ([numlanes] >= 6)
    STYLE
      COLOR 255 22 22
      WIDTH 2
    END
  END
  CLASS
    # 다른 모든 도로를 1 픽셀 너비의 어두운 색으로
    EXPRESSION ([numlanes] < 6)
    STYLE
      COLOR 205 92 82
    END
  END
END
```

이 예시에서, PostGIS 에 특화된 지시자(directive)는 다음과 같습니다:

CONNECTIONTYPE PostGIS 레이어의 경우, 언제나 "postgis"입니다.

CONNECTION 다음과 같은 표준 키 및 값의 집합인 '연결 스트링(connection string)'이 데이터베이스 연결을 관장합니다(기본값은 <>로 감싸여 있습니다).
user=<username> password=<password> dbname=<username> hostname=<server> port=<5432>
비어 있는 연결 스트링도 여전히 유효하며, 어떤 키/값 쌍이라도 생략할 수 있습니다. 최소한, 연결에 필요한 데이터베이스 명 및 사용자명은 보통 제공하기 바랍니다.

DATA 이 파라미터의 서식은 유일한 <primary key>"를 이용한 "<geocolumn> from <tablename> using srid=<srid>입니다. 이때 열은 맵에 렌더링될 공간 열을 말하고, SRID 는 열이 사용하는 SRID 이며 기본 키는 테이블 기본 키(또는 다른 어떤 유일값을 담은, 인덱스를 가진 열)입니다.
"using srid" 및 "using unique" 절(clause)을 생략할 수 있으며, 이런 경우 가능하다면 MapServer 가 자동적으로 정확한 값을 알아낼 것입니다. 그러나 각 맵을 그리기 위해 서버 상에서 어느 정도의 추가 쿼리를 실행하는 일을 감당해야 할 것입니다.

PROCESSING 기존 연결을 끊지 않고 재사용하는 복수의 레이어가 있을 경우 CLOSE_CONNECTION=DEFER 절을 삽입하면 속도가 향상됩니다. 더 자세한 설명이 필요한 경우 MapServer PostGIS Performance Tips 를 참조하십시오.

FILTER 필터란 일반적으로 SQL 쿼리 안에 있는 "WHERE" 키워드를 따르는 논리에 상응하는 유효한 SQL 스트링이어야 합니다. 따라서, 예를 들어 6 개 이상 차선을 가진 도로만을 렌더링하려면 "num_lanes >= 6"라는 필터를 사용하십시오.

3. 사용자 공간 데이터베이스 안에 사용자가 그리게 될 모든 레이어에 대해 공간(GiST) 인덱스가 빌드되어 있는지 확인하십시오.

```
CREATE INDEX [indexname] ON [tablename] USING GIST
( [geometrycolumn] );
```

4. MapServer 를 이용해서 사용자 레이어를 쿼리하게 될 경우, 사용자의 DATA 구문에 "using unique" 절을 삽입해야 할 것입니다.
쿼리 실행시 MapServer 는 각 공간 레코드에 대해 유일한 식별자를 요구하며, MapServer 의 PostGIS 모듈은 이 유일한 식별자를 제공하기 위해 사용자가 설정한 유일값을 이용합니다. 가장 좋은 방법은 테이블 기본 키를 쓰는 것입니다.

6.1.2. 자주 묻는 질문들

1. 제 맵 파일 안에 EXPRESSION 을 사용할 때, 제 테이블에 해당 값이 분명히 있는 데도 불구하고, 조건문이 항상 거짓을 반환합니다.

shapefile 과는 달리, 표현식(expression)에 참조되는 PostGIS 필드명은 소문자 여야 합니다.

```
EXPRESSION ([numlanes] >= 6)
```

2. 제 shapefile 에 사용하는 필터가 동일한 데이터를 가진 제 PostGIS 테이블에 대해서는 동작하지 않습니다.

shapefile 과는 달리, PostGIS 레이어 용 필터는 SQL 문법을 이용합니다(이 필터는 MapServer 에서 레이어를 그리기 위해 PostGIS 연결자가 생성하는 SQL 선언문에 추가됩니다).

```
FILTER "type = 'highway' and numlanes >= 4"
```

3. 제 PostGIS 레이어를 그리는 속도가 제 shapefile 레이어보다 훨씬 느린데, 정상인가요?

일반적으로, 주어진 맵에 더 많은 피처를 그릴수록 shapefile 보다 PostGIS 가 느려지게 됩니다. 상대적으로 적은 피처(수백 개)를 가진 맵의 경우, PostGIS 가 빠른 편입니다. 고밀도 피처(수천 개)를 가진 맵의 경우, 항상 PostGIS 가 더 느릴 것입니다.

그리기 작업 속도에서 상당한 문제가 있을 경우, 사용자 테이블에 공간 인덱스를 빌드하지 않았을 가능성이 있습니다.

```
postgis# CREATE INDEX geotable_gix ON geotable USING GIST ( geocolumn );
postgis# VACUUM ANALYZE;
```

4. 제 PostGIS 레이어는 잘 그려지는데, 쿼리 속도가 너무 느립니다. 뭐가 문제죠?

쿼리 속도가 빠르려면 사용자 공간 테이블에 유일 키가 있어야 하며 해당 유일 키에 대해 인덱스가 빌드되어 있어야 합니다.

사용자의 DATA 행에 USING UNIQUE 절을 삽입해서 MapServer 가 사용할 유일 키를 설정할 수 있습니다:

```
DATA "geom FROM geotable USING UNIQUE gid"
```

5. MapServer 레이어를 위한 소스로 (PostGIS 1.5 부터 지원되는) "지리형" 열을 사용할 수 있을까요?

네! MapServer 는 지리형 열을 언제나 SRID 4326 을 이용하는 도형 열과 동일하다고 인식합니다. 사용자의 DATA 구문에 "using srid=4326" 절을 삽입했는지만 확인하면 됩니다. 다른 모든 작업은 도형의 경우와 정확히 동일하게 이루어집니다.

```
DATA "geog FROM geogtable USING SRID=4326 USING UNIQUE gid"
```

6.1.3. 고급 활용

MapServer 가 좀 더 복잡한 쿼리의 결과를 이해하도록 돕기 위한 정보를 추가하는 데 USING 허위(pseudo) SQL 절을 이용합니다. 더 정확하게는, 뷰 또는 하위집합(subselect)을 (DATA 정의에서 "FROM" 오른쪽에 오는) 소스 테이블로 사용할 때, MapServer 가 각 행에 대한 유일한 식별자는 물론 테이블의 SRID 를 자동적으로 파악하기 어려워집니다. USING 절이 MapServer 에 이 두 가지 정보를 다음과 같이 제공할 수 있습니다:

```
DATA "geom FROM (
  SELECT
    table1.geom AS geom,
    table1.gid AS gid,
    table2.data AS data
  FROM table1
  LEFT JOIN table2
  ON table1.id = table2.id
) AS new_table USING UNIQUE gid USING SRID=4326"
```

USING UNIQUE <uniqueid> 맵 쿼리 실행시 어떤 행인지 식별하기 위해 MapServer 가 각 행에 대한 유일 ID 를 요구합니다. 일반적으로는 시스템 테이블에서 기본 키를 인식합니다. 하지만, 뷰 및 하위집합은 알려진 유일값 열을 자동적으로 갖추지 않습니다. MapServer 의 쿼리 기능성을 활용하고자 한다면, 사용자의 뷰 또는 하위집합이 유일값 열을 포함하는지 확인하고, USING UNIQUE 절을 통해 그 사실을 선언하십시오. 예를 들어, 이런 목적을 위해 명쾌하게 테이블의 기본 키를 선택할 수 있고, 또는 결과 출력물에서 유일할 것이 보장되는 다른 어떤 열이라도 선택할 수 있습니다.



"맵 쿼리하기"란 해당 위치의 정보를 구하기 위해 맵을 클릭하는 행위입니다. DATA 정의에 있는 SQL 쿼리를 통한 "맵 쿼리"와 착각하지 마십시오.

USING SRID=<srid> PostGIS 는, MapServer 에 정확한 데이터를 반환하기 위해, 도형이 어떤 공간 참조 시스템을 사용하는지 알아야 합니다. 일반적으로 PostGIS 데이터베이스에 있는 "geometry_columns" 테이블에서 이 정보를 찾을 수 있지만, 뷰 및 하위집합과 같이 실시간으로(on the fly) 생성된 테이블에서는 불가능합니다. 따라서 USING SRID= 옵션을 이용하면 DATA 정의 안에 정확한 SRID 를 설정할 수 있습니다.

6.1.4. 예제

간단한 예제로 시작해서 단계를 높여 가보도록 합시다. 다음 MapServer 레이어 정의를 살펴보십시오:

```
LAYER
  CONNECTIONTYPE postgis
  NAME "roads"
  CONNECTION "user=theuser password=thepass dbname=thedb host=theserver"
  DATA "geom from roads"
  STATUS ON
  TYPE LINE
  CLASS
    STYLE
      COLOR 0 0 0
    END
  END
```

```
END
```

이 레이어는 도로 테이블의 모든 도로 도형을 검정색 라인으로 표출할 것입니다.

이제 최소한 1:100,000 축척으로 확대하기 전까지는 고속도로만 보이도록 하려 한다고 해봅시다. 다음 두 레이어가 이런 효과를 낼 것입니다:

```
LAYER
  CONNECTIONTYPE postgis
  CONNECTION "user=theuser password=thepass dbname=thedb host=theserver"
  PROCESSING "CLOSE_CONNECTION=DEFER"
  DATA "geom from roads"
  MINSCALE 100000
  STATUS ON
  TYPE LINE
  FILTER "road_type = 'highway'"
  CLASS
    COLOR 0 0 0
  END
END
LAYER
  CONNECTIONTYPE postgis
  CONNECTION "user=theuser password=thepass dbname=thedb host=theserver"
  PROCESSING "CLOSE_CONNECTION=DEFER"
  DATA "geom from roads"
  MAXSCALE 100000
  STATUS ON
  TYPE LINE
  CLASSITEM road_type
  CLASS
    EXPRESSION "highway"
    STYLE
      WIDTH 2
      COLOR 255 0 0
    END
  END
  CLASS
    STYLE
      COLOR 0 0 0
    END
  END
END
```

축척이 1:100,000 을 초과할 경우 첫 번째 레이어를 사용해서 "highway" 유형의 도로만 검정색 라인으로 표출합니다. FILTER 옵션이 "highway" 유형의 도로만 표출되도록 합니다.

축척이 1:100,000 미만일 경우 두 번째 레이어를 사용해서 고속도로를 두 배 너비의 빨강색 라인으로, 다른 도로들을 일반적인 검정색 라인으로 표출 할 것입니다.

따라서, MapServer 의 기능성만을 이용해서 재미있는 작업을 두어 개 해보았습니다만, DATA SQL 선언문은 단순한 형식을 유지하고 있습니다. 도로의 명칭이 (어떤 이유로든) 또다른 테이블에 저장되어 있다고 가정하면, 도로명을 읽어 와 도로 라벨을 표출하려면 두 테이블을 연결해야 합니다.

```
LAYER
  CONNECTIONTYPE postgis
  CONNECTION "user=theuser password=thepass dbname=thedb host=theserver"
  DATA "geom FROM (SELECT roads.gid AS gid, roads.geom AS geom,
       road_names.name as name FROM roads LEFT JOIN road_names ON
       roads.road_name_id = road_names.road_name_id)
```

```
      AS named_roads USING UNIQUE gid USING SRID=4326"
  MAXSCALE 20000
  STATUS ON
  TYPE ANNOTATION
  LABELITEM name
  CLASS
    LABEL
      ANGLE auto
      SIZE 8
      COLOR 0 192 0
      TYPE truetype
      FONT arial
    END
  END
END
```

이 주석(annotation) 레이어는 축척이 1:20,000 이하일 경우 모든 도로에 초록색 라벨을 추가합니다. DATA 정의에 어떻게 SQL 연결(join)을 이용하는지도 보여주고 있습니다.

6.2. Java 클라이언트 (JDBC)

Java 클라이언트는 텍스트 표현식을 통해 직접 또는 PostGIS 번들에 포함되어 있는 JDBC 확장 프로그램을 이용해 PostgreSQL 데이터베이스 안에 있는 PostGIS "도형" 객체에 접근할 수 있습니다. 확장 프로그램 객체를 이용하려면, 사용자의 CLASSPATH 에 "postgresql.jar" JDBC 드라이버 패키지와 함께 "postgis.jar" 파일이 존재해야 합니다.

```
import java.sql.*;
import java.util.*;
import java.lang.*;
import org.postgis.*;

public class JavaGIS {

public static void main(String[] args) {

  java.sql.Connection conn;

  try {
    /*
    * JDBC 드라이버를 로드하고 연결을 확립합니다.
    */
    Class.forName("org.postgresql.Driver");
    String url = "jdbc:postgresql://localhost:5432/database";
    conn = DriverManager.getConnection(url, "postgres", "");
    /*
    * 도형 유형을 연결에 추가합니다. addDataType() 메소드를
    * 호출하기 전에 연결을 pgSQL 에 특화된 연결 실행에
    * 지정해야 한다는 점에 주의합니다.
    */

((org.postgresql.PGConnection)conn).addDataType("geometry",Class.forName("o
rg.postgis.PGgeometry"));

((org.postgresql.PGConnection)conn).addDataType("box3d",Class.forName("org.
postgis.PGbox3d"));
    /*
```

```
    * 선언문을 생성해서 선별 쿼리를 실행합니다.
    */
    Statement s = conn.createStatement();
    ResultSet r = s.executeQuery("select geom,id from geomtable");
    while( r.next() ) {
      /*
      * 도형을 객체로 얻어온 다음 해당 객체를 도형 유형에 지정합니다.
      * 도형을 출력합니다.
      */
      PGgeometry geom = (PGgeometry)r.getObject(1);
      int id = r.getInt(2);
      System.out.println("Row " + id + ":");
      System.out.println(geom.toString());
    }
    s.close();
    conn.close();
  }
catch( Exception e ) {
  e.printStackTrace();
  }
}
}
```

"PGgeometry" 객체는 포인트, 라인스트링, 폴리곤, 멀티폴리곤, 멀티라인스트링, 멀티폴리곤 등의 유형에 의존하는 특정 지형 도형 객체(추출 클래스 "도형"의 하위 클래스)를 담고 있는 래퍼(wrapper) 객체입니다.

```
PGgeometry geom = (PGgeometry)r.getObject(1);
if( geom.getType() == Geometry.POLYGON ) {
  Polygon pl = (Polygon)geom.getGeometry();
  for( int r = 0; r < pl.numRings(); r++) {
    LinearRing rng = pl.getRing(r);
    System.out.println("Ring: " + r);
    for( int p = 0; p < rng.numPoints(); p++ ) {
      Point pt = rng.getPoint(p);
      System.out.println("Point: " + p);
      System.out.println(pt.toString());
    }
  }
}
```

확장 프로그램 객체를 위한 JavaDoc 은 기하학적 객체의 다양한 데이터 접근자(accessor) 함수에 대한 참조를 제공합니다.

6.3. C 클라이언트 (libpq)

6.3.1. 텍스트 커서

6.3.2. 바이너리 커서

Chapter 7.성능 향상 비법

7.1. 대용량 도형을 담은 저용량 테이블

7.1.1. 문제점 설명

현재 PostgreSQL (8.0 포함) 버전들의 쿼리 최적화기(optimizer)에는 TOAST 테이블과 관련된 약점이 있습니다. TOAST 테이블은 일반 데이터 페이지에 적합하지 않은 (긴 텍스트, 큰 이미지 또는 수많은 꼭짓점을 가진 복잡 도형 같은) 대용량(데이터 크기 관점에서) 값을 저장하는 데 쓰이는 일종의 "확장 공간(extension room)"입니다. 자세한 정보는 the PostgreSQL Documentation for TOAST 를 참고하십시오.

상당히 대용량의 도형을 가진 테이블이지만, (전체 유럽 국가들의 고해상도 국경을 담고 있는 테이블처럼) 도형 행이 그렇게 많지 않을 경우 문제가 드러납니다. 이때 테이블 자체는 저용량이지만, 많은 TOAST 공간을 차지합니다. 이번 예제의 경우, 테이블 자체는 약 80 행을 담고 있고 데이터 페이지 3 장만 사용하지만, TOAST 테이블은 8,225 페이지를 차지합니다.

이 테이블의 행 가운데 몇 행과만 일치하는 경계 상자를 검색하기 위한 도형 연산자 &&를 이용하는 쿼리를 전송해봅시다. 이때 쿼리 최적화기는 해당 테이블이 데이터 페이지 3 장과 80 행만 가지고 있다고 판단합니다. 이런 저용량 테이블에는 인덱스보다 순차 스캔이 훨씬 빠를 거라고 추정할 것입니다. 따라서 GiST 인덱스를 무시하기로 합니다. 일반적인 경우라면 이 추정이 맞습니다. 그러나 이번 예제의 경우, && 연산자가 경계 상자를 비교하기 위해 디스크에서 모든 도형을 불러와야 하기 때문에, 결국 모든 TOAST 페이지도 읽어야 합니다.

사용자 시스템에 이 버그가 있는지 확인하려면, "EXPLAIN ANALYZE" PostgreSQL 명령어를 실행하십시오. 자세한 정보 및 기술적인 세부 사항을 알고 싶다면, PostgreSQL 성능 메일링 리스트에 있는 다음 스레드를 읽어볼 수 있습니다: http://archives.postgresql.org/pgsql-performance/2005-02/msg00030.php

7.1.2. 해결 방법

PostgreSQL 개발자들은 쿼리 추정이 TOAST 를 인식하도록 만들어 이 문제를 해결하려 하고 있습니다. 현 시점에서는, 다음 두 가지 해결 방법이 있습니다:

첫 번째 해결책은 쿼리 설계자가 인덱스를 사용하도록 강제하는 방법입니다. 쿼리를 전송하기 전에 서버로 "SET enable_seqscan TO off;"를 전송하십시오. 이렇게 하면 기본적으로 쿼리 설계자가 순차 스캔을 사용할 수 있는 경우에도 사용하지 않도록 강제합니다. 따라서 평상시처럼 GiST 인덱스를 이용할 것입니다. 그러나 이 플래그를 매번 연결할 때마다 설정해야 하고 다른 상황에서 쿼리 설계자가 잘못 추정하도록 하기 때문에, 쿼리가 끝난 후에 "SET enable_seqscan TO on;"을 전송해야 합니다.

두 번째 해결책은 쿼리 설계자가 추정하는 만큼 순차 스캔 속도를 높이는 방법입니다. 경계 상자를 "캐시"하는 추가 열을 생성한 다음, 이 추가 열을 대상으로 일치 여부를 쿼리하면 됩니다. 이번 예제의 경우, 해당 명령어는 다음과 같습니다:

```
SELECT
AddGeometryColumn('myschema','mytable','bbox','4326','GEOMETRY','2');
UPDATE mytable SET bbox = ST_Envelope(ST_Force2D(the_geom));
```

이제 geom_column 대신 bbox 에 대해 && 연산자를 이용하도록 다음과 같이 사용자 쿼리를 변경하십시오:

```
SELECT geom_column
FROM mytable
WHERE bbox && ST_SetSRID('BOX3D(0 0,1 1)'::box3d,4326);
```

물론, mytable 의 열을 변경하거나 추가할 경우, bbox 의 "일관성"을 유지해야 합니다. 일관성을 유지시키는 가장 명료한 방법은 트리거(trigger)일 것입니다. 물론 사용자 응용 프로그램을 수정해서 bbox 열의 정보를 항상 최신 상태로 유지하도록 하거나, 모든 수정 작업 후 매번 앞에서 나온 UPDATE 쿼리를 실행할 수도 있습니다.

7.2. 도형 인덱스에 대한 클러스터 작업

레코드 대부분이 읽기 전용이고, 쿼리의 대다수가 단일 인덱스를 이용하는 테이블의 경우, PostgreSQL 은 CLUSTER 명령어를 제공합니다. 이 명령어는 모든 데이터 행을 인덱스 기준과 동일한 순서로 물리적으로 재정렬합니다. 이렇게 하면 성능 향상 관련 두 가지 장점이 나타납니다. 첫째, 인덱스 범위 스캔을 하는 경우 데이터 테이블에 대한 요청 횟수가 극단적으로 줄어듭니다. 둘째, 사용자가 작업중인 집합이 인덱스 상의 조금 작은 간격에 집중하고 있을 경우 데이터 행이 더 적은 데이터 페이지를 따라 퍼져 있기 때문에 더 효율적인 캐시 작업이 가능합니다. (이 시점에서 PostgreSQL 지침서에 있는 CLUSTER 명령어 문서를 읽어보는 편이 좋습니다.)

하지만, 현재 PostgreSQL 은 PostGIS GiST 인덱스에 대한 클러스터 작업을 허용하지 않습니다. 왜냐하면 GiST 인덱스가 NULL 값을 단순히 무시하기 때문에, 다음과 같은 오류 메시지를 보게 될 것입니다:

```
lwgeom=# CLUSTER my_geom_index ON my_table;
ERROR: cannot cluster when index access method does not handle null values
HINT: You may be able to work around this by marking column "the_geom" NOT
NULL.
```

HINT 메시지의 내용대로, 테이블에 "not null" 제약조건을 추가하면 이 결점을 피해 갈 수 있습니다:

```
lwgeom=# ALTER TABLE my_table ALTER COLUMN the_geom SET not null;
ALTER TABLE
```

물론, 사용자 도형 열 안에 실제로 NULL 값이 필요한 경우 이 방법을 쓸 수는 없습니다. 또한 제약조건을 추가하려면 앞의 메소드를 이용해야 합니다. "ALTER TABLE blubb ADD CHECK (geometry is not null);"처럼 CHECK 제약조건을 사용하는 방법은 먹히지 않을 겁니다.

7.3. 차원변환 피하기

때때로 사용자 테이블에 3D 또는 4D 데이터가 있지만, 항상 2D 도형만을 출력하는 OpenGIS 준수 ST_AsText() 또는 ST_AsBinary() 함수를 통해서만 접근하는 경우가 있을 수 있습니다. 이런 경우 내부적으로 ST_Force2D() 함수를 호출하는데, 대용량 도형의 경우 이 함수는 시스템의 자원을 상당히 잡아먹습니다. 이런 자원 낭비를 피하려면, 미리 그 추가적인 차원을 완전히 삭제하는 편이 효율적일 수 있습니다.

```
UPDATE mytable SET the_geom = ST_Force_2d(the_geom);
VACUUM FULL ANALYZE mytable;
```

AddGeometryColumn() 함수를 통해 사용자 도형 열을 추가했을 경우 도형 차원에 제약조건이 걸려 있다는 사실을 주의하십시오. 이를 우회하려면 제약조건을 삭제해야 합니다. geometry_columns 테이블 내부 항목을 업데이트한 다음 제약조건을 다시 생성하는 걸 잊지 마십시오.

대용량 테이블의 경우, 사용자의 기본 키 또는 또다른 사용 가능한 기준과 함께 WHERE 절을 이용해서 테이블의 일부분만 업데이트하도록 제한한 다음 업데이트마다 간단히 "VACUUM;"을 실행하는 방식으로 이 업데이트를 작은 부분으로 나누는 편이 좋을 수 있습니다. 이렇게 하면 필요한 임시 디스크 공간이 급격히 줄어듭니다. 또한 몇 종류의 차원이 섞인 도형들을 가지고 있을 경우, "WHERE dimension(the_geom)>2"로 업데이트를 제한하면 이미 2D 인 도형을 다시 작성하는 일을 건너뛸 수 있습니다.

7.4. 사용자 설정 세부 조정

이 단원에서 소개하는 비법은 케빈 뉴펠드(Kevin Neufeld)가 FOSS4G 2007 콘퍼런스에서 발표했던 "PostGIS 파워유저를 위한 비법(Tips for the PostGIS Power User)"에서 발췌한 것입니다. 사용자의 PostGIS 사용 방식에 따라 (예를 들어 정적 데이터 및 복잡 분석이냐 자주 업데이트되는 데이터 및 복수의 사용자냐 등) 이렇게 변경하면 사용자 쿼리의 속도를 상당히 향상시킬 수 있습니다.

더 많은 비법 (및 보기 쉬운 서식)을 원한다면 다음 URL 에 원래 발표 자료가 있습니다:

http://2007.foss4g.org/presentations/view.php?abstract_id=117

7.4.1. 시작

다음 설정들은 postgresql.conf 파일 안에 설정됩니다:

checkpoint_segments:
https://www.postgresql.org/docs/current/static/runtime-config-wal.html#GUC-CHECKPOINT-SEGMENTS)

- 자동 WAL 체크포인트 사이의 로그 파일 조각의 최대 개수로, 기본값은 3 입니다.
- 작성 활동이 많은 데이터베이스의 경우 최소 10 또는 30 으로 설정하고, 데이터베이스에 대용량 로드하는 경우 더 큰 숫자로 설정하십시오. 이 문제에 대해 읽어볼 만한 다른 글을 Greg Smith: Checkpoint and Background writer 에서 볼 수 있습니다.
- xlog 를 개별 디스크 장치에 저장할 수도 있습니다.

constraint_exclusion:
http://www.postgresql.org/docs/current/static/runtime-config-wal.html#GUC-CHECKPOINT-SEGMENTS

- 기본값은 off 입니다. (PostgreSQL 8.4 미만 버전의 경우고, 8.4 이상 버전의 경우 partition 으로 설정되어 있습니다.)
- 일반적으로 테이블을 분할하는 경우 쓰입니다. PostgreSQL 8.4 미만 버전을 실행하고 있다면 쿼리 설계자가 원하는 만큼 최적화하도록 "on"으로 설정하십시오. PostgreSQL 8.4 버전부터, 이 옵션의 기본값이 "partition"으로 설정되어 PostgreSQL 8.4 이후 버전에 이상적으로 바뀌었습니다. 테이블이 상속된 계층형일 경우 쿼리 설계자가 제약조건을 검토해야 할 테이블만 분석하도록 강제하고 다른 경우엔 별다른 대가를 치르지 않기 때문입니다.

shared_buffers:
http://www.postgresql.org/docs/current/static/runtime-config-resource.html#GUC-SHARED-BUFFERS

- 기본값은 최대 32MB 입니다.
- 사용할 수 있는 RAM 용량의 1/3 에서 3/4 까지 설정하십시오.

7.4.2. 런타임

work_mem (정렬 연산 및 복잡 쿼리에 쓰이는 메모리)

- 기본값은 1MB 입니다.
- 대용량 DB, 복잡 쿼리, 많은 RAM 의 경우 상향 조정하십시오.
- 동시 사용자가 많거나 RAM 이 적을 경우 하향 조정하십시오.
- RAM 이 많고 개발자가 적을 경우:

```
SET work_mem TO 1200000;
```

maintenance_work_mem (VACUUM, CREATE INDEX 등에 쓰이는 메모리)

- 기본값은 16MB 입니다.
- 기본값이 너무 낮습니다. 메모리 스왑 도중 I/O 병목 현상이 일어나거나 객체에 랙이 걸립니다.
- RAM 이 많은 제품 서버(production server)라면 32MB 에서 256MB 까지 추천하지만, 동시 사용자 수에 따라 다릅니다. RAM 이 많고 개발자가 적을 경우:

```
SET maintainence_work_mem TO 1200000;
```

Chapter 8.PostGIS 참조 문서

이 단원에서 주어진 함수들은 PostGIS 사용자가 필요로 하는 함수들입니다. 일반 사용자에겐 쓸모없는, PostGIS 객체에 필요한 다른 지원 함수들도 있습니다.



PostGIS 는 기존 명명 규약에서 SQL-MM-중심 명명 규약으로 전환하기 시작했습니다. 그 결과, 사용자가 잘 알고 좋아하는 대부분의 함수명이 표준 공간 유형(ST) 접두사를 사용한 명칭으로 변경되었습니다. 이전 함수도 계속 사용할 수 있지만, 이 문서에 들어 있지는 않습니다. 대신 업데이트된 함수들이 이전 함수들의 자리를 차지하고 있습니다. 향후 배포판에서는 이 문서에 없는 ST_ 아닌 함수들이 더 이상 사용되지 않고 제외될 예정이므로 이전 함수의 사용을 중단해주십시오.

8.1. PostgreSQL PostGIS 도형/지리형/상자 유형

이 단원에서 PostGIS 이 설치한 PostgreSQL 데이터 유형을 소개합니다. 사용자 자신의 함수를 설계할 때 특히 중요한 이 유형들의 형변환 습성(cast behavior)을 설명한다는 점에 주의하십시오.

한 유형이 다른 유형으로 강제 형변환될 때 캐스트(cast)된다고 합니다. PostgreSQL 은 사용자 정의 유형을 위한 형변환 습성 및 형변환에 사용되는 함수들을 정의할 수 있도록 한다는 점에서 대부분의 데이터베이스보다 특별합니다. 자동 형변환을 설정할 수 있는 경우가 있는데, 오직 otherfootype 을 사용할 때만 동작하는 함수의 해당 위치에 자동 형변환이 있어서 CAST(otherfootype 으로서의 myfoo) 또는 myfoo::otherfootype 을 할 필요가 없는 경우입니다.

오버로드된 함수가 있을 경우, 자동 형변환 습성에 의존하는 경우의 위험성을 알 수 있습니다. box2d 를 사용하는 함수와 box3d 를 사용하지만 도형은 사용하지 않는 함수가 있다고 해봅시다. 이때 두 함수 모두 도형과 함께 사용하기 좋습니다. 도형이 두 함수 모두에 자동 형변환을 지원하기 때문입니다. 결국 모호한 함수 오류가 발생할 것입니다. 강제로 PostgreSQL 가 선택하게 하려면, CAST(mygeom As box3d) 또는 mygeom::box3d 를 하십시오.

적어도 PostgreSQL 8.3 기준으로 (아마도 알려지지 않은 신비한 유형 덕분에) 모든 객체를 텍스트로 형변환할 수 있습니다. 따라서 객체를 텍스트로 형변환하기 위해 별도로 정의된 캐스트는 없습니다.

box2d — xmin, ymin, xmax, ymax 로 구성된 상자입니다. 도형을 둘러싼 2D 상자를 반환하는 데 주로 쓰입니다.

box3d — xmin, ymin, zmin, xmax, ymax, zmax 로 구성된 상자입니다. 도형 또는 도형 집합의 3D 범위를 반환하는 데 주로 쓰입니다.

geometry — 평면 공간 데이터 유형

geometry_dump — (도형 객체를 담고 있는) geom 과 (덤프된 객체 내부에 있는 도형의 위치를 담고 있는 1 차원 배열인) path[], 두 개의 필드를 가진 공간 데이터 유형입니다.

geography — 타원체 공간 데이터 유형

8.1.1. box2d

box2d — xmin, ymin, xmax, ymax 로 구성된 상자입니다. 도형을 둘러싼 2D 상자를 반환하는 데 주로 쓰입니다.

설명: box2d 는 도형 또는 도형 집합을 둘러싸는 상자를 표현하는 데 쓰이는 공간 데이터 유형입니다. PostGIS 1.4 이전 버전의 ST_Extent 가 box2d 를 반환할 것입니다.

8.1.2. box3d

box3d — xmin, ymin, zmin, xmax, ymax, zmax 로 구성된 상자입니다. 도형 또는 도형 집합의 3D 범위를 반환하는 데 주로 쓰입니다.

설명: box3d 는 도형 또는 도형 집합을 둘러싸는 상자를 표현하는 데 쓰이는 PostGIS 공간 데이터 유형입니다. ST_3DExtent 가 box3d 객체를 반환할 것입니다.

형변환 습성: 이 단원에서 이런 데이터 유형에 사용할 수 있는 자동 형변환은 물론 명시적인 형변환도 소개합니다.

변환 대상	습성
box	자동적
box2d	자동적
geometry	자동적

8.1.3. geometry

geometry — 평면 공간 데이터 유형

설명: 도형은 기초적인 PostGIS 공간 데이터 유형으로, 유클리드 좌표계에 있는 피처를 표현하는 데 쓰입니다.

형변환 습성: 이 단원에서 이런 데이터 유형에 사용할 수 있는 자동 형변환은 물론 명시적인 형변환도 소개합니다.

변환 대상	습성
Box	자동적
box2d	자동적
box3d	자동적
bytea	자동적
geography	자동적
text	자동적

참고: Section 4.1, "GIS 객체"

8.1.4. geometry_dump

geometry_dump — (도형 객체를 담고 있는) geom 과 (덤프된 객체 내부에 있는 도형의 위치를 담고 있는 1 차원 배열인) path[], 두 개의 필드를 가진 공간 데이터 유형입니다.

설명: geometry_dump 은 복합 데이터 유형으로, .geom 필드가 참조하는 도형 객체와 해당 요소를 찾기 위한, 덤프된 도형 내부에 있는 탐색 경로를 정의하는 path[] 1 차원 정수 배열(1 에서 시작하므로 첫 번째 요소를 얻으려면 path[1]을 사용)로 구성됩니다. ST_Dump* 계열 함수가 더 복잡한 도형을 구성 요소의 조각들로, 그리고 해당 조각들의 위치로 분해하기 위해 이 유형을 출력물 유형으로 이용합니다.

참고: Section 14.6, "PostGIS Geometry / Geography / Raster Dump Functions"

8.1.5. geography

geography — 타원체 공간 데이터 유형

설명: 지리형은 지구 곡면(round-earth) 좌표계에 있는 피처를 표현하는 데 쓰이는 공간 데이터 유형입니다.

형변환 습성: 이 단원에서 이런 데이터 유형에 사용할 수 있는 자동 형변환은 물론 명시적인 형변환도 소개합니다.

변환 대상	습성
Geometry	명시적

참고: Section 14.4, "PostGIS Geography Support Functions", Section 4.2, "PostGIS 지리형 유형"

8.2. PostGIS GUC(Grand Unified Custom Variable)

이 단원에서 사용자 지정 PostGIS GUC(Grand Unified Custom Variable)에 대해 설명합니다. 전체(global), 데이터베이스, 세션, 또는 상호처리(transaction) 단계에서 GUC 를 설정할 수 있습니다. 전체 또는 데이터베이스 단계에서 설정하는 편이 가장 좋습니다.

postgis.backend — GEOS 와 SFCGAL 양쪽의 겹치는 함수를 서비스하는 백엔드입니다. geos 또는 sfcgal 을 선택할 수 있는데, 기본값은 geos 입니다.

postgis.gdal_datapath — GDAL 의 GDAL_DATA 옵션의 값을 할당하는 설정 옵션입니다. 설정하지 않을 경우, 환경적으로 설정된 GDAL_DATA 변수를 사용합니다.

postgis.gdal_enabled_drivers — PostGIS 환경에서 사용할 수 있는 GDAL 드라이버를 설정하는 설정 옵션입니다. GDAL 설정 변수 GDAL_SKIP 에 영향을 미칩니다.

postgis.enable_outdb_rasters — DB 외부 래스터 밴드에 접근할 수 있게 해주는 불 설정 옵션입니다.

8.2.1. postgis.backend

postgis.backend — GEOS 와 SFCGAL 양쪽의 겹치는 함수를 서비스하는 백엔드입니다. geos 또는 sfcgal 을 선택할 수 있는데, 기본값은 geos 입니다.

설명: PostGIS 를 sfcgal 지원과 함께 컴파일했을 경우에만 GUC 가 유의미해집니다. GEOS 와 SFCGAL 의 동일한 함수명을 가진 함수를 위해 이용되는 geos 백엔드가 기본값입니다. 이 변수를 변경하면 기본값을 무시하고 요청을 서비스하는 백엔드를 sfcgal 로 바꿀 수 있습니다.

2.1.0 버전부터 사용할 수 있습니다.

예시: 연결이 유지될 동안만 백엔드를 설정해봅시다.

```
set postgis.backend = sfcgal;
```

데이터베이스와의 새 연결에 백엔드를 설정해봅시다.

```
ALTER DATABASE mygisdb SET postgis.backend = sfcgal;
```

참고: Section 8.10, "SFCGAL 함수"

8.2.2. postgis.gdal_datapath

postgis.gdal_datapath — GDAL 의 GDAL_DATA 옵션의 값을 할당하는 설정 옵션입니다. 설정하지 않을 경우, 환경적으로 설정된 GDAL_DATA 변수를 사용합니다.

설명: GDAL 의 GDAL_DATA 옵션의 값을 설정하기 위한 PostgreSQL GUC 변수입니다. postgis.gdal_datapath 값은 GDAL 의 데이터 파일을 가리키는 완전한 물리적 경로여야 합니다.

이 설정 옵션은 GDAL 의 데이터 파일 경로가 쉽게 변경할 수 없도록 기록(hard-coded)되지 않은 윈도우 플랫폼에서 가장 쓸모가 있습니다. GDAL 의 데이터 파일이 GDAL 의 예상 경로에 없을 때 이 옵션을 설정해야 합니다.



PostgreSQL 의 설정 파일 postgresql.conf 안에서 이 옵션을 설정할 수 있습니다. 또 연결이나 상호처리 단계에서도 설정할 수 있습니다.

2.2.0 버전부터 사용할 수 있습니다.



GDAL 의 설정 옵션(http://trac.osgeo.org/gdal/wiki/ConfigOptions) 페이지에서 GDAL_DATA 에 대한 추가 정보를 찾아볼 수 있습니다.

예시: postgis.gdal_datapath 를 설정한 다음 재설정해보십시오.

```
SET postgis.gdal_datapath TO '/usr/local/share/gdal.hidden';
SET postgis.gdal_datapath TO default;
```

특정 데이터베이스를 대상으로 윈도우 상에서 설정해보십시오.

```
ALTER DATABASE gisdb
SET postgis.gdal_datapath = 'C:/Program Files/PostgreSQL/9.3/gdal-data';
```

참고: PostGIS_GDAL_Version, ST_Transform

8.2.3. postgis.gdal_enabled_drivers

postgis.gdal_enabled_drivers — PostGIS 환경에서 사용할 수 있는 GDAL 드라이버를 설정하는 설정 옵션입니다. GDAL 설정 변수 GDAL_SKIP 에 영향을 미칩니다.

설명: PostGIS 환경에서 사용할 수 있는 GDAL 드라이버를 설정하는 설정 옵션입니다. GDAL 설정 변수 GDAL_SKIP 에 영향을 미칩니다. PostgreSQL 의 설정 파일 postgresql.conf 안에서 이 옵션을 설정할 수 있습니다. 또 연결이나 상호처리 단계에서도 설정할 수 있습니다.

PostgreSQL 을 구동시키는 과정에서 사용할 수 있는 드라이버 목록과 함께 환경 변수 POSTGIS_GDAL_ENABLED_DRIVERS 를 패스(pass)시켜 postgis.gdal_enabled_drivers 의 초기값을 설정할 수도 있습니다.

드라이버의 축약명 또는 코드를 통해 활성화된 GDAL 특화 드라이버를 지정할 수 있습니다. 드라이버의 축약명 또는 코드는 GDAL 래스터 형식 에서 찾을 수 있습니다. 각 드라이버 사이에 공백을 삽입하면 복수의 드라이버를 지정할 수 있습니다.



postgis.gdal_enabled_drivers 를 위해 사용할 수 있는 특별 코드가 세 개 있습니다. 이 코드들은 대소문자를 구분합니다.

- DISABLE_ALL 은 모든 GDAL 드라이버를 비활성화시킵니다. DISABLE_ALL 이 있을 경우, postgis.gdal_enabled_drivers 안에 있는 다른 모든 값을 무시합니다.
- ENABLE_ALL 은 모든 GDAL 드라이버를 활성화시킵니다.
- VSICURL 은 GDAL 의 /vsicurl/ 가상 파일 시스템을 활성화시킵니다.

postgis.gdal_enabled_drivers 가 DISABLE_ALL 로 성정됐을 경우, DB 외부 래스터, ST_FromGDALRaster(), ST_AsGDALRaster(), ST_AsTIFF(), ST_AsJPEG() 그리고 ST_AsPNG()를 사용하려 한다면 모두 오류 메시지를 보게 될 것입니다.



표준 PostGIS 설치시, postgis.gdal_enabled_drivers 는 DISABLE_ALL 로 설정됩니다.

2.2.0 버전부터 사용할 수 있습니다.

예시: postgis.gdal_enabled_drivers 를 설정한 다음 재설정해보십시오.

데이터베이스와의 모든 새 연결에 백엔드를 설정해봅시다.

```
ALTER DATABASE mygisdb SET postgis.gdal_enabled_drivers TO 'GTiff PNG
JPEG';
```

서버와의 모든 새 연결에 기본 활성화된 드라이버를 설정해봅시다. 슈퍼유저 권한 및 PostgreSQL 9.4 이상 버전이 필요합니다. 또 데이터베이스, 세션, 사용자 설정 단계에서 이 설정을 무시할 수 없습니다.

```
ALTER SYSTEM SET postgis.gdal_enabled_drivers TO 'GTiff PNG JPEG';
SELECT pg_reload_conf();

SET postgis.gdal_enabled_drivers TO 'GTiff PNG JPEG';
SET postgis.gdal_enabled_drivers = default;
```

모든 GDAL 드라이버를 활성화시키십시오.

```
SET postgis.gdal_enabled_drivers = 'ENABLE_ALL';
```

모든 GDAL 드라이버를 비활성화시키십시오.

```
SET postgis.gdal_enabled_drivers = 'DISABLE_ALL';
```

참고: ST_FromGDALRaster, ST_AsGDALRaster, ST_AsTIFF, ST_AsPNG, ST_AsJPEG, postgis.enable_outdb_rasters

8.2.4. postgis.enable_outdb_rasters

postgis.enable_outdb_rasters — DB 외부 래스터 밴드에 접근할 수 있게 해주는 불 설정 옵션입니다.

설명: DB 외부 래스터 밴드에 접근할 수 있게 해주는 불 설정 옵션입니다. PostgreSQL 의 설정 파일 postgresql.conf 안에서 이 옵션을 설정할 수 있습니다. 또 연결이나 상호처리 단계에서도 설정할 수 있습니다.

PostgreSQL 을 구동시키는 과정에서 0 이 아닌 값과 함께 환경 변수 POSTGIS_ENABLE_OUTDB_RASTERS 를 패스(pass)시켜 postgis.enable_outdb_rasters 의 초기값을 설정할 수도 있습니다.



postgis.enable_outdb_rasters 가 참이라 할지라도, GUC postgis.enable_outdb_rasters 가 접근 가능한 래스터 형식을 결정합니다.



표준 PostGIS 설치시, postgis.enable_outdb_rasters 는 거짓으로 설정됩니다.

2.2.0 버전부터 사용할 수 있습니다.

예시: postgis.enable_outdb_rasters 를 설정한 다음 재설정해보십시오.

```
SET postgis.enable_outdb_rasters TO True;
SET postgis.enable_outdb_rasters = default;
SET postgis.enable_outdb_rasters = True;
SET postgis.enable_outdb_rasters = False;
```

참고: postgis.gdal_enabled_drivers

8.3. 관리 함수

AddGeometryColumn — 기존 속성 테이블에 도형 열을 추가합니다. 기본적으로 제약 조건보다는 유형 변경자를 사용해서 정의합니다. 구 버전 확인 제약조건 기반 습성(behavior)을 얻으려면 use_typmod 에 대해 거짓을 패스합니다.

DropGeometryColumn — 공간 테이블에서 도형 열을 제거합니다.

DropGeometryTable — 테이블 및 geometry_columns 에 있는 테이블의 모든 참조를 삭제합니다.

PostGIS_Full_Version — 완전한 PostGIS 버전 및 빌드 설정 정보를 반환합니다.

PostGIS_GEOS_Version — GEOS 라이브러리의 버전 숫자를 반환합니다.

PostGIS_LibXML_Version — LibXML2 라이브러리의 버전 숫자를 반환합니다.

PostGIS_Lib_Build_Date — PostGIS 라이브러리의 빌드 날짜를 반환합니다.

PostGIS_Lib_Version — PostGIS 라이브러리의 버전 숫자를 반환합니다.

PostGIS_PROJ_Version — PROJ4 라이브러리의 버전 숫자를 반환합니다.

PostGIS_Scripts_Build_Date — PostGIS 스크립트의 빌드 날짜를 반환합니다.

PostGIS_Scripts_Installed — 이 데이터베이스에 설치된 PostGIS 스크립트의 버전을 반환합니다.

PostGIS_Scripts_Released — 설치된 PostGIS 라이브러리와 함께 출시된 postgis.sql 스크립트의 버전 숫자를 반환합니다.

PostGIS_Version — **PostGIS** 버전 숫자 및 컴파일 시간 옵션을 반환합니다.

Populate_Geometry_Columns — 도형 열이 유형 변경자로 정의됐거나 적절한 공간 제약조건을 가지고 있는지 확인합니다. 이들을 확인하면 도형 열이 geometry_columns 뷰에 올바르게 등록될 것입니다. 기본적으로 유형 변경자를 가지지 않는 모든 도형 열들을 유형 변경자를 가진 도형 열들로 변환시킬 것입니다. 구 버전 습성을 원한다면 use_typmod=false 으로 설정하십시오.

UpdateGeometrySRID — 도형 열, geometry_columns 메타데이터 그리고 SRID에 있는 모든 피처의 SRID를 업데이트합니다. 제약조건으로 강제됐을 경우, 제약조건도 새로운 SRID 제약조건으로 업데이트될 것입니다. 유형 정의에 따라 구 버전을 강제했다면, 유형 정의가 변경될 것입니다.

8.3.1. AddGeometryColumn

AddGeometryColumn — 기존 속성 테이블에 도형 열을 추가합니다. 기본적으로 제약 조건보다는 유형 변경자를 사용해서 정의합니다. 구 버전 확인 제약조건 기반 습성(behavior)을 얻으려면 use_typmod 에 대해 거짓을 패스합니다.

Synopsis

text **AddGeometryColumn**(varchar *table_name*, varchar *column_name*, integer *srid*, varchar *type*, integer *dimension*, boolean *use_typmod=true*);

text **AddGeometryColumn**(varchar *schema_name*, varchar *table_name*, varchar *column_name*, integer *srid*, varchar *type*, integer *dimension*, boolean *use_typmod=true*);

text **AddGeometryColumn**(varchar *catalog_name*, varchar *schema_name*, varchar *table_name*, varchar *column_name*, integer *srid*, varchar *type*, integer *dimension*, boolean *use_typmod=true*);

설명: 기존 속성 테이블에 도형 열을 추가합니다. schema_name 은 테이블 스키마의 명칭입니다. srid 는 SPATIAL_REF_SYS 테이블 안에 있는 항목을 참조하는 정수값이어야 합니다. type 은 도형 유형과 상응하는 문자열, 예를 들어 'POLYGON'이나 'MULTILINESTRING'이어야 합니다. 스키마 명칭이 없거나 (혹은 현재 search_path 에서는 안 보이거나) 지정된 SRID, 도형 유형, 차원 등이 유효하지 않은 경우 오류가 발생합니다.



변경 사항: 2.0.0 버전. geometry_columns 가 시스템 카탈로그를 읽어오는 뷰이기 때문에 이 함수는 더 이상 geometry_columns 를 업데이트하지 않습니다. 또 기본적으로 제약조건을 생성하지도 않지만, 그 대신 PostgreSQL 내장 유형 변경자 습성을 이용합니다. 따라서 예를 들어 이 함수로 WGS84 포인트 열을 빌드하는 작업을 다음과 같이 나타낼 수 있습니다: ALTER TABLE some_table ADD COLUMN geom geometry(Point,4326);
변경 사항: 2.0.0 버전. 제약조건의 구식 습성이 필요할 경우, 기본값인 use_typmod 를 이용하되, 거짓으로 설정하십시오.



변경 사항: 2.0.0 버전. 더 이상 뷰를 직접 geometry_columns 에 등록시킬 수는 없지만, 도형 typmod 테이블 도형을 기반으로 빌드된 , 그리고 래퍼 함수 없이 사용되는 뷰는 자신의 페어런트 테이블 열의 typmod 습성을 상속하기 때문에 스스로를 정확히 등록시킵니다. 이런 뷰 도형 열을 geometry_columns 에 정확히 등록시키려면, 다른 도형을 출력하는 도형 함수를 이용하는 뷰를 typmod 도형으로 형변환시켜야 할 것입니다. Section 4.3.4, “geometry_columns 에 도형 열을 직접 등록하기” 을 참조하십시오.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

개선 사항: 2.0.0 버전. use_typmod 인자가 추가됐습니다. 제약조건 기반 도형 열 대신 기본적으로 typmod 도형 열을 생성합니다.

예제

```
-- 데이터를 담기 위한 스키마를 생성
CREATE SCHEMA my_schema;
-- 새로운 단순 PostgreSQL 테이블 생성
CREATE TABLE my_schema.my_spatial_table (id serial);

-- 단일 "id" 열을 가진 단순 테이블을 보여주는 테이블을 설명하기
postgis=# \d my_schema.my_spatial_table
                                          Table
"my_schema.my_spatial_table"
 Column |  Type   |                          Modifiers
--------+---------+------------------------------------------------------------
------------------
 id                          |     integer     |     not     null     default
nextval('my_schema.my_spatial_table_id_seq'::regclass)

-- 테이블에 공간 열을 추가
SELECT                                                      AddGeometryColumn
('my_schema','my_spatial_table','geom',4326,'POINT',2);

-- 예전 제약조건 기반 습성을 이용해서 포인트를 추가
SELECT                                                      AddGeometryColumn
('my_schema','my_spatial_table','geom_c',4326,'POINT',2, false);

-- 예전 제약조건 습성을 이용해서 만곡 폴리곤을 추가
SELECT                                                      AddGeometryColumn
('my_schema','my_spatial_table','geomcp_c',4326,'CURVEPOLYGON',2, false);

-- 새 도형을 추가했음을 보여주도록 테이블을 다시 설명하기
\d my_schema.my_spatial_table
                      addgeometrycolumn
------------------------------------------------------------------------
 my_schema.my_spatial_table.geomcp_c SRID:4326 TYPE:CURVEPOLYGON DIMS:2
(1 row)

                             Table "my_schema.my_spatial_table"
  Column  |        Type        |                          Modifiers
----------+--------------------+-------------------------------------------
----------------------------------
 id               |  integer                            |  not  null  default
nextval('my_schema.my_spatial_table_id_seq'::regclass)
 geom     | geometry(Point,4326) |
 geom_c   | geometry           |
 geomcp_c | geometry           |
Check constraints:
    "enforce_dims_geom_c" CHECK (st_ndims(geom_c) = 2)
    "enforce_dims_geomcp_c" CHECK (st_ndims(geomcp_c) = 2)
    "enforce_geotype_geom_c" CHECK (geometrytype(geom_c) = 'POINT'::text OR
geom_c IS NULL)
    "enforce_geotype_geomcp_c"      CHECK      (geometrytype(geomcp_c)      =
'CURVEPOLYGON'::text OR geomcp_c IS NULL)
    "enforce_srid_geom_c" CHECK (st_srid(geom_c) = 4326)
    "enforce_srid_geomcp_c" CHECK (st_srid(geomcp_c) = 4326)
```

```
-- geometry_columns 뷰도 새 열들을 등록합니다. --
SELECT f_geometry_column As col_name, type, srid, coord_dimension As ndims
   FROM geometry_columns
   WHERE   f_table_name   =   'my_spatial_table'   AND   f_table_schema   =
'my_schema';

 col_name |     type     | srid | ndims
----------+--------------+------+-------
 geom     | Point        | 4326 |     2
 geom_c   | Point        | 4326 |     2
 geomcp_c | CurvePolygon | 4326 |     2
```

참고: DropGeometryColumn, DropGeometryTable, Section 4.3.2, "The GEOMETRY_COLUMNS VIEW", Section 4.3.4, "geometry_columns 에 도형 열을 직접 등록하기"

8.3.2. DropGeometryColumn

DropGeometryColumn — 공간 테이블에서 도형 열을 제거합니다.

Synopsis

text **DropGeometryColumn**(varchar *table_name*, varchar *column_name*);

text **DropGeometryColumn**(varchar *schema_name*, varchar *table_name*, varchar *column_name*);

text **DropGeometryColumn**(varchar *catalog_name*, varchar *schema_name*, varchar *table_name*, varchar *column_name*);

설명: 공간 테이블에서 도형 열을 제거합니다. schema_name 이 geometry_columns 테이블에 있는 테이블 행의 f_table_schema 항목과 일치해야 한다는 점에 주의하십시오.

✓ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다.

✓ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✓ 이 방법은 원형 스트링 및 곡선을 지원합니다.



변경 사항: 2.0.0 버전. 하위 호환성을 위해 이 함수를 제공합니다. 이제는 geometry_columns 이 시스템 카탈로그를 기반으로 하는 뷰이기 때문에, 다른 어떤 테이블 열과도 마찬가지로 도형 열도 ALTER TABLE 을 이용해서 삭제할 수 있습니다.

예제

```
SELECT DropGeometryColumn ('my_schema','my_spatial_table','geom');
                    -- 결과 출력물 --
                                    dropgeometrycolumn
------------------------------------------------------
 my_schema.my_spatial_table.geom effectively removed.

-- PostGIS 2.0 이상 버전에서 위 쿼리는 표준 대체 테이블과도 동등합니다.
-- 양쪽 모두 geometry_columns 로부터 등록 해제될 것입니다.
```

```
ALTER TABLE my_schema.my_spatial_table DROP column geom;
```

참고: AddGeometryColumn, DropGeometryTable, Section 4.3.2, “The GEOMETRY_COLUMNS VIEW”

8.3.3. DropGeometryTable

DropGeometryTable — 테이블 및 geometry_columns 에 있는 테이블의 모든 참조를 삭제합니다.

Synopsis

boolean **DropGeometryTable**(varchar *table_name*);

boolean **DropGeometryTable**(varchar *schema_name*, varchar *table_name*);

boolean **DropGeometryTable**(varchar *catalog_name*, varchar *schema_name*, varchar *table_name*);

설명: 테이블 및 geometry_columns 에 있는 테이블의 모든 참조를 삭제합니다. 주의: 스키마를 제공하지 않는다면 스키마를 인식하는(schema-aware) pgSQL 설치 상에서 current_schema()를 사용해보십시오.



변경 사항: 2.0.0 버전. 하위 호환성을 위해 이 함수를 제공합니다. 이제는 geometry_columns 이 시스템 카탈로그를 기반으로 하는 뷰이기 때문에, 다른 어떤 테이블과도 마찬가지로 도형 열을 가진 테이블도 DROP TABLE 을 이용해서 삭제할 수 있습니다.

예제

```
SELECT DropGeometryTable ('my_schema','my_spatial_table');
-- 결과 출력물 --
my_schema.my_spatial_table dropped.

-- 이제 위 쿼리는 다음과 동등합니다. --
DROP TABLE my_schema.my_spatial_table;
```

참고: AddGeometryColumn, DropGeometryColumn, Section 4.3.2, “The GEOMETRY_COLUMNS VIEW”

8.3.4. PostGIS_Full_Version

PostGIS_Full_Version — 완전한 PostGIS 버전 및 빌드 설정 정보를 반환합니다.

Synopsis

text **PostGIS_Full_Version**();

설명: 완전한 PostGIS 버전 및 빌드 설정 정보를 반환합니다. 또 업그레이드가 필요한지 알 수 있는 라이브러리와 스크립트 사이의 동기화에 대해서도 알려줍니다.

예제

```
SELECT PostGIS_Full_Version();
```

```
                                                    postgis_full_version
-----------------------------------------------------------------------
---------
POSTGIS="2.2.0dev r12699" GEOS="3.5.0dev-CAPI-1.9.0 r3989" SFCGAL="1.0.4"
PROJ="Rel. 4.8.0, 6 March 2012"
GDAL="GDAL 1.11.0, released 2014/04/16" LIBXML="2.7.8" LIBJSON="0.12"
RASTER
(1 row)
```

참고: Section 2.10, "업그레이드", PostGIS_GEOS_Version, PostGIS_Lib_Version, PostGIS_LibXML_Version, PostGIS_PROJ_Version, PostGIS_Version

8.3.5. PostGIS_GEOS_Version

PostGIS_GEOS_Version — GEOS 라이브러리의 버전 숫자를 반환합니다.

Synopsis

text **PostGIS_GEOS_Version**();

설명: GEOS 라이브러리의 버전 숫자 또는 GEOS 지원이 활성화되지 않았을 경우 NULL 을 반환합니다.

예제

```
SELECT PostGIS_GEOS_Version();
 postgis_geos_version
----------------------
 3.1.0-CAPI-1.5.0
(1 row)
```

참고: PostGIS_Full_Version, PostGIS_Lib_Version, PostGIS_LibXML_Version, PostGIS_PROJ_Version, PostGIS_Version

8.3.6. PostGIS_LibXML_Version

PostGIS_LibXML_Version — LibXML2 라이브러리의 버전 숫자를 반환합니다.

Synopsis

text **PostGIS_LibXML_Version**();

설명: LibXML2 라이브러리의 버전 숫자를 반환합니다.

1.5 버전부터 사용할 수 있습니다.

예제

```
SELECT PostGIS_LibXML_Version();
 postgis_libxml_version
------------------------
 2.7.6
(1 row)
```

참고: PostGIS_Full_Version, PostGIS_Lib_Version, PostGIS_PROJ_Version, PostGIS_GEOS_Version, PostGIS_Version

8.3.7. PostGIS_Lib_Build_Date

PostGIS_Lib_Build_Date — PostGIS 라이브러리의 빌드 날짜를 반환합니다.

Synopsis

text **PostGIS_Lib_Build_Date**();

설명: PostGIS 라이브러리의 빌드 날짜를 반환합니다.

예제

```
SELECT PostGIS_Lib_Build_Date();
 postgis_lib_build_date
------------------------
 2008-06-21 17:53:21
(1 row)
```

8.3.8. PostGIS_Lib_Version

PostGIS_Lib_Version — PostGIS 라이브러리의 버전 숫자를 반환합니다.

Synopsis

text **PostGIS_Lib_Version**();

설명: PostGIS 라이브러리의 버전 숫자를 반환합니다.

예제

```
SELECT PostGIS_Lib_Version();
 postgis_lib_version
---------------------
 1.3.3
(1 row)
```

참고: PostGIS_Full_Version, PostGIS_GEOS_Version, PostGIS_LibXML_Version, PostGIS_PROJ_Version, PostGIS_Version

8.3.9. PostGIS_PROJ_Version

PostGIS_PROJ_Version — PROJ4 라이브러리의 버전 숫자를 반환합니다.

Synopsis

text **PostGIS_PROJ_Version**();

설명: PROJ4 라이브러리의 버전 숫자 혹은 PROJ4 지원이 활성화되지 않았을 경우 NULL 을 반환합니다.

예제

```
SELECT PostGIS_PROJ_Version();
  postgis_proj_version
------------------------
 Rel. 4.4.9, 29 Oct 2004
(1 row)
```

참고: PostGIS_Full_Version, PostGIS_GEOS_Version, PostGIS_Lib_Version, PostGIS_LibXML_Version, PostGIS_Version

8.3.10. PostGIS_Scripts_Build_Date

PostGIS_Scripts_Build_Date — PostGIS 스크립트의 빌드 날짜를 반환합니다.

Synopsis

text **PostGIS_Scripts_Build_Date**();

설명: PostGIS 스크립트의 빌드 날짜를 반환합니다.

1.0.0RC1 버전부터 사용할 수 있습니다.

예제

```
SELECT PostGIS_Scripts_Build_Date();
  postgis_scripts_build_date
------------------------
 2007-08-18 09:09:26
(1 row)
```

참고: PostGIS_Full_Version, PostGIS_GEOS_Version, PostGIS_Lib_Version, PostGIS_LibXML_Version, PostGIS_Version

8.3.11. PostGIS_Scripts_Installed

PostGIS_Scripts_Installed — 이 데이터베이스에 설치된 PostGIS 스크립트의 버전을 반환합니다.

Synopsis

text **PostGIS_Scripts_Installed**();

설명: 이 데이터베이스에 설치된 PostGIS 스크립트의 버전을 반환합니다.



이 함수의 출력물이 PostGIS_Scripts_Released 의 출력물과 일치하지 않는다면, 아마도 기존 데이터베이스를 제대로 업그레이드하지 못 했기 때문일 겁니다. 자세한 정보는 Upgrading 을 살펴보십시오..

0.9.0 버전부터 사용할 수 있습니다.

예제

```
SELECT PostGIS_Scripts_Installed();
  postgis_scripts_installed
-------------------------
 1.5.0SVN
(1 row)
```

참고: PostGIS_Full_Version, PostGIS_Scripts_Released, PostGIS_Version

8.3.12. PostGIS_Scripts_Released

PostGIS_Scripts_Released — 설치된 PostGIS 라이브러리와 함께 출시된 postgis.sql 스크립트의 버전 숫자를 반환합니다.

Synopsis

text **PostGIS_Scripts_Released**();

설명: 설치된 PostGIS 라이브러리와 함께 출시된 postgis.sql 스크립트의 버전 숫자를 반환합니다.



1.1.0 버전부터 이 함수는 PostGIS_Lib_Version 과 동일한 값을 반환합니다. 하위 호환성을 위해 이 함수를 유지하고 있습니다.

0.9.0 버전부터 사용할 수 있습니다.

예제

```
SELECT PostGIS_Scripts_Released();
  postgis_scripts_released
-------------------------
 1.3.4SVN
(1 row)
```

참고: PostGIS_Full_Version, PostGIS_Scripts_Installed, PostGIS_Lib_Version

8.3.13. PostGIS_Version

PostGIS_Version — PostGIS 버전 숫자 및 컴파일 시간 옵션을 반환합니다.

Synopsis

text **PostGIS_Version**();

설명: PostGIS 버전 숫자 및 컴파일 시간 옵션을 반환합니다.

예제

```
SELECT PostGIS_Version();
                  postgis_version
```

```
------------------------------------
 1.3 USE_GEOS=1 USE_PROJ=1 USE_STATS=1
(1 row)
```

참고: PostGIS_Full_Version, PostGIS_GEOS_Version, PostGIS_Lib_Version, PostGIS_LibXML_Version, PostGIS_PROJ_Version

8.3.14. Populate_Geometry_Columns

Populate_Geometry_Columns — 도형 열이 유형 변경자로 정의됐거나 적절한 공간 제약조건을 가지고 있는지 확인합니다. 이들을 확인하면 도형 열이 geometry_columns 뷰에 올바르게 등록될 것입니다. 기본적으로 유형 변경자를 가지지 않는 모든 도형 열들을 유형 변경자를 가진 도형 열들로 변환시킬 것입니다. 구 버전 습성을 원한다면 use_typmod=false 으로 설정하십시오.

Synopsis

text **Populate_Geometry_Columns**(boolean *use_typmod=true*);

int **Populate_Geometry_Columns**(oid *relation_oid*, boolean *use_typmod=true*);

설명: 도형 열이 적절한 유형 변경자를 가지고 있는지, 또는 도형 열이 geometry_columns 테이블에 정확히 등록되는지 확인하는 공간 제약조건을 가지고 있는지 확인합니다.

하위 호환성 및 각 차일드 테이블이 서로 다른 도형 유형을 가질 수도 있는 테이블 상속 같은 공간 필요성을 위해, 구 버전 확인 제약조건 습성을 계속 지원합니다. 구 버전 습성이 필요하다면, use_typmod=false 처럼 새 선택적 인자를 거짓으로 패스해야 합니다. 이렇게 하면 유형 변경자는 없지만 제약조건 3 개가 정의된 도형 열을 생성할 것입니다. 다시 말해, 모든 도형 열이 적어도 3 개의 제약조건을 가진 테이블에 종속된다는 뜻입니다:

- enforce_dims_the_geom -- 모든 도형이 동일한 차원을 가지도록 보장합니다. (ST_NDims 참조)
- enforce_geotype_the_geom -- 모든 도형이 동일한 유형이 되도록 보장합니다. (도형 유형 참조)
- enforce_srid_the_geom -- 모든 도형이 동일한 투영체를 가지도록 보장합니다. (ST_SRID 참조)

oid 테이블이 제공될 경우, 이 함수는 테이블에 있는 모든 도형 열의 SRID, 차원, 그리고 도형 유형을 결정하려 하며, 필요한 경우 제약조건을 추가하기도 합니다. 함수가 제대로 작동했다면, 올바른 행이 geometry_columns 테이블에 삽입됩니다. 아닐 경우, 예외가 잡혀 문제점을 설명하는 오류 메시지가 뜹니다.

oid 테이블과 함께 oid 뷰가 제공될 경우, 이 함수는 geometry_columns 테이블에 올바른 항목을 삽입하며 뷰에 있는 모든 도형의 SRID, 차원, 유형을 결정하려 하지만, 제약조건을 강제하지는 않습니다.

이 함수의 파라미터가 없는 변종은, 공간 제약조건을 테이블의 적절한 곳에 추가하며 먼저 데이터베이스 내부의 모든 공간 테이블 및 뷰에 대해 geometry_columns 테이블의 용량을 줄이고(truncate) 다시 채우는 파라미터가 있는 변종을 위한 단순 래퍼입니다. 파라미터가 없는 변종은 데이터베이스 내부에서 감지된 도형 열의 개수 및 geometry_columns 테이블로 삽입된 개수를 요약해서 반환합니다. 파라미터가 있는 버전은 단순히 geometry_columns 테이블로 삽입된 행의 개수를 반환합니다.

1.4.0 버전부터 사용할 수 있습니다.

변경 사항: 2.0.0 버전. 이제 도형 유형을 제한하기 위해 확인 제약조건 대신 유형 변경자를 이용합니다. 새 use_typmod 를 거짓으로 설정하면, 확인 제약조건 습성을 계속 쓸 수 있습니다.

개선 사항: 2.0.0 버전. 열 생성시 유형 변경자와 함께, 또는 확인 제약조건과 함께 생성할지 통제할 수 있는 use_typmod 의 선택적 인자가 추가됐습니다.

예제

```
CREATE TABLE public.myspatial_table(gid serial, geom geometry);
INSERT INTO myspatial_table(geom) VALUES(ST_GeomFromText('LINESTRING(1 2,
3 4)',4326) );
-- 이제 유형 수정자(typ modifier)를 이용할 것입니다. 이 쿼리가 작동하려면, 데이터가
존재해야만 합니다.
SELECT Populate_Geometry_Columns('public.myspatial_table'::regclass);

populate_geometry_columns
--------------------------
                   1


\d myspatial_table

                            Table "public.myspatial_table"
 Column |         Type          |                     Modifiers
--------+---------------------------+-----------------------------------
---------------------------
 gid       | integer                                  | not null default
nextval('myspatial_table_gid_seq'::regclass)
 geom   | geometry(LineString,4326) |
-- 도형 열이 typmod 가 아니거나 이미 제약조건을 가지고 있지 않을 경우, 제약조건을
이용하도록 도형 열을 변경할 것입니다.
-- 이 쿼리가 작동하려면, 데이터가 존재해야만 합니다.
CREATE TABLE public.myspatial_table_cs(gid serial, geom geometry);
INSERT INTO myspatial_table_cs(geom) VALUES(ST_GeomFromText('LINESTRING(1
2, 3 4)',4326) );
SELECT    Populate_Geometry_Columns('public.myspatial_table_cs'::regclass,
false);
populate_geometry_columns
--------------------------
                   1
\d myspatial_table_cs

                     Table "public.myspatial_table_cs"
 Column |   Type   |                       Modifiers
--------+----------+-------------------------------------------------------
-------------
 gid                   |     integer          |     not     null     default
nextval('myspatial_table_cs_gid_seq'::regclass)
 geom   | geometry |
Check constraints:
    "enforce_dims_geom" CHECK (st_ndims(geom) = 2)
    "enforce_geotype_geom" CHECK (geometrytype(geom) = 'LINESTRING'::text
OR geom IS NULL)
    "enforce_srid_geom" CHECK (st_srid(geom) = 4326)
```

8.3.15. UpdateGeometrySRID

UpdateGeometrySRID — 도형 열, geometry_columns 메타데이터 그리고 SRID 에 있는 모든 피처의 SRID 를 업데이트합니다. 제약조건으로 강제됐을 경우, 제약조건도 새로운 SRID 제약조건으로 업데이트될 것입니다. 유형 정의에 따라 구 버전을 강제했다면, 유형 정의가 변경될 것입니다.

Synopsis

text **UpdateGeometrySRID**(varchar *table_name*, varchar *column_name*, integer *srid*);

text **UpdateGeometrySRID**(varchar *schema_name*, varchar *table_name*, varchar *column_name*, integer *srid*);

text **UpdateGeometrySRID**(varchar *catalog_name*, varchar *schema_name*, varchar *table_name*, varchar *column_name*, integer *srid*);

설명: geometry_columns 에 있는 제약조건 및 참조를 업데이트하면서 geometry_columns 에 있는 모든 피처의 SRID 를 업데이트합니다. 주의: 스키마를 제공하지 않는다면 스키마를 인식하는(schema-aware) pgSQL 설치 상에서 current_schema()를 사용해보십시오.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

예제

다음은 도로 테이블의 이전 SRID 와 상관없이 SRID 를 4326 으로 변경할 것입니다:

```
SELECT UpdateGeometrySRID('roads','geom',4326);
```

앞의 예시는 다음 DDL 선언문과 동일합니다:

```
ALTER TABLE roads
  ALTER COLUMN geom TYPE geometry(MULTILINESTRING, 4326)
    USING ST_SetSRID(geom,4326);
```

로드 작업 중 투영체를 잘못 (또는 'unknown'으로) 적용했는데 모든 데이터를 한번에 웹 메르카토르로 변환하고 싶다면, DDL 을 통해 가능합니다. 하지만 PostGIS 관리 함수 가운데 한번에 이런 작업을 할 수 있는 동일한 함수는 없습니다.

```
ALTER TABLE roads
 ALTER COLUMN geom TYPE geometry(MULTILINESTRING, 3857) USING
ST_Transform(ST_SetSRID(geom,4326),3857) ;
```

참고: UpdateRasterSRID, ST_SetSRID, ST_Transform

8.4. 도형 작성자(constructor)

ST_BdPolyFromText — 멀티라인스트링 WKT 텍스트 표현식으로 주어진 임의의 폐쇄형 라인스트링 집합으로 폴리곤을 작성합니다.

ST_BdMPolyFromText — 멀티라인스트링 WKT 텍스트 표현식으로 주어진 임의의 폐쇄형 라인스트링 집합으로 멀티폴리곤을 작성합니다.

ST_Box2dFromGeoHash — GeoHash 스트링으로부터 BOX2D 를 반환합니다.

ST_GeogFromText — WKT (확장) 표현식으로부터 지정된 지리형 값을 반환합니다.

ST_GeographyFromText — WKT (확장) 표현식으로부터 지정된 지리형 값을 반환합니다.

ST_GeogFromWKB — WKB 도형 표현식 또는 EWKB(확장 WKB)로부터 지리형 인스턴스를 생성합니다.

ST_GeomFromTWKB — TWKB("Tiny Well-Known Binary") 도형 표현식으로부터 도형 인스턴스를 생성합니다.

ST_GeomCollFromText — 주어진 SRID 와 함께 집합 WKT 로부터 도형 집합을 만듭니다. SRID 가 주어지지 않은 경우, 기본값인 0 을 씁니다.

ST_GeomFromEWKB — EWKB(Extended Well-Known Binary)로부터 지정된 ST_Geometry 값을 반환합니다.

ST_GeomFromEWKT — EWKT(Extended Well-Known Text)로부터 지정된 ST_Geometry 값을 반환합니다.

ST_GeometryFromText — WKT(Well-Known Text)로부터 지정된 ST_Geometry 값을 반환합니다. 이 함수는 ST_GeomFromText 함수와 동일합니다.

ST_GeomFromGeoHash — GeoHash 스트링으로부터 도형을 반환합니다.

ST_GeomFromGML — 도형의 GML 표현식을 입력받아 PostGIS 도형 객체를 출력합니다.

ST_GeomFromGeoJSON — GeoJSON 표현식을 입력받아 PostGIS 도형 객체를 출력합니다.

ST_GeomFromKML — 도형의 KML 표현식을 입력받아 PostGIS 도형 객체를 출력합니다.

ST_GMLToSQL — GML 로부터 지정된 ST_Geometry 값을 반환합니다. 이 함수는 ST_GeomFromGML 함수와 동일합니다.

ST_GeomFromText — WKT 표현식으로부터 지정된 ST_Geometry 값을 반환합니다.

ST_GeomFromWKB — WKB(Well-Known Binary) 도형 표현식과 선택적인 SRID 로부터 도형 인스턴스를 생성합니다.

ST_LineFromEncodedPolyline — 인코딩된 폴리라인(polyline)으로부터 라인스트링을 생성합니다.

ST_LineFromMultiPoint — 멀티포인트 도형으로부터 라인스트링을 생성합니다.

ST_LineFromText — 주어진 SRID 와 함께 WKT 표현식으로부터 도형을 만듭니다. SRID 가 주어지지 않은 경우, 기본값인 0 을 씁니다.

ST_LineFromWKB — 주어진 SRID 와 함께 WKB 로부터 LINESTRING 을 만듭니다.

ST_LinestringFromWKB — 주어진 SRID 와 함께 WKB 로부터 도형을 만듭니다.

ST_MakeBox2D — 주어진 포인트 도형들로 정의되는 BOX2D 를 생성합니다.

ST_3DMakeBox — 주어진 3D 포인트 도형들로 정의되는 BOX3D 를 생성합니다.

ST_MakeLine — 포인트, 멀티포인트 또는 라인 도형으로부터 라인스트링을 생성합니다.

ST_MakeEnvelope — 주어진 최소값과 최대값으로 형성된 직사각형 폴리곤을 생성합니다. 입력값이 SRID 로 지정된 SRS 를 가지고 있어야 합니다.

ST_MakePolygon — 주어진 외부 구조(shell)로 형성된 폴리곤을 생성합니다. 입력 도형이 닫힌 라인스트링이어야 합니다.

ST_MakePoint — 2D, 3DZ 또는 4D 포인트 도형을 생성합니다.

ST_MakePointM — x, y, m 좌표를 가진 포인트 도형을 생성합니다.

ST_MLineFromText — WKT 표현식으로부터 지정된 ST_MultiLineString 값을 반환합니다.

ST_MPointFromText — 주어진 SRID 와 함께 WKT 표현식으로부터 도형 집합을 만듭니다. SRID 가 주어지지 않은 경우, 기본값인 0 을 씁니다.

ST_MPolyFromText — 주어진 SRID 와 함께 WKT 표현식으로부터 멀티폴리곤을 만듭니다. SRID 가 주어지지 않은 경우, 기본값인 0 을 씁니다.

ST_Point — 주어진 좌표값을 가진 ST_Point 를 반환합니다. ST_MakePoint 와 동일한 OGC 함수입니다.

ST_PointFromGeoHash — GeoHash 스트링으로부터 포인트를 반환합니다.

ST_PointFromText — 주어진 SRID 와 함께 WKT 표현식으로부터 포인트 도형을 만듭니다. SRID 가 주어지지 않은 경우, 기본값인 0 을 씁니다.

ST_PointFromWKB — 주어진 SRID 와 함께 WKB 로부터 도형을 만듭니다.

ST_Polygon — 설정된 라인스트링과 SRID 로부터 빌드된 폴리곤을 반환합니다.

ST_PolygonFromText — 주어진 SRID 와 함께 WKT 표현식으로부터 도형 집합을 만듭니다. SRID 가 주어지지 않은 경우, 기본값인 0 을 씁니다.

ST_WKBToSQL — WKB(Well-Known Binary) 표현식으로부터 지정된 ST_Geometry 값을 반환합니다. 이 함수는 SRID 를 입력받지 않는 ST_GeomFromWKB 함수와 동일합니다.

ST_WKTToSQL — WKT(Well-Known Text)로부터 지정된 ST_Geometry 값을 반환합니다. 이 함수는 ST_GeomFromText 함수와 동일합니다.

8.4.1. ST_BdPolyFromText

ST_BdPolyFromText — 멀티라인스트링 WKT 텍스트 표현식으로 주어진 임의의 폐쇄형 라인스트링 집합으로 폴리곤을 작성합니다.

Synopsis

geometry **ST_BdPolyFromText**(text *WKT*, integer *srid*);

설명: 멀티라인스트링 WKT 텍스트 표현식으로 주어진 임의의 폐쇄형 라인스트링 집합으로 폴리곤을 작성합니다.



WKT 가 멀티라인스트링이 아닐 경우 오류가 발생합니다. 출력물이 멀티폴리곤일 경우 오류가 발생하는데 이런 경우 ST_BdMPolyFromText 를 사용하거나, 또는 PostGIS 특화 접근법을 사용하려면 ST_BuildArea()를 참조하십시오..

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다. s3.2.6.2

1.1.0 버전부터 사용할 수 있습니다. GEOS 2.1.0 이상 버전이 필요합니다.

예시

```
작성중
```

참고: ST_BuildArea, ST_BdMPolyFromText

8.4.2. ST_BdMPolyFromText

ST_BdMPolyFromText — 멀티라인스트링 WKT 텍스트 표현식으로 주어진 임의의 폐쇄형 라인스트링 집합으로 멀티폴리곤을 작성합니다.

Synopsis

geometry **ST_BdMPolyFromText**(text *WKT*, integer *srid*);

설명: 멀티라인스트링 WKT 텍스트 표현식으로 주어진 임의의 폐쇄형 라인스트링 집합으로 멀티폴리곤을 작성합니다.



WKT 가 멀티라인스트링이 아닐 경우 오류가 발생합니다. 출력물이 단일 폴리곤으로 이루어졌을 경우에도 멀티폴리곤으로 강제합니다. 해당 연산시 단일 폴리곤을 출력할 것이라고 확신한다면 ST_BdPolyFromText 를 사용하거나, 또는 PostGIS 특화 접근법을 사용하려 할 경우 ST_BuildArea() 를 참조하십시오.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다. s3.2.6.2

1.1.0 버전부터 사용할 수 있습니다. GEOS 2.1.0 이상 버전이 필요합니다.

예시

```
작성중
```

참고: ST_BuildArea, ST_BdMPolyFromText

8.4.3. ST_Box2dFromGeoHash

ST_Box2dFromGeoHash — GeoHash 스트링으로부터 BOX2D 를 반환합니다.

Synopsis

box2d **ST_Box2dFromGeoHash**(text *geohash*, integer *precision=full_precision_of_geohash*);

설명: GeoHash 스트링으로부터 BOX2D 를 반환합니다.

precision 을 설정하지 않은 경우, ST_Box2dFromGeoHash 가 입력 GeoHash 스트링의 전체 정확도를 기반으로 BOX2D 를 반환합니다.

precision 을 설정한 경우, ST_Box2dFromGeoHash 가 GeoHash 에서 나온 그만큼의 문자를 이용해서 BOX2D 를 생성합니다. 정확도 값이 낮을수록 BOX2D 의 용량이 커지고 값이 높을수록 정확도가 올라갑니다.

2.1.0 버전부터 사용할 수 있습니다.

예시

```
SELECT ST_Box2dFromGeoHash('9qqj7nmxncgyy4d0dbxqz0');

             st_geomfromgeohash
------------------------------------------------
 BOX(-115.172816 36.114646,-115.172816 36.114646)

SELECT ST_Box2dFromGeoHash('9qqj7nmxncgyy4d0dbxqz0', 0);

 st_box2dfromgeohash
----------------------
 BOX(-180 -90,180 90)

 SELECT ST_Box2dFromGeoHash('9qqj7nmxncgyy4d0dbxqz0', 10);
                      st_box2dfromgeohash
-------------------------------------------------------------------------
--
 BOX(-115.17282128334 36.1146408319473,-115.172810554504 36.1146461963654)
```

참고: ST_GeoHash, ST_GeomFromGeoHash, ST_PointFromGeoHash

8.4.4. ST_GeogFromText

ST_GeogFromText — WKT (확장) 표현식으로부터 지정된 지리형 값을 반환합니다.

Synopsis

geography **ST_GeogFromText**(text *EWKT*);

설명: WKT 표현식 또는 확장 WKT 표현식으로부터 지리형 객체를 반환합니다. 투영체를 따로 설정하지 않은 경우 SRID 4326 으로 가정합니다. 이 함수는 ST_GeographyFromText 와 동일합니다. 포인트는 항상 경위도 형식으로 표현됩니다.

예시

```
--- 경위도 좌표를 지리형으로 변환하기
ALTER TABLE sometable ADD COLUMN geog geography(POINT,4326);
UPDATE sometable SET geog = ST_GeogFromText('SRID=4326;POINT(' || lon || '
' || lat || ')');
```

```
--- EPSG:4267, NAD27 을 이용해서 지리형 포인트를 지정하기
SELECT ST_AsEWKT(ST_GeogFromText('SRID=4267;POINT(-77.0092 38.889588)'));
```

참고: ST_AsText, ST_GeographyFromText

8.4.5. ST_GeographyFromText

ST_GeographyFromText — WKT (확장) 표현식으로부터 지정된 지리형 값을 반환합니다.

Synopsis

geography **ST_GeographyFromText**(text *EWKT*);

설명: WKT 표현식으로부터 지리형 객체를 반환합니다. 투영체를 따로 설정하지 않은 경우 SRID 4326 으로 가정합니다.

참고: ST_GeogFromText, ST_AsText

8.4.6. ST_GeogFromWKB

ST_GeogFromWKB — WKB 도형 표현식 또는 EWKB(확장 WKB)로부터 지리형 인스턴스를 생성합니다.

Synopsis

geography **ST_GeogFromWKB**(bytea *wkb*);

설명: ST_GeogFromWKB 함수는 도형의 WKB 표현식 또는 PostGIS 확장 WKB 를 받아 적절한 지리형 유형의 인스턴스를 생성합니다. 이 함수는 SQL 에서 도형 공장(Geometry Factory) 역할을 합니다.

SRID 를 설정하지 않은 경우, 기본값으로 4326(WGS84 경위도)을 씁니다.

✓ 이 방법은 원형 스트링 및 곡선을 지원합니다.

예시

```
-- bytea 표현식이 단일한 역슬래시(\)를 포함하긴 해도, 테이블 삽입시에는 빼놓아야 합니다.
SELECT ST_AsText(
ST_GeogFromWKB(E'\\001\\002\\000\\000\\000\\002\\000\\000\\000\\037\\205\\3
53Q\\270~\\\\\\300\\323Mb\\020X\\231C@\\020X9\\264\\310~\\\\\\300)\\\\\\217
\\302\\365\\230C@')
);
                                  st_astext
-----------------------------------------------------
 LINESTRING(-113.98 39.198,-113.981 39.195)
(1 row
```

참고: ST_GeogFromText, ST_AsBinary

8.4.7. ST_GeomFromTWKB

ST_GeomFromTWKB — TWKB("Tiny Well-Known Binary") 도형 표현식으로부터 도형 인스턴스를 생성합니다.

Synopsis

geometry **ST_GeomFromTWKB**(bytea *twkb*);

설명: ST_GeomFromTWKB 함수는 TWKB("Tiny Well-Known Binary") 도형 표현식(WKB)을 받아 적절한 도형 유형의 인스턴스를 생성합니다.

예시

```
SELECT    ST_AsText(ST_GeomFromTWKB(ST_AsTWKB('LINESTRING(126    34,    127
35)'::geometry)));

        st_astext
-----------------------------
 LINESTRING(126 34, 127 35)
(1 row)


SELECT ST_AsEWKT(
  ST_GeomFromTWKB(E'\\x620002f7f40dbce4040105')
);
                                    st_asewkt
------------------------------------------------------------
LINESTRING(-113.98 39.198,-113.981 39.195)
(1 row)
```

참고: ST_AsTWKB

8.4.8. ST_GeomCollFromText

ST_GeomCollFromText — 주어진 SRID 와 함께 집합 WKT 로부터 도형 집합을 만듭니다. SRID 가 주어지지 않은 경우, 기본값인 0 을 씁니다.

Synopsis

geometry **ST_GeomCollFromText**(text *WKT*, integer *srid*);

geometry **ST_GeomCollFromText**(text *WKT*);

설명: 주어진 SRID 와 함께 WKT 표현식으로부터 도형 집합을 만듭니다. SRID 가 주어지지 않은 경우, 기본값인 0 을 씁니다.

OGC 사양 3.2.6.2 - 선택적인 SRID 를 적합성 스위트(conformance suite)로부터 가져오십시오.

WKT 가 도형 집합(GEOMETRYCOLLECTION)이 아닌 경우 null 을 반환합니다.



사용자 WKT 도형이 모두 집합이라고 확신한다면, 이 함수를 사용하지 마십시오. 이 함수는 추가적인 유효성 검사 단계를 거치므로 ST_GeomFromText 보다 느립니다..

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다. s3.2.6.2

✔ 이 방법은 SQL/MM 사양을 도입합니다.

예시

```
SELECT ST_GeomCollFromText('GEOMETRYCOLLECTION(POINT(1 2),LINESTRING(1 2,
3 4))');
```

참고: ST_GeomFromText, ST_SRID

8.4.9. ST_GeomFromEWKB

ST_GeomFromEWKB — EWKB(Extended Well-Known Binary)로부터 지정된 ST_Geometry 값을 반환합니다.

Synopsis

geometry **ST_GeomFromEWKB**(bytea *EWKB*);

설명: OGC EWKB(Extended Well-Known Binary) 표현식으로부터 PostGIS ST_Geometry 객체를 작성합니다.



EWKB 형식이 OGC 표준은 아니지만, SRID(공간 참조 시스템 식별자)를 포함하는 PostGIS 특화 형식입니다.

개선 사항: 2.0.0 버전부터 다면체 표면(polyhedral surface) 및 TIN 을 지원합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

예시: NAD83 경위도(SRID 4269)로 투영된 LINESTRING(-71.160281 42.258729,-71.160837 42.259113,-71.161144 42.25932)의 라인스트링 바이너리 표현식



주의: 바이트 배열이 역슬래시(\)로 구분되고 작은따옴표(')를 가지고 있을 수도 있지만, standard_conforming_strings 가 비활성화돼 있을 경우 \ 및 " 둘 다 빼놓아야 합니다. 그래야만 AsEWKB 표현식과 똑같이 보이지 않기 때문입니다.

```
SELECT ST_GeomFromEWKB(E'\\001\\002\\000\\000
\\255\\020\\000\\000\\003\\000\\000\\000\\344J=
\\013B\\312Q\\300n\\303(\\010\\036!E@''\\277E''K
\\312Q\\300\\366{b\\235*!E@\\225|\\354.P\\312Q
\\300p\\231\\323e1!E@');
```



PostgreSQL 9.1 버전부터 standard_conforming_strings 가 기본적으로 활성화돼 있습니다. 구 버전에서는 따로 설정해야 했습니다. 단일 쿼리, 데이터베이스 또는 서버 레벨에서 원하는 대로 기본값을 변경할 수 있습니다. 다음은 standard_conforming_strings = on 상태에서 어떻게 할 것인지를 보여줍니다. 이 경우 작은따옴표(')를 표준 ANSI 작은따옴표(')와 함께 빼놓지만, 역슬래시는 제외하지 않습니다.

```
set standard_conforming_strings = on;
SELECT ST_GeomFromEWKB('\001\002\000\000
\255\020\000\000\003\000\000\000\344J=\012\013B
\312Q\300n\303(\010\036!E@''\277E''K\012\312Q\300\366{b\235*!E@\225|\354.P\
312Q\012\300p\231\323e1')
```

참고: ST_AsBinary, ST_AsEWKB, ST_GeomFromWKB

8.4.10. ST_GeomFromEWKT

ST_GeomFromEWKT — EWKT(Extended Well-Known Text)로부터 지정된 ST_Geometry 값을 반환합니다.

Synopsis

geometry **ST_GeomFromEWKT**(text *EWKT*);

설명: OGC EWKT(Extended Well-Known Text) 표현식으로부터 PostGIS ST_Geometry 객체를 작성합니다.



EWKT 형식이 OGC 표준은 아니지만, SRID(공간 참조 시스템 식별자)를 포함하는 PostGIS 특화 형식입니다.

개선 사항: 2.0.0 버전부터 다면체 표면(polyhedral surface) 및 TIN 을 지원합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

예시

```
SELECT     ST_GeomFromEWKT('SRID=4269;LINESTRING(-71.160281     42.258729,-
71.160837 42.259113,-71.161144 42.25932)');
SELECT  ST_GeomFromEWKT('SRID=4269;MULTILINESTRING((-71.160281  42.258729,-
71.160837 42.259113,-71.161144 42.25932))');

SELECT ST_GeomFromEWKT('SRID=4269;POINT(-71.064544 42.28787)');

SELECT                ST_GeomFromEWKT('SRID=4269;POLYGON((-71.1776585052917
42.3902909739571,-71.1776820268866 42.3903701743239,
```

```
-71.1776063012595  42.3903825660754,-71.1775826583081  42.3903033653531,-
71.1776585052917 42.3902909739571))');

SELECT        ST_GeomFromEWKT('SRID=4269;MULTIPOLYGON(((-71.1031880899493
42.3152774590236,
-71.1031627617667 42.3152960829043,-71.102923838298 42.3149156848307,
-71.1023097974109 42.3151969047397,-71.1019285062273 42.3147384934248,
-71.102505233663 42.3144722937587,-71.10277487471 42.3141658254797,
-71.103113945163 42.3142739188902,-71.10324876416 42.31402489987,
-71.1033002961013 42.3140393340215,-71.1033488797549 42.3139495090772,
-71.103396240451 42.3138632439557,-71.1041521907712 42.3141153348029,
-71.1041411411543 42.3141545014533,-71.1041287795912 42.3142114839058,
-71.1041188134329 42.3142693656241,-71.1041112482575 42.3143272556118,
-71.1041072845732 42.3143851580048,-71.1041057218871 42.3144430686681,
-71.1041065602059 42.3145009876017,-71.1041097995362 42.3145589148055,
-71.1041166403905 42.3146168544148,-71.1041258822717 42.3146748022936,
-71.1041375307579 42.3147318674446,-71.1041492906949 42.3147711126569,
-71.1041598612795 42.314808571739,-71.1042515013869 42.3151287620809,
-71.1041173835118 42.3150739481917,-71.1040809891419 42.3151344119048,
-71.1040438678912 42.3151191367447,-71.1040194562988 42.3151832057859,
-71.1038734225584 42.3151140942995,-71.1038446938243 42.3151006300338,
-71.1038315271889 42.315094347535,-71.1037393329282 42.315054824985,
-71.1035447555574 42.3152608696313,-71.1033436658644 42.3151648370544,
-71.1032580383161 42.3152269126061,-71.103223066939 42.3152517403219,
-71.1031880899493 42.3152774590236)),
((-71.1043632495873 42.315113108546,-71.1043583974082 42.3151211109857,
-71.1043443253471  42.3150676015829,-71.1043850704575  42.3150793250568,-
71.1043632495873 42.315113108546)))');
-- 3D 원형 스트링
SELECT  ST_GeomFromEWKT('CIRCULARSTRING(220268  150415  1,220227  150505
2,220227 150406 3)');
-- 다면체 표면 예시
SELECT ST_GeomFromEWKT('POLYHEDRALSURFACE(
      ((0 0 0, 0 0 1, 0 1 1, 0 1 0, 0 0 0)),
      ((0 0 0, 0 1 0, 1 1 0, 1 0 0, 0 0 0)),
      ((0 0 0, 1 0 0, 1 0 1, 0 0 1, 0 0 0)),
      ((1 1 0, 1 1 1, 1 0 1, 1 0 0, 1 1 0)),
      ((0 1 0, 0 1 1, 1 1 1, 1 1 0, 0 1 0)),
      ((0 0 1, 1 0 1, 1 1 1, 0 1 1, 0 0 1))
)');
```

참고: ST_AsEWKT, ST_GeomFromText, ST_GeomFromEWKT

8.4.11. ST_GeometryFromText

ST_GeometryFromText — WKT(Well-Known Text)로부터 지정된 ST_Geometry 값을 반환합니다. 이 함수는 ST_GeomFromText 함수와 동일합니다.

Synopsis

geometry **ST_GeometryFromText**(text *WKT*);

geometry **ST_GeometryFromText**(text *WKT*, integer *srid*);

설명:

✓ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다.

✓ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 5.1.40

참고: ST_GeomFromText

8.4.12. ST_GeomFromGeoHash

ST_GeomFromGeoHash — GeoHash 스트링으로부터 도형을 반환합니다.

Synopsis

geometry **ST_GeomFromGeoHash**(text *geohash*, integer *precision=full_precision_of_geohash*);

설명: GeoHash 스트링으로부터 도형을 반환합니다. 해당 도형은 GeoHash 범위를 표현하는 폴리곤이 될 것입니다.

precision 을 설정하지 않은 경우, ST_GeomFromGeoHash 가 입력 GeoHash 스트링의 전체 정확도를 기반으로 폴리곤을 반환합니다.

precision 을 설정한 경우, ST_GeomFromGeoHash 가 GeoHash 에서 나온 그만큼의 문자를 이용해서 폴리곤을 생성합니다.

2.1.0 버전부터 사용할 수 있습니다.

예시

```
SELECT ST_AsText(ST_GeomFromGeoHash('9qqj7nmxncgyy4d0dbxqz0'));
                                          st_astext
----------------------------------------------------------------------------
------------------------------------------------
 POLYGON((-115.172816      36.114646,-115.172816      36.114646,-115.172816
36.114646,-115.172816 36.114646,-115.172816 36.114646))

SELECT ST_AsText(ST_GeomFromGeoHash('9qqj7nmxncgyy4d0dbxqz0', 4));
                                            st_astext
----------------------------------------------------------------------------
------------------------------------------------------
 POLYGON((-115.3125      36.03515625,-115.3125      36.2109375,-114.9609375
36.2109375,-114.9609375 36.03515625,-115.3125 36.03515625))

SELECT ST_AsText(ST_GeomFromGeoHash('9qqj7nmxncgyy4d0dbxqz0', 10));

st_astext
----------------------------------------------------------------------------
----------------------------------------------------------------------------
-------------------------------------
 POLYGON((-115.17282128334               36.1146408319473,-115.17282128334
36.1146461963654,-115.172810554504      36.1146461963654,-115.172810554504
36.1146408319473,-115.17282128334 36.1146408319473))
```

참고: ST_GeoHash,ST_Box2dFromGeoHash, ST_PointFromGeoHash

8.4.13. ST_GeomFromGML

ST_GeomFromGML — 도형의 GML 표현식을 입력받아 PostGIS 도형 객체를 출력합니다.

Synopsis

geometry **ST_GeomFromGML**(text *geomgml*);

geometry **ST_GeomFromGML**(text *geomgml*, integer *srid*);

설명: OGC GML 표현식으로부터 PostGIS ST_Geometry 객체를 작성합니다.

ST_GeomFromGML 은 GML 도형 조각(geometry fragment)에 대해서만 작동합니다. 완전한 GML 문서를 사용하려 할 경우 오류가 발생합니다.

지원하는 OGC GML 버전은 다음과 같습니다:

- GML 3.2.1 네임스페이스
- GML 3.1.1 단순 피처 프로파일 SF-2 (GML 3.1.0 및 3.0.0 하위 호환)
- GML 2.1.2

OGC GML 표준: http://www.opengeospatial.org/standards/gml

1.5 버전부터 사용할 수 있습니다. LibXML2 1.6 이상 버전이 필요합니다.

개선 사항: 2.0.0 버전부터 다면체 표면(polyhedral surface) 및 TIN 을 지원합니다.

개선 사항: 2.0.0 버전에서 선택적인 기본 SRID 파라미터가 추가됐습니다.

✓ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✓ 이 함수는 다면체 표면을 지원합니다.

✓ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

GML 은 혼합 차원을 (예를 들어 동일한 멀티 도형(MultiGeometry) 안에서 2D 와 3D 를 함께) 쓸 수 있습니다. PostGIS 도형은 그렇지 않기 때문에, 사라진 Z 차원을 찾은 경우 ST_GeomFromGML 이 전체 도형을 2D 로 변환합니다.

GML 은 동일한 멀티 도형 안에서 혼합 SRS 를 지원합니다. PostGIS 도형은 그렇지 않기 때문에, 이 경우 ST_GeomFromGML 이 모든 하위 도형들을 SRS 루트 노드로 재투영합니다. GML 루트 노드로 쓸 수 있는 srsName 속성이 없을 경우, 오류가 발생합니다.

ST_GeomFromGML 은 분명한 GML 네임스페이스를 꼼꼼히 따지지 않습니다. 일반적인 사용시에는 네임스페이스를 명시적으로 언급하지 않아도 됩니다. 하지만 GML 내부에 XLink 피처를 사용하고 싶다면 그래야 합니다.



ST_GeomFromGML 함수는 SQL/MM 만곡 도형을 지원하지 않습니다.

예시: srsName 을 가진 단일 도형

```
SELECT ST_GeomFromGML('
            <gml:LineString srsName="EPSG:4269">
                  <gml:coordinates>
                        -71.16028,42.258729    -71.160837,42.259112    -
71.161143,42.25932
                  </gml:coordinates>
            </gml:LineString
>');
```

예시: XLink 사용

```
SELECT ST_GeomFromGML('
            <gml:LineString xmlns:gml="http://www.opengis.net/gml"
                        xmlns:xlink="http://www.w3.org/1999/xlink"
                        srsName="urn:ogc:def:crs:EPSG::4269">
                  <gml:pointProperty>
                        <gml:Point gml:id="p1"
><gml:pos
>42.258729 -71.16028</gml:pos
></gml:Point>
                  </gml:pointProperty>
                  <gml:pos
>42.259112 -71.160837</gml:pos>
                  <gml:pointProperty>
                        <gml:Point                    xlink:type="simple"
xlink:href="#p1"/>
                  </gml:pointProperty>
            </gml:LineString
>'););
```

예시: 다면체 표면

```
SELECT ST_AsEWKT(ST_GeomFromGML('
<gml:PolyhedralSurface>
<gml:polygonPatches>
  <gml:PolygonPatch>
    <gml:exterior>
      <gml:LinearRing
><gml:posList srsDimension="3"
>0 0 0 0 0 1 0 1 1 0 1 0 0 0 0</gml:posList
></gml:LinearRing>
    </gml:exterior>
  </gml:PolygonPatch>
  <gml:PolygonPatch>
    <gml:exterior>
       <gml:LinearRing
><gml:posList srsDimension="3"
>0 0 0 0 1 0 1 1 0 1 0 0 0 0 0</gml:posList
></gml:LinearRing>
    </gml:exterior>
  </gml:PolygonPatch>
  <gml:PolygonPatch>
    <gml:exterior>
       <gml:LinearRing
><gml:posList srsDimension="3"
>0 0 0 1 0 0 1 0 1 0 0 1 0 0 0</gml:posList
```

```
></gml:LinearRing>
    </gml:exterior>
  </gml:PolygonPatch>
  <gml:PolygonPatch>
    <gml:exterior>
       <gml:LinearRing
><gml:posList srsDimension="3"
>1 1 0 1 1 1 1 0 1 1 0 0 1 1 0</gml:posList
></gml:LinearRing>
    </gml:exterior>
  </gml:PolygonPatch>
  <gml:PolygonPatch>
    <gml:exterior>
       <gml:LinearRing
><gml:posList srsDimension="3"
>0 1 0 0 1 1 1 1 1 1 1 0 0 1 0</gml:posList
></gml:LinearRing>
    </gml:exterior>
  </gml:PolygonPatch>
  <gml:PolygonPatch>
    <gml:exterior>
       <gml:LinearRing
><gml:posList srsDimension="3"
>0 0 1 1 0 1 1 1 1 0 1 1 0 0 1</gml:posList
></gml:LinearRing>
    </gml:exterior>
  </gml:PolygonPatch>
</gml:polygonPatches>
</gml:PolyhedralSurface
>'));

-- 결과 --
 POLYHEDRALSURFACE(((0 0 0,0 0 1,0 1 1,0 1 0,0 0 0)),
 ((0 0 0,0 1 0,1 1 0,1 0 0,0 0 0)),
 ((0 0 0,1 0 0,1 0 1,0 0 1,0 0 0)),
 ((1 1 0,1 1 1,1 0 1,1 0 0,1 1 0)),
 ((0 1 0,0 1 1,1 1 1,1 1 0,0 1 0)),
 ((0 0 1,1 0 1,1 1 1,0 1 1,0 0 1)))
```

참고: Section 2.4.1, “설정”, ST_AsGML, ST_GMLToSQL

8.4.14. ST_GeomFromGeoJSON

ST_GeomFromGeoJSON — GeoJSON 표현식을 입력받아 PostGIS 도형 객체를 출력합니다.

Synopsis

geometry **ST_GeomFromGeoJSON**(text *geomjson*);

설명: GeoJSON 표현식으로부터 PostGIS 도형 객체를 작성합니다.

ST_GeomFromGML 은 JSON 도형 조각(geometry fragment)에 대해서만 작동합니다. 완전한 JSON 문서를 사용하려 할 경우 오류가 발생합니다.

2.0.0 버전부터 사용할 수 있습니다. JSON-C 0.9 이상 버전이 필요합니다.



JSON-C 지원을 활성화시키지 않았다면, 출력물 대신 오류 메시지를 보게 될 것입니다. JSON-C 를 활성화하려면, "--with-jsondir=/path/to/json-c" 인자와 함께 설정하십시오. 자세한 내용은 Section 2.4.1, “설정” 을 참조하십시오.



이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
SELECT        ST_AsText(ST_GeomFromGeoJSON('{"type":"Point","coordinates":[-
48.23456,20.12345]}')) As wkt;
wkt
------
POINT(-48.23456 20.12345)
-- a 3D linestring
SELECT
ST_AsText(ST_GeomFromGeoJSON('{"type":"LineString","coordinates":[[1,2,3],[
4,5,6],[7,8,9]]}')) As wkt;

wkt
--------------------
LINESTRING(1 2,4 5,7 8)
```

참고: ST_AsText, ST_AsGeoJSON, Section 2.4.1, “설정”

8.4.15. ST_GeomFromKML

ST_GeomFromKML — 도형의 KML 표현식을 입력받아 PostGIS 도형 객체를 출력합니다.

Synopsis

geometry **ST_GeomFromKML**(text *geomkml*);

설명: OGC KML 표현식으로부터 PostGIS ST_Geometry 객체를 작성합니다.

ST_GeomFromKML 은 KML 도형 조각(geometry fragment)에 대해서만 작동합니다. 완전한 KML 문서를 사용하려 할 경우 오류가 발생합니다.

지원하는 OGC KML 버전은 다음과 같습니다:

- KML 2.2.0 네임스페이스

OGC KML 표준: http://www.opengeospatial.org/standards/kml

1.5 버전부터 사용할 수 있습니다. LibXML2 2.6 이상 버전이 필요합니다.



이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.



ST_GeomFromKML 함수는 SQL/MM 만곡 도형을 지원하지 않습니다.

예시: srsName 을 가진 단일 도형

```
SELECT ST_GeomFromKML('
             <LineString>
                    <coordinates
>-71.1663,42.2614
                              -71.1667,42.2616</coordinates>
             </LineString
>');
```

참고: Section 2.4.1, “설정”, ST_AsKML

8.4.16. ST_GMLToSQL

ST_GMLToSQL — GML 로부터 지정된 ST_Geometry 값을 반환합니다. 이 함수는 ST_GeomFromGML 함수와 동일합니다.

Synopsis

geometry **ST_GMLToSQL**(text *geomgml*);

geometry **ST_GMLToSQL**(text *geomgml*, integer *srid*);

설명:

✓ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다. 사양 3.2.6.2 - 선택적인 SRID 를 적합성 스위트(conformance suite)로부터 가져오십시오.

✓ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 5.1.40

✓ 이 방법은 원형 스트링 및 곡선을 지원합니다.

1.5 버전부터 사용할 수 있습니다. LibXML2 1.6 이상 버전이 필요합니다.

개선 사항: 2.0.0 버전부터 다면체 표면(polyhedral surface) 및 TIN 을 지원합니다.

개선 사항: 2.0.0 버전에서 선택적인 기본 SRID 파라미터가 추가됐습니다.

참고: Section 2.4.1, “설정”, ST_GeomFromGML, ST_AsGML

8.4.17. ST_GeomFromText

ST_GeomFromText — WKT 표현식으로부터 지정된 ST_Geometry 값을 반환합니다.

Synopsis

geometry **ST_GeomFromText**(text *WKT*);

geometry **ST_GeomFromText**(text *WKT*, integer *srid*);

설명: OGC WKT(Well-Known Text) 표현식으로부터 PostGIS ST_Geometry 객체를 작성합니다.



ST_GeomFromText 함수의 변종이 2 개 있는데, 첫 번째는 SRID 를 입력받지 않고 공간 참조 시스템이 정의되지 않은(SRID=0) 도형을 반환합니다. 두 번째는 SRID 를 두 번째 인수로 입력받아 해당 SRID 를 자체 메타데이터의 일부로 포함하는 도형을 반환합니다.

✓ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다. 사양 3.2.6.2 - 선택적인 SRID 를 적합성 스위트(conformance suite)로부터 가져오십시오.

✓ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 5.1.40

✓ 이 방법은 원형 스트링 및 곡선을 지원합니다.



변경 사항: PostGIS 2.0.0 미만 버전에서는 ST_GeomFromText('GEOMETRYCOLLECTION(EMPTY)') 를 사용할 수 있었습니다. PostGIS 2.0.0 버전부터, SQL/MM 표준을 더 잘 준수하기 위해 이런 사용법은 금지됐습니다. 이제는 ST_GeomFromText('GEOMETRYCOLLECTION EMPTY') 라고 작성해야 합니다.

예시

```
SELECT     ST_GeomFromText('LINESTRING(-71.160281     42.258729,-71.160837
42.259113,-71.161144 42.25932)');
SELECT     ST_GeomFromText('LINESTRING(-71.160281     42.258729,-71.160837
42.259113,-71.161144 42.25932)',4269);

SELECT  ST_GeomFromText('MULTILINESTRING((-71.160281  42.258729,-71.160837
42.259113,-71.161144 42.25932))');

SELECT ST_GeomFromText('POINT(-71.064544 42.28787)');

SELECT    ST_GeomFromText('POLYGON((-71.1776585052917    42.3902909739571,-
71.1776820268866 42.3903701743239,
-71.1776063012595   42.3903825660754,-71.1775826583081   42.3903033653531,-
71.1776585052917 42.3902909739571))');

SELECT ST_GeomFromText('MULTIPOLYGON(((-71.1031880899493 42.3152774590236,
-71.1031627617667 42.3152960829043,-71.102923838298 42.3149156848307,
-71.1023097974109 42.3151969047397,-71.1019285062273 42.3147384934248,
-71.102505233663 42.3144722937587,-71.10277487471 42.3141658254797,
-71.103113945163 42.3142739188902,-71.10324876416 42.31402489987,
-71.1033002961013 42.3140393340215,-71.1033488797549 42.3139495090772,
-71.103396240451 42.3138632439557,-71.1041521907712 42.3141153348029,
-71.1041411411543 42.3141545014533,-71.1041287795912 42.3142114839058,
-71.1041188134329 42.3142693656241,-71.1041112482575 42.3143272556118,
-71.1041072845732 42.3143851580048,-71.1041057218871 42.3144430686681,
-71.1041065602059 42.3145009876017,-71.1041097995362 42.3145589148055,
-71.1041166403905 42.3146168544148,-71.1041258822717 42.3146748022936,
-71.1041375307579 42.3147318674446,-71.1041492906949 42.3147711126569,
-71.1041598612795 42.314808571739,-71.1042515013869 42.3151287620809,
-71.1041173835118 42.3150739481917,-71.1040809891419 42.3151344119048,
-71.1040438678912 42.3151191367447,-71.1040194562988 42.3151832057859,
-71.1038734225584 42.3151140942995,-71.1038446938243 42.3151006300338,
-71.1038315271889 42.315094347535,-71.1037393329282 42.315054824985,
-71.1035447555574 42.3152608696313,-71.1033436658644 42.3151648370544,
-71.1032580383161 42.3152269126061,-71.103223066939 42.3152517403219,
```

```
-71.1031880899493 42.3152774590236)),
((-71.1043632495873 42.315113108546,-71.1043583974082 42.3151211109857,
-71.1043443253471  42.3150676015829,-71.1043850704575  42.3150793250568,-
71.1043632495873 42.315113108546)))',4326);

SELECT ST_GeomFromText('CIRCULARSTRING(220268 150415,220227 150505,220227
150406)');
```

참고: ST_GeomFromEWKT, ST_GeomFromWKB, ST_SRID

8.4.18. ST_GeomFromWKB

ST_GeomFromWKB — WKB(Well-Known Binary) 도형 표현식과 선택적인 SRID 로부터 도형 인스턴스를 생성합니다.

Synopsis

geometry **ST_GeomFromWKB**(bytea *geom*);

geometry **ST_GeomFromWKB**(bytea *geom*, integer *srid*);

설명: ST_GeogFromWKB 함수는 도형의 WKB 표현식과 SRID(공간 참조 시스템 ID)를 받아 적절한 도형 유형의 인스턴스를 생성합니다. 이 함수는 SQL 에서 도형 공장(Geometry Factory) 역할을 합니다. 이 함수는 ST_WKBToSQL 함수를 대체할 수 있습니다.

SRID 를 설정하지 않은 경우, 기본값으로 0(unkown)을 씁니다.

✓ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다. 사양 3.2.6.2 - 선택적인 SRID 를 적합성 스위트(conformance suite)로부터 가져오십시오.

✓ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 5.1.40

✓ 이 방법은 원형 스트링 및 곡선을 지원합니다.

예시

```
-- standard_conforming_strings 이 활성화돼 있지 않은 경우
             -- bytea 표현식이 단일한 역슬래시(\)를 포함하긴 해도, 테이블 삽입시에는
빼놓아야 합니다.
SELECT ST_AsEWKT(
ST_GeomFromWKB(E'\\001\\002\\000\\000\\000\\002\\000\\000\\000\\037\\205\\3
53Q\\270~\\\\\\300\\323Mb\\020X\\231C@\\020X9\\264\\310~\\\\\\300)\\\\\\217
\\302\\365\\230C@',4326)
);
                                  st_asewkt
----------------------------------------------------
 SRID=4326;LINESTRING(-113.98 39.198,-113.981 39.195)
(1 row)

SELECT
  ST_AsText(
      ST_GeomFromWKB(
        ST_AsEWKB('POINT(2 5)'::geometry)
```

```
      )
  );
 st_astext
------------
 POINT(2 5)
(1 row)
```

참고: ST_WKBToSQL, ST_AsBinary, ST_GeomFromEWKB

8.4.19. ST_LineFromEncodedPolyline

ST_LineFromEncodedPolyline — 인코딩된 폴리라인(polyline)으로부터 라인스트링을 생성합니다.

Synopsis

geometry **ST_LineFromEncodedPolyline**(text *polyline*, integer *precision=5*);

설명: 인코딩된 폴리라인 스트링으로부터 라인스트링을 생성합니다.

참조: http://developers.google.com/maps/documentation/utilities/polylinealgorithm

2.2.0 버전부터 사용할 수 있습니다.

예시

```
-- 폴리라인으로부터 라인스트링 생성하기
SELECT
ST_AsEWKT(ST_LineFromEncodedPolyline('_p~iF~ps|U_ulLnnqC_mqNvxq`@'));
-- 결과 --
LINESTRING(-120.2 38.5,-120.95 40.7,-126.453 43.252)
```

참고: ST_AsEncodedPolyline

8.4.20. ST_LineFromMultiPoint

ST_LineFromMultiPoint — 멀티포인트 도형으로부터 라인스트링을 생성합니다.

Synopsis

geometry **ST_LineFromMultiPoint**(geometry *aMultiPoint*);

설명: 멀티포인트 도형으로부터 라인스트링을 생성합니다.

✓ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
-- 3D 멀티포인트로부터 3D 라인스트링 생성하기
SELECT  ST_AsEWKT(ST_LineFromMultiPoint(ST_GeomFromEWKT('MULTIPOINT(1  2  3,
4  5  6,  7  8  9)')));
-- 결과 --
```

```
LINESTRING(1 2 3,4 5 6,7 8 9)
```

참고: ST_AsEWKT, ST_Collect, ST_MakeLine

8.4.21. ST_LineFromText

ST_LineFromText — 주어진 SRID 와 함께 WKT 표현식으로부터 도형을 만듭니다. SRID 가 주어지지 않은 경우, 기본값인 0 을 씁니다.

Synopsis

geometry **ST_LineFromText**(text *WKT*);

geometry **ST_LineFromText**(text *WKT*, integer *srid*);

설명: 주어진 SRID 와 함께 WKT 로부터 도형을 만듭니다. SRID 가 주어지지 않은 경우, 기본값인 0 을 씁니다. 입력된 WKT 가 라인스트링이 아닌 경우, null 을 반환합니다.



OGC 사양 3.2.6.2 - 선택적인 SRID 를 적합성 스위트(conformance suite)로부터 가져오십시오.



사용자 도형이 모두 라인스트링이란 걸 알고 있다면, 그냥 ST_GeomFromText 함수를 쓰는 편이 더 효율적입니다. 이 함수는 ST_GeomFromText 만 호출하고, 라인스트링을 반환한다는 유효성 검사를 추가합니다.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다. s3.2.6.2

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 7.2.9

예시

```
SELECT    ST_LineFromText('LINESTRING(1    2,    3    4)')    AS    aline,
ST_LineFromText('POINT(1 2)') AS null_return;
aline                       | null_return
-----------------------------------------------
0102000000020000000000000000F ... | t
```

참고: ST_GeomFromText

8.4.22. ST_LineFromWKB

ST_LineFromWKB — 주어진 SRID 와 함께 WKB 로부터 LINESTRING 을 만듭니다.

Synopsis

geometry **ST_LineFromWKB**(bytea *WKB*);

geometry **ST_LineFromWKB**(bytea *WKB*, integer *srid*);

설명: ST_LineFromWKB 함수는 도형의 WKB 표현식과 SRID(공간 참조 시스템 ID)를 받아 적절한 도형 유형의 인스턴스 - 이 경우, LINESTRING 도형 - 를 생성합니다. 이 함수는 SQL 에서 도형 공장(Geometry Factory) 역할을 합니다.

SRID 를 설정하지 않은 경우, 기본값인 0 을 씁니다. 입력된 bytea 가 라인스트링이 아닌 경우, NULL 을 반환합니다.



OGC 사양 3.2.6.2 - 선택적인 SRID 를 적합성 스위트(conformance suite)로부터 가져오십시오.



사용자 도형이 모두 LINESTRING 이란 걸 알고 있다면, 그냥 ST_GeomFromWKB 함수를 쓰는 편이 더 효율적입니다. 이 함수는 ST_GeomFromWKB 만 호출하고, 라인스트링을 반환한다는 유효성 검사를 추가합니다.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다. s3.2.6.2

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 7.2.9

예시

```
SELECT   ST_LineFromWKB(ST_AsBinary(ST_GeomFromText('LINESTRING(1   2,   3
4)'))) AS aline,
            ST_LineFromWKB(ST_AsBinary(ST_GeomFromText('POINT(1 2)'))) IS
NULL AS null_return;
aline                        | null_return
---------------------------------------------
0102000000020000000000000000F ... | t
```

참고: ST_GeomFromWKB, ST_LinestringFromWKB

8.4.23. ST_LinestringFromWKB

ST_LinestringFromWKB — 주어진 SRID 와 함께 WKB 로부터 도형을 만듭니다.

Synopsis

geometry **ST_LinestringFromWKB**(bytea *WKB*);

geometry **ST_LinestringFromWKB**(bytea *WKB*, integer *srid*);

설명: ST_LinestringFromWKB 함수는 도형의 WKB 표현식과 SRID(공간 참조 시스템 ID)를 받아 적절한 도형 유형의 인스턴스 - 이 경우, LINESTRING 도형 - 를 생성합니다. 이 함수는 SQL 에서 도형 공장(Geometry Factory) 역할을 합니다.

SRID 를 설정하지 않은 경우, 기본값인 0 을 씁니다. 입력된 bytea 가 LINESTRING 도형이 아닌 경우, NULL 을 반환합니다.



OGC 사양 3.2.6.2 - 선택적인 SRID 를 적합성 스위트(conformance suite)로부터 가져오십시오.



사용자 도형이 모두 LINESTRING 이란 걸 알고 있다면, 그냥 ST_GeomFromWKB 함수를 쓰는 편이 더 효율적입니다. 이 함수는 ST_GeomFromWKB 만 호출하고, LINESTRING 을 반환한다는 유효성 검사를 추가합니다.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다. s3.2.6.2

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 7.2.9

예시

```
SELECT
  ST_LineStringFromWKB(
       ST_AsBinary(ST_GeomFromText('LINESTRING(1 2, 3 4)'))
  ) AS aline,
  ST_LinestringFromWKB(
       ST_AsBinary(ST_GeomFromText('POINT(1 2)'))
  ) IS NULL AS null_return;
   aline                        | null_return
-----------------------------------------------
0102000000020000000000000000F ... | t
```

참고: ST_GeomFromWKB, ST_LineFromWKB

8.4.24. ST_MakeBox2D

ST_MakeBox2D — 주어진 포인트 도형들로 정의되는 BOX2D 를 생성합니다.

Synopsis

box2d **ST_MakeBox2D**(geometry *pointLowLeft*, geometry *pointUpRight*);

설명: 주어진 포인트 도형들로 정의되는 BOX2D 를 생성합니다. 이 함수는 범위(range) 쿼리에 쓸모가 있습니다.

예시

```
-- 미국 전국지도 좌표 경계 상자 내부에 존재하는 모든 피처를 반환합니다.
-- 도형들이 SRID 2163(미국 등면적(equal area) 전국지도)으로 저장됐다고 가정합니다.
SELECT feature_id, feature_name, the_geom
FROM features
WHERE     the_geom     &&     ST_SetSRID(ST_MakeBox2D(ST_Point(-989502.1875,
528439.5625),
       ST_Point(-987121.375 ,529933.1875)),2163)
```

참고: ST_MakePoint, ST_Point, ST_SetSRID, ST_SRID

8.4.25. ST_3DMakeBox

ST_3DMakeBox — 주어진 3D 포인트 도형들로 정의되는 BOX3D 를 생성합니다.

Synopsis

box3d **ST_3DMakeBox**(geometry *point3DLowLeftBottom*, geometry *point3DUpRightTop*);

설명: 주어진 3D 포인트 도형 2 개로 정의되는 BOX3D 를 생성합니다.

✔ 이 함수는 3 차원을 지원하며 Z 인덱스를 삭제하지 않습니다.

변경 사항: 2.0.0 미만 버전에서는 ST_MakeBox3D 라는 명칭이었습니다.

예시

```
SELECT ST_3DMakeBox(ST_MakePoint(-989502.1875, 528439.5625, 10),
      ST_MakePoint(-987121.375 ,529933.1875, 10)) As abb3d

--bb3d--
--------
BOX3D(-989502.1875 528439.5625 10,-987121.375 529933.1875 10)
```

참고: ST_MakePoint, ST_SetSRID, ST_SRID

8.4.26. ST_MakeLine

ST_MakeLine — 포인트, 멀티포인트 또는 라인 도형으로부터 라인스트링을 생성합니다.

Synopsis

geometry **ST_MakeLine**(geometry set *geoms*);

geometry **ST_MakeLine**(geometry *geom1*, geometry *geom2*);

geometry **ST_MakeLine**(geometry[] *geoms_array*);

설명: ST_MakeLine 함수는 세 가지 형태로 나타납니다: 포인트, 멀티포인트 또는 라인 도형들의 행을 입력받아 라인스트링을 반환하는 공간 합산(spatial aggregate) 함수, 포인트, 멀티포인트 또는 라인 배열을 입력받는 함수, 그리고 포인트, 멀티포인트 또는 라인 도형 2 개를 입력받는 정규 함수. 이 함수의 합산 버전에 포인트를 입력하기 전에 하위집합을 이용해서 포인트들을 정렬하는 편이 좋습니다.

포인트, 멀티포인트, 또는 라인이 아닌 도형을 입력하면 무시할 것입니다.

라인 구성 요소를 추가할 경우 출력물에서 라인의 시작점에 있는 일반 노드를 제거합니다. 포인트 및 멀티포인트의 일반 노드는 제거되지 않습니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

1.4.0 버전부터 사용할 수 있습니다. 이 버전부터 ST_MakeLine 가 도형 배열을 입력받을 수 있습니다. 더 많은 포인트를 더 빨리 처리하기 위해 ST_MakeLine 합산 함수를 개선했습니다.

2.0.0 버전부터 라인스트링 구성 요소 입력을 지원하기 시작했습니다.

2.0.0 버전부터 멀티포인트 구성 요소 입력을 지원하기 시작했습니다.

예시: 공간 합산 버전

이 예시는 GPS 포인트 배열을 입력받아, 도형 항목이 이동 순서대로의 GPS 포인트들로 이루어진 라인스트링인 GPS 이동 하나당 한 개의 레코드를 생성합니다.

```
-- PostgreSQL 9.0 미만 버전에서는 일반적으로 동작하지만,
-- 때때로 설계자가 하위 쿼리의 순서를 무시하기로 선택하는 경우가 있습니다.
SELECT gps.gps_track, ST_MakeLine(gps.the_geom) As newgeom
      FROM (SELECT gps_track,gps_time, the_geom
                  FROM gps_points ORDER BY gps_track, gps_time) As gps
      GROUP BY gps.gps_track;
-- PostgreSQL 9.0 이상 버전을 사용중이라면
-- (합산을 위해 새로운 'ORDER BY' 지원을 사용할 수 있습니다)
-- 다음은 정확히 정렬된 라인스트링을 얻을 수 있는 확실한 방법입니다.
-- 필요할 경우 하나 이상의 열을 통해 사용자 배열을 부분적으로 정렬할 수 있습니다.
SELECT  gps.gps_track,  ST_MakeLine(gps.the_geom  ORDER  BY  gps_time)  As
newgeom
      FROM gps_points As gps
      GROUP BY gps.gps_track;
```

예시: 비공간 합산 버전

첫 번째는 포인트 2 개로 이루어진 라인스트링을 한 번에 만드는 간단한 예시입니다. 두 번째는 사용자가 그린 포인트 2 개로부터 라인스트링을 작성합니다. 세 번째는 3D 포인트 2 개를 연결해서 3 차원 공간에 라인을 한 번에 생성합니다.

```
SELECT ST_AsText(ST_MakeLine(ST_MakePoint(1,2), ST_MakePoint(3,4)));
        st_astext
---------------------
 LINESTRING(1 2,3 4)

SELECT userpoints.id, ST_MakeLine(startpoint, endpoint) As drawn_line
      FROM userpoints ;

SELECT ST_AsEWKT(ST_MakeLine(ST_MakePoint(1,2,3), ST_MakePoint(3,4,5)));
            st_asewkt
--------------------------
 LINESTRING(1 2 3,3 4 5)
```

예시: 배열 버전 사용하기

```
SELECT ST_MakeLine(ARRAY(SELECT ST_Centroid(the_geom) FROM visit_locations
ORDER BY visit_time));

-- 3D 포인트 3 개로 3D 라인 만들기
SELECT ST_AsEWKT(ST_MakeLine(ARRAY[ST_MakePoint(1,2,3),
                          ST_MakePoint(3,4,5), ST_MakePoint(6,6,6)]));
```

```
             st_asewkt
-------------------------
LINESTRING(1 2 3,3 4 5,6 6 6)
```

참고: ST_AsEWKT, ST_AsText, ST_GeomFromText, ST_MakePoint

8.4.27. ST_MakeEnvelope

ST_MakeEnvelope — 주어진 최소값과 최대값으로 형성된 직사각형 폴리곤을 생성합니다. 입력값이 SRID 로 지정된 SRS 를 가지고 있어야 합니다.

Synopsis

geometry **ST_MakeEnvelope**(double precision *xmin*, double precision *ymin*, double precision *xmax*, double precision *ymax*, integer *srid=unknown*);

설명: 최소값과 최대값으로 형성된 직사각형 폴리곤을 생성합니다. 입력값이 SRID 로 지정된 SRS 를 가지고 있어야 합니다. 지정된 SRID 가 없을 경우 알려지지 않은 공간 참조 시스템을 쓴다고 가정합니다.

1.5 버전부터 사용할 수 있습니다.

개선 사항: 2.0 버전부터 SRID 를 지정하지 않고도 직사각형(envelope)을 설정할 수 있는 기능이 추가됐습니다.

예시: 경계 상자 폴리곤을 빌드하기

```
SELECT DropGeometryTable ('my_schema','my_spatial_table');
SELECT ST_AsText(ST_MakeEnvelope(10, 10, 11, 11, 4326));

st_asewkt
-----------
POLYGON((10 10, 10 11, 11 11, 11 10, 10 10))
```

참고: ST_MakePoint, ST_MakeLine, ST_MakePolygon

8.4.28. ST_MakePolygon

ST_MakePolygon — 주어진 외부 구조(shell)로 형성된 폴리곤을 생성합니다. 입력 도형이 닫힌 라인스트링이어야 합니다.

Synopsis

geometry **ST_MakePolygon**(geometry *linestring*);

geometry **ST_MakePolygon**(geometry *outerlinestring*, geometry[] *interiorlinestrings*);

설명: 주어진 외부 구조(shell)로 형성된 폴리곤을 생성합니다. 입력 도형이 닫힌 라인스트링이어야 합니다. 두 가지 변종이 있습니다.

변종 1: 닫힌 라인스트링 하나를 입력받습니다.

변종 2: 주어진 외부 구조 및 구멍 배열로 형성된 폴리곤을 생성합니다. ST_Accum 또는 PostgreSQL ARRAY[] 및 ARRAY() 구조를 이용해서 도형 배열을 작성할 수 있습니다. 입력 도형은 닫힌 라인스트링이어야 합니다.



이 함수에 멀티라인스트링을 입력할 수는 없습니다. 라인스트링을 생성하려면 ST_LineMerge 또는 ST_Dump 를 이용하십시오..

✓ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시: 단일 폐쇄형 라인스트링

```
-- 2D 라인
SELECT  ST_MakePolygon(ST_GeomFromText('LINESTRING(75.15  29.53,77  29,77.6
29.5, 75.15 29.53)'));
-- 닫힌 라인스트링이 아닐 경우
-- 시작점을 추가해서 닫을 수 있습니다
SELECT                              ST_MakePolygon(ST_AddPoint(foo.open_line,
ST_StartPoint(foo.open_line)))
FROM (
SELECT  ST_GeomFromText('LINESTRING(75.15  29.53,77  29,77.6  29.5)')  As
open_line) As foo;

-- 닫힌 3D 라인
SELECT  ST_MakePolygon(ST_GeomFromText('LINESTRING(75.15  29.53  1,77  29
1,77.6 29.5 1, 75.15 29.53 1)'));

st_asewkt
-----------
POLYGON((75.15 29.53 1,77 29 1,77.6 29.5 1,75.15 29.53 1))

-- 측정 라인(measured line) --
SELECT  ST_MakePolygon(ST_GeomFromText('LINESTRINGM(75.15  29.53  1,77  29
1,77.6 29.5 2, 75.15 29.53 2)'));

st_asewkt
----------
POLYGONM((75.15 29.53 1,77 29 1,77.6 29.5 2,75.15 29.53 2))
```

예시: 내곽 구조를 가진 외곽 구조

개미 구멍을 가진 도넛을 빌드해봅시다.

```
SELECT ST_MakePolygon(
            ST_ExteriorRing(ST_Buffer(foo.line,10)),
      ARRAY[ST_Translate(foo.line,1,1),
            ST_ExteriorRing(ST_Buffer(ST_MakePoint(20,20),1)) ]
      )
FROM
      (SELECT ST_ExteriorRing(ST_Buffer(ST_MakePoint(10,10),10,10))
            As line )
            As foo;
```

지역 폴리곤/멀티폴리곤과 수변 라인스트링의 집합으로부터 지역 내에 있는 호수를 표현하는 구멍들을 가진 지역 경계선을 빌드해봅시다. 이 예시에서는 PostGIS ST_Accum 함수를 사용합니다.



NULL 배열을 ST_MakePolygon 에 입력하면 NULL 을 반환하기 때문에 CASE 구조를 활용합니다.



호수가 없는 경우라도 모든 지역을 반환받도록 보장하기 위해 LEFT JOIN 을 활용합니다.

```
SELECT p.gid, p.province_name,
            CASE WHEN
                  ST_Accum(w.the_geom) IS NULL THEN p.the_geom
            ELSE    ST_MakePolygon(ST_LineMerge(ST_Boundary(p.the_geom)),
ST_Accum(w.the_geom)) END
      FROM
            provinces p LEFT JOIN waterlines w
                  ON      (ST_Within(w.the_geom,      p.the_geom)      AND
ST_IsClosed(w.the_geom))
      GROUP BY p.gid, p.province_name, p.the_geom;

-- 앞과 동일한 예시이지만, 상관 하위 쿼리와
-- 행의 집합을 배열로 변환하는 PostgreSQL 내장 ARRAY() 함수를 활용합니다.

      SELECT p.gid,  p.province_name, CASE WHEN
            EXISTS(SELECT w.the_geom
                  FROM waterlines w
                  WHERE ST_Within(w.the_geom, p.the_geom)
                  AND ST_IsClosed(w.the_geom))
            THEN
            ST_MakePolygon(ST_LineMerge(ST_Boundary(p.the_geom)),
                  ARRAY(SELECT w.the_geom
                        FROM waterlines w
                        WHERE ST_Within(w.the_geom, p.the_geom)
                        AND ST_IsClosed(w.the_geom)))
            ELSE p.the_geom END As the_geom
      FROM
            provinces p;
```

참고: ST_Boundary, ST_Accum, ST_AddPoint, ST_GeometryType, ST_IsClosed, ST_LineMerge, ST_BuildArea

8.4.29. ST_MakePoint

ST_MakePoint — 2D, 3DZ 또는 4D 포인트 도형을 생성합니다.

Synopsis

geometry **ST_MakePoint**(double precision *x*, double precision *y*);

geometry **ST_MakePoint**(double precision *x*, double precision *y*, double precision *z*);

geometry **ST_MakePoint**(double precision *x*, double precision *y*, double precision *z*, double precision *m*);

설명: 2D, 3DZ 또는 4D 포인트 도형(단위를 가진 도형)을 생성합니다. ST_MakePoint 함수가 OGC 를 준수하지는 않지만, 일반적으로 ST_GeomFromText 및 ST_PointFromText 함수보다 더 빠르고 정확합니다. 또 WKT 가 아니라 처리되지 않은 좌표를 쓰는 경우, 더 쉽게 사용할 수 있습니다.


x 가 경도이고 y 가 위도라는 점을 주의하십시오.


x, y, m 좌표를 가진 포인트를 만들어야 할 경우, ST_MakePointM 함수를 이용하십시오.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
-- 알려지지 않은 SRID 를 가진 포인트를 반환
SELECT ST_MakePoint(-71.1043443253471, 42.3150676015829);

-- WGS84 경위도로 표시된 포인트를 반환
SELECT ST_SetSRID(ST_MakePoint(-71.1043443253471, 42.3150676015829),4326);

-- 3D (예를 들자면 고도를 가진) 포인트를 반환
SELECT ST_MakePoint(1, 2,1.5);

-- 포인트의 z 를 얻어오기
SELECT ST_Z(ST_MakePoint(1, 2,1.5));
result
-------
1.5
```

참고: ST_GeomFromText, ST_PointFromText, ST_SetSRID, ST_MakePointM

8.4.30. ST_MakePointM

ST_MakePointM — x, y, m 좌표를 가진 포인트 도형을 생성합니다.

Synopsis

geometry **ST_MakePointM**(float *x*, float *y*, float *m*);

설명: x, y 좌표 및 단위를 가진 포인트를 생성합니다.


x 가 경도이고 y 가 위도라는 점을 주의하십시오.

예시: 이번 예시에서 ST_AsEWKT 함수를 사용하는 이유는, ST_AsText 함수가 m 을 반환하지 못 하기 때문에 ST_AsText 대신 텍스트 표현식을 표출하기 위해서 입니다.

```
-- 알려지지 않은 SRID 를 가진 포인트의 EWKT 표현식을 반환
SELECT ST_AsEWKT(ST_MakePointM(-71.1043443253471, 42.3150676015829, 10));

-- 결과
st_asewkt
-----------------------------------------------
 POINTM(-71.1043443253471 42.3150676015829 10)

-- WGS84 경위도로 표시된 단위를 가진 포인트의 EWKT 표현식을 반환
SELECT               ST_AsEWKT(ST_SetSRID(ST_MakePointM(-71.1043443253471,
42.3150676015829,10),4326));

st_asewkt
------------------------------------------------------------
SRID=4326;POINTM(-71.1043443253471 42.3150676015829 10)

-- 3D (예를 들자면 고도를 가진) 포인트를 반환
SELECT ST_MakePoint(1, 2,1.5);

-- 포인트의 m 을 얻어오기
SELECT ST_M(ST_MakePointM(-71.1043443253471, 42.3150676015829,10));
result
-------
10
```

참고: ST_AsEWKT, ST_MakePoint, ST_SetSRID

8.4.31. ST_MLineFromText

ST_MLineFromText — WKT 표현식으로부터 지정된 ST_MultiLineString 값을 반환합니다.

Synopsis

geometry **ST_MLineFromText**(text *WKT*, integer *srid*);

geometry **ST_MLineFromText**(text *WKT*);

설명: 주어진 SRID 와 함께 WKT 표현식으로부터 도형을 만듭니다. SRID 가 주어지지 않은 경우, 기본값인 0 을 씁니다.

OGC 사양 3.2.6.2 - 선택적인 SRID 를 적합성 스위트(conformance suite)로부터 가져오십시오.

WKT 가 멀티라인스트링이 아닌 경우 null 을 반환합니다.



사용자 WKT 도형이 모두 포인트라고 확신한다면, 이 함수를 사용하지 마십시오. 이 함수는 추가적인 유효성 검사 단계를 거치므로 ST_GeomFromText 보다 느립니다.

✓ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다. s3.2.6.2

✓ 이 방법은 SQL/MM 사양을 도입합니다.SQL-MM 3: 9.4.4

예시

```
SELECT ST_MLineFromText('MULTILINESTRING((1 2, 3 4), (4 5, 6 7))');
```

참고: ST_GeomFromText

8.4.32. ST_MPointFromText

ST_MPointFromText — 주어진 SRID 와 함께 WKT 표현식으로부터 도형 집합을 만듭니다. SRID 가 주어지지 않은 경우, 기본값인 0 을 씁니다.

Synopsis

geometry **ST_MPointFromText**(text *WKT*, integer *srid*);

geometry **ST_MPointFromText**(text *WKT*);

설명: 주어진 SRID 와 함께 WKT 표현식으로부터 도형 집합을 만듭니다. SRID 가 주어지지 않은 경우, 기본값인 0 을 씁니다.

OGC 사양 3.2.6.2 - 선택적인 SRID 를 적합성 스위트(conformance suite)로부터 가져오십시오.

WKT 가 멀티포인트가 아닌 경우 null 을 반환합니다.



사용자 WKT 도형이 모두 포인트라고 확신한다면, 이 함수를 사용하지 마십시오. 이 함수는 추가적인 유효성 검사 단계를 거치므로 ST_GeomFromText 보다 느립니다.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다. s3.2.6.2

✔ 이 방법은 SQL/MM 사양을 도입합니다.SQL-MM 3: 9.4.4

예시

```
SELECT ST_MPointFromText('MULTIPOINT(1 2, 3 4)');
SELECT ST_MPointFromText('MULTIPOINT(-70.9590 42.1180, -70.9611 42.1223)',
4326);
```

참고: ST_GeomFromText

8.4.33. ST_MPolyFromText

ST_MPolyFromText — 주어진 SRID 와 함께 WKT 표현식으로부터 멀티폴리곤을 만듭니다. SRID 가 주어지지 않은 경우, 기본값인 0 을 씁니다.

Synopsis

geometry **ST_MPolyFromText**(text *WKT*, integer *srid*);

geometry **ST_MPolyFromText**(text *WKT*);

설명: 주어진 SRID 와 함께 WKT 표현식으로부터 멀티폴리곤을 만듭니다. SRID 가 주어지지 않은 경우, 기본값인 0 을 씁니다.

OGC 사양 3.2.6.2 - 선택적인 SRID 를 적합성 스위트(conformance suite)로부터 가져오십시오.

WKT 가 멀티폴리곤이 아닌 경우 오류가 발생합니다.



사용자 WKT 도형이 모두 멀티폴리곤이라고 확신한다면, 이 함수를 사용하지 마십시오. 이 함수는 추가적인 유효성 검사 단계를 거치므로 ST_GeomFromText 보다 느립니다.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다. s3.2.6.2

✔ 이 방법은 SQL/MM 사양을 도입합니다.SQL-MM 3: 9.4.4

예시

```
SELECT ST_MPolyFromText('MULTIPOLYGON(((0 0 1,20 0 1,20 20 1,0 20 1,0 0
1),(5 5 3,5 7 3,7 7 3,7 5 3,5 5 3)))');
SELECt ST_MPolyFromText('MULTIPOLYGON(((-70.916 42.1002,-70.9468 42.0946,-
70.9765   42.0872,-70.9754   42.0875,-70.9749   42.0879,-70.9752   42.0881,-
70.9754   42.0891,-70.9758   42.0894,-70.9759   42.0897,-70.9759   42.0899,-
70.9754   42.0902,-70.9756   42.0906,-70.9753   42.0907,-70.9753   42.0917,-
70.9757   42.0924,-70.9755   42.0928,-70.9755   42.0942,-70.9751   42.0948,-
70.9755   42.0953,-70.9751   42.0958,-70.9751   42.0962,-70.9759   42.0983,-
70.9767   42.0987,-70.9768   42.0991,-70.9771   42.0997,-70.9771   42.1003,-
70.9768 42.1005,-70.977 42.1011,-70.9766 42.1019,-70.9768 42.1026,-70.9769
42.1033,-70.9775    42.1042,-70.9773    42.1043,-70.9776    42.1043,-70.9778
42.1048,-70.9773    42.1058,-70.9774    42.1061,-70.9779    42.1065,-70.9782
42.1078,-70.9788    42.1085,-70.9798    42.1087,-70.9806    42.109,-70.9807
42.1093,-70.9806    42.1099,-70.9809    42.1109,-70.9808    42.1112,-70.9798
42.1116,-70.9792    42.1127,-70.979    42.1129,-70.9787    42.1134,-70.979
42.1139,-70.9791 42.1141,-70.9987 42.1116,-71.0022 42.1273,
      -70.9408 42.1513,-70.9315 42.1165,-70.916 42.1002)))',4326);
```

참고: ST_GeomFromText, ST_SRID

8.4.34. ST_Point

ST_Point — 주어진 좌표값을 가진 ST_Point 를 반환합니다. ST_MakePoint 와 동일한 OGC 함수입니다.

Synopsis

geometry **ST_Point**(float *x_lon*, float *y_lat*);

설명: 주어진 좌표값을 가진 ST_Point 를 반환합니다. MM 을 준수하며 x, y 만을 입력받는, ST_MakePoint 와 동일한 함수입니다.

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 6.1.2

예시: 도형

```
SELECT ST_SetSRID(ST_Point(-71.1043443253471, 42.3150676015829),4326)
```

예시: 지리형

```
SELECT CAST(ST_SetSRID(ST_Point(-71.1043443253471, 42.3150676015829),4326)
As geography);
```

```
-- '::'는 형변환을 의미하는 PostgreSQL 의 표기입니다.
SELECT ST_SetSRID(ST_Point(-71.1043443253471,
42.3150676015829),4326)::geography;
```

```
-- 사용자 포인트의 좌표계가 WGS84 경위도와 다를 경우, 형변환 전에 재투영해야 합니다.
-- 이 예시에서는 펜실베이니아 주의 피트 단위 평면에 있는 포인트를 WGS84 로 재투영한 다음
지리형으로 형변환합니다.
SELECT                          ST_Transform(ST_SetSRID(ST_Point(3637510,
3014852),2273),4326)::geography;
```

참고: Section 4.2.1, "지리형의 기초", ST_MakePoint, ST_SetSRID, ST_Transform

8.4.35. ST_PointFromGeoHash

ST_PointFromGeoHash — GeoHash 스트링으로부터 포인트를 반환합니다.

Synopsis

point **ST_PointFromGeoHash**(text *geohash*, integer *precision=full_precision_of_geohash*);

설명: GeoHash 스트링으로부터 포인트를 반환합니다. 해당 포인트는 GeoHash 의 중심점입니다.

precision 을 설정하지 않은 경우, ST_PointFromGeoHash 가 입력 GeoHash 스트링의 전체 정확도를 기반으로 포인트를 반환합니다.

precision 을 설정한 경우, ST_PointFromGeoHash 가 GeoHash 에서 나온 그만큼의 문자를 이용해서 포인트를 생성합니다.

2.1.0 버전부터 사용할 수 있습니다.

예시

```
SELECT ST_AsText(ST_PointFromGeoHash('9qqj7nmxncgyy4d0dbxqz0'));
        st_astext
------------------------------
 POINT(-115.172816 36.114646)

SELECT ST_AsText(ST_PointFromGeoHash('9qqj7nmxncgyy4d0dbxqz0', 4));
          st_astext
----------------------------------
 POINT(-115.13671875 36.123046875)

SELECT ST_AsText(ST_PointFromGeoHash('9qqj7nmxncgyy4d0dbxqz0', 10));
             st_astext
-------------------------------------------
 POINT(-115.172815918922 36.1146435141563)
```

참고: ST_GeoHash, ST_Box2dFromGeoHash, ST_GeomFromGeoHash

8.4.36. ST_PointFromText

ST_PointFromText — 주어진 SRID 와 함께 WKT 표현식으로부터 포인트 도형을 만듭니다. SRID 가 주어지지 않은 경우, 기본값인 0 을 씁니다.

Synopsis

geometry **ST_PointFromText**(text *WKT*);

geometry **ST_PointFromText**(text *WKT*, integer *srid*);

설명: OGC WKT 표현식으로부터 PostGIS ST_Geometry 포인트 객체를 작성합니다. SRID 가 주어지지 않은 경우, 기본값인 0('알려지지 않은')을 씁니다. 도형이 WKT 포인트 표현식이 아닌 경우 null 을 반환합니다. WKT 가 유효하지 않다면, 오류가 발생합니다.



ST_PointFromText 함수의 변종이 2 개 있는데, 첫 번째는 SRID 를 입력받지 않고 공간 참조 시스템이 정의되지 않은 도형을 반환합니다. 두 번째는 SRID 를 두 번째 인수로 입력받아 자체 메타데이터의 일부로 해당 SRID 를 포함하는 ST_Geometry 를 반환합니다. spatial_ref_sys 테이블에 정의되어 있는 SRID 여야 합니다.



사용자 WKT 도형이 모두 포인트라고 확신한다면, 이 함수를 사용하지 마십시오. 이 함수는 추가적인 유효성 검사 단계를 거치므로 ST_GeomFromText 보다 느립니다. 사용자가 경위도 좌표로부터 포인트를 빌드하고 OGC 준수 여부보다 성능 및 정확도에 더 관심이 있다면, ST_MakePoint 또는 OGC 를 준수하는 동일한 ST_Point 함수를 이용하십시오.

 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다. 사양 3.2.6.2 - 선택적인 SRID 를 적합성 스위트(conformance suite)로부터 가져오십시오.

 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 6.1.8

예시

```
SELECT ST_PointFromText('POINT(-71.064544 42.28787)');
SELECT ST_PointFromText('POINT(-71.064544 42.28787)', 4326);
```

참고: ST_GeomFromText, ST_MakePoint, ST_Point, ST_SRID

8.4.37. ST_PointFromWKB

ST_PointFromWKB — 주어진 SRID 와 함께 WKB 로부터 도형을 만듭니다.

Synopsis

geometry **ST_GeomFromWKB**(bytea *geom*);

geometry **ST_GeomFromWKB**(bytea *geom*, integer *srid*);

설명: ST_PointFromWKB 함수는 도형의 WKB 표현식과 SRID(공간 참조 시스템 ID)를 받아 적절한 도형 유형의 인스턴스 - 이 경우, POINT 도형 - 를 생성합니다. 이 함수는 SQL 에서 도형 공장(Geometry Factory) 역할을 합니다.

SRID 를 설정하지 않은 경우, 기본값인 0 을 씁니다. 입력된 bytea 가 포인트가 아닌 경우, NULL 을 반환합니다.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다. s3.2.7.2

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 6.1.9

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

예시

```
SELECT
  ST_AsText(
       ST_PointFromWKB(
         ST_AsEWKB('POINT(2 5)'::geometry)
       )
  );
 st_astext
------------
 POINT(2 5)
(1 row)

SELECT
  ST_AsText(
       ST_PointFromWKB(
         ST_AsEWKB('LINESTRING(2 5, 2 6)'::geometry)
       )
  );
 st_astext
-----------

(1 row)
```

참고: ST_GeomFromWKB, ST_LineFromWKB

8.4.38. ST_Polygon

ST_Polygon — 설정된 라인스트링과 SRID 로부터 빌드된 폴리곤을 반환합니다.

Synopsis

geometry **ST_Polygon**(geometry *aLineString*, integer *srid*);

설명: 설정된 라인스트링과 SRID 로부터 빌드된 폴리곤을 반환합니다.



폴리곤의 SRID 도 설정한다는 점을 제외하면 ST_Polygon 함수는 oST_MakePolygon 첫 번째 버전과 비슷합니다. 멀티라인스트링을 입력받지 못 하기 때문에, LineMerge 를 이용해서 멀티라인을 합치십시오. 또 구멍을 가진 폴리곤도 생성하지 못 합니다. 구멍을 가진 폴리곤을 생성하려면 ST_MakePolygon 함수를 이용하십시오.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다.

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 8.3.2

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
-- 2D 폴리곤
SELECT ST_Polygon(ST_GeomFromText('LINESTRING(75.15 29.53,77 29,77.6 29.5,
75.15 29.53)'), 4326);

-- 결과 --
POLYGON((75.15 29.53,77 29,77.6 29.5,75.15 29.53))
-- 3D 폴리곤
SELECT  ST_AsEWKT(ST_Polygon(ST_GeomFromEWKT('LINESTRING(75.15  29.53  1,77
29 1,77.6 29.5 1,  75.15 29.53 1)'), 4326));

result
------
SRID=4326;POLYGON((75.15 29.53 1,77 29 1,77.6 29.5 1,75.15 29.53 1))
```

참고: ST_AsEWKT, ST_AsText, ST_GeomFromEWKT, ST_GeomFromText, ST_LineMerge, ST_MakePolygon

8.4.39. ST_PolygonFromText

ST_PolygonFromText — 주어진 SRID 와 함께 WKT 표현식으로부터 도형 집합을 만듭니다. SRID 가 주어지지 않은 경우, 기본값인 0 을 씁니다.

Synopsis

geometry **ST_PolygonFromText**(text *WKT*);

geometry **ST_PolygonFromText**(text *WKT*, integer *srid*);

설명: 주어진 SRID 와 함께 WKT 로부터 도형을 만듭니다. SRID 가 주어지지 않은 경우, 기본값인 0 을 씁니다. 입력된 WKT 가 폴리곤이 아닌 경우, null 을 반환합니다.

OGC 사양 3.2.6.2 - 선택적인 SRID 를 적합성 스위트(conformance suite)로부터 가져오십시오.



사용자 WKT 도형이 모두 폴리곤이라고 확신한다면, 이 함수를 사용하지 마십시오. 이 함수는 추가적인 유효성 검사 단계를 거치므로 ST_GeomFromText 보다 느립니다.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다. s3.2.6.2

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 8.3.6

예시

```
SELECT  ST_PolygonFromText('POLYGON((-71.1776585052917  42.3902909739571,-
71.1776820268866 42.3903701743239,
-71.1776063012595  42.3903825660754,-71.1775826583081  42.3903033653531,-
71.1776585052917 42.3902909739571))');
st_polygonfromtext
------------------
0103000000010000000050000006...


SELECT ST_PolygonFromText('POINT(1 2)') IS NULL as point_is_notpoly;

point_is_not_poly
----------
```

참고: ST_GeomFromText

8.4.40. ST_WKBToSQL

ST_WKBToSQL — WKB(Well-Known Binary) 표현식으로부터 지정된 ST_Geometry 값을 반환합니다. 이 함수는 SRID 를 입력받지 않는 ST_GeomFromWKB 함수와 동일합니다.

Synopsis

geometry **ST_WKBToSQL**(bytea *WKB*);

설명

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 5.1.34

참고: ST_GeomFromWKB

8.5. 지오메트리 접근자

GeometryType — 도형의 유형을 스트링으로 반환합니다. 예: 'LINESTRING', 'POLYGON', 'MULTIPOINT' 등등.

ST_Boundary — 해당 도형의 결합된 범위의 닫힘 여부를 반환합니다.

ST_CoordDim — ST_Geometry 값의 좌표 차원을 반환합니다.

ST_Dimension — 좌표 차원과 동등하거나 낮아야 하는, 해당 도형 객체의 내재된 차원을 반환합니다.

ST_EndPoint — LINESTRING 또는 CIRCULARLINESTRING 도형의 마지막 포인트를 POINT 로 반환합니다.

ST_Envelope — 주어진 도형의 이중 정밀도(double precision; float8) 경계 상자를 표현하는 도형을 반환합니다.

ST_BoundingDiagonal — 주어진 도형의 경계 상자의 대각선을 반환합니다.

ST_ExteriorRing — POLYGON 도형의 외곽 고리(exterior ring)를 표현하는 라인스트링을 반환합니다. 도형이 폴리곤이 아닌 경우 NULL 을 반환합니다. 멀티폴리곤은 입력받지 못 합니다.

ST_GeometryN — 입력 도형이 도형 집합, (멀티)포인트, (멀티)라인스트링, 멀티커브(multicurve) 또는 (멀티)폴리곤이나 다면체 표면일 경우 1-기반 N 번째 도형을 반환하며, 그 외의 경우 NULL 을 반환합니다.

ST_GeometryType — ST_Geometry 값의 도형 유형을 반환합니다.

ST_InteriorRingN — 폴리곤 도형의 N 번째 내곽 라인스트링 고리를 반환합니다. 도형이 폴리곤이 아니거나 주어진 N 이 범위(range)를 벗어난 경우 NULL 을 반환합니다.

ST_IsClosed — LINESTRING 의 시작점과 종단점이 일치하는 경우 TRUE 를 반환합니다. 다면체 표면이 닫혀 (부피를 가지고) 있는 경우 TRUE 를 반환합니다.

ST_IsCollection — 인수가 집합(MULTI*, GEOMETRYCOLLECTION, ...)인 경우 TRUE 를 반환합니다.

ST_IsEmpty — 해당 도형이 텅 빈 도형 집합, 폴리곤, 포인트 등인 경우 TRUE 를 반환합니다.

ST_IsRing — LINESTRING 이 닫혀 있는 단순 도형인 경우 TRUE 를 반환합니다.

ST_IsSimple — 해당 도형이 자체 교차하거나 자체 접촉하는 이례적인 도형 포인트를 가지고 있지 않을 경우 TRUE 를 반환합니다.

ST_IsValid — ST_Geometry 가 잘 형성돼 있는 경우 TRUE 를 반환합니다.

ST_IsValidReason — 도형이 유효한지 아닌지, 그리고 유효하지 않을 경우 그 이유를 설명하는 텍스트를 반환합니다.

ST_IsValidDetail — 도형이 유효한지 아닌지, 그리고 유효하지 않을 경우 그 이유 및 위치를 설명하는 valid_detail (유효 여부, 이유, 위치) 행을 반환합니다.

ST_M — 포인트의 M 좌표를 반환합니다. M 좌표가 없는 경우 NULL 을 반환합니다. 포인트만 입력받을 수 있습니다.

ST_NDims — 도형의 좌표 차원을 2 바이트 정수형(smallint)으로 반환합니다. 값은 2, 3, 또는 4 입니다.

ST_NPoints — 도형이 가지고 있는 포인트(꼭짓점)의 개수를 반환합니다.

ST_NRings — 도형이 폴리곤 또는 멀티폴리곤인 경우 고리의 개수를 반환합니다.

ST_NumGeometries — 도형이 도형 집합 또는 멀티 도형인 경우 도형의 개수를 반환합니다. 단일 도형인 경우 1 을 반환하고, 도형 집합도 멀티 도형도 단일 도형도 아닌 경우 NULL 을 반환합니다.

ST_NumInteriorRings — 폴리곤 도형의 내곽 고리의 개수를 반환합니다.

ST_NumInteriorRing — 도형 안에 있는 폴리곤의 내곽 고리의 개수를 반환합니다. ST_NumInteriorRings 함수와 동일합니다.

ST_NumPatches — 다면체 표면 상에 있는 면의 개수를 반환합니다. 다면체 도형이 아닌 경우 NULL 을 반환할 것입니다.

ST_NumPoints — ST_LineString 또는 ST_CircularString 값에 있는 포인트의 개수를 반환합니다.

ST_PatchN — 도형이 POLYHEDRALSURFACE, POLYHEDRALSURFACEM 인 경우 1-기반 N 번째 도형 (면)을 반환합니다. 그 외의 경우 NULL 을 반환합니다.

ST_PointN — 도형 안에 있는 첫 번째 라인스트링 또는 원형 라인스트링의 N 번째 포인트를 반환합니다. 음수 값은 라인스트링의 종단점으로부터 반대로 집계된 것입니다. 도형 안에 라인스트링이 없는 경우 NULL 을 반환합니다.

ST_Points — 도형의 모든 좌표들을 담고 있는 멀티포인트를 반환합니다.

ST_SRID — spatial_ref_sys 테이블에 정의되어 있는, ST_Geometry 에 대한 공간 참조 식별자를 반환합니다.

ST_StartPoint — LINESTRING 도형의 첫 번째 포인트를 POINT 로 반환합니다.

ST_Summary — 도형의 내용을 요약한 텍스트를 반환합니다.

ST_X — 포인트의 X 좌표를 반환합니다. X 좌표가 없는 경우 NULL 을 반환합니다. 포인트만 입력받을 수 있습니다.

ST_XMax — 2 차원 경계 상자(box2d) 또는 3 차원 경계 상자(box3d) 또는 도형의 최대 X 값을 반환합니다.

ST_XMin — 2 차원 경계 상자(box2d) 또는 3 차원 경계 상자(box3d) 또는 도형의 최소 X 값을 반환합니다.

ST_Y — 포인트의 Y 좌표를 반환합니다. Y 좌표가 없는 경우 NULL 을 반환합니다. 포인트만 입력받을 수 있습니다.

ST_YMax — 2 차원 경계 상자(box2d) 또는 3 차원 경계 상자(box3d) 또는 도형의 최대 Y 값을 반환합니다.

ST_YMin — 2 차원 경계 상자(box2d) 또는 3 차원 경계 상자(box3d) 또는 도형의 최소 Y 값을 반환합니다.

ST_Z — 포인트의 Z 좌표를 반환합니다. Z 좌표가 없는 경우 NULL 을 반환합니다. 포인트만 입력받을 수 있습니다.

ST_ZMax — 2 차원 경계 상자(box2d) 또는 3 차원 경계 상자(box3d) 또는 도형의 최대 Z 값을 반환합니다.

ST_Zmflag — 도형의 ZM(의미론적 차원) 플래그를 2 바이트 정수형(smallint)으로 반환합니다. 값이 0 일 때 2D, 1 일 때 3DM, 3 일 때 3DZ, 4 일 때 4D 를 뜻합니다.

ST_ZMin — 2 차원 경계 상자(box2d) 또는 3 차원 경계 상자(box3d) 또는 도형의 최대 Z 값을 반환합니다.

8.5.1. GeometryType

GeometryType — 도형의 유형을 스트링으로 반환합니다. 예: 'LINESTRING', 'POLYGON', 'MULTIPOINT' 등등

Synopsis

text **GeometryType**(geometry *geomA*);

설명: 도형의 유형을 스트링으로 반환합니다. 예: 'LINESTRING', 'POLYGON', 'MULTIPOINT' 등등.

OGC 사양 s2.1.1.1 - 해당 도형 인스턴스가 속해 있는, 인스턴스화할 수 있는 도형 하위 유형의 명칭을 스트링으로 반환합니다.



이 함수는 'POINTM' 형식의 스트링을 반환해서 도형에 단위가 적용되었는지 여부도 보여줍니다

개선 사항: 2.0.0 버전부터 다면체 표면, 삼각형 및 TIN 을 지원하기 시작했습니다.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

예시

```
SELECT      GeometryType(ST_GeomFromText('LINESTRING(77.29      29.07,77.42
29.26,77.27 29.31,77.29 29.07)'));
 geometrytype
--------------
 LINESTRING
SELECT ST_GeometryType(ST_GeomFromEWKT('POLYHEDRALSURFACE( ((0 0 0, 0 0 1,
0 1 1, 0 1 0, 0 0 0)),
             ((0 0 0, 0 1 0, 1 1 0, 1 0 0, 0 0 0)), ((0 0 0, 1 0 0, 1 0 1,
0 0 1, 0 0 0)),
             ((1 1 0, 1 1 1, 1 0 1, 1 0 0, 1 1 0)),
             ((0 1 0, 0 1 1, 1 1 1, 1 1 0, 0 1 0)), ((0 0 1, 1 0 1, 1 1 1,
0 1 1, 0 0 1)) )'));
                   -- 결과
                   POLYHEDRALSURFACE

SELECT GeometryType(geom) as result
  FROM
    (SELECT
      ST_GeomFromEWKT('TIN (((
             0 0 0,
             0 0 1,
             0 1 0,
             0 0 0
          )), ((
             0 0 0,
             0 1 0,
             1 1 0,
```

```
            0 0 0
          ))
          )')  AS geom
   ) AS g;
 result
--------
 TIN
```

참고: ST_GeometryType

8.5.2. ST_Boundary

ST_Boundary — 해당 도형의 결합된 범위의 닫힘 여부를 반환합니다.

Synopsis

geometry **ST_Boundary**(geometry *geomA*);

설명: 해당 도형의 결합된 범위의 닫힘(closure) 여부를 반환합니다. 결합 범위(combinatorial boundary)는 OGC 사양서의 3.12.3.2 단원이 설명하는대로 정의됩니다. 이 함수의 결과가 닫힘이기 때문에, 즉 위상적(位相的)으로 폐쇄됐기 때문에, OGC 사양서 3.12.2 단원에서 설명한대로 표현적인 도형 원형(primitive)을 이용해서 결과 범위를 표현할 수 있습니다.

GEOS 모듈로 실행



2.0.0 미만 버전에서 이 함수를 GEOMETRYCOLLECTION 과 함께 사용하면 예외가 발생했습니다. 2.0.0 이후 버전은 대신 (입력을 지원하지 않는다는 의미의) NULL 을 반환합니다.



이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 만족합니다. OGC SPEC s2.1.1.1



이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 5.1.14



이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

개선 사항: 2.1.0 버전부터 삼각형을 지원하기 시작했습니다.

예시



범위 포인트들이 겹치는 라인스트링

```
SELECT ST_Boundary(geom)
FROM (SELECT 'LINESTRING(100 150,50
60, 70 80, 160 170)'::geometry As
geom) As f;

-- ST_AsText 출력
MULTIPOINT(100 150,160 170)
```



범위 멀티라인스트링을 가진 폴리곤 구멍

```
SELECT ST_Boundary(geom)
FROM (SELECT
'POLYGON (( 10 130, 50 190, 110 190,
140 150, 150 80, 100 10, 20 40, 10
130 ),
      ( 70 40, 100 50, 120 80, 80
110, 50 90, 70 40 ))'::geometry As
geom) As f;

-- ST_AsText 출력
MULTILINESTRING((10 130,50 190,110
190,140 150,150 80,100 10,20 40,10
130),
      (70 40,100 50,120 80,80
110,50 90,70 40))
```

```
SELECT  ST_AsText(ST_Boundary(ST_GeomFromText('LINESTRING(1  1,0  0,  -1
1)')));
st_astext
-----------
MULTIPOINT(1 1,-1 1)

SELECT ST_AsText(ST_Boundary(ST_GeomFromText('POLYGON((1 1,0 0, -1 1, 1
1))')));
st_astext
----------
LINESTRING(1 1,0 0,-1 1,1 1)

-- 3D 폴리곤 사용
SELECT ST_AsEWKT(ST_Boundary(ST_GeomFromEWKT('POLYGON((1 1 1,0 0 1, -1 1
1, 1 1 1))')));

st_asewkt
-----------------------------------
LINESTRING(1 1 1,0 0 1,-1 1 1,1 1 1)

- 3D 멀티라인스트링 사용하기
SELECT ST_AsEWKT(ST_Boundary(ST_GeomFromEWKT('MULTILINESTRING((1 1 1,0 0
0.5, -1 1 1),(1 1 0.5,0 0 0.5, -1 1 0.5, 1 1 0.5) )')));

st_asewkt
----------
MULTIPOINT(-1 1 1,1 1 0.75)
```

참고: ST_AsText, ST_ExteriorRing, ST_MakePolygon

8.5.3. ST_CoordDim

ST_CoordDim — ST_Geometry 값의 좌표 차원을 반환합니다.

Synopsis

integer **ST_CoordDim**(geometry *geomA*);

설명: ST_Geometry 값의 좌표 차원을 반환합니다.

이 함수는 MM 을 준수하는, ST_NDims 와 동일한 함수합니다.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 만족합니다. OGC SPEC s2.1.1.1

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 5.1.3

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

예시

```
SELECT ST_CoordDim('CIRCULARSTRING(1 2 3, 1 3 4, 5 6 7, 8 9 10, 11 12 13)');
-- 결과 --
3

SELECT ST_CoordDim(ST_Point(1,2));
-- 결과 --
2
```

참고: ST_NDims

8.5.4. ST_Dimension

ST_Dimension — 좌표 차원과 동등하거나 낮아야 하는, 해당 도형 객체의 내재된 차원을 반환합니다.

Synopsis

integer **ST_Dimension**(geometry *g*);

설명: 좌표 차원과 동등하거나 낮아야 하는, 해당 도형 객체의 내재된 차원을 반환합니다. OGC 사양서 s2.1.1.1 단원을 보면 POINT 는 0, LINESTRING 은 1, POLYGON 은 2, 그리고 GEOMETRYCOLLECTION 의 경우 구성 요소 가운데 가장 높은 차원입니다. 알려지지 않은 (텅 빈) 도형인 경우 null 을 반환합니다.

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 5.1.2

개선 사항: 2.0.0 버전부터 다면체 표면(polyhedral surface) 및 TIN 을 지원합니다. 빈 도형이 주어져도 더 이상 예외가 발생하지 않습니다.



2.0.0 미만 버전에서는 빈 도형에 대해 예외를 발생시켰습니다..

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

예시

```
SELECT ST_Dimension('GEOMETRYCOLLECTION(LINESTRING(1 1,0 0),POINT(0 0))');
ST_Dimension
------------
1
```

참고: ST_NDims

8.5.5. ST_EndPoint

ST_EndPoint — LINESTRING 또는 CIRCULARLINESTRING 도형의 마지막 포인트를 POINT 로 반환합니다.

Synopsis

boolean **ST_EndPoint**(geometry *g*);

설명: LINESTRING 도형의 마지막 포인트를 POINT 로 반환합니다. 입력 파라미터가 LINESTRING 이 아닐 경우 NULL 을 반환합니다.

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 7.1.4

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.



변경 사항: 2.0.0 버전부터 단일 도형 멀티라인스트링을 지원하지 않습니다. PostGIS 예전 버전이라면 단일 라인 멀티라인스트링을 입력받는 경우 시작점을 반환했을 겁니다. 2.0.0 버전은 다른 모든 멀티라인스트링처럼 NULL 을 반환할 뿐입니다. 구식 습성은 문서화되지 않은 기능이지만, 사용자 데이터를 라인스트링으로 저장했다고 가정한 사용자의 경우 현재 2.0 버전에서 NULL 이 반환될 수도 있습니다.

예시

```
postgis=#   SELECT   ST_AsText(ST_EndPoint('LINESTRING(1   1,   2   2,   3
3)'::geometry));
 st_astext
------------
 POINT(3 3)
(1 row)

postgis=# SELECT ST_EndPoint('POINT(1 1)'::geometry) IS NULL AS is_null;
  is_null
----------
 t
(1 row)

--3d endpoint
SELECT ST_AsEWKT(ST_EndPoint('LINESTRING(1 1 2, 1 2 3, 0 0 5)'));
  st_asewkt
--------------
 POINT(0 0 5)
(1 row)
```

참고: ST_PointN, ST_StartPoint

8.5.6. ST_Envelope

ST_Envelope — 주어진 도형의 이중 정밀도(double precision; float8) 경계 상자를 표현하는 도형을 반환합니다.

Synopsis

geometry **ST_Envelope**(geometry *g1*);

설명: 주어진 도형에 대해 float8 형 최소치 경계 상자를 도형으로 반환합니다. 해당 폴리곤은 경계 상자의 꼭짓점 포인트들로 정의됩니다((MINX, MINY), (MINX, MAXY), (MAXX, MAXY), (MAXX, MINY), (MINX, MINY)). (PostGIS 는 ZMIN/ZMAX 좌표도 추가할 것입니다.)

차원이 낮은 (수직 라인, 포인트) 경우 POLYGON 보다 낮은 차원의, 예를 들어 POINT 또는 LINESTRING 도형을 반환할 것입니다.

1.5.0 버전부터 사용할 수 있으며, float4 형 대신 이중 정밀도 형으로 출력하도록 변경되었습니다.

✓ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 만족합니다. OGC SPEC s2.1.1.1

✓ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 5.1.15

예시

```
SELECT ST_AsText(ST_Envelope('POINT(1 3)'::geometry));
 st_astext
------------
 POINT(1 3)
(1 row)


SELECT ST_AsText(ST_Envelope('LINESTRING(0 0, 1 3)'::geometry));
```

```
               st_astext
-------------------------------
 POLYGON((0 0,0 3,1 3,1 0,0 0))
(1 row)


SELECT ST_AsText(ST_Envelope('POLYGON((0 0, 0 1, 1.0000001 1, 1.0000001 0,
0 0))'::geometry));
                                          st_astext
-------------------------------------------------------------------
 POLYGON((0 0,0 1,1.00000011920929 1,1.00000011920929 0,0 0))
(1 row)
SELECT   ST_AsText(ST_Envelope('POLYGON((0   0,   0   1,   1.0000000001   1,
1.0000000001 0, 0 0))'::geometry));
                                          st_astext
-------------------------------------------------------------------
 POLYGON((0 0,0 1,1.00000011920929 1,1.00000011920929 0,0 0))
(1 row)

SELECT    Box3D(geom),    Box2D(geom),    ST_AsText(ST_Envelope(geom))    As
envelopewkt
      FROM (SELECT 'POLYGON((0 0, 0 1000012333334.34545678, 1.0000001 1,
1.0000001 0, 0 0))'::geometry As geom) As foo;
```

참고: Box2D, Box3D

8.5.7. ST_BoundingDiagonal

ST_BoundingDiagonal — 주어진 도형의 경계 상자의 대각선을 반환합니다.

Synopsis

geometry **ST_BoundingDiagonal**(geometry *geom*, boolean *fits=false*);

설명: 주어진 도형의 경계 상자의 대각선을 라인스트링으로 반환합니다. 빈 도형을 입력받은 경우 빈 대각선을 반환하게 됩니다. 빈 도형이 아니라면 포인트 2 개 중 시작점이 각 차원의 최소값들을, 종단점이 각 차원의 최대값들을 가진 라인스트링을 반환합니다.

반환된 라인스트링 도형은 언제나 입력 도형의 SRID 및 차원수를 유지합니다.

fits 파라미터는 딱 맞는(best fit) 경계 상자가 필요한지 여부를 설정합니다. 거짓으로 설정한다면 조금 넉넉한 경계 상자도 용납할 수 있습니다(수많은 꼭짓점을 가진 도형의 경우 더 빨리 처리할 수 있습니다). 어떤 경우든 반환된 대각선의 경계 상자는 항상 입력 도형을 둘러쌉니다.



차원이 낮은 (입력 도형이 단일 꼭짓점) 경우 반환되는 라인스트링이 위상적으로 유효하지 않을 (내부가 없을) 것입니다. 그렇다고 해서 반환된 도형이 의미론적으로 유효하지 않다는 뜻은 아닙니다..

2.2.0 버전부터 사용할 수 있습니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 함수는 M 좌표를 지원합니다.

예시

```
-- 포인트를 둘러싼 버퍼에서 X 의 최소값을 얻습니다.
SELECT ST_X(ST_StartPoint(ST_BoundingDiagonal(
  ST_Buffer(ST_MakePoint(0,0),10)
)));
 st_x
------
  -10
```

참고: ST_StartPoint, ST_EndPoint, ST_X, ST_Y, ST_Z, ST_M, &&&

8.5.8. ST_ExteriorRing

ST_ExteriorRing — POLYGON 도형의 외곽 고리(exterior ring)를 표현하는 라인스트링을 반환합니다. 도형이 폴리곤이 아닌 경우 NULL 을 반환합니다. 멀티폴리곤은 입력받지 못 합니다.

Synopsis

geometry **ST_ExteriorRing**(geometry *a_polygon*);

설명: POLYGON 도형의 외곽 고리(exterior ring)를 표현하는 라인스트링을 반환합니다. 도형이 폴리곤이 아닌 경우 NULL 을 반환합니다.



오직 POLYGON 도형 유형만 입력받습니다.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 만족합니다. 2.1.5.1

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 8.2.3, 8.3.3

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
-- 폴리곤 테이블이 있을 경우
SELECT gid, ST_ExteriorRing(the_geom) AS ering
FROM sometable;

-- 멀티폴리곤 테이블이며 각 폴리곤의
-- 외곽 고리들로 이루어진 멀티라인스트링을 반환하고자 할 경우
SELECT gid, ST_Collect(ST_ExteriorRing(the_geom)) AS erings
      FROM (SELECT gid, (ST_Dump(the_geom)).geom As the_geom
                  FROM sometable) As foo
GROUP BY gid;

--3d Example
SELECT ST_AsEWKT(
```

```
        ST_ExteriorRing(
        ST_GeomFromEWKT('POLYGON((0 0 1, 1 1 1, 1 2 1, 1 1 1, 0 0 1))')
        )
);

st_asewkt
---------
LINESTRING(0 0 1,1 1 1,1 2 1,1 1 1,0 0 1)
```

참고: ST_InteriorRingN, ST_Boundary, ST_NumInteriorRings

8.5.9. ST_GeometryN

ST_GeometryN — 입력 도형이 도형 집합, (멀티)포인트, (멀티)라인스트링, 멀티커브(multicurve) 또는 (멀티)폴리곤이나 다면체 표면일 경우 1-기반 N 번째 도형을 반환하며, 그 외의 경우 NULL 을 반환합니다.

Synopsis

geometry **ST_GeometryN**(geometry *geomA*, integer *n*);

설명: 입력 도형이 도형 집합, (멀티)포인트, (멀티)라인스트링, 멀티커브(multicurve) 또는 (멀티)폴리곤이나 다면체 표면일 경우 1-기반 N 번째 도형을 반환하며, 그 외의 경우 NULL 을 반환합니다.



0.8.0 버전부터 인덱스는 OGC 사양을 따라 1-기반입니다. 예전 버전은 대신 0-기반 인덱스를 시행했습니다.



모든 도형들을 추출하고자 한다면 ST_Dump 함수가 더 효율적이며, 단일 도형에 대해서도 작동할 것입니다.

개선 사항: 2.0.0 버전부터 다면체 표면, 삼각형 및 TIN 을 지원하기 시작했습니다.

변경 사항: 2.0.0 미만 버전은 단일 도형에 대해 NULL 을 반환했을 겁니다. 2.0.0 버전부터 ST_GeometryN(..,1) 경우에 대한 도형을 반환하도록 변경됐습니다.



이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 만족합니다.



이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 9.1.5



이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.



이 방법은 원형 스트링 및 곡선을 지원합니다.



이 함수는 다면체 표면을 지원합니다.



이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

표준 예시

```
-- 3D 멀티포인트에서 포인트들의 하위 집합을 추출하기
SELECT n, ST_AsEWKT(ST_GeometryN(the_geom, n)) As geomewkt
FROM (
VALUES (ST_GeomFromEWKT('MULTIPOINT(1 2 7, 3 4 7, 5 6 7, 8 9 10)') ),
( ST_GeomFromEWKT('MULTICURVE(CIRCULARSTRING(2.5 2.5,4.5 2.5, 3.5 3.5), (10
11, 12 11))') )
      )As foo(the_geom)
      CROSS JOIN generate_series(1,100) n
WHERE n <= ST_NumGeometries(the_geom);

 n |            geomewkt
---+----------------------------------------
 1 | POINT(1 2 7)
 2 | POINT(3 4 7)
 3 | POINT(5 6 7)
 4 | POINT(8 9 10)
 1 | CIRCULARSTRING(2.5 2.5,4.5 2.5,3.5 3.5)
 2 | LINESTRING(10 11,12 11)


-- 모든 도형을 추출하기(ID를 할당하려 할 때 유용합니다)
SELECT gid, n, ST_GeometryN(the_geom, n)
FROM sometable CROSS JOIN generate_series(1,100) n
WHERE n <= ST_NumGeometries(the_geom);
```

다면체 표면, TIN 및 삼각형 예시

```
-- 다면체 표면 예시
-- 다면체 표면을 각 면으로 분해하기
SELECT ST_AsEWKT(ST_GeometryN(p_geom,3)) As geom_ewkt
  FROM (SELECT ST_GeomFromEWKT('POLYHEDRALSURFACE(
((0 0 0, 0 0 1, 0 1 1, 0 1 0, 0 0 0)),
((0 0 0, 0 1 0, 1 1 0, 1 0 0, 0 0 0)),
((0 0 0, 1 0 0, 1 0 1, 0 0 1, 0 0 0)),
((1 1 0, 1 1 1, 1 0 1, 1 0 0, 1 1 0)),
((0 1 0, 0 1 1, 1 1 1, 1 1 0, 0 1 0)),
((0 0 1, 1 0 1, 1 1 1, 0 1 1, 0 0 1))
)')  AS p_geom )  AS a;

             geom_ewkt
-----------------------------------------
 POLYGON((0 0 0,1 0 0,1 0 1,0 0 1,0 0 0))
-- TIN --
SELECT ST_AsEWKT(ST_GeometryN(geom,2)) as wkt
  FROM
    (SELECT
      ST_GeomFromEWKT('TIN (((
             0 0 0,
             0 0 1,
             0 1 0,
             0 0 0
         )), ((
             0 0 0,
             0 1 0,
             1 1 0,
             0 0 0
         ))
```

```
          )')  AS geom
   ) AS g;
-- 결과 --
              wkt
------------------------------------
 TRIANGLE((0 0 0,0 1 0,1 1 0,0 0 0))
```

참고: ST_Dump, ST_NumGeometries

8.5.10. ST_GeometryType

ST_GeometryType — ST_Geometry 값의 도형 유형을 반환합니다.

Synopsis

text **ST_GeometryType**(geometry *g1*);

설명: 도형의 유형을 'ST_Linestring', 'ST_Polygon', 'ST_MultiPolygon' 등과 같은 스트링으로 반환합니다. 이 함수는 도형의 단위가 설정돼 있는지 알려주지 않는 것은 물론, 반환된 결과물이 스트링이며 접두사 ST 가 달린다는 점에서도 GeometryType(geometry) 함수와는 다릅니다.

개선 사항: 2.0.0 버전부터 다면체 표면(polyhedral surface)을 지원합니다.

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 5.1.4

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 함수는 다면체 표면을 지원합니다.

예시

```
SELECT    ST_GeometryType(ST_GeomFromText('LINESTRING(77.29    29.07,77.42
29.26,77.27 29.31,77.29 29.07)'));
-- 결과
ST_LineString
SELECT ST_GeometryType(ST_GeomFromEWKT('POLYHEDRALSURFACE( ((0 0 0, 0 0 1,
0 1 1, 0 1 0, 0 0 0)),
            ((0 0 0, 0 1 0, 1 1 0, 1 0 0, 0 0 0)), ((0 0 0, 1 0 0, 1 0 1,
0 0 1, 0 0 0)),
            ((1 1 0, 1 1 1, 1 0 1, 1 0 0, 1 1 0)),
            ((0 1 0, 0 1 1, 1 1 1, 1 1 0, 0 1 0)), ((0 0 1, 1 0 1, 1 1 1,
0 1 1, 0 0 1)) )'));
                  -- 결과
                  ST_PolyhedralSurface
SELECT ST_GeometryType(ST_GeomFromEWKT('POLYHEDRALSURFACE( ((0 0 0, 0 0 1,
0 1 1, 0 1 0, 0 0 0)),
            ((0 0 0, 0 1 0, 1 1 0, 1 0 0, 0 0 0)), ((0 0 0, 1 0 0, 1 0 1,
0 0 1, 0 0 0)),
            ((1 1 0, 1 1 1, 1 0 1, 1 0 0, 1 1 0)),
            ((0 1 0, 0 1 1, 1 1 1, 1 1 0, 0 1 0)), ((0 0 1, 1 0 1, 1 1 1,
0 1 1, 0 0 1)) )'));
                  -- 결과
                  ST_PolyhedralSurface
```

```
SELECT ST_GeometryType(geom) as result
  FROM
    (SELECT
      ST_GeomFromEWKT('TIN (((
             0 0 0,
             0 0 1,
             0 1 0,
             0 0 0
          )), ((
             0 0 0,
             0 1 0,
             1 1 0,
             0 0 0
          ))
          )')  AS geom
    ) AS g;
 result
--------
 ST_Tin
```

참고: GeometryType

8.5.11. ST_InteriorRingN

ST_InteriorRingN — 폴리곤 도형의 N 번째 내곽 라인스트링 고리를 반환합니다. 도형이 폴리곤이 아니거나 주어진 N 이 범위(range)를 벗어난 경우 NULL 을 반환합니다.

Synopsis

geometry **ST_InteriorRingN**(geometry *a_polygon*, integer *n*);

설명: 폴리곤 도형의 N 번째 내곽 라인스트링 고리를 반환합니다. 도형이 폴리곤이 아니거나 주어진 N 이 범위(range)를 벗어난 경우 NULL 을 반환합니다. 인덱스는 1 에서 시작합니다.



이 함수는 멀티폴리곤을 입력받지 못 합니다. 멀티폴리곤의 경우 ST_Dump 함수와 결합해서 이용하십시오.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 만족합니다.

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 8.2.6, 8.3.5

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
SELECT ST_AsText(ST_InteriorRingN(the_geom, 1)) As the_geom
FROM (SELECT ST_BuildArea(
             ST_Collect(ST_Buffer(ST_Point(1,2), 20,3),
                    ST_Buffer(ST_Point(1, 2), 10,3))) As the_geom
             )  as foo
```

참고: ST_ExteriorRing ST_BuildArea, ST_Collect, ST_Dump, ST_NumInteriorRing, ST_NumInteriorRings

8.5.12. ST_IsClosed

ST_IsClosed — LINESTRING 의 시작점과 종단점이 일치하는 경우 TRUE 를 반환합니다. 다면체 표면이 닫혀 (부피를 가지고) 있는 경우 TRUE 를 반환합니다.

Synopsis

boolean **ST_IsClosed**(geometry *g*);

설명: LINESTRING 의 시작점과 종단점이 일치하는 경우 TRUE 를 반환합니다. 다면체 표면의 경우, 다면체 표면이 면적(열림)을 가지고 있는지 부피(닫힘)를 가지고 있는지 알려줍니다.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 만족합니다.

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 7.1.5, 9.3.3



SQL-MM 은 ST_IsClosed(NULL) 의 결과를 0 으로 정의하지만, PostGIS 는 NULL 을 반환합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

개선 사항: 2.0.0 버전부터 다면체 표면(polyhedral surface)을 지원합니다.

✔ 이 함수는 다면체 표면을 지원합니다.

라인스트링 및 포인트 예시

```
postgis=# SELECT ST_IsClosed('LINESTRING(0 0, 1 1)'::geometry);
 st_isclosed
-------------
 f
(1 row)

postgis=# SELECT ST_IsClosed('LINESTRING(0 0, 0 1, 1 1, 0 0)'::geometry);
 st_isclosed
-------------
 t
(1 row)

postgis=# SELECT ST_IsClosed('MULTILINESTRING((0 0, 0 1, 1 1, 0 0),(0 0, 1
1))'::geometry);
 st_isclosed
-------------
 f
(1 row)

postgis=# SELECT ST_IsClosed('POINT(0 0)'::geometry);
 st_isclosed
```

```
-------------
 t
(1 row)

postgis=# SELECT ST_IsClosed('MULTIPOINT((0 0), (1 1))'::geometry);
 st_isclosed
-------------
 t
(1 row)
```

다면체 표면 예시

```
-- 입방체 --
             SELECT  ST_IsClosed(ST_GeomFromEWKT('POLYHEDRALSURFACE(  ((0  0
0, 0 0 1, 0 1 1, 0 1 0, 0 0 0)),
             ((0 0 0, 0 1 0, 1 1 0, 1 0 0, 0 0 0)), ((0 0 0, 1 0 0, 1 0 1,
0 0 1, 0 0 0)),
             ((1 1 0, 1 1 1, 1 0 1, 1 0 0, 1 1 0)),
             ((0 1 0, 0 1 1, 1 1 1, 1 1 0, 0 1 0)), ((0 0 1, 1 0 1, 1 1 1,
0 1 1, 0 0 1)) )'));

 st_isclosed
-------------
 t


 -- 입방체이지만 한 면이 없는 경우 --
 SELECT  ST_IsClosed(ST_GeomFromEWKT('POLYHEDRALSURFACE(  ((0  0  0,  0  0  1,  0
1 1, 0 1 0, 0 0 0)),
             ((0 0 0, 0 1 0, 1 1 0, 1 0 0, 0 0 0)), ((0 0 0, 1 0 0, 1 0 1,
0 0 1, 0 0 0)),
             ((1 1 0, 1 1 1, 1 0 1, 1 0 0, 1 1 0)),
             ((0 1 0, 0 1 1, 1 1 1, 1 1 0, 0 1 0)) )'));

 st_isclosed
-------------
 f
```

참고: ST_IsRing

8.5.13. ST_IsCollection

ST_IsCollection — 인수가 집합(MULTI*, GEOMETRYCOLLECTION, ...)인 경우 TRUE 를 반환합니다.

Synopsis

boolean **ST_IsCollection**(geometry *g*);

설명: 인수의 도형 유형이 다음 가운데 하나일 경우 TRUE 를 반환합니다:

- GEOMETRYCOLLECTION
- MULTI{POINT,POLYGON,LINESTRING,CURVE,SURFACE}
- COMPOUNDCURVE



이 함수는 도형의 유형을 분석합니다. 즉 비어 있거나 단일 요소를 담고 있는 집합일 경우에도 TRUE 를 반환할 것이라는 뜻입니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

예시

```
postgis=# SELECT ST_IsCollection('LINESTRING(0 0, 1 1)'::geometry);
 st_iscollection
-----------------
 f
(1 row)

postgis=# SELECT ST_IsCollection('MULTIPOINT EMPTY'::geometry);
 st_iscollection
-----------------
 t
(1 row)

postgis=# SELECT ST_IsCollection('MULTIPOINT((0 0))'::geometry);
 st_iscollection
-----------------
 t
(1 row)

postgis=# SELECT ST_IsCollection('MULTIPOINT((0 0), (42 42))'::geometry);
 st_iscollection
-----------------
 t
(1 row)

postgis=#        SELECT        ST_IsCollection('GEOMETRYCOLLECTION(POINT(0
0))'::geometry);
 st_iscollection
-----------------
 t
(1 row)
```

참고: ST_NumGeometries

8.5.14. ST_IsEmpty

ST_IsEmpty — 해당 도형이 텅 빈 도형 집합, 폴리곤, 포인트 등인 경우 TRUE 를 반환합니다.

Synopsis

boolean **ST_IsEmpty**(geometry *geomA*);

설명: 해당 도형이 텅 빈 도형인 경우 TRUE 를 반환합니다. TRUE 인 경우, 해당 도형이 텅 빈 도형 집합, 폴리곤, 포인트 등을 표현한다는 뜻입니다.

Note! SQL-MM 은 ST_IsEmpty(NULL) 의 결과를 0 으로 정의하지만, PostGIS 는 NULL 을 반환합니다.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 만족합니다. s2.1.1.1

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 5.1.7

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

Note! 변경 사항: PostGIS 2.0.0 미만 버전에서는 ST_GeomFromText('GEOMETRYCOLLECTION(EMPTY)') 를 사용할 수 있었습니다. PostGIS 2.0.0 버전부터, SQL/MM 표준을 더 잘 준수하기 위해 이런 사용법은 금지됐습니다.

예시

```
SELECT ST_IsEmpty(ST_GeomFromText('GEOMETRYCOLLECTION EMPTY'));
 st_isempty
------------
 t
(1 row)

 SELECT ST_IsEmpty(ST_GeomFromText('POLYGON EMPTY'));
 st_isempty
------------
 t
(1 row)

SELECT ST_IsEmpty(ST_GeomFromText('POLYGON((1 2, 3 4, 5 6, 1 2))'));

 st_isempty
------------
 f
(1 row)

 SELECT  ST_IsEmpty(ST_GeomFromText('POLYGON((1  2,  3  4,  5  6,  1  2))'))  =
false;
 ?column?
----------
 t
(1 row)

 SELECT ST_IsEmpty(ST_GeomFromText('CIRCULARSTRING EMPTY'));
  st_isempty
------------
 t
(1 row)
```

8.5.15. ST_IsRing

ST_IsRing — LINESTRING 이 닫혀 있는 단순 도형인 경우 TRUE 를 반환합니다.

Synopsis

boolean **ST_IsRing**(geometry *g*);

설명: 해당 LINESTRING 이 ST_IsClosed (ST_StartPoint(g) ~= ST_Endpoint(g)) 인 동시에 ST_IsSimple 인 (자체 교차하지 않는) 경우 TRUE 를 반환합니다.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 만족합니다. 2.1.5.1

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 7.1.6



SQL-MM 은 ST_IsRing(NULL) 의 결과를 0 으로 정의하지만, PostGIS 는 NULL 을 반환합니다.

예시

```
SELECT ST_IsRing(the_geom), ST_IsClosed(the_geom), ST_IsSimple(the_geom)
FROM (SELECT 'LINESTRING(0 0, 0 1, 1 1, 1 0, 0 0)'::geometry AS the_geom)
AS foo;
 st_isring | st_isclosed | st_issimple
-----------+-------------+-------------
 t         | t           | t
(1 row)

SELECT ST_IsRing(the_geom), ST_IsClosed(the_geom), ST_IsSimple(the_geom)
FROM (SELECT 'LINESTRING(0 0, 0 1, 1 0, 1 1, 0 0)'::geometry AS the_geom)
AS foo;
 st_isring | st_isclosed | st_issimple
-----------+-------------+-------------
 f         | t           | f
(1 row)
```

참고: ST_IsClosed, ST_IsSimple, ST_StartPoint, ST_EndPoint

8.5.16. ST_IsSimple

ST_IsSimple — 해당 도형이 자체 교차하거나 자체 접촉하는 이례적인 도형 포인트를 가지고 있지 않을 경우 TRUE 를 반환합니다.

Synopsis

boolean **ST_IsSimple**(geometry *geomA*);

설명: 해당 도형이 자체 교차하거나 자체 접촉하는 이례적인 도형 포인트를 가지고 있지 않을 경우 TRUE 를 반환합니다. 도형 단순성 및 유효성에 대한 OGC 의 정의를 더 자세히 알고 싶다면, "OpenGIS 의 공간도형 유효성 확인(Ensuring OpenGIS compliancy of geometries)" 을 참조하십시오.



SQL-MM 은 ST_IsSimple(NULL) 의 결과를 0 으로 정의하지만, PostGIS 는 NULL 을 반환합니다.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 만족합니다. s2.1.1.1

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 5.1.8

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
SELECT ST_IsSimple(ST_GeomFromText('POLYGON((1 2, 3 4, 5 6, 1 2))'));
 st_issimple
-------------
 t
(1 row)

 SELECT  ST_IsSimple(ST_GeomFromText('LINESTRING(1  1,2  2,2  3.5,1  3,1  2,2
1)'));
 st_issimple
-------------
 f
(1 row)
```

참고: ST_IsValid

8.5.17. ST_IsValid

ST_IsValid — ST_Geometry 가 잘 형성돼 있는 경우 TRUE 를 반환합니다.

Synopsis

boolean **ST_IsValid**(geometry *g*);

boolean **ST_IsValid**(geometry *g*, integer *flags*);

설명: ST_Geometry 값이 잘 형성돼 있는지 검사합니다. 유효하지 않은 도형인 경우, PostgreSQL NOTICE 가 그 이유에 대한 상세 정보를 제공할 것입니다. 도형 단순성 및 유효성에 대한 OGC 의 정의를 더 자세히 알고 싶다면, "OpenGIS 의 공간도형 유효성 확인(Ensuring OpenGIS compliancy of geometries)" 을 참조하십시오.


SQL-MM 은 ST_IsValid(NULL) 의 결과를 0 으로 정의하지만, PostGIS 는 NULL 을 반환합니다.

2.0.0 버전부터 플래그를 사용할 수 있으며, 이때 GEOS 3.3.0 이상 버전이 필요합니다. 이 버전의 함수는 비유효성을 설명하는 알림을 출력하지 않습니다. ST_IsValidDetail 에 사용할 수 있는 flags 가 정리돼 있습니다.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 만족합니다.

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 5.1.9


OGC-SFS 또는 SQL-MM 어떤 사양도 ST_IsValid 함수용 플래그 인수를 포함하지 않습니다. 해당 플래그는 PostGIS 확장 프로그램입니다.

예시

```
SELECT ST_IsValid(ST_GeomFromText('LINESTRING(0 0, 1 1)')) As good_line,
      ST_IsValid(ST_GeomFromText('POLYGON((0 0, 1 1, 1 2, 1 1, 0 0))')) As
bad_poly
-- 결과
NOTICE:  Self-intersection at or near point 0 0
 good_line | bad_poly
-----------+----------
 t         | f
```

참고: ST_IsSimple, ST_IsValidReason, ST_IsValidDetail, ST_Summary

8.5.18. ST_IsValidReason

ST_IsValidReason — 도형이 유효한지 아닌지, 그리고 유효하지 않을 경우 그 이유를 설명하는 텍스트를 반환합니다.

Synopsis

text **ST_IsValidReason**(geometry *geomA*);

text **ST_IsValidReason**(geometry *geomA*, integer *flags*);

설명: 도형이 유효한지 아닌지, 그리고 유효하지 않을 경우 그 이유를 설명하는 텍스트를 반환합니다.

ST_IsValid 함수와 함께 사용해서 유효하지 않은 도형 및 그 이유를 자세히 설명하는 보고서를 생성하는 데 유용합니다.

ST_IsValidDetail 에 사용할 수 있는 flags 가 정리돼 있습니다.

1.4 버전부터 사용할 수 있습니다. GEOS 3.1.0 이상 버전이 필요합니다.

플래그를 사용할 수 있는 버전은 2.0 버전부터 사용할 수 있습니다. GEOS 3.3.0 이상 버전이 필요합니다.

예시

```
-- 성공적인 다섯 쌍 실험에서 첫 거부(reject) 3 개
SELECT gid, ST_IsValidReason(the_geom) as validity_info
FROM
(SELECT   ST_MakePolygon(ST_ExteriorRing(e.buff),   ST_Accum(f.line))   As
the_geom, gid
FROM (SELECT ST_Buffer(ST_MakePoint(x1*10,y1), z1) As buff, x1*10 + y1*100
+ z1*1000 As gid
      FROM generate_series(-4,6) x1
      CROSS JOIN generate_series(2,5) y1
      CROSS JOIN generate_series(1,8) z1
      WHERE x1 > y1*0.5 AND z1 < x1*y1) As e
      INNER                          JOIN                              (SELECT
ST_Translate(ST_ExteriorRing(ST_Buffer(ST_MakePoint(x1*10,y1),  z1)),y1*1,
z1*2) As line
      FROM generate_series(-3,6) x1
      CROSS JOIN generate_series(2,5) y1
      CROSS JOIN generate_series(1,10) z1
      WHERE x1 > y1*0.75 AND z1 < x1*y1) As f
ON (ST_Area(e.buff) > 78 AND ST_Contains(e.buff, f.line))
```

```
GROUP BY gid, e.buff) As quintuplet_experiment
WHERE ST_IsValid(the_geom) = false
ORDER BY gid
LIMIT 3;

 gid  |     validity_info
------+-------------------------
 5330 | Self-intersection [32 5]
 5340 | Self-intersection [42 5]
 5350 | Self-intersection [52 5]

 -- 단순한 예시
SELECT  ST_IsValidReason('LINESTRING(220227  150406,2220227  150407,222020
150410)');

 st_isvalidreason
------------------
 Valid Geometry
```

참고: ST_IsValid, ST_Summary

8.5.19. ST_IsValidDetail

ST_IsValidDetail — 도형이 유효한지 아닌지, 그리고 유효하지 않을 경우 그 이유 및 위치를 설명하는 valid_detail (유효 여부, 이유, 위치) 행을 반환합니다.

Synopsis

valid_detail **ST_IsValidDetail**(geometry *geom*);

valid_detail **ST_IsValidDetail**(geometry *geom*, integer *flags*);

설명: 도형이 유효한지 아닌지를 설명하는 불 값(유효 여부), 유효하지 않은 이유를 설명하는 varchar(이유), 그리고 유효하지 않은 위치를 지적하는 도형(위치)으로 구성된 valid_detail 행을 반환합니다.

ST_IsValid 및 ST_IsValidReason 함수의 조합을 대신하고 개선해서 유효하지 않은 도형에 대한 상세 내용을 확인하는 데 유용합니다.

'식별자' 인수는 비트 필드 구조체(bitfield)입니다. 다음 값을 가질 수 있습니다:

- 1: 비어있는 부분을 형성하는, 자체 교차하는 고리를 유효한 것으로 간주합니다. 이 플래그는 "ESRI 식별자"로도 알려져 있습니다. OGC 모델과는 모순된다는 점을 주의하십시오.

2.0.0 버전부터 사용할 수 있습니다. GEOS 3.3.0 이상 버전이 필요합니다.

예시

```
-- 성공적인 다섯 쌍 실험에서 첫 거부(reject) 3 개
SELECT              gid,              reason(ST_IsValidDetail(the_geom)),
ST_AsText(location(ST_IsValidDetail(the_geom))) as location
FROM
(SELECT   ST_MakePolygon(ST_ExteriorRing(e.buff),   ST_Accum(f.line))   As
the_geom, gid
```

```
FROM (SELECT ST_Buffer(ST_MakePoint(x1*10,y1), z1) As buff, x1*10 + y1*100
+ z1*1000 As gid
      FROM generate_series(-4,6) x1
      CROSS JOIN generate_series(2,5) y1
      CROSS JOIN generate_series(1,8) z1
      WHERE x1 > y1*0.5 AND z1 < x1*y1) As e
      INNER                          JOIN                          (SELECT
ST_Translate(ST_ExteriorRing(ST_Buffer(ST_MakePoint(x1*10,y1),  z1)),y1*1,
z1*2) As line
      FROM generate_series(-3,6) x1
      CROSS JOIN generate_series(2,5) y1
      CROSS JOIN generate_series(1,10) z1
      WHERE x1 > y1*0.75 AND z1 < x1*y1) As f
ON (ST_Area(e.buff) > 78 AND ST_Contains(e.buff, f.line))
GROUP BY gid, e.buff) As quintuplet_experiment
WHERE ST_IsValid(the_geom) = false
ORDER BY gid
LIMIT 3;

 gid  |     reason      | location
------+-----------------+------------
 5330 | Self-intersection | POINT(32 5)
 5340 | Self-intersection | POINT(42 5)
 5350 | Self-intersection | POINT(52 5)

 -- 단순한 예시
SELECT    *    FROM    ST_IsValidDetail('LINESTRING(220227    150406,2220227
150407,222020 150410)');

 valid | reason | location
-------+--------+----------
 t     |        |
```

참고: ST_IsValid, ST_IsValidReason

8.5.20. ST_M

ST_M — 포인트의 M 좌표를 반환합니다. M 좌표가 없는 경우 NULL 을 반환합니다. 포인트만 입력받을 수 있습니다.

Synopsis

float **ST_M**(geometry *a_point*);

설명: 포인트의 M 좌표를 반환합니다. M 좌표가 없는 경우 NULL 을 반환합니다. 포인트만 입력받을 수 있습니다.

이 함수는 (아직) OGC 사양에 들어가지 않지만, 포인트 좌표 추출자(extractor) 함수 목록을 완성하기 위해 이 문서에 작성됐습니다.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 만족합니다.

✔ 이 방법은 SQL/MM 사양을 도입합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
SELECT ST_M(ST_GeomFromEWKT('POINT(1 2 3 4)'));
 st_m
------
       4
(1 row)
```

참고: ST_GeomFromEWKT, ST_X, ST_Y, ST_Z

8.5.21. ST_NDims

ST_NDims — 도형의 좌표 차원을 2 바이트 정수형(smallint)으로 반환합니다. 값은 2, 3, 또는 4 입니다.

Synopsis

integer **ST_NDims**(geometry *g1*);

설명: 도형의 좌표 차원을 반환합니다. PostGIS 는 2 - 2 차원 (x,y), 3 - 3 차원 (x,y,z), 3 - 단위를 가진 2 차원 (x,y,m), 그리고 4 - 단위를 가진 3 차원 공간 (x,y,z,m)을 지원합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
SELECT ST_NDims(ST_GeomFromText('POINT(1 1)')) As d2point,
       ST_NDims(ST_GeomFromEWKT('POINT(1 1 2)')) As d3point,
       ST_NDims(ST_GeomFromEWKT('POINTM(1 1 0.5)')) As d2pointm;

        d2point | d3point | d2pointm
---------+---------+----------
          2 |       3 |        3
```

참고: ST_CoordDim, ST_Dimension, ST_GeomFromEWKT

8.5.22. ST_NPoints

ST_NPoints — 도형이 가지고 있는 포인트(꼭짓점)의 개수를 반환합니다.

Synopsis

integer **ST_NPoints**(geometry *g1*);

설명: 도형이 가지고 있는 포인트의 개수를 반환합니다. 모든 도형을 입력받을 수 있습니다.

개선 사항: 2.0.0 버전부터 다면체 표면(polyhedral surface)을 지원합니다



1.3.4 미만 버전에서 이 함수에 만곡 도형(curve)을 담고 있는 도형을 입력하면 충돌이 일어났습니다. 1.3.4 버전부터 이 버그가 해결됐습니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다

✔ 이 함수는 다면체 표면을 지원합니다.

예시

```
SELECT      ST_NPoints(ST_GeomFromText('LINESTRING(77.29      29.07,77.42
29.26,77.27 29.31,77.29 29.07)'));
-- 결과
4

-- 3 차원 공간에 있는 폴리곤
SELECT  ST_NPoints(ST_GeomFromEWKT('LINESTRING(77.29  29.07  1,77.42  29.26
0,77.27 29.31 -1,77.29 29.07 3)'))
-- 결과
4
```

참고: ST_NumPoints

8.5.23. ST_NRings

ST_NRings — 도형이 폴리곤 또는 멀티폴리곤인 경우 고리의 개수를 반환합니다.

Synopsis

integer **ST_NRings**(geometry *geomA*);

설명: 도형이 폴리곤 또는 멀티폴리곤인 경우 고리의 개수를 반환합니다. NumInteriorRings 함수와는 달리, 외곽 고리도 개수에 포함시킵니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다

예시

```
SELECT  ST_NRings(the_geom)  As  Nrings,  ST_NumInteriorRings(the_geom)  As
ninterrings
                              FROM  (SELECT  ST_GeomFromText('POLYGON((1
2, 3 4, 5 6, 1 2))') As the_geom) As foo;
       nrings | ninterrings
--------+-------------
        1 |         0
(1 row)
```

참고: ST_NumInteriorRings

8.5.24. ST_NumGeometries

ST_NumGeometries — 도형이 도형 집합 또는 멀티 도형인 경우 도형의 개수를 반환합니다. 단일 도형인 경우 1 을 반환하고, 도형 집합도 멀티 도형도 단일 도형도 아닌 경우 NULL 을 반환합니다.

Synopsis

integer **ST_NumGeometries**(geometry *geom*);

설명: 도형의 개수를 반환합니다. 도형 집합 또는 멀티 도형인 경우 도형의 개수를, 단일 도형인 경우 1 을 반환하고, 도형 집합도 멀티 도형도 단일 도형도 아닌 경우 NULL 을 반환합니다.

개선 사항: 2.0.0 버전부터 다면체 표면, 삼각형 및 TIN 을 지원하기 시작했습니다.

변경 사항: 2.0.0 미만 버전에서 도형이 집합이나 멀티 유형이 아닐 경우 NULL 을 반환했습니다. 2.0.0 버전부터 폴리곤, 라인스트링, 포인트 같은 단일 도형에 대해 1 을 반환합니다.

✓ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 9.1.4

✓ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✓ 이 함수는 다면체 표면을 지원합니다.

✓ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

예시

```
-- 이런 경우 예전 버전은 NULL 을 반환했을 겁니다.
-- 2.0.0 버전부터 1 을 반환합니다.
SELECT    ST_NumGeometries(ST_GeomFromText('LINESTRING(77.29    29.07,77.42
29.26,77.27 29.31,77.29 29.07)'));
-- 결과
1

-- 도형 집합 예시
-- 집합 안에 있는 멀티 유형은 도형 1 개로 칩니다.
SELECT  ST_NumGeometries(ST_GeomFromEWKT('GEOMETRYCOLLECTION(MULTIPOINT(-2
3 , -2 2),
LINESTRING(5 5 ,10 10),
POLYGON((-7 4.2,-7.1 5,-7.1 4.3,-7 4.2)))'));
-- 결과
3
```

참고: ST_GeometryN, ST_Multi

8.5.25. ST_NumInteriorRings

ST_NumInteriorRings — 폴리곤 도형의 내곽 고리의 개수를 반환합니다.

Synopsis

integer **ST_NumInteriorRings**(geometry *a_polygon*);

설명: 폴리곤 도형의 내곽 고리의 개수를 반환합니다. 도형이 폴리곤이 아닐 경우 NULL 을 반환합니다.

✓ 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 8.2.5

변경 사항: 2.0.0 미만 버전에서는 멀티폴리곤을 입력하면 첫 번째 폴리곤의 내곽 고리의 개수를 반환받을 수 있었습니다.

예시

```
-- 정규 폴리곤인 경우
SELECT gid, field1, field2, ST_NumInteriorRings(the_geom) AS numholes
FROM sometable;

-- 멀티폴리곤인 경우
-- 그리고 멀티폴리곤 내부에 있는 모든 내곽 고리의 개수를 알고자 하는 경우
SELECT gid, field1, field2, SUM(ST_NumInteriorRings(the_geom)) AS numholes
FROM (SELECT gid, field1, field2, (ST_Dump(the_geom)).geom As the_geom
      FROM sometable) As foo
GROUP BY gid, field1,field2;
```

참고: ST_NumInteriorRing

8.5.26. ST_NumInteriorRing

ST_NumInteriorRing — 도형 안에 있는 폴리곤의 내곽 고리의 개수를 반환합니다. ST_NumInteriorRings 함수와 동일합니다.

Synopsis

integer **ST_NumInteriorRing**(geometry *a_polygon*);

참고: ST_NumInteriorRings

8.5.27. ST_NumPatches

ST_NumPatches — 다면체 표면 상에 있는 면의 개수를 반환합니다. 다면체 도형이 아닌 경우 NULL 을 반환할 것입니다.

Synopsis

integer **ST_NumPatches**(geometry *g1*);

설명: 다면체 표면 상에 있는 면의 개수를 반환합니다. 다면체 도형이 아닌 경우 NULL 을 반환할 것입니다. 이 함수는 ST_NumGeometries 와 동일하지만 MM 명명법을 지원합니다. MM 규약을 신경 쓰지 않는다면 ST_NumGeometries 가 더 빠릅니다.

2.0.0 버전부터 사용할 수 있습니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 만족합니다.

✔ 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: ?

✔ 이 함수는 다면체 표면을 지원합니다.

예시

```
SELECT ST_NumPatches(ST_GeomFromEWKT('POLYHEDRALSURFACE( ((0 0 0, 0 0 1, 0
1 1, 0 1 0, 0 0 0)),
            ((0 0 0, 0 1 0, 1 1 0, 1 0 0, 0 0 0)), ((0 0 0, 1 0 0, 1 0 1,
0 0 1, 0 0 0)),
            ((1 1 0, 1 1 1, 1 0 1, 1 0 0, 1 1 0)),
            ((0 1 0, 0 1 1, 1 1 1, 1 1 0, 0 1 0)), ((0 0 1, 1 0 1, 1 1 1,
0 1 1, 0 0 1)) )'));
            -- 결과
            6
```

참고: ST_GeomFromEWKT, ST_NumGeometries

8.5.28. ST_NumPoints

ST_NumPoints — ST_LineString 또는 ST_CircularString 값에 있는 포인트의 개수를 반환합니다.

Synopsis

integer **ST_NumPoints**(geometry *g1*);

설명: ST_LineString 또는 ST_CircularString 값에 있는 포인트의 개수를 반환합니다. 1.4 미만 버전에서는 사양서대로 라인스트링만 입력받았습니다. 1.4 버전부터 이 함수는 단순히 라인스트링만이 아닌, 도형의 꼭짓점의 개수를 반환하는 ST_NPoints 함수와 비슷해졌습니다. 쿼리 목적이 다양하고 많은 도형 유형을 사용할 수 있는 ST_NPoints 함수를 대신 사용하는 편이 좋습니다.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 만족합니다.

✔ 이 방법은 SQL/MM 사양을 만족합니다 SQL-MM 3: 7.2.4

예시

```
SELECT      ST_NumPoints(ST_GeomFromText('LINESTRING(77.29      29.07,77.42
29.26,77.27 29.31,77.29 29.07)'));
            -- 결과
            4
```

참고: ST_NPoints

8.5.29. ST_PatchN

ST_PatchN — 도형이 POLYHEDRALSURFACE, POLYHEDRALSURFACEM 인 경우 1-기반 N 번째 도형 (면)을 반환합니다. 그 외의 경우 NULL 을 반환합니다.

Synopsis

geometry **ST_PatchN**(geometry *geomA*, integer *n*);

설명: 도형이 POLYHEDRALSURFACE, POLYHEDRALSURFACEM 인 경우 1-기반 N 번째 도형 (면)을 반환합니다. 그 외의 경우 NULL 을 반환합니다. 이 함수는 다면체 표면에 대해 ST_GeometryN 과 동일한 답을 반환합니다. ST_GeometryN 을 이용하는 편이 더 빠릅니다.



인덱스는 1-기반입니다..



모든 도형들을 추출하고자 한다면 ST_Dump 함수가 더 효율적입니다.

2.0.0 버전부터 사용할 수 있습니다.

✔ 이 방법은 SQL/MM 사양을 만족합니다 SQL-MM 3:?

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 함수는 다면체 표면을 지원합니다.

예시

```
-- 다면체 표면의 두 번째 면을 추출합니다.
SELECT ST_AsEWKT(ST_PatchN(geom, 2)) As geomewkt
FROM (
VALUES (ST_GeomFromEWKT('POLYHEDRALSURFACE( ((0 0 0, 0 0 1, 0 1 1, 0 1 0,
0 0 0)),
       ((0 0 0, 0 1 0, 1 1 0, 1 0 0, 0 0 0)), ((0 0 0, 1 0 0, 1 0 1, 0 0 1,
0 0 0)),
       ((1 1 0, 1 1 1, 1 0 1, 1 0 0, 1 1 0)),
       ((0 1 0, 0 1 1, 1 1 1, 1 1 0, 0 1 0)), ((0 0 1, 1 0 1, 1 1 1, 0 1 1,
0 0 1)) )')) ) As foo(geom);

            geomewkt
---+----------------------------------------
 POLYGON((0 0 0,0 1 0,1 1 0,1 0 0,0 0 0))
```

참고: ST_AsEWKT, ST_GeomFromEWKT, ST_Dump, ST_GeometryN, ST_NumGeometries

8.5.30. ST_PointN

ST_PointN — 도형 안에 있는 첫 번째 라인스트링 또는 원형 라인스트링의 N 번째 포인트를 반환합니다. 음수 값은 라인스트링의 종단점으로부터 반대로 집계된 것입니다. 도형 안에 라인스트링이 없는 경우 NULL 을 반환합니다.

Synopsis

geometry **ST_PointN**(geometry *a_linestring*, integer *n*);

설명: 도형 안에 있는 단일 라인스트링 또는 원형 라인스트링의 N 번째 포인트를 반환합니다. 음수 값은 라인스트링의 종단점으로부터 반대로 집계되므로, -1 이 마지막 포인트입니다. 도형 안에 라인스트링이 없는 경우 NULL 을 반환합니다.



0.8.0 버전부터 인덱스는 OGC 사양을 따라 1-기반입니다. OGC 는 반대 방향 인덱스(음수 인덱스)를 지원하지 않습니다. 예전 버전은 대신 0-기반 인덱스를 도입했습니다.



멀티라인스트링 안에 있는 각 라인스트링의 N 번째 포인트를 얻고자 할 경우, 이 함수를 ST_Dump 함수와 연결해서 이용하십시오.



이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 만족합니다.



이 방법은 SQL/MM 사양을 만족합니다 SQL- MM 3: 7.2.5, 7.3.5



이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.



이 방법은 원형 스트링 및 곡선을 지원합니다.이 함수는 다면체 표면을 지원합니다.



변경 사항: 2.0.0 버전부터 단일 도형 멀티라인스트링을 지원하지 않습니다. PostGIS 예전 버전이라면 단일 라인 멀티라인스트링을 입력받는 경우 시작점을 반환했을 겁니다. 2.0.0 버전은 다른 모든 멀티라인스트링처럼 NULL 을 반환할 뿐입니다.
변경 사항: 2.3.0 버전부터 음수 인덱스(-1 이 마지막 포인트)를 이용할 수 있습니다.

예시

```
-- 라인스트링으로부터 모든 포인트를 추출합니다.
SELECT ST_AsText(
   ST_PointN(
         column1,
         generate_series(1, ST_NPoints(column1))
   ))
FROM ( VALUES ('LINESTRING(0 0, 1 1, 2 2)'::geometry) ) AS foo;

 st_astext
------------
 POINT(0 0)
```

```
 POINT(1 1)
 POINT(2 2)
(3 rows)

-- 원형 라인스트링 예시
SELECT  ST_AsText(ST_PointN(ST_GeomFromText('CIRCULARSTRING(1  2,  3  2,  1
2)'),2));

st_astext
----------
POINT(3 2)

SELECT st_astext(f)
FROM ST_GeometryFromtext('LINESTRING(0 0 0, 1 1 1, 2 2 2)') as g
      ,ST_PointN(g, -2) AS f -- 1 based index

st_astext
----------
"POINT Z (1 1 1)"
```

참고: ST_NPoints

8.5.31. ST_Points

ST_Points — 도형의 모든 좌표들을 담고 있는 멀티포인트를 반환합니다.

Synopsis

geometry **ST_Points**(geometry *geom*);

설명: 도형의 모든 좌표들을 담고 있는 멀티포인트를 반환합니다. 입력 도형에서 중첩되어 있는 포인트들 -- 고리 도형의 시작점과 종단점 포함 -- 을 제거하지 않습니다(이런 습성을 원하지 않을 경우, ST_RemoveRepeatedPoints 함수를 이용해서 중첩된 포인트를 제거할 수도 있습니다).

M 및 Z 좌표가 있을 경우 그대로 유지될 것입니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

2.3.0 버전부터 사용할 수 있습니다.

예시

```
SELECT ST_AsText(ST_Points('POLYGON Z ((30 10 4,10 30 5,40 40 6, 30 10))'));

-- 결과
MULTIPOINT Z (30 10 4,10 30 5,40 40 6, 30 10 4)
```

참고: ST_RemoveRepeatedPoints

8.5.32. ST_SRID

ST_SRID — spatial_ref_sys 테이블에 정의되어 있는, ST_Geometry 에 대한 공간 참조 식별자를 반환합니다.

Synopsis

integer **ST_SRID**(geometry *g1*);

설명: spatial_ref_sys 테이블에 정의되어 있는, ST_Geometry 에 대한 공간 참조 식별자를 반환합니다. Section 4.3.1, "SPATIAL_REF_SYS 테이블과 공간 참조 시스템"



spatial_ref_sys 테이블은 PostGIS 에 알려진 모든 공간 참조 시스템을 카탈로그화하는 테이블로, 어떤 공간 참조 시스템에서 다른 공간 참조 시스템으로 변환하는 작업에 쓰입니다. 따라서 사용자 도형을 변환할 계획이 있다면 올바른 공간 참조 시스템 식별자를 가지고 있는지 확인하는 일이 중요합니다.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 만족합니다. s2.1.1.1

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 5.1.5

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

예시

```
SELECT ST_SRID(ST_GeomFromText('POINT(-71.1043 42.315)',4326));
-- 결과
4326
```

참고: Section 4.3.1, "SPATIAL_REF_SYS 테이블과 공간 참조 시스템", ST_GeomFromText, ST_SetSRID, ST_Transform

8.5.33. ST_StartPoint

ST_StartPoint — LINESTRING 도형의 첫 번째 포인트를 POINT 로 반환합니다.

Synopsis

geometry **ST_StartPoint**(geometry *geomA*);

설명: LINESTRING 또는 CIRCULARLINESTRING 도형의 첫 번째 포인트를 POINT 로 반환합니다. 입력 파라미터가 LINESTRING 또는 CIRCULARLINESTRING 이 아닐 경우 NULL 을 반환합니다.

✔ 이 방법은 SQL/MM 사양을 만족합니다.SQL-MM 3: 7.1.3

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.



변경 사항: 2.0.0 버전부터 단일 도형 멀티라인스트링을 지원하지 않습니다. PostGIS 예전 버전이라면 단일 라인 멀티라인스트링을 입력받는 경우 시작점을 반환했을 겁니다. 2.0.0 버전은 다른 모든 멀티라인스트링처럼 NULL 을 반환할 뿐입니다. 구식 습성은 문서화되지 않은 기능이지만, 사용자 데이터를 라인스트링으로 저장했다고 가정한 사용자의 경우 현재 2.0 버전에서 NULL 이 반환될 수도 있습니다..

예시

```
SELECT ST_AsText(ST_StartPoint('LINESTRING(0 1, 0 2)'::geometry));
 st_astext
------------
 POINT(0 1)
(1 row)

SELECT ST_StartPoint('POINT(0 1)'::geometry) IS NULL AS is_null;
  is_null
----------
 t
(1 row)

-- 3D 라인
SELECT ST_AsEWKT(ST_StartPoint('LINESTRING(0 1 1, 0 2 2)'::geometry));
 st_asewkt
------------
 POINT(0 1 1)
(1 row)

-- 원형 라인스트링 --
SELECT ST_AsText(ST_StartPoint('CIRCULARSTRING(5 2,-3 1.999999, -2 1, -4
2, 5 2)'::geometry));
 st_astext
------------
 POINT(5 2)
```

참고: ST_EndPoint, ST_PointN

8.5.34. ST_Summary

ST_Summary — 도형의 내용을 요약한 텍스트를 반환합니다.

Synopsis

text **ST_Summary**(geometry *g*);

text **ST_Summary**(geography *g*);

설명: 도형의 내용을 요약한 텍스트를 반환합니다.

도형 유형 뒤에 대괄호로 표기된 플래그는 다음과 같은 뜻입니다:

- M: M 좌표가 존재합니다.

- Z: Z 좌표가 존재합니다.
- B: 캐쉬된 경계 상자를 가지고 있습니다.
- G: 측지형(지리형)입니다.
- S: 공간 참조 시스템을 가지고 있습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

1.2.2 버전부터 사용할 수 있습니다.

개선 사항: 2.0.0 버전부터 지리형을 지원하기 시작했습니다.

개선 사항: 2.1.0 버전. 알려진 공간 참조 시스템을 가진 경우를 나타내는 S 플래그가 추가됐습니다.

개선 사항: 2.2.0 버전부터 TIN 및 만곡 도형(curve)을 지원하기 시작했습니다.

예시

```
=# SELECT ST_Summary(ST_GeomFromText('LINESTRING(0 0, 1 1)')) as geom,
      ST_Summary(ST_GeogFromText('POLYGON((0 0, 1 1, 1 2, 1 1, 0 0))'))
geog;
          geom           |        geog
-----------------------------+-------------------------
 LineString[B] with 2 points | Polygon[BGS] with 1 rings
                       | ring 0 has 5 points
                       :
(1 row)


=#  SELECT  ST_Summary(ST_GeogFromText('LINESTRING(0  0  1,  1  1  1)'))  As
geog_line,
      ST_Summary(ST_GeomFromText('SRID=4326;POLYGON((0 0 1, 1 1 2, 1 2 3,
1 1 1, 0 0 1))')) As geom_poly;
;
         geog_line           |      geom_poly
-------------------------------- +-------------------------
 LineString[ZBGS] with 2 points | Polygon[ZBS] with 1 rings
                         :   ring 0 has 5 points
                         :
(1 row)
```

참고: PostGIS_DropBBox, PostGIS_AddBBox, ST_Force3DM, ST_Force3DZ, ST_Force2D, geography

ST_IsValid, ST_IsValid, ST_IsValidReason, ST_IsValidDetail

8.5.35. ST_X

ST_X — 포인트의 X 좌표를 반환합니다. X 좌표가 없는 경우 NULL 을 반환합니다. 포인트만 입력받을 수 있습니다.

Synopsis

float **ST_X**(geometry *a_point*);

설명: 포인트의 X 좌표를 반환합니다. X 좌표가 없는 경우 NULL 을 반환합니다. 포인트만 입력받을 수 있습니다.



어떤 도형의 최대 최소 x 값을 얻고자 하는 경우 ST_XMin 및 ST_XMax 함수를 참고하십시오.

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 6.1.3.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
SELECT ST_X(ST_GeomFromEWKT('POINT(1 2 3 4)'));
 st_x
------
       1
(1 row)

SELECT ST_Y(ST_Centroid(ST_GeomFromEWKT('LINESTRING(1 2 3 4, 1 1 1 1)')));
 st_y
------
  1.5
(1 row)
```

참고: ST_Centroid, ST_GeomFromEWKT, ST_M, ST_XMax, ST_XMin, ST_Y, ST_Z

8.5.36. ST_XMax

ST_XMax — 2 차원 경계 상자(box2d) 또는 3 차원 경계 상자(box3d) 또는 도형의 최대 X 값을 반환합니다.

Synopsis

float **ST_XMax**(box3d *aGeomorBox2DorBox3D*);

설명: 2 차원 경계 상자(box2d) 또는 3 차원 경계 상자(box3d) 또는 도형의 최대 X 값을 반환합니다.



이 함수가 box3d 를 위해서만 정의됐긴 해도, 도형 및 box2d 에 대해 정의된 자동 형변환 습성 때문에 도형 및 box2d 를 위해서도 작동합니다. 하지만 도형 또는 box2d 의 텍스트 표현식을 입력할 수는 없습니다. 텍스트 표현식은 자동 형변환의 대상이 아니기 때문입니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

예시

```
SELECT ST_XMax('BOX3D(1 2 3, 4 5 6)');
st_xmax
-------
4

SELECT ST_XMax(ST_GeomFromText('LINESTRING(1 3 4, 5 6 7)'));
st_xmax
-------
5

SELECT ST_XMax(CAST('BOX(-3 2, 3 4)' As box2d));
st_xmax
-------
3
-- 스트링 표현식을 box3d 로 자동 형변환하려 하기 때문에 오류가 발생하는 것을 지켜보십시오.
SELECT ST_XMax('LINESTRING(1 3, 5 6)');

--ERROR:  BOX3D parser - doesn't start with BOX3D(

SELECT   ST_XMax(ST_GeomFromEWKT('CIRCULARSTRING(220268   150415   1,220227
150505 2,220227 150406 3)'));
st_xmax
--------
220288.248780547
```

참고: ST_XMin, ST_YMax, ST_YMin, ST_ZMax, ST_ZMin

8.5.37. ST_XMin

ST_XMin — 2 차원 경계 상자(box2d) 또는 3 차원 경계 상자(box3d) 또는 도형의 최소 X 값을 반환합니다.

Synopsis

float **ST_XMin**(box3d *aGeomorBox2DorBox3D*);

설명: 2 차원 경계 상자(box2d) 또는 3 차원 경계 상자(box3d) 또는 도형의 최소 X 값을 반환합니다.



이 함수가 box3d 를 위해서만 정의됐긴 해도, 도형 및 box2d 에 대해 정의된 자동 형변환 습성 때문에 도형 및 box2d 를 위해서도 작동합니다. 하지만 도형 또는 box2d 의 텍스트 표현식을 입력할 수는 없습니다. 텍스트 표현식은 자동 형변환의 대상이 아니기 때문입니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

예시

```
SELECT ST_XMin('BOX3D(1 2 3, 4 5 6)');
st_xmin
-------
1

SELECT ST_XMin(ST_GeomFromText('LINESTRING(1 3 4, 5 6 7)'));
st_xmin
-------
1

SELECT ST_XMin(CAST('BOX(-3 2, 3 4)' As box2d));
st_xmin
-------
-3
-- 스트링 표현식을 box3d 로 자동 형변환하려 하기 때문에 오류가 발생하는 것을 지켜보십시오.
SELECT ST_XMin('LINESTRING(1 3, 5 6)');

--ERROR:  BOX3D parser - doesn't start with BOX3D(

SELECT   ST_XMin(ST_GeomFromEWKT('CIRCULARSTRING(220268   150415   1,220227
150505 2,220227 150406 3)'));
st_xmin
--------
220186.995121892
```

참고: ST_XMax, ST_YMax, ST_YMin, ST_ZMax, ST_ZMin

8.5.38. ST_Y

ST_Y — 포인트의 Y 좌표를 반환합니다. Y 좌표가 없는 경우 NULL 을 반환합니다. 포인트만 입력받을 수 있습니다.

Synopsis

float **ST_Y**(geometry *a_point*);

설명: 포인트의 Y 좌표를 반환합니다. Y 좌표가 없는 경우 NULL 을 반환합니다. 포인트만 입력받을 수 있습니다.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다.

✔ 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 6.1.4

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
SELECT ST_Y(ST_GeomFromEWKT('POINT(1 2 3 4)'));
 st_y
------
       2
(1 row)
```

```
SELECT ST_Y(ST_Centroid(ST_GeomFromEWKT('LINESTRING(1 2 3 4, 1 1 1 1)')));
 st_y
------
  1.5
(1 row)
```

참고: ST_Centroid, ST_GeomFromEWKT, ST_M, ST_X, ST_YMax, ST_YMin, ST_Z

8.5.39. ST_YMax

ST_YMax — 2 차원 경계 상자(box2d) 또는 3 차원 경계 상자(box3d) 또는 도형의 최대 Y 값을 반환합니다

Synopsis

float **ST_YMax**(box3d *aGeomorBox2DorBox3D*);

설명: 2 차원 경계 상자(box2d) 또는 3 차원 경계 상자(box3d) 또는 도형의 최대 Y 값을 반환합니다.



이 함수가 box3d 를 위해서만 정의됐긴 해도, 도형 및 box2d 에 대해 정의된 자동 형변환 습성 때문에 도형 및 box2d 를 위해서도 작동합니다. 하지만 도형 또는 box2d 의 텍스트 표현식을 입력할 수는 없습니다. 텍스트 표현식은 자동 형변환의 대상이 아니기 때문입니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

예시

```
SELECT ST_YMax('BOX3D(1 2 3, 4 5 6)');
st_ymax
-------
5

SELECT ST_YMax(ST_GeomFromText('LINESTRING(1 3 4, 5 6 7)'));
st_ymax
-------
6

SELECT ST_YMax(CAST('BOX(-3 2, 3 4)' As box2d));
st_ymax
-------
4
-- 스트링 표현식을 box3d 로 자동 형변환하려 하기 때문에 오류가 발생하는 것을 지켜보십시오.
SELECT ST_YMax('LINESTRING(1 3, 5 6)');

--ERROR:  BOX3D parser - doesn't start with BOX3D(

SELECT   ST_YMax(ST_GeomFromEWKT('CIRCULARSTRING(220268   150415   1,220227
150505 2,220227 150406 3)'));
st_ymax
--------
150506.126829327
```

참고: ST_XMin, ST_XMax, ST_YMin, ST_ZMax, ST_ZMin

8.5.40. ST_YMin

ST_YMin — 2 차원 경계 상자(box2d) 또는 3 차원 경계 상자(box3d) 또는 도형의 최소 Y 값을 반환합니다.

Synopsis

float **ST_YMin**(box3d *aGeomorBox2DorBox3D*);

설명: 2 차원 경계 상자(box2d) 또는 3 차원 경계 상자(box3d) 또는 도형의 최소 Y 값을 반환합니다.



이 함수가 box3d 를 위해서만 정의됐긴 해도, 도형 및 box2d 에 대해 정의된 자동 형변환 습성 때문에 도형 및 box2d 를 위해서도 작동합니다. 하지만 도형 또는 box2d 의 텍스트 표현식을 입력할 수는 없습니다. 텍스트 표현식은 자동 형변환의 대상이 아니기 때문입니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

예시

```
SELECT ST_YMin('BOX3D(1 2 3, 4 5 6)');
st_ymin
-------
2

SELECT ST_YMin(ST_GeomFromText('LINESTRING(1 3 4, 5 6 7)'));
st_ymin
-------
3

SELECT ST_YMin(CAST('BOX(-3 2, 3 4)' As box2d));
st_ymin
-------
2
-- 스트링 표현식을 box3d 로 자동 형변환하려 하기 때문에 오류가 발생하는 것을 지켜보십시오.
SELECT ST_YMin('LINESTRING(1 3, 5 6)');

--ERROR:  BOX3D parser - doesn't start with BOX3D(

SELECT   ST_YMin(ST_GeomFromEWKT('CIRCULARSTRING(220268   150415   1,220227
150505 2,220227 150406 3)'));
st_ymin
--------
150406
```

참고: ST_GeomFromEWKT, ST_XMin, ST_XMax, ST_YMax, ST_ZMax, ST_ZMin

8.5.41. ST_Z

ST_Z — 포인트의 Z 좌표를 반환합니다. Z 좌표가 없는 경우 NULL 을 반환합니다. 포인트만 입력받을 수 있습니다.

Synopsis

float **ST_Z**(geometry *a_point*);

설명: 포인트의 Z 좌표를 반환합니다. Z 좌표가 없는 경우 NULL 을 반환합니다. 포인트만 입력받을 수 있습니다.

✔ 이 방법은 SQL/MM 사양을 만족합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
SELECT ST_Z(ST_GeomFromEWKT('POINT(1 2 3 4)'));
 st_z
------
       3
(1 row)
```

참고: ST_GeomFromEWKT, ST_M, ST_X, ST_Y, ST_ZMax, ST_ZMin

8.5.42. ST_ZMax

ST_ZMax — 2 차원 경계 상자(box2d) 또는 3 차원 경계 상자(box3d) 또는 도형의 최대 Z 값을 반환합니다.

Synopsis

float **ST_ZMax**(box3d *aGeomorBox2DorBox3D*);

설명: 2 차원 경계 상자(box2d) 또는 3 차원 경계 상자(box3d) 또는 도형의 최대 Z 값을 반환합니다.



이 함수가 box3d 를 위해서만 정의됐긴 해도, 도형 및 box2d 에 대해 정의된 자동 형변환 습성 때문에 도형 및 box2d 를 위해서도 작동합니다. 하지만 도형 또는 box2d 의 텍스트 표현식을 입력할 수는 없습니다. 텍스트 표현식은 자동 형변환의 대상이 아니기 때문입니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

예시

```
SELECT ST_ZMax('BOX3D(1 2 3, 4 5 6)');
st_zmax
-------
6
```

```
SELECT ST_ZMax(ST_GeomFromEWKT('LINESTRING(1 3 4, 5 6 7)'));
st_zmax
-------
7

SELECT ST_ZMax('BOX3D(-3 2 1, 3 4 1)' );
st_zmax
-------
1
-- 스트링 표현식을 box3d 로 자동 형변환하려 하기 때문에 오류가 발생하는 것을 지켜보십시오.
SELECT ST_ZMax('LINESTRING(1 3 4, 5 6 7)');

--ERROR:  BOX3D parser - doesn't start with BOX3D(

SELECT   ST_ZMax(ST_GeomFromEWKT('CIRCULARSTRING(220268   150415   1,220227
150505 2,220227 150406 3)'));
st_zmax
--------
3
```

참고: ST_GeomFromEWKT, ST_XMin, ST_XMax, ST_YMax, ST_YMin, ST_ZMax

8.5.43. ST_Zmflag

ST_Zmflag — 도형의 ZM(의미론적 차원) 플래그를 2 바이트 정수형(smallint)으로 반환합니다. 값이 0 일 때 2D, 1 일 때 3DM, 3 일 때 3DZ, 4 일 때 4D 를 뜻합니다.

Synopsis

smallint **ST_Zmflag**(geometry *geomA*);

설명: 도형의 ZM(의미론적 차원) 플래그를 2 바이트 정수형(smallint)으로 반환합니다. 값이 0 일 때 2D, 1 일 때 3DM, 3 일 때 3DZ, 4 일 때 4D 를 뜻합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

예시

```
SELECT ST_Zmflag(ST_GeomFromEWKT('LINESTRING(1 2, 3 4)'));
 st_zmflag
-----------
             0

SELECT ST_Zmflag(ST_GeomFromEWKT('LINESTRINGM(1 2 3, 3 4 3)'));
 st_zmflag
-----------
             1

SELECT ST_Zmflag(ST_GeomFromEWKT('CIRCULARSTRING(1 2 3, 3 4 3, 5 6 3)'));
 st_zmflag
-----------
             2
SELECT ST_Zmflag(ST_GeomFromEWKT('POINT(1 2 3 4)'));
```

```
 st_zmflag
-----------
              3
```

참고: ST_CoordDim, ST_NDims, ST_Dimension

8.5.44. ST_ZMin

ST_ZMin — 2 차원 경계 상자(box2d) 또는 3 차원 경계 상자(box3d) 또는 도형의 최대 Z 값을 반환합니다.

Synopsis

float **ST_ZMin**(box3d *aGeomorBox2DorBox3D*);

설명: 2 차원 경계 상자(box2d) 또는 3 차원 경계 상자(box3d) 또는 도형의 최소 Z 값을 반환합니다.



이 함수가 box3d 를 위해서만 정의됐긴 해도, 도형 및 box2d 에 대해 정의된 자동 형변환 습성 때문에 도형 및 box2d 를 위해서도 작동합니다. 하지만 도형 또는 box2d 의 텍스트 표현식을 입력할 수는 없습니다. 텍스트 표현식은 자동 형변환의 대상이 아니기 때문입니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

예시

```
SELECT ST_ZMin('BOX3D(1 2 3, 4 5 6)');
st_zmin
-------
3

SELECT ST_ZMin(ST_GeomFromEWKT('LINESTRING(1 3 4, 5 6 7)'));
st_zmin
-------
4

SELECT ST_ZMin('BOX3D(-3 2 1, 3 4 1)' );
st_zmin
-------
1
-- 스트링 표현식을 box3d 로 자동 형변환하려 하기 때문에 오류가 발생하는 것을 지켜보십시오.
SELECT ST_ZMin('LINESTRING(1 3 4, 5 6 7)');

--ERROR:  BOX3D parser - doesn't start with BOX3D(

SELECT   ST_ZMin(ST_GeomFromEWKT('CIRCULARSTRING(220268   150415   1,220227
150505 2,220227 150406 3)'));
st_zmin
--------
1
```

참고: ST_GeomFromEWKT, ST_GeomFromText, ST_XMin, ST_XMax, ST_YMax, ST_YMin, ST_ZMax

8.6. 도형 편집자(editor)

ST_AddPoint — 라인스트링에 포인트를 추가합니다.

ST_Affine — 도형에 3 차원 아핀 변환을 적용합니다.

ST_Force2D — 도형을 "2 차원 모드"로 강제합니다.

ST_Force3D — 도형을 XYZ 모드로 강제합니다. ST_Force3DZ 라고도 합니다.

ST_Force3DZ — 도형을 XYZ 모드로 강제합니다.

ST_Force3DM — 도형을 XYM 모드로 강제합니다.

ST_Force4D — 도형을 XYZM 모드로 강제합니다.

ST_ForceCollection — 도형을 도형 집합으로 변환합니다.

ST_ForceSFS — 도형이 SFS 1.1 도형 유형만을 쓰도록 강제합니다.

ST_ForceRHR — 폴리곤 안에 있는 꼭짓점들의 방향(orientation)이 오른손 법칙(Right-Hand Rule)을 따르도록 강제합니다.

ST_ForceCurve — 적용이 가능한 경우, 도형을 해당 도형의 만곡 유형으로 상위 형변환(upcast)합니다.

ST_LineMerge — 멀티라인스트링을 함께 엮어 형성된 라인스트링(들)(의 집합)을 반환합니다.

ST_CollectionExtract — (멀티) 도형을 입력하면 지정한 유형의 요소들로만 이루어진 (멀티) 도형을 반환합니다.

ST_CollectionHomogenize — 도형 집합을 입력받아 해당 내용의 "가장 단순한" 표현식을 반환합니다.

ST_Multi — 도형을 멀티 유형 도형으로 반환합니다.

ST_Normalize — 도형을 해당 도형의 기본형으로 반환합니다.

ST_RemovePoint — 라인스트링에서 포인트를 제거합니다.

ST_Reverse — 꼭짓점들의 순서가 반대인 도형을 반환합니다.

ST_Rotate — 원점(origin)을 기준으로 도형의 rotRadians 를 반시계 방향으로 회전시킵니다.

ST_RotateX — X 축을 기준으로 도형의 rotRadians 를 회전시킵니다.

ST_RotateY — Y 축을 기준으로 도형의 rotRadians 를 회전시킵니다.

ST_RotateZ — Z 축을 기준으로 도형의 rotRadians 를 회전시킵니다.

ST_Scale — 주어진 인수로 도형의 크기를 조정합니다.

ST_Segmentize — 주어진 거리보다 더 긴 구간이 없도록 수정된 도형/지리형을 반환합니다.

ST_SetPoint — 라인스트링의 포인트를 주어진 포인트로 대체합니다.

ST_SetSRID — 도형에 적용된 SRID 를 특정 정수값으로 설정합니다.

ST_SnapToGrid — 입력 도형의 모든 포인트를 정규 그리드로 스냅(snap)시킵니다.

ST_Snap — 입력 도형의 분절 구간과 꼭짓점을 참조 도형의 꼭짓점으로 스냅시킵니다.

ST_Transform — 입력 도형의 좌표를 다른 공간 참조 시스템으로 변환시킨 새 도형을 반환합니다.

ST_Translate — 주어진 오프셋으로 도형을 변형시킵니다.

ST_TransScale — 주어진 인수 및 오프셋으로 도형을 변형시킵니다.

8.6.1. ST_AddPoint

ST_AddPoint — 라인스트링에 포인트를 추가합니다.

Synopsis

geometry **ST_AddPoint**(geometry *linestring*, geometry *point*);

geometry **ST_AddPoint**(geometry *linestring*, geometry *point*, integer *position*);

설명: 라인스트링의 포인트 <position> 앞에(0-기반 인덱스) 포인트를 추가합니다. 세 번째 파라미터는 생략하거나, 추가 작업에 대해 -1 로 설정할 수 있습니다.

1.1.0 버전부터 사용할 수 있습니다.

✓ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
-- 테이블에 있는 모든 라인스트링이 닫혔는지 확인합니다.
-- 라인스트링이 닫혀 있지 않은 경우
-- 각 라인스트링의 끝에 시작점을 추가합니다.
UPDATE sometable
SET the_geom = ST_AddPoint(the_geom, ST_StartPoint(the_geom))
FROM sometable
WHERE ST_IsClosed(the_geom) = false;

-- 3D 라인에 포인트 추가하기
SELECT ST_AsEWKT(ST_AddPoint(ST_GeomFromEWKT('LINESTRING(0 0 1, 1 1 1)'),
ST_MakePoint(1, 2, 3)));

-- 결과
st_asewkt
----------
LINESTRING(0 0 1,1 1 1,1 2 3)
```

참고: ST_RemovePoint, ST_SetPoint

8.6.2. ST_Affine

ST_Affine — 도형에 3 차원 아핀 변환을 적용합니다.

Synopsis

geometry **ST_Affine**(geometry *geomA*, float *a*, float *b*, float *c*, float *d*, float *e*, float *f*, float *g*, float *h*, float *i*, float *xoff*, float *yoff*, float *zoff*);

geometry **ST_Affine**(geometry *geomA*, float *a*, float *b*, float *d*, float *e*, float *xoff*, float *yoff*);

설명: 변형, 회전, 축척 변경 등과 같은 작업을 한번에 처리하기 위해 도형에 3 차원 아핀 변환을 적용합니다.

버전 1

```
ST_Affine(geom, a, b, c, d, e, f, g, h, i, xoff, yoff, zoff)
```

함수 호출은 변환 매트릭스

```
/ a  b  c  xoff \
| d  e  f  yoff |
| g  h  i  zoff |
\ 0  0  0    1 /
```

를 표현하며, 꼭짓점은 다음과 같이 변환됩니다:

```
x' = a*x + b*y + c*z + xoff
y' = d*x + e*y + f*z + yoff
z' = g*x + h*y + i*z + zoff
```

이 단원에 정리된 모든 번역/크기 조정 함수들은 이런 아핀 변환을 통해 표현됩니다.

버전 2: 도형에 2 차원 아핀 변환을 적용합니다.

```
ST_Affine(geom, a, b, d, e, xoff, yoff)
```

함수 호출은 변환 매트릭스

```
/  a  b  0  xoff  \      /  a  b  xoff  \
|  d  e  0  yoff  | rsp. |  d  e  yoff  |
|  0  0  1    0  |      \  0  0    1  /
\  0  0  0    1  /
```

를 표현하며, 꼭짓점은 다음과 같이 변환됩니다:

```
x' = a*x + b*y + xoff
y' = d*x + e*y + yoff
z' = z
```

이 메소드는 버전 1 3D 메소드의 하위 사례입니다.

```
SELECT DropGeometryTable ('my_schema','my_spatial_table');
```

개선 사항: 2.0.0 버전부터 2.0.0 버전부터 다면체 표면, 삼각형 및 TIN 을 지원합니다.

1.1.2 버전부터 사용할 수 있습니다. 1.2.2 버전에서 함수명을 Affine 에서 ST_Affine 으로 변경했습니다.



1.3.4 미만 버전에서 이 함수에 만곡 도형(curve)을 담고 있는 도형을 입력하면 충돌이 일어났습니다. 1.3.4 버전부터 이 버그가 해결됐습니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

예시

```
-- Z 축을 기준으로 3D 라인을 180 도 회전시킵니다. ST_Rotate() 함수를 풀어 쓴 방법이라는
점에 주의하십시오.
 SELECT   ST_AsEWKT(ST_Affine(the_geom,      cos(pi()),   -sin(pi()),   0,
sin(pi()), cos(pi()), 0,  0, 0, 1,  0, 0, 0)) As using_affine,
       ST_AsEWKT(ST_Rotate(the_geom, pi())) As using_rotate
      FROM (SELECT ST_GeomFromEWKT('LINESTRING(1 2 3, 1 4 3)') As the_geom)
As foo;
      using_affine        |       using_rotate
---------------------------+---------------------------
 LINESTRING(-1 -2 3,-1 -4 3) | LINESTRING(-1 -2 3,-1 -4 3)
(1 row)

-- X 및 Z 축에 대해 3D 라인을 180 도 회전시킵니다.
SELECT ST_AsEWKT(ST_Affine(the_geom, cos(pi()), -sin(pi()), 0, sin(pi()),
cos(pi()), -sin(pi()), 0, sin(pi()), cos(pi()), 0, 0, 0))
      FROM (SELECT ST_GeomFromEWKT('LINESTRING(1 2 3, 1 4 3)') As the_geom)
As foo;
         st_asewkt
-----------------------------
 LINESTRING(-1 -2 -3,-1 -4 -3)
(1 row)
```

참고: ST_Rotate, ST_Scale, ST_Translate, ST_TransScale

8.6.3. ST_Force2D

ST_Force2D — 도형을 "2 차원 모드"로 강제합니다.

Synopsis

geometry **ST_Force2D**(geometry *geomA*);

설명: 도형을 "2 차원 모드"로 강제해서 출력 표현식이 X 및 Y 좌표만을 가지도록 합니다. 이 함수는 (OGC 가 사양에 2 차원 도형만 있기 때문에) OGC 준수 출력물을 강제하는 데 유용합니다.

개선 사항: 2.0.0 버전부터 다면체 표면(polyhedral surface)을 지원합니다.

변경 사항: 2.1.0 미만 버전, 즉 2.0.x 버전까지 이 함수의 명칭은 ST_Force_2D 였습니다.

✓ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✓ 이 함수는 다면체 표면을 지원합니다.

✓ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
SELECT    DropGeometryTable     ('my_schema','my_spatial_table');    SELECT
ST_AsEWKT(ST_Force2D(ST_GeomFromEWKT('CIRCULARSTRING(1 1 2, 2 3 2, 4 5 2,
6 7 2, 5 6 2)')));
             st_asewkt
-----------------------------------
CIRCULARSTRING(1 1,2 3,4 5,6 7,5 6)

SELECT  ST_AsEWKT(ST_Force2D('POLYGON((0 0 2,0 5 2,5 0 2,0 0 2),(1 1 2,3 1
2,1 3 2,1 1 2))'));

                            st_asewkt
-----------------------------------------------
 POLYGON((0 0,0 5,5 0,0 0),(1 1,3 1,1 3,1 1))
```

참고: ST_Force3D

8.6.4. ST_Force3D

ST_Force3D — 도형을 XYZ 모드로 강제합니다. ST_Force3DZ 라고도 합니다.

Synopsis

geometry **ST_Force3D**(geometry *geomA*);

설명: 도형을 XYZ 모드로 강제합니다. ST_Force3DZ 함수와 동일한 함수입니다. 도형이 Z 요소를 가지고 있지 않을 경우, Z 좌표를 0 으로 고정합니다.

개선 사항: 2.0.0 버전부터 다면체 표면(polyhedral surface)을 지원합니다.

변경 사항: 2.1.0 미만 버전, 즉 2.0.x 버전까지 이 함수의 명칭은 ST_Force_3D 였습니다.

✓ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✓ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
-- 기존 3D 도형은 변경되지 않습니다.
SELECT ST_AsEWKT(ST_Force3D(ST_GeomFromEWKT('CIRCULARSTRING(1 1 2, 2 3 2,
4 5 2, 6 7 2, 5 6 2)')));
st_asewkt
-----------------------------------------------
 CIRCULARSTRING(1 1 2,2 3 2,4 5 2,6 7 2,5 6 2)


SELECT   ST_AsEWKT(ST_Force3D('POLYGON((0  0,0  5,5  0,0  0),(1  1,3  1,1  3,1
1))'));

st_asewkt
-----------------------------------------------------------------
 POLYGON((0 0 0,0 5 0,5 0 0,0 0 0),(1 1 0,3 1 0,1 3 0,1 1 0))
```

참고: ST_AsEWKT, ST_Force2D, ST_Force3DM, ST_Force3DZ

8.6.5. ST_Force3DZ

ST_Force3DZ — 도형을 XYZ 모드로 강제합니다.

Synopsis

geometry **ST_Force3DZ**(geometry *geomA*);

설명: 도형을 XYZ 모드로 강제합니다. ST_Force3D 함수와 동일한 함수입니다. 도형이 Z 요소를 가지고 있지 않을 경우, Z 좌표를 0 으로 고정합니다.

개선 사항: 2.0.0 버전부터 다면체 표면(polyhedral surface)을 지원합니다.

변경 사항: 2.1.0 미만 버전, 즉 2.0.x 버전까지 이 함수의 명칭은 ST_Force_3DZ 였습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
-- 기존 3D 도형은 변경되지 않습니다.
SELECT ST_AsEWKT(ST_Force3DZ(ST_GeomFromEWKT('CIRCULARSTRING(1 1 2, 2 3 2,
4 5 2, 6 7 2, 5 6 2)')));
st_asewkt
-----------------------------------------------
 CIRCULARSTRING(1 1 2,2 3 2,4 5 2,6 7 2,5 6 2)
```

```
SELECT  ST_AsEWKT(ST_Force3DZ('POLYGON((0 0,0 5,5 0,0 0),(1 1,3 1,1 3,1
1))'));

st_asewkt
------------------------------------------------------------------
 POLYGON((0 0 0,0 5 0,5 0 0,0 0 0),(1 1 0,3 1 0,1 3 0,1 1 0))
```

참고: ST_AsEWKT, ST_Force2D, ST_Force3DM, ST_Force3D

8.6.6. ST_Force3DM

ST_Force3DM — 도형을 XYM 모드로 강제합니다.

Synopsis

geometry **ST_Force3DM**(geometry *geomA*);

설명: 도형을 XYM 모드로 강제합니다. 도형이 M 요소를 가지고 있지 않을 경우, M 좌표를 0 으로 고정합니다. 도형이 Z 요소를 가지고 있다면 제거합니다.

변경 사항: 2.1.0 미만 버전, 즉 2.0.x 버전까지 이 함수의 명칭은 ST_Force_3DM 이었습니다.

✓ 이 방법은 원형 스트링 및 곡선을 지원합니다.

예시

```
-- 기존 3D 도형은 변경되지 않습니다.
SELECT ST_AsEWKT(ST_Force3DM(ST_GeomFromEWKT('CIRCULARSTRING(1 1 2, 2 3 2,
4 5 2, 6 7 2, 5 6 2)')));
st_asewkt
------------------------------------------------
 CIRCULARSTRINGM(1 1 0,2 3 0,4 5 0,6 7 0,5 6 0)


SELECT  ST_AsEWKT(ST_Force3DM('POLYGON((0 0 1,0 5 1,5 0 1,0 0 1),(1 1 1,3
1 1,1 3 1,1 1 1))'));

st_asewkt
------------------------------------------------------------------
 POLYGONM((0 0 0,0 5 0,5 0 0,0 0 0),(1 1 0,3 1 0,1 3 0,1 1 0))
```

참고: ST_AsEWKT, ST_Force2D, ST_Force3DM, ST_Force3D, ST_GeomFromEWKT

8.6.7. ST_Force4D

ST_Force4D — 도형을 XYZM 모드로 강제합니다.

Synopsis

geometry **ST_Force4D**(geometry *geomA*);

설명: 도형을 XYZM 모드로 강제합니다. Z 또는 M 차원이 없을 경우, Z 또는 M 차원을 0 으로 고정합니다.

변경 사항: 2.1.0 미만 버전, 즉 2.0.x 버전까지 이 함수의 명칭은 ST_Force_4D 였습니다.

✓ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✓ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
-- 기존 3D 도형은 변경되지 않습니다.
SELECT ST_AsEWKT(ST_Force4D(ST_GeomFromEWKT('CIRCULARSTRING(1 1 2, 2 3 2,
4 5 2, 6 7 2, 5 6 2)')));
st_asewkt
------------------------------------------------------------
 CIRCULARSTRING(1 1 2 0,2 3 2 0,4 5 2 0,6 7 2 0,5 6 2 0)



SELECT  ST_AsEWKT(ST_Force4D('MULTILINESTRINGM((0 0 1,0 5 2,5 0 3,0 0 4),(1
1 1,3 1 1,1 3 1,1 1 1))'));

st_asewkt
------------------------------------------------------------------------------
-------------
 MULTILINESTRING((0 0 0 1,0 5 0 2,5 0 0 3,0 0 0 4),(1 1 0 1,3 1 0 1,1 3 0
1,1 1 0 1))
```

참고: ST_AsEWKT, ST_Force2D, ST_Force3DM, ST_Force3D

8.6.8. ST_ForceCollection

ST_ForceCollection — 도형을 도형 집합으로 변환합니다.

Synopsis

geometry **ST_ForceCollection**(geometry *geomA*);

설명: 도형을 도형 집합으로 변환합니다. 이 함수는 WKB 표현식을 단순화하는 데 유용합니다.

개선 사항: 2.0.0 버전부터 다면체 표면(polyhedral surface)을 지원합니다.

1.2.2 버전부터 사용할 수 있습니다. 1.3.4 미만 버전에서 이 함수에 만곡 도형(curve)을 담고 있는 도형을 입력하면 충돌이 일어났습니다. 1.3.4 버전부터 이 버그가 해결됐습니다.

변경 사항: 2.1.0 미만 버전, 즉 2.0.x 버전까지 이 함수의 명칭은 ST_Force_Collection 이었습니다.

✓ 이 함수는 다면체 표면을 지원합니다.

✓ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✓ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
SELECT  ST_AsEWKT(ST_ForceCollection('POLYGON((0 0 1,0 5 1,5 0 1,0 0 1),(1
1 1,3 1 1,1 3 1,1 1 1))'));

st_asewkt
-----------------------------------------------------------------------------
---------
 GEOMETRYCOLLECTION(POLYGON((0 0 1,0 5 1,5 0 1,0 0 1),(1 1 1,3 1 1,1 3 1,1
1 1)))


  SELECT               ST_AsText(ST_ForceCollection('CIRCULARSTRING(220227
150406,2220227 150407,220227 150406)'));
st_astext
-----------------------------------------------------------------------------
-------
 GEOMETRYCOLLECTION(CIRCULARSTRING(220227   150406,2220227   150407,220227
150406))
(1 row)


-- 다면체 예시 --
SELECT  ST_AsEWKT(ST_ForceCollection('POLYHEDRALSURFACE(((0  0  0,0  0  1,0  1
1,0 1 0,0 0 0)),
 ((0 0 0,0 1 0,1 1 0,1 0 0,0 0 0)),
 ((0 0 0,1 0 0,1 0 1,0 0 1,0 0 0)),
 ((1 1 0,1 1 1,1 0 1,1 0 0,1 1 0)),
 ((0 1 0,0 1 1,1 1 1,1 1 0,0 1 0)),
 ((0 0 1,1 0 1,1 1 1,0 1 1,0 0 1)))'))

st_asewkt
-----------------------------------------------------------------------------
---------
GEOMETRYCOLLECTION(
  POLYGON((0 0 0,0 0 1,0 1 1,0 1 0,0 0 0)),
  POLYGON((0 0 0,0 1 0,1 1 0,1 0 0,0 0 0)),
  POLYGON((0 0 0,1 0 0,1 0 1,0 0 1,0 0 0)),
  POLYGON((1 1 0,1 1 1,1 0 1,1 0 0,1 1 0)),
  POLYGON((0 1 0,0 1 1,1 1 1,1 1 0,0 1 0)),
  POLYGON((0 0 1,1 0 1,1 1 1,0 1 1,0 0 1))
)
```

참고: ST_AsEWKT, ST_Force2D, ST_Force3DM, ST_Force3D, ST_GeomFromEWKT

8.6.9. ST_ForceSFS

ST_ForceSFS — 도형이 SFS 1.1 도형 유형만을 쓰도록 강제합니다.

Synopsis

geometry **ST_ForceSFS**(geometry *geomA*);

geometry **ST_ForceSFS**(geometry *geomA*, text *version*);

설명:

✓ 이 함수는 다면체 표면을 지원합니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

8.6.10. ST_ForceRHR

ST_ForceRHR — 폴리곤 안에 있는 꼭짓점들의 방향(orientation)이 오른손 법칙(Right-Hand Rule)을 따르도록 강제합니다.

Synopsis

geometry **ST_ForceRHR**(geometry *g*);

설명: 폴리곤 안에 있는 꼭짓점들의 방향(orientation)이 오른손 법칙(Right-Hand Rule)을 따르도록 강제합니다. GIS 용어 관점에서, 이 말은 폴리곤으로 경계가 정해진 면적이 해당 경계선(boundary)의 오른쪽에 있다는 뜻입니다. 그 중에서도 외곽 고리는 시계 방향, 내곽 고리는 반시계 방향을 향한다는 뜻입니다.

개선 사항: 2.0.0 버전부터 다면체 표면(polyhedral surface)을 지원합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 함수는 다면체 표면을 지원합니다.

예시

```
SELECT ST_AsEWKT(
  ST_ForceRHR(
       'POLYGON((0 0 2, 5 0 2, 0 5 2, 0 0 2),(1 1 2, 1 3 2, 3 1 2, 1 1 2))'
  )
);
st_asewkt
------------------------------------------------------------------
 POLYGON((0 0 2,0 5 2,5 0 2,0 0 2),(1 1 2,3 1 2,1 3 2,1 1 2))
(1 row)
```

참고: ST_BuildArea, ST_Polygonize, ST_Reverse

8.6.11. ST_ForceCurve

ST_ForceCurve — 적용이 가능한 경우, 도형을 해당 도형의 만곡 유형으로 상위 형변환(upcast)합니다.

Synopsis

geometry **ST_ForceCurve**(geometry *g*);

설명: 적용이 가능한 경우, 도형을 해당 도형의 만곡 표현식으로 바꿉니다. 라인을 복합곡선(compoundcurve)으로, 멀티라인을 멀티커브로, 폴리곤을 커브폴리곤으로, 멀티폴리곤을 멀티서페이스(multisurface)로 변환합니다. 입력 폴리곤이 이미 만곡된 표현식인 경우 입력 표현식을 그대로 반환합니다.

2.2.0 버전부터 사용할 수 있습니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

예시

```
SELECT ST_AsText(
  ST_ForceCurve(
       'POLYGON((0 0 2, 5 0 2, 0 5 2, 0 0 2),(1 1 2, 1 3 2, 3 1 2, 1 1
2))'::geometry
  )
);
st_astext
------------------------------------------------------------------------
 CURVEPOLYGON Z ((0 0 2,5 0 2,0 5 2,0 0 2),(1 1 2,1 3 2,3 1 2,1 1 2))
(1 row)
```

참고: ST_LineToCurve

8.6.12. ST_LineMerge

ST_LineMerge — 멀티라인스트링을 함께 엮어 형성된 라인스트링(들)(의 집합)을 반환합니다.

Synopsis

geometry **ST_LineMerge**(geometry *amultilinestring*);

설명: 멀티라인스트링을 구성하는 라인 조각(line work)을 함께 엮어 형성된 라인스트링(들)(의 집합)을 반환합니다.



멀티라인스트링 또는 라인스트링만 입력하십시오. 이 함수에 폴리곤이나 도형 집합을 입력하면, 텅 빈 도형 집합을 반환할 것입니다.

1.1.0 버전부터 사용할 수 있습니다.



GEOS 2.1.0 이상 버전이 필요합니다.

예시

```
SELECT ST_AsText(ST_LineMerge(
```

```
ST_GeomFromText('MULTILINESTRING((-29 -27,-30 -29.7,-36 -31,-45 -33),(-45
-33,-46 -32))')
             )
);
st_astext
------------------------------------------------------------------------
-------------------------
LINESTRING(-29 -27,-30 -29.7,-36 -31,-45 -33,-46 -32)
(1 row)

-- 합칠 수 없는 경우 원래 멀티라인스트링을 반환합니다.
SELECT ST_AsText(ST_LineMerge(
ST_GeomFromText('MULTILINESTRING((-29 -27,-30 -29.7,-36 -31,-45 -33),(-
45.2 -33.2,-46 -32))')
)
);
st_astext
----------------
MULTILINESTRING((-45.2 -33.2,-46 -32),(-29 -27,-30 -29.7,-36 -31,-45 -33))
```

참고: ST_Segmentize, ST_LineSubstring

8.6.13. ST_CollectionExtract

ST_CollectionExtract — (멀티) 도형을 입력하면 지정한 유형의 요소들로만 이루어진 (멀티) 도형을 반환합니다.

Synopsis

geometry **ST_CollectionExtract**(geometry *collection*, integer *type*);

설명: (멀티) 도형을 입력하면 지정된 유형의 요소들로만 이루어진 (멀티) 도형을 반환합니다. 지정되지 않은 유형인 하위 도형은 무시당합니다. 설정한 유형의 하위 도형이 없을 경우, 텅 빈 도형을 반환할 것입니다. 포인트, 라인, 폴리곤만 지원합니다. 유형 숫자는 1 이 포인트, 2 가 라인스트링, 3 이 폴리곤입니다.

1.5.0 버전부터 사용할 수 있습니다.



1.5.3 미만 버전에서 집합이 아닌 도형을 입력받은 경우, 이 함수는 유형에 상관없이 입력 도형을 그대로 반환했습니다. 1.5.3 버전부터 일치하지 않는 단일 도형을 입력받으면 NULL 을 반환합니다. 2.0.0 버전부터는 일치하지 않는 모든 경우에 EMPTY(텅 빈 도형)을 반환합니다..



폴리곤(3)을 지정했을 때 경계가 겹치더라도 멀티폴리곤을 반환합니다. ST_Split 의 결과에 이 함수를 적용하는 것과 같은 많은 경우, 유효하지 않은 멀티폴리곤을 반환하게 됩니다.

예시

```
-- 상수: 1 == POINT, 2 == LINESTRING, 3 == POLYGON
SELECT
ST_AsText(ST_CollectionExtract(ST_GeomFromText('GEOMETRYCOLLECTION(GEOMETRY
COLLECTION(POINT(0 0)))'),1));
st_astext
```

```
---------------
MULTIPOINT(0 0)
(1 row)

SELECT
ST_AsText(ST_CollectionExtract(ST_GeomFromText('GEOMETRYCOLLECTION(GEOMETRY
COLLECTION(LINESTRING(0 0, 1 1)),LINESTRING(2 2, 3 3))'),2));
st_astext
---------------
MULTILINESTRING((0 0, 1 1), (2 2, 3 3))
(1 row)
```

참고: ST_Multi, ST_Dump, ST_CollectionHomogenize

8.6.14. ST_CollectionHomogenize

ST_CollectionHomogenize — 도형 집합을 입력받아 해당 내용의 "가장 단순한" 표현식을 반환합니다.

Synopsis

geometry **ST_CollectionHomogenize**(geometry *collection*);

설명: 도형 집합을 입력받아 해당 내용의 "가장 단순한" 표현식을 반환합니다. 단일 개체(singleton)는 단일 개체를 반환할 것입니다. 동일 유형으로 이루어진(homogeneous) 집합은 적절한 멀티 유형으로 반환될 것입니다.



폴리곤(3)을 지정했을 때 경계가 겹치더라도 멀티폴리곤을 반환합니다. ST_Split 의 결과에 이 함수를 적용하는 것과 같은 많은 경우, 유효하지 않은 멀티폴리곤을 반환하게 됩니다.

2.0.0 버전부터 사용할 수 있습니다.

예시

```
SELECT       ST_AsText(ST_CollectionHomogenize('GEOMETRYCOLLECTION(POINT(0
0))'));

       st_astext
       ------------
        POINT(0 0)
       (1 row)

  SELECT      ST_AsText(ST_CollectionHomogenize('GEOMETRYCOLLECTION(POINT(0
0),POINT(1 1))'));

       st_astext
       ---------------------
        MULTIPOINT(0 0,1 1)
       (1 row)
```

참고: ST_Multi, ST_CollectionExtract

8.6.15. ST_Multi

ST_Multi — 도형을 멀티 유형 도형으로 반환합니다.

Synopsis

geometry **ST_Multi**(geometry *g1*);

설명: 도형을 멀티 유형의 도형으로 반환합니다. 해당 도형이 이미 멀티형(MULTI*)일 경우, 그대로 반환합니다.

예시

```
SELECT  ST_AsText(ST_Multi(ST_GeomFromText('POLYGON((743238  2967416,743238
2967450,
                  743265 2967450,743265.625 2967416,743238 2967416))')));
                  st_astext
                  ----------------------------------------------------------
-------------------------------------------
                  MULTIPOLYGON(((743238   2967416,743238   2967450,743265
2967450,743265.625 2967416,
                  743238 2967416)))
                  (1 row)
```

참고: ST_AsText

8.6.16. ST_Normalize

ST_Normalize — 도형을 해당 도형의 기본형으로 반환합니다.

Synopsis

geometry **ST_Normalize**(geometry *geom*);

설명: 도형을 해당 도형의 정규화된/기본 형태로 반환합니다. 폴리곤 고리, 폴리곤 내부 고리에 있는 꼭짓점, 멀티 유형 도형 집합체의 요소를 재정렬할 수도 있습니다.

대부분의 경우, 테스트 작업 목적으로만 쓸모가 있습니다(기대한 결과물과 반환된 결과물의 비교 등).

예시

```
SELECT ST_AsText(ST_Normalize(ST_GeomFromText(
  'GEOMETRYCOLLECTION(
    POINT(2 3),
    MULTILINESTRING((0 0, 1 1),(2 2, 3 3)),
    POLYGON(
      (0 10,0 0,10 0,10 10,0 10),
      (4 2,2 2,2 4,4 4,4 2),
      (6 8,8 8,8 6,6 6,6 8)
    )
  )'
)));
                                                          st_astext
```

```
------------------------------------------------------------------------
------------------------------------------------------------------------
--
 GEOMETRYCOLLECTION(POLYGON((0 0,0 10,10 10,10 0,0 0),(6 6,8 6,8 8,6 8,6
6),(2 2,4 2,4 4,2 4,2 2)),MULTILINESTRING((2 2,3 3),(0 0,1 1)),POINT(2 3))
(1 row)
```

참고: ST_Equals,

8.6.17. ST_RemovePoint

ST_RemovePoint — 라인스트링에서 포인트를 제거합니다.

Synopsis

geometry **ST_RemovePoint**(geometry *linestring*, integer *offset*);

설명: 라인스트링의 0-기반 인덱스를 입력받아 해당 라인스트링에서 포인트를 제거합니다. 닫힌 고리를 열린 라인스트링으로 바꾸는 데 유용합니다.

1.1.0 버전부터 사용할 수 있습니다.

✓ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
-- 종단점을 제거해서 어떤 라인스트링도 닫혀 있지 않도록 합니다.
-- 이 예시는 the_geom 의 유형이 라인스트링이라고 가정합니다.
UPDATE sometable
      SET the_geom = ST_RemovePoint(the_geom, ST_NPoints(the_geom) - 1)
      FROM sometable
      WHERE ST_IsClosed(the_geom) = true;
```

참고: ST_AddPoint, ST_NPoints, ST_NumPoints

8.6.18. ST_Reverse

ST_Reverse — 꼭짓점들의 순서가 반대인 도형을 반환합니다.

Synopsis

geometry **ST_Reverse**(geometry *g1*);

설명: 어떤 도형도 입력받을 수 있으며, 꼭짓점들의 순서를 반전시킵니다.

예시

```
SELECT ST_AsText(the_geom) as line, ST_AsText(ST_Reverse(the_geom)) As
reverseline
FROM
(SELECT ST_MakeLine(ST_MakePoint(1,2),
```

```
             ST_MakePoint(1,10)) As the_geom) as foo;
--result
             line        |    reverseline
---------------------+----------------------
LINESTRING(1 2,1 10) | LINESTRING(1 10,1 2)
```

8.6.19. ST_Rotate

ST_Rotate — 원점(origin)을 기준으로 도형의 rotRadians 를 반시계 방향으로 회전시킵니다.

Synopsis

geometry **ST_Rotate**(geometry *geomA*, float *rotRadians*);

geometry **ST_Rotate**(geometry *geomA*, float *rotRadians*, float *x0*, float *y0*);

geometry **ST_Rotate**(geometry *geomA*, float *rotRadians*, geometry *pointOrigin*);

설명: 원점(origin)을 기준으로 도형의 rotRadians 를 반시계 방향으로 회전시킵니다. 회전 원점을 포인트 도형 또는 x, y 좌표로 설정할 수 있습니다.원점을 설정하지 않을 경우, POINT(0 0)을 중심으로 도형을 회전시킵니다.

개선 사항: 2.0.0 버전부터 2.0.0 버전부터 다면체 표면, 삼각형 및 TIN 을 지원합니다.

개선 사항: 2.0.0 버전에서 회전 원점을 설정하는 추가 파라미터가 추가됐습니다.

1.1.2 버전부터 사용할 수 있습니다. 1.2.2 버전에서 함수명을 Rotate 에서 ST_Rotate 로 변경했습니다.

✓ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✓ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✓ 이 함수는 다면체 표면을 지원합니다.

✓ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

예시

```
-- 180 도 회전
SELECT ST_AsEWKT(ST_Rotate('LINESTRING (50 160, 50 50, 100 50)', pi()));
st_asewkt
---------------------------------------
 LINESTRING(-50 -160,-50 -50,-100 -50)
(1 row)

-- x=50, y=160 좌표를 중심으로 반시계 방향 30 도 회전
SELECT ST_AsEWKT(ST_Rotate('LINESTRING (50 160, 50 50, 100 50)', pi()/6,
50, 160));
st_asewkt
------------------------------------------------------------------------
--
 LINESTRING(50 160,105 64.7372055837117,148.301270189222 89.7372055837117)
(1 row)
```

```
-- 중심점 기준으로 시계 방향으로 60 도 회전
SELECT ST_AsEWKT(ST_Rotate(geom, -pi()/3, ST_Centroid(geom)))
FROM (SELECT 'LINESTRING (50 160, 50 50, 100 50)'::geometry AS geom) AS
foo;
st_asewkt
------------------------------------------------------------
 LINESTRING(116.4225 130.6721,21.1597 75.6721,46.1597 32.3708)
(1 row)
```

참고: ST_Affine, ST_RotateX, ST_RotateY, ST_RotateZ

8.6.20. ST_RotateX

ST_RotateX — X 축을 기준으로 도형의 rotRadians 를 회전시킵니다.

Synopsis

geometry **ST_RotateX**(geometry *geomA*, float *rotRadians*);

설명: X 축을 기준으로 geomA 도형의 rotRadians 를 회전시킵니다.



ST_RotateX(geomA, rotRadians) 함수는 ST_Affine(geomA, 1, 0, 0, 0, cos(rotRadians), -sin(rotRadians), 0, sin(rotRadians), cos(rotRadians), 0, 0, 0) 을 간략화한 것입니다.

개선 사항: 2.0.0 버전부터 2.0.0 버전부터 다면체 표면, 삼각형 및 TIN 을 지원합니다.

1.1.2 버전부터 사용할 수 있습니다. 1.2.2 버전에서 함수명을 RotateX 에서 ST_RotateX 로 변경했습니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

예시

```
-- X 축을 따라 라인을 90 도 회전
SELECT ST_AsEWKT(ST_RotateX(ST_GeomFromEWKT('LINESTRING(1 2 3, 1 1 1)'),
pi()/2));
st_asewkt
---------------------------
 LINESTRING(1 -3 2,1 -1 1)
```

참고: ST_Affine, ST_RotateY, ST_RotateZ

8.6.21. ST_RotateY

ST_RotateY — Y 축을 기준으로 도형의 rotRadians 를 회전시킵니다.

Synopsis

geometry **ST_RotateY**(geometry *geomA*, float *rotRadians*);

설명: Y 축을 기준으로 geomA 도형의 rotRadians 를 회전시킵니다.



ST_RotateY(geomA, rotRadians) 함수는 ST_Affine(geomA, cos(rotRadians), 0, sin(rotRadians), 0, 1, 0, -sin(rotRadians), 0, cos(rotRadians), 0, 0, 0) 을 간략화한 것입니다.

1.1.2 버전부터 사용할 수 있습니다. 1.2.2 버전에서 함수명을 RotateY 에서 ST_RotateY 로 변경했습니다.

개선 사항: 2.0.0 버전부터 2.0.0 버전부터 다면체 표면, 삼각형 및 TIN 을 지원합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

예시

```
-- Y 축을 따라 라인을 90 도 회전
 SELECT ST_AsEWKT(ST_RotateY(ST_GeomFromEWKT('LINESTRING(1 2 3, 1 1 1)'),
pi()/2));
              st_asewkt
---------------------------
 LINESTRING(3 2 -1,1 1 -1)
```

참고: ST_Affine, ST_RotateX, ST_RotateZ

8.6.22. ST_RotateZ

ST_RotateZ — Z 축을 기준으로 도형의 rotRadians 를 회전시킵니다.

Synopsis

geometry **ST_RotateZ**(geometry *geomA*, float *rotRadians*);

설명: Z 축을 기준으로 geomA 도형의 rotRadians 를 회전시킵니다.



이 함수는 ST_Rotate 함수와 동일합니다.



ST_RotateZ(geomA, rotRadians) 함수는 SELECT ST_Affine(geomA, cos(rotRadians), -sin(rotRadians), 0, sin(rotRadians), cos(rotRadians), 0, 0, 0, 1, 0, 0, 0) 을 간략화한 것입니다.

개선 사항: 2.0.0 버전부터 2.0.0 버전부터 다면체 표면, 삼각형 및 TIN 을 지원합니다.

1.1.2 버전부터 사용할 수 있습니다. 1.2.2 버전에서 함수명을 RotateZ 에서 ST_RotateZ 로 변경했습니다.



1.3.4 미만 버전에서 이 함수에 만곡 도형(curve)을 담고 있는 도형을 입력하면 충돌이 일어났습니다. 1.3.4 버전부터 이 버그가 해결됐습니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

예시

```
-- Z 축을 따라 라인을 90 도 회전
SELECT ST_AsEWKT(ST_RotateZ(ST_GeomFromEWKT('LINESTRING(1 2 3, 1 1 1)'),
pi()/2));
st_asewkt
---------------------------
 LINESTRING(-2 1 3,-1 1 1)

 -- Z 축을 기준으로 만곡 원(curved circle)을 회전
SELECT ST_AsEWKT(ST_RotateZ(the_geom, pi()/2))
FROM (SELECT ST_LineToCurve(ST_Buffer(ST_GeomFromText('POINT(234 567)'),
3)) As the_geom) As foo;


st_asewkt
------------------------------------------------------------------------
------------------------------------------------
 CURVEPOLYGON(CIRCULARSTRING(-567 237,-564.87867965644 236.12132034356,-
564 234,-569.12132034356 231.87867965644,-567 237))
```

참고: ST_Affine, ST_RotateX, ST_RotateY

8.6.23. ST_Scale

ST_Scale — 주어진 인수로 도형의 크기를 조정합니다.

Synopsis

geometry **ST_Scale**(geometry *geomA*, float *XFactor*, float *YFactor*, float *ZFactor*);

geometry **ST_Scale**(geometry *geomA*, float *XFactor*, float *YFactor*);

geometry **ST_Scale**(geometry *geom*, geometry *factor*);

설명: 좌표를 각각 대응하는 인수 파라미터로 곱해서 도형을 새로운 크기로 조정합니다.

도형을 factor 파라미터로 입력받는 버전은 2D, 3DM, 3DZ, 4D 포인트를 패스해서 지원하는 모든 차원에 대해 크기 조정(scale) 인수를 설정할 수 있습니다. factor 포인트에 없는 차원은 해당 차원의 크기 조정 대상에서 제외됩니다.



1.3.4 미만 버전에서 이 함수에 만곡 도형(curve)을 담고 있는 도형을 입력하면 충돌이 일어났습니다. 1.3.4 버전부터 이 버그가 해결됐습니다.

1.1.0 버전부터 사용할 수 있습니다.

개선 사항: 2.0.0 버전부터 2.0.0 버전부터 다면체 표면, 삼각형 및 TIN 을 지원합니다.

개선 사항: 2.2.0 버전부터 모든 차원(도형 파라미터)을 지원합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

✔ 이 함수는 M 좌표를 지원합니다.

예시

```
-- 버전 1: X, Y, Z 크기 조정
SELECT ST_AsEWKT(ST_Scale(ST_GeomFromEWKT('LINESTRING(1 2 3, 1 1 1)'), 0.5,
0.75, 0.8));
st_asewkt
--------------------------------------
 LINESTRING(0.5 1.5 2.4,0.5 0.75 0.8)

-- 버전 2: X, Y 크기 조정
 SELECT  ST_AsEWKT(ST_Scale(ST_GeomFromEWKT('LINESTRING(1  2  3,  1  1  1)'),
0.5, 0.75));
st_asewkt
---------------------------------
 LINESTRING(0.5 1.5 3,0.5 0.75 1)

-- 버전 3: X, Y, Z, M 크기 조정
 SELECT ST_AsEWKT(ST_Scale(ST_GeomFromEWKT('LINESTRING(1 2 3 4, 1 1 1 1)'),
   ST_MakePoint(0.5, 0.75, 2, -1)));
st_asewkt
-----------------------------------------
 LINESTRING(0.5 1.5 6 -4,0.5 0.75 2 -1)
```

참고: ST_Affine, ST_TransScale

8.6.24. ST_Segmentize

ST_Segmentize — 주어진 거리보다 더 긴 구간이 없도록 수정된 도형/지리형을 반환합니다.

Synopsis

geometry **ST_Segmentize**(geometry *geom*, float *max_segment_length*);

geography **ST_Segmentize**(geography *geog*, float *max_segment_length*);

설명: 주어진 max_segment_length 보다 긴 구간이 없도록 수정된 도형을 반환합니다. 거리 계산은 2D 로만 수행됩니다. 도형의 경우, 길이 단위는 공간 참조의 단위입니다. 지리형의 경우, 미터 단위를 씁니다.

1.2.2 버전부터 사용할 수 있습니다.

개선 사항: 2.1.0 버전부터 지리형을 지원하기 시작했습니다.

변경 사항: 2.1.0 버전에서 지리형을 지원하기 시작한 결과로, SELECT ST_Segmentize('LINESTRING(1 2, 3 4)',0.5); 구조는 분명하지 않은 함수 오류를 발생시킵니다. ST_GeomFromText, ST_GeogFromText 또는 SELECT ST_Segmentize('LINESTRING(1 2, 3 4)'::geometry,0.5);를 통해 도형/지리형 같은 객체의 유형을 적절히 지정해야 합니다.



이 함수는 분절 구간(segment)의 개수를 늘리기만 할 뿐입니다. 최대 길이보다 짧은 분절 구간의 길이가 늘어나지는 않습니다.

예시

```
SELECT ST_AsText(ST_Segmentize(
ST_GeomFromText('MULTILINESTRING((-29 -27,-30 -29.7,-36 -31,-45 -33),(-45
-33,-46 -32))')
             ,5)
);
st_astext
------------------------------------------------------------------------------
-------------------------
MULTILINESTRING((-29 -27,-30 -29.7,-34.886615700134 -30.758766735029,-36 -
31,
-40.8809353009198 -32.0846522890933,-45 -33),
(-45 -33,-46 -32))
(1 row)

SELECT ST_AsText(ST_Segmentize(ST_GeomFromText('POLYGON((-29 28, -30 40, -
29 28))'),10));
st_astext
----------------------
POLYGON((-29      28,-29.8304547985374      37.9654575824488,-30      40,-
29.1695452014626 30.0345424175512,-29 28))
(1 row)
```

참고: ST_LineSubstring

8.6.25. ST_SetPoint

ST_SetPoint — 라인스트링의 포인트를 주어진 포인트로 대체합니다.

Synopsis

geometry **ST_SetPoint**(geometry *linestring*, integer *zerobasedposition*, geometry *point*);

설명: 라인스트링의 N 번째 포인트를 주어진 포인트로 대체합니다. 인덱스는 0-기반입니다. 음수 인덱스는 반대로 세기 때문에, -1 이 마지막 포인트가 됩니다. 꼭짓점 하나가 움직였을 때 연결 관계를 유지하려는 경우 이 함수가 트리거 역할로 특히 유용합니다.

1.1.0 버전부터 사용할 수 있습니다.

업데이트 사항: 2.3.0 버전에서 음수 인덱스를 지원하기 시작했습니다.

✓ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
-- 라인스트링의 첫 번째 포인트를 (-1, 3)에서 (-1, 1)로 변경
SELECT ST_AsText(ST_SetPoint('LINESTRING(-1 2,-1 3)', 0, 'POINT(-1 1)'));
         st_astext
-----------------------
 LINESTRING(-1 1,-1 3)

-- 라인스트링의 마지막 포인트를 변경(이번에는 3D 라인을 다뤄봅니다)
SELECT ST_AsEWKT(ST_SetPoint(foo.the_geom, ST_NumPoints(foo.the_geom) - 1,
ST_GeomFromEWKT('POINT(-1 1 3)')))
FROM  (SELECT  ST_GeomFromEWKT('LINESTRING(-1  2  3,-1  3  4,  5  6  7)')  As
the_geom) As foo;
         st_asewkt
-----------------------
LINESTRING(-1 2 3,-1 3 4,-1 1 3)

SELECT ST_AsText(ST_SetPoint(g, -3, p))
FROM ST_GEomFromText('LINESTRING(0 0, 1 1, 2 2, 3 3, 4 4)') AS g
       , ST_PointN(g,1) as p;
         st_astext
-----------------------
LINESTRING(0 0,1 1,0 0,3 3,4 4)
```

참고: ST_AddPoint, ST_NPoints, ST_NumPoints, ST_PointN, ST_RemovePoint

8.6.26. ST_SetSRID

ST_SetSRID — 도형에 적용된 SRID 를 특정 정수값으로 설정합니다.

Synopsis

geometry **ST_SetSRID**(geometry *geom*, integer *srid*);

설명: 도형에 적용된 SRID 를 특정 정수값으로 설정합니다. 쿼리를 위해 경계 상자를 작성하는 작업에 유용합니다.



이 함수는 도형 좌표를 어떤 식으로든 변환하지 않습니다. 해당 도형에 적용됐다고 가정되는 공간 참조 시스템을 정의하는 메타데이터를 설정할 뿐입니다. 도형을 새 투영법으로 변환하고자 하는 경우엔 ST_Transform 을 이용하십시오.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

예시

-- 포인트를 WGS84 경위도로 설정 --

```
SELECT ST_SetSRID(ST_Point(-123.365556, 48.428611),4326) As wgs84long_lat;
-- (ST_AsEWKT 으로 싸인) EWKT 표현식 -
SRID=4326;POINT(-123.365556 48.428611)
```

-- 포인트를 WGS84 경위도로 설정한 다음 웹 메르카토르(구체 메르카토르)로 변환 --

```
SELECT                          ST_Transform(ST_SetSRID(ST_Point(-123.365556,
48.428611),4326),3785) As spere_merc;
-- (ST_AsEWKT 으로 싸인) EWKT 표현식 --
SRID=3785;POINT(-13732990.8753491 6178458.96425423)
```

참고: Section 4.3.1, “SPATIAL_REF_SYS 테이블과 공간 참조 시스템”, ST_AsEWKT, ST_Point, ST_SRID, ST_Transform, UpdateGeometrySRID

8.6.27. ST_SnapToGrid

ST_SnapToGrid — 입력 도형의 모든 포인트를 정규 그리드로 스냅(snap)시킵니다.

Synopsis

geometry **ST_SnapToGrid**(geometry *geomA*, float *originX*, float *originY*, float *sizeX*, float *sizeY*);

geometry **ST_SnapToGrid**(geometry *geomA*, float *sizeX*, float *sizeY*);

geometry **ST_SnapToGrid**(geometry *geomA*, float *size*);

geometry **ST_SnapToGrid**(geometry *geomA*, geometry *pointOrigin*, float *sizeX*, float *sizeY*, float *sizeZ*, float *sizeM*);

설명: 응용방법 1, 2, 3: 입력 도형의 모든 포인트를 원점과 셀(cell) 크기로 정의된 그리드로 스냅(snap)시킵니다. 동일한 셀에 떨어지는 연속된 포인트들을 제거하며, 출력 포인트들이 주어진 유형의 도형을 정의하기에 부족할 경우 결국 NULL 을 반환합니다. 도형 집합 안에서 이렇게 붕괴된 도형은 집합에서 제외됩니다. 정확도를 낮추는 데 유용합니다.

응용방법 4: 1.1.0 버전에서 추가되었습니다. 입력 도형의 모든 포인트를 원점(두 번째 인수, 포인트여야 합니다) 및 셀 크기로 정의된 그리드로 스냅시킵니다. 사용자가 그리드로 스냅시키고 싶지 않은 차원의 경우, 셀 크기를 0 으로 설정하십시오.



반환되는 도형이 단순성을 잃을 수도 있습니다(ST_IsSimple 참조).



1.1.0 버전 배포 전에는 이 함수가 항상 2 차원 도형을 반환했습니다. 1.1.0 버전부터 반환되는 도형이, 더 높은 차원값은 건드리지 않은 채, 입력 도형과 동일한 차원수를 가지게 됐습니다. 모든 그리드 차원을 정의하려면 두 번째 도형 인자를 입력받는 버전을 이용하십시오.

1.0.0RC1 버전부터 사용할 수 있습니다.

1.1.0 버전부터 Z 및 M 을 지원합니다



이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
-- 사용자 도형을 10^-3 정확도 그리드로 스냅시킵니다.
UPDATE mytable
SET the_geom = ST_SnapToGrid(the_geom, 0.001);

SELECT ST_AsText(ST_SnapToGrid(
ST_GeomFromText('LINESTRING(1.1115678 2.123, 4.111111 3.2374897, 4.11112
3.23748667)'),
0.001)
);
st_astext
-----------------------------------
 LINESTRING(1.112 2.123,4.111 3.237)

-- 4 차원 도형을 스냅시킵니다.
SELECT ST_AsEWKT(ST_SnapToGrid(
ST_GeomFromEWKT('LINESTRING(-1.1115678 2.123 2.3456 1.11111,
4.111111 3.2374897 3.1234 1.1111, -1.11111112 2.123 2.3456 1.1111112)'),
 ST_GeomFromEWKT('POINT(1.12 2.22 3.2 4.4444)'),
 0.1, 0.1, 0.1, 0.01) );
st_asewkt
-----------------------------------------------------------------------
-----
 LINESTRING(-1.08  2.12  2.3  1.1144,4.12  3.22  3.1  1.1144,-1.08  2.12  2.3
1.1144)

-- 4 차원 도형에서 ST_SnapToGrid(geom,size)가 x, y 좌표만 건드리고 m 및 z 좌표는
그대로 유지합니다.
SELECT      ST_AsEWKT(ST_SnapToGrid(ST_GeomFromEWKT('LINESTRING(-1.1115678
2.123 3 2.3456,
4.111111 3.2374897 3.1234 1.1111)'),
0.01)     );
st_asewkt
-------------------------------------------------------------
 LINESTRING(-1.11 2.12 3 2.3456,4.11 3.24 3.1234 1.1111)
```

참고: ST_Snap, ST_AsEWKT, ST_AsText, ST_GeomFromText, ST_GeomFromEWKT, ST_Simplify

8.6.28. ST_Snap

ST_Snap — 입력 도형의 분절 구간과 꼭짓점을 참조 도형의 꼭짓점으로 스냅시킵니다.

Synopsis

geometry **ST_Snap**(geometry *input*, geometry *reference*, float *tolerance*);

설명: 한 도형의 꼭짓점과 분절 구간을 또다른 도형의 꼭짓점에 스냅시킵니다. 스냅 작업이 이루어지는 위치를 조절하기 위해 스냅 허용 거리(distance tolerance)를 이용합니다.

한 도형을 또다른 도형으로 스냅시키면, (노드 및 교차점 계산 과정에서 문제를 일으킬 수 있는) 거의 일치하는 경계선을 제거함으로써 오버레이 연산을 강력하게 향상시킬 수 있습니다.

스냅을 너무 많이 하면 유효하지 않은 지형을 생성하는 결과를 가져올 수 있기 때문에, 경험식(heuristics)을 통해 어떤 경우 안전하게 스냅시킬 수 있는지 알아내서 스냅시킬 꼭짓점의 개수 및 위치를 결정합니다. 하지만 이런 방식은 몇몇 가능할 수도 있는 스냅 작업을 제외하게 될 수도 있습니다.



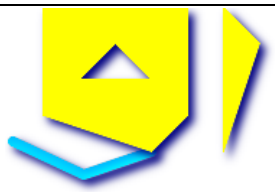
반환되는 도형이 단순성(ST_IsSimple 참조) 및 유효성(ST_IsValid 참조)을 잃을 수도 있습니다.

2.0.0 버전부터 사용할 수 있습니다. GEOS 3.3.0 이상 버전이 필요합니다.

예시



라인스트링과 함께 표출된 멀티폴리곤(스냅 작업 전)



허용 거리 1.01 을 기준으로 라인스트링에 스냅된 멀티폴리곤. 새로운 멀티폴리곤은 참조 라인스트링과 함께 표출됩니다.

```
SELECT          ST_AsText(ST_Snap(poly,line,
ST_Distance(poly,line)*1.01))             AS
polysnapped
FROM (SELECT
  ST_GeomFromText('MULTIPOLYGON(
    ((26 125, 26 200, 126 200, 126 125, 26
125 ),
    ( 51 150, 101 150, 76 175, 51 150 )),
    ((  151  100,  151  200,  176  175,  151
100 )))') As poly,
    ST_GeomFromText('LINESTRING (5 107, 54
84, 101 100)') As line

    ) As foo;

                polysnapped
-------------------------------------------
--------------------------
 MULTIPOLYGON(((26  125,26  200,126  200,126
125,101 100,26 125),
 (51  150,101  150,76  175,51  150)),((151
100,151 200,176 175,151 100)))
```



허용 거리 1.25 를 기준으로 라인스트링에 스냅된 멀티폴리곤. 새로운 멀티폴리곤은 참조 라인스트링과 함께 표출됩니다.

```
SELECT ST_AsText(
   ST_Snap(poly,line,
ST_Distance(poly,line)*1.25)
 ) AS polysnapped
FROM (SELECT
 ST_GeomFromText('MULTIPOLYGON(
   (( 26 125, 26 200, 126 200, 126 125, 26
125 ),
    ( 51 150, 101 150, 76 175, 51 150 )),
    ((  151  100,  151  200,  176  175,  151
100 )))') As poly,
    ST_GeomFromText('LINESTRING  (5  107,
54 84, 101 100)') As line

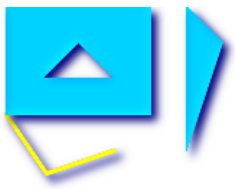
    ) As foo;

                polysnapped
-------------------------------------------
----------------------------
MULTIPOLYGON(((5  107,26  200,126  200,126
125,101 100,54 84,5 107),
(51  150,101  150,76  175,51  150)),((151
100,151 200,176 175,151 100)))
```



허용 거리 1.01 을 기준으로 원래 멀티폴리곤에 스냅된 라인스트링. 새로운 라인스트링은 참조 멀티폴리곤과 함께 표출됩니다.

```
SELECT ST_AsText(
  ST_Snap(line,                          poly,
ST_Distance(poly,line)*1.01)
 ) AS linesnapped
FROM (SELECT
 ST_GeomFromText('MULTIPOLYGON(
   ((26 125, 26 200, 126 200, 126 125, 26
125),
```



허용 거리 1.25 를 기준으로 원래 멀티폴리곤에 스냅된 라인스트링. 새로운 라인스트링은 참조 멀티폴리곤과 함께 표출됩니다.

```
SELECT ST_AsText(
 ST_Snap(line,                   poly,
ST_Distance(poly,line)*1.25)
  ) AS linesnapped
FROM (SELECT
  ST_GeomFromText('MULTIPOLYGON(
     (( 26 125, 26 200, 126 200, 126
125, 26 125 ),
```

```
    (51 150, 101 150, 76 175, 51 150 )),
    ((151 100, 151 200, 176 175, 151 100)))')
As poly,
    ST_GeomFromText('LINESTRING (5 107, 54
84, 101 100)') As line
    ) As foo;

        linesnapped
----------------------------------------
 LINESTRING(5 107,26 125,54 84,101 100)
```

```
     (51 150, 101 150, 76 175, 51
150 )),
     ((151 100, 151 200, 176 175,
151 100 )))') As poly,
      ST_GeomFromText('LINESTRING
(5 107, 54 84, 101 100)') As line
      ) As foo;
           linesnapped
----------------------------------
-----
LINESTRING(26 125,54 84,101 100)
```

참고: ST_SnapToGrid

8.6.29. ST_Transform

ST_Transform — 입력 도형의 좌표를 다른 공간 참조 시스템으로 변환시킨 새 도형을 반환합니다.

Synopsis

geometry **ST_Transform**(geometry *g1*, integer *srid*);

geometry **ST_Transform**(geometry *geom*, text *to_proj*);

geometry **ST_Transform**(geometry *geom*, text *from_proj*, text *to_proj*);

geometry **ST_Transform**(geometry *geom*, text *from_proj*, integer *to_srid*);

설명: 입력 도형의 좌표를 다른 공간 참조 시스템으로 변환시킨 새 도형을 반환합니다. 유효한 SRID 정수 파라미터가 (spatial_ref_sys 테이블에 존재해야만 합니다) 목표 공간 참조 시스템 to_srid 를 식별할 수도 있습니다. 다른 방법으로는, to_proj 그리고/또는 from_proj 에 대해 PROJ.4 문자열로 정의된 공간참조 시스템을 쓸 수도 있는데, 최적화된 방법은 아닙니다. 목표 공간 참조 시스템을 SRID 대신 PROJ.4 문자열로 표현할 경우, 산출 도형의 SRID 는 0 으로 설정될 것입니다. from_proj 파라미터를 가지고 있는 경우를 제외하면, 입력 도형은 정의된 SRID 를 가지고 있어야만 합니다.

ST_Transform 함수는 종종 ST_SetSRID()와 착각당합니다. ST_Transform 이 실제로 도형의 좌표를 한 공간 참조 시스템에서 또다른 공간 참조 시스템으로 변환시키는 반면, ST_SetSRID()는 도형의 SRID 식별자를 변경할 뿐입니다.



PostGIS 를 Proj 지원과 함께 컴파일해야 합니다. Proj 지원과 함께 컴파일됐는지 여부를 확인하려면 PostGIS_Full_Version 을 이용하십시오.



변환 작업을 한 번 이상 하는 경우, 인덱스 활용의 장점을 취하기 위해 흔히 쓰이는 변환 작업에 함수 인덱스를 설정하는 편이 좋습니다.



1.3.4 미만 버전에서 이 함수에 만곡 도형(curve)을 담고 있는 도형을 입력하면 충돌이 일어났습니다. 1.3.4 버전부터 이 버그가 해결됐습니다.

개선 사항: 2.0.0 버전부터 다면체 표면(polyhedral surface)을 지원합니다.

개선 사항: 2.3.0 버전부터 PROJ.4 텍스트를 직접 지원하기 시작했습니다.

✔ 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.6

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

예시

매사추세츠 주의 미국 피트 단위 평면 상에 있는 도형을 WGS84 경위도로 변경하십시오.

```
SELECT            ST_AsText(ST_Transform(ST_GeomFromText('POLYGON((743238
2967416,743238 2967450,
      743265 2967450,743265.625 2967416,743238 2967416))',2249),4326)) As
wgs_geom;

 wgs_geom
---------------------------
 POLYGON((-71.1776848522251             42.3902896512902,-71.1776843766326
42.3903829478009,
-71.1775844305465  42.3903826677917,-71.1775825927231  42.3902893647987,-
71.177684
8522251 42.3902896512902));
(1 row)

-- 3D 원형 스트링 예시
SELECT
ST_AsEWKT(ST_Transform(ST_GeomFromEWKT('SRID=2249;CIRCULARSTRING(743238
2967416 1,743238 2967450 2,743265 2967450 3,743265.625 2967416 3,743238
2967416 4)'),4326));

                           st_asewkt
-----------------------------------------------------------------------
-------------
 SRID=4326;CIRCULARSTRING(-71.1776848522251      42.3902896512902      1,-
71.1776843766326 42.3903829478009 2,
 -71.1775844305465 42.3903826677917 3,
 -71.1775825927231  42.3902893647987  3,-71.1776848522251  42.3902896512902
4)
```

부분 함수의 인덱스를 생성하는 예시입니다. 사용자가 모든 도형이 어디에 채워질지 확신하지 못 하는 테이블의 경우, 최선의 방법은 NULL 도형을 제외하는 부분 인덱스를 이용하는 것입니다. 이렇게 하면 공간도 절약하고 사용자의 인덱스를 더 작고 효율적으로 만들 수 있습니다.

```
CREATE INDEX idx_the_geom_26986_parcels
  ON parcels
  USING gist
  (ST_Transform(the_geom, 26986))
  WHERE the_geom IS NOT NULL;
```

예시 - 사용자 지정 공간 참조 시스템으로 변환하는 데 PROJ.4 텍스트를 이용

```
-- 북극 가까이 있는, 구심 투영(gnomonic projection)을 이용하는 폴리곤 2 개가 교차하는
지점을 찾습니다.
-- http://boundlessgeo.com/2012/02/flattening-the-peel/ 참조
 WITH data AS (
   SELECT
     ST_GeomFromText('POLYGON((170  50,170  72,-130  72,-130  50,170  50))',
4326) AS p1,
     ST_GeomFromText('POLYGON((-170 68,-170 90,-141 90,-141 68,-170 68))',
4326) AS p2,
     '+proj=gnom +ellps=WGS84 +lat_0=70 +lon_0=-160 +no_defs'::text AS gnom
 )
 SELECT ST_AsText(
   ST_Transform(
     ST_Intersection(ST_Transform(p1, gnom), ST_Transform(p2, gnom)),
   gnom, 4326))
 FROM data;
                                  st_astext
 ------------------------------------------------------------------------
--------
  POLYGON((-170 74.053793645338,-141 73.4268621378904,-141 68,-170 68,-170
74.053793645338))
```

변환 습성 설정

때때로 그리드 이동(grid-shift)과 관련된 좌표 변환이 실패할 수도 있습니다. 예를 들어 PROJ.4 가 그리드 이동 파일과 함께 빌드되지 않았거나, 좌표가 그리드 이동이 정의된 범위(range) 내부에 떨어지지 않는 경우입니다. 그리드 이동 파일이 없을 경우 PostGIS 는 기본적으로 오류를 내지만, PROJ.4 텍스트의 서로 다른 to_proj 값들을 테스트하거나 또는 spatial_ref_sys 테이블 내부의 proj4text 값을 수정해서 각 SRID 마다 이 습성을 다르게 설정할 수 있습니다.

예를 들어, proj4text 파라미터 "+datum=NAD87"은 다음 "+nadgrids" 파라미터를 간략화한 것입니다.

```
+nadgrids=@conus,@alaska,@ntv2_0.gsb,@ntv1_can.dat
```

접두사 "@"는 파일이 없는 경우에도 오류를 보고하지 말라는 뜻이지만, 제대로 된 파일 없이 목록의 마지막까지 가는 경우(예를 들어 찾아서 겹쳐졌을 경우)에는 오류를 보고합니다.

만약 반대로 사용자가 적어도 표준 파일이 존재하는 것을 확인하려 하지만 모든 파일을 스캔했는데도 발견하지 못 한 경우, 다음 NULL 변환을 적용할 수 있습니다:

```
+nadgrids=@conus,@alaska,@ntv2_0.gsb,@ntv1_can.dat,null
```

NULL 그리드 이동 파일은 전세계를 커버하며 어떤 이동도 시키지 않는 유효한 그리드 이동 파일입니다. 따라서 예시를 완전하게 하기 위해, 사용자가 PostGIS 를 수정해서 SRID 4267 로 변환시 정확한 범위 안에 떨어지지 않더라도 오류를 보고하지 않게 하려면, 다음과 같이 하면 됩니다:

```
UPDATE spatial_ref_sys SET proj4text = '+proj=longlat +ellps=clrk66
+nadgrids=@conus,@alaska,@ntv2_0.gsb,@ntv1_can.dat,null +no_defs' WHERE
srid = 4267;
```

참고: PostGIS_Full_Version, ST_AsText, ST_SetSRID, UpdateGeometrySRID

8.6.30. ST_Translate

ST_Translate — 주어진 오프셋으로 도형을 변형시킵니다.

Synopsis

geometry **ST_Translate**(geometry *g1*, float *deltax*, float *deltay*);

geometry **ST_Translate**(geometry *g1*, float *deltax*, float *deltay*, float *deltaz*);

설명: 좌표가 Δx, Δy, Δz 단위로 변형된 새로운 도형을 반환합니다. 이 단위는 해당 도형의 공간 참조(SRID)가 정의하는 단위를 기반으로 합니다.



1.3.4 미만 버전에서 이 함수에 만곡 도형(curve)을 담고 있는 도형을 입력하면 충돌이 일어났습니다. 1.3.4 버전부터 이 버그가 해결됐습니다.

1.2.2 버전부터 사용할 수 있습니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

예시

포인트를 경도 1 도 이동합니다.

```
SELECT                ST_AsText(ST_Translate(ST_GeomFromText('POINT(-71.01
42.37)',4326),1,0)) As wgs_transgeomtxt;

       wgs_transgeomtxt
       ---------------------
       POINT(-70.01 42.37)
```

라인스트링을 경도 1 도, 위도 1/2 도 이동합니다.

```
SELECT  ST_AsText(ST_Translate(ST_GeomFromText('LINESTRING(-71.01  42.37,-
71.11 42.38)',4326),1,0.5)) As wgs_transgeomtxt;
                wgs_transgeomtxt
       ---------------------------------------
       LINESTRING(-70.01 42.87,-70.11 42.88)
```

3 차원 포인트를 이동합니다.

```
SELECT ST_AsEWKT(ST_Translate(CAST('POINT(0 0 0)' As geometry), 5, 12,3));
       st_asewkt
       ---------
       POINT(5 12 3)
```

커브와 포인트를 이동합니다.

```
SELECT    ST_AsText(ST_Translate(ST_Collect('CURVEPOLYGON(CIRCULARSTRING(4
3,3.12 0.878,1 0,-1.121 5.1213,6 7, 8 9,4 3))','POINT(1 3)'),1,2));
```

```
st_astext
------------------------------------------------------------------------
-----------------------------------
 GEOMETRYCOLLECTION(CURVEPOLYGON(CIRCULARSTRING(5 5,4.12 2.878,2 2,-0.121
7.1213,7 9,9 11,5 5)),POINT(2 5))
```

참고: ST_Affine, ST_AsText, ST_GeomFromText

8.6.31. ST_TransScale

ST_TransScale — 주어진 인수 및 오프셋으로 도형을 변형시킵니다.

Synopsis

geometry **ST_TransScale**(geometry *geomA*, float *deltaX*, float *deltaY*, float *XFactor*, float *YFactor*);

설명: ⊿x 와 ⊿y 독립 변수(arg)를 이용해서 도형을 변형한 다음, XFactor, YFactor 독립 변수를 이용해 크기를 조정합니다. 2 차원에서만 작동합니다.



ST_TransScale(geomA, deltaX, deltaY, XFactor, YFactor) 는 ST_Affine(geomA, XFactor, 0, 0, 0, YFactor, 0, 0, 0, 1, deltaX*XFactor, deltaY*YFactor, 0) 을 간략화한 것입니다.



1.3.4 미만 버전에서 이 함수에 만곡 도형(curve)을 담고 있는 도형을 입력하면 충돌이 일어났습니다. 1.3.4 버전부터 이 버그가 해결됐습니다.

1.1.0 버전부터 사용할 수 있습니다.

✓ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✓ 이 방법은 원형 스트링 및 곡선을 지원합니다.

예시

```
SELECT ST_AsEWKT(ST_TransScale(ST_GeomFromEWKT('LINESTRING(1 2 3, 1 1 1)'),
0.5, 1, 1, 2));
              st_asewkt
-----------------------------
 LINESTRING(1.5 6 3,1.5 4 1)


-- 포인트에 버퍼를 부여해서 원에 가깝게 만들고, 커브로 변환한 다음 1, 2 값으로 번역하고 3,
4 값으로 크기를 조정합니다.
  SELECT        ST_AsText(ST_Transscale(ST_LineToCurve(ST_Buffer('POINT(234
567)', 3)),1,2,3,4));

st_astext
------------------------------------------------------------------------
-------------------------------------------------
```

```
 CURVEPOLYGON(CIRCULARSTRING(714                    2276,711.363961030679
2267.51471862576,705 2264,698.636038969321 2284.48528137424,714 2276))
```

참고: ST_Affine, ST_Translate

8.7. 도형 출력물(output)

ST_AsBinary — 도형/지리형의 WKB(Well-Known Binary) 표현식을 SRID 메타데이터 없이 반환합니다.

ST_AsEncodedPolyline — 라인스트링 도형으로부터 인코딩된 폴리라인을 반환합니다.

ST_AsEWKB — 도형/지리형의 WKB(Well-Known Binary) 표현식을 SRID 메타데이터와 함께 반환합니다.

ST_AsEWKT — 도형의 WKT(Well-Known Text) 표현식을 SRID 메타데이터와 함께 반환합니다.

ST_AsGeoJSON — 도형을 GeoJSON 요소로 반환합니다.

ST_AsGML — 도형을 GML 2 또는 GML 3 버전 요소로 반환합니다.

ST_AsHEXEWKB — 도형을 소 엔디안(NDR) 또는 대 엔디안(XDR) 인코딩을 통해 HEXEWKB (텍스트) 형식으로 반환합니다.

ST_AsKML — 도형을 KML 요소로 반환합니다. 응용방법이 몇 개 있습니다. 기본값은 버전 2, 정밀도 15 입니다.

ST_AsLatLonText — 주어진 포인트의 도, 분, 초 표현식을 반환합니다.

ST_AsSVG — 도형을 주어진 도형 또는 지리형의 SVG 경로 데이터로 반환합니다.

ST_AsText — 도형/지리형의 WKT(Well-Known Text) 표현식을 SRID 메타데이터 없이 반환합니다.

ST_AsTWKB — 도형을 TWKB(Tiny Well-Known Binary)로 반환합니다.

ST_AsX3D — 도형을 X3D XML 노드 요소 형식: ISO-IEC-19776-1.2-X3DEncodings-XML 로 반환합니다.

ST_GeoHash — 도형의 GeoHash 표현식을 반환합니다.

8.7.1. ST_AsBinary

ST_AsBinary — 도형/지리형의 WKB(Well-Known Binary) 표현식을 SRID 메타데이터 없이 반환합니다.

Synopsis

bytea **ST_AsBinary**(geometry *g1*);

bytea **ST_AsBinary**(geometry *g1*, text *NDR_or_XDR*);

bytea **ST_AsBinary**(geography *g1*);

bytea **ST_AsBinary**(geography *g1*, text *NDR_or_XDR*);

설명: 도형의 WKB 표현식을 반환합니다. 이 함수에는 두 가지 응용방법이 있습니다. 첫 번째 응용방법은 엔디안(endian) 인코딩 파라미터를 입력받지 않으며, 서버 머신 엔디안이 기본값인 함수입니다. 두 번째 응용방법은 소 엔디안(little-endian, 'NDR') 또는 대 엔디안(big-endian, 'XDR')을 이용해서 인코딩을 나타내는 두 번째 인수를 입력받는 함수입니다.

이 함수는 데이터베이스로부터 데이터를 문자열 표현식으로 변환하지 않고 추출하는 바이너리 커서에 유용합니다.



WKB 사양은 SRID 를 포함하지 않습니다. SRID 를 포함한 WKB 형식을 반환받으려면 ST_AsEWKB 를 이용하십시오.



ST_AsBinary 는 도형에 대한 ST_GeomFromWKB 의 역함수입니다. ST_AsBinary 표현식을 PostGIS 도형으로 변환하려면 ST_GeomFromWKB 함수를 이용하십시오.



PostgreSQL 9.0 버전에서 헥스(hex) 인코딩으로 "bytea" 데이터형을 출력하도록 기본 습성이 변경됐습니다. ST_AsBinary 는 도형에 대한 ST_GeomFromWKB 의 역함수입니다. 사용자의 GUI 도구가 예전 습성을 요구하는 경우, 사용자 데이터베이스 단계에서 SET bytea_output='escape' 라고 설정하십시오.

개선 사항: 2.0.0 버전부터 다면체 표면, 삼각형 및 TIN 을 지원하기 시작했습니다.

개선 사항: 2.0.0 버전부터 더 높은 좌표 차원을 지원합니다.

개선 사항: 2.0.0 버전부터 지리형과 함께 엔디안을 설정하는 방식을 지원합니다.

1.5.0 버전부터 지리형을 지원합니다.

변경 사항: 2.0.0 버전부터 이 함수에 알려지지 않은 유형을 입력할 수 없게 됐습니다. 반드시 도형을 입력해야 합니다. ST_AsBinary('POINT(1 2)') 같은 구조는 더 이상 유효하지 않아, n st_asbinary(unknown) is not unique error 오류가 발생합니다. 이런 코드는 ST_AsBinary('POINT(1 2)'::geometry); 로 변경돼야 합니다. 이렇게 변경할 수 없는 경우, legacy.sql 을 설치하십시오.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다. s2.1.1.1

✔ 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.37

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
SELECT ST_AsBinary(ST_GeomFromText('POLYGON((0 0,0 1,1 1,1 0,0 0))',4326));
```

```
               st_asbinary
--------------------------------
\001\003\000\000\000\001\000\000\000\005
\000\000\000\000\000\000\000\000\000\000
\000\000\000\000\000\000\000\000\000\000
\000\000\000\000\000\000\000\000\000\000
\000\000\000\360?\000\000\000\000\000\000
\360?\000\000\000\000\000\000\360?\000\000
\000\000\000\000\360?\000\000\000\000\000
\000\000\000\000\000\000\000\000\000\000\000
\000\000\000\000\000\000\000\000\000\000
(1 row)
SELECT ST_AsBinary(ST_GeomFromText('POLYGON((0 0,0 1,1 1,1 0,0 0))',4326),
'XDR');
               st_asbinary
--------------------------------
\000\000\000\000\003\000\000\000\001\000\000\000\005\000\000\000\000\000
\000\000\000\000\000\000\000\000\000\000\000\000\000\000\000\000\000\000
\000?\360\000\000\000\000\000\000?\360\000\000\000\000\000\000?\360\000\000
\000\000\000\000?\360\000\000\000\000\000\000\000\000\000\000\000\000\000\0
00
\000\000\000\000\000\000\000\000\000\000\000\000\000\000\000\000
(1 row)
```

참고: ST_GeomFromWKB, ST_AsEWKB, ST_AsTWKB, ST_AsText

8.7.2. ST_AsEncodedPolyline

ST_AsEncodedPolyline — 라인스트링 도형으로부터 인코딩된 폴리라인을 반환합니다.

Synopsis

text **ST_AsEncodedPolyline**(geometry *geom*, integer *precision=5*);

설명: 도형을 인코딩된 폴리라인으로 반환합니다. 사용자가 구글 지도를 사용하는 경우 매우 유용한 형식입니다.

2.2.0 버전부터 사용할 수 있습니다.

예시

기본

```
SELECT      ST_AsEncodedPolyline(GeomFromEWKT('SRID=4326;LINESTRING(-120.2
38.5,-120.95 40.7,-126.453 43.252)'));
       -- 결과 --
       |_p~iF~ps|U_ulLnnqC_mqNvxq`@
```

지리형 라인스트링 및 지리형 분절화(segmentize)와 결합해서 사용하고, 구글 지도에 올립니다.

```
-- 보스턴에서 샌프란시스코까지 100km 마다 분절하는 SQL 구문
       SELECT ST_AsEncodedPolyline(
              ST_Segmentize(
```

```
                    ST_GeogFromText('LINESTRING(-71.0519  42.4935,-122.4483
37.64)'),
                          100000)::geometry) As encodedFlightPath;
```

사용자가 $ 변수를 쿼리 결과로 대체한 자바스크립트는 다음과 같이 보일 것입니다.

```
<script                                                    type="text/javascript"
src="http://maps.googleapis.com/maps/api/js?libraries=geometry"
></script>
<script type="text/javascript">
       flightPath = new google.maps.Polyline({
                    path:
google.maps.geometry.encoding.decodePath("$encodedFlightPath"),
                    map: map,
                    strokeColor: '#0000CC',
                    strokeOpacity: 1.0,
                    strokeWeight: 4
             });
</script>
```

참고: ST_LineFromEncodedPolyline, ST_Segmentize

8.7.3. ST_AsEWKB

ST_AsEWKB — 도형/지리형의 WKB(Well-Known Binary) 표현식을 SRID 메타데이터와 함께 반환합니다.

Synopsis

bytea **ST_AsEWKB**(geometry *g1*);

bytea **ST_AsEWKB**(geometry *g1*, text *NDR_or_XDR*);

설명: 도형의 WKB(Well-Known Binary) 표현식을 SRID 메타데이터와 함께 반환합니다. 이 함수에는 두 가지 응용방법이 있습니다. 첫 번째 응용방법은 엔디안(endian) 인코딩 파라미터를 입력받지 않으며, 소 엔디안이 기본값인 함수입니다. 두 번째 응용방법은 소 엔디안(little-endian, 'NDR') 또는 대 엔디안(big-endian, 'XDR')을 이용해서 인코딩을 나타내는 두 번째 인수를 입력받는 함수입니다.

이 함수는 데이터베이스로부터 데이터를 문자열 표현식으로 변환하지 않고 추출하는 바이너리 커서에 유용합니다.



WKB 사양은 SRID 를 포함하지 않습니다. OGC WKB 형식을 반환받으려면 ST_AsBinary 를 이용하십시오.



ST_AsEWKB 는 ST_GeomFromEWKB 의 역함수입니다. ST_AsEWKB 표현식을 PostGIS 도형으로 변환하려면 ST_GeomFromEWKB 함수를 이용하십시오.

개선 사항: 2.0.0 버전부터 다면체 표면, 삼각형 및 TIN 을 지원하기 시작했습니다.



이 방법은 원형 스트링 및 곡선을 지원합니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
SELECT ST_AsEWKB(ST_GeomFromText('POLYGON((0 0,0 1,1 1,1 0,0 0))',4326));

                st_asewkb
--------------------------------
\001\003\000\000 \346\020\000\000\001\000
\000\000\005\000\000\000\000
\000\000\000\000\000\000\000\000
\000\000\000\000\000\000\000\000\000
\000\000\000\000\000\000\000\000\000\000
\000\000\360?\000\000\000\000\000\000\360?
\000\000\000\000\000\000\360?\000\000\000\000\000
\000\360?\000\000\000\000\000\000\000\000\000\000\000
\000\000\000\000\000\000\000\000\000\000\000\000\000
(1 row)
SELECT  ST_AsEWKB(ST_GeomFromText('POLYGON((0  0,0  1,1  1,1  0,0  0))',4326),
'XDR');
                st_asewkb
--------------------------------
\000
\000\000\003\000\000\020\346\000\000\000\001\000\000\000\005\000\000\000\00
0\
000\000\000\000\000\000\000\000\000\000\000\000\000\000\000\000\000\000\000
\000?
\360\000\000\000\000\000\000?\360\000\000\000\000\000\000?\360\000\000\000\
000
\000\000?\360\000\000\000\000\000\000\000\000\000\000\000\000\000\000\000\0
00\000
\000\000\000\000\000\000\000\000\000\000\000\000\000\000
```

참고: ST_AsBinary, ST_AsEWKT, ST_AsText, ST_GeomFromEWKT, ST_SRID

8.7.4. ST_AsEWKT

ST_AsEWKT — 도형의 WKT(Well-Known Text) 표현식을 SRID 메타데이터와 함께 반환합니다.

Synopsis

text **ST_AsEWKT**(geometry *g1*);

text **ST_AsEWKT**(geography *g1*);

설명: 앞에 SRID 가 붙은 도형의 WKT 표현식을 반환합니다.



WKT 사양은 SRID 를 포함하지 않습니다. OGC WKT 형식을 반환받으려면 ST_AsText 를 이용하십시오.

Note! WKT 형식은 정밀도를 유지하지 않으므로, 부동소수점 절단(floating truncation)을 피하려면 이동 작업(transport)에 ST_AsBinary 또는 ST_AsEWKB 형식을 이용하십시오.

Note! ST_AsEWKT 는 ST_GeomFromEWKT 의 역함수입니다. ST_AsEWKT 표현식을 PostGIS 도형으로 변환하려면 ST_GeomFromEWKT 함수를 이용하십시오.

개선 사항: 2.0.0 버전부터 지리형, 다면체 표면, 삼각형 및 TIN 을 지원하기 시작했습니다.

✓ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✓ 이 함수는 다면체 표면을 지원합니다.

✓ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

✓ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
SELECT ST_AsEWKT('0103000020E61000000100000005000000000000
                  0000000000000000000000000000000000000000000000000000
                  F03F000000000000F03F000000000000F03F000000000000F03

F000000000000000000000000000000000000000000000000'::geometry);

               st_asewkt
---------------------------------
SRID=4326;POLYGON((0 0,0 1,1 1,1 0,0 0))
(1 row)

SELECT
ST_AsEWKT('01080000800300000000000000060E30A4100000000785C0241000000000000F0
3F0000000018
E20A4100000000485F024100000000000004000000000018
E20A4100000000305C0241000000000000840')

--st_asewkt---
CIRCULARSTRING(220268 150415 1,220227 150505 2,220227 150406 3)
```

참고: ST_AsBinary, ST_AsEWKB, ST_AsText, ST_GeomFromEWKT

8.7.5. ST_AsGeoJSON

ST_AsGeoJSON — 도형을 GeoJSON 요소로 반환합니다.

Synopsis

text **ST_AsGeoJSON**(geometry *geom*, integer *maxdecimaldigits=15*, integer *options=0*);

text **ST_AsGeoJSON**(geography *geog*, integer *maxdecimaldigits=15*, integer *options=0*);

text **ST_AsGeoJSON**(integer *gj_version*, geometry *geom*, integer *maxdecimaldigits=15*, integer *options=0*);

text **ST_AsGeoJSON**(integer *gj_version*, geography *geog*, integer *maxdecimaldigits=15*, integer *options=0*);

설명: 도형을 GeoJSON(Geometry Javascript Object Notation) 요소로 반환합니다(GeoJSON 사양서 1.0 참조). 2 차원 및 3 차원 도형 둘 다 지원합니다. GeoJSON 은 SFS 1.1 도형 유형만 지원합니다(예를 들어 커브는 지원하지 않습니다).

"gj_version" 파라미터는 GeoJSON 사양의 주요 버전을 의미합니다. 이 파라미터를 설정할 경우, 1 로 설정해야 합니다. 이 숫자는 GeoJSON 의 사양 버전을 나타냅니다.

출력물에 쓰이는 소수점 이하 자릿수(decimal place)의 최대값을 줄이는 데 세 번째 인수를 사용할 수도 있습니다(기본값은 15 입니다).

GeoJSON 출력물에 Bbox 또는 Crs 를 추가하는 데 마지막 '옵션' 인수를 사용할 수 있습니다:

- 0: 옵션이 존재하지 않습니다(기본값).
- 1: GeoJSON Bbox
- 2: GeoJSON Short CRS (예: EPSG:4326)
- 4: GeoJSON Long CRS (예: urn:ogc:def:crs:EPSG::4326)

Version 1: ST_AsGeoJSON(geom) / precision=15 version=1 options=0

Version 2: ST_AsGeoJSON(geom, precision) / version=1 options=0

Version 3: ST_AsGeoJSON(geom, precision, options) / version=1

Version 4: ST_AsGeoJSON(gj_version, geom) / precision=15 options=0

Version 5: ST_AsGeoJSON(gj_version, geom, precision) /options=0

Version 6: ST_AsGeoJSON(gj_version, geom, precision,options)

1.3.4 버전부터 사용할 수 있습니다.

1.5.0 버전부터 지리형을 지원합니다.

변경 사항: 2.0.0 버전부터 기본 독립 변수(default arg) 및 명명된 독립 변수(named arg)를 지원합니다.

✓ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

AJAX 매핑 작업에 사용되는 다른 형식들보다 일반적으로 GeoJSON 형식이 더 효율적입니다. AJAX 매핑 작업을 지원하는 유명한 자바스크립트 클라이언트로 OpenLayers 가 있습니다. OpenLayers GeoJSON Example 에서 그 용례를 찾아볼 수 있습니다.

```
SELECT ST_AsGeoJSON(the_geom) from fe_edges limit 1;
                                st_asgeojson
------------------------------------------------------------------------
---------------------------------

{"type":"MultiLineString","coordinates":[[[-
89.734634999999997,31.492072000000000],
[-89.734955999999997,31.492237999999997]]]}
```

```
(1 row)
-- 3D 포인트
SELECT ST_AsGeoJSON('LINESTRING(1 2 3, 4 5 6)');

st_asgeojson
----------------------------------------------------------------------------
----------------
 {"type":"LineString","coordinates":[[1,2,3],[4,5,6]]}
```

8.7.6. ST_AsGML

ST_AsGML — 도형을 GML 2 또는 GML 3 버전 요소로 반환합니다.

Synopsis

text **ST_AsGML**(geometry *geom*, integer *maxdecimaldigits=15*, integer *options=0*);

text **ST_AsGML**(geography *geog*, integer *maxdecimaldigits=15*, integer *options=0*);

text **ST_AsGML**(integer *version*, geometry *geom*, integer *maxdecimaldigits=15*, integer *options=0*, text *nprefix=null*, text *id=null*);

text **ST_AsGML**(integer *version*, geography *geog*, integer *maxdecimaldigits=15*, integer *options=0*, text *nprefix=null*, text *id=null*);

설명: 도형을 GML(Geography Markup Language) 요소로 반환합니다. 버전 파라미터에는, 설정할 경우, 2 또는 3 을 쓸 수 있습니다. 버전 파라미터를 설정하지 않으면 기본값인 2 로 가정합니다. 출력물에 쓰이는 소수점 이라 자릿수의 최대값(maxdecimaldigits)을 줄이는 데 정밀도 인수를 사용할 수도 있습니다(기본값은 15 입니다).

GML 2 는 2.1.2 버전을, GML 3 은 3.1.1 버전을 가리킵니다.

'옵션' 인수는 비트필드(bitfield) 구조체입니다. CRS 출력 유형을 GML 출력으로 정의하고, 데이터를 위도/경도로 선언하는 데 쓸 수 있습니다.

- 0: GML Short CRS (예: EPSG:4326), 기본값
- 1: GML Long CRS (예: urn:ogc:def:crs:EPSG::4326)
- 2: GML 3 에 한해, 출력물에서 srsDimension 속성을 제거합니다.
- 4: GML 3 에 한해, 라인을 위해 <Curve> 보다 <LineString> 태그를 사용합니다.
- 16: 데이터가 위도/경도(예: srid=4326)라고 선언합니다. 기본적으로는 데이터가 평면 좌표라고 가정합니다. 이 옵션은 축의 순서(axis order)와 관련돼 있어, GML 3.1.1 출력물에 대해서만 쓸모가 있습니다. 따라서 이 옵션을 설정하면, 데이터베이스의 경도 위도 대신 위도 경도로 좌표의 순서를 바꿀 것입니다.
- 32: 도형을 둘러싼 상자(envelope)를 출력합니다.

사용자 지정 네임스페이스 접두사를 설정하거나 접두사를 사용하지 않도록 설정(비어 있는 경우)하는 데 '네임스페이스 접두사' 인수를 사용할 수도 있습니다. 이 인수가 NULL 이거나 생략된 경우 'gml' 접두사를 씁니다.

1.3.2 버전부터 사용할 수 있습니다.

1.5.0 버전부터 지리형을 지원합니다.

개선 사항: 2.0.0 버전부터 접두사를 지원합니다. 라인에 대해 커브 대신 라인스트링 태그를 이용할 수 있도록 GML 3 용 옵션 '4'가 등장했습니다. GML 3 가 다면체 표면 및 TIN 을 지원하기 시작했습니다. 상자를 출력하는 옵션 '32'도 새롭게 등장했습니다.

변경 사항: 2.0.0 버전부터 명명된 독립변수(named arg)를 기본값으로 씁니다.

개선 사항: 2.1.0 버전부터 GML 3 를 위해 ID 를 지원하기 시작했습니다.



ST_AsGML 함수의 3 이상 버전만이 다면체 표면 및 TIN 을 지원합니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시: 버전 2

```
SELECT ST_AsGML(ST_GeomFromText('POLYGON((0 0,0 1,1 1,1 0,0 0))',4326));
             st_asgml
             --------
             <gml:Polygon srsName="EPSG:4326"
><gml:outerBoundaryIs
><gml:LinearRing
><gml:coordinates
>0,0 0,1 1,1 1,0 0,0</gml:coordinates
></gml:LinearRing
></gml:outerBoundaryIs
></gml:Polygon
>
```

예시: 버전 3

```
-- 좌표의 순서를 뒤집고 확장 EPSG 를 출력합니다. (16 | 1) --
SELECT          ST_AsGML(3,          ST_GeomFromText('POINT(5.234234233242
6.34534534534)',4326), 5, 17);
                   st_asgml
                   --------
             <gml:Point srsName="urn:ogc:def:crs:EPSG::4326"
><gml:pos
>6.34535 5.23423</gml:pos
></gml:Point
>

-- 상자(envelope)를 출력합니다. (32) --
SELECT ST_AsGML(3, ST_GeomFromText('LINESTRING(1 2, 3 4, 10 20)',4326), 5,
32);
             st_asgml
             --------
      <gml:Envelope srsName="EPSG:4326">
             <gml:lowerCorner
```

```
>1 2</gml:lowerCorner>
            <gml:upperCorner
>10 20</gml:upperCorner>
      </gml:Envelope
>

-- 상자(envelope)를 출력하고 (32), 좌표를 뒤집고(경도 위도 대신 위도 경도) (16), long
SRS를 사용 (1) = 32 | 16 | 1 = 49 --
SELECT ST_AsGML(3, ST_GeomFromText('LINESTRING(1 2, 3 4, 10 20)',4326), 5,
49);
      st_asgml
      --------
<gml:Envelope srsName="urn:ogc:def:crs:EPSG::4326">
      <gml:lowerCorner
>2 1</gml:lowerCorner>
      <gml:upperCorner
>20 10</gml:upperCorner>
</gml:Envelope
>

-- 다면체 예시 --
SELECT ST_AsGML(3, ST_GeomFromEWKT('POLYHEDRALSURFACE( ((0 0 0, 0 0 1, 0 1
1, 0 1 0, 0 0 0)),
((0 0 0, 0 1 0, 1 1 0, 1 0 0, 0 0 0)), ((0 0 0, 1 0 0, 1 0 1, 0 0 1, 0 0
0)),
((1 1 0, 1 1 1, 1 0 1, 1 0 0, 1 1 0)),
((0 1 0, 0 1 1, 1 1 1, 1 1 0, 0 1 0)), ((0 0 1, 1 0 1, 1 1 1, 0 1 1, 0 0
1)) )'));
      st_asgml
      --------
 <gml:PolyhedralSurface>
<gml:polygonPatches>
  <gml:PolygonPatch>
            <gml:exterior>
                  <gml:LinearRing>
                        <gml:posList srsDimension="3"
>0 0 0 0 0 1 0 1 1 0 1 0 0 0 0</gml:posList>
                  </gml:LinearRing>
            </gml:exterior>
  </gml:PolygonPatch>
  <gml:PolygonPatch>
            <gml:exterior>
                  <gml:LinearRing>
                        <gml:posList srsDimension="3"
>0 0 0 0 1 0 1 1 0 1 0 0 0 0 0</gml:posList>
                  </gml:LinearRing>
            </gml:exterior>
  </gml:PolygonPatch>
  <gml:PolygonPatch>
            <gml:exterior>
                  <gml:LinearRing>
                        <gml:posList srsDimension="3"
>0 0 0 1 0 0 1 0 1 0 0 1 0 0 0</gml:posList>
                  </gml:LinearRing>
            </gml:exterior>
  </gml:PolygonPatch>
  <gml:PolygonPatch>
            <gml:exterior>
                  <gml:LinearRing>
                        <gml:posList srsDimension="3"
>1 1 0 1 1 1 1 0 1 1 0 0 1 1 0</gml:posList>
                  </gml:LinearRing>
```

```
            </gml:exterior>
   </gml:PolygonPatch>
   <gml:PolygonPatch>
            <gml:exterior>
                    <gml:LinearRing>
                            <gml:posList srsDimension="3"
>0 1 0 0 1 1 1 1 1 1 0 0 1 0</gml:posList>
                    </gml:LinearRing>
            </gml:exterior>
   </gml:PolygonPatch>
   <gml:PolygonPatch>
            <gml:exterior>
                    <gml:LinearRing>
                            <gml:posList srsDimension="3"
>0 0 1 1 0 1 1 1 1 0 1 1 0 0 1</gml:posList>
                    </gml:LinearRing>
            </gml:exterior>
   </gml:PolygonPatch>
</gml:polygonPatches>
</gml:PolyhedralSurface
>
```

참고: ST_GeomFromGML

8.7.7. ST_AsHEXEWKB

ST_AsHEXEWKB — 도형을 소 엔디안(NDR) 또는 대 엔디안(XDR) 인코딩을 통해 HEXEWKB (텍스트) 형식으로 반환합니다.

Synopsis

text **ST_AsHEXEWKB**(geometry *g1*, text *NDRorXDR*);

text **ST_AsHEXEWKB**(geometry *g1*);

설명: 도형을 소 엔디안(NDR) 또는 대 엔디안(XDR) 인코딩을 통해 HEXEWKB (텍스트) 형식으로 반환합니다. 인코딩을 설정하지 않으면 NDR 을 씁니다.



1.2.2 버전부터 사용할 수 있습니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

예시

```
SELECT    ST_AsHEXEWKB(ST_GeomFromText('POLYGON((0    0,0    1,1    1,1    0,0
0))',4326));
-- 이 함수는 다음 함수와 동일한 답을 출력합니다.

            SELECT    ST_GeomFromText('POLYGON((0    0,0    1,1    1,1    0,0
0))',4326)::text;
```

```
st_ashexewkb
--------
0103000020E6100000010000000500
0000000000000000000000000000000000
0000000000000000000000000000000F03F
000000000000F03F000000000000F03F000000000000F03
F0000000000000000000000000000000000000000000000000
```

8.7.8. ST_AsKML

ST_AsKML — 도형을 KML 요소로 반환합니다. 변형하는 방법이 몇 가지 있습니다. 기본값은 버전 2, 정밀도 15 입니다.

Synopsis

text **ST_AsKML**(geometry *geom*, integer *maxdecimaldigits=15*);

text **ST_AsKML**(geography *geog*, integer *maxdecimaldigits=15*);

text **ST_AsKML**(integer *version*, geometry *geom*, integer *maxdecimaldigits=15*, text *nprefix=NULL*);

text **ST_AsKML**(integer *version*, geography *geog*, integer *maxdecimaldigits=15*, text *nprefix=NULL*);

설명: 도형을 KML(Keyhole Markup Language) 요소로 반환합니다. 이 함수에는 몇 가지 변형법이 있습니다. 출력물에 쓰이는 소수점 이하 자릿수의 최대값(기본값은 15), 버전의 기본값은 2 이며 기본 네임스페이스는 접두사를 쓰지 않습니다.

버전 1: ST_AsKML(geom_or_geog, maxdecimaldigits) / version=2 / maxdecimaldigits=15

버전 2: ST_AsKML(version, geom_or_geog, maxdecimaldigits, nprefix) maxdecimaldigits=15 / nprefix=NULL



PostGIS 를 Proj 지원과 함께 컴파일해야 합니다. Proj 지원과 함께 컴파일됐는지 여부를 확인하려면 PostGIS_Full_Version 을 이용하십시오.



1.2.2 버전부터 사용할 수 있습니다. 버전 파라미터를 포함하는 최신 응용방법은 1.3.2 버전부터 등장했습니다.



개선 사항: 2.0.0 버전부터 네임스페이스에 접두사를 추가할 수 있습니다. 기본값은 접두사를 쓰지 않는 것입니다.



변경 사항: 2.0.0 버전부터 기본 독립 변수(default arg)를 사용하고 명명된 독립 변수(named arg)를 지원합니다.



ST_AsKML 출력물은 SRID 가 없는 도형과는 동작하지 않을 것입니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
SELECT ST_AsKML(ST_GeomFromText('POLYGON((0 0,0 1,1 1,1 0,0 0))',4326));

             st_askml
             --------
             <Polygon
><outerBoundaryIs
><LinearRing
><coordinates
>0,0 0,1 1,1 1,0 0,0</coordinates
></LinearRing
></outerBoundaryIs
></Polygon>

-- 3D 라인스트링
             SELECT ST_AsKML('SRID=4326;LINESTRING(1 2 3, 4 5 6)');
             <LineString
><coordinates
>1,2,3 4,5,6</coordinates
></LineString>
```

참고: ST_AsSVG, ST_AsGML

8.7.9. ST_AsLatLonText

ST_AsLatLonText — 주어진 포인트의 도, 분, 초 표현식을 반환합니다.

Synopsis

text **ST_AsLatLonText**(geometry *pt*, text *format=''*);

설명: 포인트의 도, 분, 초 표현식을 반환합니다.



이 함수는 포인트가 위도/경도 투영체 안에 있다고 가정합니다. 출력물에서 X(경도) 및 Y(위도) 좌표는 "정규" 범위(경도는 -180 도에서 180 도, 위도는 -90 도에서 90 도)로 정규화됩니다.

텍스트 파라미터는 결과물의 텍스트를 위한 형식을 담고 있는 형식 스트링으로, 날짜 형식 스트링과 비슷합니다. 유효한 토큰은 도를 나타내는 "D", 분을 나타내는 "M", 초를 나타내는 "S", 그리고 기본방향(북남동서, cardinal direction)을 나타내는 "C"입니다. D, M, S 토큰들은 원하는 폭 및 정밀도를 표시하기 위해 반복될 수도 있습니다("SSS.SSSS"는 "1.0023"처럼 표기하라는 뜻입니다).

M, S, C 는 선택적입니다. "C"가 생략된 경우, 도는 남쪽 또는 서쪽일 경우 "-" 부호와 함께 표시됩니다. "S"가 생략된 경우, 사용자가 설정한 대로의 소수점 이하 자릿수 정밀도로 분이 표시될 것입니다. "M"도 생략된 경우, 사용자가 설정한 대로의 소수점 이하 자릿수 정밀도로 도가 표시될 것입니다.

형식 스트링이 빠진 (또는 길이가 0 인) 경우 기본 형식을 쓸 것입니다.

2.0 버전부터 사용할 수 있습니다.

예시

기본형식

```
SELECT (ST_AsLatLonText('POINT (-3.2342342 -2.32498)'));
     st_aslatlontext
----------------------------
 2°19'29.928"S 3°14'3.243"W
```

(기본값과 동일한) 형식을 제공합니다.

```
SELECT (ST_AsLatLonText('POINT (-3.2342342 -2.32498)', 'D°M''S.SSS"C'));
     st_aslatlontext
----------------------------
 2°19'29.928"S 3°14'3.243"W
```

D, M, S, C 및 .이 아닌 다른 문자들은 그냥 무시됩니다.

```
SELECT  (ST_AsLatLonText('POINT  (-3.2342342  -2.32498)',  'D  degrees,  M
minutes, S seconds to the C'));
                            st_aslatlontext
-------------------------------------------------------------------------
-------------
 2 degrees, 19 minutes, 30 seconds to the S 3 degrees, 14 minutes, 3 seconds
to the W
```

기본방향 대신 부호가 붙은 도를 사용합니다.

```
SELECT (ST_AsLatLonText('POINT (-3.2342342 -2.32498)', 'D°M''S.SSS"'));
     st_aslatlontext
----------------------------
 -2°19'29.928" -3°14'3.243"
```

소수점이 붙은 도를 사용합니다.

```
SELECT  (ST_AsLatLonText('POINT  (-3.2342342  -2.32498)',  'D.DDDD  degrees
C'));
        st_aslatlontext
-----------------------------------
 2.3250 degrees S 3.2342 degrees W
```

지나치게 큰 값은 정규화됩니다.

```
SELECT (ST_AsLatLonText('POINT (-302.2342342 -792.32498)'));
       st_aslatlontext
-----------------------------
 72°19'29.928"S 57°45'56.757"E
```

8.7.10. ST_AsSVG

ST_AsSVG — 도형을 주어진 도형 또는 지리형의 SVG 경로 데이터로 반환합니다.

Synopsis

text **ST_AsSVG**(geometry *geom*, integer *rel=0*, integer *maxdecimaldigits=15*);

text **ST_AsSVG**(geography *geog*, integer *rel=0*, integer *maxdecimaldigits=15*);

설명: 도형을 SVG(Scalar Vector Graphics) 경로 데이터로 반환합니다. 상대 이동(relative move) 관점에서 구현된 경로 데이터를 얻으려면 두 번째 인수로 1 을 사용하고, 절대 이동(absolute move) 관점에서 구현된 경로 데이터를 얻으려면 기본값 0 을 사용하십시오. 출력물에 쓰인 소수점 이하 자릿수의 최대값(기본값은 15)을 줄이는 데 세 번째 인수를 쓸 수도 있습니다. 'rel' 독립 변수가 0 일 때 포인트 도형을 cx/cy 로 렌더링하고, 'rel' 독립 변수가 1 일 때 x/y 로 렌더링합니다. 멀티포인트 도형들은 쉼표(",")로 구분하고, 도형 집합의 도형들은 쌍반점(";")으로 구분합니다.



1.2.2 버전부터 사용할 수 있습니다. 1.4.0 버전에서 http://www.w3.org/TR/SVG/paths.html#PathDataBNF 를 준수하기 위해 절대 경로에 L 명령어를 포함시켰습니다.

변경 사항: 2.0.0 버전부터 기본 독립 변수(default arg)를 사용하고 명명된 독립 변수(named arg)를 지원합니다.

예시

```
SELECT ST_AsSVG(ST_GeomFromText('POLYGON((0 0,0 1,1 1,1 0,0 0))',4326));

             st_assvg
             --------
             M 0 0 L 0 -1 1 -1 1 0 Z
```

8.7.11. ST_AsText

ST_AsText — 도형/지리형의 WKT(Well-Known Text) 표현식을 SRID 메타데이터 없이 반환합니다.

Synopsis

text **ST_AsText**(geometry *g1*);

text **ST_AsText**(geography *g1*);

설명: 도형/지리형의 WKT(Well-Known Text) 표현식을 반환합니다.



WKT 사양은 SRID 를 포함하지 않습니다. SRID 를 데이터의 일부로 반환받으려면 비표준 PostGIS ST_AsEWKT 함수를 이용하십시오.



WKT 형식은 정밀도를 유지하지 않으므로, 부동소수점 절단(floating truncation)을 피하려면 이동 작업(transport)에 ST_AsBinary 또는 ST_AsEWKB 형식을 이용하십시오.



ST_AsText 는 ST_GeomFromText 의 역함수입니다. ST_AsText 표현식을 PostGIS 도형으로 변환하려면 ST_GeomFromText 함수를 이용하십시오.

1.5.0 버전부터 지리형을 지원합니다.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다. s2.1.1.1

✔ 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.25

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

예시

```
SELECT ST_AsText('0103000000010000000500000000000000000
0000000000000000000000000000000000000000000000000000
F03F000000000000F03F000000000000F03F000000000000F03
F000000000000000000000000000000000000000000000000');

                st_astext
---------------------------------
 POLYGON((0 0,0 1,1 1,1 0,0 0))
(1 row)
```

참고: ST_AsBinary, ST_AsEWKB, ST_AsEWKT, ST_GeomFromText

8.7.12. ST_AsTWKB

ST_AsTWKB — 도형을 TWKB(Tiny Well-Known Binary)로 반환합니다.

Synopsis

bytea **ST_AsTWKB**(geometry *g1*, integer *decimaldigits_xy=0*, integer *decimaldigits_z=0*, integer *decimaldigits_m=0*, boolean *include_sizes=false*, boolean *include_bounding boxes=false*);

bytea **ST_AsTWKB**(geometry[] *geometries*, bigint[] *unique_ids*, integer *decimaldigits_xy=0*, integer *decimaldigits_z=0*, integer *decimaldigits_m=0*, boolean *include_sizes=false*, boolean *include_bounding_boxes=false*);

설명: 도형을 TWKB(Tiny Well-Known Binary) 형식으로 반환합니다. TWKB 는 출력물의 용량을 최소화하는 데 집중한 압축 바이너리 형식 입니다.

소수점 이하 자릿수 파라미터가 출력물에 어느 정도의 정밀도를 저장할지 결정합니다. 기본적으로, 모든 값은 인코딩하기 전에 가장 가까운 단위로 반올림됩니다. 더 높은 정밀도를 복사하고 싶다면, 자릿수를 올리십시오. 예를 들어, 값이 1 이라면 소수점 오른쪽의 첫 번째 숫자까지 보전될 것입니다.

크기 및 경계 상자 파라미터는 객체의 인코딩된 길이 및 경계에 대한 선택적인 정보를 출력물에 포함시킬지 말지를 결정합니다. 기본적으로는 포함되지 않습니다. 사용자의 클라이언트 소프트웨어가 필요로하지 않는다면 활성화시키지 마십시오. 디스크 공간을 소비할 뿐입니다(디스크 공간을 절약하는 것이 TWKB 의 목적입니다).

이 함수의 배열 입력 형식은 도형 집합 및 유일 식별자를 식별자를 보전하는 TWKB 집합으로 변환하는 데 쓰입니다. 집합의 압축을 풀어 그 안에 있는 객체들에 대한 상세 정보에 접근하는 기능을 가진 클라이언트에 유용합니다. array_agg 함수를 이용해서 배열을 생성할 수 있습니다. 다른 파라미터들은 이 함수의 단순 형태의 경우와 동일하게 실행됩니다.



https://github.com/TWKB/Specification 에서 형식 사양서를 찾아볼 수 있으며, https://github.com/TWKB/twkb.js 에서 자바스크립트 클라이언트를 빌드하기 위한 코드를 찾을 수 있습니다.

2.2.0 버전부터 사용할 수 있습니다.

예시

```
SELECT ST_AsTWKB('LINESTRING(1 1,5 5)'::geometry);
              st_astwkb
------------------------------------------
\x02000202020808
```

식별자를 포함하는 종합 TWKB 객체를 생성하려면, 먼저 "array_agg()"를 통해 원하는 도형과 객체를 합친 다음 적절한 TWKB 함수를 호출하십시오.

```
SELECT ST_AsTWKB(array_agg(geom), array_agg(gid)) FROM mytable;
              st_astwkb
------------------------------------------
\x040402020400000202
```

참고: ST_AsBinary, ST_AsEWKB, ST_AsEWKT, ST_GeomFromText

8.7.13. ST_AsX3D

ST_AsX3D — 도형을 X3D XML 노드 요소 형식: ISO-IEC-19776-1.2-X3DEncodings-XML 로 반환합니다.

Synopsis

text **ST_AsX3D**(geometry *g1*, integer *maxdecimaldigits=15*, integer *options=0*);

설명: 도형을 http://www.web3d.org/standards/number/19776-1 에서 정의하는 X3D XML 형식화 노드 요소로 반환합니다. maxdecimaldigits (정밀도)를 설정하지 않을 경우 기본값인 15 를 씁니다.



X3D 도형 유형이 PostGIS 도형 유형을 직접 매핑하지 않기 때문에 PostGIS 도형을 X3D 도형으로 번역하는 다양한 방법이 있습니다. 또 현재 렌더링 도구 대부분이 새로운 X3D 유형을 지원하지 않기 때문에, 더 잘 매핑돼 있는 몇몇 X3D 유형을 다루지 않습니다. 이 단원에서 다루는 것이 우리가 결정한 매핑입니다. 사용자가 선호하는 매핑을 표시할 수 있게 해주는 아이디어나 방법이 떠올랐다면 부디 버그 티켓(bug ticket)을 포스팅해주십시오.
다음은 현재 PostGIS 2D/3D 유형을 X3D 유형으로 매핑하는 방법입니다.

'옵션' 인수는 비트필드 구조체입니다. PostGIS 2.2 이상 버전의 경우, 좌표를 X3D 지리좌표 지리공간 노드로 표현할지, 그리고 또 x/y 축을 뒤집을지 말지를 표시하는 데 쓰입니다. 기본적으로 ST_AsX3D 는 데이터베이스 형식 (long,lat or X,Y)로 출력하지만, X3D 의 기본값인 위도/경도, y/x 를 선호할 수도 있습니다.

- 0: 데이터베이스 순서로 정렬된 X/Y(예를 들어 경도/위도 = X,Y 가 표준 데이터베이스 순서입니다), 데이터 값, 비(非) 공간 좌표(구식 정규 좌표 태그).
- 1: X 및 Y 를 뒤집습니다. 지리좌표(GeoCoordinate) 옵션 스위치와 연결해서 사용할 경우, 출력물이 기본값인 "latitude_first"(위도 먼저)가 될 것이고 좌표도 마찬가지로 뒤집힐 것입니다.
- 2: 좌표를 지리공간 지리좌표로 출력합니다. 도형이 WGS84 경위도(SRID 4326)가 아닐 경우, 이 옵션은 오류를 발생시킬 것입니다. 현재 이 지리좌표 유형만 지원합니다. 공간 참조 시스템을 정의하는 X3D 사양 을 참조하십시오. 출력물은 기본적으로 GeoCoordinate geoSystem="GD" "WE" "longitude_first"' 가 됩니다. X3D 기본값인 GeoCoordinate geoSystem="GD" "WE" "latitude_first"' 를 선호한다면, (2 + 1) = 3 을 쓰십시오.

PostGIS 유형	2D X3D 유형	3D X3D 유형
LINESTRING	아직 구현되지 않았습니다. 구현시 PolyLine2D 가 될 것입니다.	LineSet
MULTILINESTRING	아직 구현되지 않았습니다. 구현시 PolyLine2D 가 될 것입니다.	IndexedLineSet
MULTIPOINT	Polypoint2D	PointSet
POINT	공백으로 구분된 좌표를 출력합니다.	공백으로 구분된 좌표를 출력합니다.
(MULTI) POLYGON, POLYHEDRALSURFACE	유효하지 않은 X3D 마크업(markup)입니다.	IndexedFaceSet (현재 내곽 고리를 또다른 면 집합(faceset)으로 출력합니다.)
TIN	TriangleSet2D (아직 구현되지 않았습니다.)	IndexedTriangleSet



2 차원 도형 지원이 아직 완성되지 않았습니다. 현재 내곽 고리를 그저 개별적인 폴리곤으로 그립니다. 이 부분은 현재 작업중입니다.

3 차원 공간 분야에서 많은 개선이 이루어지고 있습니다. 특히 X3D Integration with HTML5 가 말입니다.

또 렌더링된 도형을 보는 데 이용할 수 있는 멋진 오픈소스 X3D 뷰어가 있습니다. 맥, 리눅스, 윈도우 용 Free Wrl 바이너리를 http://freewrl.sourceforge.net/ 에서 다운로드할 수 있습니다. 도형을 보기 위해 패키징된 FreeWRL_Launcher 를 이용하십시오.

또 X3DDom HTML/JS 오픈소스 툴키트 및 ST_AsX3D 함수를 활용하는 PostGIS 미니멀리스트 X3D 뷰어 도 확인해보십시오.

2.0.0 버전부터 ISO-IEC-19776-1.2-X3DEncodings-XML 을 이용할 수 있습니다.

개선 사항: 2.2.0 버전부터 지리좌표 및 축(x/y, 경도/위도) 뒤집기를 지원합니다. 자세한 내용은 옵션을 살펴보십시오.

✓ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

예시: 완전히 기능하는 X3D 문서를 생성합니다. 다음 예시는 FreeWrl 및 다른 X3D 뷰어에서 볼 수 있는 정육면체를 생성할 것입니다.

```
SELECT '<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE      X3D      PUBLIC      "ISO//Web3D//DTD      X3D      3.0//EN"
"http://www.web3d.org/specifications/x3d-3.0.dtd">
<X3D>
  <Scene>
    <Transform>
      <Shape>
       <Appearance>
           <Material emissiveColor=''0 0 1''/>
       </Appearance
> ' ||
       ST_AsX3D( ST_GeomFromEWKT('POLYHEDRALSURFACE( ((0 0 0, 0 0 1, 0 1 1,
0 1 0, 0 0 0)),
((0 0 0, 0 1 0, 1 1 0, 1 0 0, 0 0 0)), ((0 0 0, 1 0 0, 1 0 1, 0 0 1, 0 0
0)),
((1 1 0, 1 1 1, 1 0 1, 1 0 0, 1 1 0)),
((0 1 0, 0 1 1, 1 1 1, 1 1 0, 0 1 0)), ((0 0 1, 1 0 1, 1 1 1, 0 1 1, 0 0
1)) )')) ||
      '</Shape>
    </Transform>
  </Scene>
</X3D
>' As x3ddoc;

             x3ddoc
             --------
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE      X3D      PUBLIC      "ISO//Web3D//DTD      X3D      3.0//EN"
"http://www.web3d.org/specifications/x3d-3.0.dtd">
<X3D>
  <Scene>
    <Transform>
      <Shape>
       <Appearance>
           <Material emissiveColor='0 0 1'/>
       </Appearance>
       <IndexedFaceSet  coordIndex='0 1 2 3 -1 4 5 6 7 -1 8 9 10 11 -1 12 13
14 15 -1 16 17 18 19 -1 20 21 22 23'>
           <Coordinate point='0 0 0 0 0 1 0 1 1 0 1 0 0 0 0 0 1 0 1 1 0 1 0
0 0 0 0 1 0 0 1 0 1 0 0 1 1 1 0 1 1 1 1 0 1 1 0 0 0 1 0 0 1 1 1 1 1 1 1 0
0 0 1 1 0 1 1 1 1 0 1 1' />
      </IndexedFaceSet>
      </Shape>
    </Transform>
  </Scene>
</X3D
>
```

예시: 소수점 정밀도 6 이며 3 단위 상승한 팔각형

```
SELECT ST_AsX3D(
```

```
ST_Translate(
   ST_Force_3d(
      ST_Buffer(ST_Point(10,10),5, 'quad_segs=2')), 0,0,
   3)
  ,6) As x3dfrag;

x3dfrag
--------
<IndexedFaceSet coordIndex="0 1 2 3 4 5 6 7">
   <Coordinate point="15 10 3 13.535534 6.464466 3 10 5 3 6.464466 6.464466
3 5 10 3 6.464466 13.535534 3 10 15 3 13.535534 13.535534 3 " />
</IndexedFaceSet
>
```

예시: 완전히 기능하는 X3D 문서를 생성합니다. 다음 예시는 FreeWrl 및 다른 X3D 뷰어에서 볼 수 있는 정육면체를 생성할 것입니다.

```
SELECT DropGeometryTable ('my_schema','my_spatial_table');
```

예시: TIN

```
SELECT DropGeometryTable ('my_schema','my_spatial_table');
SELECT ST_AsX3D(ST_GeomFromEWKT('TIN (((
            0 0 0,
            0 0 1,
            0 1 0,
            0 0 0
         )), ((
            0 0 0,
            0 1 0,
            1 1 0,
            0 0 0
         ))
         )')) As x3dfrag;

            x3dfrag
            --------
<IndexedTriangleSet  index='0 1 2 3 4 5'
><Coordinate point='0 0 0 0 0 1 0 1 0 0 0 0 0 1 0 1 1 0'/></IndexedTriangleSet
>
```

예시: 닫힌 멀티라인스트링(구멍을 가진 폴리곤의 경계선)

```
SELECT ST_AsX3D(
               ST_GeomFromEWKT('MULTILINESTRING((20 0 10,16 -12 10,0 -16
10,-12 -12 10,-20 0 10,-12 16 10,0 24 10,16 16 10,20 0 10),
  (12 0 10,8 8 10,0 12 10,-8 8 10,-8 0 10,-8 -4 10,0 -8 10,8 -4 10,12 0
10))')
) As x3dfrag;

            x3dfrag
            --------
<IndexedLineSet  coordIndex='0 1 2 3 4 5 6 7 0 -1 8 9 10 11 12 13 14 15 8'>
   <Coordinate point='20 0 10 16 -12 10 0 -16 10 -12 -12 10 -20 0 10 -12
16 10 0 24 10 16 16 10 12 0 10 8 8 10 0 12 10 -8 8 10 -8 0 10 -8 -4 10 0 -
8 10 8 -4 10 ' />
 </IndexedLineSet
>
```

8.7.14. ST_GeoHash

ST_GeoHash — 도형의 GeoHash 표현식을 반환합니다.

Synopsis

text **ST_GeoHash**(geometry *geom*, integer *maxchars=full_precision_of_point*);

설명: 도형의 GeoHash 표현식(http://en.wikipedia.org/wiki/Geohash)을 반환합니다. GeoHash 는 포인트를 접두사 작업에 기반해 정렬할 수 있고 검색할 수 있는 텍스트 형식으로 인코딩합니다. GeoHash 가 짧을수록 포인트의 정밀도 표현이 낮아집니다. GeoHash 를 실제 포인트를 담고 있는 상자로 생각할 수도 있습니다.

maxchars 를 설정하지 않은 경우 ST_GeoHash 는 입력 도형의 완전한 정밀도에 기반한 GeoHash 를 반환합니다. 포인트가 (입력의 완전한 이중 정밀도 데이터형을 담기에 충분한) 20 글자의 정밀도를 가진 GeoHash 를 반환합니다. 다른 유형들은 해당 피처의 용량에 기반한 다양한 분량의 정밀도를 가진 GeoHash 를 반환합니다. 피처 용량이 클수록 정밀도는 낮아지고, 피처 용량이 작을수록 정밀도는 높아집니다. GeoHash 가 상정하는 상자가 언제나 입력 피처를 담고 있을 것이라는 가정 때문입니다.

maxchars 를 설정한 경우 ST_GeoHash 는 최대한 설정 글자수의 정밀도를 가진 GeoHash 를 반환합니다. 따라서 입력 도형의 정밀도 표현은 낮은 편입니다. 포인트가 아닌 도형의 경우, 도형의 경계 상자의 중심점에서 계산을 시작합니다.

1.4.0 버전부터 사용할 수 있습니다.



ST_GeoHash 함수는 지리 좌표(경도/위도)를 가지지 않은 도형을 입력받지 못 합니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

예시

```
SELECT ST_GeoHash(ST_SetSRID(ST_MakePoint(-126,48),4326));

       st_geohash
----------------------
 c0w3hf1s70w3hf1s70w3

SELECT ST_GeoHash(ST_SetSRID(ST_MakePoint(-126,48),4326),5);

 st_geohash
------------
 c0w3h
```

참고: ST_GeomFromGeoHash

8.8. 연산자(operator)

&& — A 의 2D 경계 상자와 B 의 2D 경계 상자가 교차하는 경우 TRUE 를 반환합니다.

&&(geometry,box2df) — 도형의 (캐시된) 2 차원 경계 상자가 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)와 교차하는 경우 TRUE 를 반환합니다.

&&(box2df,geometry) — 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)가 도형의 (캐시된) 2 차원 경계 상자와 교차하는 경우 TRUE 를 반환합니다.

&&(box2df,box2df) — 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF) 2 개가 서로 교차하는 경우 TRUE 를 반환합니다.

&&& — A 의 n 차원 경계 상자와 B 의 n 차원 경계 상자가 교차하는 경우 TRUE 를 반환합니다.

&&&(geometry,gidx) — 도형의 (캐시된) n 차원 경계 상자가 n 차원 부동소수점 데이터형(float) 정밀도 경계 상자(GIDX)와 교차하는 경우 TRUE 를 반환합니다.

&&&(gidx,geometry) — n 차원 부동소수점 데이터형(float) 정밀도 경계 상자(GIDX)가 도형의 (캐시된) n 차원 경계 상자와 교차하는 경우 TRUE 를 반환합니다.

&&&(gidx,gidx) — n 차원 부동소수점 데이터형(float) 정밀도 경계 상자(GIDX) 2 개가 서로 교차하는 경우 TRUE 를 반환합니다.

&< — A 의 경계 상자가 B 의 경계 상자와 겹치거나 그 왼쪽에 있을 경우 TRUE 를 반환합니다.

&<| — A 의 경계 상자가 B 의 경계 상자와 겹치거나 그 아래에 있을 경우 TRUE 를 반환합니다.

&> — A 의 경계 상자가 B 의 경계 상자와 겹치거나 그 오른쪽에 있을 경우 TRUE 를 반환합니다.

<< — A 의 경계 상자가 오로지 B 의 경계 상자 왼쪽에 있을 경우에만 TRUE 를 반환합니다.

<<| — A 의 경계 상자가 오로지 B 의 경계 상자 아래에 있을 경우에만 TRUE 를 반환합니다.

= — A 의 경계 상자와 B 의 경계 상자가 동일할 경우 TRUE 를 반환합니다. 이중 정밀도 경계 상자를 이용합니다.

>> — A 의 경계 상자가 오로지 B 의 경계 상자 오른쪽에 있을 경우에만 TRUE 를 반환합니다.

@ — B 의 경계 상자가 A 의 경계 상자를 담고 있을 경우 TRUE 를 반환합니다.

@(geometry,box2df) — 도형의 2 차원 경계 상자가 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF) 안에 포함되는 경우 TRUE 를 반환합니다.

@(box2df,geometry) — 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)가 도형의 2 차원 경계 상자 안에 포함되는 경우 TRUE 를 반환합니다.

@(box2df,box2df) — 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)가 또다른 2 차원 부동소수점 데이터형 정밀도 경계 상자 안에 포함되는 경우 TRUE 를 반환합니다.

|&> — A 의 경계 상자가 B 의 경계 상자와 겹치거나 그 위에 있을 경우 TRUE 를 반환합니다.

|>> — A 의 경계 상자가 오로지 B 의 경계 상자 위에 있을 경우에만 TRUE 를 반환합니다.

~ — A 의 경계 상자가 B 의 경계 상자를 담고 있을 경우 TRUE 를 반환합니다.

~(geometry,box2df) — 도형의 2 차원 경계 상자가 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)를 포함하는 경우 TRUE 를 반환합니다.

~(box2df,geometry) — 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)가 도형의 2 차원 경계 상자를 포함하는 경우 TRUE 를 반환합니다.

~(box2df,box2df) — 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)가 또다른 2 차원 부동소수점 데이터형 정밀도 경계 상자(BOX2DF)를 포함하는 경우 TRUE 를 반환합니다.

~= — A 의 경계 상자와 B 의 경계 상자가 동일할 경우 TRUE 를 반환합니다.

<-> — A 와 B 사이의 2 차원 거리를 반환합니다.

|=| — A 와 B 사이의 최근접점(closest point of approach)을 잇는 궤도(trajectory)의 거리를 반환합니다.

<#> — A 와 B 의 경계 상자 사이의 2 차원 거리를 반환합니다.

<<->> — A 경계 상자와 B 의 경계 상자의 중심점 사이의 N 차원 거리를 반환합니다.

<<#>> — A 와 B 의 경계 상자 사이의 N 차원 거리를 반환합니다.

8.8.1. &&

&& — A 의 2D 경계 상자와 B 의 2D 경계 상자가 교차하는 경우 TRUE 를 반환합니다.

Synopsis

boolean **&&**(geometry *A* , geometry *B*);

boolean **&&**(geography *A* , geography *B*);

설명: && 연산자는 도형 A 의 2D 경계 상자와 도형 B 의 2D 경계 상자가 교차하는 경우 TRUE 를 반환합니다.



이 피연산자(operand)는 도형에서 이용할 수도 있는 모든 인덱스를 활용할 것입니다.

개선 사항: 2.0.0 버전부터 다면체 표면(polyhedral surface)을 지원합니다.

1.5.0 버전부터 지리형을 지원합니다.

✓ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✓ 이 함수는 다면체 표면을 지원합니다.

예시

```
SELECT tbl1.column1, tbl2.column1, tbl1.column2 && tbl2.column2 AS overlaps
FROM ( VALUES
       (1, 'LINESTRING(0 0, 3 3)'::geometry),
       (2, 'LINESTRING(0 1, 0 5)'::geometry)) AS tbl1,
```

```
( VALUES
      (3, 'LINESTRING(1 2, 4 6)'::geometry)) AS tbl2;

 column1 | column1 | overlaps
---------+---------+----------
       1 |       3 | t
       2 |       3 | f
(2 rows)
```

참고: |&>, &>, &<|, &<, ~, @

8.8.2. &&(geometry,box2df)

&&(geometry,box2df) — 도형의 (캐시된) 2 차원 경계 상자가 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)와 교차하는 경우 TRUE 를 반환합니다.

Synopsis

boolean **&&**(geometry *A* , box2df *B*);

설명: 도형 A 의 캐시된 2 차원 경계 상자가 2 차원 경계 상자 B 와 교차하는 경우, && 연산자가 부동소수점 정밀도를 이용해서 TRUE 를 반환합니다. 즉 B 가 (이중 정밀도) BOX2D 일 경우, B 를 내부적으로 부동소수점 정밀도 2 차원 경계 상자(BOX2DF)로 변환할 것입니다.



이 피연산자는 사용자보다 주로 BRIN 연산자가 내부적으로 활용하기 위한 것입니다.

BRIN(Block Range INdexes)을 지원하기 시작한 2.3.0 버전부터 사용할 수 있습니다.

✓ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✓ 이 함수는 다면체 표면을 지원합니다.

예시

```
SELECT      ST_MakePoint(1,1)      &&      ST_MakeBox2D(ST_MakePoint(0,0),
ST_MakePoint(2,2)) AS overlaps;

 overlaps
----------
 t
(1 row)
```

참고: &&(box2df,geometry), &&(box2df,box2df), ~(geometry,box2df), ~(box2df,geometry), ~(box2df,box2df), @(geometry,box2df), @(box2df,geometry), @(box2df,box2df)

8.8.3. &&(box2df,geometry)

&&(box2df,geometry) — 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)가 도형의 (캐시된) 2 차원 경계 상자와 교차하는 경우 TRUE 를 반환합니다.

Synopsis

boolean **&&**(box2df *A* , geometry *B*);

설명: 2 차원 경계 상자 A 가 도형 B 의 캐시된 2 차원 경계 상자와 교차하는 경우, && 연산자가 부동소수점 정밀도를 이용해서 TRUE 를 반환합니다. 즉 A 가 (이중 정밀도) BOX2D 일 경우, A 를 내부적으로 부동소수점 정밀도 2 차원 경계 상자(BOX2DF)로 변환할 것입니다.



이 피연산자는 사용자보다 주로 BRIN 연산자가 내부적으로 활용하기 위한 것입니다.

BRIN(Block Range INdexes)을 지원하기 시작한 2.3.0 버전부터 사용할 수 있습니다.

✓ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✓ 이 함수는 다면체 표면을 지원합니다.

예시

```
SELECT      ST_MakeBox2D(ST_MakePoint(0,0),      ST_MakePoint(2,2))      &&
ST_MakePoint(1,1) AS overlaps;

 overlaps
----------
 t
(1 row)
```

참고: &&(geometry,box2df), &&(box2df,box2df), ~(geometry,box2df), ~(box2df,geometry), ~(box2df,box2df), @(geometry,box2df), @(box2df,geometry), @(box2df,box2df)

8.8.4. &&(box2df,box2df)

&&(box2df,box2df) — 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF) 2 개가 서로 교차하는 경우 TRUE 를 반환합니다.

Synopsis

boolean **&&**(box2df *A* , box2df *B*);

설명: 2 차원 경계 상자 A 와 B 가 서로 교차하는 경우, && 연산자가 부동소수점 정밀도를 이용해서 TRUE 를 반환합니다. 즉 A(또는 B)가 (이중 정밀도) BOX2D 일 경우, A(또는 B)를 내부적으로 부동소수점 정밀도 2 차원 경계 상자(BOX2DF)로 변환할 것입니다.



이 피연산자는 사용자보다 주로 BRIN 연산자가 내부적으로 활용하기 위한 것입니다.

BRIN(Block Range INdexes)을 지원하기 시작한 2.3.0 버전부터 사용할 수 있습니다.

✓ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✓ 이 함수는 다면체 표면을 지원합니다.

예시

```
SELECT      ST_MakeBox2D(ST_MakePoint(0,0),      ST_MakePoint(2,2))      &&
ST_MakeBox2D(ST_MakePoint(1,1), ST_MakePoint(3,3)) AS overlaps;

 overlaps
----------
 t
(1 row)
```

참고: &&(geometry,box2df), &&(box2df,geometry), ~(geometry,box2df), ~(box2df,geometry), ~(box2df,box2df), @(geometry,box2df), @(box2df,geometry), @(box2df,box2df)

8.8.5. &&&

&&& — A 의 n 차원 경계 상자와 B 의 n 차원 경계 상자가 교차하는 경우 TRUE 를 반환합니다.

Synopsis

boolean **&&&**(geometry *A* , geometry *B*);

설명: &&& 연산자는 도형 A 의 n 차원 경계 상자와 도형 B 의 n 차원 경계 상자가 교차하는 경우 TRUE 를 반환합니다.



이 피연산자(operand)는 도형에서 이용할 수도 있는 모든 인덱스를 활용할 것입니다.

2.0.0 버전부터 사용할 수 있습니다.

✓ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✓ 이 함수는 다면체 표면을 지원합니다.

✓ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

✓ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시: 3 차원 라인스트링

```
SELECT  tbl1.column1,  tbl2.column1,  tbl1.column2  &&&  tbl2.column2  AS
overlaps_3d,
                            tbl1.column2 && tbl2.column2 AS overlaps_2d
FROM ( VALUES
       (1, 'LINESTRING Z(0 0 1, 3 3 2)'::geometry),
       (2, 'LINESTRING Z(1 2 0, 0 5 -1)'::geometry)) AS tbl1,
( VALUES
       (3, 'LINESTRING Z(1 2 1, 4 6 1)'::geometry)) AS tbl2;

 column1 | column1 | overlaps_3d | overlaps_2d
---------+---------+-------------+-------------
       1 |       3 | t           | t
       2 |       3 | f           | t
```

예시: 3DM 라인스트링

```
SELECT  tbl1.column1,  tbl2.column1,  tbl1.column2  &&&  tbl2.column2  AS
overlaps_3zm,
                            tbl1.column2 && tbl2.column2 AS overlaps_2d
FROM ( VALUES
       (1, 'LINESTRING M(0 0 1, 3 3 2)'::geometry),
       (2, 'LINESTRING M(1 2 0, 0 5 -1)'::geometry)) AS tbl1,
( VALUES
       (3, 'LINESTRING M(1 2 1, 4 6 1)'::geometry)) AS tbl2;

 column1 | column1 | overlaps_3zm | overlaps_2d
---------+---------+-------------+-------------
       1 |       3 | t           | t
       2 |       3 | f           | t
```

참고: &&

8.8.6. &&&(geometry,gidx)

&&&(geometry,gidx) — 도형의 (캐시된) n 차원 경계 상자가 n 차원 부동소수점 데이터형(float) 정밀도 경계 상자(GIDX)와 교차하는 경우 TRUE 를 반환합니다.

Synopsis

boolean **&&&**(geometry *A* , gidx *B*);

설명: 도형 A 의 캐시된 n 차원 경계 상자가 n 차원 경계 상자 B 와 교차하는 경우, &&& 연산자가 부동소수점 정밀도를 이용해서 TRUE 를 반환합니다. 즉 B 가 (이중 정밀도) BOX3D 일 경우, B 를 내부적으로 부동소수점 정밀도 3 차원 경계 상자(GIDX)로 변환할 것입니다.



이 연산자는 사용자보다 주로 BRIN 연산자가 내부적으로 활용하기 위한 것입니다

BRIN(Block Range INdexes)을 지원하기 시작한 2.3.0 버전부터 사용할 수 있습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
SELECT    ST_MakePoint(1,1,1)    &&&    ST_3DMakeBox(ST_MakePoint(0,0,0),
ST_MakePoint(2,2,2)) AS overlaps;

 overlaps
----------
 t
(1 row)
```

참고: &&&(gidx,geometry), &&&(gidx,gidx)

8.8.7. &&&(gidx,geometry)

&&&(gidx,geometry) — n 차원 부동소수점 데이터형(float) 정밀도 경계 상자(GIDX)가 도형의 (캐시된) n 차원 경계 상자와 교차하는 경우 TRUE 를 반환합니다.

Synopsis

boolean **&&&**(gidx *A* , geometry *B*);

설명: n 차원 경계 상자 A 가 도형 B 의 캐시된 n 차원 경계 상자와 교차하는 경우, &&& 연산자가 부동소수점 정밀도를 이용해서 TRUE 를 반환합니다. 즉 A 가 (이중 정밀도) BOX3D 일 경우, A 를 내부적으로 부동소수점 정밀도 3 차원 경계 상자(GIDX)로 변환할 것입니다.



이 연산자는 사용자보다 주로 BRIN 연산자가 내부적으로 활용하기 위한 것입니다

BRIN(Block Range INdexes)을 지원하기 시작한 2.3.0 버전부터 사용할 수 있습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
SELECT    ST_3DMakeBox(ST_MakePoint(0,0,0),    ST_MakePoint(2,2,2))    &&&
ST_MakePoint(1,1,1) AS overlaps;
```

```
 overlaps
----------
 t
(1 row)
```

참고: &&&(geometry,gidx), &&&(gidx,gidx)

8.8.8. &&&(gidx,gidx)

&&&(gidx,gidx) — n 차원 부동소수점 데이터형(float) 정밀도 경계 상자(GIDX) 2 개가 서로 교차하는 경우 TRUE 를 반환합니다.

Synopsis

boolean **&&&**(gidx *A* , gidx *B*);

설명: n 차원 경계 상자 A 와 B 가 서로 교차하는 경우, &&& 연산자가 부동소수점 정밀도를 이용해서 TRUE 를 반환합니다. 즉 A(또는 B)가 (이중 정밀도) BOX3D 일 경우, A(또는 B)를 내부적으로 부동소수점 정밀도 3 차원 경계 상자(GIDX)로 변환할 것입니다.



이 연산자는 사용자보다 주로 BRIN 연산자가 내부적으로 활용하기 위한 것입니다.

BRIN(Block Range INdexes)을 지원하기 시작한 2.3.0 버전부터 사용할 수 있습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.**예시**

```
SELECT    ST_3DMakeBox(ST_MakePoint(0,0,0),    ST_MakePoint(2,2,2))    &&&
ST_3DMakeBox(ST_MakePoint(1,1,1), ST_MakePoint(3,3,3)) AS overlaps;

 overlaps
----------
 t
(1 row)
```

참고: &&&(geometry,gidx), &&&(gidx,geometry)

8.8.9. &<

&< — A 의 경계 상자가 B 의 경계 상자와 겹치거나 그 왼쪽에 있을 경우 TRUE 를 반환합니다.

Synopsis

boolean **&<**(geometry *A* , geometry *B*);

설명: &< 연산자는 도형 A 의 경계 상자가 도형 B 의 경계 상자와 겹치거나 그 왼쪽에 있을 경우, 또는 더 정확히 말하자면 도형 B 의 경계 상자와 겹치거나 그 오른쪽에 있지 않을 경우, TRUE 를 반환합니다.



이 피연산자(operand)는 도형에서 이용할 수도 있는 모든 인덱스를 활용할 것입니다.

예시

```
SELECT tbl1.column1, tbl2.column1, tbl1.column2 &< tbl2.column2 AS overleft
FROM
  ( VALUES
       (1, 'LINESTRING(1 2, 4 6)'::geometry)) AS tbl1,
  ( VALUES
       (2, 'LINESTRING(0 0, 3 3)'::geometry),
       (3, 'LINESTRING(0 1, 0 5)'::geometry),
       (4, 'LINESTRING(6 0, 6 1)'::geometry)) AS tbl2;

 column1 | column1 | overleft
---------+---------+----------
         1 |       2 | f
         1 |       3 | f
         1 |       4 | t
(3 rows)
```

참고: &&, |&>, &>, &<|

8.8.10. &<|

&<| — A 의 경계 상자가 B 의 경계 상자와 겹치거나 그 아래에 있을 경우 TRUE 를 반환합니다.

Synopsis

boolean **&<|**(geometry *A* , geometry *B*);

설명: &<| 연산자는 도형 A 의 경계 상자가 도형 B 의 경계 상자와 겹치거나 그 아래에 있을 경우, 또는 더 정확히 말하자면 도형 B 의 경계 상자와 겹치거나 위에 있지 않을 경우, TRUE 를 반환합니다.



이 방법은 원형 스트링 및 곡선을 지원합니다.



이 함수는 다면체 표면을 지원합니다.



이 피연산자(operand)는 도형에서 이용할 수도 있는 모든 인덱스를 활용할 것입니다.

예시

```
SELECT  tbl1.column1,  tbl2.column1,  tbl1.column2  &<|  tbl2.column2  AS
overbelow
FROM
  ( VALUES
       (1, 'LINESTRING(6 0, 6 4)'::geometry)) AS tbl1,
  ( VALUES
       (2, 'LINESTRING(0 0, 3 3)'::geometry),
       (3, 'LINESTRING(0 1, 0 5)'::geometry),
       (4, 'LINESTRING(1 2, 4 6)'::geometry)) AS tbl2;

 column1 | column1 | overbelow
---------+---------+-----------
         1 |       2 | f
         1 |       3 | t
         1 |       4 | t
(3 rows)
```

참고: &&, |&>, &>, &<

8.8.11. &>

&> — A 의 경계 상자가 B 의 경계 상자와 겹치거나 그 오른쪽에 있을 경우 TRUE 를 반환합니다.

Synopsis

boolean **&>**(geometry *A* , geometry *B*);

설명: &> 연산자는 도형 A 의 경계 상자가 도형 B 의 경계 상자와 겹치거나 그 오른쪽에 있을 경우, 또는 더 정확히 말하자면 도형 B 의 경계 상자와 겹치거나 그 왼쪽에 있지 않을 경우, TRUE 를 반환합니다.



이 피연산자(operand)는 도형에서 이용할 수도 있는 모든 인덱스를 활용할 것입니다.

예시

```
SELECT  tbl1.column1,  tbl2.column1,  tbl1.column2  &>  tbl2.column2  AS
overright
FROM
  ( VALUES
       (1, 'LINESTRING(1 2, 4 6)'::geometry)) AS tbl1,
  ( VALUES
       (2, 'LINESTRING(0 0, 3 3)'::geometry),
       (3, 'LINESTRING(0 1, 0 5)'::geometry),
       (4, 'LINESTRING(6 0, 6 1)'::geometry)) AS tbl2;

 column1 | column1 | overright
---------+---------+-----------
         1 |       2 | t
         1 |       3 | t
         1 |       4 | f
(3 rows)
```

참고: &&, |&>, &<|, &<

8.8.12. <<

<< — A 의 경계 상자가 오로지 B 의 경계 상자 왼쪽에 있을 경우에만 TRUE 를 반환합니다.

Synopsis

boolean **<<**(geometry *A* , geometry *B*);

설명: << 연산자는 도형 A 의 경계 상자가 오로지 도형 B 의 경계 상자 왼쪽에 있을 경우에만 TRUE 를 반환합니다.



이 피연산자(operand)는 도형에서 이용할 수도 있는 모든 인덱스를 활용할 것입니다.

예시

```
SELECT tbl1.column1, tbl2.column1, tbl1.column2 << tbl2.column2 AS left
FROM
  ( VALUES
       (1, 'LINESTRING (1 2, 1 5)'::geometry)) AS tbl1,
  ( VALUES
       (2, 'LINESTRING (0 0, 4 3)'::geometry),
       (3, 'LINESTRING (6 0, 6 5)'::geometry),
       (4, 'LINESTRING (2 2, 5 6)'::geometry)) AS tbl2;

 column1 | column1 | left
---------+---------+------
         1 |       2 | f
         1 |       3 | t
         1 |       4 | t
(3 rows)
```

참고: >>, |>>, <<|

8.8.13. <<|

<<| — A 의 경계 상자가 오로지 B 의 경계 상자 아래에 있을 경우에만 TRUE 를 반환합니다.

Synopsis

boolean **<<|**(geometry *A* , geometry *B*);

설명: <<| 연산자는 도형 A 의 경계 상자가 오로지 도형 B 의 경계 상자 아래에 있을 경우에만 TRUE 를 반환합니다.



이 피연산자(operand)는 도형에서 이용할 수도 있는 모든 인덱스를 활용할 것입니다.

예시

```
SELECT tbl1.column1, tbl2.column1, tbl1.column2 <<| tbl2.column2 AS below
FROM
  ( VALUES
       (1, 'LINESTRING (0 0, 4 3)'::geometry)) AS tbl1,
  ( VALUES
       (2, 'LINESTRING (1 4, 1 7)'::geometry),
       (3, 'LINESTRING (6 1, 6 5)'::geometry),
       (4, 'LINESTRING (2 3, 5 6)'::geometry)) AS tbl2;

 column1 | column1 | below
---------+---------+-------
         1 |       2 | t
         1 |       3 | f
         1 |       4 | f
(3 rows)
```

참고: <<, >>, |>>

8.8.14. =

= — A 의 경계 상자와 B 의 경계 상자가 동일할 경우 TRUE 를 반환합니다. 이중 정밀도 경계 상자를 이용합니다.

Synopsis

boolean =(geometry *A* , geometry *B*);

boolean =(geography *A* , geography *B*);

설명: = 연산자는 도형/지리형 A 의 경계 상자와 도형/지리형 B 의 경계 상자가 동일할 경우 TRUE 를 반환합니다. PostgreSQL 은 도형을 내부 정렬하고 비교하기 위해 도형에 대해 정의된 =, <, 및 > 연산자를 이용합니다(예: GROUP BY 또는 ORDER BY 절에서).



이 함수는 많은 혼란을 야기합니다. geometryA = geometryB 를 비교할 때 두 도형이 명백히 다르더라도 두 도형의 경계 상자가 동일하기만 하면 참을 반환하기 때문입니다. 진정한 동일성을 확인하려면 ST_OrderingEquals 또는 ST_Equals 함수를 이용하십시오.



이 피연산자(operand)는 도형에서 이용할 수도 있는 어떤 인덱스도 활용하지 않을 것입니다.

✓ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✓ 이 함수는 다면체 표면을 지원합니다.

변경 사항: 2.0.0 버전부터 도형의 경계 상자가 예전의 FLOAT4 정밀도 대신 이중 정밀도를 이용하도록 바뀌었습니다. 이 변경 사항의 부작용으로, 현재 버전과는 조금 다른 예전 버전에서 참을 반환할 수도

있었던 특정 포인트들이 2.0.0 이상 버전에서는 거짓을 반환할 수도 있습니다. FLOAT4 상자는 동일하게 유지되지만 FLOAT8(이중 정밀도) 상황에서는 다를 수도 있기 때문입니다.

예시

```
SELECT 'LINESTRING(0 0, 0 1, 1 0)'::geometry = 'LINESTRING(1 1, 0
0)'::geometry;
 ?column?
----------
 t
(1 row)

SELECT ST_AsText(column1)
FROM ( VALUES
       ('LINESTRING(0 0, 1 1)'::geometry),
       ('LINESTRING(1 1, 0 0)'::geometry)) AS foo;
        st_astext
---------------------
 LINESTRING(0 0,1 1)
 LINESTRING(1 1,0 0)
(2 rows)

-- 주의: GROUP BY 절이 도형의 동치 관계를 비교하기 위해 "=" 를 이용합니다.
SELECT ST_AsText(column1)
FROM ( VALUES
       ('LINESTRING(0 0, 1 1)'::geometry),
       ('LINESTRING(1 1, 0 0)'::geometry)) AS foo
GROUP BY column1;
        st_astext
---------------------
 LINESTRING(0 0,1 1)
(1 row)

-- 2.0 미만 버전에서, 다음은 참을 반환했습니다. --
 SELECT ST_GeomFromText('POINT(1707296.37 4820536.77)') =
       ST_GeomFromText('POINT(1707296.27 4820536.87)') As pt_intersect;

--pt_intersect --
```

참고: ST_Equals, ST_OrderingEquals

8.8.15. >>

>> — A 의 경계 상자가 오로지 B 의 경계 상자 오른쪽에 있을 경우에만 TRUE 를 반환합니다.

Synopsis

boolean **>>**(geometry *A* , geometry *B*);

설명: >> 연산자는 도형 A 의 경계 상자가 오로지 도형 B 의 경계 상자 오른쪽에 있을 경우에만 TRUE 를 반환합니다.



이 피연산자(operand)는 도형에서 이용할 수도 있는 모든 인덱스를 활용할 것입니다.

예시

```
SELECT tbl1.column1, tbl2.column1, tbl1.column2 >> tbl2.column2 AS right
FROM
  ( VALUES
       (1, 'LINESTRING (2 3, 5 6)'::geometry)) AS tbl1,
  ( VALUES
       (2, 'LINESTRING (1 4, 1 7)'::geometry),
       (3, 'LINESTRING (6 1, 6 5)'::geometry),
       (4, 'LINESTRING (0 0, 4 3)'::geometry)) AS tbl2;

 column1 | column1 | right
---------+---------+-------
         1 |       2 | t
         1 |       3 | f
         1 |       4 | f
(3 rows)
```

참고: <<, |>>, <<|

8.8.16. @

@ — B 의 경계 상자가 A 의 경계 상자를 담고 있을 경우 TRUE 를 반환합니다.

Synopsis

boolean **@**(geometry *A* , geometry *B*);

설명: @ 연산자는 도형 B 의 경계 상자가 도형 A 의 경계 상자를 완전히 담고 있을 경우 TRUE 를 반환합니다.



이 피연산자(operand)는 도형에서 이용할 수도 있는 모든 인덱스를 활용할 것입니다.

예시

```
SELECT tbl1.column1, tbl2.column1, tbl1.column2 @ tbl2.column2 AS contained
FROM
  ( VALUES
       (1, 'LINESTRING (1 1, 3 3)'::geometry)) AS tbl1,
  ( VALUES
       (2, 'LINESTRING (0 0, 4 4)'::geometry),
       (3, 'LINESTRING (2 2, 4 4)'::geometry),
       (4, 'LINESTRING (1 1, 3 3)'::geometry)) AS tbl2;

 column1 | column1 | contained
---------+---------+-----------
         1 |       2 | t
         1 |       3 | f
```

```
         1 |      4 | t
(3 rows)
```

참고: ~, &&

8.8.17. @(geometry,box2df)

@(geometry,box2df) — 도형의 2 차원 경계 상자가 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF) 안에 포함되는 경우 TRUE 를 반환합니다.

Synopsis

boolean **@**(geometry *A* , box2df *B*);

설명: 도형 A 의 2 차원 경계 상자가 2 차원 경계 상자 안에 포함되는 경우, @ 연산자가 부동소수점 정밀도를 이용해서 TRUE 를 반환합니다. 즉 B 가 (이중 정밀도) BOX2D 일 경우, B 를 내부적으로 부동소수점 정밀도 2 차원 경계 상자(BOX2DF)로 변환할 것입니다.



이 피연산자는 사용자보다 주로 BRIN 연산자가 내부적으로 활용하기 위한 것입니다.

BRIN(Block Range INdexes)을 지원하기 시작한 2.3.0 버전부터 사용할 수 있습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✔ 이 함수는 다면체 표면을 지원합니다.

예시

```
SELECT DropGeometryTable ('my_schema','my_spatial_table');
SELECT       ST_Buffer(ST_GeomFromText('POINT(2       2)'),       1)       @
ST_MakeBox2D(ST_MakePoint(0,0), ST_MakePoint(5,5)) AS is_contained;

 is_contained
--------------
 t
(1 row)
```

참고: &&(geometry,box2df), &&(box2df,geometry), &&(box2df,box2df), ~(geometry,box2df), ~(box2df,geometry), ~(box2df,box2df), @(box2df,geometry), @(box2df,box2df)

8.8.18. @(box2df,geometry)

@(box2df,geometry) — 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)가 도형의 2 차원 경계 상자 안에 포함되는 경우 TRUE 를 반환합니다.

Synopsis

boolean **@**(box2df *A* , geometry *B*);

설명: 2 차원 경계 상자 A 가 도형 B 의 2 차원 경계 상자 안에 포함되는 경우, @ 연산자가 부동소수점 정밀도를 이용해서 TRUE 를 반환합니다. 즉 B 가 (이중 정밀도) BOX2D 일 경우, B 를 내부적으로 부동소수점 정밀도 2 차원 경계 상자(BOX2DF)로 변환할 것입니다.



이 피연산자는 사용자보다 주로 BRIN 연산자가 내부적으로 활용하기 위한 것입니다.

BRIN(Block Range INdexes)을 지원하기 시작한 2.3.0 버전부터 사용할 수 있습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✔ 이 함수는 다면체 표면을 지원합니다.

예시

```
S    SELECT     ST_MakeBox2D(ST_MakePoint(2,2),     ST_MakePoint(3,3))     @
ST_Buffer(ST_GeomFromText('POINT(1 1)'), 10) AS is_contained;

 is_contained
--------------
 t
(1 row)
```

참고: &&(geometry,box2df), &&(box2df,geometry), &&(box2df,box2df), ~(geometry,box2df), ~(box2df,geometry), ~(box2df,box2df), @(geometry,box2df), @(box2df,box2df)

8.8.19. @(box2df,box2df)

@(box2df,box2df) — 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)가 또다른 2 차원 부동소수점 데이터형 정밀도 경계 상자 안에 포함되는 경우 TRUE 를 반환합니다.

Synopsis

boolean **@**(box2df *A* , box2df *B*);

설명: 2 차원 경계 상자 A 가 2 차원 경계 상자 B 안에 포함되는 경우, @ 연산자가 부동소수점 정밀도를 이용해서 TRUE 를 반환합니다. 즉 A(또는 B)가 (이중 정밀도) BOX2D 일 경우, A(또는 B)를 내부적으로 부동소수점 정밀도 2 차원 경계 상자(BOX2DF)로 변환할 것입니다.



이 피연산자는 사용자보다 주로 BRIN 연산자가 내부적으로 활용하기 위한 것입니다.

BRIN(Block Range INdexes)을 지원하기 시작한 2.3.0 버전부터 사용할 수 있습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✔ 이 함수는 다면체 표면을 지원합니다.

예시

```
SELECT      ST_MakeBox2D(ST_MakePoint(2,2),      ST_MakePoint(3,3))      @
ST_MakeBox2D(ST_MakePoint(0,0), ST_MakePoint(5,5)) AS is_contained;

 is_contained
--------------
 t
(1 row)
```

참고: &&(geometry,box2df), &&(box2df,geometry), &&(box2df,box2df), ~(geometry,box2df), ~(box2df,geometry), ~(box2df,box2df), @(geometry,box2df), @(box2df,geometry)

8.8.20. |&>

|&> — A 의 경계 상자가 B 의 경계 상자와 겹치거나 그 위에 있을 경우 TRUE 를 반환합니다.

Synopsis

boolean **|&>**(geometry *A* , geometry *B*);

설명: |&> 연산자는 도형 A 의 경계 상자가 도형 B 의 경계 상자와 겹치거나 그 위에 있을 경우, 또는 더 정확히 말하자면 도형 B 의 경계 상자와 겹치거나 아래에 있지 않을 경우, TRUE 를 반환합니다.



이 피연산자(operand)는 도형에서 이용할 수도 있는 모든 인덱스를 활용할 것입니다.

예시

```
SELECT  tbl1.column1,  tbl2.column1,  tbl1.column2  |&>  tbl2.column2  AS
overabove
FROM
  ( VALUES
       (1, 'LINESTRING(6 0, 6 4)'::geometry)) AS tbl1,
  ( VALUES
       (2, 'LINESTRING(0 0, 3 3)'::geometry),
       (3, 'LINESTRING(0 1, 0 5)'::geometry),
       (4, 'LINESTRING(1 2, 4 6)'::geometry)) AS tbl2;

 column1 | column1 | overabove
---------+---------+-----------
         1 |       2 | t
         1 |       3 | f
         1 |       4 | f
(3 rows)
```

참고: &&, &>, &<|, &<

8.8.21. |>>

|>> — A 의 경계 상자가 오로지 B 의 경계 상자 위에 있을 경우에만 TRUE 를 반환합니다.

Synopsis

boolean **|>>**(geometry *A* , geometry *B*);

설명: |>> 연산자는 도형 A 의 경계 상자가 오로지 도형 B 의 경계 상자 위에 있을 경우에만 TRUE 를 반환합니다.



이 피연산자(operand)는 도형에서 이용할 수도 있는 모든 인덱스를 활용할 것입니다.

예시

```
SELECT tbl1.column1, tbl2.column1, tbl1.column2 |>> tbl2.column2 AS above
FROM
  ( VALUES
       (1, 'LINESTRING (1 4, 1 7)'::geometry)) AS tbl1,
  ( VALUES
       (2, 'LINESTRING (0 0, 4 2)'::geometry),
       (3, 'LINESTRING (6 1, 6 5)'::geometry),
       (4, 'LINESTRING (2 3, 5 6)'::geometry)) AS tbl2;

 column1 | column1 | above
---------+---------+-------
         1 |       2 | t
         1 |       3 | f
         1 |       4 | f
(3 rows)
```

참고: <<, >>, <<|

8.8.22. ~

~ — A 의 경계 상자가 B 의 경계 상자를 담고 있을 경우 TRUE 를 반환합니다.

Synopsis

boolean **~**(geometry *A* , geometry *B*);

설명: ~ 연산자는 도형 A 의 경계 상자가 도형 B 의 경계 상자를 완전히 담고 있을 경우 TRUE 를 반환합니다.



이 피연산자(operand)는 도형에서 이용할 수도 있는 모든 인덱스를 활용할 것입니다.

예시

```
SELECT tbl1.column1, tbl2.column1, tbl1.column2 ~ tbl2.column2 AS contains
FROM
  ( VALUES
       (1, 'LINESTRING (0 0, 3 3)'::geometry)) AS tbl1,
  ( VALUES
       (2, 'LINESTRING (0 0, 4 4)'::geometry),
       (3, 'LINESTRING (1 1, 2 2)'::geometry),
       (4, 'LINESTRING (0 0, 3 3)'::geometry)) AS tbl2;

 column1 | column1 | contains
---------+---------+----------
         1 |       2 | f
         1 |       3 | t
         1 |       4 | t
(3 rows)
```

참고: @, &&

8.8.23. ~(geometry,box2df)

~(geometry,box2df) — 도형의 2 차원 경계 상자가 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)를 포함하는 경우 TRUE 를 반환합니다.

Synopsis

boolean ~(geometry *A* , box2df *B*);

설명: 도형 A 의 2 차원 경계 상자가 2 차원 경계 상자 B 를 포함하는 경우, ~ 연산자가 부동소수점 정밀도를 이용해서 TRUE 를 반환합니다. 즉 B 가 (이중 정밀도) BOX2D 일 경우, B 를 내부적으로 부동소수점 정밀도 2 차원 경계 상자(BOX2DF)로 변환할 것입니다.



이 피연산자는 사용자보다 주로 BRIN 연산자가 내부적으로 활용하기 위한 것입니다.

BRIN(Block Range INdexes)을 지원하기 시작한 2.3.0 버전부터 사용할 수 있습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✔ 이 함수는 다면체 표면을 지원합니다.

예시

```
SELECT      ST_Buffer(ST_GeomFromText('POINT(1      1)'),      10)      ~
ST_MakeBox2D(ST_MakePoint(0,0), ST_MakePoint(2,2)) AS contains;

 contains
----------
 t
(1 row)
```

참고: &&(geometry,box2df), &&(box2df,geometry), &&(box2df,box2df), ~(box2df,geometry), ~(box2df,box2df), @(geometry,box2df), @(box2df,geometry), @(box2df,box2df)

8.8.24. ~(box2df,geometry)

~(box2df,geometry) — 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)가 도형의 2 차원 경계 상자를 포함하는 경우 TRUE 를 반환합니다.

Synopsis

boolean ~(box2df *A* , geometry *B*);

설명: 2 차원 경계 상자 A 가 도형 B 의 2 차원 경계 상자를 포함하는 경우, ~ 연산자가 부동소수점 정밀도를 이용해서 TRUE 를 반환합니다. 즉 A 가 (이중 정밀도) BOX2D 일 경우, A 를 내부적으로 부동소수점 정밀도 2 차원 경계 상자(BOX2DF)로 변환할 것입니다.



이 피연산자는 사용자보다 주로 BRIN 연산자가 내부적으로 활용하기 위한 것입니다.

BRIN(Block Range INdexes)을 지원하기 시작한 2.3.0 버전부터 사용할 수 있습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✔ 이 함수는 다면체 표면을 지원합니다.

예시

```
SELECT      ST_MakeBox2D(ST_MakePoint(0,0),      ST_MakePoint(5,5))      ~
ST_Buffer(ST_GeomFromText('POINT(2 2)'), 1) AS contains;

 contains
----------
 t
(1 row)
```

참고: &&(geometry,box2df), &&(box2df,geometry), &&(box2df,box2df), ~(geometry,box2df), ~(box2df,box2df), @(geometry,box2df), @(box2df,geometry), @(box2df,box2df)

8.8.25. ~(box2df,box2df)

~(box2df,box2df) — 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)가 또다른 2 차원 부동소수점 데이터형 정밀도 경계 상자(BOX2DF)를 포함하는 경우 TRUE 를 반환합니다.

Synopsis

boolean ~(box2df A , box2df B);

설명: 2 차원 경계 상자 A 가 2 차원 경계 상자 B 를 포함하는 경우, ~ 연산자가 부동소수점 정밀도를 이용해서 TRUE 를 반환합니다. 즉 A 가 (이중 정밀도) BOX2D 일 경우, A 를 내부적으로 부동소수점 정밀도 2 차원 경계 상자(BOX2DF)로 변환할 것입니다.



이 피연산자는 사용자보다 주로 BRIN 연산자가 내부적으로 활용하기 위한 것입니다.

BRIN(Block Range INdexes)을 지원하기 시작한 2.3.0 버전부터 사용할 수 있습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✔ 이 함수는 다면체 표면을 지원합니다.

예시

```
SELECT      ST_MakeBox2D(ST_MakePoint(0,0),      ST_MakePoint(5,5))      ~
ST_MakeBox2D(ST_MakePoint(2,2), ST_MakePoint(3,3)) AS contains;

 contains
----------
 t
(1 row)
```

참고: &&(geometry,box2df), &&(box2df,geometry), &&(box2df,box2df), ~(geometry,box2df), ~(box2df,geometry), @(geometry,box2df), @(box2df,geometry), @(box2df,box2df)

8.8.26. ~=

~= — A 의 경계 상자와 B 의 경계 상자가 동일할 경우 TRUE 를 반환합니다.

Synopsis

boolean **~=**(geometry *A* , geometry *B*);

설명: ~= 연산자는 도형/지리형 A 의 경계 상자와 도형/지리형 B 의 경계 상자가 동일할 경우 TRUE 를 반환합니다.



이 피연산자(operand)는 도형에서 이용할 수도 있는 모든 인덱스를 활용할 것입니다.

1.5.0 버전에서 습성이 변경됐습니다.

✔ 이 함수는 다면체 표면을 지원합니다.



이 연산자는 PostGIS 1.5 버전에서 실제 도형 동일성을 테스트하는 것에서 오직 경계 상자 동일성을 확인하는 것으로 습성이 변경됐습니다. 또 사용자가 하드 업그레이드를 했는지 소프트 업그레이드를 했는지에 따라 사용자 데이터베이스가 어떤 습성을 가질지가 달라지기

때문에 문제가 복잡해집니다. 사용자 데이터베이스가 어떤 습성을 가지고 있는지 알아보려면 다음 쿼리를 실행해보면 됩니다. 진정한 도형 동일성을 확인하려면 ST_OrderingEquals 또는 ST_Equals 함수를 이용하십시오. 경계 상자 동일성을 확인하려면 =; 연산자가 더 안전한 선택입니다.

예시

```
select 'LINESTRING(0 0, 1 1)'::geometry ~= 'LINESTRING(0 1, 1 0)'::geometry
as equality;
 equality  |
-----------------+
         t    |
```

~= 연산자가 새 습성을 가지고 있는지 예전 습성을 가지고 있는지 테스트하는 데 이 쿼리를 쓸 수 있습니다.

참고: ST_Equals, ST_OrderingEquals, =

8.8.27. <->

<-> — A 와 B 사이의 2 차원 거리를 반환합니다.

Synopsis

double precision **<->**(geometry *A* , geometry *B*);

double precision **<->**(geography *A* , geography *B*);

설명: <-> 연산자는 두 도형 사이의 2 차원 거리를 반환합니다. "ORDER BY" 절 안에서 이용하면 인덱스 지원형(index-assisted) 최근접(nearest neighbor) 결과 집합을 출력합니다. PostgreSQL 9.5 미만 버전은 경계 상자의 중심점 사이의 거리만 출력하고, 9.5 이상 버전은 도형 사이의 진정한 거리, 그리고 지리형 사이의 구면 거리(distance sphere)를 산출하는 진정한 KNN 거리 탐색을 실행합니다.



이 피연산자(operand)는 도형에 대해 이용할 수 있을지도 모르는 2 차원 GiST 인덱스를 활용할 것입니다. 연산자가 ORDER BY 절 안에 있을 때만 공간 인덱스를 쓴다는 점에서 공간 인덱스를 이용하는 다른 연산자들과는 다릅니다.



도형 가운데 하나가, 예를 들어 a.geom 이 아니라 'SRID=3005;POINT(1011102 450541)'::geometry 와 같이, (하위 쿼리/CTE(common table expression) 안에 있지 않은) 상수일 경우에만 인덱스의 효과가 나타납니다.

실제 사용례를 살펴보려면 OpenGeo workshop: Nearest-Neighbour Searching 을 참조하십시오.

개선 사항: 2.2.0 버전 -- PostgreSQL 9.5 이상 버전에서 도형 및 지리형에 대해 진정한 KNN("K nearest neighbor") 습성을 도입했습니다. 지리형 KNN 이 타원체가 아니라 구체를 기반으로 한다는 점에 주의하십시오. PostgreSQL 9.4 이하 버전에서 지리형을 쓸 수 있긴 하지만, 중심점 상자만 지원합니다.

변경 사항: 2.2.0 버전 -- PostgreSQL 9.5 버전 사용자의 경우, 예전의 하이브리드 문법(Hybrid syntax)이 느릴 수도 있기 때문에 PostGIS 2.2 이상, PostgreSQL 9.5 이상 버전에서만 사용자 코드를 실행하는 경우 이 꼼수를 제거하는 편이 좋을 수도 있습니다. 다음 예시를 참조하세요.

2.0.0 버전부터 사용할 수 있습니다. 약한 KNN 이 실제 거리 대신 도형 중심점 거리에 기반한 최근접 탐색을 제공합니다. 포인트의 경우 정확한 결과를 출력하지만, 다른 모든 유형에 대해서는 부정확한 결과를 냅니다. PostgreSQL 9.1 이상 버전에서 이용할 수 있습니다.

예시

```
SELECT ST_Distance(geom, 'SRID=3005;POINT(1011102 450541)'::geometry) as
d,edabbr, vaabbr
FROM va2005
ORDER BY d limit 10;

       d        | edabbr | vaabbr
------------------+--------+--------
              0 | ALQ    | 128
 5541.57712511724 | ALQ    | 129A
 5579.67450712005 | ALQ    | 001
  6083.4207708641 | ALQ    | 131
  7691.2205404848 | ALQ    | 003
 7900.75451037313 | ALQ    | 122
 8694.20710669982 | ALQ    | 129B
 9564.24289057111 | ALQ    | 130
  12089.665931705 | ALQ    | 127
 18472.5531479404 | ALQ    | 002
(10 rows)
```

그리고 KNN 의 가공되지 않은 답은:

```
SELECT st_distance(geom, 'SRID=3005;POINT(1011102 450541)'::geometry) as
d,edabbr, vaabbr
FROM va2005
ORDER BY geom <-> 'SRID=3005;POINT(1011102 450541)'::geometry limit 10;

       d        | edabbr | vaabbr
------------------+--------+--------
              0 | ALQ    | 128
 5541.57712511724 | ALQ    | 129A
 5579.67450712005 | ALQ    | 001
  6083.4207708641 | ALQ    | 131
  7691.2205404848 | ALQ    | 003
 7900.75451037313 | ALQ    | 122
 8694.20710669982 | ALQ    | 129B
 9564.24289057111 | ALQ    | 130
  12089.665931705 | ALQ    | 127
 18472.5531479404 | ALQ    | 002
(10 rows)
```

사용자가 이 두 쿼리에 대해 "EXPLAIN ANALYZE"를 실행하면 두 번째 쿼리의 성능이 향상되는 것을 볼 수 있을 겁니다.

PostgreSQL 9.5 미만 버전을 실행하는 사용자의 경우, 실제 최근접 거리를 찾으려면 하이브리드 쿼리를 이용하십시오. 먼저 인덱스 지원형 KNN 을 이용한 CTE(common table expression) 쿼리를 실행한 다음, 올바른 정렬 순서를 얻기 위해 정확한 쿼리를 실행합니다:

```
WITH index_query AS (
  SELECT ST_Distance(geom, 'SRID=3005;POINT(1011102 450541)'::geometry) as
d,edabbr, vaabbr
       FROM va2005
  ORDER BY geom <-> 'SRID=3005;POINT(1011102 450541)'::geometry LIMIT 100)
  SELECT *
       FROM index_query
  ORDER BY d limit 10;

       d        | edabbr | vaabbr
------------------+--------+--------
              0 | ALQ    | 128
 5541.57712511724 | ALQ    | 129A
 5579.67450712005 | ALQ    | 001
  6083.4207708641 | ALQ    | 131
  7691.2205404848 | ALQ    | 003
 7900.75451037313 | ALQ    | 122
 8694.20710669982 | ALQ    | 129B
 9564.24289057111 | ALQ    | 130
  12089.665931705 | ALQ    | 127
 18472.5531479404 | ALQ    | 002
(10 rows)
```

참고: ST_DWithin, ST_Distance, <#>

8.8.28. |=|

|=| — A 와 B 사이의 최근접점(closest point of approach)을 잇는 궤도(trajectory)의 거리를 반환합니다.

Synopsis

double precision **|=|**(geometry *A* , geometry *B*);

설명: |=| 연산자는 두 궤도(ST_IsValidTrajectory 참조) 사이의 3 차원 거리를 반환합니다. 이 연산자는 ST_DistanceCPA 함수와 동일하지만, 연산자이기 때문에 (PostgreSQL 9.5.0 이상 버전이 필요한) N 차원 인덱스를 이용하는 최근접(nearest neightbor) 탐색을 실행하는 데 사용할 수 있습니다.



이 피연산자(operand)는 도형에 대해 이용할 수 있을지도 모르는 N 차원 GiST 인덱스를 활용할 것입니다. 연산자가 ORDER BY 절 안에 있을 때만 공간 인덱스를 쓴다는 점에서 공간 인덱스를 이용하는 다른 연산자들과는 다릅니다.



도형 가운데 하나가, 예를 들어 a.geom 이 아니라 'SRID=3005;LINESTRINGM(0 0 0,0 0 1)'::geometry 와 같이, (하위 쿼리/CTE(common table expression) 안에 있지 않은) 상수일 경우에만 인덱스의 효과가 나타납니다.

2.2.0 버전부터 사용할 수 있습니다. PostgreSQL 9.5 이상 버전에서만 인덱스 지원형(index-supported)을 사용할 수 있습니다.

예시

```
-- 글자 그대로의 쿼리 궤도를 psql 변수로 저장합니다...
```

```
\set             qt             'ST_AddMeasure(ST_MakeLine(ST_MakePointM(-
350,300,0),ST_MakePointM(-410,490,0)),10,20)'
-- 쿼리를 실행합니다!
SELECT track_id, dist FROM (
  SELECT track_id, ST_DistanceCPA(tr,:qt) dist
  FROM trajectories
  ORDER BY tr |=| :qt
  LIMIT 5
) foo;
 track_id       dist
----------+-------------------
     395 | 0.576496831518066
     380 |  5.06797130410151
     390 |  7.72262293958322
     385 |   9.8004461358071
     405 |  10.9534397988433
(5 rows)
```

참고: ST_DistanceCPA, ST_ClosestPointOfApproach, ST_IsValidTrajectory

8.8.29. <#>

<#> — A 와 B 의 경계 상자 사이의 2 차원 거리를 반환합니다.

Synopsis

double precision **<#>**(geometry *A* , geometry *B*);

설명: 연산자 <#> 는 두 부동소수점 경계 상자 사이의 거리를 반환합니다. (PostgreSQL 9.1 이상인 경우) 공간 인덱스로부터 경계 상자들을 읽어 올 수도 있습니다. 이는 최근린 거리의 근사치 로 순서를 정렬하는데 유용합니다.



이 피연산자(operand)는 도형에 대해 이용할 수 있을지도 모르는 어떤 인덱스라도 활용할 것입니다. 연산자가 ORDER BY 절 안에 있을 때만 공간 인덱스를 쓴다는 점에서 공간 인덱스를 이용하는 다른 연산자들과는 다릅니다.



도형 가운데 하나가, 예를 들어 g1.geom <#>가 아니라 ORDER BY (ST_GeomFromText('POINT(12)') <#> geom)과 같이, 상수일 경우에만 인덱스의 효과가 나타납니다.

2.0.0 버전부터 사용할 수 있습니다. PostgreSQL 9.1 이상 버전에서만 KNN 을 쓸 수 있습니다.

예시

```
SELECT *
FROM (
SELECT b.tlid, b.mtfcc,
       b.geom <#
> ST_GeomFromText('LINESTRING(746149 2948672,745954 2948576,
             745787 2948499,745740 2948468,745712 2948438,
             745690 2948384,745677 2948319)',2249) As b_dist,
             ST_Distance(b.geom,        ST_GeomFromText('LINESTRING(746149
2948672,745954 2948576,
```

```
             745787 2948499,745740 2948468,745712 2948438,
             745690 2948384,745677 2948319)',2249)) As act_dist
   FROM bos_roads As b
   ORDER BY b_dist, b.tlid
   LIMIT 100) As foo
   ORDER BY act_dist, tlid LIMIT 10;

   tlid    | mtfcc |     b_dist      |    act_dist
-----------+-------+-----------------+-----------------
  85732027 | S1400 |               0 |               0
  85732029 | S1400 |               0 |               0
  85732031 | S1400 |               0 |               0
  85734335 | S1400 |               0 |               0
  85736037 | S1400 |               0 |               0
 624683742 | S1400 |               0 | 128.528874268666
  85719343 | S1400 | 260.839270432962 | 260.839270432962
  85741826 | S1400 | 164.759294123275 | 260.839270432962
  85732032 | S1400 |          277.75 | 311.830282365264
  85735592 | S1400 |          222.25 | 311.830282365264
(10 rows)
```

참고: ST_DWithin, ST_Distance, <->

8.8.30. <<->>

<<->> — A 경계 상자와 B 의 경계 상자의 중심점 사이의 N 차원 거리를 반환합니다.

Synopsis

double precision **<<->>**(geometry *A* , geometry *B*);

설명: <<->> 연산자는 두 도형의 경계 상자의 중심점 사이의 N 차 (유클리드) 거리를 반환합니다. 최근접 근사 거리의 순서를 정렬하는 데 유용합니다.



이 피연산자(operand)는 도형에 대해 이용할 수 있을지도 모르는 N 차원 GiST 인덱스를 활용할 것입니다. 연산자가 ORDER BY 절 안에 있을 때만 공간 인덱스를 쓴다는 점에서 공간 인덱스를 이용하는 다른 연산자들과는 다릅니다.



도형 가운데 하나가, 예를 들어 a.geom 이 아니라 'SRID=3005;POINT(1011102 450541)'::geometry 와 같이, (하위 쿼리/CTE(common table expression) 안에 있지 않은) 상수일 경우에만 인덱스의 효과가 나타납니다.

2.2.0 버전부터 사용할 수 있습니다. PostgreSQL 9.1 이상 버전에서만 KNN 을 쓸 수 있습니다.

참고: <<#>>, <->

8.8.31. <<#>>

<<#>> — A 와 B 의 경계 상자 사이의 N 차원 거리를 반환합니다.

Synopsis

double precision **<<#>>**(geometry *A* , geometry *B*);

설명: 연산자 <<#>> 는 두 부동소수점 경계 상자 사이의 거리를 반환합니다. (PostgreSQL 9.1 이상 버전인 경우)공간 인덱스로부터 경계 상자들을 읽어 올 수도 있습니다. 최근린 거리의 근사치 순서로 정렬하는 데 유용합니다.



이 피연산자(operand)는 도형에 대해 이용할 수 있을지도 모르는 어떤 인덱스라도 활용할 것입니다. 연산자가 ORDER BY 절 안에 있을 때만 공간 인덱스를 쓴다는 점에서 공간 인덱스를 이용하는 다른 연산자들과는 다릅니다.



도형 가운데 하나가, 예를 들어 g1.geom <<#>>가 아니라 ORDER BY (ST_GeomFromText('POINT(1 2)') <<#>> geom)과 같이, 상수일 경우에만 인덱스의 효과가 나타납니다.

2.2.0 버전부터 사용할 수 있습니다. PostgreSQL 9.1 이상 버전에서만 KNN 을 쓸 수 있습니다.

참고: <<->>, <#>

8.9. 공간 관계성 및 측정

ST_3DClosestPoint — g2 에 가장 가까운 g1 상에 있는 3 차원 포인트를 반환합니다. 해당 포인트는 3D 최단 라인의 첫 번째 포인트입니다.

ST_3DDistance — 도형 유형에 대해, 두 도형 사이의 (SRS 에 기반한) 3 차원 데카르트 최단 거리를 투영 단위로 반환합니다.

ST_3DDWithin — 3D(Z) 도형 유형에 대해, 두 도형의 3 차원 거리가 단위의 개수 내일 경우 참을 반환합니다.

ST_3DDFullyWithin — 모든 3D 도형이 서로의 설정된 거리 내에 있을 경우 참을 반환합니다.

ST_3DIntersects — 도형들이 3 차원에서 "공간적으로 교차"하는 경우 참을 반환합니다. 포인트, 라인스트링, 폴리곤, 다면체 표면(면적)만 입력받습니다. SFCGAL 백엔드가 활성화됐다면 TIN 도 지원합니다.

ST_3DLongestLine — 두 도형 사이의 3 차원 최장(longest) 라인을 반환합니다.

ST_3DMaxDistance — 도형 유형에 대해, 두 도형 사이의 (SRS 에 기반한) 3 차원 데카르트 최장 거리를 투영 단위로 반환합니다.

ST_3DShortestLine — 두 도형 사이의 3 차원 최단(shortest) 라인을 반환합니다.

ST_Area — 도형이 폴리곤이나 멀티폴리곤일 경우 표면의 면적을 반환합니다. 도형 유형의 경우, SRID 가 지정한 단위로 2 차원 데카르트 면적을 구합니다. 지리형 유형의 경우, 평방미터 단위로 곡면(curved surface)의 면적을 구합니다.

ST_Azimuth — 포인트 A 와 포인트 B 를 잇는 선의 각도를, 포인트 A 에 그은 수직선을 기준으로 시계 방향으로 라디안 단위로 각도를 측정해서 북극 기준(north-based) 방위각(azimuth)을 반환합니다.

ST_Centroid — 도형의 기하학적 중심을 반환합니다.

ST_ClosestPoint — g2 에 가장 가까운 g1 상에 있는 2 차원 포인트를 반환합니다. 해당 포인트는 최단 라인의 첫 번째 포인트입니다.

ST_ClusterDBSCAN — 각 입력 도형이 있는 클러스터에 대한 정수 ID 를 DBSCAN(Density-Based Spatial Clustering of Applications with Noise) 알고리즘의 2 차원 구현에 기반해서 반환하는, 서로 다른 데이터를 윈도를 통해 동시에 한 화면에 표시하는 함수입니다.

ST_ClusterIntersecting — 종합 함수입니다. 도형들의 집합의 연결된 구성 요소들을 가진 배열을 반환합니다.

ST_ClusterKMeans — 각 입력 도형이 있는 클러스터에 대한 정수 ID 를 반환하는, 서로 다른 데이터를 윈도를 통해 동시에 한 화면에 표시하는 함수입니다.

ST_ClusterWithin — 종합 합수입니다. 도형 집합의 배열을 반환합니다. 이때 도형 집합은 설정된 거리 미만으로 떨어진 도형들의 집합을 나타냅니다.

ST_Contains — A 외부에 놓인 B 의 포인트가 하나도 없고, 적어도 B 내부의 포인트 하나가 A 의 내부에 있는 경우 참을 반환합니다.

ST_ContainsProperly — B 가 A 의 내부와 교차하지만 A 의 경계선(또는 외부)과 교차하지는 않을 경우 참을 반환합니다. A 는 자기 자신을 제대로 담고 있지는 않지만, 자기 자신을 담고 있긴 합니다.

ST_Covers — 도형 B 안에 있는 어떤 포인트도 도형 A 외부에 없을 경우 1(참)을 반환합니다.

ST_CoveredBy — 도형/지리형 A 안에 있는 어떤 포인트도 도형/지리형 B 외부에 없을 경우 1(참)을 반환합니다.

ST_Crosses — 입력 도형들이 전부가 아닌 일부 내부 포인트를 공유하는 경우 TRUE 를 반환합니다.

ST_LineCrossingDirection — 라인스트링 2 개를 입력하면, 어떤 형태로 교차하고 있는지를 나타내는 -3 에서 3 사이의 숫자를 반환합니다. 0 은 교차하지 않음을 뜻합니다.

ST_Disjoint — 도형들이 "공간적으로 교차"하지 않을 경우 - 어떤 공간도 함께 공유하지 않을 경우 - 참을 반환합니다.

ST_Distance — 도형 유형에 대해, 두 도형 사이의 2 차원 데카르트 거리를 투영 단위(SRS 단위)로 반환합니다. 지리형 유형의 경우, 기본적으로 두 도형 사이의 측지 거리를 미터 단위로 반환합니다.

ST_MinimumClearance — 도형의 튼튼함(robustness)의 척도인 도형의 최소 여유(clearance)를 반환합니다.

ST_MinimumClearanceLine — 포인트 2 개로 이루어진, 도형의 최소 여유를 나타내는 라인스트링을 반환합니다.

ST_HausdorffDistance — 두 도형 사이의 하우스도르프 거리를 반환합니다. 기본적으로 두 도형이 얼마나 유사한지 또는 유사하지 않은지에 대한 척도입니다. 두 도형의 공간 참조 시스템 단위를 씁니다.

ST_MaxDistance — 두 도형 사이의 2 차원 최장 거리를 투영 단위로 반환합니다.

ST_DistanceSphere 경위도 도형 2 개 사이의 최단 거리를 미터 단위로 반환합니다. SRID 가 정의하는 회전타원체에서 추출한 반경을 가진 지구 구체를 이용합니다. ST_DistanceSpheroid 보다는 처리 속도가 빠르지만, 정확도는 떨어집니다. PostGIS 1.5 미만 버전에서는 포인트에 대해서만 구현돼 있었습니다.

ST_DistanceSpheroid — 특정 회전타원체가 주어진 두 경위도 도형 사이의 최단 거리를 반환합니다. PostGIS 1.5 미만 버전은 포인트만 지원했습니다.

ST_DFullyWithin — 모든 도형이 서로 설정된 거리 안에 있을 경우 참을 반환합니다.

ST_DWithin — 도형들이 서로 설정된 거리 안에 있을 경우 참을 반환합니다. 도형의 경우 공간 참조의 단위를 쓰고, 지리형의 경우 미터 단위를 씁니다. 또 지리형의 경우 측정시 use_spheroid=true(회전타원체 상에서 측정)가 기본값이며, 더 빨리 검사하려면 구체 상에서 측정하도록 use_spheroid=false 로 설정하십시오.

ST_Equals — 주어진 도형들이 동일한 도형일 경우 참을 반환합니다. 방향성은 무시합니다.

ST_GeometricMedian — 멀티 포인트의 기하학적 중심값(median)을 반환합니다.

ST_HasArc — 도형 또는 도형 집합이 원호 스트링을 담고 있을 경우 참을 반환합니다.

ST_Intersects — 도형/지리형이 "2 차원에서 공간적으로 교차"(공간의 일부라도 공유)할 경우 참을 반환합니다. 그렇지 않을 경우 (도형/지리형이 서로 분리돼 있을 경우) 거짓을 반환합니다. 지리형의 경우 허용 오차가 0.00001 미터입니다(따라서 10 마이크로미터보다 가까운 포인트들은 교차한다고 여깁니다).

ST_Length — 도형이 라인스트링 또는 멀티라인스트링일 경우 도형의 2 차원 길이를 반환합니다. 도형의 단위는 공간 참조 시스템 단위이며 지리형의 단위는 (기본값 회전타원체의) 미터 단위입니다.

ST_Length2D — 도형이 라인스트링 또는 멀티라인스트링일 경우 도형의 2 차원 길이를 반환합니다. 이 함수는 ST_Length 와 동일합니다.

ST_3DLength — 도형이 라인스트링 또는 멀티라인스트링일 경우 도형의 3 차원 또는 2 차원 길이를 반환합니다.

ST_LengthSpheroid — 타원체 상에 있는 도형의 2 차원 또는 3 차원 길이/둘레를 계산합니다. 이 함수는 도형의 좌표가 경도/위도이며 재투영하지 않고 길이를 얻고 싶을 때 유용합니다.

ST_Length2D_Spheroid — 타원체 상에 있는 도형의 2 차원 길이/둘레를 계산합니다. 이 함수는 도형의 좌표가 경도/위도이며 재투영하지 않고 길이를 얻고 싶을 때 유용합니다.

ST_LongestLine 두 도형 사이의 2 차원 최장 라인을 반환합니다. 하나 이상의 최장 라인이 있을 경우, 이 함수는 자신이 찾은 첫 번째 최장 라인만 반환할 것입니다. 반환되는 라인은 항상 g1 에서 시작해서 g2 에서 끝납니다. 이 함수가 반환하는 라인의 길이는 ST_MaxDistance 함수가 g1 과 g2 에 대해 반환하는 길이와 언제나 동일합니다.

ST_OrderingEquals — 주어진 도형들이 동일한 도형이며 포인트들의 순서가 동일한 방향일 경우 참을 반환합니다.

ST_Overlaps — 도형들이 공간을 공유하고, 동일한 차원이지만, 어느 한 쪽이 다른 쪽을 완전히 담고 있지 않을 경우 참을 반환합니다.

ST_Perimeter — ST_Surface 또는 ST_MultiSurface 도형 또는 지리형(폴리곤, 멀티폴리곤)의 경계선의 길이 측정치를 반환합니다. 도형의 단위는 공간 참조 시스템 단위이며 지리형의 단위는 미터 단위입니다.

ST_Perimeter2D — 도형이 폴리곤 또는 멀티폴리곤일 경우 도형의 2 차원 둘레를 반환합니다. 이 함수는 현재 ST_Perimeter 와 동일합니다.

ST_3DPerimeter — 도형이 폴리곤 또는 멀티폴리곤일 경우 도형의 3 차원 둘레를 반환합니다.

ST_PointOnSurface — 표면에 붙어 있게 될 POINT 를 반환합니다.

ST_Project — 시작점으로부터 미터 단위 거리 및 라디안 단위 방향(방위각)을 이용해 투영된 POINT 를 반환합니다.

ST_Relate — 도형의 내부 사이의 교차점을 확인해서 한 도형이 다른 도형과 공간 관계가 있을 경우 참을 반환합니다. 두 도형의 경계선 및 외부는 intersectionMatrixPattern 안의 값들로 설정됩니다. intersectionMatrixPattern 이 입력되지 않을 경우, 두 도형을 관계시키는 최대 intersectionMatrixPattern 을 반환합니다.

ST_RelateMatch — intersectionMattrixPattern1 이 intersectionMattrixPattern2 를 의미할 경우 참을 반환합니다.

ST_ShortestLine — 두 도형 사이의 2 차원 최단 라인을 반환합니다.

ST_Touches — 입력 도형들이 최소한 포인트 한 개를 공유하지만, 내부가 교차하지는 않을 경우 TRUE 를 반환합니다.

ST_Within — 도형 A 전체가 도형 B 안에 들어갈 경우 참을 반환합니다.

8.9.1. ST_3DClosestPoint

ST_3DClosestPoint — g2 에 가장 가까운 g1 상에 있는 3 차원 포인트를 반환합니다. 해당 포인트는 3D 최단 라인의 첫 번째 포인트입니다.

Synopsis

float ST_3DDistance(geometry *g1*, geometry *g2*);

설명: g2 에 가장 가까운 g1 상에 있는 3 차원 포인트를 반환합니다. 해당 포인트는 3D 최단 라인의 첫 번째 포인트입니다. 3D 최단 라인의 3D 길이가 3D 거리입니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 함수는 다면체 표면을 지원합니다.

2.0.0 버전부터 사용할 수 있습니다.

변경 사항: 2.2.0 버전부터 2D 도형 두 개를 입력할 경우, (존재하지 않는 Z 을 0 으로 가정하는 예전 습성 대신) 2D 포인트를 반환합니다. 2D 및 3D 의 경우, 더 이상 Z 가 없을 때 Z 를 0 으로 가정하지 않습니다.

예시

라인스트링과 포인트 -- 3D, 2D 모두의 최근접점(closest point)

```
 SELECT ST_AsEWKT(ST_3DClosestPoint(line,pt)) AS cp3d_line_pt,
         ST_AsEWKT(ST_ClosestPoint(line,pt)) As cp2d_line_pt
     FROM (SELECT 'POINT(100 100 30)'::geometry As pt,
             'LINESTRING (20 80 20, 98 190 1, 110 180 3, 50 75 1000)'::geometry As line
         ) As foo;


  cp3d_line_pt                          |        cp2d_line_pt
 ---------------------------------------+-------------------------------------------
  POINT(54.6993798867619  128.935022917228  11.5475869506606)   |  POINT(73.0769230769231
115.384615384615)
```

라인스트링과 멀티포인트 -- 3D, 2D 모두의 최근접점(closest point)

```
 SELECT ST_AsEWKT(ST_3DClosestPoint(line,pt)) AS cp3d_line_pt,
         ST_AsEWKT(ST_ClosestPoint(line,pt)) As cp2d_line_pt
     FROM (SELECT 'MULTIPOINT(100 100 30, 50 74 1000)'::geometry As pt,
             'LINESTRING (20 80 20, 98 190 1, 110 180 3, 50 75 900)'::geometry As line
         ) As foo;


             cp3d_line_pt             | cp2d_line_pt
 ---------------------------------------+-------------------------------------------
  POINT(54.6993798867619 128.935022917228 11.5475869506606) | POINT(50 75)
```

멀티라인스트링과 폴리곤 -- 3D, 2D 모두의 최근접점(closest point)

```
  SELECT ST_AsEWKT(ST_3DClosestPoint(poly, mline)) As cp3d,
     ST_AsEWKT(ST_ClosestPoint(poly, mline)) As cp2d
        FROM (SELECT  ST_GeomFromEWKT('POLYGON((175 150 5, 20 40 5, 35 45 5,
50 60 5, 100 100 5, 175 150 5))') As poly,
               ST_GeomFromEWKT('MULTILINESTRING((175 155 2, 20 40 20, 50 60
-2, 125 100 1, 175 155 1),
               (1 10 2, 5 20 1))') As mline ) As foo;
                 cp3d       |    cp2d
  ------------------------+-------------------------------
   POINT(39.993580415989 54.1889925532825 5) | POINT(20 40)
```

참고: ST_AsEWKT, ST_ClosestPoint, ST_3DDistance, ST_3DShortestLine

8.9.2. ST_3DDistance

ST_3DDistance — 도형 유형에 대해, 두 도형 사이의 (SRS 에 기반한) 3 차원 데카르트 최단 거리를 투영 단위로 반환합니다.

Synopsis

float ST_3DDistance(geometry *g1*, geometry *g2*);

설명: 도형 유형에 대해, 두 도형 사이의 3 차원 데카르트 최단 거리를 두 도형의 투영 단위(SRS 단위)로 반환합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM ?

✔ SFCGAL 백엔드도 이 방법을 제공합니다.

2.0.0 버전부터 사용할 수 있습니다.

변경 사항: 2.2.0 버전부터, 2D 및 3D 의 경우 더 이상 Z 가 없을 때 Z 를 0 으로 가정하지 않습니다.

예시

```
 -- 도형 예시 - 미터 단위 (SRID 2163 미국 등적 전도) (2D 포인트 및 라인과 비교한 3D
포인트 및 라인)
 -- 주의: 현재 수직 데이터를 지원하지 않으므로 Z 를 변환하지 않고 최종 결과물과 동일한 단위로
가정합니다.
 SELECT ST_3DDistance(
                    ST_Transform(ST_GeomFromEWKT('SRID=4326;POINT(-72.1235
42.3521 4)'),2163),
                    ST_Transform(ST_GeomFromEWKT('SRID=4326;LINESTRING(-
72.1260 42.45 15, -72.123 42.1546 20)'),2163)
              ) As dist_3d,
              ST_Distance(
                    ST_Transform(ST_GeomFromText('POINT(-72.1235
42.3521)',4326),2163),
                    ST_Transform(ST_GeomFromText('LINESTRING(-72.1260
42.45, -72.123 42.1546)', 4326),2163)
              ) As dist_2d;

     dist_3d     |    dist_2d
 ------------------+-----------------
  127.295059324629 | 126.66425605671
 -- 멀티라인스트링과 폴리곤 모두의 3D 및 2D 거리
 -- 3D 최근접점(closest point) 예시와 동일
 SELECT ST_3DDistance(poly, mline) As dist3d,
    ST_Distance(poly, mline) As dist2d
       FROM (SELECT  ST_GeomFromEWKT('POLYGON((175 150 5, 20 40 5, 35 45 5,
50 60 5, 100 100 5, 175 150 5))') As poly,
              ST_GeomFromEWKT('MULTILINESTRING((175 155 2, 20 40 20, 50 60
-2, 125 100 1, 175 155 1),
              (1 10 2, 5 20 1))') As mline ) As foo;
      dist3d      | dist2d
 -------------------+--------
  0.716635696066337 |      0
```

참고: ST_Distance, ST_3DClosestPoint, ST_3DDWithin, ST_3DMaxDistance, ST_3DShortestLine, ST_Transform

8.9.3. ST_3DDWithin

ST_3DDWithin — 3D(Z) 도형 유형에 대해, 두 도형의 3 차원 거리가 단위의 개수 내일 경우 참을 반환합니다.

Synopsis

boolean ST_3DDWithin(geometry *g1*, geometry *g2*, double precision *distance_of_srid*);

설명: 도형 유형에 대해, 두 객체 사이의 3 차원 거리가 투영 단위(SRS 단위)를 설정하는 distance_of_srid 범위 내에 있을 경우 참을 반환합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM ?

2.0.0 버전부터 사용할 수 있습니다.

예시

```
 -- 도형 예시 - 미터 단위 (SRID 2163 미국 등적 전도) (2D 포인트 및 라인과 비교한 3D
포인트 및 라인)
 -- 주의: 현재 수직 데이터를 지원하지 않으므로 Z 를 변환하지 않고 최종 결과물과 동일한 단위로
가정합니다.
 SELECT ST_3DDWithin(
                    ST_Transform(ST_GeomFromEWKT('SRID=4326;POINT(-72.1235
42.3521 4)'),2163),
                    ST_Transform(ST_GeomFromEWKT('SRID=4326;LINESTRING(-
72.1260 42.45 15, -72.123 42.1546 20)'),2163),
                    126.8
               ) As within_dist_3d,
 ST_DWithin(
                    ST_Transform(ST_GeomFromEWKT('SRID=4326;POINT(-72.1235
42.3521 4)'),2163),
                    ST_Transform(ST_GeomFromEWKT('SRID=4326;LINESTRING(-
72.1260 42.45 15, -72.123 42.1546 20)'),2163),
                    126.8
               ) As within_dist_2d;

  within_dist_3d | within_dist_2d
 ----------------+----------------
  f              | t
```

참고: ST_3DDistance, ST_Distance, ST_DWithin, ST_3DMaxDistance, ST_Transform

8.9.4. ST_3DDFullyWithin

ST_3DDFullyWithin — 모든 3D 도형이 서로의 설정된 거리 내에 있을 경우 참을 반환합니다.

Synopsis

boolean ST_3DDFullyWithin(geometry *g1*, geometry *g2*, double precision *distance*);

설명: 3D 도형들이 완전히 서로의 설정된 거리 내에 있을 경우 참을 반환합니다. 이 거리는 도형들의 공간 참조 시스템이 정의한 단위로 설정됩니다. 이 함수가 제대로 동작하려면, 소스 도형들이 둘 다 동일한 SRID 를 가진, 동일한 좌표 투영체여야 합니다.



이 함수를 호출하면 도형에서 이용할 수 있는 모든 인덱스를 활용하는 경계 상자 비교 작업을 자동적으로 포함하게 됩니다.

2.0.0 버전부터 사용할 수 있습니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 함수는 다면체 표면을 지원합니다.

예시

```
  -- 이 예시는 2D 라인/포인트의 궤적과 범위 내에 완전히 들어가는 3D 라인/포인트의 궤적을
비교하여 그 안에 완전히 들어가는 거리는 물론
               -- 완전히 들어가는 범위 및 범위 내의 거리 사이의 차이를 비교합니다.
               SELECT     ST_3DDFullyWithin(geom_a,     geom_b,     10)     as
D3DFullyWithin10, ST_3DDWithin(geom_a, geom_b, 10) as D3DWithin10,
        ST_DFullyWithin(geom_a, geom_b, 20) as D2DFullyWithin20,
        ST_3DDFullyWithin(geom_a, geom_b, 20) as D3DFullyWithin20 from
              (select ST_GeomFromEWKT('POINT(1 1 2)') as geom_a,
              ST_GeomFromEWKT('LINESTRING(1 5 2, 2 7 20, 1 9 100, 14 12 3)')
as geom_b) t1;
  d3dfullywithin10 | d3dwithin10 | d2dfullywithin20 | d3dfullywithin20
 ------------------+-------------+------------------+------------------
  f                | t           | t                | f
```

참고: ST_3DMaxDistance, ST_3DDWithin, ST_DWithin, ST_DFullyWithin

8.9.5. ST_3DIntersects

ST_3DIntersects — 도형들이 3 차원에서 "공간적으로 교차"하는 경우 참을 반환합니다. 포인트, 라인스트링, 폴리곤, 다면체 표면(면적)만 입력받습니다. SFCGAL 백엔드가 활성화됐다면 TIN 도 지원합니다.

Synopsis

boolean ST_3DIntersects(geometry *geomA* , geometry *geomB*);

설명: 겹치기(overlap), 접촉(touch), 범위 내(within) 모두 공간 교차를 의미하는 말입니다. 앞에서 언급한 함수들 가운데 어느 하나라도 참을 반환하는 경우, 해당 도형들도 공간적으로 교차하는 겁니다. 도형들이 교차하는 경우, 분리(disjoint) 함수는 거짓을 반환합니다.

2.0.0 버전부터 사용할 수 있습니다.



이 함수를 호출하면 도형에서 이용할 수 있는 모든 인덱스를 활용하는 경계 상자 비교 작업을 자동적으로 포함하게 됩니다.



TIN 지원의 장점을 취하려면, SFCGAL 백엔드를 활성화해야 합니다. 세션 단계에서 set postgis.backend = sfcgal; 로 설정하면 됩니다. 또는 또는 데이터베이스나 시스템 단계에서도 설정할 수 있습니다. 데이터베이스 단계에서는 ALTER DATABASE gisdb SET postgis.backend = sfcgal; 이라고 설정하면 됩니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

✔ SFCGAL 백엔드도 이 방법을 제공합니다.

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: ?

도형 예시

```
  SELECT ST_3DIntersects(pt, line), ST_Intersects(pt,line)
        FROM (SELECT 'POINT(0 0 2)'::geometry As pt,
              'LINESTRING (0 0 1, 0 2 3 )'::geometry As line) As foo;
   st_3dintersects | st_intersects
  -----------------+---------------
   f               | t
  (1 row)
```

TIN 예시

```
  set postgis.backend = sfcgal;
  SELECT    ST_3DIntersects('TIN(((0    0,1    0,0    1,0    0)))'::geometry,
'POINT(.1 .1)'::geometry);
   st_3dintersects
  -----------------
   t
```

참고: ST_Intersects

8.9.6. ST_3DLongestLine

ST_3DLongestLine — 두 도형 사이의 3 차원 최장(longest) 라인을 반환합니다.

Synopsis

geometry ST_3DLongestLine(geometry *g1*, geometry *g2*);

설명: 두 도형 사이의 3 차원 최장(longest) 라인을 반환합니다. 하나 이상의 최장 라인이 있을 경우, 이 함수는 첫 번째 최장 라인만 반환할 것입니다. 반환되는 라인은 항상 g1 에서 시작해서 g2 에서 끝납니다. 이 함수가 반환하는 라인의 3 차원 길이는 ST_3DMaxDistance 함수가 g1 과 g2 에 대해 반환하는 길이와 언제나 동일합니다.

2.0.0 버전부터 사용할 수 있습니다.

변경 사항: 2.2.0 버전부터 2D 도형 두 개를 입력할 경우, (존재하지 않는 Z 을 0 으로 가정하는 예전 습성 대신) 2D 포인트를 반환합니다. 2D 및 3D 의 경우, 더 이상 Z 가 없을 때 Z 를 0 으로 가정하지 않습니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 함수는 다면체 표면을 지원합니다.

예시

```
  라인스트링과 포인트 -- 3D, 2D 모두의 최장 라인
  SELECT ST_AsEWKT(ST_3DLongestLine(line,pt)) AS lol3d_line_pt,
              ST_AsEWKT(ST_LongestLine(line,pt)) As lol2d_line_pt
        FROM (SELECT 'POINT(100 100 30)'::geometry As pt,
                     'LINESTRING (20 80 20, 98 190 1, 110 180 3, 50 75
1000)'::geometry As line
              ) As foo;


          lol3d_line_pt         |      lol2d_line_pt
  ------------------------------------+----------------------------
   LINESTRING(50 75 1000,100 100 30) | LINESTRING(98 190,100 100)

  라인스트링과 멀티포인트 -- 3D, 2D 모두의 최장 라인
  SELECT ST_AsEWKT(ST_3DLongestLine(line,pt)) AS lol3d_line_pt,
              ST_AsEWKT(ST_LongestLine(line,pt)) As lol2d_line_pt
        FROM (SELECT 'MULTIPOINT(100 100 30, 50 74 1000)'::geometry As pt,
                     'LINESTRING (20 80 20, 98 190 1, 110 180 3, 50 75
900)'::geometry As line
              ) As foo;


          lol3d_line_pt        |     lol2d_line_pt
  ---------------------------------+-------------------------
   LINESTRING(98 190 1,50 74 1000) | LINESTRING(98 190,50 74)

  멀티라인스트링과 폴리곤 -- 3D, 2D 모두의 최장 라인
  SELECT ST_AsEWKT(ST_3DLongestLine(poly, mline)) As lol3d,
     ST_AsEWKT(ST_LongestLine(poly, mline)) As lol2d
        FROM (SELECT  ST_GeomFromEWKT('POLYGON((175 150 5, 20 40 5, 35 45 5,
50 60 5, 100 100 5, 175 150 5))') As poly,
              ST_GeomFromEWKT('MULTILINESTRING((175 155 2, 20 40 20, 50 60
-2, 125 100 1, 175 155 1),
              (1 10 2, 5 20 1))') As mline ) As foo;
           lol3d           |        lol2d
  -----------------------------+-------------------------
   LINESTRING(175 150 5,1 10 2) | LINESTRING(175 150,1 10)
```

참고: ST_3DClosestPoint, ST_3DDistance, ST_LongestLine, ST_3DShortestLine, ST_3DMaxDistance

8.9.7. ST_3DMaxDistance

ST_3DMaxDistance — 도형 유형에 대해, 두 도형 사이의 (SRS 에 기반한) 3 차원 데카르트 최장 거리를 투영 단위로 반환합니다.

Synopsis

float ST_3DMaxDistance(geometry *g1*, geometry *g2*);

설명: 도형 유형에 대해, 두 도형 사이의 3 차원 데카르트 최대 거리를 두 도형의 투영 단위(SRS 단위)로 반환합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 함수는 다면체 표면을 지원합니다.

2.0.0 버전부터 사용할 수 있습니다.

변경 사항: 2.2.0 버전부터, 2D 및 3D 의 경우 더 이상 Z 가 없을 때 Z 를 0 으로 가정하지 않습니다.

예시

```
 -- 도형 예시 - 미터 단위 (SRID 2163 미국 등적 전도) (2D 포인트 및 라인과 비교한 3D
포인트 및 라인)
 -- 주의: 현재 수직 데이터를 지원하지 않으므로 z 를 변환하지 않고 최종 결과물과 동일한 단위로
가정합니다.
 SELECT ST_3DMaxDistance(
                    ST_Transform(ST_GeomFromEWKT('SRID=4326;POINT(-72.1235
42.3521 10000)'),2163),
                    ST_Transform(ST_GeomFromEWKT('SRID=4326;LINESTRING(-
72.1260 42.45 15, -72.123 42.1546 20)'),2163)
                ) As dist_3d,
                ST_MaxDistance(
                    ST_Transform(ST_GeomFromEWKT('SRID=4326;POINT(-72.1235
42.3521 10000)'),2163),
                    ST_Transform(ST_GeomFromEWKT('SRID=4326;LINESTRING(-
72.1260 42.45 15, -72.123 42.1546 20)'),2163)
                ) As dist_2d;

     dist_3d     |    dist_2d
 ----------------+------------------
  24383.7467488441 | 22247.8472107251
```

참고: ST_Distance, ST_3DDWithin, ST_3DMaxDistance, ST_Transform

8.9.8. ST_3DShortestLine

ST_3DShortestLine — 두 도형 사이의 3 차원 최단(shortest) 라인을 반환합니다.

Synopsis

geometry ST_3DShortestLine(geometry *g1*, geometry *g2*);

설명: 두 도형 사이의 3 차원 최단(shortest) 라인을 반환합니다. 하나 이상의 최단 라인이 있을 경우, 이 함수는 첫 번째 최단 라인만 반환할 것입니다. g1과 g2가 단 한 개의 포인트에서만 교차할 경우, 이 함수는 교차점에서 시작하고 끝나는 라인을 반환할 것입니다. g1과 g2가 한 개 이상의 포인트에서 교차할 경우, 이 함수는 동일한 포인트에서 시작하고 끝나는 라인을 반환하지만 해당 포인트는 교차하는 포인트들 가운데 어떤 포인트라도 될 수 있습니다. 반환되는 라인은 항상 g1 에서 시작해서 g2 에서 끝납니다. 이 함수가 반환하는 라인의 3 차원 길이는 ST_3DDistance 함수가 g1 과 g2 에 대해 반환하는 길이와 언제나 동일합니다.

2.0.0 버전부터 사용할 수 있습니다.

변경 사항: 2.2.0 버전부터 2D 도형 두 개를 입력할 경우, (존재하지 않는 Z 을 0 으로 가정하는 예전 습성 대신) 2D 포인트를 반환합니다. 2D 및 3D 의 경우, 더 이상 Z 가 없을 때 Z 를 0 으로 가정하지 않습니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 함수는 다면체 표면을 지원합니다.

예시

라인스트링과 포인트 -- 3D, 2D 모두의 최단 라인

```
  SELECT ST_AsEWKT(ST_3DShortestLine(line,pt)) AS shl3d_line_pt,
                ST_AsEWKT(ST_ShortestLine(line,pt)) As shl2d_line_pt
        FROM (SELECT 'POINT(100 100 30)'::geometry As pt,
                     'LINESTRING (20 80 20, 98 190 1, 110 180 3, 50 75
1000)'::geometry As line
                ) As foo;


   shl3d_line_pt                                    |        shl2d_line_pt
  ------------------------------------------------+-------------------------------------
  LINESTRING(54.6993798867619   128.935022917228   11.5475869506606,100   100   30)      |
LINESTRING(73.0769230769231 115.384615384615,100 100)
```

라인스트링과 멀티포인트 -- 3D, 2D 모두의 최단 라인

```
  SELECT ST_AsEWKT(ST_3DShortestLine(line,pt)) AS shl3d_line_pt,
                ST_AsEWKT(ST_ShortestLine(line,pt)) As shl2d_line_pt
        FROM (SELECT 'MULTIPOINT(100 100 30, 50 74 1000)'::geometry As pt,
                     'LINESTRING (20 80 20, 98 190 1, 110 180 3, 50 75
900)'::geometry As line
                ) As foo;


             shl3d_line_pt                     | shl2d_line_pt
  ---------------------------------------------+-----------------------------------------
  LINESTRING(54.6993798867619 128.935022917228 11.5475869506606,100 100 30) | LINESTRING(50
75,50 74)
```

멀티라인스트링과 폴리곤 -- 3D, 2D 모두의 최단 라인

```
  SELECT ST_AsEWKT(ST_3DShortestLine(poly, mline)) As shl3d,
     ST_AsEWKT(ST_ShortestLine(poly, mline)) As shl2d
        FROM (SELECT  ST_GeomFromEWKT('POLYGON((175 150 5, 20 40 5, 35 45 5,
50 60 5, 100 100 5, 175 150 5))') As poly,
               ST_GeomFromEWKT('MULTILINESTRING((175 155 2, 20 40 20, 50 60
-2, 125 100 1, 175 155 1),
               (1 10 2, 5 20 1))') As mline ) As foo;
            shl3d                                |  shl2d
 ----------------------------------------------+-----------------------------------------------------------------
  LINESTRING(39.993580415989 54.1889925532825 5,40.4078575708294 53.6052383805529 5.03423778139177) | LINESTRING(20
40,20 40)
```

참고: ST_3DClosestPoint, ST_3DDistance, ST_LongestLine, ST_ShortestLine, ST_3DMaxDistance

8.9.9. ST_Area

ST_Area — 도형이 폴리곤이나 멀티폴리곤일 경우 표면의 면적을 반환합니다. 도형 유형의 경우, SRID 가 지정한 단위로 2 차원 데카르트 면적을 구합니다. 지리형 유형의 경우, 평방미터 단위로 곡면(curved surface)의 면적을 구합니다.

Synopsis

float ST_Area(geometry *g1*);

float ST_Area(geography *geog*, boolean *use_spheroid=true*);

설명: 도형이 폴리곤이나 멀티폴리곤일 경우 표면의 면적을 - ST_Surface 또는 ST_MultiSurface 함수의 면적 측정값을 - 반환합니다. 도형 유형의 경우, SRID 가 지정한 단위로 2 차원 데카르트 면적을 구합니다. 지리형 유형의 경우, 평방미터 단위로 곡면(curved surface)의 면적을 구합니다. 구체의 면적에 대해 덜 정확하지만 더 빠르게 측정하려면, ST_Area(geog,false) 함수를 이용하십시오.

개선 사항: 2.0.0 버전부터 2 차원 다면체 표면(polyhedral surface)을 지원합니다.

개선 사항: 2.2.0 버전부터 회전타원체 측정시 정확도와 강력함을 향상시키기 위해 GeographicLib 을 이용합니다. 이 새 기능의 장점을 취하려면 Proj 4.9.0 이상 버전이 필요합니다.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다.

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 8.1.2, 9.5.3

✔ 이 함수는 다면체 표면을 지원합니다.



다면체 표면의 경우, (2.5 차원이 아니라) 2 차원 다면체 표면만 지원합니다. 2.5 차원일 경우 0 이 아닌(non-zero) 답을 내놓을 수도 있지만, XY 평면상에 완벽하게 놓이는 면들의 면적만 반환하는 것입니다.

✔ SFCGAL 백엔드도 이 방법을 제공합니다.

예시

매사추세츠 토지의 부지(plot)에 대해 평방피트 단위로 면적을 반환하고, 이에 환산값을 곱해서 평방미터 면적을 구합니다. EPSG:2249 가 매사추세츠 주 피트 단위 평면이기 때문에 평방피트 단위의 값을 반환한다는 점에 주의하십시오.

```
  SELECT ST_Area(the_geom) As sqft, ST_Area(the_geom)*POWER(0.3048,2) As sqm
              FROM (SELECT
              ST_GeomFromText('POLYGON((743238 2967416,743238 2967450,
                    743265        2967450,743265.625        2967416,743238
2967416))',2249) ) As foo(the_geom);
   sqft   |    sqm
  ---------+-------------
   928.625 | 86.27208552
```

평방피트 면적을 반환하며, 평방미터 값을 구하려면 매사추세츠 주 미터 단위 평면((EPSG:26986))으로 변환해야 합니다. EPSG:2249 가 매사추세츠 주 피트 단위 평면이기 때문에 평방피트 단위의 값을 반환하고 EPSG:26986 이 매사추세츠 주 미터 단위 평면이기 때문에 변환된 면적이 평방미터 단위가 된다는 점에 주의하십시오.

```
  SELECT ST_Area(the_geom) As sqft, ST_Area(ST_Transform(the_geom,26986)) As
sqm
              FROM (SELECT
              ST_GeomFromText('POLYGON((743238 2967416,743238 2967450,
                    743265        2967450,743265.625        2967416,743238
2967416))',2249) ) As foo(the_geom);
   sqft   |      sqm
  --------+------------------
   928.625 | 86.2724304199219
```

지리형 데이터 유형을 이용해서 평방피트 및 평방미터 면적을 반환합니다. 도형을 지리형으로 변환시킨다는 점에 주의하십시오(변환하기 전에 사용자 도형의 투영체가 WGS84 4326 경위도 좌표계인지 확인해야 합니다). 지리형은 항상 미터 단위로 측정됩니다. 이 예시는 그저 비교해보기 위한 보여주기에 불과합니다. 일반적으로 이미 사용자 테이블에 지리형 데이터 유형으로 저장되어 있을 겁니다.

```
  SELECT      ST_Area(the_geog)/POWER(0.3048,2)      As      sqft_spheroid,
ST_Area(the_geog,false)/POWER(0.3048,2) As sqft_sphere, ST_Area(the_geog) As
sqm_spheroid
              FROM (SELECT
              geography(
              ST_Transform(
                    ST_GeomFromText('POLYGON((743238        2967416,743238
2967450,743265 2967450,743265.625 2967416,743238 2967416))',
                          2249
                          ) ,4326
                    )
              )
        ) As foo(the_geog);
   sqft_spheroid   |   sqft_sphere    |   sqm_spheroid
  -----------------+------------------+------------------
   928.684403538925 | 927.049336105925 | 86.2776042893529

   -- 사용자 데이터가 이미 지리형인 경우
   SELECT  ST_Area(the_geog)/POWER(0.3048,2)  As   sqft,  ST_Area(the_geog)  As
sqm
        FROM somegeogtable;
```

참고: ST_GeomFromText, ST_GeographyFromText, ST_SetSRID, ST_Transform

8.9.10. ST_Azimuth

ST_Azimuth — 포인트 A 와 포인트 B 를 잇는 선의 각도를, 포인트 A 에 그은 수직선을 기준으로 시계 방향으로 라디안 단위로 각도를 측정해서 북극 기준(north-based) 방위각(azimuth)을 반환합니다.

Synopsis

float ST_Azimuth(geometry *pointA*, geometry *pointB*);

float ST_Azimuth(geography *pointA*, geography *pointB*);

설명: 주어진 포인트 도형으로 정의된 구간의 라디안 단위로 방위각을 반환합니다. 두 포인트가 일치하는 경우 NULL 을 반환합니다. 방위각이란 북쪽을 기준으로, 시계 방향으로 양수인 각도를 말합니다. North = 0; East = $\pi/2$; South = π; West = $3\pi/2$

지리형 유형의 경우, 측지 역 문제(geodesic inverse problem)의 일부로 전방 방위각(forward azimuth)을 구합니다.

방위각은 참조 평면과 포인트 사이의 라디안 단위의 각도로 정의되는 수학적 개념입니다. PostgreSQL 내장 함수인 degrees()를 써서 라디안 단위를 도 단위로 변환할 수 있습니다. 예시 코드를 확인해보십시오.

1.1.0 버전부터 사용할 수 있습니다.

개선 사항: 2.0.0 버전부터 지리형을 지원합니다.

개선 사항: 2.2.0 버전부터 회전타원체 측정시 정확도와 강력함을 향상시키기 위해 GeographicLib 을 이용합니다. 이 새 기능의 장점을 취하려면 Proj 4.9.0 이상 버전이 필요합니다.

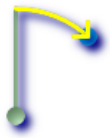
객체를 해당 객체의 수직축을 따라 이동시키기 위해 ST_Translate 함수와 방위각을 함께 이용할 때 방위각이 특히 유용합니다. 이런 내용의 예시를 보려면 Plpgsqlfunctions PostGIS wiki section 의 upgis_lineshift 단원을 살펴보십시오.

예시

도 단위의 도형 방위각

```
  SELECT degrees(ST_Azimuth(ST_Point(25, 45), ST_Point(75, 100))) AS degA_B,
           degrees(ST_Azimuth(ST_Point(75,  100),  ST_Point(25,  45)))  AS
degB_A;

      dega_b      |     degb_a
  ----------------+------------------
   42.2736890060937 | 222.273689006094
```

 *초록색: 시작점 Point(25,45)와 그 수직선 노란색: degA_B 의 궤적 경로(방위각)*	 *초록색: 시작점 Point(75,100)와 그 수직선 노란색: degB_A 의 궤적 경로(방위각)*

참고: ST_Point, ST_Translate, ST_Project, PostgreSQL Math Functions

8.9.11. ST_Centroid

ST_Centroid — 도형의 기하학적 중심을 반환합니다.

Synopsis

geometry ST_Centroid(geometry *g1*);

설명: 도형의 기하학적 중심, 또는 다른 말로, 도형의 질량 중심을 POINT 로 계산합니다. [MULTI]POINT 의 경우, 입력 좌표들의 산술 평균으로 계산합니다. [MULTI]LINESTRING 의 경우, 각 라인 선분의 가중 길이로 계산합니다. [MULTI]POLYGON 의 경우, 면적이라는 관점에서 "가중치(weight)"를 다룹니다. 텅 빈 도형을 입력할 경우, 텅 빈 GEOMETRYCOLLECTION 을 반환합니다. NULL 을 입력하면 NULL 을 반환합니다. CIRCULARSTRING 또는 COMPOUNDCURVE 를 입력할 경우, 먼저 CurveToLine 을 통해 라인스트링으로 변환한 다음, LINESTRING 에 대해서도 동일한 작업을 합니다.

개선 사항: 2.3.0 버전부터 (CurveToLine 을 이용해서) CIRCULARSTRING 및 COMPOUNDCURVE 를 지원합니다.

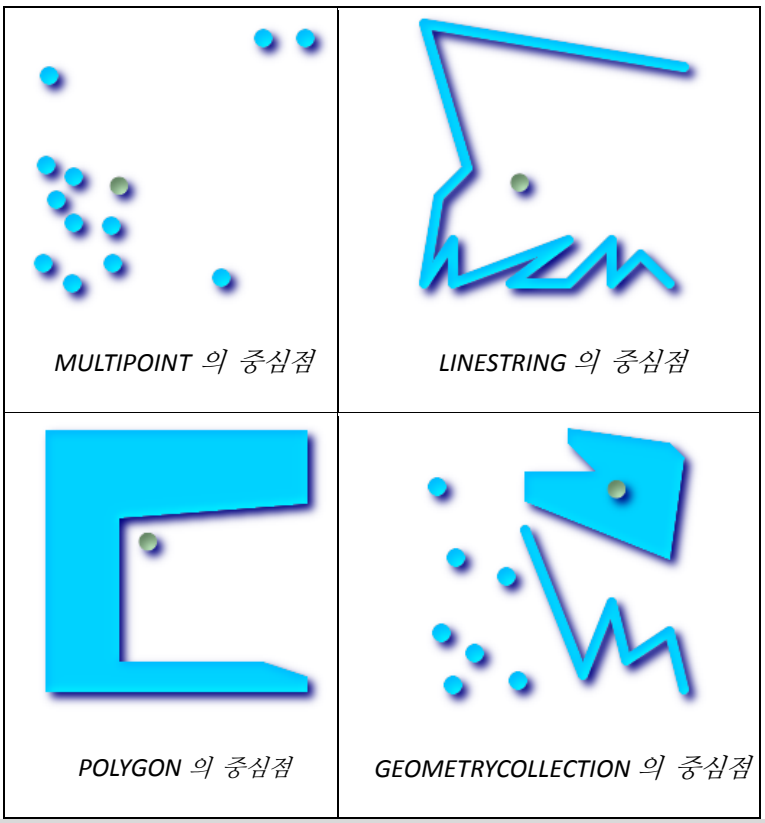
중심점은 가장 높은 차원의 구성 도형들의 집합의 중심점과 동일합니다(낮은 차원의 도형은 중심점에 가중치를 전혀 주지 않기 때문입니다).

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다.

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 8.1.4, 9.5.5

예시

다음 그림들 각각에서, 파란색 점이 입력 도형의 중심점을 나타냅니다.




```
  SELECT ST_AsText(ST_Centroid('MULTIPOINT ( -1 0, -1 2, -1 3, -1 4, -1 7, 0
1, 0 3, 1 1, 2 0, 6 0, 7 8, 9 8, 10 6 )'));
                            st_astext
  ------------------------------------------
   POINT(2.30769230769231 3.30769230769231)
  (1 row)

  SELECT ST_AsText(ST_centroid(g))
  FROM  ST_GeomFromText('CIRCULARSTRING(0 2, -1 1,0 0, 0.5 0, 1 0, 2 1, 1 2,
0.5 2, 0 2)')  AS g ;
  ----------------------------------------
  POINT(0.5 1)


  SELECT ST_AsText(ST_centroid(g))
  FROM  ST_GeomFromText('COMPOUNDCURVE(CIRCULARSTRING(0 2, -1 1,0 0),(0 0,
0.5 0, 1 0),CIRCULARSTRING( 1 0, 2 1, 1 2),(1 2, 0.5 2, 0 2))' ) AS g;
  ----------------------------------------
  POINT(0.5 1)
```

참고: ST_PointOnSurface

8.9.12. ST_ClosetPoint

ST_ClosestPoint — g2 에 가장 가까운 g1 상에 있는 2 차원 포인트를 반환합니다. 해당 포인트는 최단 라인의 첫 번째 포인트입니다.

Synopsis

geometry ST_ClosestPoint(geometry *g1*, geometry *g2*);

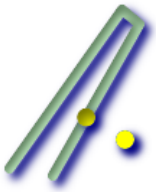
설명: g2 에 가장 가까운 g1 상에 있는 2 차원 포인트를 반환합니다. 해당 포인트는 최단 라인의 첫 번째 포인트입니다.



3 차원 도형의 경우엔 ST_3DClosestPoint 함수를 이용하는 편이 좋습니다.

1.5.0 버전부터 사용할 수 있습니다.

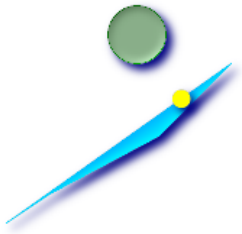
예시



포인트와 라인스트링 사이의 최근접점은 해당 포인트 자체이지만, 라인스트링과 포인트 사이의 최근접점은 라인 상에 있는 가장 가까운 포인트입니다.

```
SELECT ST_AsText(ST_ClosestPoint(pt,line))
 AS cp_pt_line,
   ST_AsText(ST_ClosestPoint(line,pt))
 As cp_line_pt
FROM (SELECT 'POINT(100 100)'::geometry As pt,
        'LINESTRING (20 80, 98 190, 110 180, 50 75 )':
    ) As foo;


 cp_pt_line      |        cp_line_pt
-----------------+-----------------------------------
POINT(100 100) | POINT(73.0769230769231 115.384615384615)
```



폴리곤 B 에 가장 가까운 폴리곤 A 상의 포인트

```
SELECT ST_AsText(
        ST_ClosestPoint(
            ST_GeomFromText('POLYGON((175 150, 20 40,
50 60, 125 100, 175 150))'),
            ST_Buffer(ST_GeomFromText('POINT(110 170)'
), 20)
            )
        ) As ptwkt;

         ptwkt
----------------------------------------
POINT(140.752120669087 125.695053378061)
```

참고: ST_3DClosestPoint,ST_Distance, ST_LongestLine, ST_ShortestLine, ST_MaxDistance

8.9.13. ST_ClusterDBSCAN

ST_ClusterDBSCAN — 각 입력 도형이 있는 클러스터에 대한 정수 ID 를 DBSCAN(Density-Based Spatial Clustering of Applications with Noise) 알고리즘의 2 차원 구현에 기반해서 반환하는, 서로 다른 데이터를 윈도를 통해 동시에 한 화면에 표시하는 함수입니다.

Synopsis

integer ST_ClusterDBSCAN(geometry winset *geom*, float8 *eps*, integer *minpoints*);

설명: 각 입력 도형의 클러스터 번호를, Density-based spatial clustering of applications with noise (DBSCAN) 알고리듬의 2 차원 구현에 기반해서 반환합니다. ST_ClusterKMeans 와 달리 클러스터의 개수를 지정할

필요는 없지만, 대신 각 클러스터를 작성하기 위한 적당한 거리(eps) 및 밀도(minpoints) 파라미터를 이용합니다.

입력 도형이 다음과 같은 경우 클러스터에 해당 도형을 추가할 것입니다:

- 적어도 minpoints 밀도의 다른 입력 도형들에서 eps 거리 안에 있는 "핵심" 도형이거나
- 핵심 도형에서 eps 거리 안에 있는 "경계(border)" 도형인 경우

경계 도형이 하나 이상의 클러스터에서 핵심 도형으로부터 eps 거리 안에 있을 수도 있다는 점에 주의하십시오. 이번 경우, 어느 쪽으로 할당해도 정확하며, 경계 도형은 가능한 클러스터 가운데 하나에 임의적으로 할당될 것입니다. 이런 경우에, 정확한 클러스터가 minpoints 밀도보다 적은 도형들과 함께 생성될 수도 있습니다. 경계 도형을 어느 쪽에 할당할지 모호한 경우, 윈도 정의에 ORDER BY 절이 포함돼 있다면 ST_ClusterDBSCAN 함수를 반복 호출했을 때 동일한 결과를 낼 것이지만, 클러스터 할당이 동일 알고리즘의 다른 구현과 달라질 수도 있습니다.

다른 어떤 클러스터에도 들어갈 기준을 만족하지 못 하는 입력 도형은 번호가 NULL 인 클러스터에 할당될 것입니다.

2.3.0 버전부터 사용할 수 있습니다. GEOS 가 필요합니다.

예시

각 획지 포인트에 클러스터 번호를 할당합니다:

```
 SELECT parcel_id, ST_ClusterDBSCAN(geom, eps := 0.5, minpoints := 5) over
() AS cid
 FROM parcels;
```

동일한 클러스터 번호를 가진 획지들을 단일 도형으로 통합합니다. 이번에는 지정한 인수 호출을 이용합니다.

```
 SELECT cid, ST_Collect(geom) AS cluster_geom, array_agg(parcel_id) AS ids_in_cluster FROM
(
   SELECT parcel_id, ST_ClusterDBSCAN(geom, eps := 0.5, minpoints := 5) over () AS cid, geom
   FROM parcels) sq
 GROUP BY cid;
```

참고: ST_ClusterKMeans, ST_ClusterIntersecting, ST_ClusterWithin

8.9.14. ST_ClusterIntersecting

ST_ClusterIntersecting — 종합 함수입니다. 도형들의 집합의 연결된 구성 요소들을 가진 배열을 반환합니다.

Synopsis

geometry[] ST_ClusterIntersecting(geometry set *g*);

설명: ST_ClusterIntersecting 은 도형 집합의 배열을 반환하는 종합 함수입니다. 이때 도형 집합은 도형들의 상호 연결된 집합을 나타냅니다.

2.2.0 버전부터 사용할 수 있습니다. GEOS 가 필요합니다.

예시

```
WITH testdata AS
 (SELECT unnest(ARRAY['LINESTRING (0 0, 1 1)'::geometry,
            'LINESTRING (5 5, 4 4)'::geometry,
            'LINESTRING (6 6, 7 7)'::geometry,
            'LINESTRING (0 0, -1 -1)'::geometry,
            'POLYGON ((0 0, 4 0, 4 4, 0 4, 0 0))'::geometry]) AS geom)

SELECT ST_AsText(unnest(ST_ClusterIntersecting(geom))) FROM testdata;

-- 결과

st_astext
---------
GEOMETRYCOLLECTION(LINESTRING(0 0,1 1),LINESTRING(5 5,4 4),LINESTRING(0 0,-1 -1),POLYGON((0
0,4 0,4 4,0 4,0 0)))
GEOMETRYCOLLECTION(LINESTRING(6 6,7 7))
```

참고: ST_ClusterDBSCAN, ST_ClusterKMeans, ST_ClusterWithin

8.9.15. ST_ClusterKMeans

ST_ClusterKMeans — 각 입력 도형이 있는 클러스터에 대한 정수 ID 를 반환하는, 서로 다른 데이터를 윈도를 통해 동시에 한 화면에 표시하는 함수입니다.

Synopsis

integer ST_ClusterKMeans(geometry winset *geom*, integer *number_of_clusters*);

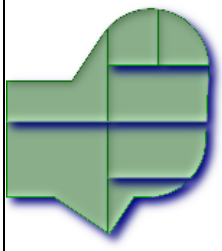
설명: 각 입력 도형에 대해 k-means 클러스터 번호에 기반한 2 차원 거리를 반환합니다. 클러스터 작업에 쓰이는 거리는 도형들의 중심점 사이의 거리입니다.

2.3.0 버전부터 사용할 수 있습니다. GEOS 가 필요합니다.

예시

예시용 가짜 획지 집합을 생성

```
CREATE TABLE parcels AS
SELECT lpad((row_number() over())::text,3,'0') As parcel_id, geom,
('{residential, commercial}'::text[])[1 + mod(row_number()OVER(),2)] As type
FROM
  ST_Subdivide(ST_Buffer('LINESTRING(40 100, 98 100, 100 150, 60 90)'::geometry,
  40, 'endcap=square'),12) As geom;
```



원래 획지



클러스터 번호(cid)를 기준으로 색상 구분된 획지

```
SELECT ST_ClusterKMeans(geom, 5) OVER() AS cid, parcel_id, geom
FROM parcels;
-- 결과
 cid | parcel_id | geom
-----+-----------+--------------
  0 | 001    | 0103000000...
  0 | 002    | 0103000000...
  1 | 003    | 0103000000...
  0 | 004    | 0103000000...
  1 | 005    | 0103000000...
  2 | 006    | 0103000000...
  2 | 007    | 0103000000...
(7 rows)
```

```
-- 유형을 기준으로 획지 클러스터 분할하기
SELECT ST_ClusterKMeans(geom,3) over (PARTITION BY type) AS cid, parcel_id, type
FROM parcels;
-- 결과
 cid | parcel_id |   type
-----+-----------+-------------
  1 | 005    | commercial
  1 | 003    | commercial
  2 | 007    | commercial
  0 | 001    | commercial
  1 | 004    | residential
  0 | 002    | residential
  2 | 006    | residential
(7 rows)
```

참고: ST_ClusterDBSCAN, ST_ClusterIntersecting, ST_ClusterWithin, ST_Subdivide

8.9.16. ST_ClusterWithin

ST_ClusterWithin — 종합 함수입니다. 도형 집합의 배열을 반환합니다. 이때 도형 집합은 설정된 거리 미만으로 떨어진 도형들의 집합을 나타냅니다.

Synopsis

geometry[] ST_ClusterWithin(geometry set *g*, float8 *distance*);

설명: ST_ClusterWithin 은 도형 집합의 배열을 반환하는 종합 함수입니다. 이때 도형 집합은 설정된 거리 미만으로 떨어진 도형들의 집합을 나타냅니다.

2.2.0 버전부터 사용할 수 있습니다. GEOS 가 필요합니다.

예시

```
 WITH testdata AS
  (SELECT unnest(ARRAY['LINESTRING (0 0, 1 1)'::geometry,
              'LINESTRING (5 5, 4 4)'::geometry,
              'LINESTRING (6 6, 7 7)'::geometry,
              'LINESTRING (0 0, -1 -1)'::geometry,
              'POLYGON ((0 0, 4 0, 4 4, 0 4, 0 0))'::geometry]) AS geom)

 SELECT ST_AsText(unnest(ST_ClusterWithin(geom, 1.4))) FROM testdata;

 -- 결과

 st_astext
 ---------
 GEOMETRYCOLLECTION(LINESTRING(0 0,1 1),LINESTRING(5 5,4 4),LINESTRING(0 0,-1 -1),POLYGON((0
0,4 0,4 4,0 4,0 0)))
 GEOMETRYCOLLECTION(LINESTRING(6 6,7 7))
```

참고: ST_ClusterDBSCAN, ST_ClusterKMeans, ST_ClusterIntersecting

8.9.17. ST_Contains

ST_Contains — A 외부에 놓인 B 의 포인트가 하나도 없고, 적어도 B 내부의 포인트 하나가 A 의 내부에 있는 경우 참을 반환합니다.

Synopsis

boolean ST_Contains(geometry *geomA*, geometry *geomB*);

설명: 도형 A 외부에 놓인 도형 B 의 포인트가 하나도 없고, 적어도 B 내부의 포인트 하나가 A 의 내부에 있을 때 도형 A 가 도형 B 를 담고 있다고 합니다. 이 정의의 중요한 섬세성은 A 가 자기 자신을 담고는 있지만, A 가 자신의 경계선을 담고 있지는 않다는 점입니다. 도형 A 가 자기 자신을 제대로 담고 있지 않은 ST_ContainsProperly 의 경우와 대조해보십시오.

도형 B 가 도형 A 안에 완전히 들어 있을 경우 참을 반환합니다. 이 함수가 제대로 동작하려면, 입력 도형들 둘 다 동일한 좌표 투영체, 즉 동일한 SRID 를 가져야 합니다. ST_Contains 는 ST_Within 의 역함수입니다. 따라서 ST_Contains(A,B)와 ST_Within(B,A)는 동일합니다. 다만 정의에 상관없이 또는 정의되지 않아 언제나 거짓을 반환하는 유효하지 않은 도형의 경우를 제외하면 말이죠.

GEOS 모듈로 실행

개선 사항: 2.3.0 버전에서 PIP 단락(short-circuit)이 포인트가 몇 개 없는 멀티포인트를 지원하도록 확장/개선됐습니다. 예전 버전은 폴리곤 내부에 있는 포인트만 지원했습니다.

 GEOMETRYCOLLECTION 을 인수로 호출하지 마십시오.

 이 함수에 유효하지 않은 도형을 입력하지 마십시오. 결과를 예측할 수 없습니다.

이 함수를 호출하면 도형에서 이용할 수 있는 모든 인덱스를 활용하는 경계 상자 비교 작업을 자동적으로 포함하게 됩니다. 인덱스 활용을 피하려면, _ST_Contains 함수를 이용하십시오.

주의: 이 함수는 정수가 아니라 불값을 반환하는, "사용할 만한" 버전입니다.

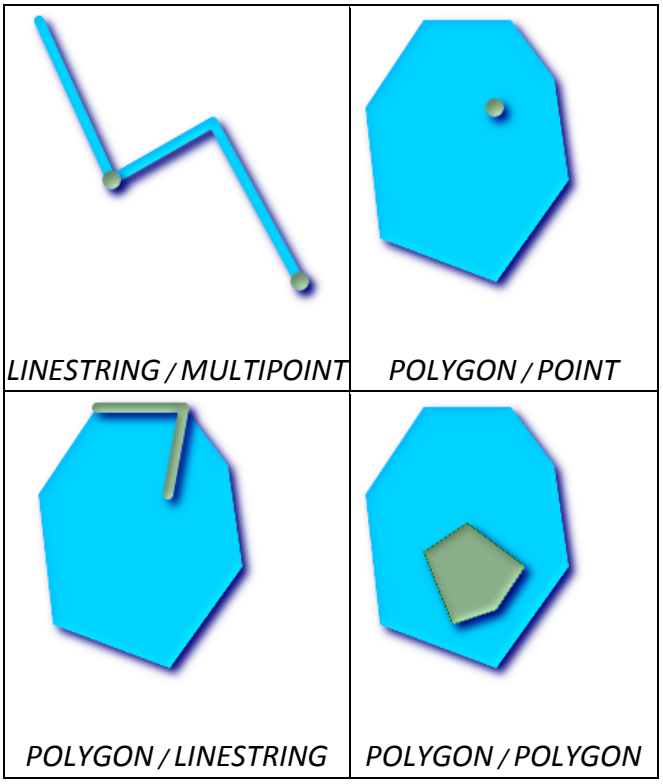
✓ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다. s2.1.1.2 // s2.1.13.3 - within(geometry B, geometry A)와 동일

✓ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 5.1.31

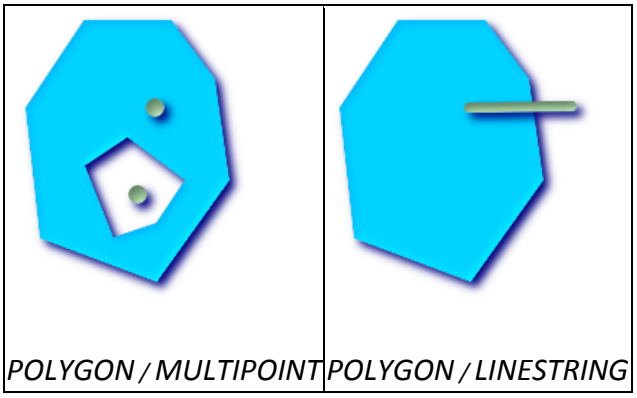
ST_Contains 및 ST_Within 함수에는 직관적으로 명백하지 않은, 어떤 섬세함들이 있습니다. 자세한 내용은 Subtleties of OGC Covers, Contains, Within 을 확인해보십시오.

예시

ST_Contains 의 술부는 다음 모든 그림들에서 TRUE 를 반환합니다.



ST_Contains 의 술부는 다음 모든 그림들에서 FALSE 를 반환합니다.



```
-- 원안의원
SELECT ST_Contains(smallc, bigc) As smallcontainsbig,
       ST_Contains(bigc,smallc) As bigcontainssmall,
       ST_Contains(bigc, ST_Union(smallc, bigc)) as bigcontainsunion,
       ST_Equals(bigc, ST_Union(smallc, bigc)) as bigisunion,
       ST_Covers(bigc, ST_ExteriorRing(bigc)) As bigcoversexterior,
       ST_Contains(bigc, ST_ExteriorRing(bigc)) As bigcontainsexterior
FROM (SELECT ST_Buffer(ST_GeomFromText('POINT(1 2)'), 10) As smallc,
             ST_Buffer(ST_GeomFromText('POINT(1 2)'), 20) As bigc) As foo;

-- 결과
 smallcontainsbig | bigcontainssmall | bigcontainsunion | bigisunion | bigcoversexterior | bigcontainsexterior
------------------+------------------+------------------+------------+-------------------+---------------------
       f          |        t         |        t         |     t      |         t         |          f

-- 담고 있는 것과 제대로 담고 있는 것의 차이점을 보여주는 예시
 SELECT ST_GeometryType(geomA) As geomtype, ST_Contains(geomA,geomA) AS
acontainsa, ST_ContainsProperly(geomA, geomA) AS acontainspropa,
    ST_Contains(geomA,       ST_Boundary(geomA))       As       acontainsba,
ST_ContainsProperly(geomA, ST_Boundary(geomA)) As acontainspropba
  FROM (VALUES ( ST_Buffer(ST_Point(1,1), 5,1) ),
                       ( ST_MakeLine(ST_Point(1,1), ST_Point(-1,-1) ) ),
                       ( ST_Point(1,1) )
         ) As foo(geomA);

   geomtype   | acontainsa | acontainspropa | acontainsba | acontainspropba
--------------+------------+----------------+-------------+-----------------
ST_Polygon    |      t     |       f        |      f      |       f
ST_LineString |      t     |       f        |      f      |       f
ST_Point      |      t     |       t        |      f      |       f
```

참고: ST_Boundary, ST_ContainsProperly, ST_Covers, ST_CoveredBy, ST_Equals, ST_Within

8.9.18. ST_ContainsProperly

ST_ContainsProperly — B 가 A 의 내부와 교차하지만 A 의 경계선(또는 외부)과 교차하지는 않을 경우 참을 반환합니다. A 는 자기 자신을 제대로 담고 있지는 않지만, 자기 자신을 담고 있긴 합니다.

Synopsis

boolean ST_ContainsProperly(geometry *geomA*, geometry *geomB*);

설명: B 가 A 의 내부와 교차하지만 A 의 경계선(또는 외부)과 교차하지는 않을 경우 참을 반환합니다.

A 는 자기 자신을 제대로 담고 있지는 않지만, 자기 자신을 담고 있긴 합니다.

다른 도형의 모든 포인트가 이 도형 내부의 포인트입니다. 이 두 도형에 대한 DE-9IM 교차점 매트릭스가 ST_Relate 에서 쓰인 [T**FF*FF*]와 일치합니다.



JTS 문서에서 살짝 말을 바꿔 인용했습니다: ST_Contains 및 ST_Intersects 대신 이 함수를 사용하는 경우의 장점은 각 개별 포인트에서 위상기하를 계산할 필요가 없어 더 효율적으로 계산할 수 있다는 점입니다. 이 함수의 용례는 대용량 폴리곤 도형을 가진 도형 집합의 교차점을 계산하는 예시입니다. 교차점을 파악하는 것이 꽤 느린 작업이기 때문에, 면 안에

> 도형 전체가 들어가는 테스트 도형을 필터링하는 containsProperly 를 이용하는 편이 더 효율적입니다. 이런 경우, 교차점이 정확히 원래 테스트 도형이라는 사실이 선험적으로 알려져 있습니다.

1.4.0 버전부터 사용할 수 있습니다. GEOS 3.1.0 이상 버전이 필요합니다.

 GEOMETRYCOLLECTION 을 인수로 호출하지 마십시오.

 이 함수에 유효하지 않은 도형을 입력하지 마십시오. 결과를 예측할 수 없습니다.

이 함수를 호출하면 도형에서 이용할 수 있는 모든 인덱스를 활용하는 경계 상자 비교 작업을 자동적으로 포함하게 됩니다. 인덱스 활용을 피하려면, _ST_ContainsProperly 함수를 이용하십시오.

예시

```
  -- 원 안의 원
        SELECT ST_ContainsProperly(smallc, bigc) As smallcontainspropbig,
        ST_ContainsProperly(bigc,smallc) As bigcontainspropsmall,
        ST_ContainsProperly(bigc,       ST_Union(smallc,       bigc))       as
bigcontainspropunion,
        ST_Equals(bigc, ST_Union(smallc, bigc)) as bigisunion,
        ST_Covers(bigc, ST_ExteriorRing(bigc)) As bigcoversexterior,
        ST_ContainsProperly(bigc,          ST_ExteriorRing(bigc))          As
bigcontainsexterior
        FROM (SELECT ST_Buffer(ST_GeomFromText('POINT(1 2)'), 10) As smallc,
        ST_Buffer(ST_GeomFromText('POINT(1 2)'), 20) As bigc) As foo;
        -- 결과
  smallcontainspropbig | bigcontainspropsmall | bigcontainspropunion | bigisunion | bigcoversexterior | bigcontainsexterior
 ----------------------+----------------------+----------------------+------------+-------------------+---------------------
      f       |       t            |      f          |      t     |       t       |      f

  -- 담고 있는 것과 제대로 담고 있는 것의 차이점을 보여주는 예시
  SELECT  ST_GeometryType(geomA)  As  geomtype,  ST_Contains(geomA,geomA)  AS
acontainsa, ST_ContainsProperly(geomA, geomA) AS acontainspropa,
  ST_Contains(geomA,        ST_Boundary(geomA))        As        acontainsba,
ST_ContainsProperly(geomA, ST_Boundary(geomA)) As acontainspropba
  FROM (VALUES ( ST_Buffer(ST_Point(1,1), 5,1) ),
               ( ST_MakeLine(ST_Point(1,1), ST_Point(-1,-1) ) ),
               ( ST_Point(1,1) )
        ) As foo(geomA);

   geomtype    | acontainsa | acontainspropa | acontainsba | acontainspropba
 --------------+------------+----------------+-------------+----------------
 ST_Polygon    |     t      |       f        |       f     |       f
 ST_LineString |     t      |       f        |       f     |       f
 ST_Point      |     t      |       t        |       f     |       f
```

참고 : ST_GeometryType, ST_Boundary, ST_Contains, ST_Covers, ST_CoveredBy, ST_Equals, ST_Relate, ST_Within

8.9.19. ST_Covers

ST_Covers — 도형 B 안에 있는 어떤 포인트도 도형 A 외부에 없을 경우 1(참)을 반환합니다.

Synopsis

boolean ST_Covers(geometry *geomA*, geometry *geomB*);

boolean ST_Covers(geography *geogpolyA*, geography *geogpointB*);

설명: 도형/지리형 B 안에 있는 어떤 포인트도 도형/지리형 A 외부에 없을 경우 1(참)을 반환합니다.

GEOS 모듈로 실행

 GEOMETRYCOLLECTION 을 인수로 호출하지 마십시오.

 지리형에 한해 포인트를 커버하는 폴리곤을 지원합니다.

 이 함수에 유효하지 않은 도형을 입력하지 마십시오. 결과를 예측할 수 없습니다.

이 함수를 호출하면 도형에서 이용할 수 있는 모든 인덱스를 활용하는 경계 상자 비교 작업을 자동적으로 포함하게 됩니다. 인덱스 활용을 피하려면, _ST_Covers 함수를 이용하십시오.

개선 사항: 2.3.0 버전에서 도형에 대한 PIP 단락(short-circuit)이 포인트가 몇 개 없는 멀티포인트를 지원하도록 확장/개선됐습니다. 예전 버전은 폴리곤 내부에 있는 포인트만 지원했습니다.

1.5.0 버전부터 지리형을 지원합니다.

1.2.2 버전부터 사용할 수 있습니다. GEOS 3.0 이상 버전이 필요합니다.

주의: 이 함수는 정수가 아니라 불값을 반환하는, "사용할 만한" 버전입니다.

OGC 표준은 아니지만, Oracle 도 이 함수를 내장하고 있습니다.

ST_Contains 및 ST_Within 함수에는 직관적으로 명백하지 않은, 어떤 섬세함들이 있습니다. 자세한 내용은 Subtleties of OGC Covers, Contains, Within 을 확인해보십시오.

예시

도형 예시

```
-- 원을 커버하는 원
SELECT ST_Covers(smallc,smallc) As smallinsmall,
       ST_Covers(smallc, bigc) As smallcoversbig,
       ST_Covers(bigc, ST_ExteriorRing(bigc)) As bigcoversexterior,
       ST_Contains(bigc, ST_ExteriorRing(bigc)) As bigcontainsexterior
FROM (SELECT ST_Buffer(ST_GeomFromText('POINT(1 2)'), 10) As smallc,
       ST_Buffer(ST_GeomFromText('POINT(1 2)'), 20) As bigc) As foo;
       -- 결과
 smallinsmall | smallcoversbig | bigcoversexterior | bigcontainsexterior
--------------+----------------+-------------------+---------------------
 t            | f              | t                 | f
(1 row)
```

지리형 예시

```
  -- 300 미터 버퍼를 준 포인트와 10 미터 버퍼를 준 포인트를 비교
  SELECT ST_Covers(geog_poly, geog_pt) As poly_covers_pt,
        ST_Covers(ST_Buffer(geog_pt,10), geog_pt) As buff_10m_covers_cent
        FROM    (SELECT    ST_Buffer(ST_GeogFromText('SRID=4326;POINT(-99.327
31.4821)'), 300) As geog_poly,
                          ST_GeogFromText('SRID=4326;POINT(-99.33 31.483)')
As geog_pt ) As foo;

   poly_covers_pt | buff_10m_covers_cent
  ----------------+----------------------
   f              | t
```

참고: ST_Contains, ST_CoveredBy, ST_Within

8.9.20. ST_CoveredBy

ST_CoveredBy — 도형/지리형 A 안에 있는 어떤 포인트도 도형/지리형 B 외부에 없을 경우 1(참)을 반환합니다.

Synopsis

boolean ST_CoveredBy(geometry *geomA*, geometry *geomB*);

boolean ST_CoveredBy(geography *geogA*, geography *geogB*);

설명: 도형/지리형 A 안에 있는 어떤 포인트도 도형/지리형 B 외부에 없을 경우 1(참)을 반환합니다.

GEOS 모듈로 실행

 GEOMETRYCOLLECTION 을 인수로 호출하지 마십시오.

 이 함수에 유효하지 않은 도형을 입력하지 마십시오. 결과를 예측할 수 없습니다.

1.2.2 버전부터 사용할 수 있습니다. GEOS 3.0 이상 버전이 필요합니다.

이 함수를 호출하면 도형에서 이용할 수 있는 모든 인덱스를 활용하는 경계 상자 비교 작업을 자동적으로 포함하게 됩니다. 인덱스 활용을 피하려면, _ST_CoveredBy 함수를 이용하십시오.

주의: 이 함수는 정수가 아니라 불값을 반환하는, "사용할 만한" 버전입니다.

OGC 표준은 아니지만, Oracle 도 이 함수를 내장하고 있습니다.

ST_Contains 및 ST_Within 함수에는 직관적으로 명백하지 않은, 어떤 섬세함들이 있습니다. 자세한 내용은 Subtleties of OGC Covers, Contains, Within 을 확인해보십시오.

예시

```
  -- 원이 커버하는 원
  SELECT ST_CoveredBy(smallc,smallc) As smallinsmall,
        ST_CoveredBy(smallc, bigc) As smallcoveredbybig,
        ST_CoveredBy(ST_ExteriorRing(bigc), bigc) As exteriorcoveredbybig,
        ST_Within(ST_ExteriorRing(bigc),bigc) As exeriorwithinbig
  FROM (SELECT ST_Buffer(ST_GeomFromText('POINT(1 2)'), 10) As smallc,
```

```
        ST_Buffer(ST_GeomFromText('POINT(1 2)'), 20) As bigc) As foo;
-- 결과
 smallinsmall | smallcoveredbybig | exteriorcoveredbybig | exeriorwithinbig
--------------+-------------------+----------------------+----------------
 t            | t                 | t                    | f
(1 row)
```

참고: ST_Contains, ST_Covers, ST_ExteriorRing, ST_Within

8.9.21. ST_Crosses

ST_Crosses — 입력 도형들이 전부가 아닌 일부 내부 포인트를 공유하는 경우 TRUE 를 반환합니다.

Synopsis

boolean ST_Crosses(geometry *g1*, geometry *g2*);

설명: ST_Crosses 는 두 도형 객체를 입력받아 두 도형이 "공간적으로 교차"하는 경우, 즉 두 도형이 전부가 아닌 일부 내부 포인트를 공유하는 경우 TRUE 를 반환합니다. 두 도형의 내부의 교차점은 절대 텅 빈 집합이어서는 안 되며 두 입력 도형의 최대 차원보다 차원수가 적어야만 합니다. 또, 두 도형의 교차점은 두 도형 어느쪽과도 동일해서는 안 됩니다. 반대의 경우, FALSE 를 반환합니다.

수학적인 관점에서, 다음과 같이 표현됩니다:

할 일: 여기에 적절한 MathML 마크업 언어를 삽입하든지 또는 GIF 를 사용할 것. 단순한 HTML 마크업 언어는 IE 와 FF 양쪽 모두에서 잘 동작하지 않음.

$$a.Crosses(b) \Leftrightarrow (dim(I(a) \cap I(b)) < max(dim(I(a)), dim(I(b)))) \wedge (a \cap b \neq a) \wedge (a \cap b \neq b)$$

두 도형에 대한 DE-9IM 교차점 매트릭스는 다음과 같습니다:

- T*T****** (포인트/라인, 포인트/면, 라인/면 상황)
- T*****T** (라인/포인트, 면/포인트, 면/라인 상황)
- 0******** (라인/라인 상황)

이 조합이 아닌 다른 차원들의 조합일 경우, 이 함수는 거짓을 반환합니다.

OpenGIS 단순 피처 사양서는 포인트/라인, 포인트/면, 라인/라인, 라인/면 조합의 경우에만 이 함수를 정의하고 있습니다. JTS/GEOS 는 라인/포인트, 면/포인트, 면/라인 조합 또한 적용되도록 정의를 확장시킵니다. 이렇게 상관 관계가 대칭을 이루었습니다.


GEOMETRYCOLLECTION 을 인수로 호출하지 마십시오.

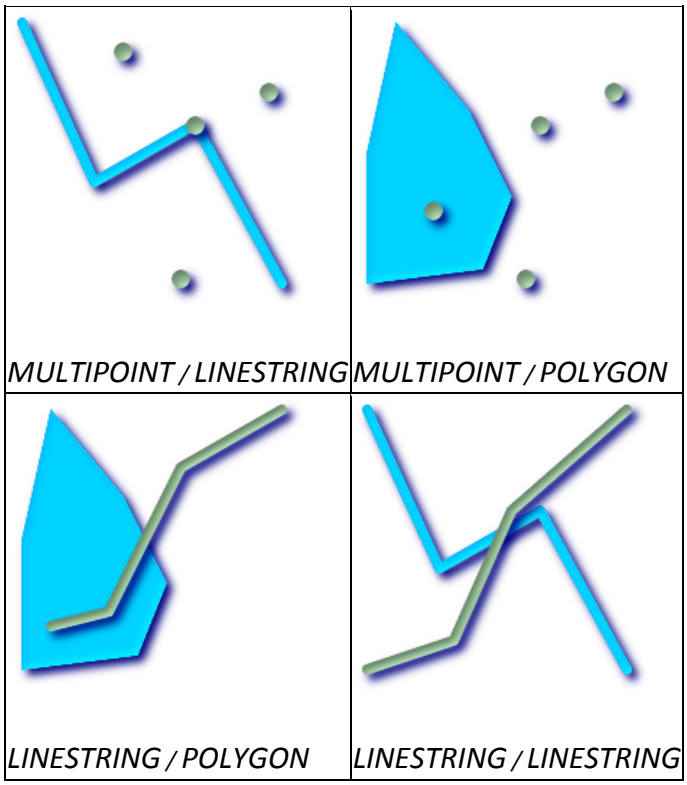

이 함수를 호출하면 도형에서 이용할 수 있는 모든 인덱스를 활용하는 경계 상자 비교 작업을 자동적으로 포함하게 됩니다.

✓ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다. s2.1.13.3

✓ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 5.1.29

예시

다음 그림들은 모두 TRUE 를 반환합니다.



사용자가 도로 테이블과 고속도로 테이블, 테이블 두 개를 가지고 있는 경우를 상정해봅시다.

```
CREATE TABLE roads (
 id serial NOT NULL,
 the_geom geometry,
 CONSTRAINT roads_pkey PRIMARY KEY (road_id)
);
```

```
CREATE TABLE highways (
  id serial NOT NULL,
  the_gem geometry,
  CONSTRAINT roads_pkey PRIMARY KEY (road_id)
  );
```

고속도로와 교차하는 도로의 목록을 구하려면, 다음과 비슷한 쿼리를 해야 합니다.

```
  SELECT roads.id
  FROM roads, highways
  WHERE ST_Crosses(roads.the_geom, highways.the_geom);
```

8.9.22. ST_LineCrossingDirection

ST_LineCrossingDirection — 라인스트링 2 개를 입력하면, 어떤 형태로 교차하고 있는지를 나타내는 -3 에서 3 사이의 숫자를 반환합니다. 0 은 교차하지 않음을 뜻합니다.

Synopsis

integer ST_LineCrossingDirection(geometry *linestringA*, geometry *linestringB*);

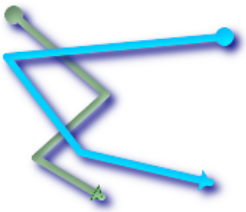
설명: 라인스트링 2 개를 입력하면, 어떤 형태로 교차하고 있는지를 나타내는 -3 에서 3 사이의 숫자를 반환합니다. 0 은 교차하지 않음을 뜻합니다. 이 함수는 LINESTRING 만 지원합니다.

정수형 상수의 정의는 다음과 같습니다:

- 0: 라인이 교차하지 않음
- -1: 라인이 왼쪽 방향으로 교차
- 1: 라인이 오른쪽 방향으로 교차
- -2: 라인이 종단점 왼쪽 방향으로 복수교차(multicross)
- 2: 라인이 종단점 오른쪽 방향으로 복수교차
- -3: 라인이 종단점과 시작점의 동일한 왼쪽 방향으로 복수교차
- 3: 라인이 종단점과 시작점의 동일한 오른쪽 방향으로 복수교차

1.4.0 버전부터 사용할 수 있습니다.

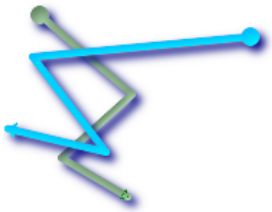
예시



라인 1(초록색), 라인 2(파란색)의 둥근점이 시작점이고, 화살표가 종단점입니다. 아래는 쿼리입니다.

```
  SELECT
ST_LineCrossingDirection(foo.line1,
foo.line2) As l1_cross_l2 ,

ST_LineCrossingDirection(foo.line2,
foo.line1) As l2_cross_l1
  FROM (
  SELECT
   ST_GeomFromText('LINESTRING(25
169,89 114,40 70,86 43)') As line1,
   ST_GeomFromText('LINESTRING(171
154,20 140,71 74,161 53)') As line2
```



라인 1(초록색), 라인 2(파란색)의 둥근점이 시작점이고, 화살표가 종단점입니다. 아래는 쿼리입니다.

```
  SELECT
ST_LineCrossingDirection(foo.line1,
foo.line2) As l1_cross_l2 ,

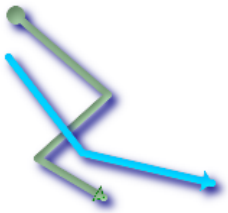
ST_LineCrossingDirection(foo.line2,
foo.line1) As l2_cross_l1
  FROM (
   SELECT
    ST_GeomFromText('LINESTRING(25
169,89 114,40 70,86 43)') As line1,
```

```
        ) As foo;

  l1_cross_l2 | l2_cross_l1
 -------------+-------------
             3 |          -3
```

```
    ST_GeomFromText('LINESTRING  (171
154, 20 140, 71 74, 2.99 90.16)') As
line2
  ) As foo;

  l1_cross_l2 | l2_cross_l1
 -------------+-------------
             2 |          -2
```



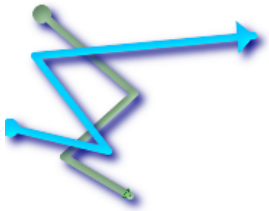
라인 1(초록색), 라인 2(파란색)의 등근점이 시작점

```
SELECT

ST_LineCrossingDirection(foo.line1,
foo.line2) As l1_cross_l2 ,

ST_LineCrossingDirection(foo.line2,
foo.line1) As l2_cross_l1
FROM (
 SELECT
  ST_GeomFromText('LINESTRING(25
169,89 114,40 70,86 43)') As line1,
  ST_GeomFromText('LINESTRING     (20
140, 71 74, 161 53)') As line2
  ) As foo;

 l1_cross_l2 | l2_cross_l1
-------------+-------------
          -1 |           1
```



라인 1(초록색), 라인 2(파란색)의 등근점이 시작점이고, 화살표가 종단점입니다. 아래는 쿼리입니다.

```
  SELECT
ST_LineCrossingDirection(foo.line1,
foo.line2) As l1_cross_l2 ,

ST_LineCrossingDirection(foo.line2,
foo.line1) As l2_cross_l1
  FROM (SELECT

ST_GeomFromText('LINESTRING(25
169,89 114,40 70,86 43)') As line1,

ST_GeomFromText('LINESTRING(2.99
90.16,71  74,20  140,171  154)')  As
line2
        ) As foo;

  l1_cross_l2 | l2_cross_l1
 -------------+-------------
           -2 |           2
```

```
  SELECT s1.gid, s2.gid, ST_LineCrossingDirection(s1.the_geom, s2.the_geom)
      FROM streets s1 CROSS JOIN streets s2 ON (s1.gid != s2.gid AND s1.the_geom &&
s2.the_geom )
  WHERE ST_CrossingDirection(s1.the_geom, s2.the_geom)
  > 0;
```

참고: ST_Crosses

8.9.23. ST_Disjoint

ST_Disjoint — 도형들이 "공간적으로 교차"하지 않을 경우 - 어떤 공간도 함께 공유하지 않을 경우 - 참을 반환합니다.

Synopsis

boolean ST_Disjoint(geometry *A* , geometry *B*);

설명: 겹치기(overlap), 접촉(touch), 범위 내(within) 모두 공간 분리(disjoint)되지 않음을 의미하는 말입니다. 앞에서 언급한 함수들 가운데 어느 하나라도 참을 반환하는 경우, 해당 도형들도 공간적으로 분리되지 않은 겁니다. 도형들이 교차하는 경우, 분리(disjoint) 함수는 거짓을 반환합니다.

GEOMETRYCOLLECTION 을 인수로 호출하지 마십시오.

GEOS 모듈로 실행

이 함수 호출은 인덱스를 활용하지 않습니다.

주의: 이 함수는 정수가 아니라 불값을 반환하는, "사용할 만한" 버전입니다.

이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다. s2.1.1.2 //s2.1.13.3 - a.Relate(b, 'FF*FF****')

이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 5.1.26

예시

```
SELECT   ST_Disjoint('POINT(0   0)'::geometry,   'LINESTRING   (   2   0,   0
2 )'::geometry);
 st_disjoint
---------------
 t
(1 row)
SELECT   ST_Disjoint('POINT(0   0)'::geometry,   'LINESTRING   (   0   0,   0
2 )'::geometry);
 st_disjoint
---------------
 f
(1 row)
```

참고: ST_IntersectsST_Intersects

8.9.24. ST_Distance

ST_Distance — 도형 유형에 대해, 두 도형 사이의 2 차원 데카르트 거리를 투영 단위(SRS 단위)로 반환합니다. 지리형 유형의 경우, 기본적으로 두 도형 사이의 측지 거리를 미터 단위로 반환합니다.

Synopsis

float ST_Distance(geometry *g1*, geometry *g2*);

float ST_Distance(geography *gg1*, geography *gg2*);

float ST_Distance(geography *gg1*, geography *gg2*, boolean *use_spheroid*);

설명

도형 유형에 대해, 두 도형 사이의 2 차원 데카르트 최단 거리를 투영 단위(SRS 단위)로 반환합니다. 지리형 유형의 경우, 기본적으로 두 도형 사이의 측지 거리를 미터 단위로 반환합니다. use_spheroid 인수를 거짓으로 설정하면, 회전타원체 대신 더 빠른 구체 계산을 이용합니다.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다.

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 5.1.23

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✔ SFCGAL 백엔드도 이 방법을 제공합니다.

1.5.0 버전부터 지리형을 지원합니다. 대용량 또는 수많은 꼭짓점을 가진 도형을 더 잘 처리하기 위해 평면에 대한 속도를 향상시켰습니다.

개선 사항: 2.1.0 버전부터 지리형에 대한 속도가 향상됐습니다. 자세한 내용은 Making Geography faster 를 참조하십시오.

개선 사항: 2.1.0 버전부터 만곡 도형을 지원하기 시작했습니다.

개선 사항: 2.2.0 버전부터 회전타원체 측정시 정확도와 강력함을 향상시키기 위해 GeographicLib 을 이용합니다. 이 새 기능의 장점을 취하려면 Proj 4.9.0 이상 버전이 필요합니다.

기본 도형 예시

```
-- 도형 예시: SRID 4326 WGS84 경위도 평면의 단위는 도입니다.
SELECT ST_Distance(
        ST_GeomFromText('POINT(-72.1235 42.3521)',4326),
        ST_GeomFromText('LINESTRING(-72.1260 42.45, -72.123 42.1546)', 4326)
    );
st_distance
-----------------
0.00150567726382282

-- 도형 예시: SRID 26986 매사추세츠 주 미터 단위 평면의 단위는 미터입니다(매사추세츠 지역에 한해 거의 정확합니다) .
SELECT ST_Distance(
            ST_Transform(ST_GeomFromText('POINT(-72.1235 42.3521)',4326),26986),
            ST_Transform(ST_GeomFromText('LINESTRING(-72.1260  42.45,  -72.123  42.1546)',
4326),26986)
        );
st_distance
-----------------
123.797937878454

-- 도형 예시: SRID 2163 미국 등적 전도의 단위는 미터입니다(가장 정확도가 떨어집니다) .
SELECT ST_Distance(
            ST_Transform(ST_GeomFromText('POINT(-72.1235 42.3521)',4326),2163),
            ST_Transform(ST_GeomFromText('LINESTRING(-72.1260  42.45,  -72.123  42.1546)',
4326),2163)
        );
```

```
st_distance
------------------
126.664256056812
```

지리형 예시

```
-- 도형 예시와 동일하지만 단위가 미터라는 점에 주의하십시오. 구체를 이용하면 좀 더 빨라지지만 덜 정확해집니다.
SELECT ST_Distance(gg1, gg2) As spheroid_dist, ST_Distance(gg1, gg2, false) As sphere_dist
FROM (SELECT
    ST_GeogFromText('SRID=4326;POINT(-72.1235 42.3521)') As gg1,
    ST_GeogFromText('SRID=4326;LINESTRING(-72.1260 42.45, -72.123 42.1546)') As gg2
    ) As foo ;

 spheroid_dist  |  sphere_dist
------------------+------------------
123.802076746848 | 123.475736916397
```

참고: ST_3DDistance, ST_DWithin, ST_DistanceSphere, ST_DistanceSpheroid, ST_MaxDistance, ST_Transform

8.9.25. ST_MinimumClearance

ST_MinimumClearance — 도형의 튼튼함(robustness)의 척도인 도형의 최소 여유(clearance)를 반환합니다.

Synopsis

float ST_MinimumClearance(geometry *g*);

설명: (폴리곤일 경우) ST_IsValid 또는 (라인일 경우) ST_IsSimple 함수에 따라 유효성에 대한 기준을 만족시키지만, 텍스트 기반 형식(WKT, KML, GML GeoJSON 등) 또는 이중 정밀도 부동소수점 좌표를 이용하지 않는 바이너리 형식(MapInfo TAB 등)으로 변환하는 과정에서 일어날 수 있는 것처럼, 꼭짓점 가운데 하나가 살짝 이동하면 유효하지 않아지는 도형이 그렇게 희귀한 것은 아닙니다.

도형의 "최소 여유"란 도형의 꼭짓점이 움직여 유효하지 않은 도형을 만드는 최소 거리를 뜻합니다. 이를 도형의 튼튼함의 정량적 척도로 생각할 수 있습니다. 최소 여유 값이 높을수록 도형은 더 튼튼한 것이죠.

도형이 e 라는 최소 여유를 가지고 있을 경우, 다음과 같이 말할 수 있습니다:

- 도형 내부의 어떤 서로 다른 꼭짓점 2 개도 e 보다 더 떨어져 있지 않습니다.
- 종단점이 아닌 한 어떤 꼭짓점도 라인 선분에 e 보다 가까이 있을 수는 없습니다.

도형에 대해 최소 여유가 존재하지 않을 경우 (예를 들어 단일 포인트 또는 포인트들이 서로 동일한 멀티포인트인 경우) ST_MinimumClearance 함수는 무한을 반환할 것입니다.

2.3.0 버전부터 사용할 수 있습니다. GEOS 3.6.0 이상 버전이 필요합니다.

예시

```
SELECT ST_MinimumClearance('POLYGON ((0 0, 1 0, 1 1, 0.5 3.2e-4, 0 0))');
 st_minimumclearance
```

```
---------------------
       0.00032
```

참고: ST_MinimumClearanceLine

8.9.26. ST_MinimumClearanceLine

ST_MinimumClearanceLine — 포인트 2 개로 이루어진, 도형의 최소 여유를 나타내는 라인스트링을 반환합니다.

Synopsis

Geometry ST_MinimumClearanceLine(geometry *g*);

설명: 포인트 2 개로 이루어진, 도형의 최소 여유를 나타내는 라인스트링을 반환합니다. 도형이 최소 여유를 가지고 있지 않을 경우, LINESTRING EMPTY 를 반환할 것입니다.

2.3.0 버전부터 사용할 수 있습니다. GEOS 3.6.0 이상 버전이 필요합니다.

예시

```
SELECT ST_AsText(ST_MinimumClearanceLine('POLYGON ((0 0, 1 0, 1 1, 0.5 3.2e-4, 0 0))'));
st_astext
------------------------------
LINESTRING(0.5 0.00032,0.5 0)
```

참고: ST_MinimumClearance

8.9.27. ST_HausdorffDistance

ST_HausdorffDistance — 두 도형 사이의 하우스도르프 거리를 반환합니다. 기본적으로 두 도형이 얼마나 유사한지 또는 유사하지 않은지에 대한 척도입니다. 두 도형의 공간 참조 시스템 단위를 씁니다.

Synopsis

float ST_HausdorffDistance(geometry *g1*, geometry *g2*);

float ST_HausdorffDistance(geometry *g1*, geometry *g2*, float *densifyFrac*);

설명: "이산 하우스도르프 거리"라고 할 수 있는 거리 척도를 계산하기 위한 알고리즘을 실행합니다. 이산 하우스도르프 거리란 도형들 가운데 하나에 대해 불연속 포인트들에 제한된 하우스도르프 거리를 말합니다. 하우스도르프 거리에 대한 위키백과 항목 캐스케이드 폴리곤 합집합 접근법의 정확도를 증명하기 위해 하우스도르프 거리 계산을 이용한 방법에 대한 마틴 데이비스의 메모

densifyFrac 인수를 설정하면, 이 함수는 구간 고밀화(segment densification) 작업을 수행한 다음 이산 하우스도르프 거리를 계산합니다. densifyFrac 파라미터가 각 구간을 고밀화하는 데 쓰이는 분수를 설정합니다. 각 구간은 동일한 길이의 하위 구간 몇 개로 나뉘며, 이 전체 길이를 나누는 분수가 주어진 분수와 가장 가깝습니다.



현재 구현된 함수는 불연속 위치로 꼭짓점만 지원합니다. 이를 임의 밀도의 포인트들을 사용할 수 있도록 확장할 수 있습니다.



이 알고리즘은 표준 하우스도르프 거리 알고리즘과 동등하지 않습니다. 하지만, 유용한 용례의 대용량 하위집합에 대해서 정확할 정도의 근사치를 계산합니다. 이 하위집합의 중요한 부분 가운데 하나가 서로 대강 평행하며 대강 같은 길이를 가진 라인스트링들입니다. 해당 하위집합이 라인을 맞춰보는 데 유용한 척도입니다.

1.5.0 버전부터 사용할 수 있습니다. GEOS 3.2.0 이상 버전이 필요합니다.

예시

각 건물에 대해, 해당 건물을 가장 잘 표현하는 획지(parcel)를 찾습니다. 먼저 도형과 교차하는 획지가 필요합니다. DISTINCT ON 이 목록에 한 번씩만 나온 각 건물을 반환해주고, ORDER BY .. ST_HausdorffDistance 가 건물과 가장 유사한 획지의 선호도를 얻게 해줍니다.

```
SELECT DISTINCT ON(buildings.gid) buildings.gid, parcels.parcel_id
  FROM buildings INNER JOIN parcels ON ST_Intersects(buildings.geom,parcels.geom)
   ORDER BY buildings.gid, ST_HausdorffDistance(buildings.geom, parcels.geom);
postgis=# SELECT ST_HausdorffDistance(
                 'LINESTRING (0 0, 2 0)'::geometry,
                 'MULTIPOINT (0 1, 1 0, 2 1)'::geometry);
 st_hausdorffdistance
 ----------------------
                    1
(1 row)

postgis=# SELECT st_hausdorffdistance('LINESTRING (130 0, 0 0, 0 150)'::geometry, 'LINESTRING
(10 10, 10 150, 130 10)'::geometry, 0.5);
 st_hausdorffdistance
 ----------------------
                   70
(1 row)
```

8.9.28. ST_MaxDistance

ST_MaxDistance — 두 도형 사이의 2 차원 최장 거리를 투영 단위로 반환합니다.

Synopsis

float ST_MaxDistance(geometry *g1*, geometry *g2*);

설명:



두 도형 사이의 2 차원 최장 거리를 투영 단위로 반환합니다. g1 과 g2 가 동일한 도형일 경우 이 함수는 해당 도형 내에서 서로 가장 멀리 있는 두 꼭짓점 사이의 거리를 반환합니다.

1.5.0 버전부터 사용할 수 있습니다.

예시

기본적으로 가장 먼 거리에 있는 포인트는 라인의 어느 부분에라도 존재할 수 있습니다.

```
postgis=# SELECT ST_MaxDistance('POINT(0 0)'::geometry, 'LINESTRING ( 2 0, 0 2 )'::geometry);
  st_maxdistance
-----------------
 2
(1 row)

postgis=# SELECT ST_MaxDistance('POINT(0 0)'::geometry, 'LINESTRING ( 2 2, 2 2 )'::geometry);
 st_maxdistance
------------------
 2.82842712474619
(1 row)
```

참고: ST_Distance, ST_LongestLine, ST_DFullyWithin

8.9.29. ST_DistanceSphere

ST_DistanceSphere — 경위도 도형 2 개 사이의 최단 거리를 미터 단위로 반환합니다. SRID 가 정의하는 회전타원체에서 추출한 반경을 가진 지구 구체를 이용합니다. ST_DistanceSpheroid 보다는 처리 속도가 빠르지만, 정확도는 떨어집니다. PostGIS 1.5 미만 버전에서는 포인트에 대해서만 구현돼 있었습니다.

Synopsis

float ST_DistanceSphere(geometry *geomlonlatA*, geometry *geomlonlatB*);

설명: 경위도 포인트 2 개 사이의 최단 거리를 미터 단위로 반환합니다. SRID 가 정의하는 회전타원체에서 추출한 반경을 가진 지구 구체를 이용합니다. ST_DistanceSpheroid 보다는 처리 속도가 빠르지만, 정확도는 떨어집니다. PostGIS 1.5 미만 버전에서는 포인트에 대해서만 구현돼 있었습니다.

1.5 버전부터 포인트가 아닌 다른 도형 유형을 지원하기 시작했습니다. 1.5 미만 버전에서는 포인트에 대해서만 구현돼 있었습니다.

변경 사항: 2.2.0 미만 버전에서는 ST_Distance_Sphere 라는 명칭이었습니다.

예시

```
SELECT    round(CAST(ST_DistanceSphere(ST_Centroid(the_geom),    ST_GeomFromText('POINT(-118
38)',4326)) As numeric),2) As dist_meters,
round(CAST(ST_Distance(ST_Transform(ST_Centroid(the_geom),32611),
        ST_Transform(ST_GeomFromText('POINT(-118  38)',  4326),32611))  As  numeric),2)  As
dist_utm11_meters,
round(CAST(ST_Distance(ST_Centroid(the_geom),  ST_GeomFromText('POINT(-118  38)',  4326))  As
numeric),5) As dist_degrees,
round(CAST(ST_Distance(ST_Transform(the_geom,32611),
        ST_Transform(ST_GeomFromText('POINT(-118  38)',  4326),32611))  As  numeric),2)  As
min_dist_line_point_meters
FROM
    (SELECT ST_GeomFromText('LINESTRING(-118.584 38.374,-118.583 38.5)', 4326) As the_geom)
as foo;
     dist_meters | dist_utm11_meters | dist_degrees | min_dist_line_point_meters
    -------------+-------------------+--------------+----------------------------
         70424.47 |      70438.00 |    0.72900 |          65871.18
```

참고: ST_Distance, ST_DistanceSpheroid

8.9.30. ST_DistanceSpheroid

ST_DistanceSpheroid — 특정 회전타원체가 주어진 두 경위도 도형 사이의 최단 거리를 반환합니다. PostGIS 1.5 미만 버전은 포인트만 지원했습니다.

Synopsis

float ST_DistanceSpheroid(geometry geomlonlatA, geometry geomlonlatB, spheroid measurement_spheroid);

설명: 특정 회전타원체가 주어진 두 경위도 도형 사이의 최단 거리를 반환합니다. 주어진 회전타원체에 대한 설명은 ST_LengthSpheroid 를 참조하십시오. PostGIS 1.5 미만 버전은 포인트만 지원했습니다.



현재 이 함수는 도형의 SRID 를 찾아보지 않고 주어진 회전타원체의 좌표로 쓰여 있다고 가정할 것입니다. 이 함수의 이전 버전은 포인트만 지원했습니다.

1.5 버전부터 포인트가 아닌 다른 도형 유형을 지원하기 시작했습니다. 1.5 미만 버전에서는 포인트에 대해서만 구현돼 있었습니다.

변경 사항: 2.2.0 미만 버전에서는 ST_Distance_Spheroid 라는 명칭이었습니다.

예시

```
SELECT round(CAST(
        ST_DistanceSpheroid(ST_Centroid(the_geom), ST_GeomFromText('POINT(-118 38)',4326),
'SPHEROID["WGS 84",6378137,298.257223563]')
            As numeric),2) As dist_meters_spheroid,
        round(CAST(ST_DistanceSphere(ST_Centroid(the_geom),    ST_GeomFromText('POINT(-118
38)',4326)) As numeric),2) As dist_meters_sphere,
round(CAST(ST_Distance(ST_Transform(ST_Centroid(the_geom),32611),
        ST_Transform(ST_GeomFromText('POINT(-118  38)',  4326),32611))  As  numeric),2)  As
dist_utm11_meters
FROM
    (SELECT ST_GeomFromText('LINESTRING(-118.584 38.374,-118.583 38.5)', 4326) As the_geom)
as foo;
 dist_meters_spheroid | dist_meters_sphere | dist_utm11_meters
----------------------+--------------------+-------------------
              70454.92 |       70424.47 |      70438.00
```

참고: ST_Distance, ST_DistanceSphere

8.9.31. ST_DFullyWithin

ST_DFullyWithin — 모든 도형이 서로 설정된 거리 안에 있을 경우 참을 반환합니다.

Synopsis

boolean ST_DFullyWithin(geometry *g1*, geometry *g2*, double precision *distance*);

설명: 도형들이 완전히 서로의 설정된 거리 내에 있을 경우 참을 반환합니다. 이 거리는 도형들의 공간 참조 시스템이 정의한 단위로 설정됩니다. 이 함수가 제대로 동작하려면, 소스 도형들이 둘 다 동일한 SRID 를 가진, 동일한 좌표 투영체여야 합니다.



이 함수를 호출하면 도형에서 이용할 수 있는 모든 인덱스를 활용하는 경계 상자 비교 작업을 자동적으로 포함하게 됩니다.

1.5.0 버전부터 사용할 수 있습니다.

예시

```
postgis=# SELECT ST_DFullyWithin(geom_a, geom_b, 10) as DFullyWithin10, ST_DWithin(geom_a,
geom_b, 10) as DWithin10, ST_DFullyWithin(geom_a, geom_b, 20) as DFullyWithin20 from
         (select ST_GeomFromText('POINT(1 1)') as geom_a,ST_GeomFromText('LINESTRING(1 5, 2
7, 1 9, 14 12)') as geom_b) t1;

-----------------
 DFullyWithin10 | DWithin10 | DFullyWithin20 |
----------------+-----------+----------------+
 f              | t         | t              |
```

참고: ST_MaxDistance, ST_DWithin

8.9.32. ST_DWithin

ST_DWithin — 도형들이 서로 설정된 거리 안에 있을 경우 참을 반환합니다. 도형의 경우 공간 참조의 단위를 쓰고, 지리형의 경우 미터 단위를 씁니다. 또 지리형의 경우 측정시 use_spheroid=true(회전타원체 상에서 측정)가 기본값이며, 더 빨리 검사하려면 구체 상에서 측정하도록 use_spheroid=false 로 설정하십시오.

Synopsis

boolean ST_DWithin(geometry *g1*, geometry *g2*, double precision *distance_of_srid*);

boolean ST_DWithin(geography *gg1*, geography *gg2*, double precision *distance_meters*);

boolean ST_DWithin(geography *gg1*, geography *gg2*, double precision *distance_meters*, boolean *use_spheroid*);

설명: 도형들이 서로 설정된 거리 안에 있을 경우 참을 반환합니다.

도형의 경우: 거리는 도형들의 공간 참조 시스템이 정의한 단위로 설정됩니다. 이 함수가 제대로 동작하려면, 소스 도형들이 둘 다 동일한 SRID 를 가진, 동일한 좌표 투영체여야 합니다.

지리형의 경우: 미터 단위를 씁니다. 또 측정시 use_spheroid=true(회전타원체 상에서 측정)가 기본값이며, 더 빨리 검사하려면 구체 상에서 측정하도록 use_spheroid=false 로 설정합니다.



이 함수를 호출하면 도형에서 이용할 수 있는 모든 인덱스를 활용하는 경계 상자 비교 작업을 자동적으로 포함하게 됩니다.



1.3 미만 버전에서는, 동일한 효과를 얻기 위해 흔히 ST_Expand 함수를 && 및 ST_Distance 와 연결해서 썼습니다. 1.3.4 이전 버전에서 이 함수는 기본적으로 해당 함수 연결의 약칭이었습니다. 1.3.4 버전부터, ST_DWithin 함수는 대용량 버퍼 지역에 대해 이전 버전보다 더 효율적으로 만들어주는 단락(short-circuit) 거리 함수를 더욱 사용하고 있습니다.



3 차원 도형의 경우 ST_3DDWithin 함수를 이용하십시오.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다.

1.5.0 버전부터 지리형을 지원합니다.

개선 사항: 2.1.0 버전부터 지리형에 대한 속도가 향상됐습니다. 자세한 내용은 Making Geography faster 를 참조하십시오.

개선 사항: 2.1.0 버전부터 만곡 도형을 지원하기 시작했습니다.

예시

```
-- 각 학교에서 3,000 단위 내에 있는 병원 가운데
-- 각 학교에서 가장 가까운 병원을 찾습니다.
-- 인덱스를 활용해서, 인덱스를 사용할 수 없는 ST_Distance 가 공간처리해야 하는
-- 검색 목록을 제한하기 위해 ST_DWithin 검색을 실행합니다.
-- 공간 참조 시스템의 단위가 미터라면 단위도 미터가 됩니다.
SELECT DISTINCT ON (s.gid) s.gid, s.school_name, s.the_geom, h.hospital_name
    FROM schools s
        LEFT JOIN hospitals h ON ST_DWithin(s.the_geom, h.the_geom, 3000)
    ORDER BY s.gid, ST_Distance(s.the_geom, h.the_geom);

-- 가까운 병원이 없는 학교들
-- 학교에서 3,000 단위 내에 병원이 없는 모든 학교를 찾습니다.
-- 이때 단위는 공간 참조 시스템의 단위입니다(예: 미터, 피트, 도)
SELECT s.gid, s.school_name
    FROM schools s
        LEFT JOIN hospitals h ON ST_DWithin(s.the_geom, h.the_geom, 3000)
    WHERE h.gid IS NULL;
```

참고: ST_Distance, ST_Expand

8.9.33. ST_Equals

ST_Equals — 주어진 도형들이 동일한 도형일 경우 참을 반환합니다. 방향성은 무시합니다.

Synopsis

boolean ST_Equals(geometry *A*, geometry *B*);

설명: 주어진 도형들이 "공간적으로 동등"한 경우 참을 반환합니다. '='보다 '나은' 답을 원한다면 이 함수를 이용하십시오. 공간적으로 동등하다는 것은 ST_Within(A,B) = true 이고 ST_Within(B,A) = true 이며 또 포인트의 정렬 순서는 다를 수 있지만 동일한 도형 구조를 표현한다는 의미입니다. 포인트의 순서가 일관된다는 사실을 검증하려면, ST_OrderingEquals 함수를 이용하십시오(ST_OrderingEquals 가 단순히 포인트 순서가 동일한지 검증하는 것보다 조금 더 엄중한 함수라는 점은 언급해야겠군요).

 이 함수는 두 도형의 바이너리가 동등할지라도 어느 한 쪽이 유효하지 않을 경우 거짓을 반환할 것입니다.

 GEOMETRYCOLLECTION 을 인수로 호출하지 마십시오.

 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다. s2.1.1.2

 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 5.1.24

예시

```
SELECT ST_Equals(ST_GeomFromText('LINESTRING(0 0, 10 10)'),
        ST_GeomFromText('LINESTRING(0 0, 5 5, 10 10)'));
 st_equals
-----------
 t
(1 row)

SELECT ST_Equals(ST_Reverse(ST_GeomFromText('LINESTRING(0 0, 10 10)')),
        ST_GeomFromText('LINESTRING(0 0, 5 5, 10 10)'));
 st_equals
-----------
 t
(1 row)
```

참고: ST_IsValid, ST_OrderingEquals, ST_Reverse, ST_Within

8.9.34. ST_GeometricMedian

ST_GeometricMedian — 멀티 포인트의 기하학적 중심값(median)을 반환합니다.

Synopsis

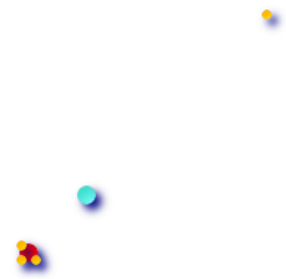
geometry ST_GeometricMedian (geometry *g* , float8 *tolerance* , int *max_iter* , boolean *fail_if_not_converged*);

설명: 바이츠펠트(Weiszfeld) 알고리즘을 이용해서 멀티포인트 도형의 기하학적 중심값의 근사치를 계산합니다. 기하학적 중심값은 중심점(centroid)보다 특이 포인트(outlier point)에 덜 민감한 중심성 척도를 제공합니다. 이 알고리즘은 연속적인 반복 과정 중 거리가 설정한 tolerance 파라미터 미만이 될 때까지 연산을 반복할 것입니다. max_iterations 횟수만큼 반복한 이후에도 이 조건을 만족시키지 못 할 경우, fail_if_not_converged 가 거짓으로 설정되지 않았다면 이 함수는 오류를 발생시키고 중단될 것입니다. tolerance 값을 설정하지 않았다면, 입력 도형의 범위에 기반한 tolerance 기본값을 계산할 것입니다.

2.3.0 버전부터 사용할 수 있습니다.

✓ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시



포인트 4 개로 이루어진 멀티포인트(노란색 포인트들)의 중심점(청록색 포인트)과 기하학적 중심값(빨간색 포인트)의 비교

```
WITH test AS (
SELECT 'MULTIPOINT((0 0), (1 1), (2 2), (200 200))'::geometry geom)
SELECT
 ST_AsText(ST_Centroid(geom)) centroid,
 ST_AsText(ST_GeometricMedian(geom)) median
FROM test;
   centroid    |          median
-------------------+-----------------------------------------
 POINT(50.75 50.75) | POINT(1.9761550281255 1.9761550281255)
(1 row)
```

참고: ST_Centroid

8.9.35. ST_HasArc

ST_HasArc — 도형 또는 도형 집합이 원호 스트링을 담고 있을 경우 참을 반환합니다.

Synopsis

boolean ST_HasArc(geometry *geomA*);

설명: 도형 또는 도형 집합이 원호 스트링을 담고 있을 경우 참을 반환합니다.

1.2.3 버전(?)부터 사용할 수 있습니다.

✓ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✓ 이 방법은 원형 스트링 및 곡선을 지원합니다.

예시

```
SELECT ST_HasArc(ST_Collect('LINESTRING(1 2, 3 4, 5 6)', 'CIRCULARSTRING(1 1, 2 3, 4 5, 6 7,
5 6)'));
        st_hasarc
        --------
        t
```

참고: ST_CurveToLine, ST_LineToCurve

8.9.36. ST_Intersects

ST_Intersects — 도형/지리형이 "2 차원에서 공간적으로 교차"(공간의 일부라도 공유)할 경우 참을 반환합니다. 그렇지 않을 경우 (도형/지리형이 서로 분리돼 있을 경우) 거짓을 반환합니다. 지리형의 경우 허용 오차가 0.00001 미터입니다(따라서 10 마이크로미터보다 가까운 포인트들은 교차한다고 여깁니다).

Synopsis

boolean ST_Intersects(geometry *geomA* , geometry *geomB*);

boolean ST_Intersects(geography *geogA* , geography *geogB*);

설명: 도형 또는 지리형이 공간의 일부라도 공유할 경우, 교차하는 것입니다. 지리형의 경우 허용 오차가 0.00001 미터입니다(따라서 10 마이크로미터보다 가까운 포인트들은 교차한다고 여깁니다).

겹치기(overlap), 접촉(touch), 범위 내(within) 모두 공간 교차를 의미하는 말입니다. 앞에서 언급한 함수들 가운데 어느 하나라도 참을 반환하는 경우, 해당 도형들도 공간적으로 교차하는 겁니다. 도형들이 교차하는 경우, 분리(disjoint) 함수는 거짓을 반환합니다.

! 이 함수의 도형 버전에 GEOMETRYCOLLECTION 을 인수로 호출하지 마십시오. 지리형 버전은 GEOMETRYCOLLECTION 이 거리 구현을 둘러싼 얇은 래퍼(wrapper)이기 때문에 도형 집합을 지원합니다.

개선 사항: 2.3.0 버전에서 PIP 단락(short-circuit)이 포인트가 몇 개 없는 멀티포인트를 지원하도록 확장/개선됐습니다. 예전 버전은 폴리곤 내부에 있는 포인트만 지원했습니다.

GEOS 모듈로 실행(도형에 대해), 지리형은 자체 지원

1.5.0 버전부터 지리형을 지원합니다.

Note! 이 함수를 호출하면 도형에서 이용할 수 있는 모든 인덱스를 활용하는 경계 상자 비교 작업을 자동적으로 포함하게 됩니다.

Note! 지리형의 경우, 이 함수는 약 0.00001 미터의 거리 오차를 허용하며 회전타원체 계산보다는 구체 계산을 이용합니다.


주의: 이 함수는 정수가 아니라 불값을 반환하는, "사용할 만한" 버전입니다.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다. s2.1.1.2 //s2.1.13.3 - ST_Intersects(g1, g2) --> Not (ST_Disjoint(g1, g2))

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 5.1.27

✔ SFCGAL 백엔드도 이 방법을 제공합니다.

도형 예시

```
SELECT ST_Intersects('POINT(0 0)'::geometry, 'LINESTRING ( 2 0, 0 2 )'::geometry);
 st_intersects
---------------
 f
(1 row)
SELECT ST_Intersects('POINT(0 0)'::geometry, 'LINESTRING ( 0 0, 0 2 )'::geometry);
 st_intersects
---------------
 t
(1 row)
```

지리형 예시

```
SELECT ST_Intersects(
         ST_GeographyFromText('SRID=4326;LINESTRING(-43.23456 72.4567,-43.23456 72.4568)'),
         ST_GeographyFromText('SRID=4326;POINT(-43.23456 72.4567772)')
         );

 st_intersects
---------------
t
```

참고: ST_3DIntersects, ST_Disjoint

8.9.37. ST_Length

ST_Length — 도형이 라인스트링 또는 멀티라인스트링일 경우 도형의 2 차원 길이를 반환합니다. 도형의 단위는 공간 참조 시스템 단위이며 지리형의 단위는 (기본값 회전타원체의) 미터 단위입니다.

Synopsis

float ST_Length(geometry *a_2dlinestring*);

float ST_Length(geography *geog*, boolean *use_spheroid=true*);

설명: 도형의 경우: 도형이 라인스트링, 멀티라인스트링, ST_Curve, ST_MultiCurve 일 경우 도형의 2 차원 데카르트 길이를 반환합니다. 면 도형의 경우 0 을 반환합니다. 면 도형에 대해서는 ST_Perimeter 를 이용하십시오. 도형 유형의 경우, 도형의 공간 참조 시스템이 해당 길이의 측정 단위를 설정합니다.

지리형의 경우: 측지 역 문제(geodesic inverse problem)를 이용해서 계산을 수행합니다. 이때 길이 단위는 미터입니다. PostGIS 를 PROJ 4.8.0 이상 버전과 함께 컴파일한 경우, SRID 가 회정타원체를 정의합니다. SRID 가 없는 경우 오직 WGS84 로 설정됩니다. use_spheroid=false 일 경우, 회전타원체 대신 구체 상에서 근사치를 계산할 것입니다.

도형의 경우 이 함수는 현재 ST_Length2D 와 동일하지만, 향후 더 높은 차원을 지원하기 위해 변경될 수도 있습니다.



변경 사항: 2.0.0 버전에서 중요한 변경이 이루어졌습니다. 2.0.0 이전 버전에서 이 함수에 폴리곤/멀티폴리곤 유형의 지리형을 입력하면 폴리곤/멀티폴리곤의 둘레를 반환했을 겁니다. 2.0.0 버전부터 도형 습성과 맞추기 위해 0 을 반환하도록 변경됐습니다. 폴리곤의 둘레를 원한다면 ST_Perimeter 함수를 이용하십시오.



지리형 측정시 기본값은 회전타원체 상의 측정입니다. 더 빠르지만 덜 정확한 구체를 이용하려면 ST_Length(gg,false); 를 쓰십시오.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다. s2.1.5.1

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 7.1.2, 9.3.4

1.5.0 버전부터 지리형을 지원합니다.

✔ SFCGAL 백엔드도 이 방법을 제공합니다.

도형 예시

라인스트링의 길이를 피트 단위로 반환합니다. 투영체 EPSG:2249 가 매사추세츠 주 피트 단위 평면이기 때문에 피트 단위라는 사실을 주의하십시오.

```
SELECT ST_Length(ST_GeomFromText('LINESTRING(743238 2967416,743238 2967450,743265 2967450,
743265.625 2967416,743238 2967416)',2249));
st_length
---------
 122.630744000095


-- WGS84 라인스트링을 매사추세츠 주 피트 단위 평면으로 변환
SELECT ST_Length(
    ST_Transform(
        ST_GeomFromEWKT('SRID=4326;LINESTRING(-72.1260 42.45, -72.1240 42.45666, -72.123
42.1546)'),
        26986
    )
);
st_length
---------
34309.4563576191
```

지리형 예시

WGS84 지리형 라인의 길이를 반환합니다.

```
-- 계산시 회전타원체보다는 구체를 이용합니다.
SELECT ST_Length(the_geog) As length_spheroid, ST_Length(the_geog,false) As length_sphere
FROM (SELECT ST_GeographyFromText(
'SRID=4326;LINESTRING(-72.1260 42.45, -72.1240 42.45666, -72.123 42.1546)') As the_geog)
As foo;
 length_spheroid  |  length_sphere
------------------+-----------------
 34310.5703627288 | 34346.206096074234309.4563576191
```

참고: ST_GeographyFromText, ST_GeomFromEWKT, ST_LengthSpheroid, ST_Perimeter, ST_Transform

8.9.38. ST_Length2D

ST_Length2D — 도형이 라인스트링 또는 멀티라인스트링일 경우 도형의 2 차원 길이를 반환합니다. 이 함수는 ST_Length 와 동일합니다.

Synopsis

float ST_Length2D(geometry *a_2dlinestring*);

설명: 도형이 라인스트링 또는 멀티라인스트링일 경우 도형의 2 차원 길이를 반환합니다. 이 함수는 ST_Length 와 동일합니다.

참고: ST_Length, ST_3DLength

8.9.39. ST_3DLength

ST_3DLength — 도형이 라인스트링 또는 멀티라인스트링일 경우 도형의 3 차원 또는 2 차원 길이를 반환합니다.

Synopsis

float ST_3DLength(geometry *a_3dlinestring*);

설명: 도형이 라인스트링 또는 멀티라인스트링일 경우 도형의 3 차원 또는 2 차원 길이를 반환합니다. 2 차원 라인의 경우 2 차원 길이만 반환할 것입니다(ST_Length 및 ST_Length2D 와 동일합니다).

✓ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

변경 사항: 2.0.0 미만 버전에서는 ST_Length3D 라는 명칭이었습니다.

예시

3 차원 케이블의 길이를 피트 단위로 반환합니다. 투영체 EPSG:2249 가 매사추세츠 주 피트 단위 평면이기 때문에 피트 단위라는 사실을 주의하십시오.

```
SELECT  ST_3DLength(ST_GeomFromText('LINESTRING(743238  2967416  1,743238  2967450  1,743265
2967450 3,
743265.625 2967416 3,743238 2967416 3)',2249));
ST_3DLength
-----------
122.704716741457
```

참고: ST_Length, ST_Length2D

8.9.40. ST_LengthSpheroid

ST_LengthSpheroid — 타원체 상에 있는 도형의 2 차원 또는 3 차원 길이/둘레를 계산합니다. 이 함수는 도형의 좌표가 경도/위도이며 재투영하지 않고 길이를 얻고 싶을 때 유용합니다.

Synopsis

float ST_LengthSpheroid(geometry *a_geometry*, spheroid *a_spheroid*);

설명: 타원체 상에 있는 도형의 길이/둘레를 계산합니다. 이 함수는 도형의 좌표가 경도/위도이며 재투영하지 않고 길이를 얻고 싶을 때 유용합니다. 이 타원체는 개별적인 데이터베이스 유형으로 다음과 같이 작성할 수 있습니다:

```
SPHEROID[<NAME>,<SEMI-MAJOR AXIS>,<INVERSE FLATTENING>]
```

```
SPHEROID["GRS_1980",6378137,298.257222101]
```

1.2.2 버전부터 사용할 수 있습니다.

변경 사항: 2.2.0 미만 버전에서는 ST_Length_Spheroid 라는 명칭이었으며, ST_3DLength_Spheroid 라는 동일 함수가 있었습니다.

✓ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
SELECT ST_LengthSpheroid( geometry_column,
               'SPHEROID["GRS_1980",6378137,298.257222101]' )
               FROM geometry_table;

SELECT ST_LengthSpheroid( the_geom, sph_m ) As tot_len,
ST_LengthSpheroid(ST_GeometryN(the_geom,1), sph_m) As len_line1,
ST_LengthSpheroid(ST_GeometryN(the_geom,2), sph_m) As len_line2
               FROM (SELECT ST_GeomFromText('MULTILINESTRING((-118.584 38.374,-118.583 38.5),
    (-71.05957 42.3589 , -71.061 43))') As the_geom,
CAST('SPHEROID["GRS_1980",6378137,298.257222101]' As spheroid) As sph_m) as foo;
    tot_len    |  len_line1   |  len_line2
-----------------+-----------------+-----------------
 85204.5207562955 | 13986.8725229309 | 71217.6482333646

 -- 3D
SELECT ST_LengthSpheroid( the_geom, sph_m ) As tot_len,
ST_LengthSpheroid(ST_GeometryN(the_geom,1), sph_m) As len_line1,
ST_LengthSpheroid(ST_GeometryN(the_geom,2), sph_m) As len_line2
```

```
              FROM  (SELECT  ST_GeomFromEWKT('MULTILINESTRING((-118.584  38.374  20,-118.583
38.5 30),
    (-71.05957 42.3589 75, -71.061 43 90))') As the_geom,
CAST('SPHEROID["GRS_1980",6378137,298.257222101]' As spheroid) As sph_m) as foo;

     tot_len    |  len_line1   |  len_line2
-----------------+-----------------+-----------------
 85204.5259107402 | 13986.876097711 | 71217.6498130292
```

참고: ST_GeometryN, ST_Length

8.9.41. ST_Length2D_Spheroid

ST_Length2D_Spheroid — 타원체 상에 있는 도형의 2 차원 길이/둘레를 계산합니다. 이 함수는 도형의 좌표가 경도/위도이며 재투영하지 않고 길이를 얻고 싶을 때 유용합니다.

Synopsis

float ST_Length2D_Spheroid(geometry *a_geometry*, spheroid *a_spheroid*);

설명: 타원체 상에 있는 도형의 2 차원 길이/둘레를 계산합니다. 이 함수는 도형의 좌표가 경도/위도이며 재투영하지 않고 길이를 얻고 싶을 때 유용합니다. 이 타원체는 개별적인 데이터베이스 유형으로 다음과 같이 작성할 수 있습니다:

```
SPHEROID[<NAME>,<SEMI-MAJOR AXIS>,<INVERSE FLATTENING>]
```

```
SPHEROID["GRS_1980",6378137,298.257222101]
```



이 함수는 계산 과정에서 Z 좌표를 무시한다는 점을 제외하면 ST_LengthSpheroid 와 거의 비슷합니다.

예시

```
SELECT ST_Length2D_Spheroid( geometry_column,
              'SPHEROID["GRS_1980",6378137,298.257222101]' )
              FROM geometry_table;

SELECT ST_Length2D_Spheroid( the_geom, sph_m ) As tot_len,
ST_Length2D_Spheroid(ST_GeometryN(the_geom,1), sph_m) As len_line1,
ST_Length2D_Spheroid(ST_GeometryN(the_geom,2), sph_m) As len_line2
              FROM (SELECT ST_GeomFromText('MULTILINESTRING((-118.584 38.374,-118.583 38.5),
    (-71.05957 42.3589 , -71.061 43))') As the_geom,
CAST('SPHEROID["GRS_1980",6378137,298.257222101]' As spheroid) As sph_m) as foo;
    tot_len    |  len_line1   |  len_line2
-----------------+-----------------+-----------------
 85204.5207562955 | 13986.8725229309 | 71217.6482333646

-- 3D 도 동일한 답을 관찰할 수 있습니다.
SELECT ST_Length2D_Spheroid( the_geom, sph_m ) As tot_len,
ST_Length2D_Spheroid(ST_GeometryN(the_geom,1), sph_m) As len_line1,
ST_Length2D_Spheroid(ST_GeometryN(the_geom,2), sph_m) As len_line2
```

```
                FROM  (SELECT  ST_GeomFromEWKT('MULTILINESTRING((-118.584  38.374  20,-118.583
38.5 30),
    (-71.05957 42.3589 75, -71.061 43 90))') As the_geom,
CAST('SPHEROID["GRS_1980",6378137,298.257222101]' As spheroid) As sph_m) as foo;

    tot_len    |  len_line1   |  len_line2
------------------+------------------+-----------------
 85204.5207562955 | 13986.8725229309 | 71217.6482333646 85204.5259107402 | 13986.876097711 |
71217.6498130292
```

참고: ST_GeometryN, ST_LengthSpheroid

8.9.42. ST_LongestLine

ST_LongestLine — 두 도형 사이의 2 차원 최장 라인을 반환합니다. 하나 이상의 최장 라인이 있을 경우, 이 함수는 자신이 찾은 첫 번째 최장 라인만 반환할 것입니다. 반환되는 라인은 항상 g1 에서 시작해서 g2 에서 끝납니다. 이 함수가 반환하는 라인의 길이는 ST_MaxDistance 함수가 g1 과 g2 에 대해 반환하는 길이와 언제나 동일합니다.

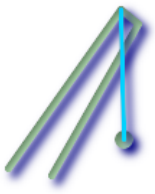
Synopsis

geometry ST_LongestLine(geometry *g1*, geometry *g2*);

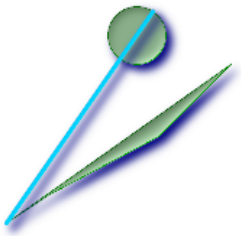
설명: 두 도형 사이의 2 차원 최장 라인을 반환합니다.

1.5.0 버전부터 사용할 수 있습니다.

예시



포인트와 라인 사이의 최장 라인



폴리곤과 폴리곤 사이의 최장 라인

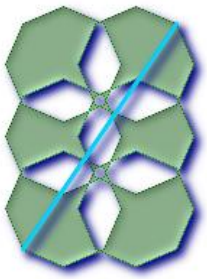
```
SELECT ST_AsText(
    ST_LongestLine(
        ST_GeomFromText('POLYGON((175 150, 20
40,
            50 60, 125 100, 175 150))'),
        ST_Buffer(ST_GeomFromText('POINT(110
170)'), 20)
        )
    ) As llinewkt;

  lline
-----------------
```

```
SELECT ST_AsText(
    ST_LongestLine('POINT(100
100)'::geometry,
        'LINESTRING (20 80, 98 190, 110
180, 50 75 )'::geometry)
    ) As lline;


 lline
-----------------
LINESTRING(100 100,98 190)
```

```
LINESTRING(20                40,121.111404660392
186.629392246051)
```



어떤 도시의 한 부분에서 다른 부분으로 여행하는 경우의 최장 직선 거리(최장 거리가 라인의 길이라는 점에 주의하십시오)

```
SELECT ST_AsText(ST_LongestLine(c.the_geom, c.the_geom)) As llinewkt,
    ST_MaxDistance(c.the_geom,c.the_geom) As max_dist,
    ST_Length(ST_LongestLine(c.the_geom, c.the_geom)) As lenll
FROM (SELECT ST_BuildArea(ST_Collect(the_geom)) As the_geom
    FROM (SELECT ST_Translate(ST_SnapToGrid(ST_Buffer(ST_Point(50 ,generate_series(50,190,
50)
            ),40, 'quad_segs=2'),1), x, 0) As the_geom
            FROM generate_series(1,100,50) As x) AS foo
) As c;

     llinewkt      |   max_dist   |    lenll
---------------------------+------------------+------------------
 LINESTRING(23 22,129 178) | 188.605408193933 | 188.605408193933
```

참고: ST_MaxDistance, ST_ShortestLine, ST_LongestLine

8.9.43. ST_OrderingEquals

ST_OrderingEquals — 주어진 도형들이 동일한 도형이며 포인트들의 순서가 동일한 방향일 경우 참을 반환합니다.

Synopsis

boolean ST_OrderingEquals(geometry *A*, geometry *B*);

설명: ST_OrderingEquals 함수는 두 도형을 비교해서 도형들이 동일하며 좌표들이 동일한 순서일 경우 t(참)를 반환하고, 그렇지 않을 경우 f(거짓)를 반환합니다.



이 함수는 SQL-MM 보다는 ArcSDE SQL 사양에 맞춰 구현됐습니다.
http://edndoc.esri.com/arcsde/9.1/sql_api/sqlapi3.htm#ST_OrderingEquals
This method implements the SQL/MM specification. SQL-MM 3: 5.1.43

예시

```
SELECT ST_OrderingEquals(ST_GeomFromText('LINESTRING(0 0, 10 10)'),
        ST_GeomFromText('LINESTRING(0 0, 5 5, 10 10)'));
 st_orderingequals
-------------------
 f
(1 row)

SELECT ST_OrderingEquals(ST_GeomFromText('LINESTRING(0 0, 10 10)'),
        ST_GeomFromText('LINESTRING(0 0, 0 0, 10 10)'));
 st_orderingequals
-------------------
 t
(1 row)

SELECT ST_OrderingEquals(ST_Reverse(ST_GeomFromText('LINESTRING(0 0, 10 10)')),
        ST_GeomFromText('LINESTRING(0 0, 0 0, 10 10)'));
 st_orderingequals
-------------------
 f
(1 row)
```

참고: ST_Equals, ST_Reverse

8.9.44. ST_Overlaps

ST_Overlaps — 도형들이 공간을 공유하고, 동일한 차원이지만, 어느 한 쪽이 다른 쪽을 완전히 담고 있지 않을 경우 참을 반환합니다.

Synopsis

boolean ST_Overlaps(geometry *A*, geometry *B*);

설명: 도형들이 "공간적으로 중첩(overlap)"할 경우 참을 반환합니다. 즉 도형들이 교차하지만, 어느 한 쪽이 다른 쪽을 완전히 담고 있지 않다는 뜻입니다.

GEOS 모듈로 실행



도형 집합을 인수로 호출하지 마십시오.

이 함수를 호출하면 도형에서 이용할 수 있는 모든 인덱스를 활용하는 경계 상자 비교 작업을 자동적으로 포함하게 됩니다. 인덱스 활용을 피하려면, _ST_Overlaps 함수를 이용하십시오.

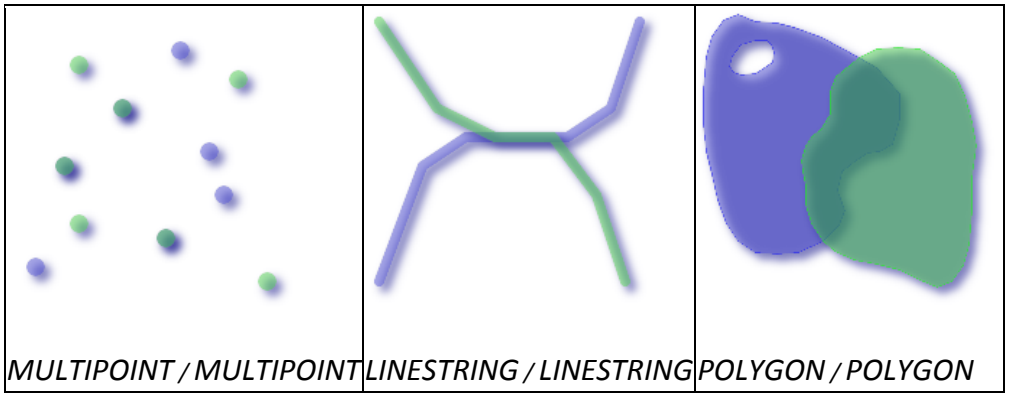
주의: 이 함수는 정수가 아니라 불값을 반환하는, "사용할 만한" 버전입니다.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다. s2.1.1.2 // s2.1.13.3

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 5.1.32

예시

다음 그림들은 모두 TRUE 를 반환합니다.



MULTIPOINT / MULTIPOINT | *LINESTRING / LINESTRING* | *POLYGON / POLYGON*

-- 라인 상에 있는 포인트는 라인에 담겨 있으며 라인보다 낮은 차원이기 때문에, 라인과 중첩하거나 교차하지 않습니다.

```
SELECT ST_Overlaps(a,b) As a_overlap_b,
    ST_Crosses(a,b) As a_crosses_b,
        ST_Intersects(a, b) As a_intersects_b, ST_Contains(b,a) As b_contains_a
FROM (SELECT ST_GeomFromText('POINT(1 0.5)') As a, ST_GeomFromText('LINESTRING(1 0, 1 1, 3
5)') As b)
    As foo

a_overlap_b | a_crosses_b | a_intersects_b | b_contains_a
------------+-------------+----------------+--------------
f       | f       | t         | t

-- 원이 부분적으로 담고 있지만 완전히 담고 있지 않은 라인은 교차한다(intersect) 또는
-- 가로지른다(cross)고 정의되지만, 차원수가 다르기 때문에 중첩하지는 않습니다.
SELECT ST_Overlaps(a,b) As a_overlap_b, ST_Crosses(a,b) As a_crosses_b,
    ST_Intersects(a, b) As a_intersects_b,
    ST_Contains(a,b) As a_contains_b
FROM     (SELECT     ST_Buffer(ST_GeomFromText('POINT(1     0.5)'),     3)          As     a,
ST_GeomFromText('LINESTRING(1 0, 1 1, 3 5)') As b)
    As foo;

 a_overlap_b | a_crosses_b | a_intersects_b | a_contains_b
-------------+-------------+----------------+--------------
 f       | t       | t         | f

-- 원과 교차하지만 원에 완전히 담겨 있지는 않은, 버퍼가 적용된 2 차원 라인스트링은
-- 두 도형이 동일한 차원이기 때문에 중첩한다고 정의되지만,
-- 두 도형의 교차점이 두 도형의 최대 차원과 동일한 차원이기 때문에
-- 가로지른다(cross)고 할 수는 없습니다.

SELECT ST_Overlaps(a,b) As a_overlap_b, ST_Crosses(a,b) As a_crosses_b, ST_Intersects(a, b)
As a_intersects_b,
```

```
ST_Contains(b,a) As b_contains_a,
ST_Dimension(a) As dim_a, ST_Dimension(b) as dim_b, ST_Dimension(ST_Intersection(a,b)) As
dima_intersection_b
FROM (SELECT ST_Buffer(ST_GeomFromText('POINT(1 0.5)'), 3) As a,
    ST_Buffer(ST_GeomFromText('LINESTRING(1 0, 1 1, 3 5)'),0.5) As b)
    As foo;

 a_overlap_b | a_crosses_b | a_intersects_b | b_contains_a | dim_a | dim_b | dima_intersection_b
-------------+-------------+----------------+--------------+-------+-------+--------------------
 t           |      f      |       t        |       f      |    2  |    2  |          2
```

참고: ST_Contains, ST_Crosses, ST_Dimension, ST_Intersects

8.9.45. ST_Perimeter

ST_Perimeter — ST_Surface 또는 ST_MultiSurface 도형 또는 지리형(폴리곤, 멀티폴리곤)의 경계선의 길이 측정치를 반환합니다. 도형의 단위는 공간 참조 시스템 단위이며 지리형의 단위는 미터 단위입니다.

Synopsis

float ST_Perimeter(geometry *g1*);

float ST_Perimeter(geography *geog*, boolean *use_spheroid=true*);

설명: 도형/지리형이 ST_Surface, ST_MultiSurface(폴리곤, 멀티폴리곤)일 경우 도형/지리형의 2 차원 둘레를 반환합니다. 면이 없는 도형의 경우 0 을 반환합니다. 선형 도형의 경우 ST_Length 를 이용하십시오. 도형 유형의 경우, 도형의 공간 참조 시스템이 해당 둘레의 측정 단위를 설정합니다.

지리형의 경우: 측지 역 문제(geodesic inverse problem)를 이용해서 계산을 수행합니다. 이때 둘레 단위는 미터입니다. PostGIS 를 PROJ 4.8.0 이상 버전과 함께 컴파일한 경우, SRID 가 회정타원체를 정의합니다. SRID 가 없는 경우 오직 WGS84 로 설정됩니다. use_spheroid=false 일 경우, 회전타원체 대신 구체 상에서 근사치를 계산할 것입니다.

이 함수는 현재 ST_Perimeter2D 와 동일하지만, 향후 더 높은 차원을 지원하기 위해 변경될 수도 있습니다.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다. s2.1.5.1

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 8.1.3, 9.5.4

개선 사항: 2.0.0 버전부터 지리형을 지원합니다.

예시: 도형

폴리곤 및 멀티폴리곤의 둘레를 피트 단위로 반환합니다. 투영체 EPSG:2249 가 매사추세츠 주 피트 단위 평면이기 때문에 피트 단위라는 사실을 주의하십시오.

```
SELECT ST_Perimeter(ST_GeomFromText('POLYGON((743238 2967416,743238 2967450,743265 2967450,
743265.625 2967416,743238 2967416))', 2249));
st_perimeter
---------
 122.630744000095
(1 row)
```

```
SELECT ST_Perimeter(ST_GeomFromText('MULTIPOLYGON(((763104.471273676 2949418.44119003,
763104.477769673 2949418.42538203,
763104.189609677 2949418.22343004,763104.471273676 2949418.44119003)),
((763104.471273676 2949418.44119003,763095.804579742 2949436.33850239,
763086.132105649 2949451.46730207,763078.452329651 2949462.11549407,
763075.354136904 2949466.17407812,763064.362142565 2949477.64291974,
763059.953961626 2949481.28983009,762994.637609571 2949532.04103014,
762990.568508415 2949535.06640477,762986.710889563 2949539.61421415,
763117.237897679 2949709.50493431,763235.236617789 2949617.95619822,
763287.718121842 2949562.20592617,763111.553321674 2949423.91664605,
763104.471273676 2949418.44119003)))', 2249));
st_perimeter
---------
 845.227713366825
(1 row)
```

예시: 지리형

폴리곤 및 멀티폴리곤의 둘레를 미터 단위로 반환합니다. 지리형이기 때문에 투영체가 WGS84 경위도라는 점에 주의하십시오.

```
SELECT ST_Perimeter(geog) As per_meters, ST_Perimeter(geog)/0.3048 As per_ft
FROM     ST_GeogFromText('POLYGON((-71.1776848522251     42.3902896512902,-71.1776843766326
42.3903829478009,
-71.1775844305465   42.3903826677917,-71.1775825927231   42.3902893647987,-71.1776848522251
42.3902896512902))') As geog;

  per_meters   |   per_ft
-----------------+------------------
37.3790462565251 | 122.634666195949


-- 멀티폴리곤 예시 --
SELECT   ST_Perimeter(geog)  As  per_meters,  ST_Perimeter(geog,false)  As  per_sphere_meters,
ST_Perimeter(geog)/0.3048 As per_ft
FROM   ST_GeogFromText('MULTIPOLYGON(((-71.1044543107478   42.340674480411,-71.1044542869917
42.3406744369506,
-71.1044553562977 42.340673886454,-71.1044543107478 42.340674480411)),
((-71.1044543107478   42.340674480411,-71.1044860600303   42.3407237015564,-71.1045215770124
42.3407653385914,
-71.1045498002983   42.3407946553165,-71.1045611902745   42.3408058316308,-71.1046016507427
42.340837442371,
-71.104617893173   42.3408475056957,-71.1048586153981   42.3409875993595,-71.1048736143677
42.3409959528211,
-71.1048878050242 42.3410084812078,-71.1044020965803 42.3414730072048,
-71.1039672113619 42.3412202916693,-71.1037740497748 42.3410666421308,
-71.1044280218456 42.3406894151355,-71.1044543107478 42.340674480411)))') As geog;

  per_meters   | per_sphere_meters |   per_ft
------------------+-------------------+------------------
257.634283683311 | 257.412311446337 | 845.256836231335
```

참고: ST_GeogFromText, ST_GeomFromText, ST_Length

8.9.46. ST_Perimeter2D

ST_Perimeter2D — 도형이 폴리곤 또는 멀티폴리곤일 경우 도형의 2 차원 둘레를 반환합니다. 이 함수는 현재 ST_Perimeter 와 동일합니다.

Synopsis

float ST_Perimeter2D(geometry *geomA*);

설명: 도형이 폴리곤 또는 멀티폴리곤일 경우 도형의 2 차원 둘레를 반환합니다.



이 함수는 현재 ST_Perimeter 와 동일합니다. 향후 버전에서 ST_Perimeter 는 도형에 대해 최고 차원의 둘레를 반환할 수도 있습니다. 이는 아직 고려 대상입니다.

참고: ST_Perimeter

8.9.47. ST_3DPerimeter

ST_3DPerimeter — 도형이 폴리곤 또는 멀티폴리곤일 경우 도형의 3 차원 둘레를 반환합니다.

Synopsis

float ST_3DPerimeter(geometry *geomA*);

설명: 도형이 폴리곤 또는 멀티폴리곤일 경우 도형의 3 차원 둘레를 반환합니다. 2 차원 도형의 경우 2 차원 둘레를 반환할 것입니다.



이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

변경 사항: 2.0.0 미만 버전에서는 ST_Perimeter3D 라는 명칭이었습니다.

예시

매사추세츠 주 피트 단위 평면에서 공중에 살짝 떠 있는 폴리곤의 둘레

```
SELECT ST_3DPerimeter(the_geom), ST_Perimeter2d(the_geom), ST_Perimeter(the_geom) FROM
             (SELECT ST_GeomFromEWKT('SRID=2249;POLYGON((743238 2967416 2,743238 2967450 1,
743265.625 2967416 1,743238 2967416 2))') As the_geom) As foo;

 ST_3DPerimeter | st_perimeter2d |  st_perimeter
------------------+------------------+------------------
105.465793597674 | 105.432997272188 | 105.432997272188 257.634283683311 | 257.412311446337
| 845.256836231335
```

참고: ST_GeomFromEWKT, ST_Perimeter, ST_Perimeter2D

8.9.48. ST_PointOnSurface

ST_PointOnSurface — 표면에 붙어 있게 될 POINT 를 반환합니다.

Synopsis

geometry ST_PointOnSurface(geometry *g1*);

설명: 표면과 교차하게 될 POINT 를 반환합니다.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다. s3.2.14.2 // s3.2.18.2

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 8.1.5, 9.5.6. 사양에 따라, ST_PointOnSurface 는 표면을 가진 도형(POLYGONs, MULTIPOLYGONS, CURVED POLYGONS)을 입력받습니다. 따라서 PostGIS 가 사양서가 허용한 범위를 확장시키는 것 같습니다. Oracle, DB II, ESRI SDE 등 대부분의 데이터베이스는 표면에 대해서만 이 함수를 지원합니다. SQL Server 2008 은, PostGIS 처럼, 모든 일반 도형을 지원합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
SELECT ST_AsText(ST_PointOnSurface('POINT(0 5)'::geometry));
 st_astext
------------
 POINT(0 5)
(1 row)

SELECT ST_AsText(ST_PointOnSurface('LINESTRING(0 5, 0 10)'::geometry));
 st_astext
------------
 POINT(0 5)
(1 row)

SELECT ST_AsText(ST_PointOnSurface('POLYGON((0 0, 0 5, 5 5, 5 0, 0 0))'::geometry));
  st_astext
----------------
 POINT(2.5 2.5)
(1 row)

SELECT ST_AsEWKT(ST_PointOnSurface(ST_GeomFromEWKT('LINESTRING(0 5 1, 0 0 1, 0 10 2)')));
  st_asewkt
----------------
 POINT(0 0 1)
(1 row)
```

참고: ST_Centroid, ST_PointInsideCircle

8.9.49. ST_Project

ST_Project — 시작점으로부터 미터 단위 거리 및 라디안 단위 방향(방위각)을 이용해 투영된 POINT 를 반환합니다.

Synopsis

geography ST_Project(geography *g1*, float *distance*, float *azimuth*);

설명: 시작점으로부터 미터 단위로 측정된 거리 및 라디안 단위로 측정된 방향(방위각)을 이용해 측지선을 따라 투영된 POINT 를 반환합니다. 이를 측지 주 문제(geodesic direct problem)라고도 합니다.

운항 분야에서 때로 방위각을 기수방위(heading) 또는 방향(bearing)으로 부르기도 합니다. 방위각은 진북(방위각 0)을 기준으로 측정됩니다. 동쪽은 방위각 90(π/2), 남쪽은 방위각 180(π), 서쪽은 방위각 270(3π/2)입니다.

거리의 단위는 미터입니다.

2.0.0 버전부터 사용할 수 있습니다.

예시: 도 단위 이용 - 100,000 미터 및 45 도 방향으로 포인트 투영

```
SELECT ST_AsText(ST_Project('POINT(0 0)'::geography, 100000, radians(45.0)));

         st_astext
-----------------------------------------
 POINT(0.635231029125537 0.639472334729198)
(1 row)
```

참고: ST_Azimuth, ST_Distance, PostgreSQL Math Functions

8.9.50. ST_Relate

ST_Relate — 도형의 내부 사이의 교차점을 확인해서 한 도형이 다른 도형과 공간 관계가 있을 경우 참을 반환합니다. 두 도형의 경계선 및 외부는 intersectionMatrixPattern 안의 값들로 설정됩니다. intersectionMatrixPattern 이 입력되지 않을 경우, 두 도형을 관계시키는 최대 intersectionMatrixPattern 을 반환합니다.

Synopsis

boolean ST_Relate(geometry *geomA*, geometry *geomB*, text *intersectionMatrixPattern*);

text ST_Relate(geometry *geomA*, geometry *geomB*);

text ST_Relate(geometry *geomA*, geometry *geomB*, integer *BoundaryNodeRule*);

설명: 버전 1: geomA, geomB, intersectionMatrix 를 입력받아 도형의 내부 사이의 교차점을 확인해서 한 도형이 다른 도형과 공간 관계가 있을 경우 1(참)을 반환합니다. 두 도형의 경계선 및 외부는 DE-9IM 매트릭스 패턴 안의 값들로 설정됩니다.

이 함수는 교차점, 가로지르기 등 복합(compound) 확인을 한번에 검사하는 데 특히 유용합니다.

도형 집합을 인수로 호출하지 마십시오.



이 함수는 정수가 아니라 불값을 반환하는, "사용할 만한" 버전입니다. OGC 사양서가 함수를 정의하고 있습니다.



이 함수는 인덱스 호출을 마술처럼 쉽게 포함하지 않습니다. 그 이유는 몇몇 관계성들이, 예를 들어, 반(反) 분리(disjoint)이기 때문입니다. 사용자가 교차점이 필요한 관계성 패턴을 사용하고 있다면, && 인덱스 호출을 포함시키십시오.

버전 2: geomA 와 geomB 를 입력받아 Section 4.3.6, "차원 확장 9 교차 모델(DE-9IM)" 을 반환합니다.

버전 3: 버전 2 와 동일하지만, 경계선 노드 규칙(1:OGC/MOD2, 2:Endpoint, 3:MultivalentEndpoint, 4:MonovalentEndpoint)을 설정할 수 있습니다.



도형 집합을 인수로 호출하지 마십시오.

OGC 사양서에는 없지만, 언급은 돼 있습니다. s2.1.13.2 단원을 참조하십시오.

GEOS 모듈로 실행

✓ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다. s2.1.1.2 // s2.1.13.3

✓ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 5.1.25

개선 사항: 2.0.0 버전부터 경계선 노드 규칙(GEOS 3.0 이상 버전 필요)을 설정할 수 있도록 지원합니다.

예시

```
-- 서로 교차하지만 내부 교차점들(poly)과 접촉하지는 않는 모든 복합 도형을 찾습니다.
SELECT l.* , b.name As poly_name
    FROM polys As b
INNER JOIN compounds As l
ON (p.the_geom && b.the_geom
AND ST_Relate(l.the_geom, b.the_geom,'T********'));

SELECT  ST_Relate(ST_GeometryFromText('POINT(1  2)'),  ST_Buffer(ST_GeometryFromText('POINT(1
2)'),2));
st_relate
-----------
0FFFFF212

SELECT        ST_Relate(ST_GeometryFromText('LINESTRING(1        2,        3        4)'),
ST_GeometryFromText('LINESTRING(5 6, 7 8)'));
st_relate
-----------
FF1FF0102


SELECT  ST_Relate(ST_GeometryFromText('POINT(1  2)'),  ST_Buffer(ST_GeometryFromText('POINT(1
2)'),2), '0FFFFF212');
st_relate
```

```
-----------
t

SELECT ST_Relate(ST_GeometryFromText('POINT(1 2)'), ST_Buffer(ST_GeometryFromText('POINT(1
2)'),2), '*FF*FF212');
st_relate
-----------
t
```

참고: ST_Crosses, Section 4.3.6, "차원 확장 9 교차 모델(DE-9IM)", ST_Disjoint, ST_Intersects, ST_Touches

8.9.51. ST_RelateMatch

ST_RelateMatch — intersectionMattrixPattern1 이 intersectionMattrixPattern2 를 의미할 경우 참을 반환합니다.

Synopsis

boolean ST_RelateMatch(text *intersectionMatrix*, text *intersectionMatrixPattern*);

설명: intersectionMatrix 와 intersectionMatrixPattern 을 입력받아 intersectionMatrix 가 intersectionMatrixPattern 을 만족할 경우 참을 반환합니다. 자세한 정보는 Section 4.3.6, "차원 확장 9 교차 모델(DE-9IM)" 을 참조하십시오.

2.0.0 버전부터 사용할 수 있습니다. GEOS 3.3.0 이상 버전이 필요합니다.

예시

```
SELECT ST_RelateMatch('101202FFF', 'TTTTTTFFF') ;
-- 결과 --
t
-- 유효하지 않은 도형 한 개가 관련된 관계성을 비교하는 (내부 및 경계선에서 교차하는 라인과 폴리곤)
-- 일반 교차점 매트릭스 패턴의 예시와 예시 매트릭스들
SELECT mat.name, pat.name, ST_RelateMatch(mat.val, pat.val) As satisfied
  FROM
    ( VALUES ('Equality', 'T1FF1FFF1'),
        ('Overlaps', 'T*T***T**'),
        ('Within', 'T*F**F***'),
        ('Disjoint', 'FF*FF****') As pat(name,val)
    CROSS JOIN
      (    VALUES ('Self intersections (invalid)', '111111111'),
          ('IE2_BI1_BB0_BE1_EI1_EE2', 'FF2101102'),
          ('IB1_IE1_BB0_BE0_EI2_EI1_EE2', 'F11F00212')
      ) As mat(name,val);
```

참고: Section 4.3.6, "차원 확장 9 교차 모델(DE-9IM)", ST_Relate

8.9.52. ST_ShortestLine

ST_ShortestLine — 두 도형 사이의 2 차원 최단 라인을 반환합니다.

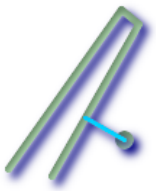
Synopsis

geometry ST_ShortestLine(geometry *g1*, geometry *g2*);

설명: 두 도형 사이의 2 차원 최단(shortest) 라인을 반환합니다. 하나 이상의 최단 라인이 있을 경우, 이 함수는 첫 번째 최단 라인만 반환할 것입니다. g1과 g2가 단 한 개의 포인트에서만 교차할 경우, 이 함수는 교차점에서 시작하고 끝나는 라인을 반환할 것입니다. g1과 g2가 한 개 이상의 포인트에서 교차할 경우, 이 함수는 동일한 포인트에서 시작하고 끝나는 라인을 반환하지만 해당 포인트는 교차하는 포인트들 가운데 어떤 포인트라도 될 수 있습니다. 반환되는 라인은 항상 g1 에서 시작해서 g2 에서 끝납니다. 이 함수가 반환하는 라인의 2 차원 길이는 ST_Distance 함수가 g1 과 g2 에 대해 반환하는 길이와 언제나 동일합니다.

1.5.0 버전부터 사용할 수 있습니다.

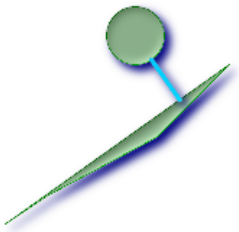
예시



포인트와 라인 사이의 최단 라인

```
SELECT ST_AsText(
    ST_ShortestLine('POINT(100 100)'::geometry,
        'LINESTRING (20 80, 98 190, 110 180, 50
75 )'::geometry)
    ) As sline;


  sline
-----------------
LINESTRING(100              100,73.0769230769231
115.384615384615)
```



폴리곤과 폴리곤 사이의 최단 라인

```
SELECT ST_AsText(
        ST_ShortestLine(
            ST_GeomFromText('POLYGON((175  150,
20 40, 50 60, 125 100, 175 150))'),

ST_Buffer(ST_GeomFromText('POINT(110    170)'),
20)
            )
        ) As slinewkt;

 LINESTRING(140.752120669087
125.695053378061,121.111404660392
153.370607753949)
```

참고: ST_ClosestPoint, ST_Distance, ST_LongestLine, ST_MaxDistance

8.9.53. ST_Touches

ST_Touches — 입력 도형들이 최소한 포인트 한 개를 공유하지만, 내부가 교차하지는 않을 경우 TRUE 를 반환합니다.

Synopsis

boolean ST_Touches(geometry *g1*, geometry *g2*);

설명: g1 과 g2 사이의 포인트만이 g1 과 g2 의 경계선들의 합집합 안에 있을 경우 TRUE 를 반환합니다. ST_Touches 관계성은 모든 면/면, 선분/선분, 선분/면, 점/면 그리고 점/선 쌍에 적용되지만 점/점 쌍에는 적용되지 않습니다.

수학적인 관점에서, 이 함수는 다음과 같이 표현됩니다:

$$a.Touches(b) \Leftrightarrow (I(a) \cap I(b) = \emptyset) \wedge (a \cap b) \neq \emptyset$$

두 도형에 대해 쓸 수 있는 DE-9IM 교차점 매트릭스는 다음과 같습니다:

- FT*******
- F**T*****
- F***T****

GEOMETRYCOLLECTION 을 인수로 호출하지 마십시오.

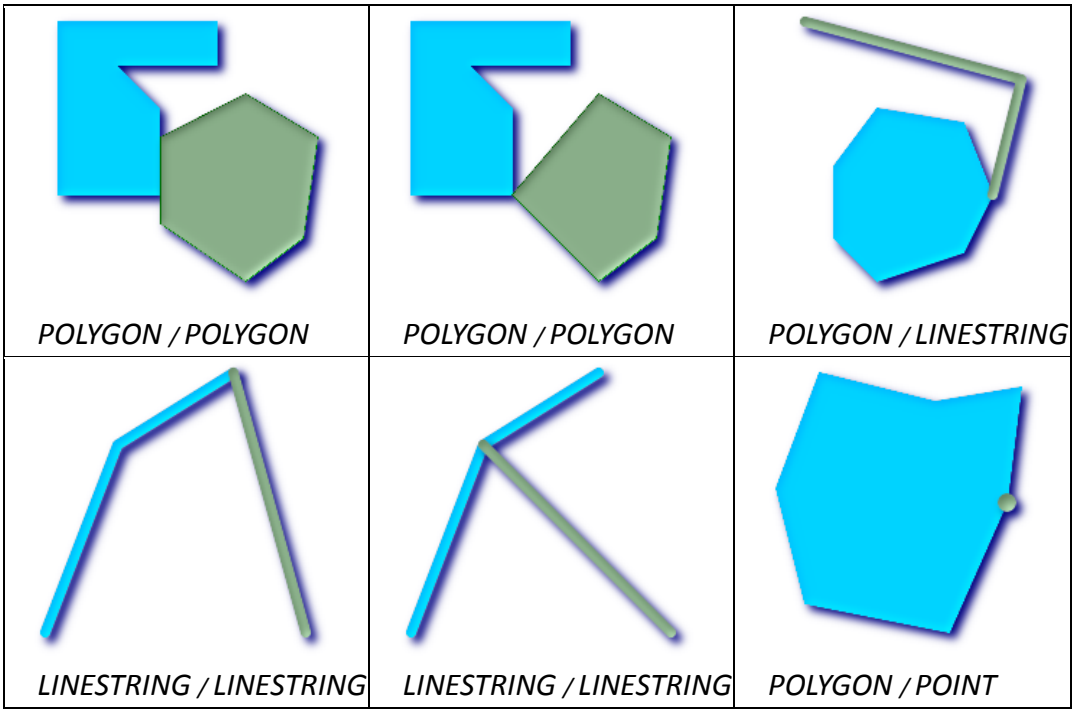
Note! 이 함수를 호출하면 도형에서 이용할 수 있는 모든 인덱스를 활용하는 경계 상자 비교 작업을 자동적으로 포함하게 됩니다. 인덱스 활용을 피하려면, _ST_Touches 함수를 대신 이용하십시오.

이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다. s2.1.1.2 // s2.1.13.3

이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 5.1.28

예시

ST_Touches 의 술부는 다음 모든 그림들에서 TRUE 를 반환합니다.



```
SELECT ST_Touches('LINESTRING(0 0, 1 1, 0 2)'::geometry, 'POINT(1 1)'::geometry);
 st_touches
------------
 f
(1 row)

SELECT ST_Touches('LINESTRING(0 0, 1 1, 0 2)'::geometry, 'POINT(0 2)'::geometry);
 st_touches
------------
 t
(1 row)
```

8.9.54. ST_Within

ST_Within — 도형 A 전체가 도형 B 안에 들어갈 경우 참을 반환합니다.

Synopsis

boolean ST_Within(geometry *A*, geometry *B*);

설명: 도형 A 가 도형 B 안에 완전히 들어 있을 경우 참을 반환합니다. 이 함수가 제대로 동작하려면, 입력 도형들 둘 다 동일한 좌표 투영체, 즉 동일한 SRID 를 가져야 합니다. ST_Within(A,B)가 참이고 ST_Contains(B,A)도 참일 경우, 두 도형은 공간적으로 동등하다고 여겨집니다.

GEOS 모듈로 실행

개선 사항: 2.3.0 버전에서 도형에 대한 PIP 단락(short-circuit)이 포인트가 몇 개 없는 멀티포인트를 지원하도록 확장/개선됐습니다. 예전 버전은 폴리곤 내부에 있는 포인트만 지원했습니다.


GEOMETRYCOLLECTION 을 인수로 호출하지 마십시오.


이 함수에 유효하지 않은 도형을 입력하지 마십시오. 결과를 예측할 수 없습니다.

이 함수를 호출하면 도형에서 이용할 수 있는 모든 인덱스를 활용하는 경계 상자 비교 작업을 자동적으로 포함하게 됩니다. 인덱스 활용을 피하려면, _ST_Within 함수를 이용하십시오.

주의: 이 함수는 정수가 아니라 불값을 반환하는, "사용할 만한" 버전입니다.


이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다. s2.1.1.2 // s2.1.13.3 - a.Relate(b, 'T*F**F***')


이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3: 5.1.30

예시

```
-- 원안에들어가는원
SELECT ST_Within(smallc,smallc) As smallinsmall,
     ST_Within(smallc, bigc) As smallinbig,
     ST_Within(bigc,smallc) As biginsmall,
     ST_Within(ST_Union(smallc, bigc), bigc) as unioninbig,
     ST_Within(bigc, ST_Union(smallc, bigc)) as biginunion,
     ST_Equals(bigc, ST_Union(smallc, bigc)) as bigisunion
FROM
(
```

```
SELECT ST_Buffer(ST_GeomFromText('POINT(50 50)'), 20) As smallc,
    ST_Buffer(ST_GeomFromText('POINT(50 50)'), 40) As bigc) As foo;
-- 결과
 smallinsmall | smallinbig | biginsmall | unioninbig | biginunion | bigisunion
--------------+------------+------------+------------+------------+------------
 t      | t     | f     | t     | t     | t
(1 row)
```



참고: ST_Contains, ST_Equals, ST_IsValid

8.10. SFCGAL 함수

SFCGAL 은 고급 2D 및 3D 함수를 제공하는 CGAL 을 둘러싼 C++ 래퍼(wrapper) 라이브러리입니다. 강력한 기능을 위해, 도형 좌표가 정밀한 유리수 표현식을 가지고 있습니다.

SFCGAL 홈페이지 http://www.sfcgal.org 에서 이 라이브러리의 설치 지침을 찾아볼 수 있습니다. 함수들을 로드하려면 확장 프로그램 postgis_sfcgal 을 생성하십시오.

SFCGAL 함수 가운데 일부가 표준 함수(ST_Intersects, ST_Intersection, ST_Difference, ST_Union, ST_Area and ST_Distance)를 대체합니다. 표준 함수와 SFCGAL 함수를 서로 바꿔가며 사용하려면:

```
SET postgis.backend = sfcgal;
```

그리고

```
SET postgis.backend = geos;
```

postgis_sfcgal_version — 실행중인 SFCGAL 의 버전을 반환합니다.

ST_Extrude — 표면을 관련 입체로 돌출시킵니다.

ST_StraightSkeleton — 도형으로부터 직선 골격(straight skeleton)을 계산합니다.

ST_ApproximateMedialAxis — 면 도형의 근사 중심축을 계산합니다.

ST_IsPlanar — 표면이 평면인지 아닌지 확인합니다.

ST_Orientation — 표면의 방향(orientation)을 결정합니다.

ST_ForceLHR — LHR(Left Hand Reverse; 시계 방향) 방향을 강제합니다.

ST_MinkowskiSum — 민코프스키 합계를 수행합니다.

ST_3DIntersection — 3 차원 교차를 수행합니다.

ST_3DDifference — 3 차원 차이를 수행합니다.

ST_3DUnion — 3 차원 통합을 수행합니다.

ST_3DArea — 3 차원 표면 도형의 면적을 계산합니다. 입체일 경우 0 을 반환할 것입니다.

ST_Tesselate — 폴리곤 또는 다면체 표면의 표면 다듬기(tesselation) 작업을 수행한 다음 TIN 또는 TIN 집합으로 반환합니다.

ST_Volume — 3 차원 입체의 부피를 계산합니다. 표면 도형을 입력하면 (닫힌 도형일지라도) 0 을 반환할 것입니다.

ST_MakeSolid — 도형을 입체로 지정합니다. 어떤 확인 작업도 수행하지 않습니다. 유효한 입체를 얻으려면, 입력 도형이 닫힌 다면체 표면 또는 닫힌 TIN 이어야만 합니다.

ST_IsSolid — 도형이 입체인지 테스트합니다. 어떤 유효성 검사도 수행하지 않습니다.

8.10.1. postgis_sfcgal_version

postgis_sfcgal_version — 실행중인 SFCGAL 의 버전을 반환합니다.

Synopsis

text postgis_sfcgal_version(void);

설명:

2.1.0 버전부터 사용할 수 있습니다.

✔ 이 방법을 쓰려면 SFCGAL 백엔드가 필요합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

8.10.2. ST_Extrude

ST_Extrude — 표면을 관련 입체로 돌출시킵니다.

Synopsis

geometry ST_Extrude(geometry *geom*, float *x*, float *y*, float *z*);

설명:

2.1.0 버전부터 사용할 수 있습니다.

✓ 이 방법을 쓰려면 SFCGAL 백엔드가 필요합니다.

✓ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

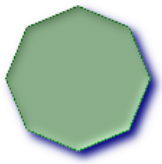
✓ 이 함수는 다면체 표면을 지원합니다.

✓ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

예시

PostGIS ST_AsX3D 를 통해 3 차원 영상을 생성한 다음 X3Dom HTML 자바스크립트 렌더링 라이브러리 를 이용해서 HTML 로 렌더링합니다.

```
SELECT
ST_Buffer(ST_GeomFromText('POINT(100
90)'),
  50, 'quad_segs=2'),0,0,30);
```



버퍼를 적용한 포인트에서 생성된 원본 팔각형

```
ST_Extrude(ST_Buffer(ST_GeomFromText('POINT(
100 90)'),
 50, 'quad_segs=2'),0,0,30);
```



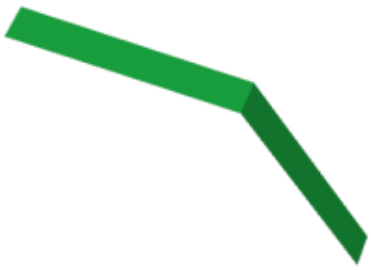
Z 축을 따라 30 단위만큼 돌출된 육각형이 다면체 표면 Z(PolyhedralSurfaceZ)을 생성합니다.

```
SELECT ST_GeomFromText('LINESTRING(50 50, 100
90, 95 150)')
```



원본 라인스트링

```
SELECT                    DropGeometryTabl
('my_schema','my_spatial_table');
```



Z 축을 따라 돌출된 라인스트링이 다면체 표면 Z(PolyhedralSurfaceZ)을 생성합니다.

참고: ST_AsX3D

8.10.3. ST_StraightSkeleton

ST_StraightSkeleton — 도형으로부터 직선 골격(straight skeleton)을 계산합니다.

Synopsis

geometry ST_StraightSkeleton(geometry *geom*);

설명:

2.1.0 버전부터 사용할 수 있습니다.

✔ 이 방법을 쓰려면 SFCGAL 백엔드가 필요합니다.

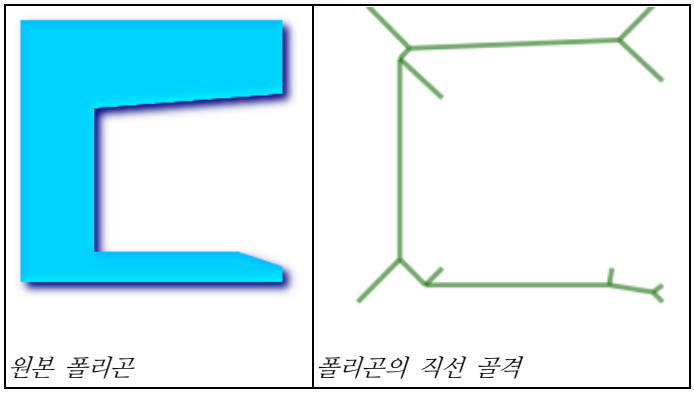
✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

예시

```
SELECT ST_StraightSkeleton(ST_GeomFromText('POLYGON (( 190 190, 10 190, 10 10, 190 10,
190 20, 160 30, 60 30, 60 130, 190 140, 190 190 ))'));
```



원본 폴리곤 | *폴리곤의 직선 골격*

8.10.4. ST_ApproximateMedialAxis

ST_ApproximateMedialAxis — 면 도형의 근사 중심축을 계산합니다.

Synopsis

geometry ST_ApproximateMedialAxis(geometry *geom*);

설명: 입력 면 도형의 직선 골격을 바탕으로 근사 중심축(approximate medial axis)을 반환합니다. 사용 가능한 버전(1.2.0 이상)에 대해 빌드했을 때 SFCGAL 에 특화된 API 를 이용합니다. 그렇지 않을 경우 이 함수는 (더 느린) ST_StraightSkeleton 함수를 둘러싼 래퍼(wrapper)에 불과합니다.

2.2.0 버전부터 사용할 수 있습니다.

✔ 이 방법을 쓰려면 SFCGAL 백엔드가 필요합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

예시

```
SELECT ST_ApproximateMedialAxis(ST_GeomFromText('POLYGON (( 190 190, 10 190, 10 10, 190
10, 190 20, 160 30, 60 30, 60 130, 190 140, 190 190 ))'));
```



폴리곤과 그 근사 중심축

참고: ST_StraightSkeleton

8.10.5. ST_IsPlanar

ST_IsPlanar — 표면이 평면인지 아닌지 확인합니다.

Synopsis

boolean ST_IsPlanar(geometry *geom*);

설명:

2.2.0 버전부터 사용할 수 있습니다. 원래 2.1.0 버전에 문서화됐지만 2.1 배포판에서는 실수로 빠졌습니다.

✔ 이 방법을 쓰려면 SFCGAL 백엔드가 필요합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

8.10.6. ST_Orientation

ST_Orientation — 표면의 방향(orientation)을 결정합니다.

Synopsis

geometry **ST_ForceLHR**(geometry *geom*);

설명: 이 함수는 폴리곤에만 적용됩니다. 폴리곤이 반시계 방향이면 -1 을, 시계 방향이면 1 을 반환합니다.

2.1.0 버전부터 사용할 수 있습니다.

✔ 이 방법을 쓰려면 SFCGAL 백엔드가 필요합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

8.10.7. ST_ForceLHR

ST_ForceLHR — LHR(Left Hand Reverse; 시계 방향) 방향을 강제합니다.

Synopsis

geometry **ST_ForceLHR**(geometry *geom*);

설명:

2.1.0 버전부터 사용할 수 있습니다.

✔ 이 방법을 쓰려면 SFCGAL 백엔드가 필요합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

8.10.8. ST_MinkowskiSum

ST_MinkowskiSum — 민코프스키 합계를 수행합니다.

Synopsis

geometry **ST_MinkowskiSum**(geometry *geom1*, geometry *geom2*);

설명: 이 함수는 폴리곤과, 포인트, 라인 또는 폴리곤의 2 차원 민코프스키 합계를 수행합니다.

두 도형 A 와 B 의 민코프스키 합계란 A 와 B 에 있는 모든 포인트를 합한, 모든 포인트들의 집합입니다. 민코프스키 합계는 운동 계획(motion planning) 및 CAD(computer-aided design) 분야에서 자주 사용됩니다. 자세한 내용은 Wikipedia Minkowski addition 을 참조하십시오.

첫 번째 파라미터는 어떤 2 차원 도형(포인트, 라인스트링, 폴리곤)도 될 수 있습니다. 만약 3 차원 도형을 입력할 경우, Z 좌표를 0 으로 치환해서 2 차원으로 변환하기 때문에 유효하지 않은 도형으로 변할 가능성이 있습니다. 두 번째 파라미터는 2 차원 폴리곤이어야만 합니다.

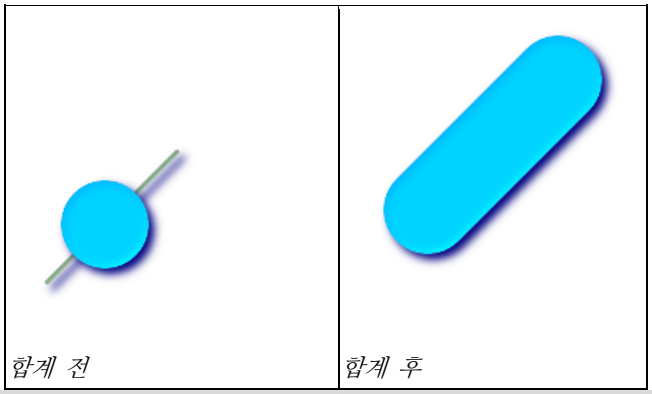
구현하는 데 CGAL 2D Minkowskisum 을 활용합니다.

2.1.0 버전부터 사용할 수 있습니다.

✔ 이 방법을 쓰려면 SFCGAL 백엔드가 필요합니다.

예시

라인스트링이 원을 뚫고 지나가는 경우 라인스트링과 원형 폴리곤의 민코프스키 합계

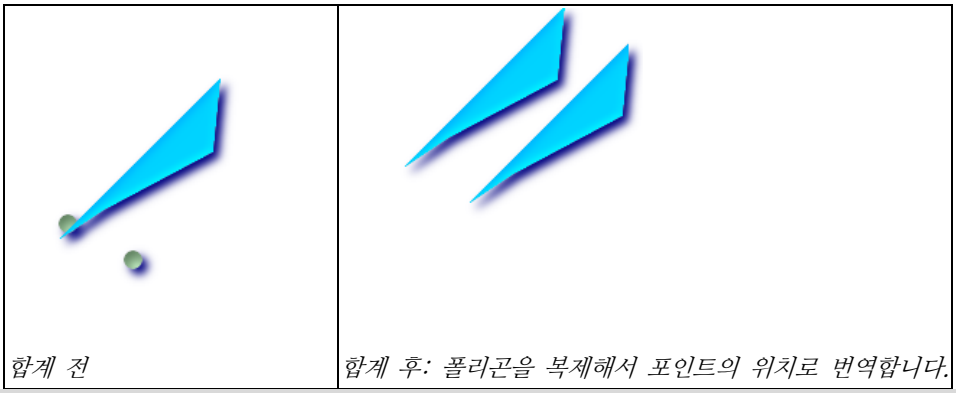


합계 전 *합계 후*

```
SELECT ST_MinkowskiSum(line, circle))
FROM (SELECT
  ST_MakeLine(ST_MakePoint(10, 10),ST_MakePoint(100, 100)) As line,
  ST_Buffer(ST_GeomFromText('POINT(50 50)'), 30) As circle) As foo;

-- WKT --
MULTIPOLYGON(((30    59.9999999999999,30.5764415879031    54.1472903395161,32.2836140246614
48.5194970290472,35.0559116309237                         43.3328930094119,38.7867965644036
38.7867965644035,43.332893009412                          35.0559116309236,48.5194970290474
32.2836140246614,54.1472903395162   30.5764415879031,60.0000000000001   30,65.8527096604839
30.5764415879031,71.4805029709527                         32.2836140246614,76.6671069905881
35.0559116309237,81.2132034355964                         38.7867965644036,171.213203435596
128.786796564404,174.944088369076                         133.332893009412,177.716385975339
138.519497029047,179.423558412097        144.147290339516,180         150,179.423558412097
155.852709660484,177.716385975339                         161.480502970953,174.944088369076
166.667106990588,171.213203435596 171.213203435596,166.667106990588 174.944088369076,
161.480502970953 177.716385975339,155.852709660484 179.423558412097,150 180,144.147290339516
179.423558412097,138.519497029047                         177.716385975339,133.332893009412
174.944088369076,128.786796564403                         171.213203435596,38.7867965644035
81.2132034355963,35.0559116309236                          76.667106990588,32.2836140246614
71.4805029709526,30.5764415879031 65.8527096604838,30 59.9999999999999)))
```

폴리곤과 멀티 포인트의 민코프스키 합계



합계 전 | *합계 후: 폴리곤을 복제해서 포인트의 위치로 번역합니다.*

```
SELECT ST_MinkowskiSum(mp, poly)
FROM (SELECT 'MULTIPOINT(25 50,70 25)'::geometry As mp,
  'POLYGON((130 150, 20 40, 50 60, 125 100, 130 150))'::geometry As poly
  ) As foo


-- WKT --
MULTIPOLYGON(
  ((70 115,100 135,175 175,225 225,70 115)),
  ((120 65,150 85,225 125,275 175,120 65))
  )
```

8.10.9. ST_3DIntersection

ST_3DIntersection — 3 차원 교차를 수행합니다.

Synopsis

geometry **ST_3DIntersection**(geometry *geom1*, geometry *geom2*);

설명: geom1 과 geom2 가 공유하는 부분을 도형으로 반환합니다.

2.1.0 버전부터 사용할 수 있습니다.

✔ 이 방법을 쓰려면 SFCGAL 백엔드가 필요합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 함수는 다면체 표면을 지원합니다.

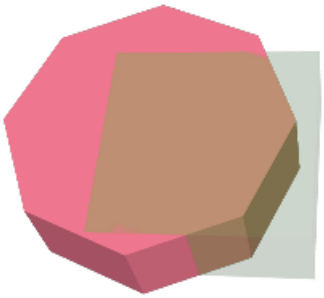
✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

예시

PostGIS ST_AsX3D 를 통해 3 차원 영상을 생성한 다음 X3Dom HTML 자바스크립트 렌더링 라이브러리 를 이용해서 HTML 로 렌더링합니다.

```
SELECT
ST_Extrude(ST_Buffer(ST_GeomFromText('POINT(1
00 90)'),
 50, 'quad_segs=2'),0,0,30) AS geom1,

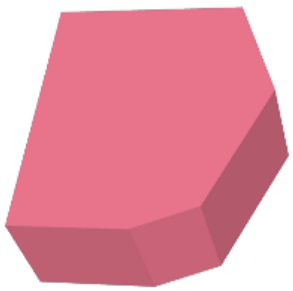
ST_Extrude(ST_Buffer(ST_GeomFromText('POINT(8
0 80)'),
 50, 'quad_segs=1'),0,0,30) AS geom2;
```



원본 3 차원 도형들이 중첩합니다. geom2 를 반투명하게 표현했습니다.

```
SELECT ST_3DIntersection(geom1,geom2)
FROM                 (                SELECT
ST_Extrude(ST_Buffer(ST_GeomFromText('POINT(
100 90)'),
 50, 'quad_segs=2'),0,0,30) AS geom1,

ST_Extrude(ST_Buffer(ST_GeomFromText('POINT(
80 80)'),
 50, 'quad_segs=1'),0,0,30) AS geom2 ) As t;
```



geom1 과 geom2 가 교차한 부분

3 차원 라인스트링과 폴리곤

```
SELECT ST_AsText(ST_3DIntersection(linestring, polygon)) As wkt
FROM ST_GeomFromText('LINESTRING Z (2 2 6,1.5 1.5 7,1 1 8,0.5 0.5 8,0 0 10)') AS linestring
 CROSS JOIN ST_GeomFromText('POLYGON((0 0 8, 0 1 8, 1 1 8, 1 0 8, 0 0 8))') AS polygon;

        wkt
-------------------------------
 LINESTRING Z (1 1 8,0.5 0.5 8)
```

정육면체(닫힌 다면체 표면)과 폴리곤 Z

```
SELECT ST_AsText(ST_3DIntersection(
        ST_GeomFromText('POLYHEDRALSURFACE Z( ((0 0 0, 0 0 1, 0 1 1, 0 1 0, 0 0 0)),
    ((0 0 0, 0 1 0, 1 1 0, 1 0 0, 0 0 0)), ((0 0 0, 1 0 0, 1 0 1, 0 0 1, 0 0 0)),
    ((1 1 0, 1 1 1, 1 0 1, 1 0 0, 1 1 0)),
    ((0 1 0, 0 1 1, 1 1 1, 1 1 0, 0 1 0)), ((0 0 1, 1 0 1, 1 1 1, 0 1 1, 0 0 1)) )'),
    'POLYGON Z ((0 0 0, 0 0 0.5, 0 0.5 0.5, 0 0.5 0, 0 0 0))'::geometry))
TIN Z (((0 0 0,0 0 0.5,0 0.5 0.5,0 0 0)),((0 0.5 0,0 0 0,0 0.5 0.5,0 0.5 0)))
```

8.10.10. ST_3DDifference

ST_3DDifference — 3 차원 차이를 수행합니다.

Synopsis

geometry **ST_3DDifference**(geometry *geom1*, geometry *geom2*);

설명: geom2 의 일부분이 아닌 geom1 의 부분을 반환합니다.

2.2.0 버전부터 사용할 수 있습니다.

✔ 이 방법을 쓰려면 SFCGAL 백엔드가 필요합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 함수는 다면체 표면을 지원합니다.

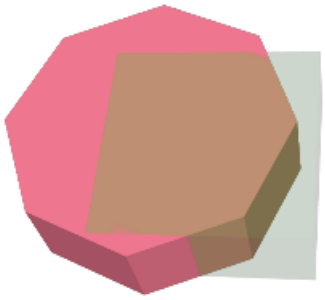
✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

예시

PostGIS ST_AsX3D 를 통해 3 차원 영상을 생성한 다음 X3Dom HTML 자바스크립트 렌더링 라이브러리 를 이용해서 HTML 로 렌더링합니다.

```
SELECT
ST_Extrude(ST_Buffer(ST_GeomFromText('POINT(
100 90)'),
 50, 'quad_segs=2'),0,0,30) AS geom1,

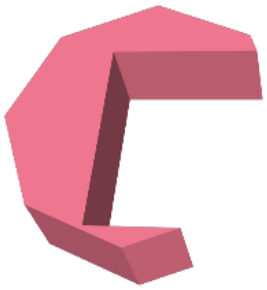
ST_Extrude(ST_Buffer(ST_GeomFromText('POINT(
80 80)'),
 50, 'quad_segs=1'),0,0,30) AS geom2;
```



원본 3 차원 도형들이 중첩합니다. geom2 가 제거될 부분을 나타냅니다.

```
SELECT ST_3DDifference(geom1,geom2)
FROM                ( SELECT
ST_Extrude(ST_Buffer(ST_GeomFromText('POINT(
100 90)'),
 50, 'quad_segs=2'),0,0,30) AS geom1,

ST_Extrude(ST_Buffer(ST_GeomFromText('POINT(
80 80)'),
 50, 'quad_segs=1'),0,0,30) AS geom2 ) As t;
```



geom2 를 제거하고 남은 부분

참고: ST_Extrude, ST_AsX3D, ST_3DIntersection ST_3DUnion

8.10.11. ST_3DUnion

ST_3DUnion — 3 차원 통합을 수행합니다.

Synopsis

geometry **ST_3DUnion**(geometry *geom1*, geometry *geom2*);

설명:

2.2.0 버전부터 사용할 수 있습니다.

✓ 이 방법을 쓰려면 SFCGAL 백엔드가 필요합니다.

✓ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✓ 이 함수는 다면체 표면을 지원합니다.

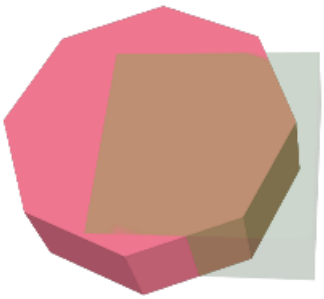
✓ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

예시

PostGIS ST_AsX3D 를 통해 3 차원 영상을 생성한 다음 X3Dom HTML 자바스크립트 렌더링 라이브러리 를 이용해서 HTML 로 렌더링합니다.

```
SELECT
ST_Extrude(ST_Buffer(ST_GeomFromText('POINT(100
90)'),
 50, 'quad_segs=2'),0,0,30) AS geom1,

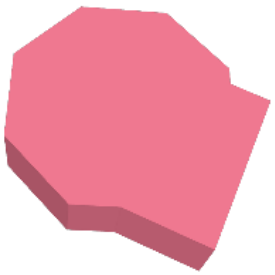
ST_Extrude(ST_Buffer(ST_GeomFromText('POINT(80
80)'),
 50, 'quad_segs=1'),0,0,30) AS geom2;
```



원본 3 차원 도형들이 중첩합니다. geom2 를 반투명하게 표현했습니다.

```
SELECT ST_3DUnion(geom1,geom2)
FROM                    (                    SELECT
ST_Extrude(ST_Buffer(ST_GeomFromText('POINT(100
90)'),
 50, 'quad_segs=2'),0,0,30) AS geom1,

ST_Extrude(ST_Buffer(ST_GeomFromText('POINT(80
80)'),
 50, 'quad_segs=1'),0,0,30) AS geom2 ) As t;
```



geom1 과 geom2 의 통합

참고: ST_Extrude, ST_AsX3D, ST_3DIntersection ST_3DDifference

8.10.12. ST_3DArea

ST_3DArea — 3 차원 표면 도형의 면적을 계산합니다. 입체일 경우 0 을 반환할 것입니다.

Synopsis

float**ST_3DArea**(geometry *geom1*);

설명:

2.1.0 버전부터 사용할 수 있습니다.

✓ 이 방법을 쓰려면 SFCGAL 백엔드가 필요합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

예시

주의: 기본적으로 KWT 로부터 빌드된 다면체 표면은 표면 도형이지 입체가 아닙니다. 따라서 표면 면적을 구할 수 있습니다. 입체로 변환하면, 면적을 구할 수 없습니다.

```
SELECT ST_3DArea(geom) As cube_surface_area,
    ST_3DArea(ST_MakeSolid(geom)) As solid_surface_area
 FROM (SELECT 'POLYHEDRALSURFACE( ((0 0 0, 0 0 1, 0 1 1, 0 1 0, 0 0 0)),
  ((0 0 0, 0 1 0, 1 1 0, 1 0 0, 0 0 0)),
  ((0 0 0, 1 0 0, 1 0 1, 0 0 1, 0 0 0)),
  ((1 1 0, 1 1 1, 1 0 1, 1 0 0, 1 1 0)),
  ((0 1 0, 0 1 1, 1 1 1, 1 1 0, 0 1 0)),
  ((0 0 1, 1 0 1, 1 1 1, 0 1 1, 0 0 1)) )'::geometry) As f(geom);

 cube_surface_area | solid_surface_area
-------------------+--------------------
         6         |          0
```

참고: ST_Area, ST_MakeSolid, ST_IsSolid, ST_Area

8.10.13. ST_Tesselate

ST_Tesselate — 폴리곤 또는 다면체 표면의 표면 다듬기(tesselation) 작업을 수행한 다음 TIN 또는 TIN 집합으로 반환합니다.

Synopsis

geometry **ST_Tesselate**(geometry *geom*);

설명: [멀티]폴리곤 또는 다면체 표면 같은 표면 도형을 입력받아 삼각형을 이용한 다듬기(모자이크) 과정을 거친 TIN 표현식을 반환합니다.

2.1.0 버전부터 사용할 수 있습니다.

✔ 이 방법을 쓰려면 SFCGAL 백엔드가 필요합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 함수는 다면체 표면을 지원합니다.

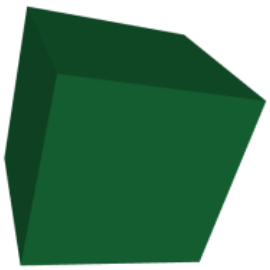
✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

예시

```
SELECT ST_3DArea(geom) As cube_surface_area,
    ST_3DArea(ST_MakeSolid(geom)) As solid_surface_area
 FROM (SELECT 'POLYHEDRALSURFACE( ((0 0 0, 0 0 1, 0 1 1, 0 1 0, 0 0 0)),
   ((0 0 0, 0 1 0, 1 1 0, 1 0 0, 0 0 0)),
   ((0 0 0, 1 0 0, 1 0 1, 0 0 1, 0 0 0)),
   ((1 1 0, 1 1 1, 1 0 1, 1 0 0, 1 1 0)),
   ((0 1 0, 0 1 1, 1 1 1, 1 1 0, 0 1 0)),
   ((0 0 1, 1 0 1, 1 1 1, 0 1 1, 0 0 1)) )'::geometry) As f(geom);

 cube_surface_area | solid_surface_area
-------------------+--------------------
         6         |          0
```

```
SELECT          ST_3DArea(geom)          As
cube_surface_area,
    ST_3DArea(ST_MakeSolid(geom))       As
SELECT  ST_GeomFromText('POLYHEDRALSURFACE
Z( ((0 0 0, 0 0 1, 0 1 1, 0 1 0, 0 0 0)),
        ((0 0 0, 0 1 0, 1 1 0, 1 0 0, 0 0
0)), ((0 0 0, 1 0 0, 1 0 1, 0 0 1, 0 0 0)),
        ((1 1 0, 1 1 1, 1 0 1, 1 0 0, 1 1
0)),
        ((0 1 0, 0 1 1, 1 1 1, 1 1 0, 0 1
0)), ((0 0 1, 1 0 1, 1 1 1, 0 1 1, 0 0
1)) )');
```



원본 정육면체

```
SELECT
ST_Tesselate(ST_GeomFromText('POLYHEDRALSURFAC
E Z( ((0 0 0, 0 0 1, 0 1 1, 0 1 0, 0 0 0)),
    ((0 0 0, 0 1 0, 1 1 0, 1 0 0, 0 0 0)), ((0
0 0, 1 0 0, 1 0 1, 0 0 1, 0 0 0)),
    ((1 1 0, 1 1 1, 1 0 1, 1 0 0, 1 1 0)),
    ((0 1 0, 0 1 1, 1 1 1, 1 1 0, 0 1 0)), ((0
0 1, 1 0 1, 1 1 1, 0 1 1, 0 0 1)) )'));
```

ST_AsText 출력물:

```
TIN Z (((0 0 0,0 0 1,0 1 1,0 0 0)),((0 1 0,0 0 0,0 1 1,0 1 0)),
   ((0 0 0,0 1 0,1 1 0,0 0 0)),
   ((1 0 0,0 0 0,1 1 0,1 0 0)),((0 0 1,1 0 0,1 0 1,0 0 1)),
   ((0 0 1,0 0 0,1 0 0,0 0 1)),
   ((1 1 0,1 1 1,1 0 1,1 1 0)),((1 0 0,1 1 0,1 0 1,1 0 0)),
   ((0 1 0,0 1 1,1 1 1,0 1 0)),((1 1 0,0 1 0,1 1 1,1 1 0)),
   ((0 1 1,1 0 1,1 1 1,0 1 1)),((0 1 1,0 0 1,1 0 1,0 1 1)))
```



삼각형을 색으로 구분한 다듬어진 정육면체

```
SELECT 'POLYGON (( 10 190, 10 70, 80 70, 80 130,
50 160, 120 160, 120 190, 10 190 ))'::geometry;
```



원본 폴리곤

```
SELECT
    ST_Tesselate('POLYGON (( 10 190, 10 70, 80 70, 80
130, 50 160, 120 160, 120 190, 10 190 ))'::geometry);
```

ST_AsText 출력물:

```
TIN(((80 130,50 160,80 70,80 130)),((50 160,10 190,10 70,50
160)),
    ((80 70,50 160,10 70,80 70)),((120 160,120 190,50 160,120
160)),
 ((120 190,10 190,50 160,120 190)))
```



다듬어진 폴리곤

8.10.14. ST_Volume

ST_Volume — 3 차원 입체의 부피를 계산합니다. 표면 도형을 입력하면 (닫힌 도형일지라도) 0 을 반환할 것입니다.

Synopsis

float **ST_Volume**(geometry *geom1*);

설명:

2.2.0 버전부터 사용할 수 있습니다.

✔ 이 방법을 쓰려면 SFCGAL 백엔드가 필요합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

예시

WKT 로 닫힌 표면을 생성했을 때, 입체라기보다는 면으로 취급됩니다. 이 닫힌 표면을 입체로 만들려면, ST_MakeSolid 함수를 이용해야 합니다. 면 도형은 부피가 없습니다. 다음은 그 사실을 보여주는 예시입니다.

```
SELECT ST_Volume(geom) As cube_surface_vol,
    ST_Volume(ST_MakeSolid(geom)) As solid_surface_vol
 FROM (SELECT 'POLYHEDRALSURFACE( ((0 0 0, 0 0 1, 0 1 1, 0 1 0, 0 0 0)),
  ((0 0 0, 0 1 0, 1 1 0, 1 0 0, 0 0 0)),
  ((0 0 0, 1 0 0, 1 0 1, 0 0 1, 0 0 0)),
  ((1 1 0, 1 1 1, 1 0 1, 1 0 0, 1 1 0)),
  ((0 1 0, 0 1 1, 1 1 1, 1 1 0, 0 1 0)),
  ((0 0 1, 1 0 1, 1 1 1, 0 1 1, 0 0 1)) )'::geometry) As f(geom);

 cube_surface_vol | solid_surface_vol
------------------+-------------------
                0 |                 1
```

참고: ST_3DArea, ST_MakeSolid, ST_IsSolid

8.10.15. ST_MakeSolid

ST_MakeSolid — 도형을 입체로 지정합니다. 어떤 확인 작업도 수행하지 않습니다. 유효한 입체를 얻으려면, 입력 도형이 닫힌 다면체 표면 또는 닫힌 TIN 이어야만 합니다.

Synopsis

boolean**ST_IsSolid**(geometry *geom1*);

설명:

2.2.0 버전부터 사용할 수 있습니다.

✔ 이 방법을 쓰려면 SFCGAL 백엔드가 필요합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

8.10.16. ST_IsSolid

ST_IsSolid — 도형이 입체인지 테스트합니다. 어떤 유효성 검사도 수행하지 않습니다.

Synopsis

boolean**ST_IsSolid**(geometry *geom1*);

설명:

2.2.0 버전부터 사용할 수 있습니다.

✔ 이 방법을 쓰려면 SFCGAL 백엔드가 필요합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

8.11. 도형 공간 처리

ST_Buffer — 입력 도형으로부터 주어진 거리 안에 있는 모든 포인트들을 커버하는 도형을 반환합니다.

ST_BuildArea — 주어진 도형의 선분 구성 요소로 이루어진 면 도형을 생성합니다.

ST_ClipByBox2D — 직사각형 안에 들어오는 도형의 부분을 반환합니다.

ST_Collect — 다른 도형들의 집합으로부터 설정된 ST_Geometry 값을 반환합니다.

ST_ConcaveHull — 도형의 함볼부(concave hull)이란 집합 내부의 모든 도형들을 둘러싸는 오목할 수도 있는 도형을 말합니다. 진공 수축 포장이라는 식으로 생각할 수 있습니다.

ST_ConvexHull — 도형의 돌출부(convex hull)은 집합 내부의 모든 도형들을 둘러싸는 최소 볼록 도형을 의미합니다.

ST_CurveToLine — 원호 스트링(CIRCULARSTRING)/만곡 폴리곤(CURVEPOLYGON)을 라인스트링/폴리곤으로 변환합니다.

ST_DelaunayTriangles — 주어진 입력 포인트들을 둘러싼 들로네 삼각망(Delaunay triangulation)을 반환합니다.

ST_Difference — 도형 A 에서 도형 B 와 교차하지 않는 부분을 표현하는 도형을 반환합니다.

ST_Dump — 도형 g1 을 구성하고 있는 geometry_dump(도형, 경로) 행들의 집합을 반환합니다.

ST_DumpPoints — 도형을 구성하는 모든 포인트의 geometry_dump(도형, 경로) 행들의 집합을 반환합니다.

ST_DumpRings — 폴리곤의 외곽 및 내곽 고리를 표현하는 geometry_dump 행들의 집합을 반환합니다.

ST_FlipCoordinates — 입력 도형을 X 축과 Y 축을 뒤집은 상태로 반환합니다. 위도/경도 피처를 빌드해서 수정해야 하는 경우 유용합니다.

ST_GeneratePoints — 폴리곤 또는 멀티폴리곤을 원본 면의 내부에 임의로 배치한 포인트들로 구성된 멀티포인트로 변환합니다.

ST_Intersection — geomA 와 geomB 의 공유 부분을 표현하는 도형을 반환합니다.

ST_LineToCurve — 라인스트링/폴리곤을 원호 스트링/만곡 폴리곤으로 변환합니다.

ST_MakeValid — 유효하지 않은 도형을 꼭짓점을 버리지 않고 유효한 도형으로 바꾸려고 시도합니다.

ST_MemUnion — 메모리 친화적이라는 점만 제외하면 ST_Union 함수와 동일합니다(메모리를 덜 사용하고 CPU 시간을 더 사용합니다).

ST_MinimumBoundingCircle — 도형 전체를 담을 수 있는 가장 작은 원형 폴리곤을 반환합니다. 기본값은 사분원 당 구간 48 개를 사용하는 것입니다.

ST_MinimumBoundingRadius — 도형 전체를 담을 수 있는 가장 작은 원의 중심 포인트 및 반지름을 반환합니다.

ST_Polygonize — 종합 함수입니다. 도형들의 집합의 선분 구성 요소로부터 형성될 수 있는 폴리곤들을 담고 있는 도형 집합을 생성합니다.

ST_Node — 라인스트링들의 집합에 노드를 적용합니다.

ST_OffsetCurve — 입력라인으로부터 주어진 거리 및 방향에 있는 오프셋 라인을 반환합니다. 중심선을 기준으로 평행한 라인들을 계산하는 데 유용합니다.

ST_RemoveRepeatedPoints — 입력 도형을 중복된 포인트들을 제거한 상태로 반환합니다.

ST_SharedPaths — 두 입력 라인스트링/멀티라인스트링이 공유하는 경로를 담고 있는 집합을 반환합니다.

ST_ShiftLongitude — 도형의 좌표를 -180 ~ 180 범위와 0 ~ 360 범위 간에 전환합니다.

ST_WrapX — X 값 근처에서 도형을 래핑합니다.

ST_Simplify — 더글러스-패커(Douglas-Peucker) 알고리즘을 이용해서 입력 도형의 "단순화"된 버전을 반환합니다.

ST_SimplifyPreserveTopology — 더글러스-패커(Douglas-Peucker) 알고리즘을 이용해서 입력 도형의 "단순화"된 버전을 반환합니다. 유효하지 않은 파생 도형(특히 폴리곤)을 생성하지 않을 것입니다.

ST_SimplifyVW — 비스베일링검-와이어트(Visvalingam-Whyatt) 알고리즘을 이용해서 입력 도형의 "단순화"된 버전을 반환합니다.

ST_SetEffectiveArea — M 세로 좌표(ordinate)에 값을 저장해서 각 꼭짓점에 대한 유효 범위(effective area)를 설정합니다. 그 다음 M 세로 좌표에 대해 필터링해서 단순화시킨 도형을 생성할 수 있습니다.

ST_Split — 도형을 분해(split)해서 나오는 도형들의 집합을 반환합니다.

ST_SymDifference — 도형 A 와 B 의 서로 교차하지 않는 부분들을 표현하는 도형을 반환합니다. ST_SymDifference(A,B) = ST_SymDifference(B,A)이기 때문에 이를 대칭차(symmetric difference)라고 합니다.

ST_Subdivide — 집합 내의 모든 도형이 설정된 개수 이하의 꼭짓점을 가지는 도형 집합을 반환합니다.

ST_SwapOrdinates — 입력 도형을 좌표값을 뒤바꾼 상태로 반환합니다.

ST_Union — 도형들의 포인트들을 통합한 집합을 표현하는 도형을 반환합니다.

ST_UnaryUnion — ST_Union 과 비슷하지만, 도형 구성 요소 레벨에서 동작합니다.

ST_Voronoi — 도형의 꼭짓점들로부터 보로노이(Voronoi) 다이어그램을 계산합니다.

8.11.1. ST_Buffer

ST_Buffer — 입력 도형으로부터 주어진 거리 안에 있는 모든 포인트들을 커버하는 도형을 반환합니다.

Synopsis

geometry **ST_Buffer**(geometry *g1*, float *radius_of_buffer*);

geometry **ST_Buffer**(geometry *g1*, float *radius_of_buffer*, integer *num_seg_quarter_circle*);

geometry **ST_Buffer**(geometry *g1*, float *radius_of_buffer*, text *buffer_style_parameters*);

geography **ST_Buffer**(geography *g1*, float *radius_of_buffer_in_meters*);

geography **ST_Buffer**(geography *g1*, float *radius_of_buffer*, integer *num_seg_quarter_circle*);

geography **ST_Buffer**(geography *g1*, float *radius_of_buffer*, text *buffer_style_parameters*);

설명: 해당 도형/지리형으로부터의 거리가 주어진 거리보다 짧거나 동등한 모든 포인트들을 표현하는 도형/지리형을 반환합니다.

도형: 도형의 공간 참조 시스템 단위로 계산합니다. 1.5 버전부터 형상을 제어하기 위한 서로 다른 끝모양(endcap) 및 마이터(mitre) 설정을 지원하고 있습니다.



음의 반경(negative radius): 폴리곤에 대해 음의 반경을 쓸 수 있습니다. 음의 반경은 폴리곤을 늘어나게 하기 보다는 줄어들게 할 것입니다.



지리형: 지리형에 대해, 이 함수는 도형 구현을 둘러싼 정말로 얇은 래퍼입니다. 이 함수는 먼저 지리형 객체의 경계 상자에 가장 어울리는 SRID 를 (UTM 또는 람베르트 북극/남극 방위정적도법(Lambert Azimuthal Equal Area)을 선호하며, 최악의 경우 메르카토르 도법에 의지해서) 결정한 다음 해당 평면 공간 참조 시스템 상에서 버퍼를 준 다음 WGS84 지리형으로 다시 변환합니다.



지리형의 경우, 두 UTM 구역에 걸치거나, 날짜변경선을 지나는 등 해당 객체가 충분히 크다면 정상적으로 동작하지 않을 수도 있습니다.

1.5 버전부터 사용할 수 있습니다. ST_Buffer 함수가 다양한 끝모양(endcap) 및 접합(join) 유형을 지원하도록 개선됐습니다. 예를 들어 도로 라인스트링을 원형(rounded) 대신 절단형(flat) 또는 돌출 사각형(square) 끝모양을 가진 폴리곤 도로로 변환시키는 데 유용합니다. 지리형을 위한 얇은 래퍼도 추가됐습니다. 고급 도형 기능성의 장점을 취하려면 GEOS 3.2 이상 버전이 필요합니다.

선택적인 (현재 도형에만 적용되는) 세 번째 파라미터는 다음과 같이 사분원(quarter circle)의 근사치를 계산하는 데 쓰이는 선분의 개수(정수형, 기본값은 8)를 설정하거나, 또는 연산을 조정하기 위한 공백으로 구분된 키=값 쌍의 목록(문자열)을 설정할 수 있습니다.

- 'quad_segs=#' : 사분원(quarter circle)의 근사치를 계산하는 데 쓰이는 선분의 개수(기본값은 8)
- 'endcap=round|flat|square' : 끝모양 스타일(기본값은 "원형(round)"으로 다른 값을 쓰려면 GEOS 3.2 이상 버전이 필요합니다). '절단형(flat)'의 동의어로 '밑동(butt)'도 쓸 수 있습니다.
- 'join=round|mitre|bevel' : 접합 스타일(기본값은 "원형(round)"으로 다른 값을 쓰려면 GEOS 3.2 이상 버전이 필요합니다). '마이터(mitre)'의 동의어로 '마이터(miter)'도 쓸 수 있습니다.
- 'mitre_limit=#.#' : 마이터 비율 제한(마이터 접합 스타일만 영향을 받습니다). 'mitre_limit'의 동의어로 'miter_limit'도 쓸 수 있습니다.

반경의 단위는 공간 참조 시스템의 단위로 측정됩니다.

포인트, 멀티포인트, 라인스트링, 멀티라인스트링, 폴리곤, 멀티폴리곤 그리고 도형 집합을 입력받을 수 있습니다.



이 함수는 3 차원 도형을 입력받더라도 세 번째 차원(Z)을 무시하고 언제나 2 차원 버퍼를 반환할 것입니다.

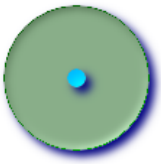
GEOS 모듈로 실행

✓ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 만족합니다. s2.1.1.3

✓ 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.17

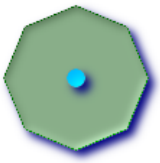
Note! 사용자들이 반경 탐색 작업을 하려 할 때 이 함수를 이용하는 실수를 하곤 합니다. 반경 탐색을 위해 버퍼를 만드는 것은 느리고 효과도 없습니다. 대신 ST_DWithin 함수를 이용하십시오..

예시



quad_segs=8 (기본값)

```
SELECT ST_Buffer(
 ST_GeomFromText('POINT(100 90)'),
 50, 'quad_segs=8');
```



quad_segs=2 (저질)

```
SELECT ST_Buffer(
 ST_GeomFromText('POINT(100 90)'),
 50, 'quad_segs=2');
```



endcap=round join=round (기본값)

```
SELECT ST_Buffer(
ST_GeomFromText(
 'LINESTRING(50 50,150 150,150 50)'
),      10,      'endcap=round join=round');
```



endcap=square

```
SELECT ST_Buffer(
 ST_GeomFromText(
  'LINESTRING(50 50,150 150,150 50)'
 ),      10,      'endcap=square join=round');
```



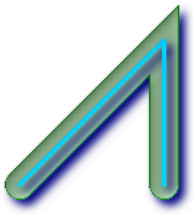
endcap=flat

```
SELECT ST_Buffer(
 ST_GeomFromText(
  'LINESTRING(50 50,150 150,150 50)'
 ), 10, 'endcap=flat join=round');)
```



join=bevel

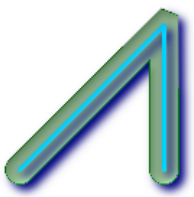
```
SELECT ST_Buffer(
 ST_GeomFromText(
  'LINESTRING(50 50,150 150,150 50)'
 ), 10, 'join=bevel');
```



join=mitre mitre_limit=5.0

(마이터 제한 기본값)

```
SELECT ST_Buffer(
 ST_GeomFromText(
  'LINESTRING(50 50,150 150,150 50)'
 ), 10, 'join=mitre mitre_limit=5.0');
```



join=mitre mitre_limit=1

```
SELECT ST_Buffer(
 ST_GeomFromText(
  'LINESTRING(50 50,150 150,150 50)'
 ), 10, 'join=mitre mitre_limit=1.0');
```

```
-- 포인트에 버퍼를 적용해서 원에 가깝게 만듭니다.
-- 원마다 포인트 2 개를 강제해서 버퍼를 적용하면
-- 변 8 개를 가진 폴리곤이 됩니다(다이어그램 참조)
SELECT       ST_NPoints(ST_Buffer(ST_GeomFromText('POINT(100       90)'),       50))       As
promisingcircle_pcount,
ST_NPoints(ST_Buffer(ST_GeomFromText('POINT(100 90)'), 50, 2)) As lamecircle_pcount;

promisingcircle_pcount | lamecircle_pcount
-----------------------+------------------
              33 |         9

-- 더 가볍지만 질이 낮은 원
-- 사분원마다 포인트 2 개를 강제하면 팔각형이 됩니다.
-- 아래는 100 미터 팔각형입니다.
-- NAD83 경위도 좌표를 매사추세츠 주 미터 단위 평면으로 변환한 다음 버퍼를 적용해서 미터 단위로 측정한다는 사실에
주의하십시오.
SELECT ST_AsText(ST_Buffer(
ST_Transform(
ST_SetSRID(ST_MakePoint(-71.063526, 42.35785),4269), 26986)
,100,2)) As octagon;
----------------------
POLYGON((236057.59057465 900908.759918696,236028.301252769 900838.049240578,235
957.59057465 900808.759918696,235886.879896532 900838.049240578,235857.59057465
900908.759918696,235886.879896532 900979.470596815,235957.59057465 901008.759918
696,236028.301252769 900979.470596815,236057.59057465 900908.759918696))
```

참고: ST_Collect, ST_DWithin, ST_SetSRID, ST_Transform, ST_Union

8.11.2. ST_BuildArea

ST_BuildArea — 주어진 도형의 선분 구성 요소로 이루어진 면 도형을 생성합니다.

Synopsis

geometry **ST_BuildArea**(geometry *A*);

설명: 주어진 도형의 선분 구성 요소로 이루어진 면 도형을 생성합니다. 반환된 유형은 입력 도형에 따라 폴리곤 또는 멀티폴리곤이 될 수 있습니다. 입력 선분이 폴리곤을 형성시키지 못 할 경우 NULL 을 반환합니다. 라인스트링, 멀티라인스트링, 폴리곤, 멀티폴리곤 그리고 도형 집합을 입력받을 수 있습니다.

이 함수는 모든 내부(inner) 도형이 구멍을 나타낸다고 가정할 것입니다.



입력 선분의 교점(node)이 정확히 구성돼야만 이 함수가 제대로 동작합니다.

1.1.0 버전부터 사용할 수 있습니다. GEOS 2.1.0 이상 버전이 필요합니다.

예시



다음 쿼리는 도넛 모양을 생성할 겁니다.

```
SELECT ST_BuildArea(ST_Collect(smallc,bigc))
FROM (SELECT
       ST_Buffer(
         ST_GeomFromText('POINT(100 90)'), 25) As smallc,
       ST_Buffer(ST_GeomFromText('POINT(100 90)'), 50) As bigc) As foo;
```



다음 쿼리는 삐죽 튀어나온 막대와 함께 원 안에 벌어진 구멍을 만들 겁니다.

```
SELECT ST_BuildArea(ST_Collect(line,circle))
FROM (SELECT
       ST_Buffer(
              ST_MakeLine(ST_MakePoint(10, 10),ST_MakePoint(190, 190)),
                          5)  As line,
       ST_Buffer(ST_GeomFromText('POINT(100 90)'), 50) As circle) As foo;

-- 다음 쿼리는 동일한 구멍을 생성하지만
-- 폴리곤 대신 라인스트링을 이용합니다.
SELECT ST_BuildArea(
       ST_Collect(ST_ExteriorRing(line),ST_ExteriorRing(circle))
       )
FROM (SELECT ST_Buffer(
       ST_MakeLine(ST_MakePoint(10, 10),ST_MakePoint(190, 190))
              ,5)  As line,
       ST_Buffer(ST_GeomFromText('POINT(100 90)'), 50) As circle) As foo;
```

참고: ST_Node, ST_MakePolygon, ST_BdPolyFromText, ST_BdMPolyFromText 이 함수에 대해 표준 OGC 인터페이스를 가진 래퍼(wrapper)들

8.11.3. ST_ClipByBox2d

ST_ClipByBox2D — 직사각형 안에 들어오는 도형의 부분을 반환합니다.

Synopsis

geometry **ST_ClipByBox2D**(geometry *geom*, box2d *box*);

설명: 빠르지만 지저분할 수도 있는 방법으로 2 차원 상자가 도형을 잘라냅니다. 출력 도형이 유효하지 않을 수도 있습니다(폴리곤의 경우 스스로 교차할 수도 있습니다). 위상기하학적으로 유효하지 않은 도형을 입력해도 예외 상태에 빠지지는 않습니다.

GEOS 모듈로 실행



GEOS 3.5.0 이상 버전이 필요합니다.

2.2.0 버전부터 사용할 수 있습니다. GEOS 3.5.0 이상 버전이 필요합니다.

예시

```
-- 두 번째 파라미터의 경우 도형을 무조건적으로 2 차원 상자로 지정한다는 사실에 의존합니다.
SELECT ST_ClipByBox2D(the_geom, ST_MakeEnvelope(0,0,10,10)) FROM mytab;
```

참고: ST_Intersection, ST_MakeBox2D, ST_MakeEnvelope

8.11.4. ST_Collect

ST_Collect — 다른 도형들의 집합으로부터 설정된 ST_Geometry 값을 반환합니다.

Synopsis

geometry ST_Collect(geometry set g1field);

geometry ST_Collect(geometry g1, geometry g2);

geometry ST_Collect(geometry[] g1_array);

설명: 멀티 도형 또는 도형 집합 유형을 출력할 수 있습니다. 두 가지 변종이 존재합니다. 변종 1 은 도형 2 개를 수집합니다. 변종 2 는 도형들의 집합을 입력받아 단일 ST_Geometry 로 수집하는 종합 함수입니다.

종합 버전: 이 함수는 도형들의 집합으로부터 도형 집합 또는 멀티 유형의 객체를 반환합니다. ST_Collect() 함수는 PostgreSQL 전문 용어로 "종합(aggregate)" 함수입니다. 즉 SUM() 및 AVG() 함수와 마찬가지로 데이터 행들을 대상으로 연산한다는 뜻입니다. 예를 들어, "SELECT ST_Collect(GEOM) FROM GEOMTABLE GROUP BY ATTRCOLUMN" 쿼리는 ATTRCOLUMN 의 각 개별적인 값에 대해 별개의 도형 집합을 반환할 것입니다.

비(非)종합 버전: 이 함수는 입력 도형 2 개의 집합인 도형을 반환합니다. 멀티 도형 또는 도형 집합 유형을 출력할 수 있습니다.



종종 ST_Collect 와 ST_Union 을 서로 바꿔 사용할 수 있습니다. 일반적인 자릿수의 경우 ST_Collect 가 ST_Union 보다 더 빠른데, ST_Collect 는 작성된 멀티폴리곤에 중첩된 부분이 없는지 입증하려 하거나 또는 경계선을 없애려 하지 않기 때문입니다. ST_Collect 는 단순히

단일 도형을 멀티 도형과 멀티 도형으로 또는 혼합 도형 유형을 도형 집합으로 만들 뿐입니다. 안타깝게도 GIS 도구들이 도형 집합을 잘 지원한다고 할 수는 없습니다. 멀티 도형을 수집하는 경우 ST_Collect 함수가 도형 집합을 반환하는 것을 막으려면, ST_Dump 함수를 활용해서 멀티 도형을 단일 도형들로 해체한 다음 다시 그룹화하는 다음 꼼수를 이용할 수 있습니다.

1.4.0 버전부터 사용할 수 있습니다. 이 버전부터 도형 배열을 입력받을 수 있습니다. 더 많은 도형들을 더 빨리 처리하도록 ST_Collect 를 개선했습니다.

✓ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✓ 이 방법은 원형 스트링 및 곡선을 지원합니다. 이 메소드는 원호 스트링 및 만곡 도형을 지원하지만, 사용자가 기대할 만한 멀티커브 또는 멀티 도형을 반환하지는 않습니다. 현재 PostGIS 는 이들을 지원하지 않습니다.

예시

종합 버전 예시(http://postgis.refractions.net/pipermail/postgis-users/2008-June/020331.html)

```
SELECT stusps,
      ST_Multi(ST_Collect(f.the_geom)) as singlegeom
     FROM (SELECT stusps, (ST_Dump(the_geom)).geom As the_geom
                FROM
                somestatetable ) As f
GROUP BY stusps
```

비(非) 종합 버전 예시

```
SELECT ST_AsText(ST_Collect(ST_GeomFromText('POINT(1 2)'),
    ST_GeomFromText('POINT(-2 3)') ));

st_astext
----------
MULTIPOINT(1 2,-2 3)

-- 2 차원 포인트 수집
SELECT ST_AsText(ST_Collect(ST_GeomFromText('POINT(1 2)'),
        ST_GeomFromText('POINT(1 2)') ) );

st_astext
----------
MULTIPOINT(1 2,1 2)

-- 3 차원 포인트 수집
SELECT ST_AsEWKT(ST_Collect(ST_GeomFromEWKT('POINT(1 2 3)'),
        ST_GeomFromEWKT('POINT(1 2 4)') ) );

        st_asewkt
-------------------------
 MULTIPOINT(1 2 3,1 2 4)

 -- 만곡 도형 예시
SELECT      ST_AsText(ST_Collect(ST_GeomFromText('CIRCULARSTRING(220268      150415,220227
150505,220227 150406)'),
ST_GeomFromText('CIRCULARSTRING(220227 150406,2220227 150407,220227 150406)')));
                                                                    st_astext
```

```
--------------------------------------------------------------------------------
 GEOMETRYCOLLECTION(CIRCULARSTRING(220268 150415,220227 150505,220227 150406),
 CIRCULARSTRING(220227 150406,2220227 150407,220227 150406))

-- 새로운 ST_Collect 배열 구조
SELECT ST_Collect(ARRAY(SELECT the_geom FROM sometable));

SELECT ST_AsText(ST_Collect(ARRAY[ST_GeomFromText('LINESTRING(1 2, 3 4)'),
             ST_GeomFromText('LINESTRING(3 4, 4 5)')])) As wktcollect;

-- WKT 수집 --
MULTILINESTRING((1 2,3 4),(3 4,4 5))
```

참고: ST_Dump, ST_Union

8.11.5. ST_ConcaveHull

ST_ConcaveHull — 도형의 오목 껍질(concave hull)이란 집합 내부의 모든 도형들을 둘러싸는 오목할 수도 있는 도형을 말합니다. 진공 수축 포장이라는 식으로 생각할 수 있습니다.

Synopsis

geometry **ST_ConcaveHull**(geometry *geomA*, float *target_percent*, boolean *allow_holes=false*);

설명: 도형의함몰부(concave hull)이란 집합 내부의 모든 도형들을 둘러싸는 오목할 수도 있는 도형을 말합니다. 구멍을 가진 폴리곤을 사용할지에 대한 파라미터의 기본값은 거짓입니다. 산출물은 절대로 단일 폴리곤을 넘지 않습니다.

target_percent 는 PostGIS 솔루션이 포기 또는 종료하기 전에 접근하려 할 볼록 껍질(convex hull) 면적의 목표 퍼센트입니다. 도형들의 집합을 둘러싼 봉지를 진공청소기로 빨아들여 봉한 도형을 오목 껍질이라고 생각해볼 수 있습니다. target_percent 를 1 로 설정하면 볼록 껍질과 동일한 답을 얻을 것입니다. target_percent 를 0 에서 0.99 사이의 값으로 설정하면 볼록 껍질보다 적은 면적을 가진 무언가를 얻을 것입니다. 오목 껍질은 도형들의 집합 주위로 고무줄을 두른 것에 가까운 볼록 껍질과 이렇게 다릅니다.

이 함수는 주로 멀티 도형 및 도형 집합을 입력받습니다. 비록 모든 상황에 맞는 함수는 아니지만, ST_Collect 또는 ST_Union 과 결합해서 포인트/라인스트링/폴리곤 집합의함몰부분 경계을 얻을 수 있습니다. 예: ST_ConcaveHull(ST_Collect(somepointfield), 0.80)

계산하는 함몰된 부분이 보다 더 오래 걸리지만 도형을 더 잘 둘러싸고 또 시각적으로 인식하는 데에도 유용합니다.

GEOS 모듈로 실행



주의: 포인트, 라인스트링 또는 도형 집합을 입력받을 경우 ST_Collect 함수를 이용하십시오. 폴리곤을 입력받을 경우, 유효하지 않은 도형이 오류를 일으킬 수도 있으므로 ST_Union 함수를 이용하십시오.



주의: 목표 퍼센트를 적게 만들수록, 처리 시간이 더 오래 걸리며 위상기하학적 예외가 발생할 가능성이 커집니다. 또 부동소수점이 많을수록 누적되는 포인트의 개수도 많아집니다. 먼저 첫 단계인 0.99 로 설정해보십시오. 보통 매우 빠르고, 어떨 때는 볼록 껍질 계산만큼 빠르며, 거의 언제나 초과 작업을 하기 때문에 일반적으로 수축(shrink)의 99%보다 더 잘 반환합니다. 두 번째 단계인 0.98 은 더 느리고, 단계가 올라갈수록 보통 2 차급수적으로(quadratically) 느려집니다. 부동소수점과 정확도를 줄이려면, ST_ConcaveHull 실행 후 ST_SimplifyPreserveTopology 또는 ST_SnapToGrid 를 이용하십시오. ST_SnapToGrid 가 조금 더 빠르지만, 유효하지 않은 도형을 반환할 수도 있는 반면 ST_SimplifyPreserveTopology 는 거의 언제나 도형의 유효성을 보전합니다.

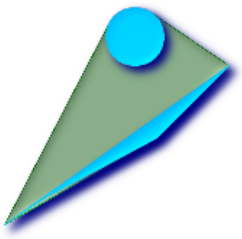
좀더 현실에 가까운 예시 및 간단한 기술 설명을 http://www.bostongis.com/postgis_concavehull.snippet 에서 찾아볼 수 있습니다.

또 Oracle 11G R2 에 소개된 사이먼 그리너(Simon Greener)의 볼록한 부분의 시연에 대한 글 http://www.spatialdbadvisor.com/oracle_spatial_tips_tricks/172/concave-hull-geometries-in-oracle-11gr2 도 확인해보십시오. 이 함수가 볼록 껍질의 목표 퍼센트 0.75 로 얻을 수 있는 모양이 사이먼 그리너가 Oracle 의 SDO_CONCAVEHULL_BOUNDARY 로 얻은 것과 비슷합니다.

2.0.0 버전부터 사용할 수 있습니다.

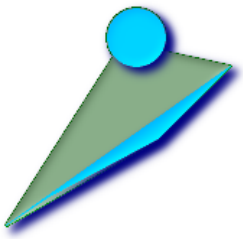
예시

```
-- 포인트 관찰에 기반한 영향 면적 추산하기
SELECT d.disease_type,
    ST_ConcaveHull(ST_Collect(d.pnt_geom), 0.99) As geom
    FROM disease_obs As d
    GROUP BY d.disease_type;
```



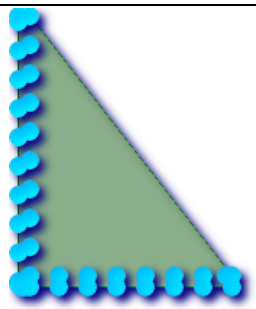
목표 100% 수축 오목 껍질에 둘러싸인 두 폴리곤의 ST_ConcaveHull

```
-- 오목 껍질과 중첩한 도형들
-- 목표 100% 수축(수축이 없으므로 볼록 껍질과 동일함)
SELECT
    ST_ConcaveHull(
        ST_Union(ST_GeomFromText('POLYGON((175
150, 20 40,
            50 60, 125 100, 175 150))'),
        ST_Buffer(ST_GeomFromText('POINT(110
170)'), 20)
        ), 1)
    As convexhull;
```



-- 볼록 껍질 면적의 목표 90%에서 오목 껍질과 중첩한 도형들

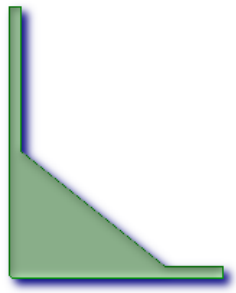
```
-- 목표 90% 수축에서 오목 껍질과 중첩한 도형들
SELECT
    ST_ConcaveHull(
        ST_Union(ST_GeomFromText('POLYGON((175
150, 20 40,
            50 60, 125 100, 175 150))'),
        ST_Buffer(ST_GeomFromText('POINT(110
170)'), 20)
        ), 0.9)
    As target_90;
```



볼록 껍질과 중첩한 L 형상 포인트들

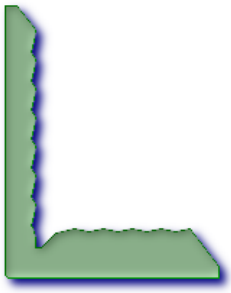
```
 -- 이 쿼리는 L 형상을 이루고 있는 포인트 42 개의 테이블을 생성합니다.
SELECT (ST_DumpPoints(ST_GeomFromText(
'MULTIPOINT(14 14,34 14,54 14,74 14,94 14,114 14,134 14,
150 14,154 14,154 6,134 6,114 6,94 6,74 6,54 6,34 6,
14 6,10 6,8 6,7 7,6 8,6 10,6 30,6 50,6 70,6 90,6 110,6 130,
6 150,6 170,6 190,6 194,14 194,14 174,14 154,14 134,14 114,
14 94,14 74,14 54,14 34,14 14)'))).geom
      INTO TABLE l_shape;

SELECT ST_ConvexHull(ST_Collect(geom))
FROM l_shape;
```



볼록 껍질의 목표 99%에서 L 형상의 ST_ConcaveHull

```
 SELECT
ST_ConcaveHull(ST_Collect(geom),
0.99)
      FROM l_shape;
```



볼록 껍질 면적의 목표 80%에서 L 형상 포인트들의 오목 껍질

```
-- 볼록 껍질의 목표 80%에서 L 형상 포인트들의 오목 껍질
SELECT
ST_ConcaveHull(ST_Collect(geom),
0.80)
      FROM l_shape;
```



볼록 껍질과 중첩한 멀티라인스트링



목표 99% 라인스트링의 오목 껍질과 중첩한 멀티라인스트링 - 첫 번째 단계

```
SELECT
ST_ConcaveHull(ST_GeomFromText('MULTILINESTRING((106 164,30 112,74 70,82 112,130 94,
    130 62,122 40,156 32,162 76,172 88),
(132 178,134 148,128 136,96 128,132 108,150 130,
170 142,174 110,156 96,158 90,158 88),
(22 64,66 28,94 38,94 68,114 76,112 30,
132 10,168 18,178 34,186 52,184 74,190 100,
190 122,182 148,178 170,176 184,156 164,146 178,
132 186,92 182,56 158,36 150,62 150,76 128,88 118))'),0.99)
```

참고: ST_Collect, ST_ConvexHull, ST_SimplifyPreserveTopology, ST_SnapToGrid

8.11.6. ST_ConvexHull

ST_ConvexHull — 도형의 볼록한 경계(convex hull)은 집합 내부의 모든 도형들을 둘러싸는 최소 볼록 도형을 의미합니다.

Synopsis

geometry **ST_ConvexHull**(geometry *geomA*);

설명: 도형의 볼록 한 경계(convex hull)는 집합 내부의 모든 도형들을 둘러싸는 최소 볼록 도형을 의미합니다.

볼록경계는 도형들의 집합 주위로 고무줄을 둘러서 얻게 되는 도형이라고 생각할 수 있습니다. 사용자 도형들을 진공 수축 포장하는 것에 가까운 함몰 경계 와는 다릅니다.

이 함수는 주로 멀티 도형 및 도형 집합을 입력받습니다. 비록 종합 함수는 아니지만, ST_Collect 과 결합해서 포인트 집합의 볼록 껍질을 얻을 수 있습니다. 예: ST_ConvexHull(ST_Collect(somepointfield))

이 함수는 포인트 집합 관찰에 기반한 영향 면적을 추산하는 데 자주 쓰입니다.

GEOS 모듈로 실행

✓ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 만족합니다. s2.1.1.3

✓ 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.16

✓ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
-- 포인트 관찰에 기반한 영향 면적 추산하기
SELECT d.disease_type,
    ST_ConvexHull(ST_Collect(d.the_geom)) As the_geom
    FROM disease_obs As d
    GROUP BY d.disease_type;
```



멀티라인스트링과 멀티포인트와 함께 표출된 멀티라인스트링 및 멀티포인트의 볼록 껍질

```
SELECT ST_AsText(ST_ConvexHull(
    ST_Collect(
        ST_GeomFromText('MULTILINESTRING((100 190,10 8),(150 10, 20 30))'),
            ST_GeomFromText('MULTIPOINT(50 5, 150 30, 50 10, 10 10)')
            )) );
-- st_astext --
POLYGON((50 5,10 8,10 10,100 190,150 30,150 10,50 5))
```

참고: ST_Collect, ST_ConcaveHull, ST_MinimumBoundingCircle

8.11.7. ST_CurveToLine

ST_CurveToLine — 원호 스트링(CIRCULARSTRING)/만곡 폴리곤(CURVEPOLYGON)을 라인스트링/폴리곤으로 변환합니다.

Synopsis

geometry **ST_CurveToLine**(geometry *curveGeom*);

geometry **ST_CurveToLine**(geometry *curveGeom*, integer *segments_per_qtr_circle*);

설명: 원호 스트링을 정규 라인스트링으로, 또는 만곡 폴리곤을 폴리곤으로 변환합니다. 원호 스트링 도형 유형을 지원하지 못 하는 상황에서 도형을 출력하는 데 유용합니다.

주어진 도형을 선형(linear) 도형으로 변환합니다. 각 만곡 도형 또는 구간을 사분원(quarter circle) 당 32 구간이라는 기본값을 통해 선형 근사치로 변환시킵니다.

1.2.2 버전(?)부터 사용할 수 있습니다.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 충족합니다.

✔ 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 7.1.7

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

예시

```
SELECT     ST_AsText(ST_CurveToLine(ST_GeomFromText('CIRCULARSTRING(220268     150415,220227
150505,220227 150406)')));

-- 결과--
 LINESTRING(220268          150415,220269.95064912          150416.539364228,220271.823415575
150418.17258804,220273.613787707 150419.895736857,
 220275.317452352     150421.704659462,220276.930305234     150423.594998003,220278.448460847
150425.562198489,
 220279.868261823      150427.60152176,220281.186287736     150429.708054909,220282.399363347
150431.876723113,
 220283.50456625      150434.10230186,220284.499233914      150436.379429536,220285.380970099
150438.702620341,220286.147650624 150441.066277505,
```

```
 220286.797428488     150443.464706771,220287.328738321     150445.892130112,220287.740300149
150448.342699654,
 220288.031122486     150450.810511759,220288.200504713     150453.289621251,220288.248038775
150455.77405574,
 220288.173610157     150458.257830005,220287.977398166     150460.734960415,220287.659875492
150463.199479347,
 220287.221807076      150465.64544956,220286.664248262     150468.066978495,220285.988542259
150470.458232479,220285.196316903 150472.81345077,
 220284.289480732     150475.126959442,220283.270218395      150477.39318505,220282.140985384
150479.606668057,
 220280.90450212       150481.762075989,220279.5637474       150483.85421628,220278.12195122
150485.87804878,
 220276.582586992     150487.828697901,220274.949363179     150489.701464356,220273.226214362
150491.491836488,
 220271.417291757     150493.195501133,220269.526953216     150494.808354014,220267.559752731
150496.326509628,
 220265.520429459      150497.746310603,220263.41389631     150499.064336517,220261.245228106
150500.277412127,
 220259.019649359      150501.38261503,220256.742521683     150502.377282695,220254.419330878
150503.259018879,
 220252.055673714     150504.025699404,220249.657244448     150504.675477269,220247.229821107
150505.206787101,
 220244.779251566      150505.61834893,220242.311439461     150505.909171266,220239.832329968
150506.078553494,
 220237.347895479     150506.126087555,220234.864121215     150506.051658938,220232.386990804
150505.855446946,
 220229.922471872      150505.537924272,220227.47650166     150505.099855856,220225.054972724
150504.542297043,
 220222.663718741 150503.86659104,220220.308500449 150503.074365683,
 220217.994991777 150502.167529512,220215.72876617 150501.148267175,
 220213.515283163 150500.019034164,220211.35987523 150498.7825509,
 220209.267734939 150497.441796181,220207.243902439 150496,
 220205.293253319      150494.460635772,220203.420486864      150492.82741196,220201.630114732
150491.104263143,
 220199.926450087     150489.295340538,220198.313597205     150487.405001997,220196.795441592
150485.437801511,
 220195.375640616      150483.39847824,220194.057614703     150481.291945091,220192.844539092
150479.123276887,220191.739336189 150476.89769814,
 220190.744668525      150474.620570464,220189.86293234     150472.297379659,220189.096251815
150469.933722495,
 220188.446473951      150467.535293229,220187.915164118      150465.107869888,220187.50360229
150462.657300346,
 220187.212779953     150460.189488241,220187.043397726     150457.710378749,220186.995863664
150455.22594426,
 220187.070292282     150452.742169995,220187.266504273     150450.265039585,220187.584026947
150447.800520653,
 220188.022095363      150445.35455044,220188.579654177      150442.933021505,220189.25536018
150440.541767521,
 220190.047585536      150438.18654923,220190.954421707     150435.873040558,220191.973684044
150433.60681495,
 220193.102917055     150431.393331943,220194.339400319     150429.237924011,220195.680155039
150427.14578372,220197.12195122 150425.12195122,
 220198.661315447      150423.171302099,220200.29453926     150421.298535644,220202.017688077
150419.508163512,220203.826610682 150417.804498867,
 220205.716949223      150416.191645986,220207.684149708      150414.673490372,220209.72347298
150413.253689397,220211.830006129 150411.935663483,
 220213.998674333      150410.722587873,220216.22425308      150409.61738497,220218.501380756
150408.622717305,220220.824571561 150407.740981121,
 220223.188228725 150406.974300596,220225.586657991 150406.324522731,220227 150406)

-- 3 차원 예시
```

```
SELECT   ST_AsEWKT(ST_CurveToLine(ST_GeomFromEWKT('CIRCULARSTRING(220268   150415   1,220227
150505 2,220227 150406 3)')));
Output
------
 LINESTRING(220268 150415 1,220269.95064912 150416.539364228 1.0181172856673,
 220271.823415575    150418.17258804    1.03623457133459,220273.613787707    150419.895736857
1.05435185700189,....AD INFINITUM ....
     220225.586657991 150406.324522731 1.32611114201132,220227 150406 3)

-- 사분원의 근사치를 구하는 데 2 구간만 사용
SELECT     ST_AsText(ST_CurveToLine(ST_GeomFromText('CIRCULARSTRING(220268     150415,220227
150505,220227 150406)'),2));
st_astext
------------------------------
 LINESTRING(220268 150415,220287.740300149 150448.342699654,220278.12195122 150485.87804878,
 220244.779251566 150505.61834893,220207.243902439 150496,220187.50360229 150462.657300346,
 220197.12195122 150425.12195122,220227 150406)
```

참고: ST_LineToCurve

8.11.8. ST_DelaunayTriangles

ST_DelaunayTriangles — 주어진 입력 포인트들을 둘러싼 들로네 삼각망(Delaunay triangulation)을 반환합니다.

Synopsis

geometry **ST_DelaunayTriangles**(geometry *g1*, float *tolerance*, int4 *flags*);

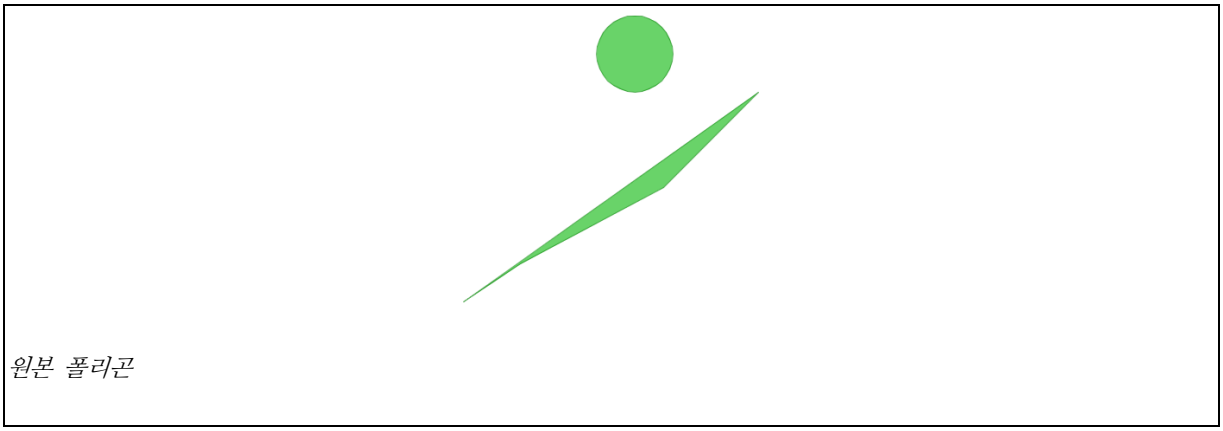
설명: 입력 도형의 꼭짓점들을 둘러싼 들로네 삼각망 을 반환합니다. 폴리곤 집합(flags=0 인 경우), 멀티라인스트링(flags=1 인 경우), 또는 TIN(flags=2 인 경우)을 출력합니다. 허용 오차가 있을 경우, 입력 꼭짓점들을 서로 스냅시키는 데 이용합니다.

2.1.0 버전부터 사용할 수 있습니다. GEOS 3.4.0 이상 버전이 필요합니다.

✓ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

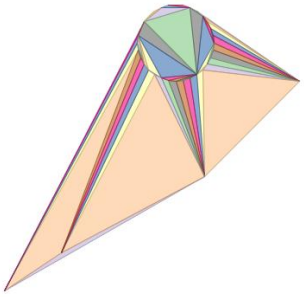
✓ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

2 차원 예시



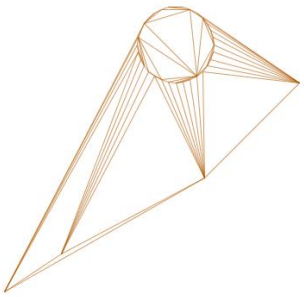
원본 폴리곤

```
-- 원본 도형 --
    ST_Union(ST_GeomFromText('POLYGON((175 150, 20 40,
            50 60, 125 100, 175 150))'),
        ST_Buffer(ST_GeomFromText('POINT(110 170)'), 20)
        )
```



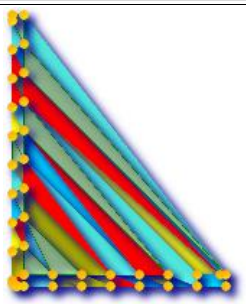
두 폴리곤의 ST_DelaunayTriangles: 각 삼각형이 서로 다른 색으로 칠해진 들로네 삼각형 폴리곤

```
-- 멀티라인스트링 삼각형과 중첩한 도형
SELECT
    ST_DelaunayTriangles(
        ST_Union(ST_GeomFromText('POLYGON((175 150, 20 40,
            50 60, 125 100, 175 150))'),
        ST_Buffer(ST_GeomFromText('POINT(110 170)'), 20)
        ))
     As dtriag;
```



-- 멀티라인스트링인 들로네 삼각형

```
SELECT
    ST_DelaunayTriangles(
        ST_Union(ST_GeomFromText('POLYGON((175 150, 20 40,
            50 60, 125 100, 175 150))'),
        ST_Buffer(ST_GeomFromText('POINT(110 170)'), 20)
        ),0.001,1)
     As dtriag;
```



-- 삼각형 폴리곤 55 개인 포인트 45 개의 들로네 삼각형

```
-- 이 쿼리는 L 형상을 이루고 있는 포인트 42 개의 테이블을 생성합니다.
```

```
SELECT (ST_DumpPoints(ST_GeomFromText(
'MULTIPOINT(14 14,34 14,54 14,74 14,94 14,114 14,134 14,
150 14,154 14,154 6,134 6,114 6,94 6,74 6,54 6,34 6,
14 6,10 6,8 6,7 7,6 8,6 10,6 30,6 50,6 70,6 90,6 110,6 130,
6 150,6 170,6 190,6 194,14 194,14 174,14 154,14 134,14 114,
14 94,14 74,14 54,14 34,14 14)'))).geom
    INTO TABLE l_shape;
-- 개별 삼각형 폴리곤으로 출력
SELECT ST_AsText((ST_Dump(geom)).geom) As wkt
FROM ( SELECT ST_DelaunayTriangles(ST_Collect(geom)) As geom
FROM l_shape) As foo;

-- WKT --
POLYGON((6 194,6 190,14 194,6 194))
POLYGON((14 194,6 190,14 174,14 194))
POLYGON((14 194,14 174,154 14,14 194))
POLYGON((154 14,14 174,14 154,154 14))
POLYGON((154 14,14 154,150 14,154 14))
POLYGON((154 14,150 14,154 6,154 14))
:
:
```

3 차원 예시

```
-- 3D 멀티포인트 --
SELECT ST_AsText(ST_DelaunayTriangles(ST_GeomFromText(
'MULTIPOINT Z(14 14 10,
150 14 100,34 6 25, 20 10 150)'))) As wkt;

-- WKT --
GEOMETRYCOLLECTION Z (POLYGON Z ((14 14 10,20 10 150,34 6 25,14 14 10))
 ,POLYGON Z ((14 14 10,34 6 25,150 14 100,14 14 10)))
```

참고: ST_ConcaveHull, ST_Dump

8.11.9. ST_Difference

ST_Difference — 도형 A 에서 도형 B 와 교차하지 않는 부분을 표현하는 도형을 반환합니다.

Synopsis

geometry **ST_Difference**(geometry *geomA*, geometry *geomB*);

설명: 도형 A 에서 도형 B 와 교차하지 않는 부분을 표현하는 도형을 반환합니다. 도형 A 에서 ST_Intersection(A,B)를 빼고 남은 부분이라고 생각할 수 있습니다. 도형 A 가 도형 B 에 완전히 담겨 있을 경우 텅 빈 도형 집합을 반환합니다.



주의: 순서가 중요합니다. B 에서 A 를 빼면 언제나 B 의 일부분을 반환할 것입니다.

GEOS 모듈로 실행



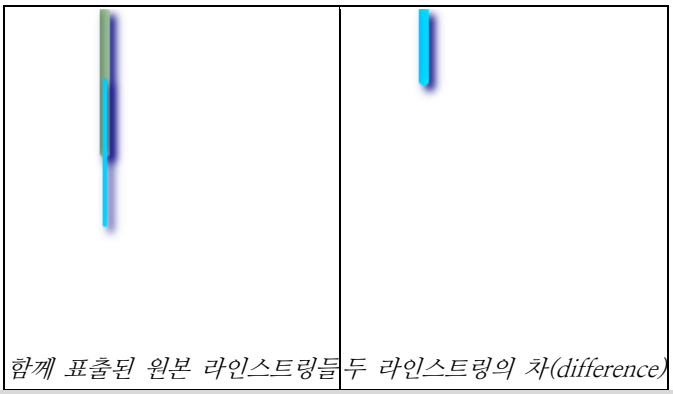
도형 집합을 인수로 호출하지 마십시오.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다. s2.1.1.3

✔ 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.20

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다. 하지만 이 함수는 빼기 작업시 XY 만 고려하고 작업후 Z-인덱스를 다시 붙이는 것으로 보입니다.

예시



함께 표출된 원본 라인스트링들 | *두 라인스트링의 차(difference)*

```
-- 2D 도형에 대해 안전합니다. ST_SymDifference 에서 보여주는 것과 동일한 도형입니다.
SELECT ST_AsText(
    ST_Difference(
            ST_GeomFromText('LINESTRING(50 100, 50 200)'),
            ST_GeomFromText('LINESTRING(50 50, 50 150)')
        )
    );

st_astext
---------
LINESTRING(50 150,50 200)
```

```
-- 3 차원 도형 입력시 제대로 동작하지 않습니다.
SELECT ST_AsEWKT(ST_Difference(ST_GeomFromEWKT('MULTIPOINT(-118.58 38.38 5,-118.60 38.329
6,-118.614 38.281 7)'), ST_GeomFromEWKT('POINT(-118.614 38.281 5)')));
st_asewkt
---------
MULTIPOINT(-118.6 38.329 6,-118.58 38.38 5)
```

참고: ST_SymDifference

8.11.10. ST_Dump

ST_Dump — 도형 g1 을 구성하고 있는 geometry_dump(도형, 경로) 행들의 집합을 반환합니다.

Synopsis

geometry_dump[] **ST_Dump**(geometry *g1*);

설명: 집합을 반환하는 함수(SRF; set-returning function)입니다. 이 함수는 도형(geom) 및 정수 배열(경로)로 이루어진 geometry_dump 행들의 집합을 반환합니다. 입력 도형이 단순 유형(포인트, 라인스트링, 폴리곤)일 경우 텅 빈 경로 배열과 geom 으로 입력 도형을 가진 단일 레코드를 반환할 것입니다. 입력 도형이 도형 집합 또는 멀티 유형일 경우, 각 집합 구성 요소 및 집합 내에서 해당 요소의 위치를 표현하는 경로로 이루어진 레코드를 반환할 것입니다.

ST_Dump는 도형들을 확장하는 데 유용합니다. 새 행들을 생성한다는 점에서 GROUP BY의 역함수라고 할 수 있습니다. 예를 들어 멀티폴리곤을 폴리곤으로 확장하는 데 이 함수를 쓸 수 있습니다.

개선 사항: 2.0.0 버전부터 다면체 표면, 삼각형 및 TIN을 지원하기 시작했습니다.

PostGIS 1.0.0RC1 버전부터 사용할 수 있습니다. PostgreSQL 7.3 이상 버전이 필요합니다.



1.3.4 미만 버전에서 이 함수에 만곡 도형(curve)을 담고 있는 도형을 입력하면 충돌이 일어났습니다. 1.3.4 버전부터 이 버그가 해결됐습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

✔ 이 함수는 3차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

표준 예시

```
SELECT sometable.field1, sometable.field1,
    (ST_Dump(sometable.the_geom)).geom AS the_geom
FROM sometable;

-- 복심곡선을 그 구성 요소인 라인스트링과 원호 스트링으로 분해합니다.
SELECT ST_AsEWKT(a.geom), ST_HasArc(a.geom)
 FROM ( SELECT (ST_Dump(p_geom)).geom AS geom
     FROM (SELECT ST_GeomFromEWKT('COMPOUNDCURVE(CIRCULARSTRING(0 0, 1 1, 1 0),(1 0, 0
1))') AS p_geom) AS b
     ) AS a;
      st_asewkt      | st_hasarc
-----------------------------+----------
 CIRCULARSTRING(0 0,1 1,1 0) | t
 LINESTRING(1 0,0 1)     | f
(2 rows)
```

다면체 표면, TIN 및 삼각형 예시

```
-- 다면체 표면 예시
-- 다면체 표면을 각 면으로 분해
SELECT (a.p_geom).path[1] As path, ST_AsEWKT((a.p_geom).geom) As geom_ewkt
 FROM (SELECT ST_Dump(ST_GeomFromEWKT('POLYHEDRALSURFACE(
((0 0 0, 0 0 1, 0 1 1, 0 1 0, 0 0 0)),
```

```
((0 0 0, 0 1 0, 1 1 0, 1 0 0, 0 0 0)), ((0 0 0, 1 0 0, 1 0 1, 0 0 1, 0 0 0)), ((1 1 0, 1
1 1, 1 0 1, 1 0 0, 1 1 0)),
((0 1 0, 0 1 1, 1 1 1, 1 1 0, 0 1 0)), ((0 0 1, 1 0 1, 1 1 1, 0 1 1, 0 0 1))
)') ) AS p_geom ) AS a;

 path |         geom_ewkt
------+-------------------------------------
   1 | POLYGON((0 0 0,0 0 1,0 1 1,0 1 0,0 0 0))
   2 | POLYGON((0 0 0,0 1 0,1 1 0,1 0 0,0 0 0))
   3 | POLYGON((0 0 0,1 0 0,1 0 1,0 0 1,0 0 0))
   4 | POLYGON((1 1 0,1 1 1,1 0 1,1 0 0,1 1 0))
   5 | POLYGON((0 1 0,0 1 1,1 1 1,1 1 0,0 1 0))
   6 | POLYGON((0 0 1,1 0 1,1 1 1,0 1 1,0 0 1))
-- TIN --
SELECT (g.gdump).path, ST_AsEWKT((g.gdump).geom) as wkt
 FROM
   (SELECT
    ST_Dump( ST_GeomFromEWKT('TIN (((
         0 0 0,
         0 0 1,
         0 1 0,
         0 0 0
       )), ((
         0 0 0,
         0 1 0,
         1 1 0,
         0 0 0
       ))
       )') ) AS gdump
   ) AS g;
-- 결과--
 path |         wkt
------+-------------------------------------
 {1} | TRIANGLE((0 0 0,0 0 1,0 1 0,0 0 0))
 {2} | TRIANGLE((0 0 0,0 1 0,1 1 0,0 0 0))
```

참고: geometry_dump, Section 14.6, “PostGIS 도형/지리형/래스터 덤프 함수”, ST_Collect, ST_Collect, ST_GeometryN

8.11.11. ST_DumpPoints

ST_DumpPoints — 도형을 구성하는 모든 포인트의 geometry_dump(도형, 경로) 행들의 집합을 반환합니다.

Synopsis

geometry_dump[]**ST_DumpPoints**(geometry *geom*);

설명: 이 집합 반환 함수(SRF)는 도형(geom) 및 정수 배열(path)로 이루어진 geometry_dump 행들의 집합을 반환합니다.

geometry_dump 의 geom 구성 요소는 모두 입력 도형을 구성하는 POINT 들입니다.

geometry_dump (integer[])의 path 구성 요소는 입력 도형의 POINT 들을 열거하는 참조 인덱스입니다. 예를 들어 LINESTRING 을 입력할 경우, i 가 LINESTRING 에 있는 nth 좌표인 {i} 의 경로를 반환합니다.

POLYGON 을 입력할 경우, i 가 고리 번호(1 이 외곽 고리이며, 내곽 고리들은 순차적으로 다음 숫자)이며 j 가 POINT 들을 (1-기반 인덱스에 대해) 열거하는 {i,j} 의 경로를 반환합니다.

개선 사항: 2.1.0 버전에서 속도가 향상됐습니다. native-C 로 재구현됐습니다.

개선 사항: 2.0.0 버전부터 다면체 표면, 삼각형 및 TIN 을 지원하기 시작했습니다.

1.5.0 버전부터 사용할 수 있습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

라인스트링 테이블을 노드로 분해하는 전형적인 예시

```
SELECT edge_id, (dp).path[1] As index, ST_AsText((dp).geom) As wktnode
FROM (SELECT 1 As edge_id
    , ST_DumpPoints(ST_GeomFromText('LINESTRING(1 2, 3 4, 10 10)')) AS dp
   UNION ALL
   SELECT 2 As edge_id
    , ST_DumpPoints(ST_GeomFromText('LINESTRING(3 5, 5 6, 9 10)')) AS dp
  ) As foo;
 edge_id | index |  wktnode
---------+-------+--------------
    1 |   1 | POINT(1 2)
    1 |   2 | POINT(3 4)
    1 |   3 | POINT(10 10)
    2 |   1 | POINT(3 5)
    2 |   2 | POINT(5 6)
    2 |   3 | POINT(9 10)
```

표준 도형 예시



```
 SELECT path, ST_AsText(geom)
FROM (
 SELECT (ST_DumpPoints(g.geom)).*
 FROM
  (SELECT
   'GEOMETRYCOLLECTION(
     POINT ( 0 1 ),
     LINESTRING ( 0 3, 3 4 ),
     POLYGON (( 2 0, 2 3, 0 2, 2 0 )),
     POLYGON (( 3 0, 3 3, 6 3, 6 0, 3 0 ),
```

```
          ( 5 1, 4 2, 5 2, 5 1 )),
     MULTIPOLYGON (
          (( 0 5, 0 8, 4 8, 4 5, 0 5 ),
          ( 1 6, 3 6, 2 7, 1 6 )),
          (( 5 4, 5 8, 6 7, 5 4 ))
     )
    )'::geometry AS geom
  ) AS g
 ) j;

  path   | st_astext
-----------+------------
 {1,1}   | POINT(0 1)
 {2,1}   | POINT(0 3)
 {2,2}   | POINT(3 4)
 {3,1,1} | POINT(2 0)
 {3,1,2} | POINT(2 3)
 {3,1,3} | POINT(0 2)
 {3,1,4} | POINT(2 0)
 {4,1,1} | POINT(3 0)
 {4,1,2} | POINT(3 3)
 {4,1,3} | POINT(6 3)
 {4,1,4} | POINT(6 0)
 {4,1,5} | POINT(3 0)
 {4,2,1} | POINT(5 1)
 {4,2,2} | POINT(4 2)
 {4,2,3} | POINT(5 2)
 {4,2,4} | POINT(5 1)
 {5,1,1,1} | POINT(0 5)
 {5,1,1,2} | POINT(0 8)
 {5,1,1,3} | POINT(4 8)
 {5,1,1,4} | POINT(4 5)
 {5,1,1,5} | POINT(0 5)
 {5,1,2,1} | POINT(1 6)
 {5,1,2,2} | POINT(3 6)
 {5,1,2,3} | POINT(2 7)
 {5,1,2,4} | POINT(1 6)
 {5,2,1,1} | POINT(5 4)
 {5,2,1,2} | POINT(5 8)
 {5,2,1,3} | POINT(6 7)
 {5,2,1,4} | POINT(5 4)
(29 rows)
```

다면체 표면, TIN 및 삼각형 예시

```
-- 다면체 표면 정육면체 --
SELECT (g.gdump).path, ST_AsEWKT((g.gdump).geom) as wkt
 FROM
   (SELECT
    ST_DumpPoints(ST_GeomFromEWKT('POLYHEDRALSURFACE( ((0 0 0, 0 0 1, 0 1 1, 0 1 0, 0 0 0)),
((0 0 0, 0 1 0, 1 1 0, 1 0 0, 0 0 0)), ((0 0 0, 1 0 0, 1 0 1, 0 0 1, 0 0 0)),
((1 1 0, 1 1 1, 1 0 1, 1 0 0, 1 1 0)),
((0 1 0, 0 1 1, 1 1 1, 1 1 0, 0 1 0)), ((0 0 1, 1 0 1, 1 1 1, 0 1 1, 0 0 1)) )') ) AS gdump
   ) AS g;
-- 결과 --
 path  |   wkt
---------+-------------
 {1,1,1} | POINT(0 0 0)
 {1,1,2} | POINT(0 0 1)
 {1,1,3} | POINT(0 1 1)
 {1,1,4} | POINT(0 1 0)
```

```
 {1,1,5} | POINT(0 0 0)
 {2,1,1} | POINT(0 0 0)
 {2,1,2} | POINT(0 1 0)
 {2,1,3} | POINT(1 1 0)
 {2,1,4} | POINT(1 0 0)
 {2,1,5} | POINT(0 0 0)
 {3,1,1} | POINT(0 0 0)
 {3,1,2} | POINT(1 0 0)
 {3,1,3} | POINT(1 0 1)
 {3,1,4} | POINT(0 0 1)
 {3,1,5} | POINT(0 0 0)
 {4,1,1} | POINT(1 1 0)
 {4,1,2} | POINT(1 1 1)
 {4,1,3} | POINT(1 0 1)
 {4,1,4} | POINT(1 0 0)
 {4,1,5} | POINT(1 1 0)
 {5,1,1} | POINT(0 1 0)
 {5,1,2} | POINT(0 1 1)
 {5,1,3} | POINT(1 1 1)
 {5,1,4} | POINT(1 1 0)
 {5,1,5} | POINT(0 1 0)
 {6,1,1} | POINT(0 0 1)
 {6,1,2} | POINT(1 0 1)
 {6,1,3} | POINT(1 1 1)
 {6,1,4} | POINT(0 1 1)
 {6,1,5} | POINT(0 0 1)
(30 rows)
-- 삼각형 --
SELECT (g.gdump).path, ST_AsText((g.gdump).geom) as wkt
 FROM
   (SELECT
    ST_DumpPoints( ST_GeomFromEWKT('TRIANGLE ((
         0 0,
         0 9,
         9 0,
         0 0
       ))') ) AS gdump
  ) AS g;
-- 결과 --
 path |  wkt
------+------------
 {1} | POINT(0 0)
 {2} | POINT(0 9)
 {3} | POINT(9 0)
 {4} | POINT(0 0)
-- TIN --
SELECT (g.gdump).path, ST_AsEWKT((g.gdump).geom) as wkt
 FROM
   (SELECT
    ST_DumpPoints( ST_GeomFromEWKT('TIN (((
         0 0 0,
         0 0 1,
         0 1 0,
         0 0 0
       )), ((
         0 0 0,
         0 1 0,
         1 1 0,
         0 0 0
       ))
       )') ) AS gdump
  ) AS g;
```

```
-- 결과 --
 path  |   wkt
-------+-------------
 {1,1,1} | POINT(0 0 0)
 {1,1,2} | POINT(0 0 1)
 {1,1,3} | POINT(0 1 0)
 {1,1,4} | POINT(0 0 0)
 {2,1,1} | POINT(0 0 0)
 {2,1,2} | POINT(0 1 0)
 {2,1,3} | POINT(1 1 0)
 {2,1,4} | POINT(0 0 0)
(8 rows)
```

참고: geometry_dump, Section 14.6, "PostGIS 도형/지리형/래스터 덤프 함수", ST_Dump, ST_DumpRings

8.11.12. ST_DumpRings

ST_DumpRings — 폴리곤의 외곽 및 내곽 고리를 표현하는 geometry_dump 행들의 집합을 반환합니다.

Synopsis

geometry_dump[] **ST_DumpRings**(geometry *a_polygon*);

설명: 집합 반환 함수(SRF)입니다. 각각 "path"와 "geom"이라는 변수명으로 불리는 integer[] 와 geometry 로 정의되는 geometry_dump 행들의 집합을 반환합니다. "path" 항목은 단일 정수를 담고 있는 폴리곤 고리 인덱스를 들고 있습니다. 외곽 고리일 경우 이 정수가 0, 구멍일 경우 양수가 됩니다. "geom" 항목은 각각 상응하는 고리를 폴리곤으로 담고 있습니다.

PostGIS 1.1.3 버전부터 사용할 수 있습니다. PostgreSQL 7.3 이상 버전이 필요합니다.



이 함수는 폴리곤만 입력받을 수 있습니다. 멀티폴리곤은 입력받지 못 합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
SELECT sometable.field1, sometable.field1,
     (ST_DumpRings(sometable.the_geom)).geom As the_geom
FROM sometableOfpolys;

SELECT ST_AsEWKT(geom) As the_geom, path
    FROM ST_DumpRings(
        ST_GeomFromEWKT('POLYGON((-8149064 5133092 1,-8149064 5132986 1,-8148996 5132839 1,-
8148972 5132767 1,-8148958 5132508 1,-8148941 5132466 1,-8148924 5132394 1,
        -8148903 5132210 1,-8148930 5131967 1,-8148992 5131978 1,-8149237 5132093 1,-8149404
5132211 1,-8149647 5132310 1,-8149757 5132394 1,
        -8150305 5132788 1,-8149064 5133092 1),
        (-8149362 5132394 1,-8149446 5132501 1,-8149548 5132597 1,-8149695 5132675 1,-
8149362 5132394 1))')
        ) as foo;
 path |                        the_geom
```

```
----------------------------------------------------------------------------------------
--------------------
 {0} | POLYGON((-8149064 5133092 1,-8149064 5132986 1,-8148996 5132839 1,-8148972 5132767
1,-8148958 5132508 1,
     |      -8148941 5132466 1,-8148924 5132394 1,
     |      -8148903 5132210 1,-8148930 5131967 1,
     |      -8148992 5131978 1,-8149237 5132093 1,
     |       -8149404 5132211 1,-8149647 5132310 1,-8149757 5132394 1,-8150305 5132788 1,-
8149064 5133092 1))
 {1} | POLYGON((-8149362 5132394 1,-8149446 5132501 1,
     |      -8149548 5132597 1,-8149695 5132675 1,-8149362 5132394 1))
```

참고: geometry_dump, Section 14.6, "PostGIS 도형/지리형/래스터 덤프 함수", ST_Dump, ST_ExteriorRing, ST_InteriorRingN

8.11.13. ST_FlipCoordinates

ST_FlipCoordinates — 입력 도형을 X 축과 Y 축을 뒤집은 상태로 반환합니다. 위도/경도 피처를 빌드해서 수정해야 하는 경우 유용합니다.

Synopsis

geometry **ST_FlipCoordinates**(geometry *geom*);

설명: 입력 도형을 X 축과 Y 축을 뒤집은 상태로 반환합니다.

2.0.0 버전부터 사용할 수 있습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 함수는 M 좌표를 지원합니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

예시

```
SELECT ST_AsEWKT(ST_FlipCoordinates(GeomFromEWKT('POINT(1 2)')));
 st_asewkt
------------
POINT(2 1)
```

참고: ST_SwapOrdinates

8.11.14. ST_GeneratePoints

ST_GeneratePoints — 폴리곤 또는 멀티폴리곤을 원본 면의 내부에 임의로 배치한 포인트들로 구성된 멀티포인트로 변환합니다.

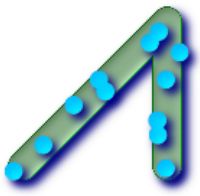
Synopsis

geometry **ST_GeneratePoints**(*g* geometry , *npoints* numeric);

설명: ST_GeneratePoints 함수는 요청한 개수가 될 때까지 입력된 면 내부에 의사무작위(pseudo-random) 포인트를 생성합니다.

2.3.0 버전부터 사용할 수 있습니다.

예시

<table>
<tr>
<td> 
원본 폴리곤</td>
<td> 
원본 폴리곤 위에 중첩한 포인트 12 개를 생성
<pre>SELECT ST_GeneratePoints(
    ST_Buffer(
        ST_GeomFromText(
        'LINESTRING(50 50,150 150,150 50)'
        ), 10, 'endcap=round join=round'), 12);</pre></td>
</tr>
</table>

8.11.15. ST_Intersection

ST_Intersection — geomA 와 geomB 의 공유 부분을 표현하는 도형을 반환합니다.

Synopsis

geometry **ST_Intersection**(geometry *geomA* , geometry *geomB*);

geography **ST_Intersection**(geography *geogA* , geography *geogB*);

설명: 도형들의 포인트 집합 교차 부분을 표현하는 도형을 반환합니다.

다시 말해서, 도형 A 와 도형 B 가 공유하는 부분을 반환합니다.

도형들이 어떤 공간도 공유하지 않을 (분리돼 있을) 경우, 텅 빈 도형 집합을 반환합니다.

ST_Intersection 함수를 ST_Intersects 함수와 결합하면, 관심 지역 또는 국가에 들어가는 도형의 일부분만 반환받고자 할 때 경계 상자, 버퍼, 지역 쿼리 내부와 같은 잘라내기(clipping) 도형을 생성하는 데 매우 유용합니다.

Note! 지리형: 지리형에 대해, 이 함수는 도형 구현을 둘러싼 정말로 얇은 래퍼입니다. 이 함수는 먼저 (지리형 객체들이 절반의 UTM 구역 내부에 있지만 동일한 UTM 이 어떤 도형도 선택하지 않을 경우) 두 지리형 객체들의 경계 상자에 가장 어울리는 SRID 를 (UTM 또는 람베르트 북극/남극 방위정적도법(Lambert Azimuthal Equal Area)을 선호하며, 최악의 경우 메르카토르 도법에 의지해서) 결정한 다음 가장 어울리는 평면 공간 참조 시스템 상에서 교차시킨 다음 WGS84 지리형으로 다시 변환합니다.

! GEOMETRYCOLLECTION 을 인수로 호출하지 마십시오.

! 3 차원 도형을 작업할 경우, 3 차원 도형에 대해 제대로 된 3 차원 교차 작업을 하는 SFCGAL 기반 ST_3DIntersection 을 이용하는 편이 좋을 수도 있습니다. 비록 이 함수가 Z 좌표를 처리하기는 하지만, postgis.backend=geos 로 설정했을 때 Z 좌표값의 평균을 냅니다. postgis.backend=sfcgal 로 설정하면, Z 좌표에 상관없이 2 차원 도형을 반환할 것입니다. 자세한 내용은 postgis.backend 를 참조하십시오.

GEOS 모듈로 실행

✓ SFCGAL 백엔드도 이 방법을 제공합니다.

1.5 버전부터 지리형 데이터형을 지원합니다.

✓ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다. s2.1.1.3

✓ 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.18

예시

```
SELECT ST_AsText(ST_Intersection('POINT(0 0)'::geometry, 'LINESTRING ( 2 0, 0
2 )'::geometry));
 st_astext
---------------
GEOMETRYCOLLECTION EMPTY
(1 row)
SELECT ST_AsText(ST_Intersection('POINT(0 0)'::geometry, 'LINESTRING ( 0 0, 0
2 )'::geometry));
 st_astext
---------------
POINT(0 0)
(1 row)

-- 자기체 경계선으로 모든 라인(길) 을 잘라냅니다(이때 자기체 도형이 폴리곤 또는 멀티폴리곤이라고 가정합니다) .
-- 주의: 라인스트링 또는 멀티라인스트링으로 출력되는 교차점만 유지합니다.
-- 포인트만을 공유하는 길에는 관심이 없기 때문입니다.
-- 도형 집합을 개별 멀티 도형 부분들로 확장하기 위해 덤프를 받아야 합니다.
-- 다음은 꽤 일반적인 쿼리로 WHERE 절을 변경하는 것만으로 여러 유형들을 작업할 수 있습니다.
SELECT clipped.gid, clipped.f_name, clipped_geom
FROM (SELECT trails.gid, trails.f_name, (ST_Dump(ST_Intersection(country.the_geom,
trails.the_geom))).geom As clipped_geom
FROM country
```

```
    INNER JOIN trails
    ON ST_Intersects(country.the_geom, trails.the_geom)) As clipped
    WHERE ST_Dimension(clipped.clipped_geom) = 1 ;

-- 폴리곤 랜드마크 같은 폴리곤의 경우, 텅 빈 도형 집합으로 출력되는 폴리곤을 제외한 (즉 폴리곤, 라인 및 포인트를 담고 있는 도형 집합의) 모든 것에 버퍼를 0.0 으로 적용하는 더 빠른 꼼수를 쓸 수도 있습니다.
-- 버퍼를 0.0 으로 적용하면 집합 구조를 없애고 폴리곤만 남기게 됩니다.
SELECT poly.gid, ST_Multi(ST_Buffer(
                 ST_Intersection(country.the_geom, poly.the_geom),
                 0.0)
                 ) As clipped_geom
FROM country
    INNER JOIN poly
    ON ST_Intersects(country.the_geom, poly.the_geom)
    WHERE Not ST_IsEmpty(ST_Buffer(ST_Intersection(country.the_geom, poly.the_geom),0.0));
```

예시: 대략 2.5 차원

따로 설정하지 않았다면 GEOS 가 기본 백엔드입니다. 이 함수는 진정한 교차가 아니라는 점에 주의하십시오. ST_3DIntersection 함수로 동일한 예시를 실행해서 비교해보십시오.

```
set postgis.backend=geos;
select ST_AsText(ST_Intersection(linestring, polygon)) As wkt
from ST_GeomFromText('LINESTRING Z (2 2 6,1.5 1.5 7,1 1 8,0.5 0.5 8,0 0 10)') AS linestring
 CROSS JOIN ST_GeomFromText('POLYGON((0 0 8, 0 1 8, 1 1 8, 1 0 8, 0 0 8))') AS polygon;

        st_astext
-------------------------------------
 LINESTRING Z (1 1 8,0.5 0.5 8,0 0 10)
```

SFCGAL 지원과 함께 사용자 PostGIS 를 컴파일한 경우, SFCGAL 을 사용하도록 선택할 수 있습니다. 그러나 기본적으로, 교차 작업을 하기 전에 도형을 2 차원으로 변환하거나, ST_Force2D 와 동등한, 2 차원 도형인 결과를 반환하거나 하는 두 가지 경우로 압축됩니다.

```
set postgis.backend=sfcgal;
select ST_AsText(ST_Intersection(linestring, polygon)) As wkt
from ST_GeomFromText('LINESTRING Z (2 2 6,1.5 1.5 7,1 1 8,0.5 0.5 8,0 0 10)') AS linestring
 CROSS JOIN ST_GeomFromText('POLYGON((0 0 8, 0 1 8, 1 1 8, 1 0 8, 0 0 8))') AS polygon;

           wkt
---------------------------------------------
 MULTILINESTRING((0.5 0.5,0 0),(1 1,0.5 0.5))
```

참고: ST_3DIntersection, ST_Difference, ST_Dimension, ST_Dump, ST_Force2D, ST_SymDifference, ST_Intersects, ST_Multi

8.11.16. ST_LineToCurve

ST_LineToCurve — 라인스트링/폴리곤을 원호 스트링/만곡 폴리곤으로 변환합니다.

Synopsis

geometry **ST_LineToCurve**(geometry *geomANoncircular*);

설명: 평범한 라인스트링/폴리곤을 각각 원호 스트링/만곡 폴리곤으로 변환합니다. 상응하는 만곡 도형을 표현하는 데 필요한 포인트가 몇 개 안 된다는 점을 눈여겨 보십시오.

1.2.2 버전(?)부터 사용할 수 있습니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

예시: 2 차원

```
SELECT  ST_AsText(ST_LineToCurve(foo.the_geom))  As  curvedastext,ST_AsText(foo.the_geom)  As
non_curvedastext
    FROM (SELECT ST_Buffer('POINT(1 3)'::geometry, 3) As the_geom) As foo;

curvedatext                                    non_curvedastext
-----------------------------------------|-----------------------------------------------
CURVEPOLYGON(CIRCULARSTRING(4    3,3.12132034355964    0.878679656440359,    |    POLYGON((4
3,3.94235584120969 2.41472903395162,3.77163859753386 1.85194970290473,
1  0,-1.12132034355965  5.12132034355963,4  3))                              |  3.49440883690764
1.33328930094119,3.12132034355964 0.878679656440359,
                                      |  2.66671069905881  0.505591163092366,2.14805029709527
0.228361402466141,
                                      | 1.58527096604839 0.0576441587903094,1 0,
                                      |          0.414729033951621     0.0576441587903077,-
0.148050297095264 0.228361402466137,
                                      |          -0.666710699058802     0.505591163092361,-
1.12132034355964 0.878679656440353,
                                      | -1.49440883690763 1.33328930094119,-1.77163859753386
1.85194970290472
                                      | --ETC-- ,3.94235584120969 3.58527096604839,4 3))
-- 3 차원 예시
SELECT ST_AsEWKT(ST_LineToCurve(ST_GeomFromEWKT('LINESTRING(1 2 3, 3 4 8, 5 6 4, 7 8 4, 9 10
4)')));

             st_asewkt
-----------------------------------
 CIRCULARSTRING(1 2 3,5 6 4,9 10 4)
```

참고: ST_CurveToLine

8.11.17. ST_MakeValid

ST_MakeValid — 유효하지 않은 도형을 꼭짓점을 버리지 않고 유효한 도형으로 바꾸려고 시도합니다.

Synopsis

geometry **ST_MakeValid**(geometry *input*);

설명: 이 함수는 주어진 유효하지 않은 도형을 입력 꼭짓점 하나도 버리지 않은 채 유효한 표현식으로 생성하려 시도합니다. 이미 유효한 도형을 입력하면 아무 것도 바꾸지 않은 채 반환합니다.

포인트, 멀티포인트, 라인스트링, 멀티라인스트링, 폴리곤, 멀티폴리곤 및 이들의 어떤 조합이건 담고 있는 도형 집합을 입력받을 수 있습니다.

전체 또는 일부 차원이 붕괴한 경우, 낮거나 동등한 차원을 가진 도형들의 집합 또는 낮은 차원의 도형을 출력할 수도 있습니다.

스스로 교차하는 경우 단일 폴리곤이 멀티 도형이 될 수도 있습니다.

2.0.0 버전부터 사용할 수 있습니다. GEOS 3.3.0 버전이 필요합니다.

개선 사항: 2.1.0 버전에서 속도가 향상됐습니다. GEOS 3.3.4 버전이 필요합니다.

개선 사항: 2.1.0 버전부터 도형 집합 및 멀티포인트를 지원하기 시작했습니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

참고: ST_IsValid ST_CollectionExtract

8.11.18. ST_MemUnion

ST_MemUnion — 메모리 친화적이라는 점만 제외하면 ST_Union 함수와 동일합니다(메모리를 덜 사용하고 CPU 시간을 더 사용합니다).

Synopsis

geometry **ST_MemUnion**(geometry set *geomfield*);

설명: 쓸 만한 설명을 추가바랍니다.



메모리 친화적이라는 점만 제외하면 ST_Union 함수와 동일합니다(메모리를 덜 사용하고 CPU 시간을 더 사용합니다). 먼저 배열을 생성한 다음 통합하는 ST_Union 종합 함수와는 달리, 이 종합 합수는 이전 단계의 결과물에 도형을 하나씩 통합하는 식으로 동작합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
ST_Union 참고
```

참고: ST_Union

8.11.19. ST_MinimumBoundingCircle

ST_MinimumBoundingCircle — 도형 전체를 담을 수 있는 가장 작은 원형 폴리곤을 반환합니다. 기본값은 사분원 당 구간 48 개를 사용하는 것입니다.

Synopsis

geometry **ST_MinimumBoundingCircle**(geometry *geomA*, integer *num_segs_per_qt_circ=48*);

설명: 도형 전체를 담을 수 있는 가장 작은 원형 폴리곤을 반환합니다.



사분원 당 구간 48 개라는 기본값으로 원에 가까운 폴리곤을 생성합니다. 이 폴리곤이 최소 경계 원(minimum bounding circle)의 근사치이기 때문에, 입력 도형 안에 있는 포인트 몇 개는 폴리곤 내부에 담기지 않을 수도 있습니다. 구간의 개수를 늘리면 성능을 거의 저하시키지 않고도 근사치를 향상시킬 수 있습니다. 폴리곤 근사치가 적당하지 않은 경우, ST_MinimumBoundingRadius 함수를 사용할 수도 있습니다.

이 함수는 주로 멀티 도형 및 도형 집합을 입력받습니다. 비록 종합 함수는 아니지만, ST_Collect 과 결합해서 도형들의 집합의 최소 경계 원을 얻을 수 있습니다. 예: ST_MinimumBoundingCircle(ST_Collect(somepointfield))

폴리곤의 면적을 최소 경계 원의 면적으로 나눈 비율을 종종 루크(Roeck) 테스트라고 부릅니다.

1.4.0 버전부터 사용할 수 있습니다. GEOS 가 필요합니다.

참고: ST_Collect, ST_MinimumBoundingRadius

예시

```
SELECT d.disease_type,
      ST_MinimumBoundingCircle(ST_Collect(d.the_geom)) As the_geom
      FROM disease_obs As d
      GROUP BY d.disease_type;
```



포인트 및 라인스트링의 최소 경계 원. 사분원의 근사치를 구하는 데 구간 8 개를 이용합니다.

```
SELECT ST_AsText(ST_MinimumBoundingCircle(
             ST_Collect(
                   ST_GeomFromEWKT('LINESTRING(55 75,125 150)'),
                         ST_Point(20, 80)), 8
                         )) As wktmbc;
wktmbc
----------
POLYGON((135.59714732062              115,134.384753327498              102.690357210921,130.79416296937
90.8537670908995,124.963360620072   79.9451031602111,117.116420743937   70.3835792560632,107.554896839789
62.5366393799277,96.6462329091006    56.70583703063,84.8096427890789     53.115246672502,72.5000000000001
51.9028526793802,60.1903572109213   53.1152466725019,48.3537670908996   56.7058370306299,37.4451031602112
62.5366393799276,27.8835792560632    70.383579256063,20.0366393799278     79.9451031602109,14.20583703063
90.8537670908993,10.615246672502          102.690357210921,9.40285267938019          115,10.6152466725019
127.309642789079,14.2058370306299    139.1462329091,20.0366393799275     150.054896839789,27.883579256063
159.616420743937,

37.4451031602108 167.463360620072,48.3537670908992
 173.29416296937,60.190357210921 176.884753327498,

72.4999999999998 178.09714732062,84.8096427890786
```

```
176.884753327498,96.6462329091003 173.29416296937,107.554896839789
167.463360620072,

117.116420743937 159.616420743937,124.963360620072
150.054896839789,130.79416296937 139.146232909101,134.384753327498
 127.309642789079,135.59714732062 115))
```

참고: ST_Collect, ST_MinimumBoundingRadius

8.11.20. ST_MinimumBoundingRadius

ST_MinimumBoundingRadius — 도형 전체를 담을 수 있는 가장 작은 원의 중심 포인트 및 반지름을 반환합니다.

Synopsis

(geometry, double precision) **ST_MinimumBoundingRadius**(geometry *geom*);

설명: 도형 전체를 담을 수 있는 가장 작은 원의 중심 포인트 및 반지름을 담고 있는 레코드를 반환합니다.

도형들의 집합의 최소 경계 원을 얻기 위해 이 함수와 ST_Collect 함수를 결합해서 이용할 수 있습니다.

2.3.0 버전부터 사용할 수 있습니다.

참고: ST_Collect, ST_MinimumBoundingCircle

예시

```
SELECT               ST_AsText(center),               radius               FROM
ST_MinimumBoundingRadius('POLYGON((26426 65078,26531 65242,26075 65136,26096
65427,26426 65078))');

             st_astext              |     radius
------------------------------------+------------------
 POINT(26284.8418027133 65267.1145090825) | 247.436045591407
```

8.11.21. ST_Polygonize

ST_Polygonize — 종합 함수입니다. 도형들의 집합의 선분 구성 요소로부터 형성될 수 있는 폴리곤들을 담고 있는 도형 집합을 생성합니다.

Synopsis

geometry **ST_Polygonize**(geometry set *geomfield*);

geometry **ST_Polygonize**(geometry[] *geom_array*);

설명: 도형들의 집합의 선분 구성 요소로부터 형성될 수 있는 폴리곤들을 담고 있는 도형 집합을 생성합니다.



도형 집합은 종종 제 3 자 도구로 처리하기 어렵기 때문에, 폴리곤들을 개별 폴리곤으로 덤프하기 위해 ST_Polygonize 를 ST_Dump 와 결합해서 이용하곤 합니다.



입력 선분의 교점(node)이 정확히 구성돼야만 이 함수가 제대로 동작합니다.

1.0.0RC1 버전부터 사용할 수 있습니다. GEOS 2.1.0 이상 버전이 필요합니다.

예시: 단일 라인스트링의 폴리곤화

```
SELECT ST_AsEWKT(ST_Polygonize(the_geom_4269)) As geomtextrep
FROM (SELECT the_geom_4269 FROM ma.suffolk_edges ORDER BY tlid LIMIT 45) As foo;

geomtextrep
------------------------------------
 SRID=4269;GEOMETRYCOLLECTION(POLYGON((-71.040878   42.285678,-71.040943   42.2856,-71.04096
42.285752,-71.040878 42.285678)),
 POLYGON((-71.17166     42.353675,-71.172026     42.354044,-71.17239     42.354358,-71.171794
42.354971,-71.170511 42.354855,
 -71.17112 42.354238,-71.17166 42.353675)))
(1 row)

-- ST_Dump를 통해 폴리곤화시킨 도형들을 개별 폴리곤으로 덤프합니다.
SELECT ST_AsEWKT((ST_Dump(foofoo.polycoll)).geom) As geomtextrep
FROM (SELECT ST_Polygonize(the_geom_4269) As polycoll
    FROM (SELECT the_geom_4269 FROM ma.suffolk_edges
        ORDER BY tlid LIMIT 45) As foo) As foofoo;

geomtextrep
------------------------
 SRID=4269;POLYGON((-71.040878 42.285678,-71.040943 42.2856,-71.04096 42.285752,
-71.040878 42.285678))
 SRID=4269;POLYGON((-71.17166 42.353675,-71.172026 42.354044,-71.17239 42.354358
,-71.171794 42.354971,-71.170511 42.354855,-71.17112 42.354238,-71.17166 42.353675))
(2 rows)
```

참고: ST_Node, ST_Dump

8.11.22. ST_Node

ST_Node — 라인스트링들의 집합에 노드를 적용합니다.

Synopsis

geometry **ST_Node**(geometry *geom*);

설명: 입력 라인스트링의 모든 노드를 보전하면서 가능한 한 가장 적은 개수의 노드를 이용해서 라인스트링 집합 전체에 노드를 적용합니다.

✓ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

2.0.0 버전부터 사용할 수 있습니다. GEOS 3.3.0 이상 버전이 필요합니다.



GEOS 3.3.1 버전까지의 버그 때문에 이 함수가 자체 교차하는 라인에 노드를 적용할 수 없었습니다. GEOS 3.3.2 버전부터 이 버그가 수정됐습니다.

예시

```
SELECT ST_AsEWKT(
         ST_Node('LINESTRINGZ(0 0 0, 10 10 10, 0 10 5, 10 0 3)'::geometry)
    ) As output;
output
------------
MULTILINESTRING((0 0 0,5 5 4.5),(5 5 4.5,10 10 10,0 10 5,5 5 4.5),(5 5 4.5,10 0 3))
```

참고: ST_UnaryUnion

8.11.23. ST_OffsetCurve

ST_OffsetCurve — 입력라인으로부터 주어진 거리 및 방향에 있는 오프셋 라인을 반환합니다. 중심선을 기준으로 평행한 라인들을 계산하는 데 유용합니다.

Synopsis

geometry **ST_OffsetCurve**(geometry *line*, float *signed_distance*, text *style_parameters=''*);

설명: 입력라인으로부터 주어진 거리 및 방향에 있는 오프셋 라인을 반환합니다. 반환된 도형의 모든 포인트들은 입력 도형으로부터 주어진 거리를 넘지 않습니다.

양의 거리를 설정한 경우 오프셋은 입력 라인의 왼쪽에 생성되며 동일한 방향을 유지할 것입니다. 음의 거리를 설정하면 오른쪽에 생성되며 반대 방향을 향할 것입니다.

2.0 버전부터 사용할 수 있습니다. GEOS 3.2 이상 버전이 필요합니다. GEOS 3.3 버전에서 개선됐습니다.

다음과 같이 연산을 조정하기 위해 선택적인 세 번째 파라미터로 공백으로 구분된 키=값 쌍의 목록을 설정할 수 있습니다:

- 'quad_segs=#' : 사분원(quarter circle)의 근사치를 계산하는 데 쓰이는 선분의 개수(기본값은 8)
- 'join=round|mitre|bevel' : 접합 스타일(기본값은 "원형(round)"). '마이터(mitre)'의 동의어로 '마이터(miter)'도 쓸 수 있습니다.
- 'mitre_limit=#.#' : 마이터 비율 제한(마이터 접합 스타일만 영향을 받습니다). 'mitre_limit'의 동의어로 'miter_limit'도 쓸 수 있습니다.

거리의 단위는 공간 참조 시스템의 단위로 측정됩니다.

라인스트링만 입력할 수 있습니다.

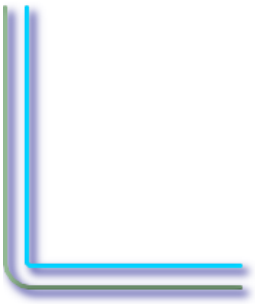
GEOS 모듈로 실행

Note! 이 함수는 3 차원 도형을 입력받더라도 세 번째 차원(Z)을 무시하고 언제나 2 차원 결과를 반환할 것입니다.

예시

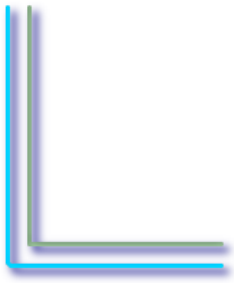
도로 주위로 열린 버퍼를 계산합니다.

```
SELECT ST_Union(
 ST_OffsetCurve(f.the_geom, f.width/2, 'quad_segs=4 join=round'),
 ST_OffsetCurve(f.the_geom, -f.width/2, 'quad_segs=4 join=round')
) as track
FROM someroadstable;
```



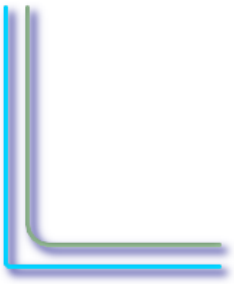
거리 15, 원본 라인에서 'quad_segs=4 join=round' 및 오프셋은 15 단위

```
SELECT ST_AsText(ST_OffsetCurve(ST_GeomFromText(
'LINESTRING(164 16,144 16,124 16,104 16,84 16,64 16,
    44 16,24 16,20 16,18 16,17 17,
    16 18,16 20,16 40,16 60,16 80,16 100,
    16 120,16 140,16 160,16 180,16 195)'),
    15, 'quad_segs=4 join=round'));
--출력물--
LINESTRING(164        1,18        1,12.2597485145232.1418070123307,
    7.39339828220179 5.39339828220179,
    5.39339828220179 7.39339828220179,
    2.14180701233067 12.2597485145237,1 18,1 195)
```



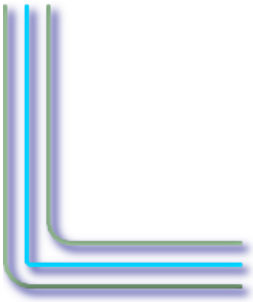
거리 -15, 원본 라인에서 'quad_segs=4 join=round' 및 오프셋은 -15 단위

```
SELECT ST_AsText(ST_OffsetCurve(geom,
    -15, 'quad_segs=4 join=round')) As notsocurvy
    FROM ST_GeomFromText(
'LINESTRING(164 16,144 16,124 16,104 16,84 16,64
16,
    44 16,24 16,20 16,18 16,17 17,
    16 18,16 20,16 40,16 60,16 80,16 100,
    16 120,16 140,16 160,16 180,16 195)') As geom;
-- notsocurvy --
LINESTRING(31 195,31 31,164 31)
```



더 둥글게 만들기 위한 이중 오프셋, 첫 번째 쿼리가 방향을 역전시킨다는 점에 주의하십시오. 즉 -30 + 15 = -15 입니다.

```
SELECT ST_AsText(ST_OffsetCurve(ST_OffsetCurve(geom,
    -30, 'quad_segs=4 join=round'), -15, 'quad_segs=
join=round')) As morecurvy
    FROM ST_GeomFromText(
```

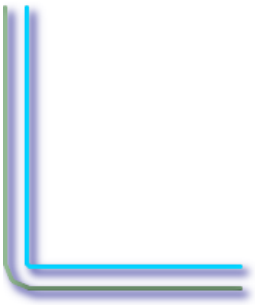


평행한 라인을 얻기 위한 정규 오프셋 15 와 결합된, 더 둥글게 만들기 위한 이중 오프셋. 원본과 중첩.

```
SELECT ST_AsText(ST_Collect(
    ST_OffsetCurve(geom,       15,       'quad_segs=
join=round'),
    ST_OffsetCurve(ST_OffsetCurve(geom,
    -30, 'quad_segs=4 join=round'), -15, 'quad_segs=
join=round')
```

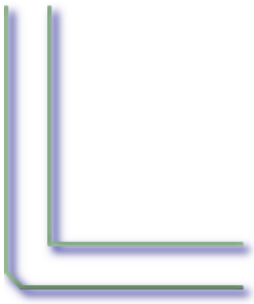
```
'LINESTRING(164 16,144 16,124 16,104 16,84 16,64 16,
    44 16,24 16,20 16,18 16,17 17,
    16 18,16 20,16 40,16 60,16 80,16 100,
    16 120,16 140,16 160,16 180,16 195)') As geom;
-- morecurvy --
LINESTRING(164        31,46        31,40.259748514523
32.1418070123307,
35.3933982822018 35.3933982822018,
32.1418070123307 40.2597485145237,31 46,31 195)
```

```
    )
) As parallel_curves
    FROM ST_GeomFromText(
'LINESTRING(164 16,144 16,124 16,104 16,84 16,64 16,
    44 16,24 16,20 16,18 16,17 17,
    16 18,16 20,16 40,16 60,16 80,16 100,
    16 120,16 140,16 160,16 180,16 195)') As geom;
-- 평행곡선 --
MULTILINESTRING((164      1,18      1,12.2597485145237
2.1418070123307,
7.39339828220179    5.39339828220179,5.39339828220179
7.39339828220179,
2.14180701233067 12.2597485145237,1 18,1 195),
(164            31,46            31,40.2597485145236
32.1418070123307,35.3933982822018 35.3933982822018,
32.1418070123307 40.2597485145237,31 46,31 195))
```



거리 15, 원본 라인과 함께 표출된 'quad_segs=4 join=round'

```
SELECT ST_AsText(ST_OffsetCurve(ST_GeomFromText(
'LINESTRING(164 16,144 16,124 16,104 16,84 16,64 16,
    44 16,24 16,20 16,18 16,17 17,
    16 18,16 20,16 40,16 60,16 80,16 100,
    16 120,16 140,16 160,16 180,16 195)'),
        15, 'quad_segs=4 join=bevel'));
-- 출력 --
LINESTRING(164        1,18        1,7.3933982822017
5.39339828220179,
    5.39339828220179 7.39339828220179,1 18,1 195)
```



수집된 15, -15. join=mitre mitre_limit=2.1

```
SELECT ST_AsText(ST_Collect(
    ST_OffsetCurve(geom, 15, 'quad_segs=4 join=mitr
mitre_limit=2.2'),
    ST_OffsetCurve(geom, -15, 'quad_segs=4 join=mitr
mitre_limit=2.2')
    ) )
    FROM ST_GeomFromText(
'LINESTRING(164 16,144 16,124 16,104 16,84 16,64 16,
    44 16,24 16,20 16,18 16,17 17,
    16 18,16 20,16 40,16 60,16 80,16 100,
    16 120,16 140,16 160,16 180,16 195)') As geom;
-- 출력물 --
MULTILINESTRING((164      1,11.7867965644036      1,
11.7867965644036,1 195),
    (31 195,31 31,164 31))
```

참고: ST_Buffer

8.11.24. ST_RemoveRepeatedPoints

ST_RemoveRepeatedPoints — 입력 도형을 중복된 포인트들을 제거한 상태로 반환합니다.

Synopsis

geometry **ST_SharedPaths**(geometry *lineal1*, geometry *lineal2*);

설명: 입력 도형을 중복된 포인트들을 제거한 상태로 반환합니다. 실제로는 [멀티]포인트, [멀티]라인, [멀티]폴리곤과만 작동하지만, 어떤 종류의 도형도 입력할 수 있다고 해도 과언은 아닙니다. 객체별 기반으로 단순화 작업을 하기 때문에 이 함수에 도형 집합도 입력할 수 있습니다.

허용 오차 파라미터를 설정할 경우, 서로 허용 오차 거리 안에 있는 꼭짓점들을 "동일"하다고 간주, 제거할 것입니다.

2.2.0 버전부터 사용할 수 있습니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

참고: ST_Simplify

8.11.25. ST_SharedPaths

ST_SharedPaths — 두 입력 라인스트링/멀티라인스트링이 공유하는 경로를 담고 있는 집합을 반환합니다.

Synopsis

geometry **ST_SharedPaths**(geometry *lineal1*, geometry *lineal2*);

설명: 두 입력 도형이 공유하는 경로를 담고 있는 집합을 반환합니다. 동일한 방향을 향하는 경로가 이 집합의 첫 번째 요소이며, 반대 방향을 향하는 경로가 두 번째 요소입니다. 경로 그 자체는 첫 번째 도형의 방향을 따릅니다.

2.0.0 버전부터 사용할 수 있습니다. GEOS 3.3.0 이상 버전이 필요합니다.

예시: 공유된 경로 찾기



멀티라인스트링과 라인스트링



멀티라인스트링과 라인스트링이 공유하는 경로와 원본 도형들을 중첩

```
SELECT ST_AsText(
 ST_SharedPaths(
  ST_GeomFromText('MULTILINESTRING((26 125,26 200,126 200,126 125,26 125),
      (51 150,101 150,76 175,51 150))'),
    ST_GeomFromText('LINESTRING(151 100,126 156.25,126 125,90 161, 76 175)')
    )
 ) As wkt

                  wkt
-------------------------------------------------------------
GEOMETRYCOLLECTION(MULTILINESTRING((126 156.25,126 125),
 (101 150,90 161),(90 161,76 175)),MULTILINESTRING EMPTY)
```

```
-- 동일한 예시에서 라인스트링의 방향만 역전
SELECT ST_AsText(
 ST_SharedPaths(
  ST_GeomFromText('LINESTRING(76 175,90 161,126 125,126 156.25,151 100)'),
  ST_GeomFromText('MULTILINESTRING((26 125,26 200,126 200,126 125,26 125),
      (51 150,101 150,76 175,51 150))')
    )
 ) As wkt

                  wkt
-------------------------------------------------------------
GEOMETRYCOLLECTION(MULTILINESTRING EMPTY,
MULTILINESTRING((76 175,90 161),(90 161,101 150),(126 125,126 156.25)))
```

참고: ST_Dump, ST_GeometryN, ST_NumGeometries

8.11.26. ST_ShiftLongitude

ST_ShiftLongitude — 도형의 좌표를 -180 ~ 180 범위와 0 ~ 360 범위 간에 전환합니다.

Synopsis

geometry **ST_ShiftLongitude**(geometry *geomA*);

설명: 도형 안에 있는 모든 피처의 모든 구성 요소 안의 모든 포인트/꼭짓점을 읽어 들여, 경도 좌표가 0 미만일 경우 360을 더합니다. 그 결과 0에서 360 사이의 값을 가지는 데이터를 180을 중심으로 하는 맵에 그리게 됩니다.



이 함수는 SRID:4326(WGS84 경위도) 같은 경위도 데이터에 대해서만 쓸모가 있습니다.



1.3.4 미만 버전에서는 버그 때문에 멀티포인트를 입력받지 못 했습니다. 1.3.4 버전부터 멀티포인트도 입력할 수 있습니다.

✓ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

개선 사항: 2.0.0 버전부터 다면체 표면(polyhedral surface) 및 TIN 을 지원합니다.

주의: 2.2.0 미만 버전에서, 이 함수의 명칭은 "ST_Shift_Longitude"였습니다.

✓ 이 함수는 다면체 표면을 지원합니다.

✓ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

예시

```
-- 3D 포인트
SELECT ST_AsEWKT(ST_ShiftLongitude(ST_GeomFromEWKT('SRID=4326;POINT(-118.58 38.38 10)'))) As
geomA,
    ST_AsEWKT(ST_ShiftLongitude(ST_GeomFromEWKT('SRID=4326;POINT(241.42  38.38  10)')))  As
geomb
geomA                                geomB
----------                           -----------
SRID=4326;POINT(241.42 38.38 10) SRID=4326;POINT(-118.58 38.38 10)

-- 정규 라인스트링
SELECT   ST_AsText(ST_ShiftLongitude(ST_GeomFromText('LINESTRING(-118.58   38.38,   -118.20
38.45)')))

st_astext
----------
LINESTRING(241.42 38.38,241.8 38.45)
```

참고: ST_WrapX

8.11.27. ST_WrapX

ST_WrapX — X 값 근처에서 도형을 래핑합니다.

Synopsis

geometry **ST_WrapX**(geometry *geom*, float8 *wrap*, float8 *move*);

설명: 이 함수는 입력 도형을 쪼갠 다음 주어진 '래핑(wrap)' 라인을 기준으로 (move 가 음일 경우) 오른쪽 또는 (move 가 양일 경우) 왼쪽에 떨어지는 모든 산출 구성 요소를 'move' 파라미터가 지정하는 방향으로 이동시키고, 마지막으로 조각들을 다시 통합시킵니다.



이 함수는 경위도 입력물이 한 편에서 다른 편으로 걸쳐 있지 않은 관심 피처를 갖도록 "다시 중앙으로 위치"시키는 데 유용합니다.

2.3.0 버전부터 사용할 수 있습니다.



이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
-- x=0~360 범위의 왼쪽에 경계 상자가 완전히 떨어지는
-- 입력 도형의 모든 구성 요소를 이동시킵니다.
select ST_WrapX(the_geom, 0, 360);

-- x=-30~360 범위의 왼쪽에 경계 상자가 완전히 떨어지는
-- 입력 도형의 모든 구성 요소를 이동시킵니다.
select ST_WrapX(the_geom, -30, 360);
```

참고: ST_ShiftLongitude

8.11.28. ST_Simplify

ST_Simplify — 더글러스-패커(Douglas-Peucker) 알고리즘을 이용해서 입력 도형의 "단순화"된 버전을 반환합니다.

Synopsis

geometry **ST_Simplify**(geometry *geomA*, float *tolerance*, boolean *preserveCollapsed*);

설명: 더글러스-패커(Douglas-Peucker) 알고리즘을 이용해서 입력 도형의 "단순화"된 버전을 반환합니다. 실제로는 [멀티]라인, [멀티]폴리곤과만 작동하지만, 어떤 종류의 도형도 입력할 수 있다고 해도 과언은 아닙니다. 객체별 기반으로 단순화 작업을 하기 때문에 이 함수에 도형 집합도 입력할 수 있습니다.

"preserve collapsed" 플래그를 설정하면 주어진 허용 오차에 비해 너무 작아 제거될 객체를 유지할 것입니다. 예를 들어, 10 미터 허용 오차를 가지고 1 미터 길이의 라인을 단순화하는 경우 말입니다. "preserve collapsed" 플래그를 설정할 경우, 이 라인이 사라지지 않을 겁니다. 이 플래그는 렌더링 엔진에

유용합니다. 수많은 작은 객체들이 맵에서 한꺼번에 사라져 놀랄 만한 자국들이 생기는 일을 막을 수 있으니까요.



반환되는 도형이 단순성을 잃을 수도 있다는 점에 주의하십시오(ST_IsSimple 참조).



위상(topology)이 보전되지 않아 유효하지 않은 도형이 반환될 수도 있습니다. 위상을 유지하려면 ST_SimplifyPreserveTopology 함수를 이용하십시오.

1.2.2 버전부터 사용할 수 있습니다.

예시

원을 너무 단순화시키면 팔각형을 거쳐 삼각형이 되고 맙니다.

```
SELECT ST_Npoints(the_geom) As np_before, ST_NPoints(ST_Simplify(the_geom,0.1)) As
np01_notbadcircle, ST_NPoints(ST_Simplify(the_geom,0.5)) As np05_notquitecircle,
ST_NPoints(ST_Simplify(the_geom,1)) As np1_octagon, ST_NPoints(ST_Simplify(the_geom,10)) As
np10_triangle,
(ST_Simplify(the_geom,100) is null) As np100_geometrygoesaway
FROM (SELECT ST_Buffer('POINT(1 3)', 10,12) As the_geom) As foo;
-- 결과 --
 np_before | np01_notbadcircle | np05_notquitecircle | np1_octagon | np10_triangle | np100_geometrygoesaway
-----------+-------------------+---------------------+-------------+---------------+------------------------
        49 |          33       |        17           |      9      |      4        |         t
```

참고: ST_IsSimple, ST_SimplifyPreserveTopology, 위상(topology) ST_Simplify

8.11.29. ST_SimplifyPreserveTopology

ST_SimplifyPreserveTopology — 더글러스-패커(Douglas-Peucker) 알고리즘을 이용해서 입력 도형의 "단순화"된 버전을 반환합니다. 유효하지 않은 파생 도형(특히 폴리곤)을 생성하지 않을 것입니다.

Synopsis

geometry **ST_SimplifyPreserveTopology**(geometry *geomA*, float *tolerance*);

설명: 더글러스-패커(Douglas-Peucker) 알고리즘을 이용해서 입력 도형의 "단순화"된 버전을 반환합니다. 유효하지 않은 파생 도형(특히 폴리곤)을 생성하지 않을 것입니다. 실제로는 [멀티]라인, [멀티]폴리곤과만 작동하지만, 어떤 종류의 도형도 입력할 수 있다고 해도 과언은 아닙니다. 객체별 기반으로 단순화 작업을 하기 때문에 이 함수에 도형 집합도 입력할 수 있습니다.

GEOS 모듈로 실행



GEOS 3.0.0 이상 버전이 필요합니다.

1.3.3 버전부터 사용할 수 있습니다.

예시

ST_Simplify 함수와 동일하지만, ST_SimplifyPreserveTopology 는 과단순화(oversimplification)를 막아준다는 사실을 알 수 있습니다. 원을 단순화해도 기껏해야 사각형에서 끝납니다.

```
SELECT                    ST_Npoints(the_geom)                    As                  np_before,
ST_NPoints(ST_SimplifyPreserveTopology(the_geom,0.1))          As          np01_notbadcircle,
ST_NPoints(ST_SimplifyPreserveTopology(the_geom,0.5)) As np05_notquitecircle,
ST_NPoints(ST_SimplifyPreserveTopology(the_geom,1))              As              np1_octagon,
ST_NPoints(ST_SimplifyPreserveTopology(the_geom,10)) As np10_square,
ST_NPoints(ST_SimplifyPreserveTopology(the_geom,100)) As np100_stillsquare
FROM (SELECT ST_Buffer('POINT(1 3)', 10,12) As the_geom) As foo;

-- 결과 --
 np_before | np01_notbadcircle | np05_notquitecircle | np1_octagon | np10_square | np100_stillsquare
-----------+-------------------+---------------------+-------------+-------------+-------------------
        49 |           33      |        17           |      9      |      5      |        5
```

참고: ST_Simplify

8.11.30. ST_SimplifyVW

ST_SimplifyVW — 비스베일링검-와이어트(Visvalingam-Whyatt) 알고리즘을 이용해서 입력 도형의 "단순화"된 버전을 반환합니다.

Synopsis

geometry **ST_SimplifyVW**(geometry *geomA*, float *tolerance*);

설명: 비스베일링검-와이어트(Visvalingam-Whyatt) 알고리즘을 이용해서 입력 도형의 "단순화"된 버전을 반환합니다. 실제로는 [멀티]라인, [멀티]폴리곤과만 작동하지만, 어떤 종류의 도형도 입력할 수 있다고 해도 과언은 아닙니다. 객체별 기반으로 단순화 작업을 하기 때문에 이 함수에 도형 집합도 입력할 수 있습니다.



반환되는 도형이 단순성을 잃을 수도 있다는 점에 주의하십시오(ST_IsSimple 참조).



위상(topology)이 보전되지 않아 유효하지 않은 도형이 반환될 수도 있습니다. 위상을 유지하려면 ST_SimplifyPreserveTopology 함수를 이용하십시오.



이 함수는 3 차원을 처리하며, 세 번째 차원이 결과에 영향을 미칠 것입니다.

2.2.0 버전부터 사용할 수 있습니다.

예시

라인스트링을 최소 임계 면적(minimum area threshold) 30 으로 단순화합니다.

```
select ST_AsText(ST_SimplifyVW(geom,30)) simplified
FROM (SELECT 'LINESTRING(5 2, 3 8, 6 20, 7 25, 10 10)'::geometry geom) As foo;
-- 결과--
 simplified
-----------+------------------+
LINESTRING(5 2,7 25,10 10)
```

참고: ST_SetEffectiveArea, ST_Simplify, ST_SimplifyPreserveTopology, 위상(topology) ST_Simplify

8.11.31. ST_SetEffectiveArea

ST_SetEffectiveArea — M 세로 좌표(ordinate)에 값을 저장해서 각 꼭짓점에 대한 유효 범위(effective area)를 설정합니다. 그 다음 M 세로 좌표에 대해 필터링해서 단순화시킨 도형을 생성할 수 있습니다.

Synopsis

geometry **ST_SetEffectiveArea**(geometry *geomA*, float *threshold = 0*, integer *set_area = 1*);

설명: 비스베일링검-와이어트 알고리즘을 이용해서 각 꼭짓점에 대한 유효 범위를 설정합니다. 유효 범위는 꼭짓점의 M 값으로 저장됩니다. 선택적인 "임계" 파라미터를 설정할 경우, 임계치 이상의 유효 범위를 가진 꼭짓점만을 담고 있는 단순화된 도형을 반환할 것입니다.

임계치를 설정할 경우 이 함수를 서버측 단순화 작업에 이용할 수 있습니다. 또다른 옵션은 임계치를 0 으로 설정하는 것입니다. 이럴 경우, 유효 범위를 M 값으로 가진 전체 도형을 반환하는데, 이 도형을 클라이언트측에서 매우 빨리 단순화하는 데 쓸 수 있습니다.

실제로는 [멀티]라인, [멀티]폴리곤과만 작동하지만, 어떤 종류의 도형도 입력할 수 있다고 해도 과언은 아닙니다. 객체별 기반으로 단순화 작업을 하기 때문에 이 함수에 도형 집합도 입력할 수 있습니다.



반환되는 도형이 단순성을 잃을 수도 있다는 점에 주의하십시오(ST_IsSimple 참조).



위상(topology)이 보전되지 않아 유효하지 않은 도형이 반환될 수도 있습니다. 위상을 유지하려면 ST_SimplifyPreserveTopology 함수를 이용하십시오.



출력 도형은 M 값으로 가지고 있던 정보를 모두 잃게 될 것입니다.



이 함수는 3 차원을 처리하며, 세 번째 차원이 유효 범위에 영향을 미칠 것입니다.

2.2.0 버전부터 사용할 수 있습니다.

예시

라인스트링의 유효 범위를 계산합니다. 임계치를 0 으로 설정하기 때문에, 입력 도형 안에 있는 모든 꼭짓점을 반환합니다.

```
select                    ST_AsText(ST_SetEffectiveArea(geom))                    all_pts,
ST_AsText(ST_SetEffectiveArea(geom,30) ) thrshld_30
FROM (SELECT 'LINESTRING(5 2, 3 8, 6 20, 7 25, 10 10)'::geometry geom) As foo;
-- 결과 --
 all_pts | thrshld_30
-----------+-------------------+
LINESTRING M (5 2 3.40282346638529e+38,3 8 29,6 20 1.5,7 25 49.5,10 10 3.40282346638529e+38)
| LINESTRING M (5 2 3.40282346638529e+38,7 25 49.5,10 10 3.40282346638529e+38)
```

참고: ST_SimplifyVW

8.11.32. ST_Split

ST_Split — 도형을 분해(split)해서 나오는 도형들의 집합을 반환합니다.

Synopsis

geometry **ST_Split**(geometry *input*, geometry *blade*);

설명: 이 함수는 [멀티]포인트, [멀티]라인, 또는 [멀티]폴리곤 경계선으로 라인을, 그리고 라인으로 [멀티]폴리곤을 분해합니다. 언제나 도형 집합을 반환합니다.

이 함수를 ST_Union 의 역함수로 생각하십시오. 이론적으로, 반환된 도형의 구성 요소에 ST_Union 을 적용하면 언제나 원본 도형을 출력해야 합니다.

2.0.0 버전부터 사용할 수 있습니다.

변경 사항: 2.2.0 버전부터 [멀티]포인트, [멀티]라인, 또는 [멀티]폴리곤 경계선으로 라인을 분해할 수 있습니다.



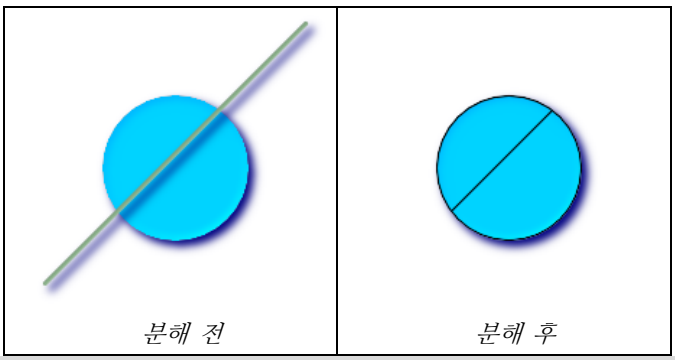
ST_Split 함수의 강력함을 향상시키려면, 매우 낮은 허용 오차를 이용해서 입력 도형을 날(blade)에 ST_Snap 시키는 편이 편리할 수도 있습니다. 그렇지 않을 경우 내부적으로 사용되는 좌표 그리드가 허용 오차 문제를 일으킬 수도 있는데, 입력 도형의 좌표와 날이 서로 일치하지 않아 입력 도형이 정확히 분해되지 않는 문제를 말합니다(#2192 참조).



[멀티]폴리곤을 날로서 입력할 경우, 폴리곤의 선형 구성 요소(경계선)가 입력 도형을 자르는 데 쓰입니다.

예시

라인으로 폴리곤을 분해



분해 전 분해 후

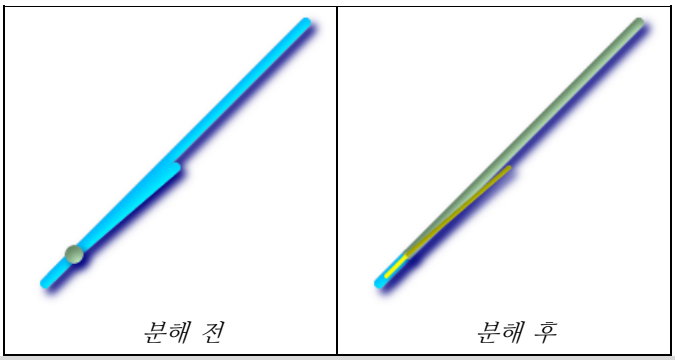
```
-- 이 쿼리는 폴리곤의 절반 2 개로 이루어진 도형 집합을 생성합니다.
-- 이 쿼리는 ST_BuildArea 함수에서 봤던 예시와 유사합니다.
SELECT ST_Split(circle, line)
FROM (SELECT
  ST_MakeLine(ST_MakePoint(10, 10),ST_MakePoint(190, 190)) As line,
  ST_Buffer(ST_GeomFromText('POINT(100 90)'), 50) As circle) As foo;

-- 결과 --
 GEOMETRYCOLLECTION(POLYGON((150    90,149.039264020162    80.2454838991936,146.193976625564
70.8658283817455,..), POLYGON(..)))

-- 개별 폴리곤들로 변환하려면, ST_Dump 또는 ST_GeometryN 함수를 쓰면 됩니다.
SELECT ST_AsText((ST_Dump(ST_Split(circle, line))).geom) As wkt
FROM (SELECT
  ST_MakeLine(ST_MakePoint(10, 10),ST_MakePoint(190, 190)) As line,
  ST_Buffer(ST_GeomFromText('POINT(100 90)'), 50) As circle) As foo;

-- 결과 --
wkt
----------------
POLYGON((150 90,149.039264020162 80.2454838991936,..))
POLYGON((60.1371179574584                                    60.1371179574584,58.4265193848728
62.2214883490198,53.8060233744357 ..))
```

포인트로 멀티라인스트링을 분해



분해 전 분해 후

```
SELECT ST_AsText(ST_Split(mline, pt)) As wktcut
    FROM (SELECT
  ST_GeomFromText('MULTILINESTRING((10 10, 190 190), (15 15, 30 30, 100 90))') As mline,
  ST_Point(30,30) As pt) As foo;

wktcut
------
GEOMETRYCOLLECTION(
  LINESTRING(10 10,30 30),
```

```
  LINESTRING(30 30,190 190),
  LINESTRING(15 15,30 30),
  LINESTRING(30 30,100 90)
)
```

참고: ST_AsText, ST_BuildArea, ST_Dump, ST_GeometryN, ST_Union, ST_Subdivide

8.11.33. ST_SymDifference

ST_SymDifference — 도형 A 와 B 의 서로 교차하지 않는 부분들을 표현하는 도형을 반환합니다. ST_SymDifference(A,B) = ST_SymDifference(B,A)이기 때문에 이를 대칭차(symmetric difference)라고 합니다.

Synopsis

geometry **ST_SymDifference**(geometry *geomA*, geometry *geomB*);

설명: 도형 A 와 B 의 서로 교차하지 않는 부분들을 표현하는 도형을 반환합니다. ST_SymDifference(A,B) = ST_SymDifference(B,A)이기 때문에 이를 대칭차(symmetric difference)라고 합니다. ST_Union(geomA,geomB)에서 ST_Intersection(A,B)를 뺀 것으로 생각할 수도 있습니다.

GEOS 모듈로 실행



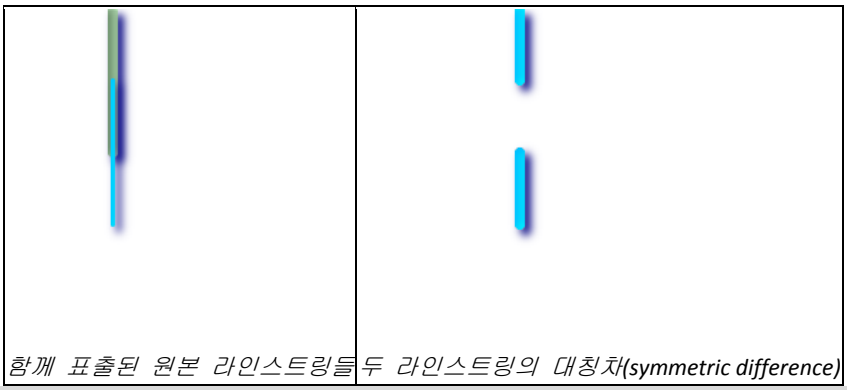
도형 집합을 인수로 호출하지 마십시오.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 만족합니다. s2.1.1.3

✔ 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.21

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다. 하지만 이 함수는 빼기 작업시 XY 만 고려하고 작업후 Z-인덱스를 다시 붙이는 것으로 보입니다.

예시



함께 표출된 원본 라인스트링들 | *두 라인스트링의 대칭차(symmetric difference)*

```
-- 2D 도형에 대해 안전합니다. 두 도형의 대칭차입니다.
SELECT ST_AsText(
    ST_SymDifference(
        ST_GeomFromText('LINESTRING(50 100, 50 200)'),
        ST_GeomFromText('LINESTRING(50 50, 50 150)')
```

```
    )
);

st_astext
---------
MULTILINESTRING((50 150,50 200),(50 50,50 100))
```

```
-- 3 차원에서 이용할 경우 제대로 동작하지 않습니다.
SELECT ST_AsEWKT(ST_SymDifference(ST_GeomFromEWKT('LINESTRING(1 2 1, 1 4 2)'),
    ST_GeomFromEWKT('LINESTRING(1 1 3, 1 3 4)')))

st_astext
------------
MULTILINESTRING((1 3 2.75,1 4 2),(1 1 3,1 2 2.25))
```

참고: ST_Difference, ST_Intersection, ST_Union

8.11.34. ST_Subdivide

ST_Subdivide — 집합 내의 모든 도형이 설정된 개수 이하의 꼭짓점을 가지는 도형 집합을 반환합니다.

Synopsis

setof geometry **ST_Subdivide**(geometry *geom*, integer *max_vertices=256*);

설명: 단일 도형을 최대 허용 개수 이하의 꼭짓점을 가지는 각 구성 요소로 이루어진 집합으로 바꿉니다. 과도하게 대용량인 폴리곤 및 다른 객체들을 데이터베이스의 페이지 용량에 맞는 작은 부분들로 변환하는 데 유용합니다. ST_ClipByBox2D 함수와 동일한 엔벨로프 클리핑을 이용하는데, 모든 부분들이 최대 허용 개수보다 적은 꼭짓점을 가질 때까지 입력 도형을 반복해서 세분화합니다. 최소 꼭짓점 허용 개수는 8 로, 8 보다 적은 수로 설정하려 하면 오류를 일으킬 것입니다.

GEOS 모듈로 클리핑 실행



GEOS 3.5.0 이상 버전이 필요합니다.

2.2.0 버전부터 사용할 수 있습니다. GEOS 3.5.0 이상 버전이 필요합니다.

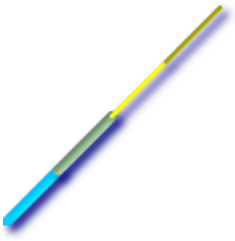
예시



최대 꼭짓점 10 개로 세분화

```
SELECT row_number() OVER() As rn, ST_AsText(geom) As wkt
FROM ( SELECT ST_SubDivide('POLYGON((132 10,119 23,85 35,68 29,66 28,49 42,32 56,22 64,32
110,40 119,36 150,
57 158,75 171,92 182,114 184,132 186,146 178,176 184,179 162,184 141,190 122,
190 100,185 79,186 56,186 52,178 34,168 18,147 13,132 10))'::geometry,10)) As f(geom);
```

```
rn |                wkt
---+-------------------------------------------------------------------------
 1 | POLYGON((22 64,29.3913043478263 98.000000000001,106.000000000001 98.00000000001,
     106.000000000001 27.5882352941173,85 35,68 29,66 28,49 42,32 56,22 64))
 2 | POLYGON((29.3913043478263 98.000000000001,32 110,40 119,36 150,57 158,
     75 11,92 182,106.000000000001 183.272727272727,106.000000000001 98.000000000001,
     29.913043478263 98.000000000001))
 3 | POLYGON((106.000000000001 27.5882352941173,106.000000000001 98.00000000000,
189.52380952381 98.000000000001,185 79,186 56,186 52,178 34,168 18,147 13,
132 0,119 23,106.000000000001 27.5882352941173))
 4 | POLYGON((106.000000000001 98.000000000001,106.000000000001 183.27272727272,
   114 184,132 186,146 178,176 184,179 162,184 141,190 122,190 100,189.5238095238
98.000000000001,106.000000000001 98.000000000001))
```



ST_Segmentize 와 결합해서 이후 분해 작업에 이용할 수 있는 추가 꼭짓점들을 생성하는 데 유용합니다.

```
SELECT ST_AsText(ST_SubDivide(ST_Segmentize('LINESTRING(0 0, 100 100, 150 150)'::geometry,10),8));
LINESTRING(0      0,7.07106781186547      7.07106781186547,14.1421356237309      14.1421356237309,21.2132034355964
21.2132034355964,28.2842712474619      28.2842712474619,35.3553390593274      35.3553390593274,37.499999999998
37.499999999998)
LINESTRING(37.499999999998          37.499999999998,42.4264068711929          42.4264068711929,49.4974746830583
49.4974746830583,56.5685424949238      56.5685424949238,63.6396103067893      63.6396103067893,70.7106781186548
70.7106781186548,74.999999999998 74.999999999998)
LINESTRING(74.999999999998          74.999999999998,77.7817459305202          77.7817459305202,84.8528137423857
84.8528137423857,91.9238815542512  91.9238815542512,98.9949493661167  98.9949493661167,100  100,107.071067811865
107.071067811865,112.499999999998 112.499999999998)
LINESTRING(112.499999999998          112.499999999998,114.142135623731          114.142135623731,121.213203435596
121.213203435596,128.284271247462      128.284271247462,135.355339059327      135.355339059327,142.426406871193
142.426406871193,149.497474683058 149.497474683058,149.999999999998 149.999999999998)
```

참고: ST_AsText, ST_ClipByBox2D, ST_Segmentize, ST_Split

8.11.35. ST_SwapOrdinates

ST_SwapOrdinates — 입력 도형을 좌표값을 뒤바꾼 상태로 반환합니다.

Synopsis

geometry **ST_SwapOrdinates**(geometry *geom*, cstring *ords*);

설명: 입력 도형을 좌표값을 뒤바꾼 상태로 반환합니다.

ords 파라미터는 뒤바꿀 좌표를 명명하는 문자 2 개 길이의 스트링입니다. 유효한 명칭은 x, y, z, 그리고 m 입니다.

2.2.0 버전부터 사용할 수 있습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 함수는 M 좌표를 지원합니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

예시

```
-- M값을 2 배로 만듭니다.
SELECT ST_AsText(
 ST_SwapOrdinates(
  ST_Scale(
   ST_SwapOrdinates(g,'xm'),
   2, 1
  ),
 'xm')
) FROM ( SELECT 'POINT ZM (0 0 0 2)'::geometry g ) foo;
   st_astext
--------------------
 POINT ZM (0 0 0 4)
```

참고: ST_FlipCoordinates

8.11.36. ST_Union

ST_Union — 도형들의 포인트들을 통합한 집합을 표현하는 도형을 반환합니다.

Synopsis

geometry **ST_Union**(geometry set *g1field*);

geometry **ST_Union**(geometry *g1*, geometry *g2*);

geometry **ST_Union**(geometry[] *g1_array*);

설명: 멀티 도형, 단일 도형 또는 도형 집합을 출력할 수 있습니다. 두 가지 응용방법이 존재합니다. 응용방법 1 은 도형 2 개를 통합해서 교차 지역이 없는 새 도형을 출력합니다. 응용방법 2 는 도형들의 집합을 입력받아 교차 지역이 없는 단일 ST_Geometry 로 통합하는 종합 함수입니다.

종합 버전: 이 함수는 도형들의 집합으로부터 멀티 또는 멀티가 아닌 도형을 반환합니다. ST_Union() 함수는 PostgreSQL 전문 용어로 "종합(aggregate)" 함수입니다. 즉 SUM() 및 AVG() 함수와 마찬가지로 데이터 행들을 대상으로 연산한다는 뜻입니다. 또 대부분의 종합 함수와 마찬가지로, NULL 도형을 무시합니다.

비(非)종합 버전: 이 함수는 입력 도형 2 개를 통합한 도형을 반환합니다. 멀티 도형, 멀티가 아닌 도형 또는 도형 집합 유형을 출력할 수 있습니다. 입력 도형 가운데 NULL 도형이 있을 경우, NULL 도형을 반환합니다.



종종 ST_Union 와 ST_Collect 를 서로 바꿔 사용할 수 있습니다. 일반적인 자릿수의 경우 ST_Union 이 ST_Collect 보다 더 느린데, ST_Union 이 작성된 멀티 도형에 교차하는 부분이 없는 것을 입증하려고 도형들을 재정렬하고 경계선을 없애려 하기 때문입니다.

GEOS 모듈로 실행

주의: 이 함수는 이전에 GeomUnion()이라는 명칭이었는데, UNION 이 SQL 이 보유한 용어이기 때문에 "Union"으로 재명명됐습니다.

1.4.0 버전부터 사용할 수 있습니다. ST_Union 함수가 개선됐습니다. 도형 배열 유형을 지원하기 시작했고, PostgreSQL 에서 집합을 더 빨리 종합할 수 있게 됐습니다. GEOS 3.1.0 이상 버전을 사용하고 있다면 ST_Union 함수가 http://blog.cleverelephant.ca/2009/01/must-faster-unions-in-postgis-14.html 에 설명돼 있는 더 빠른 종속 통합(Cascaded Union) 알고리즘을 이용할 것입니다.

✔ 이 방법은 SQL 1.1 버전을 위한 OpenGIS 단순 피처 충족 사양을 도입합니다. s2.1.1.3



종합 버전은 OGC 사양서에 명확하게 정의돼 있지 않습니다.

✔ 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.19 폴리곤이 연관된 경우 Z-인덱스(고도)

예시

쉬운예

```
SELECT stusps,
      ST_Multi(ST_Union(f.the_geom)) as singlegeom
     FROM sometable As f
GROUP BY stusps
```

응용예시

```
SELECT ST_AsText(ST_Union(ST_GeomFromText('POINT(1 2)'),
    ST_GeomFromText('POINT(-2 3)') ) )

st_astext
----------
MULTIPOINT(-2 3,1 2)


SELECT ST_AsText(ST_Union(ST_GeomFromText('POINT(1 2)'),
         ST_GeomFromText('POINT(1 2)') ) );
```

```
st_astext
----------
POINT(1 2)

-- 3D 예시: 3D를 (혼합 차원도!) 지원하는 편입니다.
SELECT ST_AsEWKT(st_union(the_geom))
FROM
(SELECT ST_GeomFromEWKT('POLYGON((-7 4.2,-7.1 4.2,-7.1 4.3,
-7 4.2))') as the_geom
UNION ALL
SELECT ST_GeomFromEWKT('POINT(5 5 5)') as the_geom
UNION ALL
    SELECT ST_GeomFromEWKT('POINT(-2 3 1)') as the_geom
UNION ALL
SELECT ST_GeomFromEWKT('LINESTRING(5 5 5, 10 10 10)') as the_geom ) as foo;

st_asewkt
---------
GEOMETRYCOLLECTION(POINT(-2 3 1),LINESTRING(5 5 5,10 10 10),POLYGON((-7 4.2 5,-7.1 4.2 5,-
7.1 4.3 5,-7 4.2 5)));

-- 3D 예시: 혼합 차원 아님
SELECT ST_AsEWKT(st_union(the_geom))
FROM
(SELECT ST_GeomFromEWKT('POLYGON((-7 4.2 2,-7.1 4.2 3,-7.1 4.3 2,
-7 4.2 2))') as the_geom
UNION ALL
SELECT ST_GeomFromEWKT('POINT(5 5 5)') as the_geom
UNION ALL
    SELECT ST_GeomFromEWKT('POINT(-2 3 1)') as the_geom
UNION ALL
SELECT ST_GeomFromEWKT('LINESTRING(5 5 5, 10 10 10)') as the_geom ) as foo;

st_asewkt
---------
GEOMETRYCOLLECTION(POINT(-2 3 1),LINESTRING(5 5 5,10 10 10),POLYGON((-7 4.2 2,-7.1 4.2 3,-
7.1 4.3 2,-7 4.2 2)))

-- 새로운 배열 구조를 이용한 예시
SELECT ST_Union(ARRAY(SELECT the_geom FROM sometable));

SELECT ST_AsText(ST_Union(ARRAY[ST_GeomFromText('LINESTRING(1 2, 3 4)'),
            ST_GeomFromText('LINESTRING(3 4, 4 5)')])) As wktunion;

--wktunion---
MULTILINESTRING((3 4,4 5),(1 2,3 4))
```

참고: ST_Collect ST_UnaryUnion

8.11.37. ST_UnaryUnion

ST_UnaryUnion — ST_Union 과 비슷하지만, 도형 구성 요소 레벨에서 동작합니다.

Synopsis

geometry **ST_UnaryUnion**(geometry *geom*);

설명: ST_Union 과는 달리, ST_UnaryUnion 은 (유효하지 않은) 멀티폴리곤의 구성 요소들 사이의 경계선을 없애지 않고 도형 집합의 구성 요소들 사이에서 통합 작업을 수행합니다. 입력 도형의 각 구성 요소를 유효하다고 가정하기 때문에, (유효하지 않은) 나비 넥타이 모양의 폴리곤으로부터 유효한 멀티폴리곤을 얻을 수는 없을 겁니다.

라인스트링들의 집합에 노드를 적용하는 데 이 함수를 쓸 수도 있습니다. ST_UnaryUnion 을 ST_Collect 와 결합해서, ST_Union 과 ST_MemUnion 사이의 균형을 맞춰서 메모리 용량 및 CPU 시간 양쪽 모두 만족시키며 한꺼번에 얼마나 많은 도형을 통합할지 상세하게 조정할 수도 있습니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

2.0.0 버전부터 사용할 수 있습니다. GEOS 3.3.0 이상 버전이 필요합니다.

참고: ST_Union, ST_MemUnion, ST_Collect, ST_Node

8.11.38. ST_Voronoi

ST_Voronoi — 도형의 꼭짓점들로부터 보로노이(Voronoi) 다이어그램을 계산합니다.

Synopsis

geometry **ST_Voronoi**(*g1* geometry , *clip* geometry , *tolerance* float8 , *return_polygons* boolean);

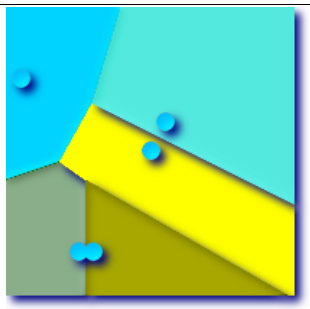
설명: ST_Voronoi 함수는 입력된 도형의 꼭짓점으로부터 2 차원 보로노이 다이어그램 을 계산합니다. 기본적으로, 입력 꼭짓점의 범위보다 더 큰 엔벨로프를 커버하는 폴리곤들의 도형 집합을 반환할 것입니다.

선택할 수 있는 파라미터:

- 'clip' : "clip" 파라미터와 함께 도형을 입력하면, 다이어그램이 "clip" 도형의 엔벨로프를 커버하도록 확장될 것입니다. 해당 엔벨로프가 기본 엔벨로프보다 작지만 않으면 말이죠. (기본값 = NULL)
- 'tolerance' : 그보다 가까운 꼭짓점들을 동일하다고 간주할 허용 오차 거리입니다. 0 이 아닌 허용 오차 거리를 설정하면 알고리즘의 강력함을 향상시킬 수 있습니다. (기본값 = 0.0)
- 'return_polygons' : 참으로 설정하면, ST_Voronoi 가 폴리곤들의 도형 집합을 출력할 것입니다. 거짓으로 설정하면, 멀티라인스트링을 출력할 것입니다. (기본값 = 참)

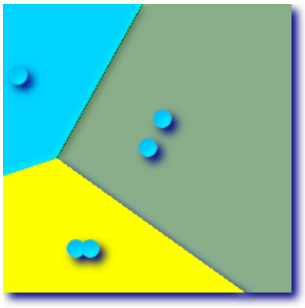
2.3.0 버전부터 사용할 수 있습니다. GEOS 3.5.0 이상 버전이 필요합니다.

예시



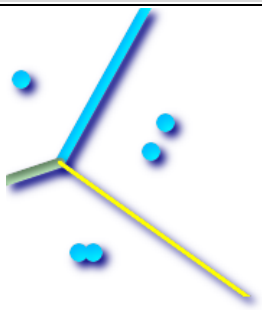
보로노이 다이어그램 위에 중첩한 포인트들

```
SELECT
     ST_Voronoi(geom) As geom
FROM (SELECT 'MULTIPOINT (50 30, 60 30, 100 100,10 150, 110 120)'::geometry As geom ) As g;
-- ST_AsText 출력물
GEOMETRYCOLLECTION(POLYGON((-110 43.3333333333333,-110 270,100.5 270,59.3478260869565
132.826086956522,36.8181818181818 92.2727272727273,-110 43.3333333333333)),
POLYGON((55 -90,-110 -90,-110 43.3333333333333,36.8181818181818 92.2727272727273,55 79.2857142857143,55 -90)),
POLYGON((230 47.5,230 -20.7142857142857,55 79.2857142857143,36.8181818181818 92.2727272727273,59.3478260869565
132.826086956522,230 47.5)),POLYGON((230 -20.7142857142857,230 -90,55 -90,55 79.2857142857143,230 -
20.7142857142857)),
POLYGON((100.5 270,230 270,230 47.5,59.3478260869565 132.826086956522,100.5 270)))
```



*허용 오차 거리가 **30** 단위인 보로노이 다이어그램*

```
SELECT ST_Voronoi(geom, null,30) As geom
FROM (SELECT 'MULTIPOINT (50 30, 60 30, 100 100,10 150, 110 120)'::geometry As geom ) As g;
-- ST_AsText 출력물
GEOMETRYCOLLECTION(POLYGON((-110 43.3333333333333,-110 270,100.5 270,59.3478260869565
132.826086956522,36.8181818181818 92.2727272727273,-110 43.3333333333333)),
POLYGON((230 47.5,230 -45.7142857142858,36.8181818181818 92.2727272727273,59.3478260869565 132.826086956522,230
47.5)),POLYGON((230 -45.7142857142858,230 -90,-110 -90,-110 43.3333333333333,36.8181818181818
92.2727272727273,230 -45.7142857142858)),
POLYGON((100.5 270,230 270,230 47.5,59.3478260869565 132.826086956522,100.5 270)))
```



*허용 오차 거리가 **30** 단위인, 멀티라인스트링 보로노이 다이어그램*

```
SELECT ST_Voronoi(geom, null,30,false) As geom
FROM (SELECT 'MULTIPOINT (50 30, 60 30, 100 100,10 150, 110 120)'::geometry As geom ) As g
-- ST_AsText 출력물
MULTILINESTRING((135.555555555556 270,36.8181818181818 92.2727272727273),(36.8181818181818 92.2727272727273,-110
43.3333333333333),(230 -45.7142857142858,36.8181818181818 92.2727272727273))
```

참고: ST_DelaunayTriangles, ST_Collect

8.12. 선형 참조(Linear Referencing)

ST_LineInterpolatePoint — 라인을 따라 보간된 포인트를 반환합니다. 두 번째 인수는 0 과 1 사이의 Float8 데이터형으로 라인스트링의 전체 길이에서 포인트가 위치해야 하는 비율을 의미합니다.

ST_LineLocatePoint — 입력 포인트에 가장 가까운 위치에 있는 라인스트링 상의 포인트를 나타내는 2 차원 라인 전체 길이의 비율을 0 에서 1 사이의 부동소수점 데이터형(float)으로 반환합니다.

ST_LineSubstring — 입력 라인스트링의 전체 2 차원 길이의 주어진 비율의 위치에서 시작해서 끝나는, 입력 라인스트링의 부분 스트링(substring)인 라인스트링을 반환합니다. 두 번째 및 세 번째 인수는 0 과 1 사이의 8 바이트 부동소수점 데이터형(float8) 값입니다.

ST_LocateAlong — 설정한 척도와 일치하는 요소들을 가진 파생 도형 집합의 값을 반환합니다. 폴리곤 구성 요소는 지원되지 않습니다.

ST_LocateBetween — 설정한 척도의 범위를 계산에 넣어서 일치하는 요소들을 가진 파생 도형 집합의 값을 반환합니다. 폴리곤 구성 요소는 지원되지 않습니다.

ST_LocateBetweenElevations — 설정한 고도의 범위를 계산에 넣어서 교차하는 요소들을 가진 파생 도형(집합)의 값을 반환합니다. 3 차원, 4 차원 라인스트링과 멀티라인스트링만 지원합니다.

ST_InterpolatePoint — 입력 포인트에 가까운 포인트에서 도형의 척도 차원의 값을 반환합니다.

ST_AddMeasure — 시작점과 종단점 사이의 선형적으로 보간된 척도 요소들을 가진 파생 도형을 반환합니다.

8.12.1. ST_LineInterpolatePoint

ST_LineInterpolatePoint — 라인을 따라 보간된 포인트를 반환합니다. 두 번째 인수는 0 과 1 사이의 Float8 데이터형으로 라인스트링의 전체 길이에서 포인트가 위치해야 하는 비율을 의미합니다.

Synopsis

geometry **ST_LineInterpolatePoint**(geometry *a_linestring*, float8 *a_fraction*);

설명: 라인을 따라 보간된 포인트를 반환합니다. 첫 번째 인수는 라인스트링이어야 합니다. 두 번째 인수는 0 과 1 사이의 Float8 데이터형으로 라인스트링의 전체 길이에서 포인트가 위치해야 하는 비율을 의미합니다.

포인트에 가장 가까운 라인의 위치를 계산하는 방법에 대해서는 ST_LineLocatePoint 를 참조하십시오.



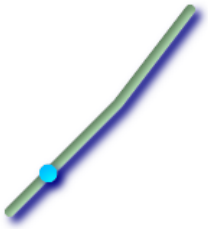
1.1.1 배포판부터 이 함수는 M 및 Z 값(이 있을 경우)도 보간합니다. 이전 배포판에서는 두 값을 0.0 으로 고정시켰습니다.

0.8.2 버전부터 사용할 수 있습니다. 1.1.1 버전에서 Z 과 M 좌표를 지원합니다.

변경 사항: 2.1.0 미만 버전, 즉 2.0.x 버전까지 이 함수의 명칭은 ST_Line_Interpolate_Point 였습니다.

✓ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시



20% 위치 (0.20)에 있는 보간된 포인트를 가진 라인스트링

```
-- 2 차원 라인을 따라 20% 위치에 있는 포인트를 반환합니다.
SELECT ST_AsEWKT(ST_LineInterpolatePoint(the_line, 0.20))
      FROM (SELECT ST_GeomFromEWKT('LINESTRING(25 50, 100 125, 150 190)') as
the_line) As foo;
   st_asewkt
----------------
 POINT(51.5974135047432 76.5974135047432)
```

```
-- 3 차원 라인의 중간에 위치한 포인트를 반환합니다.
SELECT ST_AsEWKT(ST_LineInterpolatePoint(the_line, 0.5))
      FROM (SELECT ST_GeomFromEWKT('LINESTRING(1 2 3, 4 5 6, 6 7 8)') as
the_line) As foo;

      st_asewkt
--------------------
 POINT(3.5 4.5 5.5)


-- 포인트 또는 다른 도형에 가장 가까운 라인 상의 포인트를 찾습니다.
 SELECT                      ST_AsText(ST_LineInterpolatePoint(foo.the_line,
ST_LineLocatePoint(foo.the_line, ST_GeomFromText('POINT(4 3)'))))
FROM (SELECT ST_GeomFromText('LINESTRING(1 2, 4 5, 6 7)') As the_line) As
foo;
   st_astext
----------------
 POINT(3 4)
```

참고: ST_AsText, ST_AsEWKT, ST_Length, ST_LineLocatePoint

8.12.2. ST_LineLocatePoint

ST_LineLocatePoint — 입력 포인트에 가장 가까운 위치에 있는 라인스트링 상의 포인트를 나타내는 2 차원 라인 전체 길이의 비율을 0 에서 1 사이의 부동소수점 데이터형(float)으로 반환합니다.

Synopsis

float8 **ST_LineLocatePoint**(geometry *a_linestring*, geometry *a_point*);

설명: 입력 포인트에 가장 가까운 위치에 있는 라인스트링 상의 포인트를 나타내는 2 차원 라인 전체 길이의 비율을 0 에서 1 사이의 부동소수점 데이터형(float)으로 반환합니다.

반환된 위치를 이용해서 포인트(ST_LineInterpolatePoint) 또는 부분 스트링(ST_LineSubstring)을 추출할 수 있습니다.

이 함수는 주소의 개수의 근사치를 구하는 데 유용합니다.

1.1.0 버전부터 사용할 수 있습니다.

변경 사항: 2.1.0 미만 버전, 즉 2.0.x 버전까지 이 함수의 명칭은 ST_Line_Locate_Point 였습니다.

예시

```
-- 거리를 따라 있는 포인트의 번지수를 대충 어림짐작하기
-- foo 전체가 집 중심점 및 거리처럼 보이는 가짜 데이터를 생성하기 위한 것이라는 점에 주의
-- ST_DWithin 을 이용해서 거리에서 너무 멀리 떨어진 집들을 제외
SELECT ST_AsText(house_loc) As as_text_house_loc,
      startstreet_num +
            CAST( (endstreet_num - startstreet_num)
                  * ST_LineLocatePoint(street_line, house_loc) As integer)
As street_num
FROM
(SELECT ST_GeomFromText('LINESTRING(1 2, 3 4)') As street_line,
      ST_MakePoint(x*1.01,y*1.03) As house_loc, 10 As startstreet_num,
            20 As endstreet_num
FROM generate_series(1,3) x CROSS JOIN generate_series(2,4) As y)
As foo
WHERE ST_DWithin(street_line, house_loc, 0.2);

 as_text_house_loc | street_num
-------------------+------------
 POINT(1.01 2.06)  |       10
 POINT(2.02 3.09)  |       15
 POINT(3.03 4.12)  |       20

-- 포인트 또는 다른 도형에 가장 가까운 라인 상의 포인트를 찾기
 SELECT                      ST_AsText(ST_LineInterpolatePoint(foo.the_line,
ST_LineLocatePoint(foo.the_line, ST_GeomFromText('POINT(4 3)'))))
FROM (SELECT ST_GeomFromText('LINESTRING(1 2, 4 5, 6 7)') As the_line) As
foo;
   st_astext
----------------
 POINT(3 4)
```

참고: ST_DWithin, ST_Length2D, ST_LineInterpolatePoint, ST_LineSubstring

8.12.3. ST_LineSubstring

ST_LineSubstring — 입력 라인스트링의 전체 2 차원 길이의 주어진 비율의 위치에서 시작해서 끝나는, 입력 라인스트링의 부분 스트링(substring)인 라인스트링을 반환합니다. 두 번째 및 세 번째 인수는 0 과 1 사이의 8 바이트 부동소수점 데이터형(float8) 값입니다.

Synopsis

geometry **ST_LineSubstring**(geometry *a_linestring*, float8 *startfraction*, float8 *endfraction*);

설명: 입력 라인스트링의 전체 2 차원 길이의 주어진 비율의 위치에서 시작해서 끝나는, 입력 라인스트링의 부분 스트링(substring)인 라인스트링을 반환합니다. 두 번째 및 세 번째 인수는 0과 1 사이의 8 바이트 부동소수점 데이터형(float8) 값입니다. 이 함수는 라인스트링만 입력받습니다. 근접한 멀티라인스트링을 입력하려면 ST_LineMerge 함수와 결합해서 이용하십시오.

'시작'과 '끝'이 동일한 값일 경우 이 함수는 ST_LineInterpolatePoint 함수와 같아집니다.

포인트에 가장 가까운 라인의 위치를 계산하는 방법에 대해서는 ST_LineLocatePoint 를 참조하십시오.



1.1.1 배포판부터 이 함수는 M 및 Z 값(이 있을 경우)도 보간합니다. 이전 배포판에서는 두 값을 설정하지 않았습니다.

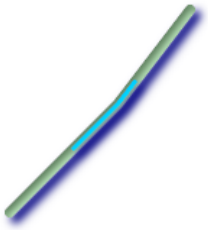
1.1.0 버전부터 사용할 수 있습니다. 1.1.1 버전부터 Z 및 M 을 지원합니다.

변경 사항: 2.1.0 미만 버전, 즉 2.0.x 버전까지 이 함수의 명칭은 ST_Line_Substring 이었습니다.



이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시



중간 1/3 범위(0.333, 0.666)를 중첩해서 출력한 라인스트링

```
-- 라인스트링의 중간 1/3 범위 부분의 근사치를 반환합니다.
SELECT  ST_AsText(ST_Line_SubString(ST_GeomFromText('LINESTRING(25  50,  100
125, 150 190)'), 0.333, 0.666));

                                                                        st_astext
--------------------------------------------------------------------------------
--------------------
LINESTRING(69.2846934853974     94.2846934853974,100     125,111.700356260683
140.210463138888)

-- 다음 쿼리는 PostgreSQL generate_series()를 이용해서
-- 테이블에 있는 모든 라인스트링을 100 단위 구간으로 잘라내는
-- SQL 의 WHILE 루프 모의 예시입니다.
-- 잘라낸 어떤 구간도 100 단위를 넘지 않습니다.
-- SRID 측정 단위로 단위를 측정합니다.
-- 또 모든 도형을 라인스트링 또는 멀티라인스트링으로,
-- 어떤 도형도 100 단위 x10,000 길이를 넘지 않는다고 가정합니다.
-- 성능 향상을 위해 사용자가 기대하는 구간의 최대 개수에 맞춰
-- 10,000 이라는 숫자를 줄일 수 있습니다.
```

```
SELECT field1, field2, ST_LineSubstring(the_geom, 100.00*n/length,
  CASE
       WHEN 100.00*(n+1) < length THEN 100.00*(n+1)/length
       ELSE 1
  END) As the_geom
FROM
  (SELECT sometable.field1, sometable.field2,
  ST_LineMerge(sometable.the_geom) AS the_geom,
  ST_Length(sometable.the_geom) As length
  FROM sometable
  ) AS t
CROSS JOIN generate_series(0,10000) AS n
WHERE n*100.00/length < 1;
```

참고: ST_Length, ST_LineInterpolatePoint, ST_LineMerge

8.12.4. ST_LocateAlong

ST_LocateAlong — 설정한 척도와 일치하는 요소들을 가진 파생 도형 집합의 값을 반환합니다. 폴리곤 구성 요소는 지원되지 않습니다.

Synopsis

geometry **ST_LocateAlong**(geometry *ageom_with_measure*, float8 *a_measure*, float8 *offset*);

설명: 설정한 척도와 일치하는 요소들을 가진 파생 도형 집합의 값을 반환합니다. 폴리곤 구성 요소는 지원되지 않습니다.

오프셋을 설정할 경우, 그 결과 입력 라인에서 설정한 단위 개수만큼 왼쪽 또는 오른쪽에 오프셋 도형 집합을 출력할 것입니다. 양의 오프셋은 왼쪽, 음의 오프셋은 오른쪽으로 출력할 것입니다.

ISO/IEC CD 13249-3:200x(E) - 지속적으로 의미를 결정하기 위하여 편집 회의를 거칠 것입니다..

1.1.0 버전부터 예전 명칭인 ST_Locate_Along_Measure 로 사용할 수 있습니다.

변경 사항: 2.0.0 미만 버전에서는 ST_Locate_Along_Measure 라는 명칭이었습니다. 예전 명칭은 더 지원되지 않고 곧 삭제될 예정이지만, 아직은 사용할 수 있습니다.



M 구성 요소를 가진 도형에 대해서만 이 함수를 이용하십시오.


이 함수는 M 좌표를 지원합니다.

예시

```
SELECT ST_AsText(the_geom)
            FROM
            (SELECT ST_LocateAlong(
                  ST_GeomFromText('MULTILINESTRINGM((1 2 3, 3 4 2, 9 4 3),
            (1 2 3, 5 4 5))'),3) As the_geom) As foo;
```

```
                                        st_asewkt
-----------------------------------------------------------
 MULTIPOINT M (1 2 3)

-- 도형 집합은 다루기 어렵기 때문에,
-- 더 쉽게 이해할 수 있도록 덤프를 받습니다.
SELECT ST_AsText((ST_Dump(the_geom)).geom)
       FROM
       (SELECT ST_LocateAlong(
                     ST_GeomFromText('MULTILINESTRINGM((1 2 3, 3 4 2, 9 4 3),
       (1 2 3, 5 4 5))'),3) As the_geom) As foo;

   st_asewkt
---------------
 POINTM(1 2 3)
 POINTM(9 4 3)
 POINTM(1 2 3)
```

참고: ST_Dump, ST_LocateBetween

8.12.5. ST_LocateBetween

ST_LocateBetween — 설정한 척도의 범위를 계산에 넣어서 일치하는 요소들을 가진 파생 도형 집합의 값을 반환합니다. 폴리곤 구성 요소는 지원되지 않습니다.

Synopsis

geometry **ST_LocateBetween**(geometry *geomA*, float8 *measure_start*, float8 *measure_end*, float8 *offset*);

설명: 설정한 척도의 범위를 계산에 넣어서 일치하는 요소들을 가진 파생 도형 집합의 값을 반환합니다. 폴리곤 구성 요소는 지원되지 않습니다.

ISO/IEC CD 13249-3:200x(E) - 지속적으로 의미를 결정하기 위하여 편집 회의를 거칠 것입니다..

1.1.0 버전부터 예전 명칭인 ST_Locate_Between_Measures 로 사용할 수 있습니다.

변경 사항: 2.0.0 미만 버전에서는 ST_Locate_Between_Measures 라는 명칭이었습니다. 예전 명칭은 더 지원되지 않고 곧 삭제될 예정이지만, 아직은 하위 호환성을 위해 사용할 수 있습니다.

✔ 이 함수는 M 좌표를 지원합니다.

예시

```
SELECT ST_AsText(the_geom)
            FROM
            (SELECT ST_LocateBetween(
                  ST_GeomFromText('MULTILINESTRING M ((1 2 3, 3 4 2, 9 4 3),
            (1 2 3, 5 4 5))'),1.5, 3) As the_geom) As foo;

                                          st_asewkt
-------------------------------------------------------------------------
 GEOMETRYCOLLECTION M (LINESTRING M (1 2 3,3 4 2,9 4 3),POINT M (1 2 3))

-- 도형 집합은 다루기 어렵기 때문에,
```

```
-- 더 쉽게 이해할 수 있도록 덤프를 받습니다.
SELECT ST_AsText((ST_Dump(the_geom)).geom)
            FROM
            (SELECT ST_LocateBetween(
                  ST_GeomFromText('MULTILINESTRING M ((1 2 3, 3 4 2, 9 4 3),
            (1 2 3, 5 4 5))'),1.5, 3) As the_geom) As foo;

              st_asewkt
-------------------------------
 LINESTRING M (1 2 3,3 4 2,9 4 3)
 POINT M (1 2 3)
```

참고: ST_Dump, ST_LocateAlong

8.12.6. ST_LocateBetweenElevations

ST_LocateBetweenElevations — 설정한 고도의 범위를 계산에 넣어서 교차하는 요소들을 가진 파생 도형(집합)의 값을 반환합니다. 3 차원, 4 차원 라인스트링과 멀티라인스트링만 지원합니다.

Synopsis

geometry **ST_LocateBetweenElevations**(geometry *geom_mline*, float8 *elevation_start*, float8 *elevation_end*);

설명: 설정한 고도의 범위를 계산에 넣어서 교차하는 요소들을 가진 파생 도형(집합)의 값을 반환합니다. 3 차원, 4 차원(3DM) 라인스트링과 멀티라인스트링만 지원합니다.

1.4.0 버전부터 사용할 수 있습니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
SELECT ST_AsEWKT(ST_LocateBetweenElevations(
                  ST_GeomFromEWKT('LINESTRING(1  2  3,  4  5  6)'),2,4))  As
ewelev;
                                                        ewelev
-------------------------------------------------------------------
      MULTILINESTRING((1 2 3,2 3 4))

SELECT ST_AsEWKT(ST_LocateBetweenElevations(
                  ST_GeomFromEWKT('LINESTRING(1 2 6, 4 5 -1, 7 8 9)'),6,9))
As ewelev;

                          ewelev
-------------------------------------------------------------------
GEOMETRYCOLLECTION(POINT(1 2 6),LINESTRING(6.1 7.1 6,7 8 9))

-- 도형 집합은 다루기 어렵기 때문에,
-- 더 쉽게 이해할 수 있도록 덤프를 받습니다.
SELECT ST_AsEWKT((ST_Dump(the_geom)).geom)
            FROM
            (SELECT ST_LocateBetweenElevations(
```

```
                    ST_GeomFromEWKT('LINESTRING(1 2 6, 4 5 -1, 7 8 9)'),6,9)
As the_geom) As foo;

                st_asewkt
---------------------------------
POINT(1 2 6)
LINESTRING(6.1 7.1 6,7 8 9)
```

참고: ST_Dump

8.12.7. ST_InterpolatePoint

ST_InterpolatePoint — 입력 포인트에 가까운 포인트에서 도형의 척도 차원의 값을 반환합니다.

Synopsis

float8 **ST_InterpolatePoint**(geometry *line*, geometry *point*);

설명: 입력 포인트에 가까운 포인트에서 도형의 척도 차원(M 차원)의 값을 반환합니다.

2.0.0 버전부터 사용할 수 있습니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
SELECT ST_InterpolatePoint('LINESTRING M (0 0 0, 10 0 20)', 'POINT(5 5)');
 st_interpolatepoint
 ---------------------
                    10
```

참고: ST_AddMeasure, ST_LocateAlong, ST_LocateBetween

8.12.8. ST_AddMeasure

ST_AddMeasure — 시작점과 종단점 사이의 선형적으로 보간된 척도 요소들을 가진 파생 도형을 반환합니다.

Synopsis

geometry **ST_AddMeasure**(geometry *geom_mline*, float8 *measure_start*, float8 *measure_end*);

설명: 시작점과 종단점 사이의 선형적으로 보간된 척도 요소들을 가진 파생 도형을 반환합니다. 도형이 척도 차원을 가지고 있지 않을 경우, 척도 차원을 추가합니다. 도형이 척도 차원을 가지고 있을 경우, 새 값들로 덮어 씁니다. 라인스트링과 멀티라인스트링만 지원합니다.

1.5.0 버전부터 사용할 수 있습니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
SELECT ST_AsText(ST_AddMeasure(
ST_GeomFromEWKT('LINESTRING(1 0, 2 0, 4 0)'),1,4)) As ewelev;
         ewelev
-------------------------------
 LINESTRINGM(1 0 1,2 0 2,4 0 4)

SELECT ST_AsText(ST_AddMeasure(
ST_GeomFromEWKT('LINESTRING(1 0 4, 2 0 4, 4 0 4)'),10,40)) As ewelev;
             ewelev
---------------------------------------
 LINESTRING(1 0 4 10,2 0 4 20,4 0 4 40)

SELECT ST_AsText(ST_AddMeasure(
ST_GeomFromEWKT('LINESTRINGM(1 0 4, 2 0 4, 4 0 4)'),10,40)) As ewelev;
             ewelev
---------------------------------------
 LINESTRINGM(1 0 10,2 0 20,4 0 40)

SELECT ST_AsText(ST_AddMeasure(
ST_GeomFromEWKT('MULTILINESTRINGM((1 0 4, 2 0 4, 4 0 4),(1 0 4, 2 0 4, 4 0
4))'),10,70)) As ewelev;
                        ewelev
-------------------------------------------------------------------
 MULTILINESTRINGM((1 0 10,2 0 20,4 0 40),(1 0 40,2 0 50,4 0 70))
```

8.13. 시계열 지원

ST_IsValidTrajectory — 도형이 유효한 궤적(trajectory)일 경우 true 를 반환합니다.

ST_ClosestPointOfApproach — 두 라인을 따라 보간된 포인트들이 가장 가까운 위치의 척도를 반환합니다.

ST_DistanceCPA — 두 궤적에서 가장 가깝게 접근한 포인트들 사이의 거리를 반환합니다.

ST_CPAWithin — 두 궤적에서 가장 가까이 접근한 포인트들 사이의 거리가 설정한 거리 이하일 경우 참을 반환합니다.

8.13.1. ST_IsValidTrajectory

ST_IsValidTrajectory — 도형이 유효한 궤적(trajectory)일 경우 true 를 반환합니다.

Synopsis

boolean **ST_IsValidTrajectory**(geometry *line*);

설명: 도형이 유효한 궤적을 인코딩하고 있는지 말해줍니다. 유효한 궤적이란 각 꼭짓점에서 다음 꼭짓점으로, 순차적으로 커지는 M 값을 가진 라인스트링으로 인코딩돼 있다는 뜻입니다.

ST_ClosestPointOfApproach 함수 같은 몇몇 시공간계열 쿼리가 유효한 궤적을 입력받을 것입니다.

2.2.0 버전부터 사용할 수 있습니다.

✓ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
-- 유효한 궤적
SELECT ST_IsValidTrajectory(ST_MakeLine(
  ST_MakePointM(0,0,1),
  ST_MakePointM(0,1,2))
);
 t

-- 유효하지 않은 궤적
SELECT                ST_IsValidTrajectory(ST_MakeLine(ST_MakePointM(0,0,1),
ST_MakePointM(0,1,0)));
NOTICE:  Measure of vertex 1 (0) not bigger than measure of vertex 0 (1)
 st_isvalidtrajectory
----------------------
 f
```

참고: ST_ClosestPointOfApproach

8.13.2. ST_ClosestPointOfApproach

ST_ClosestPointOfApproach — 두 라인을 따라 보간된 포인트들이 가장 가까운 위치의 척도를 반환합니다.

Synopsis

float8 **ST_ClosestPointOfApproach**(geometry *track1*, geometry *track2*);

설명: 입력 라인들을 따라 보간된 포인트들이 가장 가까운 거리에 있는 위치의 가장 적은 척도를 반환합니다. ST_IsValidTrajectory 로 확인한 유효한 궤적을 입력해야만 합니다. 궤적들이 M 범위 상에서 중첩하지 않을 경우 NULL 을 반환합니다.

주어진 척도의 위치에 있는 실제 포인트를 얻으려면 ST_LocateAlong 함수를 참조하십시오.

2.2.0 버전부터 사용할 수 있습니다.

✓ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
-- 10 시부터 11 시까지 움직인 두 객체가 서로 가장 가까워진 시간 및
-- 그 시점에서 두 객체의 거리를 반환합니다.
WITH inp AS ( SELECT
  ST_AddMeasure('LINESTRING Z (0 0 0, 10 0 5)'::geometry,
    extract(epoch from '2015-05-26 10:00'::timestamptz),
    extract(epoch from '2015-05-26 11:00'::timestamptz)
  ) a,
  ST_AddMeasure('LINESTRING Z (0 2 10, 12 1 2)'::geometry,
    extract(epoch from '2015-05-26 10:00'::timestamptz),
    extract(epoch from '2015-05-26 11:00'::timestamptz)
  ) b
), cpa AS (
  SELECT ST_ClosestPointOfApproach(a,b) m FROM inp
```

```
), points AS (
  SELECT ST_Force3DZ(ST_GeometryN(ST_LocateAlong(a,m),1)) pa,
       ST_Force3DZ(ST_GeometryN(ST_LocateAlong(b,m),1)) pb
  FROM inp, cpa
)
SELECT to_timestamp(m) t,
     ST_Distance(pa,pb) distance
FROM points, cpa;

            t            |    distance
-----------------------------+------------------
 2015-05-26 10:45:31.034483+02 | 1.96036833151395
```

참고: ST_IsValidTrajectory, ST_DistanceCPA, ST_LocateAlong, ST_AddMeasure

8.13.3. ST_DistanceCPA

ST_DistanceCPA — 두 궤적에서 가장 가깝게 접근한 포인트들 사이의 거리를 반환합니다.

Synopsis

float8 **ST_DistanceCPA**(geometry *track1*, geometry *track2*);

설명: 두 움직이는 객체들이 서로 가장 가까웠을 때의 최소 거리를 반환합니다. ST_IsValidTrajectory 로 확인한 유효한 궤적을 입력해야만 합니다. 궤적들이 M 범위 상에서 중첩하지 않을 경우 NULL 을 반환합니다.

2.2.0 버전부터 사용할 수 있습니다.

✓ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
-- 10 시부터 11 시까지 움직인 두 객체가 서로 가장 가까워진 최소 거리를 반환합니다.
WITH inp AS ( SELECT
  ST_AddMeasure('LINESTRING Z (0 0 0, 10 0 5)'::geometry,
    extract(epoch from '2015-05-26 10:00'::timestamptz),
    extract(epoch from '2015-05-26 11:00'::timestamptz)
  ) a,
  ST_AddMeasure('LINESTRING Z (0 2 10, 12 1 2)'::geometry,
    extract(epoch from '2015-05-26 10:00'::timestamptz),
    extract(epoch from '2015-05-26 11:00'::timestamptz)
  ) b
)
SELECT ST_DistanceCPA(a,b) distance FROM inp;

    distance
------------------
 1.96036833151395
```

참고: ST_IsValidTrajectory, ST_ClosestPointOfApproach, ST_AddMeasure, |=|

8.13.4. ST_CPAWithin

ST_CPAWithin — 두 궤적에서 가장 가까이 접근한 포인트들 사이의 거리가 설정한 거리 이하일 경우 참을 반환합니다.

Synopsis

float8 **ST_CPAWithin**(geometry *track1*, geometry *track2*, float8 *maxdist*);

설명: 움직이는 두 객체가 설정한 최대 거리 이하로 가까워진 적이 있는지 확인합니다.

ST_IsValidTrajectory 로 확인한 유효한 궤적을 입력해야만 합니다. 궤적들이 M 범위 상에서 중첩하지 않을 경우 NULL 을 반환합니다.

2.2.0 버전부터 사용할 수 있습니다.

✓ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
WITH inp AS ( SELECT
  ST_AddMeasure('LINESTRING Z (0 0 0, 10 0 5)'::geometry,
    extract(epoch from '2015-05-26 10:00'::timestamptz),
    extract(epoch from '2015-05-26 11:00'::timestamptz)
  ) a,
  ST_AddMeasure('LINESTRING Z (0 2 10, 12 1 2)'::geometry,
    extract(epoch from '2015-05-26 10:00'::timestamptz),
    extract(epoch from '2015-05-26 11:00'::timestamptz)
  ) b
)
SELECT ST_CPAWithin(a,b,2), ST_DistanceCPA(a,b) distance FROM inp;

 st_cpawithin |    distance
--------------+------------------
 t            | 1.96521473776207
```

참고: ST_IsValidTrajectory, ST_ClosestPointOfApproach, ST_DistanceCPA, |=|

8.14. 장기 트랜잭션(Long Transaction) 지원

AddAuth — 현재 트랜잭션에 이용될 승인 토큰을 추가합니다.

CheckAuth — 승인 토큰을 바탕으로, 테이블에 대해 행들의 업데이트 및 삭제를 금지/허락하는 트리거를 생성합니다.

DisableLongTransactions — 장기 실행 트랜잭션 지원을 비활성화시킵니다. 이 함수는 장기 실행 트랜잭션 지원 메타데이터 테이블을 제거하고, 잠금을 확인한(lock-checked) 테이블에 부여된 모든 트리거를 삭제합니다.

EnableLongTransactions — 장기 실행 트랜잭션 지원을 활성화시킵니다. 이 함수는 필수적인 메타데이터 테이블들을 생성하는데, 이 단원에 있는 다른 함수들을 이용하기 전에 한번 호출해줘야 합니다. 두 번 호출해도 상관없습니다.

LockRow — 테이블 내부의 특정 행에 대한 잠금/승인을 설정합니다.

UnlockRows — 특정 승인 ID 가 실행한 모든 잠금을 제거합니다. 해제된 잠금 개수를 반환합니다.

이 모듈 및 관련 PL/pgSQL 함수들은 웹 피처 서비스(Web Feature Service) 사양서가 요구하는 장기 잠금(long locking) 을 지원하기 위해 구현됐습니다.



사용자는 직렬 가능 트랜잭션 레벨(serializable transaction level) 을 이용해야 합니다. 그렇지 않으면 잠금 기제(locking mechanism)가 중단될 것입니다.

8.14.1. AddAuth

AddAuth — 현재 트랜잭션에 이용될 승인 토큰을 추가합니다.

Synopsis

boolean **AddAuth**(text *auth_token*);

설명: 현재 트랜잭션에 이용될 승인 토큰을 추가합니다.

현재 트랜잭션의 식별자 및 승인 토큰 키를 가진 temp_lock_have_table 이란 명칭의 임시 테이블을 생성/추가합니다.

1.1.3 버전부터 사용할 수 있습니다.

예시

```
SELECT LockRow('towns', '353', 'priscilla');
            BEGIN TRANSACTION;
                  SELECT AddAuth('joey');
                  UPDATE towns SET the_geom = ST_Translate(the_geom,2,2)
WHERE gid = 353;
            COMMIT;


-- 오류 --
            ERROR:   UPDATE where "gid" = '353' requires authorization
'priscilla'
```

참고: LockRow

8.14.2. CheckAuth

CheckAuth — 승인 토큰을 바탕으로, 테이블에 대해 행들의 업데이트 및 삭제를 금지/허락하는 트리거를 생성합니다.

Synopsis

integer **CheckAuth**(text *a_schema_name*, text *a_table_name*, text *a_key_column_name*);

integer **CheckAuth**(text *a_table_name*, text *a_key_column_name*);

설명: 승인 토큰을 바탕으로, 테이블에 대해 행들의 업데이트 및 삭제를 금지/허락하는 트리거를 생성합니다. <rowid_col> 열을 이용해서 행을 식별합니다.

a_schema_name 을 설정하지 않으면, 현재 스키마에서 테이블을 검색합니다.



해당 테이블에 이미 승인 트리거가 존재할 경우, 함수가 오류를 일으킵니다.
트랜잭션 지원이 활성화돼 있지 않을 경우, 함수가 예외를 발생시킵니다.

1.1.3 버전부터 사용할 수 있습니다.

예시

```
SELECT CheckAuth('public', 'towns', 'gid');
                  result
                  ------
                  0
```

참고: EnableLongTransactions

8.14.3. DisableLongTransactions

DisableLongTransactions — 장기 실행 트랜잭션 지원을 비활성화시킵니다. 이 함수는 장기 실행 트랜잭션 지원 메타데이터 테이블을 제거하고, 잠금을 확인한(lock-checked) 테이블에 부여된 모든 트리거를 삭제합니다.

Synopsis

text **DisableLongTransactions**();

설명: 장기 실행 트랜잭션 지원을 비활성화시킵니다. 이 함수는 장기 실행 트랜잭션 지원 메타데이터 테이블을 제거하고, 잠금을 확인한(lock-checked) 테이블에 부여된 모든 트리거를 삭제합니다.

authorization_table 이라는 메타 테이블, authorized_tables 라는 뷰, 그리고 checkauthtrigger 라는 모든 트리거들을 삭제합니다.

1.1.3 버전부터 사용할 수 있습니다.

예시

```
SELECT DisableLongTransactions();
-- 결과 --
Long transactions support disabled
```

참고: EnableLongTransactions

8.14.4. EnableLongTransactions

EnableLongTransactions — 장기 실행 트랜잭션 지원을 활성화시킵니다. 이 함수는 필수적인 메타데이터 테이블들을 생성하는데, 이 단원에 있는 다른 함수들을 이용하기 전에 한번 호출해줘야 합니다. 두 번 호출해도 상관없습니다.

Synopsis

text **EnableLongTransactions**();

설명: 장기 실행 트랜잭션 지원을 활성화시킵니다. 이 함수는 필수적인 메타데이터 테이블들을 생성하는데, 이 단원에 있는 다른 함수들을 이용하기 전에 한번 호출해줘야 합니다. 두 번 호출해도 상관없습니다.

authorization_table 이라는 메타 테이블 및 authorized_tables 라는 뷰를 생성합니다.

1.1.3 버전부터 사용할 수 있습니다.

예시

```
SELECT EnableLongTransactions();
-- 결과 --
Long transactions support enabled
```

참고: DisableLongTransactions

8.14.5. LockRow

LockRow — 테이블 내부의 특정 행에 대한 잠금/승인을 설정합니다.

Synopsis

integer **LockRow**(text *a_schema_name*, text *a_table_name*, text *a_row_key*, text *an_auth_token*, timestamp *expire_dt*);

integer **LockRow**(text *a_table_name*, text *a_row_key*, text *an_auth_token*, timestamp *expire_dt*);

integer **LockRow**(text *a_table_name*, text *a_row_key*, text *an_auth_token*);

설명: 테이블 내부의 특정 행에 대한 잠금/승인을 설정합니다. <authid>는 텍스트 값, <expires>는 기본값이 now()+1 시간인 타임스탬프입니다. 잠금이 할당된 경우 1 을 반환하고, 그렇지 않을 (이미 다른 승인을 받아 잠겨 있는) 경우 0 을 반환합니다.

1.1.3 버전부터 사용할 수 있습니다.

예시

```
SELECT LockRow('public', 'towns', '2', 'joey');
LockRow
-------
1
```

```
-- 조이가 이미 레코드를 잠궜기 때문에 플리실라가 잠글 수 없습니다.
SELECT LockRow('public', 'towns', '2', 'priscilla');
LockRow
-------
0
```

참고: UnlockRows

8.14.6. UnlockRows

UnlockRows — 특정 승인 ID 가 실행한 모든 잠금을 제거합니다. 해제된 잠금 개수를 반환합니다.

Synopsis

integer **UnlockRows**(text *auth_token*);

설명: 특정 승인 ID 가 실행한 모든 잠금을 제거합니다. 해제된 잠금 개수를 반환합니다.

1.1.3 버전부터 사용할 수 있습니다.

예시

```
SELECT LockRow('towns', '353', 'priscilla');
            SELECT LockRow('towns', '2', 'priscilla');
            SELECT UnLockRows('priscilla');
            UnLockRows
            ------------
            2
```

참고: LockRow

8.15. 기타 함수들

ST_Accum — 합지합 함수입니다. 도형들의 배열을 작성합니다.

Box2D — 도형의 최대 범위(maximum extent)를 나타내는 BOX2D 를 반환합니다.

Box3D — 도형의 최대 범위를 나타내는 BOX3D 를 반환합니다.

ST_EstimatedExtent — 주어진 공간 테이블의 '추정' 범위를 반환합니다. 도형 열의 통계로부터 추정치를 추출합니다. 따로 설정하지 않을 경우 현재 스키마를 이용할 것입니다.

ST_Expand — 입력 도형의 경계 상자로부터 모든 방향으로 확장된 경계 상자를 반환합니다. 이중 정밀도 데이터형을 사용합니다.

ST_Extent — 도형 행들의 경계를 이루는 경계 상자를 반환하는 종합 함수입니다.

ST_3DExtent — 도형 행들의 경계를 이루는 BOX3D 경계 상자를 반환하는 종합 함수입니다.

Find_SRID — 이 함수는 find_srid(a_db_schema, a_table, a_column)이라는 문법으로, GEOMETRY_COLUMNS 테이블을 검색해서 지정한 열의 정수형 SRID 를 반환합니다.

ST_MemSize — 도형이 차지하는 (바이트 단위) 공간 용량을 반환합니다.

ST_PointInsideCircle — center_x, center_y, radius 로 정의되는 원 내부에 들어가는 포인트 도형입니다.

8.15.1. ST_Accum

ST_Accum — 합집합 함수입니다. 도형들의 배열을 작성합니다.

Synopsis

geometry[] **ST_Accum**(geometry set *geomfield*);

설명: 합집합 함수입니다. 도형들의 배열을 작성합니다.

개선 사항: 2.0.0 버전부터 다면체 표면, 삼각형 및 TIN 을 지원하기 시작했습니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

예시

```
SELECT (ST_Accum(the_geom)) As all_em, ST_AsText((ST_Accum(the_geom))[1])
As grabone,
(ST_Accum(the_geom))[2:4] as grab_rest
                  FROM    (SELECT    ST_MakePoint(a*CAST(random()*10    As
integer), a*CAST(random()*10 As integer), a*CAST(random()*10 As integer))
As the_geom
                        FROM generate_series(1,4) a) As foo;

all_em|grabone   | grab_rest

------------------------------------------------------------------------
------+

 {0101000080000000000000014400000000000002440000000000001040:
 0101000080000000000
0001840000000000002C40000000000003040:
0101000080000000000000354000000000000038400000000000001840:
0101000080000000000000404000000000000003C400000000000003040} |
 POINT(5                                  10)                              |
{0101000080000000000000184000000000000002C400000000000003040:
 0101000080000000000000354000000000000038400000000000001840:
 0101000080000000000000404000000000000003C400000000000003040}
(1 row)
```

참고: ST_Collect

8.15.2. Box2D

Box2D — 도형의 최대 범위(maximum extent)를 나타내는 BOX2D 를 반환합니다.

Synopsis

box2d **Box2D**(geometry *geomA*);

설명: 도형의 최대 범위(maximum extent)를 나타내는 BOX2D 를 반환합니다.

개선 사항: 2.0.0 버전부터 다면체 표면, 삼각형 및 TIN 을 지원하기 시작했습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

예시

```
SELECT Box2D(ST_GeomFromText('LINESTRING(1 2, 3 4, 5 6)'));
      box2d
      ---------
      BOX(1 2,5 6)

      SELECT  Box2D(ST_GeomFromText('CIRCULARSTRING(220268  150415,220227
150505,220227 150406)'));
      box2d
      --------
      BOX(220186.984375 150406,220288.25 150506.140625)
```

참고: Box3D, ST_GeomFromText

8.15.3. Box3D

Box3D — 도형의 최대 범위를 나타내는 BOX3D 를 반환합니다.

Synopsis

box3d **Box3D**(geometry *geomA*);

설명: 도형의 최대 범위를 나타내는 BOX3D 를 반환합니다.

개선 사항: 2.0.0 버전부터 다면체 표면, 삼각형 및 TIN 을 지원하기 시작했습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
SELECT Box3D(ST_GeomFromEWKT('LINESTRING(1 2 3, 3 4 5, 5 6 5)'));
      Box3d
      ---------
      BOX3D(1 2 3,5 6 5)

      SELECT Box3D(ST_GeomFromEWKT('CIRCULARSTRING(220268 150415 1,220227
150505 1,220227 150406 1)'));
      Box3d
      --------
      BOX3D(220227 150406 1,220268 150415 1)
```

참고: Box2D, ST_GeomFromEWKT

8.15.4. ST_EstimatedExtent

ST_EstimatedExtent — 주어진 공간 테이블의 '추정' 범위를 반환합니다. 도형 열의 통계로부터 추정치를 추출합니다. 따로 설정하지 않을 경우 현재 스키마를 이용할 것입니다.

Synopsis

box2d ST_EstimatedExtent(text schema_name, text table_name, text geocolumn_name, boolean parent_ony);

box2d ST_EstimatedExtent(text schema_name, text table_name, text geocolumn_name);

box2d ST_EstimatedExtent(text table_name, text geocolumn_name);

설명: 주어진 공간 테이블의 '추정' 범위를 반환합니다. 도형 열의 통계로부터 추정치를 추출합니다. 따로 설정하지 않을 경우 현재 스키마를 이용할 것입니다. 자식 테이블(INHERITS 를 가진 테이블)이 있을 경우, 자식 테이블에서 수집한 통계도 이용하는 것이 기본 습성입니다. 'parent_ony' 파라미터를 참으로 설정한 경우, 자식 테이블을 무시하고 주어진 테이블의 통계만 이용합니다.

PostgreSQL 8.0.0 이상 버전의 경우, VACUUM ANALYZE 를 통해 통계를 수집하며 결과 범위는 실제 범위의 약 95%가 될 것입니다.



통계가 없을 경우 (빈 테이블이거나 ANALYZE 를 호출하지 않았을 경우) 이 함수는 NULL 을 반환합니다. 1.5.4 미만 버전까지는 예외를 대신 발생시켰습니다.

PostgreSQL 8.0.0 미만 버전의 경우, update_geometry_stats()를 통해 통계를 수집하며 결과 범위는 실제 범위와 일치할 것입니다.

1.0.0 버전부터 사용할 수 있습니다.

변경 사항: 2.1.0 미만 버전, 즉 2.0.x 버전까지 이 함수의 명칭은 ST_Estimated_Extent 였습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

예시

```
SELECT ST_EstimatedExtent('ny', 'edges', 'the_geom');
-- 결과 --
BOX(-8877653 4912316,-8010225.5 5589284)

SELECT ST_EstimatedExtent('feature_poly', 'the_geom');
-- 결과 --
BOX(-124.659652709961 24.6830825805664,-67.7798080444336 49.0012092590332)
```

참고: ST_Extent

8.15.5. ST_Expand

ST_Expand — 입력 도형의 경계 상자로부터 모든 방향으로 확장된 경계 상자를 반환합니다. 이중 정밀도 데이터형을 사용합니다.

Synopsis

geometry **ST_Expand**(geometry *geom*, float *units_to_expand*);

geometry **ST_Expand**(geometry *geom*, float *dx*, float *dy*, float *dz=0*, float *dm=0*);

box2d **ST_Expand**(box2d *box*, float *units_to_expand*);

box2d **ST_Expand**(box2d *box*, float *dx*, float *dy*);

box3d **ST_Expand**(box3d *box*, float *units_to_expand*);

box3d **ST_Expand**(box3d *box*, float *dx*, float *dy*, float *dz=0*);

설명: 이 함수는 입력 도형의 경계 상자로부터, 설정한 단일 거리만큼 모든 방향으로, 또는 각 방향마다 설정한 확장 거리만큼 확장된 경계 상자를 반환합니다. 이중 정밀도 데이터형을 사용합니다. 거리 쿼리를 실행하는 데 또는 공간 인덱스의 장점을 활용하도록 쿼리에 경계 상자 필터를 추가하는 데 매우 유용합니다.

ST_Expand 함수에는 가장 흔히 쓰이는 도형 버전뿐만 아니라, 내부적인 BOX2D 및 BOX3D 데이터 유형을 받아들이고 생성하는 응용방법들도 있습니다.

ST_Expand 함수는 개념상 ST_Buffer 함수와 유사하지만, ST_Buffer 가 도형을 모든 방향으로 확장하는 반면 ST_Expand 는 경계 상자를 x, y, z 단위만큼 확장합니다.

해당 단위는 SRID 로 표시된, 사용중인 공간 참조 시스템의 단위입니다.



1.3 미만 버전에서, ST_Expand 는 거리 함수와 결합해서 인덱스를 적용할 수 있는 쿼리를 실행하는 데 쓰였습니다. the_geom && ST_Expand('POINT(10 20)', 10) AND ST_Distance(the_geom, 'POINT(10 20)') < 10 와 같은 형식으로 말이죠. 1.2 버전 이후로, 이런 용법은 더 쓰기 쉬운 ST_DWithin 함수 구조로 대체됐습니다.



1.5.0 버전부터 사용할 수 있으며, float4 형 좌표 대신 이중 정밀도 형으로 출력하도록 변경되었습니다.
개선 사항: 2.0.0 버전부터 다면체 표면, 삼각형 및 TIN 을 지원하기 시작했습니다.

개선 사항: 2.3.0 버전부터 서로 다른 차원마다 서로 다른 거리만큼 확장되도록 지원합니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

예시



다음 예시는 미터 단위 투영체인 미국 등적 전도(US National Atlas Equal Area, SRID=2163)를 이용합니다.

```
-- 라인스트링의 경계 상자 주위로 10 미터 확장된 상자
SELECT  CAST(ST_Expand(ST_GeomFromText('LINESTRING(2312980  110676,2312923
110701,2312892 110714)', 2163),10) As box2d);
                                st_expand
-----------------------------------
 BOX(2312882 110666,2312990 110724)

-- 3D 상자의 10 미터 확장된 3D 상자
SELECT ST_Expand(CAST('BOX3D(778783 2951741 1,794875 2970042.61545891 10)'
As box3d),10)
                                                st_expand
-------------------------------------------------------
 BOX3D(778773 2951731 -9,794885 2970052.61545891 20)

 -- 포인트 도형 주위로 10 미터 확장된 도형의 AsText 표현식
 SELECT       ST_AsEWKT(ST_Expand(ST_GeomFromEWKT('SRID=2163;POINT(2312980
110676)'),10));

st_asewkt
------------------------------------------------------------------------
------------------------
 SRID=2163;POLYGON((2312970  110666,2312970  110686,2312990  110686,2312990
110666,2312970 110666))
```

참고: ST_AsEWKT, ST_Buffer, ST_DWithin, ST_GeomFromEWKT, ST_GeomFromText, ST_SRID

8.15.6. ST_Extent

ST_Extent — 도형 행들의 경계를 이루는 경계 상자를 반환하는 종합 함수입니다.

Synopsis

box2d **ST_Extent**(geometry set *geomfield*);

설명: ST_Extent 는 도형들의 집합을 감싸는 경계 상자를 반환합니다. ST_Extent 함수는 SQL 전문 용어로 "종합(aggregate)" 함수입니다. 즉 SUM() 및 AVG() 함수와 마찬가지로 데이터 행들을 대상으로 연산한다는 뜻입니다.

경계 상자를 반환하기 때문에, 해당 단위는 SRID 로 표시된, 사용중인 공간 참조 시스템의 단위입니다.

ST_Extent 는 개념상 Oracle Spatial/Locator 의 SDO_AGGR_MBR 과 유사합니다.



ST_Extent 가 경계 상자를 반환하기 때문에, SRID 메타데이터는 사라집니다. 도형에 SRID 메타데이터를 다시 강제하려면 ST_SetSRID 함수를 이용하십시오. 좌표는 원본 도형들의 공간 참조 단위입니다.



ST_Extent 는 입력 도형들이 (x,y,z) 좌표라 하더라도 x 및 y 요소만 가진 상자를 반환할 것입니다. (x,y,z) 좌표를 유지하려면 ST_3DExtent 를 대신 이용하십시오.



1.4.0 버전부터 사용할 수 있습니다.

개선 사항: 2.0.0 버전부터 다면체 표면, 삼각형 및 TIN 을 지원하기 시작했습니다.

✓ 이 함수는 다면체 표면을 지원합니다.

✓ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

예시



다음 예시는 매사추세츠 주 피트 단위 평면(SRID=2249) 투영체를 씁니다.

```
SELECT ST_Extent(the_geom) as bextent FROM sometable;
                                st_bextent
-----------------------------------
BOX(739651.875 2908247.25,794875.8125 2970042.75)


-- 도형들의 각 카테고리의 범위를 반환합니다.
SELECT ST_Extent(the_geom) as bextent
FROM sometable
GROUP BY category ORDER BY category;

                                  bextent                   |       name
------------------------------------------------------------+----------------
 BOX(778783.5625 2951741.25,794875.8125 2970042.75) | A
 BOX(751315.8125 2919164.75,765202.6875 2935417.25) | B
 BOX(739651.875 2917394.75,756688.375 2935866)     | C

 -- 도형에 SRID 를 다시 강제하고
 -- 해당 도형의 확장 텍스트 표현식을 렌더링합니다.
SELECT ST_SetSRID(ST_Extent(the_geom),2249) as bextent FROM sometable;

                          bextent
------------------------------------------------------------------------------
-------
 SRID=2249;POLYGON((739651.875                          2908247.25,739651.875
2970042.75,794875.8125 2970042.75,
```

```
 794875.8125 2908247.25,739651.875 2908247.25))
```

참고: ST_AsEWKT, ST_3DExtent, ST_SetSRID, ST_SRID

8.15.7. ST_3DExtent

ST_3DExtent — 도형 행들의 경계를 이루는 BOX3D 경계 상자를 반환하는 종합 함수입니다

Synopsis

box3d **ST_3DExtent**(geometry set *geomfield*);

설명: ST_3DExtent 는 도형들의 집합을 감싸는 (Z 좌표를 포함하는) BOX3D 경계 상자를 반환합니다. ST_3DExtent 함수는 SQL 전문 용어로 "종합(aggregate)" 함수입니다. 즉 SUM() 및 AVG() 함수와 마찬가지로 데이터 목록들을 대상으로 연산한다는 뜻입니다.

경계 상자를 반환하기 때문에, 해당 단위는 SRID 로 표시된, 사용중인 공간 참조 시스템의 단위입니다.



T_3DExtent 가 경계 상자를 반환하기 때문에, SRID 메타데이터는 사라집니다. 도형에 SRID 메타데이터를 다시 강제하려면 ST_SetSRID 함수를 이용하십시오. 좌표는 원본 도형들의 공간 참조 단위입니다.

개선 사항: 2.0.0 버전부터 다면체 표면, 삼각형 및 TIN 을 지원하기 시작했습니다.

변경 사항: 2.0.0 미만 버전에서는 ST_Extent3D 라는 명칭이었습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

예시

```
SELECT ST_3DExtent(foo.the_geom) As b3extent
FROM (SELECT ST_MakePoint(x,y,z) As the_geom
      FROM generate_series(1,3) As x
            CROSS JOIN generate_series(1,2) As y
            CROSS JOIN generate_series(0,2) As Z) As foo;
        b3extent
--------------------
 BOX3D(1 1 0,3 2 2)

-- 고도를 높인 원호 스트링 여러 개의 범위를 얻습니다.
SELECT ST_3DExtent(foo.the_geom) As b3extent
FROM                                                                (SELECT
ST_Translate(ST_Force_3DZ(ST_LineToCurve(ST_Buffer(ST_MakePoint(x,y),1))),0
,0,z) As the_geom
      FROM generate_series(1,3) As x
```

```
            CROSS JOIN generate_series(1,2) As y
            CROSS JOIN generate_series(0,2) As Z) As foo;

      b3extent
--------------------
 BOX3D(1 0 0,4 2 2)
```

참고: ST_Extent, ST_Force3DZ

8.15.8. Find_SRID

Find_SRID — 이 함수는 find_srid(a_db_schema, a_table, a_column)이라는 문법으로, GEOMETRY_COLUMNS 테이블을 검색해서 지정한 열의 정수형 SRID 를 반환합니다.

Synopsis

integer **Find_SRID**(varchar *a_schema_name*, varchar *a_table_name*, varchar *a_geomfield_name*);

설명: 이 함수는 find_srid(<db/schema>, <table>, <column>)이라는 문법으로, GEOMETRY_COLUMNS 테이블을 검색해서 지정한 열의 정수형 SRID 를 반환합니다. AddGeometryColumns() 함수를 통해 도형 열을 제대로 추가하지 않은 경우, 이 함수는 제대로 동작하지 않습니다.

예시

```
SELECT Find_SRID('public', 'tiger_us_state_2007', 'the_geom_4269');
find_srid
----------
4269
```

참고: ST_SRID

8.15.9. ST_MemSize

ST_MemSize — 도형이 차지하는 (바이트 단위) 공간 용량을 반환합니다.

Synopsis

integer **ST_MemSize**(geometry *geomA*);

설명: 도형이 차지하는 (바이트 단위) 공간 용량을 반환합니다.

이 함수는 PostgreSQL 내장 함수들인 pg_column_size, pg_size_pretty, pg_relation_size, pg_total_relation_size 를 멋지게 보완합니다.



테이블의 바이트 용량을 출력하는 pg_relation_size 가 ST_MemSize 보다 적은 바이트 용량을 반환할 수도 있습니다. 왜냐하면 pg_relation_size 함수는 TOAST 테이블에 저장된 대용량 도형들 및 TOAST 가 적용된 테이블이 분담하는 용량을 추가하지 않기 때문입니다.
pg_total_relation_size 함수는 테이블, TOAST 가 적용된 테이블 및 인덱스를 포함합니다.

pg_column_size 함수는 어떤 도형이 열에서 차지하게 될 공간 용량을 압축을 고려해서 반환합니다. 따라서 ST_MemSize 의 결과보다 적을 수도 있습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

✔ 이 함수는 다면체 표면을 지원합니다.

✔ 이 함수는 삼각형 및 삼각불규칙망(TIN; Triangulated Irregular Network) 표면을 지원합니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

변경 사항: 2.2.0 버전에서 명명 규약을 따라 명칭을 ST_MemSize 로 변경했습니다. 이전 버전에서 이 함수의 명칭은 ST_Mem_Size 였습니다. 이 예전 명칭은 곧 삭제될 예정이지만, 아직은 사용할 수 있습니다.

예시

```
-- 매사추세츠 주 데이터셋에서 보스턴 시가 얼마나 많은 바이트 용량을 차지할지 반환합니다.
SELECT pg_size_pretty(SUM(ST_MemSize(the_geom))) as totgeomsum,
pg_size_pretty(SUM(CASE WHEN town = 'BOSTON' THEN ST_MemSize(the_geom) ELSE
0 END)) As bossum,
CAST(SUM(CASE WHEN town = 'BOSTON' THEN ST_MemSize(the_geom) ELSE 0 END)*1.00
/
             SUM(ST_MemSize(the_geom))*100 As numeric(10,2)) As perbos
FROM towns;

totgeomsum       bossum       perbos
----------       ------       ------
1522 kB             30 kB       1.99


SELECT    ST_MemSize(ST_GeomFromText('CIRCULARSTRING(220268    150415,220227
150505,220227 150406)'));

---
73

-- 도형이 테이블 용량의 몇 퍼센트를 차지할까요?
SELECT  pg_total_relation_size('public.neighborhoods')  As  fulltable_size,
sum(ST_MemSize(the_geom)) As geomsize,
sum(ST_MemSize(the_geom))*1.00/pg_total_relation_size('public.neighborhoods
')*100 As pergeom
FROM neighborhoods;
fulltable_size geomsize  pergeom
-----------------------------------------------
262144       96238       36.71188354492187500000
```

8.15.10. ST_PointInsideCircle

ST_PointInsideCircle — center_x, center_y, radius 로 정의되는 원 내부에 들어가는 포인트 도형입니다.

Synopsis

boolean **ST_PointInsideCircle**(geometry *a_point*, float *center_x*, float *center_y*, float *radius*);

설명: 이 함수의 문법은 ST_PointInsideCircle(<geometry>,<circle_center_x>,<circle_center_y>,<radius>)입니다. 도형이 포인트이며 원 내부에 있을 경우 참을 반환합니다. 그렇지 않을 경우 거짓을 반환합니다.



함수 명칭대로 포인트 도형만 입력받습니다.

1.2 버전부터 사용할 수 있습니다.

변경 사항: 2.2.0 미만 버전에서는 ST_Point_Inside_Circle 이라는 명칭이었습니다.

예시

```
SELECT ST_PointInsideCircle(ST_Point(1,2), 0.5, 2, 3);
 st_pointinsidecircle
----------------------
 t
```

참고: ST_DWithin

8.16. 예외적인 함수들

PostGIS_AddBBox — 도형에 경계 상자를 추가합니다.

PostGIS_DropBBox — 도형으로부터 경계 상자 캐시를 삭제합니다.

PostGIS_HasBBox — 해당 도형의 경계 상자가 캐시화된 경우 참을 반환하고, 그렇지 않을 경우 거짓을 반환합니다.

이 함수들은 거의 쓰이지 않는 함수들로, 사용자 데이터에 어떤 식으로든 오류가 생겼을 경우에만 이용해야 합니다. 오류를 해결하고, 일반적인 경우라면 절대로 일어나지 않을 일들을 고치는 데 이 함수들을 이용합니다.

8.16.1. PostGIS_AddBBox

PostGIS_AddBBox — 도형에 경계 상자를 추가합니다.

Synopsis

geometry **PostGIS_AddBBox**(geometry *geomA*);

설명: 도형에 경계 상자를 추가합니다. 이렇게 하면 경계 상자 기반 쿼리의 속도가 빨라지지만, 도형의 용량은 늘어날 것입니다.



경계 상자는 도형에 자동적으로 추가되기 때문에 일반적으로 이 함수가 필요하지는 않습니다. 생성된 경계 상자에 어떤 이유로든 오류가 발생했거나, 또는 경계 상자가 없는 구버전 도형을 사용하는 경우, 옛 경계 상자를 삭제하고 다시 추가해야 합니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

예시

```
UPDATE sometable
 SET the_geom =  PostGIS_AddBBox(the_geom)
 WHERE PostGIS_HasBBox(the_geom) = false;
```

참고: PostGIS_DropBBox, PostGIS_HasBBox

8.16.2. PostGIS_DropBBox

PostGIS_DropBBox — 도형으로부터 경계 상자 캐시를 삭제합니다.

Synopsis

geometry **PostGIS_DropBBox**(geometry *geomA*);

설명: 도형으로부터 경계 상자 캐시를 삭제합니다. 이렇게 하면 도형 용량이 줄어들지만, 경계 상자 기반 쿼리는 느려집니다. 오류가 발생한 경계 상자를 삭제하는 데에도 이 함수를 이용합니다. 경계 상자 캐시에 오류가 발생했다는 숨길 수 없는 증거는, 사용자가 ST_Intersects 및 다른 관계성 쿼리를 실행했을 때 당연히 참을 반환해야 할 도형들을 배제하는 경우입니다.



경계 상자는 도형에 자동적으로 추가되어 쿼리의 속도를 향상시키기 때문에 일반적으로는 이 함수가 필요하지 않습니다. 생성된 경계 상자에 어떤 이유로든 오류가 발생했거나, 또는 경계 상자가 없는 구버전 도형을 사용하는 경우, 옛 경계 상자를 삭제하고 다시 추가해야 합니다. 8.3 에서 8.3.6 버전 사이에서, 도형을 변경했을 때 경계 상자 캐시를 항상 재계산하지 않거나 덤프/리로드 작업 없이 새로운 버전으로 업그레이드시 이미 오류가 발생한 경계 상자를 수정하지 못 하는 등 이런 종류의 오류 발생이 관찰됐습니다. 따라서 다음 쿼리를 통해 오류를 직접 수정하고 경계 상자를 다시 추가하거나, 또는 덤프/리로드 작업을 할 수 있습니다.

✔ 이 방법은 원형 스트링 및 곡선을 지원합니다.

예시

```
-- 이 예시는 오류가 발생한 경계 상자 캐시를 삭제합니다.
-- Box2D 호출 전에 ST_AsBinary 함수를 적용하면 경계 상자를 강제로 재계산하게 되고,
-- 테이블 도형 열에 Box2D 를 적용하면 언제나 캐시화된 경계 상자를 반환합니다.
                    UPDATE sometable
 SET the_geom =  PostGIS_DropBBox(the_geom)
 WHERE Not (Box2D(ST_AsBinary(the_geom)) = Box2D(the_geom));

      UPDATE sometable
 SET the_geom =  PostGIS_AddBBox(the_geom)
 WHERE Not PostGIS_HasBBOX(the_geom);
```

참고: PostGIS_AddBBox, PostGIS_HasBBox, Box2D

8.16.3. PostGIS_HasBBox

PostGIS_HasBBox — 해당 도형의 경계 상자가 캐시화된 경우 참을 반환하고, 그렇지 않을 경우 거짓을 반환합니다.

Synopsis

boolean **PostGIS_HasBBox**(geometry *geomA*);

설명: 해당 도형의 경계 상자가 캐시화된 경우 참을 반환하고, 그렇지 않을 경우 거짓을 반환합니다. 캐시 작업을 제어하려면 PostGIS_AddBBox 및 PostGIS_DropBBox 를 이용하십시오.

✓ 이 방법은 원형 스트링 및 곡선을 지원합니다.

예시

```
SELECT the_geom
FROM sometable WHERE PostGIS_HasBBox(the_geom) = false;
```

참고: PostGIS_AddBBox, PostGIS_DropBBox

Chapter 9.래스터 참조 문서

이 단원에서 주어진 함수들은 PostGIS 래스터 사용자가 필요로 하고, 현재 PostGIS 래스터에서 사용할 수 있는 함수들입니다. 일반 사용자에겐 쓸모없는, 래스터 객체에 필요한 다른 지원 함수들도 있습니다.

raster 는 래스터 데이터를 저장하고 분석하기 위한 새로운 PostGIS 유형입니다.

래스터 파일에서 래스터를 로드하려면 Section 5.1, “래스터 로드 및 생성” 를 참조하십시오.

이 참조 문서에 있는 예시들의 경우, 다음 코드로 구성된 가짜 래스터를 담은 래스터 테이블을 이용할 것입니다.

```
CREATE TABLE dummy_rast(rid integer, rast raster);
INSERT INTO dummy_rast(rid, rast)
VALUES (1,
('01' -- little endian (uint8 ndr)
||
'0000' -- version (uint16 0)
||
'0000' -- nBands (uint16 0)
||
'0000000000000040' -- scaleX (float64 2)
||
'0000000000000840' -- scaleY (float64 3)
||
'000000000000E03F' -- ipX (float64 0.5)
||
'000000000000E03F' -- ipY (float64 0.5)
||
'0000000000000000' -- skewX (float64 0)
||
'0000000000000000' -- skewY (float64 0)
||
'00000000' -- SRID (int32 0)
||
'0A00' -- width (uint16 10)
||
'1400' -- height (uint16 20)
)::raster
),
-- 래스터: 5 x 5 픽셀, 밴드 3 개, PT_8BUI 픽셀 유형, NODATA = 0
(2,  ('01000003009A9999999999A93F9A9999999999A9BF000000E02B274A' ||
'41000000007719564100000000000000000000000000000000FFFFFFFF050005000400FD
FEFDFEFEFDFEFEFDF9FAFEF' ||
'EFCF9FBFDFEFEFDFCFAFEFEFE04004E627AADD16076B4F9FE6370A9F5FE59637AB0E54F5
8617087040046566487A1506CA2E3FA5A6CAFFBFE4D566DA4CB3E454C5665')::raster);
```

9.1. 래스터 지원 데이터형

이 장에서는 PostgreSQL 의 지원 하는 래스터 데이터 유형들을 설명 합니다.

geomval — (도형 객체를 담고 있는) geom 과 (래스터 밴드의 이중 정밀도 픽셀값을 담고 있는) val, 두 개의 필드를 가진 공간 데이터형입니다.

addbandarg — 새로운 밴드의 속성 및 초기값을 정의하는 ST_AddBand 함수의 입력물로 이용되는 복합 데이터형입니다.

rastbandarg — 래스터 및 해당 래스터의 밴드 인덱스를 표현해야 할 경우 쓰이는 복합 데이터형입니다.

raster — 래스터 공간 데이터형입니다.

reclassarg — 재분류 작업의 습성을 정의하는 ST_Reclass 함수의 입력물로 쓰이는 복합 데이터형입니다.

summarystats — ST_SummaryStats 및 ST_SummaryStatsAgg 함수가 반환하는 복합 데이터형입니다.

unionarg — 공간 처리할 밴드들 및 UNION 연산의 습성을 정의하는 ST_Union 함수의 입력물로 쓰이는 복합 데이터형입니다.

9.1.1. geomval

geomval — geomval (도형 객체를 담고 있는) geom 과 (래스터 밴드의 이중 정밀도 픽셀값을 담고 있는) val, 두 개의 필드를 가진 공간 데이터형입니다.

설명: geomval 은 복합 데이터 유형으로, .geom 필드가 참조하는 도형 객체와 래스터 밴드 내부의 특정 기하 위치에 있는 픽셀값을 나타내는 이중 정밀도 값인 val 로 구성됩니다. ST_DumpAsPolygon 및 래스터 교차 계열 함수들이 래스터 밴드를 도형 폴리곤들로 분해하기 위한 출력물 유형으로 이 데이터형을 이용합니다.

참고: Section 14.6, “PostGIS Geometry / Geography / Raster Dump Functions”

9.1.2. addbandarg

addbandarg — 새로운 밴드의 속성 및 초기값을 정의하는 ST_AddBand 함수의 입력물로 이용되는 복합 데이터형입니다.

설명: 새로운 밴드의 속성 및 초기값을 정의하는 ST_AddBand 함수의 입력물로 이용되는 복합 데이터형입니다.

index **integer** — 래스터의 밴드들 사이 어떤 위치에 새 밴드를 추가할지 지시하는 1-기반 값입니다. NULL 일 경우, 래스터 밴드의 마지막에 새 밴드를 추가할 것입니다.

pixeltype **text** — ST_BandPixelType 에서 설명하고 있는 픽셀 유형들 가운데 하나입니다.

initialvalue **double precision** — 새 밴드의 모든 픽셀들을 설정하게 될 초기값입니다.

nodataval **double precision** — 새 밴드의 NODATA 값입니다. NULL 일 경우, 새 밴드에 NODATA 값을 할당하지 않을 것입니다.

참고: ST_AddBand

9.1.3. rastbandarg

rastbandarg — 래스터 및 해당 래스터의 밴드 인덱스를 표현해야 할 경우 쓰이는 복합 데이터형입니다.

설명: 래스터 및 해당 래스터의 밴드 인덱스를 표현해야 할 경우 쓰이는 복합 데이터형입니다.

rast **raster** — 해당 래스터를 지정합니다.

nband **integer** — 래스터의 밴드를 지시하는 1-기반 값입니다.

참고: ST_MapAlgebra

9.1.4. raster

raster — 래스터 공간 데이터형입니다.

설명: 래스터는 JPEG, TIFF, PNG, 수치 표고 모델(digital elevation model)에서 임포트된 것과 같은 래스터 데이터를 표현하는 데 쓰이는 공간 데이터 유형입니다. 각 래스터는 각각 픽셀 값 집합을 가지는 밴드들을 1 개 이상 가지고 있습니다. 래스터는 지리참조(georeference)될 수 있습니다.



GDAL 지원과 함께 컴파일한 PostGIS 가 필요합니다. 현재 래스터를 제약 없이 도형 유형으로 변환할 수 있지만, 이 변환 작업은 래스터의 ST_ConvexHull 을 반환합니다. 이 자동 형변환 기능은 가까운 시일 내에 제거될 수 있으므로 의존하지 않는 편이 좋습니다.

형변환 습성(Casting Behavior)

이 단원에서는 이 데이터형에 대해 사용할 수 있는 자동적임은 물론 명시적인 형변환 목록을 제시합니다.

형변환 결과	습성
도형	자동적

참고: Chapter 9, 래스터 참조 문서

9.1.5. reclassarg

reclassarg — 재분류 작업의 습성을 정의하는 ST_Reclass 함수의 입력물로 쓰이는 복합 데이터형입니다.

설명: 재분류 작업의 습성을 정의하는 ST_Reclass 함수의 입력물로 쓰이는 복합 데이터형입니다.

nband **integer** — 재분류할 밴드의 밴드 개수입니다.

reclassexpr **text** — 쉼표로 구분된 range:map_range 매핑으로 이루어진 범위 표현식입니다. ':'가 예전 밴드값을 새 밴드값으로 어떻게 매핑할 것인지를 정의하는 매핑을 정의합니다. '('는 '>'를, ')'는 미만을, ']'는 '<' 또는 동일을, '['는 '>' 또는 동일을 뜻합니다.

```
1. [a-b] = a <= x <= b

2. (a-b] = a < x <= b

3. [a-b) = a <= x < b

4. (a-b) = a < x < b
```

'(' 기호는 선택적이기 때문에 (a-b)와 a-b 의 의미는 동일합니다.

pixeltype **text** — ST_BandPixelType 에서 설명하고 있는 픽셀 유형들 가운데 하나입니다.

nodataval **double precision** — NODATA 로 취급되는 값입니다. 투명도를 지원하는 이미지 출력물의 경우, 이 값이 투명이 될 것입니다.

예: 밴드 2 를 255 가 NODATA 값인 8BUI 로 재분류하십시오.

```
SELECT ROW(2, '0-100:1-10, 101-500:11-150,501 - 10000: 151-254', '8BUI',
255)::reclassarg;
```

예: 밴드 1 을 NODATA 값이 정의되지 않은 1BB 로 재분류하십시오.

```
SELECT ROW(1, '0-100]:0, (100-255:1', '1BB', NULL)::reclassarg;
```

참고: ST_Reclass

9.1.6. summarystats

summarystats — ST_SummaryStats 및 ST_SummaryStatsAgg 함수가 반환하는 복합 데이터형입니다.

설명: ST_SummaryStats 및 ST_SummaryStatsAgg 함수가 반환하는 복합 데이터형입니다.

count **integer** — 요약 통계를 위해 집계된 픽셀 개수입니다.

sum **double precision** — 집계된 모든 픽셀 값의 합입니다.

mean **double precision** — 집계된 모든 픽셀 값의 산술 평균입니다.

stddev **double precision** — 집계된 모든 픽셀 값의 표준 편차입니다.

min **double precision** — 집계된 픽셀 값 가운데 최소값입니다.

max **double precision** — 집계된 픽셀 값 가운데 최대값입니다.

참고: ST_SummaryStats, ST_SummaryStatsAgg

9.1.7. unionarg

unionarg — 공간 처리할 밴드들 및 UNION 연산의 습성을 정의하는 ST_Union 함수의 입력물로 쓰이는 복합 데이터형입니다.

설명: 공간 처리할 밴드들 및 UNION 연산의 습성을 정의하는 ST_Union 함수의 입력물로 쓰이는 복합 데이터형입니다.

nband **integer** — 공간 처리할 각 입력 래스터의 밴드를 지시하는 1-기반 값입니다.

uniontype **text** — UNION 연산의 유형입니다. ST_Union 에서 설명하고 있는 유형들 가운데 하나입니다.

참고: ST_Union

9.2. 래스터 데이터 관리

AddRasterConstraints — 로드된 래스터 테이블에서 공간 참조 시스템, 축척, 블록 크기(blocksize), 정렬 방식, 밴드, 밴드 유형, 그리고 래스터 열이 정규적으로 블록화되어 있는지 표시하는 플래그를 제약하는 조건을 담고 있는 특정 열에 래스터 제약조건을 추가합니다. 해당 테이블에는 제약조건을 적용할 대상인 데이터가 로드돼 있어야만 합니다. 제약조건 설정이 달성될 경우 참을 반환하며, 문제가 있을 경우 안내문을 출력합니다.

DropRasterConstraints — 래스터 테이블 열을 참조하는 PostGIS 래스터 제약조건을 삭제합니다. 데이터를 다시 로드하거나 사용자 래스터 열 데이터를 업데이트해야 할 경우 유용합니다.

AddOverviewConstraints — 래스터 열을 또 다른 래스터 열의 미리보기(overview)로 태그합니다.

DropOverviewConstraints — 또 다른 래스터 열의 미리보기(overview)로서의 래스터 열을 태그 해제합니다.

PostGIS_GDAL_Version — PostGIS 가 이용하고 있는 GDAL 라이브러리의 버전을 반환합니다.

PostGIS_Raster_Lib_Build_Date — 전체 래스터 라이브러리의 빌드 날짜를 반환합니다.

PostGIS_Raster_Lib_Version — 전체 래스터 버전 및 설정 정보를 반환합니다.

ST_GDALDrivers — 사용자 GDAL 라이브러리가 지원하는 래스터 형식들의 목록을 반환합니다. 사용자가 ST_AsGDALRaster 함수를 통해 사용자 래스터를 출력할 때 쓸 수 있는 형식들입니다.

UpdateRasterSRID — 사용자가 지정한 열 및 테이블에 있는 모든 래스터의 SRID 를 변경합니다.

ST_CreateOverview — 입력 래스터 커버리지의 저해상도 버전을 생성합니다.

9.2.1. AddRasterConstraints

AddRasterConstraints — 로드된 래스터 테이블에서 공간 참조 시스템, 축척, 블록 크기(blocksize), 정렬 방식, 밴드, 밴드 유형, 그리고 래스터 열이 정규적으로 블록화되어 있는지 표시하는 플래그를 제약하는 조건을 담고 있는 특정 열에 래스터 제약조건을 추가합니다. 해당 테이블에는 제약조건을 적용할 대상인

데이터가 로드돼 있어야만 합니다. 제약조건 설정이 달성될 경우 참을 반환하며, 문제가 있을 경우 안내문을 출력합니다.

Synopsis

boolean AddRasterConstraints(name *rasttable*, name *rastcolumn*, boolean *srid*, boolean *scale_x*, boolean *scale_y*, boolean *blocksize_x*, boolean *blocksize_y*, boolean *same_alignment*, boolean *regular_blocking*, boolean *num_bands=true* , boolean *pixel_types=true* , boolean *nodata_values=true* , boolean *out_db=true* , boolean *extent=true*);

boolean AddRasterConstraints(name *rasttable*, name *rastcolumn*, text[] *VARIADIC constraints*);

boolean AddRasterConstraints(name *rastschema*, name *rasttable*, name *rastcolumn*, text[] *VARIADIC constraints*);

boolean AddRasterConstraints(name *rastschema*, name *rasttable*, name *rastcolumn*, boolean *srid=true*, boolean *scale_x=true*, boolean *scale_y=true*, boolean *blocksize_x=true*, boolean *blocksize_y=true*, boolean *same_alignment=true*, boolean *regular_blocking=false*, boolean *num_bands=true*, boolean *pixel_types=true*, boolean *nodata_values=true* , boolean *out_db=true* , boolean *extent=true*);

설명: raster_columns 래스터 카탈로그에 정보를 표출하는 데 쓰이는, 래스터 열에 대한 제약조건들을 생성합니다. rastschema 는 해당 테이블이 속해 있는 테이블 스키마의 명칭입니다. srid 는 SPATIAL_REF_SYS 테이블에 있는 항목을 참조하는 정수형 값이어야만 합니다.

raster2pgsql 로더는 래스터 테이블을 등록하는 데 이 함수를 이용합니다.

입력할 유효한 제약조건 명칭에 대한 자세한 내용은 Section 5.2.1, “래스터 열 카탈로그” 를 참조하십시오.

- blocksize 는 X 및 Y 블록 크기를 둘 다 설정합니다.
- blocksize_x 는 X 타일(각 타일의 너비 픽셀 개수)을 설정합니다.
- blocksize_y 는 Y 타일(각 타일의 높이 픽셀 개수)을 설정합니다.
- extent 는 전체 테이블의 범위를 계산하고 제약조건을 적용합니다. 모든 래스터는 해당 범위 내에 들어와야만 합니다.
- num_bands 는 밴드의 개수입니다.
- pixel_types 각 밴드에 대해 픽셀 유형의 배열을 읽어 들입니다. 모든 밴드 N 이 동일한 픽셀 유형을 가지도록 보장합니다.
- regular_blocking 은 공간적 유일성 제약조건(어떤 두 래스터도 공간적으로 동일할 수 없습니다) 및 커버리지 타일 제약조건(래스터는 커버리지에 맞춰 정렬됩니다)을 설정합니다.
- same_alignment 는 모든 타일을 동일하게 정렬하도록 보장합니다. 즉 어떤 두 타일을 비교하더라도 참을 반환할 것이라는 뜻입니다. ST_SameAlignment 를 참조하십시오. ST_SameAlignment
- srid 는 모든 타일이 동일한 SRID 를 가지도록 보장합니다.
- 기타 -- 앞의 함수들이 입력받는 제약조건은 무엇이든 추가해주십시오.



이 함수는 테이블에 이미 존재하는 데이터에 제약조건을 적용합니다. 따라서 이 함수가 작동하려면, 먼저 래스터 열을 생성한 다음 데이터를 로드해야만 합니다.



사용자가 제약조건을 적용한 다음 추가 데이터를 로드해야 할 경우, 사용자 데이터의 범위가 변경된다면 DropRasterConstraints 함수를 실행하는 편이 좋습니다.

예시: **데이터에 기반한 열에 가능한 한 모든 제약조건을 적용하기**

```
CREATE TABLE myrasters(rid SERIAL primary key, rast raster);
INSERT INTO myrasters(rast)
SELECT ST_AddBand(ST_MakeEmptyRaster(1000, 1000, 0.3, -0.3, 2, 2, 0, 0,4326), 1,
'8BSI'::text, -129, NULL);

SELECT AddRasterConstraints('myrasters'::name, 'rast'::name);


-- raster_columns 뷰에 정확히 등록됐는지 확인 --
SELECT srid, scale_x, scale_y, blocksize_x, blocksize_y, num_bands, pixel_types,
nodata_values


      FROM raster_columns
      WHERE r_table_name = 'myrasters';

 srid | scale_x | scale_y | blocksize_x | blocksize_y | num_bands | pixel_types| nodata_values
------+---------+---------+-------------+-------------+-----------+-------------+---------------
 4326 |    2    |    2    |    1000     |    1000     |     1     |   {8BSI}    |     {0}
```

예시: 단일 제약 조건을 적용하기

```
CREATE TABLE public.myrasters2(rid SERIAL primary key, rast raster);
INSERT INTO myrasters2(rast)
SELECT ST_AddBand(ST_MakeEmptyRaster(1000, 1000, 0.3, -0.3, 2, 2, 0, 0,4326), 1,
'8BSI'::text, -129, NULL);

SELECT           AddRasterConstraints('public'::name,           'myrasters2'::name,
'rast'::name,'regular_blocking', 'blocksize');
-- 안내문 출력 --
NOTICE:  Adding regular blocking constraint
NOTICE:  Adding blocksize-X constraint
NOTICE:  Adding blocksize-Y constraint
```

참고: Section 5.2.1, “래스터 열 카탈로그”, ST_AddBand, ST_MakeEmptyRaster, DropRasterConstraints, ST_BandPixelType, ST_SRID

9.2.2. AddOverviewConstraints

AddOverviewConstraints — 래스터 열을 또 다른 래스터 열의 미리보기(overview)로 태그합니다.

Synopsis

boolean AddOverviewConstraints(name *ovschema*, name *ovtable*, name *ovcolumn*, name *refschema*, name *reftable*, name *refcolumn*, int *ovfactor*);

boolean AddOverviewConstraints(name *ovtable*, name *ovcolumn*, name *reftable*, name *refcolumn*, int *ovfactor*);

설명: raster_overviews 래스터 카탈로그에 정보를 표출하는 데 쓰이는 래스터 열에 제약조건을 추가합니다.

ovfactor 파라미터는 미리보기 열에 대한 축척 승수(乘數)를 나타냅니다. ovfactor 가 클수록 해상도가 낮아집니다.

ovschema 및 refschema 파라미터를 생략할 경우, search_path 를 스캔하면서 처음 발견한 테이블을 이용할 것입니다.

2.0.0 버전부터 사용할 수 있습니다.

예시

```
CREATE TABLE res1 AS SELECT
ST_AddBand(
  ST_MakeEmptyRaster(1000, 1000, 0, 0, 2),
  1, '8BSI'::text, -129, NULL
) r1;

CREATE TABLE res2 AS SELECT
ST_AddBand(
  ST_MakeEmptyRaster(500, 500, 0, 0, 4),
  1, '8BSI'::text, -129, NULL
) r2;

SELECT AddOverviewConstraints('res2', 'r2', 'res1', 'r1', 2);

-- raster_overviews 뷰에 정확히 등록됐는지 확인 --
SELECT o_table_name ot, o_raster_column oc,
     r_table_name rt, r_raster_column rc,
     overview_factor f
FROM raster_overviews WHERE o_table_name = 'res2';
  ot  | oc |  rt  | rc | f
------+----+------+----+---
 res2 | r2 | res1 | r1 | 2
(1 row)
```

참고: Section 5.2.2, “래스터 오버뷰”, DropOverviewConstraints, ST_CreateOverview, AddRasterConstraints

9.2.3. DropRasterConstraints

DropRasterConstraints — 래스터 테이블 열을 참조하는 PostGIS 래스터 제약조건을 삭제합니다. 데이터를 다시 로드하거나 사용자 래스터 열 데이터를 업데이트해야 할 경우 유용합니다.

Synopsis

boolean DropRasterConstraints(name *rasttable*, name *rastcolumn*, boolean *srid*, boolean *scale_x*, boolean *scale_y*, boolean *blocksize_x*, boolean *blocksize_y*, boolean *same_alignment*, boolean *regular_blocking*, boolean

num_bands=true, boolean *pixel_types=true*, boolean *nodata_values=true*, boolean *out_db=true* , boolean *extent=true*);

boolean DropRasterConstraints(name *rastschema*, name *rasttable*, name *rastcolumn*, boolean *srid=true*, boolean *scale_x=true*, boolean *scale_y=true*, boolean *blocksize_x=true*, boolean *blocksize_y=true*, boolean *same_alignment=true*, boolean *regular_blocking=false*, boolean *num_bands=true*, boolean *pixel_types=true*, boolean *nodata_values=true*, boolean *out_db=true* , boolean *extent=true*);

boolean DropRasterConstraints(name *rastschema*, name *rasttable*, name *rastcolumn*, text[] *constraints*);

설명: AddRasterConstraints 가 추가한, 래스터 테이블 열을 참조하는 PostGIS 래스터 제약조건을 삭제합니다. 데이터를 다시 로드하거나 사용자 래스터 열 데이터를 업데이트해야 할 경우 유용합니다. 래스터 테이블 또는 래스터 열을 제거하려고 이 함수를 쓸 필요는 없습니다.

래스터 테이블을 삭제하는 표준적인 방법은 다음과 같습니다.

```
DROP TABLE mytable
```

래스터 열만 삭제하고 테이블의 나머지 부분은 남겨두려면, 표준 SQL 을 이용하십시오.

```
ALTER TABLE mytable DROP COLUMN rast
```

열 또는 테이블을 삭제할 경우 raster_columns 카탈로그에서 테이블이 사라질 것입니다. 하지만 제약조건들만 삭제할 경우, 래스터 열은 계속 raster_columns 카탈로그 목록에 남아 있을 것입니다. 그러나 열 및 테이블 명칭을 제외한 정보는 남아 있지 않을 것입니다.

2.0.0 버전부터 사용할 수 있습니다.

예시

```
SELECT DropRasterConstraints ('myrasters','rast');
-- 결과 출력 --
t

-- raster_columns 의 변경 사항 확인 --
SELECT srid, scale_x, scale_y, blocksize_x, blocksize_y, num_bands, pixel_types,
nodata_values
      FROM raster_columns
      WHERE r_table_name = 'myrasters';

srid | scale_x | scale_y | blocksize_x | blocksize_y | num_bands | pixel_types| nodata_values
-----+---------+---------+-------------+-------------+-----------+------------+-------------|
  0 |         |         |             |             |           |            |             |
```

참고: Section 5.2.2, “래스터 오버뷰”, DropOverviewConstraints, ST_CreateOverview, AddRasterConstraints

9.2.4. PostGIS_GDAL_Version

PostGIS_GDAL_Version — PostGIS 가 이용하고 있는 GDAL 라이브러리의 버전을 반환합니다.

Synopsis

text PostGIS_GDAL_Version();

설명: PostGIS 가 이용하고 있는 GDAL 라이브러리의 버전을 반환합니다. 또 GDAL 이 자체 데이터 파일을 찾을 수 있는지도 확인해서 보고할 것입니다.

예시

```
 SELECT PostGIS_GDAL_Version();
      postgis_gdal_version
 -----------------------------------
  GDAL 1.11dev, released 2013/04/13
```

참고: postgis.gdal_datapath

9.2.5. PostGIS_Raster_Lib_Build_Date

PostGIS_Raster_Lib_Build_Date — 전체 래스터 라이브러리의 빌드 날짜를 반환합니다.

Synopsis

Text **PostGIS_Raster_Lib_Build_Date();**

설명: 래스터의 빌드 날짜를 보고합니다.

예시

```
 SELECT PostGIS_Raster_Lib_Build_Date();
 postgis_raster_lib_version
 -------------------------------------------------------
2010-04-28 21:15:10
```

참고: PostGIS_Raster_Lib_Version

9.2.6. PostGIS_Raster_Lib_Version

PostGIS_Raster_Lib_Version – 전체 래스터 버전 및 설정 정보를 반환합니다.

Synopsis

text PostGIS_Raster_Lib_Version();

설명: PostGIS 가 이용하고 있는 GDAL 라이브러리의 버전을 반환합니다. 또 GDAL 이 자체 데이터 파일을 찾을 수 있는지도 확인해서 보고할 것입니다.

예시

```
 SELECT PostGIS_GDAL_Version();
 postgis_gdal_version
 -----------------------------------
GDAL 1.11dev, released 2013/04/13
```

참고: postgis.gdal_datapath

9.2.7. ST_GDALDrivers

ST_GDALDrivers — 사용자 GDAL 라이브러리가 지원하는 래스터 형식들의 목록을 반환합니다. 사용자가 ST_AsGDALRaster 함수를 통해 사용자 래스터를 출력할 때 쓸 수 있는 형식들입니다.

Synopsis

setof record ST_GDALDrivers(integer *OUT idx*, text *OUT short_name*, text *OUT long_name*, text *OUT create_options*);

설명: 사용자 GDAL 라이브러리가 지원하는 각 형식의 short_name, long_name 및 create_options 를 보여주는 래스터 형식 목록을 반환합니다. ST_AsGDALRaster 함수의 format 파라미터에 short_name 을 입력하십시오. 사용자 GDAL 라이브러리를 어떤 드라이버와 함께 컴파일했느냐에 따라 옵션이 달라집니다. create_options 는 특정 드라이버에 대한 각 생성 옵션의 명칭, 선택적인 type, description 그리고 VALUE 집합으로 구성된 CreationOptionList/Option 집합을 XML 서식으로 반환합니다.

변경 사항: 2.0.6, 2.1.3 버전 - GUC 또는 환경 변수 gdal_enabled_drivers 를 설정하지 않을 경우, 기본적으로 어떤 드라이버도 활성화돼 있지 않습니다.

2.0.0 버전부터 사용할 수 있습니다. GDAL 1.6.0 이상 버전이 필요합니다.

예시: 드라이브 목록

```
SET postgis.gdal_enabled_drivers = 'ENABLE_ALL';
SELECT short_name, long_name
FROM st_gdaldrivers()
ORDER BY short_name;
   short_name    |           long_name
-----------------+--------------------------------------
 AAIGrid         | Arc/Info ASCII Grid
 ARG             | Azavea Raster Grid format
 DTED            | DTED Elevation Raster
 EHdr            | ESRI .hdr Labelled
 FIT             | FIT Image
 GIF             | Graphics Interchange Format (.gif)
 GPKG            | GeoPackage
 GS7BG           | Golden Software 7 Binary Grid (.grd)
 GSAG            | Golden Software ASCII Grid (.grd)
 GSBG            | Golden Software Binary Grid (.grd)
 GTiff           | GeoTIFF
 HF2             | HF2/HFZ heightfield raster
 HFA             | Erdas Imagine Images (.img)
 ILWIS           | ILWIS Raster Map
 INGR            | Intergraph Raster
 JPEG            | JPEG JFIF
 KMLSUPEROVERLAY | Kml Super Overlay
 LCP             | FARSITE v.4 Landscape File (.lcp)
 MFF             | Vexcel MFF Raster
 NITF            | National Imagery Transmission Format
 PNG             | Portable Network Graphics
 R               | R Object Data Store
 RST             | Idrisi Raster A.1
 SAGA            | SAGA GIS Binary Grid (.sdat)
 SRTMHGT         | SRTMHGT File Format
 USGSDEM         | USGS Optional ASCII DEM (and CDED)
```

```
  VRT           | Virtual Raster
  WMS           | OGC Web Map Service
  XPM           | X11 PixMap Format
  XYZ           | ASCII Gridded XYZ
  ZMap          | ZMap Plus Grid
(31 rows)
```

예시: 각 드라이버에 대한 옵션 목록

```
-- JPEG의 생성 옵션 XML 열을 표로 출력 --
-- 이 생성 옵션들을 ST_AsGDALRaster의 인수로 사용할 수 있다는 점을 기억하십시오.
SELECT (xpath('@name', g.opt))[1]::text As oname,
      (xpath('@type', g.opt))[1]::text As otype,
      (xpath('@description', g.opt))[1]::text As descrip
FROM (SELECT unnest(xpath('/CreationOptionList/Option', create_options::xml)) As opt
FROM  st_gdaldrivers()
WHERE short_name = 'JPEG') As g;


      oname       |  otype  |     descrip
------------------+---------+-----------------------------------------------------
 PROGRESSIVE      | boolean | whether to generate a progressive JPEG
 QUALITY          | int     | good=100, bad=0, default=75
 WORLDFILE        | boolean | whether to geneate a worldfile
 INTERNAL_MASK    | boolean | whether to generate a validity mask
 COMMENT          | string  | Comment
 SOURCE_ICC_PROFILE | string  | ICC profile encoded in Base64
 EXIF_THUMBNAIL   | boolean | whether to generate an EXIF thumbnail(overview).
                            By default its max dimension will be 128
 THUMBNAIL_WIDTH  | int     | Forced thumbnail width
 THUMBNAIL_HEIGHT | int     | Forced thumbnail height
(9 rows)

-- GeoFiff에 대한 생성 옵션의 가공되지 않은 XML 출력물 --
SELECT create_options
FROM st_gdaldrivers()
WHERE short_name = 'GTiff';

<CreationOptionList>
   <Option name="COMPRESS" type="string-select">
      <Value
>NONE</Value>
      <Value
>LZW</Value>
      <Value
>PACKBITS</Value>
      <Value
>JPEG</Value>
      <Value
>CCITTRLE</Value>
      <Value
>CCITTFAX3</Value>
      <Value
>CCITTFAX4</Value>
      <Value
>DEFLATE</Value>
   </Option>
   <Option name="PREDICTOR" type="int" description="Predictor Type"/>
   <Option name="JPEG_QUALITY" type="int" description="JPEG quality 1-100" default="75"/>
   <Option  name="ZLEVEL"  type="int"  description="DEFLATE  compression  level  1-9"
default="6"/>
   <Option name="NBITS" type="int" description="BITS for sub-byte files (1-7), sub-uint16
(9-15), sub-uint32 (17-31)"/>
   <Option name="INTERLEAVE" type="string-select" default="PIXEL">
      <Value
>BAND</Value>
      <Value
>PIXEL</Value>
   </Option>
   <Option name="TILED" type="boolean" description="Switch to tiled format"/>
```

```
     <Option name="TFW" type="boolean" description="Write out world file"/>
     <Option name="RPB" type="boolean" description="Write out .RPB (RPC) file"/>
     <Option name="BLOCKXSIZE" type="int" description="Tile Width"/>
     <Option name="BLOCKYSIZE" type="int" description="Tile/Strip Height"/>
     <Option name="PHOTOMETRIC" type="string-select">
        <Value
  >MINISBLACK</Value>
        <Value
  >MINISWHITE</Value>
        <Value
  >PALETTE</Value>
        <Value
  >RGB</Value>
        <Value
  >CMYK</Value>
        <Value
  >YCBCR</Value>
        <Value
  >CIELAB</Value>
        <Value
  >ICCLAB</Value>
        <Value
  >ITULAB</Value>
     </Option>
     <Option name="SPARSE_OK" type="boolean" description="Can newly created files have
missing blocks?" default="FALSE"/>
     <Option name="ALPHA" type="boolean" description="Mark first extrasample as being
alpha"/>
     <Option name="PROFILE" type="string-select" default="GDALGeoTIFF">
        <Value
  >GDALGeoTIFF</Value>
        <Value
  >GeoTIFF</Value>
        <Value
  >BASELINE</Value>
     </Option>
     <Option name="PIXELTYPE" type="string-select">
        <Value
  >DEFAULT</Value>
        <Value
  >SIGNEDBYTE</Value>
     </Option>
     <Option name="BIGTIFF" type="string-select" description="Force creation of BigTIFF
file">
        <Value
  >YES</Value>
        <Value
  >NO</Value>
        <Value
  >IF_NEEDED</Value>
        <Value
  >IF_SAFER</Value>
     </Option>
     <Option name="ENDIANNESS" type="string-select" default="NATIVE" description="Force
endianness of created file. For DEBUG purpose mostly">
        <Value
  >NATIVE</Value>
        <Value
  >INVERTED</Value>
        <Value
  >LITTLE</Value>
        <Value
  >BIG</Value>
     </Option>
     <Option name="COPY_SRC_OVERVIEWS" type="boolean" default="NO" description="Force copy
of overviews of source dataset (CreateCopy())"/>
  </CreationOptionList
  >

  -- GeoFiff 에 대한 생성 옵션 XML 열을 표로 출력  --
  SELECT (xpath('@name', g.opt))[1]::text As oname,
        (xpath('@type', g.opt))[1]::text As otype,
        (xpath('@description', g.opt))[1]::text As descrip,
```

```
        array_to_string(xpath('Value/text()', g.opt),', ') As vals
  FROM (SELECT unnest(xpath('/CreationOptionList/Option', create_options::xml)) As opt
  FROM  st_gdaldrivers()
  WHERE short_name = 'GTiff') As g;

     oname    |  otype  |              descrip               |                vals
  ------------+---------+------------------------------------+-------------------------------------------
   COMPRESS      | string-select |                                    | NONE, LZW, PACKBITS, JPEG, CCITTRLE,
CCITTFAX3, CCITTFAX4, DEFLATE
   PREDICTOR     | int     | Predictor Type                          |
   JPEG_QUALITY    | int     | JPEG quality 1-100                         |
   ZLEVEL      | int     | DEFLATE compression level 1-9                |
   NBITS       | int     | BITS for sub-byte files (1-7), sub-uint16 (9-15), sub-uint32 (17-31) |
   INTERLEAVE    | string-select |                                  | BAND, PIXEL
   TILED       | boolean   | Switch to tiled format                     |
   TFW       | boolean   | Write out world file                       |
   RPB       | boolean   | Write out .RPB (RPC) file                    |
   BLOCKXSIZE    | int     | Tile Width                              |
   BLOCKYSIZE    | int     | Tile/Strip Height                          |
   PHOTOMETRIC     | string-select |                                     | MINISBLACK, MINISWHITE, PALETTE, RGB,
CMYK, YCBCR, CIELAB, ICCLAB, ITULAB
   SPARSE_OK     | boolean   | Can newly created files have missing blocks?          |
   ALPHA       | boolean   | Mark first extrasample as being alpha              |
   PROFILE      | string-select |                                  | GDALGeoTIFF, GeoTIFF, BASELINE
   PIXELTYPE     | string-select |                                   | DEFAULT, SIGNEDBYTE
   BIGTIFF      | string-select | Force creation of BigTIFF file                 | YES, NO, IF_NEEDED, IF_SAFER
   ENDIANNESS      | string-select | Force endianness of created file. For DEBUG purpose mostly       | NATIVE,
INVERTED, LITTLE, BIG
   COPY_SRC_OVERVIEWS | boolean   | Force copy of overviews of source dataset (CreateCopy())     |
(19 rows)
```

참고: ST_AsGDALRaster, ST_SRID, postgis.gdal_enabled_drivers

9.2.8. UpdateRasterSRID

UpdateRasterSRID — 사용자가 지정한 열 및 테이블에 있는 모든 래스터의 SRID 를 변경합니다.

Synopsis

raster UpdateRasterSRID(name *schema_name*, name *table_name*, name *column_name*, integer *new_srid*);

raster UpdateRasterSRID(name *table_name*, name *column_name*, integer *new_srid*);

설명: 사용자가 지정한 열 및 테이블에 있는 모든 래스터의 SRID 를 변경합니다. 이 함수는 지정한 열의 래스터의 SRID 를 변경하기 전에 적절한 모든 열 제약조건들(범위, 정렬 방향 및 SRID)을 삭제할 것입니다.



래스터의 데이터(밴드 픽셀 값)를 건드리지는 않습니다. 래스터의 메타데이터만 변경합니다.

2.1.0 버전부터 사용할 수 있습니다.

참고: UpdateGeometrySRID

9.2.9. ST_CreateOverview

ST_CreateOverview — 입력 래스터 커버리지의 저해상도 버전을 생성합니다.

Synopsis

regclass ST_CreateOverview(regclass *tab*, name *col*, int *factor*, text *algo='NearestNeighbor'*);

설명: 소스 테이블로부터 리샘플링한 타일들을 담은 미리보기 테이블을 생성합니다. 출력 타일은 입력 타일과 크기가 동일하고, 더 낮은 해상도로 동일한 공간 범위를 커버합니다(픽셀 크기는 양방향 모두 원본의 1/factor 이 될 것입니다).

raster_overviews 카탈로그에서 이 미리보기 테이블을 사용할 수 있게 되고, 래스터 제약조건도 강제될 것입니다.

알고리즘 옵션에는 'NearestNeighbor', 'Bilinear', 'Cubic', 'CubicSpline', 그리고 'Lanczos'가 있습니다. 자세한 내용은 GDAL Warp resampling methods 를 참조하십시오.

2.2.0 버전부터 사용할 수 있습니다.

참고: ST_Retile, AddOverviewConstraints, AddRasterConstraints, Section 5.2.2, "래스터 오버뷰"

9.3. 래스터 생성자

ST_AddBand — 입력 인덱스 위치에 입력 초기값으로 추가된 입력 유형의 새 밴드(들)을 가진 래스터를 반환합니다. 인덱스를 설정하지 않을 경우, 마지막 위치에 밴드를 추가합니다.

ST_AsRaster — PostGIS 도형을 PostGIS 래스터로 변환합니다.

ST_Band — 기존 래스터의 하나 이상의 밴드를 새 래스터로 반환합니다. 기존 래스터로부터 새 래스터를 빌드하는 데 유용합니다.

ST_MakeEmptyRaster — 설정된 차원(너비 & 높이), 좌상단 X 및 Y, 픽셀 크기, 회전(scalex, scaley, skewx & skewy) 그리고 공간 참조 시스템(SRID)를 가진 텅 빈 (밴드가 없는) 래스터를 반환합니다. 래스터를 입력할 경우, 동일한 크기, 정렬 방향 및 SRID 를 가진 새 래스터를 반환합니다. SRID 를 생략할 경우, 공간 참조 시스템을 0(unknown)으로 설정합니다.

ST_Tile — 출력 래스터에 원하는 차원을 바탕으로 입력 래스터를 분할해서 나온 래스터들의 집합을 반환합니다.

ST_Retile — 임의로 타일화된 래스터 커버리지로부터, 설정된 타일들의 집합을 반환합니다.

ST_FromGDALRaster — 지원 GDAL 래스터 파일로부터 래스터를 반환합니다.

9.3.1. ST AddBand

ST_AddBand — 입력 인덱스 위치에 입력 초기값으로 추가된 입력 유형의 새 밴드(들)을 가진 래스터를 반환합니다. 인덱스를 설정하지 않을 경우, 마지막 위치에 밴드를 추가합니다.

Synopsis

(1) raster ST_AddBand(raster *rast*, addbandarg[ ] *addbandargset*);

(2) raster ST_AddBand(raster *rast*, integer *index*, text *pixeltype*, double precision *initialvalue=0*, double precision *nodataval=NULL*);

(3) raster ST_AddBand(raster *rast*, text *pixeltype*, double precision *initialvalue=0*, double precision *nodataval=NULL*);

(4) raster ST_AddBand(raster *torast*, raster *fromrast*, integer *fromband=1*, integer *torastindex=at_end*);

(5) raster ST_AddBand(raster *torast*, raster[] *fromrasts*, integer *fromband=1*, integer *torastindex=at_end*);

(6) raster ST_AddBand(raster *rast*, integer *index*, text *outdbfile*, integer[] *outdbindex*, double precision *nodataval=NULL*);

(7) raster ST_AddBand(raster *rast*, text *outdbfile*, integer[] *outdbindex*, integer *index=at_end*, double precision *nodataval=NULL*);

설명: 입력 인덱스 위치에 입력 초기값, 입력 유형, 그리고 입력 NODATA 값의 새 밴드를 가진 래스터를 반환합니다. 인덱스를 설정하지 않을 경우, 마지막 위치에 밴드를 추가합니다. fromband 를 설정하지 않을 경우, 밴드 1 로 가정합니다. 픽셀 유형은 ST_BandPixelType 에서 지정한 픽셀 유형들 가운데 하나의 문자열 표현식입니다. 만약 기존 인덱스를 설정할 경우, 해당 인덱스보다 크거나 같은 그 이후의 모든 밴드들의 인덱스가 1 씩 증가합니다. 픽셀 유형의 최대값보다 큰 초기값을 설정할 경우, 해당 픽셀 유형이 가질 수 있는 가장 높은 값으로 초기값을 설정합니다.

addbandarg 의 배열을 입력받는 변종 1 의 경우, 특정 addbandarg 의 인덱스 값은 addbandarg 가 묘사하는 밴드가 래스터에 추가될 당시의 해당 래스터에 상대적입니다. 다음에 나오는 복수의 새로운 밴드 예시를 참고하십시오.

래스터 배열을 입력받는 변종 5 의 경우, torast 가 NULL 이라면 배열 안에 있는 각 래스터의 fromband 밴드를 새 래스터에 누계(累計)합니다.

outdbfile 을 입력받는 변종 6 및 7 의 경우, outdbfile 값이 래스터 파일을 가리키는 전체 경로를 포함해야만 합니다. PostgreSQL 서버 프로세스도 해당 파일에 접근할 수 있어야만 합니다.

개선 사항: 2.1.0 버전부터 addbandarg 를 지원합니다.

개선 사항: 2.1.0 버전부터 새로운 DB 외부 밴드를 지원합니다.

예시: 새로운 단일 밴드

```
SELECT   ROW(2,   '0-100:1-10,   101-500:11-150,501   -   10000:   151-254',   '8BUI',
255)::reclassarg; -- 픽셀 초기값을 200 으로 설정한, 부호가 없는(unsigned) 8 비트 정수형 유형의 또 다른
밴드를 추가
UPDATE dummy_rast
   SET rast = ST_AddBand(rast,'8BUI'::text,200)
WHERE rid = 1;

-- 좌상단이 딱 0 인 100x100 단위의 빈 래스터를 생성하고, (밴드 1 은 0/1 불 값 비트 스위치, 밴드 2 는 0 에서
15 사이의 값을 가질 수 있는) 밴드 2 개를 추가
-- addbandargs 를 이용
INSERT INTO dummy_rast(rid,rast)
   VALUES(10, ST_AddBand(ST_MakeEmptyRaster(100, 100, 0, 0, 1, -1, 0, 0, 0),
      ARRAY[
```

```
            ROW(1, '1BB'::text, 0, NULL),
            ROW(2, '4BUI'::text, 0, NULL)
                  ]::addbandarg[]
    )
   );

-- 모든 작업이 정상인지 확인하기 위해 래스터 밴드들의 메타데이터를 출력 --
SELECT  (bmd).*
FROM (SELECT ST_BandMetaData(rast,generate_series(1,2)) As bmd
   FROM dummy_rast WHERE rid = 10) AS foo;
 -- 결과 --
 pixeltype | nodatavalue | isoutdb | path
-----------+-------------+---------+---------+------
 1BB       |             |    f    |
 4BUI      |             |    f    |


-- 래스터의 메타데이터를 출력 --
SELECT  (rmd).width, (rmd).height, (rmd).numbands
FROM (SELECT ST_MetaData(rast) As rmd
   FROM dummy_rast WHERE rid = 10) AS foo;
-- 결과 --
 upperleftx | upperlefty | width | height | scalex | scaley | skewx | skewy | srid | numbands
------------+------------+-------+--------+--------+--------+-------+-------+------+-----------------------
     0      |     0      | 100   |  100   |    1   |  -1    |   0   |   0   |  0   |     2
```

예시: 복수의 새로운 밴드

```
SELECT
      *
FROM ST_BandMetadata(
      ST_AddBand(
            ST_MakeEmptyRaster(10, 10, 0, 0, 1, -1, 0, 0, 0),
            ARRAY[
                  ROW(NULL, '8BUI', 255, 0),
                  ROW(NULL, '16BUI', 1, 2),
                  ROW(2, '32BUI', 100, 12),
                  ROW(2, '32BF', 3.14, -1)
            ]::addbandarg[]
      ),
      ARRAY[]::integer[]
);

 bandnum | pixeltype | nodatavalue | isoutdb | path
---------+-----------+-------------+---------+------
      1  |    8BUI   |      0      | f       |
      2  |    32BF   |     -1      | f       |
      3  |    32BUI  |     12      | f       |
      4  |    16BUI  |      2      | f       |

-- 동일한 유형의 래스터들의 테이블의 첫 번째 밴드를 test_types 만큼 많은 밴드들과
-- mice 만큼 많은 행들(새 래스터들)과 함께 단일 래스터로 종합
-- 주의: PostgreSQL 9.0 이상 버전만 ORDER BY test_type 을 지원합니다.
-- 8.4 및 그 이전 버전에서는 사용자 데이터를 하위 집합으로 정렬하는 식으로 (대부분의 경우) 동작합니다.
-- test_type 의 알파벳 순서로 각 test_type 에 대한 밴드를 가진 래스터를 출력할 것입니다.
-- 동물보호론자 분들께: 이 예시에서 어떤 쥐(mice)도 희생되지 않았습니다.
SELECT
      mouse,
      ST_AddBand(NULL, array_agg(rast ORDER BY test_type), 1) As rast
FROM mice_studies
GROUP BY mouse;
```

예시: 새로운 DB 외부 밴드

```
 SELECT
       *
```

```
FROM ST_BandMetadata(
       ST_AddBand(
              ST_MakeEmptyRaster(10, 10, 0, 0, 1, -1, 0, 0, 0),
              '/home/raster/mytestraster.tif'::text, NULL::int[]
       ),
       ARRAY[]::integer[]
);

 bandnum | pixeltype | nodatavalue | isoutdb | path
---------+-----------+-------------+---------+------
   1     |    8BUI   |             | t       | /home/raster/mytestraster.tif
   2     |    8BUI   |             | t       | /home/raster/mytestraster.tif
   3     |    8BUI   |             | t       | /home/raster/mytestraster.tif
```

참고: ST_BandMetaData, ST_BandPixelType, ST_MakeEmptyRaster, ST_MetaData, ST_NumBands, ST_Reclass

9.3.2. ST_AsRaster

ST_AsRaster — PostGIS 도형을 PostGIS 래스터로 변환합니다.

Synopsis

raster ST_AsRaster(geometry *geom*, raster *ref*, text *pixeltype*, double precision *value=1*, double precision *nodataval=0*, boolean *touched=false*);

raster ST_AsRaster(geometry *geom*, raster *ref*, text[] *pixeltype=ARRAY['8BUI']*, double precision[] *value=ARRAY[1]*, double precision[] *nodataval=ARRAY[0]*, boolean *touched=false*);

raster ST_AsRaster(geometry *geom*, double precision *scalex*, double precision *scaley*, double precision *gridx*, double precision *gridy*, text *pixeltype*, double precision *value=1*, double precision *nodataval=0*, double precision *skewx=0*, double precision *skewy=0*, boolean *touched=false*);

raster ST_AsRaster(geometry *geom*, double precision *scalex*, double precision *scaley*, double precision *gridx=NULL*, double precision *gridy=NULL*, text[] *pixeltype=ARRAY['8BUI']*, double precision[] *value=ARRAY[1]*, double precision[] *nodataval=ARRAY[0]*, double precision *skewx=0*, double precision *skewy=0*, boolean *touched=false*);

raster ST_AsRaster(geometry *geom*, double precision *scalex*, double precision *scaley*, text *pixeltype*, double precision *value=1*, double precision *nodataval=0*, double precision *upperleftx=NULL*, double precision *upperlefty=NULL*, double precision *skewx=0*, double precision *skewy=0*, boolean *touched=false*);

raster ST_AsRaster(geometry *geom*, double precision *scalex*, double precision *scaley*, text[] *pixeltype*, double precision[] *value=ARRAY[1]*, double precision[] *nodataval=ARRAY[0]*, double precision *upperleftx=NULL*, double precision *upperlefty=NULL*, double precision *skewx=0*, double precision *skewy=0*, boolean *touched=false*);

raster ST_AsRaster(geometry *geom*, integer *width*, integer *height*, double precision *gridx*, double precision *gridy*, text *pixeltype*, double precision *value=1*, double precision *nodataval=0*, double precision *skewx=0*, double precision *skewy=0*, boolean *touched=false*);

raster ST_AsRaster(geometry *geom*, integer *width*, integer *height*, double precision *gridx=NULL*, double precision *gridy=NULL*, text[] *pixeltype=ARRAY['8BUI']*, double precision[] *value=ARRAY[1]*, double precision[] *nodataval=ARRAY[0]*, double precision *skewx=0*, double precision *skewy=0*, boolean *touched=false*);

raster ST_AsRaster(geometry *geom*, integer *width*, integer *height*, text *pixeltype*, double precision *value=1*, double precision *nodataval=0*, double precision *upperleftx=NULL*, double precision *upperlefty=NULL*, double precision *skewx=0*, double precision *skewy=0*, boolean *touched=false*);

raster ST_AsRaster(geometry *geom*, integer *width*, integer *height*, text[] *pixeltype*, double precision[] *value=ARRAY[1]*, double precision[] *nodataval=ARRAY[0]*, double precision *upperleftx=NULL*, double precision *upperlefty=NULL*, double precision *skewx=0*, double precision *skewy=0*, boolean *touched=false*);

설명: PostGIS 도형을 PostGIS 래스터로 변환합니다. 다양한 변종들을 통해 출력 래스터의 정렬 방향과 픽셀 크기를 크게 세 가지 방법으로 설정할 수 있습니다.

처음 두 변종으로 이루어진 첫 번째 방법은 주어진 참조 래스터와 동일한 정렬 방향(scalex, scaley 및 gridx, gridy), 픽셀 유형, NODATA 값을 가진 래스터를 생성합니다. 일반적으로, 참조 래스터를 담고 있는 테이블과 도형을 담고 있는 테이블을 결합하는 방법으로 해당 참조 래스터를 입력합니다.

네 변종으로 이루어진 두 번째 방법은 픽셀 크기(scalex & scaley 및 skewx & skewy)의 파라미터를 제공, 사용자가 래스터의 차원을 설정할 수 있도록 합니다. 결과 래스터의 width & height 가 도형의 범위에 맞도록 조정될 것입니다. 대부분의 경우, 정수형 scalex & scaley 인수들을 PostgreSQL 이 올바른 변종을 선택하도록 이중 정밀도 데이터형으로 형변환해야만 합니다.

네 변종으로 이루어진 세 번째 방법은 래스터의 차원(width & height)을 제공, 사용자가 래스터의 차원을 수정할 수 있도록 합니다. 결과 래스터의 픽셀 크기 파라미터(scalex & scaley 및 skewx & skewy)가 도형의 범위에 맞도록 조정될 것입니다.

뒤의 두 방법 가운데 각 방법의 처음 두 변종들은 사용자가 정렬 그리드의 임의의 모서리(gridx & gridy)를 설정할 수 있도록 합니다. 다른 두 변종들은 좌상단(upperleftx & upperlefty))을 입력받습니다.

각 방법의 변종은 단일 밴드 래스터 또는 복수 밴드 래스터를 생성할 수 있게 해줍니다. 복수 밴드 래스터를 생성하려면, 사용자가 픽셀 유형의 배열(pixeltype[]), 초기값의 배열(value) 그리고 NODTATA 값의 배열(nodataval)을 준비해야만 합니다. 이들을 입력하지 않으면, 기본적으로 픽셀 유형은 8BUI, 초기값은 1, NODATA 값은 0 이 됩니다.

출력 래스터는 소스 도형과 동일한 공간 참조 시스템을 가지게 됩니다. 유일한 예외는 참조 래스터를 받는 변종들뿐입니다. 이런 경우 결과 래스터는 참조 래스터와 동일한 SRID 를 가지게 됩니다.

선택적인 touched 파라미터의 기본값은 거짓으로 GDAL ALL_TOUCHED 래스터화 옵션에 매핑되는데, 이 옵션은 라인 또는 폴리곤에 닿은 픽셀을 없앨 것인지 말 것인지 결정합니다. 렌더링된 라인 경로 상에 있는 픽셀만이 아니라, 그 중심점이 폴리곤 내부에 있는 픽셀 또한 말이죠.

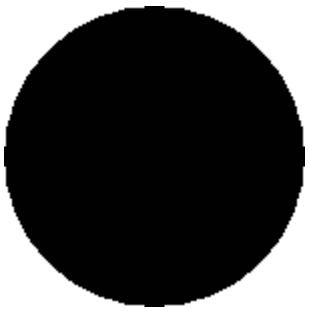
이 함수는 ST_AsPNG 및 다른 ST_AsGDALRaster 계열 함수와 결합해서 사용할 때 데이터베이스로부터 직접 가져온 도형의 JPEG 또는 PNG 를 렌더링하는 데 특히 유용합니다.

2.0.0 버전부터 사용할 수 있습니다. GDAL 1.6.0 이상 버전이 필요합니다.



아직 만곡 도형, TIN, 다면체 표면 등의 복잡 도형 유형을 렌더링할 수는 없지만, GDAL 이 관련 기능을 지원하게 되면 할 수 있게 될 것입니다.

예시: PNG 파일로 도형을 출력



검정색 원

```
  -- 이 쿼리는 150x150 픽셀을 차지하는 검정색 원을 출력할 것입니다. --
SELECT ST_AsPNG(ST_AsRaster(ST_Buffer(ST_Point(1,5),10),150, 150, '2BUI'));
```



PostGIS 만을 이용해서 렌더링된 버퍼의 예시

```
  -- RGB 밴드 - 암녹색 값 (118,154,118) - 에 매핑된 밴드들 --
 SELECT ST_AsPNG(
     ST_AsRaster(
         ST_Buffer(
             ST_GeomFromText('LINESTRING(50 50,150 150,150 50)'), 10,'join=bevel'),
             200,200,ARRAY['8BUI', '8BUI', '8BUI'], ARRAY[118,154,118], ARRAY[0,0,0]));
```

참고: ST_BandPixelType, ST_Buffer, ST_GDALDrivers, ST_AsGDALRaster, ST_AsPNG, ST_AsJPEG, ST_SRID

9.3.3. ST_Band

ST_Band — 기존 래스터의 하나 이상의 밴드를 새 래스터로 반환합니다. 기존 래스터로부터 새 래스터를 빌드하는 데 유용합니다.

Synopsis

raster ST_Band(raster rast, integer[] nbands = ARRAY[1]);

raster ST_Band(raster rast, integer nband);

raster ST_Band(raster rast, text nbands, character delimiter=,);

설명: 기존 래스터의 하나 이상의 밴드를 새 래스터로 반환합니다. 기존 래스터로부터 새 래스터를 빌드하거나, 래스터에서 선택한 밴드만 내보내거나, 래스터 내부 밴드들의 순서를 재배열하는 데

유용합니다. 어떤 밴드도 지정하지 않을 경우, 밴드 1 로 가정합니다. 밴드를 삭제하는 경우와 같은 다양한 함수에서 도우미 함수로 이용됩니다.



이 함수의 텍스트 변종으로 nbands 의 경우, 기본 구분자가 , 로 '1,2,3' 과 같은 서식으로 지정할 수 있다는 뜻입니다. 다른 구분자를 쓰고 싶은 경우 ST_Band(rast, '1@2@3', '@') 와 같은 서식을 이용하면 됩니다. 복수의 밴드를 지정하는 경우, 사용자가 이 함수의 ST_Band(rast, '{1,2,3}'::int[]); 와 같은 배열 서식을 이용하도록 강력히 권고합니다. PostGIS 향후 버전에서 밴드들의 text 목록 서식이 제거될 수도 있기 때문입니다.

2.0.0 버전부터 사용할 수 있습니다.

예시

```
  -- 새 래스터를 2 개 작성합니다: 가짜 래스터의 밴드 1 을 담고 있는 래스터 하나, 가짜 래스터의 밴드 2 를 가져와
2BUI 로 재분류한 래스터 하나
  SELECT ST_NumBands(rast1) As numb1, ST_BandPixelType(rast1) As pix1,
   ST_NumBands(rast2) As numb2,  ST_BandPixelType(rast2) As pix2
  FROM (
     SELECT ST_Band(rast) As rast1, ST_Reclass(ST_Band(rast,3), '100-200):1, [200-254:2',
'2BUI') As rast2
        FROM dummy_rast
        WHERE rid = 2) As foo;

   numb1 | pix1 | numb2 | pix2
  -------+------+-------+------
       1  |  8BUI |   1    | 2BUI

  -- 배열 형변환 문법을 이용해서 밴드 2 및 3 을 반환합니다.
  SELECT ST_NumBands(ST_Band(rast, '{2,3}'::int[])) As num_bands
     FROM dummy_rast WHERE rid=2;

  num_bands
  ----------
  2

  -- 밴드 2 및 3 을 반환합니다. 배열을 이용해 밴드를 정의합니다.
  SELECT ST_NumBands(ST_Band(rast, ARRAY[2,3])) As num_bands
     FROM dummy_rast
WHERE rid=2;
```

		
원본(rast 열)	*dupe_band*	*sing_band*

```
  -- 원본의 두 번째 밴드를 한 번, 첫 번째 밴드를 두 번 반복해서 새 래스터를 작성하고, 세 번째
밴드로 또 다른 래스터를 작성합니다.
  SELECT rast, ST_Band(rast, ARRAY[2,1,1]) As dupe_band,
        ST_Band(rast, 3) As sing_band
  FROM samples.than_chunked
WHERE rid=35;
```

참고: ST_AddBand, ST_NumBands, ST_Reclass, Chapter 9, 래스터 참조 문서

9.3.4. ST_MakeEmptyRaster

ST_MakeEmptyRaster — 설정된 차원(너비 & 높이), 좌상단 X 및 Y, 픽셀 크기, 회전(scalex, scaley, skewx & skewy) 그리고 공간 참조 시스템(SRID)를 가진 텅 빈 (밴드가 없는) 래스터를 반환합니다. 래스터를 입력할 경우, 동일한 크기, 정렬 방향 및 SRID 를 가진 새 래스터를 반환합니다. SRID 를 생략할 경우, 공간 참조 시스템을 0(unknown)으로 설정합니다.

Synopsis

raster **ST_MakeEmptyRaster**(raster *rast*);

raster **ST_MakeEmptyRaster**(integer *width*, integer *height*, float8 *upperleftx*, float8 *upperlefty*, float8 *scalex*, float8 *scaley*, float8 *skewx*, float8 *skewy*, integer *srid=unknown*);

raster **ST_MakeEmptyRaster**(integer *width*, integer *height*, float8 *upperleftx*, float8 *upperlefty*, float8 *pixelsize*);

설명: 설정된 차원(너비 & 높이), 공간 (또는 월드) 좌표로 지리참조된 좌상단 X(upperleftx) 및 좌상단 Y(upperlefty), 픽셀 크기, 회전(scalex, scaley, skewx & skewy) 그리고 공간 참조 시스템(SRID)를 가진 텅 빈 (밴드가 없는) 래스터를 반환합니다.

지난 버전은 픽셀 크기(pixelsize)를 설정하기 위한 단일 파라미터만 사용했습니다. scalex 는 이 인수로 설정됐고, scaley 는 이 인수의 음의 값으로 설정됐습니다. skewx 및 skewy 는 0 으로 설정됐습니다.

기존 래스터를 입력할 경우, 동일한 메타데이터 설정을 가진 (밴드가 없는) 새 래스터를 반환합니다.

어떤 SRID 도 설정하지 않을 경우 기본값은 0 입니다. 텅 빈 래스터를 생성한 다음 사용자가 밴드를 추가하거나 편집하려 할 수도 있습니다. 밴드를 정의하려면 ST_AddBand 를, 기본 픽셀 값을 설정하려면 ST_SetValue 를 참조하십시오.

예시

```
  INSERT INTO dummy_rast(rid,rast)
  VALUES(3, ST_MakeEmptyRaster( 100, 100, 0.0005, 0.0005, 1, 1, 0, 0, 4326) );

  -- 기존 래스터를 새 래스터를 위한 템플릿으로 사용
  INSERT INTO dummy_rast(rid,rast)
  SELECT 4, ST_MakeEmptyRaster(rast)
  FROM dummy_rast WHERE rid = 3;

  -- 방금 추가한 래스터의 메타데이터를 출력
  SELECT rid, (md).*
  FROM (SELECT rid, ST_MetaData(rast) As md
```

```
        FROM dummy_rast
        WHERE rid IN(3,4)) As foo;

 -- 출력 --
 rid  | upperleftx | upperlefty | width | height | scalex | scaley | skewx | skewy | srid | numbands
------+------------+------------+-------+--------+--------+--------+-------+-------+------+----------
  3   |   0.0005   |   0.0005   |  100  |  100   |   1    |   1    |   0   |   0   | 4326 |    0
  4   |   0.0005   |   0.0005   |  100  |  100   |   1    |   1    |   0   |   0   | 4326 |    0
```

참고: ST_AddBand, ST_MetaData, ST_ScaleX, ST_ScaleY, ST_SetValue, ST_SkewX, , ST_SkewY

9.3.5. ST_Tile

ST_Tile — 출력 래스터에 원하는 차원을 바탕으로 입력 래스터를 분할해서 나온 래스터들의 집합을 반환합니다.

Synopsis

setof raster ST_Tile(raster *rast*, int[] *nband*, integer *width*, integer *height*, boolean *padwithnodata=FALSE*, double precision *nodataval=NULL*);

setof raster ST_Tile(raster *rast*, integer *nband*, integer *width*, integer *height*, boolean *padwithnodata=FALSE*, double precision *nodataval=NULL*);

setof raster ST_Tile(raster *rast*, integer *width*, integer *height*, boolean *padwithnodata=FALSE*, double precision *nodataval=NULL*);

설명: 출력 래스터에 원하는 차원을 바탕으로 입력 래스터를 분할해서 나온 래스터들의 집합을 반환합니다.

padwithnodata = FALSE 일 경우, 래스터 우단 및 하단의 가장자리 타일들이 나머지 타일들과 다른 차원을 가지고 있을지도 모릅니다. padwithnodata = TRUE 일 경우, 모든 타일들이 동일한 차원을 가지게 되지만 가장자리 타일들이 NODATA 값으로 패딩(padding)됐을 가능성이 있습니다. 래스터 밴드(들)이 지정된 NODATA 값(들)을 가지지 않은 경우, nodataval 을 설정해서 NODATA 값을 지정할 수 있습니다.



입력 래스터의 지정된 밴드가 DB 외부 밴드일 경우, 출력 래스터의 해당 밴드 또한 DB 외부 밴드일 것입니다.

2.1.0 버전부터 사용할 수 있습니다.

예시

```
 WITH foo AS (
     SELECT ST_AddBand(ST_AddBand(ST_MakeEmptyRaster(3, 3, 0, 0, 1, -1, 0, 0, 0), 1, '8BUI', 1, 0), 2,
'8BUI', 10, 0) AS rast UNION ALL
     SELECT ST_AddBand(ST_AddBand(ST_MakeEmptyRaster(3, 3, 3, 0, 1, -1, 0, 0, 0), 1, '8BUI', 2, 0), 2,
'8BUI', 20, 0) AS rast UNION ALL
     SELECT ST_AddBand(ST_AddBand(ST_MakeEmptyRaster(3, 3, 6, 0, 1, -1, 0, 0, 0), 1, '8BUI', 3, 0), 2,
'8BUI', 30, 0) AS rast UNION ALL

     SELECT ST_AddBand(ST_AddBand(ST_MakeEmptyRaster(3, 3, 0, -3, 1, -1, 0, 0, 0), 1, '8BUI', 4, 0), 2,
'8BUI', 40, 0) AS rast UNION ALL
     SELECT ST_AddBand(ST_AddBand(ST_MakeEmptyRaster(3, 3, 3, -3, 1, -1, 0, 0, 0), 1, '8BUI', 5, 0), 2,
'8BUI', 50, 0) AS rast UNION ALL
```

```
     SELECT ST_AddBand(ST_AddBand(ST_MakeEmptyRaster(3, 3, 6, -3, 1, -1, 0, 0, 0), 1, '8BUI', 6, 0), 2,
'8BUI', 60, 0) AS rast UNION ALL

     SELECT ST_AddBand(ST_AddBand(ST_MakeEmptyRaster(3, 3, 0, -6, 1, -1, 0, 0, 0), 1, '8BUI', 7, 0), 2,
'8BUI', 70, 0) AS rast UNION ALL
     SELECT ST_AddBand(ST_AddBand(ST_MakeEmptyRaster(3, 3, 3, -6, 1, -1, 0, 0, 0), 1, '8BUI', 8, 0), 2,
'8BUI', 80, 0) AS rast UNION ALL
     SELECT ST_AddBand(ST_AddBand(ST_MakeEmptyRaster(3, 3, 6, -6, 1, -1, 0, 0, 0), 1, '8BUI', 9, 0), 2,
'8BUI', 90, 0) AS rast
 ), bar AS (
     SELECT ST_Union(rast) AS rast FROM foo
 ), baz AS (
     SELECT ST_Tile(rast, 3, 3, TRUE) AS rast FROM bar
 )
 SELECT
     ST_DumpValues(rast)
 FROM baz;

        st_dumpvalues
 ---------------------------------------
  (1,"{{1,1,1},{1,1,1},{1,1,1}}")
  (2,"{{10,10,10},{10,10,10},{10,10,10}}")
  (1,"{{2,2,2},{2,2,2},{2,2,2}}")
  (2,"{{20,20,20},{20,20,20},{20,20,20}}")
  (1,"{{3,3,3},{3,3,3},{3,3,3}}")
  (2,"{{30,30,30},{30,30,30},{30,30,30}}")
  (1,"{{4,4,4},{4,4,4},{4,4,4}}")
  (2,"{{40,40,40},{40,40,40},{40,40,40}}")
  (1,"{{5,5,5},{5,5,5},{5,5,5}}")
  (2,"{{50,50,50},{50,50,50},{50,50,50}}")
  (1,"{{6,6,6},{6,6,6},{6,6,6}}")
  (2,"{{60,60,60},{60,60,60},{60,60,60}}")
  (1,"{{7,7,7},{7,7,7},{7,7,7}}")
  (2,"{{70,70,70},{70,70,70},{70,70,70}}")
  (1,"{{8,8,8},{8,8,8},{8,8,8}}")
  (2,"{{80,80,80},{80,80,80},{80,80,80}}")
  (1,"{{9,9,9},{9,9,9},{9,9,9}}")
  (2,"{{90,90,90},{90,90,90},{90,90,90}}")
 (18 rows)

 WITH foo AS (
     SELECT ST_AddBand(ST_AddBand(ST_MakeEmptyRaster(3, 3, 0, 0, 1, -1, 0, 0, 0), 1, '8BUI', 1, 0), 2,
'8BUI', 10, 0) AS rast UNION ALL
     SELECT ST_AddBand(ST_AddBand(ST_MakeEmptyRaster(3, 3, 3, 0, 1, -1, 0, 0, 0), 1, '8BUI', 2, 0), 2,
'8BUI', 20, 0) AS rast UNION ALL
     SELECT ST_AddBand(ST_AddBand(ST_MakeEmptyRaster(3, 3, 6, 0, 1, -1, 0, 0, 0), 1, '8BUI', 3, 0), 2,
'8BUI', 30, 0) AS rast UNION ALL

     SELECT ST_AddBand(ST_AddBand(ST_MakeEmptyRaster(3, 3, 0, -3, 1, -1, 0, 0, 0), 1, '8BUI', 4, 0), 2,
'8BUI', 40, 0) AS rast UNION ALL
     SELECT ST_AddBand(ST_AddBand(ST_MakeEmptyRaster(3, 3, 3, -3, 1, -1, 0, 0, 0), 1, '8BUI', 5, 0), 2,
'8BUI', 50, 0) AS rast UNION ALL
     SELECT ST_AddBand(ST_AddBand(ST_MakeEmptyRaster(3, 3, 6, -3, 1, -1, 0, 0, 0), 1, '8BUI', 6, 0), 2,
'8BUI', 60, 0) AS rast UNION ALL

     SELECT ST_AddBand(ST_AddBand(ST_MakeEmptyRaster(3, 3, 0, -6, 1, -1, 0, 0, 0), 1, '8BUI', 7, 0), 2,
'8BUI', 70, 0) AS rast UNION ALL
     SELECT ST_AddBand(ST_AddBand(ST_MakeEmptyRaster(3, 3, 3, -6, 1, -1, 0, 0, 0), 1, '8BUI', 8, 0), 2,
'8BUI', 80, 0) AS rast UNION ALL
     SELECT ST_AddBand(ST_AddBand(ST_MakeEmptyRaster(3, 3, 6, -6, 1, -1, 0, 0, 0), 1, '8BUI', 9, 0), 2,
'8BUI', 90, 0) AS rast
 ), bar AS (
     SELECT ST_Union(rast) AS rast FROM foo
 ), baz AS (
     SELECT ST_Tile(rast, 3, 3, 2) AS rast FROM bar
 )
 SELECT
     ST_DumpValues(rast)
 FROM baz;
```

```
       st_dumpvalues
 ---------------------------------------
  (1,"{{10,10,10},{10,10,10},{10,10,10}}")
  (1,"{{20,20,20},{20,20,20},{20,20,20}}")
  (1,"{{30,30,30},{30,30,30},{30,30,30}}")
  (1,"{{40,40,40},{40,40,40},{40,40,40}}")
  (1,"{{50,50,50},{50,50,50},{50,50,50}}")
  (1,"{{60,60,60},{60,60,60},{60,60,60}}")
  (1,"{{70,70,70},{70,70,70},{70,70,70}}")
  (1,"{{80,80,80},{80,80,80},{80,80,80}}")
  (1,"{{90,90,90},{90,90,90},{90,90,90}}")
(9 rows)
```

참고: ST_Union, ST_Retile

9.3.6. ST_Retile

ST_Retile — 임의로 타일화된 래스터 커버리지로부터, 설정된 타일들의 집합을 반환합니다.

Synopsis

SETOF raster ST_Retile(regclass *tab*, name *col*, geometry *ext*, float8 *sfx*, float8 *sfy*, int *tw*, int *th*, text *algo='NearestNeighbor'*);

설명: 설정한 축척(sfx, sfy)과 최대 크기(tw, th)를 가지고, 설정한 래스터 커버리지(tab, col)에서 나온 데이터로 설정한 범위(ext)를 커버하는 타일들의 집합을 반환합니다.

알고리즘 옵션에는 'NearestNeighbor', 'Bilinear', 'Cubic', 'CubicSpline', 그리고 'Lanczos'가 있습니다. 자세한 내용은 GDAL Warp resampling methods 를 참조하십시오.

2.2.0 버전부터 사용할 수 있습니다.

참고: ST_CreateOverview

9.3.7. ST_FromGDALRaster

ST_FromGDALRaster — 지원 GDAL 래스터 파일로부터 래스터를 반환합니다.

Synopsis

raster ST_FromGDALRaster(bytea gdaldata, integer srid=NULL);

설명: 지원 GDAL 래스터 파일로부터 래스터를 반환합니다. gdaldata 는 bytea 유형으로 GDAL 래스터 파일의 내용을 담고 있어야 합니다.

srid 가 NULL 일 경우, 이 함수는 GDAL 래스터로부터 자동적으로 SRID 를 할당하려 할 것입니다. srid 를 설정한 경우, 해당 값이 자동적으로 할당된 어떤 SRID 보다 우선할 것입니다.

2.1.0 버전부터 사용할 수 있습니다.

예시

```
WITH foo AS (
```

```
      SELECT ST_AsPNG(ST_AddBand(ST_AddBand(ST_AddBand(ST_MakeEmptyRaster(2, 2, 0, 0,
0.1, -0.1, 0, 0, 4326), 1, '8BUI', 1, 0), 2, '8BUI', 2, 0), 3, '8BUI', 3, 0)) AS png
),
bar AS (
      SELECT 1 AS rid, ST_FromGDALRaster(png) AS rast FROM foo
      UNION ALL
      SELECT 2 AS rid, ST_FromGDALRaster(png, 3310) AS rast FROM foo
)
SELECT
      rid,
      ST_Metadata(rast) AS metadata,
      ST_SummaryStats(rast, 1) AS stats1,
      ST_SummaryStats(rast, 2) AS stats2,
      ST_SummaryStats(rast, 3) AS stats3
FROM bar
ORDER BY rid;


 rid |     metadata      |  stats1   |  stats2   |   stats3
-----+-------------------+-----------+-----------+-----------------------------------------
  1 | (0,0,2,2,1,-1,0,0,0,3)  | (4,4,1,0,1,1) | (4,8,2,0,2,2) | (4,12,3,0,3,3)
  2 | (0,0,2,2,1,-1,0,0,3310,3) | (4,4,1,0,1,1) | (4,8,2,0,2,2) | (4,12,3,0,3,3)
(2 rows)
```

참고: ST_AsGDALRaster

9.4. 래스터 접근자

ST_GeoReference — 월드(world) 파일에서 흔히 볼 수 있는 지리참조 메타데이터를 GDAL 또는 ESRI 형식으로 반환합니다. 기본값은 GDAL 입니다.

ST_Height — 래스터의 높이를 픽셀 개수로 반환합니다.

ST_IsEmpty — 래스터가 비어 있을 경우 (width = 0, height = 0) 참을 반환합니다. 그렇지 않을 경우 거짓을 반환합니다.

ST_MemSize — 래스터가 차지하는 공간의 용량을 (바이트 단위로) 반환합니다.

ST_MetaData — 래스터 객체의 픽셀 크기, 회전(skew), 좌상단, 좌하단 등과 같은 기본 메타데이터를 반환합니다.

ST_NumBands — 래스터 객체 내부에 있는 밴드들의 개수를 반환합니다.

ST_PixelHeight — 픽셀 높이를 공간 참조 시스템의 기하 단위로 반환합니다.

ST_PixelWidth — 픽셀 너비를 공간 참조 시스템의 기하 단위로 반환합니다.

ST_ScaleX — 픽셀 너비의 X 요소를 좌표 참조 시스템의 단위로 반환합니다.

ST_ScaleY — 픽셀 높이의 Y 요소를 좌표 참조 시스템의 단위로 반환합니다.

ST_RasterToWorldCoord — 주어진 열과 행에 있는 래스터의 좌상단의 기하학적 X, Y(경도, 위도)를 으로 반환합니다. 열과 행은 1 부터 시작합니다.

ST_RasterToWorldCoordX — 열과 행에 있는 래스터의 좌상단의 기하학적 X 좌표를 반환합니다. 열과 행의 번호는 1 부터 시작합니다.

ST_RasterToWorldCoordY — 열과 행에 있는 래스터의 좌상단의 기하학적 Y 좌표를 반환합니다. 열과 행의 번호는 1 부터 시작합니다.

ST_Rotation — 래스터의 회전각을 라디안으로 반환합니다.

ST_SkewX — 지리참조 X 기울기(skew)(또는 회전각 파라미터)를 반환합니다.

ST_SkewY — 지리참조 Y 기울기(또는 회전각 파라미터)를 반환합니다.

ST_SRID — spatial_ref_sys 테이블에 정의되어 있는, 래스터의 공간 참조 식별자를 반환합니다.

ST_Summary — 래스터의 내용을 요약한 텍스트를 반환합니다.

ST_UpperLeftX — 래스터의 좌상단 X 좌표를 투영된 공간 참조 단위로 반환합니다.

ST_UpperLeftY — 래스터의 좌상단 Y 좌표를 투영된 공간 참조 단위로 반환합니다.

ST_Width — 래스터의 너비를 픽셀 개수로 반환합니다.

ST_WorldToRasterCoord — 주어진 기하학적 X, Y(경도, 위도)의 좌상단을 열과 행으로 또는 래스터의 공간 참조 좌표 시스템 단위로 표현된 포인트 도형으로 반환합니다.

ST_WorldToRasterCoordX — 포인트 도형(pt)의 래스터 안의 열 또는 래스터의 월드 공간 참조 시스템 단위로 표현된 X, Y 월드 좌표(xw, yw)를 반환합니다.

ST_WorldToRasterCoordY — 포인트 도형(pt)의 래스터 안의 행 또는 래스터의 월드 공간 참조 시스템 단위로 표현된 X, Y 월드 좌표(xw, yw)를 반환합니다.

9.4.1. ST_GeoReference

ST_GeoReference — 월드(world) 파일에서 흔히 볼 수 있는 지리참조 메타데이터를 GDAL 또는 ESRI 형식으로 반환합니다. 기본값은 GDAL 입니다.

Synopsis

text **ST_GeoReference**(raster *rast*, text *format=GDAL*);

설명: 월드 파일 에서 흔히 보이는, 캐리지(carriage)를 포함한 지리참조 메타데이터를 GDAL 또는 ESRI 형식으로 반환합니다. 유형을 따로 설정하지 않을 경우 기본값은 GDAL 입니다. 해당 유형값은 'GDAL' 또는 'ESRI' 문자열입니다.

형식 표현식 사이의 차이점은 다음과 같습니다:

GDAL:

```
  scalex
  skewy
  skewx
  scaley
  upperleftx
upperlefty
```

ESRI:

```
  scalex
  skewy
  skewx
  scaley
  upperleftx + scalex*0.5
upperlefty + scaley*0.5
```

예시

```
  SELECT  ST_GeoReference(rast,  'ESRI')  As  esri_ref,  ST_GeoReference(rast,
'GDAL') As gdal_ref
   FROM dummy_rast WHERE rid=1;

    esri_ref   |   gdal_ref
  -------------+-------------
   2.0000000000 | 2.0000000000
   0.0000000000 : 0.0000000000
   0.0000000000 : 0.0000000000
   3.0000000000 : 3.0000000000
   1.5000000000 : 0.5000000000
 2.0000000000 : 0.5000000000
```

참고: ST_SetGeoReference, ST_ScaleX, ST_ScaleY

9.4.2. ST_HEIGHT

ST_Height — 래스터의 높이를 픽셀 개수로 반환합니다.

Synopsis

integer **ST_Height**(raster *rast*);

설명: 래스터의 높이를 반환합니다

예시

```
  SELECT rid, ST_Height(rast) As rastheight
  FROM dummy_rast;

     rid | rastheight
  -------+------------
      1  |       20
      2  |        5
```

참고: ST_Width

9.4.3. ST_IsEmpty

ST_IsEmpty — 래스터가 비어 있을 경우 (width = 0, height = 0) 참을 반환합니다. 그렇지 않을 경우 거짓을 반환합니다.

Synopsis

integer **ST_WorldToRasterCoordY**(raster *rast*, geometry *pt*);

boolean **ST_IsEmpty**(raster *rast*);

설명: 래스터가 비어 있을 경우 (width = 0, height = 0) 참을 반환합니다. 그렇지 않을 경우 거짓을 반환합니다.

2.0.0 버전부터 사용할 수 있습니다.

예시

```
  SELECT ST_IsEmpty(ST_MakeEmptyRaster(100, 100, 0, 0, 0, 0, 0, 0))
  St_isempty |
  ---------------+
  f       |
  SELECT ST_IsEmpty(ST_MakeEmptyRaster(0, 0, 0, 0, 0, 0, 0, 0))
  St_isempty |
  ---------------+
t         |
```

참고: ST_HasNoBand

9.4.4. ST_Memsize

ST_MemSize — 래스터가 차지하는 공간의 용량을 (바이트 단위로) 반환합니다.

Synopsis

integer ST_MemSize(raster *rast*);

설명: 래스터가 차지하는 공간의 용량을 (바이트 단위로) 반환합니다.

이 함수는 PostgreSQL 내장 함수들인 pg_column_size, pg_size_pretty, pg_relation_size, pg_total_relation_size 를 멋지게 보완합니다.



테이블의 바이트 용량을 출력하는 pg_relation_size 가 ST_MemSize 보다 적은 바이트 용량을 반환할 수도 있습니다. 왜냐하면 pg_relation_size 함수는 TOAST 테이블에 저장된 대용량 도형들 및 TOAST 가 적용된 테이블이 분담하는 용량을 추가하지 않기 때문입니다. pg_column_size 함수는 압축된 용량을 반환하기 때문에 더 적은 용량을 반환할 수도 있습니다.

pg_total_relation_size 함수는 테이블, TOAST 가 적용된 테이블 및 인덱스를 포함합니다.

2.2.0 버전부터 사용할 수 있습니다.

예시

```
SELECT    ST_MemSize(ST_AsRaster(ST_Buffer(ST_Point(1,5),10,1000),150,    150,    '8BUI'))As
rast_mem;

           rast_mem
```

```
 --------
22568
```

9.4.5. ST_MetaData

ST_MetaData — 래스터 객체의 픽셀 크기, 회전(skew), 좌상단, 좌하단 등과 같은 기본 메타데이터를 반환합니다.

Synopsis

record **ST_MetaData**(raster *rast*);

설명: 래스터 객체의 픽셀 크기, 회전(skew), 좌상단, 좌하단 등과 같은 기본 메타데이터를 반환합니다. 반환되는 열들은 다음과 같습니다: upperleftx | upperlefty | width | height | scalex | scaley | skewx | skewy | srid | numbands

예시

```
SELECT rid, (foo.md).*
 FROM (SELECT rid, ST_MetaData(rast) As md
FROM dummy_rast) As foo;

rid | upperleftx | upperlefty | width | height | scalex | scaley | skewx | skewy | srid | numbands
----+------------+------------+-------+--------+--------+--------+-------+-------+------+---------
 1  |    0.5     |    0.5     |  10   |   20   |   2    |   3    |   0   |   0   |  0   |    0
 2  | 3427927.75 |   5793244  |   5   |    5   |  0.05  |  -0.05 |   0   |   0   |  0   |    3
```

참고: ST_BandMetaData, ST_NumBands

9.4.6. ST_NumBands

ST_NumBands — 래스터 객체 내부에 있는 밴드들의 개수를 반환합니다.

Synopsis

integer **ST_NumBands**(raster *rast*);

설명: 래스터 객체 내부에 있는 밴드들의 개수를 반환합니다.

예시

```
SELECT rid, ST_NumBands(rast) As numbands
FROM dummy_rast;

rid | numbands
----+----------
  1 |        0
  2 |        3
```

참고: ST_Value

9.4.7. ST_PixelHeight

ST_PixelHeight — 픽셀 높이를 공간 참조 시스템의 기하 단위로 반환합니다.

Synopsis

double precision ST_PixelHeight(raster *rast*);

설명: 픽셀 높이를 공간 참조 시스템의 기하 단위로 반환합니다. 회전하지 않은 일반적인 경우, 픽셀 높이란 단순히 기하 좌표와 래스터 픽셀의 축척 비율일 뿐입니다.

이 관계성을 그림으로 시각화한 예시를 보려면 ST_PixelWidth 를 참조하십시오.

예시: 회전하지 않은 래스터

```
SELECT ST_Height(rast) As rastheight, ST_PixelHeight(rast) As pixheight,
  ST_ScaleX(rast) As scalex, ST_ScaleY(rast) As scaley, ST_SkewX(rast) As skewx,
       ST_SkewY(rast) As skewy
 FROM dummy_rast;

  rastheight | pixheight | scalex | scaley | skewx | skewy
 ------------+-----------+--------+--------+-------+----------
       20    |      3    |    2   |    3   |    0  |       0
       5     |     0.05  |   0.05 |  -0.05 |    0  |       0
```

예시: 0 값 이외의 값으로 회전한 래스터

```
 SELECT ST_Height(rast) As rastheight, ST_PixelHeight(rast) As pixheight,
  ST_ScaleX(rast) As scalex, ST_ScaleY(rast) As scaley, ST_SkewX(rast) As skewx,
       ST_SkewY(rast) As skewy
 FROM (SELECT ST_SetSKew(rast,0.5,0.5) As rast
       FROM dummy_rast) As skewed;

 rastheight |    pixheight     | scalex | scaley | skewx | skewy
 -----------+------------------+--------+--------+-------+----------
       20 |  3.04138126514911 |     2 |     3 |   0.5 |     0.5
        5 | 0.502493781056044 |   0.05 |  -0.05 |   0.5 |     0.5
```

참고: ST_PixelWidth, ST_ScaleX, ST_ScaleY, ST_SkewX, ST_SkewY

9.4.8. ST_PixelWidth

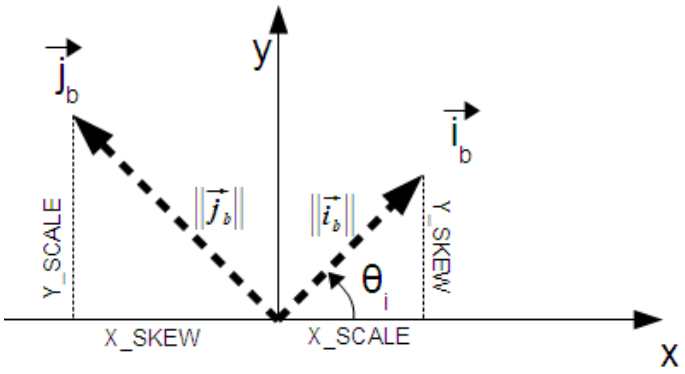
ST_PixelWidth — 픽셀 너비를 공간 참조 시스템의 기하 단위로 반환합니다.

Synopsis

double precision **ST_PixelWidth**(raster *rast*);

설명: 픽셀 너비를 공간 참조 시스템의 기하 단위로 반환합니다. 회전하지 않은 일반적인 경우, 픽셀 너비란 단순히 기하 좌표와 래스터 픽셀의 축척 비율일 뿐입니다.

다음은 해당 관계성을 보여주는 도표입니다:



픽셀 너비: *i 방향의 픽셀 크기*

픽셀 높이: j 방향의 픽셀 크기

예시: 회전하지 않은 래스터

```
SELECT ST_Width(rast) As rastwidth, ST_PixelWidth(rast) As pixwidth,
       ST_ScaleX(rast) As scalex, ST_ScaleY(rast) As scaley, ST_SkewX(rast) As skewx,
       ST_SkewY(rast) As skewy
       FROM dummy_rast;

       rastwidth | pixwidth | scalex | scaley | skewx | skewy
       ----------+----------+--------+--------+-------+----------
           10      |     2    |     2   |    3   |    0    |    0
            5      |   0.05   |   0.05  |  -0.05 |    0    |     0
```

예시: 0 값 이외의 값으로 회전한 래스터

```
SELECT ST_Width(rast) As rastwidth, ST_PixelWidth(rast) As pixwidth,
       ST_ScaleX(rast) As scalex, ST_ScaleY(rast) As scaley, ST_SkewX(rast) As skewx,
       ST_SkewY(rast) As skewy
       FROM (SELECT ST_SetSkew(rast,0.5,0.5) As rast
       FROM dummy_rast) As skewed;

       rastwidth |    pixwidth     | scalex | scaley | skewx | skewy
       ----------+-----------------+--------+--------+-------+----------
           10    |  2.06155281280883|     2  |   3    |  0.5 |     0.5
            5    | 0.502493781056044|   0.05 |  -0.05 |  0.5  |     0.5
```

참고: ST_PixelHeight, ST_ScaleX, ST_ScaleY, ST_SkewX, ST_SkewY

9.4.9. ST_ScaleX

ST_ScaleX — 픽셀 너비의 X 요소를 좌표 참조 시스템의 단위로 반환합니다.

Synopsis

float8 **ST_ScaleX**(raster *rast*);

설명: 픽셀 너비의 X 요소를 좌표 참조 시스템의 단위로 반환합니다. 자세한 내용은 월드 파일 을 참조하십시오.

변경 사항: 2.0.0 미만 WKTRaster 버전에서는 ST_PixelSizeX 라는 명칭이었습니다.

예시

```
SELECT rid, ST_ScaleX(rast) As rastpixwidth
FROM dummy_rast;

 rid | rastpixwidth
-----+--------------
   1 |           2
   2 |        0.05
```

참고: ST_Width

9.4.10. ST_ScaleY

ST_ScaleY — 픽셀 높이의 Y 요소를 좌표 참조 시스템의 단위로 반환합니다.

Synopsis

float8 **ST_ScaleY**(raster *rast*);

설명: 픽셀 높이의 Y 요소를 좌표 참조 시스템의 단위로 반환합니다. 음수일 수도 있습니다. 자세한 내용은 월드 파일 을 참조하십시오.

변경 사항: 2.0.0 미만 WKTRaster 버전에서는 ST_PixelSizeY 라는 명칭이었습니다.

예시

```
SELECT rid, ST_ScaleY(rast) As rastpixheight
FROM dummy_rast;

 rid | rastpixheight
-----+---------------
   1 |           3
 2   |       -0.05
```

참고: ST_Height

9.4.11. ST_RasterToWorldCoord

ST_RasterToWorldCoord — 주어진 열과 행에 있는 래스터의 좌상단의 기하학적 X, Y(경도, 위도)를 으로 반환합니다. 열과 행은 1 부터 시작합니다.

Synopsis

record ST_RasterToWorldCoord(raster *rast*, integer *xcolumn*, integer *yrow*);

설명: 주어진 열과 행에 있는 래스터의 좌상단의 기하학적 X, Y(경도, 위도)를 반환합니다. 반환된 X, Y 의 좌표는 지리참조된 래스터의 기하 단위입니다. 열과 행의 번호는 1 부터 시작하지만 두 파라미터 가운데 하나라도 0, 음수, 또는 래스터의 각 차원보다 큰 숫자를 입력할 경우, 래스터의 그리드가 래스터의 경계선 바깥에도 적용된다고 가정하고 래스터 외부의 좌표를 반환할 것입니다.

2.1.0 버전부터 사용할 수 있습니다.

예시

```
-- 회전하지 않은 래스터
SELECT
       rid,
       (ST_RasterToWorldCoord(rast,1, 1)).*,
       (ST_RasterToWorldCoord(rast,2, 2)).*
FROM dummy_rast

 rid | longitude  | latitude | longitude |  latitude
-----+------------+----------+-----------+------------
   1 |       0.5 |      0.5 |       2.5 |        3.5
   2 | 3427927.75 |  5793244 | 3427927.8 | 5793243.95



-- 회전한 래스터
SELECT
       rid,
       (ST_RasterToWorldCoord(rast, 1, 1)).*,
       (ST_RasterToWorldCoord(rast, 2, 3)).*
FROM (
       SELECT
              rid,
              ST_SetSkew(rast, 100.5, 0) As rast
       FROM dummy_rast
) As foo



 rid | longitude  | latitude | longitude | latitude
-----+------------+----------+-----------+-----------
   1 |     0.5    |    0.5   |    203.5  |      6.5
   2 | 3427927.75 |  5793244 | 3428128.8 | 5793243.9
```

참고: ST_RasterToWorldCoordX, ST_RasterToWorldCoordY, ST_SetSkew

9.4.12. ST_RasterToWorldCoordX

ST_RasterToWorldCoordX — 열과 행에 있는 래스터의 좌상단의 기하학적 X 좌표를 반환합니다. 열과 행의 번호는 1 부터 시작합니다.

Synopsis

float8 **ST_RasterToWorldCoordX**(raster *rast*, integer *xcolumn*);

float8 **ST_RasterToWorldCoordX**(raster *rast*, integer *xcolumn*, integer *yrow*);

설명: 래스터 열과 행의 좌상단 X 좌표를 지리참조된 래스터의 기하 단위로 반환합니다. 열과 행의 번호는 1 부터 시작하지만 두 파라미터 가운데 하나라도 음수 또는 래스터 열의 개수보다 큰 숫자를 입력할 경우, 회전 및 픽셀 크기가 선택된 래스터와 동일하다고 가정하고 래스터 파일의 왼쪽 또는 오른쪽 바깥에 있는 좌표를 반환할 것입니다.



회전하지 않은 래스터의 경우, X 열만 입력해도 충분합니다. 회전한 래스터의 경우, 지리참조된 좌표는 ST_ScaleX, ST_SkewX, 행 및 열의 함수입니다. 회전한 래스터에 대해 X 열만 입력할 경우 오류가 발생할 것입니다.

변경 사항: 2.1.0 미만 버전에서는 ST_Raster2WorldCoordx 라는 명칭이었습니다.

예시

```
-- 회전하지 않은 래스터일 경우 열만 입력해도 충분합니다.
SELECT rid, ST_RasterToWorldCoordX(rast,1) As x1coord,
      ST_RasterToWorldCoordX(rast,2) As x2coord,
      ST_ScaleX(rast) As pixelx
FROM dummy_rast;

 rid |  x1coord   |  x2coord  | pixelx
-----+-----------+----------+--------
   1  |       0.5  |      2.5  |     2
   2  | 3427927.75| 3427927.8|   0.05

-- 재미삼아 래스터를 회전시켜봅시다.
SELECT rid, ST_RasterToWorldCoordX(rast, 1, 1) As x1coord,
      ST_RasterToWorldCoordX(rast, 2, 3) As x2coord,
      ST_ScaleX(rast) As pixelx
FROM (SELECT rid, ST_SetSkew(rast, 100.5, 0) As rast FROM dummy_rast) As foo;

 rid |  x1coord   |  x2coord  | pixelx
-----+-----------+----------+--------
   1  |       0.5  |    203.5  |     2
   2  | 3427927.75| 3428128.8|   0.05
```

참고: ST_ScaleX, ST_RasterToWorldCoordY, ST_SetSkew, ST_SkewX

9.4.13. ST_RasterToWorldCoordY

ST_RasterToWorldCoordY — 열과 행에 있는 래스터의 좌상단의 기하학적 Y 좌표를 반환합니다. 열과 행의 번호는 1 부터 시작합니다.

Synopsis

float8 **ST_RasterToWorldCoordY**(raster *rast*, integer *yrow*);

float8 **ST_RasterToWorldCoordY**(raster *rast*, integer *xcolumn*, integer *yrow*);

설명: 래스터 열과 행의 좌상단 Y 좌표를 지리참조된 래스터의 기하 단위로 반환합니다. 열과 행의 번호는 1 부터 시작하지만 두 파라미터 가운데 하나라도 음수 또는 래스터 열/행의 개수보다 큰 숫자를 입력할 경우, 회전 및 픽셀 크기가 선택된 래스터와 동일하다고 가정하고 래스터 타일의 왼쪽 또는 오른쪽 바깥에 있는 좌표를 반환할 것입니다.



회전하지 않은 래스터의 경우, Y 열만 입력해도 충분합니다. 회전한 래스터의 경우, 지리참조된 좌표는 ST_ScaleY, ST_SkewY, 행 및 열의 함수입니다. 회전한 래스터에 대해 Y 행만 입력할 경우 오류가 발생할 것입니다.

변경 사항: 2.1.0 미만 버전에서는 ST_Raster2WorldCoordY 라는 명칭이었습니다.

예시

```
-- 회전하지 않은 래스터일 경우 행만 입력해도 충분합니다.
SELECT rid, ST_RasterToWorldCoordY(rast,1) As y1coord,
      ST_RasterToWorldCoordY(rast,3) As y2coord,
      ST_ScaleY(rast) As pixely
FROM dummy_rast;

 rid | y1coord |  y2coord  | pixely
-----+---------+-----------+--------
   1 |     0.5 |       6.5 |      3
   2 | 5793244 | 5793243.9 |  -0.05

-- 재미삼아 래스터를 회전시켜봅시다.
SELECT rid, ST_RasterToWorldCoordY(rast,1,1) As y1coord,
      ST_RasterToWorldCoordY(rast,2,3) As y2coord,
      ST_ScaleY(rast) As pixely
FROM (SELECT rid, ST_SetSkew(rast,0,100.5) As rast FROM dummy_rast) As foo;

 rid | y1coord |  y2coord  | pixely
-----+---------+-----------+--------
   1 |     0.5 |     107   |      3
   2 | 5793244 | 5793344.4 |  -0.05
```

참고: ST_ScaleY, ST_RasterToWorldCoordX, ST_SetSkew, ST_SkewY

9.4.14. ST_Rotation

ST_Rotation — 래스터의 회전각을 라디안으로 반환합니다.

Synopsis

float8 **ST_Rotation**(raster *rast*);

설명: 래스터의 균일한 회전각을 라디안으로 반환합니다. 래스터가 균일한 회전각으로 회전하지 않은 경우, NaN 을 반환합니다. 자세한 내용은 월드 파일 을 참조하십시오.

예시

```
 SELECT rid, ST_Rotation(ST_SetScale(ST_SetSkew(rast, sqrt(2)), sqrt(2))) as rot FROM
dummy_rast;

  rid |       rot
 -----+------------------
    1 | 0.785398163397448
  2   | 0.785398163397448
```

참고: ST_SetRotation, ST_SetScale, ST_SetSkew

9.4.15. ST_SkewX

ST_SkewX — 지리참조 X 기울기(skew)(또는 회전각 파라미터)를 반환합니다.

Synopsis

float8 **ST_SkewX**(raster *rast*);

설명: 지리참조 X 기울기(또는 회전각 파라미터)를 반환합니다. 자세한 내용은 월드 파일 을 참조하십시오.

예시

```
SELECT rid, ST_SkewX(rast) As skewx, ST_SkewY(rast) As skewy,
   ST_GeoReference(rast) as georef
FROM dummy_rast;

 rid | skewx | skewy |      georef
-----+-------+-------+--------------------
   1 |    0  |   0   | 2.0000000000
                     : 0.0000000000
                     : 0.0000000000
                     : 3.0000000000
                     : 0.5000000000
                     : 0.5000000000
                     :
   2 |    0 |    0 | 0.0500000000
                    : 0.0000000000
                    : 0.0000000000
                    : -0.0500000000
                    : 3427927.7500000000
                   : 5793244.0000000000
```

참고: ST_GeoReference, ST_SkewY, ST_SetSkew

9.4.16. ST_SkewY

ST_SkewY — 지리참조 Y 기울기(또는 회전각 파라미터)를 반환합니다.

Synopsis

float8 **ST_SkewY**(raster *rast*);

설명: 지리참조 Y 기울기(또는 회전각 파라미터)를 반환합니다. 자세한 내용은 월드 파일을 참조하십시오.

예시

```
SELECT rid, ST_SkewX(rast) As skewx, ST_SkewY(rast) As skewy,
   ST_GeoReference(rast) as georef
FROM dummy_rast;

 rid | skewx | skewy |      georef
-----+-------+-------+--------------------
   1 |    0 |    0    | 2.0000000000
                      : 0.0000000000
                      : 0.0000000000
                      : 3.0000000000
                      : 0.5000000000
                      : 0.5000000000
                      :
   2 |    0 |    0    | 0.0500000000
                      : 0.0000000000
                      : 0.0000000000

                      : -0.0500000000
                      : 3427927.7500000000
                      : 5793244.0000000000
```

참고: ST_GeoReference, ST_SkewX, ST_SetSkew

9.4.17. ST_SRID

ST_SRID — spatial_ref_sys 테이블에 정의되어 있는, 래스터의 공간 참조 식별자를 반환합니다.

Synopsis

integer **ST_SRID**(raster *rast*);

설명: spatial_ref_sys 테이블에 정의되어 있는, 래스터 객체의 공간 참조 식별자를 반환합니다.



PostGIS 2.0 버전부터, 지리참조되지 않은 래스터/도형의 SRID 가 이전 버전의 -1 대신 0 으로 바뀌었습니다.

예시

```
SELECT ST_SRID(rast) As srid
FROM dummy_rast WHERE rid=1;

srid
----------------
0
```

참고: Section 4.3.1, "SPATIAL_REF_SYS 테이블과 공간 참조 시스템", ST_SRID

9.4.18. ST_Summary

ST_Summary — 래스터의 내용을 요약한 텍스트를 반환합니다.

Synopsis

text ST_Summary(raster *rast*);

설명: 래스터의 내용을 요약한 텍스트를 반환합니다.

2.1.0 버전부터 사용할 수 있습니다.

예시

```
SELECT ST_Summary(
       ST_AddBand(
              ST_AddBand(
                     ST_AddBand(
                            ST_MakeEmptyRaster(10, 10, 0, 0, 1, -1, 0, 0, 0)
                            , 1, '8BUI', 1, 0
                     )
                     , 2, '32BF', 0, -9999
```

```
              )
              , 3, '16BSI', 0, NULL
        )
  );

                        st_summary
  ------------------------------------------------------------------
   Raster of 10x10 pixels has 3 bands and extent of BOX(0 -10,10 0)+
      band 1 of pixtype 8BUI is in-db with NODATA value of 0     +
      band 2 of pixtype 32BF is in-db with NODATA value of -9999  +
      band 3 of pixtype 16BSI is in-db with no NODATA value
(1 row)
```

참고: ST_MetaData, ST_BandMetaData, ST_Summary ST_Extent

9.4.19. ST_UpperLeftX

ST_UpperLeftX — 래스터의 좌상단 X 좌표를 투영된 공간 참조 단위로 반환합니다.

Synopsis

float8 **ST_UpperLeftX**(raster *rast*);

설명: 래스터의 좌상단 X 좌표를 투영된 공간 참조 단위로 반환합니다.

예시

```
 SELECt rid, ST_UpperLeftX(rast) As ulx
 FROM dummy_rast;

  rid |    ulx
 -----+------------
    1  |       0.5
    2  | 3427927.75
```

참고: ST_UpperLeftY, ST_GeoReference, Box3D

9.4.20. ST_UpperLeftY

ST_UpperLeftY — 래스터의 좌상단 Y 좌표를 투영된 공간 참조 단위로 반환합니다.

Synopsis

float8 **ST_UpperLeftY**(raster *rast*);

설명: 래스터의 좌상단 Y 좌표를 투영된 공간 참조 단위로 반환합니다.

예시

```
 SELECT rid, ST_UpperLeftY(rast) As uly
 FROM dummy_rast;
```

```
 rid |   uly
-----+---------
   1 |     0.5
   2 | 5793244
```

참고: ST_UpperLeftX, ST_GeoReference, Box3D

9.4.21. ST_Width

ST_Width — 래스터의 너비를 픽셀 개수로 반환합니다.

Synopsis

integer **ST_Width**(raster *rast*);

설명: 래스터의 너비를 픽셀 개수로 반환합니다.

예시

```
SELECT ST_Width(rast) As rastwidth
FROM dummy_rast WHERE rid=1;

rastwidth
----------------
10
```

참고: ST_Height

9.4.22. ST_WorldToRasterCoord

ST_WorldToRasterCoord — 주어진 기하학적 X, Y(경도, 위도)의 좌상단을 열과 행으로 또는 래스터의 공간 참조 좌표 시스템 단위로 표현된 포인트 도형으로 반환합니다.

Synopsis

record ST_WorldToRasterCoord(raster *rast*, geometry *pt*);

record ST_WorldToRasterCoord(raster *rast*, double precision *longitude*, double precision *latitude*);

설명: 주어진 기하학적 X, Y(경도, 위도)의 좌상단을 열과 행으로 또는 포인트 도형으로 반환합니다. 이 함수는 기하학적 X, Y 또는 포인트 도형이 래스터 범위 외부에 있건 말건 상관없이 동작합니다. 기하학적 X, Y는 래스터의 공간 참조 좌표 시스템 단위로 표현돼야만 합니다.

2.1.0 버전부터 사용할 수 있습니다.

예시

```
SELECT
      rid,
      (ST_WorldToRasterCoord(rast,3427927.8,20.5)).*,
```

```
        (ST_WorldToRasterCoord(rast,ST_GeomFromText('POINT(3427927.8
20.5)',ST_SRID(rast)))).*
  FROM dummy_rast;

   rid | columnx |   rowy    | columnx |   rowy
  -----+---------+---------+---------+-----------
     1  | 1713964 |       7   | 1713964 |        7
     2  |     2     | 115864471|     2    | 115864471
```

참고: ST_WorldToRasterCoordX, ST_WorldToRasterCoordY, ST_RasterToWorldCoordX, ST_RasterToWorldCoordY, ST_SRID

9.4.23. ST_WorldToRasterCoordX

ST_WorldToRasterCoordX — 포인트 도형(pt)의 래스터 안의 열 또는 래스터의 월드 공간 참조 시스템 단위로 표현된 X, Y 월드 좌표(xw, yw)를 반환합니다.

Synopsis

integer **ST_WorldToRasterCoordX**(raster *rast*, geometry *pt*);

integer **ST_WorldToRasterCoordX**(raster *rast*, double precision *xw*);

integer **ST_WorldToRasterCoordX**(raster *rast*, double precision *xw*, double precision *yw*);

설명: 포인트 도형(pt)의 래스터 안의 열 또는 X, Y 월드 좌표(xw, yw)를 반환합니다. 포인트가 필요합니다(또는 래스터가 기울어진 경우 xw 및 yw 월드 좌표가 두 개 모두 필요합니다). 래스터가 기울어지지 않은 경우 xw 좌표만으로도 충분합니다. 월드 좌표란 래스터의 공간 참조 좌표 시스템의 좌표를 말합니다.

변경 사항: 2.1.0 미만 버전에서는 ST_World2RasterCoordX 라는 명칭이었습니다.

예시

```
  SELECT rid, ST_WorldToRasterCoordX(rast,3427927.8) As xcoord,
              ST_WorldToRasterCoordX(rast,3427927.8,20.5) As xcoord_xwyw,
              ST_WorldToRasterCoordX(rast,ST_GeomFromText('POINT(3427927.8
20.5)',ST_SRID(rast))) As ptxcoord
  FROM dummy_rast;

   rid | xcoord  |  xcoord_xwyw   | ptxcoord
  -----+---------+---------+----------
     1  | 1713964 | 1713964 |  1713964
     2  |      1   |      1   |       1
```

참고: ST_RasterToWorldCoordX, ST_RasterToWorldCoordY, ST_SRID

9.4.24. ST_WorldToRasterCoordY

ST_WorldToRasterCoordY — 포인트 도형(pt)의 래스터 안의 행 또는 래스터의 월드 공간 참조 시스템 단위로 표현된 X, Y 월드 좌표(xw, yw)를 반환합니다.

Synopsis

integer **ST_WorldToRasterCoordY**(raster *rast*, geometry *pt*);

integer **ST_WorldToRasterCoordY**(raster *rast*, double precision *xw*);

integer **ST_WorldToRasterCoordY**(raster *rast*, double precision *xw*, double precision *yw*);

설명: 포인트 도형(pt)의 래스터 안의 행 또는 X, Y 월드 좌표(xw, yw)를 반환합니다. 포인트가 필요합니다(또는 래스터가 기울어진 경우 xw 및 yw 월드 좌표가 두 개 모두 필요합니다). 래스터가 기울어지지 않은 경우 xw 좌표만으로도 충분합니다. 월드 좌표란 래스터의 공간 참조 좌표 시스템의 좌표를 말합니다.

변경 사항: 2.1.0 미만 버전에서는 ST_World2RasterCoordY 라는 명칭이었습니다.

예시

```
  SELECT rid, ST_WorldToRasterCoordY(rast,20.5) As ycoord,
              ST_WorldToRasterCoordY(rast,3427927.8,20.5) As ycoord_xwyw,
              ST_WorldToRasterCoordY(rast,ST_GeomFromText('POINT(3427927.8
20.5)',ST_SRID(rast))) As ptycoord
  FROM dummy_rast;

   rid |  ycoord   | ycoord_xwyw | ptycoord
  -----+----------+-------------+-----------
     1  |        7  |           7   |         7
     2  | 115864471|   115864471 | 115864471
```

참고: ST_RasterToWorldCoordX, ST_RasterToWorldCoordY, ST_SRID

9.5. 래스터 밴드 접근자

ST_BandMetaData — 지정 래스터 밴드에 대한 기본 메타데이터를 반환합니다. 밴드를 지정하지 않을 경우 밴드 1 번이라고 가정합니다.

ST_BandNoDataValue — 입력 밴드에서 NODATA 를 나타내는 값을 반환합니다. 어떤 밴드도 지정하지 않을 경우 밴드 1 로 가정합니다.

ST_BandIsNoData — 밴드가 NODATA 값만으로 채워져 있을 경우 참을 반환합니다.

ST_BandPath — 파일 시스템에 저장된 밴드를 가리키는 시스템 파일 경로를 반환합니다. bandnum 을 설정하지 않을 경우 밴드 1 로 가정합니다.

ST_BandPixelType — 입력 밴드의 픽셀 유형을 반환합니다. bandnum 을 설정하지 않을 경우 밴드 1 로 가정합니다.

ST_HasNoBand — 입력된 밴드 번호에 밴드가 없을 경우 참을 반환합니다. 밴드 번호를 설정하지 않을 경우, 밴드 1 로 가정합니다.

9.5.1. ST_BandMetaData

ST_BandMetaData — 지정 래스터 밴드에 대한 기본 메타데이터를 반환합니다. 밴드를 지정하지 않을 경우 밴드 1 번이라고 가정합니다.

Synopsis

record ST_BandMetaData(raster *rast*, integer *bandnum=1*);

설명: 래스터 밴드에 대한 기본 메타데이터를 반환합니다. 반환되는 열들은 다음과 같습니다: pixeltype | nodatavalue | isoutdb | path



래스터가 어떤 밴드도 담고 있지 않을 경우 오류가 발생합니다.



밴드에 NODATA 값이 있을 경우, nodatavalue 는 NULL 이 될 것입니다.

예시

```
SELECT rid, (foo.md).*
 FROM (SELECT rid, ST_BandMetaData(rast,1) As md
FROM dummy_rast WHERE rid=2) As foo;

 rid | pixeltype | nodatavalue | isoutdb | path
-----+----------+--------------+---------+---------+------
   2 |    8BUI    |       0       | f        |
```

참고: ST_MetaData, ST_BandPixelType

9.5.2. ST_BandNoDataValue

ST_BandNoDataValue — 입력 밴드에서 NODATA 를 나타내는 값을 반환합니다. 어떤 밴드도 지정하지 않을 경우 밴드 1 로 가정합니다.

Synopsis

double precision ST_BandNoDataValue(raster *rast*, integer *bandnum=1*);

설명: 밴드에서 NODATA 를 나타내는 값을 반환합니다.

예시

```
SELECT ST_BandNoDataValue(rast,1) As bnval1,
   ST_BandNoDataValue(rast,2) As bnval2, ST_BandNoDataValue(rast,3) As bnval3
FROM dummy_rast
WHERE rid = 2;

 bnval1 | bnval2 | bnval3
--------+--------+--------
      0   |      0   |      0
```

참고: ST_NumBands

9.5.3. ST_BandNoDataValue

ST_BandNoDataValue — 입력 밴드에서 NODATA 를 나타내는 값을 반환합니다. 어떤 밴드도 지정하지 않을 경우 밴드 1 로 가정합니다.

Synopsis

double precision ST_BandNoDataValue(raster *rast*, integer *bandnum=1*);

설명: 밴드에서 NODATA 를 나타내는 값을 반환합니다.

예시

```
SELECT ST_BandNoDataValue(rast,1) As bnval1,
   ST_BandNoDataValue(rast,2) As bnval2, ST_BandNoDataValue(rast,3) As bnval3
FROM dummy_rast
WHERE rid = 2;

 bnval1 | bnval2 | bnval3
--------+--------+--------
      0 |      0 |      0
```

참고: ST_NumBands

9.5.4. ST_BandIsNoData

ST_BandIsNoData — 밴드가 NODATA 값만으로 채워져 있을 경우 참을 반환합니다.

Synopsis

boolean ST_BandIsNoData(raster *rast*, integer *band*, boolean *forceChecking=true*);

boolean ST_BandIsNoData(raster *rast*, boolean *forceChecking=true*);

설명: 밴드가 NODATA 값만으로 채워져 있을 경우 참을 반환합니다. 밴드를 지정하지 않으면 밴드 1 로 가정합니다. 마지막 인수가 TRUE 일 경우, 밴드의 픽셀 전체를 하나하나 확인합니다. 그렇지 않을 경우, 이 함수는 밴드에 대한 isnodata 플래그의 값을 반환할 뿐입니다. 따로 설정하지 않을 경우, 해당 파라미터의 기본값은 FALSE 입니다.

2.0.0 버전부터 사용할 수 있습니다.



플래그가 지저분할 (즉, 마지막 파라미터에 TRUE 를 설정하고 설정하지 않고에 따라 결과가 달라질) 경우, ST_SetBandNodataValue()의 마지막 인수로 TRUE 를 입력하거나, 또는

ST_SetBandIsNodata()를 이용해서 해당 플래그를 참으로 설정하도록 래스터를 업데이트해야 합니다. ST_SetBandIsNoData 를 참조하십시오.

예시

```
  -- 래스터 열 한 개를 가진 가짜 테이블을 생성
  create table dummy_rast (rid integer, rast raster);

  -- 밴드 두 개와 픽셀/밴드 하나를 가진 래스터를 추가합니다. 첫 번째 밴드의 nodatavalue =
pixel value = 3 입니다.
  -- 두 번째 밴드의 nodatavalue = 13, pixel value = 4 입니다.
  insert into dummy_rast values(1,
  (
  '01' -- little endian (uint8 ndr)
  ||
  '0000' -- version (uint16 0)
  ||
  '0200' -- nBands (uint16 0)
  ||
  '17263529ED684A3F' -- scaleX (float64 0.000805965234044584)
  ||
  'F9253529ED684ABF' -- scaleY (float64 -0.00080596523404458)
  ||
  '1C9F33CE69E352C0' -- ipX (float64 -75.5533328537098)
  ||
  '718F0E9A27A44840' -- ipY (float64 49.2824585505576)
  ||
  'ED50EB853EC32B3F' -- skewX (float64 0.000211812383858707)
  ||
  '7550EB853EC32B3F' -- skewY (float64 0.000211812383858704)
  ||
  'E6100000' -- SRID (int32 4326)
  ||
  '0100' -- width (uint16 1)
  ||
  '0100' -- height (uint16 1)
  ||
  '6' -- hasnodatavalue and isnodata value set to true.
  ||
  '2' -- first band type (4BUI)
  ||
  '03' -- novalue==3
  ||
  '03' -- pixel(0,0)==3 (same that nodata)
  ||
  '0' -- hasnodatavalue set to false
  ||
  '5' -- second band type (16BSI)
  ||
  '0D00' -- novalue==13
  ||
  '0400' -- pixel(0,0)==4
  )::raster
  );

  select st_bandisnodata(rast, 1) from dummy_rast where rid = 1; -- Expected
true
  select st_bandisnodata(rast, 2) from dummy_rast where rid = 1; -- Expected
false
```

참고: ST_BandNoDataValue, ST_NumBands, ST_SetBandNoDataValue, ST_SetBandIsNoData

9.5.5. ST_BandPath

ST_BandPath — 파일 시스템에 저장된 밴드를 가리키는 시스템 파일 경로를 반환합니다. bandnum 을 설정하지 않을 경우 밴드 1 로 가정합니다.

Synopsis

text ST_BandPath(raster *rast*, integer *bandnum=1*);

설명: 밴드를 가리키는 시스템 파일 경로를 반환합니다. DB 내부 밴드와 함께 호출할 경우 오류가 발생합니다.

9.5.6. ST_BandPixelType

ST_BandPixelType — 입력 밴드의 픽셀 유형을 반환합니다. bandnum 을 설정하지 않을 경우 밴드 1 로 가정합니다.

Synopsis

text ST_BandPixelType(raster *rast*, integer *bandnum=1*);

설명: 밴드에서 NODATA 를 나타내는 값을 반환합니다.

11 가지 픽셀 유형이 있습니다. 지원하는 픽셀 유형은 다음과 같습니다:

- 1BB - 1 비트 불 값
- 2BUI - 부호 없는 2 비트 정수형
- 4BUI - 부호 없는 4 비트 정수형
- 8BSI - 부호 있는 8 비트 정수형
- 8BUI - 부호 없는 8 비트 정수형
- 16BSI - 부호 있는 16 비트 정수형
- 16BUI - 부호 없는 16 비트 정수형
- 32BSI - 부호 있는 32 비트 정수형
- 32BUI - 부호 없는 32 비트 정수형
- 32BF - 32 비트 부동소수점 실수형
- 64BF - 64 비트 부동소수점 실수형

예시

```
SELECT ST_BandPixelType(rast,1) As btype1,
   ST_BandPixelType(rast,2) As btype2, ST_BandPixelType(rast,3) As btype3
FROM dummy_rast
WHERE rid = 2;

 btype1 | btype2 | btype3
--------+--------+--------
 8BUI   | 8BUI   | 8BUI
```

참고: ST_NumBands

9.5.7. ST_HasNoBand

ST_HasNoBand — 입력된 밴드 번호에 밴드가 없을 경우 참을 반환합니다. 밴드 번호를 설정하지 않을 경우, 밴드 1 로 가정합니다.

Synopsis

boolean ST_HasNoBand(raster *rast*, integer *bandnum=1*);

설명: 입력된 밴드 번호에 밴드가 없을 경우 참을 반환합니다. 밴드 번호를 설정하지 않을 경우, 밴드 1 로 가정합니다.

2.0.0 버전부터 사용할 수 있습니다.

예시

```
SELECT rid, ST_HasNoBand(rast) As hb1, ST_HasNoBand(rast,2) as hb2,
ST_HasNoBand(rast,4) as hb4, ST_NumBands(rast) As numbands
FROM dummy_rast;

rid | hb1 | hb2 | hb4 | numbands
----+-----+-----+-----+----------
1   | t   | t   | t   |        0
2   | f   | f   | t   |        3
```

참고: ST_NumBands

9.6. 래스터 픽셀 접근자 및 설정

ST_PixelAsPolygon — 특정 행 및 열에 대한 픽셀의 경계를 이루는 폴리곤 도형을 반환합니다.

ST_PixelAsPolygons — 래스터 밴드의 모든 픽셀의 경계를 이루는 폴리곤 도형을 각 픽셀의 값과 X, Y 래스터 좌표와 함께 반환합니다.

ST_PixelAsPoint — 픽셀의 좌상단에 위치하는 포인트 도형을 반환합니다.

ST_PixelAsPoints — 래스터 밴드의 각 픽셀에 대한 포인트 도형을 각 픽셀의 값 및 X, Y 래스터 좌표와 함께 반환합니다. 포인트 도형의 좌표는 픽셀의 좌상단 좌표입니다.

ST_PixelAsCentroid — 픽셀 하나가 차지하는 면의 중심점(포인트 도형)을 반환합니다.

ST_PixelAsCentroids — 래스터 밴드의 각 픽셀에 대한 중심점(포인트 도형)을 각 픽셀의 값 및 X, Y 래스터 좌표와 함께 반환합니다. 포인트 도형은 픽셀이 차지하는 면의 중심점입니다.

ST_Value — 입력 columnx, rowy 픽셀의 입력 밴드의 값, 또는 특정 기하학적 포인트의 밴드의 값을 반환합니다. 밴드 번호는 1 부터 시작하며, 따로 지정하지 않을 경우 1 로 가정합니다. exclude_nodata_value 를 거짓으로 설정할 경우, nodata 픽셀을 포함한 모든 픽셀이 교차한다고 가정하고 값을 반환합니다. exclude_nodata_value 를 설정하지 않은 경우, 래스터의 메타데이터에서 값을 읽어 들입니다.

ST_NearestValue — columnx 및 rowy, 또는 래스터와 동일한 공간 참조 좌표 시스템 단위로 표현된 기하학적 포인트로 지정된 입력 밴드의 픽셀에 가장 가까운 NODATA 가 아닌 값을 반환합니다.

ST_Neighborhood — columnx 및 rowy, 또는 래스터와 동일한 공간 참조 좌표 시스템 단위로 표현된 기하학적 포인트로 지정된 입력 밴드의 픽셀 주위의 NODATA 가 아닌 값들의 이중 정밀도 데이터형 2 차원 배열을 반환합니다.

ST_SetValue — 입력한 columnx, rowy 픽셀 또는 특정 도형과 교차하는 픽셀들의 위치의 밴드 값을 설정해서 나온 수정된 래스터를 반환합니다. 밴드 번호는 1 부터 시작하며, 따로 설정하지 않을 경우 1 로 가정합니다.

ST_SetValues — 지정한 밴드의 값들을 설정해서 나온 수정된 래스터를 반환합니다.

ST_DumpValues — 지정된 밴드의 값들을 2 차원 배열로 반환합니다.

ST_PixelOfValue — 검색 값과 일치하는 값을 가진 픽셀의 columnx, rowy 좌표를 반환합니다.

9.6.1. ST_PixelAsPolygon

ST_PixelAsPolygon — 특정 행 및 열에 대한 픽셀의 경계를 이루는 폴리곤 도형을 반환합니다.

Synopsis

geometry ST_PixelAsPolygon(raster rast, integer columnx, integer rowy);

설명: 특정 행 및 열에 대한 픽셀의 경계를 이루는 폴리곤 도형을 반환합니다.

2.0.0 버전부터 사용할 수 있습니다.

예시

```
-- 래스터 픽셀 폴리곤 획득
SELECT i,j, ST_AsText(ST_PixelAsPolygon(foo.rast, i,j)) As b1pgeom
FROM dummy_rast As foo
      CROSS JOIN generate_series(1,2) As i
      CROSS JOIN generate_series(1,1) As j
WHERE rid=2;

 i | j |                          b1pgeom
---+---+-------------------------------------------------------------------------------
```

```
  1 | 1 | POLYGON((3427927.75 5793244,3427927.8 5793244,3427927.8 5793243.95,...
  2 | 1 | POLYGON((3427927.8 5793244,3427927.85 5793244,3427927.85 5793243.95, ..
```

참고: ST_DumpAsPolygons, ST_PixelAsPolygons, ST_PixelAsPoint, ST_PixelAsPoints, ST_PixelAsCentroid, ST_PixelAsCentroids, ST_Intersection, ST_AsText

9.6.2. ST_PixelAsPolygons

ST_PixelAsPolygons — 래스터 밴드의 모든 픽셀의 경계를 이루는 폴리곤 도형을 각 픽셀의 값과 X, Y 래스터 좌표와 함께 반환합니다.

Synopsis

setof record **ST_PixelAsPolygons**(raster *rast*, integer *band=1*, boolean *exclude_nodata_value=TRUE*);

설명: 래스터 밴드의 모든 픽셀의 경계를 이루는 폴리곤 도형을 각 픽셀의 값(이중 정밀도 데이터형)과 X, Y 래스터 좌표(정수형)와 함께 반환합니다.



ST_PixelAsPolygons 함수는 모든 픽셀을 아우르는 폴리곤 도형 한 개를 반환합니다. 각 도형이 동일한 픽셀 값을 가지는 한 개 이상의 픽셀을 나타내는 ST_DumpAsPolygons 함수와는 다릅니다.



exclude_nodata_value = TRUE 일 경우, 픽셀 값이 NODATA 가 아닌 픽셀들만 폴리곤으로 반환합니다.

2.0.0 버전부터 사용할 수 있습니다.

개선 사항: 2.1.0 버전부터 선택적인 exclude_nodata_value 인수를 추가했습니다.

변경 사항: 2.1.1 버전에서 exclude_nodata_value 의 습성을 변경했습니다.

예시

```
 -- 래스터 픽셀 폴리곤 획득
 SELECT (gv).x, (gv).y, (gv).val, ST_AsText((gv).geom) geom
 FROM (SELECT ST_PixelAsPolygons(
         ST_SetValue(ST_SetValue(ST_AddBand(ST_MakeEmptyRaster(2, 2, 0, 0, 0.001, -0.001,
0.001, 0.001, 4269),
                           '8BUI'::text, 1, 0),
                     2, 2, 10),
               1, 1, NULL)
 ) gv
 ) foo;

 x | y | val |         geom
---+---+-----+--------------------------------------------------------------------------
 1 | 1 |     | POLYGON((0 0,0.001 0.001,0.002 0,0.001 -0.001,0 0))
 1 | 2 |   1 | POLYGON((0.001 -0.001,0.002 0,0.003 -0.001,0.002 -0.002,0.001 -0.001))
 2 | 1 |   1 | POLYGON((0.001 0.001,0.002 0.002,0.003 0.001,0.002 0,0.001 0.001))
 2 | 2 |  10 | POLYGON((0.002 0,0.003 0.001,0.004 0,0.003 -0.001,0.002 0))
```

참고: ST_DumpAsPolygons, ST_PixelAsPolygon, ST_PixelAsPoint, ST_PixelAsPoints, ST_PixelAsCentroid, ST_PixelAsCentroids, ST_AsText

9.6.3. ST_PixelAsPoint

ST_PixelAsPoint — 픽셀의 좌상단에 위치하는 포인트 도형을 반환합니다.

Synopsis

geometry ST_PixelAsPoint(raster *rast*, integer *columnx*, integer *rowy*);

설명: 픽셀의 좌상단에 위치하는 포인트 도형을 반환합니다.

2.1.0 버전부터 사용할 수 있습니다.

예시

```
SELECT ST_AsText(ST_PixelAsPoint(rast, 1, 1)) FROM dummy_rast WHERE rid = 1;

   st_astext
----------------
 POINT(0.5 0.5)
```

참고: ST_DumpAsPolygons, ST_PixelAsPolygon, ST_PixelAsPolygons, ST_PixelAsPoints, ST_PixelAsCentroid, ST_PixelAsCentroids

9.6.4. ST_Value

ST_Value — 입력 columnx, rowy 픽셀의 입력 밴드의 값, 또는 특정 기하학적 포인트의 밴드의 값을 반환합니다. 밴드 번호는 1 부터 시작하며, 따로 지정하지 않을 경우 1 로 가정합니다. exclude_nodata_value 를 거짓으로 설정할 경우, nodata 픽셀을 포함한 모든 픽셀이 교차한다고 가정하고 값을 반환합니다. exclude_nodata_value 를 설정하지 않은 경우, 래스터의 메타데이터에서 값을 읽어 들입니다.

Synopsis

double precision **ST_Value**(raster *rast*, geometry *pt*, boolean *exclude_nodata_value=true*);

double precision **ST_Value**(raster *rast*, integer *band*, geometry *pt*, boolean *exclude_nodata_value=true*);

double precision **ST_Value**(raster *rast*, integer *x*, integer *y*, boolean *exclude_nodata_value=true*);

double precision **ST_Value**(raster *rast*, integer *band*, integer *x*, integer *y*, boolean *exclude_nodata_value=true*);

설명: 입력 columnx, rowy 픽셀의 입력 밴드의 값, 또는 입력 포인트 도형의 밴드의 값을 반환합니다. 밴드 번호는 1 부터 시작하며, 따로 지정하지 않을 경우 1 로 가정합니다. exclude_nodata_value 를 참으로 설정할 경우, nodata 가 아닌 픽셀만 처리합니다. exclude_nodata_value 를 거짓으로 설정할 경우, 모든 픽셀을 처리합니다.

개선 사항: 2.0.0 버전부터 선택적인 exclude_nodata_value 인수를 추가했습니다.

예시

```
  -- 특정 PostGIS 도형 포인트 위치의 래스터 값을 얻습니다.
  -- 사용자 도형의 SRID 가 사용자 래스터와 동일해야 합니다.
  SELECT rid, ST_Value(rast, foo.pt_geom) As b1pval, ST_Value(rast, 2, foo.pt_geom) As
b2pval
  FROM dummy_rast CROSS JOIN (SELECT ST_SetSRID(ST_Point(3427927.77, 5793243.76), 0)
As pt_geom) As foo
  WHERE rid=2;

   rid | b1pval | b2pval
  -----+--------+--------
    2  |   252  |    79


  -- 실제 테이블을 이용한 일반적인 허구의 예시
  SELECT rid, ST_Value(rast, 3, sometable.geom) As b3pval
  FROM sometable
  WHERE ST_Intersects(rast,sometable.geom);

  SELECT rid, ST_Value(rast, 1, 1, 1) As b1pval,
     ST_Value(rast, 2, 1, 1) As b2pval, ST_Value(rast, 3, 1, 1) As b3pval
  FROM dummy_rast
  WHERE rid=2;

   rid | b1pval | b2pval | b3pval
  -----+--------+--------+--------
    2  |   253  |    78  |    70

  -- 각 픽셀의 밴드 1, 2, 3 의 모든 값들을 얻습니다. --
  SELECT x, y, ST_Value(rast, 1, x, y) As b1val,
        ST_Value(rast, 2, x, y) As b2val, ST_Value(rast, 3, x, y) As b3val
  FROM dummy_rast CROSS JOIN
  generate_series(1, 1000) As x CROSS JOIN generate_series(1, 1000) As y
  WHERE rid =  2 AND x <= ST_Width(rast) AND y <= ST_Height(rast);

   x | y | b1val | b2val | b3val
  ---+---+-------+-------+-------
   1 | 1 |   253 |    78 |    70
   1 | 2 |   253 |    96 |    80
   1 | 3 |   250 |    99 |    90
   1 | 4 |   251 |    89 |    77
   1 | 5 |   252 |    79 |    62
   2 | 1 |   254 |    98 |    86
   2 | 2 |   254 |   118 |   108
   :
   :

  -- 앞의 예시처럼 각 픽셀의 밴드 1, 2, 3 의 모든 값들을 얻지만 각 픽셀의 좌상단 포인트도 반환합니다. --
  SELECT ST_AsText(ST_SetSRID(
        ST_Point(ST_UpperLeftX(rast) + ST_ScaleX(rast)*x,
              ST_UpperLeftY(rast) + ST_ScaleY(rast)*y),
              ST_SRID(rast))) As uplpt
     , ST_Value(rast, 1, x, y) As b1val,
        ST_Value(rast, 2, x, y) As b2val, ST_Value(rast, 3, x, y) As b3val
  FROM dummy_rast CROSS JOIN
  generate_series(1,1000) As x CROSS JOIN generate_series(1,1000) As y
  WHERE rid =  2 AND x <= ST_Width(rast) AND y <= ST_Height(rast);

           uplpt          | b1val | b2val | b3val
  ----------------------------+-------+-------+-------
   POINT(3427929.25 5793245.5) |   253  |    78  |    70
   POINT(3427929.25 5793247)   |   253  |    96  |    80
   POINT(3427929.25 5793248.5) |   250  |    99  |    90
  :
```

```
  -- 특정 값 범위에 들어오며 특정 폴리곤과 교차하는 모든 픽셀들의 합집합으로 이루어진 폴리곤을 얻습니다.
--
  SELECT ST_AsText(ST_Union(pixpolyg)) As shadow
  FROM (SELECT ST_Translate(ST_MakeEnvelope(
              ST_UpperLeftX(rast), ST_UpperLeftY(rast),
                    ST_UpperLeftX(rast) + ST_ScaleX(rast),
                    ST_UpperLeftY(rast) + ST_ScaleY(rast), 0
                    ), ST_ScaleX(rast)*x, ST_ScaleY(rast)*y
              ) As pixpolyg, ST_Value(rast, 2, x, y) As b2val
        FROM dummy_rast CROSS JOIN
  generate_series(1,1000) As x CROSS JOIN generate_series(1,1000) As y
  WHERE rid =  2
        AND x <= ST_Width(rast) AND y <= ST_Height(rast)) As foo
  WHERE
        ST_Intersects(
              pixpolyg,
              ST_GeomFromText('POLYGON((3427928                    5793244,3427927.75
5793243.75,3427928 5793243.75,3427928 5793244))',0)
              ) AND b2val != 254;


              shadow
  ------------------------------------------------------------------------------------
-
   MULTIPOLYGON(((3427928           5793243.9,3427928           5793243.85,3427927.95
5793243.85,3427927.95 5793243.9,
   3427927.95    5793243.95,3427928    5793243.95,3427928.05    5793243.95,3427928.05
5793243.9,3427928 5793243.9)),((3427927.95 5793243.9,3427927.95 579324
  3.85,3427927.9   5793243.85,3427927.85   5793243.85,3427927.85   5793243.9,3427927.9
5793243.9,3427927.9 5793243.95,
  3427927.95   5793243.95,3427927.95   5793243.9)),((3427927.85   5793243.75,3427927.85
5793243.7,3427927.8 5793243.7,3427927.8 5793243.75
  ,3427927.8    5793243.8,3427927.8    5793243.85,3427927.85    5793243.85,3427927.85
5793243.8,3427927.85 5793243.75)),
  ((3427928.05    5793243.75,3427928.05    5793243.7,3427928    5793243.7,3427927.95
5793243.7,3427927.95 5793243.75,3427927.95 5793243.8,3427
  927.95 5793243.85,3427928 5793243.85,3427928 5793243.8,3427928.05 5793243.8,
  3427928.05   5793243.75)),((3427927.95   5793243.75,3427927.95   5793243.7,3427927.9
5793243.7,3427927.85 5793243.7,
  3427927.85    5793243.75,3427927.85    5793243.8,3427927.85    5793243.85,3427927.9
5793243.85,
  3427927.95 5793243.85,3427927.95 5793243.8,3427927.95 5793243.75)))

  -- 대용량 래스터 타일의 모든 픽셀을 확인하는 작업은 오래 걸릴 수 있습니다.
  -- 단계별로 선택할 수 있는 generate_series 파라미터를 이용해서
  -- 자릿수를 통해 정확도를 조금 줄이면 속도를 크게 향상시킬 수 있습니다.
  -- 다음 예시는 이전 예시와 같은 작업을 하지만 매 4(2x2)픽셀마다 픽셀 1 개를 확인해서 확인한 것으로
처리합니다.
  -- 확인된 픽셀의 값을 그 다음 픽셀 4 개의 값으로 저장합니다.

  SELECT ST_AsText(ST_Union(pixpolyg)) As shadow
  FROM (SELECT ST_Translate(ST_MakeEnvelope(
              ST_UpperLeftX(rast), ST_UpperLeftY(rast),
                    ST_UpperLeftX(rast) + ST_ScaleX(rast)*2,
                    ST_UpperLeftY(rast) + ST_ScaleY(rast)*2, 0
                    ), ST_ScaleX(rast)*x, ST_ScaleY(rast)*y
              ) As pixpolyg, ST_Value(rast, 2, x, y) As b2val
        FROM dummy_rast CROSS JOIN
  generate_series(1,1000,2) As x CROSS JOIN generate_series(1,1000,2) As y
  WHERE rid =  2
        AND x <= ST_Width(rast)  AND y <= ST_Height(rast)  ) As foo
  WHERE
        ST_Intersects(
              pixpolyg,
              ST_GeomFromText('POLYGON((3427928                    5793244,3427927.75
5793243.75,3427928 5793243.75,3427928 5793244))',0)
```

```
                ) AND b2val != 254;

                shadow
  ------------------------------------------------------------------------------------
-
   MULTIPOLYGON(((3427927.9 5793243.85,3427927.8 5793243.85,3427927.8 5793243.95,
   3427927.9     5793243.95,3427928     5793243.95,3427928.1     5793243.95,3427928.1
5793243.85,3427928 5793243.85,3427927.9 5793243.85)),
   ((3427927.9    5793243.65,3427927.8    5793243.65,3427927.8    5793243.75,3427927.8
5793243.85,3427927.9 5793243.85,
   3427928      5793243.85,3427928      5793243.75,3427928.1      5793243.75,3427928.1
5793243.65,3427928 5793243.65,3427927.9 5793243.65)))
```

참고: ST_SetValue, ST_DumpAsPolygons, ST_NumBands, ST_PixelAsPolygon, ST_ScaleX, ST_ScaleY, ST_UpperLeftX, ST_UpperLeftY, ST_SRID, ST_AsText, ST_Point, ST_MakeEnvelope, ST_Intersects, ST_Intersection

9.6.5. ST_NearestValue

ST_NearestValue — columnx 및 rowy, 또는 래스터와 동일한 공간 참조 좌표 시스템 단위로 표현된 기하학적 포인트로 지정된 입력 밴드의 픽셀에 가장 가까운 NODATA 가 아닌 값을 반환합니다.

Synopsis

double precision ST_NearestValue(raster *rast*, integer *bandnum*, geometry *pt*, boolean *exclude_nodata_value=true*);

double precision ST_NearestValue(raster *rast*, geometry *pt*, boolean *exclude_nodata_value=true*);

double precision ST_NearestValue(raster *rast*, integer *bandnum*, integer *columnx*, integer *rowy*, boolean *exclude_nodata_value=true*);

double precision ST_NearestValue(raster *rast*, integer *columnx*, integer *rowy*, boolean *exclude_nodata_value=true*);

설명: 입력한 columnx, rowy 픽셀, 또는 특정 기하학적 포인트 위치에 있는 입력 밴드에 가장 가까운 NODATA 가 아닌 값을 반환합니다. columnx, rowy 픽셀 또는 특정 기하학적 포인트 위치의 픽셀이 NODATA 일 경우, 이 함수는 columnx, rowy 픽셀 또는 특정 기하학적 포인트 위치에 가장 가까우며 NODATA 가 아닌 픽셀을 찾을 것입니다.

밴드 번호는 1 부터 시작하며, 따로 지정하지 않을 경우 bandnum 을 1 로 가정합니다. exclude_nodata_value 를 거짓으로 설정할 경우, nodata 픽셀을 포함한 모든 픽셀이 교차한다고 가정하고 값을 반환합니다. exclude_nodata_value 를 설정하지 않은 경우, 래스터의 메타데이터에서 값을 읽어 들입니다.

2.1.0 버전부터 사용할 수 있습니다.



ST_NearestValue 함수는 즉석에서 ST_Value 를 대체할 수 있습니다.

예시

```
-- 2x2 픽셀이 값을 가지고 있습니다.
```

```
  SELECT
        ST_Value(rast, 2, 2) AS value,
        ST_NearestValue(rast, 2, 2) AS nearestvalue
  FROM (
        SELECT
              ST_SetValue(
                    ST_SetValue(
                          ST_SetValue(
                                ST_SetValue(
                                      ST_SetValue(
                                            ST_AddBand(
                                                  ST_MakeEmptyRaster(5, 5,
-2, 2, 1, -1, 0, 0, 0),
                                                  '8BUI'::text, 1, 0
                                            ),
                                            1, 1, 0.
                                      ),
                                      2, 3, 0.
                                ),
                                3, 5, 0.
                          ),
                          4, 2, 0.
                    ),
                    5, 4, 0.
              ) AS rast
  ) AS foo

   value | nearestvalue
  -------+--------------
       1 |            1

  -- 2x3 픽셀이 NODATA 입니다.
  SELECT
        ST_Value(rast, 2, 3) AS value,
        ST_NearestValue(rast, 2, 3) AS nearestvalue
  FROM (
        SELECT
              ST_SetValue(
                    ST_SetValue(
                          ST_SetValue(
                                ST_SetValue(
                                      ST_SetValue(
                                            ST_AddBand(
                                                  ST_MakeEmptyRaster(5, 5,
-2, 2, 1, -1, 0, 0, 0),
                                                  '8BUI'::text, 1, 0
                                            ),
                                            1, 1, 0.
                                      ),
                                      2, 3, 0.
                                ),
                                3, 5, 0.
                          ),
                          4, 2, 0.
                    ),
                    5, 4, 0.
              ) AS rast
  ) AS foo

   value | nearestvalue
  -------+--------------
         |            1
```

참고: ST_Neighborhood, ST_Value

9.6.6. ST_Neighborhood

ST_Neighborhood — columnx 및 rowy, 또는 래스터와 동일한 공간 참조 좌표 시스템 단위로 표현된 기하학적 포인트로 지정된 입력 밴드의 픽셀 주위의 NODATA 가 아닌 값들의 이중 정밀도 데이터형 2 차원 배열을 반환합니다.

Synopsis

double precision[ ][ ] ST_Neighborhood(raster *rast*, integer *bandnum*, integer *columnX*, integer *rowY*, integer *distanceX*, integer *distanceY*, boolean *exclude_nodata_value=true*);

double precision[ ][ ] ST_Neighborhood(raster *rast*, integer *columnX*, integer *rowY*, integer *distanceX*, integer *distanceY*, boolean *exclude_nodata_value=true*);

double precision[ ][ ] ST_Neighborhood(raster *rast*, integer *bandnum*, geometry *pt*, integer *distanceX*, integer *distanceY*, boolean *exclude_nodata_value=true*);

double precision[ ][ ] ST_Neighborhood(raster *rast*, geometry *pt*, integer *distanceX*, integer *distanceY*, boolean *exclude_nodata_value=true*);

설명: columnx 및 rowy, 또는 래스터와 동일한 공간 참조 좌표 시스템 단위로 표현된 기하학적 포인트로 지정된 입력 밴드의 픽셀 주위의 NODATA 가 아닌 값들의 이중 정밀도 데이터형 2차원 배열을 반환합니다. distanceX 및 distanceY 파라미터가 지정한 픽셀 주위의 픽셀 개수를 X 및 Y 축으로 정의합니다. 예를 들어, 사용자 설정 픽셀 주위로 X 축을 따라 3 픽셀 거리 안 그리고 Y 축을 따라 2 픽셀 거리 안의 모든 값을 원하는 경우 말입니다. 2 차원 배열의 중심값은 columnx 및 rowy 또는 기하학적 포인트로 지정된 픽셀의 값이 될 것입니다.

밴드 번호는 1 부터 시작하며, 따로 지정하지 않을 경우 bandnum 을 1 로 가정합니다. exclude_nodata_value 를 거짓으로 설정할 경우, nodata 픽셀을 포함한 모든 픽셀이 교차한다고 가정하고 값을 반환합니다. exclude_nodata_value 를 설정하지 않은 경우, 래스터의 메타데이터에서 값을 읽어 들입니다.



반환되는 2 차원 배열의 각 축의 구성 요소 개수는 2 * (distanceX|distanceY) + 1 입니다. 따라서 distanceX 와 distanceY 가 각각 1 인 경우, 반환되는 배열은 3x3 이 될 것입니다.



ST_Min4ma, ST_Sum4ma, ST_Mean4ma 같은 어떤 래스터 공간 처리 내장 함수도 이 2 차원 배열 출력물을 입력받을 수 있습니다.

2.1.0 버전부터 사용할 수 있습니다.

예시

```
-- 2x2 픽셀이 값을 가지고 있습니다.
SELECT
      ST_Neighborhood(rast, 2, 2, 1, 1)
FROM (
      SELECT
            ST_SetValues(
                  ST_AddBand(
                        ST_MakeEmptyRaster(5, 5, -2, 2, 1, -1, 0, 0, 0),
                        '8BUI'::text, 1, 0
                  ),
                  1, 1, 1, ARRAY[
                        [0, 1, 1, 1, 1],
                        [1, 1, 1, 0, 1],
                        [1, 0, 1, 1, 1],
                        [1, 1, 1, 1, 0],
                        [1, 1, 0, 1, 1]
                  ]::double precision[],
                  1
            ) AS rast
) AS foo

       st_neighborhood
--------------------------------
 {{NULL,1,1},{1,1,NULL},{1,1,1}}

-- 2x3 픽셀이 NODATA 입니다.
SELECT
      ST_Neighborhood(rast, 2, 3, 1, 1)
FROM (
      SELECT
            ST_SetValues(
                  ST_AddBand(
                        ST_MakeEmptyRaster(5, 5, -2, 2, 1, -1, 0, 0, 0),
                        '8BUI'::text, 1, 0
                  ),
                  1, 1, 1, ARRAY[
                        [0, 1, 1, 1, 1],
                        [1, 1, 1, 0, 1],
                        [1, 0, 1, 1, 1],
                        [1, 1, 1, 1, 0],
                        [1, 1, 0, 1, 1]
                  ]::double precision[],
                  1
            ) AS rast
) AS foo

     st_neighborhood
-----------------------------
 {{1,1,1},{1,NULL,1},{1,1,1}}

-- 3x3 픽셀이 값을 가지고 있습니다.
-- exclude_nodata_value = FALSE
SELECT
      ST_Neighborhood(rast, 3, 3, 1, 1, false)
FROM (
            ST_SetValues(
                  ST_AddBand(
                        ST_MakeEmptyRaster(5, 5, -2, 2, 1, -1, 0, 0, 0),
                        '8BUI'::text, 1, 0
                  ),
                  1, 1, 1, ARRAY[
                        [0, 1, 1, 1, 1],
                        [1, 1, 1, 0, 1],
                        [1, 0, 1, 1, 1],
                        [1, 1, 1, 1, 0],
                        [1, 1, 0, 1, 1]
                  ]::double precision[],
                  1
            ) AS rast
) AS foo

     st_neighborhood
```

```
--------------------------
 {{1,0,1},{1,1,1},{0,1,1}}
```

참고: ST_NearestValue, ST_Min4ma, ST_Max4ma, ST_Sum4ma, ST_Mean4ma, ST_Range4ma, ST_Distinct4ma, ST_StdDev4ma

9.6.7. ST_SetValue

ST_SetValue — 입력한 columnx, rowy 픽셀 또는 특정 도형과 교차하는 픽셀들의 위치의 밴드 값을 설정해서 나온 수정된 래스터를 반환합니다. 밴드 번호는 1 부터 시작하며, 따로 설정하지 않을 경우 1 로 가정합니다.

Synopsis

raster **ST_SetValue**(raster *rast*, integer *bandnum*, geometry *geom*, double precision *newvalue*);

raster **ST_SetValue**(raster *rast*, geometry *geom*, double precision *newvalue*);

raster **ST_SetValue**(raster *rast*, integer *bandnum*, integer *columnx*, integer *rowy*, double precision *newvalue*);

raster **ST_SetValue**(raster *rast*, integer *columnx*, integer *rowy*, double precision *newvalue*);

설명: 래스터의 columnx, rowy 또는 도형으로 지정된 픽셀 위치의 밴드에 대해, 지정된 픽셀 값을 새 값으로 설정해서 나온 수정된 래스터를 반환합니다. 밴드를 따로 설정하지 않을 경우, 밴드 1 로 가정합니다.

개선 사항: 2.1.0 버전부터 ST_SetValue() 함수의 도형 변종이 포인트뿐만이 아니라 모든 도형 유형을 지원합니다. 이 도형 변종은 ST_SetValues()의 geomval[] 변종을 감싸는 래퍼(wrapper)입니다.

예시

```
-- 도형 예시
SELECT (foo.geomval).val, ST_AsText(ST_Union((foo.geomval).geom))
FROM (SELECT ST_DumpAsPolygons(
             ST_SetValue(rast,1,
                         ST_Point(3427927.75, 5793243.95),
                         50)
                  ) As geomval
FROM dummy_rast
where rid = 2) As foo
WHERE (foo.geomval).val < 250
GROUP BY (foo.geomval).val;

 val |                                                  st_astext
-----+--------------------------------------------------------------------------
  50 | POLYGON((3427927.75 5793244,3427927.75 5793243.95,3427927.8 579324 ...
 249 | POLYGON((3427927.95 5793243.95,3427927.95 5793243.85,3427928 57932 ...
```

```
-- 변경된 래스터를 저장 --
    UPDATE dummy_rast SET rast = ST_SetValue(rast,1, ST_Point(3427927.75, 5793243.95),100)
        WHERE rid = 2 ;
```

참고: ST_Value, ST_DumpAsPolygons

9.6.8. ST_SetValues

ST_SetValues — 지정한 밴드의 값들을 설정해서 나온 수정된 래스터를 반환합니다.

Synopsis

raster ST_SetValues(raster *rast*, integer *nband*, integer *columnx*, integer *rowy*, double precision[][] *newvalueset*, boolean[][] *noset=NULL*, boolean *keepnodata=FALSE*);

raster ST_SetValues(raster *rast*, integer *nband*, integer *columnx*, integer *rowy*, double precision[][] *newvalueset*, double precision *nosetvalue*, boolean *keepnodata=FALSE*);

raster ST_SetValues(raster *rast*, integer *nband*, integer *columnx*, integer *rowy*, integer *width*, integer *height*, double precision *newvalue*, boolean *keepnodata=FALSE*);

raster ST_SetValues(raster *rast*, integer *columnx*, integer *rowy*, integer *width*, integer *height*, double precision *newvalue*, boolean *keepnodata=FALSE*);

raster ST_SetValues(raster *rast*, integer *nband*, geomval[] *geomvalset*, boolean *keepnodata=FALSE*);

설명: 지정한 밴드에 대해, 설정한 픽셀들을 새로운 값(들)로 설정해서 나온 수정된 래스터를 반환합니다.

keepnodata 가 TRUE 일 경우, NODATA 값인 픽셀들을 newvalueset 에 있는 상응하는 값으로 설정하지 않을 것입니다.

변종 1 의 경우, columnx, rowy 픽셀 좌표 및 newvalueset 배열의 차원이 설정 대상 픽셀들을 결정합니다. 이미 newvalueset 에 있는 값들을 가진 픽셀을 중복 설정하는 것을 막기 위해 noset 을 이용할 수 있습니다(PostgreSQL 이 이가 빠진 배열을 허용하지 않기 때문입니다). 변종 1 예시를 참조하십시오.

변종 2 는 변종 1 과 비슷하지만, 불 값인 noset 배열 대신 단순한 이중 정밀도 데이터형인 nosetvalue 를 입력받습니다. newvalueset 가운데 nosetvalue 값을 가진 구성 요소를 무시할 것입니다. 변종 2 예시를 참조하십시오.

변종 3 의 경우, columnx, rowy 픽셀 좌표, width 및 height 가 설정 대상 픽셀들을 결정합니다. 변종 3 예시를 참조하십시오.

변종 4 는 rast 의 첫 번째 밴드의 픽셀들이 설정될 것이라고 가정하는 점만 제외하면 변종 3 과 동일합니다.

변종 5 의 경우, 설정 대상 픽셀들을 결정하는 데 geomval 의 배열을 이용합니다. 배열 안의 도형들이 모두 POINT 또는 MULTIPOINT 일 경우, 이 함수는 각 포인트의 경도와 위도를 써서 직접 픽셀을 설정하는

지름길을 이용합니다. 그렇지 않을 경우, 도형을 래스터로 변환한 다음 동일한 방식으로 반복합니다. 변종 5 예시를 참조하십시오.

2.1.0 버전부터 사용할 수 있습니다.

예시: 변종 1

```
/*
ST_SetValues() 함수가 다음 작업을 합니다.

+ - + - + - +        + - + - + - +
| 1 | 1 | 1 |        | 1 | 1 | 1 |
+ - + - + - +        + - + - + - +
| 1 | 1 | 1 |   =
>   | 1 | 9 | 9 |
+ - + - + - +        + - + - + - +
| 1 | 1 | 1 |        | 1 | 9 | 9 |
+ - + - + - +        + - + - + - +
*/
SELECT
       (poly).x,
       (poly).y,
       (poly).val
FROM (
SELECT
       ST_PixelAsPolygons(
              ST_SetValues(
                     ST_AddBand(
                            ST_MakeEmptyRaster(3, 3, 0, 0, 1, -1, 0, 0, 0),
                            1, '8BUI', 1, 0
                     ),
                     1, 2, 2, ARRAY[[9, 9], [9, 9]]::double precision[][]
              )
       ) AS poly
) foo
ORDER BY 1, 2;

 x | y | val
---+---+-----
 1 | 1 |   1
 1 | 2 |   1
 1 | 3 |   1
 2 | 1 |   1
 2 | 2 |   9
 2 | 3 |   9
 3 | 1 |   1
 3 | 2 |   9
 3 | 3 |   9

/*
ST_SetValues() 함수가 다음 작업을 합니다.

+ - + - + - +        + - + - + - +
| 1 | 1 | 1 |        | 9 | 9 | 9 |
+ - + - + - +        + - + - + - +
| 1 | 1 | 1 |   =
>   | 9 |   | 9 |
+ - + - + - +        + - + - + - +
| 1 | 1 | 1 |        | 9 | 9 | 9 |
+ - + - + - +        + - + - + - +
*/
SELECT
       (poly).x,
       (poly).y,
       (poly).val
FROM (
SELECT
       ST_PixelAsPolygons(
              ST_SetValues(
                     ST_AddBand(
                            ST_MakeEmptyRaster(3, 3, 0, 0, 1, -1, 0, 0, 0),
```

```
                        1, '8BUI', 1, 0
                    ),
                    1, 1, 1, ARRAY[[9, 9, 9], [9, NULL, 9], [9, 9, 9]]::double precision[][]
             )
       ) AS poly
) foo
ORDER BY 1, 2;

 x | y | val
---+---+-----
 1 | 1 |   9
 1 | 2 |   9
 1 | 3 |   9
 2 | 1 |   9
 2 | 2 |
 2 | 3 |   9
 3 | 1 |   9
 3 | 2 |   9
 3 | 3 |   9

/*
ST_SetValues() 함수가 다음 작업을 합니다.

+ - + - + - +        + - + - + - +
| 1 | 1 | 1 |        | 9 | 9 | 9 |
+ - + - + - +        + - + - + - +
| 1 | 1 | 1 |   =
>   | 1 |   | 9 |
+ - + - + - +        + - + - + - +
| 1 | 1 | 1 |        | 9 | 9 | 9 |
+ - + - + - +        + - + - + - +
*/
SELECT
       (poly).x,
       (poly).y,
       (poly).val
FROM (
SELECT
       ST_PixelAsPolygons(
              ST_SetValues(
                    ST_AddBand(
                           ST_MakeEmptyRaster(3, 3, 0, 0, 1, -1, 0, 0, 0),
                           1, '8BUI', 1, 0
                    ),
                    1, 1, 1,
                           ARRAY[[9, 9, 9], [9, NULL, 9], [9, 9, 9]]::double precision[][],
                           ARRAY[[false], [true]]::boolean[][]
              )
       ) AS poly
) foo
ORDER BY 1, 2;

 x | y | val
---+---+-----
 1 | 1 |   9
 1 | 2 |   1
 1 | 3 |   9
 2 | 1 |   9
 2 | 2 |
 2 | 3 |   9
 3 | 1 |   9
 3 | 2 |   9
 3 | 3 |   9

/*
ST_SetValues() 함수가 다음 작업을 합니다.

+ - + - + - +        + - + - + - +
|   | 1 | 1 |        |   | 9 | 9 |
+ - + - + - +        + - + - + - +
| 1 | 1 | 1 |   =
>   | 1 |   | 9 |
+ - + - + - +        + - + - + - +
```

```
  | 1 | 1 | 1 |        | 9 | 9 | 9 |
  + - + - + - +        + - + - + - +
  */
  SELECT
        (poly).x,
        (poly).y,
        (poly).val
  FROM (
  SELECT
        ST_PixelAsPolygons(
              ST_SetValues(
                    ST_SetValue(
                          ST_AddBand(
                                ST_MakeEmptyRaster(3, 3, 0, 0, 1, -1, 0, 0, 0),
                                1, '8BUI', 1, 0
                          ),
                          1, 1, 1, NULL
                    ),
                    1, 1, 1,
                          ARRAY[[9, 9, 9], [9, NULL, 9], [9, 9, 9]]::double precision[][],
                          ARRAY[[false], [true]]::boolean[][],
                          TRUE
              )
        ) AS poly
  ) foo
  ORDER BY 1, 2;

   x | y | val
  ---+---+-----
   1 | 1 |
   1 | 2 |   1
   1 | 3 |   9
   2 | 1 |   9
   2 | 2 |
   2 | 3 |   9
   3 | 1 |   9
   3 | 2 |   9
   3 | 3 |   9
```

예시: 변종 2

```
  /*
  ST_SetValues() 함수가 다음 작업을 합니다.

  + - + - + - +        + - + - + - +
  | 1 | 1 | 1 |        | 1 | 1 | 1 |
  + - + - + - +        + - + - + - +
  | 1 | 1 | 1 |   =
  >   | 1 | 9 | 9 |
  + - + - + - +        + - + - + - +
  | 1 | 1 | 1 |        | 1 | 9 | 9 |
  + - + - + - +        + - + - + - +
  */
  SELECT
        (poly).x,
        (poly).y,
        (poly).val
  FROM (
  SELECT
        ST_PixelAsPolygons(
              ST_SetValues(
                    ST_AddBand(
                          ST_MakeEmptyRaster(3, 3, 0, 0, 1, -1, 0, 0, 0),
                          1, '8BUI', 1, 0
                    ),
                    1, 1, 1, ARRAY[[-1, -1, -1], [-1, 9, 9], [-1, 9, 9]]::double precision[][],
-1
              )
        ) AS poly
  ) foo
  ORDER BY 1, 2;
```

```
 x | y | val
---+---+-----
 1 | 1 |   1
 1 | 2 |   1
 1 | 3 |   1
 2 | 1 |   1
 2 | 2 |   9
 2 | 3 |   9
 3 | 1 |   1
 3 | 2 |   9
 3 | 3 |   9

/*
이 예시는 이전 예시와 비슷하지만, nosetvalue = -1 대신 nosetvalue = NULL 을 씁니다.

ST_SetValues() 함수가 다음 작업을 합니다.

+ - + - + - +        + - + - + - +
| 1 | 1 | 1 |        | 1 | 1 | 1 |
+ - + - + - +        + - + - + - +
| 1 | 1 | 1 |   =
>   | 1 | 9 | 9 |
+ - + - + - +        + - + - + - +
| 1 | 1 | 1 |        | 1 | 9 | 9 |
+ - + - + - +        + - + - + - +
*/
SELECT
       (poly).x,
       (poly).y,
       (poly).val
FROM (
SELECT
       ST_PixelAsPolygons(
             ST_SetValues(
                   ST_AddBand(
                         ST_MakeEmptyRaster(3, 3, 0, 0, 1, -1, 0, 0, 0),
                         1, '8BUI', 1, 0
                   ),
                   1, 1, 1, ARRAY[[NULL, NULL, NULL], [NULL, 9, 9], [NULL, 9, 9]]::double
precision[][], NULL::double precision
             )
       ) AS poly
) foo
ORDER BY 1, 2;

 x | y | val
---+---+-----
 1 | 1 |   1
 1 | 2 |   1
 1 | 3 |   1
 2 | 1 |   1
 2 | 2 |   9
 2 | 3 |   9
 3 | 1 |   1
 3 | 2 |   9
 3 | 3 |   9
```

예시: 변종 3

```
/*
ST_SetValues() 함수가 다음 작업을 합니다.

+ - + - + - +        + - + - + - +
| 1 | 1 | 1 |        | 1 | 1 | 1 |
+ - + - + - +        + - + - + - +
| 1 | 1 | 1 |   =
>   | 1 | 9 | 9 |
+ - + - + - +        + - + - + - +
| 1 | 1 | 1 |        | 1 | 9 | 9 |
```

```
+ - + - + - +        + - + - + - +
*/
SELECT
       (poly).x,
       (poly).y,
       (poly).val
FROM (
SELECT
       ST_PixelAsPolygons(
             ST_SetValues(
                   ST_AddBand(
                         ST_MakeEmptyRaster(3, 3, 0, 0, 1, -1, 0, 0, 0),
                         1, '8BUI', 1, 0
                   ),
                   1, 2, 2, 2, 2, 9
             )
       ) AS poly
) foo
ORDER BY 1, 2;

 x | y | val
---+---+-----
 1 | 1 |   1
 1 | 2 |   1
 1 | 3 |   1
 2 | 1 |   1
 2 | 2 |   9
 2 | 3 |   9
 3 | 1 |   1
 3 | 2 |   9
 3 | 3 |   9

/*
ST_SetValues() 함수가 다음 작업을 합니다.

+ - + - + - +        + - + - + - +
| 1 | 1 | 1 |        | 1 | 1 | 1 |
+ - + - + - +        + - + - + - +
| 1 |   | 1 |   =
>   | 1 |   | 9 |
+ - + - + - +        + - + - + - +
| 1 | 1 | 1 |        | 1 | 9 | 9 |
+ - + - + - +        + - + - + - +
*/
SELECT
       (poly).x,
       (poly).y,
       (poly).val
FROM (
SELECT
       ST_PixelAsPolygons(
             ST_SetValues(
                   ST_SetValue(
                         ST_AddBand(
                               ST_MakeEmptyRaster(3, 3, 0, 0, 1, -1, 0, 0, 0),
                               1, '8BUI', 1, 0
                         ),
                         1, 2, 2, NULL
                   ),
                   1, 2, 2, 2, 2, 9, TRUE
             )
       ) AS poly
) foo
ORDER BY 1, 2;

 x | y | val
---+---+-----
 1 | 1 |   1
 1 | 2 |   1
 1 | 3 |   1
 2 | 1 |   1
 2 | 2 |
 2 | 3 |   9
```

```
   3 | 1 |   1
   3 | 2 |   9
   3 | 3 |   9
```

예시: 변종 5

```
  WITH foo AS (
        SELECT 1 AS rid, ST_AddBand(ST_MakeEmptyRaster(5, 5, 0, 0, 1, -1, 0, 0, 0), 1,
'8BUI', 0, 0) AS rast
  ), bar AS (
        SELECT 1 AS gid, 'SRID=0;POINT(2.5 -2.5)'::geometry geom UNION ALL
        SELECT 2 AS gid, 'SRID=0;POLYGON((1 -1, 4 -1, 4 -4, 1 -4, 1 -1))'::geometry geom
UNION ALL
        SELECT 3 AS gid, 'SRID=0;POLYGON((0 0, 5 0, 5 -1, 1 -1, 1 -4, 0 -4, 0 0))'::geometry
geom UNION ALL
        SELECT 4 AS gid, 'SRID=0;MULTIPOINT(0 0, 4 4, 4 -4)'::geometry
  )
  SELECT
        rid, gid, ST_DumpValues(ST_SetValue(rast, 1, geom, gid))
  FROM foo t1
  CROSS JOIN bar t2
  ORDER BY rid, gid;

  rid | gid |                      st_dumpvalues
 -----+-----+-----------------------------------------------------------------------------------------------------------------------
---
   1 | 1 |
(1,"{{NULL,NULL,NULL,NULL,NULL},{NULL,NULL,NULL,NULL,NULL},{NULL,NULL,1,NULL,NULL},{NULL,NULL,NULL,NULL,NULL},{NULL,NULL,NULL,NULL,NULL}}")
   1 | 2 | (1,"{{NULL,NULL,NULL,NULL,NULL},{NULL,2,2,2,NULL},{NULL,2,2,2,NULL},{NULL,2,2,2,NULL},{NULL,NULL,NULL,NULL,NULL}}")
   1 | 3 | (1,"{{3,3,3,3,3},{3,NULL,NULL,NULL,NULL},{3,NULL,NULL,NULL,NULL},{3,NULL,NULL,NULL,NULL},{NULL,NULL,NULL,NULL,NULL}}")
   1 | 4 |
(1,"{{4,NULL,NULL,NULL,NULL},{NULL,NULL,NULL,NULL,NULL},{NULL,NULL,NULL,NULL,NULL},{NULL,NULL,NULL,NULL,NULL},{NULL,NULL,NULL,NULL,4}}")
  (4 rows)
```

다음은 배열 후반에 있는 geomvals 가 전반의 geomvals 를 덮어 쓸 수 있다는 사실을 보여주는 예시입니다.

이전 예시와 반대되는 상황을 보여주는 예시입니다.

```
WITH foo AS (
      SELECT 1 AS rid, ST_AddBand(ST_MakeEmptyRaster(5, 5, 0, 0, 1, -1, 0, 0, 0), 1, '8BUI',
0, 0) AS rast
), bar AS (
      SELECT 1 AS gid, 'SRID=0;POINT(2.5 -2.5)'::geometry geom UNION ALL
      SELECT 2 AS gid, 'SRID=0;POLYGON((1 -1, 4 -1, 4 -4, 1 -4, 1 -1))'::geometry geom
UNION ALL
      SELECT 3 AS gid, 'SRID=0;POLYGON((0 0, 5 0, 5 -1, 1 -1, 1 -4, 0 -4, 0 0))'::geometry
geom UNION ALL
      SELECT 4 AS gid, 'SRID=0;MULTIPOINT(0 0, 4 4, 4 -4)'::geometry
)
SELECT
      t1.rid, t2.gid, t3.gid, ST_DumpValues(ST_SetValues(rast, 1, ARRAY[ROW(t2.geom,
t2.gid), ROW(t3.geom, t3.gid)]::geomval[]))
FROM foo t1
CROSS JOIN bar t2
CROSS JOIN bar t3
WHERE t2.gid = 2
      AND t3.gid = 1
ORDER BY t1.rid, t2.gid, t3.gid;

 rid | gid | gid |                              st_dumpvalues
-----+-----+-----+-----------------------------------------------------------------------
   1 |   2 |   1 |
(1,"{{NULL,NULL,NULL,NULL,NULL},{NULL,2,2,2,NULL},{NULL,2,1,2,NULL},{NULL,2,2,2,NULL},{NUL
L,NULL,NULL,NULL,NULL}}")
(1 row)
```

참고: ST_Value, ST_SetValue, ST_PixelAsPolygons

9.6.9. ST_DumpValues

ST_DumpValues — 지정된 밴드의 값들을 2 차원 배열로 반환합니다.

Synopsis

setof record ST_DumpValues(raster *rast* , integer[] *nband=NULL* , boolean *exclude_nodata_value=true*);

double precision[][] ST_DumpValues(raster *rast* , integer *nband* , boolean *exclude_nodata_value=true*);

설명: 지정된 밴드의 값들을 2 차원 배열로 반환합니다(첫 번째 인덱스가 행, 두 번째 인덱스가 열입니다). nband 가 NULL 이거나 설정되지 않은 경우, 래스터의 모든 밴드를 처리합니다.

2.1.0 버전부터 사용할 수 있습니다.

예시

```
  WITH foo AS (
        SELECT ST_AddBand(ST_AddBand(ST_AddBand(ST_MakeEmptyRaster(3, 3, 0, 0, 1, -1, 0,
0, 0), 1, '8BUI', 1, 0), 2, '32BF', 3, -9999), 3, '16BSI', 0, 0) AS rast
  )
  SELECT
        (ST_DumpValues(rast)).*
  FROM foo;

   nband |                  valarray
  -------+-----------------------------------------------------
       1 | {{1,1,1},{1,1,1},{1,1,1}}
       2 | {{3,3,3},{3,3,3},{3,3,3}}
       3 | {{NULL,NULL,NULL},{NULL,NULL,NULL},{NULL,NULL,NULL}}
  (3 rows)

  WITH foo AS (
        SELECT ST_AddBand(ST_AddBand(ST_AddBand(ST_MakeEmptyRaster(3, 3, 0, 0, 1, -1, 0,
0, 0), 1, '8BUI', 1, 0), 2, '32BF', 3, -9999), 3, '16BSI', 0, 0) AS rast
  )
  SELECT
        (ST_DumpValues(rast, ARRAY[3, 1])).*
  FROM foo;

   nband |                  valarray
  -------+-----------------------------------------------------
       3 | {{NULL,NULL,NULL},{NULL,NULL,NULL},{NULL,NULL,NULL}}
       1 | {{1,1,1},{1,1,1},{1,1,1}}
  (2 rows)

  WITH foo AS (
        SELECT ST_SetValue(ST_AddBand(ST_MakeEmptyRaster(3, 3, 0, 0, 1, -1, 0, 0, 0), 1,
'8BUI', 1, 0), 1, 2, 5) AS rast
  )
  SELECT
        (ST_DumpValues(rast, 1))[2][1]
  FROM foo;

   st_dumpvalues
  ---------------
               5
  (1 row)
```

참고: ST_Value, ST_SetValue, ST_SetValues

9.6.10. ST_PixelOfValue

ST_PixelOfValue — 검색 값과 일치하는 값을 가진 픽셀의 columnx, rowy 좌표를 반환합니다.

Synopsis

setof record ST_PixelOfValue(raster *rast* , integer *nband* , double precision[] *search* , boolean *exclude_nodata_value=true*);

setof record ST_PixelOfValue(raster *rast* , double precision[] *search* , boolean *exclude_nodata_value=true*);

setof record ST_PixelOfValue(raster *rast* , integer *nband* , double precision *search* , boolean *exclude_nodata_value=true*);

setof record ST_PixelOfValue(raster *rast* , double precision *search* , boolean *exclude_nodata_value=true*);

설명: 검색 값과 일치하는 값을 가진 픽셀의 columnx, rowy 좌표를 반환합니다. 밴드를 따로 설정하지 않을 경우, 밴드 1 로 가정합니다.

2.1.0 버전부터 사용할 수 있습니다.

예시

```
SELECT
       (pixels).*
FROM (
       SELECT
              ST_PixelOfValue(
                     ST_SetValue(
                            ST_SetValue(
                                   ST_SetValue(
                                          ST_SetValue(
                                                 ST_SetValue(
                                                        ST_AddBand(
                                                               ST_MakeEmptyRaster(5, 5, -2, 2,
1, -1, 0, 0, 0),
                                                               '8BUI'::text, 1, 0
                                                        ),
                                                        1, 1, 0
                                                 ),
                                                 2, 3, 0
                                          ),
                                          3, 5, 0
                                   ),
                                   4, 2, 0
                            ),
                            5, 4, 255
                     )
              , 1, ARRAY[1, 255]) AS pixels
) AS foo

 val | x | y
-----+---+---
   1 | 1 | 2
   1 | 1 | 3
   1 | 1 | 4
   1 | 1 | 5
   1 | 2 | 1
   1 | 2 | 2
   1 | 2 | 4
   1 | 2 | 5
   1 | 3 | 1
   1 | 3 | 2
   1 | 3 | 3
   1 | 3 | 4
```

```
  1 | 4 | 1
  1 | 4 | 3
  1 | 4 | 4
  1 | 4 | 5
  1 | 5 | 1
  1 | 5 | 2
  1 | 5 | 3
255 | 5 | 4
  1 | 5 | 5
```

9.7. 래스터 편집자

ST_SetGeoReference — 단일 호출로 지리참조 파라미터 6 개를 설정합니다. 숫자를 공백으로 구분해야 합니다. GDAL 또는 ESRI 서식의 입력물을 받아들입니다. 기본값은 GDAL 입니다.

ST_SetRotation — 래스터의 회전각을 라디안으로 설정합니다.

ST_SetScale — X 및 Y 픽셀 크기를 좌표 참조 시스템의 단위로 설정합니다. 단위/픽셀 너비/픽셀 높이 순서입니다.

ST_SetSkew — 지리참조 X 및 Y 기울기(skew)(또는 회전각 파라미터)를 설정합니다. 값 하나만 입력할 경우, X 와 Y 를 동일한 값으로 설정합니다.

ST_SetSRID — 래스터의 SRID 를 spatial_ref_sys 테이블에 정의된 특정 SRID 의 정수값으로 설정합니다.

ST_SetUpperLeft — 픽셀 좌상단의 값을 투영된 X, Y 좌표로 설정합니다.

ST_Resample — 특정 리샘플링 알고리즘, 새로운 차원, 임의의 그리드 모서리, 그리고 또 다른 래스터에서 정의되거나 빌려온 래스터 지리참조 속성들의 집합을 이용해서 래스터를 리샘플링합니다.

ST_Rescale — 축척(또는 픽셀 크기)만 조정해서 래스터를 리샘플링합니다. NearestNeighbor(영국 또는 미국 철자), Bilinear, Cubic, CubicSpline 또는 Lanczos 리샘플링 알고리즘을 이용해서 새 픽셀 값을 계산합니다. 기본값은 NearestNeighbor 입니다.

ST_Reskew — 기울기(또는 회전각 파라미터)만 조정해서 래스터를 리샘플링합니다. NearestNeighbor(영국 또는 미국 철자), Bilinear, Cubic, CubicSpline 또는 Lanczos 리샘플링 알고리즘을 이용해서 새 픽셀 값을 계산합니다. 기본값은 NearestNeighbor 입니다.

ST_SnapToGrid — 그리드에 래스터를 스냅시켜서 래스터를 리샘플링합니다. NearestNeighbor(영국 또는 미국 철자), Bilinear, Cubic, CubicSpline 또는 Lanczos 리샘플링 알고리즘을 이용해서 새 픽셀 값을 계산합니다. 기본값은 NearestNeighbor 입니다.

ST_Resize — 래스터의 크기를 새 너비/높이로 조정합니다.

ST_Transform — 알려진 공간 참조 시스템의 래스터를 지정한 리샘플링 알고리즘을 통해 또 다른 알려진 공간 참조 시스템으로 재투영합니다. NearestNeighbor, Bilinear, Cubic, CubicSpline, Lanczos 알고리즘을 이용할 수 있습니다. 기본값은 NearestNeighbor 입니다.

9.7.1. ST_SetGeoReference

ST_SetGeoReference — 단일 호출로 지리참조 파라미터 6 개를 설정합니다. 숫자를 공백으로 구분해야 합니다. GDAL 또는 ESRI 서식의 입력물을 받아들입니다. 기본값은 GDAL 입니다.

Synopsis

raster **ST_SetGeoReference**(raster *rast*, text *georefcoords*, text *format=GDAL*);

raster **ST_SetGeoReference**(raster *rast*, double precision *upperleftx*, double precision *upperlefty*, double precision *scalex*, double precision *scaley*, double precision *skewx*, double precision *skewy*);

설명: 단일 호출로 지리참조 파라미터 6 개를 설정합니다. 'GDAL' 또는 'ESRI' 서식의 입력물을 받아들입니다. 기본값은 GDAL 입니다. 6 개의 좌표를 입력하지 않을 경우 NULL 을 반환할 것입니다.

형식 표현식 사이의 차이점은 다음과 같습니다:

GDAL:

```
 scalex skewy skewx scaley upperleftx upperlefty
```

ESRI:

```
 scalex skewy skewx scaley upperleftx + scalex*0.5 upperlefty + scaley*0.5
```



래스터가 DB 외부 밴드를 가지고 있을 경우, 지리참조 파라미터를 변경하면 밴드의 외부 저장 데이터에 접근하는 데 오류가 발생할 수도 있습니다.

개선 사항: 2.1.0 버전에서 ST_SetGeoReference(raster, double precision, ...) 변종이 추가됐습니다.

예시

```
WITH foo AS (
	SELECT ST_MakeEmptyRaster(5, 5, 0, 0, 1, -1, 0, 0, 0) AS rast
)
SELECT
	0 AS rid, (ST_Metadata(rast)).*
FROM foo
UNION ALL
SELECT
	1, (ST_Metadata(ST_SetGeoReference(rast, '10 0 0 -10 0.1 0.1', 'GDAL'))).*
FROM foo
UNION ALL
SELECT
	2, (ST_Metadata(ST_SetGeoReference(rast, '10 0 0 -10 5.1 -4.9', 'ESRI'))).*
FROM foo
UNION ALL
SELECT
	3, (ST_Metadata(ST_SetGeoReference(rast, 1, 1, 10, -10, 0.001, 0.001))).*
FROM foo

 rid |       upperleftx       |       upperlefty       | width | height | scalex | scaley | skewx | skewy | srid | numbands
-----+------------------------+------------------------+-------+--------+--------+--------+-------+-------+------+----------
   0 |                      0 |                      0 |     5 |      5 |      1 |     -1 |     0 |     0 |    0 |        0
   1 |                    0.1 |                    0.1 |     5 |      5 |     10 |    -10 |     0 |     0 |    0 |        0
   2 | 0.0999999999999996 | 0.0999999999999996 |     5 |      5 |     10 |    -10 |     0 |     0 |    0 |        0
   3 |                      1 |                      1 |     5 |      5 |     10 |    -10 | 0.001 | 0.001 |    0 |        0
```

참고: ST_GeoReference, ST_ScaleX, ST_ScaleY, ST_UpperLeftX, ST_UpperLeftY

9.7.2. ST_SetRotation

ST_SetRotation — 래스터의 회전각을 라디안으로 설정합니다.

Synopsis

float8 **ST_SetRotation**(raster *rast*, float8 *rotation*);

설명: 래스터를 균일하게 회전시킵니다. 회전각은 라디안 단위입니다. 자세한 내용은 월드 파일 을 참조하십시오.

예시

```
   SELECT
    ST_ScaleX(rast1), ST_ScaleY(rast1), ST_SkewX(rast1), ST_SkewY(rast1),
    ST_ScaleX(rast2), ST_ScaleY(rast2), ST_SkewX(rast2), ST_SkewY(rast2)
  FROM (
    SELECT ST_SetRotation(rast, 15) AS rast1, rast as rast2 FROM dummy_rast
  ) AS foo;
    st_scalex    |    st_scaley    |    st_skewx    |    st_skewy    | st_scalex | st_scaley | st_skewx | st_skewy
 ----------------+-----------------+----------------+----------------+-----------+-----------+----------+----------
-1.51937582571764 | -2.27906373857646 | 1.95086352047135 | 1.30057568031423 |    2 |    3 |    0 |    0
-0.0379843956429411 | -0.0379843956429411 | 0.0325143920078558 | 0.0325143920078558 |   0.05 |   -0.05 |    0 |
0
```

참고: ST_Rotation, ST_ScaleX, ST_ScaleY, ST_SkewX, ST_SkewY

9.7.3. ST_SetScale

ST_SetScale — X 및 Y 픽셀 크기를 좌표 참조 시스템의 단위로 설정합니다. 단위/픽셀 너비/픽셀 높이 순서입니다.

Synopsis

raster **ST_SetScale**(raster *rast*, float8 *xy*);

raster **ST_SetScale**(raster *rast*, float8 *x*, float8 *y*);

설명: X 및 Y 픽셀 크기를 좌표 참조 시스템의 단위로 설정합니다. 단위/픽셀 너비/픽셀 높이 순서로 숫자를 입력합니다. 단위 한 개만 입력될 경우, X 와 Y 가 동일한 숫자라고 가정합니다.



ST_SetScale 은 래스터 범위에 매칭시키기 위해 래스터를 리샘플링하지 않는다는 점에서 ST_Rescale 과는 다릅니다. 원본에서 잘못 설정된 축척을 교정하기 위해 래스터의 메타데이터(또는 지리참조)를 변경할 뿐입니다. ST_Rescale 함수는 입력 데이터의 지리적 범위에 맞추기 위해 너비와 높이를 수정한 래스터를 출력합니다. ST_SetScale 함수는 래스터의 너비는 물론 높이도 수정하지 않습니다.

변경 사항: 2.0.0 미만 WKTRaster 버전에서는 ST_SetPixelSize 라는 명칭이었습니다. 2.0.0 버전에서 현재 명칭으로 변경됐습니다.

예시

```
UPDATE dummy_rast
      SET rast = ST_SetScale(rast, 1.5)
WHERE rid = 2;

SELECT ST_ScaleX(rast) As pixx, ST_ScaleY(rast) As pixy, Box3D(rast) As newbox
FROM dummy_rast
WHERE rid = 2;

 pixx | pixy |                newbox
------+------+-----------------------------------------------
  1.5 |  1.5 | BOX(3427927.75 5793244 0, 3427935.25 5793251.5 0)

UPDATE dummy_rast
      SET rast = ST_SetScale(rast, 1.5, 0.55)
WHERE rid = 2;

SELECT ST_ScaleX(rast) As pixx, ST_ScaleY(rast) As pixy, Box3D(rast) As newbox
FROM dummy_rast
WHERE rid = 2;

 pixx | pixy |               newbox
------+------+-------------------------------------------
  1.5 | 0.55 | BOX(3427927.75 5793244 0,3427935.25 5793247 0)
```

참고: ST_ScaleX, ST_ScaleY, Box3D

9.7.4. ST_SetSkew

ST_SetSkew — 지리참조 X 및 Y 기울기(skew)(또는 회전각 파라미터)를 설정합니다. 값 하나만 입력할 경우, X 와 Y 를 동일한 값으로 설정합니다.

Synopsis

raster **ST_SetSkew**(raster *rast*, float8 *skewxy*);

raster **ST_SetSkew**(raster *rast*, float8 *skewx*, float8 *skewy*);

설명: 지리참조 X 및 Y 기울기(또는 회전각 파라미터)를 설정합니다. 값 하나만 입력할 경우, X 와 Y 를 동일한 값으로 설정합니다. 자세한 내용은 월드 파일 을 참조하십시오.

예시

```
-- 예시 1
UPDATE dummy_rast SET rast = ST_SetSkew(rast,1,2) WHERE rid = 1;
SELECT rid, ST_SkewX(rast) As skewx, ST_SkewY(rast) As skewy,
   ST_GeoReference(rast) as georef
FROM dummy_rast WHERE rid = 1;

rid | skewx | skewy |   georef
----+-------+-------+--------------
  1 |     1 |     2 | 2.0000000000
                    : 2.0000000000
                    : 1.0000000000
                    : 3.0000000000
                    : 0.5000000000
                    : 0.5000000000


-- 예시 2: 두 기울기를 동일한 값으로 설정
```

```
UPDATE dummy_rast SET rast = ST_SetSkew(rast,0) WHERE rid = 1;
SELECT rid, ST_SkewX(rast) As skewx, ST_SkewY(rast) As skewy,
   ST_GeoReference(rast) as georef
FROM dummy_rast WHERE rid = 1;

 rid | skewx | skewy |   georef
-----+-------+-------+--------------
   1 |    0 |    0 | 2.0000000000
                : 0.0000000000
                : 0.0000000000
                : 3.0000000000
                : 0.5000000000
                : 0.5000000000
```

참고: ST_GeoReference, ST_SetGeoReference, ST_SkewX, ST_SkewY

9.7.5. ST_SetSRID

ST_SetSRID — 래스터의 SRID 를 spatial_ref_sys 테이블에 정의된 특정 SRID 의 정수값으로 설정합니다.

Synopsis

raster **ST_SetSRID**(raster *rast*, integer *srid*);

설명: 래스터의 SRID 를 특정 정수값으로 설정합니다.



이 함수는 래스터를 어떤 식으로든 변환하지 않습니다. 현재 좌표 참조 시스템의 공간 참조를 정의하는 메타데이터를 설정할 뿐입니다. 향후 변환 작업을 준비하는 데 유용합니다.

참고: Section 4.3.1, "SPATIAL_REF_SYS 테이블과 공간 참조 시스템", ST_SRID

9.7.6. ST_UpperLeft

ST_SetUpperLeft — 픽셀 좌상단의 값을 투영된 X, Y 좌표로 설정합니다.

Synopsis

raster **ST_SetUpperLeft**(raster *rast*, double precision *x*, double precision *y*);

설명: 래스터 좌상단의 값을 투영된 X, Y 좌표로 설정합니다.

예시

```
SELECT ST_SetUpperLeft(rast,-71.01,42.37)
FROM dummy_rast
WHERE rid = 2;
```

참고: ST_UpperLeftX, ST_UpperLeftY

9.7.7. ST_Resample

ST_Resample — 특정 리샘플링 알고리즘, 새로운 차원, 임의의 그리드 모서리, 그리고 또 다른 래스터에서 정의되거나 빌려온 래스터 지리참조 속성들의 집합을 이용해서 래스터를 리샘플링합니다.

Synopsis

raster **ST_Resample**(raster *rast*, integer *width*, integer *height*, double precision *gridx=NULL*, double precision *gridy=NULL*, double precision *skewx=0*, double precision *skewy=0*, text *algorithm=NearestNeighbour*, double precision *maxerr=0.125*);

raster **ST_Resample**(raster *rast*, double precision *scalex=0*, double precision *scaley=0*, double precision *gridx=NULL*, double precision *gridy=NULL*, double precision *skewx=0*, double precision *skewy=0*, text *algorithm=NearestNeighbor*, double precision *maxerr=0.125*);

raster **ST_Resample**(raster *rast*, raster *ref*, text *algorithm=NearestNeighbour*, double precision *maxerr=0.125*, boolean *usescale=true*);

raster **ST_Resample**(raster *rast*, raster *ref*, boolean *usescale*, text *algorithm=NearestNeighbour*, double precision *maxerr=0.125*);

설명: 특정 리샘플링 알고리즘, 새로운 차원(width & height), 그리드 모서리(gridx & gridy), 그리고 또 다른 래스터에서 정의되거나 빌려온 래스터 지리참조 속성들(scalex, scaley, skewx & skewy)의 집합을 이용해서 래스터를 리샘플링합니다. 참조 래스터를 이용할 경우, 두 래스터는 동일한 SRID 를 지고 있어야만 합니다.

NearestNeighbor(영국 또는 미국 철자), Bilinear, Cubic, CubicSpline 또는 Lanczos 리샘플링 알고리즘을 이용해서 새 픽셀 값을 계산합니다. 기본값은 가장 빠르지만 보간의 질은 가장 낮은 NearestNeighbor 입니다.

maxerr 를 설정하지 않을 경우 최대 오류 백분율 0.125 를 사용합니다.



자세한 내용은 GDAL Warp resampling methods 를 참조하십시오

2.0.0 버전부터 사용할 수 있습니다. GDAL 1.6.1 이상 버전이 필요합니다.

변경 사항: 2.1.0 버전부터 SRID 파라미터를 제거했습니다. 참조 래스터를 입력받는 변종은 더 이상 참조 래스터의 SRID 를 적용하지 않습니다. 래스터를 재투영하려면 ST_Transform()을 이용하십시오. SRID 가 없는 래스터에 대해서도 작동합니다.

예시

```
SELECT
       ST_Width(orig) AS orig_width,
       ST_Width(reduce_100) AS new_width
FROM (
       SELECT
              rast AS orig,
              ST_Resample(rast,100,100) AS reduce_100
       FROM aerials.boston
       WHERE ST_Intersects(rast,
              ST_Transform(
```

```
                    ST_MakeEnvelope(-71.128,    42.2392,-71.1277,    42.2397,
4326),26986)
        )
        LIMIT 1
  ) AS foo;

   orig_width | new_width
  ------------+-------------
        200   |       100
```

참고: ST_Rescale, ST_Resize, ST_Transform

9.7.8. ST_Rescale

ST_Rescale — 축척(또는 픽셀 크기)만 조정해서 래스터를 리샘플링합니다. NearestNeighbor(영국 또는 미국 철자), Bilinear, Cubic, CubicSpline 또는 Lanczos 리샘플링 알고리즘을 이용해서 새 픽셀 값을 계산합니다. 기본값은 NearestNeighbor 입니다.

Synopsis

raster **ST_Rescale**(raster *rast*, double precision *scalexy*, text *algorithm=NearestNeighbour*, double precision *maxerr=0.125*);

raster **ST_Rescale**(raster *rast*, double precision *scalex*, double precision *scaley*, text *algorithm=NearestNeighbour*, double precision *maxerr=0.125*);

설명: 축척(또는 픽셀 크기)만 조정해서 래스터를 리샘플링합니다. NearestNeighbor(영국 또는 미국 철자), Bilinear, Cubic, CubicSpline 또는 Lanczos 리샘플링 알고리즘을 이용해서 새 픽셀 값을 계산합니다. 기본값은 가장 빠르지만 보간의 질은 가장 낮은 NearestNeighbor 입니다.

scalex and scaley define the new pixel size. scaley must often be negative to get well oriented raster.

새 scalex 또는 scaley 가 래스터 너비 또는 높이의 나눗수가 아닌 경우, 결과 래스터의 범위가 입력 래스터의 범위를 포괄하도록 확장됩니다. 입력 범위를 정확히 유지하고자 할 경우, ST_Resize 를 참조하십시오.

maxerr 를 설정하지 않을 경우 최대 오류 백분율 0.125 를 사용합니다.



자세한 내용은 GDAL Warp resampling methods 를 참조하십시오.



ST_Rescale 은 래스터 범위에 매칭시키기 위해 래스터를 리샘플링한다는 점에서 ST_SetScale 과는 다릅니다. ST_SetScale 은 원본에서 잘못 설정된 축척을 교정하기 위해 래스터의 메타데이터(또는 지리참조)를 변경할 뿐입니다. ST_Rescale 함수는 입력 데이터의 지리적 범위에 맞추기 위해 너비와 높이를 수정한 래스터를 출력합니다. ST_SetScale 함수는 래스터의 너비는 물론 높이도 수정하지 않습니다.

예시

픽셀 크기 0.001 도에서 픽셀 크기 0.0015 도로 래스터를 재축척하는 단순한 예시입니다.

```
  -- 원본 래스터 픽셀 크기
  SELECT ST_PixelWidth(ST_AddBand(ST_MakeEmptyRaster(100, 100, 0, 0, 0.001, -
0.001, 0, 0, 4269), '8BUI'::text, 1, 0)) width

    width
  ----------
  0.001

  -- 재축척된 래스터 픽셀 크기
  SELECT ST_PixelWidth(ST_Rescale(ST_AddBand(ST_MakeEmptyRaster(100, 100, 0,
0, 0.001, -0.001, 0, 0, 4269), '8BUI'::text, 1, 0), 0.0015)) width

    width
  ----------
  0.0015
```

참고: ST_Resize, ST_Resample, ST_SetScale, ST_ScaleX, ST_ScaleY, ST_Transform

9.7.9. ST_Reskew

ST_Reskew — 기울기(또는 회전각 파라미터)만 조정해서 래스터를 리샘플링합니다. NearestNeighbor(영국 또는 미국 철자), Bilinear, Cubic, CubicSpline 또는 Lanczos 리샘플링 알고리즘을 이용해서 새 픽셀 값을 계산합니다. 기본값은 NearestNeighbor 입니다.

Synopsis

raster ST_Reskew(raster rast, double precision skewxy, text algorithm=NearestNeighbour, double precision maxerr=0.125);

raster ST_Reskew(raster rast, double precision skewx, double precision skewy, text algorithm=NearestNeighbour, double precision maxerr=0.125);

설명: 기울기(또는 회전각 파라미터)만 조정해서 래스터를 리샘플링합니다. NearestNeighbor(영국 또는 미국 철자), Bilinear, Cubic, CubicSpline 또는 Lanczos 리샘플링 알고리즘을 이용해서 새 픽셀 값을 계산합니다. 기본값은 가장 빠르지만 보간의 질은 가장 낮은 NearestNeighbor 입니다.

skewx 와 skewy 가 새 기울기를 정의합니다.

새 래스터의 범위가 입력 래스터의 범위를 포괄할 것입니다.

maxerr 를 설정하지 않을 경우 최대 오류 백분율 0.125 를 사용합니다.



자세한 내용은 GDAL Warp resampling methods 를 참조하십시오.



ST_Reskew 는 래스터 범위에 매칭시키기 위해 래스터를 리샘플링한다는 점에서 ST_SetSkew 와는 다릅니다. ST_SetSkew 는 원본에서 잘못 설정된 기울기를 교정하기 위해 래스터의

메타데이터(또는 지리참조)를 변경할 뿐입니다. ST_Reskew 함수는 입력 데이터의 지리적 범위에 맞추기 위해 너비와 높이를 수정한 래스터를 출력합니다. ST_SetSkew 함수는 래스터의 너비는 물론 높이도 수정하지 않습니다.

2.0.0 버전부터 사용할 수 있습니다. GDAL 1.6.1 이상 버전이 필요합니다.

변경 사항: 2.1.0 버전부터 SRID 가 없는 래스터도 입력받습니다.

예시

기울기 0.0 에서 기울기 0.0015 로 래스터를 다시 기울이는 단순한 예시입니다.

```
  -- 원본 래스터 기울기
  SELECT ST_Rotation(ST_AddBand(ST_MakeEmptyRaster(100, 100, 0, 0, 0.001, -
0.001, 0, 0, 4269), '8BUI'::text, 1, 0))

  -- 다시 기울인 래스터의 기울기
  SELECT ST_Rotation(ST_Reskew(ST_AddBand(ST_MakeEmptyRaster(100, 100, 0, 0,
0.001, -0.001, 0, 0, 4269), '8BUI'::text, 1, 0), 0.0015))
                  0.0015
```

참고: ST_Resample, ST_Rescale, ST_SetSkew, ST_SetRotation, ST_SkewX, ST_SkewY, ST_Transform

9.7.10. ST_SnapToGrid

ST_SnapToGrid — 그리드에 래스터를 스냅시켜서 래스터를 리샘플링합니다. NearestNeighbor(영국 또는 미국 철자), Bilinear, Cubic, CubicSpline 또는 Lanczos 리샘플링 알고리즘을 이용해서 새 픽셀 값을 계산합니다. 기본값은 NearestNeighbor 입니다.

Synopsis

raster **ST_SnapToGrid**(raster *rast*, double precision *gridx*, double precision *gridy*, text *algorithm=NearestNeighbour*, double precision *maxerr=0.125*, double precision *scalex=DEFAULT 0*, double precision *scaley=DEFAULT 0*);

raster **ST_SnapToGrid**(raster *rast*, double precision *gridx*, double precision *gridy*, double precision *scalex*, double precision *scaley*, text *algorithm=NearestNeighbour*, double precision *maxerr=0.125*);

raster **ST_SnapToGrid**(raster *rast*, double precision *gridx*, double precision *gridy*, double precision *scalexy*, text *algorithm=NearestNeighbour*, double precision *maxerr=0.125*);

설명: 임의의 픽셀 모서리(gridx & gridy)와 선택적인 픽셀 크기(scalex & scaley)로 정의되는 그리드에 래스터를 스냅시켜서 래스터를 리샘플링합니다. NearestNeighbor(영국 또는 미국 철자), Bilinear, Cubic, CubicSpline 또는 Lanczos 리샘플링 알고리즘을 이용해서 새 픽셀 값을 계산합니다. 기본값은 가장 빠르지만 보간의 질은 가장 낮은 NearestNeighbor 입니다.

gridx 및 gridy 가 새 그리드의 어떤 임의의 픽셀 모서리라도 정의합니다. 이 픽셀 모서리가 새 래스터의 좌상단일 필요도 없고, 새 래스터 범위의 경계선 또는 내부에 있어야 하지도 않습니다.

선택적으로, scalex 와 scaley 를 써서 새 그리드의 픽셀 크기를 정의할 수 있습니다.

새 래스터의 범위가 입력 래스터의 범위를 포괄할 것입니다.

maxerr 를 설정하지 않을 경우 최대 오류 백분율 0.125 를 사용합니다.



자세한 내용은 GDAL Warp resampling methods 를 참조하십시오.



그리드 파라미터를 더 세밀하게 조정해야 할 경우 ST_Resample 함수를 이용하십시오.

2.0.0 버전부터 사용할 수 있습니다. GDAL 1.6.1 이상 버전이 필요합니다.

변경 사항: 2.1.0 버전부터 SRID 가 없는 래스터도 입력받습니다.

예시

래스터를 약간 다른 그리드에 스냅시키는 단순한 예시입니다.

```
  -- 원본 래스터 밴드
  SELECT ST_UpperLeftX(ST_AddBand(ST_MakeEmptyRaster(10, 10, 0, 0, 0.001, -
0.001, 0, 0, 4269), '8BUI'::text, 1, 0))

  -- 그리드에 스냅된 래스터의 밴드
  SELECT ST_UpperLeftX(ST_SnapToGrid(ST_AddBand(ST_MakeEmptyRaster(10, 10, 0,
0, 0.001, -0.001, 0, 0, 4269), '8BUI'::text, 1, 0), 0.0002, 0.0002))
```

참고: ST_Resample, ST_Rescale, ST_UpperLeftX, ST_UpperLeftY

9.7.11. ST_Transform

ST_Transform — 알려진 공간 참조 시스템의 래스터를 지정한 리샘플링 알고리즘을 통해 또 다른 알려진 공간 참조 시스템으로 재투영합니다. NearestNeighbor, Bilinear, Cubic, CubicSpline, Lanczos 알고리즘을 이용할 수 있습니다. 기본값은 NearestNeighbor 입니다.

Synopsis

raster ST_Transform(raster rast, integer srid, text algorithm=NearestNeighbor, double precision maxerr=0.125, double precision scalex, double precision scaley);

raster ST_Transform(raster rast, integer srid, double precision scalex, double precision scaley, text algorithm=NearestNeighbor, double precision maxerr=0.125);

raster ST_Transform(raster rast, raster alignto, text algorithm=NearestNeighbor, double precision maxerr=0.125);

설명: 알려진 공간 참조 시스템의 래스터를 지정한 픽셀 왜곡(pixel warp) 알고리즘을 통해 또 다른 알려진 공간 참조 시스템으로 재투영합니다. 따로 알고리즘을 설정하지 않을 경우 기본값은 NearestNeighbor 이며, maxerror 를 설정하지 않을 경우 기본값은 백분율 0.125 입니다.

알고리즘 옵션에는 'NearestNeighbor', 'Bilinear', 'Cubic', 'CubicSpline', 그리고 'Lanczos'가 있습니다. 자세한 내용은 GDAL Warp resampling methods 를 참조하십시오.

ST_Transform 함수는 종종 ST_SetSRID()와 착각당합니다. ST_Transform 이 실제로 래스터의 좌표를 한 공간 참조 시스템에서 또다른 공간 참조 시스템으로 변환시키는 (그리고 픽셀 값을 리샘플링하는) 반면, ST_SetSRID()는 래스터의 SRID 식별자를 변경할 뿐입니다.

다른 변종과 달리, 변종 3 은 alignto 파라미터에 참조 래스터를 요구합니다. 결과 래스터는 참조 래스터의 공간 참조 시스템(SRID)으로 변환될 것이며, (ST_SameAlignment = TRUE 일 경우) 참조 래스터와 동일하게 정렬될 것입니다.



사용자의 변환 지원이 제대로 동작하지 않는다면, PROJSO 환경 변수를 사용자 PostGIS 가 이용하고 있는 .so 또는 .dll 투영 라이브러리로 설정해야 할 수도 있습니다. 파일명만 지정해주면 됩니다. 예를 들어 윈도우의 경우, 제어판 -> 시스템 -> 환경 변수로 가서 PROJSO 라는 시스템 변수를 추가한 다음 (사용자가 PROJ 4.6.1 을 이용하고 있을 경우) 해당 변수를 libproj.dll 로 설정합니다. 이렇게 변경한 다음 사용자의 PostgreSQL 서비스/데몬을 재시작해야 할 것입니다.

2.0.0 버전부터 사용할 수 있습니다. GDAL 1.6.1 이상 버전이 필요합니다.

개선 사항: 2.1.0 버전에서 ST_Transform(rast, alignto) 변종이 추가됐습니다.

예시

```
  SELECT ST_Width(mass_stm) As w_before, ST_Width(wgs_84) As w_after,
    ST_Height(mass_stm) As h_before, ST_Height(wgs_84) As h_after
        FROM
        ( SELECT rast As mass_stm, ST_Transform(rast,4326) As wgs_84
    ,  ST_Transform(rast,4326, 'Bilinear') AS wgs_84_bilin
                FROM aerials.o_2_boston
                      WHERE ST_Intersects(rast,
                            ST_Transform(ST_MakeEnvelope(-71.128,    42.2392,-
71.1277, 42.2397, 4326),26986) )
                LIMIT 1) As foo;

   w_before | w_after | h_before | h_after
  ----------+---------+----------+---------
       200 |     228 |      200 |     170
```



원본 매사추세츠 주 미터 단위 평면(mass_stm)	*WGS84 경위도(wgs_84)로 변환한 후*	*기본값 NN 대신 Bilinear 알고리즘을 통해 WGS84 경위도(wgs_84_bilin)로 변환한 후*

예시: 변종 3

다음은 ST_Transform(raster, srid)과 ST_Transform(raster, alignto)의 차이점을 보여주는 예시입니다.

```
 WITH foo AS (
     SELECT 0 AS rid, ST_AddBand(ST_MakeEmptyRaster(2, 2, -500000, 600000, 100, -100, 0, 0, 2163), 1, '16BUI',
1, 0)
  AS rast UNION ALL
     SELECT 1, ST_AddBand(ST_MakeEmptyRaster(2, 2, -499800, 600000, 100, -100, 0, 0, 2163), 1, '16BUI', 2, 0)
  AS rast UNION ALL
    SELECT 2, ST_AddBand(ST_MakeEmptyRaster(2, 2, -499600, 600000, 100, -100, 0, 0, 2163), 1, '16BUI', 3, 0)
 AS rast UNION ALL

     SELECT 3, ST_AddBand(ST_MakeEmptyRaster(2, 2, -500000, 599800, 100, -100, 0, 0, 2163), 1, '16BUI', 10, 0)
  AS rast UNION ALL
     SELECT 4, ST_AddBand(ST_MakeEmptyRaster(2, 2, -499800, 599800, 100, -100, 0, 0, 2163), 1, '16BUI', 20, 0)
  AS rast UNION ALL
     SELECT 5, ST_AddBand(ST_MakeEmptyRaster(2, 2, -499600, 599800, 100, -100, 0, 0, 2163), 1, '16BUI', 30, 0)
  AS rast UNION ALL

     SELECT 6, ST_AddBand(ST_MakeEmptyRaster(2, 2, -500000, 599600, 100, -100, 0, 0, 2163), 1, '16BUI', 100, 0)
  AS rast UNION ALL
     SELECT 7, ST_AddBand(ST_MakeEmptyRaster(2, 2, -499800, 599600, 100, -100, 0, 0, 2163), 1, '16BUI', 200, 0)
  AS rast UNION ALL
     SELECT 8, ST_AddBand(ST_MakeEmptyRaster(2, 2, -499600, 599600, 100, -100, 0, 0, 2163), 1, '16BUI', 300, 0)
  AS rast
 ), bar AS (
     SELECT
         ST_Transform(rast, 4269) AS alignto
     FROM foo
     LIMIT 1
 ), baz AS (
     SELECT
         rid,
         rast,
         ST_Transform(rast, 4269) AS not_aligned,
         ST_Transform(rast, alignto) AS aligned
     FROM foo
     CROSS JOIN bar
 )
 SELECT
     ST_SameAlignment(rast) AS rast,
     ST_SameAlignment(not_aligned) AS not_aligned,
     ST_SameAlignment(aligned) AS aligned
  FROM baz

  rast | not_aligned | aligned
 ------+-------------+---------
  t    | f           | t
```

참고: ST_Transform, ST_SetSRID

9.8. 래스터 밴드 편집기

ST_SetBandNoDataValue — NODATA 를 나타내는 입력 밴드의 값을 설정합니다. 밴드를 따로 설정하지 않을 경우 밴드 1 로 가정합니다. 밴드에 NODATA 가 없다고 표시하려면, nodata value = NULL 이라고 설정하십시오.

ST_SetBandIsNoData — 밴드의 isnodata 플래그를 참으로 설정합니다.

9.8.1. ST_SetBandNoDataValue

ST_SetBandNoDataValue — NODATA 를 나타내는 입력 밴드의 값을 설정합니다. 밴드를 따로 설정하지 않을 경우 밴드 1 로 가정합니다. 밴드에 NODATA 가 없다고 표시하려면, nodata value = NULL 이라고 설정하십시오.

Synopsis

raster **ST_SetBandNoDataValue**(raster *rast*, double precision *nodatavalue*);

raster **ST_SetBandNoDataValue**(raster *rast*, integer *band*, double precision *nodatavalue*, boolean *forcechecking=false*);

설명: 밴드에서 NODATA 를 나타내는 값을 설정합니다. 밴드를 따로 설정하지 않을 경우 밴드 1 로 가정합니다. 이 함수는 ST_Polygon, ST_DumpAsPolygons, 그리고 ST_PixelAs...() 함수의 결과물에 영향을 미칩니다.

예시

```
-- 첫 번째 밴드의 NODATA 값만 변경
UPDATE dummy_rast
       SET rast = ST_SetBandNoDataValue(rast,1, 254)
WHERE rid = 2;

-- 밴드 1, 2, 3 의 NODATA 밴드 값을 변경
UPDATE dummy_rast
       SET rast =
              ST_SetBandNoDataValue(
                     ST_SetBandNoDataValue(
                            ST_SetBandNoDataValue(
                                   rast,1, 254)
                            ,2,99),
                            3,108)
              WHERE rid = 2;

-- NODATA 값을 모두 제거합니다. 이렇게 하면 모든 공간 처리 함수가 모든 픽셀을 처리하게 될 것입니다.
UPDATE dummy_rast
       SET rast = ST_SetBandNoDataValue(rast,1, NULL)
WHERE rid = 2;
```

참고: ST_BandNoDataValue, ST_NumBands

9.8.2. ST_SetBandIsNoData

ST_SetBandIsNoData — 밴드의 isnodata 플래그를 참으로 설정합니다.

Synopsis

raster **ST_SetBandIsNoData**(raster *rast*, integer *band=1*);

설명: 밴드의 isnodata 플래그를 참으로 설정합니다. 밴드를 따로 설정하지 않을 경우 밴드 1 로 가정합니다. 플래그가 지저분하다고 여겨지는 경우에만 이 함수를 호출해야 합니다. 즉, 마지막 인수에 참을 설정한 경우와 설정하지 않을 경우 ST_BandIsNoData 함수를 호출해서 나온 결과물이 달라질 때 말입니다.

2.0.0 버전부터 사용할 수 있습니다.

예시

```
-- 래스터 열 한 개를 가진 가짜 테이블을 생성합니다.
create table dummy_rast (rid integer, rast raster);

-- 밴드 두 개와 픽셀/밴드 하나를 가진 래스터를 추가합니다. 첫 번째 밴드의 nodatavalue = pixel value = 3 입니다.
-- 두 번째 밴드의 nodatavalue = 13, pixel value = 4 입니다.
insert into dummy_rast values(1,
(
'01' -- little endian (uint8 ndr)
||
'0000' -- version (uint16 0)
||
'0200' -- nBands (uint16 0)
||
'17263529ED684A3F' -- scaleX (float64 0.000805965234044584)
||
'F9253529ED684ABF' -- scaleY (float64 -0.00080596523404458)
||
'1C9F33CE69E352C0' -- ipX (float64 -75.5533328537098)
||
'718F0E9A27A44840' -- ipY (float64 49.2824585505576)
||
'ED50EB853EC32B3F' -- skewX (float64 0.000211812383858707)
||
'7550EB853EC32B3F' -- skewY (float64 0.000211812383858704)
||
'E6100000' -- SRID (int32 4326)
||
'0100' -- width (uint16 1)
||
'0100' -- height (uint16 1)
||
'4' -- hasnodatavalue 값이 참으로, isnodata 값이 (참이어야 하는데) 거짓으로 설정됐습니다.
||
'2' -- 첫 번째 밴드 유형(4BUI)
||
'03' -- novalue==3
||
'03' -- pixel(0,0)==3 (same that nodata)
||
'0' -- hasnodatavalue 값이 거짓으로 설정됐습니다.
||
'5' -- 두 번째 밴드 유형(16BSI)
||
'0D00' -- novalue==13
||
'0400' -- pixel(0,0)==4
```

```
  )::raster
  );

  select st_bandisnodata(rast, 1) from dummy_rast where rid = 1; -- Expected false
  select st_bandisnodata(rast, 1, TRUE) from dummy_rast where rid = 1; -- Expected true

  -- isnodata 플래그가 지저분합니다. 이 플래그를 참으로 설정하겠습니다.
  update dummy_rast set rast = st_setbandisnodata(rast, 1) where rid = 1;


  select st_bandisnodata(rast, 1) from dummy_rast where rid = 1; -- Expected true
```

참고: ST_BandNoDataValue, ST_NumBands, ST_SetBandNoDataValue, ST_BandIsNoData

9.9. 래스터 밴드 통계 및 분석

ST_Count — 래스터 또는 래스터 커버리지의 입력 밴드에 있는 픽셀 개수를 반환합니다. 밴드를 따로 설정하지 않을 경우 기본값은 밴드 1 입니다. exclude_nodata_value 를 참으로 설정할 경우, NODATA 값이 아닌 픽셀의 개수만 반환할 것입니다.

ST_CountAgg — 종합 함수입니다. 래스터 집합의 입력 밴드에 있는 픽셀 개수를 반환합니다. 밴드를 따로 설정하지 않을 경우 기본값은 밴드 1 입니다. exclude_nodata_value 를 참으로 설정할 경우, NODATA 값이 아닌 픽셀의 개수만 반환할 것입니다.

ST_Histogram — 빈(bin; 히스토그램 표시에서 수직 막대로 나타나는 단위) 범위로 구분된 래스터 또는 래스터 커버리지의 데이터 분포를 요약하는 레코드 집합을 반환합니다. 따로 설정하지 않을 경우 빈의 개수를 자동으로 계산합니다.

ST_Quantile — 샘플링 또는 채우기(population)라는 맥락에서 래스터 또는 래스터 테이블 커버리지의 사분위(quantile)를 계산합니다. 따라서, 래스터의 25%, 50%, 75% 백분위(percentile) 단계로 값을 확인할 수 있습니다.

ST_SummaryStats — 입력한 래스터 밴드 또는 래스터 또는 래스터 커버리지의 count, sum, mean, stddev, min, max 로 이루어진 통계 요약을 반환합니다. 밴드를 따로 설정하지 않을 경우 밴드 1 로 가정합니다.

ST_SummaryStatsAgg — 종합 함수입니다. 래스터 집합의 입력 래스터 밴드의 count, sum, mean, stddev, min, max 로 이루어진 통계 요약을 반환합니다. 밴드를 따로 설정하지 않을 경우 밴드 1 로 가정합니다.

ST_ValueCount — 설정한 값들의 집합을 가진 래스터(또는 래스터 커버리지)의 입력 밴드에 있는 픽셀 밴드 값 및 픽셀 개수의 집계를 담고 있는 레코드 집합을 반환합니다. 밴드를 따로 설정하지 않을 경우 기본값은 밴드 1 입니다. 기본적으로 NODATA 값은 집계되지 않습니다. 픽셀의 다른 모든 값들을 출력하는데, 픽셀 밴드 값은 가장 가까운 정수로 반올림됩니다.

9.9.1. ST_Count

ST_Count — 래스터 또는 래스터 커버리지의 입력 밴드에 있는 픽셀 개수를 반환합니다. 밴드를 따로 설정하지 않을 경우 기본값은 밴드 1 입니다. exclude_nodata_value 를 참으로 설정할 경우, NODATA 값이 아닌 픽셀의 개수만 반환할 것입니다.

Synopsis

bigint **ST_Count**(raster *rast*, integer *nband=1*, boolean *exclude_nodata_value=true*);

bigint **ST_Count**(raster *rast*, boolean *exclude_nodata_value*);

bigint **ST_Count**(text *rastertable*, text *rastercolumn*, integer *nband=1*, boolean *exclude_nodata_value=true*);

bigint **ST_Count**(text *rastertable*, text *rastercolumn*, boolean *exclude_nodata_value*);

설명: 래스터 또는 래스터 커버리지의 입력 밴드에 있는 픽셀 개수를 반환합니다. 밴드를 따로 설정하지 않을 경우 nband 의 기본값은 1 입니다.

9.9.2. ST_CountAgg

ST_CountAgg — 종합 함수입니다. 래스터 집합의 입력 밴드에 있는 픽셀 개수를 반환합니다. 밴드를 따로 설정하지 않을 경우 기본값은 밴드 1 입니다. exclude_nodata_value 를 참으로 설정할 경우, NODATA 값이 아닌 픽셀의 개수만 반환할 것입니다.

Synopsis

bigint ST_CountAgg(raster *rast*, integer *nband*, boolean *exclude_nodata_value*, double precision *sample_percent*);

bigint ST_CountAgg(raster *rast*, integer *nband*, boolean *exclude_nodata_value*);

bigint ST_CountAgg(raster *rast*, boolean *exclude_nodata_value*);

설명: 래스터 집합의 입력 밴드에 있는 픽셀 개수를 반환합니다. 밴드를 따로 설정하지 않을 경우 nband 의 기본값은 1 입니다.

exclude_nodata_value 를 참으로 설정할 경우, 래스터의 nodata 값이 아닌 픽셀의 개수만 반환할 것입니다. 모든 픽셀의 개수를 구하려면 exclude_nodata_value 를 거짓으로 설정하십시오.

기본적으로 모든 픽셀을 샘플링할 것입니다. 더 빠른 속도를 원한다면, sample_percent 를 0 과 1 사이의 값으로 설정하십시오.

2.2.0 버전부터 사용할 수 있습니다.

예시

```
 WITH foo AS (
       SELECT
             rast.rast
       FROM (
             SELECT ST_SetValue(
                   ST_SetValue(
                         ST_SetValue(
                               ST_AddBand(
                                     ST_MakeEmptyRaster(10, 10, 10, 10, 2,
2, 0, 0,0)
                                     , 1, '64BF', 0, 0
                               )
                               , 1, 1, 1, -10
                         )
                         , 1, 5, 4, 0
```

```
                    )
                    , 1, 5, 5, 3.14159
              ) AS rast
       ) AS rast
       FULL JOIN (
              SELECT generate_series(1, 10) AS id
       ) AS id
              ON 1 = 1
)
SELECT
       ST_CountAgg(rast, 1, TRUE)
FROM foo;

 st_countagg
-------------
          20
(1 row)
```

참고: ST_Count, ST_SummaryStats, ST_SetBandNoDataValue

9.9.3. ST_Histogram

ST_Histogram — 빈(bin; 히스토그램 표시에서 수직 막대로 나타나는 단위) 범위로 구분된 래스터 또는 래스터 커버리지의 데이터 분포를 요약하는 레코드 집합을 반환합니다. 따로 설정하지 않을 경우 빈의 개수를 자동으로 계산합니다.

Synopsis

SETOF record **ST_Histogram**(raster *rast*, integer *nband=1*, boolean *exclude_nodata_value=true*, integer *bins=autocomputed*, double precision[] *width=NULL*, boolean *right=false*);

SETOF record **ST_Histogram**(raster *rast*, integer *nband*, integer *bins*, double precision[] *width=NULL*, boolean *right=false*);

SETOF record **ST_Histogram**(raster *rast*, integer *nband*, boolean *exclude_nodata_value*, integer *bins*, boolean *right*);

SETOF record **ST_Histogram**(raster *rast*, integer *nband*, integer *bins*, boolean *right*);

SETOF record **ST_Histogram**(text *rastertable*, text *rastercolumn*, integer *nband*, integer *bins*, boolean *right*);

SETOF record **ST_Histogram**(text *rastertable*, text *rastercolumn*, integer *nband*, boolean *exclude_nodata_value*, integer *bins*, boolean *right*);

SETOF record **ST_Histogram**(text *rastertable*, text *rastercolumn*, integer *nband=1*, boolean *exclude_nodata_value=true*, integer *bins=autocomputed*, double precision[] *width=NULL*, boolean *right=false*);

SETOF record **ST_Histogram**(text *rastertable*, text *rastercolumn*, integer *nband=1*, integer *bins*, double precision[] *width=NULL*, boolean *right=false*);

설명: 각 빈에 대해 입력 래스터 밴드의 min, max, count, percent 로 이루어진 레코드 집합을 반환합니다. 밴드를 따로 설정하지 않을 경우 nband 의 기본값은 1 입니다.



기본적으로 nodata 값이 아닌 픽셀 값만 처리합니다. 모든 픽셀의 개수를 구하려면 exclude_nodata_value 를 거짓으로 설정하십시오.

width double precision[] — width: 각 카테고리/빈의 너비를 나타내는 배열입니다. 빈 개수가 width 개수보다 큰 경우, width 를 반복합니다.

예시: 빈 9 개, width [a, b, c]는 [a, b, c, a, b, c, a, b, c]로 출력될 것입니다.

bins integer — 분류 단계(breakout)의 개수: 따로 설정할 경우 함수가 반환하는 레코드의 개수입니다. 따로 설정하지 않을 경우 분류 단계의 개수를 자동으로 계산합니다.

right boolean — 히스토그램을 왼쪽부터보다는 오른쪽부터(기본값) 계산합니다. X 값을 평가하는 기준을 [a, b) 에서 (a, b] 로 변경합니다.

2.0.0 버전부터 사용할 수 있습니다.

예시: 단일 래스터 타일 – 밴드 1,2,3 에 대한 히스토그램을 계산하고 빈을 자동으로 계산합니다.

```
SELECT band, (stats).*
FROM (SELECT rid, band, ST_Histogram(rast, band) As stats
   FROM dummy_rast CROSS JOIN generate_series(1,3) As band
    WHERE rid=2) As foo;

 band |  min  |  max  | count | percent
------+-------+-------+-------+---------
    1 |   249 |   250 |     2 |    0.08
    1 |   250 |   251 |     2 |    0.08
    1 |   251 |   252 |     1 |    0.04
    1 |   252 |   253 |     2 |    0.08
    1 |   253 |   254 |    18 |    0.72
    2 |    78 | 113.2 |    11 |    0.44
    2 | 113.2 | 148.4 |     4 |    0.16
    2 | 148.4 | 183.6 |     4 |    0.16
    2 | 183.6 | 218.8 |     1 |    0.04
    2 | 218.8 |   254 |     5 |     0.2
    3 |    62 | 100.4 |    11 |    0.44
    3 | 100.4 | 138.8 |     5 |     0.2
    3 | 138.8 | 177.2 |     4 |    0.16
    3 | 177.2 | 215.6 |     1 |    0.04
    3 | 215.6 |   254 |     4 |    0.16
```

예시: 밴드 2 의 히스토그램만 계산하고 빈 6 개를 계산합니다.

```
SELECT (stats).*
FROM (SELECT rid, ST_Histogram(rast, 2,6) As stats
   FROM dummy_rast
    WHERE rid=2) As foo;

   min    |    max    | count | percent
-----------+-----------+-------+---------
        78 | 107.333333 |     9 |    0.36
 107.333333 | 136.666667 |     6 |    0.24
 136.666667 |       166 |     0 |       0
       166 | 195.333333 |     4 |    0.16
 195.333333 | 224.666667 |     1 |    0.04
 224.666667 |       254 |     5 |     0.2
(6 rows)

-- 이전 예시와 동일하지만 각 빈의 픽셀 값 범위를 명확하게 조정합니다.
```

```
SELECT (stats).*
FROM (SELECT rid, ST_Histogram(rast, 2,6,ARRAY[0.5,1,4,100,5]) As stats
   FROM dummy_rast
    WHERE rid=2) As foo;

  min  |  max  | count | percent
-------+-------+-------+----------
    78 |  78.5 |     1 |     0.08
  78.5 |  79.5 |     1 |     0.04
  79.5 |  83.5 |     0 |        0
  83.5 | 183.5 |    17 |   0.0068
 183.5 | 188.5 |     0 |        0
 188.5 |   254 |     6 | 0.003664
(6 rows)
```

참고: ST_Count, ST_SummaryStats, ST_SummaryStatsAgg

9.9.4. ST_Quantile

ST_Quantile — 샘플링 또는 채우기(population)라는 맥락에서 래스터 또는 래스터 테이블 커버리지의 사분위(quantile)를 계산합니다. 따라서, 래스터의 25%, 50%, 75% 백분위(percentile) 단계로 값을 확인할 수 있습니다.

Synopsis

SETOF record **ST_Quantile**(raster *rast*, integer *nband=1*, boolean *exclude_nodata_value=true*, double precision[] *quantiles=NULL*);

SETOF record **ST_Quantile**(raster *rast*, double precision[] *quantiles*);

SETOF record **ST_Quantile**(raster *rast*, integer *nband*, double precision[] *quantiles*);

double precision **ST_Quantile**(raster *rast*, double precision *quantile*);

double precision **ST_Quantile**(raster *rast*, boolean *exclude_nodata_value*, double precision *quantile=NULL*);

double precision **ST_Quantile**(raster *rast*, integer *nband*, double precision *quantile*);

double precision **ST_Quantile**(raster *rast*, integer *nband*, boolean *exclude_nodata_value*, double precision *quantile*);

double precision **ST_Quantile**(raster *rast*, integer *nband*, double precision *quantile*);

SETOF record **ST_Quantile**(text *rastertable*, text *rastercolumn*, integer *nband=1*, boolean *exclude_nodata_value=true*, double precision[] *quantiles=NULL*);

SETOF record **ST_Quantile**(text *rastertable*, text *rastercolumn*, integer *nband*, double precision[] *quantiles*);

설명: 샘플링 또는 채우기(population)라는 맥락에서 래스터 또는 래스터 테이블 커버리지의 사분위(quantile)를 계산합니다. 따라서, 래스터의 25%, 50%, 75% 백분위(percentile) 단계로 값을 확인할 수 있습니다.



exclude_nodata_value 를 거짓으로 설정하면, NODATA 값의 픽셀도 집계할 것입니다.

2.0.0 버전부터 사용할 수 있습니다.

예시

```
 UPDATE dummy_rast SET rast = ST_SetBandNoDataValue(rast,249) WHERE rid=2;
 -- 이 예시 쿼리는 지정한 사분위 안에 있는 밴드 1 의 픽셀 가운데 값이 249 가 아닌 픽셀들만 처리할 것입니다. --

 SELECT (pvq).*
 FROM (SELECT ST_Quantile(rast, ARRAY[0.25,0.75]) As pvq
    FROM dummy_rast WHERE rid=2) As foo
    ORDER BY (pvq).quantile;

  quantile | value
 ----------+-------
     0.25 |   253
     0.75 |   254

 SELECT ST_Quantile(rast, 0.75) As value
    FROM dummy_rast WHERE rid=2;

 value
 ------
   254
 -- 실제 예시입니다. 밴드 2 의 픽셀 가운데 도형과 교차하는 모든 픽셀들의 사분위를 구합니다.
 SELECT rid, (ST_Quantile(rast,2)).* As pvc
    FROM o_4_boston
       WHERE ST_Intersects(rast,
          ST_GeomFromText('POLYGON((224486   892151,224486   892200,224706   892200,224706
892151,224486 892151))',26986)
          )
 ORDER BY value, quantile,rid
 ;


  rid | quantile | value
 -----+----------+-------
    1 |        0 |     0
    2 |        0 |     0
   14 |        0 |     1
   15 |        0 |     2
   14 |     0.25 |    37
    1 |     0.25 |    42
   15 |     0.25 |    47

    2 |     0.25 |    50
   14 |      0.5 |    56
    1 |      0.5 |    64
   15 |      0.5 |    66
    2 |      0.5 |    77
   14 |     0.75 |    81
   15 |     0.75 |    87
    1 |     0.75 |    94
    2 |     0.75 |   106
   14 |        1 |   199
    1 |        1 |   244
    2 |        1 |   255
   15 |        1 |   255
```

참고: ST_Count, ST_SummaryStats, ST_SummaryStatsAgg, ST_SetBandNoDataValue

9.9.5. ST_SummaryStats

ST_SummaryStats — 입력한 래스터 밴드 또는 래스터 또는 래스터 커버리지의 count, sum, mean, stddev, min, max 로 이루어진 통계 요약을 반환합니다. 밴드를 따로 설정하지 않을 경우 밴드 1 로 가정합니다.

Synopsis

summarystats **ST_SummaryStats**(raster *rast*, boolean *exclude_nodata_value*);

summarystats **ST_SummaryStats**(raster *rast*, integer *nband*, boolean *exclude_nodata_value*);

summarystats **ST_SummaryStats**(text *rastertable*, text *rastercolumn*, boolean *exclude_nodata_value*);

summarystats **ST_SummaryStats**(text *rastertable*, text *rastercolumn*, integer *nband=1*, boolean *exclude_nodata_value=true*);

설명: 입력한 래스터 밴드 또는 래스터 또는 래스터 커버리지의 count, sum, mean, stddev, min, max 로 이루어진 summarystats 을 반환합니다. 밴드를 따로 설정하지 않을 경우 nband 의 기본값은 1 입니다.

2.0.0 버전부터 사용할 수 있습니다.



기본적으로 nodata 값이 아닌 픽셀 값만 처리합니다. 모든 픽셀의 개수를 구하려면 exclude_nodata_value 를 거짓으로 설정하십시오.



기본적으로 모든 픽셀을 샘플링할 것입니다. 더 빠른 속도를 원한다면, sample_percent 를 1 보다 작은 값으로 설정하십시오.



2.2.0 버전부터 더 이상 ST_SummaryStats(rastertable, rastercolumn, ...) 변종 함수를 지원하지 않습니다. 대신 ST_SummaryStatsAgg 함수를 이용하십시오.

예시: 단일 래스터 타일

```
SELECT rid, band, (stats).*
FROM (SELECT rid, band, ST_SummaryStats(rast, band) As stats
   FROM dummy_rast CROSS JOIN generate_series(1,3) As band
    WHERE rid=2) As foo;

 rid | band | count | sum  |    mean    |  stddev   | min | max
-----+------+-------+------+------------+-----------+-----+-----
   2 |    1 |    23 | 5821 | 253.086957 |  1.248061 | 250 | 254
   2 |    2 |    25 | 3682 |    147.28 | 59.862188 |  78 | 254
   2 |    3 |    25 | 3290 |     131.6 | 61.647384 |  62 | 254
```

예시: 관심 건물과 교차하는 픽셀들을 요약

PostGIS 윈도우 64 비트 버전에서 모든 보스턴 건물들과 항공사진 타일들(각각 건물 레코드 102,000 개, 150x150 픽셀 크기의 타일 134,000 개)을 처리하는 이 예시가 574 밀리초 걸렸습니다.

```
WITH
```

```
  -- 관심 피처
    feat AS (SELECT gid As building_id, geom_26986 As geom FROM buildings AS
b
     WHERE gid IN(100, 103,150)
    ),
  -- builds 의 경계선에 맞춰 래스터 타일의 밴드 2 를 잘라낸 다음
  -- 이 잘라낸 지역의 통계를 얻습니다.
    b_stats AS
        (SELECT  building_id, (stats).*
  FROM (SELECT building_id, ST_SummaryStats(ST_Clip(rast,2,geom)) As stats
     FROM aerials.boston
              INNER JOIN feat
        ON ST_Intersects(feat.geom,rast)
   ) As foo
   )
  -- 마지막으로 통계를 요약합니다.
  SELECT building_id, SUM(count) As num_pixels
    , MIN(min) As min_pval
    ,  MAX(max) As max_pval
    , SUM(mean*count)/SUM(count) As avg_pval
        FROM b_stats
   WHERE count
  > 0
        GROUP BY building_id
        ORDER BY building_id;
   building_id | num_pixels | min_pval | max_pval |    avg_pval
  -------------+------------+----------+----------+------------------
         100 |      1090 |       1 |     255 | 61.0697247706422
         103 |       655 |       7 |     182 | 70.5038167938931
         150 |       895 |       2 |     252 | 185.642458100559
```

예시: 래스터 커버리지

```
  -- 각 밴드에 대한 통계 --
  SELECT band, (stats).*
  FROM (SELECT band, ST_SummaryStats('o_4_boston','rast', band) As stats
     FROM generate_series(1,3) As band) As foo;

   band |  count  |  sum   |      mean      |     stddev     | min | max
  ------+---------+--------+----------------+----------------+-----+-----
     1 | 8450000 | 725799 | 82.7064349112426 | 45.6800222638537 |   0 | 255
     2 | 8450000 | 700487 | 81.4197705325444 | 44.2161184161765 |   0 | 255
     3 | 8450000 | 575943 |  74.682739408284 | 44.2143885481407 |   0 | 255

  -- 테이블의 경우: 샘플링을 100% 미만으로 설정하면 속도가 향상됩니다.
  -- 이 예시에서는 25%로 설정해서 훨씬 빨리 답을 반환받습니다.
  SELECT band, (stats).*
  FROM  (SELECT  band,  ST_SummaryStats('o_4_boston','rast',  band,true,0.25)  As
stats
     FROM generate_series(1,3) As band) As foo;



   band |  count  |  sum   |      mean      |     stddev     | min | max
  ------+---------+--------+----------------+----------------+-----+-----
     1 | 2112500 | 180686 | 82.6890480473373 | 45.6961043857248 |   0 | 255
     2 | 2112500 | 174571 |  81.448503668639 | 44.2252623171821 |   0 | 255
     3 | 2112500 | 144364 | 74.6765884023669 | 44.2014869384578 |   0 | 255
```

참고: summarystats, ST_SummaryStatsAgg, ST_Count, ST_Clip

9.9.6. ST_SummaryStatsAgg

ST_SummaryStatsAgg — 종합 함수입니다. 래스터 집합의 입력 래스터 밴드의 count, sum, mean, stddev, min, max 로 이루어진 통계 요약을 반환합니다. 밴드를 따로 설정하지 않을 경우 밴드 1 로 가정합니다.

Synopsis

summarystats ST_SummaryStatsAgg(raster *rast*, integer *nband*, boolean *exclude_nodata_value*, double precision *sample_percent*);

summarystats ST_SummaryStatsAgg(raster *rast*, boolean *exclude_nodata_value*, double precision *sample_percent*);

summarystats ST_SummaryStatsAgg(raster *rast*, integer *nband*, boolean *exclude_nodata_value*);

설명: 입력한 래스터 밴드 또는 래스터 또는 래스터 커버리지의 count, sum, mean, stddev, min, max 로 이루어진 summarystats 을 반환합니다. 밴드를 따로 설정하지 않을 경우 nband 의 기본값은 1 입니다.



기본적으로 nodata 값이 아닌 픽셀 값만 처리합니다. 모든 픽셀의 개수를 구하려면 exclude_nodata_value 를 거짓으로 설정하십시오.



기본적으로 모든 픽셀을 샘플링할 것입니다. 더 빠른 속도를 원한다면, sample_percent 를 0 과 1 사이의 값으로 설정하십시오.

2.2.0 버전부터 사용할 수 있습니다.

예시

```
 WITH foo AS (
       SELECT
              rast.rast
       FROM (
              SELECT ST_SetValue(
                     ST_SetValue(
                            ST_SetValue(
                                   ST_AddBand(
                                          ST_MakeEmptyRaster(10, 10, 10, 10, 2,
2, 0, 0,0)
                                          , 1, '64BF', 0, 0
                                   )
                                   , 1, 1, 1, -10
                            )
                            , 1, 5, 4, 0
                     )
                     , 1, 5, 5, 3.14159
              ) AS rast
```

```
        ) AS rast
        FULL JOIN (
              SELECT generate_series(1, 10) AS id
        ) AS id
              ON 1 = 1
  )
  SELECT
        (stats).count,
        round((stats).sum::numeric, 3),
        round((stats).mean::numeric, 3),
        round((stats).stddev::numeric, 3),
        round((stats).min::numeric, 3),
        round((stats).max::numeric, 3)
  FROM (
        SELECT
              ST_SummaryStatsAgg(rast, 1, TRUE, 1) AS stats
        FROM foo
  ) bar;

   count |  round  | round  | round |  round  | round
  -------+---------+--------+-------+---------+-------
      20 | -68.584 | -3.429 | 6.571 | -10.000 | 3.142
  (1 row)
```

참고: summarystats, ST_SummaryStats, ST_Count, ST_Clip

9.9.7. ST_ValueCount

ST_ValueCount — 설정한 값들의 집합을 가진 래스터(또는 래스터 커버리지)의 입력 밴드에 있는 픽셀 밴드 값 및 픽셀 개수의 집계를 담고 있는 레코드 집합을 반환합니다. 밴드를 따로 설정하지 않을 경우 기본값은 밴드 1 입니다. 기본적으로 NODATA 값은 집계되지 않습니다. 픽셀의 다른 모든 값들을 출력하는데, 픽셀 밴드 값은 가장 가까운 정수로 반올림됩니다.

Synopsis

SETOF record **ST_ValueCount**(raster *rast*, integer *nband=1*, boolean *exclude_nodata_value=true*, double precision[] *searchvalues=NULL*, double precision *roundto=0*, double precision *OUT value*, integer *OUT count*);

SETOF record **ST_ValueCount**(raster *rast*, integer *nband*, double precision[] *searchvalues*, double precision *roundto=0*, double precision *OUT value*, integer *OUT count*);

SETOF record **ST_ValueCount**(raster *rast*, double precision[] *searchvalues*, double precision *roundto=0*, double precision *OUT value*, integer *OUT count*);

bigint **ST_ValueCount**(raster *rast*, double precision *searchvalue*, double precision *roundto=0*);

bigint **ST_ValueCount**(raster *rast*, integer *nband*, boolean *exclude_nodata_value*, double precision *searchvalue*, double precision *roundto=0*);

bigint **ST_ValueCount**(raster *rast*, integer *nband*, double precision *searchvalue*, double precision *roundto=0*);

SETOF record **ST_ValueCount**(text *rastertable*, text *rastercolumn*, integer *nband=1*, boolean *exclude_nodata_value=true*, double precision[] *searchvalues=NULL*, double precision *roundto=0*, double precision *OUT value*, integer *OUT count*);

SETOF record **ST_ValueCount**(text *rastertable*, text *rastercolumn*, double precision[] *searchvalues*, double precision *roundto=0*, double precision *OUT value*, integer *OUT count*);

SETOF record **ST_ValueCount**(text *rastertable*, text *rastercolumn*, integer *nband*, double precision[] *searchvalues*, double precision *roundto=0*, double precision *OUT value*, integer *OUT count*);

bigint**ST_ValueCount**(text *rastertable*, text *rastercolumn*, integer *nband*, boolean *exclude_nodata_value*, double precision *searchvalue*, double precision *roundto=0*);

bigint **ST_ValueCount**(text *rastertable*, text *rastercolumn*, double precision *searchvalue*, double precision *roundto=0*);

bigint **ST_ValueCount**(text *rastertable*, text *rastercolumn*, integer *nband*, double precision *searchvalue*, double precision *roundto=0*);

설명: 선택한 밴드의 래스터 타일 또는 래스터 커버리지에 있는 픽셀의 밴드 값 및 개수를 담고 있는 value, count 열을 가진 레코드 집합을 반환합니다.

밴드를 따로 설정하지 않을 경우 nband 의 기본값은 1 입니다. searchvalues 를 설정하지 않을 경우, 래스터 또는 래스터 커버리지에서 발견된 모든 픽셀 값을 반환할 것입니다. searchvalues 를 하나만 설정하면, 레코드 대신 해당 픽셀 밴드 값을 가진 픽셀의 개수를 나타내는 정수를 반환할 것입니다.



exclude_nodata_value 를 거짓으로 설정하면, NODATA 값의 픽셀도 집계할 것입니다.

2.0.0 버전부터 사용할 수 있습니다.

예시

```
UPDATE dummy_rast SET rast = ST_SetBandNoDataValue(rast,249) WHERE rid=2;
-- 이 예시는 밴드 1 에서 값이 249 가 아닌 픽셀들만 집계할 것입니다. --

SELECT (pvc).*
FROM (SELECT ST_ValueCount(rast) As pvc
    FROM dummy_rast WHERE rid=2) As foo
    ORDER BY (pvc).value;

 value | count
-------+-------
   250 |     2
   251 |     1
   252 |     2
   253 |     6
   254 |    12

-- 이 예시는 값 249 를 포함한 밴드 1 의 모든 픽셀을 집계할 것입니다. --
SELECT (pvc).*
FROM (SELECT ST_ValueCount(rast,1,false) As pvc
    FROM dummy_rast WHERE rid=2) As foo
    ORDER BY (pvc).value;

 value | count
-------+-------
   249 |     2
   250 |     2
   251 |     1
```

```
   252 |    2
   253 |    6
   254 |   12

 -- 이 예시는 밴드 2 가운데 NODATA 값이 아닌 픽셀들만 집계할 것입니다. --
 SELECT (pvc).*
 FROM (SELECT ST_ValueCount(rast,2) As pvc
    FROM dummy_rast WHERE rid=2) As foo
    ORDER BY (pvc).value;
  value | count
 -------+-------
    78 |    1
    79 |    1
    88 |    1
    89 |    1
    96 |    1
    97 |    1
    98 |    1
    99 |    2
   112 |    2
 :


 -- 실제 예시입니다. 도형과 교차하는 항공사진 래스터 타일의 밴드 2 에 있는 모든 픽셀을 집계한
다음
 -- 개수가 500 개를 초과하는 픽셀 밴드 값만 반환합니다.
 SELECT (pvc).value, SUM((pvc).count) As total
 FROM (SELECT ST_ValueCount(rast,2) As pvc
    FROM o_4_boston
       WHERE ST_Intersects(rast,
          ST_GeomFromText('POLYGON((224486   892151,224486   892200,224706
892200,224706 892151,224486 892151))',26986)
           )
       ) As foo
    GROUP BY (pvc).value
    HAVING SUM((pvc).count)
 > 500
    ORDER BY (pvc).value;

  value | total
 -------+-----
    51 | 502
    54 | 521
 -- 각 래스터에서 특정 도형과 교차하는 타일 가운데 값이 100 인 픽셀의 개수 집계만 반환합니다.
--
 SELECT rid, ST_ValueCount(rast,2,100) As count
    FROM o_4_boston
       WHERE ST_Intersects(rast,
          ST_GeomFromText('POLYGON((224486   892151,224486   892200,224706
892200,224706 892151,224486 892151))',26986)
           ) ;

  rid | count
 -----+-------
   1 |   56
   2 |   95
  14 |   37
  15 |   64
```

참고: ST_Count, ST_SetBandNoDataValue

9.10. 래스터 출력

ST_AsBinary — 래스터의 WKB(Well-Known Binary) 표현식을 SRID 메타데이터 없이 반환합니다.

ST_AsGDALRaster — 래스터 타일을 지정한 GDAL 래스터 형식으로 반환합니다. 래스터 형식이란 사용자가 컴파일한 라이브러리가 지원하는 형식들 가운데 하나입니다. 사용자 라이브러리가 지원하는 형식들의 목록을 보려면 ST_GDALRasters() 함수를 이용하십시오.

ST_AsJPEG — 래스터 타일에서 선택한 밴드들을 단일 JPEG(Joint Photographic Exports Group) 이미지(바이트 배열)로 반환합니다. 밴드를 따로 설정하지 않거나, 밴드가 1 개거나 또는 3 개를 초과할 경우 첫 번째 밴드를 씁니다. 밴드가 3 개뿐일 경우 밴드 3 개를 모두 써서 RGB 에 매핑시킵니다.

ST_AsPNG — 래스터 타일에서 선택한 밴드들을 단일 PNG(Portable Network Graphics) 이미지(바이트 배열)로 반환합니다. 래스터의 밴드가 1 개, 3 개, 또는 4 개이거나 따로 설정하지 않을 경우 모든 밴드를 씁니다. 밴드가 2 개 또는 4 개를 초과하며 따로 설정하지 않을 경우, 밴드 1 만 씁니다. 밴드를 RGB 또는 RGBA 스페이스에 매핑합니다.

ST_AsTIFF — 래스터 타일에서 선택한 밴드들을 단일 TIFF 이미지(바이트 배열)로 반환합니다. 밴드를 따로 설정하지 않을 경우, 모든 밴드를 쓰려 할 것입니다.

9.10.1. ST_AsBinary

ST_AsBinary — 래스터의 WKB(Well-Known Binary) 표현식을 SRID 메타데이터 없이 반환합니다.

Synopsis

bytea **ST_AsBinary**(raster *rast*, boolean *outasin=FALSE*);

설명: 래스터의 바이너리 표현식을 반환합니다. outasin 을 참으로 설정할 경우, DB 외부 밴드를 DB 내부 밴드로 취급합니다.

이 함수는 데이터베이스로부터 데이터를 문자열 표현식으로 변환하지 않고 추출하는 바이너리 커서에 유용합니다.



기본적으로, WKB 출력물은 DB 외부 밴드를 가리키는 외부 파일 경로를 담고 있습니다. 클라이언트가 DB 외부 밴드의 기저 래스터 파일에 접근하지 못 하는 경우, outasin 을 참으로 설정하십시오.

개선 사항: 2.1.0 버전에서 outasin 이 추가됐습니다.

예시

```
SELECT ST_AsBinary(rast) As rastbin FROM dummy_rast WHERE rid=1;

                       rastbin
------------------------------------------------------------------------------
\001\000\000\000\000\000\000\000\000\000\000\000@\000\000\000\000\000\000\010@\
```

```
000\000\000\000\000\000\340?\000\000\000\000\000\000\340?\000\000\000\000\000\00
0\000\000\000\000\000\000\000\000\000\000\012\000\000\000\012\000\024\000
```

9.10.2. ST_AsGDALRaster

ST_AsGDALRaster — 래스터 타일을 지정한 GDAL 래스터 형식으로 반환합니다. 래스터 형식이란 사용자가 컴파일한 라이브러리가 지원하는 형식들 가운데 하나입니다. 사용자 라이브러리가 지원하는 형식들의 목록을 보려면 ST_GDALRasters() 함수를 이용하십시오.

Synopsis

bytea **ST_AsGDALRaster**(raster *rast*, text *format*, text[] *options=NULL*, integer *srid=sameassource*);

설명: 래스터 타일을 지정한 형식으로 반환합니다. 인수들은 다음과 같습니다:

- format - 출력할 형식입니다. 사용자의 LibGDAL 라이브러리에 컴파일된 드라이버에 따라 달라질 수 있습니다. 일반적으로 사용할 수 있는 형식은 'JPEG', 'GTiff', 'PNG'입니다. 사용자 라이브러리가 지원하는 형식들의 목록을 보려면 ST_GDALDrivers 함수를 이용하십시오.
- options - GDAL 옵션들의 텍스트 배열입니다. 형식에 따라 유효한 옵션들이 달라집니다. 자세한 내용은 GDAL 래스터 형식 옵션 을 참조하십시오.
- srs - 이미지 파일에 임베딩할 proj4text 또는 (spatial_ref_sys 에서 가져온) srtext 입니다.

2.0.0 버전부터 사용할 수 있습니다. GDAL 1.6.0 이상 버전이 필요합니다.

JPEG 출력 예시

```
SELECT ST_AsGDALRaster(rast, 'JPEG') As rastjpg
FROM dummy_rast WHERE rid=1;

SELECT ST_AsGDALRaster(rast, 'JPEG', ARRAY['QUALITY=50']) As rastjpg
FROM dummy_rast WHERE rid=2;
```

GeoTiff 출력 예시

```
SELECT ST_AsGDALRaster(rast, 'GTiff') As rastjpg
FROM dummy_rast WHERE rid=2;

-- JPEG 압축한 GeoTiff를 90% 품질로 출력
SELECT ST_AsGDALRaster(rast, 'GTiff',
  ARRAY['COMPRESS=JPEG', 'JPEG_QUALITY=90'],
  4269) As rasttiff
FROM dummy_rast WHERE rid=2;
```

참고: Section 5.3, “PostGIS 래스터를 이용하는 사용자 지정 응용 프로그램 빌드하기”, ST_GDALDrivers, ST_SRID

9.10.3. ST_AsJPEG

ST_AsJPEG — 래스터 타일에서 선택한 밴드들을 단일 JPEG(Joint Photographic Exports Group) 이미지(바이트 배열)로 반환합니다. 밴드를 따로 설정하지 않거나, 밴드가 1 개거나 또는 3 개를 초과할 경우 첫 번째 밴드를 씁니다. 밴드가 3 개뿐일 경우 밴드 3 개를 모두 써서 RGB 에 매핑시킵니다.

Synopsis

bytea **ST_AsJPEG**(raster *rast*, text[] *options=NULL*);

bytea **ST_AsJPEG**(raster *rast*, integer *nband*, integer *quality*);

bytea **ST_AsJPEG**(raster *rast*, integer *nband*, text[] *options=NULL*);

bytea **ST_AsJPEG**(raster *rast*, integer[] *nbands*, text[] *options=NULL*);

bytea **ST_AsJPEG**(raster *rast*, integer[] *nbands*, integer *quality*);

설명: 래스터에서 선택한 밴드들을 단일 JPEG(Joint Photographic Exports Group) 이미지로 반환합니다. 덜 흔한 래스터 유형으로 내보내야 할 경우 ST_AsGDALRaster 함수를 이용하십시오. 밴드를 따로 설정하지 않거나, 밴드가 1 개거나 또는 3 개를 초과할 경우 첫 번째 밴드만 씁니다. 밴드가 3 개일 경우 밴드 3 개를 모두 씁니다. 이 함수에는 다음과 같은 많은 옵션을 가진 많은 변종이 있습니다.

예를 들어 ARRAY[3,2,1]은 밴드 3 을 빨간색, 밴드 2 를 초록색, 밴드 1 을 파란색에 매핑한다는 뜻입니다.

- quality - 1 부터 100 까지의 숫자입니다. 숫자가 높을수록 이미지가 선명해집니다.

- options - JPEG 에 대해 정의된 GDAL 옵션들의 텍스트 배열입니다(ST_GDALDrivers 에서 JPEG 에 대한 create_options 를 살펴보십시오). JPEG 의 경우, 유효한 옵션은 PROGRESSIVE ON/OFF 및 기본값이 75 이고 0 부터 100 까지의 범위에서 설정할 수 있는 QUALITY 입니다. 자세한 내용은 GDAL 래스터 형식 옵션 을 참조하십시오.

2.0.0 버전부터 사용할 수 있습니다. GDAL 1.6.0 이상 버전이 필요합니다.

예시: 출력

```
  -- 처음 밴드 3 개를 75% 품질로 출력
 SELECT ST_AsJPEG(rast) As rastjpg
    FROM dummy_rast WHERE rid=2;

 -- 첫 번째 밴드만 90% 품질로 출력
 SELECT ST_AsJPEG(rast,1,90) As rastjpg
    FROM dummy_rast WHERE rid=2;

 -- 처음 밴드 3 개를 출력(밴드 2 를 빨강, 밴드 1 을 녹색, 밴드 3 을 파랑으로 하고
PROGRESSIVE = ON, 품질은 90%)
 SELECT ST_AsJPEG(rast,ARRAY[2,1,3],ARRAY['QUALITY=90','PROGRESSIVE=ON']) As
rastjpg
    FROM dummy_rast WHERE rid=2;
```

참고: Section 5.3, "PostGIS 래스터를 이용하는 사용자 지정 응용 프로그램 빌드하기", ST_GDALDrivers, ST_AsGDALRaster, ST_AsPNG, ST_AsTIFF

9.10.4. ST_AsPNG

ST_AsPNG — 래스터 타일에서 선택한 밴드들을 단일 PNG(Portable Network Graphics) 이미지(바이트 배열)로 반환합니다. 래스터의 밴드가 1 개, 3 개, 또는 4 개이거나 따로 설정하지 않을 경우 모든 밴드를 씁니다. 밴드가 2 개 또는 4 개를 초과하며 따로 설정하지 않을 경우, 밴드 1 만 씁니다. 밴드를 RGB 또는 RGBA 스페이스에 매핑합니다.

Synopsis

bytea **ST_AsPNG**(raster *rast*, text[] *options=NULL*);

bytea **ST_AsPNG**(raster *rast*, integer *nband*, integer *compression*);

bytea **ST_AsPNG**(raster *rast*, integer *nband*, text[] *options=NULL*);

bytea **ST_AsPNG**(raster *rast*, integer[] *nbands*, integer *compression*);

bytea **ST_AsPNG**(raster *rast*, integer[] *nbands*, text[] *options=NULL*);

설명: 래스터에서 선택한 밴드들을 단일 PNG(Portable Network Graphics) 이미지로 반환합니다. 덜 흔한 래스터 유형으로 내보내야 할 경우 ST_AsGDALRaster 함수를 이용하십시오. 밴드를 따로 설정하지 않을 경우, 처음 3 개의 밴드를 내보냅니다. srid 를 따로 설정하지 않으면 래스터의 SRID 를 사용합니다. 이 함수에는 다음과 같은 많은 옵션을 가진 많은 변종이 있습니다:

- nband - 단일 밴드 내보내기를 위한 옵션입니다.
- nbands - 내보낼 밴드들의 배열입니다(PNG 의 경우 4 가 최대값입니다). 밴드의 순서는 RGBA 입니다. 예를 들어 ARRAY[3,2,1]은 밴드 3 을 빨간색, 밴드 2 를 초록색, 밴드 1 을 파란색에 매핑한다는 뜻입니다.
- compression - 1 부터 9 까지의 숫자입니다. 숫자가 높을수록 압축률도 높아집니다.
- options - PNG 에 대해 정의된 GDAL 옵션들의 텍스트 배열입니다(ST_GDALDrivers 에서 PNG 에 대한 create_options 를 살펴보십시오). PNG 의 경우, 유효한 옵션은 ZLEVEL(압축에 소비할 시간 - 기본값은 6)뿐으로, 예를 들어 ARRAY['ZLEVEL=9']처럼 쓰입니다. 이 함수는 출력물 2 개를 출력해야 하기 때문에 월드 파일을 사용할 수는 없습니다. 자세한 내용은 GDAL 래스터 형식 옵션을 참조하십시오.

2.0.0 버전부터 사용할 수 있습니다. GDAL 1.6.0 이상 버전이 필요합니다.

예시

```
SELECT ST_AsPNG(rast) As rastpng
FROM dummy_rast WHERE rid=2;

-- 처음 3 개의 밴드를 내보내서 밴드 3 을 빨강, 밴드 1 을 녹색, 밴드 2 를 파랑으로 매핑
SELECT ST_AsPNG(rast, ARRAY[3,1,2]) As rastpng
FROM dummy_rast WHERE rid=2;
```

참고: ST_AsGDALRaster, ST_ColorMap, ST_GDALDrivers, Section 5.3, "PostGIS 래스터를 이용하는 사용자 지정 응용 프로그램 빌드하기"

9.10.5. ST_AsTIFF

ST_AsTIFF — 래스터 타일에서 선택한 밴드들을 단일 TIFF 이미지(바이트 배열)로 반환합니다. 밴드를 따로 설정하지 않을 경우, 모든 밴드를 쓰려 할 것입니다.

Synopsis

bytea ST_AsTIFF(raster *rast*, text[ ] *options=''*, integer *srid=sameassource*);

bytea ST_AsTIFF(raster *rast*, text *compression=''*, integer *srid=sameassource*);

bytea ST_AsTIFF(raster *rast*, integer[ ] *nbands*, text *compression=''*, integer *srid=sameassource*);

bytea ST_AsTIFF(raster *rast*, integer[ ] *nbands*, text[ ] *options*, integer *srid=sameassource*);

설명: 래스터에서 선택한 밴드들을 단일 TIFF(Tagged Image File Format) 이미지로 반환합니다. 밴드를 따로 설정하지 않을 경우, 모든 밴드를 쓰려 할 것입니다. 이 함수는 ST_AsGDALRaster 를 둘러싼 래퍼입니다. 덜 흔한 래스터 유형으로 내보내야 할 경우 ST_AsGDALRaster 함수를 이용하십시오. 어떤 공간 참조 SRS 텍스트도 없을 경우, 래스터의 공간 참조를 사용합니다. 이 함수에는 다음과 같은 많은 옵션을 가진 많은 변종이 있습니다:

• nbands - 내보낼 밴드들의 배열입니다(PNG 의 경우 3 이 최대값입니다). 밴드의 순서는 RGB 입니다. 예를 들어 ARRAY[3,2,1]은 밴드 3 을 빨간색, 밴드 2 를 초록색, 밴드 1 을 파란색에 매핑한다는 뜻입니다.

• compression - 압축 표현식: JPEG90(또는 다른 퍼센트), LZW, JPEG, DEFLATE9

• options - GTiff 에 대해 정의된 GDAL 생성 옵션들의 텍스트 배열입니다(ST_GDALDrivers 에서 GTiff 에 대한 create_options 를 살펴보십시오). 자세한 내용은 GDAL 래스터 형식 옵션 을 참조하십시오.

• srid - 래스터의 spatial_ref_sys 의 SRID 입니다. 이 옵션은 지리참조 정보를 채우는 데 쓰입니다.

2.0.0 버전부터 사용할 수 있습니다. GDAL 1.6.0 이상 버전이 필요합니다.

예시: JPEG 압축 90%

```
SELECT ST_AsTIFF(rast, 'JPEG90') As rasttiff
FROM dummy_rast WHERE rid=2;
```

참고: ST_GDALDrivers, ST_AsGDALRaster, ST_SRID

9.11. 래스터 처리

9.11.1. 맵 대수(algebra)

ST_Clip — 입력 도형으로 잘라낸 래스터를 반환합니다. 밴드 번호를 지정하지 않은 경우, 모든 밴드를 처리합니다. crop 을 설정하지 않거나 참으로 설정한 경우, 잘라낸 래스터를 출력합니다.

ST_ColorMap — 소스 래스터 및 설정한 밴드로부터 8BUI 밴드(grayscale, RGB, RGBA)를 4 개까지 가지는 새 래스터를 생성합니다. 밴드를 따로 설정하지 않으면 밴드 1 로 가정합니다.

ST_Intersection — 두 래스터의 공유 부분을 표현하는, 또는 벡터화된 래스터와 도형의 기하학적 교차를 표현하는 래스터 또는 도형-픽셀값 쌍의 집합을 반환합니다.

ST_MapAlgebra — 콜백 함수 버전 - 래스터 1 개 이상, 밴드 인덱스, 그리고 사용자 지정 콜백 함수 1 개를 입력받아 밴드 1 개를 가진 래스터를 반환합니다.

ST_MapAlgebra — 표현식 버전 - 입력 래스터 1 개 또는 2 개, 밴드 인덱스, 그리고 사용자 지정 SQL 표현식 1 개 이상을 입력받아 밴드 1 개를 가진 래스터를 반환합니다.

ST_MapAlgebraExpr — 래스터 밴드 1 개 버전: 입력 래스터에 대해 유효한 PostgreSQL 대수 연산을 적용해서 형성되고, 설정한 픽셀 유형을 가진, 밴드 1 개를 가진 새 래스터를 생성합니다. 따로 밴드를 설정하지 않을 경우, 밴드 1 로 가정합니다.

ST_MapAlgebraExpr — 래스터 밴드 2 개 버전: 입력 래스터 2 개에 대해 유효한 PostgreSQL 대수 연산을 적용해서 형성되고, 설정한 픽셀 유형을 가진, 밴드 1 개를 가진 새 래스터를 생성합니다. 따로 밴드를 설정하지 않을 경우, 각 래스터의 밴드 1 로 가정합니다. 출력 래스터는 첫 번째 래스터가 정의하는 그리드 상에 (축척, 기울기 및 픽셀 모서리가) 정렬될 것입니다. extenttype 파라미터가 출력 래스터의 범위를 정의할 것입니다. extenttype 의 값은 INTERSECTION, UNION, FIRST, SECOND 가 될 수 있습니다.

ST_MapAlgebraFct — 래스터 밴드 1 개 버전: 입력 래스터에 대해 유효한 PostgreSQL 대수 연산을 적용해서 형성되고, 설정한 픽셀 유형을 가진, 밴드 1 개를 가진 새 래스터를 생성합니다. 따로 밴드를 설정하지 않을 경우, 밴드 1 로 가정합니다.

ST_MapAlgebraFct — 래스터 밴드 2 개 버전: 입력 래스터 2 개에 대해 유효한 PostgreSQL 함수를 적용해서 형성되고, 설정한 픽셀 유형을 가진, 밴드 1 개를 가진 새 래스터를 생성합니다. 따로 밴드를 설정하지 않을 경우, 밴드 1 로 가정합니다. 범위 유형을 따로 설정하지 않을 경우 기본값은 INTERSECTION 입니다.

ST_MapAlgebraFctNgb — 래스터 밴드 1 개 버전: 사용자 지정 PostgreSQL 함수를 이용하는 맵 대수 최근접 이웃(Map Algebra Nearest Neighbor)입니다. 입력 래스터 밴드의 값의 이웃(neighborhood)이 관련된 PostgreSQL 사용자 함수가 출력하는 값을 가진 래스터를 반환합니다.

ST_Reclass — 원본으로부터 재분류된 밴드 유형으로 이루어진 새 래스터를 생성합니다. nband 는 변경할 밴드를 가리킵니다. nband 를 따로 설정하지 않을 경우 밴드 1 로 가정합니다. 다른 모든 밴드들은 변경 없이 반환됩니다. 실제 사례: 보기 좋은 형식으로 더 간단하게 렌더링하기 위해 16BUI 밴드를 8BUI 등등으로 변환하십시오.

ST_Union — 래스터 타일 집합을 1 개 이상의 밴드로 이루어진 단일 래스터로 통합합니다.

9.11.1.1. ST_Clip

ST_Clip — 입력 도형으로 잘라낸 래스터를 반환합니다. 밴드 번호를 지정하지 않은 경우, 모든 밴드를 처리합니다. crop 을 설정하지 않거나 참으로 설정한 경우, 잘라낸 래스터를 출력합니다.

Synopsis

raster ST_Clip(raster *rast*, integer[] *nband*, geometry *geom*, double precision[] *nodataval=NULL*, boolean *crop=TRUE*);

raster ST_Clip(raster *rast*, integer *nband*, geometry *geom*, double precision *nodataval*, boolean *crop=TRUE*);

raster ST_Clip(raster *rast*, integer *nband*, geometry *geom*, boolean *crop*);

raster ST_Clip(raster *rast*, geometry *geom*, double precision[] *nodataval=NULL*, boolean *crop=TRUE*);

raster ST_Clip(raster *rast*, geometry *geom*, double precision *nodataval*, boolean *crop=TRUE*);

raster ST_Clip(raster *rast*, geometry *geom*, boolean *crop*);

설명: 입력 도형 geom 으로 잘라낸 래스터를 반환합니다. 밴드 인덱스를 지정하지 않을 경우, 모든 밴드를 처리합니다.

ST_Clip 이 출력한 래스터는 각 밴드에 대해 잘라낸 면에 할당된 NODATA 값을 가지고 있어야 합니다. NODATA 값을 설정하지 않고 입력 래스터에 정의된 NODATA 값이 없을 경우, 출력 래스터의 NODATA 값을 ST_MinPossibleValue(ST_BandPixelType(rast, band))로 설정합니다. 배열 내부의 NODATA 값의 개수가 밴드 개수보다 작을 경우, 배열 안의 마지막 NODATA 값을 남은 밴드의 NODATA 값으로 씁니다. NODATA 값의 개수가 밴드 개수보다 클 경우, 남는 NODATA 값을 무시합니다. NODATA 값의 배열을 입력받는 모든 변종 함수는 각 밴드에 할당될 단일 값도 입력받습니다.

crop 을 설정하지 않을 경우, 참으로 가정합니다. geom 범위와 rast 범위가 교차하는 부분을 잘라낸 래스터를 출력한다는 의미입니다. crop 을 거짓으로 설정할 경우, 새 래스터의 범위는 rast 의 범위와 동일합니다.

2.0.0 버전부터 사용할 수 있습니다.

개선 사항: 2.1.0 버전에서 C 언어로 다시 쓰였습니다.

이 예시는 MassGIS 사이트의 MassGIS Aerial Orthos 에서 다운로드할 수 있는 매사추세츠 항공사진 데이터를 이용합니다. 매사추세츠 주 미터 단위 평면의 좌표로 돼 있습니다.

예시: 밴드 1 개 잘라내기

```
-- 항공사진 타일의 첫 번째 밴드를 20 미터 버퍼로 잘라내기
SELECT ST_Clip(rast, 1,
            ST_Buffer(ST_Centroid(ST_Envelope(rast)),20)
      ) from aerials.boston
WHERE rid = 4;

-- 래스터의 마지막 차원에 대한 잘라내기의 영향을 보여줍니다.
-- 마지막 범위를 어떻게 도형의 범위로 잘라내는지 눈여겨 보십시오.
-- crop = true 일 경우
SELECT ST_XMax(ST_Envelope(ST_Clip(rast, 1, clipper, true))) As xmax_w_trim,
       ST_XMax(clipper) As xmax_clipper,
       ST_XMax(ST_Envelope(ST_Clip(rast,   1,   clipper,   false)))   As
xmax_wo_trim,
       ST_XMax(ST_Envelope(rast)) As xmax_rast_orig
FROM (SELECT rast, ST_Buffer(ST_Centroid(ST_Envelope(rast)),6) As clipper
       FROM aerials.boston
WHERE rid = 6) As foo;

    xmax_w_trim    |   xmax_clipper   |   xmax_wo_trim   |  xmax_rast_orig
-------------------+------------------+------------------+-----------------
-
  230657.436173996 | 230657.436173996 | 230666.436173996 | 230666.436173996
```



잘라내기 전의 전체 래스터 타일



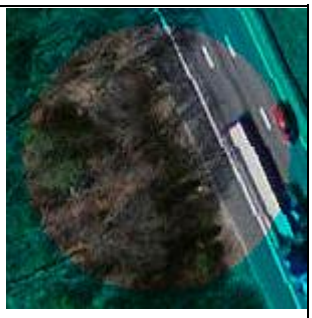
잘라낸 후

예시: crop 없이 밴드 1 개를 잘라낸 다음 다른 밴드들을 변경 없이 다시 추가하기

```
-- 이전과 동일한 예시지만, ST_AddBand 를 사용할 수 있으려면 crop 을 거짓으로 설정해야 합니다.
-- ST_AddBand 함수를 쓰려면 모든 밴드의 너비 및 높이가 동일해야 하기 때문입니다.
SELECT ST_AddBand(ST_Clip(rast, 1,
            ST_Buffer(ST_Centroid(ST_Envelope(rast)),20),false
      ), ARRAY[ST_Band(rast,2),ST_Band(rast,3)] ) from aerials.boston
WHERE rid = 6;
```



잘라내기 전의 전체 래스터 타일



잘라낸 후 - 비현실적

예시: 모든 밴드 잘라내기

```
-- 항공사진 타일의 모든 밴드를 20 미터 버퍼로 잘라내기
-- 차이점이라면 잘라낼 특정 밴드를 지정하지 않는다는 점뿐입니다.
-- 따라서 모든 밴드를 잘라냅니다.
SELECT ST_Clip(rast,
        ST_Buffer(ST_Centroid(ST_Envelope(rast)), 20),
        false
      ) from aerials.boston
WHERE rid = 4;
```



잘라내기 전의 전체 래스터 타일



잘라낸 후

참고: ST_AddBand, ST_MapAlgebra, ST_Intersection

9.11.1.2. ST_ColorMap

ST_ColorMap — 소스 래스터 및 설정한 밴드로부터 8BUI 밴드(grayscale, RGB, RGBA)를 4 개까지 가지는 새 래스터를 생성합니다. 밴드를 따로 설정하지 않으면 밴드 1 로 가정합니다.

Synopsis

raster ST_ColorMap(raster *rast*, integer *nband=1*, text *colormap=grayscale*, text *method=INTERPOLATE*);

raster ST_ColorMap(raster *rast*, text *colormap*, text *method=INTERPOLATE*);

설명: rast 의 nband 위치의 밴드에 colormap 을 적용해서 8BUI 밴드 4 개까지로 구성된 새 래스터를 출력합니다. colormap 안에 정의된 색상 구성 요소의 개수가 새 래스터의 8BUI 밴드 개수를 결정합니다.

nband 를 설정하지 않을 경우, 밴드 1 로 가정합니다.

미리 정의된 컬러맵의 키워드, 또는 값 및 색상 구성 요소를 정의하는 라인 집합이 colormap 이 될 수 있습니다.

유효한 미리 정의된 colormap 키워드:

- grayscale 또는 greyscale - 8BUI 밴드 래스터의 회색조(shades of gray)를 설정
- pseudocolor - 8BUI(RGBA) 밴드 4 개의 래스터에 대해 파랑에서 녹색으로, 녹색에서 빨강으로 변하는 색상을 설정
- fire - 8BUI(RGBA) 밴드 4 개의 래스터에 대해 검정에서 빨강으로, 빨강에서 연한 노랑으로 변하는 색상을 설정
- bluered - 8BUI(RGBA) 밴드 4 개의 래스터에 대해 파랑에서 연한 하양으로, 연한 하양에서 빨강으로 변하는 색상을 설정

사용자 지정 컬러맵을 설정하려면 사용자가 colormap 에 (한 줄에 하나씩 작성한) 항목들의 집합을 입력할 수 있습니다. 각 항목은 일반적으로 다음 5 개의 값으로 구성됩니다. 픽셀 값과 해당 픽셀의 빨강, 녹색, 파랑, 알파 구성 요소(RGBA)입니다(색상 구성 요소는 0 부터 255 사이의 값입니다). 픽셀 값 대신 해당 래스터 밴드의 최소/최대값이 0/100%인 백분율 값을 쓸 수도 있습니다. 각 값을 쉼표, 탭, 쌍점, 그리고/또는 공백으로 구분할 수 있습니다. 픽셀 값이 NODATA 값일 경우, nv, null 또는 nodata 로 설정할 수 있습니다. 다음은 그 예시입니다.

```
5 0 0 0 255
4 100:50 55 255
1 150,100 150 255
0% 255 255 255 255
nv 0 0 0 0
```

colormap 의 문법은 GDAL 의 색상 강조(color-relief) 모드 gdaldem 의 문법과 유사합니다.

유효한 method 키워드:

• INTERPOLATE - 입력한 픽셀 값들 사이의 색상을 부드럽게 섞여들게 하기 위해 선형 보간법을 이용합니다.

• EXACT - 컬러맵에 존재하는 픽셀 값과만 엄격히 매칭시킵니다. 컬러맵 항목과 일치하지 않는 값을 가진 픽셀은 0 0 0 0(RGBA)으로 설정할 것입니다.

• NEAREST - 픽셀 값과 가장 가까운 값을 가진 컬러맵 항목을 이용합니다.



컬러맵에 대해서는 ColorBrewer 를 참조하면 좋습니다.



그 결과 새 래스터의 밴드들은 NODATA 값을 가지지 않을 것입니다. NODATA 값이 필요하다면 ST_SetBandNoDataValue 를 이용해서 NODATA 값을 설정하십시오.

2.1.0 버전부터 사용할 수 있습니다.

예시

다음은 예시용 가짜 테이블입니다.

```
-- 테스트 래스터 테이블 작성 --
DROP TABLE IF EXISTS funky_shapes;
CREATE TABLE funky_shapes(rast raster);

INSERT INTO funky_shapes(rast)
WITH ref AS (
      SELECT ST_MakeEmptyRaster( 200, 200, 0, 200, 1, -1, 0, 0) AS rast
)
SELECT
      ST_Union(rast)
FROM (
      SELECT
            ST_AsRaster(
                  ST_Rotate(
                        ST_Buffer(
                              ST_GeomFromText('LINESTRING(0  2,50  50,150
150,125 50)'),
                              i*2
                        ),
                        pi() * i * 0.125, ST_Point(50,50)
                  ),
                  ref.rast, '8BUI'::text, i * 5
            ) AS rast
      FROM ref
```

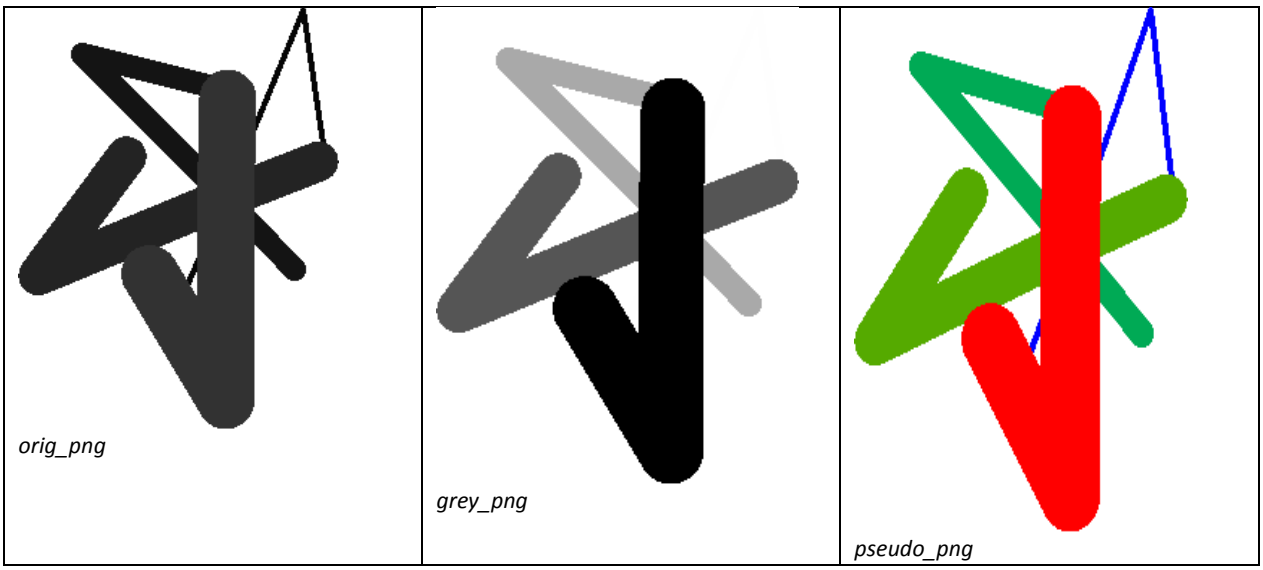
```
      CROSS JOIN generate_series(1, 10, 3) AS i
) AS shapes;

SELECT
      ST_NumBands(rast) As n_orig,
      ST_NumBands(ST_ColorMap(rast,1, 'greyscale')) As ngrey,
      ST_NumBands(ST_ColorMap(rast,1, 'pseudocolor')) As npseudo,
      ST_NumBands(ST_ColorMap(rast,1, 'fire')) As nfire,
      ST_NumBands(ST_ColorMap(rast,1, 'bluered')) As nbluered,
      ST_NumBands(ST_ColorMap(rast,1, '
100% 255   0   0
 80% 160   0   0
 50% 130   0   0
 30%  30   0   0
 20%  60   0   0
  0%   0   0   0
  nv 255 255 255
      )) As nred
FROM funky_shapes;

n_orig | ngrey | npseudo | nfire | nbluered | nred
--------+-------+---------+-------+----------+------
     1 |     1 |       4 |     4 |        4 |    3
```

예시: ST_AsPNG 를 이용해서 서로 다른 컬러맵의 외양을 비교하기

```
SELECT
      ST_AsPNG(rast) As orig_png,
      ST_AsPNG(ST_ColorMap(rast,1,'greyscale')) As grey_png,
      ST_AsPNG(ST_ColorMap(rast,1, 'pseudocolor')) As pseudo_png,
      ST_AsPNG(ST_ColorMap(rast,1, 'nfire')) As fire_png,
      ST_AsPNG(ST_ColorMap(rast,1, 'bluered')) As bluered_png,
      ST_AsPNG(ST_ColorMap(rast,1, '
100% 255   0   0
 80% 160   0   0
 50% 130   0   0
 30%  30   0   0
 20%  60   0   0
  0%   0   0   0
  nv 255 255 255
      )) As red_png
FROM funky_shapes;
```



참고: ST_AsPNG, ST_AsRaster ST_MapAlgebra, ST_NumBands, ST_Reclass, ST_SetBandNoDataValue, ST_Union

9.11.1.3. ST_Intersection

ST_Intersection — 두 래스터의 공유 부분을 표현하는, 또는 벡터화된 래스터와 도형의 기하학적 교차를 표현하는 래스터 또는 도형-픽셀값 쌍의 집합을 반환합니다.

Synopsis

setof geomval ST_Intersection(geometry *geom*, raster *rast*, integer *band_num=1*);

setof geomval ST_Intersection(raster *rast*, geometry *geom*);

setof geomval ST_Intersection(raster *rast*, integer *band*, geometry *geomin*);

raster ST_Intersection(raster *rast1*, raster *rast2*, double precision[] *nodataval*);

raster ST_Intersection(raster *rast1*, raster *rast2*, text *returnband*, double precision[] *nodataval*);

raster ST_Intersection(raster *rast1*, integer *band1*, raster *rast2*, integer *band2*, double precision[] *nodataval*);

raster ST_Intersection(raster *rast1*, integer *band1*, raster *rast2*, integer *band2*, text *returnband*, double precision[] *nodataval*);

설명: 두 래스터의 공유 부분을 표현하는, 또는 벡터화된 래스터와 도형의 기하학적 교차를 표현하는 래스터 또는 도형-픽셀값 쌍의 집합을 반환합니다.

geomval 집합을 반환하는 처음 세 변종 함수는 벡터 스페이스에서 작동합니다. 먼저 (ST_DumpAsPolygon 을 통해) 래스터를 geomval 행의 집합으로 벡터화한 다음, 해당 행을 ST_Intersection(geometry, geometry) PostGIS 함수를 이용해서 도형과 교차시킵니다. 도형이 래스터의 NODATA 값 부분과만 교차할 경우, 텅 빈 도형을 반환합니다. WHERE 절에 ST_Intersect 함수를 적절히 사용하면, 일반적으로 결과에서 이런 경우들을 제외시킬 수 있습니다.

출력되는 geomval 집합 앞뒤에 괄호를 치고 표현식 끝에 '.geom' 또는 '.val'을 붙이면 도형 및 값 부분에 접근할 수 있습니다. 예: (ST_Intersection(rast, geom)).geom

래스터를 반환하는 다른 변종들은 래스터 스페이스에서 작동합니다. 이 변종들은 교차를 수행하기 위해 ST_MapAlgebraExpr 함수의 래스터 2 개를 입력받는 버전을 이용합니다.

출력 래스터의 범위는 두 래스터 범위의 기하학적 교차 부분과 일치합니다. 출력 래스터는 returnband 파라미터가 어떻게 설정됐는지에 따라 'BAND1', 'BAND2' 또는 'BOTH' 밴드를 포함합니다. 어느 한 밴드에서 NODATA 값을 가진 부분은 출력물의 모든 밴드에서 NODATA 값을 가지게 됩니다. 즉, NODATA 값을 가진 픽셀과 교차하는 픽셀을 모두 NODATA 값을 가진 픽셀로 출력합니다.

ST_Intersection 함수가 출력하는 래스터는 교차하지 않는 부분에 할당된 NODATA 값을 가지고 있어야 합니다. 사용자가 'BAND1', 'BAND2' 또는 'BOTH' 밴드 가운데 어떤 것을 요청하느냐에 따라 1 개 또는 2 개의 NODATA 값을 가진 nodataval[] 배열을 입력해서 어떤 출력 밴드에 대해서도 NODATA 값을 정의하거나 대체할 수 있습니다. 배열의 첫 번째 값은 첫 번째 밴드의 NODATA 값을 대체하고, 두 번째 값은 두 번째 밴드의 NODATA 값을 대체합니다. 만약 입력 밴드 가운데 하나에 정의된 NODATA 값이 없고 입력된 NODATA 값 배열도 없을 경우, ST_MinPossibleValue 함수를 통해 NODATA 값을 선택합니다. NODATA 값의 배열을 입력받는 모든 변종 함수는 요청된 각 밴드에 할당될 단일 값도 입력받습니다.

모든 변종 함수에서, 밴드 번호를 따로 지정하지 않을 경우 밴드 1 로 가정합니다. 래스터와 도형 사이의 교차 부분을 래스터로 반환받아야 할 경우, ST_Clip 을 참조하십시오.



출력 범위 또는 NODATA 값을 맞닥뜨렸을 때 무엇을 반환할지를 더 세밀히 조정하려면, ST_MapAlgebraExpr 함수의 두 래스터를 입력받는 버전을 이용하십시오.



래스터 스페이스에서 래스터 밴드와 도형의 교차 부분을 계산하려면, ST_Clip 을 이용하십시오. ST_Clip 함수는 복수의 밴드를 가진 래스터를 입력받으며, 래스터화된 도형과 일치하는 밴드를 반환하지 않습니다.



ST_Intersects 및 래스터에 대한 인덱스 그리고/또는 도형 열과 결합해서 ST_Intersection 함수를 사용해야 합니다.

개선 사항: 2.0.0 부터 래스터 스페이스에서 교차 부분을 구할 수 있습니다. 2.0.0 미만 버전에서는, 벡터 스페이스에서만 교차 작업을 수행할 수 있었습니다.

예시: 도형, 래스터 -- 도형-값 쌍으로 출력

```
  SELECT
        foo.rid,
        foo.gid,
        ST_AsText((foo.geomval).geom) As geomwkt,
        (foo.geomval).val
  FROM (
        SELECT
              A.rid,
              g.gid,
              ST_Intersection(A.rast, g.geom) As geomval
        FROM dummy_rast AS A
        CROSS JOIN (
              VALUES
                    (1, ST_Point(3427928, 5793243.85) ),
                    (2,               ST_GeomFromText('LINESTRING(3427927.85
5793243.75,3427927.8 5793243.75,3427927.8 5793243.8)')),
                    (3, ST_GeomFromText('LINESTRING(1 2, 3 4)'))
        ) As g(gid,geom)
        WHERE A.rid = 2
  ) As foo;

  rid | gid |   geomwkt                                                                  | val
 -----+-----+----------------------------------------------------------------------------+-----
   2 |  1 | POINT(3427928 5793243.85)                                                  | 249
   2 |  1 | POINT(3427928 5793243.85)                                                  | 253
   2 |  2 | POINT(3427927.85 5793243.75)                                               | 254
   2 |  2 | POINT(3427927.8 5793243.8)                                                 | 251
   2 |  2 | POINT(3427927.8 5793243.8)                                                 | 253
   2 |  2 | LINESTRING(3427927.8 5793243.75,3427927.8 5793243.8)                       | 252
   2 |  2 | MULTILINESTRING((3427927.8 5793243.8,3427927.8 5793243.75),...)            | 250
   2 |  3 | GEOMETRYCOLLECTION EMPTY
```

참고: geomval, ST_Intersects, ST_MapAlgebraExpr, ST_Clip, ST_AsText

9.11.1.4. ST_MapAlgebra

ST_MapAlgebra — 콜백 함수 버전 - 래스터 1 개 이상, 밴드 인덱스, 그리고 사용자 지정 콜백 함수 1 개를 입력받아 밴드 1 개를 가진 래스터를 반환합니다.

Synopsis

raster ST_MapAlgebra(rastbandarg[] *rastbandargset*, regprocedure *callbackfunc*, text *pixeltype=NULL*, text *extenttype=INTERSECTION*, raster *customextent=NULL*, integer *distancex=0*, integer *distancey=0*, text[] *VARIADIC userargs=NULL*);

raster ST_MapAlgebra(raster *rast*, integer[] *nband*, regprocedure *callbackfunc*, text *pixeltype=NULL*, text *extenttype=FIRST*, raster *customextent=NULL*, integer *distancex=0*, integer *distancey=0*, text[] *VARIADIC userargs=NULL*);

raster ST_MapAlgebra(raster *rast*, integer *nband*, regprocedure *callbackfunc*, text *pixeltype=NULL*, text *extenttype=FIRST*, raster *customextent=NULL*, integer *distancex=0*, integer *distancey=0*, text[] *VARIADIC userargs=NULL*);

raster ST_MapAlgebra(raster *rast1*, integer *nband1*, raster *rast2*, integer *nband2*, regprocedure *callbackfunc*, text *pixeltype=NULL*, text *extenttype=INTERSECTION*, raster *customextent=NULL*, integer *distancex=0*, integer *distancey=0*, text[] *VARIADIC userargs=NULL*);

raster ST_MapAlgebra(nband *integer*, regprocedure *callbackfunc*, float8[] *mask*, boolean *weighted*, text *pixeltype=NULL*, text *extenttype=INTERSECTION*, raster *customextent=NULL*, text[] *VARIADIC userargs=NULL*);

설명: 래스터 1 개 이상, 밴드 인덱스, 그리고 사용자 지정 콜백 함수 1 개를 입력받아 밴드 1 개를 가진 래스터를 반환합니다.

rast, rast1, rast2, rastbandargset — 맵 대수(代數) 처리를 평가하는 데 쓰이는 래스터

rastbandargset 은 많은 래스터 그리고/또는 많은 밴드에 대해 맵 대수 연산을 이용할 수 있도록 해줍니다. 변종 1 예시를 참조하십시오.

nband, nband1, nband2 — 평가할 래스터의 밴드 개수입니다. nband 는 밴드를 나타내는 정수 또는 정수 배열이 될 수 있습니다. nband1 은 rast1 에 있는 밴드이며 nband2 는 래스터 2 개/밴드 2 개일 경우 rast2 에 있는 밴드입니다.

Callbackfunc — callbackfunc 파라미터는 regprocedure(회귀 처리)로 형변환된 SQL 또는 PL/pgSQL 함수의 명칭 및 서명(signature)이어야 합니다. 다음은 PL/pgSQL 함수의 예시입니다:

```
CREATE OR REPLACE FUNCTION sample_callbackfunc(value double precision[ ][ ][ ], position integer[ ][ ], VARIADIC userargs text[ ])
RETURNS double precision
AS $$
BEGIN
RETURN 0;
END;
$$ LANGUAGE 'plpgsql' IMMUTABLE;
```

callbackfunc 는 인수를 3 개 가지고 있어야만 합니다: 이중 정밀도 데이터형 3 차원 배열, 정수형 2 차원 배열, 그리고 임의로 여러 인수를 입력받는 문자열 1 차원 배열입니다. 첫 번째 인수 value 는 모든 입력 래스터에서 나온 (이중 정밀도) 값들의 집합입니다. (인덱스가 1-기반인) 3 차원이란 래스터 번호, Y 행, X 열을 말합니다. 두 번째 인수 position 은 출력 래스터 및 입력 래스터의 픽셀 위치들의 집합입니다. (인덱스가 0-기반인) 외곽 차원은 래스터 번호입니다. 외곽 차원 인덱스 0 에 있는 위치는 출력 래스터의 픽셀 위치입니다. 각 외곽 차원에 대해, 내곽 차원에 X 및 Y 를 위한 두 요소가 있습니다. 세 번째 인수 userargs 는 어떤 사용자 지정 인수라도 넘겨주기 위한 것입니다.

regprocedure 인수를 SQL 함수에 입력하려면 입력하기 위한 전체 함수 서명이 필요하고, 그 다음 regprocedure 유형으로 형변환해야 합니다. 앞의 PL/pgSQL 함수를 인수로서 넘겨주려면, 해당 인수를 위한 SQL 은 다음과 같습니다:

```
'sample_callbackfunc(double precision[ ], integer[ ], text[ ])'::regprocedure
```

해당 인수가 함수의 명칭, 함수 인수들의 유형, 명칭 및 인수 유형을 감싼 따옴표, 그리고 regprocedure 로의 형변환을 포함하고 있다는 점에 주의하십시오.

mask — 어떤 셀을 맵 대수 콜백 함수에 넘길지 필터링하는 데 쓰이는 숫자의 N 차원 배열(행렬)입니다. 0 은 이웃 셀의 값을 NODATA 로 취급해야 한다는 뜻이고, 1 은 데이터로 취급해야 한다는 뜻입니다.

가중치(weight)를 참으로 설정할 경우, 값들이 이웃 위치에 있는 해당 값의 픽셀 값을 곱하기 위한 곱수로 쓰입니다.

weighted — mask 값에 가중치를 적용해야 할지(원래 값으로 곱해야 할지) 말지(mask 를 입력받는 최초 버전에만 적용할지)를 표시하는 불 값(참/거짓)입니다.

pixeltype — pixeltype 을 설정할 경우, 새 래스터의 밴드 하나가 해당 픽셀 유형이 될 것입니다. pixeltype 이 NULL 이거나 생략된 경우, 새 래스터 밴드가 첫 번째 래스터의 지정된 밴드와(범위 유형의 경우: INTERSECTION, UNION, FIRST, CUSTOM), 또는 적절한 래스터의 지정된 밴드와(범위 유형의 경우: SECOND, LAST) 동일한 픽셀 유형을 가지게 될 것입니다. 어떤 유형인지 확신하지 못 한다면, 언제나 pixeltype 을 설정하십시오.

출력 래스터의 픽셀 유형은 ST_BandPixelType 목록에 존재하는 유형 가운데 하나이거나, 생략되거나, NULL 로 설정돼야만 합니다.

extenttype — 사용할 수 있는 값은 INTERSECTION(기본값), UNION, FIRST(래스터 1 개를 입력받는 변종들의 기본값), SECOND, LAST, CUSTOM 입니다.

customextent — extentype 이 CUSTOM 일 경우, 래스터가 customextent 를 입력받아야만 합니다. 변종 1 의 4 번째 예시를 참조하십시오.

distance — 참조 셀에서 나온 픽셀 단위의 거리입니다. 따라서 결과 매트릭스의 너비는 2*distancex + 1 일 것입니다. 따로 설정하지 않으면 참조 셀만 (이웃 셀의 개수가 0) 고려합니다.

distance — 참조 셀에서 나온 Y 방향의 픽셀 단위 거리입니다. 결과 매트릭스의 높이는 2*distancey + 1 일 것입니다. 따로 설정하지 않으면 참조 셀만 (이웃 셀의 개수가 0) 고려합니다.

userargs — callbackfunc 에 들어가는 세 번째 인수는 variadic text 배열입니다. 길고 긴 텍스트 인수들이 모두 지정된 callbackfunc 에 넘겨지며, userargs 인수에 담겨집니다.



(다양한 개수의 인수를 입력받는) VARIADIC 키워드에 대한 더 자세한 정보를 알고 싶다면, PostgreSQL 문서 가운데 Query Language (SQL) Functions 의 "SQL Functions with Variable Numbers of Arguments" 단원을 참조하십시오.



사용자가 처리하기 위해 콜백 함수에 어떤 인수를 넘겨주기로 하고 말고에 상관없이, callbackfunc 에 들어가는 text[] 인수는 필요합니다.

변종 1 은 많은 래스터 그리고/또는 많은 밴드에 대해 맵 대수 연산을 이용할 수 있도록 해주는 rastbandarg 배열을 입력받습니다. 변종 1 예시를 참조하십시오.

변종 2 및 3 은 한 래스터의 1 개 이상의 밴드에 대해 연산합니다. 변종 2 및 3 의 예시를 참조하십시오.

변종 4 는 각 래스터가 밴드 1 개씩 가지고 있는 래스터 2 개에 대해 연산합니다. 변종 4 예시를 참조하십시오.

2.2.0 버전부터 mask 를 추가할 수 있습니다.

2.1.0 버전부터 사용할 수 있습니다.

예시: 변종 1

래스터 1 개, 밴드 1 개

```
WITH foo AS (
    SELECT 1 AS rid, ST_AddBand(ST_MakeEmptyRaster(2, 2, 0, 0, 1, -1, 0, 0, 0), 1, '16BUI',
1, 0) AS rast
)
SELECT
    ST_MapAlgebra(
        ARRAY[ROW(rast, 1)]::rastbandarg[],
        'sample_callbackfunc(double precision[], int[], text[])'::regprocedure
    ) AS rast
FROM foo
```

래스터 1 개, 밴드 몇 개

```
WITH foo AS (
    SELECT 1 AS rid, ST_AddBand(ST_AddBand(ST_AddBand(ST_MakeEmptyRaster(2, 2, 0, 0, 1, -1,
0, 0, 0), 1, '16BUI', 1, 0), 2, '8BUI', 10, 0), 3, '32BUI', 100, 0) AS rast
)
SELECT
    ST_MapAlgebra(
        ARRAY[ROW(rast, 3), ROW(rast, 1), ROW(rast, 3), ROW(rast, 2)]::rastbandarg[],
        'sample_callbackfunc(double precision[], int[], text[])'::regprocedure
    ) AS rast
FROM foo
```

래스터 몇 개, 밴드 몇 개

```
WITH foo AS (
    SELECT 1 AS rid, ST_AddBand(ST_AddBand(ST_AddBand(ST_MakeEmptyRaster(2, 2, 0, 0, 1, -1,
0, 0, 0), 1, '16BUI', 1, 0), 2, '8BUI', 10, 0), 3, '32BUI', 100, 0) AS rast UNION ALL
    SELECT 2 AS rid, ST_AddBand(ST_AddBand(ST_AddBand(ST_MakeEmptyRaster(2, 2, 0, 1, 1, -1,
0, 0, 0), 1, '16BUI', 2, 0), 2, '8BUI', 20, 0), 3, '32BUI', 300, 0) AS rast
)
SELECT
    ST_MapAlgebra(
        ARRAY[ROW(t1.rast,   3),   ROW(t2.rast,   1),   ROW(t2.rast,   3),   ROW(t1.rast,
2)]::rastbandarg[],
        'sample_callbackfunc(double precision[], int[], text[])'::regprocedure
    ) AS rast
FROM foo t1
CROSS JOIN foo t2
WHERE t1.rid = 1
    AND t2.rid = 2
```

이웃을 가진 커버리지의 타일들의 완전한 예시입니다. 이 쿼리는 PostgreSQL 9.1 이상 버전에서만 작동합니다.

```
WITH foo AS (
      SELECT 0 AS rid, ST_AddBand(ST_MakeEmptyRaster(2, 2, 0, 0, 1, -1, 0, 0, 0), 1,
'16BUI', 1, 0) AS rast UNION ALL
      SELECT 1, ST_AddBand(ST_MakeEmptyRaster(2, 2, 2, 0, 1, -1, 0, 0, 0), 1, '16BUI',
2, 0) AS rast UNION ALL
      SELECT 2, ST_AddBand(ST_MakeEmptyRaster(2, 2, 4, 0, 1, -1, 0, 0, 0), 1, '16BUI',
3, 0) AS rast UNION ALL

      SELECT 3, ST_AddBand(ST_MakeEmptyRaster(2, 2, 0, -2, 1, -1, 0, 0, 0), 1, '16BUI',
10, 0) AS rast UNION ALL
      SELECT 4, ST_AddBand(ST_MakeEmptyRaster(2, 2, 2, -2, 1, -1, 0, 0, 0), 1, '16BUI',
20, 0) AS rast UNION ALL
```

```
      SELECT 5, ST_AddBand(ST_MakeEmptyRaster(2, 2, 4, -2, 1, -1, 0, 0, 0), 1, '16BUI',
30, 0) AS rast UNION ALL

      SELECT 6, ST_AddBand(ST_MakeEmptyRaster(2, 2, 0, -4, 1, -1, 0, 0, 0), 1, '16BUI',
100, 0) AS rast UNION ALL
      SELECT 7, ST_AddBand(ST_MakeEmptyRaster(2, 2, 2, -4, 1, -1, 0, 0, 0), 1, '16BUI',
200, 0) AS rast UNION ALL
      SELECT 8, ST_AddBand(ST_MakeEmptyRaster(2, 2, 4, -4, 1, -1, 0, 0, 0), 1, '16BUI',
300, 0) AS rast
)
SELECT
      t1.rid,
      ST_MapAlgebra(
            ARRAY[ROW(ST_Union(t2.rast), 1)]::rastbandarg[],
            'sample_callbackfunc(double precision[], int[], text[])'::regprocedure,
            '32BUI',
            'CUSTOM', t1.rast,
            1, 1
      ) AS rast
FROM foo t1
CROSS JOIN foo t2
WHERE t1.rid = 4
      AND t2.rid BETWEEN 0 AND 8
      AND ST_Intersects(t1.rast, t2.rast)
GROUP BY t1.rid, t1.rast
```

이웃을 가진 커버리지의 타일들의 예시와 유사하지만 PostgreSQL 9.0 에서 작동하는 쿼리입니다.

```
WITH src AS (
      SELECT 0 AS rid, ST_AddBand(ST_MakeEmptyRaster(2, 2, 0, 0, 1, -1, 0, 0, 0), 1,
'16BUI', 1, 0) AS rast UNION ALL
      SELECT 1, ST_AddBand(ST_MakeEmptyRaster(2, 2, 2, 0, 1, -1, 0, 0, 0), 1, '16BUI',
2, 0) AS rast UNION ALL
      SELECT 2, ST_AddBand(ST_MakeEmptyRaster(2, 2, 4, 0, 1, -1, 0, 0, 0), 1, '16BUI',
3, 0) AS rast UNION ALL

      SELECT 3, ST_AddBand(ST_MakeEmptyRaster(2, 2, 0, -2, 1, -1, 0, 0, 0), 1, '16BUI',
10, 0) AS rast UNION ALL
      SELECT 4, ST_AddBand(ST_MakeEmptyRaster(2, 2, 2, -2, 1, -1, 0, 0, 0), 1, '16BUI',
20, 0) AS rast UNION ALL
      SELECT 5, ST_AddBand(ST_MakeEmptyRaster(2, 2, 4, -2, 1, -1, 0, 0, 0), 1, '16BUI',
30, 0) AS rast UNION ALL

      SELECT 6, ST_AddBand(ST_MakeEmptyRaster(2, 2, 0, -4, 1, -1, 0, 0, 0), 1, '16BUI',
100, 0) AS rast UNION ALL
      SELECT 7, ST_AddBand(ST_MakeEmptyRaster(2, 2, 2, -4, 1, -1, 0, 0, 0), 1, '16BUI',
200, 0) AS rast UNION ALL
      SELECT 8, ST_AddBand(ST_MakeEmptyRaster(2, 2, 4, -4, 1, -1, 0, 0, 0), 1, '16BUI',
300, 0) AS rast
)
WITH foo AS (
      SELECT
            t1.rid,
            ST_Union(t2.rast) AS rast
      FROM src t1
      JOIN src t2
            ON ST_Intersects(t1.rast, t2.rast)
            AND t2.rid BETWEEN 0 AND 8
      WHERE t1.rid = 4
      GROUP BY t1.rid
), bar AS (
      SELECT
            t1.rid,
            ST_MapAlgebra(
                  ARRAY[ROW(t2.rast, 1)]::rastbandarg[],
                  'raster_nmapalgebra_test(double        precision[],        int[],
text[])'::regprocedure,
                  '32BUI',
```

```
                    'CUSTOM', t1.rast,
                    1, 1
              ) AS rast
       FROM src t1
       JOIN foo t2
              ON t1.rid = t2.rid
)
SELECT
       rid,
       (ST_Metadata(rast)),
       (ST_BandMetadata(rast, 1)),
       ST_Value(rast, 1, 1, 1)
FROM bar;
```

예시: 변종 2 및 3

래스터 1 개, 밴드 몇 개

```
WITH foo AS (
      SELECT 1 AS rid, ST_AddBand(ST_AddBand(ST_AddBand(ST_MakeEmptyRaster(2, 2, 0, 0, 1,
-1, 0, 0, 0), 1, '16BUI', 1, 0), 2, '8BUI', 10, 0), 3, '32BUI', 100, 0) AS rast
)
SELECT
       ST_MapAlgebra(
              rast, ARRAY[3, 1, 3, 2]::integer[],
              'sample_callbackfunc(double precision[], int[], text[])'::regprocedure
       ) AS rast
FROM foo
```

래스터 1 개, 밴드 1 개

```
WITH foo AS (
      SELECT 1 AS rid, ST_AddBand(ST_AddBand(ST_AddBand(ST_MakeEmptyRaster(2, 2, 0,
0, 1, -1, 0, 0, 0), 1, '16BUI', 1, 0), 2, '8BUI', 10, 0), 3, '32BUI', 100, 0) AS rast
)
SELECT
       ST_MapAlgebra(
              rast, 2,
              'sample_callbackfunc(double precision[], int[], text[])'::regprocedure
       ) AS rast
FROM foo
```

예시: 변종 4

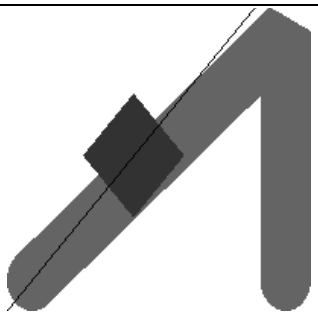
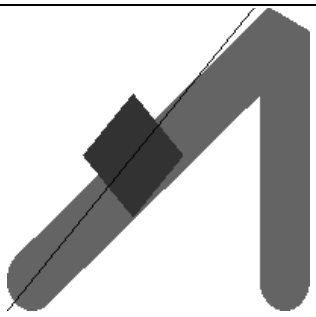
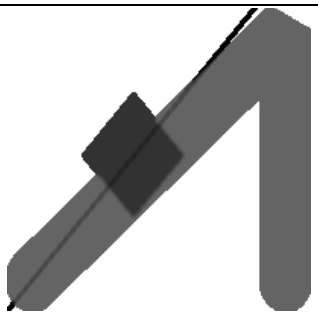
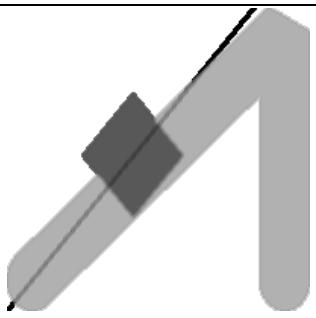
래스터 2 개, 밴드 2 개

```
WITH foo AS (
      SELECT 1 AS rid, ST_AddBand(ST_AddBand(ST_AddBand(ST_MakeEmptyRaster(2, 2, 0,
0, 1, -1, 0, 0, 0), 1, '16BUI', 1, 0), 2, '8BUI', 10, 0), 3, '32BUI', 100, 0) AS rast
UNION ALL
      SELECT 2 AS rid, ST_AddBand(ST_AddBand(ST_AddBand(ST_MakeEmptyRaster(2, 2, 0,
1, 1, -1, 0, 0, 0), 1, '16BUI', 2, 0), 2, '8BUI', 20, 0), 3, '32BUI', 300, 0) AS rast
)
SELECT
       ST_MapAlgebra(
              t1.rast, 2,
              t2.rast, 1,
              'sample_callbackfunc(double precision[], int[], text[])'::regprocedure
       ) AS rast
FROM foo t1
CROSS JOIN foo t2
WHERE t1.rid = 1
       AND t2.rid = 2
```

예시: **mask** 사용하기

```
WITH foo AS (SELECT
   ST_SetBandNoDataValue(
ST_SetValue(ST_SetValue(ST_AsRaster(
             ST_Buffer(
                   ST_GeomFromText('LINESTRING(50      50,100      90,100      50)'),
5,'join=bevel'),
                   200,200,ARRAY['8BUI'], ARRAY[100], ARRAY[0]), ST_Buffer('POINT(70
70)'::geometry,10,'quad_segs=1') ,50),
  'LINESTRING(20 20, 100 100, 150 98)'::geometry,1),0)  AS rast )
SELECT 'original' AS title, rast
FROM foo
UNION ALL
SELECT  'no  mask  mean  value'  AS  title,  ST_MapAlgebra(rast,1,'ST_mean4ma(double
precision[], int[], text[])'::regprocedure) AS rast
FROM foo
UNION ALL
SELECT   'mask   only   consider   neighbors,   exclude   center'   AS   title,
ST_MapAlgebra(rast,1,'ST_mean4ma(double precision[], int[], text[])'::regprocedure,
    '{{1,1,1}, {1,0,1}, {1,1,1}}'::double precision[], false) As rast
FROM foo

  UNION ALL
  SELECT 'mask weighted only consider neighbors, exclude center multi otehr pixel
values by 2' AS title, ST_MapAlgebra(rast,1,'ST_mean4ma(double precision[], int[],
text[])'::regprocedure,
     '{{2,2,2}, {2,0,2}, {2,2,2}}'::double precision[], true) As rast
  FROM foo;
```

<table>
<tr>
<td> 
원본</td>
<td> 
mask 평균값 없음(mask 매트릭스의 숫자가 모두 1 인 것과 동일)</td>
</tr>
<tr>
<td> 
mask 가 중심을 제외한 이웃만 고려</td>
<td> 
mask 가 중심을 제외한 다른 픽셀 값에 2 를 곱해 이웃에만 가중치를 부여</td>
</tr>
</table>

참고: rastbandarg, ST_Union, ST_MapAlgebra

9.11.1.5. ST_MapAlgebra

ST_MapAlgebra — 표현식 버전 - 입력 래스터 1 개 또는 2 개, 밴드 인덱스, 그리고 사용자 지정 SQL 표현식 1 개 이상을 입력받아 밴드 1 개를 가진 래스터를 반환합니다.

Synopsis

raster **ST_MapAlgebra**(raster *rast*, integer *nband*, text *pixeltype*, text *expression*, double precision *nodataval=NULL*);

raster **ST_MapAlgebra**(raster *rast*, text *pixeltype*, text *expression*, double precision *nodataval=NULL*);

raster **ST_MapAlgebra**(raster *rast1*, integer *nband1*, raster *rast2*, integer *nband2*, text *expression*, text *pixeltype=NULL*, text *extenttype=INTERSECTION*, text *nodata1expr=NULL*, text *nodata2expr=NULL*, double precision *nodatanodataval=NULL*);

raster **ST_MapAlgebra**(raster *rast1*, raster *rast2*, text *expression*, text *pixeltype=NULL*, text *extenttype=INTERSECTION*, text *nodata1expr=NULL*, text *nodata2expr=NULL*, double precision *nodatanodataval=NULL*);

설명: 표현식 버전 - 입력 래스터 1 개 또는 2 개, 밴드 인덱스, 그리고 사용자 지정 SQL 표현식 1 개 이상을 입력받아 밴드 1 개를 가진 래스터를 반환합니다.

2.1.0 버전부터 사용할 수 있습니다.

설명: 변종 1, 2 (래스터 1 개)

입력 래스터(rast)에 대해 expression 이 정의하는 유효한 PostgreSQL 대수 연산을 적용해서 형성된, 밴드 1 개를 가진 새 래스터를 생성합니다. nband 를 설정하지 않을 경우, 밴드 1 로 가정합니다. 이 새 래스터는 원본 래스터와 동일한 지리참조, 너비 및 높이이지만, 밴드는 1 개만 가질 것입니다.

pixeltype 을 설정할 경우, 새 래스터의 밴드가 해당 픽셀 유형이 될 것입니다. pixeltype 이 NULL 일 경우, 새 래스터의 밴드는 입력 rast 의 밴드와 동일한 픽셀 유형이 될 것입니다.

- expression 에 키워드를 쓸 수 있습니다.
    1. [rast] - 관심 픽셀의 픽셀 값
    2. [rast.val] - 관심 픽셀의 픽셀 값
    3. [rast.x] - 관심 픽셀의 1-기반 픽셀 열
    4. [rast.y] - 관심 픽셀의 1-기반 픽셀 행

설명: 변종 3, 4 (래스터 2 개)

입력 래스터 밴드 rast1, (rast2)에 대한 expression 이 정의하는 밴드 2 개에 대해, 유효한 PostgreSQL 대수 연산을 적용해서 형성된, 밴드 1 개를 가진 새 래스터를 생성합니다. band1, band2 를 설정하지 않을 경우, 밴드 1 로 가정합니다. 출력 래스터는 첫 번째 래스터가 정의하는 그리드 상에 (축척, 기울기 및 픽셀 모서리가) 정렬될 것입니다. extenttype 파라미터가 출력 래스터의 범위를 정의할 것입니다.

표현식(expression) — 래스터 2 개가 관련된 PostgreSQL 대수 표현식 및 픽셀들이 교차할 경우 픽셀 값을 정의할 PostgreSQL 정의 함수/연산자입니다. 예: (([rast1] + [rast2])/2.0)::integer

pixeltype — 출력 래스터의 픽셀 유형입니다. 이 유형은 ST_BandPixelType 목록에 존재하는 유형 가운데 하나이거나, 생략되거나, NULL 로 설정돼야만 합니다. 따로 설정하지 않거나 NULL 로 설정하지 않으면, 첫 번째 래스터의 픽셀 유형을 기본값으로 삼을 것입니다.

extenttype — 출력 래스터의 범위 제어

1. INTERSECTION - 새 래스터의 범위는 두 래스터의 교차 부분입니다. 기본값입니다.
2. UNION - 새 래스터의 범위는 두 래스터를 통합한 범위입니다.
3. FIRST - 새 래스터의 범위는 첫 번째 래스터의 범위와 동일합니다.
4. SECOND - 새 래스터의 범위는 두 번째 래스터의 범위와 동일합니다.

nodata1expr — rast1 의 픽셀들이 NODATA 값이고 공간적으로 상응하는 rast2 의 픽셀들이 값을 가지고 있을 때 어떤 것을 반환할지 정의하는 상수만, 또는 rast2 와만 관련된 대수 표현식입니다.

nodata2expr — rast2 의 픽셀들이 NODATA 값이고 공간적으로 상응하는 rast1 의 픽셀들이 값을 가지고 있을 때 어떤 것을 반환할지 정의하는 상수만, 또는 rast1 과만 관련된 대수 표현식입니다.

nodatanodataval — 공간적으로 상응하는 rast1 및 rast2 의 픽셀들이 모두 NODATA 값일 경우 반환하는 숫자 상수입니다.

- expression, nodata1expr 및 nodata2expr 에 키워드를 쓸 수 있습니다.
  1. [rast1] - rast1 에 있는 관심 픽셀의 픽셀 값
  2. [rast1.val] - rast1 에 있는 관심 픽셀의 픽셀 값
  3. [rast1.x] - rast1 에 있는 관심 픽셀의 1-기반 픽셀 열
  4. [rast1.y] - rast1 에 있는 관심 픽셀의 1-기반 픽셀 행
  5. [rast2] - rast2 에 있는 관심 픽셀의 픽셀 값
  6. [rast2.val] - rast2 에 있는 관심 픽셀의 픽셀 값
  7. [rast2.x] - rast2 에 있는 관심 픽셀의 1-기반 픽셀 열
  8. [rast2.y] - rast2 에 있는 관심 픽셀의 1-기반 픽셀 행

예시: 변종 1 및 2

```
 WITH foo AS (
       SELECT ST_AddBand(ST_MakeEmptyRaster(10, 10, 0, 0, 1, 1, 0, 0, 0),
'32BF', 1, -1) AS rast
 )
 SELECT
       ST_MapAlgebra(rast,                    1,                    NULL,
'ceil([rast]*[rast.x]/[rast.y]+[rast.val])')
 FROM foo
```

예시: 변종 3 및 4

```
 WITH foo AS (
```

```
        SELECT                    1                    AS                    rid,
ST_AddBand(ST_AddBand(ST_AddBand(ST_MakeEmptyRaster(2, 2, 0, 0, 1, -1, 0, 0,
0), 1, '16BUI', 1, 0), 2, '8BUI', 10, 0), 3, '32BUI', 100, 0) AS rast UNION
ALL
        SELECT                    2                    AS                    rid,
ST_AddBand(ST_AddBand(ST_AddBand(ST_MakeEmptyRaster(2, 2, 0, 1, 1, -1, 0, 0,
0), 1, '16BUI', 2, 0), 2, '8BUI', 20, 0), 3, '32BUI', 300, 0) AS rast
  )
  SELECT
        ST_MapAlgebra(
              t1.rast, 2,
              t2.rast, 1,
              '([rast2] + [rast1.val]) / 2'
        ) AS rast
  FROM foo t1
  CROSS JOIN foo t2
  WHERE t1.rid = 1
        AND t2.rid = 2
```

참고: rastbandarg, ST_Union, ST_MapAlgebra

9.11.1.6. ST_MapAlgebraExpr

ST_MapAlgebraExpr — 래스터 밴드 1 개 버전: 입력 래스터에 대해 유효한 PostgreSQL 대수 연산을 적용해서 형성되고, 설정한 픽셀 유형을 가진, 밴드 1 개를 가진 새 래스터를 생성합니다. 따로 밴드를 설정하지 않을 경우, 밴드 1 로 가정합니다.

Synopsis

raster **ST_MapAlgebraExpr**(raster *rast*, integer *band*, text *pixeltype*, text *expression*, double precision *nodataval=NULL*);

raster **ST_MapAlgebraExpr**(raster *rast*, text *pixeltype*, text *expression*, double precision *nodataval=NULL*);

설명:



ST_MapAlgebraExpr 2.1.0 버전부터 더 이상 이 함수를 지원하지 않습니다. 대신 ST_MapAlgebra 함수를 이용하십시오.

입력 래스터(rast)에 대해 expression 이 정의하는 유효한 PostgreSQL 대수 연산을 적용해서 형성된, 밴드 1 개를 가진 새 래스터를 생성합니다. nband 를 설정하지 않을 경우, 밴드 1 로 가정합니다. 이 새 래스터는 원본 래스터와 동일한 지리참조, 너비 및 높이이지만, 밴드는 1 개만 가질 것입니다.

pixeltype 을 설정할 경우, 새 래스터의 밴드가 해당 픽셀 유형이 될 것입니다. pixeltype 이 NULL 일 경우, 새 래스터의 밴드는 입력 rast 의 밴드와 동일한 픽셀 유형이 될 것입니다.

표현식에서 원본 밴드의 픽셀 값을 참조하는 데 [rast], 1-기반 픽셀 열 인덱스를 참조하는 데 [rast.x], 1-기반 픽셀 행 인덱스를 참조하는 데 [rast.y] 용어를 사용할 수 있습니다.

2.0.0 버전부터 사용할 수 있습니다.

예시

원본 래스터 2 개를 입력받는 모듈로(modulo) 함수인 원본으로부터 밴드 1 개를 가진 새 래스터를 생성합니다.

```
  ALTER TABLE dummy_rast ADD COLUMN map_rast raster;
  UPDATE           dummy_rast           SET           map_rast           =
ST_MapAlgebraExpr(rast,NULL,'mod([rast],2)') WHERE rid = 2;

  SELECT
        ST_Value(rast,1,i,j) As origval,
        ST_Value(map_rast, 1, i, j) As mapval
  FROM dummy_rast
  CROSS JOIN generate_series(1, 3) AS i
  CROSS JOIN generate_series(1,3) AS j
  WHERE rid = 2;

   origval | mapval
  ---------+--------
      253 |      1
      254 |      0
      253 |      1
      253 |      1
      254 |      0
      254 |      0
      250 |      0
      254 |      0
      254 |      0
```

분류를 거치고 NODATA 값을 0 으로 설정한 원본으로부터 픽셀 유형이 2BUI 인, 밴드 1 개를 가진 새 래스터를 생성합니다.

```
  ALTER TABLE dummy_rast ADD COLUMN map_rast2 raster;
  UPDATE dummy_rast SET
        map_rast2 = ST_MapAlgebraExpr(rast,'2BUI','CASE WHEN [rast] BETWEEN
100 and 250 THEN 1 WHEN [rast] = 252 THEN 2 WHEN [rast] BETWEEN 253 and 254
THEN 3 ELSE 0 END', '0')
  WHERE rid = 2;

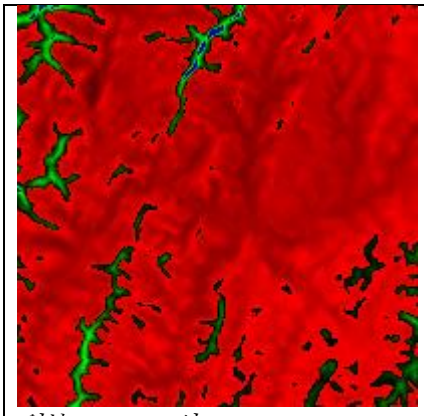
  SELECT DISTINCT
        ST_Value(rast,1,i,j) As origval,
        ST_Value(map_rast2, 1, i, j) As mapval
  FROM dummy_rast
  CROSS JOIN generate_series(1, 5) AS i
  CROSS JOIN generate_series(1,5) AS j
  WHERE rid = 2;

   origval | mapval
  ---------+--------
      249 |      1
      250 |      1
      251 |
      252 |      2
      253 |      3
      254 |      3

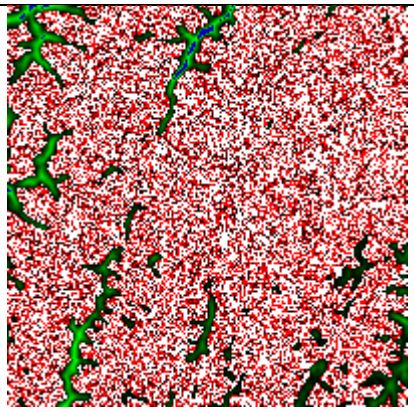
  SELECT
        ST_BandPixelType(map_rast2) As b1pixtyp
  FROM dummy_rast
  WHERE rid = 2;

   b1pixtyp
```

```
----------
 2BUI
```



원본(rast-view 열)



rast-view_ma

밴드 3 개를 가진 원본 래스터에서 맵 대수로 첫 번째 밴드를 조정하고 나머지 두 밴드는 그대로 둔 상태로, 동일한 픽셀 유형인 밴드 3 개를 가진 새 래스터를 생성합니다.

```
SELECT
       ST_AddBand(
              ST_AddBand(
                     ST_AddBand(
                            ST_MakeEmptyRaster(rast_view),
                            ST_MapAlgebraExpr(rast_view,1,NULL,'tan([rast])*[rast]')
                     ),
                     ST_Band(rast_view,2)
              ),
              ST_Band(rast_view, 3) As rast_view_ma
       )
FROM wind
WHERE rid=167;
```

참고: ST_MapAlgebraExpr, ST_MapAlgebraFct, ST_BandPixelType, ST_GeoReference, ST_Value

9.11.1.7. ST_MapAlgebraExpr

ST_MapAlgebraExpr — 래스터 밴드 2 개 버전: 입력 래스터 2 개에 대해 유효한 PostgreSQL 대수 연산을 적용해서 형성되고, 설정한 픽셀 유형을 가진, 밴드 1 개를 가진 새 래스터를 생성합니다. 따로 밴드를 설정하지 않을 경우, 각 래스터의 밴드 1 로 가정합니다. 출력 래스터는 첫 번째 래스터가 정의하는 그리드 상에 (축척, 기울기 및 픽셀 모서리가) 정렬될 것입니다. extenttype 파라미터가 출력 래스터의 범위를 정의할 것입니다. extenttype 의 값은 INTERSECTION, UNION, FIRST, SECOND 가 될 수 있습니다.

Synopsis

raster **ST_MapAlgebraFct**(raster *rast*, regprocedure *onerasteruserfunc*);

raster **ST_MapAlgebraFct**(raster *rast*, regprocedure *onerasteruserfunc*, text[] *VARIADIC args*);

raster **ST_MapAlgebraFct**(raster *rast*, text *pixeltype*, regprocedure *onerasteruserfunc*);

raster **ST_MapAlgebraFct**(raster *rast*, text *pixeltype*, regprocedure *onerasteruserfunc*, text[] *VARIADIC args*);

raster **ST_MapAlgebraFct**(raster *rast*, integer *band*, regprocedure *onerasteruserfunc*);

raster **ST_MapAlgebraFct**(raster *rast*, integer *band*, regprocedure *onerasteruserfunc*, text[] *VARIADIC args*);

raster **ST_MapAlgebraFct**(raster *rast*, integer *band*, text *pixeltype*, regprocedure *onerasteruserfunc*);

raster **ST_MapAlgebraFct**(raster *rast*, integer *band*, text *pixeltype*, regprocedure *onerasteruserfunc*, text[] *VARIADIC args*);

설명:



ST_MapAlgebraExpr 2.1.0 버전부터 더 이상 이 함수를 지원하지 않습니다. 대신 ST_MapAlgebra 함수를 이용하십시오.

입력 래스터 밴드 rast1, (rast2)에 대한 expression 이 정의하는 밴드 2 개에 대해, 유효한 PostgreSQL 대수 연산을 적용해서 형성된, 밴드 1 개를 가진 새 래스터를 생성합니다. band1, band2 를 설정하지 않을 경우, 밴드 1 로 가정합니다. 출력 래스터는 첫 번째 래스터가 정의하는 그리드 상에 (축척, 기울기 및 픽셀 모서리가) 정렬될 것입니다. extenttype 파라미터가 출력 래스터의 범위를 정의할 것입니다.

표현식(expression) — 래스터 2 개가 관련된 PostgreSQL 대수 표현식 및 픽셀들이 교차할 경우 픽셀 값을 정의할 PostgreSQL 정의 함수/연산자입니다. 예: (([rast1] + [rast2])/2.0)::integer

pixeltype — 출력 래스터의 픽셀 유형입니다. 이 유형은 ST_BandPixelType 목록에 존재하는 유형 가운데 하나이거나, 생략되거나, NULL 로 설정돼야만 합니다. 따로 설정하지 않거나 NULL 로 설정하지 않으면, 첫 번째 래스터의 픽셀 유형을 기본값으로 삼을 것입니다.

extenttype — 출력 래스터의 범위 제어

1. INTERSECTION - 새 래스터의 범위는 두 래스터의 교차 부분입니다. 기본값입니다.
2. UNION - 새 래스터의 범위는 두 래스터를 통합한 범위입니다.
3. FIRST - 새 래스터의 범위는 첫 번째 래스터의 범위와 동일합니다.
4. SECOND - 새 래스터의 범위는 두 번째 래스터의 범위와 동일합니다.

nodata1expr — rast1 의 픽셀들이 NODATA 값이고 공간적으로 상응하는 rast2 의 픽셀들이 값을 가지고 있을 때 어떤 것을 반환할지 정의하는 상수만, 또는 rast2 와만 관련된 대수 표현식입니다.

nodata2expr — rast2 의 픽셀들이 NODATA 값이고 공간적으로 상응하는 rast1 의 픽셀들이 값을 가지고 있을 때 어떤 것을 반환할지 정의하는 상수만, 또는 rast1 과만 관련된 대수 표현식입니다.

nodatanodataval — 공간적으로 상응하는 rast1 및 rast2 의 픽셀들이 모두 NODATA 값일 경우 반환하는 숫자 상수입니다.

pixeltype 을 설정할 경우, 새 래스터의 밴드가 해당 픽셀 유형이 될 것입니다. pixeltype 이 NULL 이거나 따로 설정하지 않을 경우, 새 래스터의 밴드는 입력 rast1 의 밴드와 동일한 픽셀 유형이 될 것입니다.

원본 밴드의 픽셀 값을 참조하는 데 [rast1.val], [rast2.val], 픽셀의 열/행 위치를 참조하는 데 [rast1.x], [rast1.y] 등의 용어를 사용하십시오.

2.0.0 버전부터 사용할 수 있습니다.

예시: 밴드 2 개의 교차 및 통합

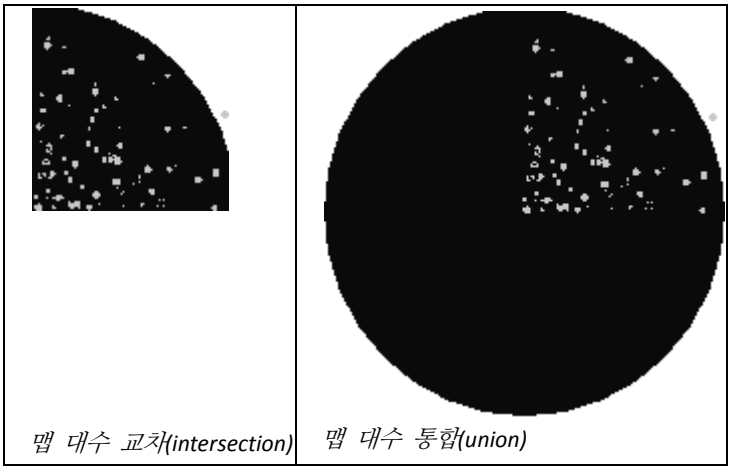
원본 래스터 2 개를 입력받는 모듈로(modulo) 함수인 원본으로부터 밴드 1 개를 가진 새 래스터를 생성합니다.

```
  -- 래스터 집합 생성 --
  DROP TABLE IF EXISTS fun_shapes;
  CREATE TABLE fun_shapes(rid serial PRIMARY KEY, fun_name text, rast raster);

  -- 매사추세츠 주 미터 단위 평면의 보스턴 주위에 몇 개의 형상을 삽입 --
  INSERT INTO fun_shapes(fun_name, rast)
  VALUES   ('ref',   ST_AsRaster(ST_MakeEnvelope(235229,   899970,   237229,
901930,26986),200,200,'8BUI',0,0));

  INSERT INTO fun_shapes(fun_name,rast)
  WITH ref(rast) AS (SELECT rast FROM fun_shapes WHERE fun_name = 'ref' )
  SELECT                  'area'                  AS                  fun_name,
ST_AsRaster(ST_Buffer(ST_SetSRID(ST_Point(236229, 900930),26986), 1000),
                    ref.rast,'8BUI', 10, 0) As rast
  FROM ref
  UNION ALL
  SELECT 'rand bubbles',
                    ST_AsRaster(
                    (SELECT ST_Collect(geom)
        FROM  (SELECT  ST_Buffer(ST_SetSRID(ST_Point(236229  +  i*random()*100,
900930 + j*random()*100),26986), random()*20) As geom
                    FROM  generate_series(1,10)  As  i,  generate_series(1,10)
As j
                    ) As foo ), ref.rast,'8BUI', 200, 0)
  FROM ref;

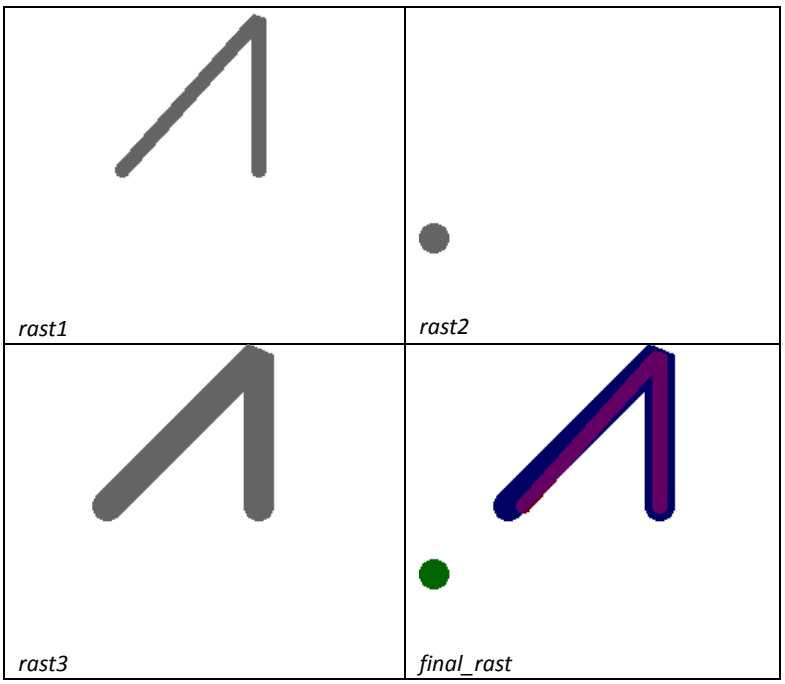
  -- 매핑 --
  SELECT  ST_MapAlgebraExpr(
              area.rast,  bub.rast,  '[rast2.val]',  '8BUI',  'INTERSECTION',
'[rast2.val]', '[rast1.val]') As interrast,
              ST_MapAlgebraExpr(
                    area.rast,  bub.rast,  '[rast2.val]',  '8BUI',  'UNION',
'[rast2.val]', '[rast1.val]') As unionrast
  FROM
    (SELECT rast FROM fun_shapes WHERE
   fun_name = 'area') As area
  CROSS JOIN  (SELECT rast
  FROM fun_shapes WHERE
   fun_name = 'rand bubbles') As bub
```



맵 대수 교차(intersection) *맵 대수 통합(union)*

예시: 캔버스 상에 래스터들을 개별 밴드로서 오버레이

```
-- 모든 단일 밴드 래스터를 회색조로 만들기 위해 ST_AsPNG 를 통해 이미지를 렌더링합니다. --
WITH mygeoms
  AS ( SELECT 2 As bnum, ST_Buffer(ST_Point(1,5),10) As geom
       UNION ALL
       SELECT 3 AS bnum,
         ST_Buffer(ST_GeomFromText('LINESTRING(50 50,150 150,150 50)'), 10,'join=bevel') As
geom
       UNION ALL
       SELECT 1 As bnum,
         ST_Buffer(ST_GeomFromText('LINESTRING(60 50,150 150,150 50)'), 5,'join=bevel') As
geom
       ),
  -- 캔버스가 도형에 1:1 픽셀이 되도록 정의
  canvas
   AS (SELECT ST_AddBand(ST_MakeEmptyRaster(200,
     200,
     ST_XMin(e)::integer, ST_YMax(e)::integer, 1, -1, 0, 0) , '8BUI'::text,0) As rast
     FROM (SELECT ST_Extent(geom) As e,
           Max(ST_SRID(geom)) As srid
           from mygeoms
           ) As foo
       ),
  rbands AS (SELECT ARRAY(SELECT ST_MapAlgebraExpr(canvas.rast, ST_AsRaster(m.geom,
canvas.rast, '8BUI', 100),
          '[rast2.val]', '8BUI', 'FIRST', '[rast2.val]', '[rast1.val]') As rast
         FROM mygeoms AS m CROSS JOIN canvas
         ORDER BY m.bnum) As rasts
         )
      SELECT rasts[1] As rast1 , rasts[2] As rast2, rasts[3] As rast3, ST_AddBand(
           ST_AddBand(rasts[1],rasts[2]), rasts[3]) As final_rast
       FROM rbands;
```



예시: 항공사진 상에 선택된 획지에 두께 2 미터의 경계선을 오버레이

```
  -- 처음 2 개의 밴드를 잘라내고 세 번째 밴드를 도형과 겹친, 밴드 3 개를 가진 새 래스터를
생성합니다.
  -- PostGIS 윈도우 64 비트 설치본에서 이 쿼리가 3.6 초 걸렸습니다.
  WITH pr AS
  -- 연산 순서에 주의하십시오: 모든 래스터를 해당 지역의 차원에 맞게 잘라냅니다.
  (SELECT ST_Clip(rast,ST_Expand(geom,50) ) As rast, g.geom
        FROM aerials.o_2_boston AS r INNER JOIN
  -- 관심 획지를 통합해서 나중에 교차시킬 수 있는 단일 도형으로 만듭니다.
               (SELECT ST_Union(ST_Transform(the_geom,26986)) AS geom
                FROM landparcels WHERE pid IN('0303890000', '0303900000')) As
g
               ON ST_Intersects(rast::geometry, ST_Expand(g.geom,50))
  ),

  -- 그리고 래스터 조각들을 통합합니다.
  -- 래스터를 ST_Union 으로 통합하는 작업은 조금 느린 편이지만, 래스터를 잘게 쪼갤수록 속도는
빨라집니다.
  -- 따라서 먼저 잘라낸 다음에 통합하는 편이 좋습니다.
  prunion AS
  (SELECT                                                    ST_AddBand(NULL,
ARRAY[ST_Union(rast,1),ST_Union(rast,2),ST_Union(rast,3)] ) As clipped,geom
  FROM pr
  GROUP BY geom)
  -- 래스터 조각들을 통합하고 획지 경계선과 겹친
  -- 결과 래스터를 반환합니다.
  -- 처음 2 개의 밴드를 추가한 다음, 세 번째 밴드와 도형에 맵 대수를 적용합니다.
  SELECT ST_AddBand(ST_Band(clipped,ARRAY[1,2])
         ,                                ST_MapAlgebraExpr(ST_Band(clipped,3),
ST_AsRaster(ST_Buffer(ST_Boundary(geom),2),clipped, '8BUI',250),
          '[rast2.val]', '8BUI', 'FIRST', '[rast2.val]', '[rast1.val]') ) As
rast
  FROM prunion;
```



파란색 라인이 선택된 획지의 경계선입니다.

참고: ST_MapAlgebraExpr, ST_AddBand, ST_AsPNG, ST_AsRaster, ST_MapAlgebraFct, ST_BandPixelType, ST_GeoReference, ST_Value, ST_Union, ST_Union

9.11.1.8. ST_MapAlgebraFct

ST_MapAlgebraFct — 래스터 밴드 1 개 버전: 입력 래스터에 대해 유효한 PostgreSQL 대수 연산을 적용해서 형성되고, 설정한 픽셀 유형을 가진, 밴드 1 개를 가진 새 래스터를 생성합니다. 따로 밴드를 설정하지 않을 경우, 밴드 1 로 가정합니다.

Synopsis

raster **ST_MapAlgebraFct**(raster *rast*, regprocedure *onerasteruserfunc*);

raster **ST_MapAlgebraFct**(raster *rast*, regprocedure *onerasteruserfunc*, text[] *VARIADIC args*);

raster **ST_MapAlgebraFct**(raster *rast*, text *pixeltype*, regprocedure *onerasteruserfunc*);

raster **ST_MapAlgebraFct**(raster *rast*, text *pixeltype*, regprocedure *onerasteruserfunc*, text[] *VARIADIC args*);

raster **ST_MapAlgebraFct**(raster *rast*, integer *band*, regprocedure *onerasteruserfunc*);

raster **ST_MapAlgebraFct**(raster *rast*, integer *band*, regprocedure *onerasteruserfunc*, text[] *VARIADIC args*);

raster **ST_MapAlgebraFct**(raster *rast*, integer *band*, text *pixeltype*, regprocedure *onerasteruserfunc*);

raster **ST_MapAlgebraFct**(raster *rast*, integer *band*, text *pixeltype*, regprocedure *onerasteruserfunc*, text[] *VARIADIC args*);

설명:



ST_MapAlgebraFct 2.1.0 버전부터 더 이상 이 함수를 지원하지 않습니다. 대신 ST_MapAlgebra 함수를 이용하십시오.

입력 래스터(rast)에 대해 onerasteruserfunc 가 정의하는 유효한 PostgreSQL 함수를 적용해서 형성된, 밴드 1 개를 가진 새 래스터를 생성합니다. band 를 설정하지 않을 경우, 밴드 1 로 가정합니다. 이 새 래스터는 원본 래스터와 동일한 지리참조, 너비 및 높이이지만, 밴드는 1 개만 가질 것입니다.

pixeltype 을 설정할 경우, 새 래스터의 밴드가 해당 픽셀 유형이 될 것입니다. pixeltype 이 NULL 일 경우, 새 래스터의 밴드는 입력 rast 의 밴드와 동일한 픽셀 유형이 될 것입니다.

onerasteruserfunc 파라미터는 regprocedure(회귀 처리)로 형변환된 SQL 또는 PL/pgSQL 함수의 명칭 및 서명(signature)이어야 합니다. 다음은 매우 간단하고 쓸모없는 PL/pgSQL 함수의 예입니다:

```
  CREATE OR REPLACE FUNCTION simple_function(pixel FLOAT, pos INTEGER[],
VARIADIC args TEXT[])
  RETURNS FLOAT
  AS $$ BEGIN
  RETURN 0.0;
  END; $$
  LANGUAGE 'plpgsql' IMMUTABLE;
```

userfunction 은 인수를 2 개 또는 3 개 받을 수 있습니다: 부동소수점 값, 선택적인 정수 배열, 그리고 임의로 여러 인수를 입력받는 문자열 배열입니다. 첫 번째 인수는 (래스터의 데이터형과 상관없이) 개별적인 래스터 셀의 값입니다. 두 번째 인수는 현재 처리중인 셀의 위치를 '{x,y}' 서식으로 표현한

것입니다. 세 번째 인수는 ST_MapAlgebraFct 의 모든 나머지 파라미터들을 userfunction 을 통해 설정해야 한다는 것을 의미합니다.

regprocedure 인수를 SQL 함수에 입력하려면 입력하기 위한 전체 함수 서명이 필요하고, 그 다음 regprocedure 유형으로 형변환해야 합니다. 앞의 PL/pgSQL 함수를 인수로서 넘겨주려면, 해당 인수를 위한 SQL 은 다음과 같습니다:

```
  'simple_function(float,integer[ ],text[ ])'::regprocedure
```

해당 인수가 함수의 명칭, 함수 인수들의 유형, 명칭 및 인수 유형을 감싼 따옴표, 그리고 regprocedure 로의 형변환을 포함하고 있다는 점에 주의하십시오.

userfunction 에 들어가는 세 번째 인수는 variadic text 배열입니다. 어떤 ST_MapAlgebraFct 함수 호출에도 입력되는 길고 긴 텍스트 인수들이 모두 지정된 userfunction 에 넘겨지며, args 인수에 담겨집니다.



(다양한 개수의 인수를 입력받는) VARIADIC 키워드에 대한 더 자세한 정보를 알고 싶다면, PostgreSQL 문서 가운데 Query Language (SQL) Functions 의 "SQL Functions with Variable Numbers of Arguments" 단원을 참조하십시오.



공간 처리를 위해 사용자 함수에 어떤 인수를 넘겨주기로 하고 말고에 상관없이, userfunction 에 들어가는 text[] 인수는 필요합니다.

2.0.0 버전부터 사용할 수 있습니다.

예시

원본 래스터 2 개를 입력받는 모듈로(modulo) 함수인 원본으로부터 밴드 1 개를 가진 새 래스터를 생성합니다.

```
  ALTER TABLE dummy_rast ADD COLUMN map_rast raster;
  CREATE FUNCTION mod_fct(pixel float, pos integer[], variadic args text[])
  RETURNS float
  AS $$
  BEGIN
     RETURN pixel::integer % 2;
  END;
  $$
  LANGUAGE 'plpgsql' IMMUTABLE;

  UPDATE dummy_rast SET map_rast =
ST_MapAlgebraFct(rast,NULL,'mod_fct(float,integer[],text[])'::regprocedure)
WHERE rid = 2;

  SELECT ST_Value(rast,1,i,j) As origval, ST_Value(map_rast, 1, i, j) As
mapval
  FROM dummy_rast CROSS JOIN generate_series(1, 3) AS i CROSS JOIN
generate_series(1,3) AS j
  WHERE rid = 2;

   origval | mapval
  ---------+--------
      253 |      1
      254 |      0
```

```
     253 |     1
     253 |     1
     254 |     0
     254 |     0
     250 |     0
     254 |     0
     254 |     0
```

재분류를 거치고 NODATA 값을 사용자 함수(0)으로 넘겨진 파라미터로 설정한 원본으로부터 픽셀 유형이 2BUI 인, 밴드 1 개를 가진 새 래스터를 생성합니다.

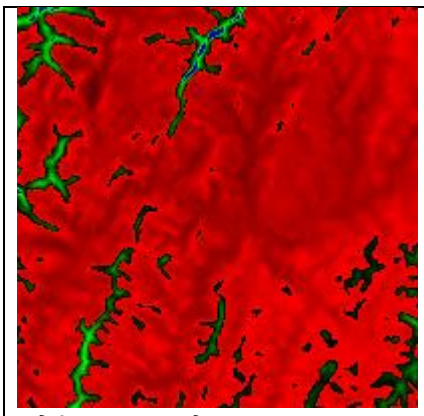
```
  ALTER TABLE dummy_rast ADD COLUMN map_rast2 raster;
  CREATE FUNCTION classify_fct(pixel float, pos integer[], variadic args
text[])
  RETURNS float
  AS
  $$
  DECLARE
     nodata float := 0;
  BEGIN
     IF NOT args[1] IS NULL THEN
        nodata := args[1];
     END IF;
     IF pixel < 251 THEN
        RETURN 1;
     ELSIF pixel = 252 THEN
        RETURN 2;
     ELSIF pixel > 252 THEN
        RETURN 3;
     ELSE
        RETURN nodata;
     END IF;
  END;
  $$
  LANGUAGE 'plpgsql';
  UPDATE dummy_rast SET map_rast2 =
ST_MapAlgebraFct(rast,'2BUI','classify_fct(float,integer[],text[])'::regproc
edure, '0') WHERE rid = 2;

  SELECT DISTINCT ST_Value(rast,1,i,j) As origval, ST_Value(map_rast2, 1, i,
j) As mapval
  FROM dummy_rast CROSS JOIN generate_series(1, 5) AS i CROSS JOIN
generate_series(1,5) AS j
  WHERE rid = 2;

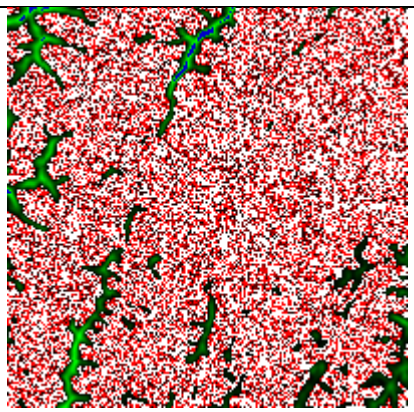
   origval | mapval
  ---------+--------
     249 |     1
     250 |     1
     251 |
     252 |     2
     253 |     3
     254 |     3

  SELECT ST_BandPixelType(map_rast2) As b1pixtyp
  FROM dummy_rast WHERE rid = 2;

   b1pixtyp
  ----------
   2BUI    254 |     0
```



원본(rast-view 열)



rast-view_ma

밴드 3 개를 가진 원본 래스터에서 맵 대수로 첫 번째 밴드를 조정하고 나머지 두 밴드는 그대로 둔 상태로, 동일한 픽셀 유형인 밴드 3 개를 가진 새 래스터를 생성합니다.

```
  CREATE FUNCTION rast_plus_tan(pixel float, pos integer[], variadic args
text[])
  RETURNS float
  AS
  $$
  BEGIN
        RETURN tan(pixel) * pixel;
  END;
  $$
  LANGUAGE 'plpgsql';

  SELECT ST_AddBand(
        ST_AddBand(
              ST_AddBand(
                    ST_MakeEmptyRaster(rast_view),

ST_MapAlgebraFct(rast_view,1,NULL,'rast_plus_tan(float,integer[],text[])'::r
egprocedure)
              ),
              ST_Band(rast_view,2)
        ),
        ST_Band(rast_view, 3) As rast_view_ma
  )
  FROM wind
  WHERE rid=167;
```

참고: ST_MapAlgebraExpr, ST_BandPixelType, ST_GeoReference, ST_SetValue

9.11.1.9. ST_MapAlgebraFct

ST_MapAlgebraFct — 래스터 밴드 2 개 버전: 입력 래스터 2 개에 대해 유효한 PostgreSQL 함수를 적용해서 형성되고, 설정한 픽셀 유형을 가진, 밴드 1 개를 가진 새 래스터를 생성합니다. 따로 밴드를 설정하지 않을 경우, 밴드 1 로 가정합니다. 범위 유형을 따로 설정하지 않을 경우 기본값은 INTERSECTION 입니다.

Synopsis

raster ST_MapAlgebraFct(raster *rast1*, raster *rast2*, regprocedure *tworastuserfunc*, text *pixeltype=same_as_rast1*, text *extenttype=INTERSECTION*, text[] *VARIADIC userargs*);

raster ST_MapAlgebraFct(raster *rast1*, integer *band1*, raster *rast2*, integer *band2*, regprocedure *tworastuserfunc*, text *pixeltype=same_as_rast1*, text *extenttype=INTERSECTION*, text[] *VARIADIC userargs*);

설명:



ST_MapAlgebraFct 2.1.0 버전부터 더 이상 이 함수를 지원하지 않습니다. 대신 ST_MapAlgebra 함수를 이용하십시오.

입력 래스터 rast1, rast2 에 대해 tworastuserfunc 가 정의하는 PostgreSQL 함수를 적용해서 형성된, 밴드 1 개를 가진 새 래스터를 생성합니다. band1 또는 band2 를 설정하지 않을 경우, 밴드 1 로 가정합니다. 이 새 래스터는 원본 래스터와 동일한 지리참조, 너비 및 높이이지만, 밴드는 1 개만 가질 것입니다.

pixeltype 을 설정할 경우, 새 래스터의 밴드가 해당 픽셀 유형이 될 것입니다. pixeltype 이 NULL 이거나 따로 설정하지 않을 경우, 새 래스터의 밴드는 입력 rast1 의 밴드와 동일한 픽셀 유형이 될 것입니다.

tworastuserfunc 파라미터는 regprocedure(회귀 처리)로 형변환된 SQL 또는 PL/pgSQL 함수의 명칭 및 서명(signature)이어야 합니다. 다음은 PL/pgSQL 함수의 예입니다:

```
  CREATE OR REPLACE FUNCTION simple_function_for_two_rasters(pixel1 FLOAT,
pixel2 FLOAT, pos INTEGER[], VARIADIC args TEXT[])
  RETURNS FLOAT
  AS $$ BEGIN
  RETURN 0.0;
  END; $$
  LANGUAGE 'plpgsql' IMMUTABLE;
```

tworastuserfunc 은 인수를 3 개 또는 4 개 받을 수 있습니다: 이중 정밀도 값, 이중 정밀도 값, 선택적인 정수형 배열, 그리고 임의로 여러 인수를 입력받는 문자열 배열입니다. 첫 번째 인수는 (래스터의 데이터형과 상관없이) rast1 에 있는 개별적인 래스터 셀의 값입니다. 두 번째 인수는 rast2 에 있는 개별적인 래스터 셀의 값입니다. 세 번째 인수는 현재 처리중인 셀의 위치를 '{x,y}' 서식으로 표현한 것입니다. 네 번째 인수는 ST_MapAlgebraFct 의 모든 나머지 파라미터들을 tworastuserfunc 을 통해 설정해야 한다는 것을 의미합니다.

regprocedure 인수를 SQL 함수에 입력하려면 입력하기 위한 전체 함수 서명이 필요하고, 그 다음 regprocedure 유형으로 형변환해야 합니다. 앞의 PL/pgSQL 함수를 인수로서 넘겨주려면, 해당 인수를 위한 SQL 은 다음과 같습니다:

```
  'simple_function(double precision, double precision, integer[], text[])'::regprocedure
```

해당 인수가 함수의 명칭, 함수 인수들의 유형, 명칭 및 인수 유형을 감싼 따옴표, 그리고 regprocedure 로의 형변환을 포함하고 있다는 점에 주의하십시오.

tworastuserfunc 에 들어가는 네 번째 인수는 variadic text 배열입니다. 어떤 ST_MapAlgebraFct 함수 호출에도 입력되는 길고 긴 텍스트 인수들이 모두 지정된 tworastuserfunc 에 넘겨지며, userargs 인수에 담깁니다.



(다양한 개수의 인수를 입력받는) VARIADIC 키워드에 대한 더 자세한 정보를 알고 싶다면, PostgreSQL 문서 가운데 Query Language (SQL) Functions 의 "SQL Functions with Variable Numbers of Arguments" 단원을 참조하십시오.



공간 처리를 위해 사용자 함수에 어떤 인수를 넘겨주기로 하고 말고에 상관없이, tworastuserfunc 에 들어가는 text[] 인수는 필요합니다.

2.0.0 버전부터 사용할 수 있습니다.

예시: 캔버스 상에 래스터들을 개별 밴드로서 오버레이

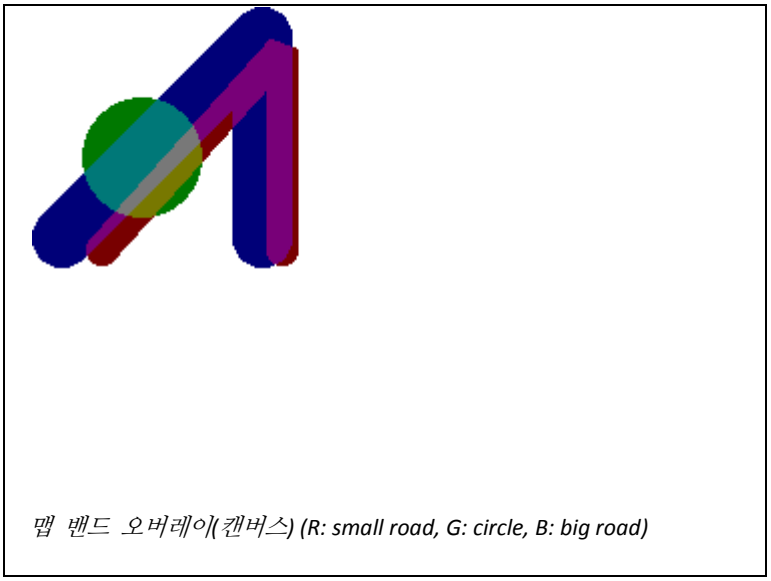
```
-- 사용자 지정 함수를 정의 --
CREATE OR REPLACE FUNCTION raster_mapalgebra_union(
    rast1 double precision,
    rast2 double precision,
  pos integer[],
    VARIADIC userargs text[]
)
    RETURNS double precision
    AS $$
    DECLARE
    BEGIN
        CASE
            WHEN rast1 IS NOT NULL AND rast2 IS NOT NULL THEN
                RETURN ((rast1 + rast2)/2.);
            WHEN rast1 IS NULL AND rast2 IS NULL THEN
                RETURN NULL;
            WHEN rast1 IS NULL THEN
                RETURN rast2;
            ELSE
                RETURN rast1;
        END CASE;

        RETURN NULL;
    END;
    $$ LANGUAGE 'plpgsql' IMMUTABLE COST 1000;

-- 래스터 테스트 테이블 준비
DROP TABLE IF EXISTS map_shapes;
CREATE TABLE map_shapes(rid serial PRIMARY KEY, rast raster, bnum integer, descrip text);
INSERT INTO map_shapes(rast,bnum, descrip)
WITH mygeoms
  AS ( SELECT 2 As bnum, ST_Buffer(ST_Point(90,90),30) As geom, 'circle' As descrip
       UNION ALL
       SELECT 3 AS bnum,
         ST_Buffer(ST_GeomFromText('LINESTRING(50 50,150 150,150 50)'), 15) As geom, 'big
road' As descrip
       UNION ALL
       SELECT 1 As bnum,
         ST_Translate(ST_Buffer(ST_GeomFromText('LINESTRING(60   50,150   150,150   50)'),
8,'join=bevel'), 10,-6) As geom, 'small road' As descrip
       ),
  -- 캔버스가 도형에 1:1 픽셀이 되도록 정의
  canvas
   AS ( SELECT ST_AddBand(ST_MakeEmptyRaster(250,
```

```
      250,
      ST_XMin(e)::integer, ST_YMax(e)::integer, 1, -1, 0, 0 ) , '8BUI'::text,0) As rast
      FROM (SELECT ST_Extent(geom) As e,
            Max(ST_SRID(geom)) As srid
            from mygeoms
            ) As foo
        )
  -- 캔버스에 정렬된 래스터를 반환
  SELECT ST_AsRaster(m.geom, canvas.rast, '8BUI', 240) As rast, bnum, descrip
          FROM mygeoms AS m CROSS JOIN canvas
  UNION ALL
  SELECT canvas.rast, 4, 'canvas'
  FROM canvas;

  -- 단일 밴드 래스터에 대해 맵 대수를 연산한 다음 ST_AddBand 로 모으기
  INSERT INTO map_shapes(rast,bnum,descrip)
  SELECT ST_AddBand(ST_AddBand(rasts[1], rasts[2]),rasts[3]), 4, 'map bands overlay fct union
(canvas)'
      FROM (SELECT ARRAY(SELECT ST_MapAlgebraFct(m1.rast, m2.rast,
              'raster_mapalgebra_union(double  precision,  double  precision,  integer[],
text[])'::regprocedure, '8BUI', 'FIRST')
          FROM map_shapes As m1 CROSS JOIN map_shapes As m2
      WHERE m1.descrip = 'canvas' AND m2.descrip <> 'canvas' ORDER BY m2.bnum) As rasts) As
foo;
```



맵 밴드 오버레이(캔버스) (R: small road, G: circle, B: big road)

추가 인수를 입력받는 사용자 지정 함수

```
  CREATE OR REPLACE FUNCTION raster_mapalgebra_userargs(
      rast1 double precision,
      rast2 double precision,
    pos integer[ ],
      VARIADIC userargs text[ ]
  )
      RETURNS double precision
      AS $$
      DECLARE
      BEGIN
          CASE
              WHEN rast1 IS NOT NULL AND rast2 IS NOT NULL THEN
                  RETURN least(userargs[1]::integer,(rast1 + rast2)/2.);
```

```
                WHEN rast1 IS NULL AND rast2 IS NULL THEN
                    RETURN userargs[2]::integer;
                WHEN rast1 IS NULL THEN
                    RETURN greatest(rast2,random()*userargs[3]::integer)::integer;
                ELSE
                    RETURN greatest(rast1, random()*userargs[4]::integer)::integer;
            END CASE;

            RETURN NULL;
        END;
        $$ LANGUAGE 'plpgsql' VOLATILE COST 1000;

    SELECT ST_MapAlgebraFct(m1.rast, 1, m1.rast, 3,
                'raster_mapalgebra_userargs(double precision, double precision, integer[],
text[])'::regprocedure,
                    '8BUI', 'INTERSECT', '100','200','200','0')
            FROM map_shapes As m1
        WHERE m1.descrip = 'map bands overlay fct union (canvas)';
```



추가 인수 및 동일한 래스터에서 가져온 다른 밴드를 받는 사용자 지정 함수

참고: ST_MapAlgebraExpr, ST_BandPixelType, ST_GeoReference, ST_SetValue

9.11.1.10. ST_MapAlgebraFctNgb

ST_MapAlgebraFctNgb — 래스터 밴드 1 개 버전: 사용자 지정 PostgreSQL 함수를 이용하는 맵 대수 최근접 이웃(Map Algebra Nearest Neighbor)입니다. 입력 래스터 밴드의 값의 이웃(neighborhood)이 관련된 PostgreSQL 사용자 함수가 출력하는 값을 가진 래스터를 반환합니다.

Synopsis

raster ST_MapAlgebraFctNgb(raster *rast*, integer *band*, text *pixeltype*, integer *ngbwidth*, integer *ngbheight*, regprocedure *onerastngbuserfunc*, text *nodatamode*, text[] *VARIADIC args*);

설명:



ST_MapAlgebraFctNgb 2.1.0 버전부터 더 이상 이 함수를 지원하지 않습니다. 대신 ST_MapAlgebra 함수를 이용하십시오.

래스터 1 개 버전: 입력 래스터 밴드 값의 이웃(neighborhood)이 관련된 PostgreSQL 사용자 함수가 출력하는 값을 가진 래스터를 반환합니다. 사용자 함수가 픽셀의 이웃 값을 숫자의 배열로 입력받아, 각 픽셀에 대해 사용자 함수의 결과값을 반환해서, 현재 처리중인 픽셀의 픽셀 값을 함수 결과값으로 대체합니다.

rast — 사용자 함수를 평가하는 데 쓰이는 래스터

band — 평가할 래스터 밴드 번호(기본값은 1)

pixeltype — 출력 래스터의 픽셀 유형입니다. 이 유형은 ST_BandPixelType 목록에 존재하는 유형 가운데 하나이거나, 생략되거나, NULL 로 설정돼야만 합니다. 따로 설정하지 않거나 NULL 로 설정하지 않으면, rast 의 픽셀 유형을 기본값으로 삼을 것입니다. 결과값이 픽셀 유형의 허용치보다 클 경우 결과값의 길이를 허용치에 맞춰 줄입니다.

ngbwidth — 이웃(neighborhood)의 셀 단위 너비

ngbheight — 이웃(neighborhood)의 셀 단위 높이

onerastngbuserfunc — 래스터의 단일 밴드의 근접 픽셀에 적용할 PL/pgSQL 또는 psql 사용자 함수입니다. 첫 번째 요소가 사각형 픽셀 이웃을 표현하는 2 차원 숫자 배열입니다.

nodatamode — NODATA 또는 NULL 인 이웃 픽셀의 경우 함수에 어떤 값을 넘겨줄지 정의합니다.

'ignore': 이웃에서 맞닥뜨린 어떤 NODATA 값도 계산을 통해 무시합니다. 사용자 콜백 함수에 이 플래그를 전송해야만 하며, 사용자 함수는 NODATA 값을 어떻게 무시할 것인지 결정합니다.

'NULL': 이웃에서 맞닥뜨린 모든 NODATA 값이 출력 픽셀을 NULL 로 만들 것입니다. 이 경우 사용자 콜백 함수를 건너뜁니다.

'value': 이웃에서 맞닥뜨린 어떤 NODATA 값도 참조 픽셀(이웃의 한가운데 있는 픽셀)로 대체합니다. 이 값이 NODATA 일 경우, (영향을 받는 이웃에 대해) 'NULL'과 동일한 습성을 보인다는 점에 주의하십시오.

args — 사용자 함수로 넘겨줄 인수들

2.0.0 버전부터 사용할 수 있습니다.

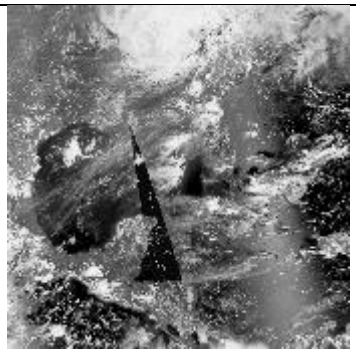
예시

다음 예시는 http://trac.osgeo.org/gdal/wiki/frmts_wtkraster.html 에서 설명하는 대로 단일 타일로 불러온 다음 ST_Rescale 예시 대로 준비한 카트리나 래스터를 활용합니다.

```
  -- 이웃에 있는 모든 값을 평균하는 단순한 '콜백' 사용자 함수입니다. --
  CREATE OR REPLACE FUNCTION rast_avg(matrix float[][], nodatamode text,
variadic args text[])
     RETURNS float AS
     $$
     DECLARE
               _matrix float[][];
        x1 integer;
        x2 integer;
        y1 integer;
        y2 integer;
        sum float;
```

```
    BEGIN
                _matrix := matrix;
        sum := 0;
        FOR x in array_lower(matrix, 1)..array_upper(matrix, 1) LOOP
            FOR y in array_lower(matrix, 2)..array_upper(matrix, 2) LOOP
                sum := sum + _matrix[x][y];
            END LOOP;
        END LOOP;
        RETURN
(sum*1.0/(array_upper(matrix,1)*array_upper(matrix,2) ))::integer ;
    END;
    $$
 LANGUAGE 'plpgsql' IMMUTABLE COST 1000;

 -- 이제 각각 서로의 X 및 Y 방향으로 픽셀 2 개 만큼의 거리 안에 래스터의 평균낸 픽셀들을
적용해봅시다. --
 SELECT ST_MapAlgebraFctNgb(rast, 1,  '8BUI', 4,4,
              'rast_avg(float[][],  text,  text[])'::regprocedure,  'NULL',
NULL) As nn_with_border
        FROM katrinas_rescaled
        limit 1;
```



래스터의 첫 번째 밴드



각각 서로의 4x4 픽셀 거리 안에 있는 픽셀들을 평균낸 후의 새 래스터

참고: ST_MapAlgebraFct, ST_MapAlgebraExpr, ST_Rescale

9.11.1.11. ST_Reclass

ST_Reclass — 원본으로부터 재분류된 밴드 유형으로 이루어진 새 래스터를 생성합니다. nband 는 변경할 밴드를 가리킵니다. nband 를 따로 설정하지 않을 경우 밴드 1 로 가정합니다. 다른 모든 밴드들은 변경 없이 반환됩니다. 실제 사례: 보기 좋은 형식으로 더 간단하게 렌더링하기 위해 16BUI 밴드를 8BUI 등등으로 변환하십시오.

Synopsis

raster ST_Reclass(raster *rast*, integer *nband*, text *reclassexpr*, text *pixeltype*, double precision *nodataval=NULL*);

raster ST_Reclass(raster *rast*, reclassarg[ ] *VARIADIC reclassargset*);

raster ST_Reclass(raster *rast*, text *reclassexpr*, text *pixeltype*);

설명: 입력 래스터(rast)에 대해 reclassexpr 이 정의하는 유효한 PostgreSQL 대수 연산을 적용해서 형성된 새 래스터를 생성합니다. band 를 설정하지 않을 경우, 밴드 1 로 가정합니다. 이 새 래스터는 원본

래스터와 동일한 지리참조, 너비 및 높이를 가질 것입니다. 지정하지 않은 밴드들은 변경 없이 반환될 것입니다. 유효한 재분류 표현식에 대한 설명이 필요한 경우 reclassarg 를 참조하십시오.

새 래스터의 밴드들은 pixeltype 픽셀 유형일 것입니다. reclassargset 를 넘겨받은 경우, 각 reclassarg(재분류 인수)가 생성된 각 밴드의 습성을 정의합니다.

2.0.0 버전부터 사용할 수 있습니다.

기본 예시

원본의 밴드 2 를 8BUI 에서 4BUI 로 변환하고 101 에서 254 까지의 모든 값을 NODATA 값으로 설정한 새 래스터를 생성합니다.

```
  ALTER TABLE dummy_rast ADD COLUMN reclass_rast raster;
  UPDATE dummy_rast SET reclass_rast = ST_Reclass(rast,2,'0-87:1-10, 88-
100:11-15, 101-254:0-0', '4BUI',0) WHERE rid = 2;

  SELECT i as col, j as row, ST_Value(rast,2,i,j) As origval,
     ST_Value(reclass_rast, 2, i, j) As reclassval,
     ST_Value(reclass_rast, 2, i, j, false) As reclassval_include_nodata
  FROM dummy_rast CROSS JOIN generate_series(1, 3) AS i CROSS JOIN
generate_series(1,3) AS j
  WHERE rid = 2;


   col | row | origval | reclassval | reclassval_include_nodata
  -----+-----+---------+------------+---------------------------
     1 |   1 |      78 |          9 |                         9
     2 |   1 |      98 |         14 |                        14
     3 |   1 |     122 |            |                         0
     1 |   2 |      96 |         14 |                        14
     2 |   2 |     118 |            |                         0
     3 |   2 |     180 |            |                         0
     1 |   3 |      99 |         15 |                        15
     2 |   3 |     112 |            |                         0
     3 |   3 |     169 |            |                         0
```

재분류 인수를 여러 개 이용하는 고급 예시

원본 래스터의 밴드 1, 2, 3 을 각각 1BB, 4BUI, 4BUI 로 변환하고 재분류한 새 래스터를 생성합니다. 이 예시가 (이론적으로 사용자가 가진 밴드의 개수만큼) 무한한 개수의 재분류 인수를 입력받을 수 있는, 다양한 개수의 인수를 입력받는 reclassarg 인수를 이용한다는 점에 주의하십시오.

```
  UPDATE dummy_rast SET reclass_rast =
     ST_Reclass(rast,
        ROW(2,'0-87]:1-10,          (87-100]:11-15,          (101-254]:0-0',
'4BUI',NULL)::reclassarg,
        ROW(1,'0-253]:1, 254:0', '1BB', NULL)::reclassarg,
        ROW(3,'0-70]:1,    (70-86:2,    [86-150):3,    [150-255:4',    '4BUI',
NULL)::reclassarg
        ) WHERE rid = 2;

  SELECT    i    as    col,    j    as    row,ST_Value(rast,1,i,j)    As    ov1,
ST_Value(reclass_rast, 1, i, j) As rv1,
     ST_Value(rast,2,i,j) As ov2, ST_Value(reclass_rast, 2, i, j) As rv2,
     ST_Value(rast,3,i,j) As ov3, ST_Value(reclass_rast, 3, i, j) As rv3
```

```
  FROM  dummy_rast  CROSS  JOIN  generate_series(1,  3)  AS  i  CROSS  JOIN
generate_series(1,3) AS j
  WHERE rid = 2;


  col | row | ov1 | rv1 | ov2 | rv2 | ov3 | rv3
  ----+-----+-----+-----+-----+-----+-----+-----
    1 |   1 | 253 |   1 |  78 |   9 |  70 |   1
    2 |   1 | 254 |   0 |  98 |  14 |  86 |   3
    3 |   1 | 253 |   1 | 122 |   0 | 100 |   3
    1 |   2 | 253 |   1 |  96 |  14 |  80 |   2
    2 |   2 | 254 |   0 | 118 |   0 | 108 |   3
    3 |   2 | 254 |   0 | 180 |   0 | 162 |   4
    1 |   3 | 250 |   1 |  99 |  15 |  90 |   3
    2 |   3 | 254 |   0 | 112 |   0 | 108 |   3
    3 |   3 | 254 |   0 | 169 |   0 | 175 |   4
```

단일 32BF 밴드를 가진 래스터를 보기 좋은 밴드 여러 개에 매핑하는 고급 예시

32BF 밴드 하나만 가진 래스터로부터 새로운 밴드 3 개((8BUI,8BUI,8BUI)를 가진 보기 좋은 새 래스터를 생성합니다.

```
  ALTER TABLE wind ADD COLUMN rast_view raster;
  UPDATE wind
        set rast_view = ST_AddBand( NULL,
        ARRAY[
        ST_Reclass(rast,           1,'0.1-10]:1-10,9-10]:11,(11-33:0'::text,
'8BUI'::text,0),
        ST_Reclass(rast,1,          '11-33):0-255,[0-32:0,(34-1000:0'::text,
'8BUI'::text,0),
        ST_Reclass(rast,1,'0-32]:0,(32-100:100-255'::text, '8BUI'::text,0)
        ]
        );
```

참고: ST_AddBand, ST_Band, ST_BandPixelType, ST_MakeEmptyRaster, reclassarg, ST_Value

9.11.1.12. ST_Union

ST_Union — 래스터 타일 집합을 1 개 이상의 밴드로 이루어진 단일 래스터로 통합합니다.

Synopsis

raster ST_Union(setof raster *rast*);

raster ST_Union(setof raster *rast*, unionarg[] *unionargset*);

raster ST_Union(setof raster *rast*, integer *nband*);

raster ST_Union(setof raster *rast*, text *uniontype*);

raster ST_Union(setof raster *rast*, integer *nband*, text *uniontype*);

설명: 래스터 타일 집합을 최소한 밴드 1 개로 이루어진 단일 래스터로 통합합니다. 출력 래스터의 범위는 전체 집합의 범위입니다. 교차 부분의 경우, uniontype 이 결과 값을 정의합니다. uniontype 의 값은 LAST(기본값), FIRST, MIN, MAX, COUNT, SUM, MEAN, RANGE 가운데 하나입니다.



래스터를 통합하려면, 무엇보다 동일하게 정렬돼 있어야 합니다. 더 자세한 내용 및 도움말은 ST_SameAlignment 와 ST_NotSameAlignmentReason 를 참조하십시오. 정렬 문제를 해결하는 방법 가운데 하나는 정렬을 위해 동일한 참조 래스터를 입력한 ST_Resample 함수를 이용하는 것입니다.

2.0.0 버전부터 사용할 수 있습니다.

개선 사항: 2.1.0 버전부터 속도가 향상됐습니다(완전히 C 언어 기반으로 변경했습니다).

2.1.0 버전부터 ST_Union(rast, unionarg) 변종을 사용할 수 있습니다.

개선 사항: 2.1.0 버전부터 ST_Union(rast) 변종 1 함수가 모든 입력 래스터의 모든 밴드를 통합합니다. PostGIS 이전 버전에서는 첫 번째 밴드로 가정했습니다.

개선 사항: 2.1.0 버전부터 ST_Union(rast, uniontype) 변종 4 함수가 모든 입력 래스터의 모든 밴드를 통합합니다.

예시: 단일 밴드를 가진 래스터 타일 뭉치를 재구성

```
  -- 원본 파일 시스템 타일을 형성하는 래스터 타일들의
  -- 첫 번째 밴드로부터 단일 밴드를 생성합니다.
  SELECT filename, ST_Union(rast,1) As file_rast
  FROM sometable WHERE filename IN('dem01', 'dem02') GROUP BY filename;
```

예시: 타일이 도형과 교차하는 부분을 통합한 다중 밴드 래스터를 반환

```
  -- 라인과 교차하는 모든 타일들을 모은 다중 밴드 래스터를 생성합니다.
  -- 주의: 2.0 버전에서 이 쿼리는 단일 밴드 래스터를 반환했겠지만
  -- 새 버전은 기본적으로 모든 밴드를 처리합니다.
  --  이  쿼리는  unionarg:  ARRAY[ROW(1,  'LAST'),  ROW(2,  'LAST'),  ROW(3,
'LAST')]::unionarg[ ]과 동일합니다.
  SELECT ST_Union(rast)
  FROM aerials.boston
  WHERE  ST_Intersects(rast,     ST_GeomFromText('LINESTRING(230486  887771,
230500 88772)',26986) );
```

예시: 타일이 도형과 교차하는 부분을 통합한 다중 밴드 래스터를 반환

다음은 밴드들의 하위 집합만을 원하거나, 또는 밴드들의 순서를 변경하고자 하는 경우 더 긴 문법을 사용하는 예시입니다.

```
  -- 라인과 교차하는 모든 타일들을 모은 다중 밴드 래스터를 생성합니다.
  SELECT   ST_Union(rast,ARRAY[ROW(2,   'LAST'),   ROW(1,   'LAST'),   ROW(3,
'LAST')]::unionarg[])
  FROM aerials.boston
  WHERE ST_Intersects(rast,  ST_GeomFromText('LINESTRING(230486 887771, 230500
88772)',26986) );
```

참고: unionarg, ST_Envelope, ST_ConvexHull, ST_Clip, ST_Union

9.11.2. 래스터 처리 내장 함수

ST_Distinct4ma — 이웃에서 유일한 픽셀 값들의 개수를 계산하는 래스터 공간 처리 함수입니다.

ST_InvDistWeight4ma — 픽셀의 이웃으로부터 픽셀 값을 보간하는 래스터 공간 처리 함수입니다.

ST_Max4ma — 이웃에서 최대 픽셀 값을 계산하는 래스터 공간 처리 함수입니다.

ST_Mean4ma — 이웃에서 평균 픽셀 값을 계산하는 래스터 공간 처리 함수입니다.

ST_Min4ma — 이웃에서 최소 픽셀 값을 계산하는 래스터 공간 처리 함수입니다.

ST_MinDist4ma — 관심 픽셀과 값을 가지고 있는 이웃 픽셀 사이의 최소 거리를 (픽셀 개수로) 반환하는 래스터 공간 처리 함수입니다.

ST_Range4ma — 이웃에 있는 픽셀값들의 범위를 계산하는 래스터 공간 처리 함수입니다.

ST_StdDev4ma — 이웃에 있는 픽셀값들의 표준 편차를 계산하는 래스터 공간 처리 함수입니다.

ST_Sum4ma — 이웃에 있는 모든 픽셀값들의 합계를 계산하는 래스터 공간 처리 함수입니다.

9.11.2.1. ST_Distinct4ma

ST_Distinct4ma — 이웃에서 유일한 픽셀 값들의 개수를 계산하는 래스터 공간 처리 함수입니다.

Synopsis

float8 ST_Distinct4ma(float8[][] matrix, text nodatamode, text[] VARIADIC args);

double precision ST_Distinct4ma(double precision[][][] value, integer[][] pos, text[] VARIADIC userargs);

설명: 픽셀의 이웃에 있는 유일한 픽셀 값들의 개수를 계산합니다.



변종 1 은 ST_MapAlgebraFctNgb 의 콜백 파라미터로 쓰이는 특화된 콜백 함수입니다.



변종 2 는 ST_MapAlgebra 의 콜백 파라미터로 쓰이는 특화된 콜백 함수입니다.



2.1.0 버전부터 ST_MapAlgebraFctNgb 를 지원하지 않기 때문에 변종 1 함수를 쓰지 않는 편이 좋습니다.

2.0.0 버전부터 사용할 수 있습니다.

개선 사항: 2.1.0 버전에서 변종 2 가 추가됐습니다.

예시

```
 SELECT
   rid,
   st_value(
     st_mapalgebrafctngb(rast,           1,           NULL,           1,           1,
'st_distinct4ma(float[][],text,text[])'::regprocedure, 'ignore', NULL), 2, 2
   )
 FROM dummy_rast
 WHERE rid = 2;
  rid | st_value
 -----+----------
   2 |     3
 (1 row)
```

참고: ST_MapAlgebraFctNgb, ST_MapAlgebra, ST_Min4ma, ST_Max4ma, ST_Sum4ma, ST_Mean4ma, ST_Distinct4ma, ST_StdDev4ma

9.11.2.2. ST_InvDistWeight4ma

ST_InvDistWeight4ma — 픽셀의 이웃으로부터 픽셀 값을 보간하는 래스터 공간 처리 함수입니다.

Synopsis

double precision **ST_InvDistWeight4ma**(double precision[][][] *value*, integer[][] *pos*, text[] *VARIADIC userargs*);

설명: 거리 역산 가중법(Inverse Distance Weighted method)을 이용해서 픽셀에 대한 보간값을 계산합니다.

userargs 를 통해 넘길 수 있는 선택적인 파라미터가 2 개 있습니다. 첫 번째 파라미터는 거리 역산 가중법 방정식에 쓰이는 0 과 1 사이의 역률(力率)입니다(다음 방정식의 k 변수). 따로 설정하지 않을 경우, 기본값은 1 입니다. 두 번째 파라미터는 관심 픽셀의 값이 이웃에서 나온 보간값에 포함될 경우에만 적용되는 가중치 백분율입니다. 관심 픽셀이 값을 가지고 있고 따로 설정하지 않을 경우, 해당 값을 반환합니다.

거리 역산 가중법의 기본 방정식은 다음과 같습니다:

$$\hat{z}(x_0) = \frac{\sum_{j=1}^{m} z(x_j) d_{ij}^{-k}}{\sum_{j=1}^{m} d_{ij}^{-k}}$$

k = 역률(power factor), 0 과 1 사이의 실수



이 함수는 ST_MapAlgebra 의 콜백 파라미터로 쓰이는 특화된 콜백 함수입니다.

2.1.0 버전부터 사용할 수 있습니다.

참고: ST_MapAlgebra, ST_MinDist4ma

9.11.2.3. ST_Max4ma

ST_Max4ma — 이웃에서 최대 픽셀 값을 계산하는 래스터 공간 처리 함수입니다.

Synopsis

float8 ST_Max4ma(float8[ ][ ] *matrix*, text *nodatamode*, text[ ] *VARIADIC args*);

double precision ST_Max4ma(double precision[ ][ ][ ] *value*, integer[ ][ ] *pos*, text[ ] *VARIADIC userargs*);

설명: 픽셀의 이웃에 있는 최대 픽셀 값을 계산합니다.

변종 2 의 경우, 해당 값을 userargs 에 넘겨서 NODATA 픽셀의 대체값을 지정할 수 있습니다.



변종 1 은 ST_MapAlgebraFctNgb 의 콜백 파라미터로 쓰이는 특화된 콜백 함수입니다.



변종 2 는 ST_MapAlgebra 의 콜백 파라미터로 쓰이는 특화된 콜백 함수입니다.



2.1.0 버전부터 ST_MapAlgebraFctNgb 를 지원하지 않기 때문에 변종 1 함수를 쓰지 않는 편이 좋습니다.

2.0.0 버전부터 사용할 수 있습니다.

개선 사항: 2.1.0 버전에서 변종 2 가 추가됐습니다.

예시

```
  SELECT
     rid,
     st_value(
        st_mapalgebrafctngb(rast,        1,        NULL,        1,        1,
'st_max4ma(float[][],text,text[])'::regprocedure, 'ignore', NULL), 2, 2
     )
  FROM dummy_rast
  WHERE rid = 2;
   rid | st_value
  -----+----------
     2 |      254
  (1 row)
```

참고: ST_MapAlgebraFctNgb, ST_MapAlgebra, ST_Min4ma, ST_Sum4ma, ST_Mean4ma, ST_Range4ma, ST_Distinct4ma, ST_StdDev4ma

9.11.2.4. ST_Mean4ma

ST_Mean4ma — 이웃에서 평균 픽셀 값을 계산하는 래스터 공간 처리 함수입니다.

Synopsis

float8 ST_Mean4ma(float8[ ][ ] *matrix*, text *nodatamode*, text[ ] *VARIADIC args*);

double precision ST_Mean4ma(double precision[ ][ ][ ] *value*, integer[ ][ ] *pos*, text[ ] *VARIADIC userargs*);

설명: 픽셀의 이웃에 있는 평균 픽셀 값을 계산합니다.

변종 2 의 경우, 해당 값을 userargs 에 넘겨서 NODATA 픽셀의 대체값을 지정할 수 있습니다.



변종 1 은 ST_MapAlgebraFctNgb 의 콜백 파라미터로 쓰이는 특화된 콜백 함수입니다.



변종 2 는 ST_MapAlgebra 의 콜백 파라미터로 쓰이는 특화된 콜백 함수입니다.



2.1.0 버전부터 ST_MapAlgebraFctNgb 를 지원하지 않기 때문에 변종 1 함수를 쓰지 않는 편이 좋습니다.

2.0.0 버전부터 사용할 수 있습니다.

개선 사항: 2.1.0 버전에서 변종 2 가 추가됐습니다.

예시: 변종 1

```
 SELECT
    rid,
    st_value(
       st_mapalgebrafctngb(rast,       1,       '32BF',       1,       1,
'st_mean4ma(float[][],text,text[])'::regprocedure, 'ignore', NULL), 2, 2
    )
 FROM dummy_rast
 WHERE rid = 2;
  rid |    st_value
 -----+------------------
   2 | 253.222229003906
 (1 row)
```

예시: 변종 2

```
 SELECT
    rid,
    st_value(
            ST_MapAlgebra(rast,   1,   'st_mean4ma(double   precision[][][],
integer[][], text[])'::regprocedure,'32BF', 'FIRST', NULL, 1, 1)
       ,  2, 2)
   FROM dummy_rast
```

```
    WHERE rid = 2;
 rid |    st_value
-----+------------------
   2 | 253.222229003906
(1 row)
```

참고: ST_MapAlgebraFctNgb, ST_MapAlgebra, ST_Min4ma, ST_Max4ma, ST_Sum4ma, ST_Range4ma, ST_StdDev4ma

9.11.2.5. ST_Min4ma

ST_Min4ma — 이웃에서 최소 픽셀 값을 계산하는 래스터 공간 처리 함수입니다.

Synopsis

float8 ST_Min4ma(float8[ ][ ] *matrix*, text *nodatamode*, text[ ] *VARIADIC args*);

double precision ST_Min4ma(double precision[ ][ ][ ] *value*, integer[ ][ ] *pos*, text[ ] *VARIADIC userargs*);

설명: 픽셀의 이웃에 있는 최소 픽셀 값을 계산합니다.

변종 2 의 경우, 해당 값을 userargs 에 넘겨서 NODATA 픽셀의 대체값을 지정할 수 있습니다.



변종 1 은 ST_MapAlgebraFctNgb 의 콜백 파라미터로 쓰이는 특화된 콜백 함수입니다.



변종 2 는 ST_MapAlgebra 의 콜백 파라미터로 쓰이는 특화된 콜백 함수입니다.



2.1.0 버전부터 ST_MapAlgebraFctNgb 를 지원하지 않기 때문에 변종 1 함수를 쓰지 않는 편이 좋습니다.

2.0.0 버전부터 사용할 수 있습니다.

개선 사항: 2.1.0 버전에서 변종 2 가 추가됐습니다.

예시

```
 SELECT
    rid,
    st_value(
       st_mapalgebrafctngb(rast,        1,        NULL,        1,        1,
'st_min4ma(float[ ][ ],text,text[ ])'::regprocedure, 'ignore', NULL), 2, 2
    )
 FROM dummy_rast
 WHERE rid = 2;
 rid | st_value
-----+----------
   2 |      250
(1 row)
```

참고: ST_MapAlgebraFctNgb, ST_MapAlgebra, ST_Max4ma, ST_Sum4ma, ST_Mean4ma, ST_Range4ma, ST_Distinct4ma, ST_StdDev4ma

9.11.2.6. ST_MinDist4ma

ST_MinDist4ma — 관심 픽셀과 값을 가지고 있는 이웃 픽셀 사이의 최소 거리를 (픽셀 개수로) 반환하는 래스터 공간 처리 함수입니다.

Synopsis

double precision ST_MinDist4ma(double precision[][][] *value*, integer[][] *pos*, text[] *VARIADIC userargs*);

설명: 관심 픽셀과 이웃에서 값을 가지고 있는 가장 가까운 픽셀 사이의 최단 거리를 (픽셀 개수로) 반환합니다.



이 함수의 목적은 ST_InvDistWeight4ma 함수가 반환하는 관심 픽셀의 보간값의 유용성을 추론하는 것을 도와주는 유익한 데이터 포인트를 제공하는 것입니다. 이웃의 밀도가 희박한 경우 이 함수가 특히 유용합니다.



이 함수는 ST_MapAlgebra 의 콜백 파라미터로 쓰이는 특화된 콜백 함수입니다.

2.1.0 버전부터 사용할 수 있습니다.

참고: ST_MapAlgebra, ST_InvDistWeight4ma

9.11.2.7. ST_Range4ma

ST_Range4ma — 이웃에 있는 픽셀값들의 범위를 계산하는 래스터 공간 처리 함수입니다.

Synopsis

float8 ST_Range4ma(float8[][] *matrix*, text *nodatamode*, text[] *VARIADIC args*);

double precision ST_Range4ma(double precision[][][] *value*, integer[][] *pos*, text[] *VARIADIC userargs*);

설명: 픽셀의 이웃에 있는 픽셀 값들의 범위를 계산합니다.

변종 2 의 경우, 해당 값을 userargs 에 넘겨서 NODATA 픽셀의 대체값을 지정할 수 있습니다.



변종 1 은 ST_MapAlgebraFctNgb 의 콜백 파라미터로 쓰이는 특화된 콜백 함수입니다.



변종 2 는 ST_MapAlgebra 의 콜백 파라미터로 쓰이는 특화된 콜백 함수입니다.



2.1.0 버전부터 ST_MapAlgebraFctNgb 를 지원하지 않기 때문에 변종 1 함수를 쓰지 않는 편이 좋습니다.

2.0.0 버전부터 사용할 수 있습니다.

개선 사항: 2.1.0 버전에서 변종 2 가 추가됐습니다.

예시

```
 SELECT
    rid,
    st_value(
       st_mapalgebrafctngb(rast,         1,         NULL,         1,         1,
'st_range4ma(float[][],text,text[])'::regprocedure, 'ignore', NULL), 2, 2
    )
 FROM dummy_rast
 WHERE rid = 2;
  rid | st_value
 -----+----------
   2 |       4
 (1 row)
```

참고: ST_MapAlgebraFctNgb, ST_MapAlgebra, ST_Min4ma, ST_Max4ma, ST_Sum4ma, ST_Mean4ma, ST_Distinct4ma, ST_StdDev4ma

9.11.2.8. ST_StdDev4ma

ST_StdDev4ma — 이웃에 있는 픽셀값들의 표준 편차를 계산하는 래스터 공간 처리 함수입니다.

Synopsis

float8 ST_StdDev4ma(float8[ ][ ] *matrix*, text *nodatamode*, text[ ] *VARIADIC args*);

double precision ST_StdDev4ma(double precision[ ][ ][ ] *value*, integer[ ][ ] *pos*, text[ ] *VARIADIC userargs*);

설명: 픽셀의 이웃에 있는 픽셀 값들의 표준 편차를 계산합니다.



변종 1 은 ST_MapAlgebraFctNgb 의 콜백 파라미터로 쓰이는 특화된 콜백 함수입니다.



변종 2 는 ST_MapAlgebra 의 콜백 파라미터로 쓰이는 특화된 콜백 함수입니다.



2.1.0 버전부터 ST_MapAlgebraFctNgb 를 지원하지 않기 때문에 변종 1 함수를 쓰지 않는 편이 좋습니다.

2.0.0 버전부터 사용할 수 있습니다.

개선 사항: 2.1.0 버전에서 변종 2 가 추가됐습니다.

예시

```
 SELECT
    rid,
    st_value(
       st_mapalgebrafctngb(rast,        1,        '32BF',        1,        1,
'st_stddev4ma(float[][],text,text[])'::regprocedure, 'ignore', NULL), 2, 2
    )
 FROM dummy_rast
 WHERE rid = 2;
  rid |     st_value
 -----+------------------
   2  | 1.30170822143555
 (1 row)
```

참고: ST_MapAlgebraFctNgb, ST_MapAlgebra, ST_Min4ma, ST_Max4ma, ST_Sum4ma, ST_Mean4ma, ST_Distinct4ma, ST_StdDev4ma

9.11.2.9. ST_Sum4ma

ST_Sum4ma — 이웃에 있는 모든 픽셀값들의 합계를 계산하는 래스터 공간 처리 함수입니다.

Synopsis

float8 ST_Sum4ma(float8[ ][ ] *matrix*, text *nodatamode*, text[ ] *VARIADIC args*);

double precision ST_Sum4ma(double precision[ ][ ][ ] *value*, integer[ ][ ] *pos*, text[ ] *VARIADIC userargs*);

설명: 픽셀의 이웃에 있는 모든 픽셀 값들의 합계를 계산합니다.

변종 2 의 경우, 해당 값을 userargs 에 넘겨서 NODATA 픽셀의 대체값을 지정할 수 있습니다.



변종 1 은 ST_MapAlgebraFctNgb 의 콜백 파라미터로 쓰이는 특화된 콜백 함수입니다.



변종 2 는 ST_MapAlgebra 의 콜백 파라미터로 쓰이는 특화된 콜백 함수입니다.



2.1.0 버전부터 ST_MapAlgebraFctNgb 를 지원하지 않기 때문에 변종 1 함수를 쓰지 않는 편이 좋습니다.

2.0.0 버전부터 사용할 수 있습니다.

개선 사항: 2.1.0 버전에서 변종 2 가 추가됐습니다.

예시

```
 SELECT
```

```
    rid,
    st_value(
       st_mapalgebrafctngb(rast,        1,        '32BF',        1,        1,
'st_sum4ma(float[ ][ ],text,text[ ])'::regprocedure, 'ignore', NULL), 2, 2
    )
  FROM dummy_rast
  WHERE rid = 2;



   rid | st_value
  -----+----------
     2 |     2279
  (1 row)
```

참고: ST_MapAlgebraFctNgb, ST_MapAlgebra, ST_Min4ma, ST_Max4ma, ST_Mean4ma, ST_Range4ma, ST_Distinct4ma, ST_StdDev4ma

9.11.3. DEM(표고)

ST_Aspect — 표고 래스터 밴드의 향(기본 단위는 도)을 반환합니다. 지형을 분석하는 데 유용합니다.

ST_HillShade — 입력한 방위각, 고도각, 밝기 및 축척을 이용해서 표고 래스터 밴드의 가상적인 음영기복을 반환합니다.

ST_Roughness — DEM 의 계산된 "거칠기(roughness)"와 함께 래스터를 반환합니다.

ST_Slope — 표고 래스터 밴드의 경사(기본 단위는 도)를 반환합니다. 지형을 분석하는 데 유용합니다.

ST_TPI — 계산된 지형위치지수(Topographic Position Index)와 함께 래스터를 반환합니다.

ST_TRI — 계산된 지형험준도지수(Terrain Ruggedness Index)와 함께 래스터를 반환합니다.

9.11.3.1. ST_Aspect

ST_Aspect — 표고 래스터 밴드의 향(기본 단위는 도)을 반환합니다. 지형을 분석하는 데 유용합니다.

Synopsis

raster ST_Aspect(raster *rast*, integer *band=1*, text *pixeltype=32BF*, text *units=DEGREES*, boolean *interpolate_nodata=FALSE*);

raster ST_Aspect(raster *rast*, integer *band*, raster *customextent*, text *pixeltype=32BF*, text *units=DEGREES*, boolean *interpolate_nodata=FALSE*);

설명: 표고 래스터 밴드의 향(기본 단위는 도)을 반환합니다. 맵 대수를 활용해서 이웃 픽셀들에 향 방정식을 적용합니다.

units 는 향의 단위를 의미합니다. RADIANS, DEGREES(기본값)를 쓸 수 있습니다.

units = RADIANS 일 경우, 북쪽에서 시계방향으로 측정한 0 과 2π 라디안 사이의 값입니다.

units = DEGREES 일 경우, 북쪽에서 시계방향으로 측정한 0 도와 360 도 사이의 값입니다.

픽셀의 경사가 0 일 경우, 픽셀의 향은 -1 입니다.



경사(slope), 향(aspect), 음영기복(hillshade)에 대한 자세한 내용을 알고 싶다면, ESRI - How hillshade works 및 ERDAS Field Guide - Aspect Images 를 참조하십시오.

2.0.0 버전부터 사용할 수 있습니다.

개선 사항: 2.1.0 버전부터 ST_MapAlgebra()를 이용하며, 선택적인 interpolate_nodata 함수 파라미터가 추가됐습니다.

변경 사항: 2.1.0 미만 버전에서는 반환되는 값이 라디안 단위였습니다. 2.1.0 버전부터 반환되는 값의 기본 단위가 도로 바뀌었습니다.

예시: 변종 1

```
  WITH foo AS (
        SELECT ST_SetValues(
              ST_AddBand(ST_MakeEmptyRaster(5, 5, 0, 0, 1, -1, 0, 0, 0), 1,
'32BF', 0, -9999),
              1, 1, 1, ARRAY[
                    [1, 1, 1, 1, 1],
                    [1, 2, 2, 2, 1],
                    [1, 2, 3, 2, 1],
                    [1, 2, 2, 2, 1],
                    [1, 1, 1, 1, 1]
              ]::double precision[][]
        ) AS rast
  )
  SELECT
        ST_DumpValues(ST_Aspect(rast, 1, '32BF'))
  FROM foo


st_dumpvalues

  ---------------------------------------------------------------------------

(1,"{{315,341.565063476562,0,18.4349479675293,45},{288.434936523438,315,0,45
,71.5650482177734},{270,270,-
1,90,90},{251.565048217773,225,180,135,108.434951782227},{225,198.43495178
  2227,180,161.565048217773,135}}")
  (1 row)
```

예시:변종 2

커버리지 타일의 완전한 예시입니다. 이 쿼리는 PostgreSQL 9.1 이상 버전에서만 작동합니다.

```
  WITH foo AS (
        SELECT ST_Tile(
              ST_SetValues(
                    ST_AddBand(
                          ST_MakeEmptyRaster(6, 6, 0, 0, 1, -1, 0, 0, 0),
                          1, '32BF', 0, -9999
                    ),
                    1, 1, 1, ARRAY[
```

```
                            [1, 1, 1, 1, 1, 1],
                            [1, 1, 1, 1, 2, 1],
                            [1, 2, 2, 3, 3, 1],
                            [1, 1, 3, 2, 1, 1],
                            [1, 2, 2, 1, 2, 1],
                            [1, 1, 1, 1, 1, 1]
                        ]::double precision[]
                ),
                2, 2
        ) AS rast
)
SELECT
        t1.rast,
        ST_Aspect(ST_Union(t2.rast), 1, t1.rast)
FROM foo t1
CROSS JOIN foo t2
WHERE ST_Intersects(t1.rast, t2.rast)
GROUP BY t1.rast;
```

참고: ST_MapAlgebra, ST_TRI, ST_TPI, ST_Roughness, ST_HillShade, ST_Slope

9.11.3.2. ST_HillShade

ST_HillShade — 입력한 방위각, 고도각, 밝기 및 축척을 이용해서 표고 래스터 밴드의 가상적인 음영기복을 반환합니다.

Synopsis

raster **ST_HillShade**(raster *rast*, integer *band=1*, text *pixeltype=32BF*, double precision *azimuth=315*, double precision *altitude=45*, double precision *max_bright=255*, double precision *scale=1.0*, boolean *interpolate_nodata=FALSE*);

raster **ST_HillShade**(raster *rast*, integer *band*, raster *customextent*, text *pixeltype=32BF*, double precision *azimuth=315*, double precision *altitude=45*, double precision *max_bright=255*, double precision *scale=1.0*, boolean *interpolate_nodata=FALSE*);

설명: 입력한 방위각, 고도각, 밝기 및 축척을 이용해서 표고 래스터 밴드의 가상적인 음영기복을 반환합니다. 맵 대수를 활용해서 이웃 픽셀들에 음영기복 방정식을 적용합니다. 반환되는 픽셀 값은 0 과 255 사이입니다.

azimuth 는 북쪽에서 시계방향으로 측정한 0 도와 360 도 사이의 값입니다.

altitude 는 0 도가 지평선, 90 도가 천정(天頂)인 0 도와 90 도 사이의 값입니다.

max_bright 는 0 이 밝기가 없고, 255 가 최대 밝기인 0 과 255 사이의 값입니다.

scale 은 수평 단위에 대한 수직 단위의 비율입니다. 피트:경위도의 경우 scale=370400, 미터:경위도의 경우 scale=111120 을 사용하십시오.

interpolate_nodata 가 참일 경우, 음영기복을 계산하기 전에 ST_InvDistWeight4ma 함수를 통해 입력 래스터에서 나온 NODATA 픽셀들의 값을 보간할 것입니다.



음영기복에 대해 더 자세히 알고 싶다면, How hillshade works 를 참조하십시오.

2.0.0 버전부터 사용할 수 있습니다.

개선 사항: 2.1.0 버전부터 ST_MapAlgebra()를 이용하며, 선택적인 interpolate_nodata 함수 파라미터가 추가됐습니다.

변경 사항: 2.1.0 미만 버전에서는 방위각 및 고도각이 라디안 단위였습니다. 2.1.0 버전부터 방위각과 고도각이 도 단위로 바뀌었습니다.

예시: 변종 1

```
 WITH foo AS (
       SELECT ST_SetValues(
             ST_AddBand(ST_MakeEmptyRaster(5, 5, 0, 0, 1, -1, 0, 0, 0), 1,
'32BF', 0, -9999),
             1, 1, 1, ARRAY[
                   [1, 1, 1, 1, 1],
                   [1, 2, 2, 2, 1],
                   [1, 2, 3, 2, 1],
                   [1, 2, 2, 2, 1],
                   [1, 1, 1, 1, 1]
             ]::double precision[][]
       ) AS rast
 )
 SELECT
       ST_DumpValues(ST_Hillshade(rast, 1, '32BF'))
 FROM foo


st_dumpvalues

 ---------------------------------------------------------------------------

(1,"{{NULL,NULL,NULL,NULL,NULL},{NULL,251.32763671875,220.749786376953,147.2
24319458008,NULL},
 {NULL,220.749786376953,180.312225341797,67.7497863769531,NULL},{NULL,147.22
4319458008
 ,67.7497863769531,43.1210060119629,NULL},{NULL,NULL,NULL,NULL,NULL}}")
 (1 row)
```

예시:변종 2

커버리지 타일의 완전한 예시입니다. 이 쿼리는 PostgreSQL 9.1 이상 버전에서만 작동합니다.

```
 WITH foo AS (
       SELECT ST_Tile(
             ST_SetValues(
                   ST_AddBand(
                         ST_MakeEmptyRaster(6, 6, 0, 0, 1, -1, 0, 0, 0),
                         1, '32BF', 0, -9999
                   ),
                   1, 1, 1, ARRAY[
                         [1, 1, 1, 1, 1, 1],
                         [1, 1, 1, 1, 2, 1],
                         [1, 2, 2, 3, 3, 1],
                         [1, 1, 3, 2, 1, 1],
                         [1, 2, 2, 1, 2, 1],
                         [1, 1, 1, 1, 1, 1]
```

```
                    ]::double precision[]
                ),
                2, 2
        ) AS rast
)
SELECT
        t1.rast,
        ST_Hillshade(ST_Union(t2.rast), 1, t1.rast)
FROM foo t1
CROSS JOIN foo t2
WHERE ST_Intersects(t1.rast, t2.rast)
GROUP BY t1.rast;
```

참고: ST_MapAlgebra, ST_TRI, ST_TPI, ST_Roughness, ST_Aspect, ST_Slope

9.11.3.3. ST_Roughness

ST_Roughness — DEM 의 계산된 "거칠기(roughness)"와 함께 래스터를 반환합니다.

Synopsis

raster **ST_Roughness**(raster *rast*, integer *nband*, raster *customextent*, text *pixeltype="32BF"* , boolean *interpolate_nodata=FALSE*);

설명: 관심 지역에 대해 최소값에서 최대값을 뺀 DEM 의 "거칠기"를 계산합니다.

2.1.0 버전부터 사용할 수 있습니다.

참고: ST_MapAlgebra, ST_TRI, ST_TPI, ST_Slope, ST_HillShade, ST_Aspect

9.11.3.4. ST_Slope

ST_Slope — 표고 래스터 밴드의 경사(기본 단위는 도)를 반환합니다. 지형을 분석하는 데 유용합니다.

Synopsis

raster **ST_Slope**(raster *rast*, integer *nband=1*, text *pixeltype=32BF*, text *units=DEGREES*, double precision *scale=1.0*, boolean *interpolate_nodata=FALSE*);

raster **ST_Slope**(raster *rast*, integer *nband*, raster *customextent*, text *pixeltype=32BF*, text *units=DEGREES*, double precision *scale=1.0*, boolean *interpolate_nodata=FALSE*);

설명: 표고 래스터 밴드의 경사(기본 단위는 도)를 반환합니다. 맵 대수를 활용해서 이웃 픽셀들에 경사 방정식을 적용합니다.

units 는 경사의 단위를 의미합니다. RADIANS, DEGREES(기본값), PERCENT 를 쓸 수 있습니다.

scale 은 수평 단위에 대한 수직 단위의 비율입니다. 피트:경위도의 경우 scale=370400, 미터:경위도의 경우 scale=111120 을 사용하십시오.

interpolate_nodata 가 참일 경우, 지표면 경사를 계산하기 전에 ST_InvDistWeight4ma 함수를 통해 입력 래스터에서 나온 NODATA 픽셀들의 값을 보간할 것입니다.



경사(slope), 향(aspect), 음영기복(hillshade)에 대한 자세한 내용을 알고 싶다면, ESRI - How hillshade works 및 ERDAS Field Guide - Slope Images 를 참조하십시오.

2.0.0 버전부터 사용할 수 있습니다.

개선 사항: 2.1.0 버전부터 ST_MapAlgebra()를 이용하며, 선택적인 units, scale, interpolate_nodata 함수 파라미터가 추가됐습니다.

변경 사항: 2.1.0 미만 버전에서는 반환되는 값이 라디안 단위였습니다. 2.1.0 버전부터 반환되는 값의 기본 단위가 도로 바뀌었습니다.

예시: 변종 1

```
 WITH foo AS (
       SELECT ST_SetValues(
              ST_AddBand(ST_MakeEmptyRaster(5, 5, 0, 0, 1, -1, 0, 0, 0), 1,
'32BF', 0, -9999),
              1, 1, 1, ARRAY[
                     [1, 1, 1, 1, 1],
                     [1, 2, 2, 2, 1],
                     [1, 2, 3, 2, 1],
                     [1, 2, 2, 2, 1],
                     [1, 1, 1, 1, 1]
              ]::double precision[][]
       ) AS rast
 )
 SELECT
       ST_DumpValues(ST_Slope(rast, 1, '32BF'))
 FROM foo

                        st_dumpvalues

 ---------------------------------------------------------------------------

(1,"{{10.0249881744385,21.5681285858154,26.5650520324707,21.5681285858154,10
.0249881744385},
 {21.5681285858154,35.2643890380859,36.8698959350586,35.2643890380859,21.568
1285858154},
 {26.5650520324707,36.8698959350586,0,36.8698959350586,26.5650520324707},
 {21.5681285858154,35.2643890380859,36.8698959350586,35.2643890380859,21.568
1285858154},
 {10.0249881744385,21.5681285858154,26.5650520324707,21.5681285858154,10.024
9881744385}}")
 (1 row)
```

예시: 변종 2

커버리지 타일의 완전한 예시입니다. 이 쿼리는 PostgreSQL 9.1 이상 버전에서만 작동합니다.

```
 WITH foo AS (
       SELECT ST_Tile(
              ST_SetValues(
                     ST_AddBand(
                            ST_MakeEmptyRaster(6, 6, 0, 0, 1, -1, 0, 0, 0),
                            1, '32BF', 0, -9999
                     ),
                     1, 1, 1, ARRAY[
```

```
                            [1, 1, 1, 1, 1, 1],
                            [1, 1, 1, 1, 2, 1],
                            [1, 2, 2, 3, 3, 1],
                            [1, 1, 3, 2, 1, 1],
                            [1, 2, 2, 1, 2, 1],
                            [1, 1, 1, 1, 1, 1]
                        ]::double precision[]
                ),
                2, 2
        ) AS rast
)
SELECT
        t1.rast,
        ST_Slope(ST_Union(t2.rast), 1, t1.rast)
FROM foo t1
CROSS JOIN foo t2
WHERE ST_Intersects(t1.rast, t2.rast)
GROUP BY t1.rast;
```

참고: ST_MapAlgebra, ST_TRI, ST_TPI, ST_Roughness, ST_HillShade, ST_Aspect

9.11.3.5. ST_TPI

ST_TPI — 계산된 지형위치지수(Topographic Position Index)와 함께 래스터를 반환합니다.

Synopsis

raster ST_TPI(raster rast, integer nband, raster customextent, text pixeltype="32BF" , boolean interpolate_nodata=FALSE);

설명: 1 반경에서 중심 셀을 뺀 중심 평균이라고 정의되는 지형위치지수(Topographic Position Index)를 계산합니다.

2.1.0 버전부터 사용할 수 있습니다.



이 함수는 1 반경 중심 평균(focalmean radius of one)만을 지원합니다.

참고: ST_MapAlgebra, ST_TRI, ST_Roughness, ST_Slope, ST_HillShade, ST_Aspect

9.11.3.6. ST_TRI

ST_TRI — 계산된 지형험준도지수(Terrain Ruggedness Index)와 함께 래스터를 반환합니다.

Synopsis

raster **ST_TRI**(raster *rast*, integer *nband*, raster *customextent*, text *pixeltype="32BF"* , boolean *interpolate_nodata=FALSE*);

설명: 중심 픽셀과 그 이웃을 비교해서 그 차의 절대값의 평균으로 지형험준도지수(Terrain Ruggedness Index)를 계산합니다.



이 함수는 1 반경 중심 평균(focalmean radius of one)만을 지원합니다.

2.1.0 버전부터 사용할 수 있습니다.

참고: ST_MapAlgebra, ST_Roughness, ST_TPI, ST_Slope, ST_HillShade, ST_Aspect

9.11.4. 래스터를 도형으로

Box3D — 래스터를 둘러싼 상자의 BOX3D 표현식을 반환합니다.

ST_ConvexHull — BandNoDataValue 와 일치하는 픽셀 값을 포함한, 래스터의 볼록 껍질 도형을 반환합니다. 정규 형상 및 기울어지지 않은 래스터의 경우, ST_Envelope 와 동일한 결과물을 출력하므로 비정규 형상 또는 기울어진 래스터에 대해서만 쓸모가 있습니다.

ST_DumpAsPolygons — 입력 래스터 밴드로부터 geomval(geom, val) 행들의 집합을 반환합니다. 밴드 번호를 설정하지 않을 경우 기본적으로 밴드 1 로 가정합니다.

ST_Envelope — 래스터 범위의 폴리곤 표현식을 반환합니다.

ST_MinConvexHull — 래스터의 NODATA 픽셀을 제외한 볼록 껍질 도형을 반환합니다.

ST_Polygon — NODATA 값이 아닌 픽셀 값을 가진 픽셀들을 통합해서 형성된 멀티폴리곤 도형을 반환합니다.

9.11.4.1. Box3D

Box3D — 래스터를 둘러싼 상자의 BOX3D 표현식을 반환합니다.

Synopsis

box3d **Box3D**(raster *rast*);

설명: 래스터의 범위를 표현하는 상자를 반환합니다.

경계 상자의 꼭짓점 포인트들((MINX, MINY), (MAXX, MAXY))이 폴리곤을 정의합니다.

변경 사항: 2.0.0 미만 버전에서는 BOX3D 대신 BOX2D 를 이용했습니다. BOX2D 가 지원이 끝난 유형이기 때문에, 2.0.0 버전에서 BOX3D 로 바뀌었습니다.

예시

```
SELECT
       rid,
       Box3D(rast) AS rastbox
FROM dummy_rast;

rid |       rastbox
----+--------------------------------------------------
1   | BOX3D(0.5 0.5 0,20.5 60.5 0)
2   | BOX3D(3427927.75 5793243.5 0,3427928 5793244 0)
```

참고: ST_Envelope

9.11.4.2. ST_ConvexHull

ST_ConvexHull — BandNoDataValue 와 일치하는 픽셀 값을 포함한, 래스터의 볼록 껍질 도형을 반환합니다. 정규 형상 및 기울어지지 않은 래스터의 경우, ST_Envelope 와 동일한 결과물을 출력하므로 비정규 형상 또는 기울어진 래스터에 대해서만 쓸모가 있습니다.

Synopsis

geometry ST_ConvexHull(raster *rast*);

설명: NoDataBandValue 밴드 픽셀을 포함한, 래스터의 볼록 껍질 도형을 반환합니다. 정규 형상 및 기울어지지 않은 래스터의 경우, ST_Envelope 와 거의 동일한 결과물을 출력하므로 비정규 형상 또는 기울어진 래스터에 대해서만 쓸모가 있습니다.



ST_Envelope 함수는 좌표를 버림(floor)하기 때문에 래스터 주위에 버퍼를 약간 주게 됩니다. 따라서 버림을 하지 않는 ST_ConvexHull 함수와는 약간 다른 결과를 냅니다.

예시

이 차이에 대한 도표를 보고 싶다면 PostGIS Raster Specification 을 참조하십시오.

```
-- 엔벨로프와 볼록 껍질이 거의 동일하다는 점에 주의하십시오.
SELECT ST_AsText(ST_ConvexHull(rast)) As convhull,
       ST_AsText(ST_Envelope(rast)) As env
FROM dummy_rast WHERE rid=1;

            convhull             |        env
---------------------------------+-----------------------------------------------------
 POLYGON((0.5 0.5,20.5 0.5,20.5 60.5,0.5 60.5,0.5 0.5)) | POLYGON((0 0,20 0,20 60,0 60,0
0))

-- 래스터를 기울입니다.
-- 이제 볼록 껍질과 엔벨로프가 어떻게 달라지는지 눈여겨 보십시오.
SELECT ST_AsText(ST_ConvexHull(rast)) As convhull,
       ST_AsText(ST_Envelope(rast)) As env
FROM (SELECT ST_SetRotation(rast, 0.1, 0.1) As rast
       FROM dummy_rast WHERE rid=1) As foo;


            convhull             |        env
---------------------------------+-----------------------------------------------------
 POLYGON((0.5 0.5,20.5 1.5,22.5 61.5,2.5 60.5,0.5 0.5)) | POLYGON((0 0,22 0,22 61,0 61,0
0))
```

참고: ST_Envelope, ST_MinConvexHull, ST_ConvexHull, ST_AsText

9.11.4.3. ST_DumpAsPolygons

ST_DumpAsPolygons — 입력 래스터 밴드로부터 geomval(geom, val) 행들의 집합을 반환합니다. 밴드 번호를 설정하지 않을 경우 기본적으로 밴드 1 로 가정합니다.

Synopsis

setof geomval ST_DumpAsPolygons(raster *rast*, integer *band_num=1*, boolean *exclude_nodata_value=TRUE*);

설명: 이 함수는 집합 반환 함수(SRF; Set-Returning Function)입니다. 도형(geom)과 픽셀 밴드 값(val)으로 이루어진 geomval 행들의 집합을 반환합니다. 각 폴리곤은 해당 밴드에서 val 이 표시하는 값과 동일한 픽셀 값을 가진 모든 픽셀들을 통합한 것입니다.

ST_DumpAsPolygon 은 래스터를 폴리곤화하는 데 유용합니다. 새 행들을 생성한다는 점에서 GROUP BY 의 역함수라고 할 수 있습니다. 예를 들어 단일 래스터를 복수의 폴리곤/멀티폴리곤으로 확장하는 데 이 함수를 쓸 수 있습니다.

GDAL 1.7 이상 버전이 필요합니다.



밴드에 대해 설정된 NODATA 값이 있을 경우, 해당 값을 가진 픽셀은 반환되지 않습니다.



래스터 안에 있는 해당 값을 가진 픽셀들의 개수에만 관심이 있다면, ST_ValueCount 함수가 더 빠릅니다.



이 함수는 픽셀 값과 상관없이 각 픽셀에 대해 도형 하나를 반환하는 ST_PixelAsPolygons 함수와 다릅니다.

예시

```
  SELECT val, ST_AsText(geom) As geomwkt
  FROM (
  SELECT (ST_DumpAsPolygons(rast)).*
  FROM dummy_rast
  WHERE rid = 2
  ) As foo
  WHERE val BETWEEN 249 and 251
  ORDER BY val;

  val |                              geomwkt
  -----+-------------------------------------------------------------------------------
249 | POLYGON((3427927.95 5793243.95,3427927.95 5793243.85,3427928 5793243.85,
          3427928 5793243.95,3427927.95 5793243.95))
  250 | POLYGON((3427927.75 5793243.9,3427927.75 5793243.85,3427927.8 5793243.85,
          3427927.8 5793243.9,3427927.75 5793243.9))
  250 | POLYGON((3427927.8 5793243.8,3427927.8 5793243.75,3427927.85 5793243.75,
          3427927.85 5793243.8, 3427927.8 5793243.8))
  251 | POLYGON((3427927.75 5793243.85,3427927.75 5793243.8,3427927.8 5793243.8,
          3427927.8 5793243.85,3427927.75 5793243.85))
```

참고: geomval, ST_Value, ST_Polygon, ST_ValueCount

9.11.4.4. ST_Envelope

ST_Envelope — 래스터 범위의 폴리곤 표현식을 반환합니다.

Synopsis

geometry **ST_Envelope**(raster *rast*);

설명: 래스터 범위의 폴리곤 표현식을 SRID 가 정의하는 공간 좌표 단위로 반환합니다. 여기에서 폴리곤 표현식이란 폴리곤으로 표현된 float8 형 최소치 경계 상자입니다.

해당 폴리곤은 경계 상자의 꼭짓점 포인트들로 정의됩니다((MINX, MINY), (MINX, MAXY), (MAXX, MAXY), (MAXX, MINY), (MINX, MINY)).

예시

```
SELECT rid, ST_AsText(ST_Envelope(rast)) As envgeomwkt
FROM dummy_rast;

 rid |                                envgeomwkt
-----+---------------------------------------------------------------------
   1 | POLYGON((0 0,20 0,20 60,0 60,0 0))
   2 | POLYGON((3427927 5793243,3427928 5793243,
             3427928 5793244,3427927 5793244, 3427927 5793243))
```

참고: ST_Envelope, ST_AsText, ST_SRID

9.11.4.5. ST_MinConvexHull

ST_MinConvexHull — 래스터의 NODATA 픽셀을 제외한 볼록 껍질 도형을 반환합니다.

Synopsis

geometry **ST_MinConvexHull**(raster *rast*, integer *nband=NULL*);

설명: 래스터의 NODATA 픽셀을 제외한 볼록 껍질 도형을 반환합니다. nband 가 NULL 일 경우, 래스터의 모든 밴드를 처리합니다.

2.1.0 버전부터 사용할 수 있습니다.

예시

```
WITH foo AS (
       SELECT
              ST_SetValues(
                     ST_SetValues(
                            ST_AddBand(ST_AddBand(ST_MakeEmptyRaster(9, 9, 0,
0, 1, -1, 0, 0, 0), 1, '8BUI', 0, 0), 2, '8BUI', 1, 0),
                            1, 1, 1,
                            ARRAY[
                                   [0, 0, 0, 0, 0, 0, 0, 0, 0],
```

```
                                    [0, 0, 0, 0, 0, 0, 0, 0, 0],
                                    [0, 0, 0, 0, 0, 0, 0, 0, 0],
                                    [0, 0, 0, 1, 0, 0, 0, 0, 1],
                                    [0, 0, 0, 1, 1, 0, 0, 0, 0],
                                    [0, 0, 0, 1, 0, 0, 0, 0, 0],
                                    [0, 0, 0, 0, 0, 0, 0, 0, 0],
                                    [0, 0, 0, 0, 0, 0, 0, 0, 0],
                                    [0, 0, 0, 0, 0, 0, 0, 0, 0]
                              ]::double precision[][]
                        ),
                        2, 1, 1,
                        ARRAY[
                              [0, 0, 0, 0, 0, 0, 0, 0, 0],
                              [0, 0, 0, 0, 0, 0, 0, 0, 0],
                              [0, 0, 0, 0, 0, 0, 0, 0, 0],
                              [1, 0, 0, 0, 0, 1, 0, 0, 0],
                              [0, 0, 0, 0, 1, 1, 0, 0, 0],
                              [0, 0, 0, 0, 0, 1, 0, 0, 0],
                              [0, 0, 0, 0, 0, 0, 0, 0, 0],
                              [0, 0, 0, 0, 0, 0, 0, 0, 0],
                              [0, 0, 1, 0, 0, 0, 0, 0, 0]
                        ]::double precision[][]
                  ) AS rast
  )
  SELECT
        ST_AsText(ST_ConvexHull(rast)) AS hull,
        ST_AsText(ST_MinConvexHull(rast)) AS mhull,
        ST_AsText(ST_MinConvexHull(rast, 1)) AS mhull_1,
        ST_AsText(ST_MinConvexHull(rast, 2)) AS mhull_2
  FROM foo

         hull        |        mhull        |        mhull_1        |        mhull_2
  -------------------+---------------------+-----------------------+---------------
   POLYGON((0 0,9 0,9 -9,0 -9,0 0)) | POLYGON((0 -3,9 -3,9 -9,0 -9,0 -3)) | POLYGON((3 -3,9 -
3,9 -6,3 -6,3 -3)) | POLYGON((0 -3,6 -3,6 -9,0 -9,0 -3))
```

참고: ST_Envelope, ST_ConvexHull, ST_ConvexHull, ST_AsText

9.11.4.6. ST_Polygon

ST_Polygon — NODATA 값이 아닌 픽셀 값을 가진 픽셀들을 통합해서 형성된 멀티폴리곤 도형을 반환합니다.

Synopsis

geometry ST_Polygon(raster *rast*, integer *band_num=1*);

설명: 0.1.6 버전부터 사용할 수 있습니다. GDAL 1.7 이상 버전이 필요합니다.

개선 사항: 2.1.0 버전부터 속도가 향상됐습니다(완전히 C 언어 기반으로 변경했습니다). 반환되는 멀티폴리곤이 유효한 도형인지 확인합니다.

변경 사항: 2.1.0 미만 버전에서 종종 폴리곤을 반환하곤 했던 것을, 언제나 멀티폴리곤을 반환하도록 바꿨습니다.

예시

```
  -- 기본적으로 NODATA 밴드 값이 0 또는 설정되지 않은 상태로, 따라서 사각형 폴리곤을 반환할 것입니다.
  SELECT ST_AsText(ST_Polygon(rast)) As geomwkt
  FROM dummy_rast
  WHERE rid = 2;

  geomwkt
  ---------------------------------------------
  MULTIPOLYGON(((3427927.75  5793244,3427928  5793244,3427928  5793243.75,3427927.75
5793243.75,3427927.75 5793244)))


  -- 이제 첫 번째 밴드의 NODATA 값을 변경합니다.
  UPDATE dummy_rast SET rast = ST_SetBandNoDataValue(rast,1,254)
  WHERE rid = 2;
  SELECt rid, ST_BandNoDataValue(rast)
  from dummy_rast where rid = 2;

  -- ST_Polygon 이 필셀 값 254 를 제외하고 멀티폴리곤을 반환합니다.
  SELECT ST_AsText(ST_Polygon(rast)) As geomwkt
  FROM dummy_rast
  WHERE rid = 2;



  geomwkt
  ------------------------------------------------------------
  MULTIPOLYGON(((3427927.9        5793243.95,3427927.85        5793243.95,3427927.85
5793244,3427927.9    5793244,3427927.9    5793243.95)),((3427928    5793243.85,3427928
5793243.8,3427927.95  5793243.8,3427927.95  5793243.85,3427927.9  5793243.85,3427927.9
5793243.9,3427927.9   5793243.95,3427927.95   5793243.95,3427928   5793243.95,3427928
5793243.85)),((3427927.8         5793243.75,3427927.75         5793243.75,3427927.75
5793243.8,3427927.75  5793243.85,3427927.75  5793243.9,3427927.75  5793244,3427927.8
5793244,3427927.8  5793243.9,3427927.8  5793243.85,3427927.85  5793243.85,3427927.85
5793243.8,3427927.85 5793243.75,3427927.8 5793243.75)))

  -- 또는 NODATA 값을 한번만 다르게 하고 싶을 경우

  SELECT ST_AsText(
        ST_Polygon(
              ST_SetBandNoDataValue(rast,1,252)
              )
        ) As geomwkt
  FROM dummy_rast
  WHERE rid =2;

  geomwkt
  ---------------------------------
  MULTIPOLYGON(((3427928  5793243.85,3427928  5793243.8,3427928  5793243.75,3427927.85
5793243.75,3427927.8  5793243.75,3427927.8  5793243.8,3427927.75  5793243.8,3427927.75
5793243.85,3427927.75   5793243.9,3427927.75   5793244,3427927.8   5793244,3427927.85
5793244,3427927.9      5793244,3427928      5793244,3427928      5793243.95,3427928
5793243.85),(3427927.9           5793243.9,3427927.9           5793243.85,3427927.95
5793243.85,3427927.95 5793243.9,3427927.9 5793243.9))
```

참고: ST_Value, ST_DumpAsPolygons

9.12. 래스터 연산자들

&& — A 의 경계 상자와 B 의 경계 상자가 교차하는 경우 TRUE 를 반환합니다.

&< — A 의 경계 상자가 B 의 경계 상자 왼쪽에 있을 경우 TRUE 를 반환합니다.

&> — A 의 경계 상자가 B 의 경계 상자 오른쪽에 있을 경우 TRUE 를 반환합니다.

= — A 의 경계 상자와 B 의 경계 상자가 동일할 경우 TRUE 를 반환합니다. 이중 정밀도 경계 상자를 이용합니다.

@ — B 의 경계 상자가 A 의 경계 상자를 담고 있을 경우 TRUE 를 반환합니다. 이중 정밀도 경계 상자를 이용합니다.

~= — A 의 경계 상자와 B 의 경계 상자가 동일할 경우 TRUE 를 반환합니다.

~ — A 의 경계 상자가 B 의 경계 상자를 담고 있을 경우 TRUE 를 반환합니다. 이중 정밀도 경계 상자를 이용합니다.

9.12.1. &&

&& — A 의 경계 상자와 B 의 경계 상자가 교차하는 경우 TRUE 를 반환합니다.

Synopsis

boolean &&(raster *A* , raster *B*);

boolean &&(raster *A* , geometry *B*);

boolean &&(geometry *B* , raster *A*);

설명: && 연산자는 래스터/도형 A 의 경계 상자와 래스터/도형 B 의 경계 상자가 교차하는 경우 TRUE 를 반환합니다.



이 피연산자(operand)는 래스터에서 이용할 수도 있는 모든 인덱스를 활용할 것입니다.

2.0.0 버전부터 사용할 수 있습니다.

예시

```
SELECT A.rid As a_rid, B.rid As b_rid, A.rast && B.rast As intersect
 FROM dummy_rast AS A CROSS JOIN dummy_rast AS B LIMIT 3;



 a_rid | b_rid | intersect
-------+-------+---------
     2 |     2 | t
     2 |     3 | f
     2 |     1 | f
```

9.12.2. &<

&< — A 의 경계 상자가 B 의 경계 상자 왼쪽에 있을 경우 TRUE 를 반환합니다.

Synopsis

boolean &>(raster *A* , raster *B*);

설명: &< 연산자는 래스터 A 의 경계 상자가 래스터 B 의 경계 상자와 겹치거나 그 왼쪽에 있을 경우, 또는 더 정확히 말하자면 래스터 B 의 경계 상자와 겹치거나 그 오른쪽에 있지 않을 경우, TRUE 를 반환합니다.



이 피연산자(operand)는 래스터에서 이용할 수도 있는 모든 인덱스를 활용할 것입니다.

예시

```
SELECT A.rid As a_rid, B.rid As b_rid, A.rast &< B.rast As overleft
 FROM dummy_rast AS A CROSS JOIN dummy_rast AS B;

a_rid | b_rid | overleft
------+-------+----------
   2 |     2 | t
   2 |     3 | f
   2 |     1 | f
   3 |     2 | t
   3 |     3 | t
   3 |     1 | f
   1 |     2 | t
   1 |     3 | t
   1 |     1 | t
```

9.12.3. &>

&> — A 의 경계 상자가 B 의 경계 상자 오른쪽에 있을 경우 TRUE 를 반환합니다.

Synopsis

boolean &>(raster *A* , raster *B*);

설명: &> 연산자는 래스터 A 의 경계 상자가 래스터 B 의 경계 상자와 겹치거나 그 오른쪽에 있을 경우, 또는 더 정확히 말하자면 래스터 B 의 경계 상자와 겹치거나 그 왼쪽에 있지 않을 경우, TRUE 를 반환합니다.



이 피연산자(operand)는 래스터에서 이용할 수도 있는 모든 인덱스를 활용할 것입니다.

예시

```
SELECT A.rid As a_rid, B.rid As b_rid, A.rast &> B.rast As overright
 FROM dummy_rast AS A CROSS JOIN dummy_rast AS B;

 a_rid | b_rid | overright
-------+-------+-----------
     2 |     2 | t
     2 |     3 | t
```

```
      2 |     1 | t
      3 |     2 | f
      3 |     3 | t
      3 |     1 | f
      1 |     2 | f
      1 |     3 | t
      1 |     1 | t
```

9.12.4. = — A

= — A 의 경계 상자와 B 의 경계 상자가 동일할 경우 TRUE 를 반환합니다. 이중 정밀도 경계 상자를 이용합니다.

Synopsis

boolean =(raster *A* , raster *B*);

설명: = 연산자는 래스터 A 의 경계 상자와 래스터 B 의 경계 상자가 동일할 경우 TRUE 를 반환합니다. PostgreSQL 은 래스터를 내부 정렬하고 비교하기 위해 도형에 대해 정의된 =, < 및 > 연산자를 이용합니다(예: GROUP BY 또는 ORDER BY 절에서).


이 피연산자(operand)는 래스터에 대해 이용할 수 있을지도 모르는 어떤 인덱스도 활용하지 않을 것입니다. 대신 ~= 함수를 이용하십시오. 이 연산자의 목적은 대부분 래스터 열로 그룹화(group by)하는 것입니다.

2.1.0 버전부터 사용할 수 있습니다.

참고: ~=

9.12.5. @ — B

@ — B 의 경계 상자가 A 의 경계 상자를 담고 있을 경우 TRUE 를 반환합니다. 이중 정밀도 경계 상자를 이용합니다.

Synopsis

boolean **@**(raster *A* , raster *B*);

boolean **@**(geometry *A* , raster *B*);

boolean **@**(raster *B* , geometry *A*);

설명: @ 연산자는 래스터/도형 B 의 경계 상자가 래스터/도형 A 의 경계 상자를 담고 있을 경우 TRUE 를 반환합니다.


이 피연산자(operand)는 래스터에 있는 공간 인덱스를 활용할 것입니다.

2.0.0 버전부터 raster @ raster, raster @ geometry 를 사용할 수 있습니다.

2.0.5 버전부터 geometry @ raster 를 지원합니다.

참고: ~

9.12.6. ~= — A

~= — A 의 경계 상자와 B 의 경계 상자가 동일할 경우 TRUE 를 반환합니다.

Synopsis

boolean **~=**(raster *A* , raster *B*);

설명: ~= 연산자는 래스터 A 의 경계 상자와 래스터 B 의 경계 상자가 동일할 경우 TRUE 를 반환합니다.



이 피연산자(operand)는 래스터에서 이용할 수도 있는 모든 인덱스를 활용할 것입니다.

2.0.0 버전부터 사용할 수 있습니다.

예시

동일한 뭉치이지만 서로 다른 주제를 표현하는 단일 밴드 래스터의 집합 2 개를 입력받아 다중 밴드 래스터를 생성하는 데 매우 유용합니다.

```
SELECT ST_AddBand(prec.rast, alt.rast) As new_rast
   FROM prec INNER JOIN alt ON (prec.rast ~= alt.rast);
```

참고: ST_AddBand, =

9.12.7. ~ — A

~ — A 의 경계 상자가 B 의 경계 상자를 담고 있을 경우 TRUE 를 반환합니다. 이중 정밀도 경계 상자를 이용합니다.

Synopsis

boolean ~(raster A , raster B);

boolean ~(geometry A , raster B);

boolean ~(raster B , geometry A);

설명: ~ 연산자는 래스터/도형 A 의 경계 상자가 래스터/도형 B 의 경계 상자를 담고 있을 경우 TRUE 를 반환합니다.



이 피연산자(operand)는 래스터에 있는 공간 인덱스를 활용할 것입니다.

2.0.0 버전부터 사용할 수 있습니다.

참고: @

9.13. 래스터 및 래스터 밴드 공간적 관계성

ST_Contains — 래스터 rastA 외부에 놓인 래스터 rastB 의 포인트가 하나도 없고, 적어도 rastB 내부의 포인트 하나가 rastA 의 내부에 있는 경우 참을 반환합니다.

ST_ContainsProperly — rastB 가 rastA 의 내부와 교차하지만 rastA 의 경계선 또는 외부와 교차하지는 않을 경우 참을 반환합니다.

ST_Covers — 래스터 rastB 의 어떤 포인트도 래스터 rastA 외부에 없을 경우 참을 반환합니다.

ST_CoveredBy — 래스터 rastA 의 어떤 포인트도 래스터 rastB 외부에 없을 경우 참을 반환합니다.

ST_Disjoint — 래스터 rastA 와 래스터 rastB 가 공간적으로 교차하지 않을 경우 참을 반환합니다.

ST_Intersects — 래스터 rastA 와 래스터 rastB 가 공간적으로 교차할 경우 참을 반환합니다.

ST_Overlaps — 래스터 rastA 와 래스터 rastB 가 교차하지만 어느 한 쪽이 다른 한 쪽을 완전히 담고 있지는 않을 경우 참을 반환합니다.

ST_Touches — 래스터 rastA 와 래스터 rastB 가 최소한 포인트 한 개를 공유하지만, 내부가 교차하지는 않을 경우 TRUE 를 반환합니다.

ST_SameAlignment — 래스터들이 동일한 기울기, 축척, 공간 참조 시스템, 그리고 오프셋을 가지고 있을 경우 (픽셀이 다른 픽셀을 침범하지 않고 동일한 그리드에 놓일 수 있을 경우) 참을 반환하고, 그렇지 않을 경우 문제점을 설명하는 안내문과 함께 거짓을 반환합니다.

ST_NotSameAlignmentReason — 래스터들이 정렬돼 있는지 아닌지, 그리고 정렬되지 않았다면 그 이유를 설명하는 텍스트를 반환합니다.

ST_Within — 래스터 rastB 외부에 놓인 래스터 rastA 의 포인트가 하나도 없고, 적어도 rastA 내부의 포인트 하나가 rastB 의 내부에 있는 경우 참을 반환합니다.

ST_DWithin — 래스터 rastA 와 래스터 rastB 가 서로 설정된 거리 안에 있을 경우 참을 반환합니다.

ST_DFullyWithin — 래스터 rastA 와 래스터 rastB 가 완전히 서로 설정된 거리 안에 있을 경우 참을 반환합니다.

9.13.1. ST_Contains

ST_Contains — 래스터 rastA 외부에 놓인 래스터 rastB 의 포인트가 하나도 없고, 적어도 rastB 내부의 포인트 하나가 rastA 의 내부에 있는 경우 참을 반환합니다.

Synopsis

boolean **ST_Contains**(raster *rastA* , integer *nbandA* , raster *rastB* , integer *nbandB*);

boolean **ST_Contains**(raster *rastA* , raster *rastB*);

설명: 래스터 rastA 외부에 놓인 래스터 rastB 의 포인트가 하나도 없고, 적어도 rastB 내부의 포인트 하나가 rastA 의 내부에 있을 경우에만 rastA 가 rastB 를 담고 있다고 합니다. 밴드 번호를 설정하지 않거나 또는 NULL 로 설정할 경우, 확인 작업중 래스터의 볼록 껍질만 고려합니다. 밴드 번호를 설정할 경우, 확인 작업중 값을 가진 (NODATA 가 아닌) 픽셀들만 고려합니다.



이 함수는 래스터에서 이용할 수도 있는 모든 인덱스를 활용할 것입니다.



래스터와 도형의 공간 관계성을 확인하려면, ST_Contains(ST_Polygon(raster), geometry) 또는 ST_Contains(geometry, ST_Polygon(raster))처럼 래스터에 ST_Polygon 함수를 이용하십시오.



ST_Contains()는 ST_Within()의 역함수입니다. 따라서, ST_Contains(rastA, rastB)의 결과와 ST_Within(rastB, rastA)의 결과는 동일할 것입니다.

2.1.0 버전부터 사용할 수 있습니다.

예시

```
  -- 밴드 번호 설정
  SELECT r1.rid, r2.rid, ST_Contains(r1.rast, 1, r2.rast, 1) FROM dummy_rast
r1 CROSS JOIN dummy_rast r2 WHERE r1.rid = 1;

  NOTICE:  The first raster provided has no bands
   rid | rid | st_contains
  -----+-----+-------------
     1 |   1 |
     1 |   2 | f

  -- 밴드 번호 설정하지 않음
  SELECT r1.rid, r2.rid, ST_Contains(r1.rast, r2.rast) FROM dummy_rast r1
CROSS JOIN dummy_rast r2 WHERE r1.rid = 1;
   rid | rid | st_contains
  -----+-----+-------------
     1 |   1 | t
     1 |   2 | f
```

참고: ST_Intersects, ST_Within

9.13.2. ST_ContainsProperly

ST_ContainsProperly — rastB 가 rastA 의 내부와 교차하지만 rastA 의 경계선 또는 외부와 교차하지는 않을 경우 참을 반환합니다.

Synopsis

boolean **ST_ContainsProperly**(raster *rastA* , integer *nbandA* , raster *rastB* , integer *nbandB*);

boolean **ST_ContainsProperly**(raster *rastA* , raster *rastB*);

설명: 래스터 rastB 가 래스터 rastA 의 내부와 교차하지만 rastA 의 경계선 또는 외부와 교차하지는 않을 경우 rastA 가 rastB 를 제대로 담고 있다고 합니다. 밴드 번호를 설정하지 않거나 또는 NULL 로 설정할 경우, 확인 작업중 래스터의 볼록 껍질만 고려합니다. 밴드 번호를 설정할 경우, 확인 작업중 값을 가진 (NODATA 가 아닌) 픽셀들만 고려합니다.

래스터 rastA 는 자기 자신을 제대로 담고 있지는 않지만, 자기 자신을 담고 있긴 합니다.



이 함수는 래스터에서 이용할 수도 있는 모든 인덱스를 활용할 것입니다.



래스터와 도형의 공간 관계성을 확인하려면, ST_ContainsProperly(ST_Polygon(raster), geometry) 또는 ST_ContainsProperly(geometry, ST_Polygon(raster))처럼 래스터에 ST_Polygon 함수를 이용하십시오.

2.1.0 버전부터 사용할 수 있습니다.

예시

```
  SELECT r1.rid, r2.rid, ST_ContainsProperly(r1.rast, 1, r2.rast, 1) FROM
dummy_rast r1 CROSS JOIN dummy_rast r2 WHERE r1.rid = 2;

  rid | rid | st_containsproperly
 -----+-----+---------------------
    2 |   1 | f
    2 |   2 | f
```

참고: ST_Intersects, ST_Contains

9.13.3. ST_Covers

ST_Covers — 래스터 rastB 의 어떤 포인트도 래스터 rastA 외부에 없을 경우 참을 반환합니다.

Synopsis

boolean **ST_CoveredBy**(raster *rastA* , integer *nbandA* , raster *rastB* , integer *nbandB*);

boolean **ST_CoveredBy**(raster *rastA* , raster *rastB*);

설명: 래스터 rastB 의 어떤 포인트도 래스터 rastA 외부에 없을 경우 rastA 가 rastB 를 커버한다고 합니다. 밴드 번호를 설정하지 않거나 또는 NULL 로 설정할 경우, 확인 작업중 래스터의 볼록 껍질만 고려합니다. 밴드 번호를 설정할 경우, 확인 작업중 값을 가진 (NODATA 가 아닌) 픽셀들만 고려합니다.



이 함수는 래스터에서 이용할 수도 있는 모든 인덱스를 활용할 것입니다.



래스터와 도형의 공간 관계성을 확인하려면, ST_Covers(ST_Polygon(raster), geometry) 또는 ST_Covers(geometry, ST_Polygon(raster))처럼 래스터에 ST_Polygon 함수를 이용하십시오.

2.1.0 버전부터 사용할 수 있습니다.

예시

```
  SELECT r1.rid, r2.rid, ST_Covers(r1.rast, 1, r2.rast, 1) FROM dummy_rast r1
CROSS JOIN dummy_rast r2 WHERE r1.rid = 2;

   rid | rid | st_covers
  -----+-----+-----------
     2 |   1 | f
     2 |   2 | t
```

참고: ST_Intersects, ST_CoveredBy

9.13.4. ST_Disjoint

ST_Disjoint — 래스터 rastA 와 래스터 rastB 가 공간적으로 교차하지 않을 경우 참을 반환합니다.

Synopsis

boolean **ST_Disjoint**(raster *rastA* , integer *nbandA* , raster *rastB* , integer *nbandB*);

boolean **ST_Disjoint**(raster *rastA* , raster *rastB*);

설명: 래스터 rastA 와 래스터 rastB 가 어떤 공간도 함께 공유하지 않을 경우 rastA 와 rastB 가 분리됐다고 합니다. 밴드 번호를 설정하지 않거나 또는 NULL 로 설정할 경우, 확인 작업중 래스터의 볼록 껍질만 고려합니다. 밴드 번호를 설정할 경우, 확인 작업중 값을 가진 (NODATA 가 아닌) 픽셀들만 고려합니다.



이 함수는 어떤 인덱스도 활용하지 않습니다.



래스터와 도형의 공간 관계성을 확인하려면, ST_Disjoint(ST_Polygon(raster), geometry)처럼 래스터에 ST_Polygon 함수를 이용하십시오.

2.1.0 버전부터 사용할 수 있습니다.

예시

```
  -- rid = 1 이 밴드를 가지고 있지 않기 때문에, 안내문(NOTICE)이 출력되고 st_disjoint 가
NULL 값을 가지게 됩니다.
  SELECT r1.rid, r2.rid, ST_Disjoint(r1.rast, 1, r2.rast, 1) FROM dummy_rast
r1 CROSS JOIN dummy_rast r2 WHERE r1.rid = 2;

  NOTICE:  The second raster provided has no bands
   rid | rid | st_disjoint
  -----+-----+-------------
     2 |   1 |
     2 |   2 | f

  -- 이번에는 밴드 번호를 설정하지 않음
  SELECT  r1.rid,  r2.rid,  ST_Disjoint(r1.rast,  r2.rast)  FROM  dummy_rast  r1
CROSS JOIN dummy_rast r2 WHERE r1.rid = 2;

   rid | rid | st_disjoint
  -----+-----+-------------
     2 |   1 | t
     2 |   2 | f
```

참고: ST_Intersects

9.13.5. ST_Intersects

ST_Intersects — 래스터 rastA 와 래스터 rastB 가 공간적으로 교차할 경우 참을 반환합니다.

Synopsis

boolean ST_Intersects(raster *rastA* , integer *nbandA* , raster *rastB* , integer *nbandB*);

boolean ST_Intersects(raster *rastA* , raster *rastB*);

boolean ST_Intersects(raster *rast* , integer *nband* , geometry *geommin*);

boolean ST_Intersects(raster *rast* , geometry *geommin* , integer *nband=NULL*);

boolean ST_Intersects(geometry *geommin* , raster *rast* , integer *nband=NULL*);

설명: 래스터 rastA 와 래스터 rastB 가 공간적으로 교차할 경우 참을 반환합니다. 밴드 번호를 설정하지 않거나 또는 NULL 로 설정할 경우, 확인 작업중 래스터의 볼록 껍질만 고려합니다. 밴드 번호를 설정할 경우, 확인 작업중 값을 가진 (NODATA 가 아닌) 픽셀들만 고려합니다.



이 함수는 래스터에서 이용할 수도 있는 모든 인덱스를 활용할 것입니다.

개선 사항: 2.0.0 버전부터 래스터/래스터 교차를 지원합니다.



변경 사항: 2.1.0 버전부터 ST_Intersects(geometry, raster)의 습성과 일치시키기 위해 ST_Intersects(raster, geometry) 변종의 습성을 바꿨습니다.

예시

```
-- 동일 래스터의 서로 다른 밴드들
SELECT ST_Intersects(rast, 2, rast, 3) FROM dummy_rast WHERE rid = 2;

 st_intersects
---------------
 t
```

참고: ST_Intersection, ST_Disjoint

9.13.6. ST_Overlaps

ST_Overlaps — 래스터 rastA 와 래스터 rastB 가 교차하지만 어느 한 쪽이 다른 한 쪽을 완전히 담고 있지는 않을 경우 참을 반환합니다.

Synopsis

boolean **ST_Overlaps**(raster *rastA* , integer *nbandA* , raster *rastB* , integer *nbandB*);

boolean **ST_Overlaps**(raster *rastA* , raster *rastB*);

설명: 래스터 rastA 와 래스터 rastB 가 공간적으로 중첩할 경우 참을 반환합니다. 다시 말해 rastA 와 rastB 가 교차하지만 어느 한 쪽이 다른 한 쪽을 완전히 담고 있지는 않다는 뜻입니다. 밴드 번호를 설정하지 않거나 또는 NULL 로 설정할 경우, 확인 작업중 래스터의 볼록 껍질만 고려합니다. 밴드 번호를 설정할 경우, 확인 작업중 값을 가진 (NODATA 가 아닌) 픽셀들만 고려합니다.



이 함수는 래스터에서 이용할 수도 있는 모든 인덱스를 활용할 것입니다.



래스터와 도형의 공간 관계성을 확인하려면, ST_Overlaps(ST_Polygon(raster), geometry)처럼 래스터에 ST_Polygon 함수를 이용하십시오.

2.1.0 버전부터 사용할 수 있습니다.

예시

```
-- 동일 래스터의 서로 다른 밴드들을 비교
SELECT ST_Overlaps(rast, 1, rast, 2) FROM dummy_rast WHERE rid = 2;

 st_overlaps
-------------
 f
```

참고: ST_Intersects

9.13.7. ST_Touches

ST_Touches — 래스터 rastA 와 래스터 rastB 가 최소한 포인트 한 개를 공유하지만, 내부가 교차하지는 않을 경우 TRUE 를 반환합니다.

Synopsis

boolean **ST_Touches**(raster *rastA* , integer *nbandA* , raster *rastB* , integer *nbandB*);

boolean **ST_Touches**(raster *rastA* , raster *rastB*);

설명: 래스터 rastA 와 래스터 rastB 가 공간적으로 접촉할 경우 참을 반환합니다. 다시 말해 rastA 와 래스터 rastB 가 최소한 포인트 한 개를 공유하지만, 내부가 교차하지는 않는다는 뜻입니다. 밴드 번호를 설정하지 않거나 또는 NULL 로 설정할 경우, 확인 작업중 래스터의 볼록 껍질만 고려합니다. 밴드 번호를 설정할 경우, 확인 작업중 값을 가진 (NODATA 가 아닌) 픽셀들만 고려합니다.



이 함수는 래스터에서 이용할 수도 있는 모든 인덱스를 활용할 것입니다.



래스터와 도형의 공간 관계성을 확인하려면, ST_Touches(ST_Polygon(raster), geometry)처럼 래스터에 ST_Polygon 함수를 이용하십시오.

2.1.0 버전부터 사용할 수 있습니다.

예시

```
  SELECT r1.rid, r2.rid, ST_Touches(r1.rast, 1, r2.rast, 1) FROM dummy_rast
r1 CROSS JOIN dummy_rast r2 WHERE r1.rid = 2;

   rid | rid | st_touches
  -----+-----+------------
     2 |   1 | f
     2 |   2 | f
```

참고: ST_Intersects

9.13.8. ST_SameAlignment

ST_SameAlignment — 래스터들이 동일한 기울기, 축척, 공간 참조 시스템, 그리고 오프셋을 가지고 있을 경우 (픽셀이 다른 픽셀을 침범하지 않고 동일한 그리드에 놓일 수 있을 경우) 참을 반환하고, 그렇지 않을 경우 문제점을 설명하는 안내문과 함께 거짓을 반환합니다.

Synopsis

boolean ST_SameAlignment(raster *rastA* , raster *rastB*);

boolean ST_SameAlignment(double precision *ulx1* , double precision *uly1* , double precision *scalex1* , double precision *scaley1* , double precision *skewx1* , double precision *skewy1* , double precision *ulx2* , double precision *uly2* , double precision *scalex2* , double precision *scaley2* , double precision *skewx2* , double precision *skewy2*);

boolean ST_SameAlignment(raster set *rastfield*);

설명: 종합 함수가 아닌 버전(변종 1, 2): (직접 입력하거나 또는 좌상단, 축척, 기울기, SRID 값을 이용해서) 두 래스터가 동일한 축척, 기울기, SRID를 가지며 한 래스터의 어떤 픽셀의 꼭짓점 4개 가운데 적어도 어느 한 꼭짓점이 다른 래스터의 그리드의 어느 모서리에 놓일 경우 참을 반환합니다. 그렇지 않을 경우 정렬 문제를 설명하는 안내문(NOTICE)과 함께 거짓을 반환합니다.

종합 함수 버전(변종 3): 래스터 집합에서, 집합 내부의 모든 래스터가 정렬돼 있을 경우 참을 반환합니다. ST_SameAlignment() 함수는 PostgreSQL 전문 용어로 "종합(aggregate)" 함수입니다. 즉 SUM() 및 AVG() 함수와 마찬가지로 데이터 행들을 대상으로 연산한다는 뜻입니다.

2.0.0 버전부터 사용할 수 있습니다.

개선 사항: 2.1.0 버전에서 종합 함수 변종이 추가됐습니다.

예시: 래스터

```
SELECT ST_SameAlignment(
      ST_MakeEmptyRaster(1, 1, 0, 0, 1, 1, 0, 0),
      ST_MakeEmptyRaster(1, 1, 0, 0, 1, 1, 0, 0)
) as sm;

sm
----
t
SELECT ST_SameAlignment(A.rast,b.rast)
 FROM dummy_rast AS A CROSS JOIN dummy_rast AS B;

 NOTICE:  The two rasters provided have different SRIDs
NOTICE:  The two rasters provided have different SRIDs
 st_samealignment
------------------
 t
 f
 f
 f
```

참고: Section 5.1, "래스터 로드 및 생성", ST_NotSameAlignmentReason, ST_MakeEmptyRaster

9.13.9. ST_NotSameAlignmentReason

ST_NotSameAlignmentReason — 래스터들이 정렬돼 있는지 아닌지, 그리고 정렬되지 않았다면 그 이유를 설명하는 텍스트를 반환합니다.

Synopsis

text **ST_NotSameAlignmentReason**(raster *rastA*, raster *rastB*);

설명: 래스터들이 정렬돼 있는지 아닌지, 그리고 정렬되지 않았다면 그 이유를 설명하는 텍스트를 반환합니다.



래스터들이 정렬되지 않은 이유가 여러 개 있을 경우, 한 가지 이유(실패한 첫 번째 확인 작업)만 반환할 것입니다.

2.1.0 버전부터 사용할 수 있습니다.

예시

```
SELECT
       ST_SameAlignment(
              ST_MakeEmptyRaster(1, 1, 0, 0, 1, 1, 0, 0),
              ST_MakeEmptyRaster(1, 1, 0, 0, 1.1, 1.1, 0, 0)
       ),
       ST_NotSameAlignmentReason(
              ST_MakeEmptyRaster(1, 1, 0, 0, 1, 1, 0, 0),
              ST_MakeEmptyRaster(1, 1, 0, 0, 1.1, 1.1, 0, 0)
       )
;

 st_samealignment |          st_notsamealignmentreason
------------------+-----------------------------------------------------
 f                | The rasters have different scales on the X axis
(1 row)
```

참고: Section 5.1, "래스터 로드 및 생성", ST_SameAlignment

9.13.10. ST_Within

ST_Within — 래스터 rastB 외부에 놓인 래스터 rastA 의 포인트가 하나도 없고, 적어도 rastA 내부의 포인트 하나가 rastB 의 내부에 있는 경우 참을 반환합니다.

Synopsis

boolean **ST_Within**(raster *rastA* , integer *nbandA* , raster *rastB* , integer *nbandB*);

boolean **ST_Within**(raster *rastA* , raster *rastB*);

설명: 래스터 rastB 외부에 놓인 래스터 rastA 의 포인트가 하나도 없고, 적어도 rastA 내부의 포인트 하나가 rastB 의 내부에 있을 경우에만 rastA 가 rastB 내부에 있다고 합니다. 밴드 번호를 설정하지 않거나 또는 NULL 로 설정할 경우, 확인 작업중 래스터의 볼록 껍질만 고려합니다. 밴드 번호를 설정할 경우, 확인 작업중 값을 가진 (NODATA 가 아닌) 픽셀들만 고려합니다.



이 피연산자(operand)는 래스터에서 이용할 수도 있는 모든 인덱스를 활용할 것입니다.



래스터와 도형의 공간 관계성을 확인하려면, ST_Within(ST_Polygon(raster), geometry) 또는 ST_Within(geometry, ST_Polygon(raster))처럼 래스터에 ST_Polygon 함수를 이용하십시오.



ST_Within()는 ST_Contains()의 역함수입니다. 따라서, ST_Within(rastA, rastB)의 결과와 ST_Contains(rastB, rastA)의 결과는 동일할 것입니다.

2.1.0 버전부터 사용할 수 있습니다.

예시

```
  SELECT r1.rid, r2.rid, ST_Within(r1.rast, 1, r2.rast, 1) FROM dummy_rast r1
CROSS JOIN dummy_rast r2 WHERE r1.rid = 2;

   rid | rid | st_within
  -----+-----+-----------
     2 |   1 | f
     2 |   2 | t
```

참고: ST_Intersects, ST_Contains, ST_DWithin, ST_DFullyWithin

9.13.11. ST_DWithin

ST_DWithin — 래스터 rastA 와 래스터 rastB 가 서로 설정된 거리 안에 있을 경우 참을 반환합니다.

Synopsis

boolean **ST_DWithin**(raster *rastA* , integer *nbandA* , raster *rastB* , integer *nbandB* , double precision *distance_of_srid*);

boolean **ST_DWithin**(raster *rastA* , raster *rastB* , double precision *distance_of_srid*);

설명: 래스터 rastA 와 래스터 rastB 가 서로 설정된 거리 안에 있을 경우 참을 반환합니다. 밴드 번호를 설정하지 않거나 또는 NULL 로 설정할 경우, 확인 작업중 래스터의 볼록 껍질만 고려합니다. 밴드 번호를 설정할 경우, 확인 작업중 값을 가진 (NODATA 가 아닌) 픽셀들만 고려합니다.

거리는 래스터들의 공간 참조 시스템이 정의한 단위로 설정됩니다. 이 함수가 제대로 동작하려면, 소스 래스터들이 둘 다 동일한 SRID 를 가진, 동일한 좌표 투영체여야 합니다.



이 피연산자(operand)는 래스터에서 이용할 수도 있는 모든 인덱스를 활용할 것입니다.



래스터와 도형의 공간 관계성을 확인하려면, ST_DWithin(ST_Polygon(raster), geometry)처럼 래스터에 ST_Polygon 함수를 이용하십시오.

2.1.0 버전부터 사용할 수 있습니다.

예시

```
  SELECT r1.rid, r2.rid, ST_DWithin(r1.rast, 1, r2.rast, 1, 3.14) FROM
dummy_rast r1 CROSS JOIN dummy_rast r2 WHERE r1.rid = 2;
```

```
 rid | rid | st_dwithin
-----+-----+------------
   2 |   1 | f
   2 |   2 | t
```

참고: ST_Within, ST_DFullyWithin

9.13.12. ST_DFullyWithin

ST_DFullyWithin — 래스터 rastA 와 래스터 rastB 가 완전히 서로 설정된 거리 안에 있을 경우 참을 반환합니다.

Synopsis

boolean **ST_DFullyWithin**(raster *rastA* , integer *nbandA* , raster *rastB* , integer *nbandB* , double precision *distance_of_srid*);

boolean **ST_DFullyWithin**(raster *rastA* , raster *rastB* , double precision *distance_of_srid*);

설명: 래스터 rastA 와 래스터 rastB 가 완전히 서로 설정된 거리 안에 있을 경우 참을 반환합니다. 밴드 번호를 설정하지 않거나 또는 NULL 로 설정할 경우, 확인 작업중 래스터의 볼록 껍질만 고려합니다. 밴드 번호를 설정할 경우, 확인 작업중 값을 가진 (NODATA 가 아닌) 픽셀들만 고려합니다.

거리는 래스터들의 공간 참조 시스템이 정의한 단위로 설정됩니다. 이 함수가 제대로 동작하려면, 소스 래스터들이 둘 다 동일한 SRID 를 가진, 동일한 좌표 투영체여야 합니다.



이 피연산자(operand)는 래스터에서 이용할 수도 있는 모든 인덱스를 활용할 것입니다.



래스터와 도형의 공간 관계성을 확인하려면, ST_DFullyWithin(ST_Polygon(raster), geometry)처럼 래스터에 ST_Polygon 함수를 이용하십시오..

2.1.0 버전부터 사용할 수 있습니다.

예시

```
 SELECT r1.rid, r2.rid, ST_DFullyWithin(r1.rast, 1, r2.rast, 1, 3.14) FROM
dummy_rast r1 CROSS JOIN dummy_rast r2 WHERE r1.rid = 2;

 rid | rid | st_dfullywithin
-----+-----+-----------------
   2 |   1 | f
   2 |   2 | t
```

참고: ST_Within, ST_DWithin

Chapter 10. PostGIS 래스터 FAQ

1. ST_FromGDALRaster 함수를 실행할 경우

ERROR: RASTER_fromGDALRaster: Could not open bytea with GDAL. Check that the bytea is of a GDAL supported format. 오류 메시지를 받거나, ST_AsPNG 또는 다른 래스터 입력 함수들을 실행하려 할 경우 ERROR: rt_raster_to_gdal: Could not load the output GDAL driver 오류 메시지를 받습니다.

PostGIS 2.1.3 및 2.0.5 버전에서, 기본적으로 모든 GDAL 드라이버 및 DB 외부 래스터를 비활성화하도록 보안 정책이 변경됐습니다. PostGIS 2.0.6, 2.1.3 security release 에서 릴리즈 노트를 볼 수 있습니다. 특정 드라이버 또는 모든 드라이버를 재활성화하거나 DB 외부 지원을 재활성화하려면, Section 2.1, "짧은 설명" 을 참조하십시오.

2. 어디에서 PostGIS 래스터 프로젝트에 대한 정보를 찾을 수 있습니까?

PostGIS 래스터 홈페이지 를 참조하십시오.

3. 이 멋진 발명품을 시작해볼 수 있는 책이나 교육 교재가 있을까요?

초급자용 정식 교육 교재 Intersecting vector buffers with large raster coverage using PostGIS Raster 가 있습니다. 호르헤 아레발로(Jorge Arévalo)가 자기 블로그에 래스터 데이터를 로드하는 방법과 함께, 해당 방법을 오라클 GeoRaster 에서의 동일한 작업과 교차 비교한 포스팅을 몇 개 올렸습니다. Jorge's PostGIS Raster / Oracle GeoRaster Series 를 확인해보십시오. PostGIS in Action - Raster chapter 처럼 무료 코드 및 다운로드 데이터를 포함해서 (35 페이지가 넘는 내용의) 한 장을 통째로 PostGIS 래스터에 집중한 교재도 있습니다. 매닝 출판사에서 나온 책으로(대량 구매시 할인율이 엄청납니다) 또는 E-Book 버전으로 PostGIS in Action 을 살 수도 있습니다. 해당 링크만이 아니라 아마존 및 다양한 서적유통 채널에서도 구입할 수 있습니다. 책을 구매하면 E-Book 버전을 다운로드할 수 있는 무료 쿠폰을 제공합니다.

PostGIS 래스터 사용자가 리뷰를 작성했습니다. PostGIS raster applied to land classification urban forestry

4. **어떻게 PostGIS 데이터베이스에 래스터 지원을 설치하나요?**

PostGIS 및 래스터용 바이너리를 다운로드하는 방법이 가장 쉽습니다. 현재 Mac OSX 의 최신 버전 및 윈도우용 바이너리를 다운로드할 수 있습니다. 먼저 PostgreSQL 8.4, 9.0, 또는 9.1 과 함께 PostGIS 2.0.0 이상 버전이 실행되고 있어야 합니다. PostGIS 2.0 버전이 PostGIS 래스터를 완전히 통합하고 있기 때문에, 사용자가 PostGIS 를 컴파일할 때 함께 컴파일될 것이라는 점에 주의하십시오.

How to Install and Configure PostGIS raster on windows 에서 윈도우 설치 및 실행에 대한 지침을 찾을 수 있습니다.

윈도우를 쓰고 있다면, 사용자가 직접 컴파일할 수도 있고, 또는 미리 컴파일된 PostGIS 래스터 윈도우 바이너리 를 이용할 수도 있습니다. Mac OSX 레퍼드(Leopard) 또는 스노 레퍼드(Snow Leopard)를 쓰고 있다면, Kyng Chaos Mac OSX PostgreSQL/GIS binaries 에서 바이너리를 다운로드할 수 있습니다.

그 다음 사용자 데이터베이스에서 래스터 지원을 활성화하려면, 사용자 데이터베이스에서 rtpostgis.sql 파일을 실행하십시오. 기존에 설치된 PostGIS 를 업그레이드할 경우 rtpostgis.sql 대신 rtpostgis_upgrade_minor.sql 파일을 실행하십시오.

다른 OS 플랫폼의 경우, 일반적으로 사용자가 직접 컴파일해야 합니다. PostGIS 및 GDAL 의존성을 가지고 있습니다. 소스로부터 컴파일하는 방법에 대한 자세한 내용은 Installing PostGIS Raster from source (in prior versions of PostGIS) 를 참조하십시오.

5. **could not load library "C:/Program Files/PostgreSQL/8.4/lib/rtpostgis.dll": The specified module could not be found 라는 오류가 발생합니다. 또는 리눅스에서 rtpostgis.sql 파일을 실행하려 하면 could not load library 오류가 발생합니다.**

rtpostgis.so/dll 은 libgdal.dll/so 에 대한 의존성을 가지고 빌드됩니다. 윈도우의 경우 사용자의 PostgreSQL 설치 폴더의 bin 폴더에 libgdal-1.dll 파일이 있는지 확인하십시오. 리눅스의 경우 사용자 경로 또는 bin 폴더에 libgdal 이 있어야 합니다.

사용자 데이터베이스에 PostGIS 를 설치하지 않았다면 서로 다른 오류들이 발생할 수도 있습니다. 래스터 지원을 설치하려 하기 전에 먼저 사용자 데이터베이스에 PostGIS 를 설치했는지를 확인하십시오.

6. 어떻게 PostGIS 로 래스터 데이터를 로드하나요?

PostGIS 최신 버전은 여러 종류의 래스터를 로드하고 또 추가 소프트웨어 없이 저해상도 미리보기를 생성할 수 있는 raster2pgsql 래스터 로더 실행 파일과 함께 패키징됩니다. 자세한 내용은 Section 5.1.1, "raster2pgsql 을 이용해 래스터를 로드하기" 를 참조하십시오. 2.0 미만 버전은 GDAL 및 NumPy 가 설치된 파이썬이 필요한 raster2pgsql.py 와 함께 패키징됩니다. 이 파이썬 버전은 더 이상 필요하지 않습니다.

7. 사용자 데이터베이스에 어떤 종류의 래스터 파일 형식을 로드할 수 있습니까?

사용자의 GDAL 라이브러리가 지원하는 모든 형식을 로드할 수 있습니다. GDAL 이 지원하는 형식은 GDAL File Formats 에 잘 정리돼 있습니다.

사용자가 설치한 특정 GDAL 이 모든 형식을 지원하지 않을 수도 있습니다. 사용자의 특정 GDAL 이 지원하는 형식들을 확인하려면, 다음 명령어를 실행하십시오:

```
raster2pgsql -G
```

8. 내 PostGIS 래스터 데이터를 다른 래스터 형식으로 내보내기할 수 있습니까?

네.

GDAL 1.7 이상 버전이 PostGIS 래스터 드라이버를 가지고 있지만, 사용자가 PostgreSQL 지원과 함께 컴파일하도록 선택할 경우에만 함께 컴파일됩니다.

사용자가 비정규적으로 블록화된 래스터를 PostGIS 래스터 데이터형으로 저장할 수 있음에도 불구하고, 해당 드라이버는 현재 비정규적으로 블록화된 래스터를 지원하지 않습니다.

사용자가 소스로부터 컴파일할 경우, 드라이버를 활성화하려면 사용자 설정에

```
  --with-pg=path/to/pg_config
```

를 포함시켜야 합니다. 여러 OS 플랫폼에 대해 GDAL 을 빌드하는 데 대한 방법을 알고 싶다면 GDAL Build Hints 를 참조하십시오.

사용자의 GDAL 버전을 PostGIS 래스터 드라이버와 함께 컴파일했다면 다음 명령어를 실행했을 때 목록에 PostGIS 래스터가 보여야 합니다:

```
  gdalinfo --formats
```

gdalinfo 를 이용해서 GDAL 을 통해 사용자 래스터에 대한 요약을 보려면 다음 명령어를 실행하십시오:

```
  gdalinfo   "PG:host=localhost port=5432 dbname='mygisdb' user='postgres'
password='whatever' schema='someschema' table=sometable"
```

데이터를 다른 래스터 형식으로 내보내기하려면, gdal_translate 를 이용하십시오. 다음 명령어는 테이블에서 모든 데이터를 10% 크기의 PNG 파일로 내보내기할 것입니다.

사용자의 픽셀 밴드 유형에 따라, 내보내기할 형식이 해당 픽셀 유형을 지원하지 않을 경우 gdal_translate 이 작동하지 않을 수도 있습니다. 예를 들어 부동소수점 밴드 유형 및 부호가 없는 32 비트 정수형은 JPG 또는 몇몇 다른 형식으로 쉽게 변환되지 않을 것입니다.

다음은 단순한 gdal_translate 작업의 예시입니다:

```
  gdal_translate  -of  PNG  -outsize  10%  10%  "PG:host=localhost  port=5432
dbname='mygisdb'  user='postgres'  password='whatever'  schema='someschema'
table=sometable" C:\somefile.png
```

사용자의 드라이버 연결 스트링에 WHERE=... 를 이용해서 내보내기 작업에 SQL WHERE 절을 사용할 수도 있습니다. 다음은 WHERE 절을 이용하는 몇몇 예시입니다:

```
  gdal_translate  -of  PNG  -outsize  10%  10%  "PG:host=localhost  port=5432
dbname='mygisdb'  user='postgres'  password='whatever'  schema='someschema'
table=sometable where='filename=\'abcd.sid\'\'" " C:\somefile.png
  gdal_translate  -of  PNG  -outsize  10%  10%  "PG:host=localhost  port=5432
dbname='mygisdb'  user='postgres'  password='whatever'  schema='someschema'
table=sometable       where='ST_Intersects(rast,       ST_SetSRID(ST_Point(-
71.032,42.3793),4326) )' " C:\intersectregion.png
```

더 많은 예시와 문법을 보려면 Reading Raster Data of PostGIS Raster section 을 참조하십시오.

9. 다운로드할 수 있는 GDAL 바이너리가 이미 PostGIS 래스터 지원과 함께 컴파일돼 있습니까?

네. GDAL Binaries 페이지를 확인해보십시오. PostgreSQL 지원과 함께 컴파일된 모든 바이너리는 PostGIS 래스터도 함께 지원할 것입니다.

PostGIS 래스터는 많은 변화를 겪고 있는 중입니다. 윈도우용 최신 "Nightly" 빌드를 다운로드하고 싶다면, GDAL 트렁크, 파이썬 바인딩 및 MapServer 실행 파일과 PostGIS 래스터 드라이버를 내장한 상태로 Visual Studio 를 통해 빌드된 터마스 세케레시(Tamás Szekeres)의 Nightly 빌드를 확인해보십시오. 그냥 SDK 배치 파일(.bat)을 클릭한 다음 사용자 명령어를 실행하십시오. http://www.gisinternals.com 에서 다운로드할 수 있으며, 또 VS 프로젝트 파일도 다운로드할 수 있습니다.

윈도우용 FWTools 의 최신 안정 버전은 래스터 지원과 함께 컴파일됩니다.

10. 어떤 도구를 사용해서 PostGIS 래스터 데이터를 표출할 수 있습니까?

GDAL 1.7 이상 버전과 PostGIS 래스터 드라이버 지원과 함께 컴파일된 MapServer 를 이용해서 래스터 데이터를 표출할 수 있습니다. PostGIS 래스터 드라이버를 설치했다면 QuantumGIS(QGIS)도 이제 PostGIS 래스터 표출을 지원합니다.

이론적으로 GDAL 을 이용해서 데이터를 렌더링하는 어떤 도구든 PostGIS 래스터 데이터를 지원하거나, 또는 최소한의 노력으로 지원할 수 있습니다. 다시 말하지만 윈도우의 경우, 사용자가 직접 컴파일해서 구성하는 수고를 들이고 싶지 않다면 터마스 세케레시의 http://www.gisinternals.com 바이너리를 선택하는 편이 좋습니다.

11. 어떻게 내 MapServer 맵에 PostGIS 래스터 레이어를 추가할 수 있습니까?

먼저 PostGIS 래스터 지원과 함께 컴파일된 GDAL 1.7 이상 버전이 필요합니다. GDAL 1.8 버전에서 많은 문제점들이 해결됐기 때문에 1.8 이상 버전을 쓰는 편이 좋습니다. 트렁크 버전에서는 더 많은 PostGIS 래스터 문제점들이 해결됐습니다.

어떤 다른 래스터와 마찬가지로 추가할 수 있습니다. MapServer 래스터 레이어와 함께 이용할 수 있는 여러 공간 처리 함수들의 목록을 보려면 MapServer Raster processing options 를 참조하십시오.

PostGIS 래스터 데이터를 특히 흥미롭게 만드는 점은, 각 타일이 다양한 표준 데이터베이스 열들을 가질 수 있기 때문에 사용자 데이터소스에서 타일을 부분으로 나눌 수 있다는 점입니다.

다음은 MapServer 에서 PostGIS 래스터 레이어를 어떻게 정의하는가에 대한 예시입니다.



타일화된 래스터의 경우 mode=2 설정이 필요한데, 이 설정은 PostGIS 2.0 및 GDAL 1.8 드라이버에 추가됐습니다. 이 설정은 GDAL 1.7 드라이버에는 존재하지 않습니다.

```
       -- 래스터를 표준 래스터 옵션으로 표출하기
  LAYER
        NAME coolwktraster
        TYPE raster
        STATUS ON
        DATA "PG:host=localhost port=5432 dbname='somedb' user='someuser'
password='whatever'
              schema='someschema' table='cooltable' mode='2'"
        PROCESSING "NODATA=0"
        PROCESSING "SCALE=AUTO"
        #... 다른 표준 래스터 공간 처리 함수를 추가
```

```
        #... 클래스는 선택적이지만 단일 밴드 데이터에 유용함
        CLASS
              NAME "boring"
              EXPRESSION ([pixel] < 20)
              COLOR 250 250 250
        END
        CLASS
              NAME "mildly interesting"
              EXPRESSION ([pixel] > 20 AND [pixel] < 1000)
              COLOR 255 0 0
        END
        CLASS
              NAME "very interesting"
              EXPRESSION ([pixel] >= 1000)
              COLOR 0 255 0
        END
  END

  -- 래스터를 표준 래스터 옵션 및 WHERE 절로 표출하기
  LAYER
        NAME soil_survey2009
        TYPE raster
        STATUS ON
        DATA "PG:host=localhost port=5432 dbname='somedb' user='someuser'
password='whatever'
              schema='someschema'                          table='cooltable'
where='survey_year=2009' mode='2'"
        PROCESSING "NODATA=0"
        #... other standard raster processing functions here
        #... classes are optional but useful for 1 band data
  END
```

토폴로지

PostGIS 토폴로지 타입들과 함수들은 면(Face), 에지(Edge), 노드(Node)와 같은 오브젝트들을 관리하기 위해 사용됩니다.

파리에서 열린 2011 년 PostGIS day 회의에서의 산드로 털리의 프리젠테이션인 Topology with PostGIS 2.0 slide deck 는 바람직한 PostGIS 토폴로지에 대한 개요 및 방향성에 대한 좋은 정보를 제공합니다.

Vincent Picave 은 State of the art of FOSS4G for topology and network analysis 에서 토폴로지란 무엇인지, 이를 어떻게 이용하는지, 이를 지원하는 다양한 FOSS4G 툴에 대한 좋은 개요와 개략적인 설명을 제공합니다.

토폴로지적인 GIS 데이터베이스의 한 예는 US Census Topologically Integrated Geographic Encoding and Reference System (TIGER) 데이터베이스 입니다. 만약 사용자가 PostGIS 토폴로지를 실험하기 위해 데이터가 좀 필요하신 경우에는 Topology_Load_Tiger 를 확인해 보십시오.

PostGIS 토폴로지 모듈은 PostGIS 의 이전 버전에도 존재했지만, PostGIS 공식문서의 일부였던 적은 없습니다. PostGIS 2.0.0 에서의 주로 정리되고 있는 부분들은, 더 이상 불필요한 모든 기능의 사용 제거, 알려진 사용성 이슈 수정, 기능과 함수들에 대한 더 나은 문서화, 새로운 기능 추가, SQL-MM 표준 준수 향상에 대한 것입니다.

이 프로젝트에 관한 보다 자세한 사항은 PostGIS Topology Wiki 에서 찾으실 수 있습니다.

이 모듈과 관련된 모든 기능과 테이블은 `topology` 라 불리는 스키마에 설치됩니다.

SQL/MM 표준에서 정의 된 함수들 앞에는 ST_라는 접두사가 붙어있고 PostGIS 에 특화된 기능들은 접두사가 붙어 있지 않습니다.

토폴로지가 지원되도록 PostGIS 2.0 를 빌드하기 위해서는 2 장에서 설명된 것처럼 `--with-topology` 옵션과 함께 컴파일 하십시오. 일부 함수들은 GEOS 3.3+에 의존성이 있습니다. 그러므로 토폴로지 지원을 완전히 활용하기 위해서는 GEOS 3.3+와 함께 컴파일 하셔야 합니다.

12. 현재 어떤 함수를 내 래스터 데이터와 함께 사용할 수 있습니까?

Chapter 9, 래스터 참조 문서 의 목록을 참조하십시오. 함수가 더 많이 있지만, 목록화를 계속 진행중입니다.

향후 어떤 함수를 기대할 수 있는지 알고 싶다면 PostGIS 래스터 로드맵 페이지를 참조하십시오.

13. ERROR: function st_intersects(raster, unknown) is not unique or st_union(geometry,text) is not unique 라는 오류가 발생합니다. 어떻게 해결해야 합니까?

사용자의 인수 가운데 하나가 도형이 아니라 도형의 문자형 표현식일 경우 'function is not unique' 오류가 발생합니다. 이런 경우 PostgreSQL 은 문자형 표현식을 알려지지 않은 유형으로 표시하는데, 즉 알려지지 않은 유형이 st_intersects(raster, geometry) 또는 st_intersects(raster,raster) 함수에 들어가 유일하지 않은(non-unique) 상태가 된다는 뜻입니다. 왜냐하면 두 함수 모두 사용자의 요청을 이론적으로는 지원할 수 있기 때문입니다. 이 오류를 방지하려면, 표현식을 도형으로 형변환해야 합니다.

예를 들어 사용자 코드가 다음처럼 보일 경우:

```
SELECT rast
 FROM my_raster
  WHERE ST_Intersects(rast, 'SRID=4326;POINT(-10 10)');
```

사용자 코드를 다음과 같이 변경해서 도형의 문자형 표현식을 도형으로 형변환하십시오:

```
SELECT rast
 FROM my_raster
  WHERE ST_Intersects(rast, 'SRID=4326;POINT(-10 10)'::geometry);
```

14. PostGIS 래스터와 오라클 GeoRaster 의 SDO_GEORASTER 및 SDO_RASTER 유형은 어떻게 다릅니까?

이 주제에 대해 더 광범위한 논의를 보고 싶다면, 호르헤 아레발로(Jorge Arévalo)의 Oracle GeoRaster and PostGIS Raster: First impressions 블로그 포스팅을 참조하십시오.

레이어 당 1 개의 지리참조(one-georeference-by-layer)에 비해 래스터 당 1 개의 지리참조(one-georeference-by-raster)가 지니는 주요한 장점은 다음과 같은 처리 방식이 가능하다는 것입니다:

* 커버리지가 반드시 사각형이 아니어도 됩니다. (넓은 범위를 커버하는 래스터 커버리지는 사각형이 아닌 경우가 많습니다. 래스터를 처리할 수 있는 방식에 대해서 문서를 찾아보십시오.)

* 래스터를 중첩시킬 수 있습니다. (벡터를 손실 없이 래스터로 변환시키는 데 필요합니다.)

오라클에서도 이런 처리 방식을 쓸 수 있지만, 같은 개수의 SDO_RASTER 테이블에 링크된 복수의 SDO_GEORASTER 객체를 저장한다는 뜻도 됩니다. 복잡 변환 작업으로 데이터베이스에 수백 개의 테이블을

생성할 수도 있습니다. PostGIS 래스터를 사용하면, 비슷한 래스터 처리 방식을 단일 테이블에 저장할 수 있습니다.

오라클의 래스터 유형은 PostGIS 가 틈(gap)이나 중첩이 없는 완전한 사각형 벡터 커버리지(위상기하학적으로 완벽한 사각형 레이어)만 저장하도록 강제하는 것과 비슷합니다. 몇몇 실제 사례에서 이는 매우 실용적이지만, 대부분의 지리학적 커버리지의 경우 그다지 현실적이거나 바람직하다고 볼 수 없습니다. 불연속적이고 사각형이 아닌 커버리지를 저장하려면 벡터 구조에 유연성이 필요합니다. 래스터 구조 또한 이런 효용을 받는 것이 대단한 장점이라고 생각합니다.

15. raster2pgsql 이 대용량 파일을 로드할 때 String of N bytes is too long for encoding conversion 이라는 오류를 내는데요?

raster2pgsql 은 로드할 파일을 생성할 때 사용자 데이터베이스와 어떤 연결도 하지 않습니다. 사용자 데이터베이스가 사용자 데이터베이스의 인코딩과 명백히 다른 클라이언트 인코딩을 설정한 경우, 대용량 (30MB 이상) 래스터 파일을 로드할 때 bytes is too long for encoding conversion 오류가 발생할 수도 있습니다.

예를 들어 사용자 데이터베이스의 인코딩이 UTF8 인데 윈도우 응용 프로그램을 지원하기 위해 클라이언트 인코딩을 WIN1252 으로 설정한 경우, 일반적으로 이 오류가 발생합니다.

이 오류를 피하려면 로드 작업중 사용자 데이터베이스의 인코딩과 클라이언트 인코딩이 동일한지 확인하십시오. 사용자의 로드 스크립트에 확실히 설정해두면 됩니다. 다음은 윈도우 예시입니다:

```
set PGCLIENTENCODING=UTF8
```

유닉스/리눅스의 경우:

```
export PGCLIENTENCODING=UTF8
```

이 문제점을 http://trac.osgeo.org/postgis/ticket/2209 에서 자세히 설명하고 있습니다.

Chapter 11. 지형(topology)

PostGIS 지형 유형 및 함수는 표면(face), 가장자리(edge), 노드(node)와 같은 위상기하학적 객체를 관리했습니다.

산드로 산틸리(Sandro Santilli)가 2011 년 파리에서 열린 PostGIS Day 컨퍼런스에서 발표한 Topology with PostGIS 2.0 slide deck 이 PostGIS 지형 유형에 대해 잘 요약하고 있으며, 향후 발전 방향에 대해서도 언급하고 있습니다.

뱅상 피카브(Vincent Picavet)의 PostGIS Topology PGConf EU 2012 는 지형 유형이란 무엇인지, 어떻게 사용하는지, 그리고 지형 유형을 지원하는 다양한 FOSS4G 도구들에 대해 잘 요약해서 보여주고 있습니다.

지형에 기반한 GIS 데이터베이스의 예로는 US Census Topologically Integrated Geographic Encoding and Referencing System (TIGER) 데이터베이스가 있습니다. PostGIS 지형 유형을 테스트해보고 싶은데 데이터가 필요하다면, Topology_Load_Tiger 를 확인해보십시오.

PostGIS 지형 모듈은 PostGIS 이전 버전에도 있었지만, 공식 PostGIS 문서에 포함된 적은 없었습니다. PostGIS 2.0.0 버전에서, 지원이 중단된 모든 함수를 제거하고, 알려진 사용성 문제들을 해결하고, 기능 및 함수들을 새로이 문서화하며, 새로운 함수들을 추가하고, SQL-MM 표준을 더 잘 준수하도록 개선하는 등 주요한 정리 작업이 이루어졌습니다.

이 프로젝트에 대한 자세한 내용을 PostGIS Topology Wiki 에서 찾아볼 수 있습니다.

이 모듈과 관련된 모든 함수 및 테이블이 topology 라는 스키마에 설치돼 있습니다.

SQL/MM 표준이 정의하는 함수들은 접두사 ST_ 가 붙은 명칭을 가지고 있으며, PostGIS 에 특화된 함수들의 명칭에는 접두사가 붙지 않습니다.

지형 지원과 함께 PostGIS 2.0 을 빌드하려면, Chapter 2, PostGIS 설치 에서 설명한 대로 --with-topology 옵션과 함께 컴파일하십시오. 일부 함수는 GEOS 3.3 이상 버전에 의존하므로, 지형 지원을 완전하게 활용하려면 GEOS 3.3 이상 버전과 함께 컴파일해야 합니다.

11.1. 지형 유형

이 단원에서 PostGIS 지형이 설치한 PostgreSQL 데이터 유형을 소개합니다. 사용자 지정 함수를 설계할 때 특히 중요한 이 유형들의 형변환 습성(cast behavior)을 설명한다는 점에 주의하십시오.

getfaceedges_returntype — 배열(sequence) 번호와 경계선(edge) 번호로 이루어진 합성 유형입니다. ST_GetFaceEdges 함수가 이 유형을 반환합니다.

TopoGeometry — 위상기하학적으로 정의된 도형을 표현하는 합성 유형입니다.

validatetopology_returntype — 오류 메시지 및 오류의 위치를 나타내는 id1 과 id2 로 이루어진 합성 유형입니다. ValidateTopology 함수가 이 유형을 반환합니다.

11.1.1. getfaceedges_returntype

getfaceedges_returntype — 배열(sequence) 번호와 경계선(edge) 번호로 이루어진 합성 유형입니다. ST_GetFaceEdges 함수가 이 유형을 반환합니다.

설명: 순차(sequence) 번호와 경계선(edge) 번호로 이루어진 합성 유형입니다. ST_GetFaceEdges 함수가 이 유형을 반환합니다.

1. sequence 는 정수형입니다: 지형 스키마 및 SRID 를 정의하는 topology.topology 테이블에 정의돼 있는 지형을 참조하십시오.

2. edge 는 정수형입니다: 경계선의 식별자입니다.

11.1.2. TopoGeometry

TopoGeometry — 위상기하학적으로 정의된 도형을 표현하는 합성 유형입니다.

설명: 특정 지형 레이어에 있는, 특정 유형 및 특정 ID 를 가진 지형 도형을 참조하는 합성 유형입니다. TopoGeometry 를 구성하는 요소는 topology_id, layer_id, id, type 정수형 속성입니다.

1. topology_id 는 정수형입니다: 지형 스키마 및 SRID 를 정의하는 topology.topology 테이블에 정의돼 있는 지형을 참조하십시오.

2. layer_id 는 정수형입니다: TopoGeometry 가 속해 있는 레이어 테이블의 layer_id 입니다. topology_id 와 layer_id 를 조합하면 topology.layers 테이블에 유일한 참조(unique reference)를 제공합니다.

3. id 는 정수형입니다: 해당 지형 레이어에 지형 도형을 유일하게 정의하는, 자동 생성된 순차 번호입니다.

4. 1 에서 4 까지의 type 정수가 도형 유형을 정의합니다. 1: [멀티]포인트, 2: [멀티]라인, 3: [멀티]폴리곤, 4: 도형 집합.

형변환 습성

이 단원에서 이런 데이터 유형에 사용할 수 있는 자동 형변환은 물론 명시적인 형변환도 소개합니다.

형변환 결과	습성
도형	자동적

참고: CreateTopoGeom

11.1.3. Validatetopology_returntype

validatetopology_returntype — 오류 메시지 및 오류의 위치를 나타내는 id1 과 id2 로 이루어진 합성 유형입니다. ValidateTopology 함수가 이 유형을 반환합니다.

설명: 오류 메시지 및 정수형 2 개로 이루어진 합성 유형입니다. ValidateTopology 함수가 확인 오류 및 해당 오류에 연관된 지형 객체들의 ID 를 표시하는 id1 과 id2 를 나타내는 이 집합을 반환합니다.

1. error 는 가변 길이 문자열(varchar)입니다: 오류의 유형을 표시합니다.

   현재 오류 설명자(descriptor)는 다음과 같습니다: coincident nodes(일치하는 노드들), edge crosses node(노드를 가로지르는 경계선), edge not simple(단순하지 않은 경계선), edge end node geometry mismatch(경계선의 종단 노드 도형 불일치), edge start node geometry mismatch(경계선의 시작 노드 도형 불일치), face overlaps face(표면 중첩 표면), face within face(표면 내부 표면)

2. id1 은 정수형입니다: 오류가 발생한 경계선(edge)/표면(face)/노드(node)의 식별자를 나타냅니다.

3. id2 는 정수형입니다: 객체 2 개와 연관된 오류일 경우, 두 번째 경계선 또는 노드를 표시합니다.

참고: ValidateTopology

11.2. 지형 도메인

이 단원에서 PostGIS 지형이 설치한 PostgreSQL 도메인을 소개합니다. 함수가 반환하는 유형 또는 테이블 열 등, 도메인을 객체 유형처럼 이용할 수 있습니다. 도메인과 유형의 차이점은, 도메인이 확인 제약조건으로 묶여 있는 기존 유형이라는 점입니다.

TopoElement — 일반적으로 TopoGeometry 구성 요소를 식별하는 데 쓰이는 정수형 2 개의 배열입니다.

TopoElementArray — TopoElement 객체들의 배열입니다.

11.2.1. TopoElement

TopoElement — 일반적으로 TopoGeometry 구성 요소를 식별하는 데 쓰이는 정수형 2 개의 배열입니다.

설명

단순 또는 계층 TopoGeometry 의 구성 요소 1 개를 표현하는 데 쓰이는 정수형 2 개의 배열입니다.

단순 TopoGeometry 의 경우, 배열의 첫 번째 요소가 지형적 원시형(topological primitive)의 식별자를 나타내고 두 번째 요소가 그 유형(1: node, 2: edge, 3: face)을 나타냅니다. 계층 TopoGeometry 의 경우 배열의 첫 번째 요소가 자식 TopoGeometry 의 식별자를 나타내고 두 번째 요소가 해당 레이어 식별자를 나타냅니다.



어떤 계층 TopoGeometry 가 주어지더라도, 모든 자식 TopoGeometry 요소들은, 해당 TopoGeometry 의 레이어에 대해 topology.layer 레코드에 지정된 동일한 자식 레이어에서 나옵니다.

예시

```
SELECT te[1] AS id, te[2] AS type FROM
( SELECT ARRAY[1,2]::topology.topoelement AS te ) f;
 id | type
----+------
  1 |    2

SELECT ARRAY[1,2]::topology.topoelement;
  te
-------
 {1,2}

-- topoelement 에 요소 3 개를 가진 배열을 넣으면 어떻게 되는지 보여주는 예시
-- 주의: topoelement 는 요소 2 개를 가진 배열이어야 하므로 차원 확인에 실패합니다.
SELECT ARRAY[1,2,3]::topology.topoelement;
ERROR:  value for domain topology.topoelement violates check constraint
"dimensions"
```

참고: GetTopoGeomElements, TopoElementArray, TopoGeometry, TopoGeom_addElement, TopoGeom_remElement

11.2.2. TopoElementArray

TopoElementArray — TopoElement 객체들의 배열입니다.

설명: 1 개 이상의 TopoElement 객체들의 배열로, 일반적으로 TopoElement 객체의 구성 요소들을 넘겨주는 데 쓰입니다.

예시

```
SELECT '{{1,2},{4,3}}'::topology.topoelementarray As tea;
  tea
-------
{{1,2},{4,3}}

-- 더 설명적인 동일한 쿼리 --
SELECT ARRAY[ARRAY[1,2], ARRAY[4,3]]::topology.topoelementarray As tea;

  tea
-------
{{1,2},{4,3}}

-- 지형과 함께 패키징된 배열 종합 함수를 이용 --
SELECT topology.TopoElementArray_Agg(ARRAY[e,t]) As tea
  FROM generate_series(1,4) As e CROSS JOIN generate_series(1,3) As t;
  tea
--------------------------------------------------------------------------------
{{1,1},{1,2},{1,3},{2,1},{2,2},{2,3},{3,1},{3,2},{3,3},{4,1},{4,2},{4,3}}
```

```
  SELECT '{{1,2,4},{3,4,5}}'::topology.topoelementarray As tea;
  ERROR:  value for domain topology.topoelementarray violates check constraint
"dimensions"
```

참고: TopoElement, GetTopoGeomElementArray, TopoElementArray_Agg

11.3. 지형 및 TopoGeometry 관리

이 단원에서 새로운 지형 스키마를 빌드하고, 지형이 유효한지 확인하고, TopoGeometry 열을 관리하기 위한 지형 함수들을 소개합니다.

AddTopoGeometryColumn — 기존 테이블에 TopoGeometry 열을 추가하고, topology.layer 에 이 새 열을 레이어로 추가하며, 새 layer_id 를 반환합니다.

DropTopology — 이용에 주의하십시오: 지형 스키마를 삭제하고 topology.topology 테이블에서 해당 참조를, 그리고 geometry_columns 테이블에서 해당 스키마에 있는 테이블의 참조를 삭제합니다.

DropTopoGeometryColumn — schema_name 스키마 안의 table_name 명칭의 테이블에서 Topogeometry 열을 삭제하고 topology.layer 테이블에서 해당 열을 등록 해제합니다.

TopologySummary — 지형 명칭을 받아 지형 내부에 있는 객체들의 유형 전체의 요약문을 반환합니다.

ValidateTopology — 지형이 가지고 있는 문제점들을 상세하게 열거하는 validatetopology_returntype 객체 집합을 반환합니다.

11.3.1. AddTopoGeometryColumn

AddTopoGeometryColumn — 기존 테이블에 TopoGeometry 열을 추가하고, topology.layer 에 이 새 열을 레이어로 추가하며, 새 layer_id 를 반환합니다.

Synopsis

integer AddTopoGeometryColumn(varchar *topology_name*, varchar *schema_name*, varchar *table_name*, varchar *column_name*, varchar *feature_type*);

integer AddTopoGeometryColumn(varchar *topology_name*, varchar *schema_name*, varchar *table_name*, varchar *column_name*, varchar *feature_type*, integer *child_layer*);

설명: 각 TopoGeometry 객체는 특정 지형의 특정 레이어에 속합니다. TopoGeometry 객체를 생성하기 전에 해당 지형 레이어를 생성해야 합니다. 지형 레이어란 지형과 피처 테이블의 조합입니다. 유형 및 계층 정보도 담고 있습니다. AddTopoGeometryColumn() 함수를 이용해서 레이어를 생성해보겠습니다:

이 함수는 요청한 열을 테이블에 추가하고 topology.layer 테이블에 레코드를 주어진 모든 정보와 함께 추가할 것입니다.

[child_layer]를 따로 설정하지 않을 (또는 NULL 로 설정할) 경우, 해당 레이어가 (원시형 지형 요소들로 이루어진) 기본 TopoGeometry 를 담을 것입니다. 그렇지 않을 경우, 이 레이어는 (child_layer 에서 나온 TopoGeometry 들로 이루어진) 계층 TopoGeometry 를 담을 것입니다.

레이어를 생성했다면 (AddTopoGeometryColumn 함수가 레이어의 ID 를 반환합니다) 해당 레이어에 TopoGeometry 객체를 작성할 준비가 된 것입니다.

유효한 feature_type 은 POINT, LINE, POLYGON, COLLECTION 입니다.

1.? 버전부터 사용할 수 있습니다.

예시

```
-- 이 예시를 위해 ma_topo 스키마에 새 테이블을 생성했다는 점에 주의하십시오.
-- 다른 스키마에 생성할 수 있었는데도 말입니다. 그랬더라면 topology_name 과 schema_name 이 달라졌을 겁니다.
CREATE SCHEMA ma;
CREATE TABLE ma.parcels(gid serial, parcel_id varchar(20) PRIMARY KEY, address text);
SELECT topology.AddTopoGeometryColumn('ma_topo', 'ma', 'parcels', 'topo', 'POLYGON');
CREATE SCHEMA ri;
CREATE TABLE ri.roads(gid serial PRIMARY KEY, road_name text);
SELECT topology.AddTopoGeometryColumn('ri_topo', 'ri', 'roads', 'topo', 'LINE');
```

참고: CreateTopology, CreateTopoGeom

11.3.2. DropToPology

DropTopology — 이용에 주의하십시오: 지형 스키마를 삭제하고 topology.topology 테이블에서 해당 참조를, 그리고 geometry_columns 테이블에서 해당 스키마에 있는 테이블의 참조를 삭제합니다.

Synopsis

integer DropTopology(varchar *topology_schema_name*);

설명: 지형 스키마를 삭제하고 topology.topology 테이블에서 해당 참조를, 그리고 geometry_columns 테이블에서 해당 스키마에 있는 테이블의 참조를 삭제합니다. 이 함수는 사용자의 관심 데이터를 파괴할 수도 있으니 주의해서 사용해야 합니다. 스키마가 존재하지 않을 경우, 해당 스키마의 명칭으로 된 참조 항목만 제거합니다.

1.? 버전부터 사용할 수 있습니다.

예시

ma_topo 스키마를 삭제하고 topology.topology 와 geometry_columns 에 있는 모든 해당 스키마의 참조를 차례로 제거합니다.

```
SELECT topology.DropTopology('ma_topo');
```

11.3.3. DropTopoGeometryColumn

DropTopoGeometryColumn — schema_name 스키마 안의 table_name 명칭의 테이블에서 Topogeometry 열을 삭제하고 topology.layer 테이블에서 해당 열을 등록 해제합니다.

Synopsis

text DropTopoGeometryColumn(varchar *schema_name*, varchar *table_name*, varchar *column_name*);

설명: schema_name 스키마 안의 table_name 명칭의 테이블에서 Topogeometry 열을 삭제하고 topology.layer 테이블에서 해당 열을 등록 해제합니다. 삭제 상태의 요약문을 반환합니다. 주의: 참조 무결성 확인 작업을 건너뛰기 위해 삭제하기 전에 먼저 모든 값을 NULL 로 설정합니다.

1.? 버전부터 사용할 수 있습니다.

예시

```
SELECT topology.DropTopoGeometryColumn('ma_topo', 'parcel_topo', 'topo');
```

참고: AddTopoGeometryColumn

11.3.4. TopologySummary

TopologySummary — 지형 명칭을 받아 지형 내부에 있는 객체들의 유형 전체의 요약문을 반환합니다.

Synopsis

text TopologySummary(varchar *topology_schema_name*);

설명: 지형 명칭을 받아 지형 내부에 있는 객체들의 유형 전체의 요약문을 반환합니다.

2.0.0 버전부터 사용할 수 있습니다.

예시

```
SELECT topology.topologysummary('city_data');
               topologysummary
-----------------------------------------------------------
 Topology city_data (329), SRID 4326, precision: 0
 22 nodes, 24 edges, 10 faces, 29 topogeoms in 5 layers
 Layer 1, type Polygonal (3), 9 topogeoms
  Deploy: features.land_parcels.feature
 Layer 2, type Puntal (1), 8 topogeoms
  Deploy: features.traffic_signs.feature
 Layer 3, type Lineal (2), 8 topogeoms
  Deploy: features.city_streets.feature
 Layer 4, type Polygonal (3), 3 topogeoms
  Hierarchy level 1, child layer 1
  Deploy: features.big_parcels.feature
 Layer 5, type Puntal (1), 1 topogeoms
  Hierarchy level 1, child layer 2
  Deploy: features.big_signs.feature
```

참고: Topology_Load_Tiger

11.3.5. ValidateTopology

ValidateTopology — 지형이 가지고 있는 문제점들을 상세하게 열거하는 validatetopology_returntype 객체 집합을 반환합니다.

Synopsis

setof validatetopology_returntype ValidateTopology(varchar *topology_schema_name*);

설명: 지형이 가지고 있는 문제점들을 상세하게 열거하는 validatetopology_returntype 객체 집합을 반환합니다. 다음 표는 가능한 오류 목록 및 반환된 ID 가 어떤 의미를 가지는지 보여줍니다.

오류	***id1***	***id2***
edge crosses node(노드를 가로지르는 경계선)	edge_id	node_id
invalid edge(유효하지 않은 경계선)	edge_id	null
edge not simple(단순하지 않은 경계선)	edge_id	null
edge crosses edge(경계선을 가로지르는 경계선)	edge_id	edge_id
edge start node geometry mismatch(경계선의 시작 노드 도형 불일치)	edge_id	node_id
edge end node geometry mis-match(경계선의 종단 노드 도형 불일치)	edge_id	node_id
face without edges(경계선이 없는 표면)	face_id	null
face has no rings(고리가 없는 표면)	face_id	null
face overlaps face(표면 중첩 표면)	face_id	face_id
face within face(표면 내부 표면)	inner face_id	outer face_id

1.0.0 버전부터 사용할 수 있습니다.

개선 사항: 2.0.0 버전에서 경계선 가로지르기 탐지를 더 효율적으로 개선하고, 이전 버전에 존재했던 긍정 오류(false positive)를 해결했습니다.

변경 사항: 2.2.0 버전에서 'edge crosses node'를 오류 설명문과 일치시키기 위해 id1 과 id2 의 값을 서로 바꿨습니다.

예시

```
SELECT * FROM  topology.ValidateTopology('ma_topo');
     error       | id1 | id2
-------------------+-----+-----
face without edges |   0 |
```

참고: validatetopology_returntype, Topology_Load_Tiger

11.4. 지형 작성자

이 단원에서 새 지형을 생성하기 위한 지형 함수에 대해 설명합니다.

CreateTopology — 새 지형 스키마를 생성하고 topology.topology 테이블에 이 새 스키마를 등록합니다.

CopyTopology — 지형 구조(노드, 경계선, 표면, 레이어 및 TopoGeometry)를 복사합니다.

ST_InitTopoGeo — 새 지형 스키마를 생성하고 topology.topology 테이블에 이 새 스키마를 등록한 다음 처리 과정의 상세한 요약문을 반환합니다.

ST_CreateTopoGeo — 텅 빈 입력 지형에 도형들의 집합을 추가한 다음 작업 성공 여부를 상세하게 설명하는 메시지를 반환합니다.

TopoGeo_AddPoint — 허용 오차를 이용해서 그리고 기존 경계선을 분할(split)하기도 해서 기존 지형에 포인트를 추가합니다.

TopoGeo_AddLineString — 허용 오차를 이용해서 그리고 기존 경계선/표면을 분할(split)하기도 해서 기존 지형에 라인스트링을 추가합니다. 경계선 식별자를 반환합니다.

TopoGeo_AddPolygon — 허용 오차를 이용해서 그리고 기존 경계선/표면을 분할(split)하기도 해서 기존 지형에 폴리곤을 추가합니다.

11.4.1. CreateTopology

CreateTopology — 새 지형 스키마를 생성하고 topology.topology 테이블에 이 새 스키마를 등록합니다.

Synopsis

integer CreateTopology(varchar *topology_schema_name*);

integer CreateTopology(varchar *topology_schema_name*, integer *srid*);

integer CreateTopology(varchar *topology_schema_name*, integer *srid*, double precision *tolerance*);

integer CreateTopology(varchar *topology_schema_name*, integer *srid*, double precision *tolerance*, boolean *hasz*);

설명: edge_data, face, node, relation 테이블로 이루어진 topology_name 명칭을 가진 새 스키마를 생성하고, topology.topology 테이블에 이 새 지형을 등록합니다. 지형 테이블에 있는 지형의 ID 를 반환합니다. SRID 는 해당 지형에 대해 spatial_ref_sys 테이블에 정의된 대로 식별된 공간 참조 시스템입니다. 지형의 명칭은 유일해야만 합니다. 공간 참조 시스템의 단위로 허용 오차를 측정합니다. 허용 오차를 따로 설정하지 않을 경우 기본값은 0 입니다.

이 함수는 SQL/MM 의 ST_InitTopoGeo 와 유사하지만 좀 더 기능적입니다. hasz 를 설정하지 않을 경우 기본값은 거짓입니다.

1.? 버전부터 사용할 수 있습니다.

예시

다음 예시는 매사추세츠 주 미터 단위 평면에 경계선, 표면, 그리고 관계성을 저장할 ma_topo 라는 새 스키마를 생성합니다. 공간 참조 시스템이 미터 단위 단위이기 때문에 허용 오차도 1/2 미터가 됩니다.

```
SELECT topology.CreateTopology('ma_topo',26986, 0.5);
```

로드아일랜드 주 피트 단위 평면에 로드아일랜드 지형을 생성합니다.

```
SELECT topology.CreateTopology('ri_topo',3438) As topoid;
topoid
------
2
```

참고: Section 4.3.1, "SPATIAL_REF_SYS 테이블과 공간 참조 시스템", ST_InitTopoGeo, Topology_Load_Tiger

11.4.2. CopyTopology

CopyTopology — 지형 구조(노드, 경계선, 표면, 레이어 및 TopoGeometry)를 복사합니다.

Synopsis

integer CopyTopology(varchar *existing_topology_name*, varchar *new_name*);

설명: existing_topology_name 에서 가져온 SRID 및 정확도, 그리고 new_topology_name 명칭을 가진 새 지형을 생성하고, existing_topology_name 의 모든 노드, 경계선, 표면을 복사한 다음 레이어 및 TopoGeometry 도 복사합니다.



topology.layer 의 새 행들이 schema_name, table_name 및 feature_column 에 대해 합성된 값들을 담을 것입니다. TopoGeometry 가 오직 정의로만 존재하고, 아직 어떤 사용자 레벨 테이블에서도 사용할 수 없기 때문입니다.

2.0.0 버전부터 사용할 수 있습니다.

예시

다음 예시는 ma_topo 라는 지형을 백업합니다.

```
SELECT topology.CopyTopology('ma_topo', 'ma_topo_bakup');
```

참고: Section 4.3.1, "SPATIAL_REF_SYS 테이블과 공간 참조 시스템", CreateTopology

11.4.3. ST_initTopoGeo

ST_InitTopoGeo — 새 지형 스키마를 생성하고 topology.topology 테이블에 이 새 스키마를 등록한 다음 처리 과정의 상세한 요약문을 반환합니다.

Synopsis

text ST_InitTopoGeo(varchar *topology_schema_name*);

설명: 이 함수는 CreateTopology 의 SQL-MM 버전이지만, CreateTopology 함수의 공간 참조 및 허용 오차 옵션이 없고 지형 ID 대신 생성 작업을 설명하는 문자열을 출력합니다.

1.? 버전부터 사용할 수 있습니다.

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3 Topo-Geo and Topo-Net 3: Routine Details: X.3.17

예시

```
 SELECT topology.ST_InitTopoGeo('topo_schema_to_create') AS topocreation;
                astopocreation
---------------------------------------------------------------
 Topology-Geometry 'topo_schema_to_create' (id:7) created.
```

참고: CreateTopology

11.4.4. ST_CreateTopoGeo

ST_CreateTopoGeo — 텅 빈 입력 지형에 도형들의 집합을 추가한 다음 작업 성공 여부를 상세하게 설명하는 메시지를 반환합니다.

Synopsis

text ST_CreateTopoGeo(varchar *atopology*, geometry *acollection*);

설명: 텅 빈 입력 지형에 도형들의 집합을 추가한 다음 작업 성공 여부를 상세하게 설명하는 메시지를 반환합니다.

텅 빈 지형을 채우는 데 유용합니다.

2.0 버전부터 사용할 수 있습니다.

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM: Topo-Geo and Topo-Net 3: Routine Details -- X.3.18

예시

```
 -- 지형을 채웁니다. --
 SELECT topology.ST_CreateTopoGeo('ri_topo',
  ST_GeomFromText('MULTILINESTRING((384744    236928,384750    236923,384769
236911,384799 236895,384811 236890,384833 236884,
   384844    236882,384866    236881,384879    236883,384954    236898,385087
236932,385117 236938,
   385167    236938,385203    236941,385224    236946,385233    236950,385241
236956,385254 236971,
   385260    236979,385268    236999,385273    237018,385273    237037,385271
237047,385267 237057,
   385225    237125,385210    237144,385192    237161,385167    237192,385162
237202,385159 237214,
   385159    237227,385162    237241,385166    237256,385196    237324,385209
237345,385234 237375,
   385237    237383,385238    237399,385236    237407,385227    237419,385213
237430,385193 237439,
   385174    237451,385170    237455,385169    237460,385171    237475,385181
237503,385190 237521,
```

```
   385200    237533,385206    237538,385213    237541,385221    237542,385235
237540,385242 237541,
   385249    237544,385260    237555,385270    237570,385289    237584,385292
237589,385291 237596,385284 237630))',3438)
   );

      st_createtopogeo
  ----------------------------
   Topology ri_topo populated



  -- 테이블 및 지형 도형을 생성합니다. --
  CREATE TABLE ri.roads(gid serial PRIMARY KEY, road_name text);

  SELECT  topology.AddTopoGeometryColumn('ri_topo',  'ri',  'roads',  'topo',
'LINE');
```

참고: AddTopoGeometryColumn, CreateTopology, DropTopology

11.4.5. TopoGeo_AddPoint

TopoGeo_AddPoint — 허용 오차를 이용해서 그리고 기존 경계선을 분할(split)하기도 해서 기존 지형에 포인트를 추가합니다.

Synopsis

integer TopoGeo_AddPoint(varchar *toponame*, geometry *apoint*, float8 *tolerance*);

설명: 기존 지형에 포인트를 추가하고 해당 포인트의 식별자를 반환합니다. 이 포인트는 입력 허용 오차 안에 있는 기존 노드 또는 경계선에 스냅할 것입니다. 스냅된 포인트가 기존 경계선을 분할할 수도 있습니다.

2.0.0 버전부터 사용할 수 있습니다.

참고: TopoGeo_AddLineString, TopoGeo_AddPolygon, AddNode, CreateTopology

11.4.6. TopoGeo_AddLineString

TopoGeo_AddLineString — 허용 오차를 이용해서 그리고 기존 경계선/표면을 분할(split)하기도 해서 기존 지형에 라인스트링을 추가합니다. 경계선 식별자를 반환합니다.

Synopsis

SETOF integer TopoGeo_AddLineString(varchar *toponame*, geometry *aline*, float8 *tolerance*);

설명: 기존 지형에 라인스트링을 추가하고 해당 라인스트링을 형성하는 경계선의 식별자 집합을 반환합니다. 이 라인스트링은 입력 허용 오차 안에 있는 기존 노드 또는 경계선에 스냅할 것입니다. 스냅된 라인스트링이 기존 경계선과 표면을 분할할 수도 있습니다.

2.0.0 버전부터 사용할 수 있습니다.

참고: TopoGeo_AddPoint, TopoGeo_AddPolygon, AddEdge, CreateTopology

11.4.7. TopoGeo_AddPolygon

TopoGeo_AddPolygon — 허용 오차를 이용해서 그리고 기존 경계선/표면을 분할(split)하기도 해서 기존 지형에 폴리곤을 추가합니다.

Synopsis

integer TopoGeo_AddPolygon(varchar *atopology*, geometry *apoly*, float8 *atolerance*);

설명: 기존 지형에 폴리곤을 추가하고 해당 폴리곤을 형성하는 표면의 식별자 집합을 반환합니다. 이 폴리곤의 외곽선은 입력 허용 오차 안에 있는 기존 노드 또는 경계선에 스냅할 것입니다. 새 폴리곤의 외곽선이 기존 경계선과 표면을 분할할 수도 있습니다.

2.0.0 버전부터 사용할 수 있습니다.

참고: TopoGeo_AddPoint, TopoGeo_AddLineString, AddFace, CreateTopology

11.5. 지형 편집자

이 단원에서 경계선, 표면, 노드를 추가하고, 이동하고, 삭제하고, 분할하기 위한 지형 함수에 대해 설명합니다. ISO SQL/MM 이 이 단원의 모든 함수를 정의합니다.

ST_AddIsoNode — 지형 안의 표면에 고립된(isolated) 노드를 추가하고 새 노드의 ID 를 반환합니다. 표면이 NULL 일 경우, 그래도 노드를 생성합니다.

ST_AddIsoEdge — 지형에 두 개의 고립된 기존 노드 anode 와 anothernode 를 연결하는 alinestring 도형이 정의하는 고립된 경계선을 추가하고 새 경계선의 ID 를 반환합니다.

ST_AddEdgeNewFaces — 새 경계선을 추가하고, 그 과정에서 경계선이 표면을 분할할 경우, 원본 표면을 삭제하고 새 표면 2 개로 교체합니다.

ST_AddEdgeModFace — 새 경계선을 추가하고, 그 과정에서 경계선이 표면을 분할할 경우, 원본 표면을 수정하고 새 표면을 추가합니다.

ST_RemEdgeNewFace — 경계선을 제거하고, 제거한 경계선이 두 표면을 분할하고 있을 경우, 원본 표면들을 삭제하고 새 표면 하나로 대체합니다.

ST_RemEdgeModFace — 경계선을 제거하고, 제거한 경계선이 두 표면을 분할하고 있을 경우, 표면 하나를 삭제하고 다른 표면을 두 표면의 공간을 차지하도록 수정합니다.

ST_ChangeEdgeGeom — 지형 구조를 건드리는 일 없이 경계선의 형상을 변경합니다.

ST_ModEdgeSplit — 기존 경계선을 따라 새 노드를 추가한 다음, 원본 경계선을 수정하고 새 경계선을 추가해서 경계선을 분할합니다.

ST_ModEdgeHeal — 두 경계선을 연결하는 노드를 삭제한 다음, 첫 번째 경계선을 수정하고 두 번째 경계선을 삭제해서 두 경계선을 붙입니다. 삭제한 노드의 ID 를 반환합니다.

ST_NewEdgeHeal — 두 경계선을 연결하는 노드를 삭제한 다음, 두 경계선을 모두 삭제하고 첫 번째 경계선과 동일한 방향인 경계선으로 교체합니다.

ST_MoveIsoNode — 지형 안에 있는 고립된 노드를 어떤 위치에서 다른 위치로 이동합니다. 새 apoint 도형의 위치에 노드가 존재할 경우 오류가 발생합니다. 이동을 설명하는 메시지를 반환합니다.

ST_NewEdgesSplit — 기존 경계선을 따라 새 노드를 추가한 다음, 원본 경계선을 삭제하고 새 경계선 2 개로 교체해서 경계선을 분할합니다. 새 경계선들을 결합하는 새로 생성된 노드의 ID 를 반환합니다.

ST_RemoveIsoNode — 고립된 노드를 제거하고 작업 내용을 설명하는 메시지를 반환합니다. 노드가 고립되지 않은 (경계선의 시작점이나 종단점인) 경우, 예외가 발생합니다.

11.5.1. ST_AddIsoNode

ST_AddIsoNode — 지형 안의 표면에 고립된(isolated) 노드를 추가하고 새 노드의 ID 를 반환합니다. 표면이 NULL 일 경우, 그래도 노드를 생성합니다.

Synopsis

integer ST_AddIsoNode(varchar *atopology*, integer *aface*, geometry *apoint*);

설명: atopology 지형의 aface ID(faceid)를 가진 기존 표면에 포인트 위치가 apoint 인 고립된 노드를 추가하고 새 노드의 ID(nodeid)를 반환합니다.

포인트 도형과 지형의 공간 참조 시스템(SRID)이 다를 경우, apoint 가 포인트 도형이 아닐 경우, 포인트가 NULL 일 경우, 또는 포인트가 기존 경계선과 (또는 외곽선일지라도) 교차하는 경우 예외가 발생합니다. 해당 포인트의 위치에 이미 노드가 존재할 경우에도 예외가 발생합니다.

aface 가 NULL 이 아닌데 apoint 가 표면 내부에 있지 않을 경우, 예외가 발생합니다.

1.? 버전부터 사용할 수 있습니다.

✓ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM: Topo-Net Routines: X+1.3.1 **참고:** AddNode, CreateTopology, DropTopology, ST_Intersects

11.5.2. ST_ AddIsoEdge

ST_AddIsoEdge — 지형에 두 개의 고립된 기존 노드 anode 와 anothernode 를 연결하는 alinestring 도형이 정의하는 고립된 경계선을 추가하고 새 경계선의 ID 를 반환합니다.

Synopsis

integer ST_AddIsoEdge(varchar *atopology*, integer *anode*, integer *anothernode*, geometry *alinestring*);

설명: 지형에 두 개의 고립된 기존 노드 anode 와 anothernode 를 연결하는 alinestring 도형이 정의하는 고립된 경계선을 추가하고 새 경계선의 ID(edgeid)를 반환합니다.

alinestring 도형과 지형의 공간 참조 시스템(SRID)이 다를 경우, 입력 인수 가운데 하나라도 NULL 일 경우, 하나 이상의 표면이 노드를 담고 있을 경우, 또는 노드가 기존 경계선의 시작 또는 종단 노드일 경우, 예외가 발생합니다.

alinestring 이 anode 와 anothernode 가 속해 있는 표면 내부에 없을 경우, 예외가 발생합니다.

anode 와 anothernode 가 alinestring 의 시작점 및 종단점이 아닐 경우 예외가 발생합니다.

1.? 버전부터 사용할 수 있습니다.

✓ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM: Topo-Geo and Topo-Net 3: Routine Details: X.3.4 **참고:** ST_AddIsoNode, ST_IsSimple, ST_Within

11.5.3. ST_AddEdgeNewFaces

ST_AddEdgeNewFaces — 새 경계선을 추가하고, 그 과정에서 경계선이 표면을 분할할 경우, 원본 표면을 삭제하고 새 표면 2 개로 교체합니다.

Synopsis

integer ST_AddEdgeNewFaces(varchar *atopology*, integer *anode*, integer *anothernode*, geometry *acurve*);

설명: 새 경계선을 추가하고, 그 과정에서 경계선이 표면을 분할할 경우, 원본 표면을 삭제하고 새 표면 2 개로 교체합니다. 새로 추가된 경계선의 ID 를 반환합니다.

모든 기존의 결합된 경계선 및 관계성을 결과에 맞춰 업데이트합니다.

어떤 인수가 NULL 일 경우, 주어진 노드들이 알려지지 않은 경우(노드가 이미 지형 스키마의 node 테이블에 존재해야 합니다), acurve 가 LINESTRING 이 아닐 경우, anode 와 anothernode 가 acurve 의 시작점 및 종단점이 아닐 경우 오류가 발생합니다.

acurve 도형과 지형의 공간 참조 시스템(SRID)이 다를 경우 예외가 발생합니다.

2.0 버전부터 사용할 수 있습니다.

✓ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM: Topo-Geo and Topo-Net 3: Routine Details: X.3.12

참고: ST_RemEdgeNewFace**,** ST_AddEdgeModFace

11.5.4. ST_AddEdgeModFace

ST_AddEdgeModFace — 새 경계선을 추가하고, 그 과정에서 경계선이 표면을 분할할 경우, 원본 표면을 수정하고 새 표면을 추가합니다.

Synopsis

integer ST_AddEdgeModFace(varchar atopology, integer anode, integer anothernode, geometry acurve);

설명: 새 경계선을 추가하고, 그 과정에서 경계선이 표면을 분할할 경우, 원본 표면을 수정하고 새 표면을 추가합니다.



가능한 경우, 새 경계선의 왼쪽에 새 표면을 생성할 것입니다. 왼쪽의 표면이 (무한한) 세계 표면(universe face)이어야 할 경우 이는 불가능할 것입니다.

새로 추가된 경계선의 ID 를 반환합니다.

모든 기존의 결합된 경계선 및 관계성을 결과에 맞춰 업데이트합니다.

어떤 인수가 NULL 일 경우, 주어진 노드들이 알려지지 않은 경우(노드가 이미 지형 스키마의 node 테이블에 존재해야 합니다), acurve 가 LINESTRING 이 아닐 경우, anode 와 anothernode 가 acurve 의 시작점 및 종단점이 아닐 경우 오류가 발생합니다.

acurve 도형과 지형의 공간 참조 시스템(SRID)이 다를 경우 예외가 발생합니다.

2.0 버전부터 사용할 수 있습니다.

✓ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM: Topo-Geo and Topo-Net 3: Routine Details: X.3.13 **참고:** ST_RemEdgeModFace, ST_AddEdgeNewFaces

11.5.5. ST_RemEdgeNewFace

ST_RemEdgeNewFace — 경계선을 제거하고, 제거한 경계선이 두 표면을 분할하고 있을 경우, 원본 표면들을 삭제하고 새 표면 하나로 대체합니다.

Synopsis

integer ST_RemEdgeNewFace(varchar *atopology*, integer *anedge*);

설명: 경계선을 제거하고, 제거한 경계선이 두 표면을 분할하고 있을 경우, 원본 표면들을 삭제하고 새 표면 하나로 대체합니다.

새로 생성된 표면의 ID 를 반환하거나, 또는 새 표면이 생성되지 않은 경우 NULL 을 반환합니다. 제거한 경계선이 세계 표면에 매달려 있거나, 고립돼 있거나, 또는 (세계 표면이 다른 쪽의 표면을 침범하게 할 수 있게) 갇혀 있을 경우 새 표면을 생성할 수 없습니다.

모든 기존의 결합된 경계선 및 관계성을 결과에 맞춰 업데이트합니다.

기존 TopoGeometry 를 정의하는 데 쓰이는 경계선을 제거하지 않습니다. 두 표면 가운데 하나만 (다른 하나가 정의하는 TopoGeometry 는 없으며) 어떤 TopoGeometry 를 정의하는 경우 두 표면을 합치지 않습니다.

어떤 인수가 NULL 일 경우, 주어진 경계선이 알려지지 않은 경우(경계선이 이미 지형 스키마의 edge 테이블에 존재해야 합니다), 지형 명칭이 유효하지 않은 경우 오류가 발생합니다.

2.0 버전부터 사용할 수 있습니다.

✓ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM: Topo-Geo and Topo-Net 3: Routine Details: X.3.14

참고: ST_RemEdgeModFace, ST_AddEdgeNewFaces

11.5.6. ST_RemEdgeModFace

ST_RemEdgeModFace — 경계선을 제거하고, 제거한 경계선이 두 표면을 분할하고 있을 경우, 표면 하나를 삭제하고 다른 표면을 두 표면의 공간을 차지하도록 수정합니다.

Synopsis

integer ST_RemEdgeModFace(varchar *atopology*, integer *anedge*);

설명: 경계선을 제거하고, 제거한 경계선이 두 표면을 분할하고 있을 경우, 표면 하나를 삭제하고 다른 표면을 두 표면의 공간을 차지하도록 수정합니다. 우선적으로 오른쪽에 있는 표면을 유지합니다. 역시 오른쪽의 표면을 유지하는 ST_AddEdgeModFace 함수와 대칭을 이루기 위해서입니다. 제거한 경계선의 위치에 남아 있는 표면의 ID 를 반환합니다.

모든 기존의 결합된 경계선 및 관계성을 결과에 맞춰 업데이트합니다.

기존 TopoGeometry 를 정의하는 데 쓰이는 경계선을 제거하지 않습니다. 두 표면 가운데 하나만 (다른 하나가 정의하는 TopoGeometry 는 없으며) 어떤 TopoGeometry 를 정의하는 경우 두 표면을 합치지 않습니다.

어떤 인수가 NULL 일 경우, 주어진 경계선이 알려지지 않은 경우(경계선이 이미 지형 스키마의 edge 테이블에 존재해야 합니다), 지형 명칭이 유효하지 않은 경우 오류가 발생합니다.

2.0 버전부터 사용할 수 있습니다.

✓ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM: Topo-Geo and Topo-Net 3: Routine Details: X.3.15

참고: ST_AddEdgeModFace**,** ST_RemEdgeNewFace

11.5.7. ST_ChangeEdgeGeom

ST_ChangeEdgeGeom — 지형 구조를 건드리는 일 없이 경계선의 형상을 변경합니다.

Synopsis

integer ST_ChangeEdgeGeom(varchar *atopology*, integer *anedge*, geometry *acurve*);

설명: 지형 구조를 건드리는 일 없이 경계선의 형상을 변경합니다.

어떤 인수가 NULL 일 경우, 주어진 경계선이 지형 스키마의 edge 테이블에 존재하지 않을 경우, acurve 가 LINESTRING 이 아닐 경우, anode 와 anothernode 가 acurve 의 시작점 및 종단점이 아닐 경우, 또는 수정 작업이 기저 지형을 변경할 경우 오류가 발생합니다.

acurve 도형과 지형의 공간 참조 시스템(SRID)이 다를 경우 예외가 발생합니다.

새 acurve 가 단순 도형이 아닐 경우, 오류가 발생합니다.

예전 위치에서 새 위치로 경계선을 이동시킬 때 장애물에 부딪히는 경우 오류가 발생합니다.

1.1.0 버전부터 사용할 수 있습니다.

개선 사항: 2.0.0 버전부터 지형의 일관성을 강제합니다.

✓ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM: Topo-Geo and Topo-Net 3: Routine Details X.3.6

예시

```
 SELECT topology.ST_ChangeEdgeGeom('ma_topo', 1,
             ST_GeomFromText('LINESTRING(227591.9         893900.4,227622.6
893844.3,227641.6 893816.6, 227704.5 893778.5)', 26986) );
  ----
  Edge 1 changed
```

참고: ST_AddEdgeModFace, ST_RemEdgeModFace, ST_ModEdgeSplit

11.5.8. ST_ModeEdgeSplit

ST_ModEdgeSplit — 기존 경계선을 따라 새 노드를 추가한 다음, 원본 경계선을 수정하고 새 경계선을 추가해서 경계선을 분할합니다.

Synopsis

integer ST_ModEdgeSplit(varchar *atopology*, integer *anedge*, geometry *apoint*);

설명: 기존 경계선을 따라 새 노드를 추가한 다음, 원본 경계선을 수정하고 새 경계선을 추가해서 경계선을 분할합니다. 기존의 모든 결합된 경계선들 및 관계성을 새로이 분할된 경계선들에 맞춰 업데이트합니다. 새로 추가된 노드의 식별자를 반환합니다.

1.? 버전부터 사용할 수 있습니다.

변경 사항: 2.0 미만 버전에서, 이 함수는 ST_ModEdgesSplit 이라는 잘못된 명칭이었습니다.

✓ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM: Topo-Geo and Topo-Net 3: Routine Details: X.3.9

예시

```
 -- 경계선 추가 --
  SELECT    topology.AddEdge('ma_topo',    ST_GeomFromText('LINESTRING(227592
893910, 227600 893910)', 26986) ) As edgeid;

 -- edgeid --
 3


 -- 경계선 분할  --
```

```
  SELECT          topology.ST_ModEdgeSplit('ma_topo',                    3,
ST_SetSRID(ST_Point(227594,893910),26986)  ) As node_id;
        node_id
  -------------------------
  7
```

참고: ST_NewEdgesSplit, ST_ModEdgeHeal, ST_NewEdgeHeal, AddEdge

11.5.9. ST_ModEdgeHeal

ST_ModEdgeHeal — 두 경계선을 연결하는 노드를 삭제한 다음, 첫 번째 경계선을 수정하고 두 번째 경계선을 삭제해서 두 경계선을 붙입니다. 삭제한 노드의 ID 를 반환합니다.

Synopsis

int ST_NewEdgeHeal(varchar *atopology*, integer *anedge*, integer *anotheredge*);

설명: 두 경계선을 연결하는 노드를 삭제한 다음, 첫 번째 경계선을 수정하고 두 번째 경계선을 삭제해서 두 경계선을 붙입니다. 삭제한 노드의 ID 를 반환합니다. 기존의 모든 결합된 경계선들 및 관계성을 삭제된 노드에 맞춰 업데이트합니다.

2.0 버전부터 사용할 수 있습니다.

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM: Topo-Geo and Topo-Net 3: Routine Details: X.3.9

참고: ST_ModEdgeSplit ST_NewEdgesSplit

11.5.10. ST_NewEdgeHeal

ST_NewEdgeHeal — 두 경계선을 연결하는 노드를 삭제한 다음, 두 경계선을 모두 삭제하고 첫 번째 경계선과 동일한 방향인 경계선으로 교체합니다.

Synopsis

int ST_NewEdgeHeal(varchar *atopology*, integer *anedge*, integer *anotheredge*);

설명: 두 경계선을 연결하는 노드를 삭제한 다음, 두 경계선을 모두 삭제하고 첫 번째 경계선과 동일한 방향인 경계선으로 교체합니다. 교체한 새 경계선의 ID 를 반환합니다. 기존의 모든 결합된 경계선들 및 관계성을 교체된 경계선에 맞춰 업데이트합니다.

2.0 버전부터 사용할 수 있습니다.

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM: Topo-Geo and Topo-Net 3: Routine Details: X.3.9

참고: ST_ModEdgeHeal ST_ModEdgeSplit ST_NewEdgesSplit

11.5.11. ST_MoveIsoNode

ST_MoveIsoNode — 지형 안에 있는 고립된 노드를 어떤 위치에서 다른 위치로 이동합니다. 새 apoint 도형의 위치에 노드가 존재할 경우 오류가 발생합니다. 이동을 설명하는 메시지를 반환합니다.

Synopsis

text ST_MoveIsoNode(varchar *atopology*, integer *anedge*, geometry *apoint*);

설명: 지형 안에 있는 고립된 노드를 어떤 위치에서 다른 위치로 이동합니다. 새 apoint 도형의 위치에 노드가 존재할 경우 오류가 발생합니다.

어떤 인수가 NULL 일 경우, apoint 가 포인트 도형이 아닐 경우, 기존 노드가 고립 노드가 아닐 (기존 경계선의 시작점 또는 종단점일) 경우, 새 노드 위치가 기존 경계선과 (종단점에서라도) 교차할 경우 예외가 발생합니다.

포인트 도형과 지형의 공간 참조 시스템(SRID)이 다를 경우 예외가 발생합니다.

1.? 버전부터 사용할 수 있습니다.

✓ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM: Topo-Net Routines: X.3.2

예시

```
  -- 표면이 없는 고립 노드를 추가 --
  SELECT          topology.ST_AddIsoNode('ma_topo',                    NULL,
ST_GeomFromText('POINT(227579 893916)', 26986) ) As nodeid;
   nodeid
  --------
       7
  -- 새 노드를 이동 --
  SELECT                topology.ST_MoveIsoNode('ma_topo',                 7,
ST_GeomFromText('POINT(227579.5 893916.5)', 26986) ) As descrip;
                   descrip
  ------------------------------------------------
  Isolated Node 7 moved to location 227579.5,893916.57
```

참고: ST_AddIsoNode

11.5.12. ST_NewEdgesSplit

ST_NewEdgesSplit — 기존 경계선을 따라 새 노드를 추가한 다음, 원본 경계선을 삭제하고 새 경계선 2 개로 교체해서 경계선을 분할합니다. 새 경계선들을 결합하는 새로 생성된 노드의 ID 를 반환합니다.

Synopsis

integer ST_NewEdgesSplit(varchar *atopology*, integer *anedge*, geometry *apoint*);

설명: 기존 경계선을 따라 apoint 포인트 위치에 새 노드를 생성한 다음, 다음, 원본 경계선을 삭제하고 새 경계선 2 개로 교체해서 경계선 ID anedge 인 경계선을 분할합니다. 새 경계선들을 결합하는 새로

생성된 노드의 ID 를 반환합니다. 기존의 모든 결합된 경계선들 및 관계성을 분할된 경계선에 맞춰 업데이트합니다.

포인트 도형과 지형의 공간 참조 시스템(SRID)이 다를 경우, apoint 가 포인트 도형이 아닐 경우, 포인트가 NULL 일 경우, 해당 포인트의 위치에 이미 노드가 존재할 경우, 경계선이 기존 경계선과 일치하지 않을 경우, 또는 포인트가 경계선 안에 있지 않을 경우 예외가 발생합니다.

1.? 버전부터 사용할 수 있습니다.

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM: Topo-Net Routines: X.3.8

예시

```
  -- 경계선 추가 --
  SELECT    topology.AddEdge('ma_topo',    ST_GeomFromText('LINESTRING(227575
893917,227592 893900)', 26986) ) As edgeid;
  -- 결과 --
  edgeid
  --------
         2
  -- 새 경계선을 분할 --
  SELECT                topology.ST_NewEdgesSplit('ma_topo',                2,
ST_GeomFromText('POINT(227578.5 893913.5)', 26986) ) As newnodeid;
   newnodeid
  ----------

         6
```

참고: ST_ModEdgeSplit ST_ModEdgeHeal ST_NewEdgeHeal AddEdge

11.5.13. ST_RemoveIsoNode

ST_RemoveIsoNode — 고립된 노드를 제거하고 작업 내용을 설명하는 메시지를 반환합니다. 노드가 고립되지 않은 (경계선의 시작점이나 종단점인) 경우, 예외가 발생합니다.

Synopsis

text ST_RemoveIsoNode(varchar *atopology*, integer *anode*);

설명: 고립된 노드를 제거하고 작업 내용을 설명하는 메시지를 반환합니다. 노드가 고립되지 않은 (경계선의 시작점이나 종단점인) 경우, 예외가 발생합니다.

1.? 버전부터 사용할 수 있습니다.

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM: Topo-Geo and Topo-Net 3: Routine Details: X+1.3.3

예시

```
-- 표면이 없는 고립 노드를 제거 --
SELECT topology.ST_RemoveIsoNode('ma_topo',  7 ) As result;
       result
-----------------------
 Isolated node 7 removed
```

참고: ST_AddIsoNode

11.6. 지형 접근자

GetEdgeByPoint — 입력 포인트와 교차하는 경계선의 경계선 ID(edgeid)를 탐색합니다.

GetFaceByPoint — 입력 포인트와 교차하는 표면의 표면 ID(faceid)를 탐색합니다.

GetNodeByPoint — 입력 포인트 위치에 있는 노드의 ID 를 탐색합니다.

GetTopologyID — 지형의 명칭을 입력받아 topology.topology 테이블에 있는 지형의 ID 를 반환합니다.

GetTopologySRID — 지형의 명칭을 입력받아 topology.topology 테이블에 있는 지형의 SRID 를 반환합니다.

GetTopologyName — 지형의 ID 를 입력받아 지형(스키마)의 명칭을 반환합니다.

ST_GetFaceEdges — aface 의 경계를 이루는 정렬된 경계선들의 집합을 반환합니다.

ST_GetFaceGeometry — 입력 지형으로부터 설정된 표면 ID 를 가진 폴리곤을 반환합니다.

GetRingEdges — 입력 경계선 쪽을 따라 탐색한 부호가 있는 경계선 식별자들의 정렬된 집합을 반환합니다.

GetNodeEdges — 입력 노드를 따르는 경계선들의 정렬된 집합을 반환합니다.

11.6.1. GetEdgeByPoint

GetEdgeByPoint — 입력 포인트와 교차하는 경계선의 경계선 ID(edgeid)를 탐색합니다.

Synopsis

integer GetEdgeByPoint(varchar *atopology*, geometry *apoint*, float8 *tol*);

포인트와 교차하는 경계선의 ID 를 검색합니다.

이 함수는 지형, 포인트, 그리고 허용 오차를 입력받아 정수(edgeid)를 반환합니다. tolerance = 0 일 경우 포인트와 경계선이 교차해야 합니다.

포인트와 경계선이 교차하지 않을 경우, 0 을 반환합니다.

0 보다 큰 허용 오차를 설정했는데 포인트 가까이에 하나 이상의 경계선이 있을 경우 예외가 발생합니다.



이 함수는 tolerance = 0 일 경우 ST_Intersects 를, 그렇지 않을 경우 ST_DWithin 을 이용합니다.

2.0.0 버전부터 사용할 수 있습니다. GEOS 3.3.0 이상 버전이 필요합니다.

예시

다음 예시들은 **AddEdge** 에서 생성한 경계선들을 사용합니다.

```
  SELECT    topology.GetEdgeByPoint('ma_topo',geom,    1)    As    with1mtol,
topology.GetEdgeByPoint('ma_topo',geom,0) As withnotol
  FROM ST_GeomFromEWKT('SRID=26986;POINT(227622.6 893843)') As geom;
   with1mtol | withnotol
  -----------+-----------
           2 |         0
  SELECT topology.GetEdgeByPoint('ma_topo',geom, 1) As nearnode
  FROM ST_GeomFromEWKT('SRID=26986;POINT(227591.9 893900.4)') As geom;

  --오류 발생 --
  ERROR:  Two or more edges found
```

참고: AddEdge, GetNodeByPoint, GetFaceByPoint

11.6.2. GetFaceByPoint

GetFaceByPoint — 입력 포인트와 교차하는 표면의 표면 ID(faceid)를 탐색합니다.

Synopsis

integer GetFaceByPoint(varchar *atopology*, geometry *apoint*, float8 *tol*);

설명: 포인트와 교차하는 표면의 ID 를 검색합니다.

이 함수는 지형, 포인트, 그리고 허용 오차를 입력받아 정수(faceid)를 반환합니다. tolerance = 0 일 경우 포인트와 표면이 교차해야 합니다.

포인트와 표면이 교차하지 않을 경우, 0 을 반환합니다.

0 보다 큰 허용 오차를 설정했는데 포인트 가까이에 하나 이상의 표면이 있을 경우 예외가 발생합니다.



이 함수는 tolerance = 0 일 경우 ST_Intersects 를, 그렇지 않을 경우 ST_DWithin 을 이용합니다.

2.0.0 버전부터 사용할 수 있습니다. GEOS 3.3.0 이상 버전이 필요합니다.

예시

다음 예시들은 AddFace 에서 생성한 표면들을 사용합니다.

```
 SELECT    topology.GetFaceByPoint('ma_topo',geom,    10)    As    with1mtol,
topology.GetFaceByPoint('ma_topo',geom,0) As withnotol
       FROM ST_GeomFromEWKT('POINT(234604.6 899382.0)') As geom;

              with1mtol | withnotol
       -------------+----------------
                     1 |     0
 SELECT topology.GetFaceByPoint('ma_topo',geom, 1) As nearnode
       FROM ST_GeomFromEWKT('POINT(227591.9 893900.4)') As geom;

 -- 오류 발생 --
 ERROR:  Two or more faces found
```

참고: AddFace, GetNodeByPoint, GetEdgeByPoint

11.6.3. GetNodeByPoint

GetNodeByPoint — 입력 포인트 위치에 있는 노드의 ID 를 탐색합니다.

Synopsis

integer GetNodeByPoint(varchar *atopology*, geometry *point*, float8 *tol*);

입력 포인트 위치에 있는 노드의 ID 를 검색합니다.

이 함수는 지형, 포인트, 그리고 허용 오차를 입력받아 정수(nodeid)를 반환합니다. tolerance = 0 일 경우 정확히 교차점을 의미하고, 그렇지 않을 경우 간격(interval)으로부터 노드를 검색합니다.

포인트 위치에 노드가 없을 경우 0 을 반환합니다.

0 보다 큰 허용 오차를 설정했는데 포인트 가까이에 하나 이상의 노드가 있을 경우 예외가 발생합니다.



이 함수는 tolerance = 0 일 경우 ST_Intersects 를, 그렇지 않을 경우 ST_DWithin 을 이용합니다.

2.0.0 버전부터 사용할 수 있습니다. GEOS 3.3.0 이상 버전이 필요합니다.

예시

다음 예시들은 AddEdge 에서 생성한 경계선들을 사용합니다.

```
 SELECT topology.GetNodeByPoint('ma_topo',geom, 1) As nearnode
  FROM ST_GeomFromEWKT('SRID=26986;POINT(227591.9 893900.4)') As geom;
   nearnode
 ----------
        2

 SELECT topology.GetNodeByPoint('ma_topo',geom, 1000) As too_much_tolerance
  FROM ST_GeomFromEWKT('SRID=26986;POINT(227591.9 893900.4)') As geom;

  -- 오류 발생 --
  ERROR:  Two or more nodes found
```

참고: AddEdge, GetEdgeByPoint, GetFaceByPoint

11.6.4. GetTopologyID

GetTopologyID — 지형의 명칭을 입력받아 topology.topology 테이블에 있는 지형의 ID 를 반환합니다.

Synopsis

integer GetTopologyID(varchar *toponame*);

설명: 지형의 명칭을 입력받아 topology.topology 테이블에 있는 지형의 ID 를 반환합니다.

1.? 버전부터 사용할 수 있습니다.

예시

```
SELECT topology.GetTopologyID('ma_topo') As topo_id;
 topo_id
---------------------------------------------------------------------------
      1
```

참고: CreateTopology, DropTopology, GetTopologyName, GetTopologySRID

11.6.5. GetTopologySRID

GetTopologySRID — 지형의 명칭을 입력받아 topology.topology 테이블에 있는 지형의 SRID 를 반환합니다.

Synopsis

integer GetTopologyID(varchar *toponame*);

설명: 지형의 명칭을 입력받아 topology.topology 테이블에 있는 지형의 공간 참조 식별자를 반환합니다.

2.0.0 버전부터 사용할 수 있습니다.

예시

```
SELECT topology.GetTopologySRID('ma_topo') As SRID;
 SRID
--------------------------
  4326
```

참고: CreateTopology, DropTopology, GetTopologyName, GetTopologyID

11.6.6. GetTopologyName

GetTopologyName — 지형의 ID 를 입력받아 지형(스키마)의 명칭을 반환합니다.

Synopsis

varchar GetTopologyName(integer *topology_id*);

설명

지형의 지형 ID 를 입력받아 topology.topology 테이블에서 지형의 지형명(스키마)을 반환합니다.

1.? 버전부터 사용할 수 있습니다.

예시

```
SELECT topology.GetTopologyName(1) As topo_name;
 topo_name
-----------
 ma_topo
```

참고: CreateTopology, DropTopology, GetTopologyID, GetTopologySRID

11.6.7. ST_GetFaceEdges

ST_GetFaceEdges — aface 의 경계를 이루는 정렬된 경계선들의 집합을 반환합니다.

Synopsis

getfaceedges_returntype ST_GetFaceEdges(varchar *atopology*, integer *aface*);

설명: aface 의 경계를 이루는 정렬된 경계선들의 집합을 반환합니다. 각 출력물은 순차(sequence) 및 경계선 ID(edgeid)로 이루어집니다. 순차 번호는 1 부터 시작합니다.

각 고리의 경계선을 가장 작은 식별자를 가진 경계선부터 열거합니다. 경계선의 순서는 왼손 법칙을 따릅니다(각 경계선의 왼쪽 방향으로 표면의 경계선을 구성하게 됩니다).

2.0 버전부터 사용할 수 있습니다.

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3 Topo-Geo and Topo-Net 3: Routine Details: X.3.5

예시

```
-- 표면 1 의 경계를 이루는 경계선들을 반환
SELECT (topology.ST_GetFaceEdges('tt', 1)).*;
-- 결과 --
 sequence | edge
----------+------
        1 |   -4
        2 |    5
        3 |    7
        4 |   -6
```

```
        5 |    1
        6 |    2
        7 |    3
  (7 rows)
  -- 표면 1 의 경계를 이루은 경계선들의 도형과
  -- 순차 및 경계선 ID 를 반환
  -- 도형과 순차만 필요할 경우, ST_GetFaceGeometry 함수를 이용할 수 있습니다.
  SELECT t.seq, t.edge, geom
  FROM topology.ST_GetFaceEdges('tt',1) As t(seq,edge)
        INNER JOIN tt.edge AS e ON abs(t.edge) = e.edge_id;
```

참고: GetRingEdges, AddFace, ST_GetFaceGeometry

11.6.8. ST_GetFaceGeometry

ST_GetFaceGeometry — 입력 지형으로부터 설정된 표면 ID 를 가진 폴리곤을 반환합니다.

Synopsis

geometry ST_GetFaceGeometry(varchar *atopology*, integer *aface*);

설명: 입력 지형으로부터 설정된 표면 ID 를 가진 폴리곤을 반환합니다. 표면을 구성하는 경계선들로부터 폴리곤을 빌드합니다.

1.? 버전부터 사용할 수 있습니다.

✔ 이 방법은 SQL/MM 사양을 도입합니다. SQL-MM 3 Topo-Geo and Topo-Net 3: Routine Details: X.3.16

예시

```
  -- AddFace 함수를 통해 추가된 폴리곤의 WKT 를 반환합니다.
  SELECT ST_AsText(topology.ST_GetFaceGeometry('ma_topo', 1)) As facegeomwkt;
  -- 결과 --
              facegeomwkt

  ------------------------------------------------------------------------------
   POLYGON((234776.9   899563.7,234896.5   899456.7,234914   899436.4,234946.6
899356.9,
  234872.5 899328.7,234891 899285.4,234992.5 899145,234890.6 899069,
  234755.2 899255.4,234612.7 899379.4,234776.9 899563.7))
```

참고: AddFace

11.6.9. GetRingEdges

GetRingEdges — 입력 경계선 쪽을 따라 탐색한 부호가 있는 경계선 식별자들의 정렬된 집합을 반환합니다.

Synopsis

getfaceedges_returntype GetRingEdges(varchar atopology, integer aring, integer max_edges=null);

설명: 입력 경계선 쪽을 따라 탐색한 부호가 있는 경계선 식별자들의 정렬된 집합을 반환합니다. 각 출력물은 순차(sequence) 및 부호가 있는 경계선 ID(edgeid)로 이루어집니다. 순차 번호는 1 부터 시작합니다.

양의 부호를 가진 경계선 ID 를 넘겨줄 경우, 해당 경계선의 왼쪽에서 탐색을 시작해서 경계선의 방향으로 진행합니다. 음의 부호를 가진 경계선 ID 를 넘겨줄 경우, 해당 경계선의 오른쪽에서 탐색을 시작해서 경계선 방향의 반대로 진행합니다.

max_edges 가 NULL 이 아닐 경우 함수가 해당 개수 이상의 레코드를 반환하지 않습니다. 유효하지 않을 수도 있는 지형을 처리할 때 이 파라미터를 안전 장치로 쓸 수 있다는 의미입니다.



이 함수는 메타데이터를 링크하는 경계선 고리를 이용합니다.

2.0.0 버전부터 사용할 수 있습니다.

참고: ST_GetFaceEdges, GetNodeEdges

11.6.10. GetNodeEdges

GetNodeEdges — 입력 노드를 따르는 경계선들의 정렬된 집합을 반환합니다.

Synopsis

getfaceedges_returntype GetNodeEdges(varchar *atopology*, integer *anode*);

설명: 입력 노드를 따르는 경계선들의 정렬된 집합을 반환합니다. 각 출력물은 순차 및 부호를 가진 경계선 ID 로 구성됩니다. 순차 번호는 1 부터 시작합니다. 양의 부호를 가진 경계선은 입력 노드로부터 시작합니다. 음의 부호를 가진 경계선은 입력 노드로 끝납니다. 폐쇄형 경계선은 두 번 (두 부호를 가지고) 표출될 것입니다. 북쪽부터 시작하는 시계 방향으로 정렬됩니다.



이 함수는 정렬 순서를 메타데이터에서 추출하기보다 계산하기 때문에 경계선 고리 링크를 빌드하는 데 유용합니다.

2.0 버전부터 사용할 수 있습니다.

참고: GetRingEdges, ST_Azimuth

11.7. 지형 공간 처리

Polygonize — 지형 경계선이 정의하는 모든 표면을 찾아 등록합니다.

AddNode — 설정한 지형 스키마의 노드 테이블에 포인트 노드를 추가하고 새 노드의 노드 ID(nodeid)를 반환합니다. 새 포인트 위치에 이미 노드가 존재할 경우, 기존 노드 ID 를 반환합니다.

AddEdge — 설정한 지형 스키마의 경계선 테이블에 라인스트링 경계선을 추가하고, 설정한 라인스트링 도형을 이용해서 포인트 노드 테이블에 경계선의 시작점 및 종단점을 추가한 다음 새 (또는 기존) 경계선의 경계선 ID(edgeid)를 반환합니다.

AddFace — 지형에 표면 원시형(face primitive)을 등록하고 그 식별자를 반환합니다.

ST_Simplify — 더글러스-패커(Douglas-Peucker) 알고리즘을 통해 입력 TopoGeometry 의 "단순화된" 도형 버전을 반환합니다.

11.7.1. Polygonize

Polygonize — 지형 경계선이 정의하는 모든 표면을 찾아 등록합니다.

Synopsis

text Polygonize(varchar *toponame*);

설명: 지형 경계선 원시형으로 확장될 수 있는 모든 표면을 등록합니다.

입력 지형이 스스로 교차하는 경계선을 가지고 있지 않다고 가정합니다.



이미 알려진 표면을 인식하기 때문에, 동일한 지형에 대해 Polygonize 함수를 반복해서 호출하는 편이 안전합니다.



이 함수는 edge 테이블의 next_left_edge 및 next_right_edge 항목을 이용하지도 설정하지도 않습니다.

2.0.0 버전부터 사용할 수 있습니다.

참고: AddFace, ST_Polygonize

11.7.2. AddNode

AddNode — 설정한 지형 스키마의 노드 테이블에 포인트 노드를 추가하고 새 노드의 노드 ID(nodeid)를 반환합니다. 새 포인트 위치에 이미 노드가 존재할 경우, 기존 노드 ID 를 반환합니다.

Synopsis

integer AddNode(varchar *toponame*, geometry *apoint*, boolean *allowEdgeSplitting=false*, boolean *computeContainingFace=false*);

설명: 설정한 지형 스키마의 노드 테이블에 포인트 노드를 추가합니다. AddEdge 함수를 호출할 때 경계선의 시작점 및 종단점을 자동적으로 추가하므로, 경계선의 노드를 따로 추가할 필요는 없습니다.

새 노드를 가로지르는 경계선이 있을 경우, allowEdgeSplitting 파라미터 값에 따라 예외가 발생하거나 또는 해당 경계선을 분할합니다.

computeContainingFace 가 참일 경우 새로 추가된 노드를 정확히 담고 있는 표면을 계산합니다.



apoint 도형의 위치에 이미 노드가 존재할 경우, 새 노드를 추가하지 않고 기존 노드 ID(nodeid)를 반환합니다.

2.0.0 버전부터 사용할 수 있습니다.

예시

```
 SELECT     topology.AddNode('ma_topo',     ST_GeomFromText('POINT(227641.6
893816.5)', 26986) ) As nodeid;
 -- 결과 --
 nodeid
 --------
  4
```

참고: AddEdge, CreateTopology

11.7.3. AddEdge

AddEdge — 설정한 지형 스키마의 경계선 테이블에 라인스트링 경계선을 추가하고, 설정한 라인스트링 도형을 이용해서 포인트 노드 테이블에 경계선의 시작점 및 종단점을 추가한 다음 새 (또는 기존) 경계선의 경계선 ID(edgeid)를 반환합니다.

Synopsis

integer AddEdge(varchar *toponame*, geometry *aline*);

설명: 설정한 라인스트링 도형을 이용해서 설정한 toponame 스키마의 경계선 테이블에 경계선을, 노드 테이블에 경계선의 노드를 추가한 다음 새 (또는 기존) 레코드의 경계선 ID(edgeid)를 반환합니다. 새로 추가된 경계선은 양쪽에 "세계(universe)" 표면을 가지며 자기 자신에게 링크합니다.



기존 라인스트링 경계선이 aline 도형을 가로지르거나, 중첩하거나, 또는 담고 있을 경우 오류가 발생하고 경계선을 추가하지 않습니다.



aline 도형의 srid 와 지형에 대해 정의된 srid 가 동일해야만 합니다. 그렇지 않을 경우 유효하지 않은 공간 참조 시스템 오류가 발생할 것입니다.

2.0.0 버전부터 사용할 수 있습니다. GEOS 3.3.0 이상 버전이 필요합니다.

예시

```
 SELECT   topology.AddEdge('ma_topo',   ST_GeomFromText('LINESTRING(227575.8
893917.2,227591.9 893900.4)', 26986) ) As edgeid;
 -- 결과 --
 edgeid
 --------
  1

 SELECT   topology.AddEdge('ma_topo',   ST_GeomFromText('LINESTRING(227591.9
893900.4,227622.6 893844.2,227641.6 893816.5,
  227704.5 893778.5)', 26986) ) As edgeid;
 -- 결과 --
 edgeid
 --------
  2

  SELECT   topology.AddEdge('ma_topo',   ST_GeomFromText('LINESTRING(227591.2
893900, 227591.9 893900.4,
   227704.5 893778.5)', 26986) ) As edgeid;
  -- 오류 발생 --
  ERROR:  Edge intersects (not on endpoints) with existing edge 1
```

참고: CreateTopology, Section 4.3.1, "SPATIAL_REF_SYS 테이블과 공간 참조 시스템"

11.7.4. AddFace

AddFace — 지형에 표면 원시형(face primitive)을 등록하고 그 식별자를 반환합니다.

Synopsis

integer AddFace(varchar *toponame*, geometry *apolygon*, boolean *force_new=false*);

설명: 지형에 표면 원시형(face primitive)을 등록하고 그 식별자를 반환합니다.

새로 추가된 표면의 left_face 및 right_face 항목이 정확한 값을 가지도록 그 외곽선을 형성하는 경계선들 및 표면이 담고 있는 경계선들을 업데이트합니다. 표면이 담고 있는 고립된 노드들도 containing_face 항목이 정확한 값을 가지도록 업데이트할 것입니다.



이 함수는 edge 테이블의 next_left_edge 및 next_right_edge 항목을 이용하지도 설정하지도 않습니다.

대상 지형이 유효하다고 (자체 교차하는 경계선을 담고 있지 않다고) 가정합니다. 기존 경계선들이 폴리곤 외곽선을 완전하게 정의하고 있지 않거나, 또는 폴리곤이 기존 표면과 중첩할 경우 예외가 발생합니다.

apolygon 도형과 동일한 표면이 이미 존재할 경우, force_new 가 (기본값인) 거짓이면 기존 표면의 표면 ID 를 반환하고, force_new 가 참이면 새로 등록된 표면에 새 ID 를 할당할 것입니다.



기존 표면을 새로 등록할 경우 (force_new = true) 경계선, 노드, 관계성 테이블에 있는 기존 표면에 대한 예전 참조들을 해결하기 위한 어떤 작동도 하지 않을 것입니다. 또 기존 표면 레코드의 MBR 항목도 업데이트하지 않을 것입니다. 호출자가 해결해야 할 문제입니다.



apolygon 도형의 srid 와 지형에 대해 정의된 srid 가 동일해야만 합니다. 그렇지 않을 경우 유효하지 않은 공간 참조 시스템 오류가 발생할 것입니다.

2.0.0 버전부터 사용할 수 있습니다.

예시

```
  -- 먼저 generate_series 를 반복자(iterator)로 이용해서 경계선을 추가합니다.
  -- (다음 쿼리는 generate_series 의 최대값 설정 때문에 포인트 10,000 개 미만인 폴리곤에
대해서만 동작할 것입니다.)
  SELECT      topology.AddEdge('ma_topo',      ST_MakeLine(ST_PointN(geom,i),
ST_PointN(geom, i + 1) )) As edgeid
    FROM (SELECT ST_NPoints(geom) AS npt, geom
         FROM
           (SELECT      ST_Boundary(ST_GeomFromText('POLYGON((234896.5      899456.7,234914
899436.4,234946.6 899356.9,234872.5 899328.7,
          234891 899285.4,234992.5 899145, 234890.6 899069,234755.2 899255.4,
          234612.7 899379.4,234776.9 899563.7,234896.5 899456.7))', 26986) ) As geom
        ) As geoms) As facen CROSS JOIN generate_series(1,10000) As i
      WHERE i < npt;

  -- 결과 --
   edgeid
  --------
       3
       4
       5
       6
       7
       8
       9
      10
      11
      12
  (10 rows)
  -- 그 다음 표면을 추가합니다. --

  SELECT topology.AddFace('ma_topo',
     ST_GeomFromText('POLYGON((234896.5   899456.7,234914   899436.4,234946.6
899356.9,234872.5 899328.7,
     234891 899285.4,234992.5 899145, 234890.6 899069,234755.2 899255.4,
     234612.7  899379.4,234776.9  899563.7,234896.5  899456.7))',  26986)  )  As
faceid;
  -- 결과 --
  faceid
  --------
   1
```

참고: AddEdge, CreateTopology, Section 4.3.1, "SPATIAL_REF_SYS 테이블과 공간 참조 시스템"

11.7.5. ST_Simplify

ST_Simplify — 더글러스-패커(Douglas-Peucker) 알고리즘을 통해 입력 TopoGeometry 의 "단순화된" 도형 버전을 반환합니다.

Synopsis

geometry **ST_Simplify**(TopoGeometry *geomA*, float *tolerance*);

설명: 각 구성 요소의 경계선에 더글러스-패커(Douglas-Peucker) 알고리즘을 작용해서 입력 TopoGeometry 의 "단순화된" 도형 버전을 반환합니다.



반환된 도형이 단순하지 않거나, 유효하지 않을 수도 있습니다.
단순성/유효성을 유지하는 데 구성 요소의 경계선을 분할하는 것이 도움이 될 수도 있습니다.

GEOS 모듈로 실행

2.1.0 버전부터 사용할 수 있습니다.

참고: 도형 ST_Simplify, ST_IsSimple, ST_IsValid, ST_ModEdgeSplit

11.8. TopoGeometry 작성자

이 단원에서 새 TopoGeometry 를 생성하기 위한 지형 함수에 대해 설명합니다.

CreateTopoGeom — 지형 요소 배열로부터 새 지형 도형 객체를 생성합니다. tg_type 은 1: [멀티]포인트, 2: [멀티]라인, 3: [멀티]폴리곤, 4: 도형 집합입니다.

toTopoGeom — 단순 도형을 지형 도형으로 변환합니다.

TopoElementArray_Agg — element_id, 유형 배열(topoelement)들의 집합에 대해 topoelementarray 를 반환합니다.

11.8.1. CreateTopoGeom

CreateTopoGeom — 지형 요소 배열로부터 새 지형 도형 객체를 생성합니다. tg_type 은 1: [멀티]포인트, 2: [멀티]라인, 3: [멀티]폴리곤, 4: 도형 집합입니다.

Synopsis

topogeometry toTopoGeom(geometry *geom*, varchar *toponame*, integer *layer_id*, float8 *tolerance*);

topogeometry toTopoGeom(geometry *geom*, topogeometry *topogeom*, float8 *tolerance*);

설명: layer_id 가 지정하는 레이어에 TopoGeometry 객체를 생성하고, toponame 스키마의 관계성 테이블에 새 객체를 등록합니다.

tg_type 은 정수형입니다. 1: [멀티]포인트(점형, punctal), 2: [멀티]라인(선형, lineal), 3: [멀티]폴리곤(면형, areal), 4: 도형 집합. layer_id 는 topology.layer 테이블에 있는 레이어 ID 입니다.

노드들의 집합이 점형 레이어를 형성하고, 경계선들의 집합이 선형 레이어를 형성하고, 표면들의 집합이 면형 레이어를 형성하며, 노드, 경계선, 표면의 혼합 집합이 도형 집합을 형성할 수 있습니다.

구성 요소 배열을 생략하면 텅 빈 TopoGeometry 객체를 생성합니다.

1.? 버전부터 사용할 수 있습니다.

예시: 기존 경계선으로부터 형성

(ST_CreateTopoGeo 에 로드한) 레이어 2(예시의 ri_roads)의 type (2) 라인의 첫 번째 경계선을 대상으로 ri_topo 스키마에 TopoGeometry 를 생성합니다.

```
INSERT INTO ri.ri_roads(road_name, topo) VALUES('Unknown',
topology.CreateTopoGeom('ri_topo',2,2,'{{1,2}}'::topology.topoelementarray);
```

예시: 면형 도형을 가장 잘 추측한 TopoGeometry 로 변환

표면들의 집합으로 형성돼야 하는 도형을 가지고 있다고 해봅시다. 예를 들어 blockgroups 테이블이 있는데 각 블록 그룹의 TopoGeometry 를 알고 싶습니다. 데이터가 완벽하게 정렬돼 있다면, 다음과 같이 할 수 있습니다:

```
-- TopoGeometry 열을 생성 --
SELECT topology.AddTopoGeometryColumn(
      'topo_boston',
      'boston', 'blockgroups', 'topo', 'POLYGON');

-- addtopgeometrycolumn --
1

-- 모든 것이 경계선과 완벽하게 정렬된다고 가정하고
-- 열을 업데이트합니다.
UPDATE boston.blockgroups AS bg
      SET topo = topology.CreateTopoGeom('topo_boston'
      ,3,1
      , foo.bfaces)
FROM (SELECT b.gid,  topology.TopoElementArray_Agg(ARRAY[f.face_id,3]) As
bfaces
      FROM boston.blockgroups As b
         INNER JOIN topo_boston.face As f ON b.geom && f.mbr
      WHERE  ST_Covers(b.geom,  topology.ST_GetFaceGeometry('topo_boston',
f.face_id))
         GROUP BY b.gid) As foo
WHERE foo.gid = bg.gid;
-- 일부 오류를 허용할 정도로 세계가 완벽한 경우는 드뭅니다.
-- 블록 그룹의 경계선으로 생각되는 범위에
-- 50% 이상 들어가는 표면의 개수를 집계합니다.
UPDATE boston.blockgroups AS bg
      SET topo = topology.CreateTopoGeom('topo_boston'
```

```
        ,3,1
        , foo.bfaces)
  FROM (SELECT b.gid,  topology.TopoElementArray_Agg(ARRAY[f.face_id,3]) As
bfaces
        FROM boston.blockgroups As b
           INNER JOIN topo_boston.face As f ON b.geom && f.mbr
        WHERE  ST_Covers(b.geom,  topology.ST_GetFaceGeometry('topo_boston',
f.face_id))
        OR
   (      ST_Intersects(b.geom,   topology.ST_GetFaceGeometry('topo_boston',
f.face_id))
           AND                                ST_Area(ST_Intersection(b.geom,
topology.ST_GetFaceGeometry('topo_boston', f.face_id) ) ) >
              ST_Area(topology.ST_GetFaceGeometry('topo_boston',
f.face_id))*0.5
              )
           GROUP BY b.gid) As foo
  WHERE foo.gid = bg.gid;

  -- TopoGeometry를 표면 및 경계선과 정렬된
  -- 비정규화 도형으로 다시 변환하고자 할 경우
  -- TopoGeometry를 도형으로 형변환합니다.
  -- 진짜 멋진 일은 새 도형들이 이제
  -- TIGER 거리 중심선과 정렬됐다는 사실입니다.
  UPDATE boston.blockgroups SET new_geom = topo::geometry;
```

참고: AddTopoGeometryColumn, toTopoGeom ST_CreateTopoGeo, ST_GetFaceGeometry, TopoElementArray, TopoElementArray_Agg

11.8.2. ToTopoGeom

toTopoGeom — 단순 도형을 지형 도형으로 변환합니다.

Synopsis

topogeometry toTopoGeom(geometry *geom*, varchar *toponame*, integer *layer_id*, float8 *tolerance*);

topogeometry toTopoGeom(geometry *geom*, topogeometry *topogeom*, float8 *tolerance*);

설명: 단순 도형을 TopoGeometry 로 변환합니다.

기저 지형에 입력 도형을 표현해야 하는 지형 원시형을 추가할 것입니다. 이 지형 원시형이 기존 지형을 분할할 수도 있습니다. 분할된 지형들은 relation 테이블에서 출력 TopoGeometry 와 관련될 것입니다.

기존 TopoGeometry 객체는 (topogeom 을 설정할 경우 제외할 수도 있습니다) 자기 형상을 유지할 것입니다.

tolerance 를 설정하면 입력 도형을 기존 원시형에 스냅시키는 데 허용 오차를 이용할 것입니다.

변종 1 의 경우 입력 지형(toponame)의 입력 레이어(layer_id)에 대해 새 TopoGeometry 를 생성할 것입니다.

변종 2 의 경우, 기존 TopoGeometry(toponame)에 작업중 생성된 원시형을 추가할 것입니다. 기존 TopoGeometry 의 최종 형상에 공간을 추가할 수도 있습니다. 새 형상이 예전 형상을 완전히 대체하도록 하려면 clearTopoGeom 을 참조하십시오.

2.0 버전부터 사용할 수 있습니다.

개선 사항: 2.1.0 버전에서 기존 TopoGeometry 를 입력받는 버전을 추가했습니다.

예시

다음은 완전히 자급자족하는 독립적인 작업 흐름(workflow)입니다.

```
 -- 아직 지형을 설정하지 않았다면 다음 쿼리를 실행하십시오.
 -- 어떤 허용 오차도 허용하지 않는 지형을 생성합니다.
 SELECT topology.CreateTopology('topo_boston_test', 2249);
 -- 새 테이블 생성
 CREATE TABLE nei_topo(gid serial primary key, nei varchar(30));
 -- 새 테이블에 TopoGeometry 열을 추가
 SELECT     topology.AddTopoGeometryColumn('topo_boston_test',     'public',
'nei_topo', 'topo', 'MULTIPOLYGON') As new_layer_id;
 new_layer_id
 -----------
 1

 -- 새 TopoGeometry 열을 채우는 데 새 레이어 ID 를 이용
 -- 새 레이어에 허용 오차가 0 인 TopoGeometry 를 추가
 INSERT INTO nei_topo(nei, topo)
 SELECT nei,  topology.toTopoGeom(geom, 'topo_boston_test', 1)
 FROM neighborhoods
 WHERE gid BETWEEN 1 and 15;

 -- 작업 내용을 확인하려면 다음 쿼리를 이용합니다. --
 SELECT * FROM
    topology.TopologySummary('topo_boston_test');

 -- 요약문 --
 Topology topo_boston_test (5), SRID 2249, precision 0
 61 nodes, 87 edges, 35 faces, 15 topogeoms in 1 layers
 Layer 1, type Polygonal (3), 15 topogeoms
  Deploy: public.nei_topo.topo
 -- 모든 TopoGeometry 폴리곤을 10 미터씩 줄입니다.
 UPDATE nei_topo SET topo = ST_Buffer(clearTopoGeom(topo), -10);

 -- 위의 연산으로 남은 아무것도 없는 토지(no-one-lands)를 집계합니다.
 -- GRASS 에서는 이걸 "polygon0 layer"라고 부를 겁니다.
 SELECT ST_GetFaceGeometry('topo_boston_test', f.face_id)
   FROM topo_boston_test.face f
   WHERE f.face_id
 > 0 -- don't consider the universe face
   AND NOT EXISTS ( -- check that no TopoGeometry references the face
     SELECT * FROM topo_boston_test.relation
     WHERE layer_id = 1 AND element_id = f.face_id
   );
```

참고: CreateTopology, AddTopoGeometryColumn, CreateTopoGeom, TopologySummary, clearTopoGeom

11.8.3. TopoElementArray_Agg

TopoElementArray_Agg — element_id, 유형 배열(topoelement)들의 집합에 대해 topoelementarray 를 반환합니다.

Synopsis

topoelementarray TopoElementArray_Agg(topoelement set *tefield*);

설명: TopoElement 집합으로부터 TopoElementArray 를 생성하는 데 쓰입니다.

2.0.0 버전부터 사용할 수 있습니다.

예시

```
SELECT topology.TopoElementArray_Agg(ARRAY[e,t]) As tea
  FROM generate_series(1,3) As e CROSS JOIN generate_series(1,4) As t;
  tea
---------------------------------------------------------------------------
{{1,1},{1,2},{1,3},{1,4},{2,1},{2,2},{2,3},{2,4},{3,1},{3,2},{3,3},{3,4}}
```

참고: TopoElement, TopoElementArray

11.9. TopoGeometry 편집자

이 단원에서 기존 지형을 편집하기 위한 지형 함수에 대해 설명합니다.

clearTopoGeom — 지형 도형의 내용을 지웁니다.

TopoGeom_addElement — TopoGeometry 의 정의에 요소를 추가합니다.

TopoGeom_remElement — TopoGeometry 의 정의에서 요소를 제거합니다.

toTopoGeom — 기존 지형 도형에 도형 형상을 추가합니다.

11.9.1. clearTopoGeom

clearTopoGeom — 지형 도형의 내용을 지웁니다.

Synopsis

topogeometry **clearTopoGeom**(topogeometry *topogeom*);

설명: TopoGeometry 의 내용을 지워서 텅 빈 TopoGeometry 로 바꿉니다. toTopoGeom 함수와 결합해서 기존 객체 및 더 높은 계층 레벨에 있는 모든 의존적인 객체의 형상을 교체하는 데 가장 유용합니다.

2.1 버전부터 사용할 수 있습니다.

예시

```
-- 모든 TopoGeometry 폴리곤을 10 미터씩 줄입니다.
UPDATE nei_topo SET topo = ST_Buffer(clearTopoGeom(topo), -10);
```

참고: toTopoGeom

11.9.2. TopoGeom_addElement

TopoGeom_addElement — TopoGeometry 의 정의에 요소를 추가합니다.

Synopsis

topogeometry **TopoGeom_addElement**(topogeometry *tg*, topoelement *el*);

설명: TopoGeometry 객체의 정의에 TopoElement 를 추가합니다. 해당 요소가 정의에 이미 존재하더라도 오류가 발생하지 않습니다.

2.3 버전부터 사용할 수 있습니다.

예시

```
-- TopoGeometry tg 에 경계선 5 를 추가합니다.
UPDATE mylayer SET tg = TopoGeom_addElement(tg, '{5,2}');
```

참고: TopoGeom_remElement, CreateTopoGeom

11.9.3. TopoGeom_remElement

TopoGeom_remElement — TopoGeometry 의 정의에서 요소를 제거합니다.

Synopsis

topogeometry **TopoGeom_remElement**(topogeometry *tg*, topoelement *el*);

설명: TopoGeometry 객체의 정의에서 TopoElement 를 제거합니다.

2.3 버전부터 사용할 수 있습니다.

예시

```
-- TopoGeometry tg 에서 표면 43 을 제거합니다.
UPDATE mylayer SET tg = TopoGeom_remElement(tg, '{43,3}');
```

참고: TopoGeom_addElement, CreateTopoGeom

11.9.4. ToTopGeom

toTopoGeom — 기존 지형 도형에 도형 형상을 추가합니다.

설명: toTopoGeom 을 참조하십시오.

11.10. TopoGeometry 접근자

GetTopoGeomElementArray — 지형의 요소 및 입력 TopoGeometry 의 유형(원시형 요소)를 담고 있는 topoelementarray (topoelement 의 배열)을 반환합니다.

GetTopoGeomElements — 입력 TopoGeometry 의 지형 element_id, element_type(원시형 요소)을 담고 있는 topoelement 객체 집합을 반환합니다.

11.10.1. GetTopoGeomElementArray

GetTopoGeomElementArray — 지형의 요소 및 입력 TopoGeometry 의 유형(원시형 요소)를 담고 있는 topoelementarray (topoelement 의 배열)을 반환합니다.

Synopsis

topoelementarray GetTopoGeomElementArray(varchar *toponame*, integer *layer_id*, integer *tg_id*);

topoelementarray topoelement GetTopoGeomElementArray(topogeometry *tg*);

설명: 지형의 요소 및 입력 TopoGeometry 의 유형(원시형 요소)를 담고 있는 TopoElementArray 를 반환합니다. 이 함수는 요소들을 데이터셋이 아니라 배열로 반환한다는 점만 제외하면 GetTopoGeomElements 함수와 비슷합니다.

tg_id 는 topology.layer 테이블에 있는 layer_id 가 지정하는 레이어 안의 지형이 가지고 있는 TopoGeometry 객체의 TopoGeometry ID 입니다.

1.? 버전부터 사용할 수 있습니다.

참고: GetTopoGeomElements, TopoElementArray

11.10.2. GetTopoGeomElements

GetTopoGeomElements — 입력 TopoGeometry 의 지형 element_id, element_type(원시형 요소)을 담고 있는 topoelement 객체 집합을 반환합니다.

Synopsis

setof topoelement GetTopoGeomElements(varchar *toponame*, integer *layer_id*, integer *tg_id*);

setof topoelement GetTopoGeomElements(topogeometry *tg*);

설명: toponame 스키마 안의 입력 TopoGeometry 객체에 대해 element_id, element_type (topoelement) 집합을 반환합니다.

tg_id 는 topology.layer 테이블에 있는 layer_id 가 지정하는 레이어 안의 지형이 가지고 있는 TopoGeometry 객체의 TopoGeometry ID 입니다.

2.0.0 버전부터 사용할 수 있습니다.

참고: GetTopoGeomElementArray, TopoElement, TopoGeom_addElement, TopoGeom_remElement

11.11. TopoGeometry 출력물

AsGML — TopoGeometry 의 GML 표현식을 반환합니다.

AsTopoJSON — TopoGeometry 의 TopoJSON 표현식을 반환합니다.

11.11.1. AsGML

AsGML — TopoGeometry 의 GML 표현식을 반환합니다.

Synopsis

text AsGML(topogeometry *tg*);

text AsGML(topogeometry *tg*, text *nsprefix_in*);

text AsGML(topogeometry *tg*, regclass *visitedTable*);

text AsGML(topogeometry *tg*, regclass *visitedTable*, text *nsprefix*);

text AsGML(topogeometry *tg*, text *nsprefix_in*, integer *precision*, integer *options*);

text AsGML(topogeometry *tg*, text *nsprefix_in*, integer *precision*, integer *options*, regclass *visitedTable*);

text AsGML(topogeometry *tg*, text *nsprefix_in*, integer *precision*, integer *options*, regclass *visitedTable*, text *idprefix*);

text AsGML(topogeometry *tg*, text *nsprefix_in*, integer *precision*, integer *options*, regclass *visitedTable*, text *idprefix*, int *gmlversion*);

설명: TopoGeometry 의 GML 표현식을 GML3 버전 서식으로 반환합니다. nsprefix_in 을 따로 설정하지 않을 경우 gml 접두사를 이용합니다. nsprefix 에 대해 텅 빈 문자열을 넘겨줄 경우 무자격(non-qualified) 네임스페이스를 반환합니다. 정확도(기본값 15) 및 옵션(기본값 1) 파라미터를 설정한 경우, 손대는 일 없이 ST_AsGML 함수에 대한 기저 호출에 그대로 넘겨줍니다.

visitedTable 파라미터는 접근한 노드 및 경계선 요소를 추적하는 데 쓰이기 때문에, 설정할 경우, 정의를 복제하는 게 아니라 교차 참조(xlink:xref)를 이용합니다. 정수형 항목 'element_type' 및 'element_id' (적어도) 2 개가 테이블에 있어야 합니다. 함수 호출자는 입력 테이블에 대해 읽기 및 쓰기 권한을 가지고 있어야만

합니다. 최상의 성능을 내려면, element_type 과 element_id 에, 이 순서대로, 인덱스를 정의해야 합니다. 항목들에 유일한 제약조건을 추가하면 이 인덱스를 자동적으로 생성할 것입니다. 다음은 그 예시입니다:

```
CREATE TABLE visited (
  element_type integer, element_id integer,
  unique(element_type, element_id)
);
```

idprefix 파라미터를 설정할 경우, 노드 및 경계선 태그 식별자에 접두사를 붙일 것입니다.

gmlver 파라미터를 설정할 경우, 기저 ST_AsGML 함수에 해당 파라미터를 넘겨줄 것입니다. 기본값은 3 입니다.

2.0.0 버전부터 사용할 수 있습니다.

예시

다음 예시는 CreateTopoGeom 에서 생성한 지형 도형을 사용합니다.

```
SELECT topology.AsGML(topo) As rdgml
 FROM ri.roads
 WHERE road_name = 'Unknown';

-- rdgml--
<gml:TopoCurve>
  <gml:directedEdge>
    <gml:Edge gml:id="E1">
      <gml:directedNode orientation="-">
        <gml:Node gml:id="N1"/>
      </gml:directedNode>
      <gml:directedNode
></gml:directedNode>
      <gml:curveProperty>
        <gml:Curve srsName="urn:ogc:def:crs:EPSG::3438">
          <gml:segments>
            <gml:LineStringSegment>
              <gml:posList srsDimension="2"
>384744 236928 384750 236923 384769 236911 384799 236895 384811 236890
                384833 236884 384844 236882 384866 236881 384879 236883 384954 236898 385087
236932 385117 236938
                385167 236938 385203 236941 385224 236946 385233 236950 385241 236956 385254
236971
                385260 236979 385268 236999 385273 237018 385273 237037 385271 237047 385267
237057 385225 237125
                385210 237144 385192 237161 385167 237192 385162 237202 385159 237214 385159
237227 385162 237241
                385166 237256 385196 237324 385209 237345 385234 237375 385237 237383 385238
237399 385236 237407
                385227 237419 385213 237430 385193 237439 385174 237451 385170 237455 385169
237460 385171 237475
                385181 237503 385190 237521 385200 237533 385206 237538 385213 237541 385221
237542 385235 237540 385242 237541
                385249 237544 385260 237555 385270 237570 385289 237584 385292 237589 385291
237596 385284 237630</gml:posList>
            </gml:LineStringSegment>
          </gml:segments>
        </gml:Curve>
      </gml:curveProperty>
    </gml:Edge>
  </gml:directedEdge>
```

```
 </gml:TopoCurve
 >
```

이전 예시와 동일하지만 네임스페이스를 쓰지 않습니다.

```
 SELECT topology.AsGML(topo,'') As rdgml
   FROM ri.roads
   WHERE road_name = 'Unknown';

 -- rdgml--
 <TopoCurve>
    <directedEdge>
       <Edge id="E1">
          <directedNode orientation="-">
             <Node id="N1"/>
          </directedNode>
          <directedNode
 ></directedNode>
          <curveProperty>
             <Curve srsName="urn:ogc:def:crs:EPSG::3438">
                <segments>
                   <LineStringSegment>
                      <posList srsDimension="2"
 >384744 236928 384750 236923 384769 236911 384799 236895 384811 236890
                      384833  236884  384844  236882  384866  236881  384879
236883 384954 236898 385087 236932 385117 236938
                      385167  236938  385203  236941  385224  236946  385233
236950 385241 236956 385254 236971
                      385260  236979  385268  236999  385273  237018  385273
237037 385271 237047 385267 237057 385225 237125
                      385210  237144  385192  237161  385167  237192  385162
237202 385159 237214 385159 237227 385162 237241
                      385166  237256  385196  237324  385209  237345  385234
237375 385237 237383 385238 237399 385236 237407
                      385227  237419  385213  237430  385193  237439  385174
237451 385170 237455 385169 237460 385171 237475
                      385181  237503  385190  237521  385200  237533  385206
237538 385213 237541 385221 237542 385235 237540 385242 237541
                      385249  237544  385260  237555  385270  237570  385289
237584 385292 237589 385291 237596 385284 237630</posList>
                    </LineStringSegment>
                </segments>
             </Curve>
          </curveProperty>
       </Edge>
    </directedEdge>
 </TopoCurve
 >
```

참고: CreateTopoGeom, ST_CreateTopoGeo

11.11.2. AsTopoJSON

AsTopoJSON — TopoGeometry 의 TopoJSON 표현식을 반환합니다.

Synopsis

text **AsTopoJSON**(topogeometry *tg*, regclass *edgeMapTable*

설명: TopoGeometry 의 TopoJSON 표현식을 반환합니다. edgeMapTable 이 NULL 이 아닐 경우, 경계선 식별자를 원호(arc) 인덱스에 매핑하는 탐색/저장(lookup/storage)으로 쓰일 것입니다. 최종 문서에 조밀한(compact) "원호" 배열을 사용할 수 있게 하기 위해서입니다.

테이블을 설정할 경우, 테이블이 "순차(serial)" 유형 "arc_id" 항목과 정수형 "edge_id" 항목을 가지고 있어야 합니다. 코드가 "edge_id"에 대해 테이블을 쿼리할 것이므로 해당 항목에 인덱스를 추가하는 편이 좋습니다.



TopoJSON 출력물에서는 원호 인덱스가 0-기반이지만 "edgeMapTable" 테이블에서는 1-기반입니다.

완전한 TopoJSON 문서는, 이 함수가 반환하는 토막 정보(snippet)에 더해, 일부 헤더와 함께 실제 원호를 담고 있어야 할 것입니다. TopoJSON 사양서 를 참조하십시오.

2.1.0 버전부터 사용할 수 있습니다.

예시

```
  CREATE TEMP TABLE edgemap(arc_id serial, edge_id int unique);

  -- 헤더
  SELECT '{ "type": "Topology", "transform": { "scale": [1,1], "translate":
[0,0] }, "objects": {'

  -- 객체
  UNION ALL SELECT '"' || feature_name || '": ' || AsTopoJSON(feature,
'edgemap')
  FROM features.big_parcels WHERE feature_name = 'P3P4';

  -- 원호
  WITH edges AS (
    SELECT m.arc_id, e.geom FROM edgemap m, city_data.edge e
    WHERE e.edge_id = m.edge_id
  ), points AS (
    SELECT arc_id, (st_dumppoints(geom)).* FROM edges
  ), compare AS (
    SELECT p2.arc_id,
          CASE WHEN p1.path IS NULL THEN p2.geom
              ELSE ST_Translate(p2.geom, -ST_X(p1.geom), -ST_Y(p1.geom))
          END AS geom
    FROM points p2 LEFT OUTER JOIN points p1
    ON ( p1.arc_id = p2.arc_id AND p2.path[1] = p1.path[1]+1 )
    ORDER BY arc_id, p2.path
  ), arcsdump AS (
    SELECT arc_id, (regexp_matches( ST_AsGeoJSON(geom), '\[.*\]'))[1] as t
    FROM compare
  ), arcs AS (
    SELECT arc_id, '[' || array_to_string(array_agg(t), ',') || ']' as a FROM
arcsdump
    GROUP BY arc_id
    ORDER BY arc_id
  )
  SELECT '}, "arcs": [' UNION ALL
  SELECT array_to_string(array_agg(a), E',\n') from arcs
```

```
  -- 꼬리말
  UNION ALL SELECT ']}'::text as t;

  -- 결과:
  { "type": "Topology", "transform": { "scale": [1,1], "translate": [0,0] },
"objects": {
  "P3P4": { "type": "MultiPolygon", "arcs": [[[-1]],[[6,5,-5,-4,-3,1]]]}
  }, "arcs": [
   [[25,30],[6,0],[0,10],[-14,0],[0,-10],[8,0]],
   [[35,6],[0,8]],
   [[35,6],[12,0]],
   [[47,6],[0,8]],
   [[47,14],[0,8]],
   [[35,22],[12,0]],
   [[35,14],[0,8]]
   ]}
```

개선 사항: 2.2.1 버전부터 점형(puntal) 입력을 지원하기 시작했습니다.

참고: ST_AsGeoJSON

11.12. 지형 공간 관계성

이 단원에서 TopoGeometry 와 지형 원시형 사이의 관계를 확인하는 데 쓰이는 지형 함수에 대해 설명합니다.

Equals — 두 TopoGeometry 가 동일한 지형 원시형으로 이루어졌을 경우 참을 반환합니다.

Intersects — 두 TopoGeometry 에서 나온 원시형의 쌍 가운데 하나라도 교차할 경우 참을 반환합니다.

11.12.1. Equals

Equals — 두 TopoGeometry 가 동일한 지형 원시형으로 이루어졌을 경우 참을 반환합니다.

Synopsis

boolean **Equals**(topogeometry *tg1*, topogeometry *tg2*);

설명: 두 TopoGeometry 가 동일한 지형 원시형(표면, 경계선, 노드)으로 이루어졌을 경우 참을 반환합니다.



이 함수는 도형 집합인 TopoGeometry 를 지원하지 않습니다. 또 서로 다른 지형으로 이루어진 두 TopoGeometry 를 비교하지 못 합니다.

1.1.0 버전부터 사용할 수 있습니다.

✔ 이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

참고: GetTopoGeomElements, ST_Equals

11.12.2. Intersects

Intersects — 두 TopoGeometry 에서 나온 원시형의 쌍 가운데 하나라도 교차할 경우 참을 반환합니다.

Synopsis

boolean **Intersects**(topogeometry *tg1*, topogeometry *tg2*);

설명: 두 TopoGeometry 에서 나온 원시형의 쌍 가운데 하나라도 교차할 경우 참을 반환합니다.



이 함수는 도형 집합인 TopoGeometry 를 지원하지 않습니다. 또 서로 다른 지형으로 이루어진 두 TopoGeometry 를 비교하지 못 합니다. 또한 현재 계층 TopoGeometry(다른 TopoGeometry 들로 구성된 TopoGeometry)도 지원하지 않습니다.

1.1.0 버전부터 사용할 수 있습니다.



이 함수는 3 차원을 지원하며 Z-인덱스를 삭제하지 않습니다.

참고: ST_Intersects

Chapter 12. 주소 표준화 도구

이 도구는 PAGC standardizer 의 한 갈래(fork)입니다(이 갈래의 원본 코드는 PAGC PostgreSQL 주소 표준화 도구 였습니다).

주소 표준화 도구는 단일 라인 주소 파싱 도구로, 주소를 입력받아 테이블에 저장된 일련의 규칙 및 도우미 어휘 목록(lexicon; lex) 테이블 과 지명 색인(gazetteer; gaz) 테이블 에 기반해서 정규화합니다.

CREATE EXTENSION address_standardizer; 명령으로 설치할 수 있는 address_standardizer 라는 단일 PostgreSQL 확장 프로그램 라이브러리로 코드를 빌드합니다. address_standardizer 확장 프로그램에 더해, address_standardizer_data_us 라는 샘플 데이터 확장 프로그램도 빌드하는데, 미국 데이터에 대한 지명 색인, 어휘 목록, 규칙 테이블을 담고 있습니다. CREATE EXTENSION address_standardizer_data_us; 명령으로 이 확장 프로그램을 설치할 수 있습니다.

PostGIS extensions/address_standardizer 에서 이 확장 프로그램을 위한 코드를 찾을 수 있으며, 현재 독립적으로 돼 있습니다.

설치에 대한 지침을 보고 싶다면 Section 2.7, “주소 표준화 도구 설치 및 활용” 를 참조하십시오.

12.1. 파싱 도구의 작동 방식

파싱 도구는 오른쪽에서 왼쪽 방향으로 먼저 우편번호, 주/도, 시군구에 해당하는 거시적(macro) 요소를 탐색한 다음, 가구의 번지수 또는 교차점 또는 랜드마크 가운데 어떤 것을 처리하고 있는지 결정하기 위한 미시적(micro) 요소를 탐색합니다. 현재 국가 코드 또는 국가명을 탐색하지는 않지만, 향후 추가될 수도 있습니다.

국가 코드 — 미국 또는 캐나다의 주/도, 미국 또는 캐나다의 우편번호를 바탕으로 미국 또는 캐나다로 가정합니다.

우편번호/집코드(zip code) — 펄(Perl) 호환 정규 표현식을 이용해서 우편번호를 인식합니다. 이 정규 표현식은 현재 parseaddress-api.c 파일에 담겨 있고, 필요한 경우 상대적으로 쉽게 변경할 수 있습니다.

주/도 — 펄(Perl) 호환 정규 표현식을 이용해서 우편번호를 인식합니다. 이 정규 표현식은 현재 parseaddress-api.c 파일에 담겨 있는데, 향후 유지보수를 더 쉽게 하기 위해 "includes"로 이동할 수도 있습니다.

12.2. 주소 표준화 도구 유형

이 단원에서 주소 표준화 도구 확장 프로그램이 설치한 PostgreSQL 데이터 유형을 소개합니다. 사용자 자신의 함수를 설계할 때 특히 중요한 이 유형들의 형변환 습성(cast behavior)을 설명한다는 점에 주의하십시오.

stdaddr — 주소의 요소들로 이루어진 합성 유형입니다. standardize_address 함수가 이 유형을 반환합니다.

설명: 주소의 요소들로 이루어진 합성 유형입니다. standardize_address 함수가 이 유형을 반환합니다. PAGC Postal Attributes 에서 요소들에 대한 몇몇 설명을 빌려왔습니다.

규칙 테이블 에서 출력 참조 번호를 표시하는 토큰 번호를 찾아볼 수 있습니다.

✓ 이 방법을 쓰려면 address_standardizer 확장 프로그램이 필요합니다.

buildingbuilding — 문자형(토큰 번호 0)입니다: 건물 번호 또는 건물명을 참조합니다. 파싱되지 않은 건물 식별자 및 유형입니다. 주소 대부분의 경우 일반적으로 비어 있습니다.

house_num — 문자형(토큰 번호 1)입니다: 도로의 번지수입니다. 예: 75 State Street 의 75 번지

predir — 문자형(토큰 번호 2)입니다: North, South, East, West 등과 같이 방향을 나타내는 도로명 접두사(STREET NAME PRE-DIRECTIONAL)입니다.

qual — 문자형(토큰 번호 3)입니다: 도로명 전치수식어(STREET NAME PRE-MODIFIER)입니다. 예: 3715 OLD HIGHWAY 99 에서 OLD

pretype — 문자형(토큰 번호 4)입니다: 도로 접두사 유형(STREET PREFIX TYPE)입니다.

name — 문자형(토큰 번호 5)입니다: 도로명(STREET NAME)입니다.

suftype — 문자형(토큰 번호 6)입니다: St, Ave, Cir 와 같은 도로 접미사 유형(STREET POST TYPE)입니다. 도로명의 몸통 뒤에 붙는 도로 유형을 뜻합니다. 예: 75 State Street 에서 STREET

sufdir — 문자형(토큰 번호 7)입니다: 방향을 나타내는 도로 접미사(STREET POST-DIRECTIONAL)입니다. 도로명 뒤에 붙어 방향을 나타내는 수식어입니다. 예: 3715 TENTH AVENUE WEST 에서 WEST

ruralroute — 문자형(토큰 번호 8)입니다: 지방 우편물 배달 노선(RURAL ROUTE)입니다. 예: RR 7 에서 7

extra — 문자형입니다: 층수 같은 추가 정보입니다.

city — 문자형(토큰 번호 10)입니다: 예: 보스턴 시

state — 문자형(토큰 번호 11)입니다: 예: 매사추세츠 주

country — 문자형(토큰 번호 12)입니다: 예: USA

postcode — 문자형(토큰 번호 13) 우편번호(postal code, zip code)입니다: 예: 02109

box — 문자형(토큰 번호 14, 15) 사서함 번호(POSTAL BOX NUMBER)입니다: 예: 02109

unit — 문자형(토큰 번호 17) 아파트 동수 또는 호수입니다: 예: APT 3B 에서 3B

12.3. 주소 표준화 도구 테이블

이 단원에서 address_standardizer 가 주소를 정규화하기 위해 이용하는 PostgreSQL 테이블 서식 목록을 소개합니다. 이 테이블들의 명칭을 여기에 소개된 대로 명명할 필요는 없다는 점에 주의하십시오. 예를 들어 각 국가 또는 사용자 지정 지오코딩 도구에 대해 서로 다른 어휘 목록(lex), 지명 색인(gaz), 규칙 테이블을 쓸 수 있습니다. 주소 표준화 도구 함수에 이 테이블들의 명칭을 넘겨줍니다.

패키징된 address_standardizer_data_us 확장 프로그램이 미국 주소를 표준화하기 위한 데이터를 담고 있습니다.

규칙 테이블 — 규칙 테이블은 주소 입력 배열 토큰을 표준화된 출력 배열 토큰에 매핑하는 일련의 규칙들을 담고 있습니다. 입력 토큰들의 집합, 그 뒤에 -1(종결자; terminator), 그 뒤에 출력 토큰들의 집합, 그 뒤에 -1, 그 뒤에 규칙의 종류를 표시하는 번호, 그 뒤에 규칙의 순위를 나열해서 하나의 규칙을 정의합니다.

어휘 목록 테이블어휘 목록(lex) 테이블은 영숫자 입력을 분류하고, 해당 입력을 (1) 입력 토큰(the section called “입력 토큰” 참조) 및 (2) 표준화된 표현식과 연관짓는 데 쓰입니다.

지명 색인 테이블지명 색인(gaz) 테이블은 지명을 표준화하고, 해당 입력을 (1) 입력 토큰(the section called “입력 토큰” 참조) 및 (2) 표준화된 표현식과 연관짓는 데 쓰입니다.

규칙 테이블 — 규칙 테이블은 주소 입력 배열 토큰을 표준화된 출력 배열 토큰에 매핑하는 일련의 규칙들을 담고 있습니다. 입력 토큰들의 집합, 그 뒤에 -1(종결자; terminator), 그 뒤에 출력 토큰들의 집합, 그 뒤에 -1, 그 뒤에 규칙의 종류를 표시하는 번호, 그 뒤에 규칙의 순위를 나열해서 하나의 규칙을 정의합니다.

설명: 규칙 테이블은 최소한 다음 열들을 가지고 있어야만 하지만, 사용자의 용처에 따라 더 많은 열을 추가할 수도 있습니다.

id — 테이블의 기본 키

rule — 규칙을 표시하는 텍스트 항목입니다. PAGC Address Standardizer Rule records 에서 자세히 설명하고 있습니다.

하나의 규칙은 입력 토큰을 나타내는 음수가 아닌 정수들의 집합, 해당 집합을 종결하는 -1, 그 다음에 우편 속성을 나타내는 동일한 개수의 음수가 아닌 정수들의 집합, 해당 집합을 종결하는 -1, 그 다음에 규칙 유형을 나타내는 정수, 그 다음에 규칙 순위를 나타내는 정수로 이루어집니다. 규칙의 순위는 (최저) 0 부터 (최고) 17 까지입니다.

따라서, 예를 들어 규칙 2 0 2 22 3 -1 5 5 6 7 3 -1 2 6 은 출력 토큰 배열 TYPE NUMBER TYPE DIRECT QUALIF 에, 해당 출력 토큰 배열은 출력 배열 STREET STREET SUFTYP SUFDIR QUALIF 에 매핑됩니다. 규칙은 6 순위의 ARC_C 규칙입니다.

stdaddr 에서 해당 출력 토큰에 대한 번호 목록을 소개하고 있습니다.

입력 토큰

각 규칙은 입력 토큰 집합과 그 다음의 종결자 -1 로 시작합니다. PAGC Input Tokens 에서 발췌한 유효한 입력 토큰들은 다음과 같습니다:

서식 기반 입력 토큰

AMPERS — (13). 앰퍼샌드(&)는 단어 "and"를 축약하는 데 자주 쓰입니다.

DASH — (9). 구두법(句讀法; punctuation) 문자입니다.

DOUBLE — (21). 문자 2 개의 배열입니다. 식별자로 종종 쓰입니다.

FRACT — (25). 분수는 가끔 가구 번호 또는 동호수에 쓰입니다.

MIXED — (23). 문자와 숫자 모두를 담고 있는 영숫자 스트링입니다. 식별자로 쓰입니다.

NUMBER — (0). 숫자 스트링입니다.

ORD — (15). "First" 또는 "1st" 같은 표현입니다. 도로명에 자주 쓰입니다.

ORD — (18). 단일 문자입니다.

WORD — (1). 임의의 길이의 문자열입니다. 단일 문자는 SINGLE 도, WORD 도 될 수 있습니다.

기능 기반 입력 토큰

BOXH — (14). 우편사서함을 표시하는 데 쓰이는 단어입니다. 예를 들어 Box 또는 PO Box 처럼 말이죠.

BUILDH — (19). 일반적으로 접두사의 형태인, 건물 또는 건물군을 표시하는 데 쓰이는 단어입니다. 예: Tower 7A 에서 Tower

BUILDT — 일반적으로 접미사의 형태인, 건물 또는 건물군을 표시하는 데 쓰이는 단어 및 약어입니다. 예: Shopping Centre

DIRECT — (22). 방향을 표시하는 데 쓰이는 단어입니다. 예: North

MILE — (20). 거리표(어떤 지점까지의 거리를 나타내는 도로가의 푯말; milepost) 주소를 표시하는 데 쓰이는 단어입니다.

ROAD — (6). 고속도로와 도로를 표시하는 데 쓰이는 단어 및 약어입니다. 예: Interstate 5 에서 Interstate

RR — (8). 지방 우편물 배달 노선(rural route)을 표시하는 데 쓰이는 단어 및 약어입니다. RR.

TYPE — (2). 도로 유형을 표시하는 데 쓰이는 단어 및 약어입니다. 예: ST 또는 AVE

UNITH — (16). 내부 하위주소를 표시하는 데 쓰이는 단어 및 약어입니다. 예: APT 또는 UNIT

우편 유형 입력 토큰

QUINT — (28). 다섯 자리 숫자입니다. 집코드(Zip Code)를 식별합니다.

QUAD — (29). 네 자리 숫자입니다. ZIP4 를 식별합니다.

PCH — (27). 문자, 숫자, 문자 순서로 이루어진 문자 3 개의 배열입니다. 캐나다 우편 번호의 처음 세 문자인 FSA 를 식별합니다.

PCT — (26). 숫자, 문자, 숫자 순서로 이루어진 문자 3 개의 배열입니다. 캐나다 우편 번호의 마지막 세 문자인 LDU 를 식별합니다.

불용어(不用語; stopword)

STOPWORD 는 WORD 와 결합합니다. 규칙 상 복수의 WORD 및 STOPWORD 스트링을 단일 WORD 토큰으로 표현할 것입니다.

STOPWORD

(7). 어휘 상 중요도가 낮아 파싱 작업시 생략할 수 있는 단어입니다. 예: THE

출력 토큰

첫 번째 -1(종결자) 다음에 출력 토큰 및 그 순서가 오고, 그 다음에 종결자 -1 이 따라옵니다. stdaddr 에서 많은 해당 출력 토큰들의 목록을 소개하고 있습니다. 규칙의 유형에 따라 사용할 수 있는 출력 토큰들도 달라집니다. the section called “규칙 유형 및 순위” 에서 각 규칙 유형에 대해 유효한 출력 토큰 목록을 볼 수 있습니다.

규칙 유형 및 순위

규칙을 구성하는 마지막 부분은 다음 가운데 하나로 표시되는 규칙 유형과 그 다음에 오는 규칙 순위입니다. 규칙의 순위는 (최저) 0 부터 (최고) 17 까지입니다.

MACRO_C

(토큰 번호 = "0"). PLACE STATE ZIP 같은 MACRO 절을 파싱하기 위한 규칙 클래스입니다.

MACRO_C 출력 토큰 (http://www.pagcgeo.org/docs/html/pagc-12.html#--r-typ-- 에서 발췌)

CITY — (토큰 번호 "10"). 예: "Albany"

STATE — (토큰 번호 "11"). 예: "NY"

NATION — (토큰 번호 "12"). 이 속성은 대부분의 참조 파일에서 쓰이지 않습니다. 예: "USA"

POSTAL — (토큰 번호 "13"). (SADS 요소 "ZIP CODE", "PLUS 4"). 미국 집코드 및 캐나다 우편 번호 모두 이 속성을 이용합니다.

MICRO_C

(토큰 번호 = "1"). (가구, 도로, sufdir, predir, pretyp, suftype, qualif 와 같은) 완전한 MICRO 절을 파싱하기 위한 규칙 클래스입니다(예: ARC_C 및 CIVIC_C). 빌드 시기에는 이 규칙들을 이용하지 않습니다.

MICRO_C 출력 토큰 (http://www.pagcgeo.org/docs/html/pagc-12.html#--r-typ-- 에서 발췌).

HOUSE — 문자형(토큰 번호 1)입니다: 도로의 번지수입니다. 예: 75 State Street 의 75 번지

Predir — 문자형(토큰 번호 2)입니다: North, South, East, West 등과 같이 방향을 나타내는 도로명 접두사(STREET NAME PRE-DIRECTIONAL)입니다.

Qual — 문자형(토큰 번호 3)입니다: 도로명 전치수식어(STREET NAME PRE-MODIFIER)입니다. 예: 3715 OLD HIGHWAY 99 에서 OLD

Pretype — 문자형(토큰 번호 4)입니다: 도로 접두사 유형(STREET PREFIX TYPE)입니다.

Street — 문자형(토큰 번호 5)입니다: 도로명(STREET NAME)입니다.

Suftype — 문자형(토큰 번호 6)입니다: St, Ave, Cir 와 같은 도로 접미사 유형(STREET POST TYPE)입니다. 도로명의 몸통 뒤에 붙는 도로 유형을 뜻합니다. 예: 75 State Street 에서 STREET

Sufdir — 문자형(토큰 번호 7)입니다: 방향을 나타내는 도로 접미사(STREET POST-DIRECTIONAL)입니다. 도로명 뒤에 붙어 방향을 나타내는 수식어입니다. 예: 3715 TENTH AVENUE WEST 에서 WEST

ARC_C

(토큰 번호 = "2"). HOUSE 속성을 제외하고 MICRO 절을 파싱하기 위한 규칙 클래스입니다. 따라서 HOUSE 토큰을 뺀 MICRO_C 출력 토큰 집합을 이용합니다.

CIVIC_C

(토큰 번호 = "3"). HOUSE 속성을 파싱하기 위한 규칙 클래스입니다.

EXTRA_C

(토큰 번호 = "4"). EXTRA 속성 - 지오코딩에서 제외된 속성 - 을 파싱하기 위한 규칙 클래스입니다. 빌드 시기에는 이 규칙들을 이용하지 않습니다.

EXTRA_C 출력 토큰 (http://www.pagcgeo.org/docs/html/pagc-12.html#--r-typ-- 에서 발췌).

BLDNG — (토큰 번호 0): 파싱되지 않은 건물 식별자 및 유형입니다.

BOXH — (토큰 번호 14): BOX 3B 에서 BOX

BOXT — (토큰 번호 15): BOX 3B 에서 3B

RR — (토큰 번호 8): RR 7 에서 RR

UNITH — (토큰 번호 16): APT 3B 에서 APT

UNITT — (토큰 번호 17): APT 3B 에서 3B

UNKNWN — (토큰 번호 9): 따로 분류되지 않은 출력물입니다.

어휘 목록 테이블 — 어휘 목록(lex) 테이블은 영숫자 입력을 분류하고, 해당 입력을 (1) 입력 토큰(the section called "입력 토큰" 참조) 및 (2) 표준화된 표현식과 연관짓는 데 쓰입니다.

설명: 어휘 목록(lexicon) 테이블은 영숫자 입력을 분류하고, 해당 입력을 (1) 입력 토큰(the section called "입력 토큰" 참조) 및 (2) 표준화된 표현식과 연관짓는 데 쓰입니다. 이 테이블에서, 예를 들어, 영단어 ONE 이 stdword 1 로 매핑된 것을 찾아볼 수 있습니다.

어휘 목록 테이블은 최소한 다음 열을 가지고 있어야 합니다. 사용자 지정 열을 추가할 수도 있습니다.

id — 테이블의 기본 키

seq — 정수형: 정의 번호?

word — 문자형: 입력 단어

stdword — 문자형: 표준화된 대체 단어

token — 정수형: 단어의 유형을 나타냅니다. 단어가 해당 맥락에서 쓰였을 경우에만 대체될 것입니다. PAGC Tokens 를 참조하십시오.

지명 색인 테이블 — 지명 색인(gaz) 테이블은 지명을 표준화하고, 해당 입력을 (1) 입력 토큰(the section called "입력 토큰" 참조) 및 (2) 표준화된 표현식과 연관짓는 데 쓰입니다.

설명: 지명 색인(gazetteer) 테이블은 지명을 표준화하고, 해당 입력을 (1) 입력 토큰(the section called "입력 토큰" 참조) 및 (2) 표준화된 표현식과 연관짓는 데 쓰입니다. 예를 들어, 미국의 경우, 주명(州名) 및 관련 약어를 가진 테이블을 로드할 수도 있습니다.

지명 색인 테이블은 최소한 다음 열을 가지고 있어야 합니다. 사용자의 용처에 따라 더 많은 열을 추가할 수도 있습니다.

id — 테이블의 기본 키

seq — 정수형: 정의 번호? - 단어의 해당 인스턴스에 쓰이는 식별자

word — 문자형: 입력 단어

stdword — 문자형: 표준화된 대체 단어

token — 정수형: 단어의 유형을 나타냅니다. 단어가 해당 맥락에서 쓰였을 경우에만 대체될 것입니다. PAGC Tokens 를 참조하십시오.

12.4. 주소 표준화 도구 함수

parse_address — 한 줄로 된 주소를 입력받아 부분들로 분해합니다.

standardize_address — 어휘 목록, 지명 색인, 규칙 테이블을 활용해서 입력 주소의 stdaddr 형식을 반환합니다.

12.4.1. parse_address

parse_address — 한 줄로 된 주소를 입력받아 부분들로 분해합니다.

Synopsis

record **parse_address**(text *address*);

설명: 주소를 입력받아 num, street, street2, address1, city, state, zip, zipplus, country 항목들로 구성된 출력 레코드를 반환합니다.

2.2.0 버전부터 사용할 수 있습니다.

✔ 이 방법을 쓰려면 address_standardizer 확장 프로그램이 필요합니다.

예시

```
단일 주소
SELECT num, street, city, zip, zipplus
      FROM parse_address('1 Devonshire Place, Boston, MA 02109-1234') AS a;
num |     street      | city  |  zip  | zipplus
-----+-----------------+--------+-------+---------
 1   | Devonshire Place | Boston | 02109 | 1234
주소 테이블
-- 기본 테이블
```

```
CREATE TABLE places(addid serial PRIMARY KEY, address text);

INSERT INTO places(address)
VALUES ('529 Main Street, Boston MA, 02129'),
 ('77 Massachusetts Avenue, Cambridge, MA 02139'),
 ('25 Wizard of Oz, Walaford, KS 99912323'),
 ('26 Capen Street, Medford, MA'),
 ('124 Mount Auburn St, Cambridge, Massachusetts 02138'),
 ('950 Main Street, Worcester, MA 01610');

 -- 주소 파싱하기
 -- 모든 항목을 원한다면 (a).*를 쓰면 됩니다.
SELECT addid, (a).num, (a).street, (a).city, (a).state, (a).zip, (a).zipplus
FROM (SELECT addid, parse_address(address) As a
 FROM places) AS p;
addid | num |       street       |   city    | state |  zip  | zipplus
-------+-----+--------------------+-----------+-------+-------+---------
    1 | 529 | Main Street        | Boston    | MA    | 02129 |
    2 | 77  | Massachusetts Avenue | Cambridge | MA    | 02139 |
    3 | 25  | Wizard of Oz       | Walaford  | KS    | 99912 | 323
    4 | 26  | Capen Street       | Medford   | MA    |       |
    5 | 124 | Mount Auburn St    | Cambridge | MA    | 02138 |
    6 | 950 | Main Street        | Worcester | MA    | 01610 |
(6 rows)
```

12.4.2. standardize_address

standardize_address — 어휘 목록, 지명 색인, 규칙 테이블을 활용해서 입력 주소의 stdaddr 형식을 반환합니다.

Synopsis

stdaddr **standardize_address**(text *lextab*, text *gaztab*, text *rultab*, text *address*);

stdaddr **standardize_address**(text *lextab*, text *gaztab*, text *rultab*, text *micro*, text *macro*);

설명: 어휘 목록 테이블, 지명 색인 테이블, 규칙 테이블 테이블명 및 주소를 활용해서 입력 주소의 stdaddr 형식을 반환합니다.

변종 1: 주소를 단일 행으로 입력받습니다.

변종 2: 주소를 두 부분으로 입력받습니다. house_num street 같은 우편 주소의 표준 첫 번째 행으로 이루어진 micro 절과, city, state postal_code country 같은 우편 주소의 표준 두 번째 행으로 이루어진 macro 절입니다.

2.2.0 버전부터 사용할 수 있습니다.

✔ 이 방법을 쓰려면 address_standardizer 확장 프로그램이 필요합니다.

예시

address_standardizer_data_us 확장 프로그램 이용

```
CREATE EXTENSION address_standardizer_data_us; -- 한 번만 실행하면 됩니다.
```

변종 1: 단일 행 주소를 입력 받습니다. 미국 주소가 아닌 경우 제대로 동작하지 않습니다.

```
 SELECT house_num, name, suftype, city, country, state, unit    FROM
standardize_address('us_lex',
                    'us_gaz', 'us_rules', 'One Devonshire Place, PH 301,
Boston, MA 02109');
 house_num |   name    | suftype |  city  | country |    state    |     unit
 ----------+-----------+---------+--------+---------+-------------+-----------
 1       | DEVONSHIRE | PLACE   | BOSTON | USA     | MASSACHUSETTS | # PENTHOUSE
301
```

TIGER 지오코딩 도구로 패키징된 테이블을 이용 (이 예시는 사용자가 postgis_tiger_geocoder 를 설치했을 경우에만 동작합니다.)

```
 SELECT * FROM standardize_address('tiger.pagc_lex',
     'tiger.pagc_gaz', 'tiger.pagc_rules', 'One Devonshire Place, PH 301, Boston, MA 02109-
1234');
```

더 알아보기 쉽게 하기 위해 hstore 확장 프로그램을 이용해서 출력물을 덤프할 것입니다. 사용자가 CREATE EXTENSION hstore; 명령으로 설치해야 합니다.

```
 SELECT (each(hstore(p))).*
  FROM standardize_address('tiger.pagc_lex', 'tiger.pagc_gaz',
    'tiger.pagc_rules', 'One Devonshire Place, PH 301, Boston, MA 02109') As
p;
 key    |     value
 -----------+-----------------
  box       |
  city      | BOSTON
  name      | DEVONSHIRE
  qual      |
  unit      | # PENTHOUSE 301
  extra     |
  state     | MA
  predir    |
  sufdir    |
  country   | USA
  pretype   |
  suftype   | PL
  building  |
  postcode  | 02109
  house_num | 1
  ruralroute |
 (16 rows)
```

변종 2: 주소를 두 부분으로 입력받습니다.

```
 SELECT (each(hstore(p))).*
  FROM standardize_address('tiger.pagc_lex', 'tiger.pagc_gaz',
    'tiger.pagc_rules', 'One Devonshire Place, PH 301', 'Boston, MA 02109,
US') As p;
 key    |     value
 -----------+-----------------
  box       |
  city      | BOSTON
  name      | DEVONSHIRE
  qual      |
  unit      | # PENTHOUSE 301
```

```
 extra     |
 state     | MA
 predir    |
 sufdir    |
 country   | USA
 pretype   |
 suftype   | PL
 building  |
 postcode  | 02109
 house_num | 1
 ruralroute |
(16 rows)
```

참고: stdaddr, 규칙 테이블, 어휘 목록 테이블, 지명 색인 테이블, Pagc_Normalize_Address

Chapter 13. PostGIS 추가 기능

이 장에서는 PostGIS 소스 TAR 파일 및 소스 저장소의 "extras" 폴더에서 찾을 수 있는 기능에 대해 문서화합니다. 이 기능들이 항상 PostGIS 바이너리 배포판에 함께 패키징되지는 않지만, 일반적으로 PL/pgSQL 기반이거나 또는 그대로 실행할 수 있는 표준 셸 스크립트입니다.

13.1. TIGER 지오코딩 도구

미국 인구조사국이 배포한 TIGER (Topologically Integrated Geographic Encoding and Referencing system) / Line and Master Address database export 과 함께 동작하도록 작성된 PL/pgSQL 기반 지오코딩 도구입니다.

지오코딩 도구는 네 가지 구성 요소로 이루어져 있습니다: 데이터 로더 함수, 주소 정규화 도구, 주소 지오코딩 도구, 그리고 역 지오코딩 도구입니다.

TIGER 지오코딩 도구가 특별히 미국을 위해 설계되긴 했지만, 대부분의 개념 및 함수를 다른 국가의 주소 및 도로망과 함께 동작하도록 조정할 수 있습니다.

스크립트가 TIGER 관련 모든 함수 및 도로 유형 접두사, 접미사, 주와 같은 재사용할 수 있는 색인 데이터, 데이터 로드를 관리하기 위한 여러 제어 테이블, 그리고 TIGER 가 로드한 모든 테이블이 승계하는, 필요한 최소한의 기본(skeleton) 기반 테이블들을 보관하는 tiger 라는 스키마를 빌드합니다.

로더가 인구조사 사이트에서 다운로드하고 데이터베이스에 로드한, 각 주에 대한 모든 인구조사 데이터를 보관하는 tiger_data 라는 또다른 스키마도 생성합니다. 현재 모델에서, 주(州) 테이블들의 각 집합에 해당 주 데이터만 강제하도록 하는 제약조건과 함께 ma_addr, ma_edges 등과 같은 주 코드 접두사를 붙입니다. 각 테이블은 tiger schema 안에 위치한 addr, faces, edges 등과 같은 테이블들을 승계합니다.

모든 지오코딩 함수는 기반 테이블들만 참조하기 때문에, 데이터 스키마를 tiger_data 라고 명명하거나 또는 다른 스키마에 데이터를 한 단계 더 분할할 수 없다거나 할 필요는 없습니다. 예를 들어 각 주에 대해 서로 다른 스키마를 쓸 수 있습니다. 모든 테이블이 tiger 스키마의 테이블을 승계하는 한 말이죠.

사용자 데이터베이스에서 확장 프로그램을 활성화하는 방법 및 이를 통해 데이터를 로드하는 방법에 대한 지침을 알고 싶다면, Section 2.8.1, “TIGER 지오코딩 도구로 사용자의 PostGIS 데이터베이스 활성화: 확장 프로그램 이용”을 참조하십시오.

TIGER 지오코딩 도구(tiger_2010)를 사용중이라면, extras/tiger 폴더에 첨부된 upgrade_geocoder.bat 또는 .sh 스크립트를 실행해서 스크립트들을 업그레이드할 수 있습니다. tiger_2010 과 tiger_2011 이상 버전의 주요 차이점은 county 및 state 테이블이 더이상 주를 기준으로 분할되지 않는다는 점입니다. tiger_2010 버전의 데이터를 가지고 있는데 tiger_2015 데이터로 변환하고 싶을 경우 Section 2.8.5, “TIGER 지오코딩 도구 설치 업그레이드”를 참조하십시오.

PostGIS 2.2.0 배포본이 TIGER 2015 데이터를 지원하기 시작했고, 주소 표준화 도구를 PostGIS 의 한 부분으로 포함했습니다.

PostGIS 2.1.0 배포본부터, PostgreSQL 9.1 이상 버전을 실행중일 경우, TIGER 지오코딩 도구를 PostgreSQL 확장 프로그램 모델과 함께 설치할 수 있게 됐습니다. 자세한 내용은 Section 2.8.1, “TIGER 지오코딩 도구로 사용자의 PostGIS 데이터베이스 활성화: 확장 프로그램 이용”을 참조하십시오.

Pagc_Normalize_Address 함수는 내장된 Normalize_Address 함수와 바로 바꿔 쓸 수 있는 대체 함수입니다. 컴파일 및 설치 지침을 알고 싶다면 Section 2.7, “주소 표준화 도구 설치 및 활용” 를 참조하십시오.

설계:

이 프로젝트의 목표는 임의의 미국 주소 스트링을 처리하고, 정규화된 TIGER 인구조사 데이터를 이용해서 주어진 주소의 위치 및 해당 위치의 가능성을 반영한 포인트 도형 및 순위(rating)를 생성할 수 있는, 모든 기능을 갖춘 지오코딩 도구를 빌드하는 것입니다. 순위 번호가 높을수록 결과가 나빠집니다.

PostGIS 2.0.0 버전에서 처음 도입된 reverse_geocode 함수는 GPS 위치의 도로 주소 및 교차로를 추출하는 데 유용합니다.

지오코딩 도구는 PostGIS 에 익숙한 사람이라면 누구나 설치하고 이용할 수 있을 만큼 단순해야 하며, PostGIS 가 지원하는 플랫폼 상에서 쉽게 설치하고 이용할 수 있어야 합니다.

지오코딩 도구는 서식 및 철자 오류가 있더라도 제대로 기능할 수 있을 만큼 강력해야 합니다.

지오코딩 도구는 향후 데이터를 업데이트, 또는 데이터소스를 대체하더라도 최소한의 코딩 변경만으로 이용할 수 있을 만큼 확장성을 갖춰야 합니다.

지오코딩 도구 함수들이 제대로 동작하려면 데이터베이스 탐색 경로에 tiger 스키마를 추가해야만 합니다.

Drop_Indexes_Generate_Script — TIGER 스키마 및 사용자 지정 스키마에 있는 기본 키가 아닌 그리고 유일하지 않은 인덱스들을 모두 삭제하는 스크립트를 생성합니다. 스키마를 따로 지정하지 않을 경우 기본값인 tiger_data 스키마를 대상으로 합니다.

Drop_Nation_Tables_Generate_Script — 지정한 스키마에서 county_all, state_all 로 시작하는 테이블, 또는 뒤에 county, state 가 붙는 주(州) 코드를 모두 삭제하는 스크립트를 생성합니다.

Drop_State_Tables_Generate_Script — 지정한 스키마에서 주(州) 명칭의 약어가 접두사로 붙는 테이블을 모두 삭제하는 스크립트를 생성합니다. 스키마를 따로 지정하지 않을 경우 기본값인 tiger_data 스키마를 대상으로 합니다.

Geocode — 주소를 스트링(또는 다른 정규화된 주소)으로 입력받아 NAD83 경위도로 투영된 포인트 도형, 각 주소에 대한 정규화된 주소 및 순위를 포함하는 가능성 있는 위치들의 집합을 출력합니다. 순위가 낮을수록 주소와 위치가 일치할 가능성이 커집니다. 결과를 순위가 낮은 순서로 정렬합니다. 결과의 최대 개수(기본값 10) 및 restrict_region(기본값 NULL) 옵션 파라미터를 넘겨줄 수 있습니다.

Geocode_Intersection — 교차하는 도로 2 개와 주, 시, 집코드를 입력받아 NAD83 경위도로 투영된 포인트 위치인 geomout, 각 위치에 대한 normalized_address (addy) 및 순위도 포함하는, 교차점에서 처음 가로지르는 도로에 있는 가능성 있는 위치들의 집합을 출력합니다. 순위가 낮을수록 주소와 위치가 일치할 가능성이 커집니다. 결과를 순위가 낮은 순서로 정렬합니다. 결과의 최대 개수(기본값 10) 옵션 파라미터를 넘겨줄 수 있습니다. TIGER 데이터(edge, face, addr) 및 PostgreSQL 퍼지 문자열 매칭(soundex, levenshtein)을 이용합니다.

Get_Geocode_Setting — tiger.geocode_settings 테이블에 저장된 특정 설정값을 반환합니다.

Get_Tract — 도형이 위치해 있는 구역(tract) 테이블로부터 인구조사 구역 또는 현장(field)을 반환합니다. 기본적으로 구역의 축약된 명칭을 반환합니다.

Install_Missing_Indexes — 지오코딩 도구의 결합(join)에 이용되는 키(key) 열을 가진 테이블을 모두 찾아서 해당 열에 쓰인 인덱스 가운데 없어진 것들에 대한 조건으로 필터링해서 없어진 인덱스들을 추가합니다.

Loader_Generate_Census_Script — 설정한 플랫폼에 대해 설정한 주(州)를 위한, TIGER 인구조사 주(州) 구역(tract), 블록 그룹(bg), 블록(tabblock) 데이터 테이블을 다운로드해서 tiger_data 스키마에 단계별로 로드하는 셸 스크립트를 생성합니다. 각 주(州)의 스크립트를 개별 레코드로 반환합니다.

Loader_Generate_Script — 설정한 플랫폼에 대해 설정한 주(州)를 위한, TIGER 데이터를 다운로드해서 tiger_data 스키마에 단계별로 로드하는 셸 스크립트를 생성합니다. 각 주(州)의 스크립트를 개별 레코드로 반환합니다. 최신 버전은 TIGER 2010 구조 변경 사항을 지원하며, 인구조사 구역, 블록 그룹, 블록 테이블도 로드합니다.

Loader_Generate_Nation_Script — 설정한 플랫폼에 대해, 군 및 주 색인 테이블을 로드하는 셸 스크립트를 생성합니다.

Missing_Indexes_Generate_Script — 지오코딩 도구의 결합(join)에 이용되는 키(key) 열을 가진 테이블 가운데 해당 열에서 없어진 인덱스를 모두 찾아서 해당 테이블에 대한 인덱스를 정의하는 SQL DDL 을 출력합니다.

Normalize_Address — 문자형 도로 주소를 입력받아, 도로 접미사, 접두사 및 유형을 표준화하고 도로, 도로명 등을 개별 필드로 분해한 norm_addy 합성 유형을 반환합니다. 이 함수는 tiger_geocoder 와 함께 패키징된 색인 데이터만 이용해서 (TIGER 인구조사 데이터는 필요없이) 작동할 것입니다.

Pagc_Normalize_Address — 문자형 도로 주소를 입력받아, 도로 접미사, 접두사 및 유형을 표준화하고 도로, 도로명 등을 개별 필드로 분해한 norm_addy 합성 유형을 반환합니다. 이 함수는 tiger_geocoder 와 함께 패키징된 색인 데이터만 이용해서 (TIGER 인구조사 데이터는 필요없이) 작동할 것입니다. address_standardizer 확장 프로그램이 필요합니다.

Pprint_Addy — norm_addy 복합 유형 객체를 입력받아, 해당 객체의 보기 좋은 인쇄용 표현을 반환합니다. 일반적으로 normalize_address 함수와 결합해서 쓰입니다.

Reverse_Geocode — 알려진 공간 참조 시스템으로 투영된 도형 포인트를 입력받아 이론적으로 가능한 주소들의 배열과 교차로의 배열을 담고 있는 레코드를 반환합니다. include_strnum_range = true 일 경우, 교차로에 있는 도로 범위를 포함합니다.

Topology_Load_Tiger — PostGIS 지형에 TIGER 데이터의 정의된 지역을 로드하면서 TIGER 데이터를 지형의 공간 참조 시스템으로 변환하고 지형의 정확도 허용 오차에 스냅시킵니다.

Set_Geocode_Setting — 지오코딩 도구 함수들의 습성에 영향을 미치는 설정값을 설정합니다.

TIGER 지오코딩 도구와 달리 여러 국가의 지오코딩을 지원한다는 장점을 가진, PostGIS 를 지원하는 몇몇 오픈소스 지오코딩 도구들이 있습니다.

- Nominatim 은 OpenStreetMap 지명 색인 데이터 형식을 이용합니다. 데이터를 로드하기 위해 osm2pgsql 이, 제대로 동작하기 위해 PostgreSQL 8.4 이상 버전 및 PostGIS 1.5 이상 버전이

필요합니다. 웹서비스 인터페이스로 패키징됐으며, 웹서비스로 호출하기 위해 설계된 것으로 보입니다. TIGER 지오코딩 도구와 마찬가지로, 지오코딩 도구 및 역 지오코딩 도구 구성 요소들을 모두 가지고 있습니다. 문서상에서 보면, Nominatim 이 TIGER 지오코딩 도구처럼 순수한 SQL 인터페이스를 가지고 있는지, 또는 웹 인터페이스에 충분한 로직을 구현했는지 불확실합니다.

- GIS Graphy 또한 PostGIS 를 활용하며 Nominatim 처럼 OSM(OpenStreetMap) 데이터를 이용합니다. OSM 데이터를 로드하기 위한 로더를 갖추고 있고, Nominatim 와 유사하게 미국 이외의 국가의 지오코딩도 할 수 있습니다. Nominatim 과 마찬가지로, 웹서비스로 실행되며 Java 1.5, Servlet apps, Solr 에 의존합니다. GIS Graphy 는 여러 플랫폼 상에서 실행되며, 몇몇 멋진 기능과 함께 역 지오코딩 도구도 가지고 있습니다.

13.1.1. Drop_Indexes_Generate_Script

Drop_Indexes_Generate_Script — TIGER 스키마 및 사용자 지정 스키마에 있는 기본 키가 아닌 그리고 유일하지 않은 인덱스들을 모두 삭제하는 스크립트를 생성합니다. 스키마를 따로 지정하지 않을 경우 기본값인 tiger_data 스키마를 대상으로 합니다.

Synopsis

text **Drop_Indexes_Generate_Script**(text *param_schema=tiger_data*);

설명: TIGER 스키마 및 사용자 지정 스키마에 있는 기본 키가 아닌 그리고 유일하지 않은 인덱스들을 모두 삭제하는 스크립트를 생성합니다. 스키마를 따로 지정하지 않을 경우 기본값인 tiger_data 스키마를 대상으로 합니다.

이 함수는 쿼리 설계자를 혼란스럽게 할 수도 있는 인덱스의 쓸데없는 팽창(bloat) 또는 불필요한 용량을 차지하는 일을 최소화합니다. Install_Missing_Indexes 함수와 결합해서 지오코딩 도구가 이용하는 인덱스들만 추가하는 데 쓰입니다.

2.0.0 버전부터 사용할 수 있습니다.

예시

```
SELECT drop_indexes_generate_script() As actionsql;
actionsql
-----------------------------------------------------------
DROP INDEX tiger.idx_tiger_countysub_lookup_lower_name;
DROP INDEX tiger.idx_tiger_edges_countyfp;
DROP INDEX tiger.idx_tiger_faces_countyfp;
DROP INDEX tiger.tiger_place_the_geom_gist;
DROP INDEX tiger.tiger_edges_the_geom_gist;
DROP INDEX tiger.tiger_state_the_geom_gist;
DROP INDEX tiger.idx_tiger_addr_least_address;
DROP INDEX tiger.idx_tiger_addr_tlid;
DROP INDEX tiger.idx_tiger_addr_zip;
DROP INDEX tiger.idx_tiger_county_countyfp;
DROP INDEX tiger.idx_tiger_county_lookup_lower_name;
DROP INDEX tiger.idx_tiger_county_lookup_snd_name;
DROP INDEX tiger.idx_tiger_county_lower_name;
DROP INDEX tiger.idx_tiger_county_snd_name;
DROP INDEX tiger.idx_tiger_county_the_geom_gist;
DROP INDEX tiger.idx_tiger_countysub_lookup_snd_name;
DROP INDEX tiger.idx_tiger_cousub_countyfp;
```

```
DROP INDEX tiger.idx_tiger_cousub_cousubfp;
DROP INDEX tiger.idx_tiger_cousub_lower_name;
DROP INDEX tiger.idx_tiger_cousub_snd_name;
DROP INDEX tiger.idx_tiger_cousub_the_geom_gist;
DROP INDEX tiger_data.idx_tiger_data_ma_addr_least_address;
DROP INDEX tiger_data.idx_tiger_data_ma_addr_tlid;
DROP INDEX tiger_data.idx_tiger_data_ma_addr_zip;
DROP INDEX tiger_data.idx_tiger_data_ma_county_countyfp;
DROP INDEX tiger_data.idx_tiger_data_ma_county_lookup_lower_name;
DROP INDEX tiger_data.idx_tiger_data_ma_county_lookup_snd_name;
DROP INDEX tiger_data.idx_tiger_data_ma_county_lower_name;
DROP INDEX tiger_data.idx_tiger_data_ma_county_snd_name;
:
:
```

참고: Install_Missing_Indexes, Missing_Indexes_Generate_Script

13.1.2. Drop_Nation_Tables_Generate_Script

Drop_Nation_Tables_Generate_Script — 지정한 스키마에서 county_all, state_all 로 시작하는 테이블, 또는 뒤에 county, state 가 붙는 주(州) 코드를 모두 삭제하는 스크립트를 생성합니다.

Synopsis

text **Drop_Nation_Tables_Generate_Script**(text *param_schema=tiger_data*);

설명: 지정한 스키마에서 county_all, state_all 로 시작하는 테이블, 또는 뒤에 county, state 가 붙는 주(州) 코드를 모두 삭제하는 스크립트를 생성합니다. tiger_2010 데이터를 tiger_2011 데이터로 업그레이드하는 경우 이 함수가 필요합니다.

2.1.0 버전부터 사용할 수 있습니다.

예시

```
SELECT drop_nation_tables_generate_script();
DROP TABLE tiger_data.county_all;
DROP TABLE tiger_data.county_all_lookup;
DROP TABLE tiger_data.state_all;
DROP TABLE tiger_data.ma_county;
DROP TABLE tiger_data.ma_state;
```

참고: Loader_Generate_Nation_Script

13.1.3. Drop_State_Tables_Generate_Script

Drop_State_Tables_Generate_Script — 지정한 스키마에서 주(州) 명칭의 약어가 접두사로 붙는 테이블을 모두 삭제하는 스크립트를 생성합니다. 스키마를 따로 지정하지 않을 경우 기본값인 tiger_data 스키마를 대상으로 합니다.

Synopsis

text **Drop_State_Tables_Generate_Script**(text *param_state*, text *param_schema=tiger_data*);

설명: 지정한 스키마에서 주(州) 명칭의 약어가 접두사로 붙는 테이블을 모두 삭제하는 스크립트를 생성합니다. 스키마를 따로 지정하지 않을 경우 기본값인 tiger_data 스키마를 대상으로 합니다. 이전 데이터 로드 과정에 어떤 문제가 생겼을 경우 주(州) 데이터를 다시 로드하기 전에 주(州) 테이블들을 삭제하는 데 유용합니다.

2.0.0 버전부터 사용할 수 있습니다.

예시

```
SELECT drop_state_tables_generate_script('PA');
DROP TABLE tiger_data.pa_addr;
DROP TABLE tiger_data.pa_county;
DROP TABLE tiger_data.pa_county_lookup;
DROP TABLE tiger_data.pa_cousub;
DROP TABLE tiger_data.pa_edges;
DROP TABLE tiger_data.pa_faces;
DROP TABLE tiger_data.pa_featnames;
DROP TABLE tiger_data.pa_place;
DROP TABLE tiger_data.pa_state;
DROP TABLE tiger_data.pa_zip_lookup_base;
DROP TABLE tiger_data.pa_zip_state;
DROP TABLE tiger_data.pa_zip_state_loc;
```

참고: Loader_Generate_Script

13.1.4. Geocode

Geocode — 주소를 스트링(또는 다른 정규화된 주소)으로 입력받아 NAD83 경위도로 투영된 포인트 도형, 각 주소에 대한 정규화된 주소 및 순위를 포함하는 가능성 있는 위치들의 집합을 출력합니다. 순위가 낮을수록 주소와 위치가 일치할 가능성이 커집니다. 결과를 순위가 낮은 순서로 정렬합니다. 결과의 최대 개수(기본값 10) 및 restrict_region(기본값 NULL) 옵션 파라미터를 넘겨줄 수 있습니다.

Synopsis

setof record **geocode**(varchar *address*, integer *max_results=10*, geometry *restrict_region=NULL*, norm_addy *OUT addy*, geometry *OUT geomout*, integer *OUT rating*);

setof record **geocode**(norm_addy *in_addy*, integer *max_results=10*, geometry *restrict_region=NULL*, norm_addy *OUT addy*, geometry *OUT geomout*, integer *OUT rating*);

설명: 주소를 스트링(또는 다른 정규화된 주소)으로 입력받아 NAD83 경위도로 투영된 포인트 도형, 각 주소에 대한 normalized_address (addy) 및 순위를 포함하는 가능성 있는 위치들의 집합을 출력합니다. 순위가 낮을수록 주소와 위치가 일치할 가능성이 커집니다. 결과를 순위가 낮은 순서로 정렬합니다. TIGER 데이터(edge, face, addr), PostgreSQL 퍼지 문자열 매칭(soundex, levenshtein), 그리고 PostGIS 라인 보간 함수를 이용해서 TIGER 경계선을 따라 주소를 보간합니다. 순위가 높을수록 지오코딩 작업이 올바를 가능성이 떨어집니다. 지오코딩된 포인트는 기본적으로 중심선에서 도로 주소가 위치한 쪽(왼쪽/오른쪽)으로 10 미터 오프셋됩니다.

개선 사항: 2.0.0 버전에서 TIGER 2010 데이터 구조를 지원하고, 속도와 지오코딩 작업의 정확도를 향상시키기 위해 몇몇 로직을 수정하고, 포인트를 중심선에서 도로 주소가 위치한 쪽으로 오프셋시키기 시작했습니다. 새로 도입된 max_results 파라미터는 결과의 개수를 설정하거나 최선의 결과만 반환하도록 하는 데 유용합니다.

기본 예시

매사추세츠(MA), 미네소타(MN), 캘리포니아(CA), 로드아일랜드(RI) 주의 TIGER 데이터를 로드한 PostgreSQL 9.1rc1/PostGIS 2.0 이 실행되고 있는 3.0 GHZ 단일 프로세서 및 2GB 램 윈도우 7 컴퓨터에서 다음 예시를 실행시키는 데 걸리는 시간을 측정했습니다.

완전 일치를 계산하는 것이 더 빠릅니다(61 밀리초).

```
 SELECT g.rating, ST_X(g.geomout) As lon, ST_Y(g.geomout) As lat,
   (addy).address As stno, (addy).streetname As street,
   (addy).streettypeabbrev As styp, (addy).location As city, (addy).stateabbrev As
st,(addy).zip
   FROM geocode('75 State Street, Boston MA 02109') As g;
 rating |    lon     |    lat     | stno | street | styp | city | st | zip
--------+------------+------------+------+--------+------+------+----+-------------
  0 | -71.0556722990239 | 42.3589914927049 |  75 | State | St  | Boston | MA | 02109
```

지오코딩 도구에 집코드를 넘겨주지 않더라도 추측할 수 있습니다(122 ~ 150 밀리초 걸립니다).

```
 SELECT g.rating, ST_AsText(ST_SnapToGrid(g.geomout,0.00001)) As wktlonlat,
   (addy).address As stno, (addy).streetname As street,
   (addy).streettypeabbrev As styp, (addy).location As city, (addy).stateabbrev As
st,(addy).zip
   FROM geocode('226 Hanover Street, Boston, MA',1) As g;
 rating |     wktlonlat     | stno | street | styp | city | st | zip
--------+-------------------+------+--------+------+--------+----+-------
    1 | POINT(-71.05528 42.36316) | 226 | Hanover | St  | Boston | MA | 02113
```

철자 오류도 처리할 수 있는데, 하나 이상의 가능성 있는 답을 순위와 함께 생성하기 때문에 시간이 더 걸립니다(500 밀리초).

```
 SELECT g.rating, ST_AsText(ST_SnapToGrid(g.geomout,0.00001)) As wktlonlat,
   (addy).address As stno, (addy).streetname As street,
   (addy).streettypeabbrev As styp, (addy).location As city, (addy).stateabbrev As
st,(addy).zip
   FROM geocode('31 - 37 Stewart Street, Boston, MA 02116') As g;
 rating |     wktlonlat     | stno | street | styp | city | st | zip
--------+-------------------+------+--------+------+--------+----+-------
   70 | POINT(-71.06459 42.35113) |  31 | Stuart | St  | Boston | MA | 02116
```

주소들을 지오코딩하는 데 배치(batch) 작업을 이용합니다. max_results = 1 로 설정하는 편이 가장 쉽습니다. 아직 지오코딩되지 않은 (순위가 없는) 주소들만 처리합니다.

```
 CREATE TABLE addresses_to_geocode(addid serial PRIMARY KEY, address text,
       lon numeric, lat numeric, new_address text, rating integer);

 INSERT INTO addresses_to_geocode(address)
 VALUES ('529 Main Street, Boston MA, 02129'),
  ('77 Massachusetts Avenue, Cambridge, MA 02139'),
  ('25 Wizard of Oz, Walaford, KS 99912323'),
  ('26 Capen Street, Medford, MA'),
```

```
   ('124 Mount Auburn St, Cambridge, Massachusetts 02138'),
   ('950 Main Street, Worcester, MA 01610');
  -- 처음 세 주소만 업데이트합니다(323 ~ 704 밀리초 - 캐싱 및 공유 메모리 영향 때문에 첫
지오코딩 작업은 항상 느립니다). --
  -- 주소가 대량일 경우 한번에 업데이트하지 않는 편이 좋습니다.
  -- 전체 지오코딩 작업을 한번에 커밋해야만 하기 때문입니다.
  -- 이 예시의 경우 LEFT JOIN 으로 재결합시키고
  -- 나쁜 주소를 다시 지오코딩 작업하지 않기 위해
  -- 일치하지 않을 경우 순위를 -1 로 설정합니다.
 UPDATE addresses_to_geocode
  SET (rating, new_address, lon, lat)
   = ( COALESCE((g.geo).rating,-1), pprint_addy((g.geo).addy),
     ST_X((g.geo).geomout)::numeric(8,5), ST_Y((g.geo).geomout)::numeric(8,5) )
 FROM (SELECT addid
   FROM addresses_to_geocode
   WHERE rating IS NULL ORDER BY addid LIMIT 3) As a
   LEFT JOIN (SELECT addid, (geocode(address,1)) As geo
   FROM addresses_to_geocode As ag
   WHERE ag.rating IS NULL ORDER BY addid LIMIT 3) As g ON a.addid = g.addid
 WHERE a.addid = addresses_to_geocode.addid;

 result
 -----
 Query returned successfully: 3 rows affected, 480 ms execution time.

 SELECT * FROM addresses_to_geocode WHERE rating is not null;

  addid |           address           |  lon   |  lat   |         new_address         | rating
 -------+-----------=---------------+--------+--------+-----------------------------+------
   1 | 529 Main Street, Boston MA, 02129      | -71.07181 | 42.38359 | 529 Main St, Boston, MA 02129      |
0
   2 | 77 Massachusetts Avenue, Cambridge, MA 02139 | -71.09428 | 42.35988 | 77 Massachusetts Ave, Cambridge,
MA 02139 |   0
   3 | 25 Wizard of Oz, Walaford, KS 99912323   |     |    |                  |  -1
```

예시: 도형 필터 이용

```
 SELECT g.rating, ST_AsText(ST_SnapToGrid(g.geomout,0.00001)) As wktlonlat,
    (addy).address As stno, (addy).streetname As street,
    (addy).streettypeabbrev As styp,
    (addy).location As city, (addy).stateabbrev As st,(addy).zip
  FROM geocode('100 Federal Street, MA',
       3,
       (SELECT ST_Union(the_geom)
          FROM place WHERE statefp = '25' AND name = 'Lynn')::geometry
       ) As g;

 rating |    wktlonlat     | stno | street | styp | city | st | zip
 -------+------------------+------+--------+------+------+----+-------
    8 | POINT(-70.96796 42.4659) | 100 | Federal | St  | Lynn | MA | 01905
 Total query runtime: 245 ms.
```

참고: Normalize_Address, Pprint_Addy, ST_AsText, ST_SnapToGrid, ST_X, ST_Y

13.1.5. Geocode_Intersection

Geocode_Intersection — 교차하는 도로 2 개와 주, 시, 집코드를 입력받아 NAD83 경위도로 투영된 포인트 위치인 geomout, 각 위치에 대한 normalized_address (addy) 및 순위도 포함하는, 교차점에서 처음 가로지르는 도로에 있는 가능성 있는 위치들의 집합을 출력합니다. 순위가 낮을수록 주소와 위치가 일치할 가능성이 커집니다. 결과를 순위가 낮은 순서로 정렬합니다. 결과의 최대 개수(기본값 10) 옵션 파라미터를 넘겨줄 수 있습니다. TIGER 데이터(edge, face, addr) 및 PostgreSQL 퍼지 문자열 매칭(soundex, levenshtein)을 이용합니다.

Synopsis

setof record **geocode_intersection**(text *roadway1*, text *roadway2*, text *in_state*, text *in_city*, text *in_zip*, integer *max_results=10*, norm_addy *OUT addy*, geometry *OUT geomout*, integer *OUT rating*);

설명: 교차하는 도로 2 개와 주, 시, 집코드를 입력받아 각 위치에 대한 normalized_address (addy), NAD83 경위도로 투영된 포인트 위치인 geomout, 그리고 순위(rating)도 포함하는, 교차점에서 처음 가로지르는 도로에 있는 가능성 있는 위치들의 집합을 출력합니다. 순위가 낮을수록 주소와 위치가 일치할 가능성이 커집니다. 결과를 순위가 낮은 순서로 정렬합니다. 결과의 최대 개수(기본값 10) 옵션 파라미터를 넘겨줄 수 있습니다. TIGER 데이터(edge, face, addr) 및 PostgreSQL 퍼지 문자열 매칭(soundex, levenshtein)을 이용합니다.

2.0.0 버전부터 사용할 수 있습니다.

기본 예시

매사추세츠(MA) 주의 모든 TIGER 데이터를 로드한 PostgreSQL 9.0/PostGIS 1.5 가 실행되고 있는 3.0 GHZ 단일 프로세서 및 2GB 램 윈도우 7 컴퓨터에서 다음 예시를 실행시키는 데 걸리는 시간을 측정했습니다. 현재 조금 느린 편입니다(3,000 밀리초).

TIGER 2011 데이터를 로드한 PostGIS 2.0 과 PostgreSQL 64 비트 버전을 실행중인 8GB 램 윈도우 2003 64 비트 컴퓨터에서 테스트한 결과는 더 빠릅니다(41 밀리초).

```
SELECT pprint_addy(addy), st_astext(geomout),rating
       FROM geocode_intersection( 'Haverford St','Germania St', 'MA', 'Boston', '02130',1);
      pprint_addy       |     st_astext      | rating
--------------------------------+----------------------------+--------
98 Haverford St, Boston, MA 02130 | POINT(-71.101375 42.31376) |   0
```

지오코딩 도구에 집코드를 넘겨주지 않더라도 추측할 수 있습니다. 윈도우 7 컴퓨터에서는 3,500 밀리초, 윈도우 2003 64 비트 컴퓨터에서는 741 밀리초 걸립니다.

```
SELECT pprint_addy(addy), st_astext(geomout),rating
         FROM geocode_intersection('Weld', 'School', 'MA', 'Boston');
     pprint_addy      |     st_astext      | rating
------------------------------+----------------------------+--------
 98 Weld Ave, Boston, MA 02119 | POINT(-71.099 42.314234) |   3
 99 Weld Ave, Boston, MA 02119 | POINT(-71.099 42.314234) |   3
```

참고: Geocode, Pprint_Addy, ST_AsText

13.1.6. Get_Geocode_Setting

Get_Geocode_Setting — tiger.geocode_settings 테이블에 저장된 특정 설정값을 반환합니다.

Synopsis

text **Get_Geocode_Setting**(text *setting_name*);

설명: tiger.geocode_settings 테이블에 저장된 특정 설정값을 반환합니다. 사용자가 함수의 디버깅을 켜고 끌 수 있게 해주는 설정값입니다. 향후 설정값으로 순위를 제어할 수 있도록 할 계획입니다. 현재 설정값 목록은 다음과 같습니다:

```
 명칭          | 설정값 | 데이터형  | 카테고리 |                                                간단한 설명
---------------+------+---------+--------+-----------------------------------------------------------
 debug_geocode_address      | false | boolean | debug   | 참일 경우 geocode_address 를 호출할 때 NOTICE 로그에 쿼리와 같은 디버그 정보를 출력합니다.
 debug_geocode_intersection | false | boolean | debug   | 참일 경우 geocode_intersection 을 호출할 때 NOTICE 로그에 쿼리와 같은 디버그 정보를 출력합니다.
 debug_normalize_address    | false | boolean | debug   | 참일 경우 normalize_address 를 호출할 때 NOTICE 로그에 쿼리 및 중간 표현식과 같은 디버그 정보를 출력합니다.
 debug_reverse_geocode      | false | boolean | debug   | 참일 경우 reverse_geocode 를 호출할 때 NOTICE 로그에 쿼리 및 중간 표현식과 같은 디버그 정보를 출력합니다.
 reverse_geocode_numbered_roads | 0   | integer | rating  | 주 및 군 고속도로에 대해, 0 - 어떤 명칭도 우선하지 않고, 1 - 번호로 된 고속도로 명칭을 우선하며, 2 - 지방의 주/군 명칭을 우선합니다.
 use_pagc_address_parser        | false   | boolean | normalize | 참으로 설정할 경우, 빌드된 TIGER normalize_address 대신 (pagc_normalize_address 를 통해) address_standardizer 확장 프로그램을 이용하려 할 것입니다.
```

변경 사항: 2.2.0 버전부터 geocode_settings_default 테이블에 기본 설정값을 저장합니다. 사용자 지정 설정값은 geocode_settings 테이블에 있으며, 이 geocode_settings 테이블에는 사용자가 설정한 설정값만 저장합니다.

2.1.0 버전부터 사용할 수 있습니다.

예시: 디버깅 설정값 반환

```
SELECT get_geocode_setting('debug_geocode_address) As result;
result
---------
false
```

참고: Set_Geocode_Setting

13.1.7. Get_Tract

Get_Tract — 도형이 위치해 있는 구역(tract) 테이블로부터 인구조사 구역 또는 현장(field)을 반환합니다. 기본적으로 구역의 축약된 명칭을 반환합니다.

Synopsis

text **get_tract**(geometry *loc_geom*, text *output_field=name*);

설명: 도형을 입력받아 해당 도형이 위치한 인구조사 구역을 반환합니다. 공간 참조 시스템을 따로 설정하지 않을 경우 NAD83 경위도라고 가정합니다.

2.0.0 버전부터 사용할 수 있습니다.

기본 예시

```
SELECT get_tract(ST_Point(-71.101375, 42.31376) ) As tract_name;
tract_name
---------
1203.01

-- 이 쿼리는 TIGER 도형 ID(geoid)를 반환합니다.
SELECT get_tract(ST_Point(-71.101375, 42.31376), 'tract_id' ) As tract_id;
tract_id
---------
25025120301
```

참고: Geocode >

13.1.8. Install_Missing_Indexes

Install_Missing_Indexes — 지오코딩 도구의 결합(join)에 이용되는 키(key) 열을 가진 테이블을 모두 찾아서 해당 열에 쓰인 인덱스 가운데 없어진 것들에 대한 조건으로 필터링해서 없어진 인덱스들을 추가합니다.

Synopsis

boolean **Install_Missing_Indexes**();

설명: tiger 및 tiger_data 스키마에서 지오코딩 도구의 결합에 이용되는 키 열을 가진 테이블을 모두 찾은 다음, 해당 열에서 없어진 인덱스들을 필터링해서, 해당 테이블들을 위한 인덱스를 정의하기 위한 SQL DDL 을 출력하고, 생성된 스크립트를 실행합니다. 이 함수는 로드 과정에서 없어졌을 수도 있는, 쿼리를 더 빨리 하기 위해 필요한 새 인덱스들을 추가하는 도우미 함수입니다. 이 함수는 인덱스 생성 스크립트를 작성하는 Missing_Indexes_Generate_Script 함수와 동일한 계열로, 작성된 스크립트를 실행까지 합니다. update_geocode.sql 업그레이드 스크립트의 일부로 이 함수를 호출합니다.

2.0.0 버전부터 사용할 수 있습니다.

예시

```
SELECT install_missing_indexes();
       install_missing_indexes
-------------------------
 t
```

참고: Loader_Generate_Script, Missing_Indexes_Generate_Script

13.1.9. Loader_Generate_Census_Script

Loader_Generate_Census_Script — 설정한 플랫폼에 대해 설정한 주(州)를 위한, TIGER 인구조사 주(州) 구역(tract), 블록 그룹(bg), 블록(tabblock) 데이터 테이블을 다운로드해서 tiger_data 스키마에 단계별로 로드하는 셸 스크립트를 생성합니다. 각 주(州)의 스크립트를 개별 레코드로 반환합니다.

Synopsis

setof text **loader_generate_census_script**(text[] *param_states*, text *os*);

설명: 설정한 플랫폼에 대해 설정한 주(州)를 위한, TIGER 인구조사 주(州) tract, bg, tabblocks 데이터 테이블을 다운로드해서 tiger_data 스키마에 단계별로 로드하는 셸 스크립트를 생성합니다. 각 주(州)의 스크립트를 개별 레코드로 반환합니다.

리눅스 상에서 unzip을 (윈도우 상에서는 기본적으로 7-zip을) 사용하고, 다운로드를 위해 wget을 씁니다. 데이터를 로드하는 데 Section 4.4.2, "shp2pgsql: ESRI shapefile 로더 이용하기"를 이용합니다. 이 함수가 작업에 이용하는 가장 작은 단위가 주(州) 전체라는 점에 주의하십시오. 이 함수는 "staging" 및 "temp" 폴더에 있는 파일들만 처리할 것입니다.

처리 과정 및 서로 다른 OS 셸 문법 변수들을 제어하기 위해 다음 제어 테이블들을 이용합니다.

1. loader_variables - 인구조사 현장, 연도, 데이터 및 단계별(staging) 스키마들과 같은 여러 변수들을 추적합니다.

2. loader_platform - 여러 플랫폼 및 여러 실행 파일들의 위치에 대한 개요를 작성합니다. 기본적으로 윈도우와 리눅스를 가지고 있습니다. 더 많은 플랫폼을 추가할 수 있습니다.

3. loader_lookuptables - 각 레코드가 테이블 유형(주, 군), 테이블에서 레코드를 처리할지 여부, 테이블에 레코드를 로드하는 방법을 정의합니다. 각 단계별로 데이터를 임포트하고 준비하며, 각 단계에 대해 열, 인덱스, 제약조건을 추가하고 제거하는 단계들을 정의합니다. 각 테이블의 접두사는 주명(州名)이며, TIGER 스키마 안의 테이블을 승계합니다. 예: tiger.faces 를 승계하는 tiger_data.ma_faces 테이블을 생성하십시오.

2.0.0 버전부터 사용할 수 있습니다.



Loader_Generate_Script 는 이 로직을 포함하지만, PostGIS 2.0.0 alpha5 이전 버전에 TIGER 지오코딩 도구를 설치한 경우, 이런 추가적인 테이블을 얻으려면 사용자가 이미 처리한 주(州)에 대해 이 함수를 실행해야 할 것입니다.

예시

선택한 주들을 위한 데이터를 윈도우 셸 스크립트 형식으로 로드하는 스크립트를 생성합니다.

```
  SELECT loader_generate_census_script(ARRAY['MA'], 'windows');
  -- 결과 --
  set
STATEDIR="\gisdata\www2.census.gov\geo\pvs\tiger2010st\25_Massachusetts"
  set TMPDIR=\gisdata\temp\
  set UNZIPTOOL="C:\Program Files\7-Zip\7z.exe"
  set WGETTOOL="C:\wget\wget.exe"
```

```
  set PGBIN=C:\projects\pg\pg91win\bin\
  set PGPORT=5432
  set PGHOST=localhost
  set PGUSER=postgres
  set PGPASSWORD=yourpasswordhere
  set PGDATABASE=tiger_postgis20
  set PSQL="%PGBIN%psql"
  set SHP2PGSQL="%PGBIN%shp2pgsql"
  cd \gisdata

  %WGETTOOL% http://www2.census.gov/geo/pvs/tiger2010st/25_Massachusetts/25/
--no-parent --relative --accept=*bg10.zip,*tract10.zip,*tabblock10.zip --
mirror --reject=html
  del %TMPDIR%\*.* /Q
  %PSQL% -c "DROP SCHEMA tiger_staging CASCADE;"
  %PSQL% -c "CREATE SCHEMA tiger_staging;"
  cd %STATEDIR%
  for /r %%z in (*.zip) do %UNZIPTOOL% e %%z -o%TMPDIR%
  cd %TMPDIR%
  %PSQL% -c "CREATE TABLE tiger_data.MA_tract(CONSTRAINT pk_MA_tract PRIMARY
KEY (tract_id) ) INHERITS(tiger.tract); "
  %SHP2PGSQL% -c -s 4269 -g the_geom -W "latin1" tl_2010_25_tract10.dbf
tiger_staging.ma_tract10 | %PSQL%
  %PSQL% -c "ALTER TABLE tiger_staging.MA_tract10 RENAME geoid10 TO tract_id;
SELECT loader_load_staged_data(lower('MA_tract10'), lower('MA_tract')); "
  %PSQL% -c "CREATE INDEX tiger_data_MA_tract_the_geom_gist ON
tiger_data.MA_tract USING gist(the_geom);"
  %PSQL% -c "VACUUM ANALYZE tiger_data.MA_tract;"
  %PSQL% -c "ALTER TABLE tiger_data.MA_tract ADD CONSTRAINT chk_statefp CHECK
(statefp = '25');"
  :
```

.sh 스크립트를 생성합니다.

```
  STATEDIR="/gisdata/www2.census.gov/geo/pvs/tiger2010st/25_Massachusetts"
  TMPDIR="/gisdata/temp/"
  UNZIPTOOL=unzip
  WGETTOOL="/usr/bin/wget"
  export PGBIN=/usr/pgsql-9.0/bin
  export PGPORT=5432
  export PGHOST=localhost
  export PGUSER=postgres
  export PGPASSWORD=yourpasswordhere
  export PGDATABASE=geocoder
  PSQL=${PGBIN}/psql
  SHP2PGSQL=${PGBIN}/shp2pgsql
  cd /gisdata

  wget http://www2.census.gov/geo/pvs/tiger2010st/25_Massachusetts/25/ --
no-parent --relative --accept=*bg10.zip,*tract10.zip,*tabblock10.zip --
mirror --reject=html
  rm -f ${TMPDIR}/*.*
  ${PSQL} -c "DROP SCHEMA tiger_staging CASCADE;"
  ${PSQL} -c "CREATE SCHEMA tiger_staging;"
  cd $STATEDIR
  for z in *.zip; do $UNZIPTOOL -o -d $TMPDIR $z; done
  :
  :
```

참고: Loader_Generate_Script

13.1.10. Loader_Generate_Script

Loader_Generate_Script — 설정한 플랫폼에 대해 설정한 주(州)를 위한, TIGER 데이터를 다운로드해서 tiger_data 스키마에 단계별로 로드하는 셸 스크립트를 생성합니다. 각 주(州)의 스크립트를 개별 레코드로 반환합니다. 최신 버전은 TIGER 2010 구조 변경 사항을 지원하며, 인구조사 구역, 블록 그룹, 블록 테이블도 로드합니다.

Synopsis

setof text **loader_generate_script**(text[] *param_states*, text *os*);

설명: 설정한 플랫폼에 대해 설정한 주(州)를 위한, TIGER 데이터를 다운로드해서 tiger_data 스키마에 단계별로 로드하는 셸 스크립트를 생성합니다. 각 주(州)의 스크립트를 개별 레코드로 반환합니다.

리눅스 상에서 unzip을 (윈도우 상에서는 기본적으로 7-zip을) 사용하고, 다운로드를 위해 wget을 씁니다. 데이터를 로드하는 데 Section 4.4.2, “shp2pgsql: ESRI shapefile 로더 이용하기” 를 이용합니다. 이 함수가 작업에 이용하는 가장 작은 단위가 주(州) 전체라는 점에 주의하십시오. 다만 사용자가 직접 파일들을 다운로드해서 이 단위를 덮어 쓸 수 있습니다. 이 함수는 "staging" 및 "temp" 폴더에 있는 파일들만 처리할 것입니다.

처리 과정 및 서로 다른 OS 셸 문법 변수들을 제어하기 위해 다음 제어 테이블들을 이용합니다.

1. loader_variables - 인구조사 현장, 연도, 데이터 및 단계별(staging) 스키마들과 같은 여러 변수들을 추적합니다.
2. loader_platform - 여러 플랫폼 및 여러 실행 파일들의 위치에 대한 개요를 작성합니다. 기본적으로 윈도우와 리눅스를 가지고 있습니다. 더 많은 플랫폼을 추가할 수 있습니다.
3. loader_lookuptables - 각 레코드가 테이블 유형(주, 군), 테이블에서 레코드를 처리할지 여부, 테이블에 레코드를 로드하는 방법을 정의합니다. 각 단계별로 데이터를 임포트하고 준비하며, 각 단계에 대해 열, 인덱스, 제약조건을 추가하고 제거하는 단계들을 정의합니다. 각 테이블의 접두사는 주명(州名)이며, TIGER 스키마 안의 테이블을 승계합니다. 예: tiger.faces 를 승계하는 tiger_data.ma_faces 테이블을 생성하십시오.

2.0.0 버전부터 사용할 수 있습니다. TIGER 2010 데이터 구조를 지원하며 인구조사 구역(tract), 블록 그룹(bg), 블록(tabblock) 테이블을 로드합니다.



If you are using pgAdmin 3, be warned that by default pgAdmin 3 truncates long text. To fix, change File -> Options -> Query Tool -> Query Editor - > Max. characters per column to larger than 50000 characters.

예시

```
  Using psql where gistest is your database and /gisdata/data_load.sh is the
file to create with the shell commands to run.
  psql -U postgres -h localhost -d gistest -A -t \
   -c    "SELECT    Loader_Generate_Script(ARRAY['MA'],    'gistest')"    >
/gisdata/data_load.sh;
```

```
  주(州) 2곳을 위한 데이터를 윈도우 셸 스크립트 형식으로 로드하는 스크립트를 생성합니다.
  SELECT loader_generate_script(ARRAY['MA','RI'], 'windows') AS result;
  -- result --
  set TMPDIR=\gisdata\temp\
  set UNZIPTOOL="C:\Program Files\7-Zip\7z.exe"
  set WGETTOOL="C:\wget\wget.exe"
  set PGBIN=C:\Program Files\PostgreSQL\9.4\bin\
  set PGPORT=5432
  set PGHOST=localhost
  set PGUSER=postgres
  set PGPASSWORD=yourpasswordhere
  set PGDATABASE=geocoder
  set PSQL="%PGBIN%psql"
  set SHP2PGSQL="%PGBIN%shp2pgsql"
  cd \gisdata

  cd \gisdata
  %WGETTOOL% ftp://ftp2.census.gov/geo/tiger/TIGER2015/PLACE/tl_*_25_* --no-
parent --relative --recursive --level=2 --accept=zip --mirror --reject=html
  cd \gisdata/ftp2.census.gov/geo/tiger/TIGER2015/PLACE
  :
  :
```

.sh 스크립트를 생성합니다.

```
  SELECT loader_generate_script(ARRAY['MA','RI'], 'sh') AS result;
  -- result --
  TMPDIR="/gisdata/temp/"
  UNZIPTOOL=unzip
  WGETTOOL="/usr/bin/wget"
  export PGBIN=/usr/lib/postgresql/9.4/bin
  export PGPORT=5432
  export PGHOST=localhost
  export PGUSER=postgres
  export PGPASSWORD=yourpasswordhere
  export PGDATABASE=geocoder
  PSQL=${PGBIN}/psql
  SHP2PGSQL=${PGBIN}/shp2pgsql
  cd /gisdata

  cd /gisdata
  wget ftp://ftp2.census.gov/geo/tiger/TIGER2015/PLACE/tl_*_25_* --no-parent
--relative --recursive --level=2 --accept=zip --mirror --reject=html
  cd /gisdata/ftp2.census.gov/geo/tiger/TIGER2015/PLACE
  rm -f ${TMPDIR}/*.*
  :
  :
```

참고: Section 2.8.1, “TIGER 지오코딩 도구로 사용자의 PostGIS 데이터베이스 활성화: 확장 프로그램 이용”, Loader_Generate_Nation_Script

13.1.11. Loader_Generate_Nation_Script

Loader_Generate_Nation_Script — 설정한 플랫폼에 대해, 군 및 주 색인 테이블을 로드하는 셸 스크립트를 생성합니다.

Synopsis

text **loader_generate_nation_script**(text *os*);

설명: 설정한 플랫폼에 대해, tiger_data 스키마로 county_all, county_all_lookup, state_all 테이블을 로드하는 셸 스크립트를 생성합니다. 이 테이블들은 각각 tiger 스키마의 county, county_lookup, state 테이블을 승계합니다.

리눅스 상에서 unzip을 (윈도우 상에서는 기본적으로 7-zip을) 사용하고, 다운로드를 위해 wget을 씁니다. 데이터를 로드하는 데 Section 4.4.2, "shp2pgsql: ESRI shapefile 로더 이용하기"를 이용합니다.

처리 과정 및 서로 다른 OS 셸 문법 변수들을 제어하기 위해 다음 제어 테이블 tiger.loader_platform, tiger.loader_variables, 그리고 tiger.loader_lookuptables 를 이용합니다.

1. loader_variables - 인구조사 현장, 연도, 데이터 및 단계별(staging) 스키마들과 같은 여러 변수들을 추적합니다.
2. loader_platform - 여러 플랫폼 및 여러 실행 파일들의 위치에 대한 개요를 작성합니다. 기본적으로 윈도우와 리눅스/유닉스를 가지고 있습니다. 더 많은 플랫폼을 추가할 수 있습니다.
3. loader_lookuptables - 각 레코드가 테이블 유형(주, 군), 테이블에서 레코드를 처리할지 여부, 테이블에 레코드를 로드하는 방법을 정의합니다. 각 단계별로 데이터를 임포트하고 준비하며, 각 단계에 대해 열, 인덱스, 제약조건을 추가하고 제거하는 단계들을 정의합니다. 각 테이블의 접두사는 주명(州名)이며, TIGER 스키마 안의 테이블을 승계합니다. 예: tiger.faces 를 승계하는 tiger_data.ma_faces 테이블을 생성하십시오.

2.1.0 버전부터 사용할 수 있습니다.



사용자가 tiger_2010 버전을 사용중이었는데 주(州)를 tiger_2011 버전으로 다시 로드하려 할 경우, 이 스크립트를 실행하기 전에 제일 먼저 삭제 선언문 Drop_Nation_Tables_Generate_Script 를 생성하고 실행해야 할 것입니다.

예시

윈도우 상에서 국가 데이터를 로드하기 위한 스크립트를 생성합니다.

```
SELECT loader_generate_nation_script('windows');
```

리눅스/유닉스 시스템 상에서 데이터를 로드하기 위한 스크립트를 생성합니다.

```
SELECT loader_generate_nation_script('sh');
```

참고: Loader_Generate_Script

13.1.12. Missing_Indexes_Generate_Script

Missing_Indexes_Generate_Script — 지오코딩 도구의 결합(join)에 이용되는 키(key) 열을 가진 테이블 가운데 해당 열에서 없어진 인덱스를 모두 찾아서 해당 테이블에 대한 인덱스를 정의하는 SQL DDL 을 출력합니다.

Synopsis

text **Missing_Indexes_Generate_Script**();

설명: tiger 및 tiger_data 스키마에서 지오코딩 도구의 결합(join)에 이용되는 키(key) 열을 가진 테이블 가운데 해당 열에서 없어진 인덱스를 모두 찾아서 해당 테이블에 대한 인덱스를 정의하는 SQL DDL 을 출력합니다. 이 함수는 로드 과정에서 없어졌을 수도 있는, 쿼리를 더 빨리 하기 위해 필요한 새 인덱스들을 추가하는 도우미 함수입니다. 지오코딩 도구가 향상되면, 사용되는 새 인덱스를 수용하기 위해 이 함수도 업데이트될 것입니다. 이 함수가 아무것도 출력하지 않을 경우, 모든 테이블이 이미 키 인덱스로 간주되는 것을 가지고 있다는 뜻입니다.

2.0.0 버전부터 사용할 수 있습니다.

예시

```
  SELECT missing_indexes_generate_script();
  -- 출력물: 로드 작업 스크립트에 많은 수정이 가해지기 전에 생성된 데이터베이스 상에서 실행한 것입니다. --
  CREATE INDEX idx_tiger_county_countyfp ON tiger.county USING btree(countyfp);
  CREATE INDEX idx_tiger_cousub_countyfp ON tiger.cousub USING btree(countyfp);
  CREATE INDEX idx_tiger_edges_tfidr ON tiger.edges USING btree(tfidr);
  CREATE INDEX idx_tiger_edges_tfidl ON tiger.edges USING btree(tfidl);
  CREATE INDEX idx_tiger_zip_lookup_all_zip ON tiger.zip_lookup_all USING btree(zip);
  CREATE    INDEX    idx_tiger_data_ma_county_countyfp    ON    tiger_data.ma_county    USING
btree(countyfp);
  CREATE    INDEX    idx_tiger_data_ma_cousub_countyfp    ON    tiger_data.ma_cousub    USING
btree(countyfp);
  CREATE    INDEX    idx_tiger_data_ma_edges_countyfp    ON    tiger_data.ma_edges    USING
btree(countyfp);
  CREATE    INDEX    idx_tiger_data_ma_faces_countyfp    ON    tiger_data.ma_faces    USING
btree(countyfp);
```

참고: Loader_Generate_Script, Install_Missing_Indexes

13.1.13. Normalize_Address

Normalize_Address — 문자형 도로 주소를 입력받아, 도로 접미사, 접두사 및 유형을 표준화하고 도로, 도로명 등을 개별 필드로 분해한 norm_addy 합성 유형을 반환합니다. 이 함수는 tiger_geocoder 와 함께 패키징된 색인 데이터만 이용해서 (TIGER 인구조사 데이터는 필요없이) 작동할 것입니다.

Synopsis

norm_addy **normalize_address**(varchar *in_address*);

설명: 문자형 도로 주소를 입력받아, 도로 접미사, 접두사 및 유형을 표준화하고 도로, 도로명 등을 개별 필드로 분해한 norm_addy 합성 유형을 반환합니다. 모든 주소를 정규화된 우편 형식으로 변환하기 위한 지오코딩 과정의 첫 번째 단계입니다. 지오코딩 도구와 함께 패키징된 데이터 외에 다른 데이터는 필요하지 않습니다.

이 함수는 tiger 스키마에 있고 tiger_geocoder 와 함께 미리 로드돼 있는 여러 방향/주(州)/접미사 색인 테이블만 이용합니다. 따라서 이 함수를 이용하기 위해 TIGER 인구조사 데이터 또는 다른 추가적인 데이터를 다운로드할 필요가 없습니다. tiger 스키마에 있는 여러 색인 테이블에 더 많은 약어 또는 대체 명칭들을 추가해야 할 필요가 있을 수도 있습니다.

이 함수는 입력 주소를 정규화하기 위해 tiger 스키마에 있는 여러 제어 색인 테이블을 이용합니다.

이 함수는 norm_addy 유형 객체 안의 필드들을 다음과 같은 순서로 반환합니다. 이때 ()가 지오코딩 도구가 요구하는 필드를 나타내고, []는 선택적인 필드를 나타냅니다:

(address) [predirAbbrev] (streetName) [streetTypeAbbrev] [postdirAbbrev] [internal] [location] [stateAbbrev] [zip]

1. address 는 정수형입니다: 도로 번지수입니다.

2. predirAbbrev 는 varchar 형입니다: N, S, E, W 등과 같은 도로의 방향을 의미하는 접두사입니다. direction_lookup 테이블을 이용해서 이 필드를 제어합니다.

3. streetName 은 varchar 형입니다.

4. streetTypeAbbrev 는 varchar 형으로, St, Ave, Cir 처럼 도로 유형의 축약 버전입니다. street_type_lookup 테이블을 이용해서 이 필드를 제어합니다.

5. postdirAbbrev 는 varchar 형으로, N, S, E, W 등과 같은 도로의 방향을 의미하는 접미사의 축약 버전입니다. direction_lookup 테이블을 이용해서 이 필드를 제어합니다.

6. internal 은 varchar 형입니다. 아파트 또는 빌라의 동호수와 같은 내부 주소입니다.

7. location 은 varchar 형으로, 일반적으로 도시 또는 지자체를 나타냅니다.

8. stateAbbrev 는 varchar 형으로, MA, NY, MI 처럼 두 글자로 표현한 미국의 주명(州名)입니다. state_lookup 테이블을 이용해서 이 필드를 제어합니다.

9. zip 은 varchar 형입니다. 02109 같은 다섯 자리 집코드입니다.

10.parsed 는 불(boolean)형입니다. 정규화 처리해서 주소를 형성했는지 여부를 표시합니다. normalize_address 함수는 주소를 반환하기 전에 이 필드를 참으로 설정합니다.

예시

선택한 필드를 출력합니다. 보기 좋은 문자형 출력물을 원한다면 Pprint_Addy 함수를 이용하십시오.

```
SELECT address As orig, (g.na).streetname, (g.na).streettypeabbrev
 FROM (SELECT address, normalize_address(address) As na
       FROM addresses_to_geocode) As g;

             orig              | streetname  | streettypeabbrev
-------------------------------+-------------+------------------
 28 Capen Street, Medford, MA             | Capen     | St
 124 Mount Auburn St, Cambridge, Massachusetts 02138 | Mount Auburn | St
 950 Main Street, Worcester, MA 01610         | Main      | St
 529 Main Street, Boston MA, 02129          | Main      | St
 77 Massachusetts Avenue, Cambridge, MA 02139    | Massachusetts | Ave
 25 Wizard of Oz, Walaford, KS 99912323        | Wizard of Oz |
```

참고: Geocode, Pprint_Addy

13.1.14. Pagc_Normalize_Address

Pagc_Normalize_Address — 문자형 도로 주소를 입력받아, 도로 접미사, 접두사 및 유형을 표준화하고 도로, 도로명 등을 개별 필드로 분해한 norm_addy 합성 유형을 반환합니다. 이 함수는 tiger_geocoder 와 함께 패키징된 색인 데이터만 이용해서 (TIGER 인구조사 데이터는 필요없이) 작동할 것입니다. address_standardizer 확장 프로그램이 필요합니다.

Synopsis

norm_addy **pagc_normalize_address**(varchar *in_address*);

설명: 문자형 도로 주소를 입력받아, 도로 접미사, 접두사 및 유형을 표준화하고 도로, 도로명 등을 개별 필드로 분해한 norm_addy 합성 유형을 반환합니다. 모든 주소를 정규화된 우편 형식으로 변환하기 위한 지오코딩 과정의 첫 번째 단계입니다. 지오코딩 도구와 함께 패키징된 데이터 외에 다른 데이터는 필요하지 않습니다.

이 함수는 tiger 스키마에 있고 tiger_geocoder 와 함께 미리 로드돼 있는 여러 pagc_* 색인 테이블만 이용합니다. 따라서 이 함수를 이용하기 위해 TIGER 인구조사 데이터 또는 다른 추가적인 데이터를 다운로드할 필요가 없습니다. tiger 스키마에 있는 여러 색인 테이블에 더 많은 약어 또는 대체 명칭들을 추가해야 할 필요가 있을 수도 있습니다.

이 함수는 입력 주소를 정규화하기 위해 tiger 스키마에 있는 여러 제어 색인 테이블을 이용합니다.

이 함수는 norm_addy 유형 객체 안의 필드들을 다음과 같은 순서로 반환합니다. 이때 ()가 지오코딩 도구가 요구하는 필드를 나타내고, []는 선택적인 필드를 나타냅니다:

Normalize_Address 에 대해 대소문자 규칙 및 서식이 살짝 다른 변종들이 있습니다.

2.1.0 버전부터 사용할 수 있습니다.

This method needs address_standardizer extension.

(address) [predirAbbrev] (streetName) [streetTypeAbbrev] [postdirAbbrev] [internal] [location] [stateAbbrev] [zip]

이 시점에서 address_standardizer 확장 프로그램의 원형 standardaddr 는 norm_addy 보다 약간 다채롭습니다. (국가를 포함한) 국제 주소를 지원하도록 설계됐기 때문입니다. standardaddr 의 상응하는 필드들은 다음과 같습니다:

house_num, predir, name, suftype, sufdir, unit, city, state, postcode

1. address 는 정수형입니다: 도로 번지수입니다.

2. predirAbbrev 는 varchar 형입니다: N, S, E, W 등과 같은 도로의 방향을 의미하는 접두사입니다. direction_lookup 테이블을 이용해서 이 필드를 제어합니다.

3. streetName 은 varchar 형입니다.

4. streetTypeAbbrev 는 varchar 형으로, St, Ave, Cir 처럼 도로 유형의 축약 버전입니다. street_type_lookup 테이블을 이용해서 이 필드를 제어합니다.

5. postdirAbbrev 는 varchar 형으로, N, S, E, W 등과 같은 도로의 방향을 의미하는 접미사의 축약 버전입니다. direction_lookup 테이블을 이용해서 이 필드를 제어합니다.

6. internal 은 varchar 형입니다. 아파트 또는 빌라의 동호수와 같은 내부 주소입니다.

7. location 은 varchar 형으로, 일반적으로 도시 또는 지자체를 나타냅니다.

8. stateAbbrev 는 varchar 형으로, MA, NY, MI 처럼 두 글자로 표현한 미국의 주명(州名)입니다. state_lookup 테이블을 이용해서 이 필드를 제어합니다.

9. zip 은 varchar 형입니다. 02109 같은 다섯 자리 집코드입니다.

10.parsed 는 불(boolean)형입니다. 정규화 처리해서 주소를 형성했는지 여부를 표시합니다. normalize_address 함수는 주소를 반환하기 전에 이 필드를 참으로 설정합니다.

예시

단일 호출 예시

```
 SELECT addy.*
 FROM pagc_normalize_address('9000 E ROO ST STE 999, Springfield, CO') AS addy;

  address | predirabbrev | streetname | streettypeabbrev | postdirabbrev | internal | location  | stateabbrev | zip |
parsed
 ---------+--------------+------------+------------------+---------------+----------+-----------+-------------+-----
+--------
   9000 | E      | ROO    | ST       |       | SUITE 999 | SPRINGFIELD | CO    |  | t
```

배치 호출 예시. 현재 postgis_tiger_geocoder 가 address_standardizer 를 감싸는 방식에 대한 속도 관련 문제점들이 있습니다. 바라건대 향후 버전에서 이 문제점들을 해결할 수 있을 것입니다. 이 문제점들을 피하려면, 배치 지오코딩 작업이 배치 모드에서 normaddy 를 생성하도록 호출하는 데 속도를 높힐 필요가 있을 경우, Geocode 에서 생성한 데이터를 이용하는 Normalize_Address 의 예시와 유사한 다음 예시에서처럼 address_standardizer 및 standardize_address 함수를 직접 호출하도록 권장합니다.

```
 WITH g AS (SELECT address, ROW((sa).house_num, (sa).predir, (sa).name
   ,   (sa).suftype,   (sa).sufdir,   (sa).unit   ,   (sa).city,   (sa).state,
(sa).postcode, true)::norm_addy As na
  FROM (SELECT address, standardize_address('tiger.pagc_lex'
       , 'tiger.pagc_gaz'
       , 'tiger.pagc_rules', address) As sa
        FROM addresses_to_geocode) As g)
 SELECT address As orig, (g.na).streetname, (g.na).streettypeabbrev
  FROM  g;

 orig                          | streetname  | streettypeabbrev
 ------------------------------+-------------+------------------
 529 Main Street, Boston MA, 02129          | MAIN      | ST
 77 Massachusetts Avenue, Cambridge, MA 02139     | MASSACHUSETTS | AVE
 25 Wizard of Oz, Walaford, KS 99912323        | WIZARD OF   |
 26 Capen Street, Medford, MA             | CAPEN     | ST
 124 Mount Auburn St, Cambridge, Massachusetts 02138 | MOUNT AUBURN | ST
 950 Main Street, Worcester, MA 01610         | MAIN      | ST
```

참고: Normalize_Address, Geocode

13.1.15. Pprint_Addy

Pprint_Addy — norm_addy 복합 유형 객체를 입력받아, 해당 객체의 보기 좋은 인쇄용 표현을 반환합니다. 일반적으로 normalize_address 함수와 결합해서 쓰입니다.

Synopsis

varchar **pprint_addy**(norm_addy *in_addy*);

설명: norm_addy 복합 유형 객체를 입력받아, 해당 객체의 보기 좋은 인쇄용 표현을 반환합니다. 지오코딩 도구와 함께 패키징된 데이터 이외에 다른 데이터는 불필요합니다.

일반적으로 Normalize_Address 함수와 결합해서 쓰입니다.

예시

단일 주소의 보기 좋은 인쇄용 출력

```
  SELECT pprint_addy(normalize_address('202 East Fremont Street, Las Vegas,
Nevada 89101')) As pretty_address;
           pretty_address
  ---------------------------------------
   202 E Fremont St, Las Vegas, NV 89101

  주소 테이블의 보기 좋은 인쇄용 출력
  SELECT   address   As   orig,   pprint_addy(normalize_address(address))   As
pretty_address
        FROM addresses_to_geocode;

             orig             |       pretty_address
  ----------------------------+-----------------------------------
  529 Main Street, Boston MA, 02129          | 529 Main St, Boston MA, 02129
  77 Massachusetts Avenue, Cambridge, MA 02139      | 77 Massachusetts Ave, Cambridge, MA
02139
  28 Capen Street, Medford, MA             | 28 Capen St, Medford, MA
  124 Mount Auburn St, Cambridge, Massachusetts 02138 | 124 Mount Auburn St, Cambridge, MA
02138
  950 Main Street, Worcester, MA 01610         | 950 Main St, Worcester, MA 01610
```

참고: Normalize_Address

13.1.16. Reverse_Geocode

Reverse_Geocode — 알려진 공간 참조 시스템으로 투영된 도형 포인트를 입력받아 이론적으로 가능한 주소들의 배열과 교차로의 배열을 담고 있는 레코드를 반환합니다. include_strnum_range = true 일 경우, 교차로에 있는 도로 범위를 포함합니다.

Synopsis

record **Reverse_Geocode**(geometry *pt*, boolean *include_strnum_range=false*, geometry[] *OUT intpt*, norm_addy[] *OUT addy*, varchar[] *OUT street*);

설명: 알려진 공간 참조 시스템으로 투영된 도형 포인트를 입력받아 이론적으로 가능한 주소들의 배열과 교차로의 배열을 담고 있는 레코드를 반환합니다. include_strnum_range = true 일 경우, 교차로에 있는 도로 범위를 포함합니다. 따로 설정하지 않을 경우 include_strnum_range 의 기본값은 거짓입니다. 포인트가 어떤 도로에 가장 가까운지에 따라 주소를 정렬하기 때문에 첫 번째 주소가 대부분 맞는 편입니다.

실제 주소라고 하는 대신 이론적이라고 하는 이유가 뭘까요. TIGER 데이터는 실제 주소를 가지고 있지 않고, 다만 도로 범위만 가지고 있습니다. 그러므로 이론적 주소란 도로 범위에 기반해서 보간된 주소입니다. 예를 들어 어떤 실제 주소를 보간하면 26 Court Sq. 라는 주소가 실재하지 않는데도 불구하고 26 Court St. 와 26 Court Sq. 를 반환합니다. 왜냐하면 포인트가 두 도로의 교차점에 위치할 수도 있으므로 로직이 두 도로 모두를 따라 보간하기 때문입니다. 이 로직은 또 주소들이 도로를 따라 균등한 간격으로 위치한다고 가정하는데, 물론 지자체 건물이 도로 범위의 큰 부분을 차지하고 나머지 건물들이 도로 끝 부분에 몰려 있을 수도 있기 때문에 이 가정은 틀렸습니다.

주의: 이 함수는 TIGER 데이터에 의존합니다. 해당 포인트의 지역을 커버하는 데이터를 로드하지 않았을 경우, NULL 로 채워진 레코드를 반환할 것입니다.

반환된 레코드를 구성하는 요소는 다음과 같습니다:

1. intpt 은 포인트 배열입니다: 입력 포인트에 가장 가까운 도로 상의 중심선 포인트들입니다. 주소의 개수 만큼 많은 포인트들이 있습니다.

2. addy 는 norm_addy(정규화된 주소) 배열입니다: 입력 포인트에 적합한, 가능한 주소들의 배열입니다. 배열의 첫 번째 주소가 가장 올바를 가능성이 큽니다. 일반적으로, 포인트가 도로 2 개 또는 3 개의 교차점에 위치한 경우 또는 포인트가 도로의 어느 한 편이 아니라 도로 상에 위치한 경우를 제외하면, 주소 1 개만 있어야 합니다.

3. street 는 varchar 형 배열입니다: 교차로의 도로들(또는 도로 1 개)입니다(교차하는 도로들 또는 포인트가 그 위에 투영된 도로입니다).

2.0.0 버전부터 사용할 수 있습니다.

예시

포인트가 두 도로의 교차점에 있지만, 어느 한 도로에 더 가까운 경우의 예시입니다. 이 포인트는 MIT - 77 Massachusetts Ave, Cambridge, MA 02139 - 에 매우 가까운 위치에 있습니다. 도로 3 개의 경우는 아니지만, PostgreSQL 이 상한(上限; upper bound)을 넘어서는 항목에 대해 NULL 을 반환할 것이기 때문에 안전하게 이용할 수 있다는 점에 주의하십시오. 도로 범위도 포함합니다.

```
  SELECT pprint_addy(r.addy[1]) As st1, pprint_addy(r.addy[2]) As st2, pprint_addy(r.addy[3])
As st3,
        array_to_string(r.street, ',') As cross_streets
      FROM reverse_geocode(ST_GeomFromText('POINT(-71.093902 42.359446)',4269),true) As r;

  result
  ------
     st1                   | st2 | st3 |        cross_streets
  -------------------------+-----+-----+---------------------------------------------
   67 Massachusetts Ave, Cambridge, MA 02139 |     |     | 67 - 127 Massachusetts Ave,32 - 88
Vassar St
```

다음 예시는 교차로에 대해 주소 범위를 포함하지 않도록 선택하고, 두 도로의 교차점에 정말로 가까운 위치를 골라서 서로 다른 두 주소들을 반환할 수 있도록 합니다.

```
SELECT pprint_addy(r.addy[1]) As st1, pprint_addy(r.addy[2]) As st2,
pprint_addy(r.addy[3]) As st3, array_to_string(r.street, ',') As cross_str
FROM reverse_geocode(ST_GeomFromText('POINT(-71.06941 42.34225)',4269)) As r;

result
--------
        st1         |         st2         | st3 | cross_str
--------------------+---------------------+-----+------------------------
 5 Bradford St, Boston, MA 02118 | 49 Waltham St, Boston, MA 02118 |   | Waltham St
```

다음 예시는 Geocode 의 지오코딩된 예시를 다시 이용해서 도로가 최대한 2 개만 교차하는 교차로에 있는 최선의 주소만을 반환하도록 합니다.

```
SELECT actual_addr, lon, lat, pprint_addy((rg).addy[1]) As int_addr1,
  (rg).street[1] As cross1, (rg).street[2] As cross2
FROM (SELECT address As actual_addr, lon, lat,
  reverse_geocode( ST_SetSRID(ST_Point(lon,lat),4326) ) As rg
  FROM addresses_to_geocode WHERE rating
> -1) As foo;

          actual_addr          |  lon  |  lat  |        int_addr1        |   cross1   | cross2
-------------------------------+-------+-------+-------------------------+------------+------------
  529 Main Street, Boston MA, 02129           | -71.07181 | 42.38359 | 527 Main St, Boston, MA 02129        |
Medford St   |
  77 Massachusetts Avenue, Cambridge, MA 02139     | -71.09428 | 42.35988 | 77 Massachusetts Ave, Cambridge, MA
02139 | Vassar St    |
  26 Capen Street, Medford, MA            | -71.12377 | 42.41101 | 9 Edison Ave, Medford, MA 02155      | Capen
St    | Tesla Ave
  124 Mount Auburn St, Cambridge, Massachusetts 02138 | -71.12304 | 42.37328 | 3 University Rd, Cambridge, MA
02138   | Mount Auburn St |
  950 Main Street, Worcester, MA 01610         | -71.82368 | 42.24956 | 3 Maywood St, Worcester, MA 01603     |
Main St     | Maywood Pl
```

참고: Pprint_Addy, Geocode

13.1.17. Topology_Load_Tiger

Topology_Load_Tiger — PostGIS 지형에 TIGER 데이터의 정의된 지역을 로드하면서 TIGER 데이터를 지형의 공간 참조 시스템으로 변환하고 지형의 정확도 허용 오차에 스냅시킵니다.

Synopsis

text **Topology_Load_Tiger**(varchar *topo_name*, varchar *region_type*, varchar *region_id*);

설명: PostGIS 지형에 TIGER 데이터의 정의된 지역을 로드합니다. 면, 노드 및 경계선을 대상 지형의 공간 참조 시스템으로 변환하고 대상 지형의 허용 오차에 포인트들을 스냅시킵니다. 이렇게 생성된 면, 노드, 경계선이 원본 TIGER 데이터의 면, 노드, 경계선과 동일한 ID 를 유지하기 때문에, 이 데이터셋은 향후 TIGER 데이터와 더 쉽게 함께 쓰일 수 있습니다. 처리 과정에 대한 요약 설명문을 반환합니다.

이 함수는, 예를 들어, 도로의 중심선을 따라 새로 형성된 폴리곤이 필요한데 이 폴리곤들이 서로 중첩해서는 안 될 경우, 데이터를 제약하는 데 유용합니다.



이 함수는 TIGER 데이터는 물론 PostGIS 지형의 설치에 의존적입니다. 더 자세한 정보를 원한다면 Chapter 11, 지형(topology) 및 Section 2.4.1, “설정” 을 참조하십시오. 관심지역을

커버하는 데이터를 로드하지 않았을 경우, 어떤 지형 레코드도 생성하지 않을 것입니다. 또한 지형 함수를 이용해서 지형을 생성하지 않았을 경우에도 오류를 발생시킬 것입니다.



지형 확인 오류 대부분은 변환 작업 후 경계선 및 포인트가 제대로 정렬되지 않거나 중첩되지 않는 허용 오차 문제입니다. 지형 확인 오류가 발생한 경우, 이런 상황을 해결하려면 정확도를 높이거나 낮추는 편이 좋을 수도 있습니다.

필수 인수:

1. topo_name - 데이터를 로드할 기존 PostGIS 지형의 명칭입니다.
2. region_type - 경계를 이루는 지역의 유형입니다. 현재 place 와 county 만 지원합니다. 몇몇 유형을 더 지원할 예정입니다. 이 인수는 tiger.place, tiger.county 처럼 지역 경계를 정의하려면 살펴봐야 할 테이블입니다.
3. region_id - TIGER 가 지리 ID(geoid)라고 부르는 식별자입니다. 테이블에 있는 지역의 유일한 식별자입니다. place 의 경우 tiger.place 테이블에 있는 plcidfp 열입니다. county 의 경우 tiger.county 테이블에 있는 cntyidfp 열입니다.

2.0.0 버전부터 사용할 수 있습니다.

예시: 매사추세츠 주 보스턴 시의 지형

매사추세츠 주 피트 단위 평면(2249)에서 매사추세츠 주 보스턴 시에 대해 허용 오차 0.25 피트를 가진 지형을 생성한 다음, 보스턴 시의 TIGER 면, 경계선, 노드를 로드합니다.

```
  SELECT topology.CreateTopology('topo_boston', 2249, 0.25);
  createtopology
  --------------
     15
  -- 윈도우 7 데스크탑에서 9.1 버전을 (5 개 주의 TIGER 데이터를 로드한 상태로) 실행했을 때
60,902 밀리초 ~ 1 분 소요
  SELECT tiger.topology_load_tiger('topo_boston', 'place', '2507000');
  -- topology_loader_tiger --
  29722 edges holding in temporary. 11108 faces added. 1875 edges of faces
added.  20576 nodes added.
  19962 nodes contained in a face.  0 edge start end corrected.  31597 edges
added.

  -- 41 밀리초 소요 --
  SELECT topology.TopologySummary('topo_boston');
   -- topologysummary--
  Topology topo_boston (15), SRID 2249, precision 0.25
  20576 nodes, 31597 edges, 11109 faces, 0 topogeoms in 0 layers

  -- 확인 작업에 28,797 밀리초 소요, 발생한 오류 없음 --
  SELECT * FROM
     topology.ValidateTopology('topo_boston');

       error      |   id1    |    id2
  ----------------+---------+--------------------
```

예시: 매사추세츠 주 서퍽 군의 지형

매사추세츠 주 미터 단위 평면(26986)에서 매사추세츠 주 서퍽 군에 대해 허용 오차 0.25 미터를 가진 지형을 생성한 다음, 서퍽 군의 TIGER 면, 경계선, 노드를 로드합니다.

```
  SELECT topology.CreateTopology('topo_suffolk', 26986, 0.25);
  -- 윈도우 7 32 비트 상에서 5 개 주의 TIGER 데이터를 로드한 상태로 실행했을 때 56,275 밀리초
~ 1 분 소요
  -- 보스턴 데이터를 로드한 다음 예열됐음이 분명함
  SELECT tiger.topology_load_tiger('topo_suffolk', 'county', '25025');
  -- topology_loader_tiger --
   36003 edges holding in temporary. 13518 faces added. 2172 edges of faces
added.
   24761 nodes added.  24075 nodes contained in a face.  0 edge start end
corrected.  38175 edges added.
  -- 31 밀리초 소요 --
  SELECT topology.TopologySummary('topo_suffolk');
   -- topologysummary--
   Topology topo_suffolk (14), SRID 26986, precision 0.25
  24761 nodes, 38175 edges, 13519 faces, 0 topogeoms in 0 layers

  -- 확인 작업에 33,606 밀리초 소요 --
  SELECT * FROM
     topology.ValidateTopology('topo_suffolk');

        error      |   id1    |    id2
  ----------------+----------+-----------
   coincident nodes  | 81045651 |  81064553
   edge crosses node | 81045651 |  85737793
   edge crosses node | 81045651 |  85742215
   edge crosses node | 81045651 | 620628939
   edge crosses node | 81064553 |  85697815
   edge crosses node | 81064553 |  85728168
   edge crosses node | 81064553 |  85733413
```

참고: CreateTopology, CreateTopoGeom, TopologySummary, ValidateTopology

13.1.18. Set_Geocode_Setting

Set_Geocode_Setting — 지오코딩 도구 함수들의 습성에 영향을 미치는 설정값을 설정합니다.

Synopsis

text **Set_Geocode_Setting**(text *setting_name*, text *setting_value*);

설명: tiger.geocode_settings 테이블에 저장된 특정 설정값의 값을 설정합니다. 사용자가 함수의 디버깅을 켜고 끌 수 있게 해주는 설정값입니다. 향후 설정값으로 순위를 제어할 수 있도록 할 계획입니다. Get_Geocode_Setting 에서 현재 설정값 목록을 볼 수 있습니다.

2.1.0 버전부터 사용할 수 있습니다.

예시: 디버깅 설정값 반환

이 함수가 참일 때 Geocode 를 실행할 경우, NOTICE 로그가 소요 시간 및 쿼리를 출력할 것입니다.

```
SELECT set_geocode_setting('debug_geocode_address', 'true') As result;
result
---------
true
```

참고: Get_Geocode_Setting

Chapter 14. PostGIS 공간 함수 색인

14.1. PostGIS 종합 함수

다음 함수들은 Sum, Average 같은 다른 어떤 SQL 종합 함수처럼 사용할 수 있는, PostGIS 가 제공하는 공간 종합 함수입니다.

- ST_3DExtent - 도형 행들의 경계를 이루는 BOX3D 경계 상자를 반환하는 종합 함수입니다.
- ST_Accum - 종합 함수입니다. 도형들의 배열을 작성합니다.
- ST_ClusterIntersecting - 종합 함수입니다. 도형들의 집합의 연결된 구성 요소들을 가진 배열을 반환합니다.
- ST_ClusterWithin - 종합 함수입니다. 도형 집합의 배열을 반환합니다. 이때 도형 집합은 설정된 거리 미만으로 떨어진 도형들의 집합을 나타냅니다.
- ST_Collect - 다른 도형들의 집합으로부터 설정된 ST_Geometry 값을 반환합니다.
- ST_Extent - 도형 행들의 경계를 이루는 경계 상자를 반환하는 종합 함수입니다.
- ST_MakeLine - 포인트, 멀티포인트 또는 라인 도형으로부터 라인스트링을 생성합니다.
- ST_MemUnion - 메모리 친화적이라는 점만 제외하면 ST_Union 함수와 동일합니다(메모리를 덜 사용하고 CPU 시간을 더 사용합니다).
- ST_Polygonize - 종합 함수입니다. 도형들의 집합의 선분 구성 요소로부터 형성될 수 있는 폴리곤들을 담고 있는 도형 집합을 생성합니다.
- ST_SameAlignment - 래스터들이 동일한 기울기, 축척, 공간 참조 시스템, 그리고 오프셋을 가지고 있을 경우 (픽셀이 다른 픽셀을 침범하지 않고 동일한 그리드에 놓일 수 있을 경우) 참을 반환하고, 그렇지 않을 경우 문제점을 설명하는 안내문과 함께 거짓을 반환합니다.
- ST_Union - 도형들의 포인트들을 통합한 집합을 표현하는 도형을 반환합니다.
- TopoElementArray_Agg - element_id, 유형 배열(topoelement)들의 집합에 대해 topoelementarray 를 반환합니다.

14.2. PostGIS Window Functions

다음 함수들은 row_numer(), lead(), lag() 같은 다른 어떤 SQL 윈도우 함수처럼 사용할 수 있는, PostGIS 가 제공하는 공간 윈도우 함수입니다. 이 함수들은 모두 SQL OVER() 절을 필요로 합니다.

- ST_ClusterDBSCAN - 각 입력 도형이 있는 클러스터에 대한 정수 ID 를 DBSCAN(Density-Based Spatial Clustering of Applications with Noise) 알고리즘의 2 차원 구현에 기반해서 반환하는, 서로 다른 데이터를 윈도를 통해 동시에 한 화면에 표시하는 함수입니다.
- ST_ClusterKMeans - 각 입력 도형이 있는 클러스터에 대한 정수 ID 를 반환하는, 서로 다른 데이터를 윈도를 통해 동시에 한 화면에 표시하는 함수입니다.

14.3. PostGIS SQL-MM 호환 함수

다음 함수들은 SQL/MM 3 표준을 따르는 PostGIS 함수입니다.



SQL-MM 은 모든 도형 구성자의 기본 SRID 를 0 으로 정의합니다. PostGIS 는 SRID 기본값으로 -1 을 씁니다.

- ST_3DDWithin - 3D(Z) 도형 유형에 대해, 두 도형의 3 차원 거리가 단위의 개수 내일 경우 참을 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM ?
- ST_3DDistance - 도형 유형에 대해, 두 도형 사이의 (SRS 에 기반한) 3 차원 데카르트 최단 거리를 투영 단위로 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM ?
- ST_3DIntersects - 도형들이 3 차원에서 "공간적으로 교차"하는 경우 참을 반환합니다. 포인트, 라인스트링, 폴리곤, 다면체 표면(면적)만 입력받습니다. SFCGAL 백엔드가 활성화됐다면 TIN 도 지원합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: ?
- ST_AddEdgeModFace - 새 경계선을 추가하고, 그 과정에서 경계선이 표면을 분할할 경우, 원본 표면을 수정하고 새 표면을 추가합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM: Topo-Geo and Topo-Net 3: Routine Details: X.3.13
- ST_AddEdgeNewFaces - 새 경계선을 추가하고, 그 과정에서 경계선이 표면을 분할할 경우, 원본 표면을 삭제하고 새 표면 2 개로 교체합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM: Topo-Geo and Topo-Net 3: Routine Details: X.3.12
- ST_AddIsoEdge - 지형에 두 개의 고립된 기존 노드 anode 와 anothernode 를 연결하는 alinestring 도형이 정의하는 고립된 경계선을 추가하고 새 경계선의 ID 를 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM: Topo-Geo and Topo-Net 3: Routine Details: X.3.4
- ST_AddIsoNode - 지형 안의 표면에 고립된(isolated) 노드를 추가하고 새 노드의 ID 를 반환합니다. 표면이 NULL 일 경우, 그래도 노드를 생성합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM: Topo-Net Routines: X+1.3.1
- ST_Area - 도형이 폴리곤이나 멀티폴리곤일 경우 표면의 면적을 반환합니다. 도형 유형의 경우, SRID 가 지정한 단위로 2 차원 데카르트 면적을 구합니다. 지리형 유형의 경우, 평방미터 단위로 곡면(curved surface)의 면적을 구합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 8.1.2, 9.5.3
- ST_AsBinary - 도형/지리형의 WKB(Well-Known Binary) 표현식을 SRID 메타데이터 없이 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.37
- ST_AsText - 도형/지리형의 WKT(Well-Known Text) 표현식을 SRID 메타데이터 없이 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.25

- ST_Boundary - 해당 도형의 결합된 범위의 닫힘 여부를 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.14
- ST_Buffer - 입력 도형으로부터 주어진 거리 안에 있는 모든 포인트들을 커버하는 도형을 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.17
- ST_Centroid - 도형의 기하학적 중심을 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 8.1.4, 9.5.5
- ST_ChangeEdgeGeom - 지형 구조를 건드리는 일 없이 경계선의 형상을 변경합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM: Topo-Geo and Topo-Net 3: Routine Details X.3.6
- ST_Contains - A 외부에 놓인 B 의 포인트가 하나도 없고, 적어도 B 내부의 포인트 하나가 A 의 내부에 있는 경우 참을 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.31
- ST_ConvexHull - 도형의 볼록 껍질(convex hull)은 집합 내부의 모든 도형들을 둘러싸는 최소 볼록 도형을 의미합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.16
- ST_CoordDim - ST_Geometry 값의 좌표 차원을 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.3
- ST_CreateTopoGeo - 텅 빈 입력 지형에 도형들의 집합을 추가한 다음 작업 성공 여부를 상세하게 설명하는 메시지를 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM: Topo-Geo and Topo-Net 3: Routine Details -- X.3.18
- ST_Crosses - 입력 도형들이 전부가 아닌 일부 내부 포인트를 공유하는 경우 TRUE 를 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.29
- ST_CurveToLine - 원호 스트링(CIRCULARSTRING)/만곡 폴리곤(CURVEPOLYGON)을 라인스트링/폴리곤으로 변환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 7.1.7
- ST_Difference - 도형 A 에서 도형 B 와 교차하지 않는 부분을 표현하는 도형을 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.20
- ST_Dimension - 좌표 차원과 동등하거나 낮아야 하는, 해당 도형 객체의 내재된 차원을 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.2
- ST_Disjoint - 도형들이 "공간적으로 교차"하지 않을 경우 - 어떤 공간도 함께 공유하지 않을 경우 - 참을 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.26
- ST_Distance - 도형 유형에 대해, 두 도형 사이의 2 차원 데카르트 거리를 투영 단위(SRS 단위)로 반환합니다. 지리형 유형의 경우, 기본적으로 두 도형 사이의 측지 거리를 미터 단위로 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.23
- ST_EndPoint - LINESTRING 또는 CIRCULARLINESTRING 도형의 마지막 포인트를 POINT 로 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 7.1.4
- ST_Envelope - 주어진 도형의 이중 정밀도(double precision; float8) 경계 상자를 표현하는 도형을 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.15
- ST_Equals - 주어진 도형들이 동일한 도형일 경우 참을 반환합니다. 방향성은 무시합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.24

- ST_ExteriorRing - POLYGON 도형의 외곽 고리(exterior ring)를 표현하는 라인스트링을 반환합니다. 도형이 폴리곤이 아닌 경우 NULL 을 반환합니다. 멀티폴리곤은 입력받지 못 합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 8.2.3, 8.3.3

- ST_GMLToSQL - GML 로부터 지정된 ST_Geometry 값을 반환합니다. 이 함수는 ST_GeomFromGML 함수와 동일합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.50 (만곡 도형 지원 제외)

- ST_GeomCollFromText - 주어진 SRID 와 함께 집합 WKT 로부터 도형 집합을 만듭니다. SRID 가 주어지지 않은 경우, 기본값인 0 을 씁니다. 이 방법은 SQL/MM 사양을 만족합니다.

- ST_GeomFromText - WKT 표현식으로부터 지정된 ST_Geometry 값을 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.40

- ST_GeomFromWKB - WKB(Well-Known Binary) 도형 표현식과 선택적인 SRID 로부터 도형 인스턴스를 생성합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.41

- ST_GeometryFromText - WKT(Well-Known Text)로부터 지정된 ST_Geometry 값을 반환합니다. 이 함수는 ST_GeomFromText 함수와 동일합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.40

- ST_GeometryN - 입력 도형이 도형 집합, (멀티)포인트, (멀티)라인스트링, 멀티커브(multicurve) 또는 (멀티)폴리곤이나 다면체 표면일 경우 1-기반 N 번째 도형을 반환하며, 그 외의 경우 NULL 을 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 9.1.5

- ST_GeometryType - ST_Geometry 값의 도형 유형을 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.4

- ST_GetFaceEdges - aface 의 경계를 이루는 정렬된 경계선들의 집합을 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3 Topo-Geo and Topo-Net 3: Routine Details: X.3.5

- ST_GetFaceGeometry - 입력 지형으로부터 설정된 표면 ID 를 가진 폴리곤을 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3 Topo-Geo and Topo-Net 3: Routine Details: X.3.16

- ST_InitTopoGeo - 새 지형 스키마를 생성하고 topology.topology 테이블에 이 새 스키마를 등록한 다음 처리 과정의 상세한 요약문을 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3 Topo-Geo and Topo-Net 3: Routine Details: X.3.17

- ST_InteriorRingN - 폴리곤 도형의 N 번째 내곽 라인스트링 고리를 반환합니다. 도형이 폴리곤이 아니거나 주어진 N 이 범위(range)를 벗어난 경우 NULL 을 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 8.2.6, 8.3.5

- ST_Intersection - geomA 와 geomB 의 공유 부분을 표현하는 도형을 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.18

- ST_Intersects - 도형/지리형이 "2 차원에서 공간적으로 교차"(공간의 일부라도 공유)할 경우 참을 반환합니다. 그렇지 않을 경우 (도형/지리형이 서로 분리돼 있을 경우) 거짓을 반환합니다. 지리형의 경우 허용 오차가 0.00001 미터입니다(따라서 10 마이크로미터보다 가까운 포인트들은 교차한다고 여깁니다). 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.27

- ST_IsClosed - LINESTRING 의 시작점과 종단점이 일치하는 경우 TRUE 를 반환합니다. 다면체 표면이 닫혀 (부피를 가지고) 있는 경우 TRUE 를 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 7.1.5, 9.3.3

- ST_IsEmpty - 해당 도형이 텅 빈 도형 집합, 폴리곤, 포인트 등인 경우 TRUE 를 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.7
- ST_IsRing - LINESTRING 이 닫혀 있는 단순 도형인 경우 TRUE 를 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 7.1.6
- ST_IsSimple - 해당 도형이 자체 교차하거나 자체 접촉하는 이례적인 도형 포인트를 가지고 있지 않을 경우 TRUE 를 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.8
- ST_IsValid - ST_Geometry 가 잘 형성돼 있는 경우 TRUE 를 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.9
- ST_Length - 도형이 라인스트링 또는 멀티라인스트링일 경우 도형의 2 차원 길이를 반환합니다. 도형의 단위는 공간 참조 시스템 단위이며 지리형의 단위는 (기본값 회전타원체의) 미터 단위입니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 7.1.2, 9.3.4
- ST_LineFromText - 주어진 SRID 와 함께 WKT 표현식으로부터 도형을 만듭니다. SRID 가 주어지지 않은 경우, 기본값인 0 을 씁니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 7.2.8
- ST_LineFromWKB - 주어진 SRID 와 함께 WKB 로부터 LINESTRING 을 만듭니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 7.2.9
- ST_LinestringFromWKB - 주어진 SRID 와 함께 WKB 로부터 도형을 만듭니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 7.2.9
- ST_M - 포인트의 M 좌표를 반환합니다. M 좌표가 없는 경우 NULL 을 반환합니다. 포인트만 입력받을 수 있습니다. 이 방법은 SQL/MM 사양을 만족합니다.
- ST_MLineFromText - WKT 표현식으로부터 지정된 ST_MultiLineString 값을 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다.SQL-MM 3: 9.4.4
- ST_MPointFromText - 주어진 SRID 와 함께 WKT 표현식으로부터 도형 집합을 만듭니다. SRID 가 주어지지 않은 경우, 기본값인 0 을 씁니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 9.2.4
- ST_MPolyFromText - 주어진 SRID 와 함께 WKT 표현식으로부터 멀티폴리곤을 만듭니다. SRID 가 주어지지 않은 경우, 기본값인 0 을 씁니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 9.6.4
- ST_ModEdgeHeal - 두 경계선을 연결하는 노드를 삭제한 다음, 첫 번째 경계선을 수정하고 두 번째 경계선을 삭제해서 두 경계선을 붙입니다. 삭제한 노드의 ID 를 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM: Topo-Geo and Topo-Net 3: Routine Details: X.3.9
- ST_ModEdgeSplit - 기존 경계선을 따라 새 노드를 추가한 다음, 원본 경계선을 수정하고 새 경계선을 추가해서 경계선을 분할합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM: Topo-Geo and Topo-Net 3: Routine Details: X.3.9
- ST_MoveIsoNode - 지형 안에 있는 고립된 노드를 어떤 위치에서 다른 위치로 이동합니다. 새 apoint 도형의 위치에 노드가 존재할 경우 오류가 발생합니다. 이동을 설명하는 메시지를 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM: Topo-Net Routines: X.3.2
- ST_NewEdgeHeal - 두 경계선을 연결하는 노드를 삭제한 다음, 두 경계선을 모두 삭제하고 첫 번째 경계선과 동일한 방향인 경계선으로 교체합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM: Topo-Geo and Topo-Net 3: Routine Details: X.3.9

- ST_NewEdgesSplit - 기존 경계선을 따라 새 노드를 추가한 다음, 원본 경계선을 삭제하고 새 경계선 2 개로 교체해서 경계선을 분할합니다. 새 경계선들을 결합하는 새로 생성된 노드의 ID 를 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM: Topo-Net Routines: X.3.8

- ST_NumGeometries - 도형이 도형 집합 또는 멀티 도형인 경우 도형의 개수를 반환합니다. 단일 도형인 경우 1 을 반환하고, 도형 집합도 멀티 도형도 단일 도형도 아닌 경우 NULL 을 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 9.1.4

- ST_NumInteriorRings - 폴리곤 도형의 내곽 고리의 개수를 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 8.2.5

- ST_NumPatches - 다면체 표면 상에 있는 면의 개수를 반환합니다. 다면체 도형이 아닌 경우 NULL 을 반환할 것입니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: ?

- ST_NumPoints - ST_LineString 또는 ST_CircularString 값에 있는 포인트의 개수를 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 7.2.4

- ST_OrderingEquals - 주어진 도형들이 동일한 도형이며 포인트들의 순서가 동일한 방향일 경우 참을 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.43

- ST_Overlaps - 도형들이 공간을 공유하고, 동일한 차원이지만, 어느 한 쪽이 다른 쪽을 완전히 담고 있지 않을 경우 참을 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.32

- ST_PatchN - 도형이 POLYHEDRALSURFACE, POLYHEDRALSURFACEM 인 경우 1-기반 N 번째 도형 (면)을 반환합니다. 그 외의 경우 NULL 을 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: ?

- ST_Perimeter - ST_Surface 또는 ST_MultiSurface 도형 또는 지리형(폴리곤, 멀티폴리곤)의 경계선의 길이 측정치를 반환합니다. 도형의 단위는 공간 참조 시스템 단위이며 지리형의 단위는 미터 단위입니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 8.1.3, 9.5.4

- ST_Point - 주어진 좌표값을 가진 ST_Point 를 반환합니다. ST_MakePoint 와 동일한 OGC 함수입니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 6.1.2

- ST_PointFromText - 주어진 SRID 와 함께 WKT 표현식으로부터 포인트 도형을 만듭니다. SRID 가 주어지지 않은 경우, 기본값인 0 을 씁니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 6.1.8

- ST_PointFromWKB - 주어진 SRID 와 함께 WKB 로부터 도형을 만듭니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 6.1.9

- ST_PointN - 도형 안에 있는 첫 번째 라인스트링 또는 원형 라인스트링의 N 번째 포인트를 반환합니다. 음수 값은 라인스트링의 종단점으로부터 반대로 집계된 것입니다. 도형 안에 라인스트링이 없는 경우 NULL 을 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 7.2.5, 7.3.5

- ST_PointOnSurface - 표면에 붙어 있게 될 POINT 를 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 8.1.5, 9.5.6. 사양에 따라, ST_PointOnSurface 는 표면을 가진 도형(POLYGONs, MULTIPOLYGONS, CURVED POLYGONS)을 입력받습니다. 따라서 PostGIS 가 사양서가 허용한 범위를 확장시키는 것 같습니다. Oracle, DB II, ESRI SDE 등 대부분의 데이터베이스는 표면에 대해서만 이 함수를 지원합니다. SQL Server 2008 은, PostGIS 처럼, 모든 일반 도형을 지원합니다.

- ST_Polygon - 설정된 라인스트링과 SRID 로부터 빌드된 폴리곤을 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 8.3.2

- ST_PolygonFromText - 주어진 SRID 와 함께 WKT 표현식으로부터 도형 집합을 만듭니다. SRID 가 주어지지 않은 경우, 기본값인 0 을 씁니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 8.3.6

- ST_Relate - 도형의 내부 사이의 교차점을 확인해서 한 도형이 다른 도형과 공간 관계가 있을 경우 참을 반환합니다. 두 도형의 경계선 및 외부는 intersectionMatrixPattern 안의 값들로 설정됩니다. intersectionMatrixPattern 이 입력되지 않을 경우, 두 도형을 관계시키는 최대 intersectionMatrixPattern 을 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.25

- ST_RemEdgeModFace - 경계선을 제거하고, 제거한 경계선이 두 표면을 분할하고 있을 경우, 표면 하나를 삭제하고 다른 표면을 두 표면의 공간을 차지하도록 수정합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM: Topo-Geo and Topo-Net 3: Routine Details: X.3.15

- ST_RemEdgeNewFace - 경계선을 제거하고, 제거한 경계선이 두 표면을 분할하고 있을 경우, 원본 표면들을 삭제하고 새 표면 하나로 대체합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM: Topo-Geo and Topo-Net 3: Routine Details: X.3.14

- ST_RemoveIsoNode - 고립된 노드를 제거하고 작업 내용을 설명하는 메시지를 반환합니다. 노드가 고립되지 않은 (경계선의 시작점이나 종단점인) 경우, 예외가 발생합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM: Topo-Geo and Topo-Net 3: Routine Details: X+1.3.3

- ST_SRID - spatial_ref_sys 테이블에 정의되어 있는, ST_Geometry 에 대한 공간 참조 식별자를 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.5

- ST_StartPoint - LINESTRING 도형의 첫 번째 포인트를 POINT 로 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 7.1.3

- ST_SymDifference - 도형 A 와 B 의 서로 교차하지 않는 부분들을 표현하는 도형을 반환합니다. ST_SymDifference(A,B) = ST_SymDifference(B,A)이기 때문에 이를 대칭차(symmetric difference)라고 합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.21

- ST_Touches - 입력 도형들이 최소한 포인트 한 개를 공유하지만, 내부가 교차하지는 않을 경우 TRUE 를 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.28

- ST_Transform - 입력 도형의 좌표를 다른 공간 참조 시스템으로 변환시킨 새 도형을 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.6

- ST_Union - 도형들의 포인트들을 통합한 집합을 표현하는 도형을 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.19 폴리곤이 연관된 경우 Z-인덱스(고도)

- ST_WKBToSQL - WKB(Well-Known Binary) 표현식으로부터 지정된 ST_Geometry 값을 반환합니다. 이 함수는 SRID 를 입력받지 않는 ST_GeomFromWKB 함수와 동일합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.36

- ST_WKTToSQL - WKT(Well-Known Text)로부터 지정된 ST_Geometry 값을 반환합니다. 이 함수는 ST_GeomFromText 함수와 동일합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.34

- ST_Within - 도형 A 전체가 도형 B 안에 들어갈 경우 참을 반환합니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 5.1.30

- ST_X - 포인트의 X 좌표를 반환합니다. X 좌표가 없는 경우 NULL 을 반환합니다. 포인트만 입력받을 수 있습니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 6.1.3

- ST_Y - 포인트의 Y 좌표를 반환합니다. Y 좌표가 없는 경우 NULL 을 반환합니다. 포인트만 입력받을 수 있습니다. 이 방법은 SQL/MM 사양을 만족합니다. SQL-MM 3: 6.1.4
- ST_Z - 포인트의 Z 좌표를 반환합니다. Z 좌표가 없는 경우 NULL 을 반환합니다. 포인트만 입력받을 수 있습니다. 이 방법은 SQL/MM 사양을 만족합니다.

14.4. PostGIS 지리형 지원 함수

다음 함수 및 연산자들은 지리형 데이터형 객체를 입력받거나 산출물로 반환하는 PostGIS 함수/연산자입니다.



(T)가 붙는 함수들은 태생적인 측지 함수가 아니며, 연산을 하기 위해 지리형에서 또는 지리형으로 호출할 때 ST_Transform 함수를 이용합니다. 따라서 날짜변경선을 넘거나, 극을 넘거나, 하나 이상의 UTM 영역을 커버하는 넓은 도형 또는 지리형 쌍을 처리할 때, 예상대로 동작하지 않을 수도 있습니다. UTM 과 람베르트 방위도법(북/남)을 선호하며 최악의 경우 메르카토르 도법으로 후퇴하는 기본 변형 방법을 쓰는 편이 좋습니다.

- ST_Area - 도형이 폴리곤이나 멀티폴리곤일 경우 표면의 면적을 반환합니다. 도형 유형의 경우, SRID 가 지정한 단위로 2 차원 데카르트 면적을 구합니다. 지리형 유형의 경우, 평방미터 단위로 곡면(curved surface)의 면적을 구합니다.
- ST_AsBinary - 도형/지리형의 WKB(Well-Known Binary) 표현식을 SRID 메타데이터 없이 반환합니다.
- ST_AsEWKT - 도형의 WKT(Well-Known Text) 표현식을 SRID 메타데이터와 함께 반환합니다.
- ST_AsGML - 도형을 GML 2 또는 GML 3 버전 요소로 반환합니다.
- ST_AsGeoJSON - 도형을 GeoJSON 요소로 반환합니다.
- ST_AsKML - 도형을 KML 요소로 반환합니다. 응용방법이 몇 개 있습니다. 기본값은 버전 2, 정밀도 15 입니다.
- ST_AsSVG - 도형을 주어진 도형 또는 지리형의 SVG 경로 데이터로 반환합니다.
- ST_AsText - 도형/지리형의 WKT(Well-Known Text) 표현식을 SRID 메타데이터 없이 반환합니다.
- ST_Azimuth - 포인트 A 와 포인트 B 를 잇는 선의 각도를, 포인트 A 에 그은 수직선을 기준으로 시계 방향으로 라디안 단위로 각도를 측정해서 북극 기준(north-based) 방위각(azimuth)을 반환합니다.
- ST_Buffer - 입력 도형으로부터 주어진 거리 안에 있는 모든 포인트들을 커버하는 도형을 반환합니다.
- ST_CoveredBy - 도형/지리형 A 안에 있는 어떤 포인트도 도형/지리형 B 외부에 없을 경우 1(참)을 반환합니다.
- ST_Covers - 도형 B 안에 있는 어떤 포인트도 도형 A 외부에 없을 경우 1(참)을 반환합니다.
- ST_DWithin - 도형들이 서로 설정된 거리 안에 있을 경우 참을 반환합니다. 도형의 경우 공간 참조의 단위를 쓰고, 지리형의 경우 미터 단위를 씁니다. 또 지리형의 경우 측정시 use_spheroid=true(회전타원체 상에서 측정)가 기본값이며, 더 빨리 검사하려면 구체 상에서 측정하도록 use_spheroid=false 로 설정하십시오.

- ST_Distance - 도형 유형에 대해, 두 도형 사이의 2 차원 데카르트 거리를 투영 단위(SRS 단위)로 반환합니다. 지리형 유형의 경우, 기본적으로 두 도형 사이의 측지 거리를 미터 단위로 반환합니다.
- ST_GeogFromText - WKT (확장) 표현식으로부터 지정된 지리형 값을 반환합니다.
- ST_GeogFromWKB - WKB 도형 표현식 또는 EWKB(확장 WKB)로부터 지리형 인스턴스를 생성합니다.
- ST_GeographyFromText - WKT (확장) 표현식으로부터 지정된 지리형 값을 반환합니다.
- = - A 의 경계 상자와 B 의 경계 상자가 동일할 경우 TRUE 를 반환합니다. 이중 정밀도 경계 상자를 이용합니다.
- ST_Intersection - geomA 와 geomB 의 공유 부분을 표현하는 도형을 반환합니다.
- ST_Intersects - 도형/지리형이 "2 차원에서 공간적으로 교차"(공간의 일부라도 공유)할 경우 참을 반환합니다. 그렇지 않을 경우 (도형/지리형이 서로 분리돼 있을 경우) 거짓을 반환합니다. 지리형의 경우 허용 오차가 0.00001 미터입니다(따라서 10 마이크로미터보다 가까운 포인트들은 교차한다고 여깁니다).
- ST_Length - 도형이 라인스트링 또는 멀티라인스트링일 경우 도형의 2 차원 길이를 반환합니다. 도형의 단위는 공간 참조 시스템 단위이며 지리형의 단위는 (기본값 회전타원체의) 미터 단위입니다.
- ST_Perimeter - ST_Surface 또는 ST_MultiSurface 도형 또는 지리형(폴리곤, 멀티폴리곤)의 경계선의 길이 측정치를 반환합니다. 도형의 단위는 공간 참조 시스템 단위이며 지리형의 단위는 미터 단위입니다.
- ST_Project - 시작점으로부터 미터 단위 거리 및 라디안 단위 방향(방위각)을 이용해 투영된 POINT 를 반환합니다.
- ST_Segmentize - 주어진 거리보다 더 긴 구간이 없도록 수정된 도형/지리형을 반환합니다.
- ST_Summary - 도형의 내용을 요약한 텍스트를 반환합니다.
- <-> - A 와 B 사이의 2 차원 거리를 반환합니다.
- && - A 의 2D 경계 상자와 B 의 2D 경계 상자가 교차하는 경우 TRUE 를 반환합니다.

14.5. PostGIS 래스터 지원 함수

다음 함수 및 연산자들은 래스터 데이터형 객체를 입력받거나 산출물로 반환하는 PostGIS 함수/연산자입니다. 알파벳 순서로 정렬했습니다.

- Box3D - 래스터를 둘러싼 상자의 BOX3D 표현식을 반환합니다.
- @ - B 의 경계 상자가 A 의 경계 상자를 담고 있을 경우 TRUE 를 반환합니다. 이중 정밀도 경계 상자를 이용합니다.
- ~ - A 의 경계 상자가 B 의 경계 상자를 담고 있을 경우 TRUE 를 반환합니다. 이중 정밀도 경계 상자를 이용합니다.
- = - A 의 경계 상자와 B 의 경계 상자가 동일할 경우 TRUE 를 반환합니다. 이중 정밀도 경계 상자를 이용합니다.

- && - A 의 경계 상자와 B 의 경계 상자가 교차하는 경우 TRUE 를 반환합니다.
- &< - A 의 경계 상자가 B 의 경계 상자 왼쪽에 있을 경우 TRUE 를 반환합니다.
- &> - A 의 경계 상자가 B 의 경계 상자 오른쪽에 있을 경우 TRUE 를 반환합니다.
- ~= - A 의 경계 상자와 B 의 경계 상자가 동일할 경우 TRUE 를 반환합니다.
- ST_Retile - 임의로 타일화된 래스터 커버리지로부터, 설정된 타일들의 집합을 반환합니다.
- ST_AddBand - 입력 인덱스 위치에 입력 초기값으로 추가된 입력 유형의 새 밴드(들)을 가진 래스터를 반환합니다. 인덱스를 설정하지 않을 경우, 마지막 위치에 밴드를 추가합니다.
- ST_AsBinary - 래스터의 WKB(Well-Known Binary) 표현식을 SRID 메타데이터 없이 반환합니다.
- ST_AsGDALRaster - 래스터 타일을 지정한 GDAL 래스터 형식으로 반환합니다. 래스터 형식이란 사용자가 컴파일한 라이브러리가 지원하는 형식들 가운데 하나입니다. 사용자 라이브러리가 지원하는 형식들의 목록을 보려면 ST_GDALRasters() 함수를 이용하십시오.
- ST_AsJPEG - 래스터 타일에서 선택한 밴드들을 단일 JPEG(Joint Photographic Exports Group) 이미지(바이트 배열)로 반환합니다. 밴드를 따로 설정하지 않거나, 밴드가 1 개거나 또는 3 개를 초과할 경우 첫 번째 밴드를 씁니다. 밴드가 3 개뿐일 경우 밴드 3 개를 모두 써서 RGB 에 매핑시킵니다.
- ST_AsPNG - 래스터 타일에서 선택한 밴드들을 단일 PNG(Portable Network Graphics) 이미지(바이트 배열)로 반환합니다. 래스터의 밴드가 1 개, 3 개, 또는 4 개이거나 따로 설정하지 않을 경우 모든 밴드를 씁니다. 밴드가 2 개 또는 4 개를 초과하며 따로 설정하지 않을 경우, 밴드 1 만 씁니다. 밴드를 RGB 또는 RGBA 스페이스에 매핑합니다.
- ST_AsRaster - PostGIS 도형을 PostGIS 래스터로 변환합니다.
- ST_AsTIFF - 래스터 타일에서 선택한 밴드들을 단일 TIFF 이미지(바이트 배열)로 반환합니다. 밴드를 따로 설정하지 않을 경우, 모든 밴드를 쓰려 할 것입니다.
- ST_Aspect - 표고 래스터 밴드의 향(기본 단위는 도)을 반환합니다. 지형을 분석하는 데 유용합니다.
- ST_Band - 기존 래스터의 하나 이상의 밴드를 새 래스터로 반환합니다. 기존 래스터로부터 새 래스터를 빌드하는 데 유용합니다.
- ST_BandIsNoData - 밴드가 NODATA 값만으로 채워져 있을 경우 참을 반환합니다.
- ST_BandMetaData - 지정 래스터 밴드에 대한 기본 메타데이터를 반환합니다. 밴드를 지정하지 않을 경우 밴드 1 번이라고 가정합니다.
- ST_BandNoDataValue - 입력 밴드에서 NODATA 를 나타내는 값을 반환합니다. 어떤 밴드도 지정하지 않을 경우 밴드 1 로 가정합니다.
- ST_BandPath - 파일 시스템에 저장된 밴드를 가리키는 시스템 파일 경로를 반환합니다. bandnum 을 설정하지 않을 경우 밴드 1 로 가정합니다.
- ST_BandPixelType - 입력 밴드의 픽셀 유형을 반환합니다. bandnum 을 설정하지 않을 경우 밴드 1 로 가정합니다.

- ST_Clip - 입력 도형으로 잘라낸 래스터를 반환합니다. 밴드 번호를 지정하지 않은 경우, 모든 밴드를 처리합니다. crop 을 설정하지 않거나 참으로 설정한 경우, 잘라낸 래스터를 출력합니다.
- ST_ColorMap - 소스 래스터 및 설정한 밴드로부터 8BUI 밴드(grayscale, RGB, RGBA)를 4 개까지 가지는 새 래스터를 생성합니다. 밴드를 따로 설정하지 않으면 밴드 1 로 가정합니다.
- ST_Contains - 래스터 rastA 외부에 놓인 래스터 rastB 의 포인트가 하나도 없고, 적어도 rastB 내부의 포인트 하나가 rastA 의 내부에 있는 경우 참을 반환합니다.
- ST_ContainsProperly - rastB 가 rastA 의 내부와 교차하지만 rastA 의 경계선 또는 외부와 교차하지는 않을 경우 참을 반환합니다.
- ST_ConvexHull - BandNoDataValue 와 일치하는 픽셀 값을 포함한, 래스터의 볼록 껍질 도형을 반환합니다. 정규 형상 및 기울어지지 않은 래스터의 경우, ST_Envelope 와 동일한 결과물을 출력하므로 비정규 형상 또는 기울어진 래스터에 대해서만 쓸모가 있습니다.
- ST_Count - 래스터 또는 래스터 커버리지의 입력 밴드에 있는 픽셀 개수를 반환합니다. 밴드를 따로 설정하지 않을 경우 기본값은 밴드 1 입니다. exclude_nodata_value 를 참으로 설정할 경우, NODATA 값이 아닌 픽셀의 개수만 반환할 것입니다.
- ST_CountAgg - 종합 함수입니다. 래스터 집합의 입력 밴드에 있는 픽셀 개수를 반환합니다. 밴드를 따로 설정하지 않을 경우 기본값은 밴드 1 입니다. exclude_nodata_value 를 참으로 설정할 경우, NODATA 값이 아닌 픽셀의 개수만 반환할 것입니다.
- ST_CoveredBy - 래스터 rastA 의 어떤 포인트도 래스터 rastB 외부에 없을 경우 참을 반환합니다.
- ST_Covers - 래스터 rastB 의 어떤 포인트도 래스터 rastA 외부에 없을 경우 참을 반환합니다.
- ST_DFullyWithin - 래스터 rastA 와 래스터 rastB 가 완전히 서로 설정된 거리 안에 있을 경우 참을 반환합니다.
- ST_DWithin - 래스터 rastA 와 래스터 rastB 가 서로 설정된 거리 안에 있을 경우 참을 반환합니다.
- ST_Disjoint - 래스터 rastA 와 래스터 rastB 가 공간적으로 교차하지 않을 경우 참을 반환합니다.
- ST_DumpAsPolygons - 입력 래스터 밴드로부터 geomval(geom, val) 행들의 집합을 반환합니다. 밴드 번호를 설정하지 않을 경우 기본적으로 밴드 1 로 가정합니다.
- ST_DumpValues - 지정된 밴드의 값들을 2 차원 배열로 반환합니다.
- ST_Envelope - 래스터 범위의 폴리곤 표현식을 반환합니다.
- ST_FromGDALRaster - 지원 GDAL 래스터 파일로부터 래스터를 반환합니다.
- ST_GeoReference - 월드(world) 파일에서 흔히 볼 수 있는 지리참조 메타데이터를 GDAL 또는 ESRI 형식으로 반환합니다. 기본값은 GDAL 입니다.
- ST_HasNoBand - 입력된 밴드 번호에 밴드가 없을 경우 참을 반환합니다. 밴드 번호를 설정하지 않을 경우, 밴드 1 로 가정합니다.
- ST_Height - 래스터의 높이를 픽셀 개수로 반환합니다.
- ST_HillShade - 입력한 방위각, 고도각, 밝기 및 축척을 이용해서 표고 래스터 밴드의 가상적인 음영기복을 반환합니다.

- ST_Histogram - 빈(bin; 히스토그램 표시에서 수직 막대로 나타나는 단위) 범위로 구분된 래스터 또는 래스터 커버리지의 데이터 분포를 요약하는 레코드 집합을 반환합니다. 따로 설정하지 않을 경우 빈의 개수를 자동으로 계산합니다.

- ST_Intersection - 두 래스터의 공유 부분을 표현하는, 또는 벡터화된 래스터와 도형의 기하학적 교차를 표현하는 래스터 또는 도형-픽셀값 쌍의 집합을 반환합니다.

- ST_Intersects - 래스터 rastA 와 래스터 rastB 가 공간적으로 교차할 경우 참을 반환합니다.

- ST_IsEmpty - 래스터가 비어 있을 경우 (width = 0, height = 0) 참을 반환합니다. 그렇지 않을 경우 거짓을 반환합니다.

- ST_MakeEmptyRaster - 설정된 차원(너비 & 높이), 좌상단 X 및 Y, 픽셀 크기, 회전(scalex, scaley, skewx & skewy) 그리고 공간 참조 시스템(SRID)를 가진 텅 빈 (밴드가 없는) 래스터를 반환합니다. 래스터를 입력할 경우, 동일한 크기, 정렬 방향 및 SRID 를 가진 새 래스터를 반환합니다. SRID 를 생략할 경우, 공간 참조 시스템을 0(unknown)으로 설정합니다.

- ST_MapAlgebra - 콜백 함수 버전 - 래스터 1 개 이상, 밴드 인덱스, 그리고 사용자 지정 콜백 함수 1 개를 입력받아 밴드 1 개를 가진 래스터를 반환합니다.

- ST_MapAlgebraExpr - 래스터 밴드 1 개 버전: 입력 래스터에 대해 유효한 PostgreSQL 대수 연산을 적용해서 형성되고, 설정한 픽셀 유형을 가진, 밴드 1 개를 가진 새 래스터를 생성합니다. 따로 밴드를 설정하지 않을 경우, 밴드 1 로 가정합니다.

- ST_MapAlgebraExpr - 래스터 밴드 2 개 버전: 입력 래스터 2 개에 대해 유효한 PostgreSQL 대수 연산을 적용해서 형성되고, 설정한 픽셀 유형을 가진, 밴드 1 개를 가진 새 래스터를 생성합니다. 따로 밴드를 설정하지 않을 경우, 각 래스터의 밴드 1 로 가정합니다. 출력 래스터는 첫 번째 래스터가 정의하는 그리드 상에 (축척, 기울기 및 픽셀 모서리가) 정렬될 것입니다. extenttype 파라미터가 출력 래스터의 범위를 정의할 것입니다. extenttype 의 값은 INTERSECTION, UNION, FIRST, SECOND 가 될 수 있습니다.

- ST_MapAlgebraFct - 래스터 밴드 1 개 버전: 입력 래스터에 대해 유효한 PostgreSQL 대수 연산을 적용해서 형성되고, 설정한 픽셀 유형을 가진, 밴드 1 개를 가진 새 래스터를 생성합니다. 따로 밴드를 설정하지 않을 경우, 밴드 1 로 가정합니다.

- ST_MapAlgebraFct - 래스터 밴드 2 개 버전: 입력 래스터 2 개에 대해 유효한 PostgreSQL 함수를 적용해서 형성되고, 설정한 픽셀 유형을 가진, 밴드 1 개를 가진 새 래스터를 생성합니다. 따로 밴드를 설정하지 않을 경우, 밴드 1 로 가정합니다. 범위 유형을 따로 설정하지 않을 경우 기본값은 INTERSECTION 입니다.

- ST_MapAlgebraFctNgb - 래스터 밴드 1 개 버전: 사용자 지정 PostgreSQL 함수를 이용하는 맵 대수 최근접 이웃(Map Algebra Nearest Neighbor)입니다. 입력 래스터 밴드의 값의 이웃(neighborhood)이 관련된 PostgreSQL 사용자 함수가 출력하는 값을 가진 래스터를 반환합니다.

- ST_MapAlgebra - 표현식 버전 - 입력 래스터 1 개 또는 2 개, 밴드 인덱스, 그리고 사용자 지정 SQL 표현식 1 개 이상을 입력받아 밴드 1 개를 가진 래스터를 반환합니다.

- ST_MemSize - 래스터가 차지하는 공간의 용량을 (바이트 단위로) 반환합니다.

- ST_MetaData - 래스터 객체의 픽셀 크기, 회전(skew), 좌상단, 좌하단 등과 같은 기본 메타데이터를 반환합니다.

- ST_MinConvexHull - 래스터의 NODATA 픽셀을 제외한 볼록 껍질 도형을 반환합니다.
- ST_NearestValue - columnx 및 rowy, 또는 래스터와 동일한 공간 참조 좌표 시스템 단위로 표현된 기하학적 포인트로 지정된 입력 밴드의 픽셀에 가장 가까운 NODATA 가 아닌 값을 반환합니다.
- ST_Neighborhood - columnx 및 rowy, 또는 래스터와 동일한 공간 참조 좌표 시스템 단위로 표현된 기하학적 포인트로 지정된 입력 밴드의 픽셀 주위의 NODATA 가 아닌 값들의 이중 정밀도 데이터형 2 차원 배열을 반환합니다.
- ST_NotSameAlignmentReason - 래스터들이 정렬돼 있는지 아닌지, 그리고 정렬되지 않았다면 그 이유를 설명하는 텍스트를 반환합니다.
- ST_NumBands - 래스터 객체 내부에 있는 밴드들의 개수를 반환합니다.
- ST_Overlaps - 래스터 rastA 와 래스터 rastB 가 교차하지만 어느 한 쪽이 다른 한 쪽을 완전히 담고 있지는 않을 경우 참을 반환합니다.
- ST_PixelAsCentroid - 픽셀 하나가 차지하는 면의 중심점(포인트 도형)을 반환합니다.
- ST_PixelAsCentroids - 래스터 밴드의 각 픽셀에 대한 중심점(포인트 도형)을 각 픽셀의 값 및 X, Y 래스터 좌표와 함께 반환합니다. 포인트 도형은 픽셀이 차지하는 면의 중심점입니다.
- ST_PixelAsPoint - 픽셀의 좌상단에 위치하는 포인트 도형을 반환합니다.
- ST_PixelAsPoints - 래스터 밴드의 각 픽셀에 대한 포인트 도형을 각 픽셀의 값 및 X, Y 래스터 좌표와 함께 반환합니다. 포인트 도형의 좌표는 픽셀의 좌상단 좌표입니다.
- ST_PixelAsPolygon - 특정 행 및 열에 대한 픽셀의 경계를 이루는 폴리곤 도형을 반환합니다.
- ST_PixelAsPolygons - 래스터 밴드의 모든 픽셀의 경계를 이루는 폴리곤 도형을 각 픽셀의 값과 X, Y 래스터 좌표와 함께 반환합니다.
- ST_PixelHeight - 픽셀 높이를 공간 참조 시스템의 기하 단위로 반환합니다.
- ST_PixelOfValue - 검색 값과 일치하는 값을 가진 픽셀의 columnx, rowy 좌표를 반환합니다.
- ST_PixelWidth - 픽셀 너비를 공간 참조 시스템의 기하 단위로 반환합니다.
- ST_Polygon - NODATA 값이 아닌 픽셀 값을 가진 픽셀들을 통합해서 형성된 멀티폴리곤 도형을 반환합니다.
- ST_Quantile - 샘플링 또는 채우기(population)라는 맥락에서 래스터 또는 래스터 테이블 커버리지의 사분위(quantile)를 계산합니다. 따라서, 래스터의 25%, 50%, 75% 백분위(percentile) 단계로 값을 확인할 수 있습니다.
- ST_RasterToWorldCoord - 주어진 열과 행에 있는 래스터의 좌상단의 기하학적 X, Y(경도, 위도)를 으로 반환합니다. 열과 행은 1 부터 시작합니다.
- ST_RasterToWorldCoordX - 열과 행에 있는 래스터의 좌상단의 기하학적 X 좌표를 반환합니다. 열과 행의 번호는 1 부터 시작합니다.
- ST_RasterToWorldCoordY - 열과 행에 있는 래스터의 좌상단의 기하학적 Y 좌표를 반환합니다. 열과 행의 번호는 1 부터 시작합니다.

- ST_Reclass - 원본으로부터 재분류된 밴드 유형으로 이루어진 새 래스터를 생성합니다. nband 는 변경할 밴드를 가리킵니다. nband 를 따로 설정하지 않을 경우 밴드 1 로 가정합니다. 다른 모든 밴드들은 변경 없이 반환됩니다. 실제 사례: 보기 좋은 형식으로 더 간단하게 렌더링하기 위해 16BUI 밴드를 8BUI 등등으로 변환하십시오.
- ST_Resample - 특정 리샘플링 알고리즘, 새로운 차원, 임의의 그리드 모서리, 그리고 또 다른 래스터에서 정의되거나 빌려온 래스터 지리참조 속성들의 집합을 이용해서 래스터를 리샘플링합니다.
- ST_Rescale - 축척(또는 픽셀 크기)만 조정해서 래스터를 리샘플링합니다. NearestNeighbor(영국 또는 미국 철자), Bilinear, Cubic, CubicSpline 또는 Lanczos 리샘플링 알고리즘을 이용해서 새 픽셀 값을 계산합니다. 기본값은 NearestNeighbor 입니다.
- ST_Resize - 래스터의 크기를 새 너비/높이로 조정합니다.
- ST_Reskew - 기울기(또는 회전각 파라미터)만 조정해서 래스터를 리샘플링합니다. NearestNeighbor(영국 또는 미국 철자), Bilinear, Cubic, CubicSpline 또는 Lanczos 리샘플링 알고리즘을 이용해서 새 픽셀 값을 계산합니다. 기본값은 NearestNeighbor 입니다.
- ST_Rotation - 래스터의 회전각을 라디안으로 반환합니다.
- ST_Roughness - DEM 의 계산된 "거칠기(roughness)"와 함께 래스터를 반환합니다.
- ST_SRID - spatial_ref_sys 테이블에 정의되어 있는, 래스터의 공간 참조 식별자를 반환합니다.
- ST_SameAlignment - 래스터들이 동일한 기울기, 축척, 공간 참조 시스템, 그리고 오프셋을 가지고 있을 경우 (픽셀이 다른 픽셀을 침범하지 않고 동일한 그리드에 놓일 수 있을 경우) 참을 반환하고, 그렇지 않을 경우 문제점을 설명하는 안내문과 함께 거짓을 반환합니다.
- ST_ScaleX - 픽셀 너비의 X 요소를 좌표 참조 시스템의 단위로 반환합니다.
- ST_ScaleY - 픽셀 높이의 Y 요소를 좌표 참조 시스템의 단위로 반환합니다.
- ST_SetBandIsNoData - 밴드의 isnodata 플래그를 참으로 설정합니다.
- ST_SetBandNoDataValue - NODATA 를 나타내는 입력 밴드의 값을 설정합니다. 밴드를 따로 설정하지 않을 경우 밴드 1 로 가정합니다. 밴드에 NODATA 가 없다고 표시하려면, nodata value = NULL 이라고 설정하십시오.
- ST_SetGeoReference - 단일 호출로 지리참조 파라미터 6 개를 설정합니다. 숫자를 공백으로 구분해야 합니다. GDAL 또는 ESRI 서식의 입력물을 받아들입니다. 기본값은 GDAL 입니다.
- ST_SetRotation - 래스터의 회전각을 라디안으로 설정합니다.
- ST_SetSRID - 래스터의 SRID 를 spatial_ref_sys 테이블에 정의된 특정 SRID 의 정수값으로 설정합니다.
- ST_SetScale - X 및 Y 픽셀 크기를 좌표 참조 시스템의 단위로 설정합니다. 단위/픽셀 너비/픽셀 높이 순서입니다.
- ST_SetSkew - 지리참조 X 및 Y 기울기(skew)(또는 회전각 파라미터)를 설정합니다. 값 하나만 입력할 경우, X 와 Y 를 동일한 값으로 설정합니다.
- ST_SetUpperLeft - 픽셀 좌상단의 값을 투영된 X, Y 좌표로 설정합니다.

- ST_SetValue - 입력한 columnx, rowy 픽셀 또는 특정 도형과 교차하는 픽셀들의 위치의 밴드 값을 설정해서 나온 수정된 래스터를 반환합니다. 밴드 번호는 1 부터 시작하며, 따로 설정하지 않을 경우 1 로 가정합니다.
- ST_SetValues - 지정한 밴드의 값들을 설정해서 나온 수정된 래스터를 반환합니다.
- ST_SkewX - 지리참조 X 기울기(skew)(또는 회전각 파라미터)를 반환합니다.
- ST_SkewY - 지리참조 Y 기울기(또는 회전각 파라미터)를 반환합니다.
- ST_Slope - 표고 래스터 밴드의 경사(기본 단위는 도)를 반환합니다. 지형을 분석하는 데 유용합니다.
- ST_SnapToGrid - 그리드에 래스터를 스냅시켜서 래스터를 리샘플링합니다. NearestNeighbor(영국 또는 미국 철자), Bilinear, Cubic, CubicSpline 또는 Lanczos 리샘플링 알고리즘을 이용해서 새 픽셀 값을 계산합니다. 기본값은 NearestNeighbor 입니다.
- ST_Summary - 래스터의 내용을 요약한 텍스트를 반환합니다.
- ST_SummaryStats - 입력한 래스터 밴드 또는 래스터 또는 래스터 커버리지의 count, sum, mean, stddev, min, max 로 이루어진 통계 요약을 반환합니다. 밴드를 따로 설정하지 않을 경우 밴드 1 로 가정합니다.
- ST_SummaryStatsAgg - 종합 함수입니다. 래스터 집합의 입력 래스터 밴드의 count, sum, mean, stddev, min, max 로 이루어진 통계 요약을 반환합니다. 밴드를 따로 설정하지 않을 경우 밴드 1 로 가정합니다.
- ST_TPI - 계산된 지형위치지수(Topographic Position Index)와 함께 래스터를 반환합니다.
- ST_TRI - 계산된 지형험준도지수(Terrain Ruggedness Index)와 함께 래스터를 반환합니다.
- ST_Tile - 출력 래스터에 원하는 차원을 바탕으로 입력 래스터를 분할해서 나온 래스터들의 집합을 반환합니다.
- ST_Touches - 래스터 rastA 와 래스터 rastB 가 최소한 포인트 한 개를 공유하지만, 내부가 교차하지는 않을 경우 TRUE 를 반환합니다.
- ST_Transform - 알려진 공간 참조 시스템의 래스터를 지정한 리샘플링 알고리즘을 통해 또 다른 알려진 공간 참조 시스템으로 재투영합니다. NearestNeighbor, Bilinear, Cubic, CubicSpline, Lanczos 알고리즘을 이용할 수 있습니다. 기본값은 NearestNeighbor 입니다.
- ST_Union - 래스터 타일 집합을 1 개 이상의 밴드로 이루어진 단일 래스터로 통합합니다.
- ST_UpperLeftX - 래스터의 좌상단 X 좌표를 투영된 공간 참조 단위로 반환합니다.
- ST_UpperLeftY - 래스터의 좌상단 Y 좌표를 투영된 공간 참조 단위로 반환합니다.
- ST_Value - 입력 columnx, rowy 픽셀의 입력 밴드의 값, 또는 특정 기하학적 포인트의 밴드의 값을 반환합니다. 밴드 번호는 1 부터 시작하며, 따로 지정하지 않을 경우 1 로 가정합니다. exclude_nodata_value 를 거짓으로 설정할 경우, nodata 픽셀을 포함한 모든 픽셀이 교차한다고 가정하고 값을 반환합니다. exclude_nodata_value 를 설정하지 않은 경우, 래스터의 메타데이터에서 값을 읽어 들입니다.

- ST_ValueCount - 설정한 값들의 집합을 가진 래스터(또는 래스터 커버리지)의 입력 밴드에 있는 픽셀 밴드 값 및 픽셀 개수의 집계를 담고 있는 레코드 집합을 반환합니다. 밴드를 따로 설정하지 않을 경우 기본값은 밴드 1 입니다. 기본적으로 NODATA 값은 집계되지 않습니다. 픽셀의 다른 모든 값들을 출력하는데, 픽셀 밴드 값은 가장 가까운 정수로 반올림됩니다.

- ST_Width - 래스터의 너비를 픽셀 개수로 반환합니다.

- ST_Within - 래스터 rastB 외부에 놓인 래스터 rastA 의 포인트가 하나도 없고, 적어도 rastA 내부의 포인트 하나가 rastB 의 내부에 있는 경우 참을 반환합니다.

- ST_WorldToRasterCoord - 주어진 기하학적 X, Y(경도, 위도)의 좌상단을 열과 행으로 또는 래스터의 공간 참조 좌표 시스템 단위로 표현된 포인트 도형으로 반환합니다.

- ST_WorldToRasterCoordX - 포인트 도형(pt)의 래스터 안의 열 또는 래스터의 월드 공간 참조 시스템 단위로 표현된 X, Y 월드 좌표(xw, yw)를 반환합니다.

- ST_WorldToRasterCoordY - 포인트 도형(pt)의 래스터 안의 행 또는 래스터의 월드 공간 참조 시스템 단위로 표현된 X, Y 월드 좌표(xw, yw)를 반환합니다.

- UpdateRasterSRID - 사용자가 지정한 열 및 테이블에 있는 모든 래스터의 SRID 를 변경합니다.

14.6. PostGIS 도형/지리형/래스터 덤프 함수

다음 함수들은 geometry_dump 또는 geomval 데이터형 객체를 단일 또는 집합으로 입력받거나 산출물로 반환하는 PostGIS 함수입니다.

- ST_DumpAsPolygons - 입력 래스터 밴드로부터 geomval(geom, val) 행들의 집합을 반환합니다. 밴드 번호를 설정하지 않을 경우 기본적으로 밴드 1 로 가정합니다.

- ST_Intersection - 두 래스터의 공유 부분을 표현하는, 또는 벡터화된 래스터와 도형의 기하학적 교차를 표현하는 래스터 또는 도형-픽셀값 쌍의 집합을 반환합니다.

- ST_Dump - 도형 g1 을 구성하고 있는 geometry_dump(도형, 경로) 행들의 집합을 반환합니다.

- ST_DumpPoints - 도형을 구성하는 모든 포인트의 geometry_dump(도형, 경로) 행들의 집합을 반환합니다.

- ST_DumpRings - 폴리곤의 외곽 및 내곽 고리를 표현하는 geometry_dump 행들의 집합을 반환합니다.

14.7. PostGIS 상자 함수

다음 함수들은 상자 계열의 PostGIS 공간 유형을 입력받거나 산출물로 반환하는 PostGIS 함수입니다. 이 상자 계열 유형은 BOX2D, 그리고 BOX3D 로 구성되어 있습니다.

- Box2D - 도형의 최대 범위(maximum extent)를 나타내는 BOX2D 를 반환합니다.

- Box3D - 도형의 최대 범위를 나타내는 BOX3D 를 반환합니다.

- Box3D - 래스터를 둘러싼 상자의 BOX3D 표현식을 반환합니다.

- ST_3DExtent - 도형 행들의 경계를 이루는 BOX3D 경계 상자를 반환하는 종합 함수입니다.
- ST_3DMakeBox - 주어진 3D 포인트 도형들로 정의되는 BOX3D 를 생성합니다.
- ST_AsTWKB - 도형을 TWKB(Tiny Well-Known Binary)로 반환합니다.
- ST_Box2dFromGeoHash - GeoHash 스트링으로부터 BOX2D 를 반환합니다.
- ST_ClipByBox2D - 직사각형 안에 들어오는 도형의 부분을 반환합니다.
- ST_EstimatedExtent - 주어진 공간 테이블의 '추정' 범위를 반환합니다. 도형 열의 통계로부터 추정치를 추출합니다. 따로 설정하지 않을 경우 현재 스키마를 이용할 것입니다.
- ST_Expand - 입력 도형의 경계 상자로부터 모든 방향으로 확장된 경계 상자를 반환합니다. 이중 정밀도 데이터형을 사용합니다.
- ST_Extent - 도형 행들의 경계를 이루는 경계 상자를 반환하는 종합 함수입니다.
- ST_MakeBox2D - 주어진 포인트 도형들로 정의되는 BOX2D 를 생성합니다.
- ST_XMax - 2 차원 경계 상자(box2d) 또는 3 차원 경계 상자(box3d) 또는 도형의 최대 X 값을 반환합니다.
- ST_XMin - 2 차원 경계 상자(box2d) 또는 3 차원 경계 상자(box3d) 또는 도형의 최소 X 값을 반환합니다.
- ST_YMax - 2 차원 경계 상자(box2d) 또는 3 차원 경계 상자(box3d) 또는 도형의 최대 Y 값을 반환합니다.
- ST_YMin - 2 차원 경계 상자(box2d) 또는 3 차원 경계 상자(box3d) 또는 도형의 최소 Y 값을 반환합니다.
- ST_ZMax - 2 차원 경계 상자(box2d) 또는 3 차원 경계 상자(box3d) 또는 도형의 최대 Z 값을 반환합니다.
- ST_ZMin - 2 차원 경계 상자(box2d) 또는 3 차원 경계 상자(box3d) 또는 도형의 최대 Z 값을 반환합니다.
- ~(box2df,box2df) - 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)가 또다른 2 차원 부동소수점 데이터형 정밀도 경계 상자(BOX2DF)를 포함하는 경우 TRUE 를 반환합니다.
- ~(box2df,geometry) - 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)가 도형의 2 차원 경계 상자를 포함하는 경우 TRUE 를 반환합니다.
- ~(geometry,box2df) - 도형의 2 차원 경계 상자가 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)를 포함하는 경우 TRUE 를 반환합니다.
- @(box2df,box2df) - 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)가 또다른 2 차원 부동소수점 데이터형 정밀도 경계 상자 안에 포함되는 경우 TRUE 를 반환합니다.
- @(box2df,geometry) - 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)가 도형의 2 차원 경계 상자 안에 포함되는 경우 TRUE 를 반환합니다.
- @(geometry,box2df) - 도형의 2 차원 경계 상자가 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF) 안에 포함되는 경우 TRUE 를 반환합니다.
- &&(box2df,box2df) - 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF) 2 개가 서로 교차하는 경우 TRUE 를 반환합니다.

- &&(box2df,geometry) - 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)가 도형의 (캐시된) 2 차원 경계 상자와 교차하는 경우 TRUE 를 반환합니다.
- &&(geometry,box2df) - 도형의 (캐시된) 2 차원 경계 상자가 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)와 교차하는 경우 TRUE 를 반환합니다.

14.8.3 차원을 지원하는 PostGIS 함수

다음 함수들은 Z-인덱스를 삭제하지 않는 PostGIS 함수입니다.

- AddGeometryColumn - 기존 속성 테이블에 도형 열을 추가합니다. 기본적으로 제약 조건보다는 유형 변경자를 사용해서 정의합니다. 구 버전 확인 제약조건 기반 습성(behavior)을 얻으려면 use_typmod 에 대해 거짓을 패스합니다.
- Box3D - 도형의 최대 범위를 나타내는 BOX3D 를 반환합니다.
- DropGeometryColumn - 공간 테이블에서 도형 열을 제거합니다.
- GeometryType - 도형의 유형을 스트링으로 반환합니다. 예: 'LINESTRING', 'POLYGON', 'MULTIPOINT' 등등.
- ST_3DArea - 3 차원 표면 도형의 면적을 계산합니다. 입체일 경우 0 을 반환할 것입니다.
- ST_3DClosestPoint - g2 에 가장 가까운 g1 상에 있는 3 차원 포인트를 반환합니다. 해당 포인트는 3D 최단 라인의 첫 번째 포인트입니다.
- ST_3DDFullyWithin - 모든 3D 도형이 서로의 설정된 거리 내에 있을 경우 참을 반환합니다.
- ST_3DDWithin - 3D(Z) 도형 유형에 대해, 두 도형의 3 차원 거리가 단위의 개수 내일 경우 참을 반환합니다.
- ST_3DDifference - 3 차원 차이를 수행합니다.
- ST_3DDistance - 도형 유형에 대해, 두 도형 사이의 (SRS 에 기반한) 3 차원 데카르트 최단 거리를 투영 단위로 반환합니다.
- ST_3DExtent - 도형 행들의 경계를 이루는 BOX3D 경계 상자를 반환하는 종합 함수입니다.
- ST_3DIntersection - 3 차원 교차를 수행합니다.
- ST_3DIntersects - 도형들이 3 차원에서 "공간적으로 교차"하는 경우 참을 반환합니다. 포인트, 라인스트링, 폴리곤, 다면체 표면(면적)만 입력받습니다. SFCGAL 백엔드가 활성화됐다면 TIN 도 지원합니다.
- ST_3DLength - 도형이 라인스트링 또는 멀티라인스트링일 경우 도형의 3 차원 또는 2 차원 길이를 반환합니다.
- ST_3DLongestLine - 두 도형 사이의 3 차원 최장(longest) 라인을 반환합니다.
- ST_3DMaxDistance - 도형 유형에 대해, 두 도형 사이의 (SRS 에 기반한) 3 차원 데카르트 최장 거리를 투영 단위로 반환합니다.
- ST_3DPerimeter - 도형이 폴리곤 또는 멀티폴리곤일 경우 도형의 3 차원 둘레를 반환합니다.

- ST_3DShortestLine - 두 도형 사이의 3 차원 최단(shortest) 라인을 반환합니다.
- ST_3DUnion - 3 차원 통합을 수행합니다.
- ST_Accum - 종합 함수입니다. 도형들의 배열을 작성합니다.
- ST_AddMeasure - 시작점과 종단점 사이의 선형적으로 보간된 척도 요소들을 가진 파생 도형을 반환합니다.
- ST_AddPoint - 라인스트링에 포인트를 추가합니다.
- ST_Affine - 도형에 3 차원 아핀 변환을 적용합니다.
- ST_ApproximateMedialAxis - 면 도형의 근사 중심축을 계산합니다.
- ST_AsBinary - 도형/지리형의 WKB(Well-Known Binary) 표현식을 SRID 메타데이터 없이 반환합니다.
- ST_AsEWKB - 도형/지리형의 WKB(Well-Known Binary) 표현식을 SRID 메타데이터와 함께 반환합니다.
- ST_AsEWKT - 도형의 WKT(Well-Known Text) 표현식을 SRID 메타데이터와 함께 반환합니다.
- ST_AsGML - 도형을 GML 2 또는 GML 3 버전 요소로 반환합니다.
- ST_AsGeoJSON - 도형을 GeoJSON 요소로 반환합니다.
- ST_AsHEXEWKB - 도형을 소 엔디안(NDR) 또는 대 엔디안(XDR) 인코딩을 통해 HEXEWKB (텍스트) 형식으로 반환합니다.
- ST_AsKML - 도형을 KML 요소로 반환합니다. 응용방법이 몇 개 있습니다. 기본값은 버전 2, 정밀도 15 입니다.
- ST_AsX3D - 도형을 X3D XML 노드 요소 형식: ISO-IEC-19776-1.2-X3DEncodings-XML 로 반환합니다.
- ST_Boundary - 해당 도형의 결합된 범위의 닫힘 여부를 반환합니다.
- ST_BoundingDiagonal - 주어진 도형의 경계 상자의 대각선을 반환합니다.
- ST_CPAWithin - 두 궤적에서 가장 가까이 접근한 포인트들 사이의 거리가 설정한 거리 이하일 경우 참을 반환합니다.
- ST_ClosestPointOfApproach - 두 라인을 따라 보간된 포인트들이 가장 가까운 위치의 척도를 반환합니다.
- ST_Collect - 다른 도형들의 집합으로부터 설정된 ST_Geometry 값을 반환합니다.
- ST_ConvexHull - 도형의 볼록 껍질(convex hull)은 집합 내부의 모든 도형들을 둘러싸는 최소 볼록 도형을 의미합니다.
- ST_CoordDim - ST_Geometry 값의 좌표 차원을 반환합니다.
- ST_CurveToLine - 원호 스트링(CIRCULARSTRING)/만곡 폴리곤(CURVEPOLYGON)을 라인스트링/폴리곤으로 변환합니다.
- ST_DelaunayTriangles - 주어진 입력 포인트들을 둘러싼 들로네 삼각망(Delaunay triangulation)을 반환합니다.

- ST_Difference - 도형 A 에서 도형 B 와 교차하지 않는 부분을 표현하는 도형을 반환합니다.
- ST_DistanceCPA - 두 궤적에서 가장 가깝게 접근한 포인트들 사이의 거리를 반환합니다.
- ST_Dump - 도형 g1 을 구성하고 있는 geometry_dump(도형, 경로) 행들의 집합을 반환합니다.
- ST_DumpPoints - 도형을 구성하는 모든 포인트의 geometry_dump(도형, 경로) 행들의 집합을 반환합니다.
- ST_DumpRings - 폴리곤의 외곽 및 내곽 고리를 표현하는 geometry_dump 행들의 집합을 반환합니다.
- ST_EndPoint - LINESTRING 또는 CIRCULARLINESTRING 도형의 마지막 포인트를 POINT 로 반환합니다.
- ST_ExteriorRing - POLYGON 도형의 외곽 고리(exterior ring)를 표현하는 라인스트링을 반환합니다. 도형이 폴리곤이 아닌 경우 NULL 을 반환합니다. 멀티폴리곤은 입력받지 못 합니다.
- ST_Extrude - 표면을 관련 입체로 돌출시킵니다.
- ST_FlipCoordinates - 입력 도형을 X 축과 Y 축을 뒤집은 상태로 반환합니다. 위도/경도 피처를 빌드해서 수정해야 하는 경우 유용합니다.
- ST_Force2D - 도형을 "2 차원 모드"로 강제합니다.
- ST_ForceCurve - 적용이 가능한 경우, 도형을 해당 도형의 만곡 유형으로 상위 형변환(upcast)합니다.
- ST_ForceLHR - LHR(Left Hand Reverse; 시계 방향) 방향을 강제합니다.
- ST_ForceRHR - 폴리곤 안에 있는 꼭짓점들의 방향(orientation)이 오른손 법칙(Right-Hand Rule)을 따르도록 강제합니다.
- ST_ForceSFS - 도형이 SFS 1.1 도형 유형만을 쓰도록 강제합니다.
- ST_Force_3D - 도형을 XYZ 모드로 강제합니다. ST_Force3DZ 라고도 합니다.
- ST_Force_3DZ - 도형을 XYZ 모드로 강제합니다.
- ST_Force_4D - 도형을 XYZM 모드로 강제합니다.
- ST_Force_Collection - 도형을 도형 집합으로 변환합니다.
- ST_GeomFromEWKB - EWKB(Extended Well-Known Binary)로부터 지정된 ST_Geometry 값을 반환합니다.
- ST_GeomFromEWKT - EWKT(Extended Well-Known Text)로부터 지정된 ST_Geometry 값을 반환합니다.
- ST_GeomFromGML - 도형의 GML 표현식을 입력받아 PostGIS 도형 객체를 출력합니다.
- ST_GeomFromGeoJSON - GeoJSON 표현식을 입력받아 PostGIS 도형 객체를 출력합니다.
- ST_GeomFromKML - 도형의 KML 표현식을 입력받아 PostGIS 도형 객체를 출력합니다.
- ST_GeometricMedian - 멀티 포인트의 기하학적 중심값(median)을 반환합니다.
- ST_GeometryN - 입력 도형이 도형 집합, (멀티)포인트, (멀티)라인스트링, 멀티커브(multicurve) 또는 (멀티)폴리곤이나 다면체 표면일 경우 1-기반 N 번째 도형을 반환하며, 그 외의 경우 NULL 을 반환합니다.

- ST_GeometryType - ST_Geometry 값의 도형 유형을 반환합니다.
- ST_HasArc - 도형 또는 도형 집합이 원호 스트링을 담고 있을 경우 참을 반환합니다.
- ST_InteriorRingN - 폴리곤 도형의 N 번째 내곽 라인스트링 고리를 반환합니다. 도형이 폴리곤이 아니거나 주어진 N 이 범위(range)를 벗어난 경우 NULL 을 반환합니다.
- ST_InterpolatePoint - 입력 포인트에 가까운 포인트에서 도형의 척도 차원의 값을 반환합니다.
- ST_IsClosed - LINESTRING 의 시작점과 종단점이 일치하는 경우 TRUE 를 반환합니다. 다면체 표면이 닫혀 (부피를 가지고) 있는 경우 TRUE 를 반환합니다.
- ST_IsCollection - 인수가 집합(MULTI*, GEOMETRYCOLLECTION, ...)인 경우 TRUE 를 반환합니다.
- ST_IsPlanar - 표면이 평면인지 아닌지 확인합니다.
- ST_IsSimple - 해당 도형이 자체 교차하거나 자체 접촉하는 이례적인 도형 포인트를 가지고 있지 않을 경우 TRUE 를 반환합니다.
- ST_IsSolid - 도형이 입체인지 테스트합니다. 어떤 유효성 검사도 수행하지 않습니다.
- ST_IsValidTrajectory - 도형이 유효한 궤적(trajectory)일 경우 true 를 반환합니다.
- ST_Length_Spheroid - 타원체 상에 있는 도형의 2 차원 또는 3 차원 길이/둘레를 계산합니다. 이 함수는 도형의 좌표가 경도/위도이며 재투영하지 않고 길이를 얻고 싶을 때 유용합니다.
- ST_LineFromMultiPoint - 멀티포인트 도형으로부터 라인스트링을 생성합니다.
- ST_LineInterpolatePoint - 라인을 따라 보간된 포인트를 반환합니다. 두 번째 인수는 0 과 1 사이의 Float8 데이터형으로 라인스트링의 전체 길이에서 포인트가 위치해야 하는 비율을 의미합니다.
- ST_LineSubstring - 입력 라인스트링의 전체 2 차원 길이의 주어진 비율의 위치에서 시작해서 끝나는, 입력 라인스트링의 부분 스트링(substring)인 라인스트링을 반환합니다. 두 번째 및 세 번째 인수는 0 과 1 사이의 8 바이트 부동소수점 데이터형(float8) 값입니다.
- ST_LineToCurve - 라인스트링/폴리곤을 원호 스트링/만곡 폴리곤으로 변환합니다.
- ST_LocateBetweenElevations - 설정한 고도의 범위를 계산에 넣어서 교차하는 요소들을 가진 파생 도형(집합)의 값을 반환합니다. 3 차원, 4 차원 라인스트링과 멀티라인스트링만 지원합니다.
- ST_M - 포인트의 M 좌표를 반환합니다. M 좌표가 없는 경우 NULL 을 반환합니다. 포인트만 입력받을 수 있습니다.
- ST_MakeLine - 포인트, 멀티포인트 또는 라인 도형으로부터 라인스트링을 생성합니다.
- ST_MakePoint - 2D, 3DZ 또는 4D 포인트 도형을 생성합니다.
- ST_MakePolygon - 주어진 외부 구조(shell)로 형성된 폴리곤을 생성합니다. 입력 도형이 닫힌 라인스트링이어야 합니다.
- ST_MakeSolid - 도형을 입체로 지정합니다. 어떤 확인 작업도 수행하지 않습니다. 유효한 입체를 얻으려면, 입력 도형이 닫힌 다면체 표면 또는 닫힌 TIN 이어야만 합니다.
- ST_MakeValid - 유효하지 않은 도형을 꼭짓점을 버리지 않고 유효한 도형으로 바꾸려고 시도합니다.

- ST_MemSize - 도형이 차지하는 (바이트 단위) 공간 용량을 반환합니다.
- ST_MemUnion - 메모리 친화적이라는 점만 제외하면 ST_Union 함수와 동일합니다(메모리를 덜 사용하고 CPU 시간을 더 사용합니다).
- ST_NDims - 도형의 좌표 차원을 2 바이트 정수형(smallint)으로 반환합니다. 값은 2, 3, 또는 4 입니다.
- ST_NPoints - 도형이 가지고 있는 포인트(꼭짓점)의 개수를 반환합니다.
- ST_NRings - 도형이 폴리곤 또는 멀티폴리곤인 경우 고리의 개수를 반환합니다.
- ST_Node - 라인스트링들의 집합에 노드를 적용합니다.
- ST_NumGeometries - 도형이 도형 집합 또는 멀티 도형인 경우 도형의 개수를 반환합니다. 단일 도형인 경우 1 을 반환하고, 도형 집합도 멀티 도형도 단일 도형도 아닌 경우 NULL 을 반환합니다.
- ST_NumPatches - 다면체 표면 상에 있는 면의 개수를 반환합니다. 다면체 도형이 아닌 경우 NULL 을 반환할 것입니다.
- ST_Orientation - 표면의 방향(orientation)을 결정합니다.
- ST_PatchN - 도형이 POLYHEDRALSURFACE, POLYHEDRALSURFACEM 인 경우 1-기반 N 번째 도형 (면)을 반환합니다. 그 외의 경우 NULL 을 반환합니다.
- ST_PointFromWKB - 주어진 SRID 와 함께 WKB 로부터 도형을 만듭니다.
- ST_PointN - 도형 안에 있는 첫 번째 라인스트링 또는 원형 라인스트링의 N 번째 포인트를 반환합니다. 음수 값은 라인스트링의 종단점으로부터 반대로 집계된 것입니다. 도형 안에 라인스트링이 없는 경우 NULL 을 반환합니다.
- ST_PointOnSurface - 표면에 붙어 있게 될 POINT 를 반환합니다.
- ST_Points - 도형의 모든 좌표들을 담고 있는 멀티포인트를 반환합니다.
- ST_Polygon - 설정된 라인스트링과 SRID 로부터 빌드된 폴리곤을 반환합니다.
- ST_RemovePoint - 라인스트링에서 포인트를 제거합니다.
- ST_RemoveRepeatedPoints - 입력 도형을 중복된 포인트들을 제거한 상태로 반환합니다.
- ST_Rotate - 원점(origin)을 기준으로 도형의 rotRadians 를 반시계 방향으로 회전시킵니다.
- ST_RotateX - X 축을 기준으로 도형의 rotRadians 를 회전시킵니다.
- ST_RotateY - Y 축을 기준으로 도형의 rotRadians 를 회전시킵니다.
- ST_RotateZ - Z 축을 기준으로 도형의 rotRadians 를 회전시킵니다.
- ST_Scale - 주어진 인수로 도형의 크기를 조정합니다.
- ST_SetPoint - 라인스트링의 포인트를 주어진 포인트로 대체합니다.
- ST_Shift_Longitude - 도형의 좌표를 -180 ~ 180 범위와 0 ~ 360 범위 간에 전환합니다.
- ST_SnapToGrid - 입력 도형의 모든 포인트를 정규 그리드로 스냅(snap)시킵니다.

- ST_StartPoint - LINESTRING 도형의 첫 번째 포인트를 POINT 로 반환합니다.
- ST_StraightSkeleton - 도형으로부터 직선 골격(straight skeleton)을 계산합니다.
- ST_SwapOrdinates - 입력 도형을 좌표값을 뒤바꾼 상태로 반환합니다.
- ST_SymDifference - 도형 A 와 B 의 서로 교차하지 않는 부분들을 표현하는 도형을 반환합니다. ST_SymDifference(A,B) = ST_SymDifference(B,A)이기 때문에 이를 대칭차(symmetric difference)라고 합니다.
- ST_Tesselate - 폴리곤 또는 다면체 표면의 표면 다듬기(tesselation) 작업을 수행한 다음 TIN 또는 TIN 집합으로 반환합니다.
- ST_TransScale - 주어진 인수 및 오프셋으로 도형을 변형시킵니다.
- ST_Translate - 주어진 오프셋으로 도형을 변형시킵니다.
- ST_UnaryUnion - ST_Union 과 비슷하지만, 도형 구성 요소 레벨에서 동작합니다.
- ST_Volume - 3 차원 입체의 부피를 계산합니다. 표면 도형을 입력하면 (닫힌 도형일지라도) 0 을 반환할 것입니다.
- ST_WrapX - X 값 근처에서 도형을 래핑합니다.
- ST_X - 포인트의 X 좌표를 반환합니다. X 좌표가 없는 경우 NULL 을 반환합니다. 포인트만 입력받을 수 있습니다.
- ST_XMax - 2 차원 경계 상자(box2d) 또는 3 차원 경계 상자(box3d) 또는 도형의 최대 X 값을 반환합니다.
- ST_XMin - 2 차원 경계 상자(box2d) 또는 3 차원 경계 상자(box3d) 또는 도형의 최소 X 값을 반환합니다.
- ST_Y - 포인트의 Y 좌표를 반환합니다. Y 좌표가 없는 경우 NULL 을 반환합니다. 포인트만 입력받을 수 있습니다.
- ST_YMax - 2 차원 경계 상자(box2d) 또는 3 차원 경계 상자(box3d) 또는 도형의 최대 Y 값을 반환합니다.
- ST_YMin - 2 차원 경계 상자(box2d) 또는 3 차원 경계 상자(box3d) 또는 도형의 최소 Y 값을 반환합니다.
- ST_Z - 포인트의 Z 좌표를 반환합니다. Z 좌표가 없는 경우 NULL 을 반환합니다. 포인트만 입력받을 수 있습니다.
- ST_ZMax - 2 차원 경계 상자(box2d) 또는 3 차원 경계 상자(box3d) 또는 도형의 최대 Z 값을 반환합니다.
- ST_ZMin - 2 차원 경계 상자(box2d) 또는 3 차원 경계 상자(box3d) 또는 도형의 최대 Z 값을 반환합니다.
- ST_Zmflag - 도형의 ZM(의미론적 차원) 플래그를 2 바이트 정수형(smallint)으로 반환합니다. 값이 0 일 때 2D, 1 일 때 3DM, 3 일 때 3DZ, 4 일 때 4D 를 뜻합니다.
- TG_Equals - 두 TopoGeometry 가 동일한 지형 원시형으로 이루어졌을 경우 참을 반환합니다.
- TG_Intersects - 두 TopoGeometry 에서 나온 원시형의 쌍 가운데 하나라도 교차할 경우 참을 반환합니다.

- UpdateGeometrySRID - 도형 열, geometry_columns 메타데이터 그리고 SRID 에 있는 모든 피처의 SRID 를 업데이트합니다. 제약조건으로 강제됐을 경우, 제약조건도 새로운 SRID 제약조건으로 업데이트될 것입니다. 유형 정의에 따라 구 버전을 강제했다면, 유형 정의가 변경될 것입니다.
- geometry_overlaps_nd - A 의 n 차원 경계 상자와 B 의 n 차원 경계 상자가 교차하는 경우 TRUE 를 반환합니다.
- overlaps_nd_geometry_gidx - 도형의 (캐시된) n 차원 경계 상자가 n 차원 부동소수점 데이터형(float) 정밀도 경계 상자(GIDX)와 교차하는 경우 TRUE 를 반환합니다.
- overlaps_nd_gidx_geometry - n 차원 부동소수점 데이터형(float) 정밀도 경계 상자(GIDX)가 도형의 (캐시된) n 차원 경계 상자와 교차하는 경우 TRUE 를 반환합니다.
- overlaps_nd_gidx_gidx - n 차원 부동소수점 데이터형(float) 정밀도 경계 상자(GIDX) 2 개가 서로 교차하는 경우 TRUE 를 반환합니다.
- postgis_sfcgal_version - 실행중인 SFCGAL 의 버전을 반환합니다.

14.9. PostGIS 폐합 도형 지원 함수

다음 함수들은 CIRCULARSTRING, CURVEPOLYGON 및 다른 만곡 도형 유형들을 이용할 수 있는 PostGIS 함수입니다.

- AddGeometryColumn - 기존 속성 테이블에 도형 열을 추가합니다. 기본적으로 제약 조건보다는 유형 변경자를 사용해서 정의합니다. 구 버전 확인 제약조건 기반 습성(behavior)을 얻으려면 use_typmod 에 대해 거짓을 패스합니다.
- Box2D - 도형의 최대 범위(maximum extent)를 나타내는 BOX2D 를 반환합니다.
- Box3D - 도형의 최대 범위를 나타내는 BOX3D 를 반환합니다.
- DropGeometryColumn - 공간 테이블에서 도형 열을 제거합니다.
- GeometryType - 도형의 유형을 스트링으로 반환합니다. 예: 'LINESTRING', 'POLYGON', 'MULTIPOINT' 등등.
- PostGIS_AddBBox - 도형에 경계 상자를 추가합니다.
- PostGIS_DropBBox - 도형으로부터 경계 상자 캐시를 삭제합니다.
- PostGIS_HasBBox - 해당 도형의 경계 상자가 캐시화된 경우 참을 반환하고, 그렇지 않을 경우 거짓을 반환합니다.
- ST_3DExtent - 도형 행들의 경계를 이루는 BOX3D 경계 상자를 반환하는 종합 함수입니다.
- ST_Accum - 종합 함수입니다. 도형들의 배열을 작성합니다.
- ST_Affine - 도형에 3 차원 아핀 변환을 적용합니다.
- ST_AsBinary - 도형/지리형의 WKB(Well-Known Binary) 표현식을 SRID 메타데이터 없이 반환합니다.
- ST_AsEWKB - 도형/지리형의 WKB(Well-Known Binary) 표현식을 SRID 메타데이터와 함께 반환합니다.
- ST_AsEWKT - 도형의 WKT(Well-Known Text) 표현식을 SRID 메타데이터와 함께 반환합니다.

- ST_AsHEXEWKB - 도형을 소 엔디안(NDR) 또는 대 엔디안(XDR) 인코딩을 통해 HEXEWKB (텍스트) 형식으로 반환합니다.
- ST_AsText - 도형/지리형의 WKT(Well-Known Text) 표현식을 SRID 메타데이터 없이 반환합니다.
- ST_Collect - 다른 도형들의 집합으로부터 설정된 ST_Geometry 값을 반환합니다.
- ST_CoordDim - ST_Geometry 값의 좌표 차원을 반환합니다.
- ST_CurveToLine - 원호 스트링(CIRCULARSTRING)/만곡 폴리곤(CURVEPOLYGON)을 라인스트링/폴리곤으로 변환합니다.
- ST_Distance - 도형 유형에 대해, 두 도형 사이의 2 차원 데카르트 거리를 투영 단위(SRS 단위)로 반환합니다. 지리형 유형의 경우, 기본적으로 두 도형 사이의 측지 거리를 미터 단위로 반환합니다.
- ST_Dump - 도형 g1 을 구성하고 있는 geometry_dump(도형, 경로) 행들의 집합을 반환합니다.
- ST_DumpPoints - 도형을 구성하는 모든 포인트의 geometry_dump(도형, 경로) 행들의 집합을 반환합니다.
- ST_EndPoint - LINESTRING 또는 CIRCULARLINESTRING 도형의 마지막 포인트를 POINT 로 반환합니다.
- ST_EstimatedExtent - 주어진 공간 테이블의 '추정' 범위를 반환합니다. 도형 열의 통계로부터 추정치를 추출합니다. 따로 설정하지 않을 경우 현재 스키마를 이용할 것입니다.
- ST_FlipCoordinates - 입력 도형을 X 축과 Y 축을 뒤집은 상태로 반환합니다. 위도/경도 피처를 빌드해서 수정해야 하는 경우 유용합니다.
- ST_Force2D - 도형을 "2 차원 모드"로 강제합니다.
- ST_ForceCurve - 적용이 가능한 경우, 도형을 해당 도형의 만곡 유형으로 상위 형변환(upcast)합니다.
- ST_ForceSFS - 도형이 SFS 1.1 도형 유형만을 쓰도록 강제합니다.
- ST_Force3D - 도형을 XYZ 모드로 강제합니다. ST_Force3DZ 라고도 합니다.
- ST_Force3DM - 도형을 XYM 모드로 강제합니다.
- ST_Force3DZ - 도형을 XYZ 모드로 강제합니다.
- ST_Force4D - 도형을 XYZM 모드로 강제합니다.
- ST_ForceCollection - 도형을 도형 집합으로 변환합니다.
- ST_GeoHash - 도형의 GeoHash 표현식을 반환합니다.
- ST_GeogFromWKB - WKB 도형 표현식 또는 EWKB(확장 WKB)로부터 지리형 인스턴스를 생성합니다.
- ST_GeomFromEWKB - EWKB(Extended Well-Known Binary)로부터 지정된 ST_Geometry 값을 반환합니다.
- ST_GeomFromEWKT - EWKT(Extended Well-Known Text)로부터 지정된 ST_Geometry 값을 반환합니다.
- ST_GeomFromText - WKT 표현식으로부터 지정된 ST_Geometry 값을 반환합니다.

- ST_GeomFromWKB - WKB(Well-Known Binary) 도형 표현식과 선택적인 SRID 로부터 도형 인스턴스를 생성합니다.
- ST_GeometryN - 입력 도형이 도형 집합, (멀티)포인트, (멀티)라인스트링, 멀티커브(multicurve) 또는 (멀티)폴리곤이나 다면체 표면일 경우 1-기반 N 번째 도형을 반환하며, 그 외의 경우 NULL 을 반환합니다.
- = - A 의 경계 상자와 B 의 경계 상자가 동일할 경우 TRUE 를 반환합니다. 이중 정밀도 경계 상자를 이용합니다.
- &<| - A 의 경계 상자가 B 의 경계 상자와 겹치거나 그 아래에 있을 경우 TRUE 를 반환합니다.
- ST_HasArc - 도형 또는 도형 집합이 원호 스트링을 담고 있을 경우 참을 반환합니다.
- ST_IsClosed - LINESTRING 의 시작점과 종단점이 일치하는 경우 TRUE 를 반환합니다. 다면체 표면이 닫혀 (부피를 가지고) 있는 경우 TRUE 를 반환합니다.
- ST_IsCollection - 인수가 집합(MULTI*, GEOMETRYCOLLECTION, ...)인 경우 TRUE 를 반환합니다.
- ST_IsEmpty - 해당 도형이 텅 빈 도형 집합, 폴리곤, 포인트 등인 경우 TRUE 를 반환합니다.
- ST_LineToCurve - 라인스트링/폴리곤을 원호 스트링/만곡 폴리곤으로 변환합니다.
- ST_MemSize - 도형이 차지하는 (바이트 단위) 공간 용량을 반환합니다.
- ST_NPoints - 도형이 가지고 있는 포인트(꼭짓점)의 개수를 반환합니다.
- ST_NRings - 도형이 폴리곤 또는 멀티폴리곤인 경우 고리의 개수를 반환합니다.
- ST_PointFromWKB - 주어진 SRID 와 함께 WKB 로부터 도형을 만듭니다.
- ST_PointN - 도형 안에 있는 첫 번째 라인스트링 또는 원형 라인스트링의 N 번째 포인트를 반환합니다. 음수 값은 라인스트링의 종단점으로부터 반대로 집계된 것입니다. 도형 안에 라인스트링이 없는 경우 NULL 을 반환합니다.
- ST_Points - 도형의 모든 좌표들을 담고 있는 멀티포인트를 반환합니다.
- ST_Rotate - 원점(origin)을 기준으로 도형의 rotRadians 를 반시계 방향으로 회전시킵니다.
- ST_RotateZ - Z 축을 기준으로 도형의 rotRadians 를 회전시킵니다.
- ST_SRID - spatial_ref_sys 테이블에 정의되어 있는, ST_Geometry 에 대한 공간 참조 식별자를 반환합니다.
- ST_Scale - 주어진 인수로 도형의 크기를 조정합니다.
- ST_SetSRID - 도형에 적용된 SRID 를 특정 정수값으로 설정합니다.
- ST_StartPoint - LINESTRING 도형의 첫 번째 포인트를 POINT 로 반환합니다.
- ST_Summary - 도형의 내용을 요약한 텍스트를 반환합니다.
- ST_SwapOrdinates - 입력 도형을 좌표값을 뒤바꾼 상태로 반환합니다.
- ST_TransScale - 주어진 인수 및 오프셋으로 도형을 변형시킵니다.

- ST_Transform - 입력 도형의 좌표를 다른 공간 참조 시스템으로 변환시킨 새 도형을 반환합니다.
- ST_Translate - 주어진 오프셋으로 도형을 변형시킵니다.
- ST_XMax - 2 차원 경계 상자(box2d) 또는 3 차원 경계 상자(box3d) 또는 도형의 최대 X 값을 반환합니다.
- ST_XMin - 2 차원 경계 상자(box2d) 또는 3 차원 경계 상자(box3d) 또는 도형의 최소 X 값을 반환합니다.
- ST_YMax - 2 차원 경계 상자(box2d) 또는 3 차원 경계 상자(box3d) 또는 도형의 최대 Y 값을 반환합니다.
- ST_YMin - 2 차원 경계 상자(box2d) 또는 3 차원 경계 상자(box3d) 또는 도형의 최소 Y 값을 반환합니다.
- ST_ZMax - 2 차원 경계 상자(box2d) 또는 3 차원 경계 상자(box3d) 또는 도형의 최대 Z 값을 반환합니다.
- ST_ZMin - 2 차원 경계 상자(box2d) 또는 3 차원 경계 상자(box3d) 또는 도형의 최대 Z 값을 반환합니다.
- ST_Zmflag - 도형의 ZM(의미론적 차원) 플래그를 2 바이트 정수형(smallint)으로 반환합니다. 값이 0 일 때 2D, 1 일 때 3DM, 3 일 때 3DZ, 4 일 때 4D 를 뜻합니다.
- UpdateGeometrySRID - 도형 열, geometry_columns 메타데이터 그리고 SRID 에 있는 모든 피처의 SRID 를 업데이트합니다. 제약조건으로 강제됐을 경우, 제약조건도 새로운 SRID 제약조건으로 업데이트될 것입니다. 유형 정의에 따라 구 버전을 강제했다면, 유형 정의가 변경될 것입니다.
- ~(box2df,box2df) - R2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)가 또다른 2 차원 부동소수점 데이터형 정밀도 경계 상자(BOX2DF)를 포함하는 경우 TRUE 를 반환합니다.
- ~(box2df,geometry) - 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)가 도형의 2 차원 경계 상자를 포함하는 경우 TRUE 를 반환합니다.
- ~(geometry,box2df) - 도형의 2 차원 경계 상자가 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)를 포함하는 경우 TRUE 를 반환합니다.
- && - A 의 2D 경계 상자와 B 의 2D 경계 상자가 교차하는 경우 TRUE 를 반환합니다.
- &&& - A 의 n 차원 경계 상자와 B 의 n 차원 경계 상자가 교차하는 경우 TRUE 를 반환합니다.
- @(box2df,box2df) - 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)가 또다른 2 차원 부동소수점 데이터형 정밀도 경계 상자 안에 포함되는 경우 TRUE 를 반환합니다.
- @(box2df,geometry) - 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)가 도형의 2 차원 경계 상자 안에 포함되는 경우 TRUE 를 반환합니다.
- @(geometry,box2df) - 도형의 2 차원 경계 상자가 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF) 안에 포함되는 경우 TRUE 를 반환합니다.
- &&(box2df,box2df) - 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF) 2 개가 서로 교차하는 경우 TRUE 를 반환합니다.
- &&(box2df,geometry) - 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)가 도형의 (캐시된) 2 차원 경계 상자와 교차하는 경우 TRUE 를 반환합니다.

- &&(geometry,box2df) - 도형의 (캐시된) 2 차원 경계 상자가 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)와 교차하는 경우 TRUE 를 반환합니다.
- &&&(geometry,gidx) - 도형의 (캐시된) n 차원 경계 상자가 n 차원 부동소수점 데이터형(float) 정밀도 경계 상자(GIDX)와 교차하는 경우 TRUE 를 반환합니다.
- &&&(gidx,geometry) - n 차원 부동소수점 데이터형(float) 정밀도 경계 상자(GIDX)가 도형의 (캐시된) n 차원 경계 상자와 교차하는 경우 TRUE 를 반환합니다.
- &&&(gidx,gidx) - n 차원 부동소수점 데이터형(float) 정밀도 경계 상자(GIDX) 2 개가 서로 교차하는 경우 TRUE 를 반환합니다.

14.10. PostGIS 다면체 표면 지원 함수

다음 함수들은 POLYHEDRALSURFACE, POLYHEDRALSURFACEM 도형 유형들을 이용할 수 있는 PostGIS 함수입니다.

- Box2D - 도형의 최대 범위(maximum extent)를 나타내는 BOX2D 를 반환합니다.
- Box3D - 도형의 최대 범위를 나타내는 BOX3D 를 반환합니다.
- GeometryType - 도형의 유형을 스트링으로 반환합니다. 예: 'LINESTRING', 'POLYGON', 'MULTIPOINT' 등등.
- ST_3DArea - 3 차원 표면 도형의 면적을 계산합니다. 입체일 경우 0 을 반환할 것입니다.
- ST_3DClosestPoint - g2 에 가장 가까운 g1 상에 있는 3 차원 포인트를 반환합니다. 해당 포인트는 3D 최단 라인의 첫 번째 포인트입니다.
- ST_3DDFullyWithin - 모든 3D 도형이 서로의 설정된 거리 내에 있을 경우 참을 반환합니다.
- ST_3DDWithin - 3D(Z) 도형 유형에 대해, 두 도형의 3 차원 거리가 단위의 개수 내일 경우 참을 반환합니다.
- ST_3DDifference - 3 차원 차이를 수행합니다.
- ST_3DDistance - 도형 유형에 대해, 두 도형 사이의 (SRS 에 기반한) 3 차원 데카르트 최단 거리를 투영 단위로 반환합니다.
- ST_3DExtent - 도형 행들의 경계를 이루는 BOX3D 경계 상자를 반환하는 종합 함수입니다.
- ST_3DIntersection - 3 차원 교차를 수행합니다.
- ST_3DIntersects - 도형들이 3 차원에서 "공간적으로 교차"하는 경우 참을 반환합니다. 포인트, 라인스트링, 폴리곤, 다면체 표면(면적)만 입력받습니다. SFCGAL 백엔드가 활성화됐다면 TIN 도 지원합니다.
- ST_3DLongestLine - 두 도형 사이의 3 차원 최장(longest) 라인을 반환합니다.
- ST_3DMaxDistance - 도형 유형에 대해, 두 도형 사이의 (SRS 에 기반한) 3 차원 데카르트 최장 거리를 투영 단위로 반환합니다.
- ST_3DShortestLine - 두 도형 사이의 3 차원 최단(shortest) 라인을 반환합니다.

- ST_3DUnion - 3 차원 통합을 수행합니다.
- ST_Accum - 종합 함수입니다. 도형들의 배열을 작성합니다.
- ST_Affine - 도형에 3 차원 아핀 변환을 적용합니다.
- ST_ApproximateMedialAxis - 면 도형의 근사 중심축을 계산합니다.
- ST_Area - 도형이 폴리곤이나 멀티폴리곤일 경우 표면의 면적을 반환합니다. 도형 유형의 경우, SRID 가 지정한 단위로 2 차원 데카르트 면적을 구합니다. 지리형 유형의 경우, 평방미터 단위로 곡면(curved surface)의 면적을 구합니다.
- ST_AsBinary - 도형/지리형의 WKB(Well-Known Binary) 표현식을 SRID 메타데이터 없이 반환합니다.
- ST_AsEWKB - 도형/지리형의 WKB(Well-Known Binary) 표현식을 SRID 메타데이터와 함께 반환합니다.
- ST_AsEWKT - 도형의 WKT(Well-Known Text) 표현식을 SRID 메타데이터와 함께 반환합니다.
- ST_AsGML - 도형을 GML 2 또는 GML 3 버전 요소로 반환합니다.
- ST_AsX3D - 도형을 X3D XML 노드 요소 형식: ISO-IEC-19776-1.2-X3DEncodings-XML 로 반환합니다.
- ST_CoordDim - ST_Geometry 값의 좌표 차원을 반환합니다.
- ST_Dimension - 좌표 차원과 동등하거나 낮아야 하는, 해당 도형 객체의 내재된 차원을 반환합니다.
- ST_Dump - 도형 g1 을 구성하고 있는 geometry_dump(도형, 경로) 행들의 집합을 반환합니다.
- ST_DumpPoints - 도형을 구성하는 모든 포인트의 geometry_dump(도형, 경로) 행들의 집합을 반환합니다.
- ST_Expand - 입력 도형의 경계 상자로부터 모든 방향으로 확장된 경계 상자를 반환합니다. 이중 정밀도 데이터형을 사용합니다.
- ST_Extent - 도형 행들의 경계를 이루는 경계 상자를 반환하는 종합 함수입니다.
- ST_Extrude - 표면을 관련 입체로 돌출시킵니다.
- ST_FlipCoordinates - 입력 도형을 X 축과 Y 축을 뒤집은 상태로 반환합니다. 위도/경도 피처를 빌드해서 수정해야 하는 경우 유용합니다.
- ST_Force2D - 도형을 "2 차원 모드"로 강제합니다.
- ST_ForceLHR - LHR(Left Hand Reverse; 시계 방향) 방향을 강제합니다.
- ST_ForceRHR - 폴리곤 안에 있는 꼭짓점들의 방향(orientation)이 오른손 법칙(Right-Hand Rule)을 따르도록 강제합니다.
- ST_ForceSFS - 도형이 SFS 1.1 도형 유형만을 쓰도록 강제합니다.
- ST_Force3D - 도형을 XYZ 모드로 강제합니다. ST_Force3DZ 라고도 합니다.
- ST_Force3DZ - 도형을 XYZ 모드로 강제합니다.
- ST_ForceCollection - 도형을 도형 집합으로 변환합니다.

- ST_GeomFromEWKB - EWKB(Extended Well-Known Binary)로부터 지정된 ST_Geometry 값을 반환합니다.
- ST_GeomFromEWKT - EWKT(Extended Well-Known Text)로부터 지정된 ST_Geometry 값을 반환합니다.
- ST_GeomFromGML - 도형의 GML 표현식을 입력받아 PostGIS 도형 객체를 출력합니다.
- ST_GeometryN - 입력 도형이 도형 집합, (멀티)포인트, (멀티)라인스트링, 멀티커브(multicurve) 또는 (멀티)폴리곤이나 다면체 표면일 경우 1-기반 N 번째 도형을 반환하며, 그 외의 경우 NULL 을 반환합니다.
- ST_GeometryType - ST_Geometry 값의 도형 유형을 반환합니다.
- = - A 의 경계 상자와 B 의 경계 상자가 동일할 경우 TRUE 를 반환합니다. 이중 정밀도 경계 상자를 이용합니다.
- &<| - A 의 경계 상자가 B 의 경계 상자와 겹치거나 그 아래에 있을 경우 TRUE 를 반환합니다.
- ~= - A 의 경계 상자와 B 의 경계 상자가 동일할 경우 TRUE 를 반환합니다.
- ST_IsClosed - LINESTRING 의 시작점과 종단점이 일치하는 경우 TRUE 를 반환합니다. 다면체 표면이 닫혀 (부피를 가지고) 있는 경우 TRUE 를 반환합니다.
- ST_IsPlanar - 표면이 평면인지 아닌지 확인합니다.
- ST_IsSolid - 도형이 입체인지 테스트합니다. 어떤 유효성 검사도 수행하지 않습니다.
- ST_MakeSolid - 도형을 입체로 지정합니다. 어떤 확인 작업도 수행하지 않습니다. 유효한 입체를 얻으려면, 입력 도형이 닫힌 다면체 표면 또는 닫힌 TIN 이어야만 합니다.
- ST_MemSize - 도형이 차지하는 (바이트 단위) 공간 용량을 반환합니다.
- ST_NPoints - 도형이 가지고 있는 포인트(꼭짓점)의 개수를 반환합니다.
- ST_NumGeometries - 도형이 도형 집합 또는 멀티 도형인 경우 도형의 개수를 반환합니다. 단일 도형인 경우 1 을 반환하고, 도형 집합도 멀티 도형도 단일 도형도 아닌 경우 NULL 을 반환합니다.
- ST_NumPatches - 다면체 표면 상에 있는 면의 개수를 반환합니다. 다면체 도형이 아닌 경우 NULL 을 반환할 것입니다.
- ST_PatchN - 도형이 POLYHEDRALSURFACE, POLYHEDRALSURFACEM 인 경우 1-기반 N 번째 도형 (면)을 반환합니다. 그 외의 경우 NULL 을 반환합니다.
- ST_RemoveRepeatedPoints - 입력 도형을 중복된 포인트들을 제거한 상태로 반환합니다.
- ST_Rotate - 원점(origin)을 기준으로 도형의 rotRadians 를 반시계 방향으로 회전시킵니다.
- ST_RotateX - X 축을 기준으로 도형의 rotRadians 를 회전시킵니다.
- ST_RotateY - Y 축을 기준으로 도형의 rotRadians 를 회전시킵니다.
- ST_RotateZ - Z 축을 기준으로 도형의 rotRadians 를 회전시킵니다.
- ST_Scale - 주어진 인수로 도형의 크기를 조정합니다.
- ST_ShiftLongitude - 도형의 좌표를 -180 ~ 180 범위와 0 ~ 360 범위 간에 전환합니다.

- ST_StraightSkeleton - 도형으로부터 직선 골격(straight skeleton)을 계산합니다.
- ST_Summary - 도형의 내용을 요약한 텍스트를 반환합니다.
- ST_SwapOrdinates - 입력 도형을 좌표값을 뒤바꾼 상태로 반환합니다.
- ST_Tesselate - 폴리곤 또는 다면체 표면의 표면 다듬기(tesselation) 작업을 수행한 다음 TIN 또는 TIN 집합으로 반환합니다.
- ST_Transform - 입력 도형의 좌표를 다른 공간 참조 시스템으로 변환시킨 새 도형을 반환합니다.
- ST_Volume - 3 차원 입체의 부피를 계산합니다. 표면 도형을 입력하면 (닫힌 도형일지라도) 0 을 반환할 것입니다.
- ~(box2df,box2df) - R2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)가 또다른 2 차원 부동소수점 데이터형 정밀도 경계 상자(BOX2DF)를 포함하는 경우 TRUE 를 반환합니다.
- ~(box2df,geometry) - 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)가 도형의 2 차원 경계 상자를 포함하는 경우 TRUE 를 반환합니다.
- ~(geometry,box2df) - 도형의 2 차원 경계 상자가 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)를 포함하는 경우 TRUE 를 반환합니다.
- && - A 의 2D 경계 상자와 B 의 2D 경계 상자가 교차하는 경우 TRUE 를 반환합니다.
- &&& - A 의 n 차원 경계 상자와 B 의 n 차원 경계 상자가 교차하는 경우 TRUE 를 반환합니다.
- @(box2df,box2df) - 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)가 또다른 2 차원 부동소수점 데이터형 정밀도 경계 상자 안에 포함되는 경우 TRUE 를 반환합니다.
- @(box2df,geometry) - 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)가 도형의 2 차원 경계 상자 안에 포함되는 경우 TRUE 를 반환합니다.
- @(geometry,box2df) - 도형의 2 차원 경계 상자가 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF) 안에 포함되는 경우 TRUE 를 반환합니다.
- &&(box2df,box2df) - 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF) 2 개가 서로 교차하는 경우 TRUE 를 반환합니다.
- &&(box2df,geometry) - 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)가 도형의 (캐시된) 2 차원 경계 상자와 교차하는 경우 TRUE 를 반환합니다.
- &&(geometry,box2df) - 도형의 (캐시된) 2 차원 경계 상자가 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)와 교차하는 경우 TRUE 를 반환합니다.
- &&&(geometry,gidx) - 도형의 (캐시된) n 차원 경계 상자가 n 차원 부동소수점 데이터형(float) 정밀도 경계 상자(GIDX)와 교차하는 경우 TRUE 를 반환합니다.
- &&&(gidx,geometry) - n 차원 부동소수점 데이터형(float) 정밀도 경계 상자(GIDX)가 도형의 (캐시된) n 차원 경계 상자와 교차하는 경우 TRUE 를 반환합니다.
- &&&(gidx,gidx) - n 차원 부동소수점 데이터형(float) 정밀도 경계 상자(GIDX) 2 개가 서로 교차하는 경우 TRUE 를 반환합니다.

- postgis_sfcgal_version - 실행중인 SFCGAL 의 버전을 반환합니다.

14.11. PostGIS 함수 지원 매트릭스

다음은 PostGIS 의 공간 특화 함수들의 알파벳 순서 목록으로, 각 함수가 사용할 수 있는 공간 유형의 종류 또는 각 함수가 따라야 하는 OGC/SQL 호환성의 종류를 보여줍니다.

- ✓는 해당 함수가 태생적으로 해당 유형 또는 하위유형을 사용할 수 있다는 뜻입니다.
-  표시는 해당 함수가 해당 유형을 사용할 수 있긴 하지만, 내장된 변환자를 통해 도형으로 변환한 후 "최적의 **SRID**" 공간 참조 시스템으로 변경한 다음 원래대로 변환해야 한다는 뜻입니다. 광대한 지역 또는 극지역의 경우 기대한 대로의 결과가 나오지 않을 수도 있으며 부동소수점 찌꺼기를 축적할 수도 있습니다.
-  표시는 해당 함수가 직접 해당 유형을 지원하기보다 **BOX3D** 같은 다른 유형으로 자동 변환하기 때문에 해당 유형을 사용할 수 있다는 뜻입니다.
-  표시는 **PostGIS** 를 **SFCGAL** 지원과 함께 컴파일한 경우에만 해당 함수가 해당 유형을 사용할 수 있다는 뜻입니다.
-  표시는 **PostGIS** 를 **SFCGAL** 지원과 함께 컴파일한 경우 **SFCGAL** 이 함수를 지원하고, 그렇지 않을 경우 **GEOS**/내장 지원이 함수를 지원한다는 뜻입니다.
- **geom** – 기본 2 차원 도형 지원 **(x,y).**
- **geog** – 기본 2 차원 지리형 지원 **(x,y).**
- **2.5D** – 3 차원/4 차원 공간 내부의 기본 2 차원 도형(Z 또는 M 좌표 있음).
- **PS** – 다면체 표면
- **T** – 삼각형 및 **TIN(Triangulated Irregular Network surfaces)**

Function	*geom*	*geog*	*2.5D*	*Curves*	*SQL MM*	*PS*	*T*
Box2D	✓			✓		✓	✓
Box3D	✓		✓	✓		✓	✓
Find_SRID							
도형 유형	✓		✓	✓		✓	✓
ST_3DArea							
ST_3DClosestPoint	✓		✓			✓	
ST_3DDFullyWithin	✓		✓			✓	
ST_3DDWithin	✓		✓		✓	✓	
ST_3DDifference							
ST_3DDistance	✓						
ST_3DExtent	✓		✓	✓		✓	✓
ST_3DIntersection							
ST_3DIntersects	✓						
ST_3DLength	✓		✓				
ST_3DLongestLine	✓		✓			✓	
ST_3DMakeBox	✓						
ST_3DMaxDistance	✓		✓			✓	
ST_3DPerimeter	✓		✓				
ST_3DShortestLine	✓		✓			✓	

Function	*geom*	*geog*	*2.5D*	*Curves*	*SQL MM*	*PS*	*T*
ST_3DUnion							
ST_Accum	✓		✓	✓		✓	✓
ST_AddMeasure	✓		✓				
ST_AddPoint	✓		✓				
ST_Affine	✓		✓	✓		✓	✓
ST_ApproximateMedialAxis							
ST_Area	✓	✓					
ST_AsBinary	✓	✓	✓	✓	✓	✓	✓
ST_AsEWKB	✓		✓	✓		✓	✓
ST_AsEWKT	✓	✓	✓	✓		✓	✓
ST_AsEncodedPolyline	✓						
ST_AsGML	✓	✓	✓			✓	✓
ST_AsGeoJSON	✓	✓	✓				
ST_AsHEXEWKB	✓		✓	✓			
ST_AsKML	✓	✓	✓				
ST_AsLatLonText	✓						
ST_AsSVG	✓	✓					
ST_AsTWKB	✓						
ST_AsText	✓	✓		✓	✓		
ST_AsX3D	✓		✓			✓	✓
ST_Azimuth	✓	✓					
ST_BdMPolyFromText	✓						
ST_BdPolyFromText	✓						
ST_Boundary	✓		✓		✓		
ST_BoundingDiagonal	✓		✓				
ST_Box2dFromGeoHash							
ST_Buffer	✓	✓			✓		
ST_BuildArea	✓						
ST_CPAWithin	✓		✓				
ST_Centroid	✓				✓		
ST_ClipByBox2D	✓						
ST_ClosestPoint	✓						
ST_ClosestPointOfApproach	✓		✓				
ST_ClusterDBSCAN	✓						
ST_ClusterIntersecting	✓						
ST_ClusterKMeans	✓						
ST_ClusterWithin	✓						
ST_Collect	✓		✓	✓			
ST_CollectionExtract	✓						
ST_CollectionHomogenize	✓						
ST_ConcaveHull	✓						
ST_Contains	✓				✓		
ST_ContainsProperly	✓						
ST_ConvexHull	✓		✓		✓		
ST_CoordDim	✓		✓	✓	✓	✓	✓
ST_CoveredBy	✓	✓					
ST_Covers	✓	✓					

Function	geom	geog	2.5D	Curves	SQL MM	PS	T
ST_Crosses	✓				✓		
ST_CurveToLine	✓		✓	✓	✓		
ST_DFullyWithin	✓						
ST_DWithin	✓	✓					
ST_DelaunayTriangles	✓		✓				✓
ST_Difference	✓		✓		✓		
ST_Dimension	✓				✓	✓	✓
ST_Disjoint	✓				✓		
ST_Distance	✓	✓		✓			
ST_DistanceCPA	✓		✓				
ST_DistanceSphere	✓						
ST_DistanceSpheroid	✓						
ST_Dump	✓		✓	✓		✓	✓
ST_DumpPoints	✓		✓	✓		✓	✓
ST_DumpRings	✓		✓				
ST_EndPoint	✓		✓	✓	✓		
ST_Envelope	✓				✓		
ST_Equals	✓				✓		
ST_EstimatedExtent				✓			
ST_Expand	✓					✓	✓
ST_Extent	✓					✓	✓
ST_ExteriorRing	✓		✓		✓		
ST_Extrude							
ST_FlipCoordinates	✓		✓	✓		✓	✓
ST_Force2D	✓		✓	✓		✓	
ST_ForceCurve	✓		✓	✓			
ST_ForceLHR							
ST_ForceRHR	✓		✓			✓	
ST_ForceSFS	✓		✓	✓		✓	✓
ST_Force3D	✓		✓	✓		✓	
ST_Force3DM	✓			✓			
ST_Force3DZ	✓		✓	✓		✓	
ST_Force4D	✓		✓	✓			
ST_ForceCollection	✓		✓	✓		✓	
ST_GMLToSQL	✓				✓		
ST_GeneratePoints	✓						
ST_GeoHash	✓			✓			
ST_GeogFromText		✓					
ST_GeogFromWKB		✓		✓			
ST_GeographyFromText		✓					
ST_GeomCollFromText	✓				✓		
ST_GeomFromEWKB	✓		✓	✓		✓	✓
ST_GeomFromEWKT	✓		✓	✓		✓	✓
ST_GeomFromGML	✓		✓			✓	✓
ST_GeomFromGeoHash	✓						
ST_GeomFromGeoJSON	✓		✓				
ST_GeomFromKML	✓		✓				

Function	*geom*	*geog*	*2.5D*	*Curves*	*SQL MM*	*PS*	*T*
ST_GeomFromTWKB	✓						
ST_GeomFromText	✓			✓	✓		
ST_GeomFromWKB	✓			✓	✓		
ST_GeometricMedian	✓		✓				
ST_GeometryFromText	✓				✓		
ST_GeometryN	✓		✓	✓	✓	✓	✓
ST_GeometryType	✓		✓		✓	✓	
|>>	✓						
<<|	✓						
~	✓						
@	✓						
=	✓	✓		✓		✓	
<<	✓						
|&>	✓						
&<|	✓			✓		✓	
&<	✓						
&>	✓						
>>	✓						
~=	✓					✓	
ST_HasArc	✓		✓	✓			
ST_HausdorffDistance	✓						
ST_InteriorRingN	✓		✓		✓		
ST_InterpolatePoint	✓		✓				
ST_Intersection	✓	✓					
ST_Intersects	✓	✓					
ST_IsClosed	✓		✓	✓	✓	✓	
ST_IsCollection	✓		✓	✓			
ST_IsEmpty	✓			✓	✓		
ST_IsPlanar							
ST_IsRing	✓				✓		
ST_IsSimple	✓		✓		✓		
ST_IsSolid							
ST_IsValid	✓				✓		
ST_IsValidDetail	✓						
ST_IsValidReason	✓						
ST_IsValidTrajectory	✓		✓				
ST_Length	✓	✓					
ST_Length2D	✓						
ST_Length2D_Spheroid	✓						
ST_LengthSpheroid	✓		✓				
ST_LineCrossingDirection	✓						
ST_LineFromEncodedPolyline	✓						
ST_LineFromMultiPoint	✓		✓				
ST_LineFromText	✓				✓		
ST_LineFromWKB	✓				✓		
ST_LineInterpolatePoint	✓		✓				

Function	geom	geog	2.5D	Curves	SQL MM	PS	T
ST_LineLocatePoint	✓						
ST_LineMerge	✓						
ST_LineSubstring	✓		✓				
ST_LineToCurve	✓		✓	✓			
ST_LinestringFromWKB	✓				✓		
ST_LocateAlong	✓						
ST_LocateBetween	✓						
ST_LocateBetweenElevations	✓		✓				
ST_LongestLine	✓						
ST_M	✓		✓		✓		
ST_MLineFromText	✓				✓		
ST_MPointFromText	✓				✓		
ST_MPolyFromText	✓				✓		
ST_MakeBox2D	✓						
ST_MakeEnvelope	✓						
ST_MakeLine	✓		✓				
ST_MakePoint	✓		✓				
ST_MakePointM	✓						
ST_MakePolygon	✓		✓				
ST_MakeSolid							
ST_MakeValid	✓		✓				
ST_MaxDistance	✓						
ST_MemSize	✓		✓	✓		✓	✓
ST_MemUnion	✓		✓				
ST_MinimumBoundingCircle	✓						
ST_MinimumBoundingRadius	✓						
ST_MinimumClearance	✓						
ST_MinimumClearanceLine	✓						
ST_MinkowskiSum							
ST_Multi	✓						
ST_NDims	✓		✓				
ST_NPoints	✓		✓	✓		✓	
ST_NRings	✓		✓	✓			
ST_Node	✓		✓				
ST_Normalize	✓						
ST_NumGeometries	✓		✓		✓	✓	✓
ST_NumInteriorRing	✓						
ST_NumInteriorRings	✓				✓		
ST_NumPatches	✓		✓		✓	✓	
ST_NumPoints	✓				✓		
ST_OffsetCurve	✓						
ST_OrderingEquals	✓				✓		
ST_Orientation							
ST_Overlaps	✓				✓		
ST_PatchN	✓		✓		✓	✓	
ST_Perimeter	✓	✓			✓		
ST_Perimeter2D	✓						

Function	*geom*	*geog*	*2.5D*	*Curves*	*SQL MM*	*PS*	*T*
ST_Point	✓				✓		
ST_PointFromGeoHash							
ST_PointFromText	✓				✓		
ST_PointFromWKB	✓		✓	✓	✓		
ST_PointN	✓		✓	✓	✓		
ST_PointOnSurface	✓		✓		✓		
ST_PointInsideCircle	✓						
ST_Points	✓		✓	✓			
ST_Polygon	✓		✓		✓		
ST_PolygonFromText	✓				✓		
ST_Polygonize	✓						
ST_Project		✓					
ST_Relate	✓				✓		
ST_RelateMatch							
ST_RemovePoint	✓		✓				
ST_RemoveRepeatedPoints	✓		✓			✓	
ST_Reverse	✓						
ST_Rotate	✓		✓	✓		✓	✓
ST_RotateX	✓		✓			✓	✓
ST_RotateY	✓		✓			✓	✓
ST_RotateZ	✓		✓	✓		✓	✓
ST_SRID	✓			✓	✓		
ST_Scale	✓		✓	✓		✓	✓
ST_Segmentize	✓	✓					
ST_SetEffectiveArea	✓						
ST_SetPoint	✓		✓				
ST_SetSRID	✓			✓			
ST_SharedPaths	✓						
ST_ShiftLongitude	✓		✓			✓	✓
ST_ShortestLine	✓						
ST_Simplify	✓						
ST_SimplifyPreserveTopology	✓						
ST_SimplifyVW	✓						
ST_Snap	✓						
ST_SnapToGrid	✓		✓				
ST_Split	✓						
ST_StartPoint	✓		✓	✓	✓		
ST_StraightSkeleton							
ST_Subdivide	✓						
ST_Summary	✓	✓		✓		✓	✓
ST_SwapOrdinates	✓		✓	✓		✓	✓
ST_SymDifference	✓		✓		✓		
ST_Tesselate							
ST_Touches	✓				✓		
ST_TransScale	✓		✓	✓			
ST_Transform	✓			✓	✓	✓	
ST_Translate	✓		✓	✓			

Function	geom	geog	2.5D	Curves	SQL MM	PS	T
ST_UnaryUnion	✓		✓				
ST_Union	✓				✓		
ST_Volume							
ST_Voronoi	✓						
ST_WKBToSQL	✓				✓		
ST_WKTToSQL	✓				✓		
ST_Within	✓				✓		
ST_WrapX	✓		✓				
ST_X	✓		✓		✓		
ST_XMax			✓	✓			
ST_XMin			✓	✓			
ST_Y	✓		✓		✓		
ST_YMax			✓	✓			
ST_YMin			✓	✓			
ST_Z	✓		✓		✓		
ST_ZMax			✓	✓			
ST_ZMin			✓	✓			
ST_Zmflag	✓		✓	✓			
~(box2df,box2df)				✓		✓	
~(box2df,geometry)	✓			✓		✓	
~(geometry,box2df)	✓			✓		✓	
<#>	✓						
<<#>>	✓						
<<->>	✓						
|=|	✓						
<->	✓	✓					
&&	✓	✓		✓		✓	
&&&	✓		✓	✓		✓	✓
@(box2df,box2df)				✓		✓	
@(box2df,geometry)	✓			✓		✓	
@(geometry,box2df)	✓			✓		✓	
&&(box2df,box2df)				✓		✓	
&&(box2df,geometry)	✓			✓		✓	
&&(geometry,box2df)	✓			✓		✓	
&&&(geometry,gidx)	✓		✓	✓		✓	✓
&&&(gidx,geometry)	✓		✓	✓		✓	✓
&&&(gidx,gidx)			✓	✓		✓	✓
postgis.backend							
postgis.enable_outdb_rasters							
postgis.gdal_datapath							
postgis.gdal_enabled_drivers							
postgis_sfcgal_version							

14.12. 신규, 개선, 변경된 PostGIS 함수

14.12.1. 2.3 버전에서 신규 또는 개선된 PostGIS 함수

다음 함수들은 추가 또는 개선된 PostGIS 함수입니다.

PostGIS 2.3 버전 신규 함수들:

- &&&(geometry,gidx) - BRIN(Block Range INdexes)을 지원하기 시작한 2.3.0 버전부터 사용할 수 있습니다. 도형의 (캐시된) n 차원 경계 상자가 n 차원 부동소수점 데이터형(float) 정밀도 경계 상자(GIDX)와 교차하는 경우 TRUE 를 반환합니다.
- &&&(gidx,geometry) - BRIN(Block Range INdexes)을 지원하기 시작한 2.3.0 버전부터 사용할 수 있습니다. n 차원 부동소수점 데이터형(float) 정밀도 경계 상자(GIDX)가 도형의 (캐시된) n 차원 경계 상자와 교차하는 경우 TRUE 를 반환합니다.
- &&&(gidx,gidx) - BRIN(Block Range INdexes)을 지원하기 시작한 2.3.0 버전부터 사용할 수 있습니다. n 차원 부동소수점 데이터형(float) 정밀도 경계 상자(GIDX) 2 개가 서로 교차하는 경우 TRUE 를 반환합니다.
- &&(box2df,box2df) - BRIN(Block Range INdexes)을 지원하기 시작한 2.3.0 버전부터 사용할 수 있습니다. 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF) 2 개가 서로 교차하는 경우 TRUE 를 반환합니다.
- &&(box2df,geometry) - BRIN(Block Range INdexes)을 지원하기 시작한 2.3.0 버전부터 사용할 수 있습니다. 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)가 도형의 (캐시된) 2 차원 경계 상자와 교차하는 경우 TRUE 를 반환합니다.
- &&(geometry,box2df) - BRIN(Block Range INdexes)을 지원하기 시작한 2.3.0 버전부터 사용할 수 있습니다. 도형의 (캐시된) 2 차원 경계 상자가 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)와 교차하는 경우 TRUE 를 반환합니다.
- @(box2df,box2df) - BRIN(Block Range INdexes)을 지원하기 시작한 2.3.0 버전부터 사용할 수 있습니다. 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)가 또다른 2 차원 부동소수점 데이터형 정밀도 경계 상자 안에 포함되는 경우 TRUE 를 반환합니다.
- @(box2df,geometry) - BRIN(Block Range INdexes)을 지원하기 시작한 2.3.0 버전부터 사용할 수 있습니다. 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)가 도형의 2 차원 경계 상자 안에 포함되는 경우 TRUE 를 반환합니다.
- @(geometry,box2df) - BRIN(Block Range INdexes)을 지원하기 시작한 2.3.0 버전부터 사용할 수 있습니다. 도형의 2 차원 경계 상자가 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF) 안에 포함되는 경우 TRUE 를 반환합니다.
- ~(box2df,box2df) - BRIN(Block Range INdexes)을 지원하기 시작한 2.3.0 버전부터 사용할 수 있습니다. R2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)가 또다른 2 차원 부동소수점 데이터형 정밀도 경계 상자(BOX2DF)를 포함하는 경우 TRUE 를 반환합니다.

- ~(box2df,geometry) - BRIN(Block Range INdexes)을 지원하기 시작한 2.3.0 버전부터 사용할 수 있습니다. 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)가 도형의 2 차원 경계 상자를 포함하는 경우 TRUE 를 반환합니다.
- ~(geometry,box2df) - BRIN(Block Range INdexes)을 지원하기 시작한 2.3.0 버전부터 사용할 수 있습니다. 도형의 2 차원 경계 상자가 2 차원 부동소수점 데이터형(float) 정밀도 경계 상자(BOX2DF)를 포함하는 경우 TRUE 를 반환합니다.

다음 함수들은 PostGIS 2.3 버전에서 개선된 PostGIS 함수입니다.

14.12.2. 2.2 버전에서 신규 또는 개선된 PostGIS 함수

다음 함수들은 추가 또는 개선된 PostGIS 함수입니다.



이제 CREATE EXTENSION postgis_sfcgal; 을 입력해서 postgis_sfcgal 을 확장 프로그램으로 설치할 수 있습니다.



PostGIS 2.2.0 버전부터 TIGER 2015 데이터를 사용할 수 있도록 Tiger Geocoder 를 업그레이드했습니다.



주소 데이터를 표준화하기 위한 address_standardizer, address_standardizer_data_us 확장 프로그램에 대해 자세히 알고 싶다면 12. “주소 표준화 도구”를 참조하십시오.



보다 향상된 성능을 위해 많은 지형 관련 함수를 C 함수로 재작성했습니다.

PostGIS 2.2 버전 신규 함수들:

다음 함수들은 PostGIS 2.2 버전에서 개선된 PostGIS 함수입니다.

14.12.3. 2.2 버전에서 크게 변경된 PostGIS 함수

다음 함수들은 PostGIS 2.2 버전에서 크게 변경되었을 수 있는 PostGIS 함수입니다. 이 함수들 가운데 하나라도 사용하고 있다면, 사용자의 기존 코드를 확인해봐야 할지도 모릅니다.

14.12.4. 2.1 버전에서 신규 또는 개선된 PostGIS 함수

다음 함수들은 추가 또는 개선된 PostGIS 함수입니다.



지형 관련 성능이 더욱 향상되었습니다. 자세한 내용은 11. "지형(topology)"을 참조하십시오.



(특히 대역외 out-of-band 래스터 처리 관련) 버그를 수정하고, 많은 신규 함수들(일반 작업을 수행하기 위해 입력해야 할 코드를 짧게 줄임)을 추가했으며, 래스터 기능의 획기적인 속도 향상을 이루었습니다. 자세한 내용은 9. "래스터 참조 문서"를 참조하십시오.



PostGIS 2.1.0 버전부터 TIGER 2012 인구조사 데이터를 사용할 수 있도록 Tiger Geocoder 를 업그레이드했습니다. 순위 선호도를 디버그하고 조정하기 위해 geocode_settings 를 추가했고, 로더를 정비하여 로드할 테이블만 다운로드하도록 했습니다. PostGIS 2.1.1 버전부터 TIGER 2013 데이터를 사용할 수 있도록 Tiger Geocoder 를 업그레이드했습니다. 자세한 내용은 13.1 "TIGER 지오코딩 도구"를 참조하십시오.

PostGIS 2.1 버전 신규 함수들:

다음 함수들은 PostGIS 2.1 버전에서 개선된 PostGIS 함수입니다.

14.12.5. 2.1 버전에서 크게 변경된 PostGIS 함수

다음 함수들은 PostGIS 2.1 버전에서 크게 변경되었을 수 있는 PostGIS 함수입니다. 이 함수들 가운데 하나라도 사용하고 있다면, 사용자의 기존 코드를 확인해봐야 할지도 모릅니다.

14.12.6. 2.0 버전에서 신규, 습성 변경, 또는 개선된 PostGIS 함수

다음 함수들은 PostGIS 2.0 배포판에서 추가, 개선, 또는 14.12.7 "2.0 버전에서 습성 변경된 PostGIS 함수"로 크게 변경된 PostGIS 함수입니다.

신규 도형 유형들: PostGIS 2.0 버전은 TIN 및 다면체 표면을 도입했습니다.



지형에 대한 지원이 크게 향상되었습니다. 자세한 내용은 11. "지형(topology)"을 참조하십시오.



PostGIS 2.0 버전에서, 래스터 유형과 래스터 기능성을 통합했습니다. 신규 래스터 함수들을 여기에 나열하기에는 너무 많고 또 모두 신규 함수이기 때문에, 사용할 수 있는 래스터 함수에 대한 자세한 내용은 9. "래스터 참조 문서"를 참조해주십시오. 2.0 이전 버전은 raster_columns 및 raster_overviews 를 실재 테이블로 가지고 있었습니다. 이들은 배포 전에 뷰로 변경되었습니다. ST_AddRasterColumn 같은 함수들은 제거되어 AddRasterConstraints, DropRasterConstraints 로 대체되었습니다. 그 결과 래스터 테이블을 생성하는 일부 확장 프로그램도 수정해야 할 수 있습니다.



TIGER 2010 인구조사 데이터를 사용할 수 있도록 Tiger Geocoder 를 업그레이드했으며, 이제 핵심 PostGIS 문서에 포함되었습니다. 역(逆) 지오코더 함수도 추가되었습니다. 자세한 내용은 13.1 "TIGER 지오코딩 도구"를 참조하십시오.

다음 함수들은 PostGIS 2.0 버전에서 개선된 PostGIS 함수입니다.

14.12.7. 2.0 버전에서 습성 변경된 PostGIS 함수

다음 함수들은 PostGIS 2.0 버전에서 습성이 변경된 PostGIS 함수로, 확장 프로그램을 수정해야 할 수도 있습니다.



중요도가 떨어져 더 이상 사용되지 않는 함수들을 대부분 제거했습니다. 이들은 1.2 버전 이후로 문서화되지 않은 함수이거나 한번도 문서화된 일이 없는 내부 함수들입니다. 사용자가 문서에서 찾을 수 없는 함수를 이용하고 있다면, 해당 함수는 아마도 중요도가 떨어져 더 이상 사용되지 않는 함수이거나, 곧 중요도가 떨어져 더 이상 사용되지 않을 함수이거나, 또는 내부 함수로 사용을 피해야 할 함수입니다. 중요도가 떨어져 더 이상 사용되지 않는 함수에 의존적인 확장 프로그램 또는 도구를 사용하고 있다면, 자세한 내용을 Q: 3.2 에서 참조해주십시오.



도형의 경계 상자가 4 비트 부동소수점 데이터 유형(float4)에서 이중 정밀도(float8)로 변경되었습니다. 이 변경은 경계 상자 연산자를 이용해서 얻는 답과 경계 상자를 도형으로 형변환하는 작업에 영향을 미칩니다. 예를 들어 ST_SetSRID(abbox) 함수는 PostGIS 2.0 이상 버전에서 이전 버전에서와는 다르지만 좀 더 정확한 답을 반환하게 될 것입니다. 또 포트(port) 쿼리를 뷰(view)하는 답도 아주 달라질 수 있습니다.



hasnodata 인수가 exclude_nodata_value 인수로 대체되었습니다. exclude_nodata_value 는 기존 hasnodata 와 동일한 의미를 지니지만 그 목적이 더 뚜렷합니다.

14.12.8. 1.5 버전에서 신규, 습성 변경,또는 개선된 PostGIS 함수

다음 함수들은 이번 마이너 배포판에서 도입되거나 개선된 PostGIS 함수입니다.

14.12.9. 1.4 버전에서 신규, 습성 변경, 또는 개선된 PostGIS 함수

다음 함수들은 PostGIS 1.4 버전 배포판에서 도입되거나 개선된 PostGIS 함수입니다.

14.12.10. 1.3 버전의 신규 PostGIS 함수

다음 함수들은 PostGIS 1.3 버전 배포판에서 도입된 PostGIS 함수입니다.

Chapter 15. 문제 보고

15.1. 소프트웨어 버그 보고하기

버그를 보고하는 것은 실질적으로 PostGIS 개발을 도와주는 근본적인 방법입니다. 가장 효과적인 버그 리포트는 PostGIS 개발자들이 해당 버그를 다시 재현할 수 있도록 하는 것이기 때문에, 이상적으로는 버그를 유발한 스크립트 및 버그가 발생한 환경에 관련된 모든 정보를 담고 있어야 할 것입니다. (PostGIS 의 경우) SELECT postgis_full_version() 을, 그리고 (PostgreSQL 의 경우) SELECT version() 을 실행하면 쓸 만한 정보를 추출할 수 있습니다.

최신 배포판을 사용하지 않을 경우, 먼저 release changelog 를 살펴보고 사용자의 버그가 이미 수정됐는지 알아보는 것도 중요합니다.

PostGIS bug tracker 를 이용하면 사용자의 보고가 무시되지 않으며, 사용자에게 어떻게 처리되고 있는지 계속 알려줄 것입니다. 새 버그를 보고하기 전에 이미 알려진 버그인지 아닌지 알아보기 위해 데이터베이스를 쿼리해보십시오. 그리고 진행중인 버그일 경우, 사용자가 어떤 새로운 정보라도 가지고 있다면 추가해주십시오.

새 보고를 작성하기 전에 효과적으로 버그를 보고하는 방법 에 대한 사이먼 테이섬(Simon Tatham)의 글을 읽어보는 편이 좋습니다.

15.2. 문서 문제점 보고하기

문서는 소프트웨어의 기능 및 습성을 정확하게 반영해야 합니다. 그렇지 않을 경우, 소프트웨어 버그 때문이거나 문서에 오류 또는 결함이 있기 때문일 수 있습니다.

문서 문제점 또한 PostGIS bug tracker 로 보고할 수 있습니다.

사용자의 정정 사항이 사소할 경우, 그냥 문서의 어떤 위치에 있는지 정확하게 밝힌 상태로 새 버그 트래커 이슈에 설명한 다음 발행하십시오.

사용자의 정정 사항이 좀 더 광범위할 경우, Subversion 패치를 이용하는 편이 훨씬 좋습니다. 다음은 유닉스 상의 4 단계 과정입니다(사용자가 이미 Subversion 을 설치했다고 가정합니다):

1. PostGIS 의 Subversion 트렁크(trunk)의 복사본을 확인하십시오. 유닉스 상에서 다음과 같이 입력합니다:

```
svn checkout http://svn.osgeo.org/postgis/trunk/
```

이 복사본을 "./trunk" 디렉터리에 저장할 것입니다.

2. 문서에 대한 사용자의 정정 사항을 원하는 편집기로 수정하십시오. 유닉스 상에서 (예를 들어) 다음과 같이 입력합니다:

```
vim trunk/doc/postgis.xml
```

문서가 HTML 이 아니라 DocBook XML 서식으로 작성된다는 점에 주의하십시오. 따라서 사용자가 DocBook XML 에 익숙하지 않다면, 문서의 다른 부분에 있는 예를 따라주십시오.

3. 문서의 복사본 원본으로부터 변경 사항을 담고 있는 패치 파일을 만드십시오. 유닉스 상에서 다음과 같이 입력합니다:

```
svn diff trunk/doc/postgis.xml > doc.patch
```

4. 새 버그 트래커 이슈에 패치 파일을 첨부하십시오.